

r-type
0.0.0

Generated by Doxygen 1.11.0

1 R-Type	1
1.1 Supported Platforms	1
1.2 Project Structure	1
1.3 Prerequisites	2
1.4 Build and Run	2
1.4.1 Unix (Linux, macOS)	2
1.4.2 Windows	2
1.5 Documentation	2
1.6 Technology Stack Analysis	2
1.7 External Libraries	3
1.8 Contributing	3
1.9 License	3
2 R-Type Client	5
2.1 Configuration	5
2.2 Plugins	5
2.3 Troubleshooting / Tips	5
3 R-Type Server	7
3.1 Configuration	7
3.2 Plugins	7
3.3 Networking / Protocol	7
3.4 Troubleshooting / Tips	7
4 How to create your own plugin	9
5 Audio plugin	11
5.1 How to add a new audio plugin	11
6 Game plugin	13
6.1 How to add a new game plugin	13
6.1.1 Client	13
6.1.2 Server	14
7 Network plugin	17
7.1 How to add a new network plugin	17
7.1.1 Client	17
7.1.2 Server	18
8 Renderer plugin	21
8.1 How to add a new renderer plugin	21
9 Technical Analysis & Technology Comparison	23
9.1 Evaluation Criteria	23
9.2 Network: Asio vs Alternatives	23
9.2.1 Selected: Asio (Standalone)	23

9.3 Graphics & Audio: SFML vs Alternatives	23
9.3.1 Selected: SFML (Simple and Fast Multimedia Library)	23
9.4 Testing: GoogleTest vs Alternatives	24
9.4.1 Selected: GoogleTest	24
9.5 Architectural Summary	24
9.5.1 Final Technology Stack	24
9.5.2 Stack Advantages	24
10 Network Protocol Specification	25
10.1 1. Introduction	25
10.2 2. Terminology	25
10.3 3. Transport	25
10.4 4. Packet Header	25
10.5 5. Packet Types	25
10.6 6. Reliability & Fragmentation	26
10.7 7. Disconnect & Error Codes	26
10.8 8. Event Types	26
10.9 9. Security	26
10.10 10. Timing	27
10.11 Appendix A: Example Packets	27
11 Contributing to R-Type	29
11.1 Contribution workflow	29
11.1.1 Create / Find an Issue	29
11.1.2 Branching & PR Naming	29
11.1.3 Commit Norms	29
11.2 Code Style	30
11.2.1 C++	30
11.2.1.1 Naming Convention	30
11.2.2 Yaml	30
12 LICENSE	31
13 Namespace Index	33
13.1 Namespace List	33
14 Hierarchical Index	35
14.1 Class Hierarchy	35
15 Class Index	39
15.1 Class List	39
16 File Index	45
16.1 File List	45
17 Namespace Documentation	51

17.1 cli Namespace Reference	51
17.1.1 Typedef Documentation	51
17.1.1.1 json	51
17.2 cli::GameConfig Namespace Reference	51
17.3 cli::GameConfig::Animation Namespace Reference	52
17.3.1 Variable Documentation	52
17.3.1.1 FRAME_DURATION	52
17.4 cli::GameConfig::Beam Namespace Reference	52
17.4.1 Variable Documentation	52
17.4.1.1 BAR_HEIGHT	52
17.4.1.2 BAR_WIDTH	52
17.4.1.3 BAR_X	52
17.4.1.4 BAR_Y	52
17.4.1.5 CHARGE_RATE	52
17.4.1.6 MAX_CHARGE	52
17.5 cli::GameConfig::Enemy Namespace Reference	53
17.6 cli::GameConfig::Enemy::Easy Namespace Reference	53
17.6.1 Variable Documentation	53
17.6.1.1 ANIMATION_DURATION	53
17.6.1.2 ANIMATION_FRAMES	53
17.6.1.3 DAMAGE	53
17.6.1.4 FRAMES_PER_ROW	53
17.6.1.5 HEALTH	53
17.6.1.6 SCALE	53
17.6.1.7 SHOOT_COOLDOWN	53
17.6.1.8 SPAWN_RATE	54
17.6.1.9 SPEED	54
17.6.1.10 SPRITE_HEIGHT	54
17.6.1.11 SPRITE_WIDTH	54
17.7 cli::GameConfig::Explosion Namespace Reference	54
17.7.1 Variable Documentation	54
17.7.1.1 ANIMATION_DURATION	54
17.7.1.2 ANIMATION_FRAMES	54
17.7.1.3 FRAMES_PER_ROW	54
17.7.1.4 LIFETIME	54
17.7.1.5 SCALE	54
17.7.1.6 SPRITE_HEIGHT	55
17.7.1.7 SPRITE_WIDTH	55
17.8 cli::GameConfig::Hitbox Namespace Reference	55
17.8.1 Variable Documentation	55
17.8.1.1 ENEMY_RADIUS	55
17.8.1.2 PLAYER_RADIUS	55
17.8.1.3 PROJECTILE_BASIC_RADIUS	55

17.8.1.4 PROJECTILE_SUPERCHARGED_RADIUS	55
17.9 cli::GameConfig::LoadingAnimation Namespace Reference	55
17.9.1 Variable Documentation	55
17.9.1.1 ANIMATION_DURATION	55
17.9.1.2 ANIMATION_FRAMES	56
17.9.1.3 OFFSET_X	56
17.9.1.4 OFFSET_Y	56
17.9.1.5 SPRITE_HEIGHT	56
17.9.1.6 SPRITE_WIDTH	56
17.10 cli::GameConfig::Player Namespace Reference	56
17.10.1 Variable Documentation	56
17.10.1.1 DIAGONAL_SPEED_MULTIPLIER	56
17.10.1.2 FRAMES_PER_ROW	56
17.10.1.3 SCALE	56
17.10.1.4 SPEED	56
17.10.1.5 SPRITE_HEIGHT	57
17.10.1.6 SPRITE_WIDTH	57
17.10.1.7 TOTAL_FRAMES	57
17.11 cli::GameConfig::Projectile Namespace Reference	57
17.12 cli::GameConfig::Projectile::Basic Namespace Reference	57
17.12.1 Variable Documentation	57
17.12.1.1 DAMAGE	57
17.12.1.2 FIRE_COOLDOWN	57
17.12.1.3 LIFETIME	57
17.12.1.4 SCALE	57
17.12.1.5 SPEED	57
17.12.1.6 SPRITE_HEIGHT	58
17.12.1.7 SPRITE_WIDTH	58
17.13 cli::GameConfig::Projectile::Supercharged Namespace Reference	58
17.13.1 Variable Documentation	58
17.13.1.1 ANIMATION_DURATION	58
17.13.1.2 ANIMATION_FRAMES	58
17.13.1.3 CHARGE_TIME	58
17.13.1.4 DAMAGE	58
17.13.1.5 FIRE_COOLDOWN	58
17.13.1.6 LIFETIME	58
17.13.1.7 SCALE	58
17.13.1.8 SPEED	59
17.13.1.9 SPRITE_HEIGHT	59
17.13.1.10 SPRITE_WIDTH	59
17.14 cli::GameConfig::Screen Namespace Reference	59
17.14.1 Variable Documentation	59
17.14.1.1 MAX_Y	59

17.14.1.2	MIN_Y	59
17.14.1.3	REMOVE_MAX_Y	59
17.14.1.4	REMOVE_MIN_Y	59
17.14.1.5	REMOVE_X	59
17.14.1.6	SPAWN_X	59
17.15	cli::GameConfig::Stage Namespace Reference	60
17.15.1	Variable Documentation	60
17.15.1.1	CEILING_OFFSET_Y	60
17.15.1.2	FLOOR_OFFSET_Y	60
17.16	ecs Namespace Reference	60
17.16.1	Typedef Documentation	61
17.16.1.1	Entity	61
17.16.2	Variable Documentation	61
17.16.2.1	INVALID_ENTITY	61
17.17	eng Namespace Reference	61
17.17.1	Typedef Documentation	62
17.17.1.1	id	62
17.17.2	Enumeration Type Documentation	62
17.17.2.1	ConnectionState	62
17.17.2.2	EventType	63
17.17.2.3	Key	63
17.17.2.4	State	64
17.17.2.5	Status	64
17.18	gme Namespace Reference	64
17.18.1	Enumeration Type Documentation	66
17.18.1.1	EnemyBehavior	66
17.18.1.2	LevelState	66
17.18.1.3	ServerEntityType	67
17.18.1.4	State	67
17.19	gme::Utils Namespace Reference	67
17.19.1	Function Documentation	67
17.19.1.1	calculateHitboxOffsets()	67
17.20	rnp Namespace Reference	68
17.20.1	Typedef Documentation	70
17.20.1.1	AckHandler	70
17.20.1.2	ConnectAcceptHandler	70
17.20.1.3	ConnectHandler	70
17.20.1.4	DisconnectHandler	70
17.20.1.5	EntityEventHandler	70
17.20.1.6	ErrorHandler	70
17.20.1.7	GameStartHandler	71
17.20.1.8	LobbyCreateHandler	71
17.20.1.9	LobbyCreateResponseHandler	71

17.20.1.10 LobbyJoinHandler	71
17.20.1.11 LobbyJoinResponseHandler	71
17.20.1.12 LobbyLeaveHandler	71
17.20.1.13 LobbyListRequestHandler	71
17.20.1.14 LobbyListResponseHandler	71
17.20.1.15 LobbyUpdateHandler	71
17.20.1.16 PingHandler	71
17.20.1.17 PongHandler	71
17.20.1.18 StartGameRequestHandler	72
17.20.1.19 WorldStateHandler	72
17.20.2 Enumeration Type Documentation	72
17.20.2.1 DisconnectReason	72
17.20.2.2 EntitySubtype	72
17.20.2.3 EntityType	72
17.20.2.4 ErrorCode	73
17.20.2.5 EventType	73
17.20.2.6 HandlerResult	73
17.20.2.7 LobbyStatus	74
17.20.2.8 PacketFlags	74
17.20.2.9 PacketType	74
17.20.3 Variable Documentation	75
17.20.3.1 MAX_PAYLOAD	75
17.21 srv Namespace Reference	75
17.21.1 Typedef Documentation	76
17.21.1.1 id	76
17.21.1.2 json	76
17.21.2 Enumeration Type Documentation	76
17.21.2.1 LobbyStatus	76
17.21.3 Variable Documentation	76
17.21.3.1 MAX_CLIENTS	76
17.21.3.2 MAX_IP_LENGTH	76
17.21.3.3 MAX_LEN_RECV_BUFFER	76
17.22 srv::Config Namespace Reference	76
17.23 srv::Config::Network Namespace Reference	77
17.23.1 Variable Documentation	77
17.23.1.1 DEFAULT_MAX_CLIENT	77
17.23.1.2 DEFAULT_NETWORK_HOST	77
17.23.1.3 DEFAULT_NETWORK_PORT	77
17.24 srv::Game Namespace Reference	77
17.24.1 Variable Documentation	77
17.24.1.1 DEFAULT_TICK_RATE	77
17.24.1.2 DEFAULT_UPDATE_INTERVAL	77
17.25 srv::Path Namespace Reference	77

17.26	srv::Path::Plugin Namespace Reference	77
17.26.1	Variable Documentation	78
17.26.1.1	PLUGINS_GAME_RTYPE_SERVER	78
17.26.1.2	PLUGINS_NETWORK_ASIO_SERVER	78
17.27	utl Namespace Reference	78
17.27.1	Typedef Documentation	79
17.27.1.1	EntryPointFn	79
17.27.1.2	LibHandle	80
17.27.2	Enumeration Type Documentation	80
17.27.2.1	EventPriority	80
17.27.2.2	EventType	80
17.27.2.3	LogLevel	81
17.27.2.4	PluginType	81
17.27.3	Function Documentation	81
17.27.3.1	calculateHitboxOffsets()	81
17.27.3.2	calculateHitboxOffsetsRelative()	82
17.27.3.3	calculateSpriteCenter()	83
17.27.3.4	getEnvMap()	83
17.27.3.5	readFile()	83
17.27.4	Variable Documentation	84
17.27.4.1	GAME_LOGIC	84
17.27.4.2	NETWORK_CLIENT	84
17.27.4.3	NETWORK_SERVER	84
17.27.4.4	RENDERING_ENGINE	84
17.28	utl::cli Namespace Reference	84
17.29	utl::Config Namespace Reference	84
17.30	utl::Config::Audio Namespace Reference	84
17.30.1	Variable Documentation	84
17.30.1.1	DEFAULT_AUDIO_MUTED	84
17.30.1.2	DEFAULT_AUDIO_VOLUME	85
17.31	utl::Config::Color Namespace Reference	85
17.31.1	Variable Documentation	85
17.31.1.1	BLUE	85
17.31.1.2	BLUE_SECOND	85
17.31.1.3	CYAN_ELECTRIC	85
17.31.1.4	CYAN_ELECTRIC_FOREGROUND	86
17.31.1.5	CYAN_ELECTRIC_PARTICLES	86
17.31.1.6	DARK	86
17.31.1.7	GRAY_BLUE_SUBTLE	86
17.31.1.8	GREEN	86
17.31.1.9	GREEN_READY	86
17.31.1.10	INFO_TEXT_COLOR	86
17.31.1.11	PURPLE	86

17.31.1.12	STAR_BG	86
17.31.1.13	STAR_MID	87
17.31.1.14	TEXT_VALUE_COLOR	87
17.31.1.15	WHITE	87
17.31.1.16	WHITE_LOW	87
17.31.1.17	WHITE_TRANS	87
17.31.1.18	YELLOW	87
17.32	utl::Config::Game Namespace Reference	87
17.32.1	Variable Documentation	87
17.32.1.1	DEFAULT_CONTROL_SCHEME	87
17.32.1.2	DEFAULT_SKIN_INDEX	87
17.32.1.3	DEFAULT_VIDEO_QUALITY	88
17.33	utl::Config::Network Namespace Reference	88
17.33.1	Variable Documentation	88
17.33.1.1	DEFAULT_NETWORK_HOST	88
17.33.1.2	DEFAULT_NETWORK_PORT	88
17.34	utl::Config::Player Namespace Reference	88
17.34.1	Variable Documentation	88
17.34.1.1	DEFAULT_PLAYER_NAME	88
17.35	utl::Config::Window Namespace Reference	88
17.35.1	Variable Documentation	88
17.35.1.1	DEFAULT_WINDOW_FRAME_LIMIT	88
17.35.1.2	DEFAULT_WINDOW_FULLSCREEN	88
17.35.1.3	DEFAULT_WINDOW_HEIGHT	89
17.35.1.4	DEFAULT_WINDOW_WIDTH	89
17.36	utl::GameConfig Namespace Reference	89
17.37	utl::GameConfig::Animation Namespace Reference	89
17.37.1	Variable Documentation	89
17.37.1.1	FRAME_DURATION	89
17.38	utl::GameConfig::Beam Namespace Reference	89
17.38.1	Variable Documentation	89
17.38.1.1	BAR_HEIGHT	89
17.38.1.2	BAR_WIDTH	90
17.38.1.3	BAR_X	90
17.38.1.4	BAR_Y	90
17.38.1.5	CHARGE_RATE	90
17.38.1.6	MAX_CHARGE	90
17.39	utl::GameConfig::Enemy Namespace Reference	90
17.40	utl::GameConfig::Enemy::Easy Namespace Reference	90
17.40.1	Variable Documentation	90
17.40.1.1	ANIMATION_DURATION	90
17.40.1.2	ANIMATION_FRAMES	91
17.40.1.3	DAMAGE	91

17.40.1.4	FRAMES_PER_ROW	91
17.40.1.5	HEALTH	91
17.40.1.6	SCALE	91
17.40.1.7	SHOOT_COOLDOWN	91
17.40.1.8	SPAWN_RATE	91
17.40.1.9	SPEED	91
17.40.1.10	SPRITE_HEIGHT	91
17.40.1.11	SPRITE_WIDTH	91
17.41	utl::GameConfig::Explosion Namespace Reference	92
17.41.1	Variable Documentation	92
17.41.1.1	ANIMATION_DURATION	92
17.41.1.2	ANIMATION_FRAMES	92
17.41.1.3	FRAMES_PER_ROW	92
17.41.1.4	LIFETIME	92
17.41.1.5	SCALE	92
17.41.1.6	SPRITE_HEIGHT	92
17.41.1.7	SPRITE_WIDTH	92
17.42	utl::GameConfig::Hitbox Namespace Reference	92
17.42.1	Variable Documentation	93
17.42.1.1	ENEMY_RADIUS	93
17.42.1.2	PLAYER_RADIUS	93
17.42.1.3	PROJECTILE_BASIC_RADIUS	93
17.42.1.4	PROJECTILE_SUPERCHARGED_RADIUS	93
17.43	utl::GameConfig::LoadingAnimation Namespace Reference	93
17.43.1	Variable Documentation	93
17.43.1.1	ANIMATION_DURATION	93
17.43.1.2	ANIMATION_FRAMES	93
17.43.1.3	OFFSET_X	93
17.43.1.4	OFFSET_Y	94
17.43.1.5	SPRITE_HEIGHT	94
17.43.1.6	SPRITE_WIDTH	94
17.44	utl::GameConfig::Player Namespace Reference	94
17.44.1	Variable Documentation	94
17.44.1.1	DIAGONAL_SPEED_MULTIPLIER	94
17.44.1.2	FRAMES_PER_ROW	94
17.44.1.3	HEIGHT	94
17.44.1.4	SCALE	94
17.44.1.5	SPEED	95
17.44.1.6	SPRITE_HEIGHT	95
17.44.1.7	SPRITE_WIDTH	95
17.44.1.8	WIDTH	95
17.45	utl::GameConfig::Projectile Namespace Reference	95
17.46	utl::GameConfig::Projectile::Basic Namespace Reference	95

17.46.1 Variable Documentation	95
17.46.1.1 DAMAGE	95
17.46.1.2 FIRE_COOLDOWN	95
17.46.1.3 LIFETIME	96
17.46.1.4 SCALE	96
17.46.1.5 SPEED	96
17.46.1.6 SPRITE_HEIGHT	96
17.46.1.7 SPRITE_WIDTH	96
17.47 utl::GameConfig::Projectile::Supercharged Namespace Reference	96
17.47.1 Variable Documentation	96
17.47.1.1 ANIMATION_DURATION	96
17.47.1.2 ANIMATION_FRAMES	96
17.47.1.3 CHARGE_TIME	97
17.47.1.4 DAMAGE	97
17.47.1.5 FIRE_COOLDOWN	97
17.47.1.6 LIFETIME	97
17.47.1.7 SCALE	97
17.47.1.8 SPEED	97
17.47.1.9 SPRITE_HEIGHT	97
17.47.1.10 SPRITE_WIDTH	97
17.48 utl::GameConfig::Screen Namespace Reference	97
17.48.1 Variable Documentation	98
17.48.1.1 MAX_Y	98
17.48.1.2 MIN_Y	98
17.48.1.3 REMOVE_MAX_Y	98
17.48.1.4 REMOVE_MIN_Y	98
17.48.1.5 REMOVE_X	98
17.48.1.6 SPAWN_X	98
17.49 utl::GameConfig::Server Namespace Reference	98
17.49.1 Variable Documentation	98
17.49.1.1 SCREEN_HEIGHT	98
17.49.1.2 SCREEN_WIDTH	99
17.49.1.3 WORLD_MARGIN	99
17.50 utl::GameConfig::Server::Enemy Namespace Reference	99
17.51 utl::GameConfig::Server::Enemy::Advanced Namespace Reference	99
17.51.1 Variable Documentation	99
17.51.1.1 DAMAGE	99
17.51.1.2 HEALTH	99
17.51.1.3 HITBOX_RADIUS	99
17.51.1.4 SHOOT_COOLDOWN	99
17.51.1.5 SINE_AMPLITUDE	99
17.51.1.6 SINE_FREQUENCY	100
17.51.1.7 SPEED	100

17.52	utl::GameConfig::Server::Enemy::AI Namespace Reference	100
17.52.1	Variable Documentation	100
17.52.1.1	AGGRESSIVE_ACTIVATION_DISTANCE	100
17.52.1.2	MAX_SHOOT_DISTANCE	100
17.52.1.3	SHOOT_CHANCE	100
17.52.1.4	ZIGZAG_PERIOD	100
17.52.1.5	ZIGZAG_SPEED_X	100
17.52.1.6	ZIGZAG_SPEED_Y	100
17.53	utl::GameConfig::Server::Enemy::Basic Namespace Reference	100
17.53.1	Variable Documentation	101
17.53.1.1	DAMAGE	101
17.53.1.2	HEALTH	101
17.53.1.3	HITBOX_RADIUS	101
17.53.1.4	SHOOT_COOLDOWN	101
17.53.1.5	SPEED	101
17.54	utl::GameConfig::Server::Enemy::Boss Namespace Reference	101
17.54.1	Variable Documentation	101
17.54.1.1	DAMAGE	101
17.54.1.2	HEALTH	102
17.54.1.3	HITBOX_RADIUS	102
17.54.1.4	PHASE1_COOLDOWN	102
17.54.1.5	PHASE1_SINE_AMPLITUDE	102
17.54.1.6	PHASE1_SINE_FREQUENCY	102
17.54.1.7	PHASE1_SPEED	102
17.54.1.8	PHASE2_COOLDOWN	102
17.54.1.9	PHASE3_AGGRESSION	102
17.54.1.10	PHASE3_COOLDOWN	102
17.54.1.11	PHASE3_SPEED	102
17.54.1.12	PHASE_2_THRESHOLD	102
17.54.1.13	PHASE_3_THRESHOLD	102
17.54.1.14	RAM_DAMAGE	103
17.54.1.15	SPREAD_ANGLE_DEGREES	103
17.54.1.16	SPREAD_INTERVAL	103
17.54.1.17	SPREAD_SPEED	103
17.55	utl::GameConfig::Server::Network Namespace Reference	103
17.55.1	Variable Documentation	103
17.55.1.1	BROADCAST_INTERVAL	103
17.55.1.2	BROADCAST_RATE	103
17.56	utl::GameConfig::Server::Projectile Namespace Reference	103
17.56.1	Variable Documentation	103
17.56.1.1	ENEMY_DAMAGE	103
17.56.1.2	ENEMY_LIFETIME	103
17.56.1.3	ENEMY_SPEED	104

17.57 utl::GameConfig::Stage Namespace Reference	104
17.57.1 Variable Documentation	104
17.57.1.1 CEILING_OFFSET_Y	104
17.57.1.2 FLOOR_OFFSET_Y	104
17.58 utl::Path Namespace Reference	104
17.59 utl::Path::Audio Namespace Reference	104
17.59.1 Variable Documentation	104
17.59.1.1 AUDIO_BATTLE_THEME	104
17.59.1.2 AUDIO_BEGIN	105
17.59.1.3 AUDIO_BOSS	105
17.59.1.4 AUDIO_COIN	105
17.59.1.5 AUDIO_DEATH_ALLIES	105
17.59.1.6 AUDIO_DEATH_ENEMIES	105
17.59.1.7 AUDIO_GOOFY	105
17.59.1.8 AUDIO_INPUT	105
17.59.1.9 AUDIO_SUPERCHARGED_SHOT	105
17.59.1.10 AUDIO_TITLE	105
17.60 utl::Path::Font Namespace Reference	105
17.60.1 Variable Documentation	106
17.60.1.1 FONTS_RTYPE	106
17.61 utl::Path::Icons Namespace Reference	106
17.61.1 Variable Documentation	106
17.61.1.1 ICON_APP	106
17.62 utl::Path::Plugin Namespace Reference	106
17.62.1 Variable Documentation	106
17.62.1.1 PLUGIN_AUDIO_SFML	106
17.62.1.2 PLUGIN_GAME_MULTI	106
17.62.1.3 PLUGIN_GAME_SOLO	106
17.62.1.4 PLUGIN_NETWORK_ASIO_CLIENT	107
17.62.1.5 PLUGIN_RENDERER_SFML	107
17.63 utl::Path::Texture Namespace Reference	107
17.63.1 Variable Documentation	107
17.63.1.1 ICON_PATH	107
17.63.1.2 TEXTURE_BOSS	107
17.63.1.3 TEXTURE_ENEMY_ADVANCED	108
17.63.1.4 TEXTURE_ENEMY_BASIC	108
17.63.1.5 TEXTURE_ENEMY_EASY	108
17.63.1.6 TEXTURE_ENEMY_PROJECTILE	108
17.63.1.7 TEXTURE_EXPLOSION	108
17.63.1.8 TEXTURE_PLAYER	108
17.63.1.9 TEXTURE_SCORE_COUNTER_BG	108
17.63.1.10 TEXTURE_SCORE_DIGIT_0	108
17.63.1.11 TEXTURE_SCORE_DIGIT_1	108

17.63.1.12 TEXTURE_SCORE_DIGIT_2	108
17.63.1.13 TEXTURE_SCORE_DIGIT_3	109
17.63.1.14 TEXTURE_SCORE_DIGIT_4	109
17.63.1.15 TEXTURE_SCORE_DIGIT_5	109
17.63.1.16 TEXTURE_SCORE_DIGIT_6	109
17.63.1.17 TEXTURE_SCORE_DIGIT_7	109
17.63.1.18 TEXTURE_SCORE_DIGIT_8	109
17.63.1.19 TEXTURE_SCORE_DIGIT_9	109
17.63.1.20 TEXTURE_SHOOT	109
17.63.1.21 TEXTURE_SHOOT_CHARGED	109
17.63.1.22 TEXTURE_SHOOT_LOADING	109
17.63.1.23 TEXTURE_STAGE1_CEILING	110
17.63.1.24 TEXTURE_STAGE1_FLOOR	110
17.64 utl::srv Namespace Reference	110
18 Class Documentation	111
18.1 ecs::Animation Struct Reference	111
18.1.1 Detailed Description	113
18.1.2 Member Data Documentation	113
18.1.2.1 current_frame	113
18.1.2.2 current_time	113
18.1.2.3 frame_duration	113
18.1.2.4 frame_height	113
18.1.2.5 frame_width	113
18.1.2.6 frames_per_row	113
18.1.2.7 total_frames	113
18.2 ecs::AnimationSystem Class Reference	113
18.2.1 Detailed Description	116
18.2.2 Constructor & Destructor Documentation	116
18.2.2.1 AnimationSystem() [1/3]	116
18.2.2.2 ~AnimationSystem()	116
18.2.2.3 AnimationSystem() [2/3]	116
18.2.2.4 AnimationSystem() [3/3]	116
18.2.3 Member Function Documentation	116
18.2.3.1 operator=() [1/2]	116
18.2.3.2 operator=() [2/2]	116
18.2.3.3 update()	116
18.3 gme::AnimationSystem Class Reference	117
18.3.1 Detailed Description	119
18.3.2 Constructor & Destructor Documentation	119
18.3.2.1 AnimationSystem() [1/3]	119
18.3.2.2 ~AnimationSystem()	119
18.3.2.3 AnimationSystem() [2/3]	119

18.3.2.4 AnimationSystem() [3/3]	119
18.3.3 Member Function Documentation	119
18.3.3.1 operator=() [1/2]	119
18.3.3.2 operator=() [2/2]	119
18.3.3.3 update()	119
18.4 utl::cli::AppConfig Struct Reference	120
18.4.1 Detailed Description	121
18.4.2 Member Data Documentation	121
18.4.2.1 audioVolume	121
18.4.2.2 controlScheme	121
18.4.2.3 frameLimit	121
18.4.2.4 fullscreen	121
18.4.2.5 height	121
18.4.2.6 host	121
18.4.2.7 player_name	121
18.4.2.8 port	121
18.4.2.9 skinIndex	121
18.4.2.10 videoQuality	122
18.4.2.11 width	122
18.5 utl::srv::AppConfig Struct Reference	122
18.5.1 Detailed Description	122
18.5.2 Member Data Documentation	123
18.5.2.1 host	123
18.5.2.2 port	123
18.6 cli::ArgsConfig Struct Reference	123
18.6.1 Detailed Description	125
18.6.2 Member Function Documentation	125
18.6.2.1 fromFile()	125
18.6.3 Member Data Documentation	125
18.6.3.1 audio_lib_path	125
18.6.3.2 exit	125
18.6.3.3 frameLimit	125
18.6.3.4 fullscreen	125
18.6.3.5 game_multi_lib_path	125
18.6.3.6 game_solo_lib_path	126
18.6.3.7 height	126
18.6.3.8 host	126
18.6.3.9 network_lib_path	126
18.6.3.10 player_name	126
18.6.3.11 port	126
18.6.3.12 renderer_lib_path	126
18.6.3.13 width	126
18.7 srv::ArgsConfig Struct Reference	126

18.7.1 Detailed Description	127
18.7.2 Member Function Documentation	127
18.7.2.1 fromFile()	127
18.7.3 Member Data Documentation	128
18.7.3.1 exit	128
18.7.3.2 game_lib_path	128
18.7.3.3 host	128
18.7.3.4 network_lib_path	128
18.7.3.5 port	128
18.8 cli::ArgsHandler Class Reference	128
18.8.1 Detailed Description	129
18.8.2 Constructor & Destructor Documentation	129
18.8.2.1 ArgsHandler() [1/3]	129
18.8.2.2 ~ArgsHandler()	129
18.8.2.3 ArgsHandler() [2/3]	129
18.8.2.4 ArgsHandler() [3/3]	130
18.8.3 Member Function Documentation	130
18.8.3.1 operator=() [1/2]	130
18.8.3.2 operator=() [2/2]	130
18.8.3.3 ParseArgs()	130
18.8.3.4 ParseEnv()	130
18.9 srv::ArgsHandler Class Reference	131
18.9.1 Detailed Description	132
18.9.2 Constructor & Destructor Documentation	132
18.9.2.1 ArgsHandler() [1/3]	132
18.9.2.2 ~ArgsHandler()	132
18.9.2.3 ArgsHandler() [2/3]	132
18.9.2.4 ArgsHandler() [3/3]	132
18.9.3 Member Function Documentation	132
18.9.3.1 operator=() [1/2]	132
18.9.3.2 operator=() [2/2]	132
18.9.3.3 ParseArgs()	132
18.9.3.4 ParseEnv()	133
18.10 eng::AScene Class Reference	133
18.10.1 Detailed Description	135
18.10.2 Constructor & Destructor Documentation	135
18.10.2.1 AScene() [1/3]	135
18.10.2.2 ~AScene()	135
18.10.2.3 AScene() [2/3]	135
18.10.2.4 AScene() [3/3]	135
18.10.3 Member Function Documentation	135
18.10.3.1 addSystem()	135
18.10.3.2 getId()	136

18.10.3.3	getName()	136
18.10.3.4	getRegistry()	136
18.10.3.5	operator=() [1/2]	136
18.10.3.6	operator=() [2/2]	136
18.10.3.7	setName()	136
18.10.3.8	updateSystems()	136
18.10.4	Member Data Documentation	136
18.10.4.1	m_eventBus	136
18.10.4.2	m_eventComponentId	136
18.10.4.3	m_id	137
18.10.4.4	m_name	137
18.10.4.5	m_registry	137
18.10.4.6	m_systems	137
18.11	srv::AScene Class Reference	137
18.11.1	Detailed Description	140
18.11.2	Constructor & Destructor Documentation	140
18.11.2.1	AScene() [1/3]	140
18.11.2.2	~AScene()	140
18.11.2.3	AScene() [2/3]	140
18.11.2.4	AScene() [3/3]	140
18.11.3	Member Function Documentation	140
18.11.3.1	getId()	140
18.11.3.2	getName()	140
18.11.3.3	getRegistry()	140
18.11.3.4	operator=() [1/2]	140
18.11.3.5	operator=() [2/2]	141
18.11.3.6	setName()	141
18.11.4	Member Data Documentation	141
18.11.4.1	m_id	141
18.11.4.2	m_name	141
18.11.4.3	m_registry	141
18.11.4.4	s_nextId	141
18.12	eng::AsioClient Class Reference	141
18.12.1	Detailed Description	146
18.12.2	Constructor & Destructor Documentation	146
18.12.2.1	AsioClient()	146
18.12.2.2	~AsioClient()	147
18.12.3	Member Function Documentation	147
18.12.3.1	connect()	147
18.12.3.2	createLobby()	148
18.12.3.3	disconnect()	149
18.12.3.4	generatePingNonce()	150
18.12.3.5	getConnectionState()	150

18.12.3.6	getCurrentLobbyId()	150
18.12.3.7	getLatency()	150
18.12.3.8	getName()	150
18.12.3.9	getServerTickRate()	150
18.12.3.10	getSessionId()	150
18.12.3.11	getStats()	151
18.12.3.12	getType()	151
18.12.3.13	handleConnectAccept()	151
18.12.3.14	handleDisconnect()	152
18.12.3.15	handleEntityEvent()	152
18.12.3.16	handleError()	153
18.12.3.17	handleGameStart()	154
18.12.3.18	handleLobbyCreateResponse()	154
18.12.3.19	handleLobbyJoinResponse()	155
18.12.3.20	handleLobbyListResponse()	156
18.12.3.21	handleLobbyUpdate()	157
18.12.3.22	handlePing()	157
18.12.3.23	handlePong()	158
18.12.3.24	handleReceive()	159
18.12.3.25	handleWorldState()	159
18.12.3.26	isConnected()	160
18.12.3.27	isInLobby()	160
18.12.3.28	joinLobby()	160
18.12.3.29	leaveLobby()	161
18.12.3.30	networkThreadLoop()	162
18.12.3.31	processBusEvent()	162
18.12.3.32	processSendQueue()	163
18.12.3.33	requestLobbyList()	164
18.12.3.34	requestStartGame()	165
18.12.3.35	sendConnect()	165
18.12.3.36	sendDisconnect()	166
18.12.3.37	sendPacketImmediate()	167
18.12.3.38	sendPing()	167
18.12.3.39	sendPong()	168
18.12.3.40	sendToServer()	169
18.12.3.41	setClientCapabilities()	169
18.12.3.42	setOnGameStart()	170
18.12.3.43	setOnLobbyCreated()	170
18.12.3.44	setOnLobbyJoined()	170
18.12.3.45	setOnLobbyListReceived()	170
18.12.3.46	setOnLobbyUpdated()	170
18.12.3.47	setPlayerName()	171
18.12.3.48	setupPacketHandlers()	171

18.12.3.49	<code>startReceive()</code>	172
18.12.3.50	<code>update()</code>	173
18.12.3.51	<code>updateConnectionManagement()</code>	173
18.12.4	Member Data Documentation	174
18.12.4.1	<code>m_clientCaps</code>	174
18.12.4.2	<code>m_componentId</code>	174
18.12.4.3	<code>m_connectionState</code>	174
18.12.4.4	<code>m_connectionTimeout</code>	174
18.12.4.5	<code>m_currentLobbyId</code>	174
18.12.4.6	<code>m_eventBus</code>	175
18.12.4.7	<code>m_ioContext</code>	175
18.12.4.8	<code>m_lastPingNonce</code>	175
18.12.4.9	<code>m_lastPingTime</code>	175
18.12.4.10	<code>m_lastServerResponse</code>	175
18.12.4.11	<code>m_latency</code>	175
18.12.4.12	<code>m_networkThread</code>	175
18.12.4.13	<code>m_onGameStart</code>	175
18.12.4.14	<code>m_onLobbyCreated</code>	175
18.12.4.15	<code>m_onLobbyJoined</code>	176
18.12.4.16	<code>m_onLobbyListReceived</code>	176
18.12.4.17	<code>m_onLobbyUpdated</code>	176
18.12.4.18	<code>m_packetHandler</code>	176
18.12.4.19	<code>m_pingInterval</code>	176
18.12.4.20	<code>m_playerName</code>	176
18.12.4.21	<code>m_recvBuffer</code>	176
18.12.4.22	<code>m_running</code>	176
18.12.4.23	<code>m_senderEndpoint</code>	176
18.12.4.24	<code>m_sendQueue</code>	176
18.12.4.25	<code>m_sendQueueMutex</code>	177
18.12.4.26	<code>m_serverEndpoint</code>	177
18.12.4.27	<code>m_serverHost</code>	177
18.12.4.28	<code>m_serverPort</code>	177
18.12.4.29	<code>m_serverTickRate</code>	177
18.12.4.30	<code>m_sessionId</code>	177
18.12.4.31	<code>m_socket</code>	177
18.12.4.32	<code>m_stats</code>	177
18.13	<code>srv::AsioServer</code> Class Reference	177
18.13.1	Detailed Description	182
18.13.2	Constructor & Destructor Documentation	182
18.13.2.1	<code>AsioServer()</code>	182
18.13.2.2	<code>~AsioServer()</code>	183
18.13.3	Member Function Documentation	183
18.13.3.1	<code>broadcastGameStart()</code>	183

18.13.3.2 broadcastLobbyUpdate()	184
18.13.3.3 cleanupEmptyLobbies()	185
18.13.3.4 createLobby()	186
18.13.3.5 disconnectClient()	186
18.13.3.6 endpointToString()	187
18.13.3.7 generateSessionId()	187
18.13.3.8 getClientCount()	188
18.13.3.9 getConnectedSessions()	188
18.13.3.10 getName()	188
18.13.3.11 getType()	188
18.13.3.12 handleBroadcastEvent()	188
18.13.3.13 handleConnect()	189
18.13.3.14 handleDisconnect()	190
18.13.3.15 handleEntityEvent()	191
18.13.3.16 handleLobbyCreate()	192
18.13.3.17 handleLobbyJoin()	192
18.13.3.18 handleLobbyLeave()	193
18.13.3.19 handleLobbyListRequest()	194
18.13.3.20 handlePing()	195
18.13.3.21 handlePong()	196
18.13.3.22 handleReceive()	196
18.13.3.23 handleSendEntityEventToClients()	197
18.13.3.24 handleSendToClientEvent()	198
18.13.3.25 handleStartGameRequest()	198
18.13.3.26 init()	199
18.13.3.27 isRunning()	199
18.13.3.28 joinLobby()	199
18.13.3.29 leaveLobby()	200
18.13.3.30 lobbyToLobbyInfo()	201
18.13.3.31 networkThreadLoop()	201
18.13.3.32 processEventBusEvents()	201
18.13.3.33 processSendQueue()	202
18.13.3.34 sendConnectAccept()	203
18.13.3.35 sendError()	203
18.13.3.36 sendLobbyCreateResponse()	204
18.13.3.37 sendLobbyJoinResponse()	205
18.13.3.38 sendLobbyList()	206
18.13.3.39 sendPacketImmediate()	207
18.13.3.40 sendPong()	208
18.13.3.41 sendToAllClients()	208
18.13.3.42 sendToClient()	209
18.13.3.43 setServerCapabilities()	209
18.13.3.44 setTickRate()	209

18.13.3.45	setupPacketHandlers()	209
18.13.3.46	start()	210
18.13.3.47	startReceive()	211
18.13.3.48	stop()	211
18.13.3.49	update()	212
18.13.3.50	updateClientManagement()	213
18.13.4	Member Data Documentation	213
18.13.4.1	m_clients	213
18.13.4.2	m_clientsMutex	214
18.13.4.3	m_clientTimeout	214
18.13.4.4	m_componentId	214
18.13.4.5	m_endpointToSession	214
18.13.4.6	m_eventBus	214
18.13.4.7	m_host	214
18.13.4.8	m_ioContext	214
18.13.4.9	m_lastPingTime	214
18.13.4.10	m_lobbies	214
18.13.4.11	m_lobbiesMutex	215
18.13.4.12	m_networkThread	215
18.13.4.13	m_nextLobbyId	215
18.13.4.14	m_nextSessionId	215
18.13.4.15	m_packetHandler	215
18.13.4.16	m_pingInterval	215
18.13.4.17	m_port	215
18.13.4.18	m_recvBuffer	215
18.13.4.19	m_running	215
18.13.4.20	m_senderEndpoint	215
18.13.4.21	m_sendQueue	216
18.13.4.22	m_sendQueueMutex	216
18.13.4.23	m_serverCaps	216
18.13.4.24	m_socket	216
18.13.4.25	m_socketMutex	216
18.13.4.26	m_started	216
18.13.4.27	m_tickRate	216
18.14	ecs::ASystem Class Reference	216
18.14.1	Detailed Description	217
18.14.2	Member Function Documentation	217
18.14.2.1	isEnabled()	217
18.14.2.2	setEnabled()	218
18.14.3	Member Data Documentation	218
18.14.3.1	m_isEnable	218
18.15	ecs::Audio Struct Reference	218
18.15.1	Detailed Description	219

18.15.2 Member Data Documentation	220
18.15.2.1 loop	220
18.15.2.2 path	220
18.15.2.3 play	220
18.15.2.4 volume	220
18.16 ecs::AudioSystem Class Reference	220
18.16.1 Detailed Description	223
18.16.2 Constructor & Destructor Documentation	223
18.16.2.1 AudioSystem() [1/3]	223
18.16.2.2 ~AudioSystem()	223
18.16.2.3 AudioSystem() [2/3]	223
18.16.2.4 AudioSystem() [3/3]	223
18.16.3 Member Function Documentation	223
18.16.3.1 operator=() [1/2]	223
18.16.3.2 operator=() [2/2]	224
18.16.3.3 update()	224
18.16.4 Member Data Documentation	224
18.16.4.1 m_audio	224
18.16.4.2 m_audioVolume	224
18.16.4.3 m_playSelectionMusic	224
18.16.4.4 m_registry	224
18.16.4.5 m_selectionSoundEntity	225
18.16.4.6 m_selectionSoundName	225
18.17 ecs::BeamCharge Struct Reference	225
18.17.1 Detailed Description	226
18.17.2 Member Data Documentation	226
18.17.2.1 current_charge	226
18.17.2.2 max_charge	227
18.18 ecs::BeamSystem Class Reference	227
18.18.1 Detailed Description	229
18.18.2 Constructor & Destructor Documentation	229
18.18.2.1 BeamSystem() [1/3]	229
18.18.2.2 ~BeamSystem()	229
18.18.2.3 BeamSystem() [2/3]	229
18.18.2.4 BeamSystem() [3/3]	229
18.18.3 Member Function Documentation	229
18.18.3.1 operator=() [1/2]	229
18.18.3.2 operator=() [2/2]	229
18.18.3.3 update()	229
18.18.4 Member Data Documentation	230
18.18.4.1 m_renderer	230
18.19 gme::BeamSystem Class Reference	230
18.19.1 Detailed Description	233

18.19.2 Constructor & Destructor Documentation	233
18.19.2.1 BeamSystem() [1/3]	233
18.19.2.2 ~BeamSystem()	233
18.19.2.3 BeamSystem() [2/3]	233
18.19.2.4 BeamSystem() [3/3]	233
18.19.3 Member Function Documentation	233
18.19.3.1 operator=() [1/2]	233
18.19.3.2 operator=() [2/2]	233
18.19.3.3 update()	233
18.19.4 Member Data Documentation	234
18.19.4.1 m_renderer	234
18.20 ecs::Ceiling Struct Reference	234
18.20.1 Detailed Description	235
18.21 eng::CircleShape Struct Reference	235
18.21.1 Detailed Description	236
18.21.2 Member Data Documentation	236
18.21.2.1 color	236
18.21.2.2 name	237
18.21.2.3 outline_color	237
18.21.2.4 outline_thickness	237
18.21.2.5 radius	237
18.21.2.6 x	237
18.21.2.7 y	237
18.22 cli::Client Class Reference	237
18.22.1 Detailed Description	239
18.22.2 Constructor & Destructor Documentation	239
18.22.2.1 Client() [1/3]	239
18.22.2.2 ~Client()	239
18.22.2.3 Client() [2/3]	239
18.22.2.4 Client() [3/3]	239
18.22.3 Member Function Documentation	240
18.22.3.1 handleEvents()	240
18.22.3.2 operator=() [1/2]	240
18.22.3.3 operator=() [2/2]	240
18.22.3.4 run()	240
18.22.3.5 setupConfig()	240
18.22.3.6 setupScenes()	241
18.22.3.7 stop()	241
18.22.3.8 updateKeyboardInput()	241
18.22.4 Member Data Documentation	242
18.22.4.1 m_config	242
18.22.4.2 m_engine	242
18.22.4.3 m_gameMulti	242

18.22.4.4	<code>m_gameSolo</code>	242
18.22.4.5	<code>m_keysPressed</code>	242
18.22.4.6	<code>m_pluginLoader</code>	242
18.22.4.7	<code>m_showDebug</code>	242
18.23	<code>srv::ClientSession</code> Struct Reference	243
18.23.1	Detailed Description	244
18.23.2	Constructor & Destructor Documentation	244
18.23.2.1	<code>ClientSession()</code>	244
18.23.3	Member Data Documentation	244
18.23.3.1	<code>clientCaps</code>	244
18.23.3.2	<code>currentLobbyId</code>	244
18.23.3.3	<code>endpoint</code>	244
18.23.3.4	<code>isConnected</code>	244
18.23.3.5	<code>lastPingSent</code>	244
18.23.3.6	<code>lastSeen</code>	244
18.23.3.7	<code>latency</code>	244
18.23.3.8	<code>playerName</code>	244
18.23.3.9	<code>sessionId</code>	245
18.24	<code>utl::Clock</code> Class Reference	245
18.24.1	Detailed Description	246
18.24.2	Member Typedef Documentation	246
18.24.2.1	<code>Duration</code>	246
18.24.2.2	<code>TimePoint</code>	246
18.24.3	Constructor & Destructor Documentation	246
18.24.3.1	<code>Clock()</code> [1/3]	246
18.24.3.2	<code>~Clock()</code>	246
18.24.3.3	<code>Clock()</code> [2/3]	246
18.24.3.4	<code>Clock()</code> [3/3]	247
18.24.4	Member Function Documentation	247
18.24.4.1	<code>getDeltaSeconds()</code>	247
18.24.4.2	<code>getElapsed()</code>	247
18.24.4.3	<code>now()</code>	247
18.24.4.4	<code>operator=()</code> [1/2]	248
18.24.4.5	<code>operator=()</code> [2/2]	248
18.24.4.6	<code>pause()</code>	248
18.24.4.7	<code>restart()</code>	248
18.24.4.8	<code>resume()</code>	249
18.24.5	Friends And Related Symbol Documentation	249
18.24.5.1	<code>operator<<</code>	249
18.24.6	Member Data Documentation	249
18.24.6.1	<code>m_isPaused</code>	249
18.24.6.2	<code>m_pausedDuration</code>	249
18.24.6.3	<code>m_pausedTime</code>	249

18.24.6.4 m_start	250
18.25 gme::CollisionPair Struct Reference	250
18.25.1 Detailed Description	250
18.25.2 Member Data Documentation	250
18.25.2.1 entityId1	250
18.25.2.2 entityId2	250
18.25.2.3 overlapDistance	250
18.26 gme::CollisionSystem Class Reference	251
18.26.1 Detailed Description	254
18.26.2 Constructor & Destructor Documentation	254
18.26.2.1 CollisionSystem() [1/9]	254
18.26.2.2 ~CollisionSystem() [1/3]	254
18.26.2.3 CollisionSystem() [2/9]	254
18.26.2.4 CollisionSystem() [3/9]	254
18.26.2.5 CollisionSystem() [4/9]	254
18.26.2.6 ~CollisionSystem() [2/3]	254
18.26.2.7 CollisionSystem() [5/9]	254
18.26.2.8 CollisionSystem() [6/9]	254
18.26.2.9 CollisionSystem() [7/9]	255
18.26.2.10 ~CollisionSystem() [3/3]	255
18.26.2.11 CollisionSystem() [8/9]	255
18.26.2.12 CollisionSystem() [9/9]	255
18.26.3 Member Function Documentation	255
18.26.3.1 applyDamageToEnemy()	255
18.26.3.2 applyDamageToPlayer()	256
18.26.3.3 checkCircleCollision()	256
18.26.3.4 checkCircularCollision()	257
18.26.3.5 createExplosion()	257
18.26.3.6 ensureEnemyDeathChannel()	258
18.26.3.7 ensurePlayerDeathAudio()	258
18.26.3.8 getCollisionCount()	259
18.26.3.9 getCollisionInfo()	259
18.26.3.10 handleEnemyProjectilePlayerCollision()	260
18.26.3.11 handlePlayerEnemyCollision()	260
18.26.3.12 handlePlayerPowerupCollision()	261
18.26.3.13 handlePlayerProjectileEnemyCollision()	261
18.26.3.14 isEnabled() [1/2]	261
18.26.3.15 isEnabled() [2/2]	262
18.26.3.16 operator=() [1/6]	262
18.26.3.17 operator=() [2/6]	262
18.26.3.18 operator=() [3/6]	262
18.26.3.19 operator=() [4/6]	262
18.26.3.20 operator=() [5/6]	262

18.26.3.21	operator=() [6/6]	262
18.26.3.22	playEnemyDeathSound()	262
18.26.3.23	playPlayerDeathSound()	263
18.26.3.24	removeEnemy()	263
18.26.3.25	removeProjectile()	264
18.26.3.26	resetCollisionCount()	265
18.26.3.27	setEnabled() [1/2]	265
18.26.3.28	setEnabled() [2/2]	265
18.26.3.29	update() [1/3]	265
18.26.3.30	update() [2/3]	265
18.26.3.31	update() [3/3]	265
18.26.4	Member Data Documentation	266
18.26.4.1	m_collisionCount	266
18.26.4.2	m_enemyDeathAudioEntities	266
18.26.4.3	m_entityManager	266
18.26.4.4	m_nextEnemyDeathChannel	266
18.26.4.5	m_playerDeathAudioEntity	266
18.26.4.6	m_registry	266
18.26.4.7	m_renderer	266
18.26.4.8	m_showDebug	267
18.26.4.9	m_wasPlayerPresent	267
18.27	ecs::Color Struct Reference	267
18.27.1	Detailed Description	268
18.27.2	Member Data Documentation	269
18.27.2.1	a	269
18.27.2.2	b	269
18.27.2.3	g	269
18.27.2.4	r	269
18.28	eng::Color Struct Reference	269
18.28.1	Detailed Description	269
18.28.2	Member Data Documentation	270
18.28.2.1	a	270
18.28.2.2	b	270
18.28.2.3	g	270
18.28.2.4	r	270
18.29	gme::ConfigMulti Class Reference	270
18.29.1	Detailed Description	273
18.29.2	Constructor & Destructor Documentation	273
18.29.2.1	ConfigMulti() [1/3]	273
18.29.2.2	~ConfigMulti()	273
18.29.2.3	ConfigMulti() [2/3]	273
18.29.2.4	ConfigMulti() [3/3]	273
18.29.3	Member Function Documentation	273

18.29.3.1	event()	273
18.29.3.2	operator=() [1/2]	274
18.29.3.3	operator=() [2/2]	274
18.29.3.4	playMusic()	274
18.29.3.5	update()	274
18.29.4	Member Data Documentation	274
18.29.4.1	m_animationTime	274
18.29.4.2	m_keysPressed	274
18.29.4.3	m_menuOptions	274
18.29.4.4	m_playMusic	274
18.29.4.5	m_selectedIndex	274
18.29.4.6	m_titleEntity	275
18.29.4.7	m_titlePulseTime	275
18.29.4.8	onOptionSelected	275
18.30	gme::ConfigSolo Class Reference	275
18.30.1	Detailed Description	278
18.30.2	Constructor & Destructor Documentation	278
18.30.2.1	ConfigSolo() [1/3]	278
18.30.2.2	~ConfigSolo()	278
18.30.2.3	ConfigSolo() [2/3]	278
18.30.2.4	ConfigSolo() [3/3]	278
18.30.3	Member Function Documentation	278
18.30.3.1	event()	278
18.30.3.2	operator=() [1/2]	279
18.30.3.3	operator=() [2/2]	279
18.30.3.4	playMusic()	279
18.30.3.5	update()	279
18.30.4	Member Data Documentation	279
18.30.4.1	m_animationTime	279
18.30.4.2	m_fpsEntity	279
18.30.4.3	m_keysPressed	279
18.30.4.4	m_menuOptions	279
18.30.4.5	m_playMusic	279
18.30.4.6	m_selectedIndex	280
18.30.4.7	m_titleEntity	280
18.30.4.8	m_titlePulseTime	280
18.30.4.9	onOptionSelected	280
18.31	eng::ConnectionStats Struct Reference	280
18.31.1	Detailed Description	281
18.31.2	Member Data Documentation	281
18.31.2.1	bytesTransferred	281
18.31.2.2	connectionTime	281
18.31.2.3	packetsLost	281

18.31.2.4 packetsReceived	281
18.31.2.5 packetsSent	281
18.32 gme::CreateRoomScene Class Reference	281
18.32.1 Detailed Description	284
18.32.2 Constructor & Destructor Documentation	284
18.32.2.1 CreateRoomScene() [1/3]	284
18.32.2.2 ~CreateRoomScene()	284
18.32.2.3 CreateRoomScene() [2/3]	285
18.32.2.4 CreateRoomScene() [3/3]	285
18.32.3 Member Function Documentation	285
18.32.3.1 createRoom()	285
18.32.3.2 event()	285
18.32.3.3 operator=() [1/2]	285
18.32.3.4 operator=() [2/2]	285
18.32.3.5 playMusic()	286
18.32.3.6 processEventBus()	286
18.32.3.7 update()	286
18.32.3.8 updateValueDisplay()	286
18.32.4 Member Data Documentation	286
18.32.4.1 m_animationTime	286
18.32.4.2 m_maxPlayers	286
18.32.4.3 m_maxPlayersValueEntity	286
18.32.4.4 m_options	286
18.32.4.5 m_playMusic	287
18.32.4.6 m_roomName	287
18.32.4.7 m_roomNameValueEntity	287
18.32.4.8 m_selectedIndex	287
18.32.4.9 m_titleEntity	287
18.32.4.10 onBackToMulti	287
18.32.4.11 onCreate	287
18.32.4.12 onRoomCreated	287
18.33 ecs::DebugSystem Class Reference	287
18.33.1 Detailed Description	290
18.33.2 Constructor & Destructor Documentation	290
18.33.2.1 DebugSystem() [1/3]	290
18.33.2.2 ~DebugSystem()	290
18.33.2.3 DebugSystem() [2/3]	290
18.33.2.4 DebugSystem() [3/3]	290
18.33.3 Member Function Documentation	290
18.33.3.1 operator=() [1/2]	290
18.33.3.2 operator=() [2/2]	290
18.33.3.3 update()	290
18.33.4 Member Data Documentation	291

18.33.4.1	<code>m_renderer</code>	291
18.33.4.2	<code>m_showDebug</code>	291
18.34	<code>rnpp::EndianUtils</code> Class Reference	291
18.34.1	Detailed Description	292
18.34.2	Member Function Documentation	292
18.34.2.1	<code>hostToNetwork16()</code>	292
18.34.2.2	<code>hostToNetwork32()</code>	293
18.34.2.3	<code>hostToNetworkFloat()</code>	294
18.34.2.4	<code>networkToHost16()</code>	295
18.34.2.5	<code>networkToHost32()</code>	296
18.34.2.6	<code>networkToHostFloat()</code>	297
18.35	<code>ecs::Enemy</code> Struct Reference	297
18.35.1	Detailed Description	300
18.35.2	Member Data Documentation	300
18.35.2.1	<code>damage</code>	300
18.35.2.2	<code>health</code>	300
18.35.2.3	<code>last_shot_time</code>	300
18.35.2.4	<code>max_health</code>	300
18.35.2.5	<code>shoot_cooldown</code>	300
18.35.2.6	<code>speed</code>	300
18.36	<code>gme::EnemyAISystem</code> Class Reference	300
18.36.1	Detailed Description	303
18.36.2	Constructor & Destructor Documentation	303
18.36.2.1	<code>EnemyAISystem()</code> [1/3]	303
18.36.2.2	<code>~EnemyAISystem()</code>	304
18.36.2.3	<code>EnemyAISystem()</code> [2/3]	304
18.36.2.4	<code>EnemyAISystem()</code> [3/3]	304
18.36.3	Member Function Documentation	304
18.36.3.1	<code>applyAggressiveMovement()</code>	304
18.36.3.2	<code>applySineWaveMovement()</code>	304
18.36.3.3	<code>applyZigzagMovement()</code>	305
18.36.3.4	<code>canShoot()</code>	305
18.36.3.5	<code>clampToScreen()</code>	306
18.36.3.6	<code>findNearestPlayer()</code>	306
18.36.3.7	<code>getAggressiveness()</code>	307
18.36.3.8	<code>getDistanceToNearestPlayer()</code>	307
18.36.3.9	<code>operator=()</code> [1/2]	308
18.36.3.10	<code>operator=()</code> [2/2]	308
18.36.3.11	<code>setAggressiveness()</code>	308
18.36.3.12	<code>tryShoot()</code>	308
18.36.3.13	<code>update()</code>	309
18.36.3.14	<code>updateAdvancedEnemyAI()</code>	309
18.36.3.15	<code>updateBasicEnemyAI()</code>	310

18.36.3.16	updateBossAI()	311
18.36.4	Member Data Documentation	311
18.36.4.1	m_aggressiveness	311
18.36.4.2	m_bossSpreadTimers	312
18.36.4.3	m_enemyMovementTimers	312
18.36.4.4	m_entityManager	312
18.36.4.5	m_registry	312
18.36.4.6	m_rng	312
18.36.4.7	m_shootChance	312
18.37	gme::EnemySpawn Struct Reference	312
18.37.1	Detailed Description	313
18.37.2	Member Data Documentation	313
18.37.2.1	health	313
18.37.2.2	spawnDelay	313
18.37.2.3	type	313
18.37.2.4	x	313
18.37.2.5	y	314
18.38	gme::EnemySpawnSystem Class Reference	314
18.38.1	Detailed Description	317
18.38.2	Constructor & Destructor Documentation	317
18.38.2.1	EnemySpawnSystem() [1/3]	317
18.38.2.2	~EnemySpawnSystem()	318
18.38.2.3	EnemySpawnSystem() [2/3]	318
18.38.2.4	EnemySpawnSystem() [3/3]	318
18.38.3	Member Function Documentation	318
18.38.3.1	getCurrentWaveIndex()	318
18.38.3.2	getLevelTime()	318
18.38.3.3	getRandomY()	318
18.38.3.4	getSpawnX()	318
18.38.3.5	getTotalEnemiesSpawned()	319
18.38.3.6	initializeWaves()	319
18.38.3.7	isEnabled()	319
18.38.3.8	operator=() [1/2]	319
18.38.3.9	operator=() [2/2]	319
18.38.3.10	processWave()	319
18.38.3.11	reset()	320
18.38.3.12	setEnabled()	320
18.38.3.13	update()	320
18.38.4	Member Data Documentation	321
18.38.4.1	m_currentWaveIndex	321
18.38.4.2	m_enabled	321
18.38.4.3	m_entityManager	321
18.38.4.4	m_levelTime	321

18.38.4.5	m_rng	321
18.38.4.6	m_totalEnemiesSpawned	321
18.38.4.7	m_waves	321
18.38.4.8	m_waveSpawnTimer	321
18.38.4.9	m_yDistribution	322
18.39	gme::EnemySystem Class Reference	322
18.39.1	Detailed Description	324
18.39.2	Constructor & Destructor Documentation	324
18.39.2.1	EnemySystem() [1/3]	324
18.39.2.2	~EnemySystem()	324
18.39.2.3	EnemySystem() [2/3]	324
18.39.2.4	EnemySystem() [3/3]	324
18.39.3	Member Function Documentation	324
18.39.3.1	isEnabled()	324
18.39.3.2	operator=() [1/2]	324
18.39.3.3	operator=() [2/2]	324
18.39.3.4	setEnabled()	324
18.39.3.5	update()	325
18.39.4	Member Data Documentation	325
18.39.4.1	m_renderer	325
18.40	eng::Engine Class Reference	325
18.40.1	Detailed Description	327
18.40.2	Constructor & Destructor Documentation	327
18.40.2.1	Engine() [1/3]	327
18.40.2.2	~Engine()	327
18.40.2.3	Engine() [2/3]	327
18.40.2.4	Engine() [3/3]	327
18.40.3	Member Function Documentation	327
18.40.3.1	getAudio()	327
18.40.3.2	getClock()	328
18.40.3.3	getNetwork()	328
18.40.3.4	getRenderer()	328
18.40.3.5	getSceneManager()	328
18.40.3.6	getState()	328
18.40.3.7	operator=() [1/2]	329
18.40.3.8	operator=() [2/2]	329
18.40.3.9	render()	329
18.40.3.10	setState()	329
18.40.3.11	stop()	329
18.40.4	Member Data Documentation	329
18.40.4.1	m_audio	329
18.40.4.2	m_clock	329
18.40.4.3	m_network	329

18.40.4.4	<code>m_renderer</code>	330
18.40.4.5	<code>m_sceneManager</code>	330
18.40.4.6	<code>m_state</code>	330
18.41	<code>ecs::Registry::EntityBuilder</code> Class Reference	330
18.41.1	Detailed Description	332
18.41.2	Constructor & Destructor Documentation	332
18.41.2.1	<code>EntityBuilder()</code>	332
18.41.3	Member Function Documentation	332
18.41.3.1	<code>build()</code>	332
18.41.3.2	<code>with()</code>	332
18.41.4	Member Data Documentation	333
18.41.4.1	<code>m_entity</code>	333
18.41.4.2	<code>m_registry</code>	334
18.42	<code>gme::EntityManager</code> Class Reference	334
18.42.1	Detailed Description	335
18.42.2	Constructor & Destructor Documentation	336
18.42.2.1	<code>EntityManager()</code> [1/3]	336
18.42.2.2	<code>~EntityManager()</code>	336
18.42.2.3	<code>EntityManager()</code> [2/3]	336
18.42.2.4	<code>EntityManager()</code> [3/3]	336
18.42.3	Member Function Documentation	336
18.42.3.1	<code>cleanupDestroyedEntities()</code>	336
18.42.3.2	<code>clear()</code>	336
18.42.3.3	<code>createAdvancedEnemy()</code>	337
18.42.3.4	<code>createBasicEnemy()</code>	337
18.42.3.5	<code>createBoss()</code>	338
18.42.3.6	<code>createEnemyProjectile()</code>	338
18.42.3.7	<code>createPlayer()</code>	339
18.42.3.8	<code>createPlayerProjectile()</code>	339
18.42.3.9	<code>createPowerup()</code>	340
18.42.3.10	<code>destroyEnemy()</code>	340
18.42.3.11	<code>destroyPlayer()</code>	341
18.42.3.12	<code>destroyPowerup()</code>	341
18.42.3.13	<code>destroyProjectile()</code>	341
18.42.3.14	<code>getAllEntityStates()</code>	342
18.42.3.15	<code>getEnemies()</code>	342
18.42.3.16	<code>getEnemy()</code>	343
18.42.3.17	<code>getEntityMetadata()</code> [1/2]	343
18.42.3.18	<code>getEntityMetadata()</code> [2/2]	344
18.42.3.19	<code>getNetworkEntityType()</code>	344
18.42.3.20	<code>getNetworkIdForEntity()</code>	344
18.42.3.21	<code>getPlayer()</code>	344
18.42.3.22	<code>getPlayers()</code>	345

18.42.3.23	getProjectile()	345
18.42.3.24	getProjectiles()	345
18.42.3.25	getTotalEntityCount()	345
18.42.3.26	hasPlayer()	346
18.42.3.27	operator=() [1/2]	346
18.42.3.28	operator=() [2/2]	346
18.42.3.29	processDestroyQueue()	346
18.42.3.30	registerEntity()	347
18.42.3.31	unregisterEntity()	347
18.42.3.32	updateLifetimes()	348
18.42.4	Member Data Documentation	348
18.42.4.1	m_destroyQueue	348
18.42.4.2	m_enemyEntities	348
18.42.4.3	m_entityMetadata	348
18.42.4.4	m_entityToNetworkId	348
18.42.4.5	m_nextEnemyId	348
18.42.4.6	m_nextPowerupId	348
18.42.4.7	m_nextProjectileId	349
18.42.4.8	m_playerEntities	349
18.42.4.9	m_powerupEntities	349
18.42.4.10	m_projectileEntities	349
18.42.4.11	m_registry	349
18.43	gme::EntityMetadata Struct Reference	349
18.43.1	Detailed Description	350
18.43.2	Member Data Documentation	350
18.43.2.1	currentLife	350
18.43.2.2	entity	350
18.43.2.3	isActive	350
18.43.2.4	lifetime	351
18.43.2.5	networkId	351
18.43.2.6	ownerId	351
18.43.2.7	type	351
18.44	rnpp::EntityState Struct Reference	351
18.44.1	Detailed Description	352
18.44.2	Member Data Documentation	352
18.44.2.1	healthPercent	352
18.44.2.2	id	352
18.44.2.3	stateFlags	352
18.44.2.4	subtype	352
18.44.2.5	type	352
18.44.2.6	vx	352
18.44.2.7	vy	353
18.44.2.8	x	353

18.44.2.9 y	353
18.45 cli::EnvConfig Struct Reference	353
18.45.1 Detailed Description	353
18.46 srv::EnvConfig Struct Reference	353
18.46.1 Detailed Description	354
18.47 eng::Event Struct Reference	354
18.47.1 Detailed Description	354
18.47.2 Member Data Documentation	354
18.47.2.1 key	354
18.47.2.2 type	355
18.48 utl::Event Class Reference	355
18.48.1 Detailed Description	356
18.48.2 Constructor & Destructor Documentation	356
18.48.2.1 Event() [1/4]	356
18.48.2.2 Event() [2/4]	356
18.48.2.3 Event() [3/4]	356
18.48.2.4 Event() [4/4]	357
18.48.3 Member Function Documentation	357
18.48.3.1 getAge()	357
18.48.3.2 hasExpired()	357
18.48.3.3 isBroadcast()	358
18.48.3.4 isTargeted()	358
18.48.4 Member Data Documentation	358
18.48.4.1 data	358
18.48.4.2 priority	359
18.48.4.3 sourceId	359
18.48.4.4 targetId	359
18.48.4.5 timestamp	359
18.48.4.6 type	359
18.49 utl::EventBus Class Reference	359
18.49.1 Detailed Description	362
18.49.2 Constructor & Destructor Documentation	362
18.49.2.1 EventBus() [1/3]	362
18.49.2.2 ~EventBus()	362
18.49.2.3 EventBus() [2/3]	362
18.49.2.4 EventBus() [3/3]	362
18.49.3 Member Function Documentation	362
18.49.3.1 clear()	362
18.49.3.2 clearStats()	363
18.49.3.3 consume()	363
18.49.3.4 consumeForTarget()	364
18.49.3.5 consumeType()	364
18.49.3.6 filterEventType()	365

18.49.3.7	getInstance()	365
18.49.3.8	getQueueSize()	366
18.49.3.9	getStats()	366
18.49.3.10	isRunning()	367
18.49.3.11	operator=() [1/2]	367
18.49.3.12	operator=() [2/2]	367
18.49.3.13	publish() [1/2]	367
18.49.3.14	publish() [2/2]	368
18.49.3.15	registerComponent()	369
18.49.3.16	setMaxQueueSize()	370
18.49.3.17	stop()	370
18.49.3.18	subscribe()	370
18.49.3.19	unfilterEventType()	371
18.49.3.20	unregisterComponent()	372
18.49.3.21	unsubscribe()	372
18.49.3.22	updateStats()	373
18.49.3.23	waitForEvents()	373
18.49.4	Member Data Documentation	374
18.49.4.1	m_componentMutex	374
18.49.4.2	m_componentNames	374
18.49.4.3	m_eventQueue	374
18.49.4.4	m_filteredTypes	374
18.49.4.5	m_filterMutex	374
18.49.4.6	m_maxQueueSize	374
18.49.4.7	m_priorityQueue	374
18.49.4.8	m_running	374
18.49.4.9	m_stats	375
18.49.4.10	m_statsMutex	375
18.49.4.11	m_subscribers	375
18.49.4.12	m_subscribersMutex	375
18.50	utl::EventConnect Struct Reference	375
18.50.1	Detailed Description	376
18.50.2	Member Data Documentation	376
18.50.2.1	playerName	376
18.50.2.2	serverIP	376
18.50.2.3	serverPort	376
18.51	utl::EventBus::EventQueue Struct Reference	377
18.51.1	Detailed Description	377
18.51.2	Member Data Documentation	378
18.51.2.1	cv	378
18.51.2.2	events	378
18.51.2.3	mutex	378
18.51.2.4	totalEvents	378

18.52 rnp::EventRecord Struct Reference	378
18.52.1 Detailed Description	379
18.52.2 Member Data Documentation	379
18.52.2.1 data	379
18.52.2.2 entityId	379
18.52.2.3 type	379
18.53 utl::EventStats Class Reference	379
18.53.1 Detailed Description	381
18.53.2 Member Function Documentation	381
18.53.2.1 getEventsPerSecond()	381
18.53.2.2 reset()	381
18.53.3 Member Data Documentation	381
18.53.3.1 currentQueueSize	381
18.53.3.2 eventTypeCount	381
18.53.3.3 maxQueueSize	381
18.53.3.4 priorityCount	382
18.53.3.5 sourceCount	382
18.53.3.6 startTime	382
18.53.3.7 targetCount	382
18.53.3.8 totalEventsConsumed	382
18.53.3.9 totalEventsExpired	382
18.53.3.10 totalEventsPublished	382
18.54 ecs::Explosion Struct Reference	382
18.54.1 Detailed Description	385
18.54.2 Member Data Documentation	385
18.54.2.1 current_frame	385
18.54.2.2 current_lifetime	385
18.54.2.3 current_time	385
18.54.2.4 frame_duration	385
18.54.2.5 frame_height	385
18.54.2.6 frame_width	385
18.54.2.7 frames_per_row	385
18.54.2.8 lifetime	385
18.54.2.9 total_frames	385
18.55 gme::ExplosionSystem Class Reference	386
18.55.1 Detailed Description	388
18.55.2 Constructor & Destructor Documentation	388
18.55.2.1 ExplosionSystem() [1/6]	388
18.55.2.2 ~ExplosionSystem() [1/2]	388
18.55.2.3 ExplosionSystem() [2/6]	388
18.55.2.4 ExplosionSystem() [3/6]	388
18.55.2.5 ExplosionSystem() [4/6]	388
18.55.2.6 ~ExplosionSystem() [2/2]	388

18.55.2.7 ExplosionSystem() [5/6]	389
18.55.2.8 ExplosionSystem() [6/6]	389
18.55.3 Member Function Documentation	389
18.55.3.1 isEnabled() [1/2]	389
18.55.3.2 isEnabled() [2/2]	389
18.55.3.3 operator=() [1/4]	389
18.55.3.4 operator=() [2/4]	389
18.55.3.5 operator=() [3/4]	389
18.55.3.6 operator=() [4/4]	389
18.55.3.7 removeExplosion() [1/2]	389
18.55.3.8 removeExplosion() [2/2]	390
18.55.3.9 setEnable() [1/2]	390
18.55.3.10 setEnable() [2/2]	390
18.55.3.11 update() [1/2]	390
18.55.3.12 update() [2/2]	391
18.55.4 Member Data Documentation	391
18.55.4.1 m_renderer	391
18.56 gme::FlappyBirdClientSolo Class Reference	391
18.56.1 Detailed Description	394
18.56.2 Constructor & Destructor Documentation	394
18.56.2.1 FlappyBirdClientSolo() [1/3]	394
18.56.2.2 ~FlappyBirdClientSolo()	394
18.56.2.3 FlappyBirdClientSolo() [2/3]	394
18.56.2.4 FlappyBirdClientSolo() [3/3]	394
18.56.3 Member Function Documentation	394
18.56.3.1 getMainSceneId()	394
18.56.3.2 getName()	394
18.56.3.3 getType()	395
18.56.3.4 init()	395
18.56.3.5 operator=() [1/2]	395
18.56.3.6 operator=() [2/2]	395
18.56.3.7 setupScenes()	395
18.56.3.8 update()	396
18.56.4 Member Data Documentation	396
18.56.4.1 m_appConfig	396
18.56.4.2 m_engine	396
18.56.4.3 m_mainSceneId	396
18.57 ecs::Floor Struct Reference	396
18.57.1 Detailed Description	397
18.58 ecs::Font Struct Reference	397
18.58.1 Detailed Description	399
18.58.2 Member Data Documentation	399
18.58.2.1 path	399

18.59 gme::Game Class Reference	399
18.59.1 Detailed Description	402
18.59.2 Constructor & Destructor Documentation	402
18.59.2.1 Game() [1/3]	402
18.59.2.2 ~Game()	403
18.59.2.3 Game() [2/3]	403
18.59.2.4 Game() [3/3]	403
18.59.3 Member Function Documentation	403
18.59.3.1 createPipePair()	403
18.59.3.2 createPlayer()	403
18.59.3.3 event()	404
18.59.3.4 operator=() [1/2]	404
18.59.3.5 operator=() [2/2]	404
18.59.3.6 playMusic()	404
18.59.3.7 resetGame()	404
18.59.3.8 update()	405
18.59.4 Member Data Documentation	405
18.59.4.1 m_audio	405
18.59.4.2 m_flapSound	405
18.59.4.3 m_gameOver	405
18.59.4.4 m_gameOverShown	405
18.59.4.5 m_keysPressed	405
18.59.4.6 m_looseSound	405
18.59.4.7 m_looseText	406
18.59.4.8 m_pipes	406
18.59.4.9 m_playerEntity	406
18.59.4.10 m_playMusic	406
18.59.4.11 m_renderer	406
18.59.4.12 m_showDebug	406
18.60 gme::GameMulti Class Reference	406
18.60.1 Detailed Description	410
18.60.2 Constructor & Destructor Documentation	410
18.60.2.1 GameMulti() [1/3]	410
18.60.2.2 ~GameMulti()	410
18.60.2.3 GameMulti() [2/3]	410
18.60.2.4 GameMulti() [3/3]	410
18.60.3 Member Function Documentation	411
18.60.3.1 event()	411
18.60.3.2 handleWorldStateUpdate()	411
18.60.3.3 operator=() [1/2]	411
18.60.3.4 operator=() [2/2]	411
18.60.3.5 playMusic()	411
18.60.3.6 preloadCommonTextures()	411

18.60.3.7 processEventBus()	412
18.60.3.8 setupEventSubscriptions()	412
18.60.3.9 update()	413
18.60.3.10 updateInterpolation() [1/2]	413
18.60.3.11 updateInterpolation() [2/2]	413
18.60.3.12 updatePlayerSkin()	413
18.60.4 Member Data Documentation	413
18.60.4.1 BOSS_MUSIC_DURATION	413
18.60.4.2 ENEMY_SMOOTH_FACTOR	413
18.60.4.3 m_audio	413
18.60.4.4 m_beginSoundEntity	414
18.60.4.5 m_bossMusicEntity	414
18.60.4.6 m_bossMusicStarted	414
18.60.4.7 m_bossMusicTimer	414
18.60.4.8 m_enemyData	414
18.60.4.9 m_enemyEntities	414
18.60.4.10 m_eventComponentId	414
18.60.4.11 m_firstWorldState	414
18.60.4.12 m_inputHistory	414
18.60.4.13 m_keysPressed	414
18.60.4.14 m_lastAckSeqId	414
18.60.4.15 m_loadedFonts	414
18.60.4.16 m_loadedTextures	415
18.60.4.17 m_localPlayerEntity	415
18.60.4.18 m_playerController	415
18.60.4.19 m_playerSkinMap	415
18.60.4.20 m_playMusic	415
18.60.4.21 m_projectileData	415
18.60.4.22 m_projectileEntities	415
18.60.4.23 m_remotePlayerData	415
18.60.4.24 m_remotePlayers	415
18.60.4.25 m_renderer	415
18.60.4.26 m_sessionId	415
18.60.4.27 m_showDebug	416
18.60.4.28 m_skinIndex	416
18.60.4.29 m_stageManager	416
18.60.4.30 m_starfieldCreated	416
18.60.4.31 PROJECTILE_SMOOTH_FACTOR	416
18.60.4.32 REMOTE_PLAYER_SMOOTH_FACTOR	416
18.61 gme::GameSolo Class Reference	416
18.61.1 Detailed Description	419
18.61.2 Constructor & Destructor Documentation	419
18.61.2.1 GameSolo() [1/3]	419

18.61.2.2	~GameSolo()	420
18.61.2.3	GameSolo() [2/3]	420
18.61.2.4	GameSolo() [3/3]	420
18.61.3	Member Function Documentation	420
18.61.3.1	createPlayer()	420
18.61.3.2	event()	420
18.61.3.3	handlePlayerInputs()	420
18.61.3.4	operator=() [1/2]	421
18.61.3.5	operator=() [2/2]	421
18.61.3.6	playMusic()	421
18.61.3.7	preloadCommonTextures()	421
18.61.3.8	update()	421
18.61.3.9	updatePlayerSkin()	422
18.61.4	Member Data Documentation	422
18.61.4.1	m_audio	422
18.61.4.2	m_keysPressed	422
18.61.4.3	m_lastAppliedSkinIndex	422
18.61.4.4	m_loadedFonts	422
18.61.4.5	m_loadedTextures	422
18.61.4.6	m_playerEntity	422
18.61.4.7	m_playmusic	422
18.61.4.8	m_renderer	422
18.61.4.9	m_showDebug	422
18.61.4.10	m_skinIndex	422
18.61.4.11	m_stageManager	423
18.62	rnp::HandlerPacket Class Reference	423
18.62.1	Detailed Description	426
18.62.2	Constructor & Destructor Documentation	426
18.62.2.1	HandlerPacket() [1/2]	426
18.62.2.2	~HandlerPacket()	426
18.62.2.3	HandlerPacket() [2/2]	426
18.62.3	Member Function Documentation	427
18.62.3.1	clearAllSessions()	427
18.62.3.2	clearSession()	427
18.62.3.3	getStats()	427
18.62.3.4	isRateLimited()	427
18.62.3.5	onConnect()	428
18.62.3.6	onConnectAccept()	428
18.62.3.7	onDisconnect()	429
18.62.3.8	onEntityEvent()	429
18.62.3.9	onError()	430
18.62.3.10	onGameStart()	430
18.62.3.11	onLobbyCreate()	431

18.62.3.12	onLobbyCreateResponse()	431
18.62.3.13	onLobbyJoin()	431
18.62.3.14	onLobbyJoinResponse()	432
18.62.3.15	onLobbyLeave()	432
18.62.3.16	onLobbyListRequest()	433
18.62.3.17	onLobbyListResponse()	433
18.62.3.18	onLobbyUpdate()	433
18.62.3.19	onPing()	434
18.62.3.20	onPong()	434
18.62.3.21	onStartGameRequest()	435
18.62.3.22	onWorldState()	435
18.62.3.23	operator=()	436
18.62.3.24	parseEntityEvents()	436
18.62.3.25	processPacket()	436
18.62.3.26	resetStats()	438
18.62.3.27	setRateLimitInterval()	438
18.62.3.28	updateStats()	438
18.62.4	Member Data Documentation	439
18.62.4.1	ackHandler_	439
18.62.4.2	connectAcceptHandler_	439
18.62.4.3	connectHandler_	439
18.62.4.4	disconnectHandler_	439
18.62.4.5	entityEventHandler_	439
18.62.4.6	errorHandler_	439
18.62.4.7	gameStartHandler_	439
18.62.4.8	lastPacketTime_	439
18.62.4.9	lobbyCreateHandler_	440
18.62.4.10	lobbyCreateResponseHandler_	440
18.62.4.11	lobbyJoinHandler_	440
18.62.4.12	lobbyJoinResponseHandler_	440
18.62.4.13	lobbyLeaveHandler_	440
18.62.4.14	lobbyListRequestHandler_	440
18.62.4.15	lobbyListResponseHandler_	440
18.62.4.16	lobbyUpdateHandler_	440
18.62.4.17	pingHandler_	440
18.62.4.18	pongHandler_	440
18.62.4.19	rateLimitInterval_	441
18.62.4.20	startGameRequestHandler_	441
18.62.4.21	stats_	441
18.62.4.22	worldStateHandler_	441
18.63	rnp::HandlerStats Struct Reference	441
18.63.1	Detailed Description	442
18.63.2	Member Data Documentation	442

18.63.2.1 packetTypeCount	442
18.63.2.2 resultCount	442
18.63.2.3 totalBytesReceived	442
18.63.2.4 totalPacketsDropped	442
18.63.2.5 totalPacketsProcessed	442
18.63.2.6 totalPacketsReceived	442
18.64 ecs::Hitbox Struct Reference	442
18.64.1 Detailed Description	444
18.64.2 Member Data Documentation	444
18.64.2.1 offsetX	444
18.64.2.2 offsetY	445
18.64.2.3 radius	445
18.65 gme::HUDSystem Class Reference	445
18.65.1 Detailed Description	448
18.65.2 Constructor & Destructor Documentation	448
18.65.2.1 HUDSystem() [1/3]	448
18.65.2.2 ~HUDSystem()	448
18.65.2.3 HUDSystem() [2/3]	448
18.65.2.4 HUDSystem() [3/3]	448
18.65.3 Member Function Documentation	449
18.65.3.1 createScoreHUD()	449
18.65.3.2 operator=() [1/2]	449
18.65.3.3 operator=() [2/2]	449
18.65.3.4 update()	449
18.65.3.5 updateScore()	450
18.65.4 Member Data Documentation	450
18.65.4.1 m_registry	450
18.65.4.2 m_renderer	450
18.65.4.3 m_scoreBgEntity	450
18.65.4.4 m_scoreDigitEntities	450
18.66 eng::IAudio Interface Reference	450
18.66.1 Detailed Description	452
18.66.2 Member Function Documentation	452
18.66.2.1 createAudio()	452
18.66.2.2 isPlaying()	453
18.66.2.3 playAudio()	453
18.66.2.4 setLoop()	453
18.66.2.5 setVolume()	454
18.66.2.6 stopAudio()	454
18.67 ecs::IComponent Struct Reference	454
18.67.1 Detailed Description	455
18.67.2 Member Data Documentation	455
18.67.2.1 id	455

18.68 gme::IGameClient Interface Reference	455
18.68.1 Detailed Description	457
18.68.2 Member Function Documentation	457
18.68.2.1 getMainSceneId()	457
18.68.2.2 init()	457
18.68.2.3 update()	458
18.69 gme::IGameServer Interface Reference	458
18.69.1 Detailed Description	460
18.69.2 Member Function Documentation	460
18.69.2.1 getState()	460
18.69.2.2 start()	460
18.69.2.3 stop()	461
18.69.2.4 update()	461
18.70 eng::INetworkClient Interface Reference	461
18.70.1 Detailed Description	464
18.70.2 Constructor & Destructor Documentation	464
18.70.2.1 ~INetworkClient()	464
18.70.3 Member Function Documentation	464
18.70.3.1 connect()	464
18.70.3.2 disconnect()	464
18.70.3.3 getConnectionState()	464
18.70.3.4 getLatency()	464
18.70.3.5 getServerTickRate()	464
18.70.3.6 getSessionId()	464
18.70.3.7 isConnected()	464
18.70.3.8 sendToServer()	464
18.70.3.9 setClientCapabilities()	464
18.70.3.10 setPlayerName()	465
18.70.3.11 update()	465
18.71 srv::INetworkServer Interface Reference	465
18.71.1 Detailed Description	468
18.71.2 Constructor & Destructor Documentation	468
18.71.2.1 ~INetworkServer()	468
18.71.3 Member Function Documentation	468
18.71.3.1 disconnectClient()	468
18.71.3.2 getClientCount()	468
18.71.3.3 getConnectedSessions()	468
18.71.3.4 init()	468
18.71.3.5 isRunning()	468
18.71.3.6 sendToAllClients()	468
18.71.3.7 sendToClient()	468
18.71.3.8 setServerCapabilities()	469
18.71.3.9 setTickRate()	469

18.71.3.10	start()	469
18.71.3.11	stop()	469
18.71.3.12	update()	469
18.72	gme::InterpolationData Struct Reference	469
18.72.1	Detailed Description	470
18.72.2	Member Data Documentation	470
18.72.2.1	currentRotation	470
18.72.2.2	currentX	470
18.72.2.3	currentY	470
18.72.2.4	smoothFactor	470
18.72.2.5	targetRotation	470
18.72.2.6	targetVx	470
18.72.2.7	targetVy	470
18.72.2.8	targetX	470
18.72.2.9	targetY	471
18.73	cli::Intro Class Reference	471
18.73.1	Detailed Description	474
18.73.2	Constructor & Destructor Documentation	474
18.73.2.1	Intro() [1/3]	474
18.73.2.2	~Intro()	474
18.73.2.3	Intro() [2/3]	474
18.73.2.4	Intro() [3/3]	474
18.73.3	Member Function Documentation	474
18.73.3.1	event()	474
18.73.3.2	operator=() [1/2]	474
18.73.3.3	operator=() [2/2]	475
18.73.3.4	update()	475
18.73.4	Member Data Documentation	475
18.73.4.1	m_audio	475
18.73.4.2	m_clock	475
18.73.4.3	m_elapsedTime	475
18.73.4.4	m_logoEntity	475
18.73.4.5	m_renderer	475
18.73.4.6	m_soundEntity	475
18.73.4.7	onLeave	475
18.74	utl::IPlugin Interface Reference	475
18.74.1	Detailed Description	477
18.74.2	Constructor & Destructor Documentation	477
18.74.2.1	~IPlugin()	477
18.74.3	Member Function Documentation	477
18.74.3.1	getName()	477
18.74.3.2	getType()	477
18.75	ecs::Registry::IPool Class Reference	478

18.75.1 Detailed Description	478
18.75.2 Constructor & Destructor Documentation	478
18.75.2.1 ~IPool()	478
18.75.3 Member Function Documentation	479
18.75.3.1 remove()	479
18.76 eng::IRenderer Interface Reference	479
18.76.1 Detailed Description	481
18.76.2 Member Function Documentation	481
18.76.2.1 clearWindow()	481
18.76.2.2 closeWindow()	481
18.76.2.3 createCircleShape()	481
18.76.2.4 createFont()	481
18.76.2.5 createSprite()	482
18.76.2.6 createText()	482
18.76.2.7 createTexture()	482
18.76.2.8 createWindow()	482
18.76.2.9 displayWindow()	482
18.76.2.10 drawCircleShape()	482
18.76.2.11 drawPoint()	483
18.76.2.12 drawSprite()	483
18.76.2.13 drawText()	484
18.76.2.14 getWindowSize()	484
18.76.2.15 pollEvent()	485
18.76.2.16 renderGui()	485
18.76.2.17 rotateSprite()	485
18.76.2.18 setCircleShapePosition()	485
18.76.2.19 setFrameLimit()	486
18.76.2.20 setSpriteColor()	486
18.76.2.21 setSpriteFrame()	486
18.76.2.22 setSpriteOrigin()	486
18.76.2.23 setSpritePosition()	486
18.76.2.24 setSpriteRotation()	487
18.76.2.25 setSpriteScale()	487
18.76.2.26 setSpriteTexture()	487
18.76.2.27 setTextColor()	488
18.76.2.28 setTextContent()	488
18.76.2.29 setTextPosition()	488
18.76.2.30 setWindowIcon()	489
18.76.2.31 windowIsOpen()	489
18.77 eng::IScene Interface Reference	489
18.77.1 Detailed Description	490
18.77.2 Constructor & Destructor Documentation	490
18.77.2.1 ~IScene()	490

18.77.3 Member Function Documentation	490
18.77.3.1 addSystem()	490
18.77.3.2 event()	491
18.77.3.3 getId()	491
18.77.3.4 getName()	491
18.77.3.5 getRegistry()	491
18.77.3.6 setName()	491
18.77.3.7 update()	491
18.77.3.8 updateSystems()	491
18.78 srv::IScene Class Reference	491
18.78.1 Detailed Description	493
18.78.2 Constructor & Destructor Documentation	493
18.78.2.1 ~IScene()	493
18.78.3 Member Function Documentation	493
18.78.3.1 getId()	493
18.78.3.2 getName()	493
18.78.3.3 getRegistry()	493
18.78.3.4 setName()	494
18.78.3.5 update()	494
18.79 ecs::ISystem Interface Reference	494
18.79.1 Detailed Description	494
18.79.2 Constructor & Destructor Documentation	495
18.79.2.1 ~ISystem()	495
18.79.3 Member Function Documentation	495
18.79.3.1 isEnabled()	495
18.79.3.2 setEnable()	495
18.79.3.3 update()	495
18.80 gme::JoinRoomScene Class Reference	495
18.80.1 Detailed Description	498
18.80.2 Constructor & Destructor Documentation	498
18.80.2.1 JoinRoomScene() [1/3]	498
18.80.2.2 ~JoinRoomScene()	499
18.80.2.3 JoinRoomScene() [2/3]	499
18.80.2.4 JoinRoomScene() [3/3]	499
18.80.3 Member Function Documentation	499
18.80.3.1 clearRoomEntities()	499
18.80.3.2 event()	499
18.80.3.3 handleLobbyJoinResponse()	499
18.80.3.4 handleLobbyListResponse()	499
18.80.3.5 operator=() [1/2]	500
18.80.3.6 operator=() [2/2]	500
18.80.3.7 playMusic()	500
18.80.3.8 processEventBus()	500

18.80.3.9	refreshRoomList()	500
18.80.3.10	setRooms()	500
18.80.3.11	setupEventSubscriptions()	500
18.80.3.12	update()	501
18.80.3.13	updateRoomDisplay()	501
18.80.4	Member Data Documentation	501
18.80.4.1	m_animationTime	501
18.80.4.2	m_noRoomsEntity	501
18.80.4.3	m_playMusic	501
18.80.4.4	m_roomEntities	502
18.80.4.5	m_rooms	502
18.80.4.6	m_selectedIndex	502
18.80.4.7	onBackToMulti	502
18.80.4.8	onJoin	502
18.80.4.9	onRefreshRequest	502
18.81	ecs::KeyboardInput Struct Reference	502
18.81.1	Detailed Description	504
18.81.2	Member Data Documentation	505
18.81.2.1	down_pressed	505
18.81.2.2	left_pressed	505
18.81.2.3	right_pressed	505
18.81.2.4	space_pressed	505
18.81.2.5	up_pressed	505
18.82	ecs::Layer Struct Reference	505
18.82.1	Detailed Description	506
18.82.2	Member Data Documentation	506
18.82.2.1	layer	506
18.83	ecs::LoadingAnimation Struct Reference	507
18.83.1	Detailed Description	509
18.83.2	Member Data Documentation	509
18.83.2.1	current_frame	509
18.83.2.2	current_time	509
18.83.2.3	frame_duration	509
18.83.2.4	frame_height	509
18.83.2.5	frame_width	509
18.83.2.6	frames_per_row	509
18.83.2.7	total_frames	509
18.84	gme::LoadingAnimationSystem Class Reference	509
18.84.1	Detailed Description	512
18.84.2	Constructor & Destructor Documentation	512
18.84.2.1	LoadingAnimationSystem() [1/6]	512
18.84.2.2	~LoadingAnimationSystem() [1/2]	512
18.84.2.3	LoadingAnimationSystem() [2/6]	512

18.84.2.4 LoadingAnimationSystem() [3/6]	512
18.84.2.5 LoadingAnimationSystem() [4/6]	512
18.84.2.6 ~LoadingAnimationSystem() [2/2]	512
18.84.2.7 LoadingAnimationSystem() [5/6]	512
18.84.2.8 LoadingAnimationSystem() [6/6]	513
18.84.3 Member Function Documentation	513
18.84.3.1 operator=() [1/4]	513
18.84.3.2 operator=() [2/4]	513
18.84.3.3 operator=() [3/4]	513
18.84.3.4 operator=() [4/4]	513
18.84.3.5 update() [1/2]	513
18.84.3.6 update() [2/2]	513
18.84.4 Member Data Documentation	514
18.84.4.1 m_renderer	514
18.85 srv::Lobby Struct Reference	514
18.85.1 Detailed Description	515
18.85.2 Constructor & Destructor Documentation	516
18.85.2.1 Lobby() [1/2]	516
18.85.2.2 Lobby() [2/2]	516
18.85.3 Member Data Documentation	516
18.85.3.1 createdTime	516
18.85.3.2 gameMode	516
18.85.3.3 hostSessionId	516
18.85.3.4 lobbyId	516
18.85.3.5 lobbyName	516
18.85.3.6 maxPlayers	516
18.85.3.7 playerSessions	516
18.85.3.8 status	517
18.86 rnp::LobbyInfo Struct Reference	517
18.86.1 Detailed Description	518
18.86.2 Member Data Documentation	518
18.86.2.1 currentPlayers	518
18.86.2.2 gameMode	518
18.86.2.3 hostSessionId	518
18.86.2.4 lobbyId	518
18.86.2.5 lobbyName	518
18.86.2.6 maxPlayers	518
18.86.2.7 playerNames	518
18.86.2.8 status	519
18.87 utl::Logger Class Reference	519
18.87.1 Detailed Description	520
18.87.2 Member Enumeration Documentation	520
18.87.2.1 ColorIndex	520

18.87.3 Constructor & Destructor Documentation	520
18.87.3.1 Logger() [1/3]	520
18.87.3.2 Logger() [2/3]	520
18.87.3.3 Logger() [3/3]	520
18.87.3.4 ~Logger()	521
18.87.4 Member Function Documentation	521
18.87.4.1 formatLogMessage()	521
18.87.4.2 getColorForDuration()	522
18.87.4.3 init()	523
18.87.4.4 log()	523
18.87.4.5 logExecutionTime()	524
18.87.4.6 operator=() [1/2]	524
18.87.4.7 operator=() [2/2]	524
18.87.5 Member Data Documentation	525
18.87.5.1 LOG_LEVEL_COLOR	525
18.87.5.2 LOG_LEVEL_STRING	525
18.88 cli::Menu Class Reference	525
18.88.1 Detailed Description	528
18.88.2 Constructor & Destructor Documentation	528
18.88.2.1 Menu() [1/3]	528
18.88.2.2 ~Menu()	528
18.88.2.3 Menu() [2/3]	528
18.88.2.4 Menu() [3/3]	528
18.88.3 Member Function Documentation	529
18.88.3.1 event()	529
18.88.3.2 operator=() [1/2]	529
18.88.3.3 operator=() [2/2]	529
18.88.3.4 playMusic()	529
18.88.3.5 update()	529
18.88.4 Member Data Documentation	529
18.88.4.1 m_animationTime	529
18.88.4.2 m_contributors	529
18.88.4.3 m_contributorsEntity	529
18.88.4.4 m_contributorsOffset	529
18.88.4.5 m_keysPressed	530
18.88.4.6 m_menuOptions	530
18.88.4.7 m_playMusic	530
18.88.4.8 m_selectedIndex	530
18.88.4.9 m_titleEntity	530
18.88.4.10 m_titlePulseTime	530
18.88.4.11 onOptionSelected	530
18.89 rnp::PacketConnect Struct Reference	530
18.89.1 Detailed Description	531

18.89.2 Member Data Documentation	531
18.89.2.1 clientCaps	531
18.89.2.2 nameLen	531
18.89.2.3 playerName	531
18.90 rnp::PacketConnectAccept Struct Reference	532
18.90.1 Detailed Description	532
18.90.2 Member Data Documentation	532
18.90.2.1 mtuPayloadBytes	532
18.90.2.2 serverCaps	532
18.90.2.3 sessionId	532
18.90.2.4 tickRateHz	532
18.91 rnp::PacketContext Struct Reference	533
18.91.1 Detailed Description	533
18.91.2 Member Data Documentation	534
18.91.2.1 isCompressed	534
18.91.2.2 isReliable	534
18.91.2.3 receiveTime	534
18.91.2.4 senderAddress	534
18.91.2.5 senderPort	534
18.91.2.6 sessionId	534
18.92 rnp::PacketDisconnect Struct Reference	534
18.92.1 Detailed Description	535
18.92.2 Member Data Documentation	535
18.92.2.1 reasonCode	535
18.93 rnp::PacketError Struct Reference	535
18.93.1 Detailed Description	536
18.93.2 Member Data Documentation	536
18.93.2.1 description	536
18.93.2.2 errorCode	536
18.93.2.3 msgLen	537
18.94 rnp::PacketGameStart Struct Reference	537
18.94.1 Detailed Description	537
18.94.2 Member Data Documentation	537
18.94.2.1 lobbyId	537
18.95 rnp::PacketHeader Struct Reference	537
18.95.1 Detailed Description	538
18.95.2 Member Data Documentation	538
18.95.2.1 length	538
18.95.2.2 sessionId	538
18.95.2.3 type	538
18.96 rnp::PacketLobbyCreate Struct Reference	539
18.96.1 Detailed Description	539
18.96.2 Member Data Documentation	539

18.96.2.1	gameMode	539
18.96.2.2	lobbyName	540
18.96.2.3	maxPlayers	540
18.96.2.4	nameLen	540
18.97	rnp::PacketLobbyCreateResponse Struct Reference	540
18.97.1	Detailed Description	540
18.97.2	Member Data Documentation	541
18.97.2.1	errorCode	541
18.97.2.2	lobbyId	541
18.97.2.3	success	541
18.98	rnp::PacketLobbyJoin Struct Reference	541
18.98.1	Detailed Description	541
18.98.2	Member Data Documentation	541
18.98.2.1	lobbyId	541
18.99	rnp::PacketLobbyJoinResponse Struct Reference	542
18.99.1	Detailed Description	543
18.99.2	Member Data Documentation	543
18.99.2.1	errorCode	543
18.99.2.2	lobbyId	543
18.99.2.3	lobbyInfo	543
18.99.2.4	success	543
18.100	rnp::PacketLobbyListResponse Struct Reference	543
18.100.1	Detailed Description	544
18.100.2	Member Data Documentation	545
18.100.2.1	lobbies	545
18.100.2.2	lobbyCount	545
18.101	rnp::PacketLobbyUpdate Struct Reference	545
18.101.1	Detailed Description	546
18.101.2	Member Data Documentation	546
18.101.2.1	lobbyInfo	546
18.102	rnp::PacketPingPong Struct Reference	547
18.102.1	Detailed Description	547
18.102.2	Member Data Documentation	547
18.102.2.1	nonce	547
18.102.2.2	sendTimeMs	547
18.103	rnp::PacketStartGameRequest Struct Reference	547
18.103.1	Detailed Description	548
18.103.2	Member Data Documentation	548
18.103.2.1	lobbyId	548
18.104	rnp::PacketWorldState Struct Reference	548
18.104.1	Detailed Description	549
18.104.2	Member Data Documentation	549
18.104.2.1	entities	549

18.104.2.2 entityCount	550
18.104.2.3 serverTick	550
18.105 gme::GameMulti::PendingInput Struct Reference	550
18.105.1 Detailed Description	550
18.105.2 Member Data Documentation	551
18.105.2.1 dt	551
18.105.2.2 inputData	551
18.105.2.3 seqId	551
18.106 gme::PlayerControllerMulti::PendingInput Struct Reference	551
18.106.1 Detailed Description	551
18.106.2 Member Data Documentation	552
18.106.2.1 dt	552
18.106.2.2 inputData	552
18.106.2.3 seqId	552
18.107 ecs::Pixel Struct Reference	552
18.107.1 Detailed Description	553
18.108 ecs::Player Struct Reference	553
18.108.1 Detailed Description	555
18.108.2 Member Data Documentation	555
18.108.2.1 is_cli	555
18.109 gme::PlayerControllerMulti Class Reference	555
18.109.1 Detailed Description	558
18.109.2 Constructor & Destructor Documentation	558
18.109.2.1 PlayerControllerMulti() [1/3]	558
18.109.2.2 ~PlayerControllerMulti()	558
18.109.2.3 PlayerControllerMulti() [2/3]	558
18.109.2.4 PlayerControllerMulti() [3/3]	558
18.109.3 Member Function Documentation	558
18.109.3.1 createPlayer()	558
18.109.3.2 handleInput()	559
18.109.3.3 isSpacePressed()	559
18.109.3.4 operator=() [1/2]	559
18.109.3.5 operator=() [2/2]	559
18.109.3.6 sendInputsIfChanged()	559
18.109.3.7 sendInputToServer()	559
18.109.3.8 update()	560
18.109.4 Member Data Documentation	560
18.109.4.1 INPUT_THROTTLE_INTERVAL	560
18.109.4.2 m_componentId	560
18.109.4.3 m_eventBus	560
18.109.4.4 m_keysPressed	560
18.109.4.5 m_lastAckSeqId	560
18.109.4.6 m_nextSeqId	561

18.109.4.7	<code>m_playerEntity</code>	561
18.109.4.8	<code>m_renderer</code>	561
18.110	<code>gme::PlayerDirectionSystem</code> Class Reference	561
18.110.1	Detailed Description	564
18.110.2	Constructor & Destructor Documentation	564
18.110.2.1	<code>PlayerDirectionSystem()</code> [1/6]	564
18.110.2.2	<code>~PlayerDirectionSystem()</code> [1/2]	564
18.110.2.3	<code>PlayerDirectionSystem()</code> [2/6]	564
18.110.2.4	<code>PlayerDirectionSystem()</code> [3/6]	564
18.110.2.5	<code>PlayerDirectionSystem()</code> [4/6]	564
18.110.2.6	<code>~PlayerDirectionSystem()</code> [2/2]	564
18.110.2.7	<code>PlayerDirectionSystem()</code> [5/6]	564
18.110.2.8	<code>PlayerDirectionSystem()</code> [6/6]	565
18.110.3	Member Function Documentation	565
18.110.3.1	<code>operator=()</code> [1/4]	565
18.110.3.2	<code>operator=()</code> [2/4]	565
18.110.3.3	<code>operator=()</code> [3/4]	565
18.110.3.4	<code>operator=()</code> [4/4]	565
18.110.3.5	<code>update()</code> [1/2]	565
18.110.3.6	<code>update()</code> [2/2]	565
18.110.4	Member Data Documentation	566
18.110.4.1	<code>m_skinIndex</code>	566
18.111	<code>utl::PluginLoader</code> Class Reference	566
18.111.1	Detailed Description	568
18.111.2	Constructor & Destructor Documentation	568
18.111.2.1	<code>PluginLoader()</code> [1/3]	568
18.111.2.2	<code>~PluginLoader()</code>	568
18.111.2.3	<code>PluginLoader()</code> [2/3]	568
18.111.2.4	<code>PluginLoader()</code> [3/3]	568
18.111.3	Member Function Documentation	568
18.111.3.1	<code>getEntryPoint()</code>	568
18.111.3.2	<code>loadLibrary()</code>	569
18.111.3.3	<code>loadPlugin()</code>	569
18.111.3.4	<code>operator=()</code> [1/2]	570
18.111.3.5	<code>operator=()</code> [2/2]	570
18.111.4	Member Data Documentation	570
18.111.4.1	<code>m_handles</code>	570
18.111.4.2	<code>m_mutex</code>	570
18.111.4.3	<code>m_plugins</code>	570
18.112	<code>ecs::Registry::Pool< T ></code> Class Template Reference	570
18.112.1	Detailed Description	571
18.112.2	Member Function Documentation	572
18.112.2.1	<code>add()</code>	572

18.112.2.2	get()	572
18.112.2.3	has()	572
18.112.2.4	remove()	572
18.112.3	Member Data Documentation	572
18.112.3.1	data	572
18.113	utl::EventBus::PriorityEventQueue Struct Reference	572
18.113.1	Detailed Description	574
18.113.2	Constructor & Destructor Documentation	574
18.113.2.1	PriorityEventQueue()	574
18.113.3	Member Data Documentation	574
18.113.3.1	events	574
18.113.3.2	mutex	574
18.114	ecs::Projectile Struct Reference	574
18.114.1	Detailed Description	576
18.114.2	Member Enumeration Documentation	576
18.114.2.1	Type	576
18.114.3	Member Data Documentation	576
18.114.3.1	current_lifetime	576
18.114.3.2	damage	576
18.114.3.3	lifetime	576
18.114.3.4	pierce_remaining	576
18.114.3.5	type	576
18.115	gme::ProjectileManager Class Reference	576
18.115.1	Detailed Description	578
18.115.2	Constructor & Destructor Documentation	578
18.115.2.1	ProjectileManager() [1/6]	578
18.115.2.2	~ProjectileManager() [1/2]	578
18.115.2.3	ProjectileManager() [2/6]	578
18.115.2.4	ProjectileManager() [3/6]	578
18.115.2.5	ProjectileManager() [4/6]	578
18.115.2.6	~ProjectileManager() [2/2]	578
18.115.2.7	ProjectileManager() [5/6]	578
18.115.2.8	ProjectileManager() [6/6]	578
18.115.3	Member Function Documentation	578
18.115.3.1	createBasicProjectile()	578
18.115.3.2	createProjectile()	579
18.115.3.3	createSuperchargedProjectile()	580
18.115.3.4	operator=() [1/4]	581
18.115.3.5	operator=() [2/4]	581
18.115.3.6	operator=() [3/4]	581
18.115.3.7	operator=() [4/4]	581
18.116	gme::ProjectileSystem Class Reference	582
18.116.1	Detailed Description	584

18.116.2 Constructor & Destructor Documentation	584
18.116.2.1 ProjectileSystem() [1/6]	584
18.116.2.2 ~ProjectileSystem() [1/2]	584
18.116.2.3 ProjectileSystem() [2/6]	584
18.116.2.4 ProjectileSystem() [3/6]	584
18.116.2.5 ProjectileSystem() [4/6]	584
18.116.2.6 ~ProjectileSystem() [2/2]	584
18.116.2.7 ProjectileSystem() [5/6]	584
18.116.2.8 ProjectileSystem() [6/6]	585
18.116.3 Member Function Documentation	585
18.116.3.1 operator=() [1/4]	585
18.116.3.2 operator=() [2/4]	585
18.116.3.3 operator=() [3/4]	585
18.116.3.4 operator=() [4/4]	585
18.116.3.5 update() [1/2]	585
18.116.3.6 update() [2/2]	585
18.117 eng::QueuedPacket Struct Reference	586
18.117.1 Detailed Description	587
18.117.2 Constructor & Destructor Documentation	587
18.117.2.1 QueuedPacket()	587
18.117.3 Member Data Documentation	587
18.117.3.1 data	587
18.117.3.2 reliable	587
18.118 srv::QueuedPacket Struct Reference	587
18.118.1 Detailed Description	588
18.118.2 Constructor & Destructor Documentation	588
18.118.2.1 QueuedPacket()	588
18.118.3 Member Data Documentation	588
18.118.3.1 data	588
18.118.3.2 destination	588
18.118.3.3 reliable	588
18.119 ecs::Rect Struct Reference	588
18.119.1 Detailed Description	590
18.119.2 Member Data Documentation	591
18.119.2.1 pos_x	591
18.119.2.2 pos_y	591
18.119.2.3 size_x	591
18.119.2.4 size_y	591
18.120 ecs::Registry Class Reference	591
18.120.1 Detailed Description	593
18.120.2 Constructor & Destructor Documentation	593
18.120.2.1 Registry() [1/3]	593
18.120.2.2 ~Registry()	593

18.120.2.3 Registry() [2/3]	593
18.120.2.4 Registry() [3/3]	593
18.120.3 Member Function Documentation	593
18.120.3.1 addComponent()	593
18.120.3.2 createEntity()	594
18.120.3.3 getAll()	596
18.120.3.4 getComponent()	598
18.120.3.5 getPool()	600
18.120.3.6 hasComponent()	601
18.120.3.7 onComponentAdded()	602
18.120.3.8 operator=() [1/2]	603
18.120.3.9 operator=() [2/2]	603
18.120.3.10 removeComponent()	603
18.120.4 Member Data Documentation	604
18.120.4.1 m_components	604
18.120.4.2 m_entities	604
18.120.4.3 m_lastEntity	604
18.120.4.4 m_onComponentAddedCallbacks	604
18.121 gme::RTypeClientMulti Class Reference	604
18.121.1 Detailed Description	607
18.121.2 Constructor & Destructor Documentation	607
18.121.2.1 RTypeClientMulti() [1/3]	607
18.121.2.2 ~RTypeClientMulti()	607
18.121.2.3 RTypeClientMulti() [2/3]	607
18.121.2.4 RTypeClientMulti() [3/3]	607
18.121.3 Member Function Documentation	607
18.121.3.1 getMainSceneId()	607
18.121.3.2 getName()	607
18.121.3.3 getType()	608
18.121.3.4 init()	608
18.121.3.5 operator=() [1/2]	608
18.121.3.6 operator=() [2/2]	608
18.121.3.7 setupScenes()	608
18.121.3.8 update()	609
18.121.4 Member Data Documentation	609
18.121.4.1 m_appConfig	609
18.121.4.2 m_engine	609
18.121.4.3 m_mainSceneId	609
18.121.4.4 m_showDebug	609
18.122 gme::RTypeClientSolo Class Reference	609
18.122.1 Detailed Description	612
18.122.2 Constructor & Destructor Documentation	612
18.122.2.1 RTypeClientSolo() [1/3]	612

18.122.2.2 ~RTypeClientSolo()	612
18.122.2.3 RTypeClientSolo() [2/3]	612
18.122.2.4 RTypeClientSolo() [3/3]	612
18.122.3 Member Function Documentation	612
18.122.3.1 getMainSceneId()	612
18.122.3.2 getName()	612
18.122.3.3 getType()	613
18.122.3.4 init()	613
18.122.3.5 operator=() [1/2]	613
18.122.3.6 operator=() [2/2]	613
18.122.3.7 setupScenes()	613
18.122.3.8 update()	614
18.122.4 Member Data Documentation	614
18.122.4.1 m_appConfig	614
18.122.4.2 m_engine	614
18.122.4.3 m_mainSceneId	614
18.123 gme::RTypeServer Class Reference	615
18.123.1 Detailed Description	617
18.123.2 Constructor & Destructor Documentation	617
18.123.2.1 RTypeServer() [1/3]	617
18.123.2.2 ~RTypeServer()	617
18.123.2.3 RTypeServer() [2/3]	617
18.123.2.4 RTypeServer() [3/3]	617
18.123.3 Member Function Documentation	618
18.123.3.1 broadcastWorldState()	618
18.123.3.2 getName()	618
18.123.3.3 getState()	618
18.123.3.4 getType()	618
18.123.3.5 handlePlayerShooting()	619
18.123.3.6 operator=() [1/2]	619
18.123.3.7 operator=() [2/2]	619
18.123.3.8 processGameStartEvent()	619
18.123.3.9 processPlayerInputEvent()	620
18.123.3.10 processServerStartEvent()	620
18.123.3.11 start()	621
18.123.3.12 stop()	621
18.123.3.13 update()	622
18.123.3.14 updateEntities()	622
18.123.3.15 updateSystems()	623
18.123.4 Member Data Documentation	623
18.123.4.1 BROADCAST_INTERVAL	623
18.123.4.2 m_collisionSystem	623
18.123.4.3 m_enemyAISystem	623

18.123.4.4	m_enemySpawnSystem	623
18.123.4.5	m_entityManager	623
18.123.4.6	m_eventBus	624
18.123.4.7	m_gameState	624
18.123.4.8	m_lastBroadcastTime	624
18.123.4.9	m_lastShotTime	624
18.123.4.10	m_levelState	624
18.123.4.11	m_playerShooting	624
18.123.4.12	m_registry	624
18.123.4.13	m_waveManager	624
18.123.4.14	PROJECTILE_COOLDOWN	624
18.124	ecs::Scale Struct Reference	625
18.124.1	Detailed Description	626
18.124.2	Member Data Documentation	626
18.124.2.1	x	626
18.124.2.2	y	627
18.125	eng::SceneManager Class Reference	627
18.125.1	Detailed Description	629
18.125.2	Constructor & Destructor Documentation	629
18.125.2.1	SceneManager() [1/3]	629
18.125.2.2	~SceneManager()	629
18.125.2.3	SceneManager() [2/3]	629
18.125.2.4	SceneManager() [3/3]	629
18.125.3	Member Function Documentation	629
18.125.3.1	addScene()	629
18.125.3.2	generateNextId()	629
18.125.3.3	getCurrentScene()	629
18.125.3.4	getScene()	630
18.125.3.5	operator=() [1/2]	630
18.125.3.6	operator=() [2/2]	630
18.125.3.7	switchToScene()	630
18.125.4	Member Data Documentation	630
18.125.4.1	m_currentSceneId	630
18.125.4.2	m_nextId	630
18.125.4.3	m_scenes	630
18.126	srv::SceneManager Class Reference	630
18.126.1	Detailed Description	632
18.126.2	Constructor & Destructor Documentation	632
18.126.2.1	SceneManager() [1/3]	632
18.126.2.2	~SceneManager()	632
18.126.2.3	SceneManager() [2/3]	632
18.126.2.4	SceneManager() [3/3]	632
18.126.3	Member Function Documentation	632

18.126.3.1	addScene()	632
18.126.3.2	getCurrentScene()	632
18.126.3.3	getScene()	632
18.126.3.4	operator=() [1/2]	632
18.126.3.5	operator=() [2/2]	633
18.126.3.6	switchToScene()	633
18.126.4	Member Data Documentation	633
18.126.4.1	m_currentSceneId	633
18.126.4.2	m_scenes	633
18.127	ecs::Score Struct Reference	633
18.127.1	Detailed Description	634
18.127.2	Member Data Documentation	634
18.127.2.1	value	634
18.128	ecs::Scrolling Struct Reference	635
18.128.1	Detailed Description	636
18.128.2	Member Data Documentation	637
18.128.2.1	fit_width	637
18.128.2.2	original_height	637
18.128.2.3	original_width	637
18.128.2.4	speed_x	637
18.129	gme::ScrollingSystem Class Reference	637
18.129.1	Detailed Description	640
18.129.2	Constructor & Destructor Documentation	640
18.129.2.1	ScrollingSystem() [1/6]	640
18.129.2.2	~ScrollingSystem() [1/2]	640
18.129.2.3	ScrollingSystem() [2/6]	640
18.129.2.4	ScrollingSystem() [3/6]	640
18.129.2.5	ScrollingSystem() [4/6]	640
18.129.2.6	~ScrollingSystem() [2/2]	640
18.129.2.7	ScrollingSystem() [5/6]	640
18.129.2.8	ScrollingSystem() [6/6]	641
18.129.3	Member Function Documentation	641
18.129.3.1	operator=() [1/4]	641
18.129.3.2	operator=() [2/4]	641
18.129.3.3	operator=() [3/4]	641
18.129.3.4	operator=() [4/4]	641
18.129.3.5	update() [1/2]	641
18.129.3.6	update() [2/2]	641
18.129.4	Member Data Documentation	642
18.129.4.1	m_renderer	642
18.130	rnp::Serializer Class Reference	642
18.130.1	Detailed Description	646
18.130.2	Constructor & Destructor Documentation	646

18.130.2.1 Serializer() [1/3]	646
18.130.2.2 Serializer() [2/3]	646
18.130.2.3 Serializer() [3/3]	647
18.130.3 Member Function Documentation	647
18.130.3.1 createDamageEvent()	647
18.130.3.2 createDespawnEvent()	647
18.130.3.3 createInputEvent()	648
18.130.3.4 createScoreEvent()	648
18.130.3.5 createSpawnEvent()	649
18.130.3.6 deserializeConnect()	649
18.130.3.7 deserializeConnectAccept()	650
18.130.3.8 deserializeDisconnect()	651
18.130.3.9 deserializeEntityEvents()	651
18.130.3.10 deserializeEntityState()	652
18.130.3.11 deserializeError()	652
18.130.3.12 deserializeEventRecord()	653
18.130.3.13 deserializeGameStart()	654
18.130.3.14 deserializeHeader()	654
18.130.3.15 deserializeInputData()	655
18.130.3.16 deserializeLobbyCreate()	655
18.130.3.17 deserializeLobbyCreateResponse()	656
18.130.3.18 deserializeLobbyInfo()	657
18.130.3.19 deserializeLobbyJoin()	657
18.130.3.20 deserializeLobbyJoinResponse()	658
18.130.3.21 deserializeLobbyListResponse()	659
18.130.3.22 deserializeLobbyUpdate()	659
18.130.3.23 deserializePingPong()	660
18.130.3.24 deserializeStartGameRequest()	661
18.130.3.25 deserializeWorldState()	661
18.130.3.26 getData()	662
18.130.3.27 getSize()	663
18.130.3.28 readByte()	664
18.130.3.29 readBytes()	665
18.130.3.30 readFloat()	666
18.130.3.31 readString()	667
18.130.3.32 readUInt16()	667
18.130.3.33 readUInt32()	668
18.130.3.34 reset()	669
18.130.3.35 serializeConnect()	669
18.130.3.36 serializeConnectAccept()	670
18.130.3.37 serializeDisconnect()	671
18.130.3.38 serializeEntityEvents()	671
18.130.3.39 serializeEntityState()	672

18.130.3.40	serializeError()	673
18.130.3.41	serializeEventRecord()	673
18.130.3.42	serializeGameStart()	674
18.130.3.43	serializeHeader()	675
18.130.3.44	serializeInputData()	676
18.130.3.45	serializeLobbyCreate()	677
18.130.3.46	serializeLobbyCreateResponse()	678
18.130.3.47	serializeLobbyInfo()	678
18.130.3.48	serializeLobbyJoin()	679
18.130.3.49	serializeLobbyJoinResponse()	680
18.130.3.50	serializeLobbyListResponse()	680
18.130.3.51	serializeLobbyUpdate()	681
18.130.3.52	serializePingPong()	682
18.130.3.53	serializeStartGameRequest()	682
18.130.3.54	serializeWorldState()	683
18.130.3.55	writeByte()	684
18.130.3.56	writeBytes()	684
18.130.3.57	writeFloat()	685
18.130.3.58	writeString()	686
18.130.3.59	writeUInt16()	687
18.130.3.60	writeUInt32()	688
18.130.4	Member Data Documentation	689
18.130.4.1	buffer__	689
18.130.4.2	readPos__	689
18.130.4.3	writePos__	690
18.131	srv::Server Class Reference	690
18.131.1	Detailed Description	692
18.131.2	Constructor & Destructor Documentation	692
18.131.2.1	Server() [1/3]	692
18.131.2.2	~Server()	692
18.131.2.3	Server() [2/3]	692
18.131.2.4	Server() [3/3]	692
18.131.3	Member Function Documentation	692
18.131.3.1	operator=() [1/2]	692
18.131.3.2	operator=() [2/2]	692
18.131.3.3	run()	693
18.131.3.4	setupConfig()	693
18.131.4	Member Data Documentation	693
18.131.4.1	m_clock	693
18.131.4.2	m_config	693
18.131.4.3	m_game	693
18.131.4.4	m_network	693
18.131.4.5	m_pluginLoader	694

18.131.4.6 m_sceneManager	694
18.132 gme::ServerScene Class Reference	694
18.132.1 Detailed Description	697
18.132.2 Constructor & Destructor Documentation	697
18.132.2.1 ServerScene() [1/3]	697
18.132.2.2 ~ServerScene()	698
18.132.2.3 ServerScene() [2/3]	698
18.132.2.4 ServerScene() [3/3]	698
18.132.3 Member Function Documentation	698
18.132.3.1 connectServer()	698
18.132.3.2 event()	698
18.132.3.3 getCurrentEditField()	698
18.132.3.4 operator=() [1/2]	699
18.132.3.5 operator=() [2/2]	699
18.132.3.6 playMusic()	699
18.132.3.7 update()	699
18.132.3.8 updateValueDisplay()	699
18.132.4 Member Data Documentation	699
18.132.4.1 m_animationTime	699
18.132.4.2 m_host	699
18.132.4.3 m_playerName	699
18.132.4.4 m_playerNameValueEntity	699
18.132.4.5 m_playMusic	699
18.132.4.6 m_port	700
18.132.4.7 m_selectedIndex	700
18.132.4.8 m_serverIPValueEntity	700
18.132.4.9 m_serverOptions	700
18.132.4.10 m_serverPortValueEntity	700
18.132.4.11 onBackToMenu	700
18.132.4.12 onConnect	700
18.133 cli::Settings Class Reference	700
18.133.1 Detailed Description	703
18.133.2 Constructor & Destructor Documentation	703
18.133.2.1 Settings() [1/3]	703
18.133.2.2 ~Settings()	704
18.133.2.3 Settings() [2/3]	704
18.133.2.4 Settings() [3/3]	704
18.133.3 Member Function Documentation	704
18.133.3.1 applySkinChange()	704
18.133.3.2 applyVideoQuality()	704
18.133.3.3 event()	704
18.133.3.4 loadFromConfig()	704
18.133.3.5 operator=() [1/2]	705

18.133.3.6	<code>operator=()</code> [2/2]	705
18.133.3.7	<code>playMusic()</code>	705
18.133.3.8	<code>update()</code>	705
18.133.3.9	<code>updateSettingsDisplay()</code>	705
18.133.4	Member Data Documentation	705
18.133.4.1	<code>m_animationTime</code>	705
18.133.4.2	<code>m_appConfig</code>	705
18.133.4.3	<code>m_audioVolume</code>	705
18.133.4.4	<code>m_controlScheme</code>	706
18.133.4.5	<code>m_controlValueEntity</code>	706
18.133.4.6	<code>m_keysPressed</code>	706
18.133.4.7	<code>m_playMusic</code>	706
18.133.4.8	<code>m_qualityValueEntity</code>	706
18.133.4.9	<code>m_renderer</code>	706
18.133.4.10	<code>m_selectedIndex</code>	706
18.133.4.11	<code>m_settingsOptions</code>	706
18.133.4.12	<code>m_skinIndex</code>	706
18.133.4.13	<code>m_skinSpriteEntity</code>	706
18.133.4.14	<code>m_titleEntity</code>	707
18.133.4.15	<code>m_titlePulseTime</code>	707
18.133.4.16	<code>m_videoQuality</code>	707
18.133.4.17	<code>m_volumeValueEntity</code>	707
18.133.4.18	<code>onLeave</code>	707
18.134	<code>utl::SharedLib</code> Struct Reference	707
18.134.1	Detailed Description	708
18.134.2	Constructor & Destructor Documentation	708
18.134.2.1	<code>SharedLib()</code> [1/3]	708
18.134.2.2	<code>~SharedLib()</code>	708
18.134.2.3	<code>SharedLib()</code> [2/3]	708
18.134.2.4	<code>SharedLib()</code> [3/3]	708
18.134.3	Member Function Documentation	709
18.134.3.1	<code>close()</code>	709
18.134.3.2	<code>operator bool()</code>	709
18.134.3.3	<code>operator=()</code> [1/2]	709
18.134.3.4	<code>operator=()</code> [2/2]	709
18.134.4	Member Data Documentation	709
18.134.4.1	<code>handle</code>	709
18.135	<code>gme::SpawnSystem</code> Class Reference	710
18.135.1	Detailed Description	712
18.135.2	Constructor & Destructor Documentation	712
18.135.2.1	<code>SpawnSystem()</code> [1/3]	712
18.135.2.2	<code>~SpawnSystem()</code>	712
18.135.2.3	<code>SpawnSystem()</code> [2/3]	712

18.135.2.4 SpawnSystem() [3/3]	712
18.135.3 Member Function Documentation	712
18.135.3.1 isEnabled()	712
18.135.3.2 operator=() [1/2]	712
18.135.3.3 operator=() [2/2]	712
18.135.3.4 setEnable()	713
18.135.3.5 spawnEnemy()	713
18.135.3.6 spawnWave()	713
18.135.3.7 update()	714
18.135.4 Member Data Documentation	714
18.135.4.1 m_enemySpawnTimer	714
18.135.4.2 m_renderer	714
18.135.4.3 m_waveTimer	714
18.136 gme::SpawnWave Struct Reference	715
18.136.1 Detailed Description	715
18.136.2 Member Data Documentation	715
18.136.2.1 advancedEnemyCount	715
18.136.2.2 basicEnemyCount	715
18.136.2.3 bossSpawn	716
18.136.2.4 spawnedCount	716
18.136.2.5 spawnInterval	716
18.136.2.6 spawnTime	716
18.137 ecs::SpriteSystem Class Reference	716
18.137.1 Detailed Description	719
18.137.2 Constructor & Destructor Documentation	719
18.137.2.1 SpriteSystem() [1/3]	719
18.137.2.2 ~SpriteSystem()	719
18.137.2.3 SpriteSystem() [2/3]	719
18.137.2.4 SpriteSystem() [3/3]	719
18.137.3 Member Function Documentation	719
18.137.3.1 operator=() [1/2]	719
18.137.3.2 operator=() [2/2]	719
18.137.3.3 update()	719
18.137.4 Member Data Documentation	720
18.137.4.1 m_renderer	720
18.138 gme::StageManager Class Reference	720
18.138.1 Detailed Description	722
18.138.2 Constructor & Destructor Documentation	722
18.138.2.1 StageManager() [1/6]	722
18.138.2.2 ~StageManager() [1/2]	722
18.138.2.3 StageManager() [2/6]	722
18.138.2.4 StageManager() [3/6]	723
18.138.2.5 StageManager() [4/6]	723

18.138.2.6	<code>~StageManager()</code> [2/2]	723
18.138.2.7	<code>StageManager()</code> [5/6]	723
18.138.2.8	<code>StageManager()</code> [6/6]	723
18.138.3	Member Function Documentation	723
18.138.3.1	<code>getCurrentWave()</code> [1/2]	723
18.138.3.2	<code>getCurrentWave()</code> [2/2]	723
18.138.3.3	<code>getTotalGameTime()</code> [1/2]	723
18.138.3.4	<code>getTotalGameTime()</code> [2/2]	723
18.138.3.5	<code>getWaveTimer()</code> [1/2]	723
18.138.3.6	<code>getWaveTimer()</code> [2/2]	723
18.138.3.7	<code>isStageSpawned()</code> [1/2]	724
18.138.3.8	<code>isStageSpawned()</code> [2/2]	724
18.138.3.9	<code>operator=()</code> [1/4]	724
18.138.3.10	<code>operator=()</code> [2/4]	724
18.138.3.11	<code>operator=()</code> [3/4]	724
18.138.3.12	<code>operator=()</code> [4/4]	724
18.138.3.13	<code>spawnStage()</code> [1/2]	724
18.138.3.14	<code>spawnStage()</code> [2/2]	725
18.138.3.15	<code>stopScrolling()</code>	725
18.138.3.16	<code>update()</code> [1/2]	725
18.138.3.17	<code>update()</code> [2/2]	725
18.138.4	Member Data Documentation	726
18.138.4.1	<code>m_currentWave</code>	726
18.138.4.2	<code>m_stageSpawned</code>	726
18.138.4.3	<code>m_stageTimer</code>	726
18.138.4.4	<code>m_totalGameTimer</code>	726
18.138.4.5	<code>m_waveTimer</code>	726
18.138.4.6	<code>ORIGINAL_WIDTH</code>	726
18.138.4.7	<code>SCROLL_SPEED</code>	726
18.138.4.8	<code>SPRITE_HEIGHT</code>	726
18.138.4.9	<code>STAGE_SPAWN_DELAY</code>	726
18.138.4.10	<code>TOTAL_WAVES</code>	726
18.138.4.11	<code>WAVE_DURATION</code>	727
18.139	<code>ecs::StarfieldSystem</code> Class Reference	727
18.139.1	Detailed Description	730
18.139.2	Constructor & Destructor Documentation	730
18.139.2.1	<code>StarfieldSystem()</code> [1/3]	730
18.139.2.2	<code>~StarfieldSystem()</code>	730
18.139.2.3	<code>StarfieldSystem()</code> [2/3]	731
18.139.2.4	<code>StarfieldSystem()</code> [3/3]	731
18.139.3	Member Function Documentation	731
18.139.3.1	<code>createComets()</code>	731
18.139.3.2	<code>createNebulae()</code>	731

18.139.3.3 createPlanets()	732
18.139.3.4 createShootingStars()	732
18.139.3.5 createStarfield()	733
18.139.3.6 createStars()	734
18.139.3.7 createStarsVaried()	734
18.139.3.8 operator=() [1/2]	735
18.139.3.9 operator=() [2/2]	735
18.139.3.10 update()	735
18.139.4 Member Data Documentation	736
18.139.4.1 m_renderer	736
18.139.4.2 m_timeAccumulator	736
18.140 ecs::Text Struct Reference	736
18.140.1 Detailed Description	737
18.140.2 Member Data Documentation	737
18.140.2.1 content	737
18.140.2.2 font_size	738
18.141 eng::Text Struct Reference	738
18.141.1 Detailed Description	739
18.141.2 Member Data Documentation	739
18.141.2.1 color	739
18.141.2.2 content	739
18.141.2.3 font_name	739
18.141.2.4 name	739
18.141.2.5 size	739
18.141.2.6 x	739
18.141.2.7 y	739
18.142 ecs::TextSystem Class Reference	739
18.142.1 Detailed Description	742
18.142.2 Constructor & Destructor Documentation	742
18.142.2.1 TextSystem() [1/3]	742
18.142.2.2 ~TextSystem()	742
18.142.2.3 TextSystem() [2/3]	742
18.142.2.4 TextSystem() [3/3]	742
18.142.3 Member Function Documentation	742
18.142.3.1 operator=() [1/2]	742
18.142.3.2 operator=() [2/2]	742
18.142.3.3 update()	742
18.142.4 Member Data Documentation	743
18.142.4.1 m_renderer	743
18.143 ecs::Texture Struct Reference	743
18.143.1 Detailed Description	744
18.143.2 Member Data Documentation	744
18.143.2.1 path	744

18.144	ecs::Transform Struct Reference	745
18.144.1	Detailed Description	746
18.144.2	Member Data Documentation	746
18.144.2.1	rotation	746
18.144.2.2	x	747
18.144.2.3	y	747
18.145	ecs::Velocity Struct Reference	747
18.145.1	Detailed Description	748
18.145.2	Member Data Documentation	748
18.145.2.1	x	748
18.145.2.2	y	749
18.146	gme::WaitingRoomScene Class Reference	749
18.146.1	Detailed Description	753
18.146.2	Constructor & Destructor Documentation	753
18.146.2.1	WaitingRoomScene() [1/3]	753
18.146.2.2	~WaitingRoomScene()	753
18.146.2.3	WaitingRoomScene() [2/3]	753
18.146.2.4	WaitingRoomScene() [3/3]	753
18.146.3	Member Function Documentation	753
18.146.3.1	clearPlayerEntities()	753
18.146.3.2	event()	753
18.146.3.3	getLobbyId()	754
18.146.3.4	handleGameStart()	754
18.146.3.5	handleLobbyUpdate()	754
18.146.3.6	isHost()	754
18.146.3.7	leaveLobby()	755
18.146.3.8	operator=() [1/2]	755
18.146.3.9	operator=() [2/2]	755
18.146.3.10	playMusic()	755
18.146.3.11	processEventBus()	755
18.146.3.12	setIsHost()	756
18.146.3.13	setLobbyId()	756
18.146.3.14	setLobbyInfo()	757
18.146.3.15	setupEventSubscriptions()	757
18.146.3.16	startGame()	757
18.146.3.17	update()	758
18.146.3.18	updatePlayerDisplay()	758
18.146.4	Member Data Documentation	758
18.146.4.1	BUTTON_COUNT	758
18.146.4.2	BUTTON_LEAVE	758
18.146.4.3	BUTTON_READY	758
18.146.4.4	BUTTON_START	758
18.146.4.5	m_animationTime	759

18.146.4.6	m_currentLobbyInfo	759
18.146.4.7	m_hasLobbyInfo	759
18.146.4.8	m_isHost	759
18.146.4.9	m_leaveButtonEntity	759
18.146.4.10	m_lobbyId	759
18.146.4.11	m_lobbyIdEntity	759
18.146.4.12	m_playerCountEntity	759
18.146.4.13	m_playerEntities	759
18.146.4.14	m_playMusic	759
18.146.4.15	m_readyButtonEntity	759
18.146.4.16	m_renderer	760
18.146.4.17	m_selectedButton	760
18.146.4.18	m_startButtonEntity	760
18.146.4.19	m_statusEntity	760
18.146.4.20	onGameStart	760
18.146.4.21	onLeaveLobby	760
18.147	gme::Wave Struct Reference	760
18.147.1	Detailed Description	761
18.147.2	Member Data Documentation	761
18.147.2.1	duration	761
18.147.2.2	enemies	762
18.147.2.3	onComplete	762
18.147.2.4	waitForClear	762
18.147.2.5	waveNumber	762
18.148	gme::WaveManager Class Reference	762
18.148.1	Detailed Description	765
18.148.2	Constructor & Destructor Documentation	765
18.148.2.1	WaveManager() [1/3]	765
18.148.2.2	~WaveManager()	765
18.148.2.3	WaveManager() [2/3]	765
18.148.2.4	WaveManager() [3/3]	765
18.148.3	Member Function Documentation	765
18.148.3.1	addWave()	765
18.148.3.2	clearWaves()	765
18.148.3.3	createBossWave()	766
18.148.3.4	createWave1()	766
18.148.3.5	createWave2()	766
18.148.3.6	createWave3()	767
18.148.3.7	createWave4()	767
18.148.3.8	getCurrentWave()	767
18.148.3.9	getTotalWaves()	768
18.148.3.10	isActive()	768
18.148.3.11	isCompleted()	768

18.148.3.12	isReady()	768
18.148.3.13	isWaveCleared()	768
18.148.3.14	operator=() [1/2]	769
18.148.3.15	operator=() [2/2]	769
18.148.3.16	processSpawns()	769
18.148.3.17	reset()	769
18.148.3.18	setupDefaultWaves()	770
18.148.3.19	start()	770
18.148.3.20	startNextWave()	770
18.148.3.21	update()	771
18.148.4	Member Data Documentation	771
18.148.4.1	INITIAL_DELAY	771
18.148.4.2	m_active	771
18.148.4.3	m_completed	771
18.148.4.4	m_currentWaveIndex	772
18.148.4.5	m_enemiesSpawns	772
18.148.4.6	m_entityManager	772
18.148.4.7	m_initialDelayPassed	772
18.148.4.8	m_initialDelayTimer	772
18.148.4.9	m_spawnTimer	772
18.148.4.10	m_waveInProgress	772
18.148.4.11	m_waves	772
18.148.4.12	m_waveTimer	772
18.149	gme::WeaponSystem Class Reference	773
18.149.1	Detailed Description	775
18.149.2	Constructor & Destructor Documentation	776
18.149.2.1	WeaponSystem() [1/6]	776
18.149.2.2	~WeaponSystem() [1/2]	776
18.149.2.3	WeaponSystem() [2/6]	776
18.149.2.4	WeaponSystem() [3/6]	776
18.149.2.5	WeaponSystem() [4/6]	776
18.149.2.6	~WeaponSystem() [2/2]	776
18.149.2.7	WeaponSystem() [5/6]	776
18.149.2.8	WeaponSystem() [6/6]	776
18.149.3	Member Function Documentation	776
18.149.3.1	ensureSuperShotAudio() [1/2]	776
18.149.3.2	ensureSuperShotAudio() [2/2]	777
18.149.3.3	hideLoadingAnimation() [1/2]	777
18.149.3.4	hideLoadingAnimation() [2/2]	777
18.149.3.5	operator=() [1/4]	778
18.149.3.6	operator=() [2/4]	778
18.149.3.7	operator=() [3/4]	778
18.149.3.8	operator=() [4/4]	778

18.149.3.9	reset() [1/2]	778
18.149.3.10	reset() [2/2]	778
18.149.3.11	showLoadingAnimation() [1/2]	778
18.149.3.12	showLoadingAnimation() [2/2]	779
18.149.3.13	tryFireBasic()	779
18.149.3.14	tryFireSupercharged()	780
18.149.3.15	update() [1/2]	780
18.149.3.16	update() [2/2]	781
18.149.4	Member Data Documentation	781
18.149.4.1	m_fireCooldown	781
18.149.4.2	m_isCharging	781
18.149.4.3	m_superShotAudioEntity	781
18.150	eng::WindowSize Struct Reference	782
18.150.1	Detailed Description	782
18.150.2	Member Data Documentation	782
18.150.2.1	height	782
18.150.2.2	width	782
19	File Documentation	783
19.1	/home/masina/Projects/Epitech/rtype/client/include/Client/ArgsHandler.hpp File Reference	783
19.1.1	Detailed Description	784
19.2	ArgsHandler.hpp	784
19.3	/home/masina/Projects/Epitech/rtype/server/include/Server/ArgsHandler.hpp File Reference	785
19.3.1	Detailed Description	786
19.4	ArgsHandler.hpp	786
19.5	/home/masina/Projects/Epitech/rtype/client/include/Client/Client.hpp File Reference	787
19.5.1	Detailed Description	788
19.6	Client.hpp	788
19.7	/home/masina/Projects/Epitech/rtype/client/include/Client/Generated/Version.hpp File Reference	789
19.7.1	Macro Definition Documentation	789
19.7.1.1	BUILD_TYPE	789
19.7.1.2	GIT_COMMIT_HASH	789
19.7.1.3	GIT_TAG	789
19.7.1.4	PROJECT_NAME	789
19.7.1.5	PROJECT_VERSION	790
19.7.1.6	PROJECT_VERSION_MAJOR	790
19.7.1.7	PROJECT_VERSION_MINOR	790
19.7.1.8	PROJECT_VERSION_PATCH	790
19.8	Version.hpp	790
19.9	/home/masina/Projects/Epitech/rtype/server/include/Server/Generated/Version.hpp File Reference	791

19.9.1 Macro Definition Documentation	791
19.9.1.1 BUILD_TYPE	791
19.9.1.2 GIT_COMMIT_HASH	791
19.9.1.3 GIT_TAG	791
19.9.1.4 PROJECT_NAME	791
19.9.1.5 PROJECT_VERSION	792
19.9.1.6 PROJECT_VERSION_MAJOR	792
19.9.1.7 PROJECT_VERSION_MINOR	792
19.9.1.8 PROJECT_VERSION_PATCH	792
19.10 Version.hpp	792
19.11 /home/masina/Projects/Epitech/rtype/client/include/Client/Scenes/Intro.hpp File Reference	792
19.11.1 Detailed Description	793
19.12 Intro.hpp	793
19.13 /home/masina/Projects/Epitech/rtype/client/include/Client/Scenes/Menu.hpp File Reference	794
19.13.1 Detailed Description	795
19.14 Menu.hpp	795
19.15 /home/masina/Projects/Epitech/rtype/client/include/Client/Scenes/Settings.hpp File Reference	795
19.15.1 Detailed Description	796
19.16 Settings.hpp	796
19.17 /home/masina/Projects/Epitech/rtype/client/README.md File Reference	798
19.18 /home/masina/Projects/Epitech/rtype/plugins/Audio/README.md File Reference	798
19.19 /home/masina/Projects/Epitech/rtype/plugins/Game/README.md File Reference	798
19.20 /home/masina/Projects/Epitech/rtype/plugins/Network/README.md File Reference	798
19.21 /home/masina/Projects/Epitech/rtype/plugins/README.md File Reference	798
19.22 /home/masina/Projects/Epitech/rtype/plugins/Renderer/README.md File Reference	798
19.23 /home/masina/Projects/Epitech/rtype/README.md File Reference	798
19.24 /home/masina/Projects/Epitech/rtype/server/README.md File Reference	798
19.25 /home/masina/Projects/Epitech/rtype/client/src/argsHandler.cpp File Reference	798
19.25.1 Macro Definition Documentation	799
19.25.1.1 APP_EXTENSION	799
19.25.2 Variable Documentation	799
19.25.2.1 HELP_MESSAGE	799
19.25.2.2 VERSION_MESSAGE	799
19.26 argsHandler.cpp	799
19.27 /home/masina/Projects/Epitech/rtype/server/src/argsHandler.cpp File Reference	801
19.27.1 Macro Definition Documentation	802
19.27.1.1 APP_EXTENSION	802
19.27.2 Variable Documentation	802
19.27.2.1 HELP_MESSAGE	802
19.27.2.2 VERSION_MESSAGE	802
19.28 argsHandler.cpp	802

19.29	/home/masina/Projects/Epitech/rtype/client/src/client.cpp File Reference	803
19.30	client.cpp	804
19.31	/home/masina/Projects/Epitech/rtype/client/src/event.cpp File Reference	806
19.32	event.cpp	806
19.33	/home/masina/Projects/Epitech/rtype/client/src/main.cpp File Reference	807
19.33.1	Function Documentation	807
19.33.1.1	main()	807
19.34	main.cpp	807
19.35	/home/masina/Projects/Epitech/rtype/server/src/main.cpp File Reference	808
19.35.1	Function Documentation	808
19.35.1.1	main()	808
19.36	main.cpp	809
19.37	/home/masina/Projects/Epitech/rtype/client/src/scenes/intro.cpp File Reference	809
19.38	intro.cpp	810
19.39	/home/masina/Projects/Epitech/rtype/client/src/scenes/menu.cpp File Reference	811
19.40	menu.cpp	811
19.41	/home/masina/Projects/Epitech/rtype/client/src/scenes/settings.cpp File Reference	814
19.42	settings.cpp	814
19.43	/home/masina/Projects/Epitech/rtype/CONTRIBUTING.md File Reference	818
19.44	/home/masina/Projects/Epitech/rtype/documentation/comparative_study.md File Reference	818
19.45	/home/masina/Projects/Epitech/rtype/documentation/network_protocol_specification.md File Reference	818
19.46	/home/masina/Projects/Epitech/rtype/LICENSE.md File Reference	818
19.47	/home/masina/Projects/Epitech/rtype/modules/ECS/include/ECS/Component.hpp File Reference	818
19.47.1	Detailed Description	820
19.48	Component.hpp	820
19.49	/home/masina/Projects/Epitech/rtype/modules/ECS/include/ECS/Entity.hpp File Reference	822
19.49.1	Detailed Description	822
19.50	Entity.hpp	823
19.51	/home/masina/Projects/Epitech/rtype/modules/ECS/include/ECS/Interfaces/ISystems.hpp File Reference	823
19.51.1	Detailed Description	824
19.52	ISystems.hpp	824
19.53	/home/masina/Projects/Epitech/rtype/modules/ECS/include/ECS/Registry.hpp File Reference	824
19.53.1	Detailed Description	825
19.54	Registry.hpp	825
19.55	/home/masina/Projects/Epitech/rtype/modules/ECS/include/ECS/Systems/Animation.hpp File Reference	827
19.56	Animation.hpp	828
19.57	/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/include/RTypeClientMulti/Systems/Animation.hpp File Reference	829

19.57.1 Detailed Description	829
19.58 Animation.hpp	829
19.59 /home/masina/Projects/Epitech/rtype/modules/ECS/include/ECS/Systems/Audio.hpp File Reference	830
19.59.1 Detailed Description	831
19.60 Audio.hpp	831
19.61 /home/masina/Projects/Epitech/rtype/modules/ECS/include/ECS/Systems/Beam.hpp File Reference	833
19.61.1 Detailed Description	834
19.62 Beam.hpp	834
19.63 /home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/include/↵ RTypeClientMulti/Systems/Beam.hpp File Reference	835
19.63.1 Detailed Description	835
19.64 Beam.hpp	836
19.65 /home/masina/Projects/Epitech/rtype/modules/ECS/include/ECS/Systems/Debug.hpp File Reference	837
19.65.1 Detailed Description	838
19.66 Debug.hpp	838
19.67 /home/masina/Projects/Epitech/rtype/modules/ECS/include/ECS/Systems/Sprite.hpp File Reference	839
19.67.1 Detailed Description	840
19.68 Sprite.hpp	840
19.69 /home/masina/Projects/Epitech/rtype/modules/ECS/include/ECS/Systems/Starfield.hpp File Reference	841
19.69.1 Detailed Description	842
19.70 Starfield.hpp	842
19.71 /home/masina/Projects/Epitech/rtype/modules/ECS/include/ECS/Systems/Text.hpp File Reference	845
19.72 Text.hpp	846
19.73 /home/masina/Projects/Epitech/rtype/modules/Engine/include/Engine/Engine.hpp File Reference	847
19.73.1 Detailed Description	848
19.74 Engine.hpp	848
19.75 /home/masina/Projects/Epitech/rtype/modules/Engine/src/engine.cpp File Reference	849
19.76 engine.cpp	849
19.77 /home/masina/Projects/Epitech/rtype/modules/Interfaces/include/Interfaces/IAudio.hpp File Reference	849
19.77.1 Detailed Description	850
19.78 IAudio.hpp	850
19.79 /home/masina/Projects/Epitech/rtype/modules/Interfaces/include/Interfaces/IGame↵ Client.hpp File Reference	851
19.79.1 Detailed Description	852
19.80 IGameClient.hpp	852
19.81 /home/masina/Projects/Epitech/rtype/modules/Interfaces/include/Interfaces/IGame↵ Server.hpp File Reference	852
19.81.1 Detailed Description	853

19.82 IGameServer.hpp	853
19.83 /home/masina/Projects/Epitech/rtype/modules/Interfaces/include/Interfaces/INetwork↵ Client.hpp File Reference	854
19.83.1 Detailed Description	855
19.84 INetworkClient.hpp	855
19.85 /home/masina/Projects/Epitech/rtype/modules/Interfaces/include/Interfaces/INetwork↵ Server.hpp File Reference	856
19.85.1 Detailed Description	857
19.86 INetworkServer.hpp	857
19.87 /home/masina/Projects/Epitech/rtype/modules/Interfaces/include/Interfaces/IRenderer.hpp File Reference	858
19.87.1 Detailed Description	859
19.88 IRenderer.hpp	859
19.89 /home/masina/Projects/Epitech/rtype/modules/Interfaces/include/Interfaces/Protocol/↵ HandlerPacket.hpp File Reference	861
19.89.1 Detailed Description	863
19.90 HandlerPacket.hpp	863
19.91 /home/masina/Projects/Epitech/rtype/modules/Interfaces/include/Interfaces/Protocol/↵ Protocol.hpp File Reference	870
19.91.1 Detailed Description	872
19.92 Protocol.hpp	872
19.93 /home/masina/Projects/Epitech/rtype/modules/Interfaces/include/Interfaces/Protocol/↵ Serializer.hpp File Reference	876
19.93.1 Detailed Description	877
19.94 Serializer.hpp	877
19.95 /home/masina/Projects/Epitech/rtype/modules/Utils/include/Utils/AppConfig↵ Client.hpp File Reference	888
19.95.1 Detailed Description	889
19.96 AppConfigClient.hpp	889
19.97 /home/masina/Projects/Epitech/rtype/modules/Utils/include/Utils/AppConfig↵ Server.hpp File Reference	889
19.97.1 Detailed Description	891
19.98 AppConfigServer.hpp	891
19.99 /home/masina/Projects/Epitech/rtype/modules/Utils/include/Utils/Clock.hpp File Ref- erence	892
19.99.1 Detailed Description	892
19.100 Clock.hpp	892
19.101 /home/masina/Projects/Epitech/rtype/modules/Utils/include/Utils/Event.hpp File Reference	894
19.101.1 Detailed Description	895
19.102 Event.hpp	895
19.103 /home/masina/Projects/Epitech/rtype/modules/Utils/include/Utils/EventBus.hpp File Reference	897
19.103.1 Detailed Description	898
19.104 EventBus.hpp	898

19.105	/home/masina/Projects/Epitech/rtype/modules/Utils/include/Utils/HitboxUtils.hpp File Reference	905
19.105.1	Detailed Description	906
19.106	HitboxUtils.hpp	906
19.107	/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/include/↵ RTypeClientMulti/Utils/HitboxUtils.hpp File Reference	907
19.108	HitboxUtils.hpp	908
19.109	/home/masina/Projects/Epitech/rtype/modules/Utils/include/Utils/Interfaces/↵ IPlugin.hpp File Reference	908
19.109.1	Detailed Description	909
19.109.2	Macro Definition Documentation	909
19.109.2.1	PLUGIN_EXPORT	909
19.110	IPlugin.hpp	909
19.111	/home/masina/Projects/Epitech/rtype/modules/Utils/include/Utils/Logger.hpp File Reference	910
19.111.1	Detailed Description	910
19.112	Logger.hpp	910
19.113	/home/masina/Projects/Epitech/rtype/modules/Utils/include/Utils/PluginLoader.hpp File Reference	912
19.113.1	Detailed Description	913
19.113.2	Macro Definition Documentation	913
19.113.2.1	PLUGINS_EXTENSION	913
19.114	PluginLoader.hpp	913
19.115	/home/masina/Projects/Epitech/rtype/modules/Utils/include/Utils/RTypeShared/↵ GameConfig.hpp File Reference	915
19.115.1	Detailed Description	918
19.116	GameConfig.hpp	918
19.117	/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/include/↵ RTypeClientMulti/GameConfig.hpp File Reference	920
19.117.1	Detailed Description	922
19.118	GameConfig.hpp	922
19.119	/home/masina/Projects/Epitech/rtype/modules/Utils/include/Utils/Utils.hpp File Ref- erence	924
19.119.1	Detailed Description	925
19.120	Utils.hpp	925
19.121	/home/masina/Projects/Epitech/rtype/modules/Utils/src/logger.cpp File Reference	925
19.122	logger.cpp	925
19.123	/home/masina/Projects/Epitech/rtype/modules/Utils/src/utils.cpp File Reference	926
19.124	utils.cpp	926
19.125	/home/masina/Projects/Epitech/rtype/plugins/Game/FlappyBird/Client/Solo/include/↵ FlappyBirdClientSolo/FlappyBirdClientSolo.hpp File Reference	927
19.125.1	Detailed Description	928
19.126	FlappyBirdClientSolo.hpp	928
19.127	/home/masina/Projects/Epitech/rtype/plugins/Game/FlappyBird/Client/Solo/include/↵ FlappyBirdClientSolo/Scenes/Game.hpp File Reference	929
19.127.1	Detailed Description	930

19.128	Game.hpp	930
19.129	/home/masina/Projects/Epitech/rtype/plugins/Game/FlappyBird/Client/Solo/src/entrypoint.cpp	
	File Reference	930
19.129.1	Function Documentation	931
19.129.1.1	entryPoint()	931
19.130	entrypoint.cpp	931
19.131	/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/src/entrypoint.cpp	
	File Reference	931
19.131.1	Function Documentation	932
19.131.1.1	entryPoint()	932
19.132	entrypoint.cpp	932
19.133	/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Solo/src/entrypoint.cpp	
	File Reference	932
19.133.1	Function Documentation	932
19.133.1.1	entryPoint()	932
19.134	entrypoint.cpp	932
19.135	/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Server/src/entrypoint.cpp	
	File Reference	933
19.135.1	Function Documentation	933
19.135.1.1	entryPoint()	933
19.136	entrypoint.cpp	933
19.137	/home/masina/Projects/Epitech/rtype/plugins/Network/Asio/Client/src/entrypoint.cpp	
	File Reference	933
19.137.1	Function Documentation	934
19.137.1.1	entryPoint()	934
19.138	entrypoint.cpp	934
19.139	/home/masina/Projects/Epitech/rtype/plugins/Network/Asio/Server/src/entrypoint.cpp	
	File Reference	934
19.139.1	Function Documentation	934
19.139.1.1	entryPoint()	934
19.140	entrypoint.cpp	934
19.141	/home/masina/Projects/Epitech/rtype/plugins/Game/FlappyBird/Client/Solo/src/flappyBirdClientSolo.cpp	
	File Reference	935
19.142	flappyBirdClientSolo.cpp	935
19.143	/home/masina/Projects/Epitech/rtype/plugins/Game/FlappyBird/Client/Solo/src/scenes/game.cpp	
	File Reference	935
19.143.1	Enumeration Type Documentation	936
19.143.1.1	PipeType	936
19.143.2	Function Documentation	936
19.143.2.1	getGapSize()	936
19.143.2.2	randomPipeType()	936
19.144	game.cpp	937
19.145	/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/include/RTypeClientMulti/Managers/ProjectileManager.hpp	
	File Reference	941
19.145.1	Detailed Description	941

19.146	ProjectileManager.hpp	941
19.147	/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Solo/include/↵ RTypeClientSolo/Managers/ProjectileManager.hpp File Reference	942
19.147.1	Detailed Description	943
19.148	ProjectileManager.hpp	943
19.149	/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/include/↵ RTypeClientMulti/Managers/StageManager.hpp File Reference	944
19.150	StageManager.hpp	944
19.151	/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Solo/include/↵ RTypeClientSolo/Managers/StageManager.hpp File Reference	945
19.151.1	Detailed Description	946
19.152	StageManager.hpp	946
19.153	/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/include/↵ RTypeClientMulti/RTypeClientMulti.hpp File Reference	947
19.153.1	Detailed Description	948
19.154	RTypeClientMulti.hpp	948
19.155	/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/include/↵ RTypeClientMulti/Scenes/ConfigMulti.hpp File Reference	949
19.155.1	Detailed Description	949
19.156	ConfigMulti.hpp	949
19.157	/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/include/↵ RTypeClientMulti/Scenes/CreateRoom.hpp File Reference	950
19.157.1	Detailed Description	951
19.158	CreateRoom.hpp	951
19.159	/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/include/↵ RTypeClientMulti/Scenes/GameMulti.hpp File Reference	952
19.159.1	Detailed Description	953
19.160	GameMulti.hpp	953
19.161	/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/include/↵ RTypeClientMulti/Scenes/JoinRoom.hpp File Reference	955
19.161.1	Detailed Description	956
19.162	JoinRoom.hpp	956
19.163	/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/include/↵ RTypeClientMulti/Scenes/ServerScene.hpp File Reference	956
19.163.1	Detailed Description	957
19.164	ServerScene.hpp	957
19.165	/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/include/↵ RTypeClientMulti/Scenes/WaitingRoom.hpp File Reference	958
19.165.1	Detailed Description	959
19.166	WaitingRoom.hpp	959
19.167	/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/include/↵ RTypeClientMulti/Systems/Collision.hpp File Reference	960
19.168	Collision.hpp	962
19.169	/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Solo/include/↵ RTypeClientSolo/Systems/Collision.hpp File Reference	963
19.169.1	Detailed Description	964

19.170 Collision.hpp	964
19.171 /home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/include/↵ RTypeClientMulti/Systems/Explosion.hpp File Reference	969
19.171.1 Detailed Description	970
19.172 Explosion.hpp	970
19.173 /home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Solo/include/↵ RTypeClientSolo/Systems/Explosion.hpp File Reference	971
19.173.1 Detailed Description	972
19.174 Explosion.hpp	972
19.175 /home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/include/↵ RTypeClientMulti/Systems/LoadingAnimation.hpp File Reference	974
19.176 LoadingAnimation.hpp	975
19.177 /home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Solo/include/↵ RTypeClientSolo/Systems/LoadingAnimation.hpp File Reference	976
19.177.1 Detailed Description	977
19.178 LoadingAnimation.hpp	977
19.179 /home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/include/↵ RTypeClientMulti/Systems/PlayerControllerMulti.hpp File Reference	978
19.179.1 Detailed Description	978
19.180 PlayerControllerMulti.hpp	979
19.181 /home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/include/↵ RTypeClientMulti/Systems/PlayerDirection.hpp File Reference	979
19.181.1 Macro Definition Documentation	981
19.181.1.1 M_PI	981
19.182 PlayerDirection.hpp	981
19.183 /home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Solo/include/↵ RTypeClientSolo/Systems/PlayerDirection.hpp File Reference	982
19.183.1 Detailed Description	983
19.183.2 Macro Definition Documentation	983
19.183.2.1 M_PI	983
19.184 PlayerDirection.hpp	983
19.185 /home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/include/↵ RTypeClientMulti/Systems/Projectile.hpp File Reference	985
19.185.1 Detailed Description	986
19.186 Projectile.hpp	986
19.187 /home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Solo/include/↵ RTypeClientSolo/Systems/Projectile.hpp File Reference	987
19.187.1 Detailed Description	988
19.188 Projectile.hpp	988
19.189 /home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/include/↵ RTypeClientMulti/Systems/Scrolling.hpp File Reference	989
19.190 Scrolling.hpp	990
19.191 /home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Solo/include/↵ RTypeClientSolo/Systems/Scrolling.hpp File Reference	991
19.191.1 Detailed Description	992
19.192 Scrolling.hpp	993

19.193	/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/include/↵ RTypeClientMulti/Systems/Weapon.hpp File Reference	994
19.193.1	Detailed Description	995
19.194	Weapon.hpp	995
19.195	/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Solo/include/↵ RTypeClientSolo/Systems/Weapon.hpp File Reference	996
19.195.1	Detailed Description	997
19.196	Weapon.hpp	997
19.197	/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/src/managers/stage↵ Manager.cpp File Reference	998
19.198	stageManager.cpp	999
19.199	/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Solo/src/managers/stage↵ Manager.cpp File Reference	1000
19.200	stageManager.cpp	1000
19.201	/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/src/rtype↵ ClientMulti.cpp File Reference	1001
19.202	rtypeClientMulti.cpp	1001
19.203	/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/src/scenes/config↵ Multi.cpp File Reference	1004
19.204	configMulti.cpp	1004
19.205	/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/src/scenes/create↵ Room.cpp File Reference	1006
19.205.1	Function Documentation	1006
19.205.1.1	keyToChar()	1006
19.206	createRoom.cpp	1007
19.207	/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/src/scenes/game↵ Multi.cpp File Reference	1011
19.208	gameMulti.cpp	1011
19.209	/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/src/scenes/join↵ Room.cpp File Reference	1020
19.210	joinRoom.cpp	1020
19.211	/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/src/scenes/↵ ServerScene.cpp File Reference	1025
19.211.1	Function Documentation	1025
19.211.1.1	keyToChar()	1025
19.212	ServerScene.cpp	1026
19.213	/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/src/scenes/waiting↵ Room.cpp File Reference	1030
19.214	waitingRoom.cpp	1030
19.215	/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/src/↵ Systems/playerControllerMulti.cpp File Reference	1035
19.216	playerControllerMulti.cpp	1036
19.217	/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/src/↵ Systems/projectile.cpp File Reference	1038
19.218	projectile.cpp	1039
19.219	/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/src/↵ Systems/weapon.cpp File Reference	1040

19.220	weapon.cpp	1040
19.221	/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Solo/src/systems/weapon.cpp File Reference	1042
19.222	weapon.cpp	1042
19.223	/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Solo/include/↵ RTypeClientSolo/RTypeClientSolo.hpp File Reference	1044
19.223.1	Detailed Description	1045
19.224	RTypeClientSolo.hpp	1045
19.225	/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Solo/include/↵ RTypeClientSolo/Scenes/ConfigSolo.hpp File Reference	1046
19.225.1	Detailed Description	1047
19.226	ConfigSolo.hpp	1047
19.227	/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Solo/include/↵ RTypeClientSolo/Scenes/GameSolo.hpp File Reference	1048
19.227.1	Detailed Description	1049
19.228	GameSolo.hpp	1049
19.229	/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Solo/include/↵ RTypeClientSolo/Systems/Enemy.hpp File Reference	1050
19.229.1	Detailed Description	1051
19.230	Enemy.hpp	1051
19.231	/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Solo/include/↵ RTypeClientSolo/Systems/HUD.hpp File Reference	1053
19.231.1	Detailed Description	1054
19.232	HUD.hpp	1054
19.233	/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Solo/include/↵ RTypeClientSolo/Systems/Spawn.hpp File Reference	1055
19.233.1	Detailed Description	1056
19.234	Spawn.hpp	1056
19.235	/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Solo/src/managers/projectile↵ Manager.cpp File Reference	1058
19.236	projectileManager.cpp	1058
19.237	/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Solo/src/rtype↵ ClientSolo.cpp File Reference	1059
19.238	rtypeClientSolo.cpp	1059
19.239	/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Solo/src/scenes/config↵ Solo.cpp File Reference	1060
19.240	configSolo.cpp	1061
19.241	/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Solo/src/scenes/game↵ Solo.cpp File Reference	1063
19.242	gameSolo.cpp	1064
19.243	/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Solo/src/systems/↵ HUD.cpp File Reference	1067
19.244	HUD.cpp	1067
19.245	/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Server/include/RType↵ Server/EntityManager.hpp File Reference	1068
19.245.1	Detailed Description	1069

19.246	EntityManager.hpp	1069
19.247	/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Server/include/RType↵ Server/RTypeServer.hpp File Reference	1071
19.247.1	Detailed Description	1072
19.248	RTypeServer.hpp	1072
19.249	/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Server/include/RType↵ Server/Systems/CollisionSystem.hpp File Reference	1073
19.249.1	Detailed Description	1074
19.250	CollisionSystem.hpp	1075
19.251	/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Server/include/RType↵ Server/Systems/EnemyAISystem.hpp File Reference	1076
19.251.1	Detailed Description	1077
19.252	EnemyAISystem.hpp	1077
19.253	/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Server/include/RType↵ Server/Systems/EnemySpawnSystem.hpp File Reference	1078
19.253.1	Detailed Description	1079
19.254	EnemySpawnSystem.hpp	1079
19.255	/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Server/include/RType↵ Server/WaveManager.hpp File Reference	1080
19.255.1	Detailed Description	1081
19.256	WaveManager.hpp	1082
19.257	/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Server/src/Entity↵ Manager.cpp File Reference	1083
19.257.1	Detailed Description	1083
19.258	EntityManager.cpp	1084
19.259	/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Server/src/rtype↵ Server.cpp File Reference	1090
19.259.1	Detailed Description	1091
19.260	rtypeServer.cpp	1091
19.261	/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Server/src/Systems/↵ CollisionSystem.cpp File Reference	1096
19.261.1	Detailed Description	1097
19.262	CollisionSystem.cpp	1097
19.263	/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Server/src/Systems/↵ EnemyAISystem.cpp File Reference	1100
19.263.1	Detailed Description	1100
19.264	EnemyAISystem.cpp	1100
19.265	/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Server/src/Systems/↵ EnemySpawnSystem.cpp File Reference	1105
19.265.1	Detailed Description	1106
19.266	EnemySpawnSystem.cpp	1106
19.267	/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Server/src/Wave↵ Manager.cpp File Reference	1108
19.267.1	Detailed Description	1109
19.268	WaveManager.cpp	1109

19.269	/home/masina/Projects/Epitech/rtype/plugins/Network/Asio/Client/include/AsioClient/AsioClient.hpp File Reference	1113
19.269.1	Detailed Description	1114
19.270	AsioClient.hpp	1114
19.271	/home/masina/Projects/Epitech/rtype/plugins/Network/Asio/Client/src/asioClient.cpp File Reference	1119
19.272	asioClient.cpp	1120
19.273	/home/masina/Projects/Epitech/rtype/plugins/Network/Asio/Server/include/AsioServer/AsioServer.hpp File Reference	1132
19.273.1	Detailed Description	1133
19.274	AsioServer.hpp	1133
19.275	/home/masina/Projects/Epitech/rtype/plugins/Network/Asio/Server/src/asioServer.cpp File Reference	1139
19.276	asioServer.cpp	1139
19.277	/home/masina/Projects/Epitech/rtype/modules/Utils/include/Utils/Common.hpp File Reference	1158
19.277.1	Detailed Description	1161
19.278	Common.hpp	1161
19.279	/home/masina/Projects/Epitech/rtype/server/include/Server/Common.hpp File Reference	1162
19.279.1	Detailed Description	1164
19.280	Common.hpp	1164
19.281	/home/masina/Projects/Epitech/rtype/modules/Engine/include/Engine/Interfaces/IScene.hpp File Reference	1164
19.281.1	Detailed Description	1165
19.282	IScene.hpp	1165
19.283	/home/masina/Projects/Epitech/rtype/server/include/Server/Interfaces/IScene.hpp File Reference	1166
19.283.1	Detailed Description	1168
19.284	IScene.hpp	1168
19.285	/home/masina/Projects/Epitech/rtype/modules/Engine/include/Engine/SceneManager.hpp File Reference	1169
19.285.1	Detailed Description	1169
19.286	SceneManager.hpp	1169
19.287	/home/masina/Projects/Epitech/rtype/server/include/Server/SceneManager.hpp File Reference	1170
19.287.1	Detailed Description	1171
19.288	SceneManager.hpp	1171
19.289	/home/masina/Projects/Epitech/rtype/server/include/Server/Scenes/Level_0.hpp File Reference	1172
19.289.1	Detailed Description	1172
19.290	Level_0.hpp	1172
19.291	/home/masina/Projects/Epitech/rtype/server/include/Server/Scenes/Level_1.hpp File Reference	1172
19.291.1	Detailed Description	1173
19.292	Level_1.hpp	1173

19.293	/home/masina/Projects/Epitech/rtype/server/include/Server/Server.hpp File Reference	1173
19.293.1	Detailed Description	1174
19.294	Server.hpp	1174
19.295	/home/masina/Projects/Epitech/rtype/modules/ECS/include/ECS/Systems/Systems.hpp File Reference	1175
19.295.1	Detailed Description	1176
19.296	Systems.hpp	1176
19.297	/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/include/↵ RTypeClientMulti/Systems/Systems.hpp File Reference	1176
19.297.1	Detailed Description	1177
19.298	Systems.hpp	1177
19.299	/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Solo/include/↵ RTypeClientSolo/Systems/Systems.hpp File Reference	1177
19.299.1	Detailed Description	1178
19.300	Systems.hpp	1178
19.301	/home/masina/Projects/Epitech/rtype/server/include/Server/Systems/Systems.hpp File Reference	1179
19.301.1	Detailed Description	1179
19.302	Systems.hpp	1179
19.303	/home/masina/Projects/Epitech/rtype/server/src/server.cpp File Reference	1180
19.304	server.cpp	1180
Index		1183

Chapter 1

R-Type

This project reimagines R-Type using a modular game engine entirely written in C++23. It features:

- A multithreaded server
- A graphical client
- A plugin-based engine (audio, rendering, networking)
- A custom Entity-Component-System (ECS)

Each subsystem (client, server, and engine) is completely decoupled and communicates via well-defined interfaces.

1.1 Supported Platforms

Platform	Compiler	Status
Linux	g++	
macOS	g++	
Windows	MSVC	

1.2 Project Structure

```
flowchart LR
    subgraph App
        subgraph client [Client]
            A[Client]
            A -->|.a/.lib| B[Engine]
            B -->|.a/.lib| C[ECS]
            A -->|.so/.dll| D[IGameClient]
            B -->|.so/.dll| E[IAudio]
            B -->|.so/.dll| F[INetworkClient]
            B -->|.so/.dll| G[IRenderer]
        end

        subgraph server [Server]
            H[Server]
            H -->|.so/.dll| I[IGameServer]
            H -->|.so/.dll| J[INetworkServer]
        end

        A <==>|TCP/UDP| H
    end
```

1.3 Prerequisites

Make sure you have the following dependencies installed on your system:

- [CMake 4.0.0](#)
- [C++23](#)

1.4 Build and Run

Important

When cloning the project, you should also initialize the submodules:

```
git clone --recurse-submodules git@github.com:bobis33/R-Type.git
```

If you already cloned the project, you can initialize the submodules with:

```
git submodule update --init --recursive
```

1.4.1 Unix (Linux, macOS)

```
./scripts/unix/build.sh release
## Or
cmake -S . -B cmake-build-release -G "Ninja" -DCMAKE_BUILD_TYPE=Release -DCMAKE_CXX_COMPILER=g++
      -DCMAKE_C_COMPILER=gcc
cmake --build cmake-build-release -- -j4
## Then
./cmake-build-release/bin/r-type_client ## client
./cmake-build-release/bin/r-type_server ## server
```

1.4.2 Windows

```
cmake -S . -B cmake-build-release -G "Visual Studio 17 2022" -A x64 -DCMAKE_BUILD_TYPE=Release
cmake --build cmake-build-release --config Release
## Then
cmake-build-release\bin\r-type_client.exe ## client
cmake-build-release\bin\r-type_server.exe ## server
```

1.5 Documentation

API documentation is generated using Doxygen and deployed on [GitHub Pages](#). You can find the same documentation as PDF [here](#). More specific documentation for each part of the project can be found in their respective directories:

- [Client documentation](#)
- [Server documentation](#)
- [Plugins documentation](#)
- [Modules documentation](#)

1.6 Technology Stack Analysis

For a detailed comparative study of the technologies used in this project and the reasoning behind our architectural choices, see: [Comparative study](#)

1.7 External Libraries

All dependencies are included as submodules in the [third-party](#) directory.

1.8 Contributing

Want to contribute? See [CONTRIBUTING.md](#).

1.9 License

This project is licensed under the MIT License - see the [LICENSE](#) file for details.

Chapter 2

R-Type | Client

This document details how to configure and run the R-Type client.

2.1 Configuration

You can configure the client using a JSON file:

```
./r-type_client --config config-client.json
```

Here is an example configuration file:

```
{
  "client": {
    "server_ip": "0.0.0.0",
    "server_port": 4242,
    "player_name": "Player"
  },
  "plugins": {
    "audio": "/plugins/my_audio_plugin.so",
    "network": "/plugins/my_network_client_plugin.so",
    "renderer": "/plugins/my_renderer_plugin.so"
  },
  "window": {
    "fullscreen": false,
    "width": 920,
    "height": 540,
    "frame_limit": 240
  }
}
```

2.2 Plugins

The client uses three main plugin types:

- Audio: Implements IAudio
- Renderer: Implements IRenderer
- Network client: Implements INetworkClient

Refer to the plugin folder and the interfaces documentation to implement custom plugins.

2.3 Troubleshooting / Tips

- Ensure your JSON paths are correct for plugins.

Chapter 3

R-Type | Server

This document details how to configure and run the R-Type server.

3.1 Configuration

You can configure the server using a JSON file:

```
./r-type_server --config config-server.json
```

Here is an example configuration file:

```
{
  "host": "0.0.0.0",
  "port": 2560,
  "plugins": {
    "network": "/plugins/my_network_server_plugin.so"
  }
}
```

3.2 Plugins

The server requires a network plugin implementing `INetworkServer`.

Refer to the plugin folder and the interfaces documentation to implement custom plugins.

3.3 Networking / Protocol

- The server communicates with clients using UDP.
- Clients must use the correct server IP and port.
- Session management, player events, and game state updates are handled through the network plugin.

3.4 Troubleshooting / Tips

- Ensure your JSON paths are correct for plugins.

Chapter 4

How to create your own plugin

If you want u can implement your own plugin. For more information, check readme in the specific directory, depending on your plugins' implementation:

- [Audio](#)
- [Game](#)
- [Network](#)
- [Renderer](#)

Chapter 5

Audio plugin

5.1 How to add a new audio plugin

First you will have to create a directory corresponding on ur implementation inside the plugins/Audio directory. For example, if you are implementing an audio plugin for the "SuperSound" audio library, you would create a directory named SuperSound. Inside this directory, you will need to create at least:

- CMakeLists.txt

```
project(audio_SuperSound
    DESCRIPTION "Audio SuperSound Plugin"
    LANGUAGES C CXX
)

set(SRC_DIR "${PROJECT_SOURCE_DIR}/src")
set(INCLUDE_DIR "${PROJECT_SOURCE_DIR}/include")

file(GLOB_RECURSE SOURCES "${SRC_DIR}/*.cpp")
file(GLOB_RECURSE HEADERS "${INCLUDE_DIR}/*.hpp")

add_library(${PROJECT_NAME} SHARED ${SOURCES} ${HEADERS})
target_include_directories(${PROJECT_NAME} PRIVATE
    ${INCLUDE_DIR}
)
target_compile_options(${PROJECT_NAME} PRIVATE ${WARNING_FLAGS})
target_link_libraries(${PROJECT_NAME} PRIVATE r-type_modules_client)
target_compile_features(${PROJECT_NAME} PRIVATE cxx_std_23)
set_target_properties(${PROJECT_NAME} PROPERTIES
    POSITION_INDEPENDENT_CODE ON
    PREFIX ""
)
```

If needed, link to your dependency libraries using `target_link_libraries`.

- Include dir with your header files (e.g., `include/SuperSound/AudioSuperSound.hpp`). Dont forget to make your class inherit from `IAudio`.

```
++
///
/// @file SuperSoundAudio.hpp
/// @brief SuperSoundAudio class declaration
/// @namespace eng
///

#pragma once

#include "Interfaces/IAudio.hpp"

namespace eng
{
    ///
    /// @class SuperSoundAudio
    /// @brief Class for audio management
    /// @namespace eng
    ///
    class SuperSoundAudio final : public IAudio
    {
    public:
        SuperSoundAudio() = default;
        ~SuperSoundAudio() override = default;

        SuperSoundAudio(const SuperSoundAudio &) = delete;
    };
}
```

```

SuperSoundAudio &operator=(const SuperSoundAudio &) = delete;
SuperSoundAudio(SuperSoundAudio &&) = delete;
SuperSoundAudio &operator=(SuperSoundAudio &&) = delete;

[[nodiscard]] const std::string getName() const override { return "Audio_SuperSound"; }
[[nodiscard]] utl::PluginType getType() const override { return utl::PluginType::AUDIO; }

void createAudio(const std::string &path, float volume, bool loop, const std::string &name) override;
void playAudio(const std::string &name) override;
void setVolume(const std::string &name, float volume) override;
void setLoop(const std::string &name, bool loop) override;
void stopAudio(const std::string &name) override;
Status isPlaying(const std::string &name) override;

private:

}; // class SuperSoundAudio

} // namespace eng

```

- Source dir with your implementation files (e.g., src/SuperSoundAudio.cpp), and your entrypoint:

```

++
#include <memory>

#include "SFMLAudio/SFMLAudio.hpp"

extern "C"
{
    PLUGIN_EXPORT eng::IAudio *entryPoint() { return std::make_unique<eng::SuperSoundAudio>().release(); }
}

```

Then your setup is done! You just need to add in the plugin/audio CMakeLists.txt the directory of your plugin:

```
add_subdirectory(SuperSound)
```

Chapter 6

Game plugin

6.1 How to add a new game plugin

First you will have to create a directory corresponding to your implementation inside the plugins/Network directory. For example, if you are implementing a game plugin for the "Snake" game, you would create a directory named Snake. Inside this directory, you will need to create at least:

- Client and Server directory
- CMakeLists.txt
add_subdirectory(Client)
add_subdirectory(Server)

6.1.1 Client

- Client/CMakeLists.txt

```
project(game_snakeClient
    DESCRIPTION "Game Snake client Plugin"
    LANGUAGES C CXX
)

set(SRC_DIR "${PROJECT_SOURCE_DIR}/src")
set(INCLUDE_DIR "${PROJECT_SOURCE_DIR}/include")

file(GLOB_RECURSE SOURCES "${SRC_DIR}/*.cpp")
file(GLOB_RECURSE HEADERS "${INCLUDE_DIR}/*.hpp")

add_library(${PROJECT_NAME} SHARED ${SOURCES} ${HEADERS})
target_include_directories(${PROJECT_NAME} PRIVATE
    ${INCLUDE_DIR}
)
target_compile_options(${PROJECT_NAME} PRIVATE ${WARNING_FLAGS})
target_link_libraries(${PROJECT_NAME} PRIVATE
    r-type_modules_client
    r-type_game_client_shared
)
target_compile_features(${PROJECT_NAME} PRIVATE cxx_std_23)
set_target_properties(${PROJECT_NAME} PROPERTIES
    POSITION_INDEPENDENT_CODE ON
    PREFIX ""
)
```

If needed, link to your dependency libraries using target_link_libraries.

- Include dir with your header files (e.g., include/UnixSocketClient/UnixSocketClient.hpp). Don't forget to make your class inherit from INetworkClient.

```
++
///
/// @file SnakeClient.hpp
/// @brief snake client game class declaration
/// @namespace eng
///
#pragma once

#include "Interfaces/IGameClient.hpp"

namespace eng
{
```

```

///
/// @class SnakeClient
/// @brief Class for snake game client
/// @namespace eng
///
class SnakeClient final : public IGameClient
{
public:
    SnakeClient() = default;
    ~SnakeClient() override = default;

    SnakeClient(const SnakeClient &) = delete;
    SnakeClient &operator=(const SnakeClient &) = delete;
    SnakeClient(SnakeClient &&) = delete;
    SnakeClient &operator=(SnakeClient &&) = delete;

    void update(float deltaTime, unsigned int width, unsigned int height) override;
    void init(eng::Engine &engine, utl::cli::AppConfig &appConfig, bool &showDebug,
              const eng::id menuSceneId) override
    {
        m_engine = &engine;
        m_appConfig = &appConfig;
        setupScenes(showDebug, menuSceneId);
    }

    [[nodiscard]] unsigned int getMainSceneId() const override { return m_mainSceneId; }

private:
    eng::Engine *m_engine = nullptr;
    utl::cli::AppConfig *m_appConfig = nullptr;
    eng::id m_mainSceneId;

    void setupScenes(bool &showDebug, eng::id menuSceneId);
}; // class SnakeClient
} // namespace eng

```

- Source dir with your implementation files (e.g., src/SnakeClient.cpp), and your entrypoint:

```

++
#include <memory>

#include "SnakeClient/SnakeClient.hpp"

extern "C"
{
    PLUGIN_EXPORT eng::IGameClient *entryPoint() { return std::make_unique<eng::SnakeClient>().release(); }
}

```

6.1.2 Server

- Server/CMakeLists.txt

```

project(game_SnakeServer
    DESCRIPTION "Game snake server Plugin"
    LANGUAGES C CXX
)

set(SRC_DIR "${PROJECT_SOURCE_DIR}/src")
set(INCLUDE_DIR "${PROJECT_SOURCE_DIR}/include")

file(GLOB_RECURSE SOURCES "${SRC_DIR}/*.cpp")
file(GLOB_RECURSE HEADERS "${INCLUDE_DIR}/*.hpp")

add_library(${PROJECT_NAME} SHARED ${SOURCES} ${HEADERS})
target_include_directories(${PROJECT_NAME} PRIVATE
    ${INCLUDE_DIR}
)
target_compile_options(${PROJECT_NAME} PRIVATE ${WARNING_FLAGS})
target_link_libraries(${PROJECT_NAME} PRIVATE
    r-type_modules_server
    r-type_game_client_shared
)
target_compile_features(${PROJECT_NAME} PRIVATE cxx_std_23)
set_target_properties(${PROJECT_NAME} PROPERTIES
    POSITION_INDEPENDENT_CODE ON
    PREFIX ""
)

```

If needed, link to your dependency libraries using `target_link_libraries`.

- Include dir with your header files (e.g., include/SnakeServer/SnakeServer.hpp). Don't forget to make your class inherit from `IGameServer`.

```

++
///
/// @file INetworkServer.hpp
/// @brief Snake Server network class declaration
/// @namespace eng
///

#pragma once

#include "Interfaces/IGameServer.hpp"

namespace eng
{
    ///
    /// @class SnakeServer
    /// @brief Class for snake game server
    /// @namespace eng
    ///
    class SnakeServer final : public IGameServer
    {
    public:
        SnakeServer() = default;
        ~SnakeServer() override = default;

        SnakeServer(const SnakeServer &) = delete;
        SnakeServer &operator=(const SnakeServer &) = delete;
        SnakeServer(SnakeServer &&) = delete;
        SnakeServer &operator=(SnakeServer &&) = delete;

        [[nodiscard]] State getState() const override;

        void start() override;
        void stop() override;
        void update(float deltaTime) override;

    }; // class SnakeServer
} // namespace eng

```

- Source dir with your implementation files (e.g., src/SnakeServer.cpp), and your entrypoint:

```

++
#include <memory>

#include "SnakeServer/SnakeServer.hpp"

extern "C"
{
    PLUGIN_EXPORT eng::IGameServer *entryPoint() { return std::make_unique<eng::SnakeServer>().release(); }
}

```


Chapter 7

Network plugin

7.1 How to add a new network plugin

First you will have to create a directory corresponding on ur implementation inside the plugins/Network directory. For example, if you are implementing a network plugin for the "Unix socket" library, you would create a directory named UnixSocket. Inside this directory, you will need to create at least:

- Client and Server directory
- CMakeLists.txt
add_subdirectory(Client)
add_subdirectory(Server)

7.1.1 Client

- Client/CMakeLists.txt

```
project(network_UnixSocketClient
    DESCRIPTION "Network Unix Socket client Plugin"
    LANGUAGES C CXX
)

set(SRC_DIR "${PROJECT_SOURCE_DIR}/src")
set(INCLUDE_DIR "${PROJECT_SOURCE_DIR}/include")

file(GLOB_RECURSE SOURCES "${SRC_DIR}/*.cpp")
file(GLOB_RECURSE HEADERS "${INCLUDE_DIR}/*.hpp")

add_library(${PROJECT_NAME} SHARED ${SOURCES} ${HEADERS})
target_include_directories(${PROJECT_NAME} PRIVATE ${INCLUDE_DIR})
target_compile_options(${PROJECT_NAME} PRIVATE ${WARNING_FLAGS})
target_link_libraries(${PROJECT_NAME} PRIVATE r-type_modules_client)
target_compile_features(${PROJECT_NAME} PRIVATE cxx_std_23)
set_target_properties(${PROJECT_NAME} PROPERTIES
    POSITION_INDEPENDENT_CODE ON
    PREFIX ""
)
```

If needed, link to your dependency libraries using target_link_libraries.

- Include dir with your header files (e.g., include/UnixSocketClient/UnixSocketClient.hpp). Don't forget to make your class inherit from INetworkClient.

```
++
///
/// @file UnixSocketClient.hpp
/// @brief Unix Socket client network class declaration
/// @namespace eng
///

#pragma once

#include "Interfaces/INetworkClient.hpp"

namespace eng
{
    ///
    /// @class UnixSocketClient
    /// @brief Class for network with unix socket
    /// @namespace eng
```

```

///
class UnixSocketClient final : public INetworkClient
{
public:
    UnixSocketClient() = default;
    ~UnixSocketClient() override = default;

    UnixSocketClient(const UnixSocketClient &) = delete;
    UnixSocketClient &operator=(const UnixSocketClient &) = delete;
    UnixSocketClient(UnixSocketClient &&) = delete;
    UnixSocketClient &operator=(UnixSocketClient &&) = delete;

    [[nodiscard]] bool isConnected() const override;
    [[nodiscard]] ConnectionState getConnectionState() const override;
    [[nodiscard]] std::uint32_t getSessionId() const override;
    [[nodiscard]] std::uint16_t getServerTickRate() const override;
    [[nodiscard]] std::uint32_t getLatency() const override;

    void setPlayerName(const std::string &playerName) override;
    void setClientCapabilities(std::uint32_t caps) override;

}; // class UnixSocketClient

} // namespace eng

```

- Source dir with your implementation files (e.g., src/UnixSocketClient.cpp), and your entrypoint:

```

++
#include <memory>

#include "UnixSocketClient/UnixSocketClient.hpp"

extern "C"
{
    PLUGIN_EXPORT eng::INetworkClient *entryPoint() { return std::make_unique<eng::UnixSocketClient>().release(); }
}

```

7.1.2 Server

- Server/CMakeLists.txt

```

project(network UnixSocketServer
    DESCRIPTION "Network Unix Socket server Plugin"
    LANGUAGES C CXX
)

set(SRC_DIR "${PROJECT_SOURCE_DIR}/src")
set(INCLUDE_DIR "${PROJECT_SOURCE_DIR}/include")

file(GLOB_RECURSE SOURCES "${SRC_DIR}/*.cpp")
file(GLOB_RECURSE HEADERS "${INCLUDE_DIR}/*.hpp")

add_library(${PROJECT_NAME} SHARED ${SOURCES} ${HEADERS})
target_include_directories(${PROJECT_NAME} PRIVATE
    ${INCLUDE_DIR}
)
target_compile_options(${PROJECT_NAME} PRIVATE ${WARNING_FLAGS})
target_link_libraries(${PROJECT_NAME} PRIVATE r-type_modules_server)
target_compile_features(${PROJECT_NAME} PRIVATE cxx_std_23)
set_target_properties(${PROJECT_NAME} PROPERTIES
    POSITION_INDEPENDENT_CODE ON
    PREFIX ""
)

```

If needed, link to your dependency libraries using `target_link_libraries`.

- Include dir with your header files (e.g., include/UnixSocketServer/UnixSocketServer.hpp). Don't forget to make your class inherit from `INetworkServer`.

```

++
///
/// @file INetworkServer.hpp
/// @brief Unix Socket server network class declaration
/// @namespace eng
///

#pragma once

#include "Interfaces/INetworkServer.hpp"

namespace eng
{
    ///
    /// @class UnixSocketServer

```

```

/// @brief Class for network with unix socket
/// @namespace eng
///
class UnixSocketServer final : public INetworkServer
{
public:
    UnixSocketServer() = default;
    ~UnixSocketServer() override = default;

    UnixSocketServer(const UnixSocketServer &) = delete;
    UnixSocketServer &operator=(const UnixSocketServer &) = delete;
    UnixSocketServer(UnixSocketServer &&) = delete;
    UnixSocketServer &operator=(UnixSocketServer &&) = delete;

    void init(const std::string &host, std::uint16_t port) override;
    void start() override;
    void stop() override;
    void update() override;
    void sendToClient(std::uint32_t sessionId, const std::vector<std::uint8_t> &data,
        bool reliable = false) override;
    void sendToAllClients(const std::vector<std::uint8_t> &data, bool reliable = false) override;
    void disconnectClient(std::uint32_t sessionId) override;

    [[nodiscard]] std::size_t getClientCount() const override;
    [[nodiscard]] std::vector<std::uint32_t> getConnectedSessions() const override;
    [[nodiscard]] bool isRunning() const override;

    void setTickRate(std::uint16_t tickRate) override;
    void setServerCapabilities(std::uint32_t caps) override;

}; // class UnixSocketServer

} // namespace eng

```

- Source dir with your implementation files (e.g., src/UnixSocketServer.cpp), and your entrypoint:

```

++
#include <memory>

#include "UnixSocketServer/UnixSocketServer.hpp"

extern "C"
{
    PLUGIN_EXPORT eng::INetworkClient *entryPoint() { return std::make_unique<eng::UnixSocketServer>().release(); }
}

```


Chapter 8

Renderer plugin

8.1 How to add a new renderer plugin

First you will have to create a directory corresponding on ur implementation inside the plugins/Renderer directory. For example, if you are implementing a renderer plugin for the "SDL" graphic library, you would create a directory named SDL. Inside this directory, you will need to create at least:

- CMakeLists.txt

```
project(renderer_SDL
    DESCRIPTION "Renderer SDL Plugin"
    LANGUAGES C CXX
)

set(SRC_DIR "${PROJECT_SOURCE_DIR}/src")
set(INCLUDE_DIR "${PROJECT_SOURCE_DIR}/include")

file(GLOB_RECURSE SOURCES "${SRC_DIR}/*.cpp")
file(GLOB_RECURSE HEADERS "${INCLUDE_DIR}/*.hpp")

add_library(${PROJECT_NAME} SHARED ${SOURCES} ${HEADERS})
target_include_directories(${PROJECT_NAME} PRIVATE
    ${INCLUDE_DIR}
)
target_compile_options(${PROJECT_NAME} PRIVATE ${WARNING_FLAGS})
target_link_libraries(${PROJECT_NAME} PRIVATE r-type_modules_client)
target_compile_features(${PROJECT_NAME} PRIVATE cxx_std_23)
set_target_properties(${PROJECT_NAME} PROPERTIES
    POSITION_INDEPENDENT_CODE ON
    PREFIX ""
)
```

If needed, link to your dependency libraries using `target_link_libraries`.

- Include dir with your header files (e.g., `include/SDL/SDL.hpp`). Don't forget to make your class inherit from `IRenderer`.

```
++
///
/// @file SDL.hpp
/// @brief SDL renderer class declaration
/// @namespace eng
///

#pragma once

#include "Interfaces/IRenderer.hpp"

namespace eng
{
    ///
    /// @class SDL
    /// @brief Class for rendering with SDL
    /// @namespace eng
    ///
    class SDL final : public IRenderer
    {
    public:
        SDL() = default;
        ~SDL() override = default;

        SDL(const SDL &) = delete;
    };
}
```

```

SDL &operator=(const SDL &) = delete;
SDL(SDL &&) = delete;
SDL &operator=(SDL &&) = delete;

[[nodiscard]] const std::string getName() const override { return "Renderer_SFML"; }
[[nodiscard]] utl::PluginType getType() const override { return utl::PluginType::RENDERER; }

void createWindow(const std::string &title, unsigned int height, unsigned int width,
                 unsigned int frameLimit, bool fullscreen) override;
bool windowIsOpen() const override;
void closeWindow() override;
void clearWindow(Color color) override;
void displayWindow() override;
WindowSize getWindowSize() override;
void setWindowIcon(const std::string &path) override;

bool pollEvent(Event &event) override;
void setFrameLimit(unsigned int frameLimit) override;

void createFont(const std::string &name, const std::string &path) override;
void createText(Text text) override;
void setTextContent(const std::string &name, const std::string &content) override;
void setTextPosition(const std::string &name, float x, float y) override;
void setTextColor(const std::string &name, Color color) override;

void drawText(const std::string &name) override;
void createTexture(const std::string &name, const std::string &path) override;
void createSprite(const std::string &name, const std::string &textureName, float x, float y, float scale_x,
                 float scale_y, int fx, int fy, int fnx, int fny) override;
void setSpritePosition(const std::string &name, float x, float y) override;
void setSpriteTexture(const std::string &name, const std::string &path) override;
void setSpriteFrame(const std::string &name, int fx, int fy, int fnx, int fny) override;
void setSpriteScale(const std::string &name, int x, int y) override;
void setSpriteColor(const std::string &name, Color color) override;
void drawSprite(const std::string &name) override;
void setSpriteRotation(const std::string &name, float rotation) override;
void rotateSprite(const std::string &name, float rotation) override;
void setSpriteOrigin(const std::string &name) override;

void createCircleShape(CircleShape circleShape) override;
void setCircleShapePosition(const std::string &name, float x, float y) override;
void drawCircleShape(const std::string &name) override;

void drawPoint(float x, float y, Color color) override;
void renderGui(const WindowSize &>windowSize) override;

}; // class SDL

} // namespace eng

```

- Source dir with your implementation files (e.g., src/SDL.cpp), and your entrypoint:

```

++
#include <memory>

#include "SFMLAudio/SFMLAudio.hpp"

extern "C"
{
    PLUGIN_EXPORT eng::IRenderer *entryPoint() { return std::make_unique<eng::SDL>().release(); }
}

```

Then your setup is done! You just need to add in the plugin/Renderer CMakeLists.txt the directory of your plugin:

```
add_subdirectory(SDL)
```

Chapter 9

Technical Analysis & Technology Comparison

9.1 Evaluation Criteria

Our primary criteria for technology selection were:

- Portability: Cross-platform compatibility (Windows, Linux, macOS)
 - Performance: Execution efficiency and optimization
 - Ease of Use: Learning curve and integration simplicity
 - Modern C++ Compatibility: Support for C++17/20/23
 - Ecosystem: Documentation, community, and maintenance
-

9.2 Network: Asio vs Alternatives

9.2.1 Selected: Asio (Standalone)

Why Asio?

- Header-only: Lightweight, no separate compilation required
- Asynchronous approach: Efficient handling of multiple connections
- Simple to use: Intuitive API for network operations
- High performance: Optimized non-blocking I/O
- Modern C++: Native use of C++17/20 concepts

Why not the alternatives?

- SFML Network: Too lightweight, simple communication protocol, limited modularity
 - UNIX Sockets: Limited to UNIX systems, adds cross-platform complexity
 - Boost.Asio: Heavy Boost dependency vs. lightweight header-only standalone
-

9.3 Graphics & Audio: SFML vs Alternatives

9.3.1 Selected: SFML (Simple and Fast Multimedia Library)

Why SFML?

- Popular graphics API: Wide adoption in the community
 - Simple to integrate: Straightforward installation and configuration
 - High performance: Good balance between performance and ease of use
-

- Team familiarity: Mastered by all team members
- Complete multimedia: Graphics, Audio, Network, Window in one package

Why not the alternatives?

- Raylib: Performant API but too minimalistic, limited audio features
- SDL2: Complete and performant, but more complex to integrate and unfamiliar to the team
- Direct OpenGL: Too low-level, prohibitive development time

9.4 Testing: GoogleTest vs Alternatives

9.4.1 Selected: GoogleTest

Why GoogleTest?

- Modern and complete framework: Advanced features (mocking, parameterized tests)
- Extremely simple to use: Intuitive and clear syntax
- Industry standard: Used by Google and numerous open-source projects
- CMake integration: Excellent native support
- Detailed reports: Formatted and informative output

Why not the alternatives?

- Boost.Test: Too heavy for just unit testing
- Catch2: Lighter, but more limited unit testing features
- CPPUnit: Aging API and poor documentation

9.5 Architectural Summary

9.5.1 Final Technology Stack

R-Type	
Network Layer	Asio (Standalone)
Graphics/Audio	SFML 2.6+
Testing	GoogleTest
Build System	CMake 3.16+
Language	C++17/20

9.5.2 Stack Advantages

- Coherence: Technologies that integrate naturally
- Maintainability: Stable and well-documented APIs
- Scalability: Support for modern C++ standards
- Team Experience: Reduced learning time

Chapter 10

Network Protocol Specification

Author: Elliot Mas Date: 2025-10-02

10.1 1. Introduction

This document specifies the R-Type multiplayer game protocol. It targets fast, bandwidth-efficient client/server communication over UDP with:

- application-level reliability,
- compact binary payloads,
- extensibility for future features.

10.2 2. Terminology

Client : a game instance controlled by a player Server : the authoritative game host managing the world state Packet : a datagram transmitted between client and server Session : an authenticated association between client and server, identified by a Session ID

10.3 3. Transport

Protocol : UDP Default Port : 4567 Byte Order : Big Endian (network byte order) Max Msg Size : 512 bytes (after compression) Reliability : Implemented at application level Rate Limits : ≤ 200 packets/sec per client recommended

10.4 4. Packet Header

All packets share this structure:

Type (8b) : packet type Length (16b) : payload length in bytes Flags (16b) : bitfield 0x0001 = ACK↔
_REQ 0x0002 = RELIABLE 0x0004 = FRAG 0x0008 = COMPRESSED Reserved (16b) : must be 0 Sequence Number (32b): per-session, monotonic Session ID (32b) : server-assigned after CONNECT Payload : type-specific data

All multi-byte fields are encoded in big endian.

10.5 5. Packet Types

0x01 - CONNECT 0x02 - DISCONNECT 0x03 - WORLD_STATE 0x04 - PING 0x05 - PONG 0x06 - ERROR 0x07 - ACK 0x08 - ENTITY_EVENT 0x09 - CONNECT_ACCEPT

CONNECT (0x01) Payload: uint8 name_len bytes player_name[name_len] uint32 client_caps

CONNECT_ACCEPT (0x09) Payload: uint32 session_id uint16 tick_rate_hz uint16 mtu_payload_↔
bytes uint32 server_caps

DISCONNECT (0x02) Payload: uint16 reason_code

WORLD_STATE (0x03) Payload: uint32 server_tick uint16 entity_count repeated entity { uint32 id uint16 type float32 x, y, vx, vy uint8 state_flags }

ENTITY_EVENT (0x08) Payload: uint32 server_tick uint16 event_count repeated event { uint8 event_type // see EventType enum below uint32 entity_id uint8 data_len bytes data[data_len] }

PING (0x04) / PONG (0x05) Payload: uint32 nonce uint32 send_time_ms

ACK (0x07) Payload: uint32 cumulative_ack uint32 ack_bits (32-bit SACK window)

ERROR (0x06) Payload: uint16 error_code uint16 msg_len bytes description[msg_len]

10.6 6. Reliability & Fragmentation

- Sequence numbers: 32-bit, wraparound
- ACK packets: selective ACK
- Retransmission timeout: 200ms -> 1.6s max
- Max retries: 6
- Fragmentation if FRAG flag set: uint16 frag_id uint16 frag_index uint16 frag_count

10.7 7. Disconnect & Error Codes

Disconnect Reasons: 0 unspecified 1 client_request 2 timeout 3 protocol_error 4 server_shutdown 5 server_full 6 banned

Error Codes: 1 invalid_payload 2 unauthorized_session 3 rate_limited 4 internal_error

10.8 8. Event Types

enum class EventType : uint8 { SPAWN = 0x01, DESPAWN = 0x02, DAMAGE = 0x03, SCORE = 0x04, POWERUP = 0x05, INPUT = 0x06, CUSTOM = 0xFF };

Each event in ENTITY_EVENT payload has:

- event_type (uint8)
- entity_id (uint32)
- data_len (uint8)
- data[data_len] (event-specific content)

Example per-type data:

- SPAWN: { uint16 entity_type; float32 x; float32 y; }
- DESPAWN: { uint8 reason; }
- DAMAGE: { uint16 amount; uint32 source_id; }
- SCORE: { uint16 points; }
- POWERUP: { uint16 powerup_type; }
- INPUT: { uint16 buttons; uint8 direction; uint8 shooting; uint32 client_time_ms; }
- CUSTOM: opaque blob

10.9 9. Security

- Session ID bound to (IP, port)
- Sequence numbers prevent replay

10.10 10. Timing

- Tick rate: advertised by server
- Keepalive: PING every 5s, disconnect if no PONG after 15s
- Clients interpolate WORLD_STATE, corrected by ENTITY_EVENT

10.11 Appendix A: Example Packets

CONNECT (client → server) Type=0x01, Flags=0, Seq=1, SID=0 Payload: [05][“Elliot”][0x00000001]
CONNECT_ACCEPT (server → client) Type=0x09, Flags=RELIABLE|ACK_REQ Payload: session↔
_id=0xA1B2C3D4, tick=50, mtu=508, caps=...
ENTITY_EVENT (client → server, example INPUT) Type=0x08 Payload: tick=1234, event_count=1
Event[0]: INPUT, entity_id=42, data={buttons=0x01, dir=4, shoot=1, time=56789}
ENTITY_EVENT (server → client, example SPAWN) Type=0x08 Payload: tick=1250, event_count=1
Event[0]: SPAWN, entity_id=99, data={type=enemy, x=100.0, y=200.0}

Chapter 11

Contributing to R-Type

This guide ensures contributions are consistent, readable, and maintainable.

11.1 Contribution workflow

We strictly use GitHub Issues and Pull Requests (PRs). No direct commits to main are allowed.

11.1.1 Create / Find an Issue

Before contributing, make sure there is an open issue for your task. If not, create one using the provided templates:

- Bug Report
- Feature Request

11.1.2 Branching & PR Naming

Each contribution must be developed in a dedicated branch. Branches and Pull Requests must follow this naming convention:

<Type>/<Scope>/<Sub-scope>

- Type → Feature | Fix | Refactor | Docs | Test | CI
- Scope → Client | Server
- Sub-scope → Engine | ECS | Game | etc.

This ensures consistency across branches and PRs, making it easy to identify the purpose of each contribution at a glance.

11.1.3 Commit Norms

Commit Type	Description
build	Changes that affect the build system or external dependencies (npm, make, etc.)
ci	Changes related to integration files and scripts or configuration (Travis, Ansible, BrowserStack, etc.)
feat	Addition of a new feature
fix	Bug fix
perf	Performance improvements
refactor	Modification that neither adds a new feature nor improves performance
style	Change that does not affect functionality or semantics (indentation, formatting, adding space, renaming a variable, etc.)
docs	Writing or updating documentation

test	Addition or modification of tests
------	-----------------------------------

11.2 Code Style

11.2.1 C++

- Use clang-format with the provided configuration
- Follow RAII principles
- Use `std::unique_ptr`/`std::shared_ptr` instead of raw pointers
- Header files: `.hpp` - Implementation : `.cpp`
- Namespace prefixes:
 - `eng::` → Engine
 - `ecs::` → ECS
 - `cli::` → Client
 - `gme::` → Game logic
 - `utl::` → Utils library

11.2.1.1 Naming Convention

- Classes / Structs / Enums: PascalCase
`class` PlayerEntity { ... };
`struct` TransformComponent { ... };
- Methods / Variables / Functions: camelCase
`void` updatePosition();
`int` playerHealth;
- Private members:
`unsigned int` m_privateMember
- Constants / Macros: UPPER_CASE_SNAKE_CASE
`const int` MAX_PLAYERS = 4;
- Namespaces: lowercase
`namespace` ecs { ... }
- Types / struct member: snake_case
`using` entity_id = unsigned int;
`struct` MyStruct {
 `std::string` my_member;
}

11.2.2 Yaml

- Extention : `.yaml`

Chapter 12

LICENSE

MIT License

Copyright (c) 2025 Masina Elliot, Piquet Moana, Comet Arthur and Groux Aaron

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

Chapter 13

Namespace Index

13.1 Namespace List

Here is a list of all namespaces with brief descriptions:

cli	51
cli::GameConfig	51
cli::GameConfig::Animation	52
cli::GameConfig::Beam	52
cli::GameConfig::Enemy	53
cli::GameConfig::Enemy::Easy	53
cli::GameConfig::Explosion	54
cli::GameConfig::Hitbox	55
cli::GameConfig::LoadingAnimation	55
cli::GameConfig::Player	56
cli::GameConfig::Projectile	57
cli::GameConfig::Projectile::Basic	57
cli::GameConfig::Projectile::Supercharged	58
cli::GameConfig::Screen	59
cli::GameConfig::Stage	60
ecs	60
eng	61
gme	64
gme::Utils	67
rnp	68
srv	75
srv::Config	76
srv::Config::Network	77
srv::Game	77
srv::Path	77
srv::Path::Plugin	77
utl	78
utl::cli	84
utl::Config	84
utl::Config::Audio	84
utl::Config::Color	85
utl::Config::Game	87
utl::Config::Network	88
utl::Config::Player	88
utl::Config::Window	88
utl::GameConfig	89
utl::GameConfig::Animation	89
utl::GameConfig::Beam	89
utl::GameConfig::Enemy	90
utl::GameConfig::Enemy::Easy	90

utl::GameConfig::Explosion	92
utl::GameConfig::Hitbox	92
utl::GameConfig::LoadingAnimation	93
utl::GameConfig::Player	94
utl::GameConfig::Projectile	95
utl::GameConfig::Projectile::Basic	95
utl::GameConfig::Projectile::Supercharged	96
utl::GameConfig::Screen	97
utl::GameConfig::Server	98
utl::GameConfig::Server::Enemy	99
utl::GameConfig::Server::Enemy::Advanced	99
utl::GameConfig::Server::Enemy::AI	100
utl::GameConfig::Server::Enemy::Basic	100
utl::GameConfig::Server::Enemy::Boss	101
utl::GameConfig::Server::Network	103
utl::GameConfig::Server::Projectile	103
utl::GameConfig::Stage	104
utl::Path	104
utl::Path::Audio	104
utl::Path::Font	105
utl::Path::Icons	106
utl::Path::Plugin	106
utl::Path::Texture	107
utl::srv	110

Chapter 14

Hierarchical Index

14.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

utl::cli::AppConfig	120
utl::srv::AppConfig	122
cli::ArgsConfig	123
srv::ArgsConfig	126
cli::ArgsHandler	128
srv::ArgsHandler	131
eng::CircleShape	235
cli::Client	237
srv::ClientSession	243
utl::Clock	245
gme::CollisionPair	250
eng::Color	269
eng::ConnectionStats	280
rnp::EndianUtils	291
gme::EnemySpawn	312
eng::Engine	325
ecs::Registry::EntityBuilder	330
gme::EntityManager	334
gme::EntityMetadata	349
rnp::EntityState	351
cli::EnvConfig	353
srv::EnvConfig	353
eng::Event	354
utl::Event	355
utl::EventBus	359
utl::EventConnect	375
utl::EventBus::EventQueue	377
rnp::EventRecord	378
utl::EventStats	379
rnp::HandlerPacket	423
rnp::HandlerStats	441
ecs::IComponent	454
ecs::Animation	111
ecs::Audio	218
ecs::BeamCharge	225
ecs::Ceiling	234
ecs::Color	267
ecs::Enemy	297
ecs::Explosion	382
ecs::Floor	396

ecs::Font	397
ecs::Hitbox	442
ecs::KeyboardInput	502
ecs::Layer	505
ecs::LoadingAnimation	507
ecs::Pixel	552
ecs::Player	553
ecs::Projectile	574
ecs::Rect	588
ecs::Scale	625
ecs::Score	633
ecs::Scrolling	635
ecs::Text	736
ecs::Texture	743
ecs::Transform	745
ecs::Velocity	747
gme::InterpolationData	469
utl::IPlugin	475
eng::IAudio	450
eng::INetworkClient	461
eng::AsioClient	141
eng::IRenderer	479
gme::IGameClient	455
gme::FlappyBirdClientSolo	391
gme::RTypeClientMulti	604
gme::RTypeClientSolo	609
gme::IGameServer	458
gme::RTypeServer	615
srv::INetworkServer	465
srv::AsioServer	177
ecs::Registry::IPool	478
ecs::Registry::Pool< T >	570
eng::IScene	489
eng::AScene	133
cli::Intro	471
cli::Menu	525
cli::Settings	700
gme::ConfigMulti	270
gme::ConfigSolo	275
gme::CreateRoomScene	281
gme::Game	399
gme::GameMulti	406
gme::GameSolo	416
gme::JoinRoomScene	495
gme::ServerScene	694
gme::WaitingRoomScene	749
srv::IScene	491
srv::AScene	137
ecs::ISystem	494
ecs::ASystem	216
ecs::AnimationSystem	113
ecs::AudioSystem	220
ecs::BeamSystem	227
ecs::DebugSystem	287
ecs::SpriteSystem	716
ecs::StarfieldSystem	727

ecs::TextSystem	739
gme::AnimationSystem	117
gme::BeamSystem	230
gme::CollisionSystem	251
gme::CollisionSystem	251
gme::CollisionSystem	251
gme::EnemyAISystem	300
gme::EnemySpawnSystem	314
gme::EnemySystem	322
gme::ExplosionSystem	386
gme::ExplosionSystem	386
gme::HUDSystem	445
gme::LoadingAnimationSystem	509
gme::LoadingAnimationSystem	509
gme::PlayerControllerMulti	555
gme::PlayerDirectionSystem	561
gme::PlayerDirectionSystem	561
gme::ProjectileSystem	582
gme::ProjectileSystem	582
gme::ScrollingSystem	637
gme::ScrollingSystem	637
gme::SpawnSystem	710
gme::WeaponSystem	773
gme::WeaponSystem	773
srv::Lobby	514
rnp::LobbyInfo	517
utl::Logger	519
rnp::PacketConnect	530
rnp::PacketConnectAccept	532
rnp::PacketContext	533
rnp::PacketDisconnect	534
rnp::PacketError	535
rnp::PacketGameStart	537
rnp::PacketHeader	537
rnp::PacketLobbyCreate	539
rnp::PacketLobbyCreateResponse	540
rnp::PacketLobbyJoin	541
rnp::PacketLobbyJoinResponse	542
rnp::PacketLobbyListResponse	543
rnp::PacketLobbyUpdate	545
rnp::PacketPingPong	547
rnp::PacketStartGameRequest	547
rnp::PacketWorldState	548
gme::GameMulti::PendingInput	550
gme::PlayerControllerMulti::PendingInput	551
utl::PluginLoader	566
utl::EventBus::PriorityEventQueue	572
gme::ProjectileManager	576
eng::QueuedPacket	586
srv::QueuedPacket	587
ecs::Registry	591
eng::SceneManager	627
srv::SceneManager	630
rnp::Serializer	642
srv::Server	690
utl::SharedLib	707
gme::SpawnWave	715
gme::StageManager	720

eng::Text	738
gme::Wave	760
gme::WaveManager	762
eng::WindowSize	782

Chapter 15

Class Index

15.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

ecs::Animation	111
ecs::AnimationSystem	113
gme::AnimationSystem	117
utl::cli::AppConfig	120
utl::srv::AppConfig	122
cli::ArgsConfig	123
srv::ArgsConfig	126
cli::ArgsHandler	
Class to handle command line arguments	128
srv::ArgsHandler	
Class to handle command line arguments	131
eng::AScene	
Abstract class for scene	133
srv::AScene	
Class for scene	137
eng::AsioClient	
Asio UDP client implementation	141
srv::AsioServer	
Asio UDP server implementation	177
ecs::ASystem	
Abstract class for system	216
ecs::Audio	218
ecs::AudioSystem	
Class for audio system	220
ecs::BeamCharge	225
ecs::BeamSystem	227
gme::BeamSystem	230
ecs::Ceiling	234
eng::CircleShape	235
cli::Client	
Class for the client	237
srv::ClientSession	
Client session information	243
utl::Clock	
Class for clock	245
gme::CollisionPair	
Collision pair for tracking collisions	250
gme::CollisionSystem	
Handles collision detection and resolution on server	251
ecs::Color	267

eng::Color	269
gme::ConfigMulti	
ConfigMulti scene	270
gme::ConfigSolo	
ConfigSolo scene	275
eng::ConnectionStats	
Connection statistics	280
gme::CreateRoomScene	
Create room scene	281
ecs::DebugSystem	
Class for debug system	287
rnp::EndianUtils	
Utility class for endianness conversion	291
ecs::Enemy	297
gme::EnemyAISystem	
Manages enemy behavior, movement patterns, and shooting	300
gme::EnemySpawn	
Enemy spawn definition for a wave	312
gme::EnemySpawnSystem	
Manages enemy spawning based on level progression	314
gme::EnemySystem	322
eng::Engine	
Class for the engine	325
ecs::Registry::EntityBuilder	330
gme::EntityManager	
Manages entity lifecycle and provides factory methods	334
gme::EntityMetadata	
Entity metadata for tracking	349
rnp::EntityState	
Entity state for WORLD_STATE packet	351
cli::EnvConfig	353
srv::EnvConfig	353
eng::Event	354
utl::Event	
Event structure for inter-component communication	355
utl::EventBus	
Thread-safe event bus for decoupled component communication	359
utl::EventConnect	375
utl::EventBus::EventQueue	
Internal event queue structure	377
rnp::EventRecord	
Event record for ENTITY_EVENT packets (TLV format)	378
utl::EventStats	
Event statistics structure	379
ecs::Explosion	382
gme::ExplosionSystem	386
gme::FlappyBirdClientSolo	
Class for the FlappyBirdClientSolo plugin	391
ecs::Floor	396
ecs::Font	397
gme::Game	
Game solo scene	399
gme::GameMulti	
GameMulti scene	406
gme::GameSolo	
GameSolo scene	416
rnp::HandlerPacket	
Main packet handler class	423

rnp::HandlerStats	
Statistics for packet handling	441
ecs::Hitbox	442
gme::HUDSystem	445
eng::IAudio	
Interface for the audio	450
ecs::IComponent	454
gme::IGameClient	
Interface for the games	455
gme::IGameServer	
Interface for the games	458
eng::INetworkClient	
Interface for the client network	461
srv::INetworkServer	
Interface for the server network	465
gme::InterpolationData	469
cli::Intro	
Intro scene	471
utl::IPlugin	
Interface for plugins	475
ecs::Registry::IPool	478
eng::IRenderer	
Interface for the renderer	479
eng::IScene	
Interface class for scene	489
srv::IScene	
Interface class for scene	491
ecs::ISystem	
Interface class for system	494
gme::JoinRoomScene	
Join room scene	495
ecs::KeyboardInput	502
ecs::Layer	505
ecs::LoadingAnimation	507
gme::LoadingAnimationSystem	509
srv::Lobby	
Lobby structure	514
rnp::LobbyInfo	
Lobby information structure	517
utl::Logger	
Class for logging	519
cli::Menu	
Menu scene	525
rnp::PacketConnect	
CONNECT packet payload	530
rnp::PacketConnectAccept	
CONNECT_ACCEPT packet payload	532
rnp::PacketContext	
Context information for packet processing	533
rnp::PacketDisconnect	
DISCONNECT packet payload	534
rnp::PacketError	
ERROR packet payload	535
rnp::PacketGameStart	
GAME_START packet payload	537
rnp::PacketHeader	
Packet header according to RNP specification (Big Endian) Total size: 7 bytes (1 + 2 + 4)	537

rnp::PacketLobbyCreate	
LOBBY_CREATE packet payload	539
rnp::PacketLobbyCreateResponse	
LOBBY_CREATE_RESPONSE packet payload	540
rnp::PacketLobbyJoin	
LOBBY_JOIN packet payload	541
rnp::PacketLobbyJoinResponse	
LOBBY_JOIN_RESPONSE packet payload	542
rnp::PacketLobbyListResponse	
LOBBY_LIST_RESPONSE packet payload	543
rnp::PacketLobbyUpdate	
LOBBY_UPDATE packet payload	545
rnp::PacketPingPong	
PING/PONG packet payload	547
rnp::PacketStartGameRequest	
START_GAME_REQUEST packet payload (client requests to start game)	547
rnp::PacketWorldState	
WORLD_STATE packet payload	548
gme::GameMulti::PendingInput	550
gme::PlayerControllerMulti::PendingInput	551
ecs::Pixel	552
ecs::Player	553
gme::PlayerControllerMulti	555
gme::PlayerDirectionSystem	561
utl::PluginLoader	
Modern, type-safe plugin loader	566
ecs::Registry::Pool< T >	570
utl::EventBus::PriorityEventQueue	
Priority queue for events	572
ecs::Projectile	574
gme::ProjectileManager	
Handles projectile creation and management	576
gme::ProjectileSystem	582
eng::QueuedPacket	
Queued packet for sending	586
srv::QueuedPacket	
Queued packet for sending	587
ecs::Rect	588
ecs::Registry	
Class for managing entities and their components	591
gme::RTypeClientMulti	
Class for the RTypeClientMulti plugin	604
gme::RTypeClientSolo	
Class for the RTypeClientSolo plugin	609
gme::RTypeServer	
Class for the RTypeServer plugin	615
ecs::Scale	625
eng::SceneManager	
Class for managing scenes	627
srv::SceneManager	
Class for managing scenes	630
ecs::Score	633
ecs::Scrolling	635
gme::ScrollingSystem	637
rnp::Serializer	
Binary serializer for RNP protocol packets	642
srv::Server	
Class for the server	690

gme::ServerScene	
Server configuration scene	694
cli::Settings	
Settings scene	700
utl::SharedLib	
Handle to a dynamic library with RAI	707
gme::SpawnSystem	710
gme::SpawnWave	
Spawn wave configuration	715
ecs::SpriteSystem	
Class for sprite system	716
gme::StageManager	720
ecs::StarfieldSystem	727
ecs::Text	736
eng::Text	738
ecs::TextSystem	
Class for text system	739
ecs::Texture	743
ecs::Transform	745
ecs::Velocity	747
gme::WaitingRoomScene	
Waiting room scene where players wait before game starts	749
gme::Wave	
Wave definition containing multiple enemy spawns	760
gme::WaveManager	
Manages wave-based enemy spawning with configurable patterns	762
gme::WeaponSystem	
Manages weapon firing and charging (client-side visual feedback only)	773
eng::WindowSize	782

Chapter 16

File Index

16.1 File List

Here is a list of all files with brief descriptions:

/home/masina/Projects/Epitech/rtype/client/include/Client/ ArgsHandler.hpp	
This file contains the ArgsHandler class declaration	783
/home/masina/Projects/Epitech/rtype/client/include/Client/ Client.hpp	
This file contains the Client class declaration	787
/home/masina/Projects/Epitech/rtype/client/include/Client/Generated/ Version.hpp	789
/home/masina/Projects/Epitech/rtype/client/include/Client/Scenes/ Intro.hpp	
This file contains the intro scene	792
/home/masina/Projects/Epitech/rtype/client/include/Client/Scenes/ Menu.hpp	
This file contains the menu scene	794
/home/masina/Projects/Epitech/rtype/client/include/Client/Scenes/ Settings.hpp	
This file contains the settings scene	795
/home/masina/Projects/Epitech/rtype/client/src/ argsHandler.cpp	798
/home/masina/Projects/Epitech/rtype/client/src/ client.cpp	803
/home/masina/Projects/Epitech/rtype/client/src/ event.cpp	806
/home/masina/Projects/Epitech/rtype/client/src/ main.cpp	807
/home/masina/Projects/Epitech/rtype/client/src/scenes/ intro.cpp	809
/home/masina/Projects/Epitech/rtype/client/src/scenes/ menu.cpp	811
/home/masina/Projects/Epitech/rtype/client/src/scenes/ settings.cpp	814
/home/masina/Projects/Epitech/rtype/modules/ECS/include/ECS/ Component.hpp	
This file contains the component definitions	818
/home/masina/Projects/Epitech/rtype/modules/ECS/include/ECS/ Entity.hpp	
This file contains the entity definitions	822
/home/masina/Projects/Epitech/rtype/modules/ECS/include/ECS/ Registry.hpp	
This file contains the Registry class declaration	824
/home/masina/Projects/Epitech/rtype/modules/ECS/include/ECS/Interfaces/ ISystems.hpp	
This file contains the interface for systems	823
/home/masina/Projects/Epitech/rtype/modules/ECS/include/ECS/Systems/ Animation.hpp	827
/home/masina/Projects/Epitech/rtype/modules/ECS/include/ECS/Systems/ Audio.hpp	
This file contains the audio system definition	830
/home/masina/Projects/Epitech/rtype/modules/ECS/include/ECS/Systems/ Beam.hpp	
This file contains the beam system definitions	833
/home/masina/Projects/Epitech/rtype/modules/ECS/include/ECS/Systems/ Debug.hpp	
This file contains the debug system definition	837
/home/masina/Projects/Epitech/rtype/modules/ECS/include/ECS/Systems/ Sprite.hpp	
This file contains the sprite system definition	839
/home/masina/Projects/Epitech/rtype/modules/ECS/include/ECS/Systems/ Starfield.hpp	
Starfield System for managing background stars and visual effects	841
/home/masina/Projects/Epitech/rtype/modules/ECS/include/ECS/Systems/ Systems.hpp	
This file contains the system definitions	1175
/home/masina/Projects/Epitech/rtype/modules/ECS/include/ECS/Systems/ Text.hpp	845

/home/masina/Projects/Epitech/rtype/modules/Engine/include/Engine/ Engine.hpp	
This file contains the Engine class declaration	847
/home/masina/Projects/Epitech/rtype/modules/Engine/include/Engine/ SceneManager.hpp	
This file contains the SceneManager class declaration	1169
/home/masina/Projects/Epitech/rtype/modules/Engine/include/Engine/Interfaces/ IScene.hpp	
This file contains the IScene class	1164
/home/masina/Projects/Epitech/rtype/modules/Engine/src/ engine.cpp	849
/home/masina/Projects/Epitech/rtype/modules/Interfaces/include/Interfaces/ IAudio.hpp	
This file contains the Audio interface	849
/home/masina/Projects/Epitech/rtype/modules/Interfaces/include/Interfaces/ IGameClient.hpp	
This file contains the Game interface	851
/home/masina/Projects/Epitech/rtype/modules/Interfaces/include/Interfaces/ IGameServer.hpp	
This file contains the Game interface	852
/home/masina/Projects/Epitech/rtype/modules/Interfaces/include/Interfaces/ INetworkClient.hpp	
This file contains the client network interface	854
/home/masina/Projects/Epitech/rtype/modules/Interfaces/include/Interfaces/ INetworkServer.hpp	
This file contains the server network interface	856
/home/masina/Projects/Epitech/rtype/modules/Interfaces/include/Interfaces/ IRenderer.hpp	
This file contains the IRenderer class declaration	858
/home/masina/Projects/Epitech/rtype/modules/Interfaces/include/Interfaces/Protocol/ HandlerPacket.hpp	
Network packet handler for RNP protocol	861
/home/masina/Projects/Epitech/rtype/modules/Interfaces/include/Interfaces/Protocol/ Protocol.hpp	
This file contains the network protocol	870
/home/masina/Projects/Epitech/rtype/modules/Interfaces/include/Interfaces/Protocol/ Serializer.hpp	
Network packet serializer for RNP protocol	876
/home/masina/Projects/Epitech/rtype/modules/Utils/include/Utils/ AppConfigClient.hpp	
This file contains the app config structure for client	888
/home/masina/Projects/Epitech/rtype/modules/Utils/include/Utils/ AppConfigServer.hpp	
This file contains the app config structure for server	889
/home/masina/Projects/Epitech/rtype/modules/Utils/include/Utils/ Clock.hpp	
This file contains the Clock class	892
/home/masina/Projects/Epitech/rtype/modules/Utils/include/Utils/ Common.hpp	
This file contains common definitions and constants	1158
/home/masina/Projects/Epitech/rtype/modules/Utils/include/Utils/ Event.hpp	
Event structures and types for event-driven communication	894
/home/masina/Projects/Epitech/rtype/modules/Utils/include/Utils/ EventBus.hpp	
Thread-safe event bus implementation for inter-component communication	897
/home/masina/Projects/Epitech/rtype/modules/Utils/include/Utils/ HitboxUtils.hpp	
Utility functions for hitbox calculations	905
/home/masina/Projects/Epitech/rtype/modules/Utils/include/Utils/ Logger.hpp	
This file contains the Logger class	910
/home/masina/Projects/Epitech/rtype/modules/Utils/include/Utils/ PluginLoader.hpp	
Modern, cross-platform plugin loader	912
/home/masina/Projects/Epitech/rtype/modules/Utils/include/Utils/ Utils.hpp	
This file contains utility functions	924
/home/masina/Projects/Epitech/rtype/modules/Utils/include/Utils/Interfaces/ IPlugin.hpp	
This file contains the plugin interface	908
/home/masina/Projects/Epitech/rtype/modules/Utils/include/Utils/RTypeShared/ GameConfig.hpp	
Configuration constants for the multiplayer game	915
/home/masina/Projects/Epitech/rtype/modules/Utils/src/ logger.cpp	925
/home/masina/Projects/Epitech/rtype/modules/Utils/src/ utils.cpp	926
/home/masina/Projects/Epitech/rtype/plugins/Game/FlappyBird/Client/Solo/include/ ↵	
FlappyBirdClientSolo/ FlappyBirdClientSolo.hpp	
This file contains the FlappyBirdClientSolo class declaration	927
/home/masina/Projects/Epitech/rtype/plugins/Game/FlappyBird/Client/Solo/include/ ↵	
FlappyBirdClientSolo/Scenes/ Game.hpp	
This file contains the solo Game scene	929

/home/masina/Projects/Epitech/rtype/plugins/Game/FlappyBird/Client/Solo/src/entrypoint.cpp	930
/home/masina/Projects/Epitech/rtype/plugins/Game/FlappyBird/Client/Solo/src/flappyBirdClientSolo.cpp	935
/home/masina/Projects/Epitech/rtype/plugins/Game/FlappyBird/Client/Solo/src/scenes/game.cpp	935
/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/include/RType↵ ClientMulti/GameConfig.hpp	
Configuration constants for the game	920
/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/include/RType↵ ClientMulti/RTypeClientMulti.hpp	
This file contains the RTypeClientMulti class declaration	947
/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/include/RType↵ ClientMulti/Managers/ProjectileManager.hpp	
Manages projectile creation and configuration for multiplayer	941
/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/include/RType↵ ClientMulti/Managers/StageManager.hpp	
.	944
/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/include/RType↵ ClientMulti/Scenes/ConfigMulti.hpp	
This file contains the multiplayer configuration scene	949
/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/include/RType↵ ClientMulti/Scenes/CreateRoom.hpp	
This file contains the create room scene	950
/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/include/RType↵ ClientMulti/Scenes/GameMulti.hpp	
This file contains the multiplayer Game scene	952
/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/include/RType↵ ClientMulti/Scenes/JoinRoom.hpp	
This file contains the join room scene	955
/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/include/RType↵ ClientMulti/Scenes/ServerScene.hpp	
This file contains the server configuration scene	956
/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/include/RType↵ ClientMulti/Scenes/WaitingRoom.hpp	
This file contains the waiting room scene	958
/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/include/RType↵ ClientMulti/Systems/Animation.hpp	
Animation system for multiplayer	829
/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/include/RType↵ ClientMulti/Systems/Beam.hpp	
Beam system for multiplayer	835
/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/include/RType↵ ClientMulti/Systems/Collision.hpp	
.	960
/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/include/RType↵ ClientMulti/Systems/Explosion.hpp	
This file contains the explosion system definitions	969
/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/include/RType↵ ClientMulti/Systems>LoadingAnimation.hpp	
.	974
/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/include/RType↵ ClientMulti/Systems/PlayerControllerMulti.hpp	
Player Controller System for multiplayer (sends inputs to server)	978
/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/include/RType↵ ClientMulti/Systems/PlayerDirection.hpp	
.	979
/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/include/RType↵ ClientMulti/Systems/Projectile.hpp	
Projectile system for multiplayer	985
/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/include/RType↵ ClientMulti/Systems/Scrolling.hpp	
.	989

/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/include/RType↵ ClientMulti/Systems/ Systems.hpp This file contains the system definitions	1176
/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/include/RType↵ ClientMulti/Systems/ Weapon.hpp Handles weapon firing logic for multiplayer	994
/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/include/RType↵ ClientMulti/Utils/ HitboxUtils.hpp	907
/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/src/ entrypoint.cpp 931	
/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/src/ rtypeClientMulti.cpp 1001	
/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/src/managers/ stageManager.cpp 998	
/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/src/scenes/ configMulti.cpp 1004	
/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/src/scenes/ createRoom.cpp 1006	
/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/src/scenes/ gameMulti.cpp 1011	
/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/src/scenes/ joinRoom.cpp 1020	
/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/src/scenes/ ServerScene.cpp 1025	
/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/src/scenes/ waitingRoom.cpp 1030	
/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/src/Systems/ playerControllerMulti.cpp 1035	
/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/src/Systems/ projectile.cpp 1038	
/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/src/Systems/ weapon.cpp 1040	
/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Solo/include/RType↵ ClientSolo/ RTypeClientSolo.hpp This file contains the RTypeClientSolo class declaration	1044
/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Solo/include/RType↵ ClientSolo/Managers/ ProjectileManager.hpp Manages projectile creation and configuration	942
/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Solo/include/RType↵ ClientSolo/Managers/ StageManager.hpp Stage Manager for handling floor and ceiling spawning	945
/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Solo/include/RType↵ ClientSolo/Scenes/ ConfigSolo.hpp This file contains the solo configuration scene	1046
/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Solo/include/RType↵ ClientSolo/Scenes/ GameSolo.hpp This file contains the solo Game scene	1048
/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Solo/include/RType↵ ClientSolo/Systems/ Collision.hpp This file contains the collision system definitions	963
/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Solo/include/RType↵ ClientSolo/Systems/ Enemy.hpp This file contains the enemy system definitions	1050
/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Solo/include/RType↵ ClientSolo/Systems/ Explosion.hpp This file contains the explosion system definitions	971

/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Solo/include/RType↵ ClientSolo/Systems/ HUD.hpp	
HUD System for managing score display and UI elements	1053
/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Solo/include/RType↵ ClientSolo/Systems/ LoadingAnimation.hpp	
This file contains the loading animation system definitions	976
/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Solo/include/RType↵ ClientSolo/Systems/ PlayerDirection.hpp	
This file contains the player direction system definitions	982
/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Solo/include/RType↵ ClientSolo/Systems/ Projectile.hpp	
This file contains the projectile system definitions	987
/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Solo/include/RType↵ ClientSolo/Systems/ Scrolling.hpp	
Horizontal scrolling system for large floor/ceiling sprites	991
/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Solo/include/RType↵ ClientSolo/Systems/ Spawn.hpp	
This file contains the spawn system definitions	1055
/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Solo/include/RType↵ ClientSolo/Systems/ Systems.hpp	
This file contains the system definitions	1177
/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Solo/include/RType↵ ClientSolo/Systems/ Weapon.hpp	
Handles weapon firing logic	996
/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Solo/src/ entrypoint.cpp	932
/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Solo/src/ rtypeClientSolo.cpp	1059
/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Solo/src/managers/ projectileManager.cpp	1058
/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Solo/src/managers/ stageManager.cpp	1000
/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Solo/src/scenes/ configSolo.cpp	1060
/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Solo/src/scenes/ gameSolo.cpp	1063
/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Solo/src/systems/ HUD.cpp	1067
/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Solo/src/systems/ weapon.cpp	1042
/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Server/include/RTypeServer/ EntityManager.hpp	
Centralized entity management for RTypeServer	1068
/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Server/include/RTypeServer/ RTypeServer.hpp	
This file contains the RTypeServer class declaration	1071
/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Server/include/RTypeServer/ WaveManager.hpp	
Wave-based enemy spawning system for RTypeServer	1080
/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Server/include/RTypeServer/↵ Systems/ CollisionSystem.hpp	
Server-side collision detection system	1073
/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Server/include/RTypeServer/↵ Systems/ EnemyAISystem.hpp	
Enemy AI and behavior system for server	1076
/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Server/include/RTypeServer/↵ Systems/ EnemySpawnSystem.hpp	
Enemy spawning system for server	1078
/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Server/src/ EntityManager.cpp	
Implementation of EntityManager for RTypeServer	1083
/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Server/src/ entrypoint.cpp	933

/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Server/src/ rtypeServer.cpp	
Implementation of RTypeServer with new entity management system	1090
/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Server/src/ WaveManager.cpp	
Implementation of wave-based enemy spawning system	1108
/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Server/src/Systems/ CollisionSystem.cpp	
Implementation of server-side collision detection system	1096
/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Server/src/Systems/ EnemyAISystem.cpp	
Implementation of enemy AI and behavior system	1100
/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Server/src/Systems/ EnemySpawnSystem.cpp	
Implementation of enemy spawning system	1105
/home/masina/Projects/Epitech/rtype/plugins/Network/Asio/Client/include/AsioClient/ AsioClient.hpp	
Asio-based implementation of INetworkClient interface	1113
/home/masina/Projects/Epitech/rtype/plugins/Network/Asio/Client/src/ asioClient.cpp . . .	1119
/home/masina/Projects/Epitech/rtype/plugins/Network/Asio/Client/src/ entrypoint.cpp . . .	933
/home/masina/Projects/Epitech/rtype/plugins/Network/Asio/Server/include/AsioServer/ AsioServer.hpp	
Asio-based implementation of INetworkServer interface	1132
/home/masina/Projects/Epitech/rtype/plugins/Network/Asio/Server/src/ asioServer.cpp . . .	1139
/home/masina/Projects/Epitech/rtype/plugins/Network/Asio/Server/src/ entrypoint.cpp . . .	934
/home/masina/Projects/Epitech/rtype/server/include/Server/ ArgsHandler.hpp	
This file contains the ArgsHandler class declaration	785
/home/masina/Projects/Epitech/rtype/server/include/Server/ Common.hpp	
This file contains common definitions and constants	1162
/home/masina/Projects/Epitech/rtype/server/include/Server/ SceneManager.hpp	
This file contains the SceneManager class declaration	1170
/home/masina/Projects/Epitech/rtype/server/include/Server/ Server.hpp	
This file contains the Server class declaration	1173
/home/masina/Projects/Epitech/rtype/server/include/Server/Generated/ Version.hpp	791
/home/masina/Projects/Epitech/rtype/server/include/Server/Interfaces/ IScene.hpp	
This file contains the IScene class	1166
/home/masina/Projects/Epitech/rtype/server/include/Server/Scenes/ Level_0.hpp	
This file contains the level_0 definitions	1172
/home/masina/Projects/Epitech/rtype/server/include/Server/Scenes/ Level_1.hpp	
This file contains the level_0 definitions	1172
/home/masina/Projects/Epitech/rtype/server/include/Server/Systems/ Systems.hpp	
This file contains the system definitions	1179
/home/masina/Projects/Epitech/rtype/server/src/ argsHandler.cpp	801
/home/masina/Projects/Epitech/rtype/server/src/ main.cpp	808
/home/masina/Projects/Epitech/rtype/server/src/ server.cpp	1180

Chapter 17

Namespace Documentation

17.1 cli Namespace Reference

Namespaces

- namespace [GameConfig](#)

Classes

- struct [ArgsConfig](#)
- class [ArgsHandler](#)
Class to handle command line arguments.
- class [Client](#)
Class for the client.
- struct [EnvConfig](#)
- class [Intro](#)
[Intro](#) scene.
- class [Menu](#)
[Menu](#) scene.
- class [Settings](#)
[Settings](#) scene.

Typedefs

- using [json](#) = nlohmann::json

17.1.1 Typedef Documentation

17.1.1.1 json

using [cli::json](#) = nlohmann::json

Definition at line [16](#) of file [ArgsHandler.hpp](#).

17.2 cli::GameConfig Namespace Reference

Namespaces

- namespace [Animation](#)
- namespace [Beam](#)
- namespace [Enemy](#)
- namespace [Explosion](#)
- namespace [Hitbox](#)
- namespace [LoadingAnimation](#)
- namespace [Player](#)

- namespace [Projectile](#)
- namespace [Screen](#)
- namespace [Stage](#)

17.3 cli::GameConfig::Animation Namespace Reference

Variables

- constexpr float [FRAME_DURATION](#) = 0.1f

17.3.1 Variable Documentation

17.3.1.1 FRAME_DURATION

float cli::GameConfig::Animation::FRAME_DURATION = 0.1f [inline], [constexpr]

Definition at line 61 of file [GameConfig.hpp](#).

17.4 cli::GameConfig::Beam Namespace Reference

Variables

- constexpr float [MAX_CHARGE](#) = 1.0f
- constexpr float [CHARGE_RATE](#) = 1.0f
- constexpr float [BAR_WIDTH](#) = 120.0f
- constexpr float [BAR_HEIGHT](#) = 12.0f
- constexpr float [BAR_X](#) = 10.0f
- constexpr float [BAR_Y](#) = 10.0f

17.4.1 Variable Documentation

17.4.1.1 BAR_HEIGHT

float cli::GameConfig::Beam::BAR_HEIGHT = 12.0f [inline], [constexpr]

Definition at line 68 of file [GameConfig.hpp](#).

17.4.1.2 BAR_WIDTH

float cli::GameConfig::Beam::BAR_WIDTH = 120.0f [inline], [constexpr]

Definition at line 67 of file [GameConfig.hpp](#).

17.4.1.3 BAR_X

float cli::GameConfig::Beam::BAR_X = 10.0f [inline], [constexpr]

Definition at line 69 of file [GameConfig.hpp](#).

17.4.1.4 BAR_Y

float cli::GameConfig::Beam::BAR_Y = 10.0f [inline], [constexpr]

Definition at line 70 of file [GameConfig.hpp](#).

17.4.1.5 CHARGE_RATE

float cli::GameConfig::Beam::CHARGE_RATE = 1.0f [inline], [constexpr]

Definition at line 66 of file [GameConfig.hpp](#).

17.4.1.6 MAX_CHARGE

float cli::GameConfig::Beam::MAX_CHARGE = 1.0f [inline], [constexpr]

Definition at line 65 of file [GameConfig.hpp](#).

17.5 cli::GameConfig::Enemy Namespace Reference

Namespaces

- namespace [Easy](#)

17.6 cli::GameConfig::Enemy::Easy Namespace Reference

Variables

- constexpr float [HEALTH](#) = 1.0f
- constexpr float [DAMAGE](#) = 5.0f
- constexpr float [SPEED](#) = 80.0f
- constexpr float [SPRITE_WIDTH](#) = 32.0f
- constexpr float [SPRITE_HEIGHT](#) = 32.0f
- constexpr float [SCALE](#) = 2.0f
- constexpr float [SHOOT_COOLDOWN](#) = 2.0f
- constexpr float [SPAWN_RATE](#) = 2.0f
- constexpr int [ANIMATION_FRAMES](#) = 4
- constexpr float [ANIMATION_DURATION](#) = 0.5f
- constexpr int [FRAMES_PER_ROW](#) = 4

17.6.1 Variable Documentation

17.6.1.1 ANIMATION_DURATION

float cli::GameConfig::Enemy::Easy::ANIMATION_DURATION = 0.5f [inline], [constexpr]

Definition at line 99 of file [GameConfig.hpp](#).

17.6.1.2 ANIMATION_FRAMES

int cli::GameConfig::Enemy::Easy::ANIMATION_FRAMES = 4 [inline], [constexpr]

Definition at line 98 of file [GameConfig.hpp](#).

17.6.1.3 DAMAGE

float cli::GameConfig::Enemy::Easy::DAMAGE = 5.0f [inline], [constexpr]

Definition at line 91 of file [GameConfig.hpp](#).

17.6.1.4 FRAMES_PER_ROW

int cli::GameConfig::Enemy::Easy::FRAMES_PER_ROW = 4 [inline], [constexpr]

Definition at line 100 of file [GameConfig.hpp](#).

17.6.1.5 HEALTH

float cli::GameConfig::Enemy::Easy::HEALTH = 1.0f [inline], [constexpr]

Definition at line 90 of file [GameConfig.hpp](#).

17.6.1.6 SCALE

float cli::GameConfig::Enemy::Easy::SCALE = 2.0f [inline], [constexpr]

Definition at line 95 of file [GameConfig.hpp](#).

17.6.1.7 SHOOT_COOLDOWN

float cli::GameConfig::Enemy::Easy::SHOOT_COOLDOWN = 2.0f [inline], [constexpr]

Definition at line 96 of file [GameConfig.hpp](#).

17.6.1.8 SPAWN_RATE

float cli::GameConfig::Enemy::Easy::SPAWN_RATE = 2.0f [inline], [constexpr]
Definition at line 97 of file [GameConfig.hpp](#).

17.6.1.9 SPEED

float cli::GameConfig::Enemy::Easy::SPEED = 80.0f [inline], [constexpr]
Definition at line 92 of file [GameConfig.hpp](#).

17.6.1.10 SPRITE_HEIGHT

float cli::GameConfig::Enemy::Easy::SPRITE_HEIGHT = 32.0f [inline], [constexpr]
Definition at line 94 of file [GameConfig.hpp](#).

17.6.1.11 SPRITE_WIDTH

float cli::GameConfig::Enemy::Easy::SPRITE_WIDTH = 32.0f [inline], [constexpr]
Definition at line 93 of file [GameConfig.hpp](#).

17.7 cli::GameConfig::Explosion Namespace Reference

Variables

- constexpr float [SPRITE_WIDTH](#) = 32.0f
- constexpr float [SPRITE_HEIGHT](#) = 32.0f
- constexpr int [ANIMATION_FRAMES](#) = 4
- constexpr float [ANIMATION_DURATION](#) = 0.1f
- constexpr int [FRAMES_PER_ROW](#) = 4
- constexpr float [LIFETIME](#) = 0.4f
- constexpr float [SCALE](#) = 2.0f

17.7.1 Variable Documentation

17.7.1.1 ANIMATION_DURATION

float cli::GameConfig::Explosion::ANIMATION_DURATION = 0.1f [inline], [constexpr]
Definition at line 108 of file [GameConfig.hpp](#).

17.7.1.2 ANIMATION_FRAMES

int cli::GameConfig::Explosion::ANIMATION_FRAMES = 4 [inline], [constexpr]
Definition at line 107 of file [GameConfig.hpp](#).

17.7.1.3 FRAMES_PER_ROW

int cli::GameConfig::Explosion::FRAMES_PER_ROW = 4 [inline], [constexpr]
Definition at line 109 of file [GameConfig.hpp](#).

17.7.1.4 LIFETIME

float cli::GameConfig::Explosion::LIFETIME = 0.4f [inline], [constexpr]
Definition at line 110 of file [GameConfig.hpp](#).

17.7.1.5 SCALE

float cli::GameConfig::Explosion::SCALE = 2.0f [inline], [constexpr]
Definition at line 111 of file [GameConfig.hpp](#).

17.7.1.6 SPRITE_HEIGHT

float cli::GameConfig::Explosion::SPRITE_HEIGHT = 32.0f [inline], [constexpr]
Definition at line 106 of file [GameConfig.hpp](#).

17.7.1.7 SPRITE_WIDTH

float cli::GameConfig::Explosion::SPRITE_WIDTH = 32.0f [inline], [constexpr]
Definition at line 105 of file [GameConfig.hpp](#).

17.8 cli::GameConfig::Hitbox Namespace Reference

Variables

- constexpr float [PLAYER_RADIUS](#) = 20.0f
- constexpr float [ENEMY_RADIUS](#) = 15.0f
- constexpr float [PROJECTILE_BASIC_RADIUS](#) = 5.0f
- constexpr float [PROJECTILE_SUPERCHARGED_RADIUS](#) = 8.0f

17.8.1 Variable Documentation

17.8.1.1 ENEMY_RADIUS

float cli::GameConfig::Hitbox::ENEMY_RADIUS = 15.0f [inline], [constexpr]
Definition at line 116 of file [GameConfig.hpp](#).

17.8.1.2 PLAYER_RADIUS

float cli::GameConfig::Hitbox::PLAYER_RADIUS = 20.0f [inline], [constexpr]
Definition at line 115 of file [GameConfig.hpp](#).

17.8.1.3 PROJECTILE_BASIC_RADIUS

float cli::GameConfig::Hitbox::PROJECTILE_BASIC_RADIUS = 5.0f [inline], [constexpr]
Definition at line 117 of file [GameConfig.hpp](#).

17.8.1.4 PROJECTILE_SUPERCHARGED_RADIUS

float cli::GameConfig::Hitbox::PROJECTILE_SUPERCHARGED_RADIUS = 8.0f [inline], [constexpr]
Definition at line 118 of file [GameConfig.hpp](#).

17.9 cli::GameConfig::LoadingAnimation Namespace Reference

Variables

- constexpr float [SPRITE_WIDTH](#) = 29.0f
- constexpr float [SPRITE_HEIGHT](#) = 24.0f
- constexpr int [ANIMATION_FRAMES](#) = 4
- constexpr float [ANIMATION_DURATION](#) = 0.15f
- constexpr float [OFFSET_X](#) = 60.0f
- constexpr float [OFFSET_Y](#) = 6.0f

17.9.1 Variable Documentation

17.9.1.1 ANIMATION_DURATION

float cli::GameConfig::LoadingAnimation::ANIMATION_DURATION = 0.15f [inline], [constexpr]
Definition at line 77 of file [GameConfig.hpp](#).

17.9.1.2 ANIMATION_FRAMES

int cli::GameConfig::LoadingAnimation::ANIMATION_FRAMES = 4 [inline], [constexpr]
 Definition at line 76 of file [GameConfig.hpp](#).

17.9.1.3 OFFSET_X

float cli::GameConfig::LoadingAnimation::OFFSET_X = 60.0f [inline], [constexpr]
 Definition at line 78 of file [GameConfig.hpp](#).

17.9.1.4 OFFSET_Y

float cli::GameConfig::LoadingAnimation::OFFSET_Y = 6.0f [inline], [constexpr]
 Definition at line 79 of file [GameConfig.hpp](#).

17.9.1.5 SPRITE_HEIGHT

float cli::GameConfig::LoadingAnimation::SPRITE_HEIGHT = 24.0f [inline], [constexpr]
 Definition at line 75 of file [GameConfig.hpp](#).

17.9.1.6 SPRITE_WIDTH

float cli::GameConfig::LoadingAnimation::SPRITE_WIDTH = 29.0f [inline], [constexpr]
 Definition at line 74 of file [GameConfig.hpp](#).

17.10 cli::GameConfig::Player Namespace Reference

Variables

- constexpr float [SPEED](#) = 500.0f
- constexpr float [DIAGONAL_SPEED_MULTIPLIER](#) = 0.707f
- constexpr float [SPRITE_WIDTH](#) = 33.0f
- constexpr float [SPRITE_HEIGHT](#) = 17.0f
- constexpr float [SCALE](#) = 2.0f
- constexpr int [FRAMES_PER_ROW](#) = 5
- constexpr int [TOTAL_FRAMES](#) = 5

17.10.1 Variable Documentation

17.10.1.1 DIAGONAL_SPEED_MULTIPLIER

float cli::GameConfig::Player::DIAGONAL_SPEED_MULTIPLIER = 0.707f [inline], [constexpr]
 Definition at line 25 of file [GameConfig.hpp](#).

17.10.1.2 FRAMES_PER_ROW

int cli::GameConfig::Player::FRAMES_PER_ROW = 5 [inline], [constexpr]
 Definition at line 29 of file [GameConfig.hpp](#).

17.10.1.3 SCALE

float cli::GameConfig::Player::SCALE = 2.0f [inline], [constexpr]
 Definition at line 28 of file [GameConfig.hpp](#).

17.10.1.4 SPEED

float cli::GameConfig::Player::SPEED = 500.0f [inline], [constexpr]
 Definition at line 24 of file [GameConfig.hpp](#).

17.10.1.5 SPRITE_HEIGHT

float cli::GameConfig::Player::SPRITE_HEIGHT = 17.0f [inline], [constexpr]

Definition at line 27 of file [GameConfig.hpp](#).

17.10.1.6 SPRITE_WIDTH

float cli::GameConfig::Player::SPRITE_WIDTH = 33.0f [inline], [constexpr]

Definition at line 26 of file [GameConfig.hpp](#).

17.10.1.7 TOTAL_FRAMES

int cli::GameConfig::Player::TOTAL_FRAMES = 5 [inline], [constexpr]

Definition at line 30 of file [GameConfig.hpp](#).

17.11 cli::GameConfig::Projectile Namespace Reference

Namespaces

- namespace [Basic](#)
- namespace [Supercharged](#)

17.12 cli::GameConfig::Projectile::Basic Namespace Reference

Variables

- constexpr float [SPEED](#) = 800.0f
- constexpr float [DAMAGE](#) = 10.0f
- constexpr float [LIFETIME](#) = 3.0f
- constexpr float [SCALE](#) = 1.0f
- constexpr float [FIRE_COOLDOWN](#) = 0.3f
- constexpr float [SPRITE_WIDTH](#) = 16.0f
- constexpr float [SPRITE_HEIGHT](#) = 8.0f

17.12.1 Variable Documentation

17.12.1.1 DAMAGE

float cli::GameConfig::Projectile::Basic::DAMAGE = 10.0f [inline], [constexpr]

Definition at line 37 of file [GameConfig.hpp](#).

17.12.1.2 FIRE_COOLDOWN

float cli::GameConfig::Projectile::Basic::FIRE_COOLDOWN = 0.3f [inline], [constexpr]

Definition at line 40 of file [GameConfig.hpp](#).

17.12.1.3 LIFETIME

float cli::GameConfig::Projectile::Basic::LIFETIME = 3.0f [inline], [constexpr]

Definition at line 38 of file [GameConfig.hpp](#).

17.12.1.4 SCALE

float cli::GameConfig::Projectile::Basic::SCALE = 1.0f [inline], [constexpr]

Definition at line 39 of file [GameConfig.hpp](#).

17.12.1.5 SPEED

float cli::GameConfig::Projectile::Basic::SPEED = 800.0f [inline], [constexpr]

Definition at line 36 of file [GameConfig.hpp](#).

17.12.1.6 SPRITE_HEIGHT

float cli::GameConfig::Projectile::Basic::SPRITE_HEIGHT = 8.0f [inline], [constexpr]
Definition at line 42 of file [GameConfig.hpp](#).

17.12.1.7 SPRITE_WIDTH

float cli::GameConfig::Projectile::Basic::SPRITE_WIDTH = 16.0f [inline], [constexpr]
Definition at line 41 of file [GameConfig.hpp](#).

17.13 cli::GameConfig::Projectile::Supercharged Namespace Reference

Variables

- constexpr float [SPEED](#) = 1200.0f
- constexpr float [DAMAGE](#) = 25.0f
- constexpr float [LIFETIME](#) = 5.0f
- constexpr float [SCALE](#) = 1.5f
- constexpr float [FIRE_COOLDOWN](#) = 0.2f
- constexpr float [CHARGE_TIME](#) = 0.5f
- constexpr float [SPRITE_WIDTH](#) = 29.0f
- constexpr float [SPRITE_HEIGHT](#) = 24.0f
- constexpr int [ANIMATION_FRAMES](#) = 4
- constexpr float [ANIMATION_DURATION](#) = 0.15f

17.13.1 Variable Documentation

17.13.1.1 ANIMATION_DURATION

float cli::GameConfig::Projectile::Supercharged::ANIMATION_DURATION = 0.15f [inline], [constexpr]
Definition at line 56 of file [GameConfig.hpp](#).

17.13.1.2 ANIMATION_FRAMES

int cli::GameConfig::Projectile::Supercharged::ANIMATION_FRAMES = 4 [inline], [constexpr]
Definition at line 55 of file [GameConfig.hpp](#).

17.13.1.3 CHARGE_TIME

float cli::GameConfig::Projectile::Supercharged::CHARGE_TIME = 0.5f [inline], [constexpr]
Definition at line 52 of file [GameConfig.hpp](#).

17.13.1.4 DAMAGE

float cli::GameConfig::Projectile::Supercharged::DAMAGE = 25.0f [inline], [constexpr]
Definition at line 48 of file [GameConfig.hpp](#).

17.13.1.5 FIRE_COOLDOWN

float cli::GameConfig::Projectile::Supercharged::FIRE_COOLDOWN = 0.2f [inline], [constexpr]
Definition at line 51 of file [GameConfig.hpp](#).

17.13.1.6 LIFETIME

float cli::GameConfig::Projectile::Supercharged::LIFETIME = 5.0f [inline], [constexpr]
Definition at line 49 of file [GameConfig.hpp](#).

17.13.1.7 SCALE

float cli::GameConfig::Projectile::Supercharged::SCALE = 1.5f [inline], [constexpr]
Definition at line 50 of file [GameConfig.hpp](#).

17.13.1.8 SPEED

float cli::GameConfig::Projectile::Supercharged::SPEED = 1200.0f [inline], [constexpr]

Definition at line 47 of file [GameConfig.hpp](#).

17.13.1.9 SPRITE_HEIGHT

float cli::GameConfig::Projectile::Supercharged::SPRITE_HEIGHT = 24.0f [inline], [constexpr]

Definition at line 54 of file [GameConfig.hpp](#).

17.13.1.10 SPRITE_WIDTH

float cli::GameConfig::Projectile::Supercharged::SPRITE_WIDTH = 29.0f [inline], [constexpr]

Definition at line 53 of file [GameConfig.hpp](#).

17.14 cli::GameConfig::Screen Namespace Reference

Variables

- constexpr float [SPAWN_X](#) = 1950.0f
- constexpr float [MIN_Y](#) = 50.0f
- constexpr float [MAX_Y](#) = 1030.0f
- constexpr float [REMOVE_X](#) = -100.0f
- constexpr float [REMOVE_MIN_Y](#) = -50.0f
- constexpr float [REMOVE_MAX_Y](#) = 1130.0f

17.14.1 Variable Documentation

17.14.1.1 MAX_Y

float cli::GameConfig::Screen::MAX_Y = 1030.0f [inline], [constexpr]

Definition at line 17 of file [GameConfig.hpp](#).

17.14.1.2 MIN_Y

float cli::GameConfig::Screen::MIN_Y = 50.0f [inline], [constexpr]

Definition at line 16 of file [GameConfig.hpp](#).

17.14.1.3 REMOVE_MAX_Y

float cli::GameConfig::Screen::REMOVE_MAX_Y = 1130.0f [inline], [constexpr]

Definition at line 20 of file [GameConfig.hpp](#).

17.14.1.4 REMOVE_MIN_Y

float cli::GameConfig::Screen::REMOVE_MIN_Y = -50.0f [inline], [constexpr]

Definition at line 19 of file [GameConfig.hpp](#).

17.14.1.5 REMOVE_X

float cli::GameConfig::Screen::REMOVE_X = -100.0f [inline], [constexpr]

Definition at line 18 of file [GameConfig.hpp](#).

17.14.1.6 SPAWN_X

float cli::GameConfig::Screen::SPAWN_X = 1950.0f [inline], [constexpr]

Definition at line 15 of file [GameConfig.hpp](#).

17.15 cli::GameConfig::Stage Namespace Reference

Variables

- constexpr float [FLOOR_OFFSET_Y](#) = 16.0f
- constexpr float [CEILING_OFFSET_Y](#) = -1.0f

17.15.1 Variable Documentation

17.15.1.1 CEILING_OFFSET_Y

float cli::GameConfig::Stage::CEILING_OFFSET_Y = -1.0f [inline], [constexpr]

Definition at line 84 of file [GameConfig.hpp](#).

17.15.1.2 FLOOR_OFFSET_Y

float cli::GameConfig::Stage::FLOOR_OFFSET_Y = 16.0f [inline], [constexpr]

Definition at line 83 of file [GameConfig.hpp](#).

17.16 ecs Namespace Reference

Classes

- struct [Animation](#)
- class [AnimationSystem](#)
- class [ASystem](#)

Abstract class for system.

- struct [Audio](#)
- class [AudioSystem](#)

Class for audio system.

- struct [BeamCharge](#)
- class [BeamSystem](#)
- struct [Ceiling](#)
- struct [Color](#)
- class [DebugSystem](#)

Class for debug system.

- struct [Enemy](#)
- struct [Explosion](#)
- struct [Floor](#)
- struct [Font](#)
- struct [Hitbox](#)
- struct [IComponent](#)
- interface [ISystem](#)

Interface class for system.

- struct [KeyboardInput](#)
- struct [Layer](#)
- struct [LoadingAnimation](#)
- struct [Pixel](#)
- struct [Player](#)
- struct [Projectile](#)
- struct [Rect](#)
- class [Registry](#)

Class for managing entities and their components.

- struct [Scale](#)
- struct [Score](#)
- struct [Scrolling](#)

- class [SpriteSystem](#)
Class for sprite system.
- class [StarfieldSystem](#)
- struct [Text](#)
- class [TextSystem](#)
Class for text system.
- struct [Texture](#)
- struct [Transform](#)
- struct [Velocity](#)

Typedefs

- using [Entity](#) = std::uint32_t

Variables

- constexpr [Entity](#) [INVALID_ENTITY](#) = 0

17.16.1 Typedef Documentation

17.16.1.1 Entity

using [ecs::Entity](#) = std::uint32_t

Definition at line 13 of file [Entity.hpp](#).

17.16.2 Variable Documentation

17.16.2.1 INVALID_ENTITY

[Entity](#) [ecs::INVALID_ENTITY](#) = 0 [constexpr]

Definition at line 14 of file [Entity.hpp](#).

Referenced by [gme::EnemyAISystem::applyAggressiveMovement\(\)](#), [gme::CollisionSystem::applyDamageToEnemy\(\)](#), [gme::CollisionSystem::CollisionSystem\(\)](#), [gme::CollisionSystem::ensureEnemyDeathChannel\(\)](#), [gme::CollisionSystem::ensureEnemyDeathChannel\(\)](#), [gme::WeaponSystem::ensureSuperShotAudio\(\)](#), [gme::EnemyAISystem::findNearestPlayer\(\)](#), [gme::EntityManager::getEnemy\(\)](#), [gme::EntityManager::getPlayer\(\)](#), [gme::EntityManager::getProjectile\(\)](#), [gme::RTypeServer::handlePlayerShooting\(\)](#), [gme::CollisionSystem::playEnemyDeathSound\(\)](#), [gme::CollisionSystem::playPlayerDeathSound\(\)](#), [gme::WeaponSystem::tryFireSupercharged\(\)](#), [gme::EnemyAISystem::tryShoot\(\)](#), and [gme::PlayerControllerMulti::update\(\)](#).

17.17 eng Namespace Reference

Classes

- class [AScene](#)
Abstract class for scene.
- class [AsioClient](#)
Asio UDP client implementation.
- struct [CircleShape](#)
- struct [Color](#)
- struct [ConnectionStats](#)
Connection statistics.
- class [Engine](#)
Class for the engine.
- struct [Event](#)
- interface [IAudio](#)
Interface for the audio.
- interface [INetworkClient](#)
Interface for the client network.

- interface [IRenderer](#)
Interface for the renderer.
- interface [IScene](#)
interface class for scene
- struct [QueuedPacket](#)
Queued packet for sending.
- class [SceneManager](#)
Class for managing scenes.
- struct [Text](#)
- struct [WindowSize](#)

Typedefs

- using [id](#) = unsigned int

Enumerations

- enum [State](#) : unsigned char { [STOP](#) = 0 , [RUN](#) = 1 , [DEFAULT](#) = 2 }
- enum class [Status](#) { [Stopped](#) , [Paused](#) , [Playing](#) }
- enum class [ConnectionState](#) { [DISCONNECTED](#) , [CONNECTING](#) , [CONNECTED](#) , [DISCONNECTING](#) }
Connection state enumeration.
- enum class [Key](#) {
[Unknown](#) , [Escape](#) , [Enter](#) , [Space](#) ,
[Up](#) , [Down](#) , [Left](#) , [Right](#) ,
[Delete](#) , [Dot](#) , [A](#) , [B](#) ,
[C](#) , [D](#) , [E](#) , [F](#) ,
[G](#) , [H](#) , [I](#) , [J](#) ,
[K](#) , [L](#) , [M](#) , [N](#) ,
[O](#) , [P](#) , [Q](#) , [R](#) ,
[S](#) , [T](#) , [U](#) , [V](#) ,
[W](#) , [X](#) , [Y](#) , [Z](#) ,
[Num0](#) , [Num1](#) , [Num2](#) , [Num3](#) ,
[Num4](#) , [Num5](#) , [Num6](#) , [Num7](#) ,
[Num8](#) , [Num9](#) , [LControl](#) }
- enum class [EventType](#) { [Closed](#) , [KeyPressed](#) , [KeyReleased](#) , [None](#) }

17.17.1 Typedef Documentation

17.17.1.1 id

using [eng::id](#) = unsigned int

Definition at line 20 of file [IScene.hpp](#).

17.17.2 Enumeration Type Documentation

17.17.2.1 ConnectionState

enum class [eng::ConnectionState](#) [strong]

Connection state enumeration.

Enumerator

DISCONNECTED	
CONNECTING	
CONNECTED	
DISCONNECTING	

Definition at line 21 of file [INetworkClient.hpp](#).

17.17.2.2 EventType

enum class [eng::EventType](#) [strong]

Enumerator

Closed	
KeyPressed	
KeyReleased	
None	

Definition at line 94 of file [IRenderer.hpp](#).

17.17.2.3 Key

enum class [eng::Key](#) [strong]

Enumerator

Unknown	
Escape	
Enter	
Space	
Up	
Down	
Left	
Right	
Delete	
Dot	
A	
B	
C	
D	
E	
F	
G	
H	
I	
J	
K	
L	
M	
N	
O	
P	
Q	
R	
S	
T	
U	
V	
W	
X	

Enumerator

Y	
Z	
Num0	
Num1	
Num2	
Num3	
Num4	
Num5	
Num6	
Num7	
Num8	
Num9	
LControl	

Definition at line 44 of file [IRenderer.hpp](#).

17.17.2.4 State

enum [eng::State](#) : unsigned char

Enumerator

STOP	
RUN	
DEFAULT	

Definition at line 21 of file [Engine.hpp](#).

17.17.2.5 Status

enum class [eng::Status](#) [strong]

Enumerator

Stopped	
Paused	
Playing	

Definition at line 16 of file [IAudio.hpp](#).

17.18 gme Namespace Reference

Namespaces

- namespace [Utils](#)

Classes

- class [AnimationSystem](#)
- class [BeamSystem](#)
- struct [CollisionPair](#)
 - Collision pair for tracking collisions.
- class [CollisionSystem](#)

- Handles collision detection and resolution on server.
- class [ConfigMulti](#)
[ConfigMulti](#) scene.
- class [ConfigSolo](#)
[ConfigSolo](#) scene.
- class [CreateRoomScene](#)
Create room scene.
- class [EnemyAISystem](#)
Manages enemy behavior, movement patterns, and shooting.
- struct [EnemySpawn](#)
Enemy spawn definition for a wave.
- class [EnemySpawnSystem](#)
Manages enemy spawning based on level progression.
- class [EnemySystem](#)
- class [EntityManager](#)
Manages entity lifecycle and provides factory methods.
- struct [EntityMetadata](#)
Entity metadata for tracking.
- class [ExplosionSystem](#)
- class [FlappyBirdClientSolo](#)
Class for the [FlappyBirdClientSolo](#) plugin.
- class [Game](#)
[Game](#) solo scene.
- class [GameMulti](#)
[GameMulti](#) scene.
- class [GameSolo](#)
[GameSolo](#) scene.
- class [HUDSystem](#)
- interface [IGameClient](#)
Interface for the games.
- interface [IGameServer](#)
Interface for the games.
- struct [InterpolationData](#)
- class [JoinRoomScene](#)
Join room scene.
- class [LoadingAnimationSystem](#)
- class [PlayerControllerMulti](#)
- class [PlayerDirectionSystem](#)
- class [ProjectileManager](#)
Handles projectile creation and management.
- class [ProjectileSystem](#)
- class [RTypeClientMulti](#)
Class for the [RTypeClientMulti](#) plugin.
- class [RTypeClientSolo](#)
Class for the [RTypeClientSolo](#) plugin.
- class [RTypeServer](#)
Class for the [RTypeServer](#) plugin.
- class [ScrollingSystem](#)
- class [ServerScene](#)
Server configuration scene.
- class [SpawnSystem](#)

- struct [SpawnWave](#)
Spawn wave configuration.
- class [StageManager](#)
- class [WaitingRoomScene](#)
Waiting room scene where players wait before game starts.
- struct [Wave](#)
[Wave](#) definition containing multiple enemy spawns.
- class [WaveManager](#)
Manages wave-based enemy spawning with configurable patterns.
- class [WeaponSystem](#)
Manages weapon firing and charging (client-side visual feedback only)

Enumerations

- enum class [State](#) : uint8_t { [PLAYING](#) = 0 , [WIN](#) = 1 , [LOSE](#) = 2 }
- enum class [ServerEntityType](#) : uint8_t {
[PLAYER](#) = 0 , [ENEMY_BASIC](#) = 1 , [ENEMY_ADVANCED](#) = 2 , [BOSS](#) = 3 ,
[PROJECTILE_PLAYER](#) = 4 , [PROJECTILE_ENEMY](#) = 5 , [POWERUP](#) = 6 }
- Entity type enumeration for server-side management.
- enum class [LevelState](#) : uint8_t { [WAITING_FOR_PLAYERS](#) = 0 , [IN_PROGRESS](#) = 1 ,
[COMPLETED](#) = 2 , [LOOSE](#) = 3 }
- enum class [EnemyBehavior](#) : uint8_t {
[STRAIGHT](#) = 0 , [SINE_WAVE](#) = 1 , [AGGRESSIVE](#) = 2 , [STATIONARY](#) = 3 ,
[ZIGZAG](#) = 4 }
- Enemy behavior pattern types.

17.18.1 Enumeration Type Documentation

17.18.1.1 EnemyBehavior

enum class [gme::EnemyBehavior](#) : uint8_t [strong]

Enemy behavior pattern types.

Enumerator

STRAIGHT	
SINE_WAVE	
AGGRESSIVE	
STATIONARY	
ZIGZAG	

Definition at line 22 of file [EnemyAISystem.hpp](#).

17.18.1.2 LevelState

enum class [gme::LevelState](#) : uint8_t [strong]

Enumerator

WAITING_FOR_PLAYERS	
IN_PROGRESS	
COMPLETED	
LOOSE	

Definition at line 26 of file [RTypeServer.hpp](#).

17.18.1.3 ServerEntityType

enum class [gme::ServerEntityType](#) : uint8_t [strong]
Entity type enumeration for server-side management.

Enumerator

PLAYER	
ENEMY_BASIC	
ENEMY_ADVANCED	
BOSS	
PROJECTILE_PLAYER	
PROJECTILE_ENEMY	
POWERUP	

Definition at line 21 of file [EntityManager.hpp](#).

17.18.1.4 State

enum class [gme::State](#) : uint8_t [strong]

Enumerator

PLAYING	
WIN	
LOSE	

Definition at line 16 of file [IGameServer.hpp](#).

17.19 gme::Utils Namespace Reference

Functions

- `std::tuple< float, float >` [calculateHitboxOffsets](#) (float spriteWidth, float spriteHeight, float scale)

17.19.1 Function Documentation

17.19.1.1 calculateHitboxOffsets()

`std::tuple< float, float >` [gme::Utils::calculateHitboxOffsets](#) (
float spriteWidth,
float spriteHeight,
float scale) [inline]

Definition at line 9 of file [HitboxUtils.hpp](#).

Referenced by [gme::PlayerControllerMulti::createPlayer\(\)](#).

Here is the caller graph for this function:



17.20 rnp Namespace Reference

Classes

- class [EndianUtils](#)
Utility class for endianness conversion.
- struct [EntityState](#)
Entity state for WORLD_STATE packet.
- struct [EventRecord](#)
Event record for ENTITY_EVENT packets (TLV format)
- class [HandlerPacket](#)
Main packet handler class.
- struct [HandlerStats](#)
Statistics for packet handling.
- struct [LobbyInfo](#)
Lobby information structure.
- struct [PacketConnect](#)
CONNECT packet payload.
- struct [PacketConnectAccept](#)
CONNECT_ACCEPT packet payload.
- struct [PacketContext](#)
Context information for packet processing.
- struct [PacketDisconnect](#)
DISCONNECT packet payload.
- struct [PacketError](#)
ERROR packet payload.
- struct [PacketGameStart](#)
GAME_START packet payload.
- struct [PacketHeader](#)
Packet header according to RNP specification (Big Endian) Total size: 7 bytes (1 + 2 + 4)
- struct [PacketLobbyCreate](#)
LOBBY_CREATE packet payload.
- struct [PacketLobbyCreateResponse](#)
LOBBY_CREATE_RESPONSE packet payload.
- struct [PacketLobbyJoin](#)
LOBBY_JOIN packet payload.
- struct [PacketLobbyJoinResponse](#)
LOBBY_JOIN_RESPONSE packet payload.
- struct [PacketLobbyListResponse](#)
LOBBY_LIST_RESPONSE packet payload.
- struct [PacketLobbyUpdate](#)
LOBBY_UPDATE packet payload.
- struct [PacketPingPong](#)
PING/PONG packet payload.
- struct [PacketStartGameRequest](#)
START_GAME_REQUEST packet payload (client requests to start game)
- struct [PacketWorldState](#)
WORLD_STATE packet payload.
- class [Serializer](#)
Binary serializer for RNP protocol packets.

Typedefs

- using [ConnectHandler](#) = std::function<[HandlerResult](#)(const [PacketConnect](#) &, const [PacketContext](#) &>>
- using [ConnectAcceptHandler](#) = std::function<[HandlerResult](#)(const [PacketConnectAccept](#) &, const [PacketContext](#) &>>
- using [DisconnectHandler](#) = std::function<[HandlerResult](#)(const [PacketDisconnect](#) &, const [PacketContext](#) &>>
- using [WorldStateHandler](#) = std::function<[HandlerResult](#)(const [PacketWorldState](#) &, const [PacketContext](#) &>>
- using [PingHandler](#) = std::function<[HandlerResult](#)(const [PacketPingPong](#) &, const [PacketContext](#) &>>
- using [PongHandler](#) = std::function<[HandlerResult](#)(const [PacketPingPong](#) &, const [PacketContext](#) &>>
- using [ErrorHandler](#) = std::function<[HandlerResult](#)(const [PacketError](#) &, const [PacketContext](#) &>>
- using [EntityEventHandler](#) = std::function<[HandlerResult](#)(const std::vector<[EventRecord](#)> &, const [PacketContext](#) &>>
- using [AckHandler](#) = std::function<[HandlerResult](#)(std::uint32_t sequenceId, const [PacketContext](#) &>>
- using [LobbyListRequestHandler](#) = std::function<[HandlerResult](#)(const [PacketContext](#) &>>
- using [LobbyListResponseHandler](#)
- using [LobbyCreateHandler](#) = std::function<[HandlerResult](#)(const [PacketLobbyCreate](#) &, const [PacketContext](#) &>>
- using [LobbyCreateResponseHandler](#)
- using [LobbyJoinHandler](#) = std::function<[HandlerResult](#)(const [PacketLobbyJoin](#) &, const [PacketContext](#) &>>
- using [LobbyJoinResponseHandler](#)
- using [LobbyLeaveHandler](#) = std::function<[HandlerResult](#)(const [PacketContext](#) &>>
- using [LobbyUpdateHandler](#) = std::function<[HandlerResult](#)(const [PacketLobbyUpdate](#) &, const [PacketContext](#) &>>
- using [GameStartHandler](#) = std::function<[HandlerResult](#)(const [PacketGameStart](#) &, const [PacketContext](#) &>>
- using [StartGameRequestHandler](#) = std::function<[HandlerResult](#)(const [PacketStartGameRequest](#) &, const [PacketContext](#) &>>

Enumerations

- enum class [HandlerResult](#) {
[SUCCESS](#) , [INVALID_PACKET](#) , [UNSUPPORTED_TYPE](#) , [PROCESSING_ERROR](#) ,
[SESSION_INVALID](#) , [RATE_LIMITED](#) }
 Packet processing result.
- enum class [PacketType](#) : std::uint8_t {
[CONNECT](#) = 0x01 , [DISCONNECT](#) = 0x02 , [WORLD_STATE](#) = 0x03 , [PING](#) = 0x04 ,
[PONG](#) = 0x05 , [PACKET_ERROR](#) = 0x06 , [ENTITY_EVENT](#) = 0x07 , [CONNECT_ACCEPT](#)
 = 0x08 ,
[LOBBY_LIST_REQUEST](#) = 0x09 , [LOBBY_LIST_RESPONSE](#) = 0x0A , [LOBBY_CREATE](#)
 = 0x0B , [LOBBY_CREATE_RESPONSE](#) = 0x0C ,
[LOBBY_JOIN](#) = 0x0D , [LOBBY_JOIN_RESPONSE](#) = 0x0E , [LOBBY_LEAVE](#) = 0x0F ,
[LOBBY_UPDATE](#) = 0x10 ,
[GAME_START](#) = 0x11 , [START_GAME_REQUEST](#) = 0x12 }
 Packet types according to RNP specification.
- enum class [PacketFlags](#) : std::uint16_t { [NONE](#) = 0x0000 , [RELIABLE](#) = 0x0001 , [COMPRESSED](#)
 = 0x0002 }
 Packet flags for reliability and fragmentation.
- enum class [DisconnectReason](#) : std::uint16_t {
[UNSPECIFIED](#) = 0 , [CLIENT_REQUEST](#) = 1 , [TIMEOUT](#) = 2 , [PROTOCOL_ERROR](#) = 3 ,
[SERVER_SHUTDOWN](#) = 4 , [SERVER_FULL](#) = 5 , [BANNED](#) = 6 }

Disconnect reason codes.

- enum class [ErrorCode](#) : std::uint16_t {
[NONE](#) = 0 , [INVALID_PAYLOAD](#) = 1 , [UNAUTHORIZED_SESSION](#) = 2 , [RATE_LIMITED](#) = 3 ,
[INTERNAL_ERROR](#) = 4 , [LOBBY_NOT_FOUND](#) = 5 , [LOBBY_FULL](#) = 6 , [ALREADY_IN_LOBBY](#) = 7 ,
[NOT_IN_LOBBY](#) = 8 , [NOT_LOBBY_HOST](#) = 9 }

Error codes.

- enum class [EventType](#) : std::uint8_t {
[SPAWN](#) = 0x01 , [DESPAWN](#) = 0x02 , [DAMAGE](#) = 0x03 , [SCORE](#) = 0x04 ,
[POWERUP](#) = 0x05 , [INPUT](#) = 0x06 , [CUSTOM](#) = 0xFF }

Event types for ENTITY_EVENT packets.

- enum class [EntityType](#) : std::uint16_t { [PLAYER](#) = 0x01 , [ENEMY](#) = 0x02 , [PROJECTILE](#) = 0x03 , [BOSS](#) = 0x04 }

Entity types for world state.

- enum class [EntitySubtype](#) : std::uint8_t {
[NONE](#) = 0 , [ENEMY_BASIC](#) = 1 , [ENEMY_ADVANCED](#) = 2 , [ENEMY_BOSS](#) = 3 ,
[PROJECTILE_PLAYER](#) = 10 , [PROJECTILE_PLAYER_SUPERCHARGED](#) = 11 ,
[PROJECTILE_ENEMY](#) = 12 }

Entity subtype enumeration.

- enum class [LobbyStatus](#) : std::uint8_t { [WAITING](#) = 0 , [IN_GAME](#) = 1 , [FINISHED](#) = 2 }

Lobby status enumeration.

Variables

- constexpr std::size_t [MAX_PAYLOAD](#) = 512

17.20.1 Typedef Documentation

17.20.1.1 AckHandler

using [rnp::AckHandler](#) = std::function<[HandlerResult](#)(std::uint32_t sequenceId, const [PacketContext](#) &)>

Definition at line 59 of file [HandlerPacket.hpp](#).

17.20.1.2 ConnectAcceptHandler

using [rnp::ConnectAcceptHandler](#) = std::function<[HandlerResult](#)(const [PacketConnectAccept](#) &, const [PacketContext](#) &)>

Definition at line 52 of file [HandlerPacket.hpp](#).

17.20.1.3 ConnectHandler

using [rnp::ConnectHandler](#) = std::function<[HandlerResult](#)(const [PacketConnect](#) &, const [PacketContext](#) &)>

Definition at line 51 of file [HandlerPacket.hpp](#).

17.20.1.4 DisconnectHandler

using [rnp::DisconnectHandler](#) = std::function<[HandlerResult](#)(const [PacketDisconnect](#) &, const [PacketContext](#) &)>

Definition at line 53 of file [HandlerPacket.hpp](#).

17.20.1.5 EntityEventHandler

using [rnp::EntityEventHandler](#) = std::function<[HandlerResult](#)(const std::vector<[EventRecord](#)> &, const [PacketContext](#) &)>

Definition at line 58 of file [HandlerPacket.hpp](#).

17.20.1.6 ErrorHandler

using [rnp::ErrorHandler](#) = std::function<[HandlerResult](#)(const [PacketError](#) &, const [PacketContext](#) &)>

Definition at line 57 of file [HandlerPacket.hpp](#).

17.20.1.7 GameStartHandler

using `rnp::GameStartHandler` = std::function<HandlerResult(const PacketGameStart &, const PacketContext &)>
Definition at line 73 of file [HandlerPacket.hpp](#).

17.20.1.8 LobbyCreateHandler

using `rnp::LobbyCreateHandler` = std::function<HandlerResult(const PacketLobbyCreate &, const PacketContext &)>
Definition at line 65 of file [HandlerPacket.hpp](#).

17.20.1.9 LobbyCreateResponseHandler

using `rnp::LobbyCreateResponseHandler`

Initial value:

std::function<HandlerResult(const PacketLobbyCreateResponse &, const PacketContext &)>

Definition at line 66 of file [HandlerPacket.hpp](#).

17.20.1.10 LobbyJoinHandler

using `rnp::LobbyJoinHandler` = std::function<HandlerResult(const PacketLobbyJoin &, const PacketContext &)>

Definition at line 68 of file [HandlerPacket.hpp](#).

17.20.1.11 LobbyJoinResponseHandler

using `rnp::LobbyJoinResponseHandler`

Initial value:

std::function<HandlerResult(const PacketLobbyJoinResponse &, const PacketContext &)>

Definition at line 69 of file [HandlerPacket.hpp](#).

17.20.1.12 LobbyLeaveHandler

using `rnp::LobbyLeaveHandler` = std::function<HandlerResult(const PacketContext &)>

Definition at line 71 of file [HandlerPacket.hpp](#).

17.20.1.13 LobbyListRequestHandler

using `rnp::LobbyListRequestHandler` = std::function<HandlerResult(const PacketContext &)>

Definition at line 62 of file [HandlerPacket.hpp](#).

17.20.1.14 LobbyListResponseHandler

using `rnp::LobbyListResponseHandler`

Initial value:

std::function<HandlerResult(const PacketLobbyListResponse &, const PacketContext &)>

Definition at line 63 of file [HandlerPacket.hpp](#).

17.20.1.15 LobbyUpdateHandler

using `rnp::LobbyUpdateHandler` = std::function<HandlerResult(const PacketLobbyUpdate &, const PacketContext &)>

Definition at line 72 of file [HandlerPacket.hpp](#).

17.20.1.16 PingHandler

using `rnp::PingHandler` = std::function<HandlerResult(const PacketPingPong &, const PacketContext &)>

Definition at line 55 of file [HandlerPacket.hpp](#).

17.20.1.17 PongHandler

using `rnp::PongHandler` = std::function<HandlerResult(const PacketPingPong &, const PacketContext &)>

Definition at line 56 of file [HandlerPacket.hpp](#).

17.20.1.18 StartGameRequestHandler

using [rnp::StartGameRequestHandler](#) = std::function<[HandlerResult](#)(const [PacketStartGameRequest](#) &, const [PacketContext](#) &>

Definition at line 74 of file [HandlerPacket.hpp](#).

17.20.1.19 WorldStateHandler

using [rnp::WorldStateHandler](#) = std::function<[HandlerResult](#)(const [PacketWorldState](#) &, const [PacketContext](#) &>

Definition at line 54 of file [HandlerPacket.hpp](#).

17.20.2 Enumeration Type Documentation

17.20.2.1 DisconnectReason

enum class [rnp::DisconnectReason](#) : std::uint16_t [strong]

Disconnect reason codes.

Enumerator

UNSPECIFIED	
CLIENT_REQUEST	
TIMEOUT	
PROTOCOL_ERROR	
SERVER_SHUTDOWN	
SERVER_FULL	
BANNED	

Definition at line 59 of file [Protocol.hpp](#).

17.20.2.2 EntitySubtype

enum class [rnp::EntitySubtype](#) : std::uint8_t [strong]

Entity subtype enumeration.

Enumerator

NONE	
ENEMY_BASIC	
ENEMY_ADVANCED	
ENEMY_BOSS	
PROJECTILE_PLAYER	
PROJECTILE_PLAYER_SUPERCHARGED	
PROJECTILE_ENEMY	

Definition at line 165 of file [Protocol.hpp](#).

17.20.2.3 EntityType

enum class [rnp::EntityType](#) : std::uint16_t [strong]

Entity types for world state.

Enumerator

PLAYER	
ENEMY	
PROJECTILE	

BOSS	
------	--

Definition at line 104 of file [Protocol.hpp](#).

17.20.2.4 ErrorCode

enum class [rnp::ErrorCode](#) : std::uint16_t [strong]
Error codes.

Enumerator

NONE	
INVALID_PAYLOAD	
UNAUTHORIZED_SESSION	
RATE_LIMITED	
INTERNAL_ERROR	
LOBBY_NOT_FOUND	
LOBBY_FULL	
ALREADY_IN_LOBBY	
NOT_IN_LOBBY	
NOT_LOBBY_HOST	

Definition at line 73 of file [Protocol.hpp](#).

17.20.2.5 EventType

enum class [rnp::EventType](#) : std::uint8_t [strong]
Event types for ENTITY_EVENT packets.

Enumerator

SPAWN	
DESPAWN	
DAMAGE	
SCORE	
POWERUP	
INPUT	
CUSTOM	

Definition at line 90 of file [Protocol.hpp](#).

17.20.2.6 HandlerResult

enum class [rnp::HandlerResult](#) [strong]
Packet processing result.

Enumerator

SUCCESS	
INVALID_PACKET	
UNSUPPORTED_TYPE	
PROCESSING_ERROR	
SESSION_INVALID	
RATE_LIMITED	

Definition at line 23 of file [HandlerPacket.hpp](#).

17.20.2.7 LobbyStatus

enum class [rnp::LobbyStatus](#) : std::uint8_t [strong]
 Lobby status enumeration.

Enumerator

WAITING	
IN_GAME	
FINISHED	

Definition at line 224 of file [Protocol.hpp](#).

17.20.2.8 PacketFlags

enum class [rnp::PacketFlags](#) : std::uint16_t [strong]
 Packet flags for reliability and fragmentation.

Enumerator

NONE	
RELIABLE	
COMPRESSED	

Definition at line 49 of file [Protocol.hpp](#).

17.20.2.9 PacketType

enum class [rnp::PacketType](#) : std::uint8_t [strong]
 Packet types according to RNP specification.

Enumerator

CONNECT	
DISCONNECT	
WORLD_STATE	
PING	
PONG	
PACKET_ERROR	
ENTITY_EVENT	
CONNECT_ACCEPT	
LOBBY_LIST_REQUEST	
LOBBY_LIST_RESPONSE	
LOBBY_CREATE	
LOBBY_CREATE_RESPONSE	
LOBBY_JOIN	
LOBBY_JOIN_RESPONSE	
LOBBY_LEAVE	
LOBBY_UPDATE	
GAME_START	

START_GAME_REQUEST

Definition at line 23 of file [Protocol.hpp](#).

17.20.3 Variable Documentation

17.20.3.1 MAX_PAYLOAD

std::size_t rnp::MAX_PAYLOAD = 512 [inline], [constexpr]

Definition at line 18 of file [Protocol.hpp](#).

Referenced by [rnp::HandlerPacket::processPacket\(\)](#), [srv::AsioServer::sendConnectAccept\(\)](#), [rnp::Serializer::Serializer\(\)](#), and [rnp::Serializer::writeBytes\(\)](#).

17.21 srv Namespace Reference

Namespaces

- namespace [Config](#)
- namespace [Game](#)
- namespace [Path](#)

Classes

- struct [ArgsConfig](#)
- class [ArgsHandler](#)
Class to handle command line arguments.
- class [AScene](#)
Class for scene.
- class [AsioServer](#)
Asio UDP server implementation.
- struct [ClientSession](#)
Client session information.
- struct [EnvConfig](#)
- interface [INetworkServer](#)
Interface for the server network.
- class [IScene](#)
interface class for scene
- struct [Lobby](#)
[Lobby](#) structure.
- struct [QueuedPacket](#)
Queued packet for sending.
- class [SceneManager](#)
Class for managing scenes.
- class [Server](#)
Class for the server.

Typedefs

- using [json](#) = nlohmann::json
- using [id](#) = unsigned int

Enumerations

- enum class [LobbyStatus](#) : std::uint8_t {
 [SUCCESS](#) = 0 , [WAITING](#) = 1 , [IN_GAME](#) = 2 , [NOT_FOUND](#) = 3 ,
 [FULL](#) = 4 , [ALREADY_IN_LOBBY](#) = 5 }

Variables

- constexpr size_t [MAX_CLIENTS](#) = 16
- constexpr size_t [MAX_IP_LENGTH](#) = 8
- constexpr size_t [MAX_LEN_RECV_BUFFER](#) = 1024

17.21.1 Typedef Documentation

17.21.1.1 id

using [srv::id](#) = unsigned int

Definition at line [16](#) of file [IScene.hpp](#).

17.21.1.2 json

using [srv::json](#) = nlohmann::json

Definition at line [18](#) of file [ArgsHandler.hpp](#).

17.21.2 Enumeration Type Documentation

17.21.2.1 LobbyStatus

enum class [srv::LobbyStatus](#) : std::uint8_t [strong]

Enumerator

SUCCESS	
WAITING	
IN_GAME	
NOT_FOUND	
FULL	
ALREADY_IN_LOBBY	

Definition at line [49](#) of file [AsioServer.hpp](#).

17.21.3 Variable Documentation

17.21.3.1 MAX_CLIENTS

size_t [srv::MAX_CLIENTS](#) = 16 [constexpr]

Definition at line [18](#) of file [INetworkServer.hpp](#).

Referenced by [srv::AsioServer::handleConnect\(\)](#).

17.21.3.2 MAX_IP_LENGTH

size_t [srv::MAX_IP_LENGTH](#) = 8 [constexpr]

Definition at line [19](#) of file [INetworkServer.hpp](#).

17.21.3.3 MAX_LEN_RECV_BUFFER

size_t [srv::MAX_LEN_RECV_BUFFER](#) = 1024 [constexpr]

Definition at line [20](#) of file [INetworkServer.hpp](#).

17.22 srv::Config Namespace Reference

Namespaces

- namespace [Network](#)

17.23 srv::Config::Network Namespace Reference

Variables

- constexpr auto [DEFAULT_NETWORK_HOST](#) = "0.0.0.0"
- constexpr auto [DEFAULT_NETWORK_PORT](#) = 2560
- constexpr auto [DEFAULT_MAX_CLIENT](#) = 4

17.23.1 Variable Documentation

17.23.1.1 DEFAULT_MAX_CLIENT

auto srv::Config::Network::DEFAULT_MAX_CLIENT = 4 [inline], [constexpr]

Definition at line 25 of file [Common.hpp](#).

17.23.1.2 DEFAULT_NETWORK_HOST

auto srv::Config::Network::DEFAULT_NETWORK_HOST = "0.0.0.0" [inline], [constexpr]

Definition at line 23 of file [Common.hpp](#).

17.23.1.3 DEFAULT_NETWORK_PORT

auto srv::Config::Network::DEFAULT_NETWORK_PORT = 2560 [inline], [constexpr]

Definition at line 24 of file [Common.hpp](#).

17.24 srv::Game Namespace Reference

Variables

- constexpr auto [DEFAULT_TICK_RATE](#) = 60
- constexpr auto [DEFAULT_UPDATE_INTERVAL](#) = 1 / 20.F

17.24.1 Variable Documentation

17.24.1.1 DEFAULT_TICK_RATE

auto srv::Game::DEFAULT_TICK_RATE = 60 [inline], [constexpr]

Definition at line 29 of file [Common.hpp](#).

17.24.1.2 DEFAULT_UPDATE_INTERVAL

auto srv::Game::DEFAULT_UPDATE_INTERVAL = 1 / 20.F [inline], [constexpr]

Definition at line 30 of file [Common.hpp](#).

17.25 srv::Path Namespace Reference

Namespaces

- namespace [Plugin](#)

17.26 srv::Path::Plugin Namespace Reference

Variables

- auto [PLUGINS_NETWORK_ASIO_SERVER](#)
- auto [PLUGINS_GAME_RTYPE_SERVER](#)

17.26.1 Variable Documentation

17.26.1.1 PLUGINS_GAME_RTYPE_SERVER

auto srv::Path::Plugin::PLUGINS_GAME_RTYPE_SERVER [inline]

Initial value:

=

std::filesystem::path(PLUGINS_DIR) / ("game_rtype_server" + std::string(PLUGINS_EXTENSION))

Definition at line 36 of file [Common.hpp](#).

17.26.1.2 PLUGINS_NETWORK_ASIO_SERVER

auto srv::Path::Plugin::PLUGINS_NETWORK_ASIO_SERVER [inline]

Initial value:

=

std::filesystem::path(PLUGINS_DIR) / ("network_asio_server" + std::string(PLUGINS_EXTENSION))

Definition at line 34 of file [Common.hpp](#).

17.27 utl Namespace Reference

Namespaces

- namespace [cli](#)
- namespace [Config](#)
- namespace [GameConfig](#)
- namespace [Path](#)
- namespace [srv](#)

Classes

- class [Clock](#)
Class for clock.
- class [Event](#)
[Event](#) structure for inter-component communication.
- class [EventBus](#)
Thread-safe event bus for decoupled component communication.
- struct [EventConnect](#)
- class [EventStats](#)
[Event](#) statistics structure.
- interface [IPlugin](#)
Interface for plugins.
- class [Logger](#)
Class for logging.
- class [PluginLoader](#)
Modern, type-safe plugin loader.
- struct [SharedLib](#)
Handle to a dynamic library with RAII.

Typedefs

- using [LibHandle](#)
- using [EntryPointFn](#) = [IPlugin](#) *(*)()

Enumerations

- enum class [EventType](#) : std::uint32_t {
[WORLD_STATE_RECEIVED](#) = 0x1000 , [ENTITY_EVENTS_RECEIVED](#) = 0x1001 ,
[PLAYER_CONNECTED](#) = 0x1002 , [PLAYER_DISCONNECTED](#) = 0x1003 ,
[PLAYER_INPUT_RECEIVED](#) = 0x1010 , [ENTITY_EVENT_RECEIVED](#) = 0x1011 ,
[CONNECTION_ACCEPTED](#) = 0x1020 , [CONNECTION_REJECTED](#) = 0x1021 ,
[SERVER_DISCONNECTED](#) = 0x1022 , [SEND_PLAYER_INPUT](#) = 0x2000 , [SEND_ENTITY_EVENT](#)
= 0x2001 , [SEND_ENTITY_EVENTS](#) = 0x2002 ,
[BROADCAST_WORLD_STATE](#) = 0x2003 , [BROADCAST_TO_CLIENTS](#) = 0x2004 ,
[SEND_TO_CLIENT](#) = 0x2010 , [DISCONNECT_CLIENT](#) = 0x2011 ,
[REQUEST_CONNECT](#) = 0x2020 , [REQUEST_DISCONNECT](#) = 0x2021 , [SERVER_START](#) =
0x3000 , [SERVER_STOP](#) = 0x3001 ,
[SERVER_STATUS_UPDATE](#) = 0x3002 , [LOBBY_LIST_REQUEST](#) = 0x4000 , [LOBBY_LIST_RESPONSE](#)
= 0x4001 , [LOBBY_CREATE](#) = 0x4010 ,
[LOBBY_CREATE_RESPONSE](#) = 0x4011 , [LOBBY_JOIN](#) = 0x4020 , [LOBBY_JOIN_RESPONSE](#)
= 0x4021 , [LOBBY_LEAVE](#) = 0x4030 ,
[LOBBY_UPDATE](#) = 0x4040 , [GAME_START](#) = 0x4050 }
Event types for inter-component communication.
- enum class [EventPriority](#) : std::uint8_t { [LOW](#) = 0 , [NORMAL](#) = 1 , [HIGH](#) = 2 , [CRITICAL](#) =
3 }
Event priority levels.
- enum class [PluginType](#) : uint8_t {
[AUDIO](#) = 0 , [NETWORK_CLIENT](#) = 1 , [NETWORK_SERVER](#) = 1 , [RENDERER](#) = 2 ,
[GAME_CLIENT](#) = 3 , [GAME_SERVER](#) = 4 , [UNKNOWN](#) = 255 }
- enum class [LogLevel](#) : uint8_t { [INFO](#) , [WARNING](#) }

Functions

- std::pair< float, float > [calculateSpriteCenter](#) (const float x, const float y, const float spriteWidth,
const float spriteHeight, const float scale)
Calculate the center position of a sprite.
- std::pair< float, float > [calculateHitboxOffsets](#) (const float spriteWidth, const float spriteHeight,
const float scale)
Calculate hitbox offsets to center the hitbox on the sprite.
- std::pair< float, float > [calculateHitboxOffsetsRelative](#) (const float x, const float y, const float
spriteWidth, const float spriteHeight, const float scale)
Calculate hitbox offsets relative to sprite position.
- std::vector< char > [readFile](#) (const std::string &filename)
- std::unordered_map< std::string, std::string > [getEnvMap](#) (const char *const *env)

Variables

- static constexpr std::uint32_t [NETWORK_CLIENT](#) = 1
- static constexpr std::uint32_t [NETWORK_SERVER](#) = 2
- static constexpr std::uint32_t [GAME_LOGIC](#) = 3
- static constexpr std::uint32_t [RENDERING_ENGINE](#) = 4

17.27.1 Typedef Documentation

17.27.1.1 EntryPointFn

using [utl::EntryPointFn](#) = IPlugin *(*)()

Definition at line 78 of file [PluginLoader.hpp](#).

17.27.1.2 LibHandle

using [utl::LibHandle](#)

Initial value:

`void *`

Definition at line 34 of file [PluginLoader.hpp](#).

17.27.2 Enumeration Type Documentation

17.27.2.1 EventPriority

enum class [utl::EventPriority](#) : `std::uint8_t` [strong]

[Event](#) priority levels.

Enumerator

LOW	
NORMAL	
HIGH	
CRITICAL	

Definition at line 69 of file [Event.hpp](#).

17.27.2.2 EventType

enum class [utl::EventType](#) : `std::uint32_t` [strong]

[Event](#) types for inter-component communication.

Enumerator

WORLD_STATE_RECEIVED	
ENTITY_EVENTS_RECEIVED	
PLAYER_CONNECTED	
PLAYER_DISCONNECTED	
PLAYER_INPUT_RECEIVED	
ENTITY_EVENT_RECEIVED	
CONNECTION_ACCEPTED	
CONNECTION_REJECTED	
SERVER_DISCONNECTED	
SEND_PLAYER_INPUT	
SEND_ENTITY_EVENT	
SEND_ENTITY_EVENTS	
BROADCAST_WORLD_STATE	
BROADCAST_TO_CLIENTS	
SEND_TO_CLIENT	
DISCONNECT_CLIENT	
REQUEST_CONNECT	
REQUEST_DISCONNECT	
SERVER_START	
SERVER_STOP	
SERVER_STATUS_UPDATE	
LOBBY_LIST_REQUEST	

Enumerator

LOBBY_LIST_RESPONSE	
LOBBY_CREATE	
LOBBY_CREATE_RESPONSE	
LOBBY_JOIN	
LOBBY_JOIN_RESPONSE	
LOBBY_LEAVE	
LOBBY_UPDATE	
GAME_START	

Definition at line 25 of file [Event.hpp](#).

17.27.2.3 LogLevel

enum class [utl::LogLevel](#) : uint8_t [strong]

Enumerator

INFO	
WARNING	

Definition at line 17 of file [Logger.hpp](#).

17.27.2.4 PluginType

enum class [utl::PluginType](#) : uint8_t [strong]

Enumerator

AUDIO	
NETWORK_CLIENT	
NETWORK_SERVER	
RENDERER	
GAME_CLIENT	
GAME_SERVER	
UNKNOWN	

Definition at line 21 of file [IPlugin.hpp](#).

17.27.3 Function Documentation

17.27.3.1 calculateHitboxOffsets()

```
std::pair< float, float > utl::calculateHitboxOffsets (
    const float spriteWidth,
    const float spriteHeight,
    const float scale) [inline]
```

Calculate hitbox offsets to center the hitbox on the sprite.

Parameters

spriteWidth	Sprite width
spriteHeight	Sprite height
scale	Sprite scale

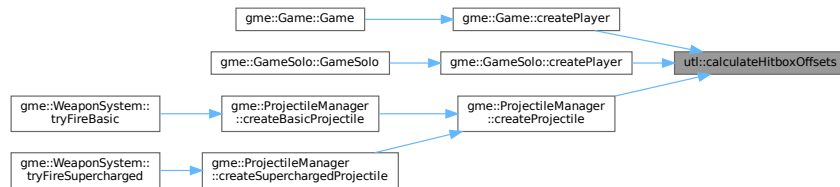
Returns

Pair of (offsetX, offsetY)

Definition at line 33 of file [HitboxUtils.hpp](#).

Referenced by [gme::Game::createPlayer\(\)](#), [gme::GameSolo::createPlayer\(\)](#), and [gme::ProjectileManager::createProjectile\(\)](#).

Here is the caller graph for this function:



17.27.3.2 calculateHitboxOffsetsRelative()

```

std::pair< float, float > utl::calculateHitboxOffsetsRelative (
    const float x,
    const float y,
    const float spriteWidth,
    const float spriteHeight,
    const float scale) [inline]
  
```

Calculate hitbox offsets relative to sprite position.

Parameters

x	Sprite X position
y	Sprite Y position
spriteWidth	Sprite width
spriteHeight	Sprite height
scale	Sprite scale

Returns

Pair of (offsetX, offsetY) relative to sprite position

Definition at line 48 of file [HitboxUtils.hpp](#).

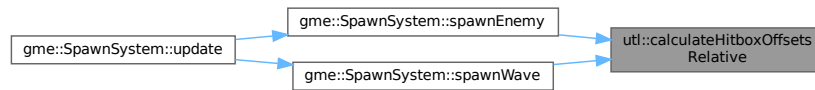
References [calculateSpriteCenter\(\)](#).

Referenced by [gme::SpawnSystem::spawnEnemy\(\)](#), and [gme::SpawnSystem::spawnWave\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



17.27.3.3 calculateSpriteCenter()

```
std::pair< float, float > utl::calculateSpriteCenter (
    const float x,
    const float y,
    const float spriteWidth,
    const float spriteHeight,
    const float scale) [inline]
```

Calculate the center position of a sprite.

Parameters

x	Sprite X position
y	Sprite Y position
spriteWidth	Sprite width
spriteHeight	Sprite height
scale	Sprite scale

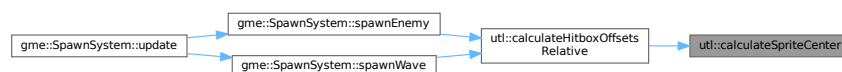
Returns

Pair of (centerX, centerY)

Definition at line 20 of file [HitboxUtils.hpp](#).

Referenced by [calculateHitboxOffsetsRelative\(\)](#).

Here is the caller graph for this function:



17.27.3.4 getEnvMap()

```
std::unordered_map< std::string, std::string > utl::getEnvMap (
    const char *const * env) [nodiscard]
```

Definition at line 30 of file [utils.cpp](#).

17.27.3.5 readFile()

```
std::vector< char > utl::readFile (
    const std::string & filename) [nodiscard]
```

Definition at line 9 of file [utils.cpp](#).

17.27.4 Variable Documentation

17.27.4.1 GAME_LOGIC

`std::uint32_t utl::GAME_LOGIC = 3` [static], [constexpr]

Definition at line 19 of file [Event.hpp](#).

Referenced by [srv::AsioServer::broadcastGameStart\(\)](#), [srv::AsioServer::handleConnect\(\)](#), [srv::AsioServer::handleDisconnect\(\)](#), [srv::AsioServer::handleEntityEvent\(\)](#), [srv::AsioServer::handleStartGameRequest\(\)](#), [gme::RTypeServer::RTypeServer\(\)](#), and [gme::RTypeServer::update\(\)](#).

17.27.4.2 NETWORK_CLIENT

`std::uint32_t utl::NETWORK_CLIENT = 1` [static], [constexpr]

Definition at line 17 of file [Event.hpp](#).

Referenced by [gme::ServerScene::connectServer\(\)](#), [gme::CreateRoomScene::createRoom\(\)](#), [gme::JoinRoomScene::event\(\)](#), [gme::WaitingRoomScene::leaveLobby\(\)](#), [gme::JoinRoomScene::refreshRoomList\(\)](#), [gme::PlayerControllerMulti::sendInput\(\)](#), and [gme::WaitingRoomScene::startGame\(\)](#).

17.27.4.3 NETWORK_SERVER

`std::uint32_t utl::NETWORK_SERVER = 2` [static], [constexpr]

Definition at line 18 of file [Event.hpp](#).

Referenced by [srv::AsioServer::broadcastGameStart\(\)](#), and [gme::RTypeServer::broadcastWorldState\(\)](#).

17.27.4.4 RENDERING_ENGINE

`std::uint32_t utl::RENDERING_ENGINE = 4` [static], [constexpr]

Definition at line 20 of file [Event.hpp](#).

Referenced by [eng::AsioClient::handleConnectAccept\(\)](#).

17.28 utl::cli Namespace Reference

Classes

- struct [AppConfig](#)

17.29 utl::Config Namespace Reference

Namespaces

- namespace [Audio](#)
- namespace [Color](#)
- namespace [Game](#)
- namespace [Network](#)
- namespace [Player](#)
- namespace [Window](#)

17.30 utl::Config::Audio Namespace Reference

Variables

- constexpr auto [DEFAULT_AUDIO_VOLUME](#) = 0.5F
- constexpr auto [DEFAULT_AUDIO_MUTED](#) = false

17.30.1 Variable Documentation

17.30.1.1 DEFAULT_AUDIO_MUTED

`auto utl::Config::Audio::DEFAULT_AUDIO_MUTED = false` [inline], [constexpr]

Definition at line 33 of file [Common.hpp](#).

17.30.1.2 DEFAULT_AUDIO_VOLUME

auto utl::Config::Audio::DEFAULT_AUDIO_VOLUME = 0.5F [inline], [constexpr]

Definition at line 32 of file [Common.hpp](#).

Referenced by [cli::Client::setupConfig\(\)](#).

17.31 utl::Config::Color Namespace Reference

Variables

- static constexpr [eng::Color DARK](#) = {.r = 0U, .g = 0U, .b = 0U, .a = 255U}
- static constexpr [eng::Color CYAN_ELECTRIC](#) = {.r = 0U, .g = 191U, .b = 255U, .a = 255U}
- static constexpr [eng::Color GRAY_BLUE_SUBTLE](#) = {.r = 160U, .g = 160U, .b = 160U, .a = 255U}
- static constexpr [eng::Color STAR_BG](#) = {.r = 100U, .g = 100U, .b = 150U, .a = 80U}
- static constexpr [eng::Color STAR_MID](#) = {.r = 150U, .g = 150U, .b = 200U, .a = 120U}
- static constexpr [eng::Color YELLOW](#) = {.r = 255U, .g = 255U, .b = 200U, .a = 200U}
- static constexpr [eng::Color CYAN_ELECTRIC_PARTICLES](#) = {.r = 0U, .g = 191U, .b = 255U, .a = 100U}
- static constexpr [eng::Color CYAN_ELECTRIC_FOREGROUND](#) = {.r = 0U, .g = 191U, .b = 255U, .a = 180U}
- static constexpr [eng::Color INFO_TEXT_COLOR](#) = {.r = 180U, .g = 180U, .b = 180U, .a = 200U}
- static constexpr [eng::Color WHITE](#) = {.r = 255U, .g = 255U, .b = 255U, .a = 255U}
- static constexpr [eng::Color WHITE_LOW](#) = {.r = 255U, .g = 255U, .b = 255U, .a = 0U}
- static constexpr [eng::Color TEXT_VALUE_COLOR](#) = {.r = 200U, .g = 200U, .b = 255U, .a = 255U}
- static constexpr [eng::Color WHITE_TRANS](#) = {.r = 255U, .g = 255U, .b = 255U, .a = 100U}
- static constexpr [eng::Color BLUE](#) = {.r = 200U, .g = 200U, .b = 255U, .a = 150U}
- static constexpr [eng::Color BLUE_SECOND](#) = {.r = 50U, .g = 100U, .b = 200U, .a = 60U}
- static constexpr [eng::Color PURPLE](#) = {.r = 100U, .g = 50U, .b = 150U, .a = 80U}
- static constexpr [eng::Color GREEN](#) = {.r = 200U, .g = 255U, .b = 200U, .a = 180U}
- static constexpr [eng::Color GREEN_READY](#) = {0U, 255U, 100U, 255U}

17.31.1 Variable Documentation

17.31.1.1 BLUE

[eng::Color](#) utl::Config::Color::BLUE = {.r = 200U, .g = 200U, .b = 255U, .a = 150U} [static], [constexpr]

Definition at line 68 of file [Common.hpp](#).

Referenced by [ecs::StarfieldSystem::createStarfield\(\)](#).

17.31.1.2 BLUE_SECOND

[eng::Color](#) utl::Config::Color::BLUE_SECOND = {.r = 50U, .g = 100U, .b = 200U, .a = 60U} [static], [constexpr]

Definition at line 69 of file [Common.hpp](#).

Referenced by [ecs::StarfieldSystem::createNebulae\(\)](#).

17.31.1.3 CYAN_ELECTRIC

[eng::Color](#) utl::Config::Color::CYAN_ELECTRIC = {.r = 0U, .g = 191U, .b = 255U, .a = 255U} [static], [constexpr]

Definition at line 56 of file [Common.hpp](#).

Referenced by [gme::ConfigMulti::ConfigMulti\(\)](#), [gme::ConfigSolo::ConfigSolo\(\)](#), [gme::CreateRoomScene::CreateRoomScene\(\)](#), [gme::Game::Game\(\)](#), [gme::JoinRoomScene::JoinRoomScene\(\)](#), [cli::Menu::Menu\(\)](#), [gme::ServerScene::ServerScene\(\)](#), [cli::Settings::Settings\(\)](#), [cli::Menu::update\(\)](#), [cli::Settings::update\(\)](#), [gme::ConfigMulti::update\(\)](#), [gme::ConfigSolo::update\(\)](#), [gme::WaitingRoomScene::update\(\)](#), and [gme::WaitingRoomScene::WaitingRoomScene\(\)](#).

17.31.1.4 CYAN_ELECTRIC_FOREGROUND

`eng::Color` `utl::Config::Color::CYAN_ELECTRIC_FOREGROUND = {.r = 0U, .g = 191U, .b = 255U, .a = 180U}` `[static]`, `[constexpr]`

Definition at line 62 of file [Common.hpp](#).

17.31.1.5 CYAN_ELECTRIC_PARTICLES

`eng::Color` `utl::Config::Color::CYAN_ELECTRIC_PARTICLES = {.r = 0U, .g = 191U, .b = 255U, .a = 100U}` `[static]`, `[constexpr]`

Definition at line 61 of file [Common.hpp](#).

17.31.1.6 DARK

`eng::Color` `utl::Config::Color::DARK = {.r = 0U, .g = 0U, .b = 0U, .a = 255U}` `[static]`, `[constexpr]`

Definition at line 55 of file [Common.hpp](#).

Referenced by [cli::Client::run\(\)](#).

17.31.1.7 GRAY_BLUE_SUBTLE

`eng::Color` `utl::Config::Color::GRAY_BLUE_SUBTLE = {.r = 160U, .g = 160U, .b = 160U, .a = 255U}` `[static]`, `[constexpr]`

Definition at line 57 of file [Common.hpp](#).

Referenced by [gme::ConfigMulti::ConfigMulti\(\)](#), [gme::CreateRoomScene::CreateRoomScene\(\)](#), [gme::JoinRoomScene::JoinRoomScene\(\)](#), [cli::Menu::Menu\(\)](#), [gme::ServerScene::ServerScene\(\)](#), [cli::Settings::Settings\(\)](#), [cli::Menu::update\(\)](#), [cli::Settings::update\(\)](#), [gme::ConfigMulti::update\(\)](#), [gme::ConfigSolo::update\(\)](#), [gme::CreateRoomScene::update\(\)](#), [gme::JoinRoomScene::update\(\)](#), [gme::ServerScene::update\(\)](#), [gme::WaitingRoomScene::update\(\)](#), and [gme::WaitingRoomScene::WaitingRoomScene\(\)](#).

17.31.1.8 GREEN

`eng::Color` `utl::Config::Color::GREEN = {.r = 200U, .g = 255U, .b = 200U, .a = 180U}` `[static]`, `[constexpr]`

Definition at line 71 of file [Common.hpp](#).

Referenced by [ecs::StarfieldSystem::createComets\(\)](#), and [ecs::StarfieldSystem::createShootingStars\(\)](#).

17.31.1.9 GREEN_READY

`eng::Color` `utl::Config::Color::GREEN_READY = {0U, 255U, 100U, 255U}` `[static]`, `[constexpr]`

Definition at line 72 of file [Common.hpp](#).

Referenced by [gme::WaitingRoomScene::updatePlayerDisplay\(\)](#).

17.31.1.10 INFO_TEXT_COLOR

`eng::Color` `utl::Config::Color::INFO_TEXT_COLOR = {.r = 180U, .g = 180U, .b = 180U, .a = 200U}` `[static]`, `[constexpr]`

Definition at line 63 of file [Common.hpp](#).

Referenced by [gme::JoinRoomScene::JoinRoomScene\(\)](#), [cli::Settings::Settings\(\)](#), [gme::WaitingRoomScene::updatePlayerDisplay\(\)](#), and [gme::WaitingRoomScene::WaitingRoomScene\(\)](#).

17.31.1.11 PURPLE

`eng::Color` `utl::Config::Color::PURPLE = {.r = 100U, .g = 50U, .b = 150U, .a = 80U}` `[static]`, `[constexpr]`

Definition at line 70 of file [Common.hpp](#).

Referenced by [ecs::StarfieldSystem::createPlanets\(\)](#).

17.31.1.12 STAR_BG

`eng::Color` `utl::Config::Color::STAR_BG = {.r = 100U, .g = 100U, .b = 150U, .a = 80U}` `[static]`, `[constexpr]`

Definition at line 58 of file [Common.hpp](#).

17.31.1.13 STAR_MID

[eng::Color](#) utl::Config::Color::STAR_MID = {.r = 150U, .g = 150U, .b = 200U, .a = 120U} [static], [constexpr]

Definition at line 59 of file [Common.hpp](#).

17.31.1.14 TEXT_VALUE_COLOR

[eng::Color](#) utl::Config::Color::TEXT_VALUE_COLOR = {.r = 200U, .g = 200U, .b = 255U, .a = 255U} [static], [constexpr]

Definition at line 66 of file [Common.hpp](#).

Referenced by [gme::CreateRoomScene::CreateRoomScene\(\)](#), [gme::ServerScene::ServerScene\(\)](#), [cli::Settings::Settings\(\)](#), [gme::JoinRoomScene::update\(\)](#), [gme::JoinRoomScene::updateRoomDisplay\(\)](#), and [gme::WaitingRoomScene::WaitingRoomScene\(\)](#).

17.31.1.15 WHITE

[eng::Color](#) utl::Config::Color::WHITE = {.r = 255U, .g = 255U, .b = 255U, .a = 255U} [static], [constexpr]

Definition at line 64 of file [Common.hpp](#).

Referenced by [gme::ConfigSolo::ConfigSolo\(\)](#), and [cli::Settings::Settings\(\)](#).

17.31.1.16 WHITE_LOW

[eng::Color](#) utl::Config::Color::WHITE_LOW = {.r = 255U, .g = 255U, .b = 255U, .a = 0U} [static], [constexpr]

Definition at line 65 of file [Common.hpp](#).

Referenced by [cli::Intro::Intro\(\)](#).

17.31.1.17 WHITE_TRANS

[eng::Color](#) utl::Config::Color::WHITE_TRANS = {.r = 255U, .g = 255U, .b = 255U, .a = 100U} [static], [constexpr]

Definition at line 67 of file [Common.hpp](#).

Referenced by [ecs::StarfieldSystem::createStarfield\(\)](#).

17.31.1.18 YELLOW

[eng::Color](#) utl::Config::Color::YELLOW = {.r = 255U, .g = 255U, .b = 200U, .a = 200U} [static], [constexpr]

Definition at line 60 of file [Common.hpp](#).

Referenced by [ecs::StarfieldSystem::createStarfield\(\)](#).

17.32 utl::Config::Game Namespace Reference

Variables

- constexpr auto [DEFAULT_VIDEO_QUALITY](#) = 1
- constexpr auto [DEFAULT_CONTROL_SCHEME](#) = 2
- constexpr auto [DEFAULT_SKIN_INDEX](#) = 0

17.32.1 Variable Documentation

17.32.1.1 DEFAULT_CONTROL_SCHEME

auto utl::Config::Game::DEFAULT_CONTROL_SCHEME = 2 [inline], [constexpr]

Definition at line 50 of file [Common.hpp](#).

Referenced by [cli::Client::setupConfig\(\)](#).

17.32.1.2 DEFAULT_SKIN_INDEX

auto utl::Config::Game::DEFAULT_SKIN_INDEX = 0 [inline], [constexpr]

Definition at line 51 of file [Common.hpp](#).

Referenced by [cli::Client::setupConfig\(\)](#).

17.32.1.3 DEFAULT_VIDEO_QUALITY

auto utl::Config::Game::DEFAULT_VIDEO_QUALITY = 1 [inline], [constexpr]

Definition at line 49 of file [Common.hpp](#).

Referenced by [cli::Client::setupConfig\(\)](#).

17.33 utl::Config::Network Namespace Reference

Variables

- constexpr auto [DEFAULT_NETWORK_HOST](#) = "127.0.0.1"
- constexpr auto [DEFAULT_NETWORK_PORT](#) = 2560

17.33.1 Variable Documentation

17.33.1.1 DEFAULT_NETWORK_HOST

auto utl::Config::Network::DEFAULT_NETWORK_HOST = "127.0.0.1" [inline], [constexpr]

Definition at line 37 of file [Common.hpp](#).

17.33.1.2 DEFAULT_NETWORK_PORT

auto utl::Config::Network::DEFAULT_NETWORK_PORT = 2560 [inline], [constexpr]

Definition at line 38 of file [Common.hpp](#).

17.34 utl::Config::Player Namespace Reference

Variables

- constexpr auto [DEFAULT_PLAYER_NAME](#) = "player"

17.34.1 Variable Documentation

17.34.1.1 DEFAULT_PLAYER_NAME

auto utl::Config::Player::DEFAULT_PLAYER_NAME = "player" [inline], [constexpr]

Definition at line 28 of file [Common.hpp](#).

17.35 utl::Config::Window Namespace Reference

Variables

- constexpr auto [DEFAULT_WINDOW_WIDTH](#) = 1920
- constexpr auto [DEFAULT_WINDOW_HEIGHT](#) = 1080
- constexpr auto [DEFAULT_WINDOW_FRAME_LIMIT](#) = 240
- constexpr auto [DEFAULT_WINDOW_FULLSCREEN](#) = false

17.35.1 Variable Documentation

17.35.1.1 DEFAULT_WINDOW_FRAME_LIMIT

auto utl::Config::Window::DEFAULT_WINDOW_FRAME_LIMIT = 240 [inline], [constexpr]

Definition at line 44 of file [Common.hpp](#).

17.35.1.2 DEFAULT_WINDOW_FULLSCREEN

auto utl::Config::Window::DEFAULT_WINDOW_FULLSCREEN = false [inline], [constexpr]

Definition at line 45 of file [Common.hpp](#).

17.35.1.3 DEFAULT_WINDOW_HEIGHT

auto utl::Config::Window::DEFAULT_WINDOW_HEIGHT = 1080 [inline], [constexpr]
Definition at line 43 of file [Common.hpp](#).

17.35.1.4 DEFAULT_WINDOW_WIDTH

auto utl::Config::Window::DEFAULT_WINDOW_WIDTH = 1920 [inline], [constexpr]
Definition at line 42 of file [Common.hpp](#).

17.36 utl::GameConfig Namespace Reference

Namespaces

- namespace [Animation](#)
- namespace [Beam](#)
- namespace [Enemy](#)
- namespace [Explosion](#)
- namespace [Hitbox](#)
- namespace [LoadingAnimation](#)
- namespace [Player](#)
- namespace [Projectile](#)
- namespace [Screen](#)
- namespace [Server](#)
- namespace [Stage](#)

17.37 utl::GameConfig::Animation Namespace Reference

Variables

- constexpr float [FRAME_DURATION](#) = 0.1f

17.37.1 Variable Documentation

17.37.1.1 FRAME_DURATION

float utl::GameConfig::Animation::FRAME_DURATION = 0.1f [inline], [constexpr]
Definition at line 62 of file [GameConfig.hpp](#).

17.38 utl::GameConfig::Beam Namespace Reference

Variables

- constexpr float [MAX_CHARGE](#) = 1.0f
- constexpr float [CHARGE_RATE](#) = 1.0f
- constexpr float [BAR_WIDTH](#) = 120.0f
- constexpr float [BAR_HEIGHT](#) = 12.0f
- constexpr float [BAR_X](#) = 10.0f
- constexpr float [BAR_Y](#) = 10.0f

17.38.1 Variable Documentation

17.38.1.1 BAR_HEIGHT

float utl::GameConfig::Beam::BAR_HEIGHT = 12.0f [inline], [constexpr]
Definition at line 69 of file [GameConfig.hpp](#).
Referenced by [ecs::BeamSystem::update\(\)](#).

17.38.1.2 BAR_WIDTH

float utl::GameConfig::Beam::BAR_WIDTH = 120.0f [inline], [constexpr]

Definition at line 68 of file [GameConfig.hpp](#).

Referenced by [ecs::BeamSystem::update\(\)](#).

17.38.1.3 BAR_X

float utl::GameConfig::Beam::BAR_X = 10.0f [inline], [constexpr]

Definition at line 70 of file [GameConfig.hpp](#).

17.38.1.4 BAR_Y

float utl::GameConfig::Beam::BAR_Y = 10.0f [inline], [constexpr]

Definition at line 71 of file [GameConfig.hpp](#).

17.38.1.5 CHARGE_RATE

float utl::GameConfig::Beam::CHARGE_RATE = 1.0f [inline], [constexpr]

Definition at line 67 of file [GameConfig.hpp](#).

Referenced by [gme::WeaponSystem::update\(\)](#).

17.38.1.6 MAX_CHARGE

float utl::GameConfig::Beam::MAX_CHARGE = 1.0f [inline], [constexpr]

Definition at line 66 of file [GameConfig.hpp](#).

Referenced by [gme::GameSolo::createPlayer\(\)](#), and [gme::PlayerControllerMulti::createPlayer\(\)](#).

17.39 utl::GameConfig::Enemy Namespace Reference

Namespaces

- namespace [Easy](#)

17.40 utl::GameConfig::Enemy::Easy Namespace Reference

Variables

- constexpr float [HEALTH](#) = 1.0f
- constexpr float [DAMAGE](#) = 5.0f
- constexpr float [SPEED](#) = 80.0f
- constexpr float [SPRITE_WIDTH](#) = 32.0f
- constexpr float [SPRITE_HEIGHT](#) = 32.0f
- constexpr float [SCALE](#) = 2.0f
- constexpr float [SHOOT_COOLDOWN](#) = 2.0f
- constexpr float [SPAWN_RATE](#) = 2.0f
- constexpr int [ANIMATION_FRAMES](#) = 4
- constexpr float [ANIMATION_DURATION](#) = 0.5f
- constexpr int [FRAMES_PER_ROW](#) = 4

17.40.1 Variable Documentation

17.40.1.1 ANIMATION_DURATION

float utl::GameConfig::Enemy::Easy::ANIMATION_DURATION = 0.5f [inline], [constexpr]

Definition at line 100 of file [GameConfig.hpp](#).

Referenced by [gme::SpawnSystem::spawnEnemy\(\)](#), and [gme::SpawnSystem::spawnWave\(\)](#).

17.40.1.2 ANIMATION_FRAMES

int utl::GameConfig::Enemy::Easy::ANIMATION_FRAMES = 4 [inline], [constexpr]

Definition at line 99 of file [GameConfig.hpp](#).

Referenced by [gme::SpawnSystem::spawnEnemy\(\)](#), and [gme::SpawnSystem::spawnWave\(\)](#).

17.40.1.3 DAMAGE

float utl::GameConfig::Enemy::Easy::DAMAGE = 5.0f [inline], [constexpr]

Definition at line 92 of file [GameConfig.hpp](#).

Referenced by [gme::SpawnSystem::spawnEnemy\(\)](#), and [gme::SpawnSystem::spawnWave\(\)](#).

17.40.1.4 FRAMES_PER_ROW

int utl::GameConfig::Enemy::Easy::FRAMES_PER_ROW = 4 [inline], [constexpr]

Definition at line 101 of file [GameConfig.hpp](#).

Referenced by [gme::SpawnSystem::spawnEnemy\(\)](#), and [gme::SpawnSystem::spawnWave\(\)](#).

17.40.1.5 HEALTH

float utl::GameConfig::Enemy::Easy::HEALTH = 1.0f [inline], [constexpr]

Definition at line 91 of file [GameConfig.hpp](#).

Referenced by [gme::SpawnSystem::spawnEnemy\(\)](#), and [gme::SpawnSystem::spawnWave\(\)](#).

17.40.1.6 SCALE

float utl::GameConfig::Enemy::Easy::SCALE = 2.0f [inline], [constexpr]

Definition at line 96 of file [GameConfig.hpp](#).

Referenced by [gme::SpawnSystem::spawnEnemy\(\)](#), and [gme::SpawnSystem::spawnWave\(\)](#).

17.40.1.7 SHOOT_COOLDOWN

float utl::GameConfig::Enemy::Easy::SHOOT_COOLDOWN = 2.0f [inline], [constexpr]

Definition at line 97 of file [GameConfig.hpp](#).

Referenced by [gme::SpawnSystem::spawnEnemy\(\)](#), and [gme::SpawnSystem::spawnWave\(\)](#).

17.40.1.8 SPAWN_RATE

float utl::GameConfig::Enemy::Easy::SPAWN_RATE = 2.0f [inline], [constexpr]

Definition at line 98 of file [GameConfig.hpp](#).

Referenced by [gme::SpawnSystem::update\(\)](#).

17.40.1.9 SPEED

float utl::GameConfig::Enemy::Easy::SPEED = 80.0f [inline], [constexpr]

Definition at line 93 of file [GameConfig.hpp](#).

Referenced by [gme::SpawnSystem::spawnEnemy\(\)](#), and [gme::SpawnSystem::spawnWave\(\)](#).

17.40.1.10 SPRITE_HEIGHT

float utl::GameConfig::Enemy::Easy::SPRITE_HEIGHT = 32.0f [inline], [constexpr]

Definition at line 95 of file [GameConfig.hpp](#).

Referenced by [gme::SpawnSystem::spawnEnemy\(\)](#), and [gme::SpawnSystem::spawnWave\(\)](#).

17.40.1.11 SPRITE_WIDTH

float utl::GameConfig::Enemy::Easy::SPRITE_WIDTH = 32.0f [inline], [constexpr]

Definition at line 94 of file [GameConfig.hpp](#).

Referenced by [gme::SpawnSystem::spawnEnemy\(\)](#), [gme::SpawnSystem::spawnWave\(\)](#), and [gme::EnemySystem::update\(\)](#).

17.41 utl::GameConfig::Explosion Namespace Reference

Variables

- constexpr float [SPRITE_WIDTH](#) = 32.0f
- constexpr float [SPRITE_HEIGHT](#) = 32.0f
- constexpr int [ANIMATION_FRAMES](#) = 4
- constexpr float [ANIMATION_DURATION](#) = 0.1f
- constexpr int [FRAMES_PER_ROW](#) = 4
- constexpr float [LIFETIME](#) = 0.4f
- constexpr float [SCALE](#) = 2.0f

17.41.1 Variable Documentation

17.41.1.1 ANIMATION_DURATION

float utl::GameConfig::Explosion::ANIMATION_DURATION = 0.1f [inline], [constexpr]

Definition at line 109 of file [GameConfig.hpp](#).

Referenced by [gme::CollisionSystem::createExplosion\(\)](#), and [gme::GameMulti::handleWorldStateUpdate\(\)](#).

17.41.1.2 ANIMATION_FRAMES

int utl::GameConfig::Explosion::ANIMATION_FRAMES = 4 [inline], [constexpr]

Definition at line 108 of file [GameConfig.hpp](#).

Referenced by [gme::CollisionSystem::createExplosion\(\)](#), and [gme::GameMulti::handleWorldStateUpdate\(\)](#).

17.41.1.3 FRAMES_PER_ROW

int utl::GameConfig::Explosion::FRAMES_PER_ROW = 4 [inline], [constexpr]

Definition at line 110 of file [GameConfig.hpp](#).

Referenced by [gme::CollisionSystem::createExplosion\(\)](#), and [gme::GameMulti::handleWorldStateUpdate\(\)](#).

17.41.1.4 LIFETIME

float utl::GameConfig::Explosion::LIFETIME = 0.4f [inline], [constexpr]

Definition at line 111 of file [GameConfig.hpp](#).

Referenced by [gme::CollisionSystem::createExplosion\(\)](#), and [gme::GameMulti::handleWorldStateUpdate\(\)](#).

17.41.1.5 SCALE

float utl::GameConfig::Explosion::SCALE = 2.0f [inline], [constexpr]

Definition at line 112 of file [GameConfig.hpp](#).

Referenced by [gme::CollisionSystem::createExplosion\(\)](#), and [gme::GameMulti::handleWorldStateUpdate\(\)](#).

17.41.1.6 SPRITE_HEIGHT

float utl::GameConfig::Explosion::SPRITE_HEIGHT = 32.0f [inline], [constexpr]

Definition at line 107 of file [GameConfig.hpp](#).

Referenced by [gme::CollisionSystem::createExplosion\(\)](#), and [gme::GameMulti::handleWorldStateUpdate\(\)](#).

17.41.1.7 SPRITE_WIDTH

float utl::GameConfig::Explosion::SPRITE_WIDTH = 32.0f [inline], [constexpr]

Definition at line 106 of file [GameConfig.hpp](#).

Referenced by [gme::CollisionSystem::createExplosion\(\)](#), and [gme::GameMulti::handleWorldStateUpdate\(\)](#).

17.42 utl::GameConfig::Hitbox Namespace Reference

Variables

- constexpr float [PLAYER_RADIUS](#) = 20.0f

- constexpr float [ENEMY_RADIUS](#) = 15.0f
- constexpr float [PROJECTILE_BASIC_RADIUS](#) = 5.0f
- constexpr float [PROJECTILE_SUPERCHARGED_RADIUS](#) = 8.0f

17.42.1 Variable Documentation

17.42.1.1 ENEMY_RADIUS

float utl::GameConfig::Hitbox::ENEMY_RADIUS = 15.0f [inline], [constexpr]

Definition at line 117 of file [GameConfig.hpp](#).

Referenced by [gme::SpawnSystem::spawnEnemy\(\)](#), and [gme::SpawnSystem::spawnWave\(\)](#).

17.42.1.2 PLAYER_RADIUS

float utl::GameConfig::Hitbox::PLAYER_RADIUS = 20.0f [inline], [constexpr]

Definition at line 116 of file [GameConfig.hpp](#).

Referenced by [gme::GameSolo::createPlayer\(\)](#), and [gme::PlayerControllerMulti::createPlayer\(\)](#).

17.42.1.3 PROJECTILE_BASIC_RADIUS

float utl::GameConfig::Hitbox::PROJECTILE_BASIC_RADIUS = 5.0f [inline], [constexpr]

Definition at line 118 of file [GameConfig.hpp](#).

Referenced by [gme::ProjectileManager::createProjectile\(\)](#).

17.42.1.4 PROJECTILE_SUPERCHARGED_RADIUS

float utl::GameConfig::Hitbox::PROJECTILE_SUPERCHARGED_RADIUS = 8.0f [inline], [constexpr]

Definition at line 119 of file [GameConfig.hpp](#).

Referenced by [gme::ProjectileManager::createProjectile\(\)](#).

17.43 utl::GameConfig::LoadingAnimation Namespace Reference

Variables

- constexpr float [SPRITE_WIDTH](#) = 29.0f
- constexpr float [SPRITE_HEIGHT](#) = 24.0f
- constexpr int [ANIMATION_FRAMES](#) = 4
- constexpr float [ANIMATION_DURATION](#) = 0.15f
- constexpr float [OFFSET_X](#) = 60.0f
- constexpr float [OFFSET_Y](#) = 6.0f

17.43.1 Variable Documentation

17.43.1.1 ANIMATION_DURATION

float utl::GameConfig::LoadingAnimation::ANIMATION_DURATION = 0.15f [inline], [constexpr]

Definition at line 78 of file [GameConfig.hpp](#).

Referenced by [gme::WeaponSystem::showLoadingAnimation\(\)](#).

17.43.1.2 ANIMATION_FRAMES

int utl::GameConfig::LoadingAnimation::ANIMATION_FRAMES = 4 [inline], [constexpr]

Definition at line 77 of file [GameConfig.hpp](#).

Referenced by [gme::WeaponSystem::showLoadingAnimation\(\)](#).

17.43.1.3 OFFSET_X

float utl::GameConfig::LoadingAnimation::OFFSET_X = 60.0f [inline], [constexpr]

Definition at line 79 of file [GameConfig.hpp](#).

Referenced by [gme::WeaponSystem::showLoadingAnimation\(\)](#).

17.43.1.4 OFFSET_Y

float utl::GameConfig::LoadingAnimation::OFFSET_Y = 6.0f [inline], [constexpr]

Definition at line 80 of file [GameConfig.hpp](#).

Referenced by [gme::WeaponSystem::showLoadingAnimation\(\)](#).

17.43.1.5 SPRITE_HEIGHT

float utl::GameConfig::LoadingAnimation::SPRITE_HEIGHT = 24.0f [inline], [constexpr]

Definition at line 76 of file [GameConfig.hpp](#).

Referenced by [gme::WeaponSystem::showLoadingAnimation\(\)](#).

17.43.1.6 SPRITE_WIDTH

float utl::GameConfig::LoadingAnimation::SPRITE_WIDTH = 29.0f [inline], [constexpr]

Definition at line 75 of file [GameConfig.hpp](#).

Referenced by [gme::WeaponSystem::showLoadingAnimation\(\)](#).

17.44 utl::GameConfig::Player Namespace Reference

Variables

- constexpr float [SPEED](#) = 500.0f
- constexpr float [DIAGONAL_SPEED_MULTIPLIER](#) = 0.707f
- constexpr float [SPRITE_WIDTH](#) = 33.0f
- constexpr float [SPRITE_HEIGHT](#) = 17.0f
- constexpr float [WIDTH](#) = 66.0f
- constexpr float [HEIGHT](#) = 17.0f
- constexpr float [SCALE](#) = 2.0f
- constexpr int [FRAMES_PER_ROW](#) = 5

17.44.1 Variable Documentation

17.44.1.1 DIAGONAL_SPEED_MULTIPLIER

float utl::GameConfig::Player::DIAGONAL_SPEED_MULTIPLIER = 0.707f [inline], [constexpr]

Definition at line 25 of file [GameConfig.hpp](#).

Referenced by [gme::GameSolo::handlePlayerInputs\(\)](#).

17.44.1.2 FRAMES_PER_ROW

int utl::GameConfig::Player::FRAMES_PER_ROW = 5 [inline], [constexpr]

Definition at line 31 of file [GameConfig.hpp](#).

Referenced by [gme::PlayerDirectionSystem::update\(\)](#).

17.44.1.3 HEIGHT

float utl::GameConfig::Player::HEIGHT = 17.0f [inline], [constexpr]

Definition at line 29 of file [GameConfig.hpp](#).

17.44.1.4 SCALE

float utl::GameConfig::Player::SCALE = 2.0f [inline], [constexpr]

Definition at line 30 of file [GameConfig.hpp](#).

Referenced by [gme::GameSolo::createPlayer\(\)](#), [gme::PlayerControllerMulti::createPlayer\(\)](#), [gme::GameSolo::handlePlayerInput\(\)](#) and [gme::GameMulti::handleWorldStateUpdate\(\)](#).

17.44.1.5 SPEED

float utl::GameConfig::Player::SPEED = 500.0f [inline], [constexpr]

Definition at line 24 of file [GameConfig.hpp](#).

Referenced by [gme::GameSolo::handlePlayerInputs\(\)](#).

17.44.1.6 SPRITE_HEIGHT

float utl::GameConfig::Player::SPRITE_HEIGHT = 17.0f [inline], [constexpr]

Definition at line 27 of file [GameConfig.hpp](#).

Referenced by [gme::GameSolo::createPlayer\(\)](#), [gme::PlayerControllerMulti::createPlayer\(\)](#), [gme::GameMulti::GameMulti\(\)](#), [gme::GameSolo::handlePlayerInputs\(\)](#), [gme::GameMulti::handleWorldStateUpdate\(\)](#), [gme::PlayerDirectionSystem::update\(\)](#), [gme::GameMulti::updatePlayerSkin\(\)](#), and [gme::GameSolo::updatePlayerSkin\(\)](#).

17.44.1.7 SPRITE_WIDTH

float utl::GameConfig::Player::SPRITE_WIDTH = 33.0f [inline], [constexpr]

Definition at line 26 of file [GameConfig.hpp](#).

Referenced by [gme::GameSolo::createPlayer\(\)](#), [gme::PlayerControllerMulti::createPlayer\(\)](#), [gme::GameSolo::handlePlayerInputs\(\)](#), [gme::GameMulti::handleWorldStateUpdate\(\)](#), [ecs::BeamSystem::update\(\)](#), and [gme::PlayerDirectionSystem::update\(\)](#).

17.44.1.8 WIDTH

float utl::GameConfig::Player::WIDTH = 66.0f [inline], [constexpr]

Definition at line 28 of file [GameConfig.hpp](#).

17.45 utl::GameConfig::Projectile Namespace Reference

Namespaces

- namespace [Basic](#)
- namespace [Supercharged](#)

17.46 utl::GameConfig::Projectile::Basic Namespace Reference

Variables

- constexpr float [SPEED](#) = 800.0f
- constexpr float [DAMAGE](#) = 10.0f
- constexpr float [LIFETIME](#) = 3.0f
- constexpr float [SCALE](#) = 1.0f
- constexpr float [FIRE_COOLDOWN](#) = 0.3f
- constexpr float [SPRITE_WIDTH](#) = 16.0f
- constexpr float [SPRITE_HEIGHT](#) = 8.0f

17.46.1 Variable Documentation

17.46.1.1 DAMAGE

float utl::GameConfig::Projectile::Basic::DAMAGE = 10.0f [inline], [constexpr]

Definition at line 38 of file [GameConfig.hpp](#).

Referenced by [gme::ProjectileManager::createProjectile\(\)](#).

17.46.1.2 FIRE_COOLDOWN

float utl::GameConfig::Projectile::Basic::FIRE_COOLDOWN = 0.3f [inline], [constexpr]

Definition at line 41 of file [GameConfig.hpp](#).

Referenced by [gme::WeaponSystem::tryFireBasic\(\)](#).

17.46.1.3 LIFETIME

float utl::GameConfig::Projectile::Basic::LIFETIME = 3.0f [inline], [constexpr]

Definition at line 39 of file [GameConfig.hpp](#).

Referenced by [gme::ProjectileManager::createProjectile\(\)](#).

17.46.1.4 SCALE

float utl::GameConfig::Projectile::Basic::SCALE = 1.0f [inline], [constexpr]

Definition at line 40 of file [GameConfig.hpp](#).

Referenced by [gme::ProjectileManager::createProjectile\(\)](#).

17.46.1.5 SPEED

float utl::GameConfig::Projectile::Basic::SPEED = 800.0f [inline], [constexpr]

Definition at line 37 of file [GameConfig.hpp](#).

Referenced by [gme::WeaponSystem::tryFireBasic\(\)](#).

17.46.1.6 SPRITE_HEIGHT

float utl::GameConfig::Projectile::Basic::SPRITE_HEIGHT = 8.0f [inline], [constexpr]

Definition at line 43 of file [GameConfig.hpp](#).

Referenced by [gme::ProjectileManager::createProjectile\(\)](#).

17.46.1.7 SPRITE_WIDTH

float utl::GameConfig::Projectile::Basic::SPRITE_WIDTH = 16.0f [inline], [constexpr]

Definition at line 42 of file [GameConfig.hpp](#).

Referenced by [gme::ProjectileManager::createProjectile\(\)](#).

17.47 utl::GameConfig::Projectile::Supercharged Namespace Reference

Variables

- constexpr float [SPEED](#) = 1200.0f
- constexpr float [DAMAGE](#) = 25.0f
- constexpr float [LIFETIME](#) = 5.0f
- constexpr float [SCALE](#) = 1.5f
- constexpr float [FIRE_COOLDOWN](#) = 0.2f
- constexpr float [CHARGE_TIME](#) = 0.5f
- constexpr float [SPRITE_WIDTH](#) = 29.0f
- constexpr float [SPRITE_HEIGHT](#) = 24.0f
- constexpr int [ANIMATION_FRAMES](#) = 4
- constexpr float [ANIMATION_DURATION](#) = 0.15f

17.47.1 Variable Documentation

17.47.1.1 ANIMATION_DURATION

float utl::GameConfig::Projectile::Supercharged::ANIMATION_DURATION = 0.15f [inline], [constexpr]

Definition at line 57 of file [GameConfig.hpp](#).

Referenced by [gme::ProjectileManager::createProjectile\(\)](#).

17.47.1.2 ANIMATION_FRAMES

int utl::GameConfig::Projectile::Supercharged::ANIMATION_FRAMES = 4 [inline], [constexpr]

Definition at line 56 of file [GameConfig.hpp](#).

Referenced by [gme::ProjectileManager::createProjectile\(\)](#).

17.47.1.3 CHARGE_TIME

float utl::GameConfig::Projectile::Supercharged::CHARGE_TIME = 0.5f [inline], [constexpr]
Definition at line 53 of file [GameConfig.hpp](#).

17.47.1.4 DAMAGE

float utl::GameConfig::Projectile::Supercharged::DAMAGE = 25.0f [inline], [constexpr]
Definition at line 49 of file [GameConfig.hpp](#).
Referenced by [gme::ProjectileManager::createProjectile\(\)](#).

17.47.1.5 FIRE_COOLDOWN

float utl::GameConfig::Projectile::Supercharged::FIRE_COOLDOWN = 0.2f [inline], [constexpr]
Definition at line 52 of file [GameConfig.hpp](#).

17.47.1.6 LIFETIME

float utl::GameConfig::Projectile::Supercharged::LIFETIME = 5.0f [inline], [constexpr]
Definition at line 50 of file [GameConfig.hpp](#).
Referenced by [gme::ProjectileManager::createProjectile\(\)](#).

17.47.1.7 SCALE

float utl::GameConfig::Projectile::Supercharged::SCALE = 1.5f [inline], [constexpr]
Definition at line 51 of file [GameConfig.hpp](#).
Referenced by [gme::ProjectileManager::createProjectile\(\)](#).

17.47.1.8 SPEED

float utl::GameConfig::Projectile::Supercharged::SPEED = 1200.0f [inline], [constexpr]
Definition at line 48 of file [GameConfig.hpp](#).
Referenced by [gme::WeaponSystem::tryFireSupercharged\(\)](#).

17.47.1.9 SPRITE_HEIGHT

float utl::GameConfig::Projectile::Supercharged::SPRITE_HEIGHT = 24.0f [inline], [constexpr]
Definition at line 55 of file [GameConfig.hpp](#).
Referenced by [gme::ProjectileManager::createProjectile\(\)](#).

17.47.1.10 SPRITE_WIDTH

float utl::GameConfig::Projectile::Supercharged::SPRITE_WIDTH = 29.0f [inline], [constexpr]
Definition at line 54 of file [GameConfig.hpp](#).
Referenced by [gme::ProjectileManager::createProjectile\(\)](#).

17.48 utl::GameConfig::Screen Namespace Reference

Variables

- constexpr float [SPAWN_X](#) = 1950.0f
- constexpr float [MIN_Y](#) = 50.0f
- constexpr float [MAX_Y](#) = 1030.0f
- constexpr float [REMOVE_X](#) = -100.0f
- constexpr float [REMOVE_MIN_Y](#) = -50.0f
- constexpr float [REMOVE_MAX_Y](#) = 1130.0f

17.48.1 Variable Documentation

17.48.1.1 MAX_Y

float utl::GameConfig::Screen::MAX_Y = 1030.0f [inline], [constexpr]

Definition at line 17 of file [GameConfig.hpp](#).

Referenced by [gme::SpawnSystem::spawnEnemy\(\)](#), and [gme::SpawnSystem::spawnWave\(\)](#).

17.48.1.2 MIN_Y

float utl::GameConfig::Screen::MIN_Y = 50.0f [inline], [constexpr]

Definition at line 16 of file [GameConfig.hpp](#).

Referenced by [gme::SpawnSystem::spawnEnemy\(\)](#), and [gme::SpawnSystem::spawnWave\(\)](#).

17.48.1.3 REMOVE_MAX_Y

float utl::GameConfig::Screen::REMOVE_MAX_Y = 1130.0f [inline], [constexpr]

Definition at line 20 of file [GameConfig.hpp](#).

Referenced by [gme::EnemySystem::update\(\)](#).

17.48.1.4 REMOVE_MIN_Y

float utl::GameConfig::Screen::REMOVE_MIN_Y = -50.0f [inline], [constexpr]

Definition at line 19 of file [GameConfig.hpp](#).

Referenced by [gme::EnemySystem::update\(\)](#).

17.48.1.5 REMOVE_X

float utl::GameConfig::Screen::REMOVE_X = -100.0f [inline], [constexpr]

Definition at line 18 of file [GameConfig.hpp](#).

Referenced by [gme::EnemySystem::update\(\)](#).

17.48.1.6 SPAWN_X

float utl::GameConfig::Screen::SPAWN_X = 1950.0f [inline], [constexpr]

Definition at line 15 of file [GameConfig.hpp](#).

Referenced by [gme::SpawnSystem::spawnEnemy\(\)](#), and [gme::SpawnSystem::spawnWave\(\)](#).

17.49 utl::GameConfig::Server Namespace Reference

Namespaces

- namespace [Enemy](#)
- namespace [Network](#)
- namespace [Projectile](#)

Variables

- constexpr float [SCREEN_WIDTH](#) = 1920.0f
- constexpr float [SCREEN_HEIGHT](#) = 1080.0f
- constexpr float [WORLD_MARGIN](#) = 200.0f

17.49.1 Variable Documentation

17.49.1.1 SCREEN_HEIGHT

float utl::GameConfig::Server::SCREEN_HEIGHT = 1080.0f [inline], [constexpr]

Definition at line 124 of file [GameConfig.hpp](#).

Referenced by [gme::EnemyAISystem::clampToScreen\(\)](#), [gme::EntityManager::createPlayer\(\)](#), and [gme::RTYPEServer::updateEntities\(\)](#).

17.49.1.2 SCREEN_WIDTH

float utl::GameConfig::Server::SCREEN_WIDTH = 1920.0f [inline], [constexpr]

Definition at line 123 of file [GameConfig.hpp](#).

Referenced by [gme::EnemyAISystem::clampToScreen\(\)](#), [gme::EntityManager::createPlayer\(\)](#), and [gme::RTypeServer::updateEntities\(\)](#).

17.49.1.3 WORLD_MARGIN

float utl::GameConfig::Server::WORLD_MARGIN = 200.0f [inline], [constexpr]

Definition at line 125 of file [GameConfig.hpp](#).

Referenced by [gme::RTypeServer::updateEntities\(\)](#).

17.50 utl::GameConfig::Server::Enemy Namespace Reference

Namespaces

- namespace [Advanced](#)
- namespace [AI](#)
- namespace [Basic](#)
- namespace [Boss](#)

17.51 utl::GameConfig::Server::Enemy::Advanced Namespace Reference

Variables

- constexpr float [HEALTH](#) = 100.0f
- constexpr float [DAMAGE](#) = 15.0f
- constexpr float [SPEED](#) = 150.0f
- constexpr float [SHOOT_COOLDOWN](#) = 1.5f
- constexpr float [HITBOX_RADIUS](#) = 15.0f
- constexpr float [SINE_FREQUENCY](#) = 1.5f
- constexpr float [SINE_AMPLITUDE](#) = 100.0f

17.51.1 Variable Documentation

17.51.1.1 DAMAGE

float utl::GameConfig::Server::Enemy::Advanced::DAMAGE = 15.0f [inline], [constexpr]

Definition at line 141 of file [GameConfig.hpp](#).

17.51.1.2 HEALTH

float utl::GameConfig::Server::Enemy::Advanced::HEALTH = 100.0f [inline], [constexpr]

Definition at line 140 of file [GameConfig.hpp](#).

17.51.1.3 HITBOX_RADIUS

float utl::GameConfig::Server::Enemy::Advanced::HITBOX_RADIUS = 15.0f [inline], [constexpr]

Definition at line 144 of file [GameConfig.hpp](#).

17.51.1.4 SHOOT_COOLDOWN

float utl::GameConfig::Server::Enemy::Advanced::SHOOT_COOLDOWN = 1.5f [inline], [constexpr]

Definition at line 143 of file [GameConfig.hpp](#).

17.51.1.5 SINE_AMPLITUDE

float utl::GameConfig::Server::Enemy::Advanced::SINE_AMPLITUDE = 100.0f [inline], [constexpr]

Definition at line 146 of file [GameConfig.hpp](#).

17.51.1.6 SINE_FREQUENCY

float utl::GameConfig::Server::Enemy::Advanced::SINE_FREQUENCY = 1.5f [inline], [constexpr]

Definition at line 145 of file [GameConfig.hpp](#).

17.51.1.7 SPEED

float utl::GameConfig::Server::Enemy::Advanced::SPEED = 150.0f [inline], [constexpr]

Definition at line 142 of file [GameConfig.hpp](#).

17.52 utl::GameConfig::Server::Enemy::AI Namespace Reference

Variables

- constexpr float [SHOOT_CHANCE](#) = 0.3f
- constexpr float [MAX_SHOOT_DISTANCE](#) = 1500.0f
- constexpr float [AGGRESSIVE_ACTIVATION_DISTANCE](#) = 400.0f
- constexpr float [ZIGZAG_PERIOD](#) = 1.5f
- constexpr float [ZIGZAG_SPEED_X](#) = 150.0f
- constexpr float [ZIGZAG_SPEED_Y](#) = 100.0f

17.52.1 Variable Documentation

17.52.1.1 AGGRESSIVE_ACTIVATION_DISTANCE

float utl::GameConfig::Server::Enemy::AI::AGGRESSIVE_ACTIVATION_DISTANCE = 400.0f [inline], [constexpr]

Definition at line 174 of file [GameConfig.hpp](#).

17.52.1.2 MAX_SHOOT_DISTANCE

float utl::GameConfig::Server::Enemy::AI::MAX_SHOOT_DISTANCE = 1500.0f [inline], [constexpr]

Definition at line 173 of file [GameConfig.hpp](#).

17.52.1.3 SHOOT_CHANCE

float utl::GameConfig::Server::Enemy::AI::SHOOT_CHANCE = 0.3f [inline], [constexpr]

Definition at line 172 of file [GameConfig.hpp](#).

17.52.1.4 ZIGZAG_PERIOD

float utl::GameConfig::Server::Enemy::AI::ZIGZAG_PERIOD = 1.5f [inline], [constexpr]

Definition at line 175 of file [GameConfig.hpp](#).

17.52.1.5 ZIGZAG_SPEED_X

float utl::GameConfig::Server::Enemy::AI::ZIGZAG_SPEED_X = 150.0f [inline], [constexpr]

Definition at line 176 of file [GameConfig.hpp](#).

17.52.1.6 ZIGZAG_SPEED_Y

float utl::GameConfig::Server::Enemy::AI::ZIGZAG_SPEED_Y = 100.0f [inline], [constexpr]

Definition at line 177 of file [GameConfig.hpp](#).

17.53 utl::GameConfig::Server::Enemy::Basic Namespace Reference

Variables

- constexpr float [HEALTH](#) = 50.0f
- constexpr float [DAMAGE](#) = 10.0f
- constexpr float [SPEED](#) = 200.0f

- constexpr float [SHOOT_COOLDOWN](#) = 2.0f
- constexpr float [HITBOX_RADIUS](#) = 15.0f

17.53.1 Variable Documentation

17.53.1.1 DAMAGE

float utl::GameConfig::Server::Enemy::Basic::DAMAGE = 10.0f [inline], [constexpr]

Definition at line 132 of file [GameConfig.hpp](#).

17.53.1.2 HEALTH

float utl::GameConfig::Server::Enemy::Basic::HEALTH = 50.0f [inline], [constexpr]

Definition at line 131 of file [GameConfig.hpp](#).

17.53.1.3 HITBOX_RADIUS

float utl::GameConfig::Server::Enemy::Basic::HITBOX_RADIUS = 15.0f [inline], [constexpr]

Definition at line 135 of file [GameConfig.hpp](#).

17.53.1.4 SHOOT_COOLDOWN

float utl::GameConfig::Server::Enemy::Basic::SHOOT_COOLDOWN = 2.0f [inline], [constexpr]

Definition at line 134 of file [GameConfig.hpp](#).

17.53.1.5 SPEED

float utl::GameConfig::Server::Enemy::Basic::SPEED = 200.0f [inline], [constexpr]

Definition at line 133 of file [GameConfig.hpp](#).

17.54 utl::GameConfig::Server::Enemy::Boss Namespace Reference

Variables

- constexpr float [HEALTH](#) = 1000.0f
- constexpr float [DAMAGE](#) = 50.0f
- constexpr float [HITBOX_RADIUS](#) = 50.0f
- constexpr float [RAM_DAMAGE](#) = 20.0f
- constexpr float [PHASE_2_THRESHOLD](#) = 0.66f
- constexpr float [PHASE_3_THRESHOLD](#) = 0.33f
- constexpr float [PHASE1_SPEED](#) = 30.0f
- constexpr float [PHASE1_COOLDOWN](#) = 1.5f
- constexpr float [PHASE1_SINE_FREQUENCY](#) = 0.5f
- constexpr float [PHASE1_SINE_AMPLITUDE](#) = 150.0f
- constexpr float [PHASE2_COOLDOWN](#) = 1.0f
- constexpr float [PHASE3_SPEED](#) = 50.0f
- constexpr float [PHASE3_COOLDOWN](#) = 0.5f
- constexpr float [PHASE3_AGGRESSION](#) = 80.0f
- constexpr float [SPREAD_INTERVAL](#) = 1.0f
- constexpr float [SPREAD_ANGLE_DEGREES](#) = 15.0f
- constexpr float [SPREAD_SPEED](#) = 300.0f

17.54.1 Variable Documentation

17.54.1.1 DAMAGE

float utl::GameConfig::Server::Enemy::Boss::DAMAGE = 50.0f [inline], [constexpr]

Definition at line 152 of file [GameConfig.hpp](#).

17.54.1.2 HEALTH

float utl::GameConfig::Server::Enemy::Boss::HEALTH = 1000.0f [inline], [constexpr]

Definition at line 151 of file [GameConfig.hpp](#).

17.54.1.3 HITBOX_RADIUS

float utl::GameConfig::Server::Enemy::Boss::HITBOX_RADIUS = 50.0f [inline], [constexpr]

Definition at line 153 of file [GameConfig.hpp](#).

17.54.1.4 PHASE1_COOLDOWN

float utl::GameConfig::Server::Enemy::Boss::PHASE1_COOLDOWN = 1.5f [inline], [constexpr]

Definition at line 158 of file [GameConfig.hpp](#).

17.54.1.5 PHASE1_SINE_AMPLITUDE

float utl::GameConfig::Server::Enemy::Boss::PHASE1_SINE_AMPLITUDE = 150.0f [inline], [constexpr]

Definition at line 160 of file [GameConfig.hpp](#).

17.54.1.6 PHASE1_SINE_FREQUENCY

float utl::GameConfig::Server::Enemy::Boss::PHASE1_SINE_FREQUENCY = 0.5f [inline], [constexpr]

Definition at line 159 of file [GameConfig.hpp](#).

17.54.1.7 PHASE1_SPEED

float utl::GameConfig::Server::Enemy::Boss::PHASE1_SPEED = 30.0f [inline], [constexpr]

Definition at line 157 of file [GameConfig.hpp](#).

17.54.1.8 PHASE2_COOLDOWN

float utl::GameConfig::Server::Enemy::Boss::PHASE2_COOLDOWN = 1.0f [inline], [constexpr]

Definition at line 161 of file [GameConfig.hpp](#).

17.54.1.9 PHASE3_AGGRESSION

float utl::GameConfig::Server::Enemy::Boss::PHASE3_AGGRESSION = 80.0f [inline], [constexpr]

Definition at line 164 of file [GameConfig.hpp](#).

17.54.1.10 PHASE3_COOLDOWN

float utl::GameConfig::Server::Enemy::Boss::PHASE3_COOLDOWN = 0.5f [inline], [constexpr]

Definition at line 163 of file [GameConfig.hpp](#).

17.54.1.11 PHASE3_SPEED

float utl::GameConfig::Server::Enemy::Boss::PHASE3_SPEED = 50.0f [inline], [constexpr]

Definition at line 162 of file [GameConfig.hpp](#).

17.54.1.12 PHASE_2_THRESHOLD

float utl::GameConfig::Server::Enemy::Boss::PHASE_2_THRESHOLD = 0.66f [inline], [constexpr]

Definition at line 155 of file [GameConfig.hpp](#).

17.54.1.13 PHASE_3_THRESHOLD

float utl::GameConfig::Server::Enemy::Boss::PHASE_3_THRESHOLD = 0.33f [inline], [constexpr]

Definition at line 156 of file [GameConfig.hpp](#).

17.54.1.14 RAM_DAMAGE

float utl::GameConfig::Server::Enemy::Boss::RAM_DAMAGE = 20.0f [inline], [constexpr]

Definition at line 154 of file [GameConfig.hpp](#).

17.54.1.15 SPREAD_ANGLE_DEGREES

float utl::GameConfig::Server::Enemy::Boss::SPREAD_ANGLE_DEGREES = 15.0f [inline], [constexpr]

Definition at line 166 of file [GameConfig.hpp](#).

17.54.1.16 SPREAD_INTERVAL

float utl::GameConfig::Server::Enemy::Boss::SPREAD_INTERVAL = 1.0f [inline], [constexpr]

Definition at line 165 of file [GameConfig.hpp](#).

17.54.1.17 SPREAD_SPEED

float utl::GameConfig::Server::Enemy::Boss::SPREAD_SPEED = 300.0f [inline], [constexpr]

Definition at line 167 of file [GameConfig.hpp](#).

17.55 utl::GameConfig::Server::Network Namespace Reference

Variables

- constexpr float [BROADCAST_RATE](#) = 60.0f
- constexpr float [BROADCAST_INTERVAL](#) = 1.0f / [BROADCAST_RATE](#)

17.55.1 Variable Documentation

17.55.1.1 BROADCAST_INTERVAL

float utl::GameConfig::Server::Network::BROADCAST_INTERVAL = 1.0f / [BROADCAST_RATE](#) [inline], [constexpr]

Definition at line 191 of file [GameConfig.hpp](#).

17.55.1.2 BROADCAST_RATE

float utl::GameConfig::Server::Network::BROADCAST_RATE = 60.0f [inline], [constexpr]

Definition at line 190 of file [GameConfig.hpp](#).

17.56 utl::GameConfig::Server::Projectile Namespace Reference

Variables

- constexpr float [ENEMY_SPEED](#) = 500.0f
- constexpr float [ENEMY_DAMAGE](#) = 15.0f
- constexpr float [ENEMY_LIFETIME](#) = 10.0f

17.56.1 Variable Documentation

17.56.1.1 ENEMY_DAMAGE

float utl::GameConfig::Server::Projectile::ENEMY_DAMAGE = 15.0f [inline], [constexpr]

Definition at line 184 of file [GameConfig.hpp](#).

17.56.1.2 ENEMY_LIFETIME

float utl::GameConfig::Server::Projectile::ENEMY_LIFETIME = 10.0f [inline], [constexpr]

Definition at line 185 of file [GameConfig.hpp](#).

17.56.1.3 ENEMY_SPEED

float utl::GameConfig::Server::Projectile::ENEMY_SPEED = 500.0f [inline], [constexpr]
 Definition at line 183 of file [GameConfig.hpp](#).

17.57 utl::GameConfig::Stage Namespace Reference

Variables

- constexpr float [FLOOR_OFFSET_Y](#) = 16.0f
- constexpr float [CEILING_OFFSET_Y](#) = -1.0f

17.57.1 Variable Documentation

17.57.1.1 CEILING_OFFSET_Y

float utl::GameConfig::Stage::CEILING_OFFSET_Y = -1.0f [inline], [constexpr]
 Definition at line 85 of file [GameConfig.hpp](#).
 Referenced by [gme::ScrollingSystem::update\(\)](#).

17.57.1.2 FLOOR_OFFSET_Y

float utl::GameConfig::Stage::FLOOR_OFFSET_Y = 16.0f [inline], [constexpr]
 Definition at line 84 of file [GameConfig.hpp](#).
 Referenced by [gme::ScrollingSystem::update\(\)](#).

17.58 utl::Path Namespace Reference

Namespaces

- namespace [Audio](#)
- namespace [Font](#)
- namespace [Icons](#)
- namespace [Plugin](#)
- namespace [Texture](#)

17.59 utl::Path::Audio Namespace Reference

Variables

- constexpr auto [AUDIO_TITLE](#) = "assets/audio/title.mp3"
- constexpr auto [AUDIO_COIN](#) = "assets/audio/coin.mp3"
- constexpr auto [AUDIO_BATTLE_THEME](#) = "assets/audio/battle_theme.mp3"
- constexpr auto [AUDIO_INPUT](#) = "assets/audio/input.wav"
- constexpr auto [AUDIO_DEATH_ALLIES](#) = "assets/audio/deathallies.wav"
- constexpr auto [AUDIO_DEATH_ENEMIES](#) = "assets/audio/deathennemies.wav"
- constexpr auto [AUDIO_SUPERCHARGED_SHOT](#) = "assets/audio/shoot.wav"
- constexpr auto [AUDIO_BEGIN](#) = "assets/audio/begin.wav"
- constexpr auto [AUDIO_GOOFY](#) = "assets/audio/goofy.wav"
- constexpr auto [AUDIO_BOSS](#) = "assets/audio/Bossmusic.mp3"

17.59.1 Variable Documentation

17.59.1.1 AUDIO_BATTLE_THEME

auto utl::Path::Audio::AUDIO_BATTLE_THEME = "assets/audio/battle_theme.mp3" [inline], [constexpr]
 Definition at line 86 of file [Common.hpp](#).

17.59.1.2 AUDIO_BEGIN

auto utl::Path::Audio::AUDIO_BEGIN = "assets/audio/begin.wav" [inline], [constexpr]

Definition at line 91 of file [Common.hpp](#).

Referenced by [gme::GameMulti::GameMulti\(\)](#), and [gme::GameSolo::GameSolo\(\)](#).

17.59.1.3 AUDIO_BOSS

auto utl::Path::Audio::AUDIO_BOSS = "assets/audio/Bossmusic.mp3" [inline], [constexpr]

Definition at line 93 of file [Common.hpp](#).

Referenced by [gme::GameMulti::update\(\)](#).

17.59.1.4 AUDIO_COIN

auto utl::Path::Audio::AUDIO_COIN = "assets/audio/coin.mp3" [inline], [constexpr]

Definition at line 85 of file [Common.hpp](#).

17.59.1.5 AUDIO_DEATH_ALLIES

auto utl::Path::Audio::AUDIO_DEATH_ALLIES = "assets/audio/deathallies.wav" [inline], [constexpr]

Definition at line 88 of file [Common.hpp](#).

Referenced by [gme::CollisionSystem::ensurePlayerDeathAudio\(\)](#).

17.59.1.6 AUDIO_DEATH_ENEMIES

auto utl::Path::Audio::AUDIO_DEATH_ENEMIES = "assets/audio/deathennemies.wav" [inline], [constexpr]

Definition at line 89 of file [Common.hpp](#).

Referenced by [gme::CollisionSystem::ensureEnemyDeathChannel\(\)](#).

17.59.1.7 AUDIO_GOOFY

auto utl::Path::Audio::AUDIO_GOOFY = "assets/audio/goofy.wav" [inline], [constexpr]

Definition at line 92 of file [Common.hpp](#).

17.59.1.8 AUDIO_INPUT

auto utl::Path::Audio::AUDIO_INPUT = "assets/audio/input.wav" [inline], [constexpr]

Definition at line 87 of file [Common.hpp](#).

Referenced by [ecs::AudioSystem::AudioSystem\(\)](#).

17.59.1.9 AUDIO_SUPERCHARGED_SHOT

auto utl::Path::Audio::AUDIO_SUPERCHARGED_SHOT = "assets/audio/shoot.wav" [inline], [constexpr]

Definition at line 90 of file [Common.hpp](#).

Referenced by [gme::WeaponSystem::ensureSuperShotAudio\(\)](#).

17.59.1.10 AUDIO_TITLE

auto utl::Path::Audio::AUDIO_TITLE = "assets/audio/title.mp3" [inline], [constexpr]

Definition at line 84 of file [Common.hpp](#).

17.60 utl::Path::Font Namespace Reference

Variables

- constexpr auto [FONTS_RTYPE](#) = "assets/fonts/r-type.otf"

17.60.1 Variable Documentation

17.60.1.1 FONTS_RTYPE

auto utl::Path::Font::FONTS_RTYPE = "assets/fonts/r-type.otf" [inline], [constexpr]

Definition at line 97 of file [Common.hpp](#).

Referenced by [gme::ConfigMulti::ConfigMulti\(\)](#), [gme::ConfigSolo::ConfigSolo\(\)](#), [gme::CreateRoomScene::CreateRoomScene\(\)](#), [gme::Game::Game\(\)](#), [gme::JoinRoomScene::JoinRoomScene\(\)](#), [cli::Menu::Menu\(\)](#), [gme::ServerScene::ServerScene\(\)](#), [cli::Settings::Settings\(\)](#), [gme::WaitingRoomScene::updatePlayerDisplay\(\)](#), [gme::JoinRoomScene::updateRoomDisplay\(\)](#), and [gme::WaitingRoomScene::WaitingRoomScene\(\)](#).

17.61 utl::Path::Icons Namespace Reference

Variables

- constexpr auto [ICON_APP](#) = "assets/icons/icon.png"

17.61.1 Variable Documentation

17.61.1.1 ICON_APP

auto utl::Path::Icons::ICON_APP = "assets/icons/icon.png" [inline], [constexpr]

Definition at line 80 of file [Common.hpp](#).

Referenced by [cli::Intro::Intro\(\)](#).

17.62 utl::Path::Plugin Namespace Reference

Variables

- auto [PLUGIN_AUDIO_SFML](#)
- auto [PLUGIN_NETWORK_ASIO_CLIENT](#)
- auto [PLUGIN_RENDERER_SFML](#)
- auto [PLUGIN_GAME_SOLO](#)
- auto [PLUGIN_GAME_MULTI](#)

17.62.1 Variable Documentation

17.62.1.1 PLUGIN_AUDIO_SFML

auto utl::Path::Plugin::PLUGIN_AUDIO_SFML [inline]

Initial value:

=

std::filesystem::path(PLUGINS_DIR) / ("audio_sfml" + std::string([PLUGINS_EXTENSION](#)))

Definition at line 101 of file [Common.hpp](#).

Referenced by [cli::Client::Client\(\)](#).

17.62.1.2 PLUGIN_GAME_MULTI

auto utl::Path::Plugin::PLUGIN_GAME_MULTI [inline]

Initial value:

=

std::filesystem::path(PLUGINS_DIR) / ("game_rtype_client_multi" + std::string([PLUGINS_EXTENSION](#)))

Definition at line 109 of file [Common.hpp](#).

Referenced by [cli::Client::Client\(\)](#).

17.62.1.3 PLUGIN_GAME_SOLO

auto utl::Path::Plugin::PLUGIN_GAME_SOLO [inline]

Initial value:

=

std::filesystem::path(PLUGINS_DIR) / ("game_rtype_client_solo" + std::string([PLUGINS_EXTENSION](#)))

Definition at line 107 of file [Common.hpp](#).

Referenced by [cli::Client::Client\(\)](#).

17.62.1.4 PLUGIN_NETWORK_ASIO_CLIENT

auto utl::Path::Plugin::PLUGIN_NETWORK_ASIO_CLIENT [inline]

Initial value:

```
=
    std::filesystem::path(PLUGINS_DIR) / ("network_asio_client" + std::string(PLUGINS_EXTENSION))
```

Definition at line 103 of file [Common.hpp](#).

Referenced by [cli::Client::Client\(\)](#).

17.62.1.5 PLUGIN_RENDERER_SFML

auto utl::Path::Plugin::PLUGIN_RENDERER_SFML [inline]

Initial value:

```
=
    std::filesystem::path(PLUGINS_DIR) / ("renderer_sfml" + std::string(PLUGINS_EXTENSION))
```

Definition at line 105 of file [Common.hpp](#).

Referenced by [cli::Client::Client\(\)](#).

17.63 utl::Path::Texture Namespace Reference

Variables

- constexpr auto [TEXTURE_PLAYER](#) = "assets/sprites/r-typesheet42.gif"
- constexpr auto [TEXTURE_SHOOT](#) = "assets/sprites/shoot.gif"
- constexpr auto [TEXTURE_SHOOT_CHARGED](#) = "assets/sprites/shootcharged.gif"
- constexpr auto [TEXTURE_SHOOT_LOADING](#) = "assets/sprites/shootchargedloading.gif"
- constexpr auto [TEXTURE_ENEMY_EASY](#) = "assets/sprites/r-typesheet5.gif"
- constexpr auto [TEXTURE_ENEMY_BASIC](#) = "assets/sprites/r-typesheet5.gif"
- constexpr auto [TEXTURE_ENEMY_ADVANCED](#) = "assets/sprites/r-typesheet3.gif"
- constexpr auto [TEXTURE_BOSS](#) = "assets/sprites/r-typesheet14.gif"
- constexpr auto [TEXTURE_ENEMY_PROJECTILE](#) = "assets/sprites/r-typesheet20.gif"
- constexpr auto [TEXTURE_EXPLOSION](#) = "assets/sprites/r-typesheet44.gif"
- constexpr auto [TEXTURE_STAGE1_FLOOR](#) = "assets/sprites/floorstage1.png"
- constexpr auto [TEXTURE_STAGE1_CEILING](#) = "assets/sprites/ceillingstage1.png"
- constexpr auto [TEXTURE_SCORE_COUNTER_BG](#) = "assets/sprites/counter.png"
- constexpr auto [TEXTURE_SCORE_DIGIT_0](#) = "assets/sprites/counter0.png"
- constexpr auto [TEXTURE_SCORE_DIGIT_1](#) = "assets/sprites/counter1.png"
- constexpr auto [TEXTURE_SCORE_DIGIT_2](#) = "assets/sprites/counter2.png"
- constexpr auto [TEXTURE_SCORE_DIGIT_3](#) = "assets/sprites/counter3.png"
- constexpr auto [TEXTURE_SCORE_DIGIT_4](#) = "assets/sprites/counter4.png"
- constexpr auto [TEXTURE_SCORE_DIGIT_5](#) = "assets/sprites/counter5.png"
- constexpr auto [TEXTURE_SCORE_DIGIT_6](#) = "assets/sprites/counter6.png"
- constexpr auto [TEXTURE_SCORE_DIGIT_7](#) = "assets/sprites/counter7.png"
- constexpr auto [TEXTURE_SCORE_DIGIT_8](#) = "assets/sprites/counter8.png"
- constexpr auto [TEXTURE_SCORE_DIGIT_9](#) = "assets/sprites/counter9.png"
- constexpr auto [ICON_PATH](#) = "assets/icons/icon.png"

17.63.1 Variable Documentation

17.63.1.1 ICON_PATH

auto utl::Path::Texture::ICON_PATH = "assets/icons/icon.png" [inline], [constexpr]

Definition at line 139 of file [Common.hpp](#).

Referenced by [cli::Client::Client\(\)](#).

17.63.1.2 TEXTURE_BOSS

auto utl::Path::Texture::TEXTURE_BOSS = "assets/sprites/r-typesheet14.gif" [inline], [constexpr]

Definition at line 121 of file [Common.hpp](#).

Referenced by [gme::GameMulti::handleWorldStateUpdate\(\)](#), and [gme::GameMulti::preloadCommonTextures\(\)](#).

17.63.1.3 TEXTURE_ENEMY_ADVANCED

auto utl::Path::Texture::TEXTURE_ENEMY_ADVANCED = "assets/sprites/r-typesheet3.gif" [inline], [constexpr]

Definition at line 120 of file [Common.hpp](#).

Referenced by [gme::GameMulti::handleWorldStateUpdate\(\)](#), and [gme::GameMulti::preloadCommonTextures\(\)](#).

17.63.1.4 TEXTURE_ENEMY_BASIC

auto utl::Path::Texture::TEXTURE_ENEMY_BASIC = "assets/sprites/r-typesheet5.gif" [inline], [constexpr]

Definition at line 119 of file [Common.hpp](#).

Referenced by [gme::GameMulti::handleWorldStateUpdate\(\)](#), and [gme::GameMulti::preloadCommonTextures\(\)](#).

17.63.1.5 TEXTURE_ENEMY_EASY

auto utl::Path::Texture::TEXTURE_ENEMY_EASY = "assets/sprites/r-typesheet5.gif" [inline], [constexpr]

Definition at line 118 of file [Common.hpp](#).

Referenced by [gme::SpawnSystem::spawnEnemy\(\)](#), and [gme::SpawnSystem::spawnWave\(\)](#).

17.63.1.6 TEXTURE_ENEMY_PROJECTILE

auto utl::Path::Texture::TEXTURE_ENEMY_PROJECTILE = "assets/sprites/r-typesheet20.gif" [inline], [constexpr]

Definition at line 122 of file [Common.hpp](#).

Referenced by [gme::GameMulti::handleWorldStateUpdate\(\)](#), and [gme::GameMulti::preloadCommonTextures\(\)](#).

17.63.1.7 TEXTURE_EXPLOSION

auto utl::Path::Texture::TEXTURE_EXPLOSION = "assets/sprites/r-typesheet44.gif" [inline], [constexpr]

Definition at line 123 of file [Common.hpp](#).

Referenced by [gme::CollisionSystem::createExplosion\(\)](#), [gme::GameMulti::handleWorldStateUpdate\(\)](#), and [gme::GameMulti::preloadCommonTextures\(\)](#).

17.63.1.8 TEXTURE_PLAYER

auto utl::Path::Texture::TEXTURE_PLAYER = "assets/sprites/r-typesheet42.gif" [inline], [constexpr]

Definition at line 114 of file [Common.hpp](#).

Referenced by [gme::Game::createPlayer\(\)](#), [gme::GameSolo::createPlayer\(\)](#), [gme::PlayerControllerMulti::createPlayer\(\)](#), [gme::GameMulti::handleWorldStateUpdate\(\)](#), [gme::GameMulti::preloadCommonTextures\(\)](#), and [gme::GameSolo::preloadCommonTextures\(\)](#).

17.63.1.9 TEXTURE_SCORE_COUNTER_BG

auto utl::Path::Texture::TEXTURE_SCORE_COUNTER_BG = "assets/sprites/counter.png" [inline], [constexpr]

Definition at line 127 of file [Common.hpp](#).

Referenced by [gme::HUDSystem::createScoreHUD\(\)](#).

17.63.1.10 TEXTURE_SCORE_DIGIT_0

auto utl::Path::Texture::TEXTURE_SCORE_DIGIT_0 = "assets/sprites/counter0.png" [inline], [constexpr]

Definition at line 128 of file [Common.hpp](#).

Referenced by [gme::HUDSystem::createScoreHUD\(\)](#), and [gme::HUDSystem::updateScore\(\)](#).

17.63.1.11 TEXTURE_SCORE_DIGIT_1

auto utl::Path::Texture::TEXTURE_SCORE_DIGIT_1 = "assets/sprites/counter1.png" [inline], [constexpr]

Definition at line 129 of file [Common.hpp](#).

Referenced by [gme::HUDSystem::updateScore\(\)](#).

17.63.1.12 TEXTURE_SCORE_DIGIT_2

auto utl::Path::Texture::TEXTURE_SCORE_DIGIT_2 = "assets/sprites/counter2.png" [inline], [constexpr]

Definition at line 130 of file [Common.hpp](#).

Referenced by [gme::HUDSystem::updateScore\(\)](#).

17.63.1.13 TEXTURE_SCORE_DIGIT_3

auto utl::Path::Texture::TEXTURE_SCORE_DIGIT_3 = "assets/sprites/counter3.png" [inline], [constexpr]

Definition at line 131 of file [Common.hpp](#).

Referenced by [gme::HUDSystem::updateScore\(\)](#).

17.63.1.14 TEXTURE_SCORE_DIGIT_4

auto utl::Path::Texture::TEXTURE_SCORE_DIGIT_4 = "assets/sprites/counter4.png" [inline], [constexpr]

Definition at line 132 of file [Common.hpp](#).

Referenced by [gme::HUDSystem::updateScore\(\)](#).

17.63.1.15 TEXTURE_SCORE_DIGIT_5

auto utl::Path::Texture::TEXTURE_SCORE_DIGIT_5 = "assets/sprites/counter5.png" [inline], [constexpr]

Definition at line 133 of file [Common.hpp](#).

Referenced by [gme::HUDSystem::updateScore\(\)](#).

17.63.1.16 TEXTURE_SCORE_DIGIT_6

auto utl::Path::Texture::TEXTURE_SCORE_DIGIT_6 = "assets/sprites/counter6.png" [inline], [constexpr]

Definition at line 134 of file [Common.hpp](#).

Referenced by [gme::HUDSystem::updateScore\(\)](#).

17.63.1.17 TEXTURE_SCORE_DIGIT_7

auto utl::Path::Texture::TEXTURE_SCORE_DIGIT_7 = "assets/sprites/counter7.png" [inline], [constexpr]

Definition at line 135 of file [Common.hpp](#).

Referenced by [gme::HUDSystem::updateScore\(\)](#).

17.63.1.18 TEXTURE_SCORE_DIGIT_8

auto utl::Path::Texture::TEXTURE_SCORE_DIGIT_8 = "assets/sprites/counter8.png" [inline], [constexpr]

Definition at line 136 of file [Common.hpp](#).

Referenced by [gme::HUDSystem::updateScore\(\)](#).

17.63.1.19 TEXTURE_SCORE_DIGIT_9

auto utl::Path::Texture::TEXTURE_SCORE_DIGIT_9 = "assets/sprites/counter9.png" [inline], [constexpr]

Definition at line 137 of file [Common.hpp](#).

Referenced by [gme::HUDSystem::updateScore\(\)](#).

17.63.1.20 TEXTURE_SHOOT

auto utl::Path::Texture::TEXTURE_SHOOT = "assets/sprites/shoot.gif" [inline], [constexpr]

Definition at line 115 of file [Common.hpp](#).

Referenced by [gme::ProjectileManager::createProjectile\(\)](#), [gme::GameMulti::handleWorldStateUpdate\(\)](#), [gme::GameMulti::preloadCommonTextures\(\)](#), and [gme::GameSolo::preloadCommonTextures\(\)](#).

17.63.1.21 TEXTURE_SHOOT_CHARGED

auto utl::Path::Texture::TEXTURE_SHOOT_CHARGED = "assets/sprites/shootcharged.gif" [inline], [constexpr]

Definition at line 116 of file [Common.hpp](#).

Referenced by [gme::ProjectileManager::createProjectile\(\)](#), [gme::GameMulti::handleWorldStateUpdate\(\)](#), [gme::GameMulti::preloadCommonTextures\(\)](#), and [gme::GameSolo::preloadCommonTextures\(\)](#).

17.63.1.22 TEXTURE_SHOOT_LOADING

auto utl::Path::Texture::TEXTURE_SHOOT_LOADING = "assets/sprites/shootchargedloading.gif" [inline], [constexpr]

Definition at line 117 of file [Common.hpp](#).

Referenced by [gme::WeaponSystem::showLoadingAnimation\(\)](#).

17.63.1.23 TEXTURE_STAGE1_CEILING

auto utl::Path::Texture::TEXTURE_STAGE1_CEILING = "assets/sprites/ceillingstage1.png" [inline], [constexpr]

Definition at line 125 of file [Common.hpp](#).

Referenced by [gme::GameSolo::preloadCommonTextures\(\)](#), and [gme::StageManager::spawnStage\(\)](#).

17.63.1.24 TEXTURE_STAGE1_FLOOR

auto utl::Path::Texture::TEXTURE_STAGE1_FLOOR = "assets/sprites/floorstage1.png" [inline], [constexpr]

Definition at line 124 of file [Common.hpp](#).

Referenced by [gme::GameSolo::preloadCommonTextures\(\)](#), and [gme::StageManager::spawnStage\(\)](#).

17.64 utl::srv Namespace Reference

Classes

- struct [AppConfig](#)

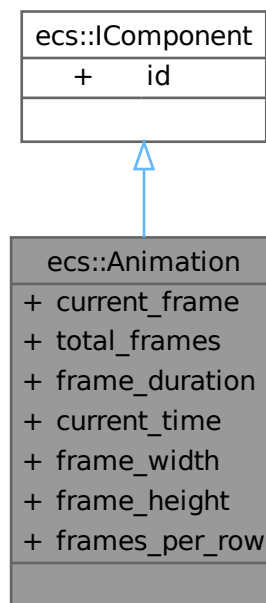
Chapter 18

Class Documentation

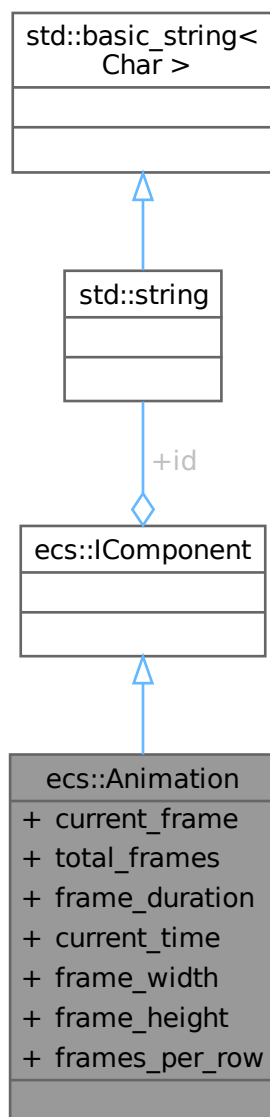
18.1 ecs::Animation Struct Reference

#include <Component.hpp>

Inheritance diagram for ecs::Animation:



Collaboration diagram for `ecs::Animation`:



Public Attributes

- `int current_frame {}`
- `int total_frames {}`
- `float frame_duration {}`
- `float current_time {}`
- `int frame_width {}`
- `int frame_height {}`
- `int frames_per_row {}`

Public Attributes inherited from `ecs::IComponent`

- `std::string id`

18.1.1 Detailed Description

Definition at line 72 of file [Component.hpp](#).

18.1.2 Member Data Documentation

18.1.2.1 current_frame

```
int ecs::Animation::current_frame {}
```

Definition at line 74 of file [Component.hpp](#).

18.1.2.2 current_time

```
float ecs::Animation::current_time {}
```

Definition at line 77 of file [Component.hpp](#).

18.1.2.3 frame_duration

```
float ecs::Animation::frame_duration {}
```

Definition at line 76 of file [Component.hpp](#).

18.1.2.4 frame_height

```
int ecs::Animation::frame_height {}
```

Definition at line 79 of file [Component.hpp](#).

18.1.2.5 frame_width

```
int ecs::Animation::frame_width {}
```

Definition at line 78 of file [Component.hpp](#).

18.1.2.6 frames_per_row

```
int ecs::Animation::frames_per_row {}
```

Definition at line 80 of file [Component.hpp](#).

18.1.2.7 total_frames

```
int ecs::Animation::total_frames {}
```

Definition at line 75 of file [Component.hpp](#).

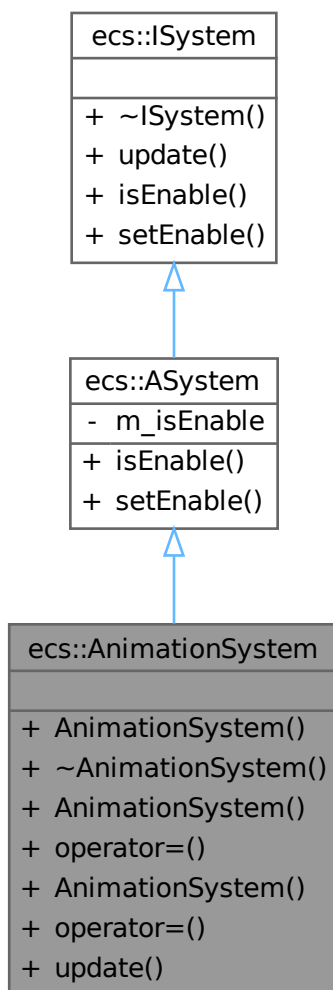
The documentation for this struct was generated from the following file:

- [/home/masina/Projects/Epitech/rtype/modules/ECS/include/ECS/Component.hpp](#)

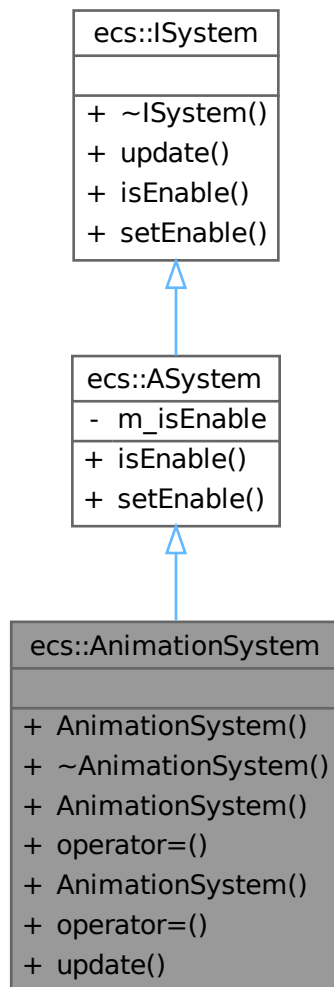
18.2 ecs::AnimationSystem Class Reference

```
#include <Animation.hpp>
```

Inheritance diagram for ecs::AnimationSystem:



Collaboration diagram for ecs::AnimationSystem:



Public Member Functions

- [AnimationSystem](#) (const std::shared_ptr< [eng::IRenderer](#) > &)
- [~AnimationSystem](#) () override=default
- [AnimationSystem](#) (const [AnimationSystem](#) &)=delete
- [AnimationSystem](#) & operator= (const [AnimationSystem](#) &)=delete
- [AnimationSystem](#) ([AnimationSystem](#) &&)=delete
- [AnimationSystem](#) & operator= ([AnimationSystem](#) &&)=delete
- void [update](#) ([Registry](#) ®istry, const float dt) override

Public Member Functions inherited from [ecs::ASystem](#)

- bool [isEnabled](#) () override
- void [setEnable](#) (const bool enable) override

Public Member Functions inherited from [ecs::ISystem](#)

- virtual [~ISystem](#) ()=default

18.2.1 Detailed Description

Definition at line 17 of file [Animation.hpp](#).

18.2.2 Constructor & Destructor Documentation

18.2.2.1 AnimationSystem() [1/3]

```
ecs::AnimationSystem::AnimationSystem (
    const std::shared_ptr< eng::IRenderer > & ) [inline], [explicit]
```

Definition at line 20 of file [Animation.hpp](#).

18.2.2.2 ~AnimationSystem()

```
ecs::AnimationSystem::~AnimationSystem () [override], [default]
```

18.2.2.3 AnimationSystem() [2/3]

```
ecs::AnimationSystem::AnimationSystem (
    const AnimationSystem & ) [delete]
```

18.2.2.4 AnimationSystem() [3/3]

```
ecs::AnimationSystem::AnimationSystem (
    AnimationSystem && ) [delete]
```

18.2.3 Member Function Documentation

18.2.3.1 operator=() [1/2]

```
AnimationSystem & ecs::AnimationSystem::operator= (
    AnimationSystem && ) [delete]
```

18.2.3.2 operator=() [2/2]

```
AnimationSystem & ecs::AnimationSystem::operator= (
    const AnimationSystem & ) [delete]
```

18.2.3.3 update()

```
void ecs::AnimationSystem::update (
    Registry & registry,
    const float dt) [inline], [override], [virtual]
```

Implements [ecs::ISystem](#).

Definition at line 28 of file [Animation.hpp](#).

References [ecs::Registry::getAll\(\)](#), and [ecs::Registry::getComponent\(\)](#).

Here is the call graph for this function:



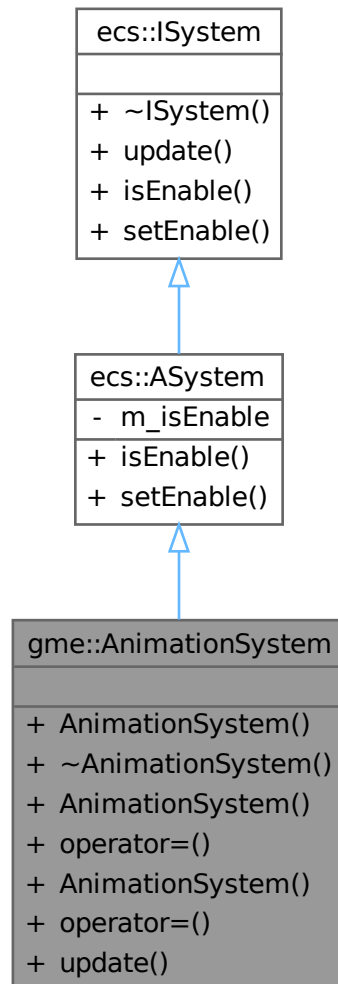
The documentation for this class was generated from the following file:

- [/home/masina/Projects/Epitech/rtype/modules/ECS/include/ECS/Systems/Animation.hpp](#)

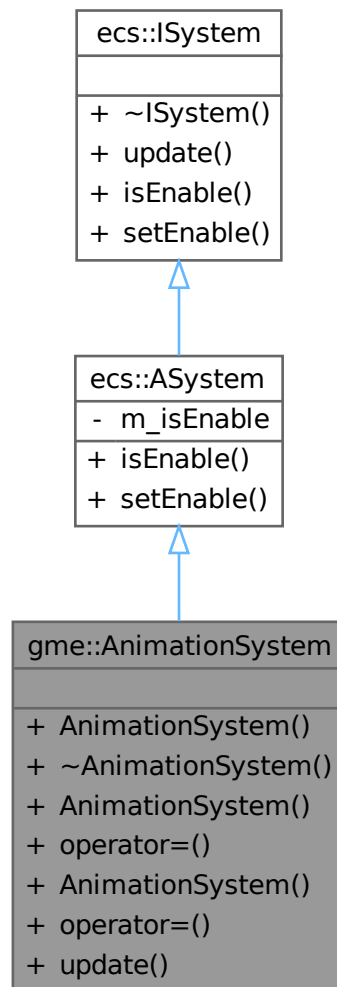
18.3 gme::AnimationSystem Class Reference

#include <Animation.hpp>

Inheritance diagram for gme::AnimationSystem:



Collaboration diagram for gme::AnimationSystem:



Public Member Functions

- [AnimationSystem](#) (const std::shared_ptr< [eng::IRenderer](#) > &)
- [~AnimationSystem](#) () override=default
- [AnimationSystem](#) (const [AnimationSystem](#) &)=delete
- [AnimationSystem](#) & operator= (const [AnimationSystem](#) &)=delete
- [AnimationSystem](#) ([AnimationSystem](#) &&)=delete
- [AnimationSystem](#) & operator= (const [AnimationSystem](#) &&)=delete
- void [update](#) ([ecs::Registry](#) ®istry, float dt) override

Public Member Functions inherited from [ecs::ASystem](#)

- bool [isEnabled](#) () override
- void [setEnable](#) (const bool enable) override

Public Member Functions inherited from [ecs::ISystem](#)

- virtual [~ISystem](#) ()=default

18.3.1 Detailed Description

Definition at line 17 of file [Animation.hpp](#).

18.3.2 Constructor & Destructor Documentation

18.3.2.1 AnimationSystem() [1/3]

```
gme::AnimationSystem::AnimationSystem (
    const std::shared_ptr< eng::IRenderer > & ) [inline], [explicit]
```

Definition at line 20 of file [Animation.hpp](#).

18.3.2.2 ~AnimationSystem()

```
gme::AnimationSystem::~AnimationSystem () [override], [default]
```

18.3.2.3 AnimationSystem() [2/3]

```
gme::AnimationSystem::AnimationSystem (
    const AnimationSystem & ) [delete]
```

18.3.2.4 AnimationSystem() [3/3]

```
gme::AnimationSystem::AnimationSystem (
    AnimationSystem && ) [delete]
```

18.3.3 Member Function Documentation

18.3.3.1 operator=() [1/2]

```
AnimationSystem & gme::AnimationSystem::operator= (
    const AnimationSystem && ) [delete]
```

18.3.3.2 operator=() [2/2]

```
AnimationSystem & gme::AnimationSystem::operator= (
    const AnimationSystem & ) [delete]
```

18.3.3.3 update()

```
void gme::AnimationSystem::update (
    ecs::Registry & registry,
    float dt) [inline], [override], [virtual]
```

Implements [ecs::ISystem](#).

Definition at line 28 of file [Animation.hpp](#).

References [ecs::Registry::getAll\(\)](#), and [ecs::Registry::getComponent\(\)](#).

Here is the call graph for this function:



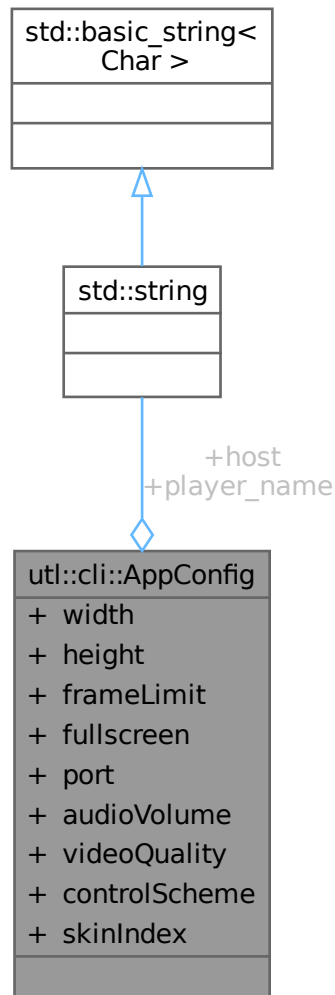
The documentation for this class was generated from the following file:

- `/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/include/RType↔ ClientMulti/Systems/Animation.hpp`

18.4 utl::cli::AppConfig Struct Reference

#include <AppConfigClient.hpp>

Collaboration diagram for utl::cli::AppConfig:



Public Attributes

- unsigned int **width** = `Config::Window::DEFAULT_WINDOW_WIDTH`
- unsigned int **height** = `Config::Window::DEFAULT_WINDOW_HEIGHT`
- unsigned int **frameLimit** = `Config::Window::DEFAULT_WINDOW_FRAME_LIMIT`
- bool **fullscreen** = `Config::Window::DEFAULT_WINDOW_FULLSCREEN`
- std::string **host** = `Config::Network::DEFAULT_NETWORK_HOST`
- unsigned int **port** = `Config::Network::DEFAULT_NETWORK_PORT`
- std::string **player_name** = `Config::Player::DEFAULT_PLAYER_NAME`
- float **audioVolume** = `Config::Audio::DEFAULT_AUDIO_VOLUME`
- int **videoQuality** = `Config::Game::DEFAULT_VIDEO_QUALITY`
- int **controlScheme** = `Config::Game::DEFAULT_CONTROL_SCHEME`
- int **skinIndex** = `Config::Game::DEFAULT_SKIN_INDEX`

18.4.1 Detailed Description

Definition at line 15 of file [AppConfigClient.hpp](#).

18.4.2 Member Data Documentation

18.4.2.1 audioVolume

float utl::cli::AppConfig::audioVolume = [Config::Audio::DEFAULT_AUDIO_VOLUME](#)

Definition at line 24 of file [AppConfigClient.hpp](#).

Referenced by [cli::Client::setupConfig\(\)](#), and [gme::RTypeClientSolo::setupScenes\(\)](#).

18.4.2.2 controlScheme

int utl::cli::AppConfig::controlScheme = [Config::Game::DEFAULT_CONTROL_SCHEME](#)

Definition at line 26 of file [AppConfigClient.hpp](#).

Referenced by [cli::Client::setupConfig\(\)](#).

18.4.2.3 frameLimit

unsigned int utl::cli::AppConfig::frameLimit = [Config::Window::DEFAULT_WINDOW_FRAME_LIMIT](#)

Definition at line 19 of file [AppConfigClient.hpp](#).

Referenced by [cli::Client::setupConfig\(\)](#).

18.4.2.4 fullscreen

bool utl::cli::AppConfig::fullscreen = [Config::Window::DEFAULT_WINDOW_FULLSCREEN](#)

Definition at line 20 of file [AppConfigClient.hpp](#).

Referenced by [cli::Client::setupConfig\(\)](#).

18.4.2.5 height

unsigned int utl::cli::AppConfig::height = [Config::Window::DEFAULT_WINDOW_HEIGHT](#)

Definition at line 18 of file [AppConfigClient.hpp](#).

Referenced by [cli::Client::setupConfig\(\)](#).

18.4.2.6 host

std::string utl::cli::AppConfig::host = [Config::Network::DEFAULT_NETWORK_HOST](#)

Definition at line 21 of file [AppConfigClient.hpp](#).

Referenced by [cli::Client::setupConfig\(\)](#).

18.4.2.7 player_name

std::string utl::cli::AppConfig::player_name = [Config::Player::DEFAULT_PLAYER_NAME](#)

Definition at line 23 of file [AppConfigClient.hpp](#).

Referenced by [cli::Client::setupConfig\(\)](#).

18.4.2.8 port

unsigned int utl::cli::AppConfig::port = [Config::Network::DEFAULT_NETWORK_PORT](#)

Definition at line 22 of file [AppConfigClient.hpp](#).

Referenced by [cli::Client::setupConfig\(\)](#).

18.4.2.9 skinIndex

int utl::cli::AppConfig::skinIndex = [Config::Game::DEFAULT_SKIN_INDEX](#)

Definition at line 27 of file [AppConfigClient.hpp](#).

Referenced by [cli::Client::setupConfig\(\)](#), and [gme::RTypeClientSolo::setupScenes\(\)](#).

18.4.2.10 videoQuality

int utl::cli::AppConfig::videoQuality = Config::Game::DEFAULT_VIDEO_QUALITY

Definition at line 25 of file AppConfigClient.hpp.

Referenced by cli::Client::setupConfig().

18.4.2.11 width

unsigned int utl::cli::AppConfig::width = Config::Window::DEFAULT_WINDOW_WIDTH

Definition at line 17 of file AppConfigClient.hpp.

Referenced by cli::Client::setupConfig().

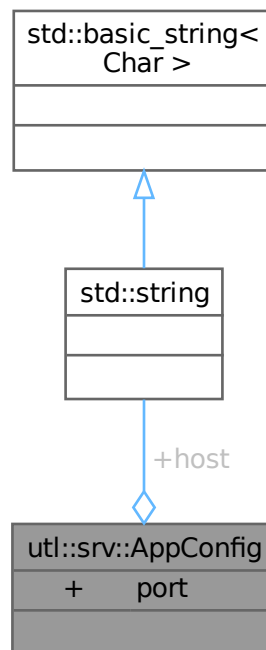
The documentation for this struct was generated from the following file:

- /home/masina/Projects/Epitech/rtype/modules/Utils/include/Utils/AppConfigClient.hpp

18.5 utl::srv::AppConfig Struct Reference

#include <AppConfigServer.hpp>

Collaboration diagram for utl::srv::AppConfig:



Public Attributes

- std::string [host](#)
- uint16_t [port](#)

18.5.1 Detailed Description

Definition at line 15 of file AppConfigServer.hpp.

18.5.2 Member Data Documentation

18.5.2.1 host

std::string utl::srv::AppConfig::host

Definition at line 17 of file [AppConfigServer.hpp](#).

Referenced by [srv::Server::setupConfig\(\)](#).

18.5.2.2 port

uint16_t utl::srv::AppConfig::port

Definition at line 18 of file [AppConfigServer.hpp](#).

Referenced by [srv::Server::setupConfig\(\)](#).

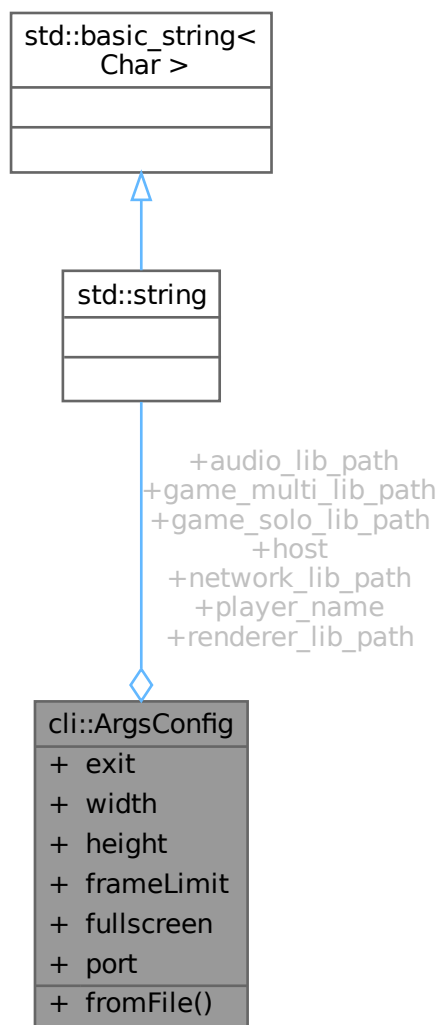
The documentation for this struct was generated from the following file:

- [/home/masina/Projects/Epitech/rtype/modules/Utils/include/Utils/AppConfigServer.hpp](#)

18.6 cli::ArgsConfig Struct Reference

#include <ArgsHandler.hpp>

Collaboration diagram for cli::ArgsConfig:



Static Public Member Functions

- static [ArgsConfig fromFile](#) (const std::string &path)

Public Attributes

- bool [exit](#) = false
- unsigned int [width](#) = utl::Config::Window::DEFAULT_WINDOW_WIDTH
- unsigned int [height](#) = utl::Config::Window::DEFAULT_WINDOW_HEIGHT
- unsigned int [frameLimit](#) = utl::Config::Window::DEFAULT_WINDOW_FRAME_LIMIT
- bool [fullscreen](#) = utl::Config::Window::DEFAULT_WINDOW_FULLSCREEN
- std::string [host](#) = utl::Config::Network::DEFAULT_NETWORK_HOST
- unsigned int [port](#) = utl::Config::Network::DEFAULT_NETWORK_PORT
- std::string [player_name](#) = utl::Config::Player::DEFAULT_PLAYER_NAME
- std::string [audio_lib_path](#) = utl::Path::Plugin::PLUGIN_AUDIO_SFML.string()

- `std::string network_lib_path = utl::Path::Plugin::PLUGIN_NETWORK_ASIO_CLIENT.string()`
- `std::string renderer_lib_path = utl::Path::Plugin::PLUGIN_RENDERER_SFML.string()`
- `std::string game_solo_lib_path = utl::Path::Plugin::PLUGIN_GAME_SOLO.string()`
- `std::string game_multi_lib_path = utl::Path::Plugin::PLUGIN_GAME_MULTI.string()`

18.6.1 Detailed Description

Definition at line 18 of file [ArgsHandler.hpp](#).

18.6.2 Member Function Documentation

18.6.2.1 fromFile()

`cli::ArgsConfig cli::ArgsConfig::fromFile (const std::string & path) [static]`

Definition at line 26 of file [argsHandler.cpp](#).

References [audio_lib_path](#), [frameLimit](#), [fullscreen](#), [game_multi_lib_path](#), [game_solo_lib_path](#), [height](#), [host](#), [network_lib_path](#), [player_name](#), [port](#), [renderer_lib_path](#), and [width](#).

Referenced by [cli::ArgsHandler::ParseArgs\(\)](#).

Here is the caller graph for this function:



18.6.3 Member Data Documentation

18.6.3.1 audio_lib_path

`std::string cli::ArgsConfig::audio_lib_path = utl::Path::Plugin::PLUGIN_AUDIO_SFML.string()`

Definition at line 28 of file [ArgsHandler.hpp](#).

Referenced by [cli::Client::Client\(\)](#), and [fromFile\(\)](#).

18.6.3.2 exit

`bool cli::ArgsConfig::exit = false`

Definition at line 20 of file [ArgsHandler.hpp](#).

Referenced by [main\(\)](#).

18.6.3.3 frameLimit

`unsigned int cli::ArgsConfig::frameLimit = utl::Config::Window::DEFAULT_WINDOW_FRAME_LIMIT`

Definition at line 23 of file [ArgsHandler.hpp](#).

Referenced by [fromFile\(\)](#), and [cli::Client::setupConfig\(\)](#).

18.6.3.4 fullscreen

`bool cli::ArgsConfig::fullscreen = utl::Config::Window::DEFAULT_WINDOW_FULLSCREEN`

Definition at line 24 of file [ArgsHandler.hpp](#).

Referenced by [fromFile\(\)](#), and [cli::Client::setupConfig\(\)](#).

18.6.3.5 game_multi_lib_path

`std::string cli::ArgsConfig::game_multi_lib_path = utl::Path::Plugin::PLUGIN_GAME_MULTI.string()`

Definition at line 32 of file [ArgsHandler.hpp](#).

Referenced by [cli::Client::Client\(\)](#), and [fromFile\(\)](#).

18.6.3.6 game_solo_lib_path

`std::string cli::ArgsConfig::game_solo_lib_path = utl::Path::Plugin::PLUGIN_GAME_SOLO.string()`

Definition at line 31 of file [ArgsHandler.hpp](#).

Referenced by [cli::Client::Client\(\)](#), and [fromFile\(\)](#).

18.6.3.7 height

`unsigned int cli::ArgsConfig::height = utl::Config::Window::DEFAULT_WINDOW_HEIGHT`

Definition at line 22 of file [ArgsHandler.hpp](#).

Referenced by [fromFile\(\)](#), and [cli::Client::setupConfig\(\)](#).

18.6.3.8 host

`std::string cli::ArgsConfig::host = utl::Config::Network::DEFAULT_NETWORK_HOST`

Definition at line 25 of file [ArgsHandler.hpp](#).

Referenced by [fromFile\(\)](#), and [cli::Client::setupConfig\(\)](#).

18.6.3.9 network_lib_path

`std::string cli::ArgsConfig::network_lib_path = utl::Path::Plugin::PLUGIN_NETWORK_ASIO_CLIENT.string()`

Definition at line 29 of file [ArgsHandler.hpp](#).

Referenced by [cli::Client::Client\(\)](#), and [fromFile\(\)](#).

18.6.3.10 player_name

`std::string cli::ArgsConfig::player_name = utl::Config::Player::DEFAULT_PLAYER_NAME`

Definition at line 27 of file [ArgsHandler.hpp](#).

Referenced by [fromFile\(\)](#), and [cli::Client::setupConfig\(\)](#).

18.6.3.11 port

`unsigned int cli::ArgsConfig::port = utl::Config::Network::DEFAULT_NETWORK_PORT`

Definition at line 26 of file [ArgsHandler.hpp](#).

Referenced by [fromFile\(\)](#), and [cli::Client::setupConfig\(\)](#).

18.6.3.12 renderer_lib_path

`std::string cli::ArgsConfig::renderer_lib_path = utl::Path::Plugin::PLUGIN_RENDERER_SFML.string()`

Definition at line 30 of file [ArgsHandler.hpp](#).

Referenced by [cli::Client::Client\(\)](#), and [fromFile\(\)](#).

18.6.3.13 width

`unsigned int cli::ArgsConfig::width = utl::Config::Window::DEFAULT_WINDOW_WIDTH`

Definition at line 21 of file [ArgsHandler.hpp](#).

Referenced by [fromFile\(\)](#), and [cli::Client::setupConfig\(\)](#).

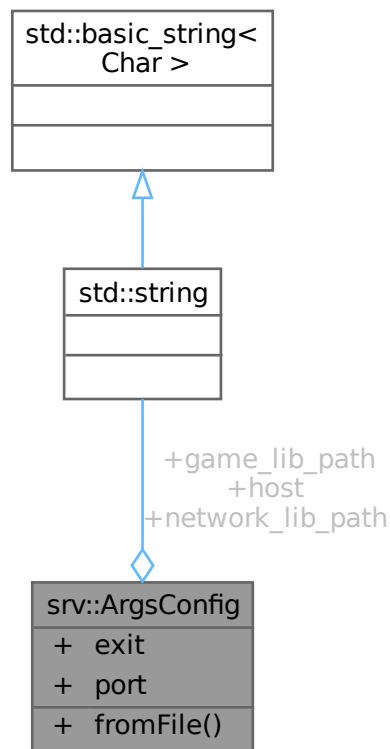
The documentation for this struct was generated from the following files:

- [/home/masina/Projects/Epitech/rtype/client/include/Client/ArgsHandler.hpp](#)
- [/home/masina/Projects/Epitech/rtype/client/src/argsHandler.cpp](#)

18.7 srv::ArgsConfig Struct Reference

`#include <ArgsHandler.hpp>`

Collaboration diagram for `srv::ArgsConfig`:



Static Public Member Functions

- static [ArgsConfig fromFile](#) (const std::string &path)

Public Attributes

- bool [exit](#) = false
- std::string [host](#) = [Config::Network::DEFAULT_NETWORK_HOST](#)
- uint16_t [port](#) = [Config::Network::DEFAULT_NETWORK_PORT](#)
- std::string [network_lib_path](#) = [Path::Plugin::PLUGINS_NETWORK_ASIO_SERVER.string\(\)](#)
- std::string [game_lib_path](#) = [Path::Plugin::PLUGINS_GAME_RTYPE_SERVER.string\(\)](#)

18.7.1 Detailed Description

Definition at line 20 of file [ArgsHandler.hpp](#).

18.7.2 Member Function Documentation

18.7.2.1 fromFile()

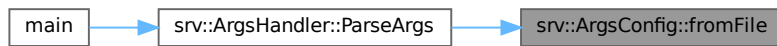
[srv::ArgsConfig](#) [srv::ArgsConfig::fromFile](#) (
const std::string & path) [static]

Definition at line 26 of file [argsHandler.cpp](#).

References [host](#), [network_lib_path](#), and [port](#).

Referenced by [srv::ArgsHandler::ParseArgs\(\)](#).

Here is the caller graph for this function:



18.7.3 Member Data Documentation

18.7.3.1 exit

`bool srv::ArgsConfig::exit = false`

Definition at line 22 of file [ArgsHandler.hpp](#).

Referenced by [main\(\)](#).

18.7.3.2 game_lib_path

`std::string srv::ArgsConfig::game_lib_path = Path::Plugin::PLUGINS_GAME_RTYPE_SERVER.string()`

Definition at line 26 of file [ArgsHandler.hpp](#).

18.7.3.3 host

`std::string srv::ArgsConfig::host = Config::Network::DEFAULT_NETWORK_HOST`

Definition at line 23 of file [ArgsHandler.hpp](#).

Referenced by [fromFile\(\)](#), [srv::Server::Server\(\)](#), and [srv::Server::setupConfig\(\)](#).

18.7.3.4 network_lib_path

`std::string srv::ArgsConfig::network_lib_path = Path::Plugin::PLUGINS_NETWORK_ASIO_SERVER.string()`

Definition at line 25 of file [ArgsHandler.hpp](#).

Referenced by [fromFile\(\)](#).

18.7.3.5 port

`uint16_t srv::ArgsConfig::port = Config::Network::DEFAULT_NETWORK_PORT`

Definition at line 24 of file [ArgsHandler.hpp](#).

Referenced by [fromFile\(\)](#), [srv::Server::Server\(\)](#), and [srv::Server::setupConfig\(\)](#).

The documentation for this struct was generated from the following files:

- [/home/masina/Projects/Epitech/rtype/server/include/Server/ArgsHandler.hpp](#)
- [/home/masina/Projects/Epitech/rtype/server/src/argsHandler.cpp](#)

18.8 cli::ArgsHandler Class Reference

Class to handle command line arguments.

`#include <ArgsHandler.hpp>`

Collaboration diagram for cli::ArgsHandler:

cli::ArgsHandler
+ ArgsHandler() + ~ArgsHandler() + ArgsHandler() + operator=() + ArgsHandler() + operator=() + ParseArgs() + ParseEnv()

Public Member Functions

- [ArgsHandler](#) ()=default
- [~ArgsHandler](#) ()=default
- [ArgsHandler](#) (const [ArgsHandler](#) &)=delete
- [ArgsHandler](#) & [operator=](#) (const [ArgsHandler](#) &)=delete
- [ArgsHandler](#) ([ArgsHandler](#) &&)=delete
- [ArgsHandler](#) & [operator=](#) ([ArgsHandler](#) &&)=delete

Static Public Member Functions

- static [ArgsConfig ParseArgs](#) (int argc, const char *const argv[])
- static [EnvConfig ParseEnv](#) (const char *const env[])

18.8.1 Detailed Description

Class to handle command line arguments.

Definition at line 45 of file [ArgsHandler.hpp](#).

18.8.2 Constructor & Destructor Documentation

18.8.2.1 ArgsHandler() [1/3]

cli::ArgsHandler::ArgsHandler () [default]

18.8.2.2 ~ArgsHandler()

cli::ArgsHandler::~~ArgsHandler () [default]

18.8.2.3 ArgsHandler() [2/3]

cli::ArgsHandler::ArgsHandler (
const [ArgsHandler](#) &) [delete]

18.8.2.4 ArgsHandler() [3/3]

```
cli::ArgsHandler::ArgsHandler (
    ArgsHandler && ) [delete]
```

18.8.3 Member Function Documentation

18.8.3.1 operator=() [1/2]

```
ArgsHandler & cli::ArgsHandler::operator= (
    ArgsHandler && ) [delete]
```

18.8.3.2 operator=() [2/2]

```
ArgsHandler & cli::ArgsHandler::operator= (
    const ArgsHandler & ) [delete]
```

18.8.3.3 ParseArgs()

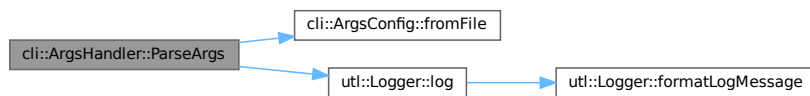
```
cli::ArgsConfig cli::ArgsHandler::ParseArgs (
    int argc,
    const char *const argv[]) [static]
```

Definition at line 95 of file [argsHandler.cpp](#).

References [cli::ArgsConfig::fromFile\(\)](#), [HELP_MESSAGE](#), [utl::INFO](#), [utl::Logger::log\(\)](#), and [VERSION_MESSAGE](#).

Referenced by [main\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.8.3.4 ParseEnv()

```
cli::EnvConfig cli::ArgsHandler::ParseEnv (
    const char *const env[]) [static]
```

Definition at line 151 of file [argsHandler.cpp](#).

Referenced by [main\(\)](#).

Here is the caller graph for this function:



The documentation for this class was generated from the following files:

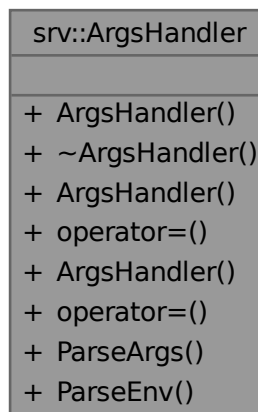
- [/home/masina/Projects/Epitech/rtype/client/include/Client/ArgsHandler.hpp](#)
- [/home/masina/Projects/Epitech/rtype/client/src/argsHandler.cpp](#)

18.9 srv::ArgsHandler Class Reference

Class to handle command line arguments.

#include <ArgsHandler.hpp>

Collaboration diagram for srv::ArgsHandler:



Public Member Functions

- [ArgsHandler](#) ()=default
- [~ArgsHandler](#) ()=default
- [ArgsHandler](#) (const [ArgsHandler](#) &)=delete
- [ArgsHandler](#) & operator= (const [ArgsHandler](#) &)=delete
- [ArgsHandler](#) ([ArgsHandler](#) &&)=delete
- [ArgsHandler](#) & operator= ([ArgsHandler](#) &&)=delete

Static Public Member Functions

- static [ArgsConfig ParseArgs](#) (int argc, const char *const argv[])
- static [EnvConfig ParseEnv](#) (const char *const env[])

18.9.1 Detailed Description

Class to handle command line arguments.
Definition at line 39 of file [ArgsHandler.hpp](#).

18.9.2 Constructor & Destructor Documentation

18.9.2.1 ArgsHandler() [1/3]

srv::ArgsHandler::ArgsHandler () [default]

18.9.2.2 ~ArgsHandler()

srv::ArgsHandler::~~ArgsHandler () [default]

18.9.2.3 ArgsHandler() [2/3]

srv::ArgsHandler::ArgsHandler (
const [ArgsHandler](#) &) [delete]

18.9.2.4 ArgsHandler() [3/3]

srv::ArgsHandler::ArgsHandler (
[ArgsHandler](#) &&) [delete]

18.9.3 Member Function Documentation

18.9.3.1 operator=() [1/2]

[ArgsHandler](#) & srv::ArgsHandler::operator= (
[ArgsHandler](#) &&) [delete]

18.9.3.2 operator=() [2/2]

[ArgsHandler](#) & srv::ArgsHandler::operator= (
const [ArgsHandler](#) &) [delete]

18.9.3.3 ParseArgs()

[srv::ArgsConfig](#) srv::ArgsHandler::ParseArgs (
int argc,
const char *const argv[]) [static]

Definition at line 53 of file [argsHandler.cpp](#).

References [srv::ArgsConfig::fromFile\(\)](#), [HELP_MESSAGE](#), [utl::INFO](#), [utl::Logger::log\(\)](#), and [VERSION_MESSAGE](#).

Referenced by [main\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.9.3.4 ParseEnv()

```
srv::EnvConfig srv::ArgsHandler::ParseEnv (  
    const char *const env[]) [static]
```

Definition at line 106 of file [argsHandler.cpp](#).

Referenced by [main\(\)](#).

Here is the caller graph for this function:



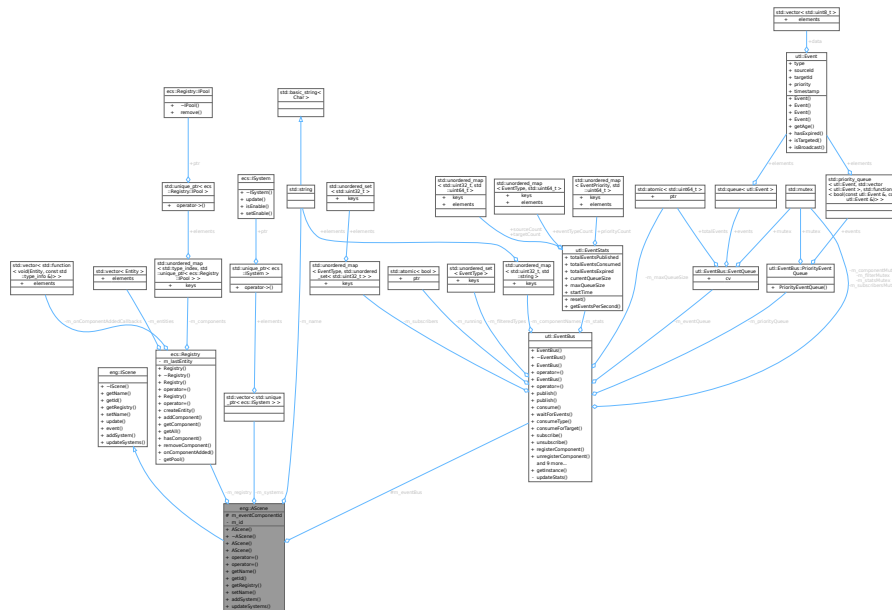
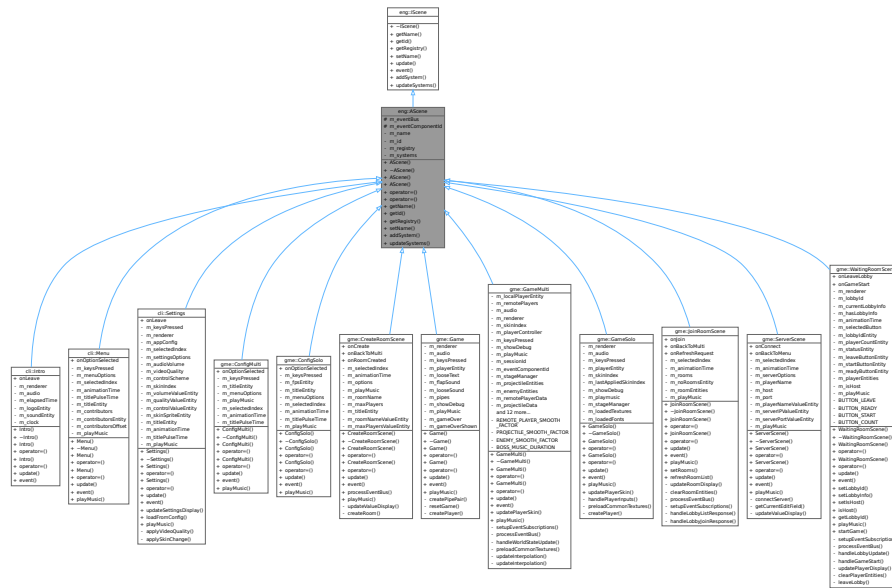
The documentation for this class was generated from the following files:

- [/home/masina/Projects/Epitech/rtype/server/include/Server/ArgsHandler.hpp](#)
- [/home/masina/Projects/Epitech/rtype/server/src/argsHandler.cpp](#)

18.10 eng::AScene Class Reference

Abstract class for scene.

```
#include <IScene.hpp>
```



- `AScene` (const `id` assignedId)
- `~AScene` () override=default
- `AScene` (const `AScene` &other)=delete
- `AScene` (`AScene` &&other)=delete
- `AScene` & `operator=` (const `AScene` &other)=delete
- `AScene` & `operator=` (`AScene` &&other)=delete
- `std::string` & `getName` () override
- `id` `getId` () const override
- `ecs::Registry` & `getRegistry` () override

- void [setName](#) (const std::string &newName) override
- void [addSystem](#) (std::unique_ptr< [ecs::ISystem](#) > system) override
- void [updateSystems](#) (const float dt) override

Public Member Functions inherited from [eng::IScene](#)

- virtual [~IScene](#) ()=default
- virtual void [update](#) (float dt, const [WindowSize](#) &size)=0
- virtual void [event](#) (const [Event](#) &event)=0

Protected Attributes

- [utl::EventBus](#) & [m_eventBus](#) = [utl::EventBus::getInstance\(\)](#)
- std::uint32_t [m_eventComponentId](#) = 0

Private Attributes

- std::string [m_name](#) = "default_name"
- [id](#) [m_id](#) = 1
- [ecs::Registry](#) [m_registry](#)
- std::vector< std::unique_ptr< [ecs::ISystem](#) > > [m_systems](#)

18.10.1 Detailed Description

Abstract class for scene.

Definition at line 51 of file [IScene.hpp](#).

18.10.2 Constructor & Destructor Documentation

18.10.2.1 AScene() [1/3]

[eng::AScene::AScene](#) (
const [id](#) assignedId) [inline], [explicit]

Definition at line 54 of file [IScene.hpp](#).

18.10.2.2 ~AScene()

[eng::AScene::~~AScene](#) () [override], [default]

18.10.2.3 AScene() [2/3]

[eng::AScene::AScene](#) (
const [AScene](#) & other) [delete]

18.10.2.4 AScene() [3/3]

[eng::AScene::AScene](#) (
[AScene](#) && other) [delete]

18.10.3 Member Function Documentation

18.10.3.1 addSystem()

void [eng::AScene::addSystem](#) (
std::unique_ptr< [ecs::ISystem](#) > system) [inline], [override], [virtual]

Implements [eng::IScene](#).

Definition at line 68 of file [IScene.hpp](#).

References [m_systems](#).

18.10.3.2 getId()

`id` `eng::AScene::getId () const` [inline], [nodiscard], [override], [virtual]

Implements [eng::IScene](#).

Definition at line 63 of file [IScene.hpp](#).

References [m_id](#).

18.10.3.3 getName()

`std::string & eng::AScene::getName ()` [inline], [nodiscard], [override], [virtual]

Implements [eng::IScene](#).

Definition at line 62 of file [IScene.hpp](#).

References [m_name](#).

18.10.3.4 getRegistry()

`ecs::Registry & eng::AScene::getRegistry ()` [inline], [nodiscard], [override], [virtual]

Implements [eng::IScene](#).

Definition at line 64 of file [IScene.hpp](#).

References [m_registry](#).

18.10.3.5 operator=() [1/2]

[AScene](#) & `eng::AScene::operator= (`
 [AScene](#) && other) [delete]

18.10.3.6 operator=() [2/2]

[AScene](#) & `eng::AScene::operator= (`
 const [AScene](#) & other) [delete]

18.10.3.7 setName()

`void eng::AScene::setName (`
 const std::string & newName) [inline], [override], [virtual]

Implements [eng::IScene](#).

Definition at line 66 of file [IScene.hpp](#).

References [m_name](#).

18.10.3.8 updateSystems()

`void eng::AScene::updateSystems (`
 const float dt) [inline], [override], [virtual]

Implements [eng::IScene](#).

Definition at line 69 of file [IScene.hpp](#).

References [m_registry](#), and [m_systems](#).

18.10.4 Member Data Documentation

18.10.4.1 m_eventBus

[utl::EventBus](#)& `eng::AScene::m_eventBus = utl::EventBus::getInstance()` [protected]

Definition at line 84 of file [IScene.hpp](#).

Referenced by [gme::CreateRoomScene::CreateRoomScene\(\)](#), [gme::JoinRoomScene::JoinRoomScene\(\)](#), [gme::ServerScene::ServerScene\(\)](#), and [gme::WaitingRoomScene::WaitingRoomScene\(\)](#).

18.10.4.2 m_eventComponentId

`std::uint32_t eng::AScene::m_eventComponentId = 0` [protected]

Definition at line 85 of file [IScene.hpp](#).

Referenced by [gme::CreateRoomScene::CreateRoomScene\(\)](#), [gme::JoinRoomScene::JoinRoomScene\(\)](#), [gme::ServerScene::ServerScene\(\)](#), and [gme::WaitingRoomScene::WaitingRoomScene\(\)](#).

18.10.4.3 m_id

`id` `eng::AScene::m_id = 1` [private]

Definition at line 79 of file [IScene.hpp](#).

Referenced by [getId\(\)](#).

18.10.4.4 m_name

`std::string` `eng::AScene::m_name = "default_name"` [private]

Definition at line 78 of file [IScene.hpp](#).

Referenced by [getName\(\)](#), and [setName\(\)](#).

18.10.4.5 m_registry

`ecs::Registry` `eng::AScene::m_registry` [private]

Definition at line 80 of file [IScene.hpp](#).

Referenced by [getRegistry\(\)](#), and [updateSystems\(\)](#).

18.10.4.6 m_systems

`std::vector<std::unique_ptr<ecs::ISystem>>` `eng::AScene::m_systems` [private]

Definition at line 81 of file [IScene.hpp](#).

Referenced by [addSystem\(\)](#), and [updateSystems\(\)](#).

The documentation for this class was generated from the following file:

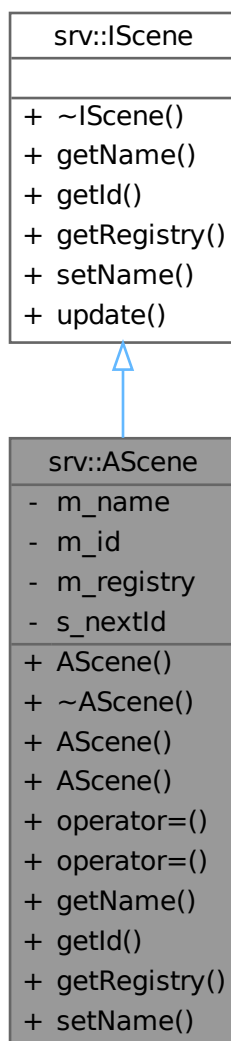
- [/home/masina/Projects/Epitech/rtype/modules/Engine/include/Engine/Interfaces/IScene.hpp](#)

18.11 srv::AScene Class Reference

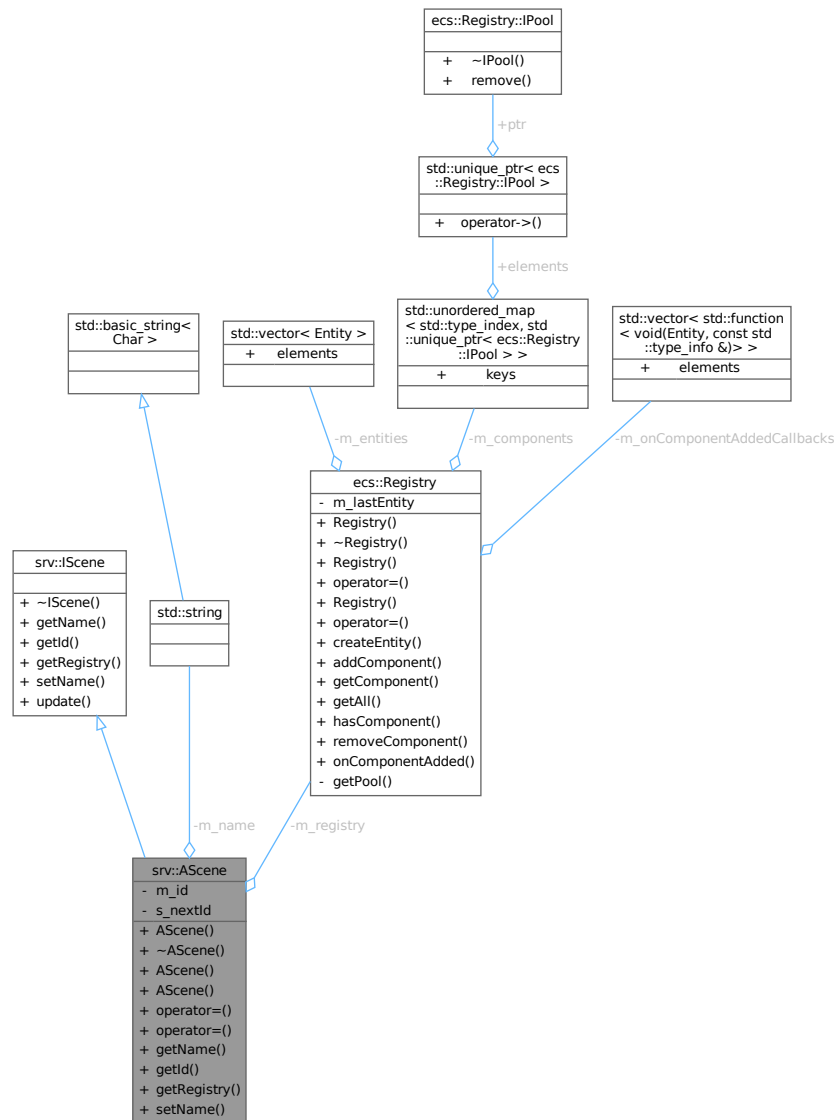
Class for scene.

`#include <IScene.hpp>`

Inheritance diagram for `srv::AScene`:



Collaboration diagram for srv::AScene:



Public Member Functions

- [AScene](#) ()
- [~AScene](#) () override=default
- [AScene](#) (const [AScene](#) &other)=delete
- [AScene](#) ([AScene](#) &&other)=delete
- [AScene](#) & operator= (const [AScene](#) &other)=delete
- [AScene](#) & operator= ([AScene](#) &&other)=delete
- `std::string` & [getName](#) () override
- `id` [getId](#) () const override
- `ecs::Registry` & [getRegistry](#) () override
- void [setName](#) (const `std::string` &newName) override

Public Member Functions inherited from [srv::IScene](#)

- virtual [~IScene](#) ()=default
- virtual void [update](#) (float dt)=0

Private Attributes

- `std::string m_name = "default_name"`
- `id m_id = 1`
- `ecs::Registry m_registry`

Static Private Attributes

- `static id s_nextId = 1`

18.11.1 Detailed Description

Class for scene.

Definition at line 42 of file [IScene.hpp](#).

18.11.2 Constructor & Destructor Documentation

18.11.2.1 AScene() [1/3]

`srv::AScene::AScene ()` [inline]

Definition at line 45 of file [IScene.hpp](#).

18.11.2.2 ~AScene()

`srv::AScene::~~AScene ()` [override], [default]

18.11.2.3 AScene() [2/3]

`srv::AScene::AScene (`
 `const AScene & other)` [delete]

18.11.2.4 AScene() [3/3]

`srv::AScene::AScene (`
 `AScene && other)` [delete]

18.11.3 Member Function Documentation

18.11.3.1 getId()

`id srv::AScene::getId () const` [inline], [nodiscard], [override], [virtual]

Implements [srv::IScene](#).

Definition at line 54 of file [IScene.hpp](#).

References [m_id](#).

18.11.3.2 getName()

`std::string & srv::AScene::getName ()` [inline], [nodiscard], [override], [virtual]

Implements [srv::IScene](#).

Definition at line 53 of file [IScene.hpp](#).

References [m_name](#).

18.11.3.3 getRegistry()

`ecs::Registry & srv::AScene::getRegistry ()` [inline], [nodiscard], [override], [virtual]

Implements [srv::IScene](#).

Definition at line 55 of file [IScene.hpp](#).

References [m_registry](#).

18.11.3.4 operator=() [1/2]

`AScene & srv::AScene::operator= (`
 `AScene && other)` [delete]

18.11.3.5 operator=() [2/2]

[AScene](#) & srv::AScene::operator= (
 const [AScene](#) & other) [delete]

18.11.3.6 setName()

void srv::AScene::setName (
 const std::string & newName) [inline], [override], [virtual]

Implements [srv::IScene](#).

Definition at line 57 of file [IScene.hpp](#).

References [m_name](#).

18.11.4 Member Data Documentation

18.11.4.1 m_id

[id](#) srv::AScene::m_id = 1 [private]

Definition at line 61 of file [IScene.hpp](#).

Referenced by [getId\(\)](#).

18.11.4.2 m_name

std::string srv::AScene::m_name = "default_name" [private]

Definition at line 60 of file [IScene.hpp](#).

Referenced by [getName\(\)](#), and [setName\(\)](#).

18.11.4.3 m_registry

[ecs::Registry](#) srv::AScene::m_registry [private]

Definition at line 62 of file [IScene.hpp](#).

Referenced by [getRegistry\(\)](#).

18.11.4.4 s_nextId

[id](#) srv::AScene::s_nextId = 1 [inline], [static], [private]

Definition at line 63 of file [IScene.hpp](#).

The documentation for this class was generated from the following file:

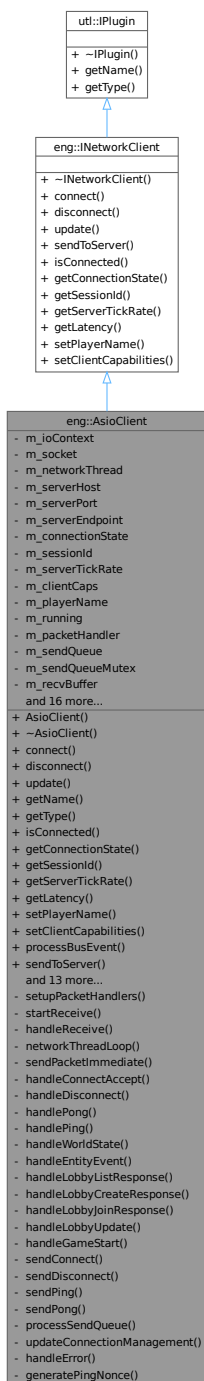
- [/home/masina/Projects/Epitech/rtype/server/include/Server/Interfaces/IScene.hpp](#)

18.12 eng::AsioClient Class Reference

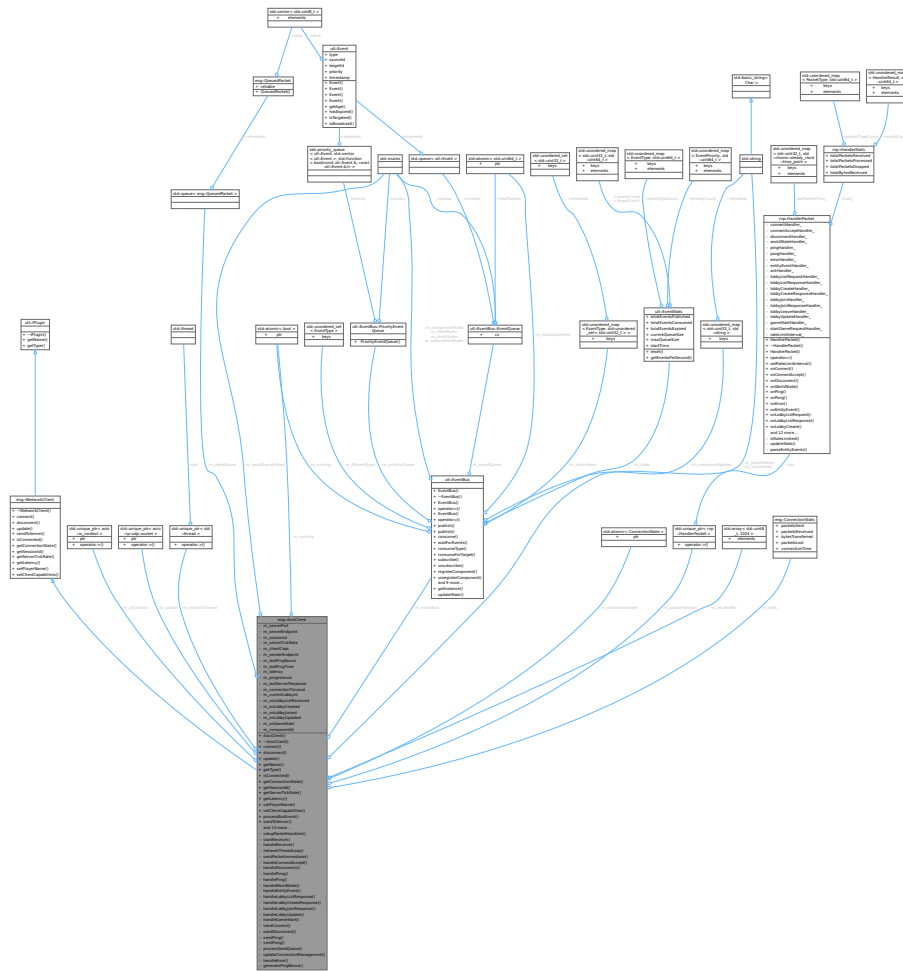
Asio UDP client implementation.

#include <AsioClient.hpp>

Inheritance diagram for eng::AsioClient:



Collaboration diagram for eng::AsioClient:



Public Member Functions

- [AsioClient](#) ()
Constructor.
- [~AsioClient](#) () override
Destructor.
- void [connect](#) (const std::string &host, std::uint16_t port) override
- void [disconnect](#) () override
- void [update](#) () override
- const std::string [getName](#) () const override
- utl::PluginType [getType](#) () const override
- bool [isConnected](#) () const override
- ConnectionState [getConnectionState](#) () const override
- std::uint32_t [getSessionId](#) () const override
- std::uint16_t [getServerTickRate](#) () const override
- std::uint32_t [getLatency](#) () const override
- void [setPlayerName](#) (const std::string &playerName) override
- void [setClientCapabilities](#) (std::uint32_t caps) override
- void [processBusEvent](#) ()
- void [sendToServer](#) (const std::vector< std::uint8_t > &data, bool reliable=false) override
Send custom packet to server.
- const [ConnectionStats](#) & [getStats](#) () const

- `Get connection statistics.`
- `void requestLobbyList ()`
Request lobby list from server.
- `void createLobby (rnp::PacketLobbyCreate lobbyCreate)`
Create a new lobby.
- `void joinLobby (std::uint32_t lobbyId)`
Join an existing lobby.
- `void leaveLobby ()`
Leave current lobby.
- `void requestStartGame (std::uint32_t lobbyId)`
Request to start the game (host only)
- `std::uint32_t getCurrentLobbyId () const`
Get current lobby ID.
- `bool isInLobby () const`
Check if player is in a lobby.
- `void setOnLobbyListReceived (std::function< void(const std::vector< rnp::LobbyInfo > &)> callback)`
Set callback for lobby list received.
- `void setOnLobbyCreated (std::function< void(std::uint32_t, bool, rnp::ErrorCode)> callback)`
Set callback for lobby created.
- `void setOnLobbyJoined (std::function< void(std::uint32_t, bool, rnp::ErrorCode, const rnp::LobbyInfo *)> callback)`
Set callback for lobby joined.
- `void setOnLobbyUpdated (std::function< void(const rnp::LobbyInfo &)> callback)`
Set callback for lobby updated.
- `void setOnGameStart (std::function< void(std::uint32_t, std::uint32_t)> callback)`
Set callback for game start.

Public Member Functions inherited from [eng::INetworkClient](#)

- `virtual ~INetworkClient ()=default`
- `virtual void connect (const std::string &host, uint16_t port)=0`

Public Member Functions inherited from [utl::IPlugin](#)

- `virtual ~IPlugin ()=default`

Private Member Functions

- `void setupPacketHandlers ()`
Initialize packet handlers.
- `void startReceive ()`
Start receiving packets asynchronously.
- `void handleReceive (std::size_t bytesReceived)`
Handle received packet.
- `void networkThreadLoop () const`
Network thread main loop.
- `void sendPacketImmediate (const std::vector< std::uint8_t > &data)`
Send packet immediately.
- `rnp::HandlerResult handleConnectAccept (const rnp::PacketConnectAccept &packet, const rnp::PacketContext &context)`
Handle CONNECT_ACCEPT packet.

- [rnp::HandlerResult handleDisconnect](#) (const [rnp::PacketDisconnect](#) &packet, const [rnp::PacketContext](#) &context)
Handle DISCONNECT packet.
- [rnp::HandlerResult handlePong](#) (const [rnp::PacketPingPong](#) &packet, const [rnp::PacketContext](#) &context)
Handle PONG packet.
- [rnp::HandlerResult handlePing](#) (const [rnp::PacketPingPong](#) &packet, const [rnp::PacketContext](#) &context)
Handle PING packet.
- [rnp::HandlerResult handleWorldState](#) (const [rnp::PacketWorldState](#) &packet, const [rnp::PacketContext](#) &context) const
Handle WORLD_STATE packet.
- [rnp::HandlerResult handleEntityEvent](#) (const std::vector< [rnp::EventRecord](#) > &events, const [rnp::PacketContext](#) &context) const
Handle ENTITY_EVENT packet.
- [rnp::HandlerResult handleLobbyListResponse](#) (const [rnp::PacketLobbyListResponse](#) &packet, const [rnp::PacketContext](#) &context) const
Handle LOBBY_LIST_RESPONSE packet.
- [rnp::HandlerResult handleLobbyCreateResponse](#) (const [rnp::PacketLobbyCreateResponse](#) &packet, const [rnp::PacketContext](#) &context)
Handle LOBBY_CREATE_RESPONSE packet.
- [rnp::HandlerResult handleLobbyJoinResponse](#) (const [rnp::PacketLobbyJoinResponse](#) &packet, const [rnp::PacketContext](#) &context)
Handle LOBBY_JOIN_RESPONSE packet.
- [rnp::HandlerResult handleLobbyUpdate](#) (const [rnp::PacketLobbyUpdate](#) &packet, const [rnp::PacketContext](#) &context) const
Handle LOBBY_UPDATE packet.
- [rnp::HandlerResult handleGameStart](#) (const [rnp::PacketGameStart](#) &packet, const [rnp::PacketContext](#) &context) const
Handle GAME_START packet.
- void [sendConnect](#) ()
Send CONNECT packet.
- void [sendDisconnect](#) ()
Send DISCONNECT packet.
- void [sendPing](#) ()
Send PING packet.
- void [sendPong](#) (std::uint32_t nonce)
Send PONG response.
- void [processSendQueue](#) ()
Process send queue.
- void [updateConnectionManagement](#) ()
Update connection management (timeouts, pings)

Static Private Member Functions

- static [rnp::HandlerResult handleError](#) (const [rnp::PacketError](#) &packet, const [rnp::PacketContext](#) &context)
Handle ERROR packet.
- static std::uint32_t [generatePingNonce](#) ()
Generate ping nonce.

Private Attributes

- `std::unique_ptr< asio::io_context > m_ioContext`
- `std::unique_ptr< asio::ip::udp::socket > m_socket`
- `std::unique_ptr< std::thread > m_networkThread`
- `std::string m_serverHost`
- `std::uint16_t m_serverPort`
- `asio::ip::udp::endpoint m_serverEndpoint`
- `std::atomic< ConnectionState > m_connectionState`
- `std::uint32_t m_sessionId`
- `std::uint16_t m_serverTickRate`
- `std::uint32_t m_clientCaps`
- `std::string m_playerName`
- `std::atomic< bool > m_running`
- `std::unique_ptr< rnp::HandlerPacket > m_packetHandler`
- `std::queue< QueuedPacket > m_sendQueue`
- `std::mutex m_sendQueueMutex`
- `std::array< std::uint8_t, 1024 > m_recvBuffer`
- `asio::ip::udp::endpoint m_senderEndpoint`
- `std::uint32_t m_lastPingNonce`
- `std::chrono::steady_clock::time_point m_lastPingTime`
- `std::uint32_t m_latency`
- `std::chrono::milliseconds m_pingInterval`
- `std::chrono::steady_clock::time_point m_lastServerResponse`
- `std::chrono::milliseconds m_connectionTimeout`
- `ConnectionStats m_stats`
- `std::uint32_t m_currentLobbyId`
- `std::function< void(const std::vector< rnp::LobbyInfo > &)> m_onLobbyListReceived`
- `std::function< void(std::uint32_t, bool, rnp::ErrorCode)> m_onLobbyCreated`
- `std::function< void(std::uint32_t, bool, rnp::ErrorCode, const rnp::LobbyInfo *)> m_onLobbyJoined`
- `std::function< void(const rnp::LobbyInfo &)> m_onLobbyUpdated`
- `std::function< void(std::uint32_t, std::uint32_t)> m_onGameStart`
- `utl::EventBus & m_eventBus`
- `std::uint32_t m_componentId = utl::NETWORK_CLIENT`

18.12.1 Detailed Description

Asio UDP client implementation.

Definition at line 58 of file [AsioClient.hpp](#).

18.12.2 Constructor & Destructor Documentation

18.12.2.1 AsioClient()

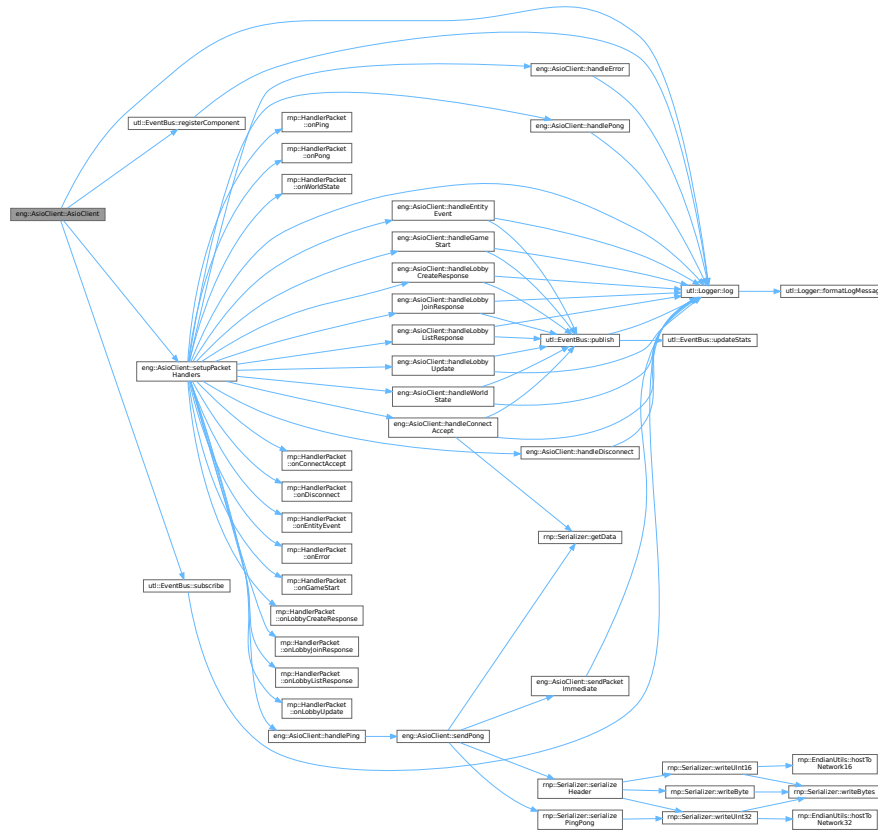
`eng::AsioClient::AsioClient ()`

Constructor.

Definition at line 21 of file [asioClient.cpp](#).

References [eng::ConnectionStats::connectionTime](#), [utl::INFO](#), [utl::LOBBY_CREATE](#), [utl::LOBBY_JOIN](#), [utl::LOBBY_LEAVE](#), [utl::LOBBY_LIST_REQUEST](#), [utl::Logger::log\(\)](#), [m_componentId](#), [m_eventBus](#), [m_stats](#), [utl::EventBus::registerComponent\(\)](#), [utl::REQUEST_CONNECT](#), [utl::REQUEST_DISCONNECT](#), [utl::SEND_ENTITY_EVENT](#), [utl::SEND_PLAYER_INPUT](#), [setupPacketHandlers\(\)](#), and [utl::EventBus::subscribe\(\)](#).

Here is the call graph for this function:



18.12.2.2 ~AsioClient()

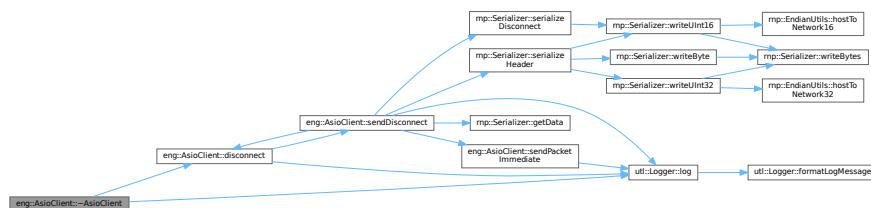
`eng::AsioClient::~~AsioClient()` [override]

Destructor.

Definition at line 46 of file `asioClient.cpp`.

References `disconnect()`, `utl::INFO`, and `utl::Logger::log()`.

Here is the call graph for this function:



18.12.3 Member Function Documentation

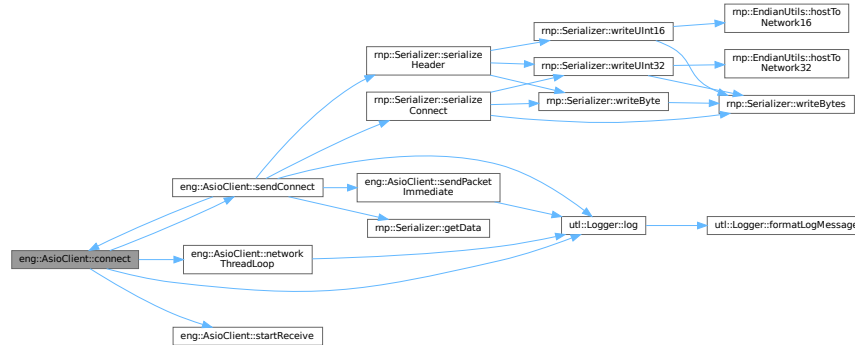
18.12.3.1 connect()

```
void eng::AsioClient::connect (
    const std::string & host,
    std::uint16_t port) [override]
```

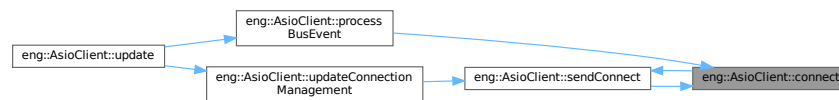
Definition at line 52 of file `asioClient.cpp`.

References [eng::CONNECTING](#), [eng::DISCONNECTED](#), [utl::INFO](#), [utl::Logger::log\(\)](#), [m_connectionState](#), [m_ioContext](#), [m_lastServerResponse](#), [m_networkThread](#), [m_running](#), [m_serverEndpoint](#), [m_serverHost](#), [m_serverPort](#), [m_socket](#), [networkThreadLoop\(\)](#), [sendConnect\(\)](#), [startReceive\(\)](#), and [utl::WARNING](#).
 Referenced by [processBusEvent\(\)](#), and [sendConnect\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.12.3.2 createLobby()

void [eng::AsioClient::createLobby](#) (
 [rnp::PacketLobbyCreate](#) lobbyCreate)

Create a new lobby.

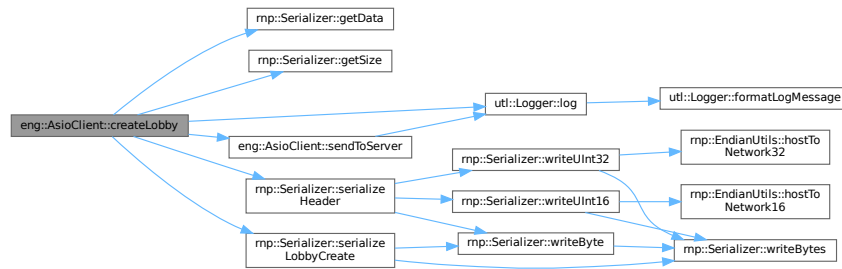
Parameters

name	Lobby name
maxPlayers	Maximum number of players
gameMode	Game mode identifier

Definition at line 813 of file [asioClient.cpp](#).

References [eng::CONNECTED](#), [rnp::PacketLobbyCreate::gameMode](#), [rnp::Serializer::getData\(\)](#), [rnp::Serializer::getSize\(\)](#), [utl::INFO](#), [rnp::PacketHeader::length](#), [rnp::LOBBY_CREATE](#), [rnp::PacketLobbyCreate::lobbyName](#), [utl::Logger::log\(\)](#), [m_connectionState](#), [m_currentLobbyId](#), [m_sessionId](#), [rnp::PacketLobbyCreate::maxPlayers](#), [rnp::PacketLobbyCreate::nameLen](#), [sendToServer\(\)](#), [rnp::Serializer::serializeHeader\(\)](#), [rnp::Serializer::serializeLobbyCreate\(\)](#), [rnp::PacketHeader::sessionId](#), [rnp::PacketHeader::type](#), and [utl::WARNING](#).
 Referenced by [processBusEvent\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.12.3.3 disconnect()

void eng::AsioClient::disconnect () [override], [virtual]

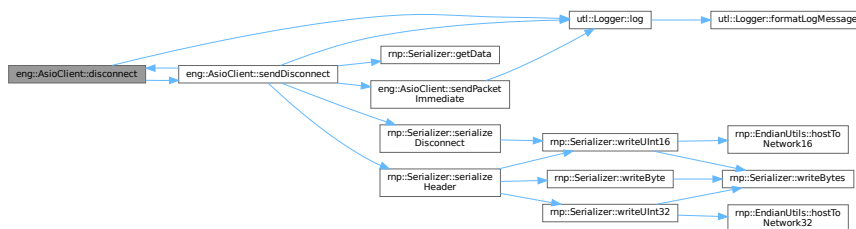
Implements [eng::INetworkClient](#).

Definition at line 98 of file [asioClient.cpp](#).

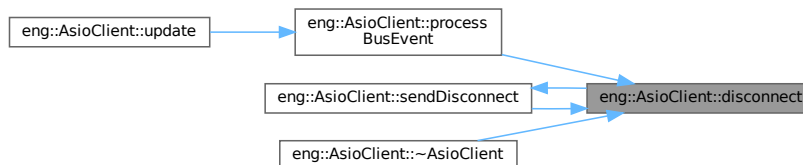
References [eng::DISCONNECTED](#), [eng::DISCONNECTING](#), [utl::INFO](#), [utl::Logger::log\(\)](#), [m_connectionState](#), [m_ioContext](#), [m_latency](#), [m_networkThread](#), [m_running](#), [m_sendQueue](#), [m_sendQueueMutex](#), [m_serverTickRate](#), [m_sessionId](#), [m_socket](#), and [sendDisconnect\(\)](#).

Referenced by [processBusEvent\(\)](#), [sendDisconnect\(\)](#), and [~AsioClient\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.12.3.4 generatePingNonce()

`std::uint32_t eng::AsioClient::generatePingNonce () [static], [private]`

Generate ping nonce.

Returns

New ping nonce

Definition at line 651 of file [asioClient.cpp](#).

Referenced by [sendPing\(\)](#).

Here is the caller graph for this function:



18.12.3.5 getConnectionState()

`ConnectionState eng::AsioClient::getConnectionState () const [nodiscard], [override], [virtual]`

Implements [eng::INetworkClient](#).

Definition at line 174 of file [asioClient.cpp](#).

References [m_connectionState](#).

18.12.3.6 getCurrentLobbyId()

`std::uint32_t eng::AsioClient::getCurrentLobbyId () const [inline]`

Get current lobby ID.

Returns

Current lobby ID (0 if not in lobby)

Definition at line 139 of file [AsioClient.hpp](#).

References [m_currentLobbyId](#).

18.12.3.7 getLatency()

`std::uint32_t eng::AsioClient::getLatency () const [nodiscard], [override], [virtual]`

Implements [eng::INetworkClient](#).

Definition at line 180 of file [asioClient.cpp](#).

References [m_latency](#).

18.12.3.8 getName()

`const std::string eng::AsioClient::getName () const [inline], [nodiscard], [override], [virtual]`

Implements [utl::IPlugin](#).

Definition at line 77 of file [AsioClient.hpp](#).

18.12.3.9 getServerTickRate()

`std::uint16_t eng::AsioClient::getServerTickRate () const [nodiscard], [override], [virtual]`

Implements [eng::INetworkClient](#).

Definition at line 178 of file [asioClient.cpp](#).

References [m_serverTickRate](#).

18.12.3.10 getSessionId()

`std::uint32_t eng::AsioClient::getSessionId () const [nodiscard], [override], [virtual]`

Implements [eng::INetworkClient](#).

Definition at line 176 of file [asioClient.cpp](#).

References [m_sessionId](#).

18.12.3.11 getStats()

const [ConnectionStats](#) & eng::AsioClient::getStats () const [inline]
Get connection statistics.

Returns

Current connection stats

Definition at line 102 of file [AsioClient.hpp](#).

References [m_stats](#).

18.12.3.12 getType()

[utl::PluginType](#) eng::AsioClient::getType () const [inline], [nodiscard], [override], [virtual]

Implements [utl::IPlugin](#).

Definition at line 78 of file [AsioClient.hpp](#).

References [utl::NETWORK_CLIENT](#).

18.12.3.13 handleConnectAccept()

[rnp::HandlerResult](#) eng::AsioClient::handleConnectAccept (
const [rnp::PacketConnectAccept](#) & packet,
const [rnp::PacketContext](#) & context) [private]

Handle CONNECT_ACCEPT packet.

Parameters

packet	Connect accept packet
context	Packet context

Returns

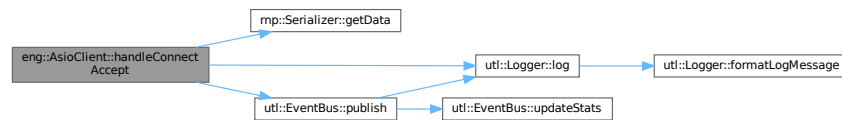
Handler result

Definition at line 376 of file [asioClient.cpp](#).

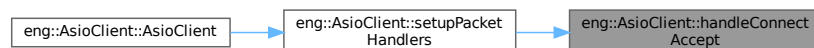
References [eng::CONNECTED](#), [utl::CONNECTION_ACCEPTED](#), [rnp::Serializer::getData\(\)](#), [utl::INFO](#), [utl::Logger::log\(\)](#), [m_componentId](#), [m_connectionState](#), [m_eventBus](#), [m_serverTickRate](#), [m_sessionId](#), [utl::EventBus::publish\(\)](#), [utl::RENDERING_ENGINE](#), [rnp::PacketConnectAccept::sessionId](#), [rnp::SUCCESS](#), and [rnp::PacketConnectAccept::tickRateHz](#).

Referenced by [setupPacketHandlers\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.12.3.14 handleDisconnect()

```

rnp::HandlerResult eng::AsioClient::handleDisconnect (
    const rnp::PacketDisconnect & packet,
    const rnp::PacketContext & context) [private]

```

Handle DISCONNECT packet.

Parameters

packet	Disconnect packet
context	Packet context

Returns

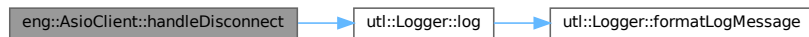
Handler result

Definition at line 393 of file [asioClient.cpp](#).

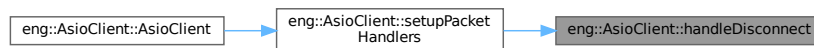
References [rnp::BANNED](#), [rnp::CLIENT_REQUEST](#), [eng::DISCONNECTED](#), [utl::INFO](#), [utl::Logger::log\(\)](#), [m_connectionState](#), [m_sessionId](#), [rnp::PROTOCOL_ERROR](#), [rnp::PacketDisconnect::reasonCode](#), [rnp::SERVER_FULL](#), [rnp::SERVER_SHUTDOWN](#), [rnp::SUCCESS](#), [rnp::TIMEOUT](#), and [rnp::UNSPECIFIED](#).

Referenced by [setupPacketHandlers\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.12.3.15 handleEntityEvent()

```

rnp::HandlerResult eng::AsioClient::handleEntityEvent (
    const std::vector< rnp::EventRecord > & events,
    const rnp::PacketContext & context) const [private]

```

Handle ENTITY_EVENT packet.

Parameters

events	Vector of event records
context	Packet context

Returns

Handler result

Definition at line 777 of file [asioClient.cpp](#).

References [utl::ENTITY_EVENT_RECEIVED](#), [utl::INFO](#), [utl::Logger::log\(\)](#), [m_componentId](#), [m_eventBus](#), [utl::EventBus::publish\(\)](#), and [rnp::SUCCESS](#).

Referenced by [setupPacketHandlers\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.12.3.16 handleError()

[rnp::HandlerResult](#) [eng::AsioClient::handleError](#) (
 const [rnp::PacketError](#) & packet,
 const [rnp::PacketContext](#) & context) [static], [private]

Handle ERROR packet.

Parameters

packet	Error packet
context	Packet context

Returns

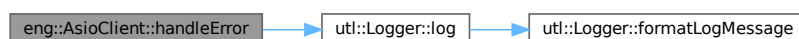
Handler result

Definition at line 454 of file [asioClient.cpp](#).

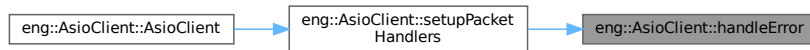
References [rnp::PacketError::description](#), [rnp::PacketError::errorCode](#), [rnp::INTERNAL_ERROR](#), [rnp::INVALID_PAYLOAD](#), [utl::Logger::log\(\)](#), [rnp::RATE_LIMITED](#), [rnp::SUCCESS](#), [rnp::UNAUTHORIZED_SESSION](#), and [utl::WARNING](#).

Referenced by [setupPacketHandlers\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.12.3.17 handleGameStart()

rnp::HandlerResult eng::AsioClient::handleGameStart (
 const **rnp::PacketGameStart** & packet,
 const **rnp::PacketContext** & context) const [private]
 Handle GAME_START packet.

Parameters

packet	Game start packet
context	Packet context

Returns

Handler result

Definition at line 1046 of file asioClient.cpp.

References [utl::Event::data](#), [utl::GAME_START](#), [utl::INFO](#), [rnp::PacketGameStart::lobbyId](#), [utl::Logger::log\(\)](#), [m_componentId](#), [m_eventBus](#), [m_onGameStart](#), [utl::EventBus::publish\(\)](#), [rnp::PacketContext::sessionId](#), and [rnp::SUCCESS](#).

Referenced by [setupPacketHandlers\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.12.3.18 handleLobbyCreateResponse()

rnp::HandlerResult eng::AsioClient::handleLobbyCreateResponse (
 const **rnp::PacketLobbyCreateResponse** & packet,
 const **rnp::PacketContext** & context) [private]
 Handle LOBBY_CREATE_RESPONSE packet.

Parameters

packet	Lobby create response packet
context	Packet context

Returns

Handler result

Definition at line 983 of file [asioClient.cpp](#).

References [rnp::PacketLobbyCreateResponse::errorCode](#), [utl::INFO](#), [utl::LOBBY_CREATE_RESPONSE](#), [rnp::PacketLobbyCreateResponse::lobbyId](#), [utl::Logger::log\(\)](#), [m_componentId](#), [m_currentLobbyId](#), [m_eventBus](#), [m_onLobbyCreated](#), [utl::EventBus::publish\(\)](#), [rnp::SUCCESS](#), and [rnp::PacketLobbyCreateResponse::success](#).
Referenced by [setupPacketHandlers\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.12.3.19 handleLobbyJoinResponse()

```

rnp::HandlerResult eng::AsioClient::handleLobbyJoinResponse (
    const rnp::PacketLobbyJoinResponse & packet,
    const rnp::PacketContext & context) [private]

```

Handle LOBBY_JOIN_RESPONSE packet.

Parameters

packet	Lobby join response packet
context	Packet context

Returns

Handler result

Definition at line 1005 of file [asioClient.cpp](#).

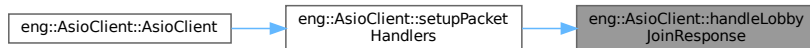
References [rnp::PacketLobbyJoinResponse::errorCode](#), [utl::INFO](#), [utl::LOBBY_JOIN_RESPONSE](#), [rnp::PacketLobbyJoinResponse::lobbyId](#), [rnp::PacketLobbyJoinResponse::lobbyInfo](#), [utl::Logger::log\(\)](#), [m_componentId](#), [m_currentLobbyId](#), [m_eventBus](#), [m_onLobbyJoined](#), [utl::EventBus::publish\(\)](#), [rnp::SUCCESS](#), and [rnp::PacketLobbyJoinResponse::success](#).

Referenced by [setupPacketHandlers\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.12.3.20 handleLobbyListResponse()

```

rnp::HandlerResult eng::AsioClient::handleLobbyListResponse (
    const rnp::PacketLobbyListResponse & packet,
    const rnp::PacketContext & context) const [private]

```

Handle LOBBY_LIST_RESPONSE packet.

Parameters

packet	Lobby list response packet
context	Packet context

Returns

Handler result

Definition at line 968 of file [asioClient.cpp](#).

References [utl::INFO](#), [rnp::PacketLobbyListResponse::lobbies](#), [utl::LOBBY_LIST_RESPONSE](#), [rnp::PacketLobbyListResponse::lobbyCount](#), [utl::Logger::log\(\)](#), [m_componentId](#), [m_eventBus](#), [m_onLobbyListReceived](#), [utl::EventBus::publish\(\)](#), and [rnp::SUCCESS](#).

Referenced by [setupPacketHandlers\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.12.3.21 handleLobbyUpdate()

```

rnp::HandlerResult eng::AsioClient::handleLobbyUpdate (
    const rnp::PacketLobbyUpdate & packet,
    const rnp::PacketContext & context) const [private]

```

Handle LOBBY_UPDATE packet.

Parameters

packet	Lobby update packet
context	Packet context

Returns

Handler result

Definition at line 1029 of file `asioClient.cpp`.

References `utl::INFO`, `utl::LOBBY_UPDATE`, `rnp::LobbyInfo::lobbyId`, `rnp::PacketLobbyUpdate::lobbyInfo`, `utl::Logger::log()`, `m_componentId`, `m_eventBus`, `m_onLobbyUpdated`, `utl::EventBus::publish()`, and `rnp::SUCCESS`.

Referenced by `setupPacketHandlers()`.

Here is the call graph for this function:



Here is the caller graph for this function:



18.12.3.22 handlePing()

```

rnp::HandlerResult eng::AsioClient::handlePing (
    const rnp::PacketPingPong & packet,
    const rnp::PacketContext & context) [private]

```

Handle PING packet.

Parameters

packet	Ping packet
context	Packet context

Returns

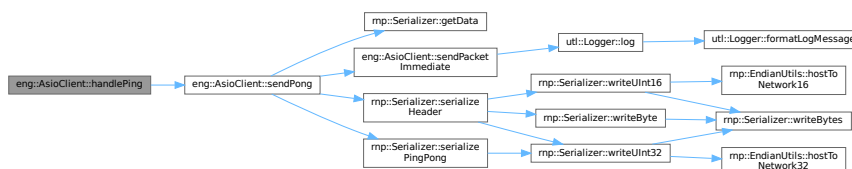
Handler result

Definition at line 447 of file [asioClient.cpp](#).

References [rnp::PacketPingPong::nonce](#), [sendPong\(\)](#), and [rnp::SUCCESS](#).

Referenced by [setupPacketHandlers\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.12.3.23 handlePong()

```

rnp::HandlerResult eng::AsioClient::handlePong (
    const rnp::PacketPingPong & packet,
    const rnp::PacketContext & context) [private]

```

Handle PONG packet.

Parameters

packet	Pong packet
context	Packet context

Returns

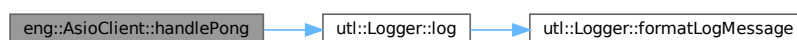
Handler result

Definition at line 432 of file [asioClient.cpp](#).

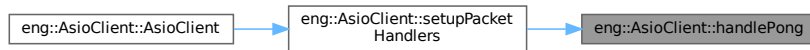
References [utl::INFO](#), [utl::Logger::log\(\)](#), [m_lastPingNonce](#), [m_lastPingTime](#), [m_latency](#), [rnp::PacketPingPong::nonce](#), and [rnp::SUCCESS](#).

Referenced by [setupPacketHandlers\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.12.3.24 handleReceive()

```
void eng::AsioClient::handleReceive (
    std::size_t bytesReceived) [private]
Handle received packet.
```

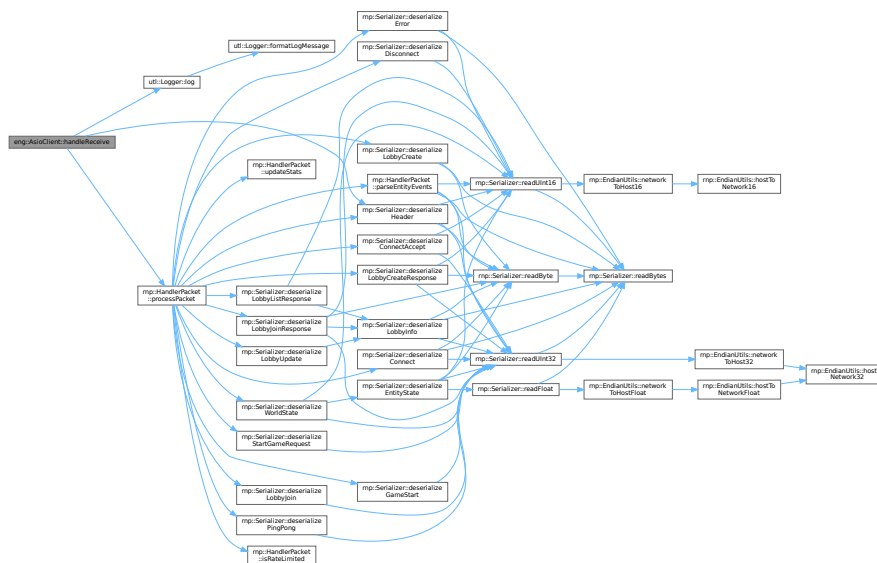
Parameters

bytesReceived	Number of bytes received
---------------	--------------------------

Definition at line 283 of file [asioClient.cpp](#).

References [eng::ConnectionStats::bytesTransferred](#), [rnp::Serializer::deserializeHeader\(\)](#), [utl::Logger::log\(\)](#), [m_lastServerResponse](#), [m_packetHandler](#), [m_rcvBuffer](#), [m_senderEndpoint](#), [m_stats](#), [eng::ConnectionStats::packetsReceived](#), [rnp::HandlerPacket::processPacket\(\)](#), [rnp::PacketContext::receiveTime](#), [rnp::PacketContext::senderAddress](#), [rnp::PacketContext::senderPort](#), [rnp::PacketContext::sessionId](#), [rnp::PacketHeader::sessionId](#), [rnp::SUCCESS](#), and [utl::WARNING](#).

Here is the call graph for this function:



18.12.3.25 handleWorldState()

```
rnp::HandlerResult eng::AsioClient::handleWorldState (
    const rnp::PacketWorldState & packet,
    const rnp::PacketContext & context) const [private]
Handle WORLD_STATE packet.
```

Parameters

packet	World state packet
context	Packet context

Returns

Handler result

Definition at line 483 of file [asioClient.cpp](#).

References [rnp::PacketWorldState::entityCount](#), [util::INFO](#), [util::Logger::log\(\)](#), [m_componentId](#), [m_eventBus](#), [util::EventBus::publish\(\)](#), [rnp::PacketWorldState::serverTick](#), [rnp::SUCCESS](#), and [util::WORLD_STATE_RECEIVED](#).

Referenced by [setupPacketHandlers\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.12.3.26 isConnected()

`bool eng::AsioClient::isConnected () const` [nodiscard], [override], [virtual]

Implements [eng::INetworkClient](#).

Definition at line 172 of file [asioClient.cpp](#).

References [eng::CONNECTED](#), and [m_connectionState](#).

18.12.3.27 isInLobby()

`bool eng::AsioClient::isInLobby () const` [inline]

Check if player is in a lobby.

Returns

True if in lobby

Definition at line 145 of file [AsioClient.hpp](#).

References [m_currentLobbyId](#).

18.12.3.28 joinLobby()

`void eng::AsioClient::joinLobby (`

`std::uint32_t lobbyId)`

Join an existing lobby.

Parameters

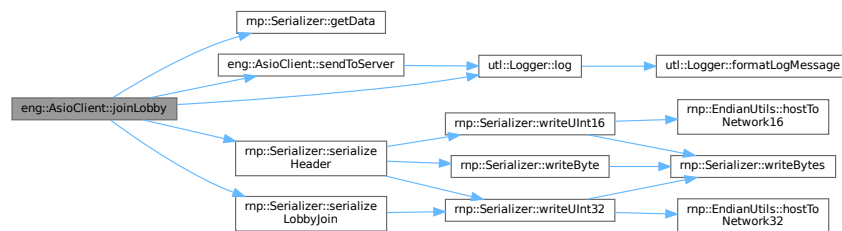
lobbyId	ID of the lobby to join
---------	-------------------------

Definition at line 852 of file [asioClient.cpp](#).

References [eng::CONNECTED](#), [rnp::Serializer::getData\(\)](#), [utl::INFO](#), [rnp::PacketHeader::length](#), [rnp::LOBBY_JOIN](#), [rnp::PacketLobbyJoin::lobbyId](#), [utl::Logger::log\(\)](#), [m_connectionState](#), [m_currentLobbyId](#), [m_sessionId](#), [sendToServer\(\)](#), [rnp::Serializer::serializeHeader\(\)](#), [rnp::Serializer::serializeLobbyJoin\(\)](#), [rnp::PacketHeader::sessionId](#), [rnp::PacketHeader::type](#), and [utl::WARNING](#).

Referenced by [processBusEvent\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.12.3.29 leaveLobby()

void [eng::AsioClient::leaveLobby\(\)](#)

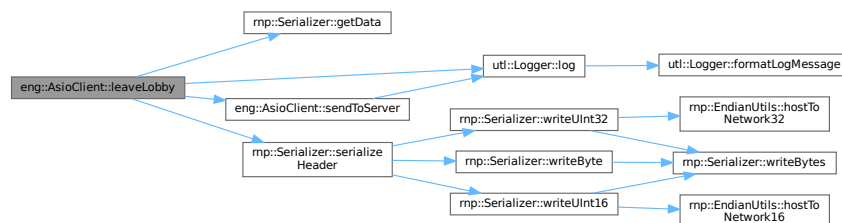
Leave current lobby.

Definition at line 884 of file [asioClient.cpp](#).

References [eng::CONNECTED](#), [rnp::Serializer::getData\(\)](#), [utl::INFO](#), [rnp::PacketHeader::length](#), [rnp::LOBBY_LEAVE](#), [utl::Logger::log\(\)](#), [m_connectionState](#), [m_currentLobbyId](#), [m_sessionId](#), [sendToServer\(\)](#), [rnp::Serializer::serializeHeader\(\)](#), [rnp::PacketHeader::sessionId](#), [rnp::PacketHeader::type](#), and [utl::WARNING](#).

Referenced by [processBusEvent\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.12.3.30 networkThreadLoop()

void eng::AsioClient::networkThreadLoop () const [private]

Network thread main loop.

Definition at line 324 of file [asioClient.cpp](#).

References [utl::INFO](#), [utl::Logger::log\(\)](#), [m_ioContext](#), [m_running](#), and [utl::WARNING](#).

Referenced by [connect\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.12.3.31 processBusEvent()

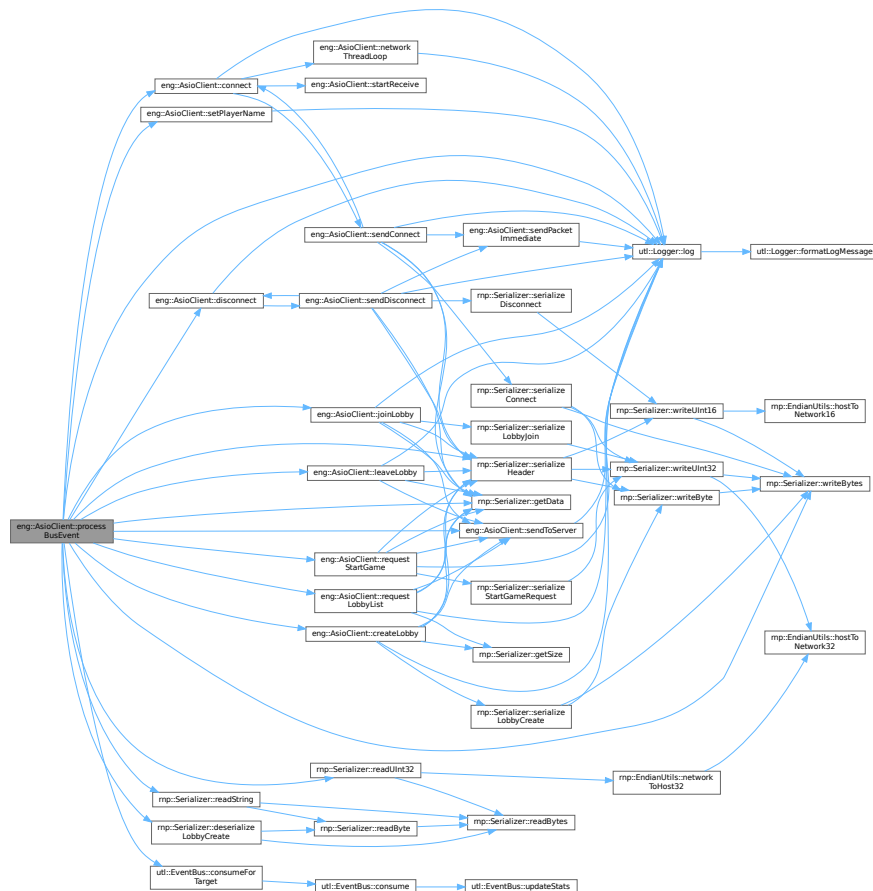
void eng::AsioClient::processBusEvent ()

Definition at line 659 of file [asioClient.cpp](#).

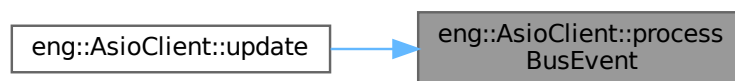
References [connect\(\)](#), [eng::CONNECTED](#), [utl::EventBus::consumeForTarget\(\)](#), [createLobby\(\)](#), [rnp::Serializer::deserializeLobbyCreate\(\)](#), [disconnect\(\)](#), [eng::DISCONNECTED](#), [rnp::ENTITY_EVENT](#), [rnp::PacketLobbyCreate::gameMode](#), [rnp::Serializer::getData\(\)](#), [utl::INFO](#), [joinLobby\(\)](#), [leaveLobby\(\)](#), [rnp::PacketHeader::length](#), [utl::LOBBY_CREATE](#), [utl::LOBBY_JOIN](#), [utl::LOBBY_LEAVE](#), [utl::LOBBY_LIST_REQUEST](#), [rnp::PacketLobbyCreate::lobbyName](#), [utl::Logger::log\(\)](#), [m_componentId](#), [m_connectionState](#), [m_eventBus](#), [m_sessionId](#), [rnp::PacketLobbyCreate::maxPlayers](#), [rnp::PacketLobbyCreate::nameLen](#), [rnp::Serializer::readString\(\)](#), [rnp::Serializer::readUInt32\(\)](#), [utl::REQUEST_CONNECT](#), [utl::REQUEST_DISCONNECT](#), [requestLobbyList\(\)](#), [requestStartGame\(\)](#), [utl::SEND_ENTITY_EVENT](#), [utl::SEND_PLAYER_INPUT](#), [sendToServer\(\)](#), [rnp::Serializer::serializeHeader\(\)](#), [rnp::PacketHeader::sessionId](#), [setPlayerName\(\)](#), [rnp::PacketHeader::type](#), [utl::WARNING](#), [utl::WORLD_STATE_RECEIVED](#), and [rnp::Serializer::writeBytes\(\)](#).

Referenced by [update\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.12.3.32 processSendQueue()

void `eng::AsioClient::processSendQueue()` [private]

Process send queue.

Definition at line 597 of file `asioClient.cpp`.

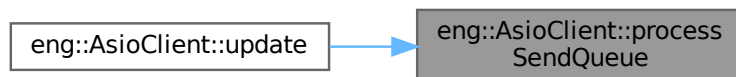
References `eng::QueuedPacket::data`, `util::INFO`, `util::Logger::log()`, `m_sendQueue`, `m_sendQueueMutex`, and `sendPacketImmediate()`.

Referenced by `update()`.

Here is the call graph for this function:



Here is the caller graph for this function:



18.12.3.33 requestLobbyList()

void eng::AsioClient::requestLobbyList ()

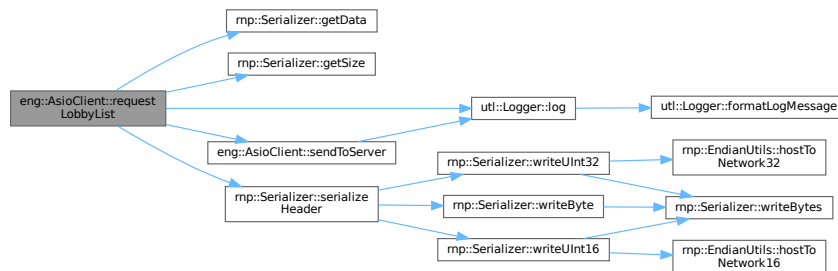
Request lobby list from server.

Definition at line 790 of file [asioClient.cpp](#).

References [eng::CONNECTED](#), [rnp::Serializer::getData\(\)](#), [rnp::Serializer::getSize\(\)](#), [utl::INFO](#), [rnp::PacketHeader::length](#), [rnp::LOBBY_LIST_REQUEST](#), [utl::Logger::log\(\)](#), [m_connectionState](#), [m_sessionId](#), [sendToServer\(\)](#), [rnp::Serializer::serializeHeader\(\)](#), [rnp::PacketHeader::sessionId](#), [rnp::PacketHeader::type](#), and [utl::WARNING](#).

Referenced by [processBusEvent\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.12.3.34 requestStartGame()

void eng::AsioClient::requestStartGame (
 std::uint32_t lobbyId)
 Request to start the game (host only)

Parameters

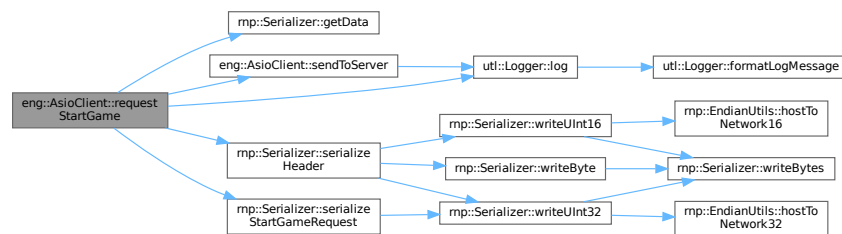
lobbyId	Lobby ID to start
---------	-------------------

Definition at line 914 of file [asioClient.cpp](#).

References [eng::CONNECTED](#), [rnp::Serializer::getData\(\)](#), [utl::INFO](#), [rnp::PacketHeader::length](#), [rnp::PacketStartGameRequest::lobbyId](#), [utl::Logger::log\(\)](#), [m_connectionState](#), [m_sessionId](#), [sendToServer\(\)](#), [rnp::Serializer::serializeHeader\(\)](#), [rnp::Serializer::serializeStartGameRequest\(\)](#), [rnp::PacketHeader::sessionId](#), [rnp::START_GAME_REQUEST](#), [rnp::PacketHeader::type](#), and [utl::WARNING](#).

Referenced by [processBusEvent\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.12.3.35 sendConnect()

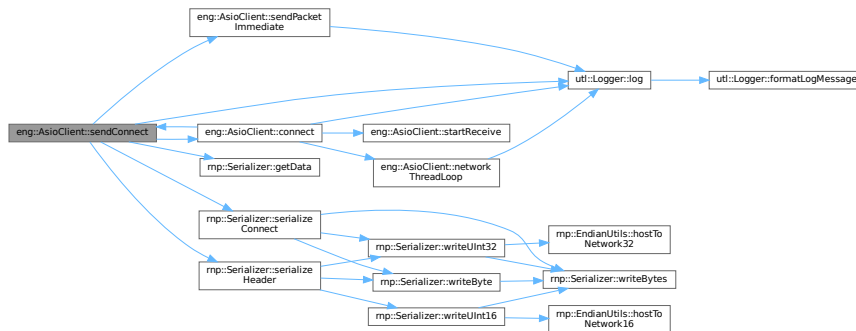
void eng::AsioClient::sendConnect () [private]
 Send CONNECT packet.

Definition at line 497 of file [asioClient.cpp](#).

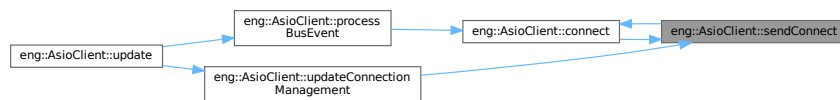
References [rnp::CONNECT](#), [connect\(\)](#), [rnp::Serializer::getData\(\)](#), [utl::INFO](#), [utl::Logger::log\(\)](#), [m_clientCaps](#), [m_playerName](#), [rnp::PacketConnect::nameLen](#), [sendPacketImmediate\(\)](#), [rnp::Serializer::serializeConnect\(\)](#), [rnp::Serializer::serializeHeader\(\)](#), and [rnp::PacketHeader::type](#).

Referenced by [connect\(\)](#), and [updateConnectionManagement\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.12.3.36 sendDisconnect()

void eng::AsioClient::sendDisconnect () [private]

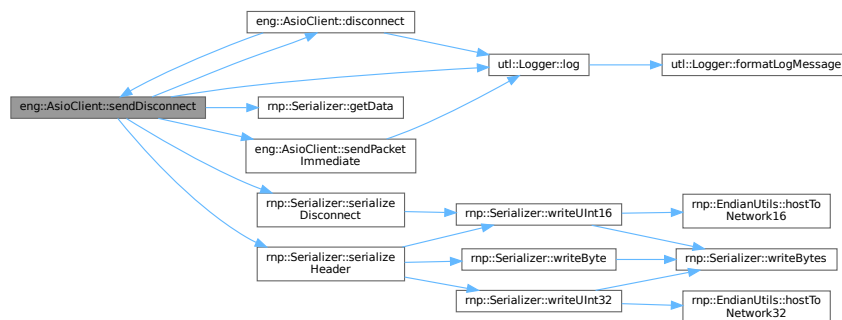
Send DISCONNECT packet.

Definition at line 522 of file [asioClient.cpp](#).

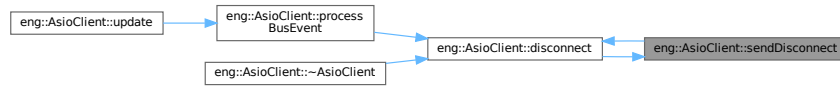
References [rnp::CLIENT_REQUEST](#), [rnp::DISCONNECT](#), [disconnect\(\)](#), [rnp::Serializer::getData\(\)](#), [utl::INFO](#), [utl::Logger::log\(\)](#), [m_sessionId](#), [rnp::PacketDisconnect::reasonCode](#), [sendPacketImmediate\(\)](#), [rnp::Serializer::serializeDisconnect\(\)](#), [rnp::Serializer::serializeHeader\(\)](#), and [rnp::PacketHeader::type](#).

Referenced by [disconnect\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.12.3.37 sendPacketImmediate()

```
void eng::AsioClient::sendPacketImmediate (
    const std::vector< std::uint8_t > & data) [private]
```

Send packet immediately.

Parameters

data	Packet data
------	-------------

Definition at line 350 of file [asioClient.cpp](#).

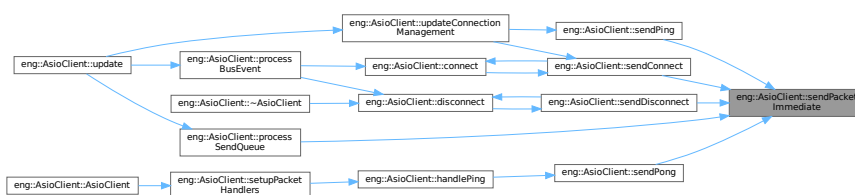
References [eng::ConnectionStats::bytesTransferred](#), [utl::INFO](#), [utl::Logger::log\(\)](#), [m_serverEndpoint](#), [m_socket](#), [m_stats](#), [eng::ConnectionStats::packetsSent](#), and [utl::WARNING](#).

Referenced by [processSendQueue\(\)](#), [sendConnect\(\)](#), [sendDisconnect\(\)](#), [sendPing\(\)](#), and [sendPong\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.12.3.38 sendPing()

```
void eng::AsioClient::sendPing () [private]
```

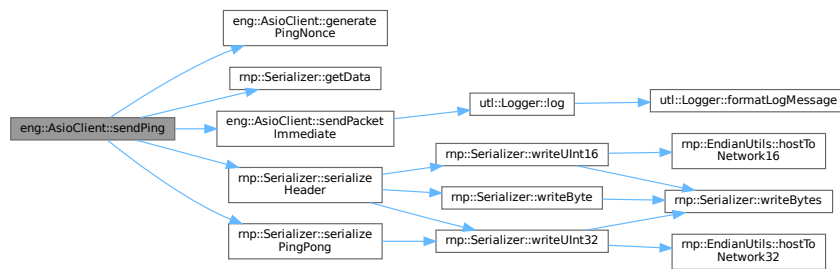
Send PING packet.

Definition at line 545 of file [asioClient.cpp](#).

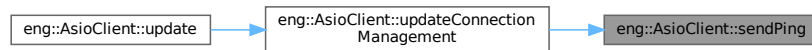
References [generatePingNonce\(\)](#), [rnp::Serializer::getData\(\)](#), [m_lastPingNonce](#), [m_lastPingTime](#), [m_sessionId](#), [rnp::PacketPingPong::nonce](#), [rnp::PING](#), [sendPacketImmediate\(\)](#), [rnp::Serializer::serializeHeader\(\)](#), [rnp::Serializer::serializePingPong\(\)](#), and [rnp::PacketHeader::type](#).

Referenced by [updateConnectionManagement\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.12.3.39 sendPong()

```
void eng::AsioClient::sendPong (
    std::uint32_t nonce) [private]
Send PONG response.
```

Parameters

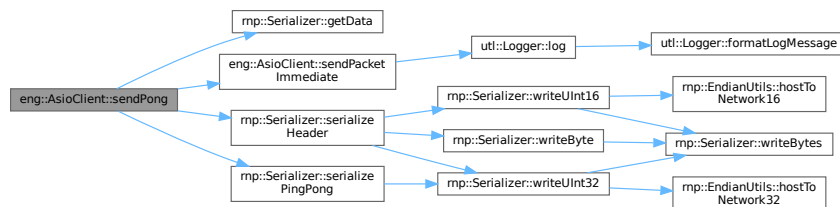
nonce	Ping nonce to echo back
-------	-------------------------

Definition at line 572 of file [asioClient.cpp](#).

References [rnp::Serializer::getData\(\)](#), [m_sessionId](#), [rnp::PacketPingPong::nonce](#), [rnp::PONG](#), [sendPacketImmediate\(\)](#), [rnp::Serializer::serializeHeader\(\)](#), [rnp::Serializer::serializePingPong\(\)](#), and [rnp::PacketHeader::type](#).

Referenced by [handlePing\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.12.3.40 sendToServer()

```
void eng::AsioClient::sendToServer (
    const std::vector< std::uint8_t > & data,
    bool reliable = false) [override], [virtual]
```

Send custom packet to server.

Parameters

data	Packet data
reliable	Whether packet should be reliable

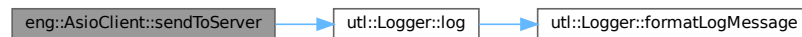
Implements [eng::INetworkClient](#).

Definition at line 194 of file [asioClient.cpp](#).

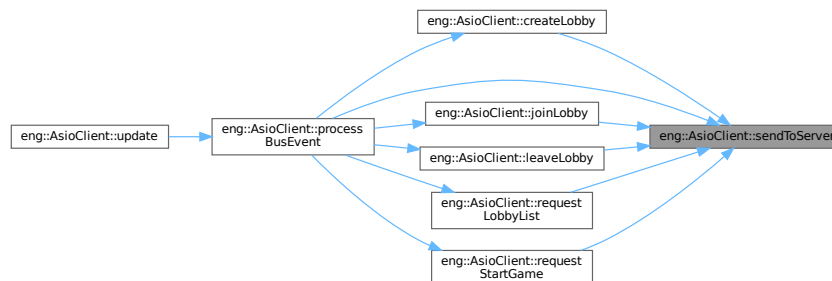
References [eng::CONNECTED](#), [utl::Logger::log\(\)](#), [m_connectionState](#), [m_sendQueue](#), [m_sendQueueMutex](#), and [utl::WARNING](#).

Referenced by [createLobby\(\)](#), [joinLobby\(\)](#), [leaveLobby\(\)](#), [processBusEvent\(\)](#), [requestLobbyList\(\)](#), and [requestStartGame\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.12.3.41 setClientCapabilities()

```
void eng::AsioClient::setClientCapabilities (
    std::uint32_t caps) [override], [virtual]
```

Implements [eng::INetworkClient](#).

Definition at line 188 of file [asioClient.cpp](#).

References [utl::INFO](#), [utl::Logger::log\(\)](#), and [m_clientCaps](#).

Here is the call graph for this function:



18.12.3.42 setOnGameStart()

```
void eng::AsioClient::setOnGameStart (
    std::function< void(std::uint32_t, std::uint32_t)> callback)
```

Set callback for game start.

Parameters

callback	Function to call when game starts
----------	-----------------------------------

Definition at line 963 of file [asioClient.cpp](#).

References [m_onGameStart](#).

18.12.3.43 setOnLobbyCreated()

```
void eng::AsioClient::setOnLobbyCreated (
    std::function< void(std::uint32_t, bool, rnp::ErrorCode)> callback)
```

Set callback for lobby created.

Parameters

callback	Function to call when lobby creation response is received
----------	---

Definition at line 947 of file [asioClient.cpp](#).

References [m_onLobbyCreated](#).

18.12.3.44 setOnLobbyJoined()

```
void eng::AsioClient::setOnLobbyJoined (
    std::function< void(std::uint32_t, bool, rnp::ErrorCode, const rnp::LobbyInfo *)> callback)
```

Set callback for lobby joined.

Parameters

callback	Function to call when lobby join response is received
----------	---

Definition at line 952 of file [asioClient.cpp](#).

References [m_onLobbyJoined](#).

18.12.3.45 setOnLobbyListReceived()

```
void eng::AsioClient::setOnLobbyListReceived (
    std::function< void(const std::vector< rnp::LobbyInfo > &)> callback)
```

Set callback for lobby list received.

Parameters

callback	Function to call when lobby list is received
----------	--

Definition at line 942 of file [asioClient.cpp](#).

References [m_onLobbyListReceived](#).

18.12.3.46 setOnLobbyUpdated()

```
void eng::AsioClient::setOnLobbyUpdated (
    std::function< void(const rnp::LobbyInfo &)> callback)
```

Set callback for lobby updated.

Parameters

callback	Function to call when lobby is updated
----------	--

Definition at line 958 of file [asioClient.cpp](#).

References [m_onLobbyUpdated](#).

18.12.3.47 setPlayerName()

void eng::AsioClient::setPlayerName (const std::string & playerName) [override], [virtual]

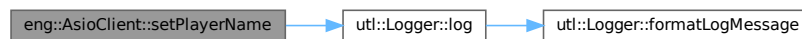
Implements [eng::INetworkClient](#).

Definition at line 182 of file [asioClient.cpp](#).

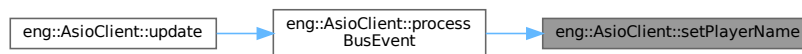
References [utl::INFO](#), [utl::Logger::log\(\)](#), and [m_playerName](#).

Referenced by [processBusEvent\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.12.3.48 setupPacketHandlers()

void eng::AsioClient::setupPacketHandlers () [private]

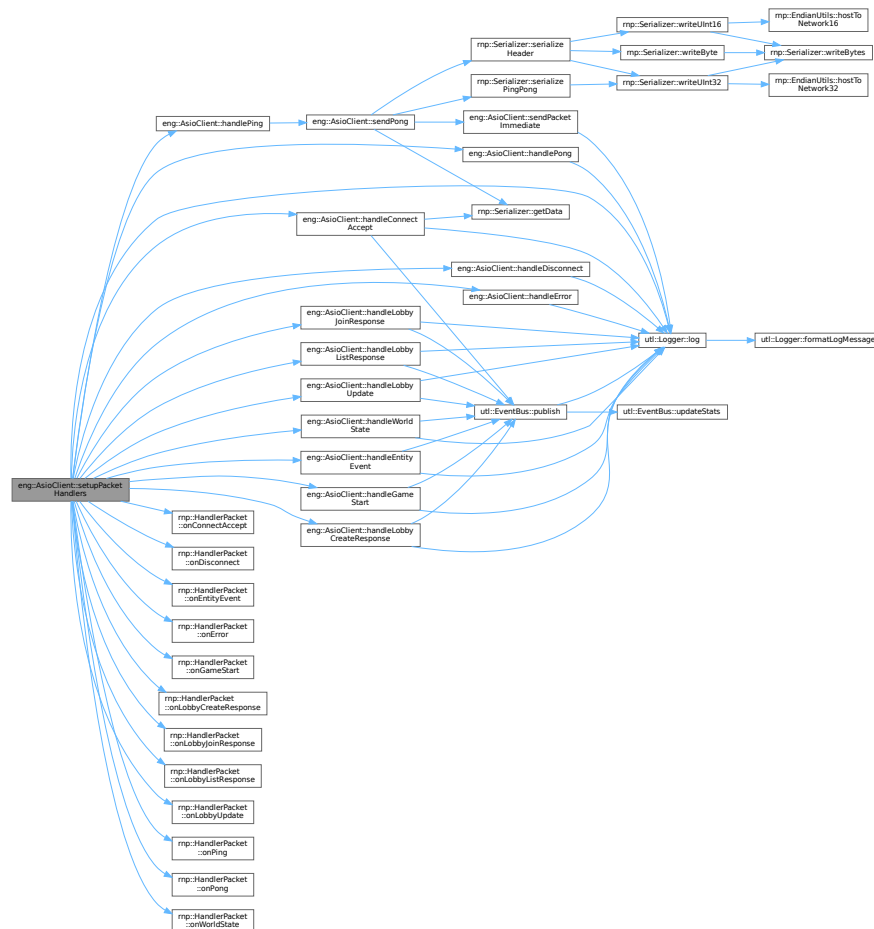
Initialize packet handlers.

Definition at line 206 of file [asioClient.cpp](#).

References [handleConnectAccept\(\)](#), [handleDisconnect\(\)](#), [handleEntityEvent\(\)](#), [handleError\(\)](#), [handleGameStart\(\)](#), [handleLobbyCreateResponse\(\)](#), [handleLobbyJoinResponse\(\)](#), [handleLobbyListResponse\(\)](#), [handleLobbyUpdate\(\)](#), [handlePing\(\)](#), [handlePong\(\)](#), [handleWorldState\(\)](#), [utl::INFO](#), [utl::Logger::log\(\)](#), [m_packetHandler](#), [rnp::HandlerPacket::onConnectAccept\(\)](#), [rnp::HandlerPacket::onDisconnect\(\)](#), [rnp::HandlerPacket::onEntityEvent\(\)](#), [rnp::HandlerPacket::onError\(\)](#), [rnp::HandlerPacket::onGameStart\(\)](#), [rnp::HandlerPacket::onLobbyCreateResponse\(\)](#), [rnp::HandlerPacket::onLobbyJoinResponse\(\)](#), [rnp::HandlerPacket::onLobbyListResponse\(\)](#), [rnp::HandlerPacket::onLobbyUpdate\(\)](#), [rnp::HandlerPacket::onPing\(\)](#), [rnp::HandlerPacket::onPong\(\)](#), and [rnp::HandlerPacket::onWorldState\(\)](#).

Referenced by [AsioClient\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.12.3.49 startReceive()

void eng::AsioClient::startReceive () [private]

Start receiving packets asynchronously.

Definition at line 264 of file [asioClient.cpp](#).

References [m_recvBuffer](#), [m_running](#), [m_senderEndpoint](#), and [m_socket](#).

Referenced by [connect\(\)](#).

Here is the caller graph for this function:



18.12.3.50 update()

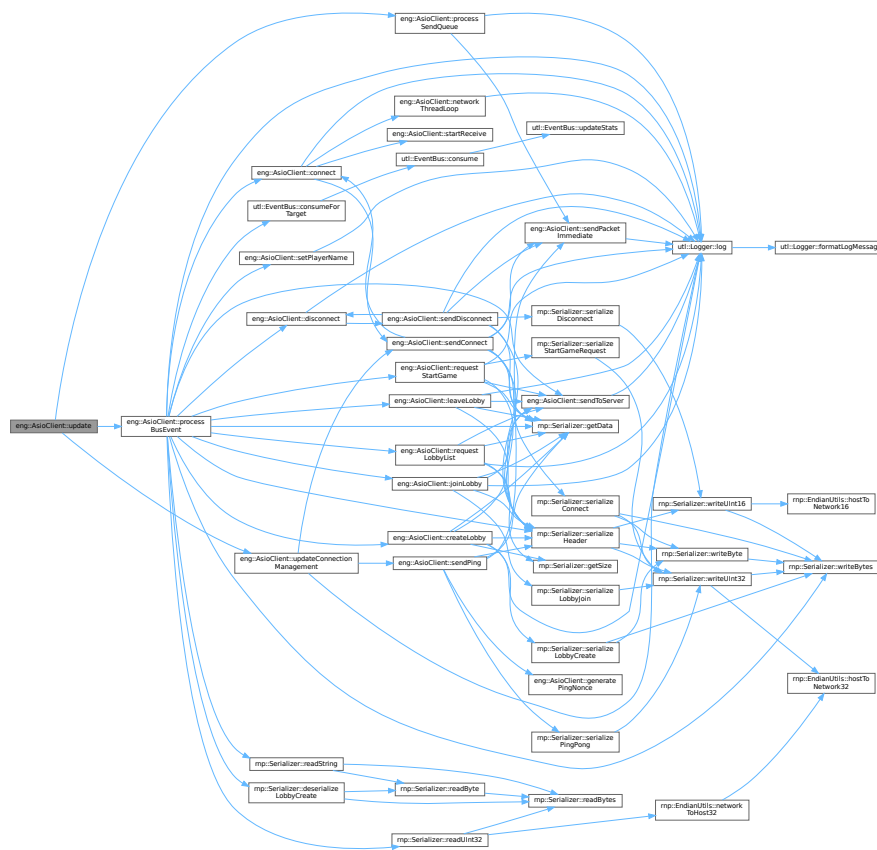
void eng::AsioClient::update () [override], [virtual]

Implements [eng::INetworkClient](#).

Definition at line 152 of file [asioClient.cpp](#).

References [eng::CONNECTED](#), [m_connectionState](#), [processBusEvent\(\)](#), [processSendQueue\(\)](#), and [updateConnectionManagement\(\)](#).

Here is the call graph for this function:



18.12.3.51 updateConnectionManagement()

void eng::AsioClient::updateConnectionManagement () [private]

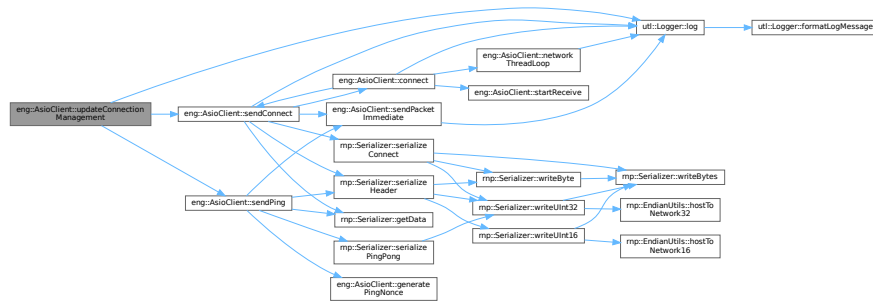
Update connection management (timeouts, pings)

Definition at line 614 of file [asioClient.cpp](#).

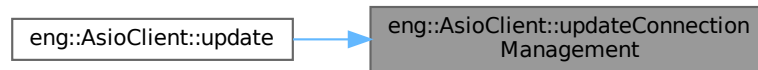
References [eng::CONNECTED](#), [eng::CONNECTING](#), [eng::DISCONNECTED](#), [utl::Logger::log\(\)](#), [m_connectionState](#), [m_connectionTimeout](#), [m_lastPingTime](#), [m_lastServerResponse](#), [m_pingInterval](#), [m_sessionId](#), [sendConnect\(\)](#), [sendPing\(\)](#), and [utl::WARNING](#).

Referenced by [update\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.12.4 Member Data Documentation

18.12.4.1 m_clientCaps

std::uint32_t eng::AsioClient::m_clientCaps [private]

Definition at line 194 of file [AsioClient.hpp](#).

Referenced by [sendConnect\(\)](#), and [setClientCapabilities\(\)](#).

18.12.4.2 m_componentId

std::uint32_t eng::AsioClient::m_componentId = [util::NETWORK_CLIENT](#) [private]

Definition at line 235 of file [AsioClient.hpp](#).

Referenced by [AsioClient\(\)](#), [handleConnectAccept\(\)](#), [handleEntityEvent\(\)](#), [handleGameStart\(\)](#), [handleLobbyCreateResponse\(\)](#), [handleLobbyJoinResponse\(\)](#), [handleLobbyListResponse\(\)](#), [handleLobbyUpdate\(\)](#), [handleWorldState\(\)](#), and [processBusEvent\(\)](#).

18.12.4.3 m_connectionState

std::atomic<[ConnectionState](#)> eng::AsioClient::m_connectionState [private]

Definition at line 191 of file [AsioClient.hpp](#).

Referenced by [connect\(\)](#), [createLobby\(\)](#), [disconnect\(\)](#), [getConnectionState\(\)](#), [handleConnectAccept\(\)](#), [handleDisconnect\(\)](#), [isConnected\(\)](#), [joinLobby\(\)](#), [leaveLobby\(\)](#), [processBusEvent\(\)](#), [requestLobbyList\(\)](#), [requestStartGame\(\)](#), [sendToServer\(\)](#), [update\(\)](#), and [updateConnectionManagement\(\)](#).

18.12.4.4 m_connectionTimeout

std::chrono::milliseconds eng::AsioClient::m_connectionTimeout [private]

Definition at line 219 of file [AsioClient.hpp](#).

Referenced by [updateConnectionManagement\(\)](#).

18.12.4.5 m_currentLobbyId

std::uint32_t eng::AsioClient::m_currentLobbyId [private]

Definition at line 225 of file [AsioClient.hpp](#).

Referenced by [createLobby\(\)](#), [getCurrentLobbyId\(\)](#), [handleLobbyCreateResponse\(\)](#), [handleLobbyJoinResponse\(\)](#), [isInLobby\(\)](#), [joinLobby\(\)](#), and [leaveLobby\(\)](#).

18.12.4.6 m_eventBus

[utl::EventBus](#)& eng::AsioClient::m_eventBus [private]

Definition at line 234 of file [AsioClient.hpp](#).

Referenced by [AsioClient\(\)](#), [handleConnectAccept\(\)](#), [handleEntityEvent\(\)](#), [handleGameStart\(\)](#), [handleLobbyCreateResponse\(\)](#), [handleLobbyJoinResponse\(\)](#), [handleLobbyListResponse\(\)](#), [handleLobbyUpdate\(\)](#), [handleWorldState\(\)](#), and [processBusEvent\(\)](#).

18.12.4.7 m_ioContext

std::unique_ptr<asio::io_context> eng::AsioClient::m_ioContext [private]

Definition at line 181 of file [AsioClient.hpp](#).

Referenced by [connect\(\)](#), [disconnect\(\)](#), and [networkThreadLoop\(\)](#).

18.12.4.8 m_lastPingNonce

std::uint32_t eng::AsioClient::m_lastPingNonce [private]

Definition at line 212 of file [AsioClient.hpp](#).

Referenced by [handlePong\(\)](#), and [sendPing\(\)](#).

18.12.4.9 m_lastPingTime

std::chrono::steady_clock::time_point eng::AsioClient::m_lastPingTime [private]

Definition at line 213 of file [AsioClient.hpp](#).

Referenced by [handlePong\(\)](#), [sendPing\(\)](#), and [updateConnectionManagement\(\)](#).

18.12.4.10 m_lastServerResponse

std::chrono::steady_clock::time_point eng::AsioClient::m_lastServerResponse [private]

Definition at line 218 of file [AsioClient.hpp](#).

Referenced by [connect\(\)](#), [handleReceive\(\)](#), and [updateConnectionManagement\(\)](#).

18.12.4.11 m_latency

std::uint32_t eng::AsioClient::m_latency [private]

Definition at line 214 of file [AsioClient.hpp](#).

Referenced by [disconnect\(\)](#), [getLatency\(\)](#), and [handlePong\(\)](#).

18.12.4.12 m_networkThread

std::unique_ptr<std::thread> eng::AsioClient::m_networkThread [private]

Definition at line 183 of file [AsioClient.hpp](#).

Referenced by [connect\(\)](#), and [disconnect\(\)](#).

18.12.4.13 m_onGameStart

std::function<void(std::uint32_t, std::uint32_t)> eng::AsioClient::m_onGameStart [private]

Definition at line 232 of file [AsioClient.hpp](#).

Referenced by [handleGameStart\(\)](#), and [setOnGameStart\(\)](#).

18.12.4.14 m_onLobbyCreated

std::function<void(std::uint32_t, bool, [rnp::ErrorCode](#))> eng::AsioClient::m_onLobbyCreated [private]

Definition at line 229 of file [AsioClient.hpp](#).

Referenced by [handleLobbyCreateResponse\(\)](#), and [setOnLobbyCreated\(\)](#).

18.12.4.15 `m_onLobbyJoined`

`std::function<void(std::uint32_t, bool, rnp::ErrorCode, const rnp::LobbyInfo *)> eng::AsioClient::m_onLobbyJoined` [private]

Definition at line 230 of file [AsioClient.hpp](#).

Referenced by [handleLobbyJoinResponse\(\)](#), and [setOnLobbyJoined\(\)](#).

18.12.4.16 `m_onLobbyListReceived`

`std::function<void(const std::vector<rnp::LobbyInfo> &)> eng::AsioClient::m_onLobbyListReceived` [private]

Definition at line 228 of file [AsioClient.hpp](#).

Referenced by [handleLobbyListResponse\(\)](#), and [setOnLobbyListReceived\(\)](#).

18.12.4.17 `m_onLobbyUpdated`

`std::function<void(const rnp::LobbyInfo &)> eng::AsioClient::m_onLobbyUpdated` [private]

Definition at line 231 of file [AsioClient.hpp](#).

Referenced by [handleLobbyUpdate\(\)](#), and [setOnLobbyUpdated\(\)](#).

18.12.4.18 `m_packetHandler`

`std::unique_ptr<rnp::HandlerPacket> eng::AsioClient::m_packetHandler` [private]

Definition at line 203 of file [AsioClient.hpp](#).

Referenced by [handleReceive\(\)](#), and [setupPacketHandlers\(\)](#).

18.12.4.19 `m_pingInterval`

`std::chrono::milliseconds eng::AsioClient::m_pingInterval` [private]

Definition at line 215 of file [AsioClient.hpp](#).

Referenced by [updateConnectionManagement\(\)](#).

18.12.4.20 `m_playerName`

`std::string eng::AsioClient::m_playerName` [private]

Definition at line 197 of file [AsioClient.hpp](#).

Referenced by [sendConnect\(\)](#), and [setPlayerName\(\)](#).

18.12.4.21 `m_recvBuffer`

`std::array<std::uint8_t, 1024> eng::AsioClient::m_recvBuffer` [private]

Definition at line 208 of file [AsioClient.hpp](#).

Referenced by [handleReceive\(\)](#), and [startReceive\(\)](#).

18.12.4.22 `m_running`

`std::atomic<bool> eng::AsioClient::m_running` [private]

Definition at line 200 of file [AsioClient.hpp](#).

Referenced by [connect\(\)](#), [disconnect\(\)](#), [networkThreadLoop\(\)](#), and [startReceive\(\)](#).

18.12.4.23 `m_senderEndpoint`

`asio::ip::udp::endpoint eng::AsioClient::m_senderEndpoint` [private]

Definition at line 209 of file [AsioClient.hpp](#).

Referenced by [handleReceive\(\)](#), and [startReceive\(\)](#).

18.12.4.24 `m_sendQueue`

`std::queue<QueuedPacket> eng::AsioClient::m_sendQueue` [private]

Definition at line 204 of file [AsioClient.hpp](#).

Referenced by [disconnect\(\)](#), [processSendQueue\(\)](#), and [sendToServer\(\)](#).

18.12.4.25 m_sendQueueMutex

std::mutex eng::AsioClient::m_sendQueueMutex [private]

Definition at line 205 of file [AsioClient.hpp](#).

Referenced by [disconnect\(\)](#), [processSendQueue\(\)](#), and [sendToServer\(\)](#).

18.12.4.26 m_serverEndpoint

asio::ip::udp::endpoint eng::AsioClient::m_serverEndpoint [private]

Definition at line 188 of file [AsioClient.hpp](#).

Referenced by [connect\(\)](#), and [sendPacketImmediate\(\)](#).

18.12.4.27 m_serverHost

std::string eng::AsioClient::m_serverHost [private]

Definition at line 186 of file [AsioClient.hpp](#).

Referenced by [connect\(\)](#).

18.12.4.28 m_serverPort

std::uint16_t eng::AsioClient::m_serverPort [private]

Definition at line 187 of file [AsioClient.hpp](#).

Referenced by [connect\(\)](#).

18.12.4.29 m_serverTickRate

std::uint16_t eng::AsioClient::m_serverTickRate [private]

Definition at line 193 of file [AsioClient.hpp](#).

Referenced by [disconnect\(\)](#), [getServerTickRate\(\)](#), and [handleConnectAccept\(\)](#).

18.12.4.30 m_sessionId

std::uint32_t eng::AsioClient::m_sessionId [private]

Definition at line 192 of file [AsioClient.hpp](#).

Referenced by [createLobby\(\)](#), [disconnect\(\)](#), [getSessionId\(\)](#), [handleConnectAccept\(\)](#), [handleDisconnect\(\)](#), [joinLobby\(\)](#), [leaveLobby\(\)](#), [processBusEvent\(\)](#), [requestLobbyList\(\)](#), [requestStartGame\(\)](#), [sendDisconnect\(\)](#), [sendPing\(\)](#), [sendPong\(\)](#), and [updateConnectionManagement\(\)](#).

18.12.4.31 m_socket

std::unique_ptr<asio::ip::udp::socket> eng::AsioClient::m_socket [private]

Definition at line 182 of file [AsioClient.hpp](#).

Referenced by [connect\(\)](#), [disconnect\(\)](#), [sendPacketImmediate\(\)](#), and [startReceive\(\)](#).

18.12.4.32 m_stats

[ConnectionStats](#) eng::AsioClient::m_stats [private]

Definition at line 222 of file [AsioClient.hpp](#).

Referenced by [AsioClient\(\)](#), [getStats\(\)](#), [handleReceive\(\)](#), and [sendPacketImmediate\(\)](#).

The documentation for this class was generated from the following files:

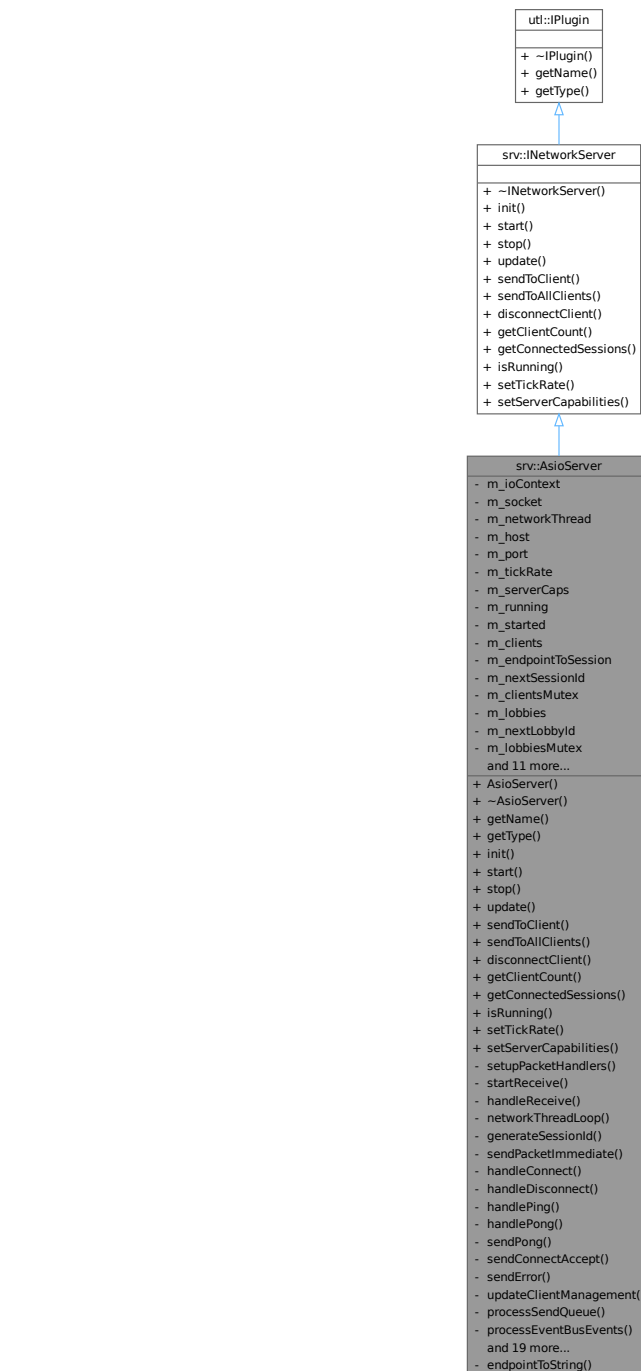
- [/home/masina/Projects/Epitech/rtype/plugins/Network/Asio/Client/include/AsioClient/AsioClient.hpp](#)
- [/home/masina/Projects/Epitech/rtype/plugins/Network/Asio/Client/src/asioClient.cpp](#)

18.13 srv::AsioServer Class Reference

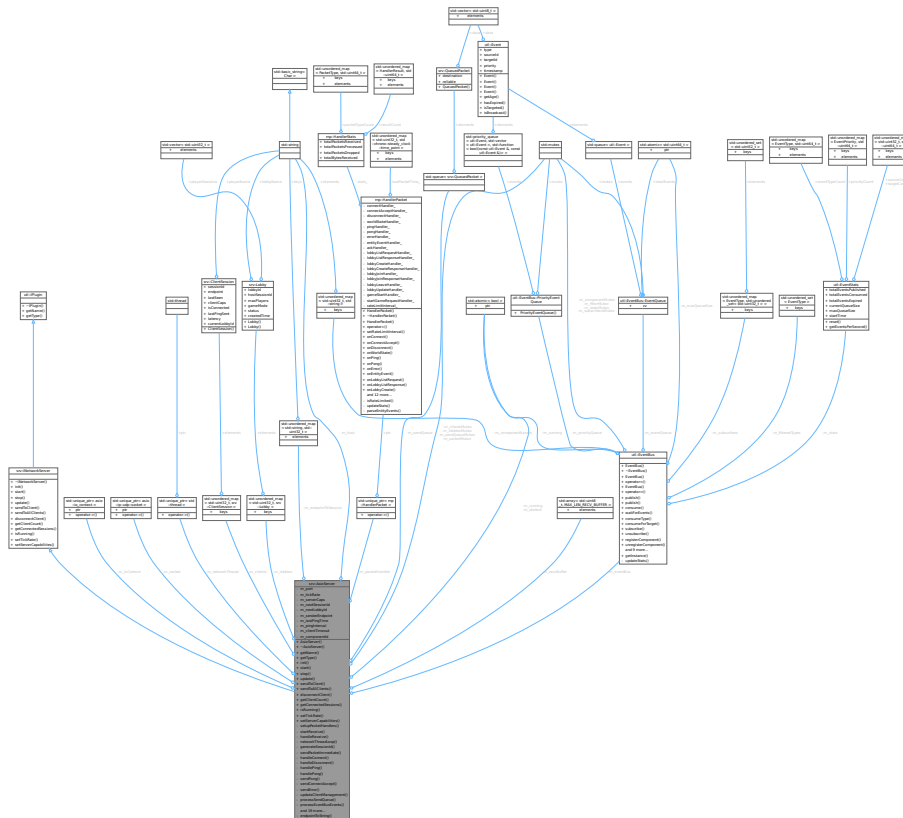
Asio UDP server implementation.

#include <AsioServer.hpp>

Inheritance diagram for `srv::AsioServer`:



Collaboration diagram for srv::AsioServer:



Public Member Functions

- [AsioServer](#) ()
Constructor.
- [~AsioServer](#) () override
Destructor.
- const std::string [getName](#) () const override
- [utl::PluginType](#) [getType](#) () const override
- void [init](#) (const std::string &host, std::uint16_t port) override
- void [start](#) () override
- void [stop](#) () override
- void [update](#) () override
- void [sendToClient](#) (std::uint32_t sessionId, const std::vector< std::uint8_t > &data, bool reliable=false) override
- void [sendToAllClients](#) (const std::vector< std::uint8_t > &data, bool reliable=false) override
- void [disconnectClient](#) (std::uint32_t sessionId) override
- std::size_t [getClientCount](#) () const override
- std::vector< std::uint32_t > [getConnectedSessions](#) () const override
- bool [isRunning](#) () const override
- void [setTickRate](#) (std::uint16_t tickRate) override
- void [setServerCapabilities](#) (std::uint32_t caps) override

Public Member Functions inherited from [srv::INetworkServer](#)

- virtual [~INetworkServer](#) ()=default
- virtual void [init](#) (const std::string &host, uint16_t port)=0

Public Member Functions inherited from [utl::IPlugin](#)

- virtual [~IPlugin](#) ()=default

Private Member Functions

- void [setupPacketHandlers](#) ()
Initialize packet handlers.
- void [startReceive](#) ()
Start receiving packets asynchronously.
- void [handleReceive](#) (std::size_t bytesReceived)
Handle received packet.
- void [networkThreadLoop](#) () const
Network thread main loop.
- std::uint32_t [generateSessionId](#) () const
Generate unique session ID.
- void [sendPacketImmediate](#) (const std::vector< std::uint8_t > &data, const asio::ip::udp::endpoint &destination)
Send packet immediately.
- [rnp::HandlerResult](#) [handleConnect](#) (const [rnp::PacketConnect](#) &packet, const [rnp::PacketContext](#) &context)
Handle CONNECT packet.
- [rnp::HandlerResult](#) [handleDisconnect](#) (const [rnp::PacketDisconnect](#) &packet, const [rnp::PacketContext](#) &context)
Handle DISCONNECT packet.
- [rnp::HandlerResult](#) [handlePing](#) (const [rnp::PacketPingPong](#) &packet, const [rnp::PacketContext](#) &context)
Handle PING packet.
- [rnp::HandlerResult](#) [handlePong](#) (const [rnp::PacketPingPong](#) &packet, const [rnp::PacketContext](#) &context)
Handle PONG packet.
- void [sendPong](#) (std::uint32_t nonce, const asio::ip::udp::endpoint &destination, std::uint32_t sessionId)
Send PONG response.
- void [sendConnectAccept](#) (std::uint32_t sessionId, const asio::ip::udp::endpoint &destination)
Send CONNECT_ACCEPT response.
- void [sendError](#) ([rnp::ErrorCode](#) errorCode, const std::string &description, const asio::ip::udp::endpoint &destination, std::uint32_t sessionId)
Send ERROR packet.
- void [updateClientManagement](#) ()
Update client timeouts and send pings.
- void [processSendQueue](#) ()
Process send queue.
- void [processEventBusEvents](#) ()
Process EventBus events for network operations.
- void [handleSendToClientEvent](#) (const [utl::Event](#) &event)
Handle send to client event from EventBus.
- void [handleBroadcastEvent](#) (const [utl::Event](#) &event)
Handle broadcast event from EventBus.
- void [handleSendEntityEventToClients](#) (const [utl::Event](#) &event)
Handle send entity event to clients from EventBus.
- [rnp::HandlerResult](#) [handleEntityEvent](#) (const std::vector< [rnp::EventRecord](#) > &events, const [rnp::PacketContext](#) &context) const

- Handle entity event packet (including player inputs)
- [rnp::HandlerResult handleLobbyListRequest](#) (const [rnp::PacketContext](#) &context)
 - Handle LOBBY_LIST_REQUEST packet.
- [rnp::HandlerResult handleLobbyCreate](#) (const [rnp::PacketLobbyCreate](#) &packet, const [rnp::PacketContext](#) &context)
 - Handle LOBBY_CREATE packet.
- [rnp::HandlerResult handleLobbyJoin](#) (const [rnp::PacketLobbyJoin](#) &packet, const [rnp::PacketContext](#) &context)
 - Handle LOBBY_JOIN packet.
- [rnp::HandlerResult handleLobbyLeave](#) (const [rnp::PacketContext](#) &context)
 - Handle LOBBY_LEAVE packet.
- void [sendLobbyList](#) (std::uint32_t sessionId)
 - Send lobby list to client.
- std::uint32_t [createLobby](#) (const std::string &name, std::uint32_t hostSession, std::uint8_t maxPlayers, std::uint8_t gameMode)
 - Create new lobby.
- [LobbyStatus joinLobby](#) (std::uint32_t lobbyId, std::uint32_t sessionId)
 - Join player to lobby.
- void [leaveLobby](#) (std::uint32_t sessionId)
 - Remove player from lobby.
- void [broadcastLobbyUpdate](#) (std::uint32_t lobbyId)
 - Broadcast lobby update to all lobby members.
- [rnp::HandlerResult handleStartGameRequest](#) (const [rnp::PacketStartGameRequest](#) &packet, const [rnp::PacketContext](#) &context)
 - Handle START_GAME_REQUEST packet.
- void [broadcastGameStart](#) (std::uint32_t lobbyId)
 - Broadcast game start to all players in lobby.
- [rnp::LobbyInfo lobbyToLobbyInfo](#) (const [Lobby](#) &lobby)
 - Convert [Lobby](#) to [LobbyInfo](#) for network transmission.
- void [sendLobbyCreateResponse](#) (std::uint32_t sessionId, std::uint32_t lobbyId, bool success, [rnp::ErrorCode](#) errorCode=[rnp::ErrorCode::INTERNAL_ERROR](#))
 - Send lobby create response.
- void [sendLobbyJoinResponse](#) (std::uint32_t sessionId, std::uint32_t lobbyId, bool success, [rnp::ErrorCode](#) errorCode, const [rnp::LobbyInfo](#) *lobbyInfo=nullptr)
 - Send lobby join response.
- void [cleanupEmptyLobbies](#) ()
 - Clean up empty lobbies.

Static Private Member Functions

- static std::string [endpointToString](#) (const asio::ip::udp::endpoint &endpoint)
 - Get endpoint string representation.

Private Attributes

- std::unique_ptr< asio::io_context > [m_ioContext](#)
- std::unique_ptr< asio::ip::udp::socket > [m_socket](#)
- std::unique_ptr< std::thread > [m_networkThread](#)
- std::string [m_host](#)
- std::uint16_t [m_port](#)
- std::uint16_t [m_tickRate](#)
- std::uint32_t [m_serverCaps](#)
- std::atomic< bool > [m_running](#)

- `std::atomic< bool > m_started`
- `std::unordered_map< std::uint32_t, ClientSession > m_clients`
- `std::unordered_map< std::string, std::uint32_t > m_endpointToSession`
- `std::uint32_t m_nextSessionId`
- `std::mutex m_clientsMutex`
- `std::unordered_map< std::uint32_t, Lobby > m_lobbies`
- `std::uint32_t m_nextLobbyId`
- `std::mutex m_lobbiesMutex`
- `std::unique_ptr< rnp::HandlerPacket > m_packetHandler`
- `std::queue< QueuedPacket > m_sendQueue`
- `std::mutex m_sendQueueMutex`
- `std::mutex m_socketMutex`
- `std::array< std::uint8_t, MAX_LEN_RECV_BUFFER > m_recvBuffer`
- `asio::ip::udp::endpoint m_senderEndpoint`
- `std::chrono::steady_clock::time_point m_lastPingTime`
- `std::chrono::milliseconds m_pingInterval`
- `std::chrono::milliseconds m_clientTimeout`
- `std::uint32_t m_componentId`
- `utl::EventBus & m_eventBus`

18.13.1 Detailed Description

Asio UDP server implementation.

Definition at line 105 of file [AsioServer.hpp](#).

18.13.2 Constructor & Destructor Documentation

18.13.2.1 AsioServer()

`srv::AsioServer::AsioServer ()`

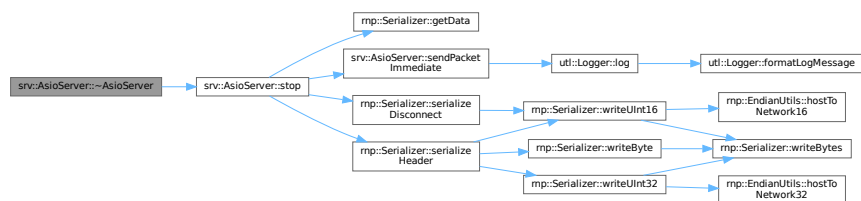
Constructor.

Definition at line 21 of file [asioServer.cpp](#).

References [utl::BROADCAST_WORLD_STATE](#), [m_componentId](#), [m_eventBus](#), [utl::EventBus::registerComponent\(\)](#), [utl::SEND_ENTITY_EVENTS](#), [utl::SEND_TO_CLIENT](#), [setupPacketHandlers\(\)](#), and [utl::EventBus::subscribe\(\)](#).

[illegible]

srv::~asioServer::~asioServer () [override]
Destructor.
Definition at line 43 of file [asioServer.cpp](#).
References [stop\(\)](#).
Here is the call graph for this function:



18.13.3.1 broadcastGameStart()

Generated by Doxygen

Parameters

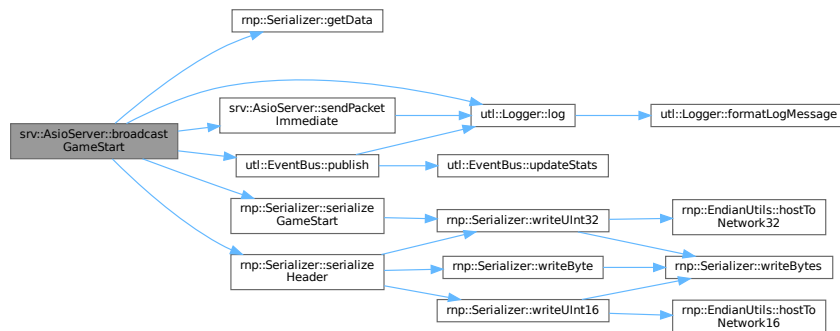
lobbyId	Target lobby ID
---------	-----------------

Definition at line 1544 of file asioServer.cpp.

References [utl::GAME_LOGIC](#), [rnp::GAME_START](#), [utl::GAME_START](#), [rnp::Serializer::getData\(\)](#), [utl::INFO](#), [rnp::PacketHeader::length](#), [rnp::PacketGameStart::lobbyId](#), [utl::Logger::log\(\)](#), [m_clients](#), [m_clientsMutex](#), [m_eventBus](#), [m_lobbies](#), [m_lobbiesMutex](#), [utl::NETWORK_SERVER](#), [utl::EventBus::publish\(\)](#), [sendPacketImmediate\(\)](#), [rnp::Serializer::serializeGameStart\(\)](#), [rnp::Serializer::serializeHeader\(\)](#), [rnp::PacketHeader::sessionId](#) and [rnp::PacketHeader::type](#).

Referenced by [handleStartGameRequest\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.13.3.2 broadcastLobbyUpdate()

```
void srv::AsioServer::broadcastLobbyUpdate (
    std::uint32_t lobbyId) [private]
```

Broadcast lobby update to all lobby members.

Parameters

lobbyId	Target lobby ID
---------	-----------------

Definition at line 1312 of file asioServer.cpp.

References [rnp::Serializer::getData\(\)](#), [utl::INFO](#), [rnp::PacketHeader::length](#), [rnp::LOBBY_UPDATE](#), [rnp::PacketLobbyUpdate::lobbyInfo](#), [lobbyToLobbyInfo\(\)](#), [utl::Logger::log\(\)](#), [m_clients](#), [m_clientsMutex](#), [m_lobbies](#), [m_lobbiesMutex](#), [sendPacketImmediate\(\)](#), [rnp::Serializer::serializeHeader\(\)](#), [rnp::Serializer::serializeLobbyUpdate\(\)](#), [rnp::PacketHeader::sessionId](#), [rnp::PacketHeader::type](#), and [update\(\)](#).

Referenced by [disconnectClient\(\)](#), [handleLobbyCreate\(\)](#), [handleLobbyJoin\(\)](#), and [handleLobbyLeave\(\)](#).

[illegible]

```

sequenceDiagram
    participant AS as srv::AsioServer::AsioServer
    participant U as srv::AsioServer::update
    participant UCM as srv::AsioServer::updateClient Management
    participant DC as srv::AsioServer::disconnect Client
    participant H as srv::AsioServer::handleLobby Leave
    participant B as srv::AsioServer::broadcast LobbyUpdate
    participant SH as srv::AsioServer::setupPacket Handlers
    participant CL as srv::AsioServer::handleLobby Create
    participant HLJ as srv::AsioServer::handleLobbyJoin

    AS --> U
    U --> UCM
    UCM --> DC
    DC --> B
    H --> B
    AS --> SH
    SH --> CL
    SH --> HLJ
    CL --> B
    HLJ --> B
    B --> U
  
```

void srv::asioServer::cleanupEmptyLobbies () [private]
Clean up empty lobbies.
Definition at line 1443 of file [asioServer.cpp](#).
References [util::INFO](#), [util::Logger::log\(\)](#), and [m_lobbies](#).
Referenced by [leaveLobby\(\)](#).

```

graph LR
    A["srv::AsioServer::cleanup  
EmptyLobbies"] --> B["utl::Logger::log"]
    B --> C["utl::Logger::formatLogMessage"]

```

```

graph LR
    A[svc.AssServer.AssServer] --> B[svc.AssServer.updatePhysical  
Handlers]
    B --> C[svc.AssServer.handleLiberty  
Create]
    B --> D[svc.AssServer.handleLiberty  
Join]
    C --> E[svc.AssServer.handleLiberty  
Update]
    D --> E
    E --> F[svc.AssServer.handleLiberty  
Update]
    F --> G[svc.AssServer.updateClient  
Management]
    F --> H[svc.AssServer.updateClient]
    G --> I[svc.AssServer.updateClient]
    I --> J[svc.AssServer.handleLiberty  
EmptyLiberty]

```

18.13.3.4 createLobby()

```
std::uint32_t srv::AsioServer::createLobby (
    const std::string & name,
    std::uint32_t hostSession,
    std::uint8_t maxPlayers,
    std::uint8_t gameMode) [private]
```

Create new lobby.

Parameters

name	Lobby name
hostSession	Host session ID
maxPlayers	Maximum players
gameMode	Game mode

Returns

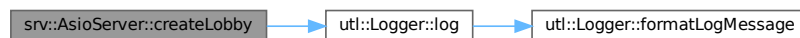
New lobby ID

Definition at line 1209 of file [asioServer.cpp](#).

References [utl::INFO](#), [utl::Logger::log\(\)](#), [m_lobbies](#), [m_lobbiesMutex](#), [m_nextLobbyId](#), and [srv::Lobby::playerSessions](#).

Referenced by [handleLobbyCreate\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.13.3.5 disconnectClient()

```
void srv::AsioServer::disconnectClient (
    std::uint32_t sessionId) [override], [virtual]
```

Implements [srv::INetworkServer](#).

Definition at line 250 of file [asioServer.cpp](#).

References [broadcastLobbyUpdate\(\)](#), [rnp::HandlerPacket::clearSession\(\)](#), [rnp::CLIENT_REQUEST](#), [rnp::DISCONNECT](#), [endpointToString\(\)](#), [rnp::Serializer::getData\(\)](#), [utl::INFO](#), [leaveLobby\(\)](#), [rnp::PacketHeader::length](#), [utl::Logger::log\(\)](#), [m_clients](#), [m_clientsMutex](#), [m_endpointToSession](#), [m_packetHandler](#), [rnp::PacketDisconnect::reasonC](#), [sendPacketImmediate\(\)](#), [rnp::Serializer::serializeDisconnect\(\)](#), [rnp::Serializer::serializeHeader\(\)](#), [rnp::PacketHeader::session](#) and [rnp::PacketHeader::type](#).

Referenced by [updateClientManagement\(\)](#).

```

sequenceDiagram
    participant A as svr:asioServer:asioServer
    participant B as svr:asioServer:setupPacketHandlers
    participant C1 as svr:asioServer:handleLobbyCreate
    participant C2 as svr:asioServer:handleLobbyJoin
    participant C3 as svr:asioServer:handleLobbyLeave
    participant D as svr:asioServer:broadcastLobbyUpdate
    participant E as svr:asioServer:update
    participant F as svr:asioServer:updateClientManagement
    participant G as svr:asioServer:disconnectClient

    A->>B
    B->>C1
    B->>C2
    B->>C3
    C1->>D
    C2->>D
    C3->>D
    D->>E
    E->>F
    F->>G
  
```

```
std::string srv::AsioServer::endpointToString (
    const asio::ip::udp::endpoint & endpoint)  [static], [private]
Get endpoint string representation.
```

endpoint	UDP endpoint
----------	--------------

String representation

```

sequenceDiagram
    participant A as svc-AioServer:AioServer
    participant B as svc-AioServer:setupPacketHandlers
    participant C as svc-AioServer:handleLibbyCreate
    participant D as svc-AioServer:handleLibbyJoin
    participant E as svc-AioServer:handleLibbyLeave
    participant F as svc-AioServer:broadcastLibbyUpdate
    participant G as svc-AioServer:update
    participant H as svc-AioServer:updateClientManagement
    participant I as svc-AioServer:disconnectClient
    participant J as svc-AioServer:handleDisconnect
    participant K as svc-AioServer:handleConnect
    participant L as svc-AioServer:disconnectClient

    A->>B
    B->>C
    B->>D
    B->>E
    D->>F
    F->>G
    G->>H
    H->>I
    E->>J
    J->>K
    K->>L
    L->>L
  
```

```
std::uint32_t srv::AsioServer::generateSessionId () const [private]
Generate unique session ID.
```

Returns

New session ID

Definition at line 510 of file [asioServer.cpp](#).

References [m_clients](#).

Referenced by [handleConnect\(\)](#).

Here is the caller graph for this function:



18.13.3.8 getClientCount()

`std::size_t srv::AsioServer::getClientCount () const` [nodiscard], [override], [virtual]

Implements [srv::INetworkServer](#).

Definition at line 307 of file [asioServer.cpp](#).

References [m_clients](#), and [m_clientsMutex](#).

18.13.3.9 getConnectedSessions()

`std::vector< std::uint32_t > srv::AsioServer::getConnectedSessions () const` [nodiscard], [override], [virtual]

Implements [srv::INetworkServer](#).

Definition at line 313 of file [asioServer.cpp](#).

References [m_clients](#), and [m_clientsMutex](#).

18.13.3.10 getName()

`const std::string srv::AsioServer::getName () const` [inline], [nodiscard], [override], [virtual]

Implements [utl::IPlugin](#).

Definition at line 119 of file [AsioServer.hpp](#).

18.13.3.11 getType()

[utl::PluginType](#) `srv::AsioServer::getType () const` [inline], [nodiscard], [override], [virtual]

Implements [utl::IPlugin](#).

Definition at line 120 of file [AsioServer.hpp](#).

References [utl::NETWORK_CLIENT](#).

18.13.3.12 handleBroadcastEvent()

`void srv::AsioServer::handleBroadcastEvent (`
 `const utl::Event & event)` [private]

Handle broadcast event from EventBus.

Parameters

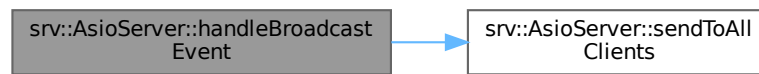
event	Broadcast event
-------	-----------------

Definition at line 899 of file [asioServer.cpp](#).

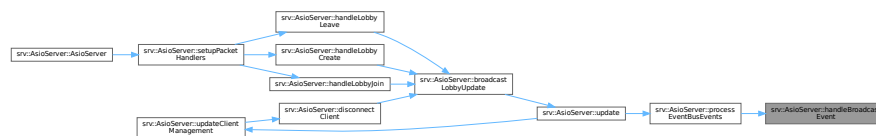
References [utl::Event::data](#), and [sendToAllClients\(\)](#).

Referenced by [processEventBusEvents\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.13.3.13 handleConnect()

```

rnp::HandlerResult srv::AsioServer::handleConnect (
    const rnp::PacketConnect & packet,
    const rnp::PacketContext & context) [private]
  
```

Handle CONNECT packet.

Parameters

packet	Connect packet
context	Packet context

Returns

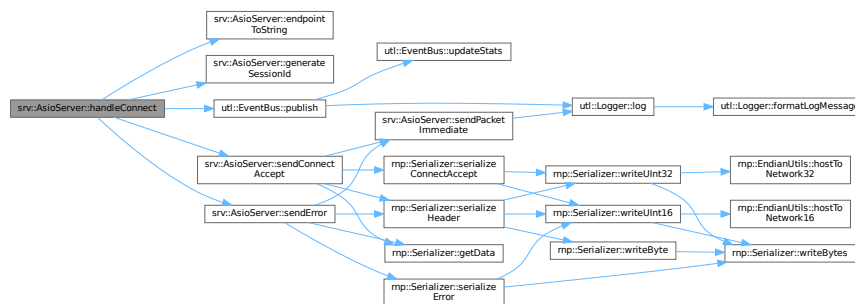
Handler result

Definition at line 562 of file asioServer.cpp.

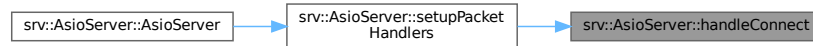
References [rnp::PacketConnect::clientCaps](#), [srv::ClientSession::clientCaps](#), [srv::ClientSession::endpoint](#), [endpointToString\(\)](#), [utl::GAME_LOGIC](#), [generateSessionId\(\)](#), [srv::ClientSession::isConnected](#), [srv::ClientSession::lastSeen](#), [m_clients](#), [m_clientsMutex](#), [m_componentId](#), [m_endpointToSession](#), [m_eventBus](#), [m_senderEndpoint](#), [srv::MAX_CLIENTS](#), [rnp::PacketConnect::nameLen](#), [utl::PLAYER_CONNECTED](#), [rnp::PacketConnect::playerName](#), [srv::ClientSession::playerName](#), [utl::EventBus::publish\(\)](#), [rnp::RATE_LIMITED](#), [rnp::PacketContext::receiveTime](#), [sendConnectAccept\(\)](#), [sendError\(\)](#), [srv::ClientSession::sessionId](#), and [rnp::SUCCESS](#).

Referenced by [setupPacketHandlers\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.13.3.14 handleDisconnect()

```

rnp::HandlerResult srv::AsioServer::handleDisconnect (
    const rnp::PacketDisconnect & packet,
    const rnp::PacketContext & context) [private]

```

Handle DISCONNECT packet.

Parameters

packet	Disconnect packet
context	Packet context

Returns

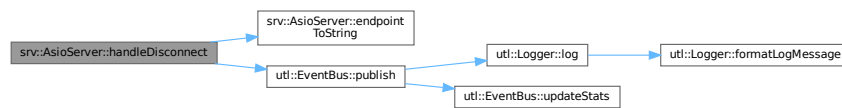
Handler result

Definition at line 636 of file asioServer.cpp.

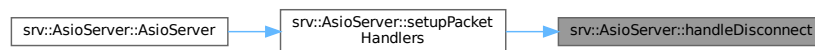
References [endpointToString\(\)](#), [utl::GAME_LOGIC](#), [m_clients](#), [m_clientsMutex](#), [m_componentId](#), [m_endpointToSession](#), [m_eventBus](#), [utl::PLAYER_DISCONNECTED](#), [utl::EventBus::publish\(\)](#), [rnp::PacketContext::sessionId](#), and [rnp::SUCCESS](#).

Referenced by [setupPacketHandlers\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.13.3.15 handleEntityEvent()

```

rnp::HandlerResult srv::AsioServer::handleEntityEvent (
    const std::vector< rnp::EventRecord > & events,
    const rnp::PacketContext & context) const [private]

```

Handle entity event packet (including player inputs)

Parameters

events	Entity event records
context	Packet context

Returns

Handler result

Definition at line 919 of file [asioServer.cpp](#).

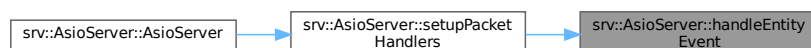
References [utl::ENTITY_EVENT_RECEIVED](#), [utl::GAME_LOGIC](#), [utl::INFO](#), [rnp::INPUT](#), [utl::Logger::log\(\)](#), [m_componentId](#), [m_eventBus](#), [utl::PLAYER_INPUT_RECEIVED](#), [utl::EventBus::publish\(\)](#), [rnp::PacketContext::sessionId](#), and [rnp::SUCCESS](#).

Referenced by [setupPacketHandlers\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:

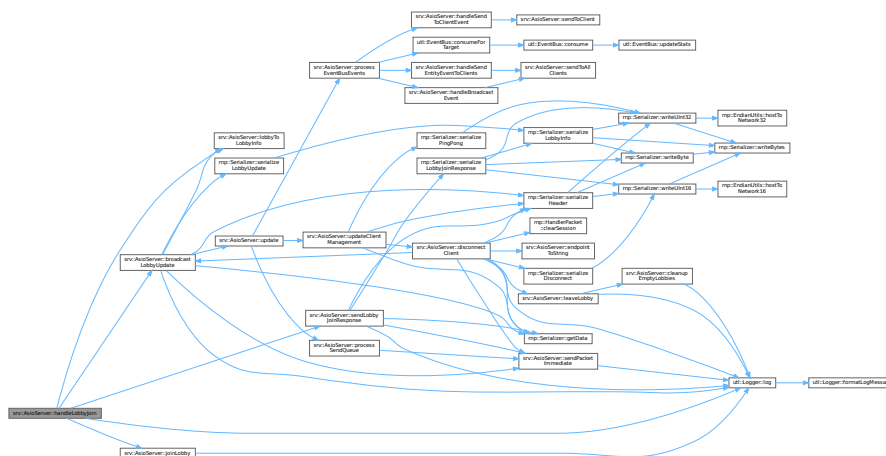


packet	Lobby join packet
context	Packet context

Handler result

References `rnpg::ALREADY_IN_LOBBY`, `broadcastLobbyUpdate()`, `rnpg::LobbyInfo::currentPlayers`, `utl::INFO`, `joinLobby()`, `rnpg::LOBBY_NOT_FOUND`, `rnpg::PacketLobbyJoin::lobbyId`, `lobbyToLobbyInfo()`, `utl::Logger::log()`, `m_clients`, `m_clientsMutex`, `m_lobbies`, `m_lobbiesMutex`, `rnpg::LobbyInfo::maxPlayers`, `rnpg::NONE`, `rnpg::PROCESSING_ERROR`, `sendLobbyJoinResponse()`, `rnpg::PacketContext::sessionId`, `rnpg::SUCCESS`, `srv::SUCCESS`, `rnpg::UNAUTHORIZED_SESSION`, and `utl::WARNING`.

Here is the call graph for this function:



```

graph LR
    A[srv::AsioServer::AsioServer] --> B[srv::AsioServer::setupPacket  
Handlers]
    B --> C[srv::AsioServer::handleLobbyJoin]

```

```

np::HandlerResult srv::AsioServer::handleLobbyLeave (
    const np::PacketContext & context) [private]
Handle LOBBY_LEAVE packet.

```

context	Packet context
---------	----------------

Returns

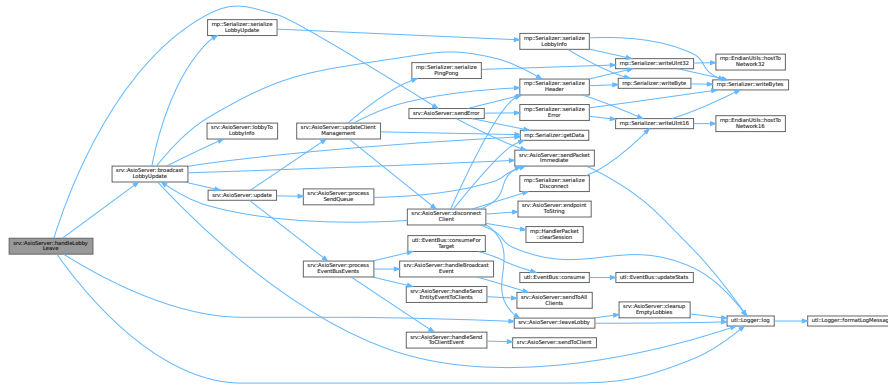
Handler result

Definition at line 1129 of file asioServer.cpp.

References [broadcastLobbyUpdate\(\)](#), [utl::INFO](#), [leaveLobby\(\)](#), [utl::Logger::log\(\)](#), [m_clients](#), [m_clientsMutex](#), [m_senderEndpoint](#), [rnp::NOT_IN_LOBBY](#), [rnp::PROCESSING_ERROR](#), [sendError\(\)](#), [rnp::PacketContext::sessionId](#), [rnp::SUCCESS](#), and [rnp::UNAUTHORIZED_SESSION](#).

Referenced by [setupPacketHandlers\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.13.3.19 handleLobbyListRequest()

`rnp::HandlerResult` `srv::AsioServer::handleLobbyListRequest` (
 const `rnp::PacketContext` & context) [private]
Handle LOBBY_LIST_REQUEST packet.

Parameters

context	Packet context
---------	----------------

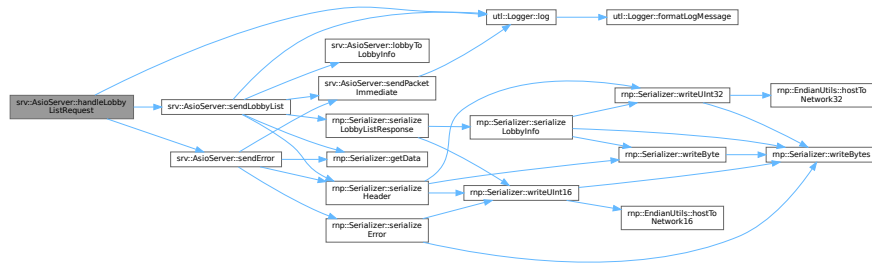
Returns

Handler result

Definition at line 960 of file asioServer.cpp.

References [utl::INFO](#), [utl::Logger::log\(\)](#), [m_clients](#), [m_clientsMutex](#), [m_senderEndpoint](#), [rnp::PROCESSING_ERROR](#), [sendError\(\)](#), [sendLobbyList\(\)](#), [rnp::PacketContext::sessionId](#), [rnp::SUCCESS](#), and [rnp::UNAUTHORIZED_SESSION](#).
Referenced by [setupPacketHandlers\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.13.3.20 handlePing()

```

rnp::HandlerResult srv::AsioServer::handlePing (
    const rnp::PacketPingPong & packet,
    const rnp::PacketContext & context) [private]

```

Handle PING packet.

Parameters

packet	Ping packet
context	Packet context

Returns

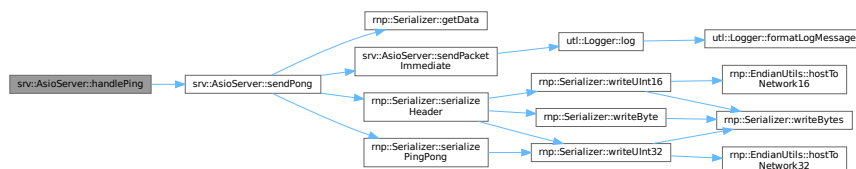
Handler result

Definition at line 679 of file [asioServer.cpp](#).

References [m_clients](#), [m_clientsMutex](#), [m_senderEndpoint](#), [rnp::PacketPingPong::nonce](#), [rnp::PacketContext::receiveTime](#), [sendPong\(\)](#), [rnp::PacketContext::sessionId](#), and [rnp::SUCCESS](#).

Referenced by [setupPacketHandlers\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.13.3.21 handlePong()

`rnp::HandlerResult` `srv::AsioServer::handlePong` (
 const `rnp::PacketPingPong` & packet,
 const `rnp::PacketContext` & context) [private]
 Handle PONG packet.

Parameters

packet	Pong packet
context	Packet context

Returns

Handler result

Definition at line 696 of file `asioServer.cpp`.

References `m_clients`, `m_clientsMutex`, `rnp::PacketContext::receiveTime`, `rnp::PacketPingPong::sendTimeMs`, `rnp::PacketContext::sessionId`, and `rnp::SUCCESS`.

Referenced by `setupPacketHandlers()`.

Here is the caller graph for this function:



18.13.3.22 handleReceive()

`void` `srv::AsioServer::handleReceive` (
 std::size_t bytesReceived) [private]
 Handle received packet.

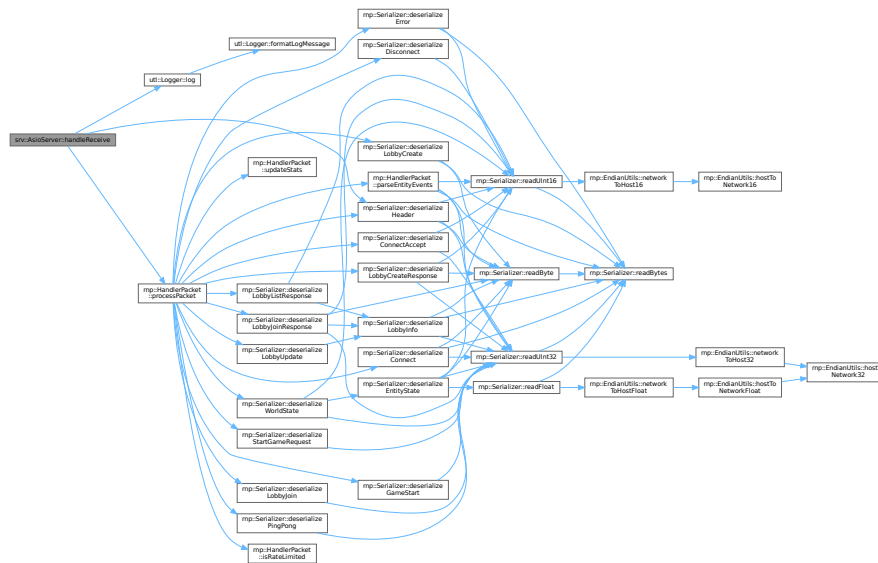
Parameters

bytesReceived	Number of bytes received
---------------	--------------------------

Definition at line 424 of file `asioServer.cpp`.

References `rnp::Serializer::deserializeHeader()`, `utl::INFO`, `utl::Logger::log()`, `m_packetHandler`, `m_recvBuffer`, `m_senderEndpoint`, `rnp::HandlerPacket::processPacket()`, `rnp::PacketContext::receiveTime`, `rnp::PacketContext::senderAddress`, `rnp::PacketContext::senderPort`, `rnp::PacketContext::sessionId`, `rnp::PacketHeader::sessionId`, `rnp::SUCCESS`, `rnp::PacketHeader::type`, and `utl::WARNING`.

Here is the call graph for this function:



18.13.3.23 handleSendEntityEventToClients()

```
void srv::AsioServer::handleSendEntityEventToClients (
    const utl::Event & event) [private]
Handle send entity event to clients from EventBus.
```

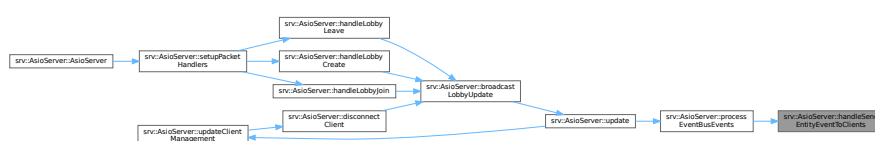
Parameters

event	Entity event to send to clients
-------	---------------------------------

Definition at line 909 of file [asioServer.cpp](#).
References [utl::Event::data](#), and [sendToAllClients\(\)](#).
Referenced by [processEventBusEvents\(\)](#).
Here is the call graph for this function:



Here is the caller graph for this function:



18.13.3.24 handleSendToClientEvent()

```
void srv::AsioServer::handleSendToClientEvent (
    const utl::Event & event) [private]
Handle send to client event from EventBus.
```

Parameters

event	Send to client event
-------	----------------------

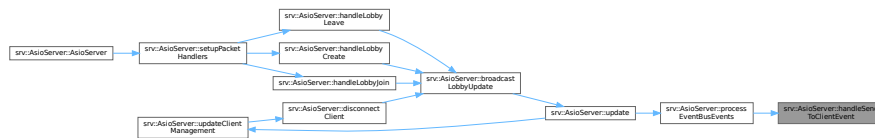
Definition at line 880 of file asioServer.cpp.

References [utl::Event::data](#), and [sendToClient\(\)](#).Referenced by [processEventBusEvents\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.13.3.25 handleStartGameRequest()

```
rnp::HandlerResult srv::AsioServer::handleStartGameRequest (
    const rnp::PacketStartGameRequest & packet,
    const rnp::PacketContext & context) [private]
Handle START_GAME_REQUEST packet.
```

Parameters

packet	Start game request packet
context	Packet context

Returns

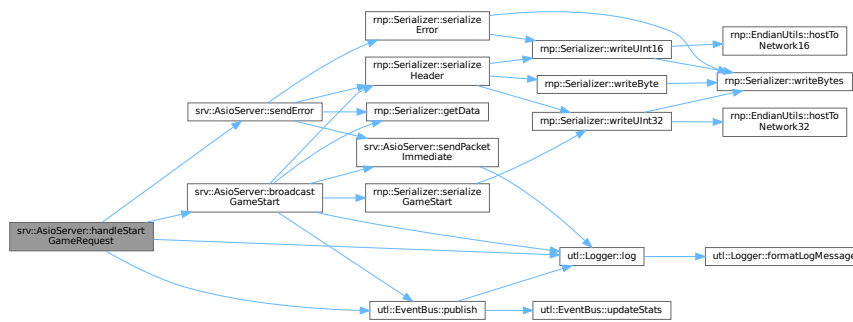
Handler result

Definition at line 1462 of file asioServer.cpp.

References [broadcastGameStart\(\)](#), [utl::Event::data](#), [utl::GAME_LOGIC](#), [rnp::IN_GAME](#), [utl::INFO](#), [rnp::LOBBY_NOT_FOUND](#), [rnp::PacketStartGameRequest::lobbyId](#), [utl::Logger::log\(\)](#), [m_clients](#), [m_clientsMutex](#), [m_componentId](#), [m_eventBus](#), [m_lobbies](#), [m_lobbiesMutex](#), [m_senderEndpoint](#), [rnp::NOT_IN_LOBBY](#), [rnp::PROCESSING_ERROR](#), [utl::EventBus::publish\(\)](#), [sendError\(\)](#), [utl::SERVER_START](#), [rnp::PacketContext::sessionId](#), [rnp::SUCCESS](#), and [rnp::UNAUTHORIZED_SESSION](#).

Referenced by [setupPacketHandlers\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.13.3.26 init()

```
void srv::AsioServer::init (
    const std::string & host,
    std::uint16_t port) [override]
```

Definition at line 50 of file [asioServer.cpp](#).

References [m_host](#), and [m_port](#).

18.13.3.27 isRunning()

```
bool srv::AsioServer::isRunning () const [nodiscard], [override], [virtual]
```

Implements [srv::INetworkServer](#).

Definition at line 327 of file [asioServer.cpp](#).

References [m_running](#).

18.13.3.28 joinLobby()

```
LobbyStatus srv::AsioServer::joinLobby (
    std::uint32_t lobbyId,
    std::uint32_t sessionId) [private]
```

Join player to lobby.

Parameters

lobbyId	Target lobby ID
sessionId	Player session ID

Returns

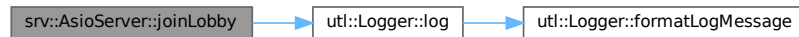
Success status

Definition at line 1227 of file `asioServer.cpp`.

References `srv::ALREADY_IN_LOBBY`, `srv::FULL`, `srv::IN_GAME`, `util::Logger::log()`, `m_lobbies`, `m_lobbiesMutex`, `srv::Lobby::maxPlayers`, `srv::NOT_FOUND`, `srv::Lobby::playerSessions`, `srv::Lobby::status`, `srv::SUCCESS`, `np::WAITING`, and `util::WARNING`.

Referenced by `handleLobbyJoin()`.

Here is the call graph for this function:



Here is the caller graph for this function:



18.13.3.29 leaveLobby()

```
void srv::AsioServer::leaveLobby (
    std::uint32_t sessionId) [private]
```

Remove player from lobby.

Parameters

sessionId	Player session ID
-----------	-------------------

Definition at line 1273 of file `asioServer.cpp`.

References `cleanupEmptyLobbies()`, `util::INFO`, `util::Logger::log()`, `m_lobbies`, and `m_lobbiesMutex`.

Referenced by `disconnectClient()`, and `handleLobbyLeave()`.

Here is the call graph for this function:



Here is the caller graph for this function:



18.13.3.30 lobbyToLobbyInfo()

`rnp::LobbyInfo` `srv::AsioServer::lobbyToLobbyInfo (`
 const `Lobby` & lobby) [private]

Convert `Lobby` to `LobbyInfo` for network transmission.

Parameters

lobby	<code>Lobby</code> structure
-------	------------------------------

Returns

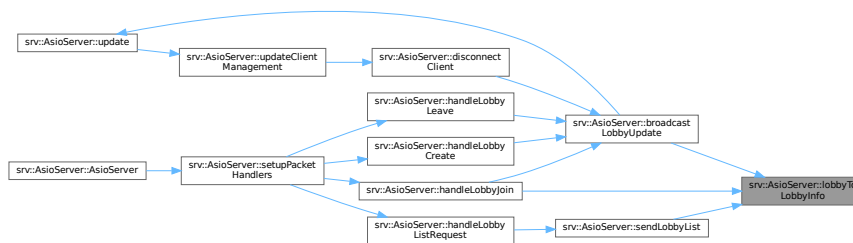
`LobbyInfo` structure

Definition at line 1358 of file `asioServer.cpp`.

References `rnp::LobbyInfo::currentPlayers`, `rnp::LobbyInfo::gameMode`, `srv::Lobby::gameMode`, `rnp::LobbyInfo::hostSessionId`, `srv::Lobby::hostSessionId`, `rnp::LobbyInfo::lobbyId`, `srv::Lobby::lobbyId`, `rnp::LobbyInfo::lobbyName`, `srv::Lobby::lobbyName`, `m_clients`, `m_clientsMutex`, `rnp::LobbyInfo::maxPlayers`, `srv::Lobby::maxPlayers`, `rnp::LobbyInfo::playerNames`, `srv::Lobby::playerSessions`, `rnp::LobbyInfo::status`, and `srv::Lobby::status`.

Referenced by `broadcastLobbyUpdate()`, `handleLobbyJoin()`, and `sendLobbyList()`.

Here is the caller graph for this function:



18.13.3.31 networkThreadLoop()

`void` `srv::AsioServer::networkThreadLoop ()` const [private]

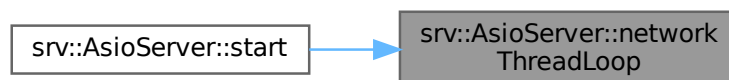
Network thread main loop.

Definition at line 475 of file `asioServer.cpp`.

References `m_ioContext`, and `m_running`.

Referenced by `start()`.

Here is the caller graph for this function:



18.13.3.32 processEventBusEvents()

`void` `srv::AsioServer::processEventBusEvents ()` [private]

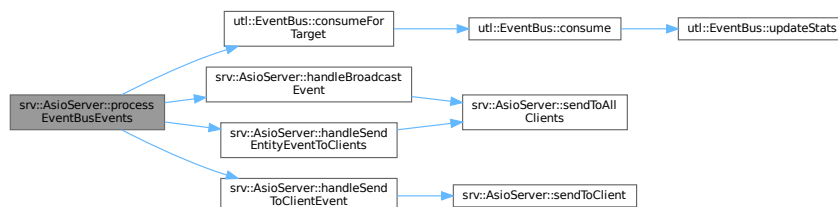
Process EventBus events for network operations.

Definition at line 850 of file [asioServer.cpp](#).

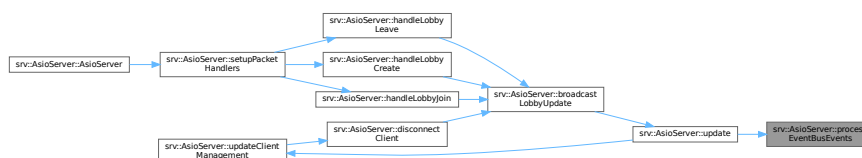
References [utl::BROADCAST_WORLD_STATE](#), [utl::EventBus::consumeForTarget\(\)](#), [handleBroadcastEvent\(\)](#), [handleSendEntityEventToClients\(\)](#), [handleSendToClientEvent\(\)](#), [m_componentId](#), [m_eventBus](#), [utl::SEND_ENTITY_EVENTS](#), and [utl::SEND_TO_CLIENT](#).

Referenced by [update\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.13.3.33 processSendQueue()

```
void srv::AsioServer::processSendQueue () [private]
```

Process send queue.

Definition at line 839 of file [asioServer.cpp](#).

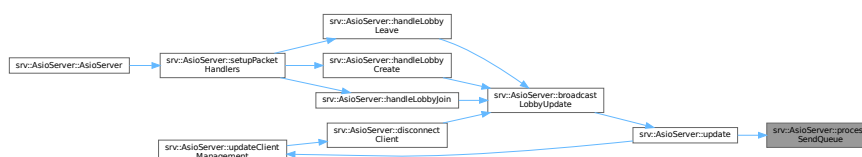
References [srv::QueuedPacket::data](#), [srv::QueuedPacket::destination](#), [m_sendQueue](#), [m_sendQueueMutex](#), and [sendPacketImmediate\(\)](#).

Referenced by [update\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.13.3.34 sendConnectAccept()

```
void srv::AsioServer::sendConnectAccept (
    std::uint32_t sessionId,
    const asio::ip::udp::endpoint & destination) [private]
Send CONNECT_ACCEPT response.
```

Parameters

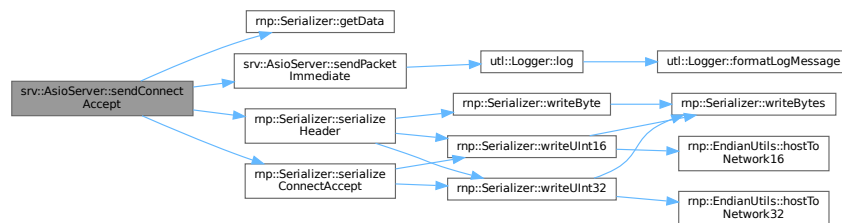
sessionId	New session ID
destination	Target endpoint

Definition at line 736 of file [asioServer.cpp](#).

References [rnp::CONNECT_ACCEPT](#), [rnp::Serializer::getData\(\)](#), [m_serverCaps](#), [m_tickRate](#), [rnp::MAX_PAYLOAD](#), [sendPacketImmediate\(\)](#), [rnp::Serializer::serializeConnectAccept\(\)](#), [rnp::Serializer::serializeHeader\(\)](#), [rnp::PacketConnectAccept::sessionId](#), and [rnp::PacketHeader::type](#).

Referenced by [handleConnect\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.13.3.35 sendError()

```
void srv::AsioServer::sendError (
    rnp::ErrorCode errorCode,
    const std::string & description,
    const asio::ip::udp::endpoint & destination,
    std::uint32_t sessionId) [private]
Send ERROR packet.
```

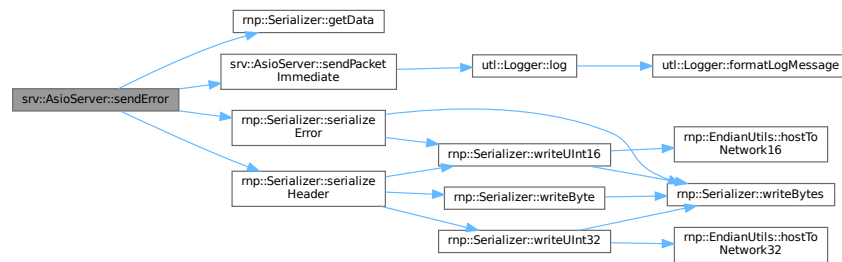
Parameters

errorCode	Error code
description	Error description
destination	Target endpoint
sessionId	Session ID

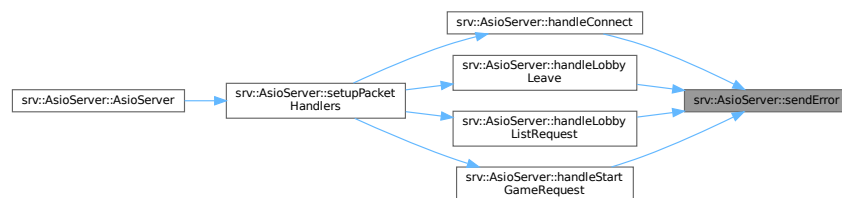
Definition at line 764 of file [asioServer.cpp](#).

References [rnp::PacketError::errorCode](#), [rnp::Serializer::getData\(\)](#), [rnp::PACKET_ERROR](#), [sendPacketImmediate\(\)](#), [rnp::Serializer::serializeError\(\)](#), [rnp::Serializer::serializeHeader\(\)](#), and [rnp::PacketHeader::type](#).

Referenced by [handleConnect\(\)](#), [handleLobbyLeave\(\)](#), [handleLobbyListRequest\(\)](#), and [handleStartGameRequest\(\)](#).
Here is the call graph for this function:



Here is the caller graph for this function:



18.13.3.36 sendLobbyCreateResponse()

```

void srv::AsioServer::sendLobbyCreateResponse (
    std::uint32_t sessionId,
    std::uint32_t lobbyId,
    bool success,
    rnp::ErrorCode errorCode = rnp::ErrorCode::INTERNAL_ERROR) [private]
  
```

Send lobby create response.

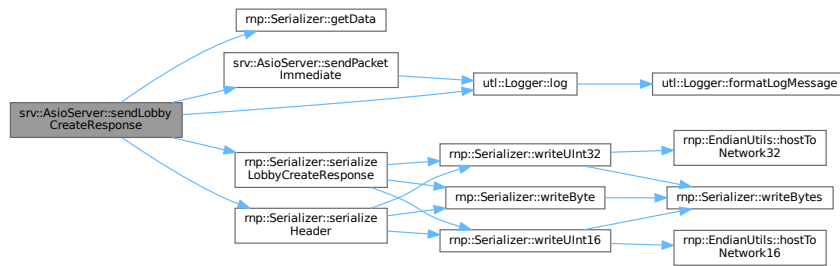
Parameters

sessionId	Target client session ID
lobbyId	Created lobby ID (0 if failed)
success	Success status
errorCode	Error code if failed

Definition at line 1389 of file [asioServer.cpp](#).

References [rnp::PacketLobbyCreateResponse::errorCode](#), [rnp::Serializer::getData\(\)](#), [utl::INFO](#), [rnp::PacketHeader::length](#), [rnp::LOBBY_CREATE_RESPONSE](#), [rnp::PacketLobbyCreateResponse::lobbyId](#), [utl::Logger::log\(\)](#), [m_senderEndpoint](#), [sendPacketImmediate\(\)](#), [rnp::Serializer::serializeHeader\(\)](#), [rnp::Serializer::serializeLobbyCreateResponse\(\)](#), [rnp::PacketHeader::sessionId](#), [rnp::PacketLobbyCreateResponse::success](#), and [rnp::PacketHeader::type](#).
Referenced by [handleLobbyCreate\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.13.3.37 sendLobbyJoinResponse()

```

void srv::AsioServer::sendLobbyJoinResponse (
    std::uint32_t sessionId,
    std::uint32_t lobbyId,
    bool success,
    rnp::ErrorCode errorCode,
    const rnp::LobbyInfo * lobbyInfo = nullptr) [private]

```

Send lobby join response.

Parameters

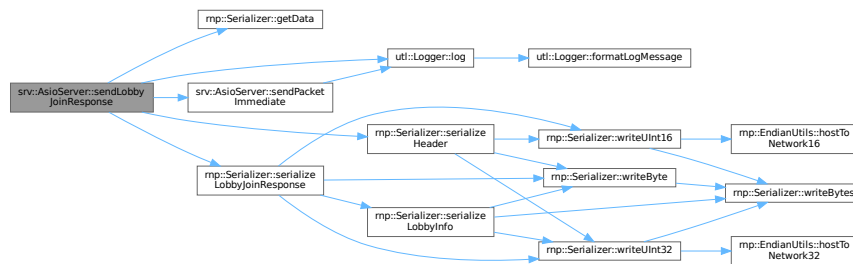
sessionId	Target client session ID
lobbyId	Target lobby ID
success	Success status
errorCode	Error code if failed
lobbyInfo	Lobby information if successful

Definition at line 1414 of file [asioServer.cpp](#).

References [rnp::PacketLobbyJoinResponse::errorCode](#), [rnp::Serializer::getData\(\)](#), [utl::INFO](#), [rnp::PacketHeader::length](#), [rnp::LOBBY_JOIN_RESPONSE](#), [rnp::PacketLobbyJoinResponse::lobbyId](#), [rnp::PacketLobbyJoinResponse::lobbyInfo](#), [utl::Logger::log\(\)](#), [m_senderEndpoint](#), [sendPacketImmediate\(\)](#), [rnp::Serializer::serializeHeader\(\)](#), [rnp::Serializer::serializeLobbyJoinResponse\(\)](#), [rnp::PacketHeader::sessionId](#), [rnp::PacketLobbyJoinResponse::success](#), and [rnp::PacketHeader::type](#).

Referenced by [handleLobbyJoin\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.13.3.38 sendLobbyList()

```
void srv::asioServer::sendLobbyList (
    std::uint32_t sessionId) [private]
Send lobby list to client.
```

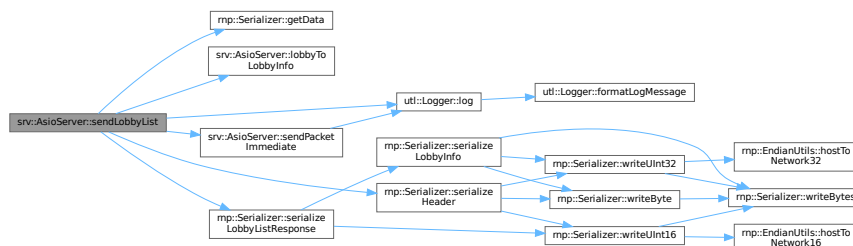
Parameters

sessionId	Target client session ID
-----------	--------------------------

Definition at line 1169 of file [asioServer.cpp](#).

References [rnp::LobbyInfo::currentPlayers](#), [rnp::Serializer::getData\(\)](#), [utl::INFO](#), [rnp::PacketHeader::length](#), [rnp::PacketLobbyListResponse::lobbies](#), [rnp::LOBBY_LIST_RESPONSE](#), [rnp::PacketLobbyListResponse::lobbyCount](#), [rnp::LobbyInfo::lobbyId](#), [lobbyToLobbyInfo\(\)](#), [utl::Logger::log\(\)](#), [m_lobbies](#), [m_lobbiesMutex](#), [m_senderEndpoint](#), [rnp::LobbyInfo::maxPlayers](#), [sendPacketImmediate\(\)](#), [rnp::Serializer::serializeHeader\(\)](#), [rnp::Serializer::serializeLobbyListResponse\(\)](#), [rnp::PacketHeader::sessionId](#), and [rnp::PacketHeader::type](#).
 Referenced by [handleLobbyListRequest\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.13.3.39 sendPacketImmediate()

```
void srv::AsioServer::sendPacketImmediate (
    const std::vector< std::uint8_t > & data,
    const asio::ip::udp::endpoint & destination) [private]
```

Send packet immediately.

Parameters

data	Packet data
destination	Target endpoint

Definition at line 530 of file [asioServer.cpp](#).

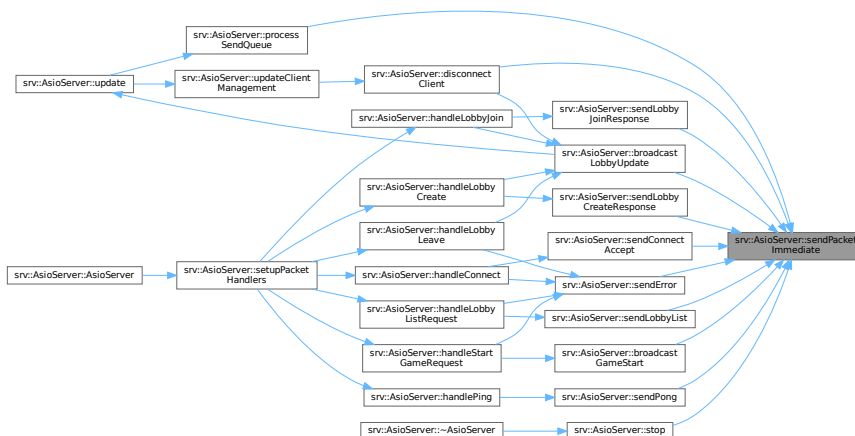
References [utl::INFO](#), [utl::Logger::log\(\)](#), [m_running](#), [m_socket](#), [m_socketMutex](#), and [utl::WARNING](#).

Referenced by [broadcastGameStart\(\)](#), [broadcastLobbyUpdate\(\)](#), [disconnectClient\(\)](#), [processSendQueue\(\)](#), [sendConnectAccept\(\)](#), [sendError\(\)](#), [sendLobbyCreateResponse\(\)](#), [sendLobbyJoinResponse\(\)](#), [sendLobbyList\(\)](#), [sendPong\(\)](#), and [stop\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.13.3.40 sendPong()

```
void srv::AsioServer::sendPong (
    std::uint32_t nonce,
    const asio::ip::udp::endpoint & destination,
    std::uint32_t sessionId) [private]
```

Send PONG response.

Parameters

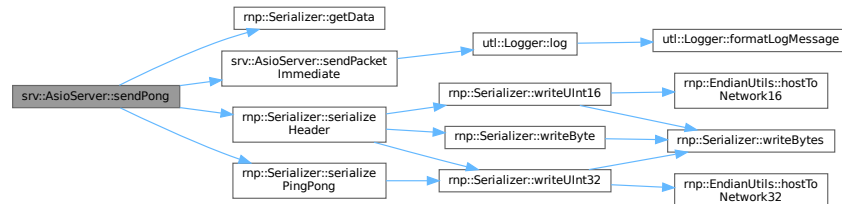
nonce	Ping nonce
destination	Target endpoint
sessionId	Client session ID

Definition at line 716 of file [asioServer.cpp](#).

References [rnp::Serializer::getData\(\)](#), [rnp::PacketPingPong::nonce](#), [rnp::PONG](#), [sendPacketImmediate\(\)](#), [rnp::Serializer::serializeHeader\(\)](#), [rnp::Serializer::serializePingPong\(\)](#), and [rnp::PacketHeader::type](#).

Referenced by [handlePing\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.13.3.41 sendToAllClients()

```
void srv::AsioServer::sendToAllClients (
    const std::vector< std::uint8_t > & data,
    bool reliable = false) [override], [virtual]
```

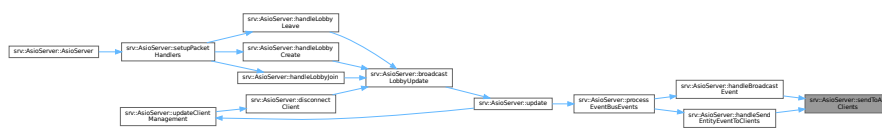
Implements [srv::INetworkServer](#).

Definition at line 236 of file [asioServer.cpp](#).

References [m_clients](#), [m_clientsMutex](#), [m_sendQueue](#), and [m_sendQueueMutex](#).

Referenced by [handleBroadcastEvent\(\)](#), and [handleSendEntityEventToClients\(\)](#).

Here is the caller graph for this function:




```
void srv::asioServer::sendToClient (
    std::uint32_t sessionId,
    const std::vector< std::uint8_t > & data,
    bool reliable = false) [override], [virtual]
```

Definition at line 221 of file asioServer.cpp.

Referenced by [handleSendToClientEvent\(\)](#).

```

graph LR
    A[srv::AsioServer::AsioServer] --> B[srv::AsioServer::setUpSocketHandlers]
    B --> C[srv::AsioServer::handshakeLobbyLeave]
    B --> D[srv::AsioServer::handshakeLobbyCreate]
    B --> E[srv::AsioServer::handshakeLobbyJoin]
    C --> F[srv::AsioServer::broadcastLobbyUpdate]
    D --> F
    E --> G[srv::AsioServer::disconnectClient]
    F --> H[srv::AsioServer::update]
    G --> H
    I[srv::AsioServer::updateClientManagement] --> J[srv::AsioServer::updateClient]
    J --> K[srv::AsioServer::processEventAndEvents]
    K --> L[srv::AsioServer::handshakeToServer]
    L --> M[srv::AsioServer::sendToClient]
  
```

```
void srv::AsioServer::setServerCapabilities (
    std::uint32_t caps) [override], [virtual]
```

Definition at line 335 of file asioServer.cpp.

```
void srv::AsioServer::setTickRate (
    std::uint16_t tickRate) [override], [virtual]
```

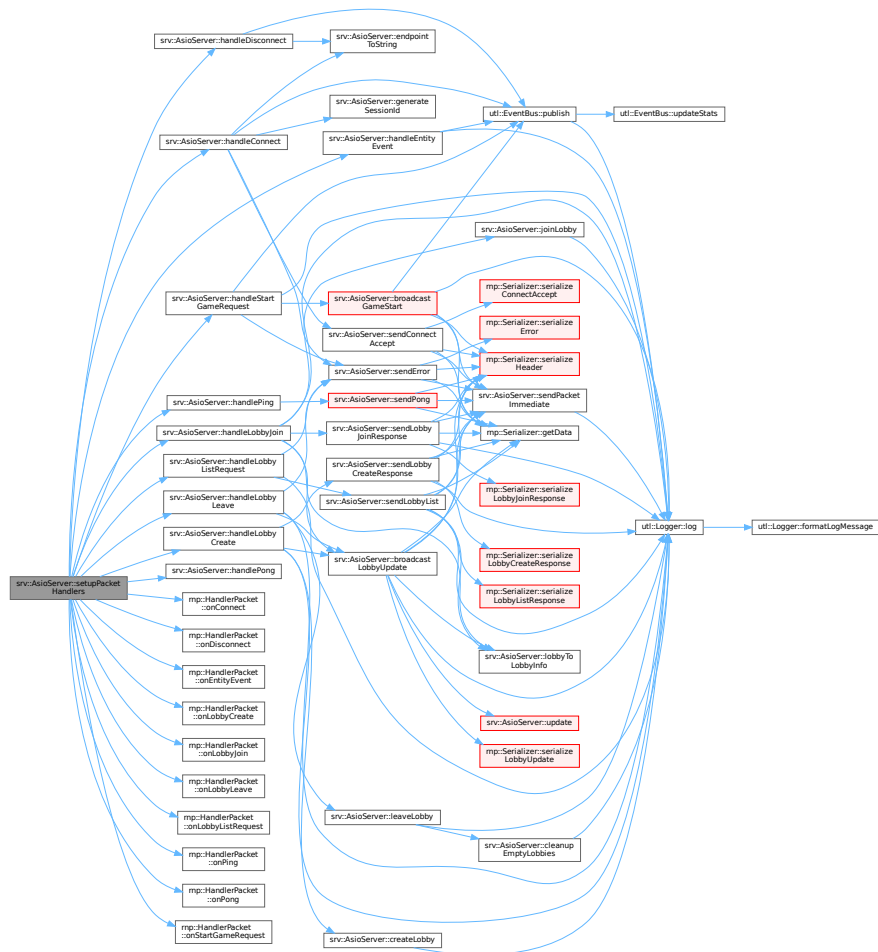
Definition at line 329 of file asioServer.cpp.

```
void srv::AsioServer::setupPacketHandlers () [private]
Initialize packet handlers.
```

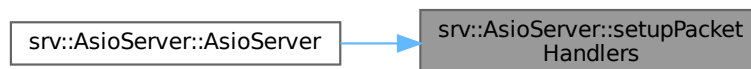
Definition at line 341 of file asioServer.cpp.

References [handleConnect\(\)](#), [handleDisconnect\(\)](#), [handleEntityEvent\(\)](#), [handleLobbyCreate\(\)](#), [handleLobbyJoin\(\)](#), [handleLobbyLeave\(\)](#), [handleLobbyListRequest\(\)](#), [handlePing\(\)](#), [handlePong\(\)](#), [handleStartGameRequest\(\)](#), [m_packetHandler](#), [rnp::HandlerPacket::onConnect\(\)](#), [rnp::HandlerPacket::onDisconnect\(\)](#), [rnp::HandlerPacket::onEntityEvent\(\)](#), [rnp::HandlerPacket::onLobbyCreate\(\)](#), [rnp::HandlerPacket::onLobbyJoin\(\)](#), [rnp::HandlerPacket::onLobbyLeave\(\)](#), [rnp::HandlerPacket::onLobbyListRequest\(\)](#), [rnp::HandlerPacket::onPing\(\)](#), [rnp::HandlerPacket::onPong\(\)](#), and [rnp::HandlerPacket::onStartGameRequest\(\)](#).
Referenced by [AsioServer\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.13.3.46 start()

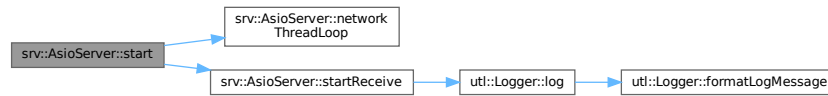
void srv::AsioServer::start () [override], [virtual]

Implements [srv::INetworkServer](#).

Definition at line 59 of file [asioServer.cpp](#).

References [m_host](#), [m_networkThread](#), [m_port](#), [m_running](#), [m_socket](#), [m_started](#), [networkThreadLoop\(\)](#), and [startReceive\(\)](#).

Here is the call graph for this function:



18.13.3.47 startReceive()

void srv::AsioServer::startReceive () [private]

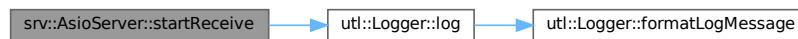
Start receiving packets asynchronously.

Definition at line 387 of file [asioServer.cpp](#).

References [utl::Logger::log\(\)](#), [m_recvBuffer](#), [m_running](#), [m_senderEndpoint](#), [m_socket](#), and [utl::WARNING](#).

Referenced by [start\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.13.3.48 stop()

void srv::AsioServer::stop () [override], [virtual]

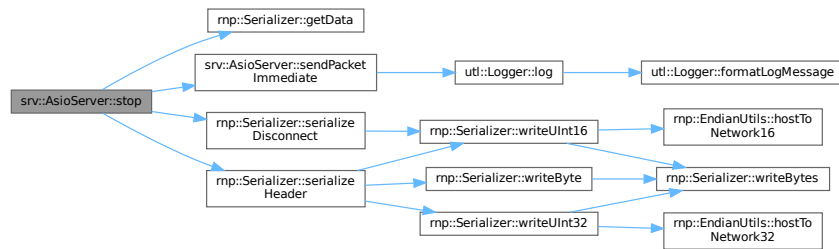
Implements [srv::INetworkServer](#).

Definition at line 109 of file [asioServer.cpp](#).

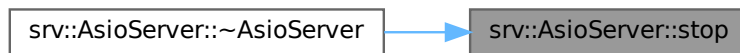
References [rnp::DISCONNECT](#), [rnp::Serializer::getData\(\)](#), [m_clients](#), [m_clientsMutex](#), [m_endpointToSession](#), [m_ioContext](#), [m_networkThread](#), [m_running](#), [m_socket](#), [m_socketMutex](#), [m_started](#), [rnp::PacketDisconnect::reasonCode](#), [sendPacketImmediate\(\)](#), [rnp::Serializer::serializeDisconnect\(\)](#), [rnp::Serializer::serializeHeader\(\)](#), [rnp::SERVER_SHUTDOWN](#), and [rnp::PacketHeader::type](#).

Referenced by [~AsioServer\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.13.3.49 update()

void srv::AsioServer::update () [override], [virtual]

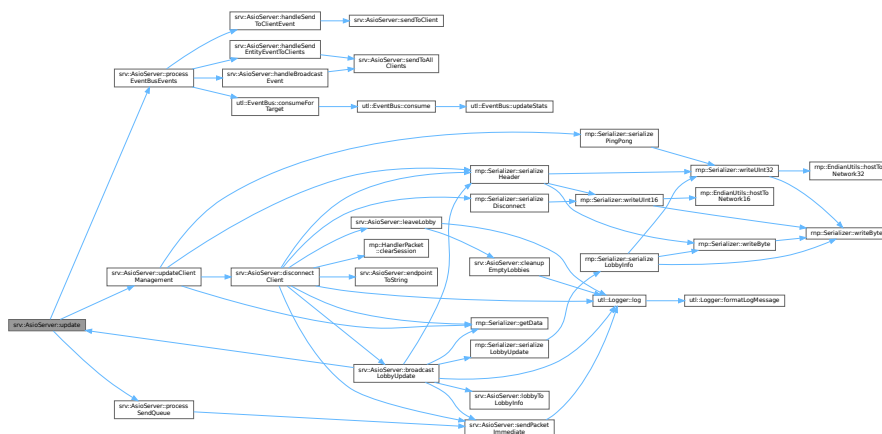
Implements [srv::INetworkServer](#).

Definition at line 205 of file [asioServer.cpp](#).

References [m_running](#), [processEventBusEvents\(\)](#), [processSendQueue\(\)](#), and [updateClientManagement\(\)](#).

Referenced by [broadcastLobbyUpdate\(\)](#).

Here is the call graph for this function:



```

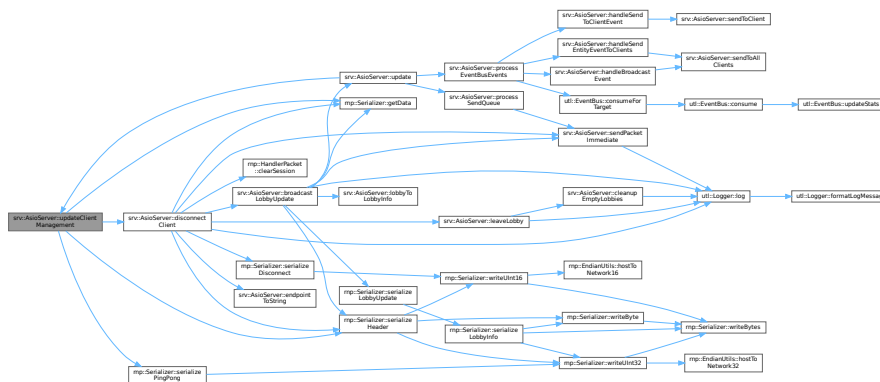
sequenceDiagram
    participant A as srv::AsioServer:AsioServer
    participant B as srv::AsioServer:setupPacket Handlers
    participant C as srv::AsioServer:handleLobby Leave
    participant D as srv::AsioServer:handleLobby Create
    participant E as srv::AsioServer:handleLobbyJoin
    participant F as srv::AsioServer:broadcast LobbyUpdate
    participant G as srv::AsioServer:update
    participant H as srv::AsioServer:disconnect Client
    participant I as srv::AsioServer:updateClient Management

    A->>B
    B->>C
    B->>D
    B->>E
    C->>F
    E->>F
    F->>G
    G->>H
    H->>I
  
```

```
void srv::AsioServer::updateClientManagement () [private]
```

Definition at line 784 of file asioServer.cpp.

Here is the call graph for this function:



```

sequenceDiagram
    participant A as srv:AsioServer:AsioServer
    participant B as srv:AsioServer:setupPacket Handling
    participant C as srv:AsioServer:disconnect Client
    participant D as srv:AsioServer:handleLobby Create
    participant E as srv:AsioServer:handleLobbyJoin
    participant F as srv:AsioServer:handleLobby Leave
    participant G as srv:AsioServer:broadcast Lobby Update
    participant H as srv:AsioServer:update
    participant I as srv:AsioServer:updateClient Management

    A->>B
    B->>C
    B->>D
    B->>E
    B->>F
    C->>I
    D->>G
    E->>G
    F->>G
    G->>H
    H->>I
  
```

18.13.4.1 m clients

Definition at line 156 of file [AsioServer.hpp](#).

Generated by Doxygen

18.13.4.2 m_clientsMutex

std::mutex srv::AsioServer::m_clientsMutex [mutable], [private]

Definition at line 159 of file [AsioServer.hpp](#).

Referenced by [broadcastGameStart\(\)](#), [broadcastLobbyUpdate\(\)](#), [disconnectClient\(\)](#), [getClientCount\(\)](#), [getConnectedSessions\(\)](#), [handleConnect\(\)](#), [handleDisconnect\(\)](#), [handleLobbyCreate\(\)](#), [handleLobbyJoin\(\)](#), [handleLobbyLeave\(\)](#), [handleLobbyListRequest\(\)](#), [handlePing\(\)](#), [handlePong\(\)](#), [handleStartGameRequest\(\)](#), [lobbyToLobbyInfo\(\)](#), [sendToAllClients\(\)](#), [sendToClient\(\)](#), [stop\(\)](#), and [updateClientManagement\(\)](#).

18.13.4.3 m_clientTimeout

std::chrono::milliseconds srv::AsioServer::m_clientTimeout [private]

Definition at line 179 of file [AsioServer.hpp](#).

Referenced by [updateClientManagement\(\)](#).

18.13.4.4 m_componentId

std::uint32_t srv::AsioServer::m_componentId [private]

Definition at line 182 of file [AsioServer.hpp](#).

Referenced by [AsioServer\(\)](#), [handleConnect\(\)](#), [handleDisconnect\(\)](#), [handleEntityEvent\(\)](#), [handleStartGameRequest\(\)](#), and [processEventBusEvents\(\)](#).

18.13.4.5 m_endpointToSession

std::unordered_map<std::string, std::uint32_t> srv::AsioServer::m_endpointToSession [private]

Definition at line 157 of file [AsioServer.hpp](#).

Referenced by [disconnectClient\(\)](#), [handleConnect\(\)](#), [handleDisconnect\(\)](#), and [stop\(\)](#).

18.13.4.6 m_eventBus

utl::EventBus& srv::AsioServer::m_eventBus [private]

Definition at line 183 of file [AsioServer.hpp](#).

Referenced by [AsioServer\(\)](#), [broadcastGameStart\(\)](#), [handleConnect\(\)](#), [handleDisconnect\(\)](#), [handleEntityEvent\(\)](#), [handleStartGameRequest\(\)](#), and [processEventBusEvents\(\)](#).

18.13.4.7 m_host

std::string srv::AsioServer::m_host [private]

Definition at line 146 of file [AsioServer.hpp](#).

Referenced by [init\(\)](#), and [start\(\)](#).

18.13.4.8 m_ioContext

std::unique_ptr<asio::io_context> srv::AsioServer::m_ioContext [private]

Definition at line 141 of file [AsioServer.hpp](#).

Referenced by [networkThreadLoop\(\)](#), and [stop\(\)](#).

18.13.4.9 m_lastPingTime

std::chrono::steady_clock::time_point srv::AsioServer::m_lastPingTime [private]

Definition at line 177 of file [AsioServer.hpp](#).

Referenced by [updateClientManagement\(\)](#).

18.13.4.10 m_lobbies

std::unordered_map<std::uint32_t, Lobby> srv::AsioServer::m_lobbies [private]

Definition at line 162 of file [AsioServer.hpp](#).

Referenced by [broadcastGameStart\(\)](#), [broadcastLobbyUpdate\(\)](#), [cleanupEmptyLobbies\(\)](#), [createLobby\(\)](#), [handleLobbyJoin\(\)](#), [handleStartGameRequest\(\)](#), [joinLobby\(\)](#), [leaveLobby\(\)](#), and [sendLobbyList\(\)](#).

18.13.4.11 m_lobbiesMutex

std::mutex srv::AsioServer::m_lobbiesMutex [mutable], [private]

Definition at line 164 of file [AsioServer.hpp](#).

Referenced by [broadcastGameStart\(\)](#), [broadcastLobbyUpdate\(\)](#), [createLobby\(\)](#), [handleLobbyJoin\(\)](#), [handleStartGameRequest\(\)](#), [joinLobby\(\)](#), [leaveLobby\(\)](#), and [sendLobbyList\(\)](#).

18.13.4.12 m_networkThread

std::unique_ptr<std::thread> srv::AsioServer::m_networkThread [private]

Definition at line 143 of file [AsioServer.hpp](#).

Referenced by [start\(\)](#), and [stop\(\)](#).

18.13.4.13 m_nextLobbyId

std::uint32_t srv::AsioServer::m_nextLobbyId [private]

Definition at line 163 of file [AsioServer.hpp](#).

Referenced by [createLobby\(\)](#), and [handleLobbyCreate\(\)](#).

18.13.4.14 m_nextSessionId

std::uint32_t srv::AsioServer::m_nextSessionId [private]

Definition at line 158 of file [AsioServer.hpp](#).

18.13.4.15 m_packetHandler

std::unique_ptr<rng::HandlerPacket> srv::AsioServer::m_packetHandler [private]

Definition at line 167 of file [AsioServer.hpp](#).

Referenced by [disconnectClient\(\)](#), [handleReceive\(\)](#), and [setupPacketHandlers\(\)](#).

18.13.4.16 m_pingInterval

std::chrono::milliseconds srv::AsioServer::m_pingInterval [private]

Definition at line 178 of file [AsioServer.hpp](#).

Referenced by [updateClientManagement\(\)](#).

18.13.4.17 m_port

std::uint16_t srv::AsioServer::m_port [private]

Definition at line 147 of file [AsioServer.hpp](#).

Referenced by [init\(\)](#), and [start\(\)](#).

18.13.4.18 m_recvBuffer

std::array<std::uint8_t, MAX_LEN_RECV_BUFFER> srv::AsioServer::m_recvBuffer [private]

Definition at line 173 of file [AsioServer.hpp](#).

Referenced by [handleReceive\(\)](#), and [startReceive\(\)](#).

18.13.4.19 m_running

std::atomic<bool> srv::AsioServer::m_running [private]

Definition at line 152 of file [AsioServer.hpp](#).

Referenced by [isRunning\(\)](#), [networkThreadLoop\(\)](#), [sendPacketImmediate\(\)](#), [start\(\)](#), [startReceive\(\)](#), [stop\(\)](#), and [update\(\)](#).

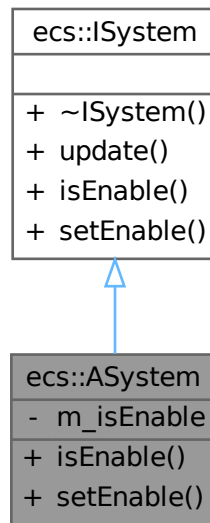
18.13.4.20 m_senderEndpoint

asio::ip::udp::endpoint srv::AsioServer::m_senderEndpoint [private]

Definition at line 174 of file [AsioServer.hpp](#).

Referenced by [handleConnect\(\)](#), [handleLobbyLeave\(\)](#), [handleLobbyListRequest\(\)](#), [handlePing\(\)](#), [handleReceive\(\)](#), [handleStartGameRequest\(\)](#), [sendLobbyCreateResponse\(\)](#), [sendLobbyJoinResponse\(\)](#), [sendLobbyList\(\)](#), and [startReceive\(\)](#).

Collaboration diagram for ecs::ASystem:



Public Member Functions

- bool `isEnabled` () override
- void `setEnable` (const bool enable) override

Public Member Functions inherited from `ecs::ISystem`

- virtual `~ISystem` ()=default
- virtual void `update` (`Registry` ®istry, float dt)=0

Private Attributes

- bool `m_isEnable` = true

18.14.1 Detailed Description

Abstract class for system.

Definition at line 33 of file `ISystems.hpp`.

18.14.2 Member Function Documentation

18.14.2.1 `isEnabled()`

bool `ecs::ASystem::isEnabled` () [inline], [override], [virtual]

Implements `ecs::ISystem`.

Reimplemented in `gme::CollisionSystem`, `gme::CollisionSystem`, `gme::EnemySystem`, `gme::ExplosionSystem`, `gme::ExplosionSystem`, and `gme::SpawnSystem`.

Definition at line 36 of file `ISystems.hpp`.

References `m_isEnable`.

18.14.2.2 setEnable()

```
void ecs::ASystem::setEnable (
    const bool enable) [inline], [override], [virtual]
```

Implements [ecs::ISystem](#).

Reimplemented in [gme::CollisionSystem](#), [gme::CollisionSystem](#), [gme::EnemySystem](#), [gme::ExplosionSystem](#), [gme::ExplosionSystem](#), and [gme::SpawnSystem](#).

Definition at line 37 of file [ISystems.hpp](#).

References [m_isEnable](#).

18.14.3 Member Data Documentation

18.14.3.1 m_isEnable

```
bool ecs::ASystem::m_isEnable = true [private]
```

Definition at line 40 of file [ISystems.hpp](#).

Referenced by [isEnabled\(\)](#), and [setEnable\(\)](#).

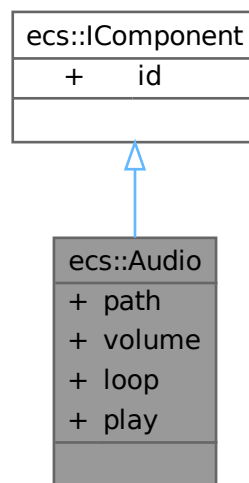
The documentation for this class was generated from the following file:

- [/home/masina/Projects/Epitech/rtype/modules/ECS/include/ECS/Interfaces/ISystems.hpp](#)

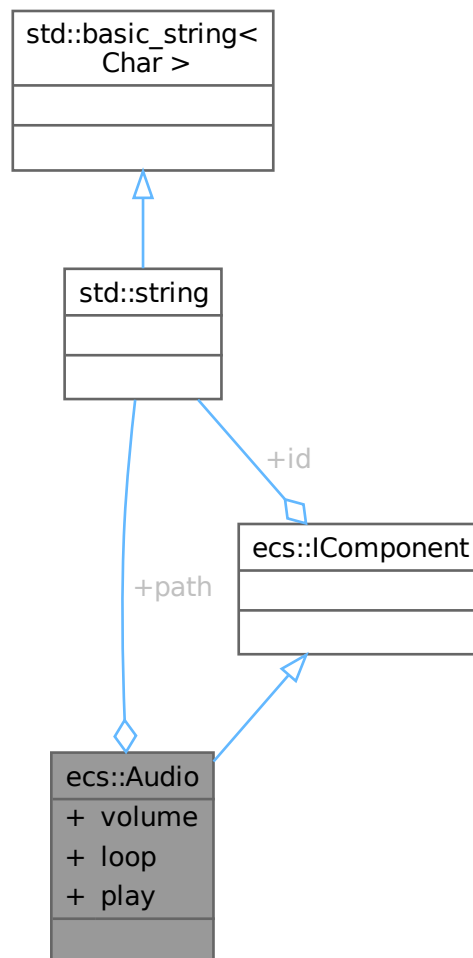
18.15 ecs::Audio Struct Reference

```
#include <Component.hpp>
```

Inheritance diagram for `ecs::Audio`:



Collaboration diagram for `ecs::Audio`:



Public Attributes

- `std::string` [path](#)
- `float` [volume](#)
- `bool` [loop](#)
- `bool` [play](#)

Public Attributes inherited from [ecs::IComponent](#)

- `std::string` [id](#)

18.15.1 Detailed Description

Definition at line 17 of file [Component.hpp](#).

18.15.2 Member Data Documentation

18.15.2.1 loop

bool ecs::Audio::loop

Definition at line 21 of file [Component.hpp](#).

18.15.2.2 path

std::string ecs::Audio::path

Definition at line 19 of file [Component.hpp](#).

18.15.2.3 play

bool ecs::Audio::play

Definition at line 22 of file [Component.hpp](#).

Referenced by [gme::Game::event\(\)](#).

18.15.2.4 volume

float ecs::Audio::volume

Definition at line 20 of file [Component.hpp](#).

The documentation for this struct was generated from the following file:

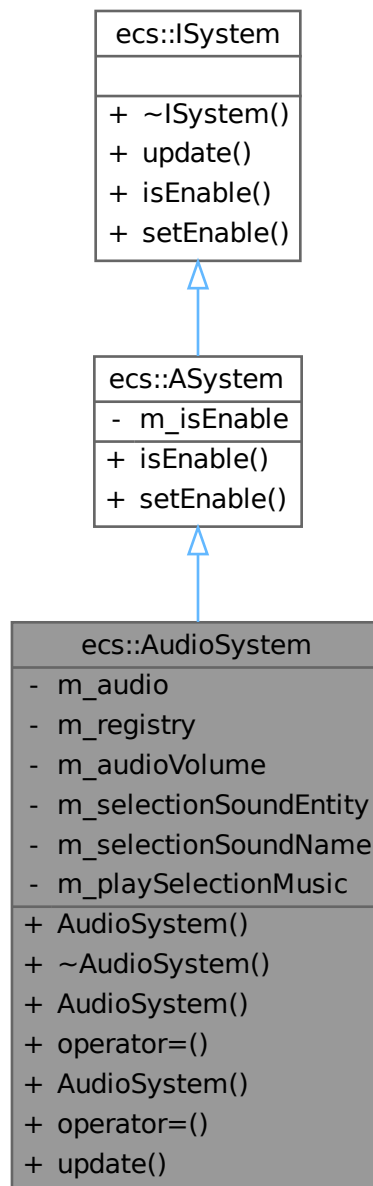
- [/home/masina/Projects/Epitech/rtype/modules/ECS/include/ECS/Component.hpp](#)

18.16 ecs::AudioSystem Class Reference

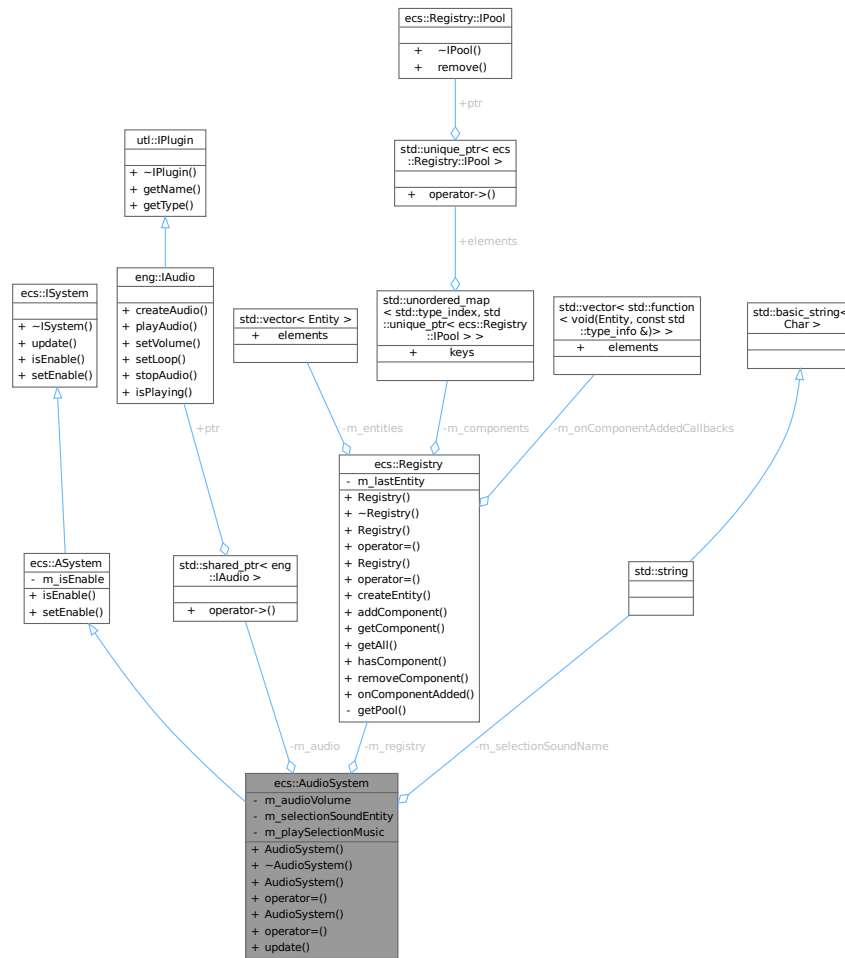
Class for audio system.

#include <Audio.hpp>

Inheritance diagram for ecs::AudioSystem:



Collaboration diagram for `ecs::AudioSystem`:



Public Member Functions

- `AudioSystem` (const `std::shared_ptr< eng::IAudio >` &audio, const float &audioVolume, `Registry` ®istry, bool &playSelectionMusic)
- `~AudioSystem` () override=default
- `AudioSystem` (const `AudioSystem` &)=delete
- `AudioSystem` & operator= (const `AudioSystem` &)=delete
- `AudioSystem` (`AudioSystem` &&)=delete
- `AudioSystem` & operator= (`AudioSystem` &&)=delete
- void `update` (`Registry` ®istry, float) override

Public Member Functions inherited from `ecs::ASystem`

- bool `isEnabled` () override
- void `setEnable` (const bool enable) override

Public Member Functions inherited from `ecs::ISystem`

- virtual `~ISystem` ()=default

Private Attributes

- `const std::shared_ptr< eng::IAudio > & m_audio`
- `Registry & m_registry`
- `const float & m_audioVolume`
- `Entity m_selectionSoundEntity {}`
- `std::string m_selectionSoundName`
- `bool & m_playSelectionMusic`

18.16.1 Detailed Description

Class for audio system.

Definition at line 23 of file [Audio.hpp](#).

18.16.2 Constructor & Destructor Documentation

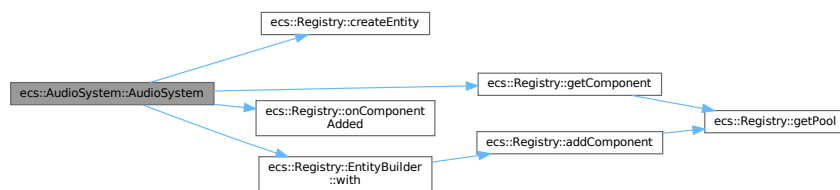
18.16.2.1 AudioSystem() [1/3]

```
ecs::AudioSystem::AudioSystem (
    const std::shared_ptr< eng::IAudio > & audio,
    const float & audioVolume,
    Registry & registry,
    bool & playSelectionMusic) [inline], [explicit]
```

Definition at line 26 of file [Audio.hpp](#).

References [utl::Path::Audio::AUDIO_INPUT](#), [ecs::Registry::createEntity\(\)](#), [ecs::Registry::getComponent\(\)](#), [m_selectionSoundEntity](#), [m_selectionSoundName](#), [ecs::Registry::onComponentAdded\(\)](#), and [ecs::Registry::EntityBuilder::with](#)

Here is the call graph for this function:



18.16.2.2 ~AudioSystem()

```
ecs::AudioSystem::~AudioSystem () [override], [default]
```

18.16.2.3 AudioSystem() [2/3]

```
ecs::AudioSystem::AudioSystem (
    const AudioSystem & ) [delete]
```

18.16.2.4 AudioSystem() [3/3]

```
ecs::AudioSystem::AudioSystem (
    AudioSystem && ) [delete]
```

18.16.3 Member Function Documentation

18.16.3.1 operator=() [1/2]

```
AudioSystem & ecs::AudioSystem::operator= (
    AudioSystem && ) [delete]
```

18.16.3.2 operator=() [2/2]

[AudioSystem](#) & ecs::AudioSystem::operator= (
const [AudioSystem](#) &) [delete]

18.16.3.3 update()

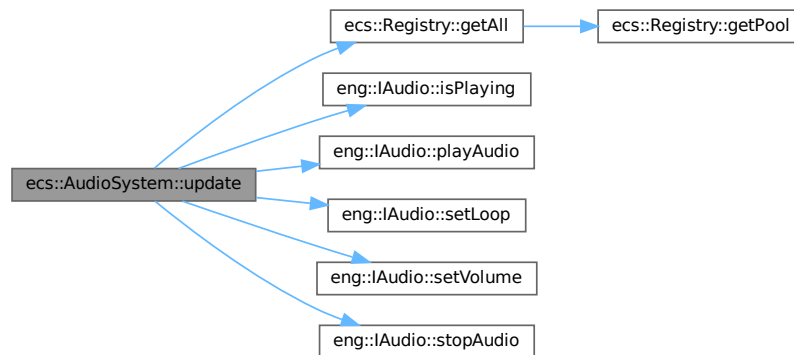
void ecs::AudioSystem::update (
Registry & registry,
float) [inline], [override], [virtual]

Implements [ecs::ISystem](#).

Definition at line 59 of file [Audio.hpp](#).

References [ecs::Registry::getAll\(\)](#), [eng::IAudio::isPlaying\(\)](#), [m_audio](#), [m_audioVolume](#), [m_playSelectionMusic](#), [m_selectionSoundName](#), [eng::IAudio::playAudio\(\)](#), [eng::Playing](#), [eng::IAudio::setLoop\(\)](#), [eng::IAudio::setVolume\(\)](#), [eng::IAudio::stopAudio\(\)](#), and [eng::Stopped](#).

Here is the call graph for this function:



18.16.4 Member Data Documentation

18.16.4.1 m_audio

const std::shared_ptr<[eng::IAudio](#)>& ecs::AudioSystem::m_audio [private]

Definition at line 99 of file [Audio.hpp](#).

Referenced by [update\(\)](#).

18.16.4.2 m_audioVolume

const float& ecs::AudioSystem::m_audioVolume [private]

Definition at line 101 of file [Audio.hpp](#).

Referenced by [update\(\)](#).

18.16.4.3 m_playSelectionMusic

bool& ecs::AudioSystem::m_playSelectionMusic [private]

Definition at line 105 of file [Audio.hpp](#).

Referenced by [update\(\)](#).

18.16.4.4 m_registry

[Registry](#)& ecs::AudioSystem::m_registry [private]

Definition at line 100 of file [Audio.hpp](#).

18.16.4.5 m_selectionSoundEntity

Entity ecs::AudioSystem::m_selectionSoundEntity {} [private]

Definition at line 103 of file [Audio.hpp](#).

Referenced by [AudioSystem\(\)](#).

18.16.4.6 m_selectionSoundName

std::string ecs::AudioSystem::m_selectionSoundName [private]

Definition at line 104 of file [Audio.hpp](#).

Referenced by [AudioSystem\(\)](#), and [update\(\)](#).

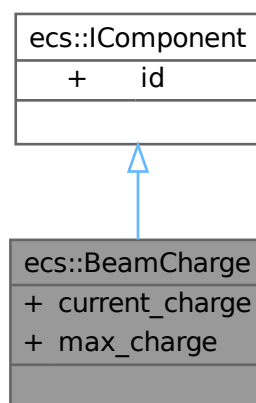
The documentation for this class was generated from the following file:

- [/home/masina/Projects/Epitech/rtype/modules/ECS/include/ECS/Systems/Audio.hpp](#)

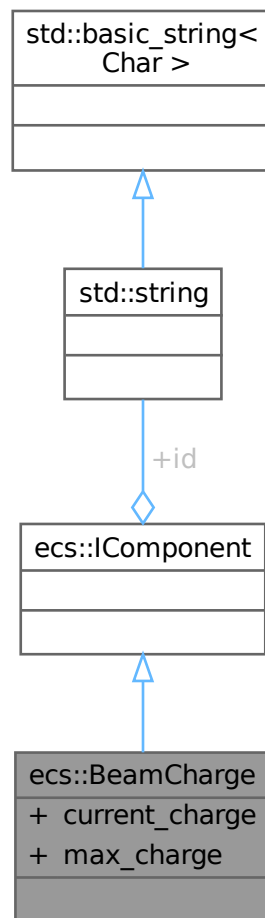
18.17 ecs::BeamCharge Struct Reference

```
#include <Component.hpp>
```

Inheritance diagram for ecs::BeamCharge:



Collaboration diagram for `ecs::BeamCharge`:



Public Attributes

- float [current_charge](#)
- float [max_charge](#)

Public Attributes inherited from [ecs::IComponent](#)

- `std::string` [id](#)

18.17.1 Detailed Description

Definition at line 96 of file [Component.hpp](#).

18.17.2 Member Data Documentation

18.17.2.1 `current_charge`

float `ecs::BeamCharge::current_charge`

Definition at line 98 of file [Component.hpp](#).

Referenced by [gme::RTypeServer::handlePlayerShooting\(\)](#).

18.17.2.2 max_charge

float ecs::BeamCharge::max_charge

Definition at line 99 of file [Component.hpp](#).

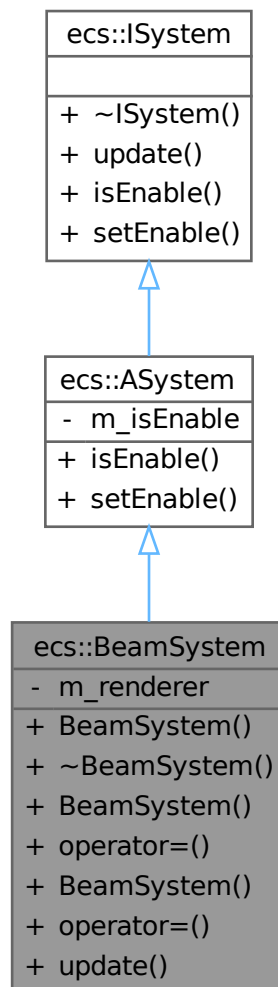
The documentation for this struct was generated from the following file:

- [/home/masina/Projects/Epitech/rtype/modules/ECS/include/ECS/Component.hpp](#)

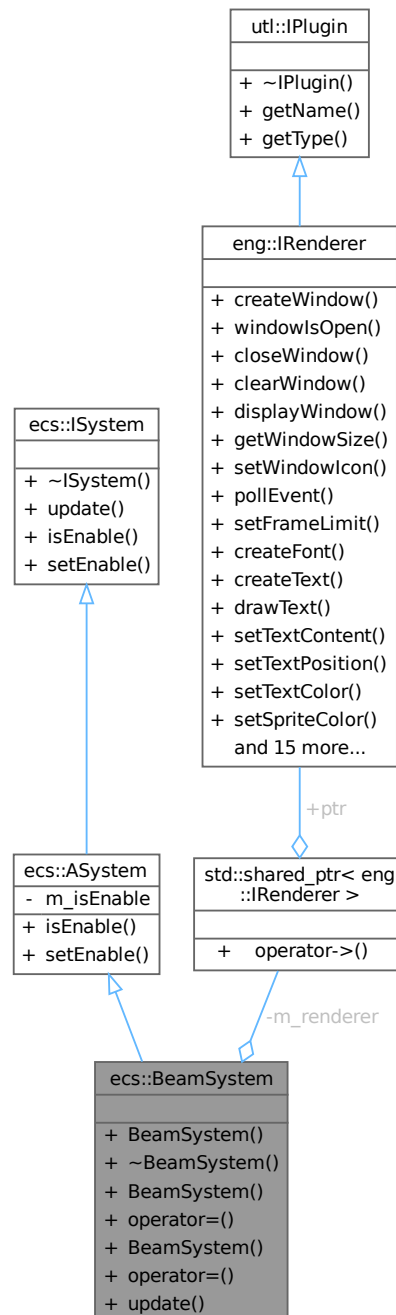
18.18 ecs::BeamSystem Class Reference

#include <Beam.hpp>

Inheritance diagram for ecs::BeamSystem:



Collaboration diagram for `ecs::BeamSystem`:



Public Member Functions

- `BeamSystem` (`const std::shared_ptr< eng::IRenderer > &renderer`)
- `~BeamSystem` () override=default
- `BeamSystem` (`const BeamSystem &`)=delete
- `BeamSystem & operator=` (`const BeamSystem &`)=delete
- `BeamSystem` (`BeamSystem &&`)=delete
- `BeamSystem & operator=` (`BeamSystem &&`)=delete
- `void update` (`Registry ®istry`, `float`) override

Public Member Functions inherited from [ecs::ASystem](#)

- bool [isEnabled](#) () override
- void [setEnabled](#) (const bool enable) override

Public Member Functions inherited from [ecs::ISystem](#)

- virtual [~ISystem](#) ()=default

Private Attributes

- const std::shared_ptr< [eng::IRenderer](#) > & [m_renderer](#)

18.18.1 Detailed Description

Definition at line 18 of file [Beam.hpp](#).

18.18.2 Constructor & Destructor Documentation

18.18.2.1 BeamSystem() [1/3]

```
ecs::BeamSystem::BeamSystem (
    const std::shared_ptr< eng::IRenderer > & renderer) [inline], [explicit]
```

Definition at line 21 of file [Beam.hpp](#).

18.18.2.2 ~BeamSystem()

```
ecs::BeamSystem::~BeamSystem () [override], [default]
```

18.18.2.3 BeamSystem() [2/3]

```
ecs::BeamSystem::BeamSystem (
    const BeamSystem & ) [delete]
```

18.18.2.4 BeamSystem() [3/3]

```
ecs::BeamSystem::BeamSystem (
    BeamSystem && ) [delete]
```

18.18.3 Member Function Documentation

18.18.3.1 operator=() [1/2]

```
BeamSystem & ecs::BeamSystem::operator= (
    BeamSystem && ) [delete]
```

18.18.3.2 operator=() [2/2]

```
BeamSystem & ecs::BeamSystem::operator= (
    const BeamSystem & ) [delete]
```

18.18.3.3 update()

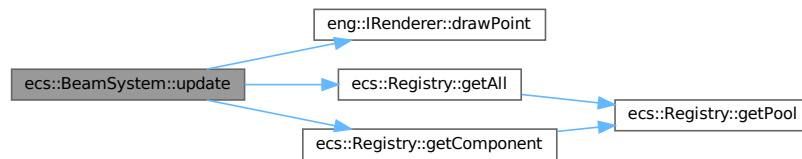
```
void ecs::BeamSystem::update (
    Registry & registry,
    float ) [inline], [override], [virtual]
```

Implements [ecs::ISystem](#).

Definition at line 29 of file [Beam.hpp](#).

References [utl::GameConfig::Beam::BAR_HEIGHT](#), [utl::GameConfig::Beam::BAR_WIDTH](#), [eng::IRenderer::drawPoint](#), [ecs::Registry::getAll\(\)](#), [ecs::Registry::getComponent\(\)](#), [m_renderer](#), [eng::Color::r](#), [utl::GameConfig::Player::SPRITE_WIDTH](#) and [ecs::Transform::x](#).

Here is the call graph for this function:



18.18.4 Member Data Documentation

18.18.4.1 m_renderer

`const std::shared_ptr<eng::IRenderer>& ecs::BeamSystem::m_renderer` [private]

Definition at line 93 of file [Beam.hpp](#).

Referenced by [update\(\)](#).

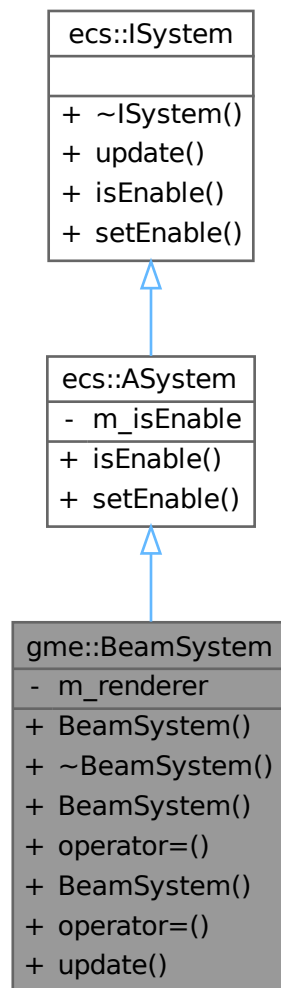
The documentation for this class was generated from the following file:

- [/home/masina/Projects/Epitech/rtype/modules/ECS/include/ECS/Systems/Beam.hpp](#)

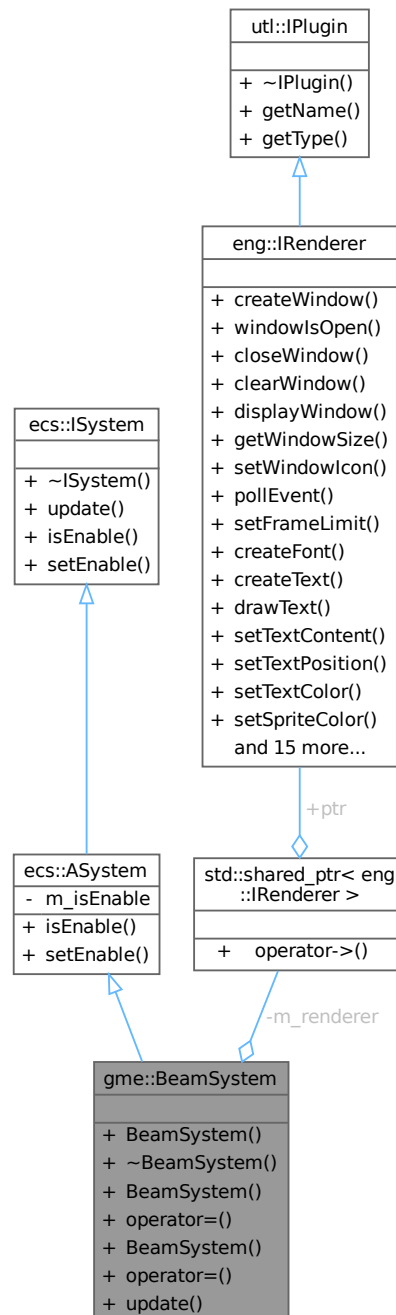
18.19 gme::BeamSystem Class Reference

`#include <Beam.hpp>`

Inheritance diagram for gme::BeamSystem:



Collaboration diagram for gme::BeamSystem:



Public Member Functions

- `BeamSystem` (`const std::shared_ptr< eng::IRenderer > &renderer`)
- `~BeamSystem` () override=default
- `BeamSystem` (`const BeamSystem &`)=delete
- `BeamSystem & operator=` (`const BeamSystem &`)=delete
- `BeamSystem` (`BeamSystem &&`)=delete
- `BeamSystem & operator=` (`BeamSystem &&`)=delete
- `void update` (`ecs::Registry ®istry`, `float`) override

Public Member Functions inherited from [ecs::ASystem](#)

- bool [isEnabled](#) () override
- void [setEnabled](#) (const bool enable) override

Public Member Functions inherited from [ecs::ISystem](#)

- virtual [~ISystem](#) ()=default

Private Attributes

- const std::shared_ptr< [eng::IRenderer](#) > & [m_renderer](#)

18.19.1 Detailed Description

Definition at line 18 of file [Beam.hpp](#).

18.19.2 Constructor & Destructor Documentation

18.19.2.1 BeamSystem() [1/3]

```
gme::BeamSystem::BeamSystem (
    const std::shared_ptr< eng::IRenderer > & renderer) [inline], [explicit]
```

Definition at line 21 of file [Beam.hpp](#).

18.19.2.2 ~BeamSystem()

```
gme::BeamSystem::~BeamSystem () [override], [default]
```

18.19.2.3 BeamSystem() [2/3]

```
gme::BeamSystem::BeamSystem (
    const BeamSystem & ) [delete]
```

18.19.2.4 BeamSystem() [3/3]

```
gme::BeamSystem::BeamSystem (
    BeamSystem && ) [delete]
```

18.19.3 Member Function Documentation

18.19.3.1 operator=() [1/2]

```
BeamSystem & gme::BeamSystem::operator= (
    BeamSystem && ) [delete]
```

18.19.3.2 operator=() [2/2]

```
BeamSystem & gme::BeamSystem::operator= (
    const BeamSystem & ) [delete]
```

18.19.3.3 update()

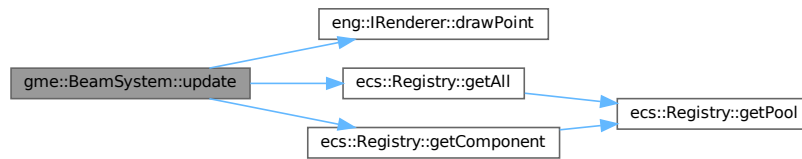
```
void gme::BeamSystem::update (
    ecs::Registry & registry,
    float ) [inline], [override], [virtual]
```

Implements [ecs::ISystem](#).

Definition at line 29 of file [Beam.hpp](#).

References [eng::IRenderer::drawPoint\(\)](#), [ecs::Registry::getAll\(\)](#), [ecs::Registry::getComponent\(\)](#), [m_renderer](#), [eng::Color::r](#), and [ecs::Transform::x](#).

Here is the call graph for this function:



18.19.4 Member Data Documentation

18.19.4.1 m_renderer

`const std::shared_ptr<eng::IRenderer>& gme::BeamSystem::m_renderer` [private]

Definition at line 94 of file [Beam.hpp](#).

Referenced by [update\(\)](#).

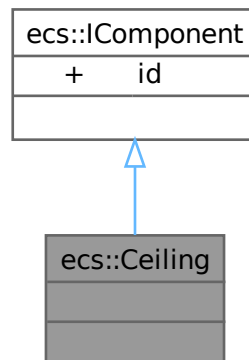
The documentation for this class was generated from the following file:

- `/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/include/RTypeClientMulti/Systems/Beam.hpp`

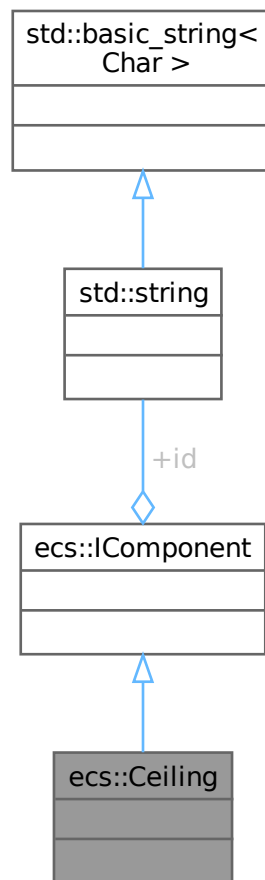
18.20 ecs::Ceiling Struct Reference

`#include <Component.hpp>`

Inheritance diagram for `ecs::Ceiling`:



Collaboration diagram for `ecs::Ceiling`:



Additional Inherited Members

Public Attributes inherited from [ecs::IComponent](#)

- `std::string id`

18.20.1 Detailed Description

Definition at line 156 of file [Component.hpp](#).

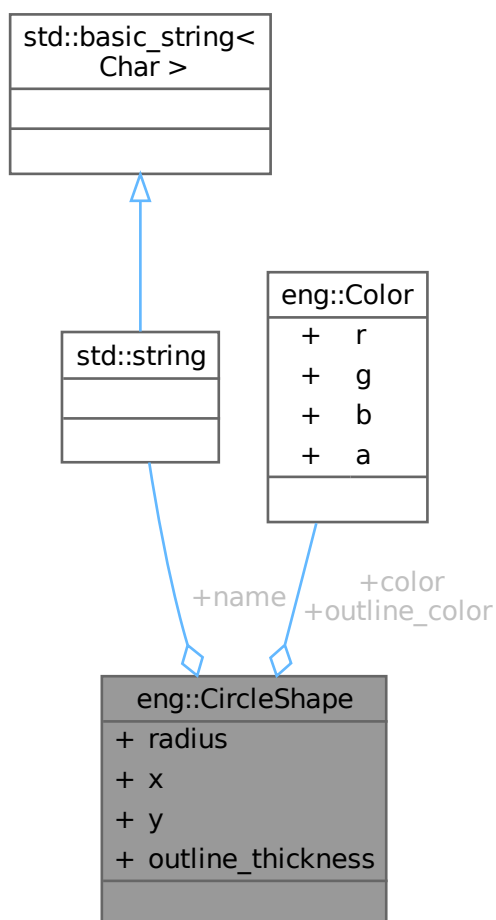
The documentation for this struct was generated from the following file:

- `/home/masina/Projects/Epitech/rtype/modules/ECS/include/ECS/Component.hpp`

18.21 eng::CircleShape Struct Reference

`#include <IRenderer.hpp>`

Collaboration diagram for `eng::CircleShape`:



Public Attributes

- `std::string` [name](#)
- `float` [radius](#)
- `Color` [color](#)
- `float` [x](#)
- `float` [y](#)
- `float` [outline_thickness](#)
- `Color` [outline_color](#)

18.21.1 Detailed Description

Definition at line 33 of file [IRenderer.hpp](#).

18.21.2 Member Data Documentation

18.21.2.1 `color`

`Color` `eng::CircleShape::color`

Definition at line 37 of file [IRenderer.hpp](#).

18.21.2.2 name

std::string eng::CircleShape::name

Definition at line 35 of file [IRenderer.hpp](#).

18.21.2.3 outline_color

[Color](#) eng::CircleShape::outline_color

Definition at line 41 of file [IRenderer.hpp](#).

18.21.2.4 outline_thickness

float eng::CircleShape::outline_thickness

Definition at line 40 of file [IRenderer.hpp](#).

18.21.2.5 radius

float eng::CircleShape::radius

Definition at line 36 of file [IRenderer.hpp](#).

18.21.2.6 x

float eng::CircleShape::x

Definition at line 38 of file [IRenderer.hpp](#).

18.21.2.7 y

float eng::CircleShape::y

Definition at line 39 of file [IRenderer.hpp](#).

The documentation for this struct was generated from the following file:

- [/home/masina/Projects/Epitech/rtype/modules/Interfaces/include/Interfaces/IRenderer.hpp](#)

18.22 cli::Client Class Reference

Class for the client.

#include <Client.hpp>

- void [updateKeyboardInput](#) (ecs::Registry ®istry)
- void [setupScenes](#) ()

Static Private Member Functions

- static [utl::cli::AppConfig setupConfig](#) (const [ArgsConfig](#) &cfg)

Private Attributes

- std::unique_ptr< [utl::PluginLoader](#) > [m_pluginLoader](#)
- std::unique_ptr< [eng::Engine](#) > [m_engine](#)
- std::shared_ptr< [gme::IGameClient](#) > [m_gameSolo](#)
- std::shared_ptr< [gme::IGameClient](#) > [m_gameMulti](#)
- std::unordered_map< [eng::Key](#), bool > [m_keysPressed](#)
- [utl::cli::AppConfig](#) [m_config](#)
- bool [m_showDebug](#) = false

18.22.1 Detailed Description

Class for the client.

Definition at line 25 of file [Client.hpp](#).

18.22.2 Constructor & Destructor Documentation

18.22.2.1 Client() [1/3]

[cli::Client::Client](#) (
 const [ArgsConfig](#) & cfg) [explicit]

Definition at line 30 of file [client.cpp](#).

References [cli::ArgsConfig::audio_lib_path](#), [BUILD_TYPE](#), [cli::ArgsConfig::game_multi_lib_path](#), [cli::ArgsConfig::game_solo_lib_path](#), [GIT_COMMIT_HASH](#), [GIT_TAG](#), [utl::Path::Texture::ICON_PATH](#), [utl::INFO](#), [utl::Logger::log\(\)](#), [cli::ArgsConfig::network_lib_path](#), [utl::Path::Plugin::PLUGIN_AUDIO_SFML](#), [utl::Path::Plugin::PLUGIN_GAME_MULTI](#), [utl::Path::Plugin::PLUGIN_GAME_SOLO](#), [utl::Path::Plugin::PLUGIN_RENDERER_SFML](#), [PROJECT_NAME](#), [PROJECT_VERSION](#), and [cli::ArgsConfig::renderer_lib_path](#).

Here is the call graph for this function:



18.22.2.2 ~Client()

[cli::Client::~~Client](#) () [default]

18.22.2.3 Client() [2/3]

[cli::Client::Client](#) (
 const [Client](#) &) [delete]

18.22.2.4 Client() [3/3]

[cli::Client::Client](#) (
 [Client](#) &&) [delete]

18.22.3 Member Function Documentation

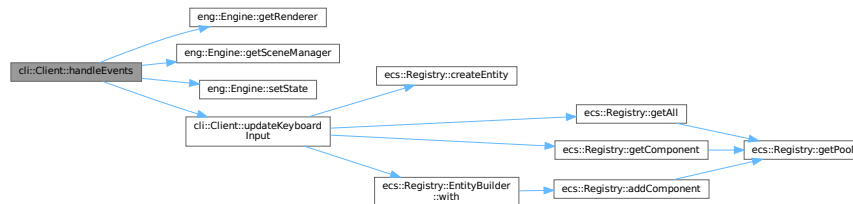
18.22.3.1 handleEvents()

void cli::Client::handleEvents (
 eng::Event & event) [private]

Definition at line 4 of file [event.cpp](#).

References [eng::Closed](#), [eng::Engine::getRenderer\(\)](#), [eng::Engine::getSceneManager\(\)](#), [eng::Event::key](#), [eng::KeyPressed](#), [eng::KeyReleased](#), [eng::LControl](#), [m_engine](#), [m_keysPressed](#), [m_showDebug](#), [eng::Engine::setState\(\)](#), [eng::STOP](#), [eng::Event::type](#), and [updateKeyboardInput\(\)](#).

Here is the call graph for this function:



18.22.3.2 operator=() [1/2]

[Client](#) & cli::Client::operator= (
 [Client](#) &&) [delete]

18.22.3.3 operator=() [2/2]

[Client](#) & cli::Client::operator= (
 const [Client](#) &) [delete]

18.22.3.4 run()

void cli::Client::run ()

Definition at line 68 of file [client.cpp](#).

References [utl::Config::Color::DARK](#), and [eng::RUN](#).

Referenced by [main\(\)](#).

Here is the caller graph for this function:



18.22.3.5 setupConfig()

[utl::cli::AppConfig](#) cli::Client::setupConfig (
 const [ArgsConfig](#) & cfg) [static], [private]

Definition at line 10 of file [client.cpp](#).

References [utl::cli::AppConfig::audioVolume](#), [utl::cli::AppConfig::controlScheme](#), [utl::Config::Audio::DEFAULT_AUDIO_](#), [utl::Config::Game::DEFAULT_CONTROL_SCHEME](#), [utl::Config::Game::DEFAULT_SKIN_INDEX](#),

utl::Config::Game::DEFAULT_VIDEO_QUALITY, cli::ArgsConfig::frameLimit, utl::cli::AppConfig::frameLimit, cli::ArgsConfig::fullscreen, utl::cli::AppConfig::fullscreen, cli::ArgsConfig::height, utl::cli::AppConfig::height, cli::ArgsConfig::host, utl::cli::AppConfig::host, cli::ArgsConfig::player_name, utl::cli::AppConfig::player_name, cli::ArgsConfig::port, utl::cli::AppConfig::port, utl::cli::AppConfig::skinIndex, utl::cli::AppConfig::videoQuality, cli::ArgsConfig::width, and utl::cli::AppConfig::width.

18.22.3.6 setupScenes()

void cli::Client::setupScenes () [private]
Definition at line 89 of file [client.cpp](#).

18.22.3.7 stop()

void cli::Client::stop () const
Definition at line 83 of file [client.cpp](#).
Referenced by [main\(\)](#).
Here is the caller graph for this function:



18.22.3.8 updateKeyboardInput()

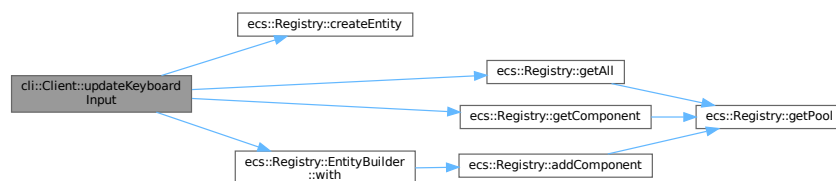
void cli::Client::updateKeyboardInput (
 [ecs::Registry](#) & registry) [private]

Definition at line 38 of file [event.cpp](#).

References [ecs::Registry::createEntity\(\)](#), [eng::Down](#), [ecs::Registry::getAll\(\)](#), [ecs::Registry::getComponent\(\)](#), [eng::Left](#), [eng::Right](#), [eng::Space](#), [eng::Up](#), and [ecs::Registry::EntityBuilder::with\(\)](#).

Referenced by [handleEvents\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.22.4 Member Data Documentation

18.22.4.1 m_config

[utl::cli::AppConfig](#) cli::Client::m_config [private]
 Definition at line 52 of file [Client.hpp](#).

18.22.4.2 m_engine

std::unique_ptr<[eng::Engine](#)> cli::Client::m_engine [private]
 Definition at line 47 of file [Client.hpp](#).
 Referenced by [handleEvents\(\)](#).

18.22.4.3 m_gameMulti

std::shared_ptr<[gme::IGameClient](#)> cli::Client::m_gameMulti [private]
 Definition at line 49 of file [Client.hpp](#).

18.22.4.4 m_gameSolo

std::shared_ptr<[gme::IGameClient](#)> cli::Client::m_gameSolo [private]
 Definition at line 48 of file [Client.hpp](#).

18.22.4.5 m_keysPressed

std::unordered_map<[eng::Key](#), bool> cli::Client::m_keysPressed [private]
 Definition at line 50 of file [Client.hpp](#).
 Referenced by [handleEvents\(\)](#).

18.22.4.6 m_pluginLoader

std::unique_ptr<[utl::PluginLoader](#)> cli::Client::m_pluginLoader [private]
 Definition at line 46 of file [Client.hpp](#).

18.22.4.7 m_showDebug

bool cli::Client::m_showDebug = false [private]
 Definition at line 53 of file [Client.hpp](#).
 Referenced by [handleEvents\(\)](#).

The documentation for this class was generated from the following files:

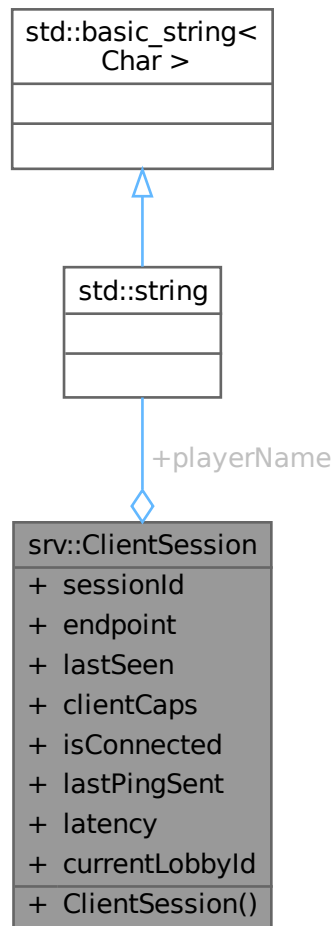
- /home/masina/Projects/Epitech/rtype/client/include/Client/[Client.hpp](#)
- /home/masina/Projects/Epitech/rtype/client/src/[client.cpp](#)
- /home/masina/Projects/Epitech/rtype/client/src/[event.cpp](#)

18.23 srv::ClientSession Struct Reference

Client session information.

```
#include <AsioServer.hpp>
```

Collaboration diagram for srv::ClientSession:



Public Member Functions

- [ClientSession](#) ()

Public Attributes

- `std::uint32_t` [sessionId](#)
- `asio::ip::udp::endpoint` [endpoint](#)
- `std::chrono::steady_clock::time_point` [lastSeen](#)
- `std::string` [playerName](#)
- `std::uint32_t` [clientCaps](#)
- `bool` [isConnected](#)
- `std::uint32_t` [lastPingSent](#)
- `std::uint32_t` [latency](#)
- `std::uint32_t` [currentLobbyId](#)

18.23.1 Detailed Description

Client session information.

Definition at line 30 of file [AsioServer.hpp](#).

18.23.2 Constructor & Destructor Documentation

18.23.2.1 ClientSession()

`srv::ClientSession::ClientSession () [inline]`

Definition at line 42 of file [AsioServer.hpp](#).

18.23.3 Member Data Documentation

18.23.3.1 clientCaps

`std::uint32_t srv::ClientSession::clientCaps`

Definition at line 36 of file [AsioServer.hpp](#).

Referenced by [srv::AsioServer::handleConnect\(\)](#).

18.23.3.2 currentLobbyId

`std::uint32_t srv::ClientSession::currentLobbyId`

Definition at line 40 of file [AsioServer.hpp](#).

18.23.3.3 endpoint

`asio::ip::udp::endpoint srv::ClientSession::endpoint`

Definition at line 33 of file [AsioServer.hpp](#).

Referenced by [srv::AsioServer::handleConnect\(\)](#).

18.23.3.4 isConnected

`bool srv::ClientSession::isConnected`

Definition at line 37 of file [AsioServer.hpp](#).

Referenced by [srv::AsioServer::handleConnect\(\)](#).

18.23.3.5 lastPingSent

`std::uint32_t srv::ClientSession::lastPingSent`

Definition at line 38 of file [AsioServer.hpp](#).

18.23.3.6 lastSeen

`std::chrono::steady_clock::time_point srv::ClientSession::lastSeen`

Definition at line 34 of file [AsioServer.hpp](#).

Referenced by [srv::AsioServer::handleConnect\(\)](#).

18.23.3.7 latency

`std::uint32_t srv::ClientSession::latency`

Definition at line 39 of file [AsioServer.hpp](#).

18.23.3.8 playerName

`std::string srv::ClientSession::playerName`

Definition at line 35 of file [AsioServer.hpp](#).

Referenced by [srv::AsioServer::handleConnect\(\)](#).

18.23.3.9 sessionId

std::uint32_t srv::ClientSession::sessionId

Definition at line 32 of file [AsioServer.hpp](#).

Referenced by [srv::AsioServer::handleConnect\(\)](#).

The documentation for this struct was generated from the following file:

- [/home/masina/Projects/Epitech/rtype/plugins/Network/Asio/Server/include/AsioServer/AsioServer.hpp](#)

18.24 utl::Clock Class Reference

Class for clock.

#include <Clock.hpp>

Collaboration diagram for utl::Clock:

utl::Clock
- m_start - m_pausedTime - m_pausedDuration - m_isPaused
+ Clock() + ~Clock() + Clock() + operator=() + Clock() + operator=() + restart() + pause() + resume() + getDeltaSeconds() + getElapsed() + now()

Public Types

- using [TimePoint](#) = std::chrono::time_point<std::chrono::high_resolution_clock>

Public Member Functions

- [Clock](#) (const bool startNow=true)
- [~Clock](#) ()=default
- [Clock](#) (const [Clock](#) &)=delete
- [Clock](#) & [operator=](#) (const [Clock](#) &)=delete
- [Clock](#) ([Clock](#) &&)=delete
- [Clock](#) & [operator=](#) ([Clock](#) &&)=delete
- void [restart](#) ()

- void [pause](#) ()
- void [resume](#) ()
- float [getDeltaSeconds](#) () const
- template<typename [Duration](#) = std::chrono::seconds>
auto [getElapsed](#) () const

Static Public Member Functions

- static [TimePoint](#) [now](#) ()

Private Types

- using [Duration](#) = std::chrono::high_resolution_clock::duration

Private Attributes

- [TimePoint](#) [m_start](#)
- [TimePoint](#) [m_pausedTime](#)
- [Duration](#) [m_pausedDuration](#)
- bool [m_isPaused](#) {false}

Friends

- std::ostream & [operator<<](#) (std::ostream &os, const [Clock](#) &clock)

18.24.1 Detailed Description

Class for clock.

Definition at line 20 of file [Clock.hpp](#).

18.24.2 Member Typedef Documentation

18.24.2.1 Duration

using [utl::Clock::Duration](#) = std::chrono::high_resolution_clock::duration [private]

Definition at line 78 of file [Clock.hpp](#).

18.24.2.2 TimePoint

using [utl::Clock::TimePoint](#) = std::chrono::time_point<std::chrono::high_resolution_clock>

Definition at line 24 of file [Clock.hpp](#).

18.24.3 Constructor & Destructor Documentation

18.24.3.1 Clock() [1/3]

```
utl::Clock::Clock (
    const bool startNow = true) [inline], [explicit]
```

Definition at line 26 of file [Clock.hpp](#).

18.24.3.2 ~Clock()

```
utl::Clock::~Clock () [default]
```

18.24.3.3 Clock() [2/3]

```
utl::Clock::Clock (
    const Clock & ) [delete]
```

18.24.3.4 Clock() [3/3]

```
utl::Clock::Clock (  
    Clock && ) [delete]
```

18.24.4 Member Function Documentation

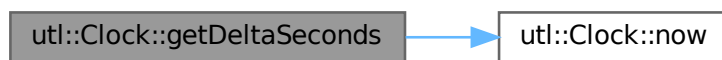
18.24.4.1 getDeltaSeconds()

```
float utl::Clock::getDeltaSeconds () const [inline], [nodiscard]
```

Definition at line 63 of file [Clock.hpp](#).

References [m_isPaused](#), [m_pausedDuration](#), [m_pausedTime](#), [m_start](#), and [now\(\)](#).

Here is the call graph for this function:



18.24.4.2 getElapsed()

```
template<typename Duration = std::chrono::seconds>
```

```
auto utl::Clock::getElapsed () const [inline], [nodiscard]
```

Definition at line 72 of file [Clock.hpp](#).

References [m_pausedDuration](#), [m_start](#), and [now\(\)](#).

Here is the call graph for this function:



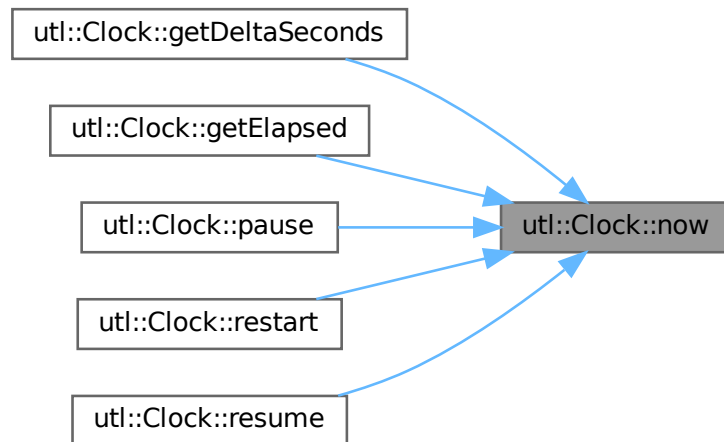
18.24.4.3 now()

```
static TimePoint utl::Clock::now () [inline], [static]
```

Definition at line 40 of file [Clock.hpp](#).

Referenced by [getDeltaSeconds\(\)](#), [getElapsed\(\)](#), [pause\(\)](#), [restart\(\)](#), and [resume\(\)](#).

Here is the caller graph for this function:



18.24.4.4 operator=() [1/2]

[Clock](#) & utl::Clock::operator= (
[Clock](#) &&) [delete]

18.24.4.5 operator=() [2/2]

[Clock](#) & utl::Clock::operator= (
 const [Clock](#) &) [delete]

18.24.4.6 pause()

void utl::Clock::pause () [inline]

Definition at line 47 of file [Clock.hpp](#).

References [m_isPaused](#), [m_pausedTime](#), and [now\(\)](#).

Here is the call graph for this function:



18.24.4.7 restart()

void utl::Clock::restart () [inline]

Definition at line 41 of file [Clock.hpp](#).

References [m_isPaused](#), [m_pausedDuration](#), [m_start](#), and [now\(\)](#).

Here is the call graph for this function:



18.24.4.8 resume()

`void utl::Clock::resume ()` [inline]

Definition at line 55 of file [Clock.hpp](#).

References [m_isPaused](#), [m_pausedDuration](#), [m_pausedTime](#), and [now\(\)](#).

Here is the call graph for this function:



18.24.5 Friends And Related Symbol Documentation

18.24.5.1 operator<<

```
std::ostream & operator<< (
    std::ostream & os,
    const Clock & clock) [friend]
```

Definition at line 34 of file [Clock.hpp](#).

18.24.6 Member Data Documentation

18.24.6.1 m_isPaused

`bool utl::Clock::m_isPaused {false}` [private]

Definition at line 83 of file [Clock.hpp](#).

Referenced by [getDeltaSeconds\(\)](#), [pause\(\)](#), [restart\(\)](#), and [resume\(\)](#).

18.24.6.2 m_pausedDuration

[Duration](#) `utl::Clock::m_pausedDuration` [private]

Definition at line 82 of file [Clock.hpp](#).

Referenced by [getDeltaSeconds\(\)](#), [getElapsed\(\)](#), [restart\(\)](#), and [resume\(\)](#).

18.24.6.3 m_pausedTime

[TimePoint](#) `utl::Clock::m_pausedTime` [private]

Definition at line 81 of file [Clock.hpp](#).

Referenced by [getDeltaSeconds\(\)](#), [pause\(\)](#), and [resume\(\)](#).

18.24.6.4 m_start

[TimePoint](#) utl::Clock::m_start [private]

Definition at line 80 of file [Clock.hpp](#).

Referenced by [getDeltaSeconds\(\)](#), [getElapsed\(\)](#), and [restart\(\)](#).

The documentation for this class was generated from the following file:

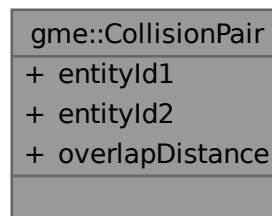
- [/home/masina/Projects/Epitech/rtype/modules/Utils/include/Utils/Clock.hpp](#)

18.25 gme::CollisionPair Struct Reference

Collision pair for tracking collisions.

`#include <CollisionSystem.hpp>`

Collaboration diagram for gme::CollisionPair:



Public Attributes

- `std::uint32_t` [entityId1](#)
- `std::uint32_t` [entityId2](#)
- `float` [overlapDistance](#)

18.25.1 Detailed Description

Collision pair for tracking collisions.

Definition at line 22 of file [CollisionSystem.hpp](#).

18.25.2 Member Data Documentation

18.25.2.1 entityId1

`std::uint32_t` gme::CollisionPair::entityId1

Definition at line 24 of file [CollisionSystem.hpp](#).

18.25.2.2 entityId2

`std::uint32_t` gme::CollisionPair::entityId2

Definition at line 25 of file [CollisionSystem.hpp](#).

18.25.2.3 overlapDistance

`float` gme::CollisionPair::overlapDistance

Definition at line 26 of file [CollisionSystem.hpp](#).

The documentation for this struct was generated from the following file:

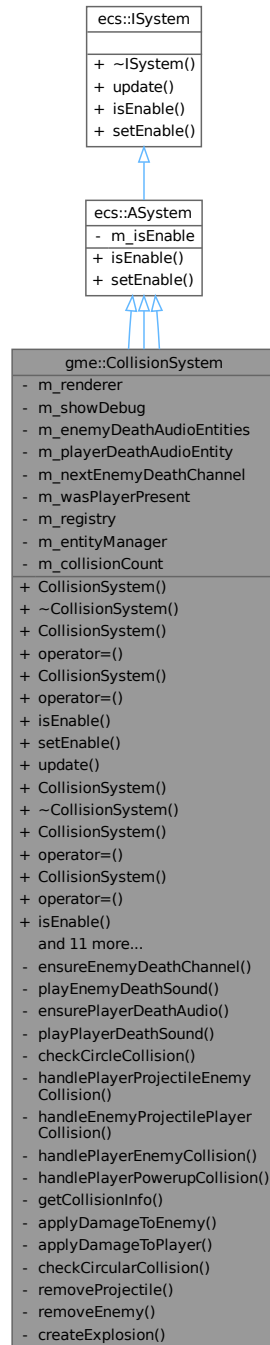
- [/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Server/include/RTypeServer/Systems/CollisionSystem.hpp](#)

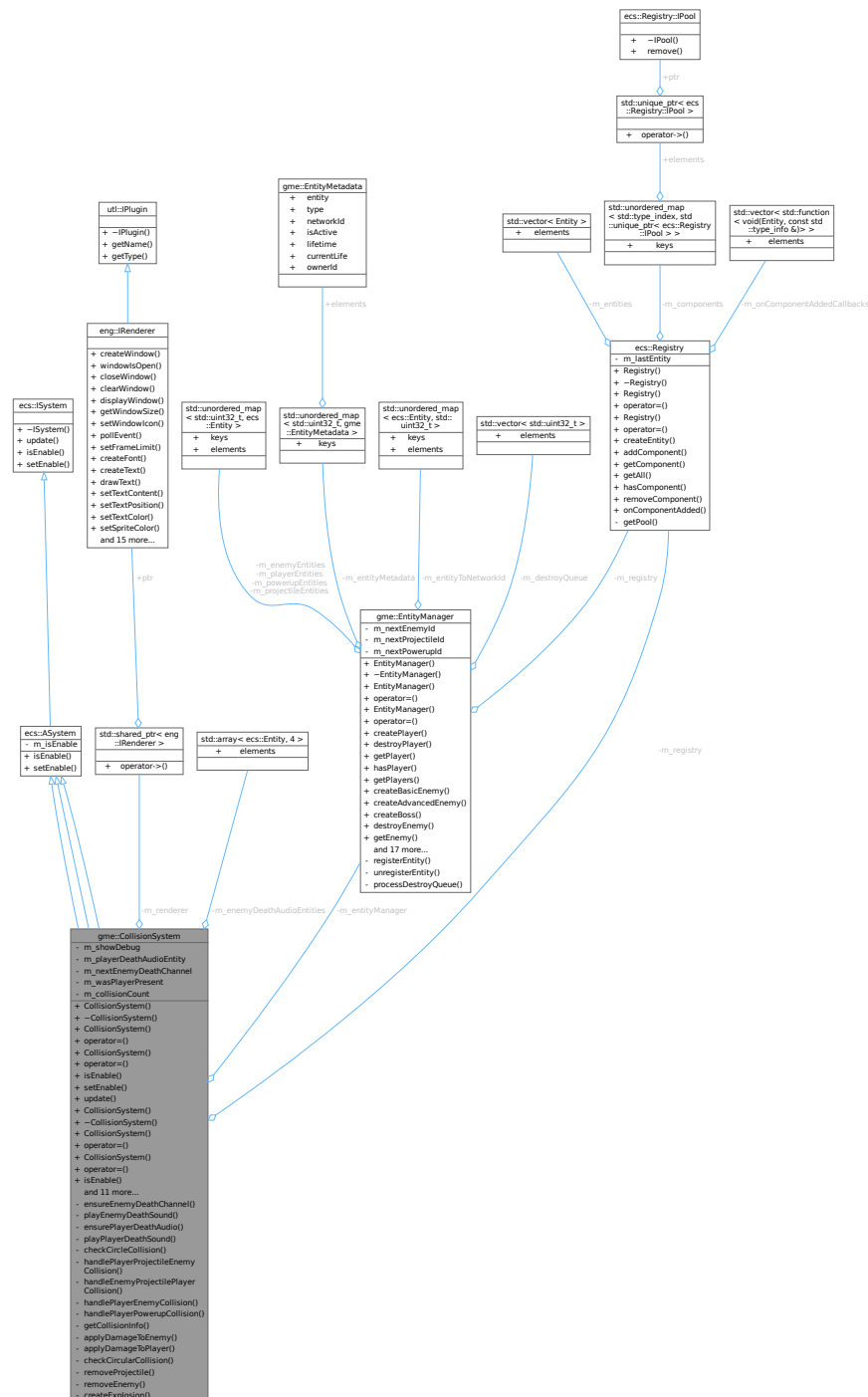
18.26 gme::CollisionSystem Class Reference

Handles collision detection and resolution on server.

```
#include <Collision.hpp>
```

Inheritance diagram for gme::CollisionSystem:





- `CollisionSystem` (const std::shared_ptr< eng::IRenderer > &renderer, bool &showDebug)
- `~CollisionSystem` () override=default
- `CollisionSystem` (const `CollisionSystem` &)=delete
- `CollisionSystem` & `operator=` (const `CollisionSystem` &)=delete
- `CollisionSystem` (`CollisionSystem` &&)=delete
- `CollisionSystem` & `operator=` (const `CollisionSystem` &&)=delete

- bool `isEnabled` () override
- void `setEnabled` (bool enable) override
- void `update` (ecs::Registry ®istry, float dt) override
- `CollisionSystem` (const std::shared_ptr< eng::IRenderer > &renderer, bool &showDebug)
- `~CollisionSystem` () override=default
- `CollisionSystem` (const `CollisionSystem` &)=delete
- `CollisionSystem` & `operator=` (const `CollisionSystem` &)=delete
- `CollisionSystem` (`CollisionSystem` &&)=delete
- `CollisionSystem` & `operator=` (`CollisionSystem` &&)=delete
- bool `isEnabled` () override
- void `setEnabled` (const bool enable) override
- void `update` (ecs::Registry ®istry, float dt) override
- `CollisionSystem` (ecs::Registry ®istry, `EntityManager` &entityManager)
- `~CollisionSystem` () override=default
- `CollisionSystem` (const `CollisionSystem` &)=delete
- `CollisionSystem` & `operator=` (const `CollisionSystem` &)=delete
- `CollisionSystem` (`CollisionSystem` &&)=delete
- `CollisionSystem` & `operator=` (`CollisionSystem` &&)=delete
- void `update` (ecs::Registry ®istry, float deltaTime) override
- size_t `getCollisionCount` () const
- void `resetCollisionCount` ()

Public Member Functions inherited from `ecs::ISystem`

- virtual `~ISystem` ()=default

Private Member Functions

- void `ensureEnemyDeathChannel` (ecs::Registry ®istry, const std::size_t channelIndex)
- void `playEnemyDeathSound` (ecs::Registry ®istry)
- void `ensurePlayerDeathAudio` (ecs::Registry ®istry)
- void `playPlayerDeathSound` (ecs::Registry ®istry)
- bool `checkCircleCollision` (float x1, float y1, float r1, float x2, float y2, float r2, float *overlap←Dist=nullptr) const
- void `handlePlayerProjectileEnemyCollision` ()
- void `handleEnemyProjectilePlayerCollision` ()
- void `handlePlayerEnemyCollision` ()
- void `handlePlayerPowerupCollision` ()
- bool `getCollisionInfo` (ecs::Entity entity, float &x, float &y, float &radius) const
- void `applyDamageToEnemy` (std::uint32_t enemyId, float damage)
- void `applyDamageToPlayer` (std::uint32_t playerId, float damage)

Static Private Member Functions

- static bool `checkCircularCollision` (const ecs::Transform &transform1, const ecs::Hitbox &hitbox1, const ecs::Transform &transform2, const ecs::Hitbox &hitbox2)
- static void `removeProjectile` (ecs::Registry ®istry, ecs::Entity entity)
- static void `removeEnemy` (ecs::Registry ®istry, ecs::Entity entity)
- static void `createExplosion` (ecs::Registry ®istry, float x, float y)

Private Attributes

- `const std::shared_ptr< eng::IRenderer > & m_renderer`
- `bool & m_showDebug`
- `std::array< ecs::Entity, 4 > m_enemyDeathAudioEntities {}`
- `ecs::Entity m_playerDeathAudioEntity = ecs::INVALID_ENTITY`
- `std::size_t m_nextEnemyDeathChannel = 0`
- `bool m_wasPlayerPresent = false`
- `ecs::Registry & m_registry`
- `EntityManager & m_entityManager`
- `size_t m_collisionCount = 0`

18.26.1 Detailed Description

Handles collision detection and resolution on server.

Definition at line 34 of file [CollisionSystem.hpp](#).

18.26.2 Constructor & Destructor Documentation

18.26.2.1 CollisionSystem() [1/9]

```
gme::CollisionSystem::CollisionSystem (
    const std::shared_ptr< eng::IRenderer > & renderer,
    bool & showDebug) [inline], [explicit]
```

Definition at line 15 of file [Collision.hpp](#).

18.26.2.2 ~CollisionSystem() [1/3]

```
gme::CollisionSystem::~~CollisionSystem () [override], [default]
```

18.26.2.3 CollisionSystem() [2/9]

```
gme::CollisionSystem::CollisionSystem (
    const CollisionSystem & ) [delete]
```

18.26.2.4 CollisionSystem() [3/9]

```
gme::CollisionSystem::CollisionSystem (
    CollisionSystem && ) [delete]
```

18.26.2.5 CollisionSystem() [4/9]

```
gme::CollisionSystem::CollisionSystem (
    const std::shared_ptr< eng::IRenderer > & renderer,
    bool & showDebug) [inline], [explicit]
```

Definition at line 28 of file [Collision.hpp](#).

References [ecs::INVALID_ENTITY](#), and [m_enemyDeathAudioEntities](#).

18.26.2.6 ~CollisionSystem() [2/3]

```
gme::CollisionSystem::~~CollisionSystem () [override], [default]
```

18.26.2.7 CollisionSystem() [5/9]

```
gme::CollisionSystem::CollisionSystem (
    const CollisionSystem & ) [delete]
```

18.26.2.8 CollisionSystem() [6/9]

```
gme::CollisionSystem::CollisionSystem (
    CollisionSystem && ) [delete]
```

18.26.2.9 CollisionSystem() [7/9]

```
gme::CollisionSystem::CollisionSystem (
    ecs::Registry & registry,
    EntityManager & entityManager) [inline], [explicit]
```

Definition at line 37 of file [CollisionSystem.hpp](#).

18.26.2.10 ~CollisionSystem() [3/3]

```
gme::CollisionSystem::~~CollisionSystem () [override], [default]
```

18.26.2.11 CollisionSystem() [8/9]

```
gme::CollisionSystem::CollisionSystem (
    const CollisionSystem & ) [delete]
```

18.26.2.12 CollisionSystem() [9/9]

```
gme::CollisionSystem::CollisionSystem (
    CollisionSystem && ) [delete]
```

18.26.3 Member Function Documentation

18.26.3.1 applyDamageToEnemy()

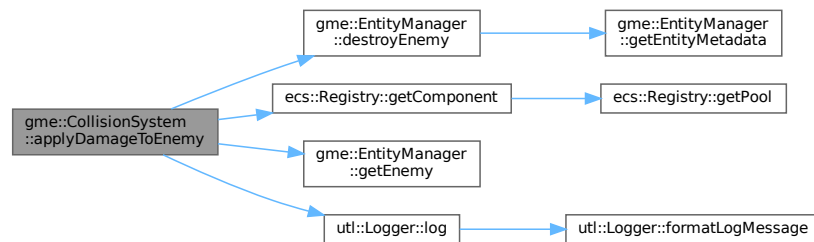
```
void gme::CollisionSystem::applyDamageToEnemy (
    std::uint32_t enemyId,
    float damage) [private]
```

Definition at line 193 of file [CollisionSystem.cpp](#).

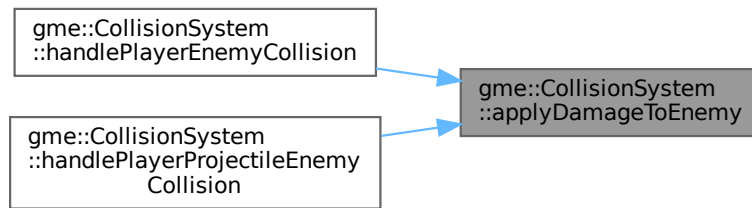
References [gme::EntityManager::destroyEnemy\(\)](#), [ecs::Registry::getComponent\(\)](#), [gme::EntityManager::getEnemy\(\)](#), [ecs::Enemy::health](#), [utl::INFO](#), [ecs::INVALID_ENTITY](#), [utl::Logger::log\(\)](#), [m_entityManager](#), and [m_registry](#).

Referenced by [handlePlayerEnemyCollision\(\)](#), and [handlePlayerProjectileEnemyCollision\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



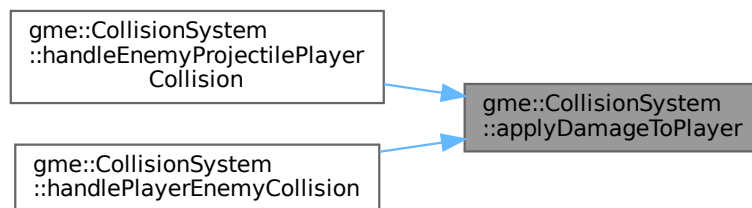
18.26.3.2 `applyDamageToPlayer()`

```
void gme::CollisionSystem::applyDamageToPlayer (
    std::uint32_t playerId,
    float damage) [private]
```

Definition at line 221 of file [CollisionSystem.cpp](#).

Referenced by [handleEnemyProjectilePlayerCollision\(\)](#), and [handlePlayerEnemyCollision\(\)](#).

Here is the caller graph for this function:



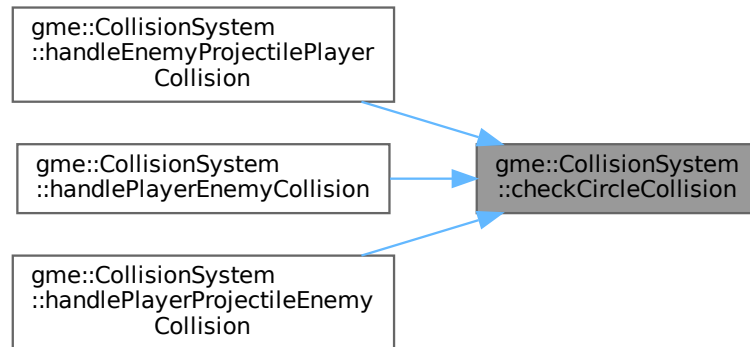
18.26.3.3 `checkCircleCollision()`

```
bool gme::CollisionSystem::checkCircleCollision (
    float x1,
    float y1,
    float r1,
    float x2,
    float y2,
    float r2,
    float * overlapDist = nullptr) const [inline], [private]
```

Definition at line 77 of file [CollisionSystem.hpp](#).

Referenced by [handleEnemyProjectilePlayerCollision\(\)](#), [handlePlayerEnemyCollision\(\)](#), and [handlePlayerProjectileEnemyCollision\(\)](#).

Here is the caller graph for this function:



18.26.3.4 checkCircularCollision()

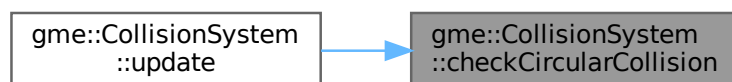
```
static bool gme::CollisionSystem::checkCircularCollision (
    const ecs::Transform & transform1,
    const ecs::Hitbox & hitbox1,
    const ecs::Transform & transform2,
    const ecs::Hitbox & hitbox2) [inline], [static], [private]
```

Definition at line 203 of file [Collision.hpp](#).

References [ecs::Hitbox::offsetX](#), [ecs::Hitbox::offsetY](#), [ecs::Hitbox::radius](#), [ecs::Transform::x](#), and [ecs::Transform::y](#).

Referenced by [update\(\)](#).

Here is the caller graph for this function:



18.26.3.5 createExplosion()

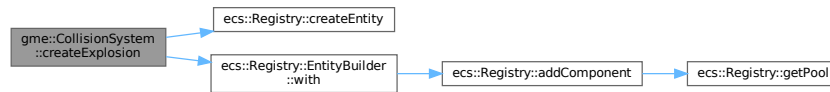
```
static void gme::CollisionSystem::createExplosion (
    ecs::Registry & registry,
    float x,
    float y) [inline], [static], [private]
```

Definition at line 279 of file [Collision.hpp](#).

References [utl::GameConfig::Explosion::ANIMATION_DURATION](#), [utl::GameConfig::Explosion::ANIMATION_FRAME](#), [ecs::Registry::createEntity\(\)](#), [utl::GameConfig::Explosion::FRAMES_PER_ROW](#), [utl::GameConfig::Explosion::LIFETIME](#), [utl::GameConfig::Explosion::SCALE](#), [utl::GameConfig::Explosion::SPRITE_HEIGHT](#), [utl::GameConfig::Explosion::SPR](#), [utl::Path::Texture::TEXTURE_EXPLOSION](#), and [ecs::Registry::EntityBuilder::with\(\)](#).

Referenced by [update\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.26.3.6 ensureEnemyDeathChannel()

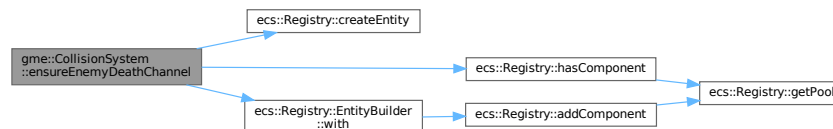
```
void gme::CollisionSystem::ensureEnemyDeathChannel (
    ecs::Registry & registry,
    const std::size_t channelIndex) [inline], [private]
```

Definition at line 297 of file [Collision.hpp](#).

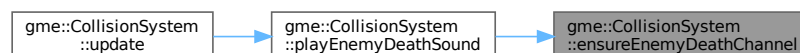
References [utl::Path::Audio::AUDIO_DEATH_ENEMIES](#), [ecs::Registry::createEntity\(\)](#), [ecs::Registry::hasComponent\(\)](#), [ecs::INVALID_ENTITY](#), [m_enemyDeathAudioEntities](#), and [ecs::Registry::EntityBuilder::with\(\)](#).

Referenced by [playEnemyDeathSound\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.26.3.7 ensurePlayerDeathAudio()

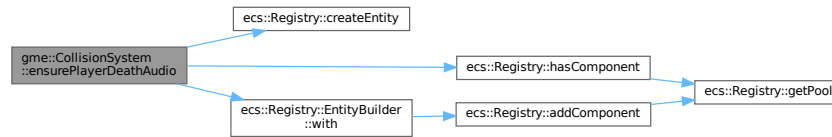
```
void gme::CollisionSystem::ensurePlayerDeathAudio (
    ecs::Registry & registry) [inline], [private]
```

Definition at line 333 of file [Collision.hpp](#).

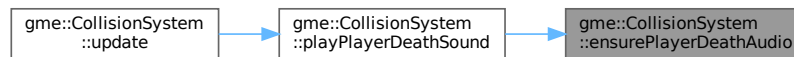
References [utl::Path::Audio::AUDIO_DEATH_ALLIES](#), [ecs::Registry::createEntity\(\)](#), [ecs::Registry::hasComponent\(\)](#), [ecs::INVALID_ENTITY](#), [m_playerDeathAudioEntity](#), and [ecs::Registry::EntityBuilder::with\(\)](#).

Referenced by [playPlayerDeathSound\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.26.3.8 getCollisionCount()

`size_t gme::CollisionSystem::getCollisionCount () const` [inline]

Definition at line 51 of file [CollisionSystem.hpp](#).

References [m_collisionCount](#).

18.26.3.9 getCollisionInfo()

`bool gme::CollisionSystem::getCollisionInfo (`
 [ecs::Entity](#) entity,
 float & x,
 float & y,
 float & radius) `const` [inline], [private]

Definition at line 93 of file [CollisionSystem.hpp](#).

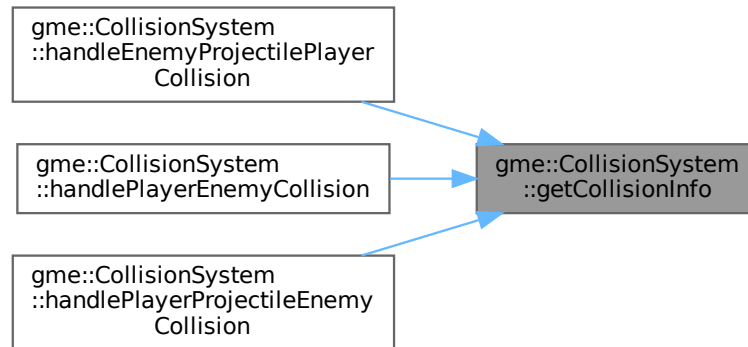
References [ecs::Registry::getComponent\(\)](#), [m_registry](#), and [ecs::Transform::y](#).

Referenced by [handleEnemyProjectilePlayerCollision\(\)](#), [handlePlayerEnemyCollision\(\)](#), and [handlePlayerProjectileEnemy](#)

Here is the call graph for this function:



Here is the caller graph for this function:



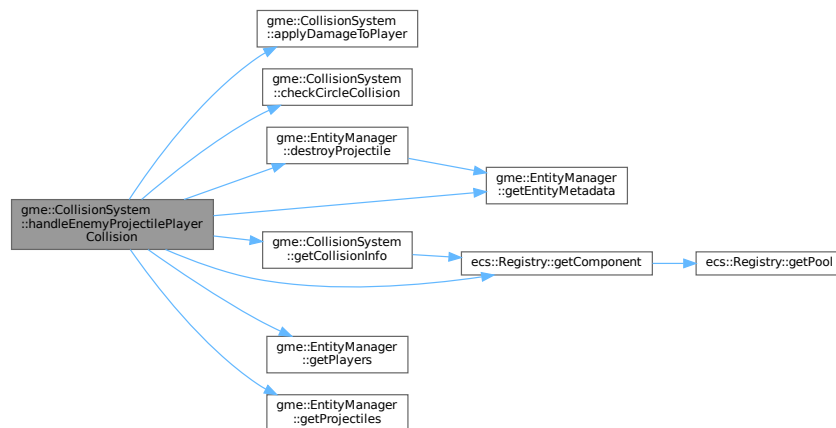
18.26.3.10 `handleEnemyProjectilePlayerCollision()`

`void gme::CollisionSystem::handleEnemyProjectilePlayerCollision ()` [private]

Definition at line 90 of file [CollisionSystem.cpp](#).

References [applyDamageToPlayer\(\)](#), [checkCircleCollision\(\)](#), [gme::EntityManager::destroyProjectile\(\)](#), [getCollisionInfo\(\)](#), [ecs::Registry::getComponent\(\)](#), [gme::EntityManager::getEntityMetadata\(\)](#), [gme::EntityManager::getPlayers\(\)](#), [gme::EntityManager::getProjectiles\(\)](#), [m_collisionCount](#), [m_entityManager](#), [m_registry](#), and [gme::PROJECTILE_ENEMY](#).

Here is the call graph for this function:



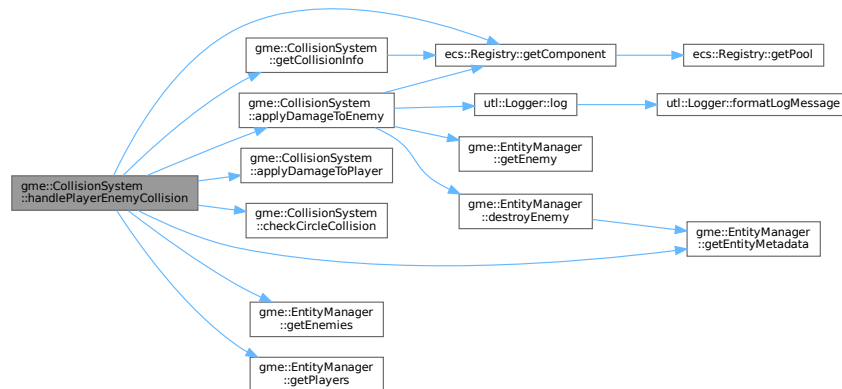
18.26.3.11 `handlePlayerEnemyCollision()`

`void gme::CollisionSystem::handlePlayerEnemyCollision ()` [private]

Definition at line 141 of file [CollisionSystem.cpp](#).

References [applyDamageToEnemy\(\)](#), [applyDamageToPlayer\(\)](#), [checkCircleCollision\(\)](#), [ecs::Enemy::damage](#), [getCollisionInfo\(\)](#), [ecs::Registry::getComponent\(\)](#), [gme::EntityManager::getEnemies\(\)](#), [gme::EntityManager::getEntityMetadata\(\)](#), [gme::EntityManager::getPlayers\(\)](#), [m_collisionCount](#), [m_entityManager](#), and [m_registry](#).

Here is the call graph for this function:



18.26.3.12 handlePlayerPowerupCollision()

void gme::CollisionSystem::handlePlayerPowerupCollision () [private]

Definition at line 187 of file [CollisionSystem.cpp](#).

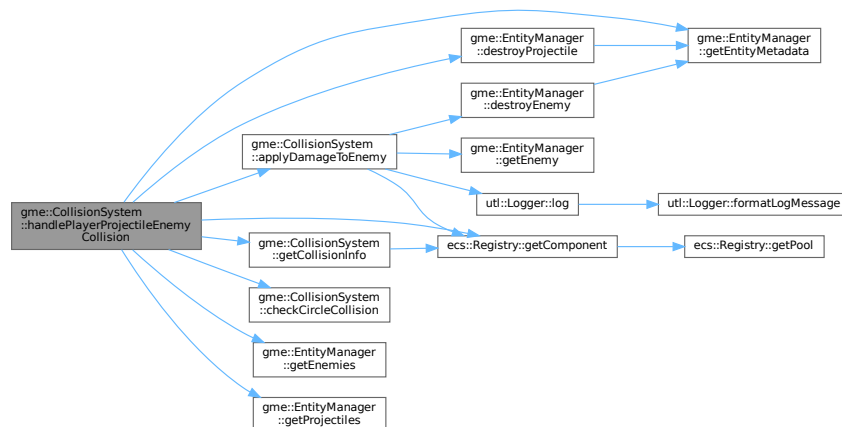
18.26.3.13 handlePlayerProjectileEnemyCollision()

void gme::CollisionSystem::handlePlayerProjectileEnemyCollision () [private]

Definition at line 27 of file [CollisionSystem.cpp](#).

References [applyDamageToEnemy\(\)](#), [checkCircleCollision\(\)](#), [gme::EntityManager::destroyProjectile\(\)](#), [getCollisionInfo\(\)](#), [ecs::Registry::getComponent\(\)](#), [gme::EntityManager::getEnemies\(\)](#), [gme::EntityManager::getEntityMetadata\(\)](#), [gme::EntityManager::getProjectiles\(\)](#), [m_collisionCount](#), [m_entityManager](#), [m_registry](#), and [gme::PROJECTILE_PLAYERS](#).

Here is the call graph for this function:



18.26.3.14 isEnabled() [1/2]

bool gme::CollisionSystem::isEnabled () [inline], [override], [virtual]

Reimplemented from [ecs::ASystem](#).

Definition at line 26 of file [Collision.hpp](#).

18.26.3.15 isEnabled() [2/2]

bool gme::CollisionSystem::isEnabled() [inline], [override], [virtual]

Reimplemented from [ecs::ASystem](#).

Definition at line 40 of file [Collision.hpp](#).

18.26.3.16 operator=() [1/6]

[CollisionSystem](#) & gme::CollisionSystem::operator= (
 [CollisionSystem](#) &&) [delete]

18.26.3.17 operator=() [2/6]

[CollisionSystem](#) & gme::CollisionSystem::operator= (
 [CollisionSystem](#) &&) [delete]

18.26.3.18 operator=() [3/6]

[CollisionSystem](#) & gme::CollisionSystem::operator= (
 const [CollisionSystem](#) &&) [delete]

18.26.3.19 operator=() [4/6]

[CollisionSystem](#) & gme::CollisionSystem::operator= (
 const [CollisionSystem](#) &) [delete]

18.26.3.20 operator=() [5/6]

[CollisionSystem](#) & gme::CollisionSystem::operator= (
 const [CollisionSystem](#) &) [delete]

18.26.3.21 operator=() [6/6]

[CollisionSystem](#) & gme::CollisionSystem::operator= (
 const [CollisionSystem](#) &) [delete]

18.26.3.22 playEnemyDeathSound()

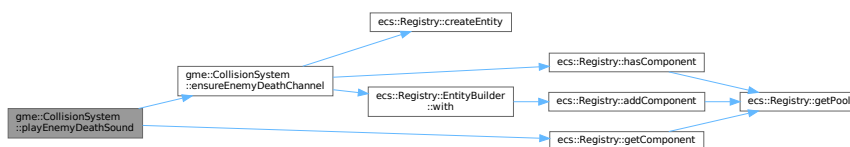
void gme::CollisionSystem::playEnemyDeathSound (
 [ecs::Registry](#) & registry) [inline], [private]

Definition at line 316 of file [Collision.hpp](#).

References [ensureEnemyDeathChannel\(\)](#), [ecs::Registry::getComponent\(\)](#), [ecs::INVALID_ENTITY](#), [m_enemyDeathAudioEntities](#), and [m_nextEnemyDeathChannel](#).

Referenced by [update\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.26.3.23 playPlayerDeathSound()

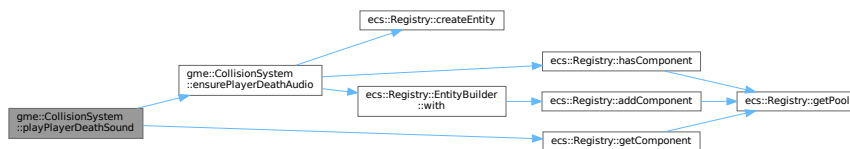
```
void gme::CollisionSystem::playPlayerDeathSound (
    ecs::Registry & registry) [inline], [private]
```

Definition at line 347 of file [Collision.hpp](#).

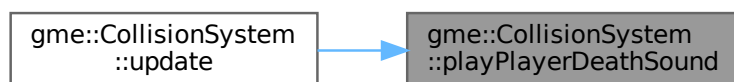
References [ensurePlayerDeathAudio\(\)](#), [ecs::Registry::getComponent\(\)](#), [ecs::INVALID_ENTITY](#), and [m_playerDeathAudioEntity](#).

Referenced by [update\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.26.3.24 removeEnemy()

```
static void gme::CollisionSystem::removeEnemy (
    ecs::Registry & registry,
    ecs::Entity entity) [inline], [static], [private]
```

Definition at line 247 of file [Collision.hpp](#).

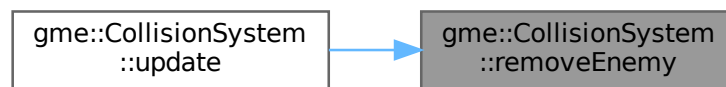
References [ecs::Registry::hasComponent\(\)](#), and [ecs::Registry::removeComponent\(\)](#).

Referenced by [update\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.26.3.25 removeProjectile()

```
static void gme::CollisionSystem::removeProjectile (
    ecs::Registry & registry,
    ecs::Entity entity) [inline], [static], [private]
```

Definition at line 219 of file [Collision.hpp](#).

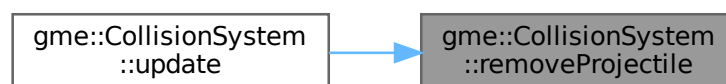
References [ecs::Registry::hasComponent\(\)](#), and [ecs::Registry::removeComponent\(\)](#).

Referenced by [update\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.26.3.26 resetCollisionCount()

void gme::CollisionSystem::resetCollisionCount () [inline]

Definition at line 52 of file [CollisionSystem.hpp](#).

References [m_collisionCount](#).

18.26.3.27 setEnable() [1/2]

void gme::CollisionSystem::setEnable (
 bool enable) [inline], [override], [virtual]

Reimplemented from [ecs::ASystem](#).

Definition at line 27 of file [Collision.hpp](#).

18.26.3.28 setEnable() [2/2]

void gme::CollisionSystem::setEnable (
 const bool enable) [inline], [override], [virtual]

Reimplemented from [ecs::ASystem](#).

Definition at line 41 of file [Collision.hpp](#).

18.26.3.29 update() [1/3]

void gme::CollisionSystem::update (
 [ecs::Registry](#) & registry,
 float deltaTime) [override], [virtual]

Implements [ecs::ISystem](#).

18.26.3.30 update() [2/3]

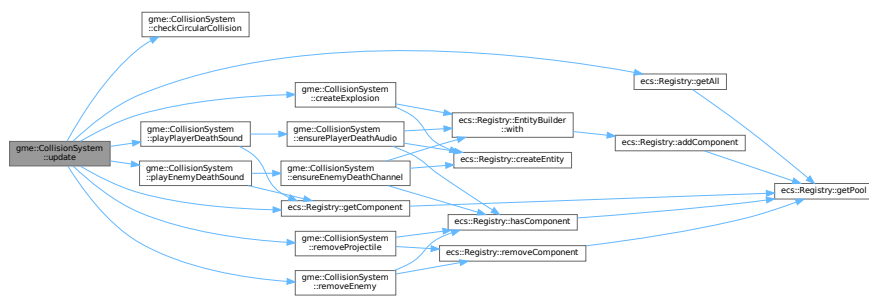
void gme::CollisionSystem::update (
 [ecs::Registry](#) & registry,
 float dt) [inline], [override], [virtual]

Implements [ecs::ISystem](#).

Definition at line 29 of file [Collision.hpp](#).

References [ecs::Registry::getAll\(\)](#), [ecs::Registry::getComponent\(\)](#), [ecs::Transform::y](#), and [ecs::Velocity::y](#).

Here is the call graph for this function:



18.26.3.31 update() [3/3]

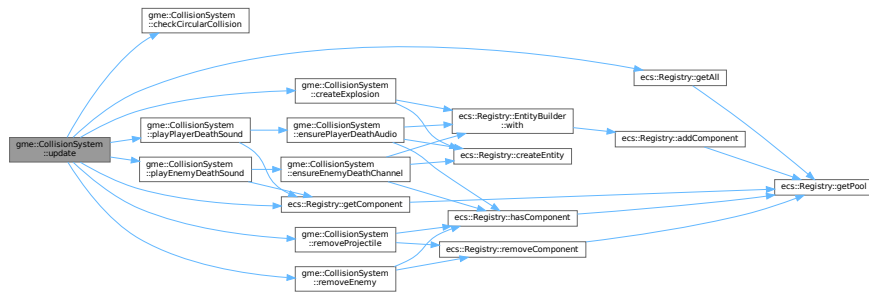
void gme::CollisionSystem::update (
 [ecs::Registry](#) & registry,
 float dt) [inline], [override], [virtual]

Implements [ecs::ISystem](#).

Definition at line 43 of file [Collision.hpp](#).

References [checkCircularCollision\(\)](#), [createExplosion\(\)](#), [ecs::Registry::getAll\(\)](#), [ecs::Registry::getComponent\(\)](#), [m_wasPlayerPresent](#), [playEnemyDeathSound\(\)](#), [playPlayerDeathSound\(\)](#), [removeEnemy\(\)](#), [removeProjectile\(\)](#), [ecs::Projectile::SUPERCHARGED](#), [ecs::Transform::y](#), and [ecs::Velocity::y](#).

Here is the call graph for this function:



18.26.4 Member Data Documentation

18.26.4.1 m_collisionCount

size_t gme::CollisionSystem::m_collisionCount = 0 [private]

Definition at line 57 of file [CollisionSystem.hpp](#).

Referenced by [getCollisionCount\(\)](#), [handleEnemyProjectilePlayerCollision\(\)](#), [handlePlayerEnemyCollision\(\)](#), [handlePlayerProjectileEnemyCollision\(\)](#), and [resetCollisionCount\(\)](#).

18.26.4.2 m_enemyDeathAudioEntities

std::array<ecs::Entity, 4> gme::CollisionSystem::m_enemyDeathAudioEntities {} [private]

Definition at line 197 of file [Collision.hpp](#).

Referenced by [CollisionSystem\(\)](#), [ensureEnemyDeathChannel\(\)](#), and [playEnemyDeathSound\(\)](#).

18.26.4.3 m_entityManager

EntityManager& gme::CollisionSystem::m_entityManager [private]

Definition at line 56 of file [CollisionSystem.hpp](#).

Referenced by [applyDamageToEnemy\(\)](#), [handleEnemyProjectilePlayerCollision\(\)](#), [handlePlayerEnemyCollision\(\)](#), and [handlePlayerProjectileEnemyCollision\(\)](#).

18.26.4.4 m_nextEnemyDeathChannel

std::size_t gme::CollisionSystem::m_nextEnemyDeathChannel = 0 [private]

Definition at line 199 of file [Collision.hpp](#).

Referenced by [playEnemyDeathSound\(\)](#).

18.26.4.5 m_playerDeathAudioEntity

ecs::Entity gme::CollisionSystem::m_playerDeathAudioEntity = ecs::INVALID_ENTITY [private]

Definition at line 198 of file [Collision.hpp](#).

Referenced by [ensurePlayerDeathAudio\(\)](#), and [playPlayerDeathSound\(\)](#).

18.26.4.6 m_registry

ecs::Registry& gme::CollisionSystem::m_registry [private]

Definition at line 55 of file [CollisionSystem.hpp](#).

Referenced by [applyDamageToEnemy\(\)](#), [getCollisionInfo\(\)](#), [handleEnemyProjectilePlayerCollision\(\)](#), [handlePlayerEnemyCollision\(\)](#), and [handlePlayerProjectileEnemyCollision\(\)](#).

18.26.4.7 m_renderer

const std::shared_ptr<eng::IRenderer> & gme::CollisionSystem::m_renderer [private]

Definition at line 99 of file [Collision.hpp](#).

18.26.4.8 m_showDebug

bool & gme::CollisionSystem::m_showDebug [private]

Definition at line 100 of file [Collision.hpp](#).

18.26.4.9 m_wasPlayerPresent

bool gme::CollisionSystem::m_wasPlayerPresent = false [private]

Definition at line 200 of file [Collision.hpp](#).

Referenced by [update\(\)](#).

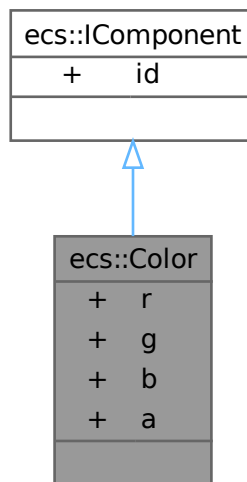
The documentation for this class was generated from the following files:

- [/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/include/RTypeClientMulti/Systems/Collision.hpp](#)
- [/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Solo/include/RTypeClientSolo/Systems/Collision.hpp](#)
- [/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Server/include/RTypeServerSystems/CollisionSystem.hpp](#)
- [/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Server/src/Systems/CollisionSystem.cpp](#)

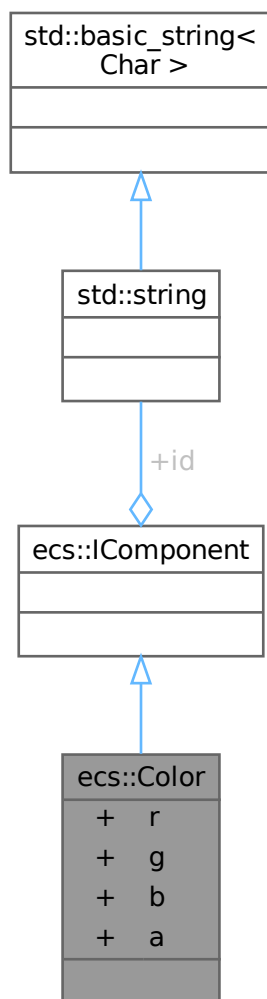
18.27 ecs::Color Struct Reference

#include <Component.hpp>

Inheritance diagram for ecs::Color:



Collaboration diagram for `ecs::Color`:



Public Attributes

- unsigned char `r` {}
- unsigned char `g` {}
- unsigned char `b` {}
- unsigned char `a` {}

Public Attributes inherited from `ecs::IComponent`

- std::string `id`

18.27.1 Detailed Description

Definition at line 24 of file [Component.hpp](#).

18.27.2 Member Data Documentation

18.27.2.1 a

unsigned char ecs::Color::a {}

Definition at line 29 of file [Component.hpp](#).

Referenced by [cli::Intro::update\(\)](#).

18.27.2.2 b

unsigned char ecs::Color::b {}

Definition at line 28 of file [Component.hpp](#).

Referenced by [gme::WaitingRoomScene::update\(\)](#).

18.27.2.3 g

unsigned char ecs::Color::g {}

Definition at line 27 of file [Component.hpp](#).

Referenced by [gme::WaitingRoomScene::update\(\)](#).

18.27.2.4 r

unsigned char ecs::Color::r {}

Definition at line 26 of file [Component.hpp](#).

Referenced by [cli::Menu::update\(\)](#), and [gme::WaitingRoomScene::update\(\)](#).

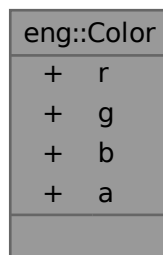
The documentation for this struct was generated from the following file:

- [/home/masina/Projects/Epitech/rtype/modules/ECS/include/ECS/Component.hpp](#)

18.28 eng::Color Struct Reference

```
#include <IRenderer.hpp>
```

Collaboration diagram for eng::Color:



Public Attributes

- unsigned char [r](#)
- unsigned char [g](#)
- unsigned char [b](#)
- unsigned char [a](#)

18.28.1 Detailed Description

Definition at line 15 of file [IRenderer.hpp](#).

18.28.2 Member Data Documentation

18.28.2.1 a

unsigned char eng::Color::a

Definition at line 20 of file [IRenderer.hpp](#).

Referenced by [gme::ConfigMulti::ConfigMulti\(\)](#), [gme::ConfigSolo::ConfigSolo\(\)](#), [ecs::StarfieldSystem::createComets\(\)](#), [ecs::StarfieldSystem::createNebulae\(\)](#), [ecs::StarfieldSystem::createPlanets\(\)](#), [gme::CreateRoomScene::CreateRoomScene\(\)](#), [ecs::StarfieldSystem::createShootingStars\(\)](#), [ecs::StarfieldSystem::createStars\(\)](#), [gme::Game::Game\(\)](#), [cli::Intro::Intro\(\)](#), [gme::JoinRoomScene::JoinRoomScene\(\)](#), [cli::Menu::Menu\(\)](#), [gme::ServerScene::ServerScene\(\)](#), [cli::Settings::Settings\(\)](#), [gme::JoinRoomScene::updateRoomDisplay\(\)](#), and [gme::WaitingRoomScene::WaitingRoomScene\(\)](#).

18.28.2.2 b

unsigned char eng::Color::b

Definition at line 19 of file [IRenderer.hpp](#).

Referenced by [gme::ConfigMulti::ConfigMulti\(\)](#), [gme::ConfigSolo::ConfigSolo\(\)](#), [ecs::StarfieldSystem::createComets\(\)](#), [ecs::StarfieldSystem::createNebulae\(\)](#), [ecs::StarfieldSystem::createPlanets\(\)](#), [gme::CreateRoomScene::CreateRoomScene\(\)](#), [ecs::StarfieldSystem::createShootingStars\(\)](#), [ecs::StarfieldSystem::createStars\(\)](#), [gme::Game::Game\(\)](#), [cli::Intro::Intro\(\)](#), [gme::JoinRoomScene::JoinRoomScene\(\)](#), [cli::Menu::Menu\(\)](#), [gme::ServerScene::ServerScene\(\)](#), [cli::Settings::Settings\(\)](#), [cli::Menu::update\(\)](#), [cli::Settings::update\(\)](#), [gme::ConfigMulti::update\(\)](#), [gme::ConfigSolo::update\(\)](#), [gme::CreateRoomScene::update\(\)](#), [gme::JoinRoomScene::update\(\)](#), [gme::ServerScene::update\(\)](#), [gme::WaitingRoomScene::update\(\)](#), [gme::JoinRoomScene::updateRoomDisplay\(\)](#), and [gme::WaitingRoomScene::WaitingRoomScene\(\)](#).

18.28.2.3 g

unsigned char eng::Color::g

Definition at line 18 of file [IRenderer.hpp](#).

Referenced by [gme::ConfigMulti::ConfigMulti\(\)](#), [gme::ConfigSolo::ConfigSolo\(\)](#), [ecs::StarfieldSystem::createComets\(\)](#), [ecs::StarfieldSystem::createNebulae\(\)](#), [ecs::StarfieldSystem::createPlanets\(\)](#), [gme::CreateRoomScene::CreateRoomScene\(\)](#), [ecs::StarfieldSystem::createShootingStars\(\)](#), [ecs::StarfieldSystem::createStars\(\)](#), [gme::Game::Game\(\)](#), [cli::Intro::Intro\(\)](#), [gme::JoinRoomScene::JoinRoomScene\(\)](#), [cli::Menu::Menu\(\)](#), [gme::ServerScene::ServerScene\(\)](#), [cli::Settings::Settings\(\)](#), [cli::Menu::update\(\)](#), [cli::Settings::update\(\)](#), [gme::ConfigMulti::update\(\)](#), [gme::ConfigSolo::update\(\)](#), [gme::CreateRoomScene::update\(\)](#), [gme::JoinRoomScene::update\(\)](#), [gme::ServerScene::update\(\)](#), [gme::WaitingRoomScene::update\(\)](#), [gme::JoinRoomScene::updateRoomDisplay\(\)](#), and [gme::WaitingRoomScene::WaitingRoomScene\(\)](#).

18.28.2.4 r

unsigned char eng::Color::r

Definition at line 17 of file [IRenderer.hpp](#).

Referenced by [gme::ConfigMulti::ConfigMulti\(\)](#), [gme::ConfigSolo::ConfigSolo\(\)](#), [ecs::StarfieldSystem::createComets\(\)](#), [ecs::StarfieldSystem::createNebulae\(\)](#), [ecs::StarfieldSystem::createPlanets\(\)](#), [gme::CreateRoomScene::CreateRoomScene\(\)](#), [ecs::StarfieldSystem::createShootingStars\(\)](#), [ecs::StarfieldSystem::createStars\(\)](#), [gme::Game::Game\(\)](#), [cli::Intro::Intro\(\)](#), [gme::JoinRoomScene::JoinRoomScene\(\)](#), [cli::Menu::Menu\(\)](#), [gme::ServerScene::ServerScene\(\)](#), [cli::Settings::Settings\(\)](#), [cli::Menu::update\(\)](#), [cli::Settings::update\(\)](#), [ecs::BeamSystem::update\(\)](#), [gme::BeamSystem::update\(\)](#), [gme::ConfigMulti::update\(\)](#), [gme::ConfigSolo::update\(\)](#), [gme::CreateRoomScene::update\(\)](#), [gme::JoinRoomScene::update\(\)](#), [gme::ServerScene::update\(\)](#), [gme::WaitingRoomScene::update\(\)](#), [gme::JoinRoomScene::updateRoomDisplay\(\)](#), and [gme::WaitingRoomScene::WaitingRoomScene\(\)](#).

The documentation for this struct was generated from the following file:

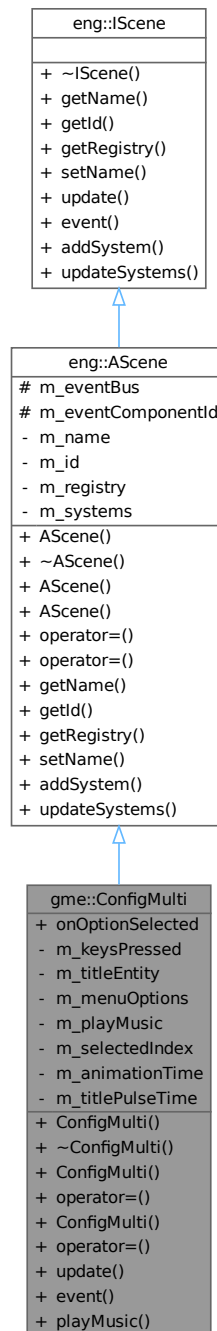
- [/home/masina/Projects/Epitech/rtype/modules/Interfaces/include/Interfaces/IRenderer.hpp](#)

18.29 gme::ConfigMulti Class Reference

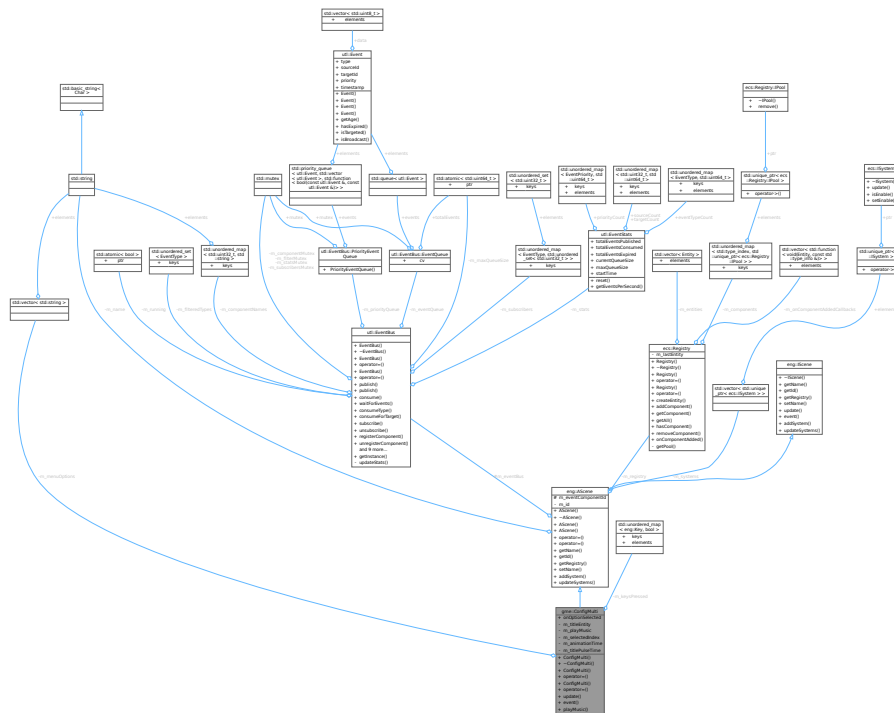
[ConfigMulti](#) scene.

```
#include <ConfigMulti.hpp>
```

Inheritance diagram for gme::ConfigMulti:



Collaboration diagram for gme::ConfigMulti:



Public Member Functions

- [ConfigMulti](#) ([eng::id](#) assignedId, const std::shared_ptr< [eng::IRenderer](#) > &renderer)
- [~ConfigMulti](#) () override=default
- [ConfigMulti](#) (const [ConfigMulti](#) &other)=delete
- [ConfigMulti](#) & operator= (const [ConfigMulti](#) &other)=delete
- [ConfigMulti](#) ([ConfigMulti](#) &&other)=delete
- [ConfigMulti](#) & operator= ([ConfigMulti](#) &&other)=delete
- void [update](#) (float dt, const [eng::WindowSize](#) &size) override
- void [event](#) (const [eng::Event](#) &event) override
- bool & [playMusic](#) ()

Public Member Functions inherited from [eng::AScene](#)

- [AScene](#) (const [id](#) assignedId)
- [~AScene](#) () override=default
- [AScene](#) (const [AScene](#) &other)=delete
- [AScene](#) ([AScene](#) &&other)=delete
- [AScene](#) & operator= (const [AScene](#) &other)=delete
- [AScene](#) & operator= ([AScene](#) &&other)=delete
- std::string & [getName](#) () override
- [id](#) [getId](#) () const override
- [ecs::Registry](#) & [getRegistry](#) () override
- void [setName](#) (const std::string &newName) override
- void [addSystem](#) (std::unique_ptr< [ecs::ISystem](#) > system) override
- void [updateSystems](#) (const float dt) override

Public Member Functions inherited from [eng::IScene](#)

- virtual [~IScene](#) ()=default

Public Attributes

- `std::function< void(const std::string &option)>` [onOptionSelected](#)

Private Attributes

- `std::unordered_map< eng::Key, bool >` [m_keysPressed](#)
- `ecs::Entity` [m_titleEntity](#)
- `const std::vector< std::string >` [m_menuOptions](#) = {"Create room", "Join room", "Go back to menu"}
- `bool` [m_playMusic](#) = false
- `int` [m_selectedIndex](#) = 0
- `float` [m_animationTime](#) = 0.0f
- `float` [m_titlePulseTime](#) = 0.0f

Additional Inherited Members

Protected Attributes inherited from [eng::AScene](#)

- `utl::EventBus &` [m_eventBus](#) = `utl::EventBus::getInstance()`
- `std::uint32_t` [m_eventComponentId](#) = 0

18.29.1 Detailed Description

[ConfigMulti](#) scene.

Definition at line 20 of file [ConfigMulti.hpp](#).

18.29.2 Constructor & Destructor Documentation

18.29.2.1 [ConfigMulti\(\)](#) [1/3]

```
gme::ConfigMulti::ConfigMulti (
    eng::id assignedId,
    const std::shared_ptr< eng::IRenderer > & renderer)
```

Definition at line 7 of file [configMulti.cpp](#).

References [eng::Color::a](#), [eng::Color::b](#), [utl::Config::Color::CYAN_ELECTRIC](#), [utl::Path::Font::FONTS_RTYPE](#), [eng::Color::g](#), [utl::Config::Color::GRAY_BLUE_SUBTLE](#), [ecs::IComponent::id](#), [m_menuOptions](#), [m_selectedIndex](#), [m_titleEntity](#), [eng::Color::r](#), and [ecs::Scale::x](#).

18.29.2.2 [~ConfigMulti\(\)](#)

```
gme::ConfigMulti::~~ConfigMulti () [override], [default]
```

18.29.2.3 [ConfigMulti\(\)](#) [2/3]

```
gme::ConfigMulti::ConfigMulti (
    const ConfigMulti & other) [delete]
```

18.29.2.4 [ConfigMulti\(\)](#) [3/3]

```
gme::ConfigMulti::ConfigMulti (
    ConfigMulti && other) [delete]
```

18.29.3 Member Function Documentation

18.29.3.1 [event\(\)](#)

```
void gme::ConfigMulti::event (
    const eng::Event & event) [override], [virtual]
```

Implements [eng::IScene](#).

Definition at line 128 of file [configMulti.cpp](#).

References [eng::Down](#), [eng::Enter](#), [eng::Event::key](#), [eng::KeyPressed](#), [eng::KeyReleased](#), [eng::Event::type](#), and [eng::Up](#).

18.29.3.2 operator=() [1/2]

[ConfigMulti](#) & gme::ConfigMulti::operator= (
[ConfigMulti](#) && other) [delete]

18.29.3.3 operator=() [2/2]

[ConfigMulti](#) & gme::ConfigMulti::operator= (
const [ConfigMulti](#) & other) [delete]

18.29.3.4 playMusic()

bool & gme::ConfigMulti::playMusic () [inline]

Definition at line 33 of file [ConfigMulti.hpp](#).

References [m_playMusic](#).

18.29.3.5 update()

void gme::ConfigMulti::update (
float dt,
const [eng::WindowSize](#) & size) [override], [virtual]

Implements [eng::IScene](#).

Definition at line 86 of file [configMulti.cpp](#).

References [eng::Color::b](#), [utl::Config::Color::CYAN_ELECTRIC](#), [eng::Color::g](#), [utl::Config::Color::GRAY_BLUE_SUBT](#) and [eng::Color::r](#).

18.29.4 Member Data Documentation

18.29.4.1 m_animationTime

float gme::ConfigMulti::m_animationTime = 0.0f [private]

Definition at line 43 of file [ConfigMulti.hpp](#).

18.29.4.2 m_keysPressed

std::unordered_map<[eng::Key](#), bool> gme::ConfigMulti::m_keysPressed [private]

Definition at line 38 of file [ConfigMulti.hpp](#).

18.29.4.3 m_menuOptions

const std::vector<std::string> gme::ConfigMulti::m_menuOptions = {"Create room", "Join room", "Go back to menu"}
[private]

Definition at line 40 of file [ConfigMulti.hpp](#).

Referenced by [ConfigMulti\(\)](#).

18.29.4.4 m_playMusic

bool gme::ConfigMulti::m_playMusic = false [private]

Definition at line 41 of file [ConfigMulti.hpp](#).

Referenced by [playMusic\(\)](#).

18.29.4.5 m_selectedIndex

int gme::ConfigMulti::m_selectedIndex = 0 [private]

Definition at line 42 of file [ConfigMulti.hpp](#).

Referenced by [ConfigMulti\(\)](#).

18.29.4.6 m_titleEntity

[ecs::Entity](#) gme::ConfigMulti::m_titleEntity [private]

Definition at line 39 of file [ConfigMulti.hpp](#).

Referenced by [ConfigMulti\(\)](#).

18.29.4.7 m_titlePulseTime

float gme::ConfigMulti::m_titlePulseTime = 0.0f [private]

Definition at line 44 of file [ConfigMulti.hpp](#).

18.29.4.8 onOptionSelected

std::function<void(const std::string &option)> gme::ConfigMulti::onOptionSelected

Definition at line 35 of file [ConfigMulti.hpp](#).

The documentation for this class was generated from the following files:

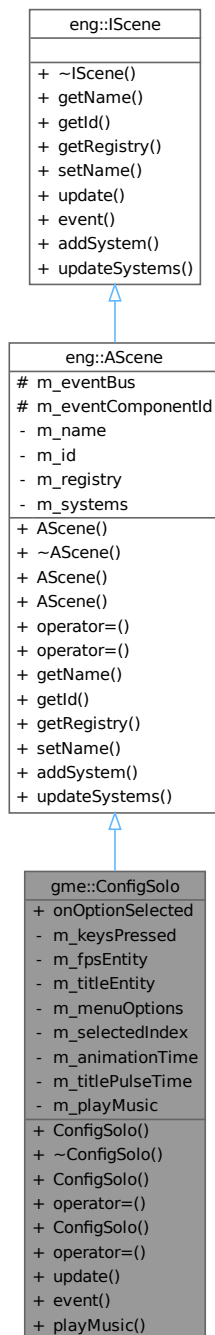
- [/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/include/RType↵ClientMulti/Scenes/ConfigMulti.hpp](#)
- [/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/src/scenes/configMulti.cpp](#)

18.30 gme::ConfigSolo Class Reference

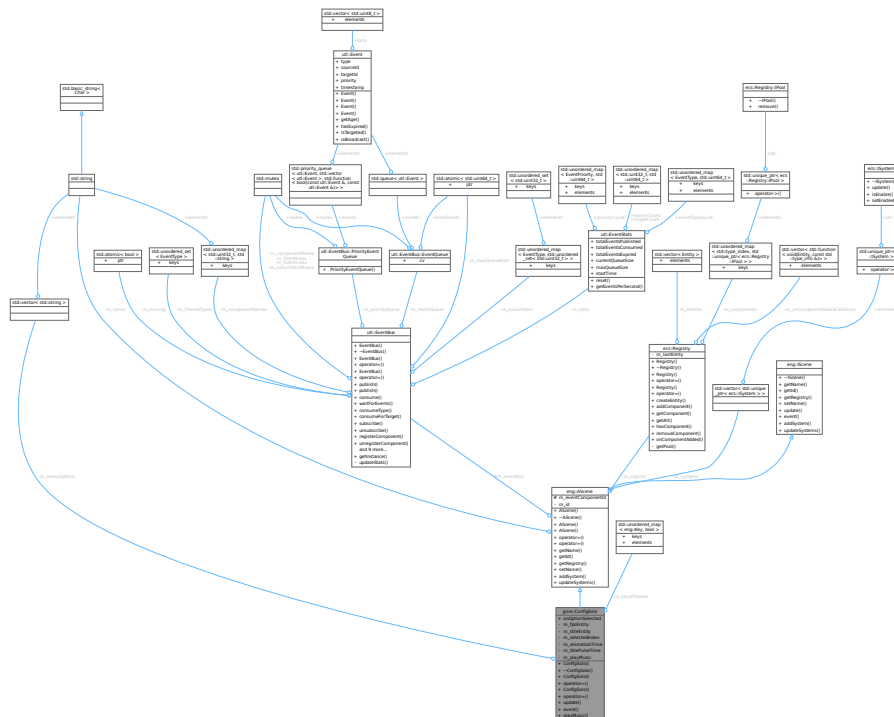
[ConfigSolo](#) scene.

#include <ConfigSolo.hpp>

Inheritance diagram for gme::ConfigSolo:



Collaboration diagram for gme::ConfigSolo:



Public Member Functions

- `ConfigSolo` (`eng::id` assignedId, const std::shared_ptr< `eng::IRenderer` > &renderer)
- `~ConfigSolo` () override=default
- `ConfigSolo` (const `ConfigSolo` &other)=delete
- `ConfigSolo` & `operator=` (const `ConfigSolo` &other)=delete
- `ConfigSolo` (`ConfigSolo` &&other)=delete
- `ConfigSolo` & `operator=` (`ConfigSolo` &&other)=delete
- void `update` (float dt, const `eng::WindowSize` &size) override
- void `event` (const `eng::Event` &event) override
- bool & `playMusic` ()

Public Member Functions inherited from [eng::AScene](#)

- `AScene` (const `id` assignedId)
- `~AScene` () override=default
- `AScene` (const `AScene` &other)=delete
- `AScene` (`AScene` &&other)=delete
- `AScene` & `operator=` (const `AScene` &other)=delete
- `AScene` & `operator=` (`AScene` &&other)=delete
- `std::string` & `getName` () override
- `id` `getId` () const override
- `ecs::Registry` & `getRegistry` () override
- void `setName` (const `std::string` &newName) override
- void `addSystem` (`std::unique_ptr`< `ecs::ISystem` > system) override
- void `updateSystems` (const float dt) override

Public Member Functions inherited from `eng::IScene`

- virtual $\sim$ IScene ()=default

Public Attributes

- `std::function< void(const std::string &option)>` [onOptionSelected](#)

Private Attributes

- `std::unordered_map< eng::Key, bool >` [m_keysPressed](#)
- `ecs::Entity` [m_fpsEntity](#)
- `ecs::Entity` [m_titleEntity](#)
- `const std::vector< std::string >` [m_menuOptions](#) = {"Level easy", "Level medium", "Go back to menu"}
- `int` [m_selectedIndex](#) = 0
- `float` [m_animationTime](#) = 0.0f
- `float` [m_titlePulseTime](#) = 0.0f
- `bool` [m_playMusic](#) = false

Additional Inherited Members

Protected Attributes inherited from [eng::AScene](#)

- `utl::EventBus &` [m_eventBus](#) = `utl::EventBus::getInstance()`
- `std::uint32_t` [m_eventComponentId](#) = 0

18.30.1 Detailed Description

[ConfigSolo](#) scene.

Definition at line 20 of file [ConfigSolo.hpp](#).

18.30.2 Constructor & Destructor Documentation

18.30.2.1 [ConfigSolo\(\)](#) [1/3]

```
gme::ConfigSolo::ConfigSolo (
    eng::id assignedId,
    const std::shared_ptr< eng::IRenderer > & renderer)
```

Definition at line 7 of file [configSolo.cpp](#).

References [eng::Color::a](#), [eng::Color::b](#), [utl::Config::Color::CYAN_ELECTRIC](#), [utl::Path::Font::FONTS_RTYPE](#), [eng::Color::g](#), [ecs::IComponent::id](#), [m_menuOptions](#), [m_selectedIndex](#), [m_titleEntity](#), [eng::Color::r](#), [utl::Config::Color::WHITE](#), and [ecs::Scale::x](#).

18.30.2.2 [~ConfigSolo\(\)](#)

```
gme::ConfigSolo::~ConfigSolo () [override], [default]
```

18.30.2.3 [ConfigSolo\(\)](#) [2/3]

```
gme::ConfigSolo::ConfigSolo (
    const ConfigSolo & other) [delete]
```

18.30.2.4 [ConfigSolo\(\)](#) [3/3]

```
gme::ConfigSolo::ConfigSolo (
    ConfigSolo && other) [delete]
```

18.30.3 Member Function Documentation

18.30.3.1 [event\(\)](#)

```
void gme::ConfigSolo::event (
    const eng::Event & event) [override], [virtual]
```

Implements [eng::IScene](#).

Definition at line 142 of file [configSolo.cpp](#).

References [eng::Down](#), [eng::Enter](#), [eng::Event::key](#), [eng::KeyPressed](#), [eng::KeyReleased](#), [eng::Left](#), [eng::Right](#), [eng::Space](#), [eng::Event::type](#), and [eng::Up](#).

18.30.3.2 operator=() [1/2]

[ConfigSolo](#) & gme::ConfigSolo::operator= (
[ConfigSolo](#) && other) [delete]

18.30.3.3 operator=() [2/2]

[ConfigSolo](#) & gme::ConfigSolo::operator= (
const [ConfigSolo](#) & other) [delete]

18.30.3.4 playMusic()

bool & gme::ConfigSolo::playMusic () [inline]

Definition at line 33 of file [ConfigSolo.hpp](#).

References [m_playMusic](#).

18.30.3.5 update()

void gme::ConfigSolo::update (
float dt,
const [eng::WindowSize](#) & size) [override], [virtual]

Implements [eng::IScene](#).

Definition at line 84 of file [configSolo.cpp](#).

References [eng::Color::b](#), [utl::Config::Color::CYAN_ELECTRIC](#), [eng::Color::g](#), [utl::Config::Color::GRAY_BLUE_SUBT](#), and [eng::Color::r](#).

18.30.4 Member Data Documentation

18.30.4.1 m_animationTime

float gme::ConfigSolo::m_animationTime = 0.0f [private]

Definition at line 44 of file [ConfigSolo.hpp](#).

18.30.4.2 m_fpsEntity

[ecs::Entity](#) gme::ConfigSolo::m_fpsEntity [private]

Definition at line 39 of file [ConfigSolo.hpp](#).

18.30.4.3 m_keysPressed

std::unordered_map<[eng::Key](#), bool> gme::ConfigSolo::m_keysPressed [private]

Definition at line 38 of file [ConfigSolo.hpp](#).

18.30.4.4 m_menuOptions

const std::vector<std::string> gme::ConfigSolo::m_menuOptions = {"Level easy", "Level medium", "Go back to menu"}
[private]

Definition at line 41 of file [ConfigSolo.hpp](#).

Referenced by [ConfigSolo\(\)](#).

18.30.4.5 m_playMusic

bool gme::ConfigSolo::m_playMusic = false [private]

Definition at line 46 of file [ConfigSolo.hpp](#).

Referenced by [playMusic\(\)](#).

18.30.4.6 m_selectedIndex

int gme::ConfigSolo::m_selectedIndex = 0 [private]

Definition at line 43 of file [ConfigSolo.hpp](#).

Referenced by [ConfigSolo\(\)](#).

18.30.4.7 m_titleEntity

[ecs::Entity](#) gme::ConfigSolo::m_titleEntity [private]

Definition at line 40 of file [ConfigSolo.hpp](#).

Referenced by [ConfigSolo\(\)](#).

18.30.4.8 m_titlePulseTime

float gme::ConfigSolo::m_titlePulseTime = 0.0f [private]

Definition at line 45 of file [ConfigSolo.hpp](#).

18.30.4.9 onOptionSelected

std::function<void(const std::string &option)> gme::ConfigSolo::onOptionSelected

Definition at line 35 of file [ConfigSolo.hpp](#).

The documentation for this class was generated from the following files:

- /home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Solo/include/RTypeClient↔ Solo/Scenes/[ConfigSolo.hpp](#)
- /home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Solo/src/scenes/[configSolo.cpp](#)

18.31 eng::ConnectionStats Struct Reference

Connection statistics.

#include <AsioClient.hpp>

Collaboration diagram for eng::ConnectionStats:

eng::ConnectionStats
+ packetsSent
+ packetsReceived
+ bytesTransferred
+ packetsLost
+ connectionTime

Public Attributes

- std::uint32_t [packetsSent](#) = 0
- std::uint32_t [packetsReceived](#) = 0
- std::uint32_t [bytesTransferred](#) = 0
- std::uint32_t [packetsLost](#) = 0
- std::chrono::steady_clock::time_point [connectionTime](#)

18.31.1 Detailed Description

Connection statistics.

Definition at line 44 of file [AsioClient.hpp](#).

18.31.2 Member Data Documentation

18.31.2.1 bytesTransferred

`std::uint32_t eng::ConnectionStats::bytesTransferred = 0`

Definition at line 48 of file [AsioClient.hpp](#).

Referenced by [eng::AsioClient::handleReceive\(\)](#), and [eng::AsioClient::sendPacketImmediate\(\)](#).

18.31.2.2 connectionTime

`std::chrono::steady_clock::time_point eng::ConnectionStats::connectionTime`

Definition at line 50 of file [AsioClient.hpp](#).

Referenced by [eng::AsioClient::AsioClient\(\)](#).

18.31.2.3 packetsLost

`std::uint32_t eng::ConnectionStats::packetsLost = 0`

Definition at line 49 of file [AsioClient.hpp](#).

18.31.2.4 packetsReceived

`std::uint32_t eng::ConnectionStats::packetsReceived = 0`

Definition at line 47 of file [AsioClient.hpp](#).

Referenced by [eng::AsioClient::handleReceive\(\)](#).

18.31.2.5 packetsSent

`std::uint32_t eng::ConnectionStats::packetsSent = 0`

Definition at line 46 of file [AsioClient.hpp](#).

Referenced by [eng::AsioClient::sendPacketImmediate\(\)](#).

The documentation for this struct was generated from the following file:

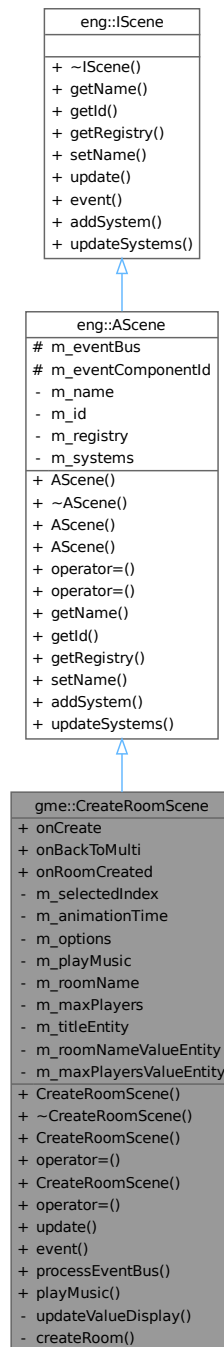
- [/home/masina/Projects/Epitech/rtype/plugins/Network/Asio/Client/include/AsioClient/AsioClient.hpp](#)

18.32 gme::CreateRoomScene Class Reference

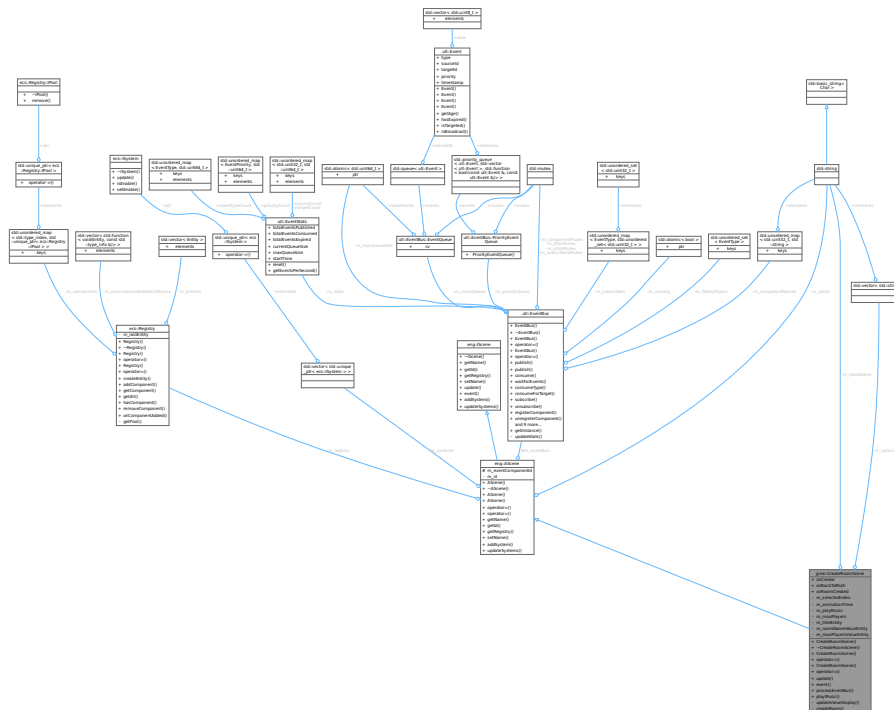
Create room scene.

`#include <CreateRoom.hpp>`

Inheritance diagram for gme::CreateRoomScene:



Collaboration diagram for gme::CreateRoomScene:



Public Member Functions

- `CreateRoomScene (eng::id assignedId, const std::shared_ptr< eng::IRenderer > &renderer)`
- `~CreateRoomScene ()` override=default
- `CreateRoomScene (const CreateRoomScene &other)=delete`
- `CreateRoomScene & operator= (const CreateRoomScene &other)=delete`
- `CreateRoomScene (CreateRoomScene &&other)=delete`
- `CreateRoomScene & operator= (CreateRoomScene &&other)=delete`
- `void update (float dt, const eng::WindowSize &size)` override
- `void event (const eng::Event &event)` override
- `void processEventBus ()` const
- `bool & playMusic ()`

Public Member Functions inherited from `eng::AScene`

- `AScene` (const `id` assignedId)
- `~AScene` () override=default
- `AScene` (const `AScene` &other)=delete
- `AScene` (`AScene` &&other)=delete
- `AScene` & `operator=` (const `AScene` &other)=delete
- `AScene` & `operator=` (`AScene` &&other)=delete
- `std::string` & `getName` () override
- `id` `getId` () const override
- `ecs::Registry` & `getRegistry` () override
- void `setName` (const `std::string` &newName) override
- void `addSystem` (`std::unique_ptr`< `ecs::ISystem` > system) override
- void `updateSystems` (const float dt) override

Public Member Functions inherited from [eng::IScene](#)

- virtual $\sim$ IScene ()=default

Public Attributes

- `std::function< void(const std::string &roomName, int maxPlayers)>` [onCreate](#)
- `std::function< void()>` [onBackToMulti](#)
- `std::function< void(int lobbyId, const rnp::LobbyInfo *lobbyInfo)>` [onRoomCreated](#)

Private Member Functions

- `void` [updateValueDisplay](#) ()
- `void` [createRoom](#) () `const`

Private Attributes

- `size_t` [m_selectedIndex](#) = 0
- `float` [m_animationTime](#) = 0.0f
- `const std::vector< std::string >` [m_options](#) = {"Room Name", "Max Players", "Create", "Back"}
- `bool` [m_playMusic](#) = false
- `std::string` [m_roomName](#) = "My Room"
- `int` [m_maxPlayers](#) = 4
- `ecs::Entity` [m_titleEntity](#) = 0
- `ecs::Entity` [m_roomNameValueEntity](#) = 0
- `ecs::Entity` [m_maxPlayersValueEntity](#) = 0

Additional Inherited Members

Protected Attributes inherited from [eng::AScene](#)

- `utl::EventBus` & [m_eventBus](#) = `utl::EventBus::getInstance()`
- `std::uint32_t` [m_eventComponentId](#) = 0

18.32.1 Detailed Description

Create room scene.

Definition at line 18 of file [CreateRoom.hpp](#).

18.32.2 Constructor & Destructor Documentation

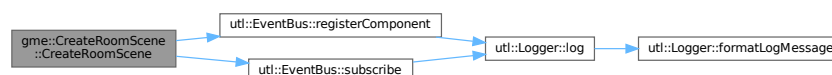
18.32.2.1 CreateRoomScene() [1/3]

```
gme::CreateRoomScene::CreateRoomScene (
    eng::id assignedId,
    const std::shared_ptr< eng::IRenderer > & renderer)
```

Definition at line 98 of file [createRoom.cpp](#).

References [eng::Color::a](#), [eng::Color::b](#), [utl::Config::Color::CYAN_ELECTRIC](#), [utl::Path::Font::FONTS_RTYPE](#), [eng::Color::g](#), [utl::Config::Color::GRAY_BLUE_SUBTLE](#), [ecs::IComponent::id](#), [utl::LOBBY_CREATE_RESPONSE](#), [eng::AScene::m_eventBus](#), [eng::AScene::m_eventComponentId](#), [m_maxPlayers](#), [m_maxPlayersValueEntity](#), [m_options](#), [m_roomName](#), [m_roomNameValueEntity](#), [m_titleEntity](#), [eng::Color::r](#), [utl::EventBus::registerComponent\(\)](#), [utl::EventBus::subscribe\(\)](#), and [utl::Config::Color::TEXT_VALUE_COLOR](#).

Here is the call graph for this function:



18.32.2.2 ~CreateRoomScene()

```
gme::CreateRoomScene::~CreateRoomScene () [override], [default]
```

18.32.2.3 CreateRoomScene() [2/3]

`gme::CreateRoomScene::CreateRoomScene (`
 const [CreateRoomScene](#) & other) [delete]

18.32.2.4 CreateRoomScene() [3/3]

`gme::CreateRoomScene::CreateRoomScene (`
 [CreateRoomScene](#) && other) [delete]

18.32.3 Member Function Documentation

18.32.3.1 createRoom()

`void gme::CreateRoomScene::createRoom () const` [private]

Definition at line 310 of file [createRoom.cpp](#).

References [utl::INFO](#), [utl::LOBBY_CREATE](#), [utl::Logger::log\(\)](#), [rnp::PacketLobbyCreate::maxPlayers](#), and [utl::NETWORK_CLIENT](#).

Here is the call graph for this function:



18.32.3.2 event()

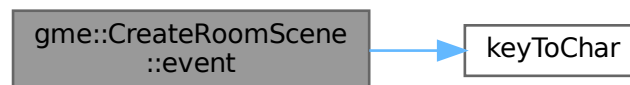
`void gme::CreateRoomScene::event (`
 const [eng::Event](#) & event) [override], [virtual]

Implements [eng::IScene](#).

Definition at line 210 of file [createRoom.cpp](#).

References [eng::Delete](#), [eng::Down](#), [eng::Enter](#), [eng::Escape](#), [eng::Event::key](#), [eng::KeyPressed](#), [eng::KeyReleased](#), [keyToChar\(\)](#), [eng::Left](#), [eng::Right](#), [eng::Event::type](#), and [eng::Up](#).

Here is the call graph for this function:



18.32.3.3 operator=() [1/2]

[CreateRoomScene](#) & `gme::CreateRoomScene::operator= (`
 const [CreateRoomScene](#) & other) [delete]

18.32.3.4 operator=() [2/2]

[CreateRoomScene](#) & `gme::CreateRoomScene::operator= (`
 [CreateRoomScene](#) && other) [delete]

18.32.3.5 playMusic()

bool & gme::CreateRoomScene::playMusic () [inline]

Definition at line 32 of file [CreateRoom.hpp](#).

References [m_playMusic](#).

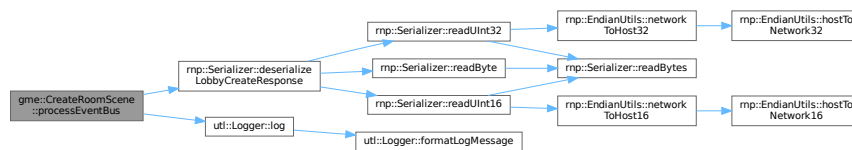
18.32.3.6 processEventBus()

void gme::CreateRoomScene::processEventBus () const

Definition at line 332 of file [createRoom.cpp](#).

References [rmp::Serializer::deserializeLobbyCreateResponse\(\)](#), [utl::INFO](#), [utl::LOBBY_CREATE_RESPONSE](#), [utl::Logger::log\(\)](#), and [utl::WARNING](#).

Here is the call graph for this function:



18.32.3.7 update()

void gme::CreateRoomScene::update (
 float dt,
 const [eng::WindowSize](#) & size) [override], [virtual]

Implements [eng::IScene](#).

Definition at line 173 of file [createRoom.cpp](#).

References [eng::Color::b](#), [eng::Color::g](#), [utl::Config::Color::GRAY_BLUE_SUBTLE](#), and [eng::Color::r](#).

18.32.3.8 updateValueDisplay()

void gme::CreateRoomScene::updateValueDisplay () [private]

Definition at line 295 of file [createRoom.cpp](#).

18.32.4 Member Data Documentation

18.32.4.1 m_animationTime

float gme::CreateRoomScene::m_animationTime = 0.0f [private]

Definition at line 40 of file [CreateRoom.hpp](#).

18.32.4.2 m_maxPlayers

int gme::CreateRoomScene::m_maxPlayers = 4 [private]

Definition at line 44 of file [CreateRoom.hpp](#).

Referenced by [CreateRoomScene\(\)](#).

18.32.4.3 m_maxPlayersValueEntity

[ecs::Entity](#) gme::CreateRoomScene::m_maxPlayersValueEntity = 0 [private]

Definition at line 48 of file [CreateRoom.hpp](#).

Referenced by [CreateRoomScene\(\)](#).

18.32.4.4 m_options

const std::vector<std::string> gme::CreateRoomScene::m_options = {"Room Name", "Max Players", "Create", "Back"} [private]

Definition at line 41 of file [CreateRoom.hpp](#).
 Referenced by [CreateRoomScene\(\)](#).

18.32.4.5 m_playMusic

bool gme::CreateRoomScene::m_playMusic = false [private]
 Definition at line 42 of file [CreateRoom.hpp](#).
 Referenced by [playMusic\(\)](#).

18.32.4.6 m_roomName

std::string gme::CreateRoomScene::m_roomName = "My Room" [private]
 Definition at line 43 of file [CreateRoom.hpp](#).
 Referenced by [CreateRoomScene\(\)](#).

18.32.4.7 m_roomNameValueEntity

[ecs::Entity](#) gme::CreateRoomScene::m_roomNameValueEntity = 0 [private]
 Definition at line 47 of file [CreateRoom.hpp](#).
 Referenced by [CreateRoomScene\(\)](#).

18.32.4.8 m_selectedIndex

size_t gme::CreateRoomScene::m_selectedIndex = 0 [private]
 Definition at line 39 of file [CreateRoom.hpp](#).

18.32.4.9 m_titleEntity

[ecs::Entity](#) gme::CreateRoomScene::m_titleEntity = 0 [private]
 Definition at line 46 of file [CreateRoom.hpp](#).
 Referenced by [CreateRoomScene\(\)](#).

18.32.4.10 onBackToMulti

std::function<void()> gme::CreateRoomScene::onBackToMulti
 Definition at line 35 of file [CreateRoom.hpp](#).

18.32.4.11 onCreate

std::function<void(const std::string &roomName, int maxPlayers)> gme::CreateRoomScene::onCreate
 Definition at line 34 of file [CreateRoom.hpp](#).

18.32.4.12 onRoomCreated

std::function<void(int lobbyId, const [rnp::LobbyInfo](#) *lobbyInfo)> gme::CreateRoomScene::onRoomCreated
 Definition at line 36 of file [CreateRoom.hpp](#).

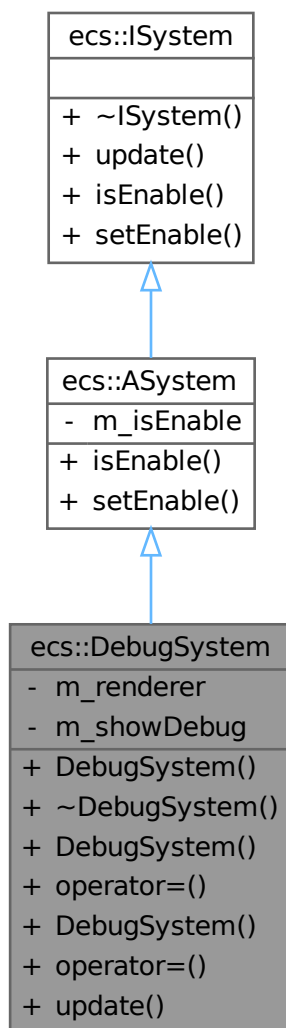
The documentation for this class was generated from the following files:

- /home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/include/RType↵ ClientMulti/Scenes/[CreateRoom.hpp](#)
- /home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/src/scenes/[createRoom.cpp](#)

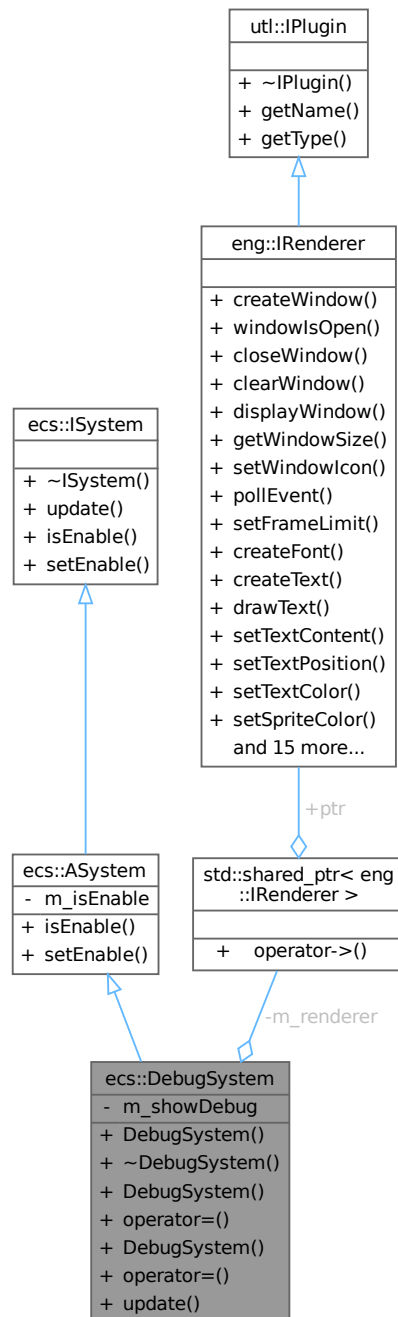
18.33 ecs::DebugSystem Class Reference

Class for debug system.
 #include <Debug.hpp>

Inheritance diagram for ecs::DebugSystem:



Collaboration diagram for ecs::DebugSystem:



Public Member Functions

- [DebugSystem](#) (const std::shared_ptr< [eng::IRenderer](#) > &renderer, bool &showDebug)
- [~DebugSystem](#) () override=default
- [DebugSystem](#) (const [DebugSystem](#) &)=delete
- [DebugSystem](#) & operator= (const [DebugSystem](#) &)=delete
- [DebugSystem](#) ([DebugSystem](#) &&)=delete
- [DebugSystem](#) & operator= ([DebugSystem](#) &&)=delete
- void [update](#) ([Registry](#) ®istry, float dt) override

Public Member Functions inherited from [ecs::ASystem](#)

- bool [isEnabled](#) () override
- void [setEnabled](#) (const bool enable) override

Public Member Functions inherited from [ecs::ISystem](#)

- virtual [~ISystem](#) ()=default

Private Attributes

- const std::shared_ptr< [eng::IRenderer](#) > & [m_renderer](#)
- bool & [m_showDebug](#)

18.33.1 Detailed Description

Class for debug system.

Definition at line 23 of file [Debug.hpp](#).

18.33.2 Constructor & Destructor Documentation

18.33.2.1 DebugSystem() [1/3]

```
ecs::DebugSystem::DebugSystem (
    const std::shared_ptr< eng::IRenderer > & renderer,
    bool & showDebug) [inline], [explicit]
```

Definition at line 26 of file [Debug.hpp](#).

18.33.2.2 ~DebugSystem()

```
ecs::DebugSystem::~~DebugSystem () [override], [default]
```

18.33.2.3 DebugSystem() [2/3]

```
ecs::DebugSystem::DebugSystem (
    const DebugSystem & ) [delete]
```

18.33.2.4 DebugSystem() [3/3]

```
ecs::DebugSystem::DebugSystem (
    DebugSystem && ) [delete]
```

18.33.3 Member Function Documentation

18.33.3.1 operator=() [1/2]

```
DebugSystem & ecs::DebugSystem::operator= (
    const DebugSystem & ) [delete]
```

18.33.3.2 operator=() [2/2]

```
DebugSystem & ecs::DebugSystem::operator= (
    DebugSystem && ) [delete]
```

18.33.3.3 update()

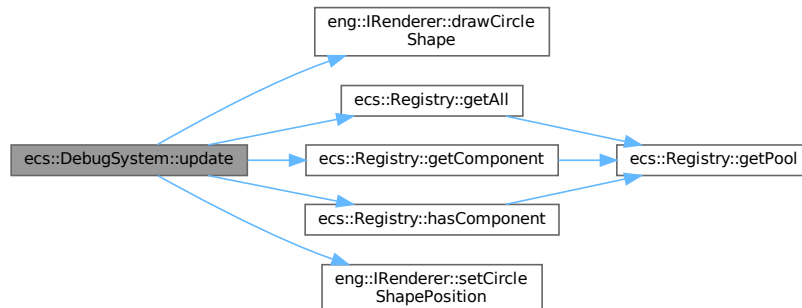
```
void ecs::DebugSystem::update (
    Registry & registry,
    float dt) [inline], [override], [virtual]
```

Implements [ecs::ISystem](#).

Definition at line 38 of file [Debug.hpp](#).

References [eng::IRenderer::drawCircleShape\(\)](#), [ecs::Registry::getAll\(\)](#), [ecs::Registry::getComponent\(\)](#), [ecs::Registry::hasComponent\(\)](#), [m_renderer](#), [m_showDebug](#), [eng::IRenderer::setCircleShapePosition\(\)](#), and [ecs::Transform::y](#).

Here is the call graph for this function:



18.33.4 Member Data Documentation

18.33.4.1 m_renderer

`const std::shared_ptr<eng::IRenderer>& ecs::DebugSystem::m_renderer` [private]

Definition at line 68 of file [Debug.hpp](#).

Referenced by [update\(\)](#).

18.33.4.2 m_showDebug

`bool& ecs::DebugSystem::m_showDebug` [private]

Definition at line 69 of file [Debug.hpp](#).

Referenced by [update\(\)](#).

The documentation for this class was generated from the following file:

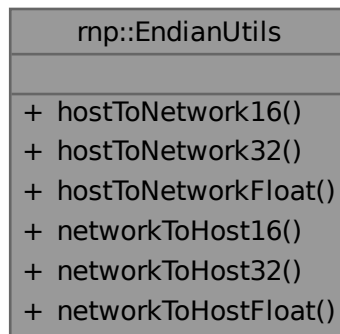
- [/home/masina/Projects/Epitech/rtype/modules/ECS/include/ECS/Systems/Debug.hpp](#)

18.34 rnp::EndianUtils Class Reference

Utility class for endianness conversion.

`#include <Serializer.hpp>`

Collaboration diagram for `rnmp::EndianUtils`:



Static Public Member Functions

- static `std::uint16_t` [hostToNetwork16](#) (`std::uint16_t` hostValue)
- static `std::uint32_t` [hostToNetwork32](#) (`std::uint32_t` hostValue)
- static `float` [hostToNetworkFloat](#) (`float` hostValue)
- static `std::uint16_t` [networkToHost16](#) (`std::uint16_t` networkValue)
- static `std::uint32_t` [networkToHost32](#) (`std::uint32_t` networkValue)
- static `float` [networkToHostFloat](#) (`float` networkValue)

18.34.1 Detailed Description

Utility class for endianness conversion.

Definition at line 21 of file [Serializer.hpp](#).

18.34.2 Member Function Documentation

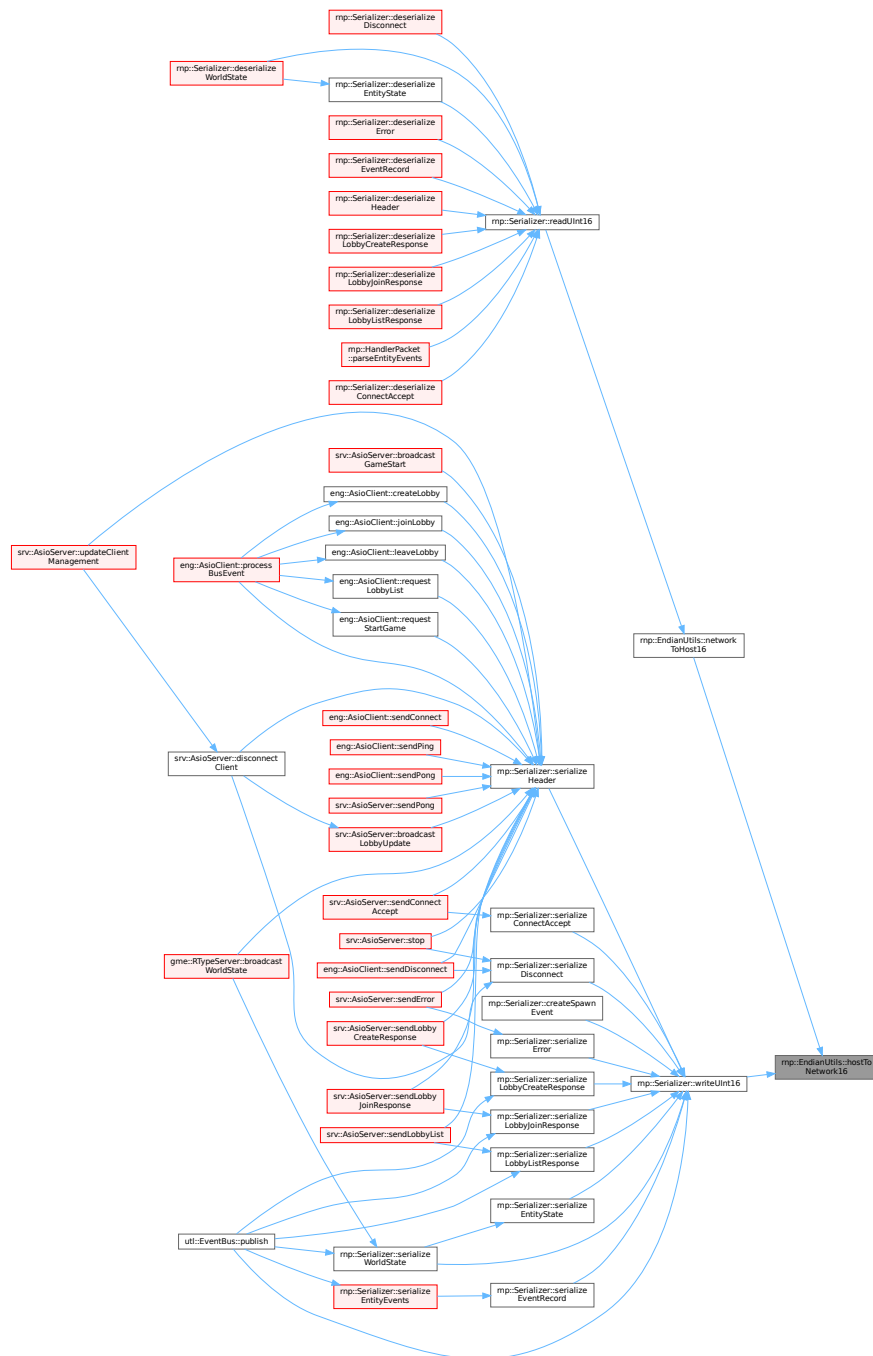
18.34.2.1 `hostToNetwork16()`

static `std::uint16_t` `rnmp::EndianUtils::hostToNetwork16` (
`std::uint16_t` hostValue) [inline], [static]

Definition at line 24 of file [Serializer.hpp](#).

Referenced by [networkToHost16\(\)](#), and [rnmp::Serializer::writeUInt16\(\)](#).

Here is the caller graph for this function:



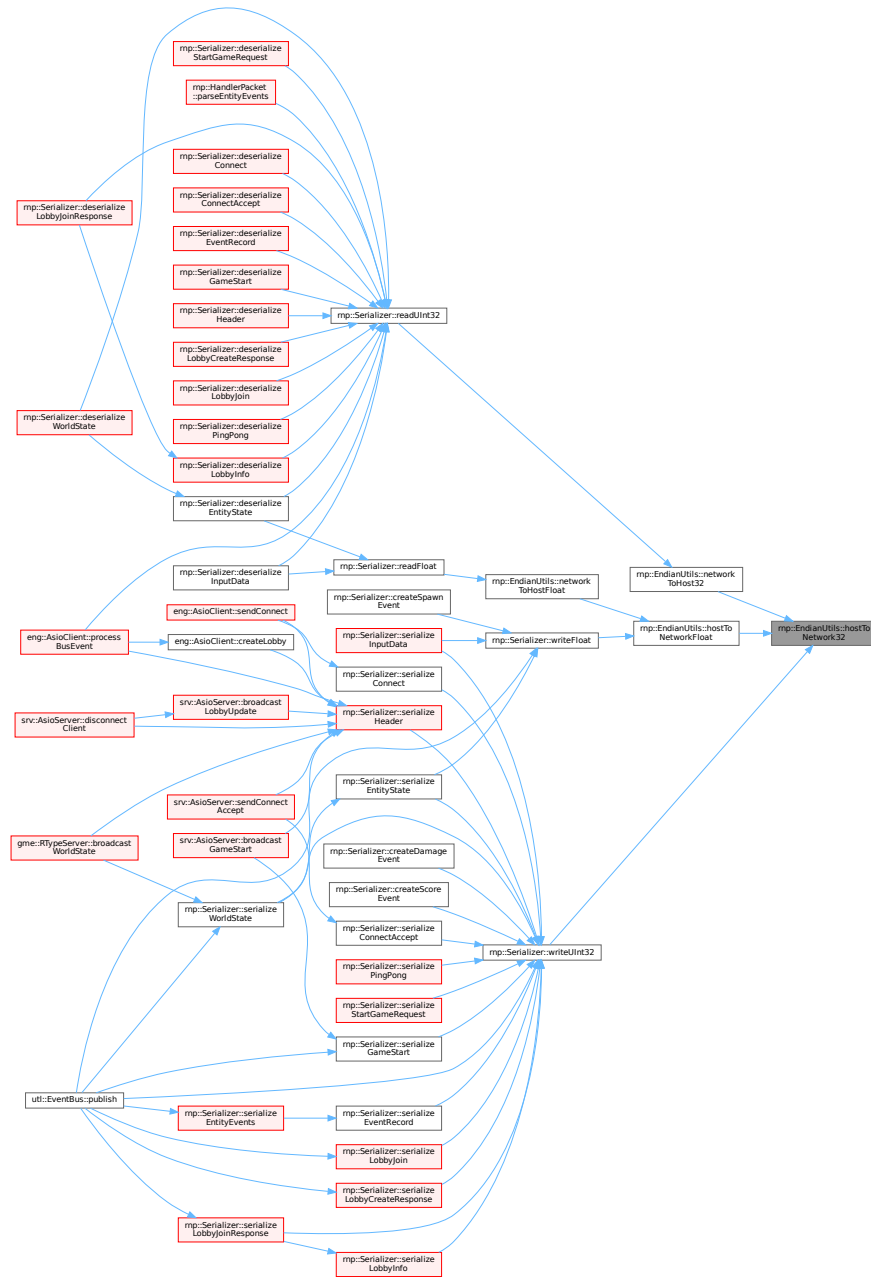
18.34.2.2 hostToNetwork32()

```
static std::uint32_t rnp::EndianUtils::hostToNetwork32 (
    std::uint32_t hostValue) [inline], [static]
```

Definition at line 29 of file [Serializer.hpp](#).

Referenced by [hostToNetworkFloat\(\)](#), [networkToHost32\(\)](#), and [rnp::Serializer::writeUInt32\(\)](#).

Here is the caller graph for this function:



18.34.2.3 hostToNetworkFloat()

```
static float mp::EndianUtils::hostToNetworkFloat (
    float hostValue) [inline], [static]
```

Definition at line 35 of file [Serializer.hpp](#).

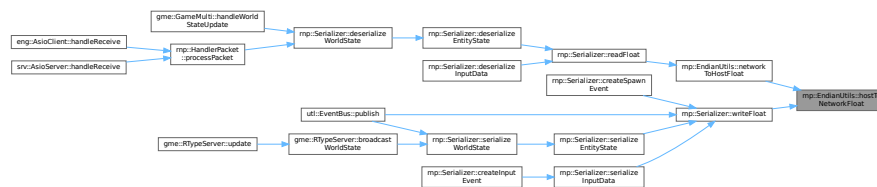
References [hostToNetwork32\(\)](#).

Referenced by [networkToHostFloat\(\)](#), and [mp::Serializer::writeFloat\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.34.2.4 networkToHost16()

```
static std::uint16_t rnp::EndianUtils::networkToHost16 (
    std::uint16_t networkValue) [inline], [static]
```

Definition at line 45 of file [Serializer.hpp](#).

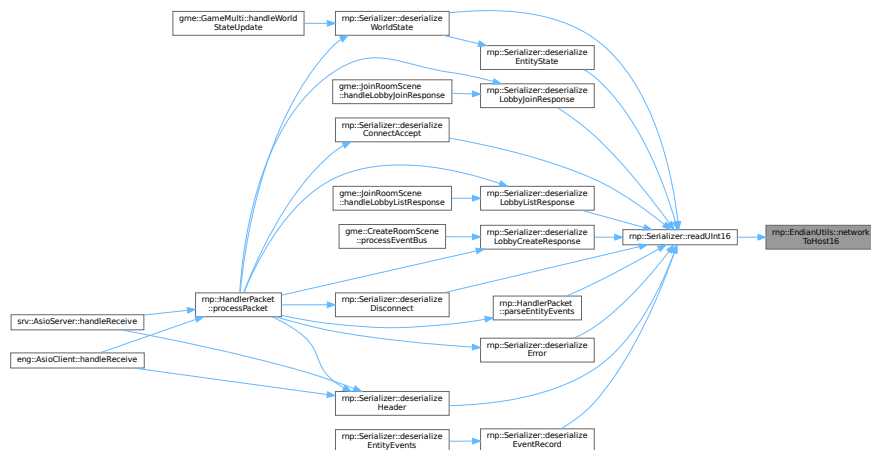
References [hostToNetwork16\(\)](#).

Referenced by [rnp::Serializer::readUInt16\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.34.2.5 networkToHost32()

```
static std::uint32_t rnp::EndianUtils::networkToHost32 (
    std::uint32_t networkValue) [inline], [static]
```

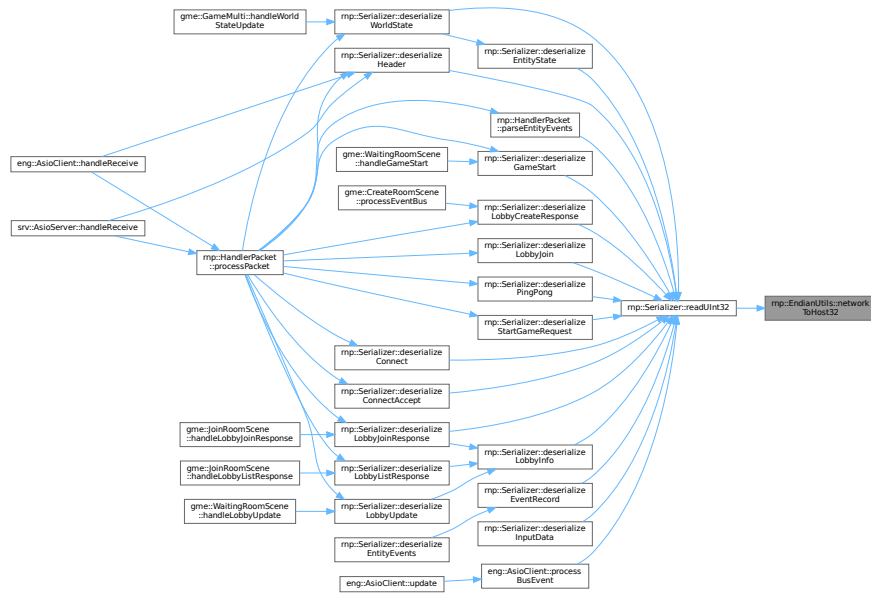
Definition at line 50 of file Serializer.hpp.

References [hostToNetwork32\(\)](#).Referenced by [rnp::Serializer::readUInt32\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.34.2.6 networkToHostFloat()

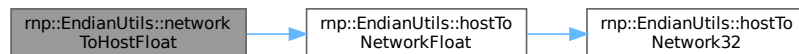
```
static float mp::EndianUtils::networkToHostFloat (
    float networkValue) [inline], [static]
```

Definition at line 55 of file [Serializer.hpp](#).

References [hostToNetworkFloat\(\)](#).

Referenced by [mp::Serializer::readFloat\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



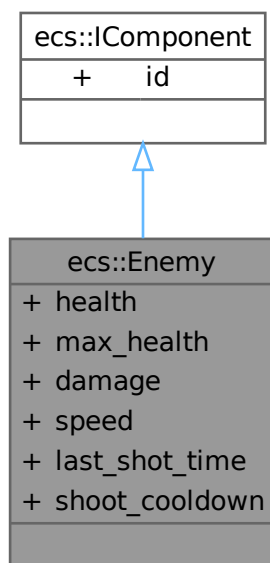
The documentation for this class was generated from the following file:

- [/home/masina/Projects/Epitech/rtype/modules/Interfaces/include/Interfaces/Protocol/Serializer.hpp](#)

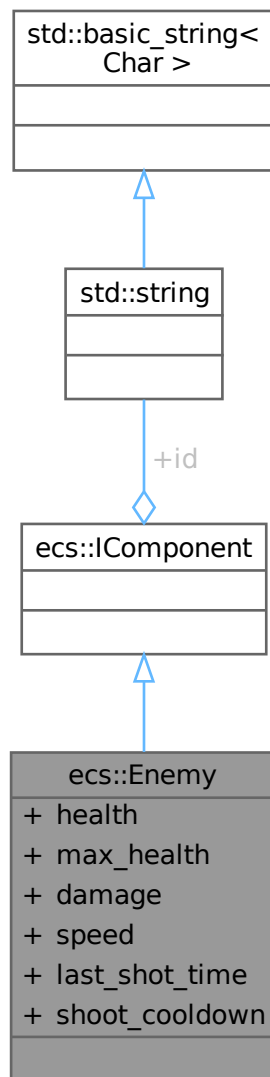
18.35 ecs::Enemy Struct Reference

```
#include <Component.hpp>
```

Inheritance diagram for ecs::Enemy:



Collaboration diagram for `ecs::Enemy`:



Public Attributes

- float [health](#)
- float [max_health](#)
- float [damage](#)
- float [speed](#)
- float [last_shot_time](#)
- float [shoot_cooldown](#)

Public Attributes inherited from [ecs::IComponent](#)

- std::string [id](#)

18.35.1 Detailed Description

Definition at line 113 of file [Component.hpp](#).

18.35.2 Member Data Documentation

18.35.2.1 damage

float ecs::Enemy::damage

Definition at line 117 of file [Component.hpp](#).

Referenced by [gme::CollisionSystem::handlePlayerEnemyCollision\(\)](#).

18.35.2.2 health

float ecs::Enemy::health

Definition at line 115 of file [Component.hpp](#).

Referenced by [gme::CollisionSystem::applyDamageToEnemy\(\)](#).

18.35.2.3 last_shot_time

float ecs::Enemy::last_shot_time

Definition at line 119 of file [Component.hpp](#).

Referenced by [gme::EnemyAISystem::canShoot\(\)](#).

18.35.2.4 max_health

float ecs::Enemy::max_health

Definition at line 116 of file [Component.hpp](#).

18.35.2.5 shoot_cooldown

float ecs::Enemy::shoot_cooldown

Definition at line 120 of file [Component.hpp](#).

18.35.2.6 speed

float ecs::Enemy::speed

Definition at line 118 of file [Component.hpp](#).

The documentation for this struct was generated from the following file:

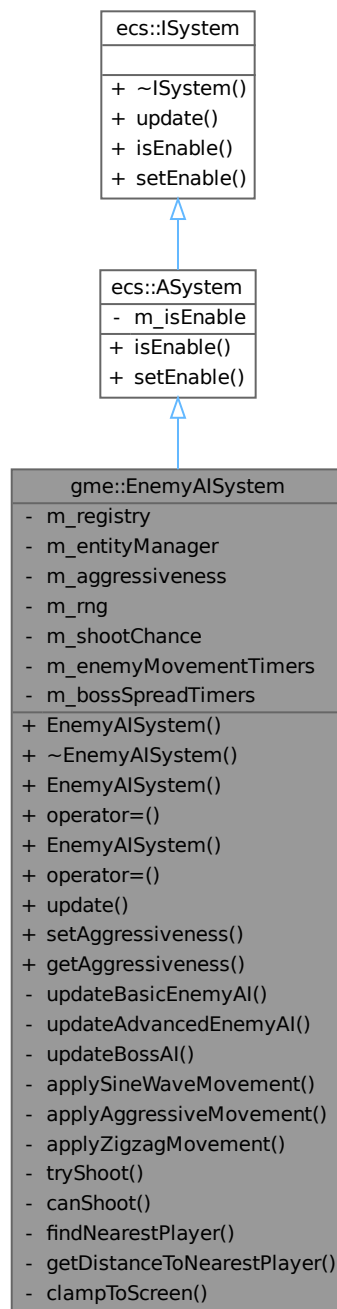
- [/home/masina/Projects/Epitech/rtype/modules/ECS/include/ECS/Component.hpp](#)

18.36 gme::EnemyAISystem Class Reference

Manages enemy behavior, movement patterns, and shooting.

#include <EnemyAISystem.hpp>

Inheritance diagram for gme::EnemyAISystem:



Public Member Functions inherited from [ecs::ASystem](#)

- bool [isEnabled](#) () override
- void [setEnabled](#) (const bool enable) override

Public Member Functions inherited from [ecs::ISystem](#)

- virtual [~ISystem](#) ()=default

Private Member Functions

- void [updateBasicEnemyAI](#) (std::uint32_t enemyId, [ecs::Entity](#) enemy, float deltaTime)
- void [updateAdvancedEnemyAI](#) (std::uint32_t enemyId, [ecs::Entity](#) enemy, float deltaTime)
- void [updateBossAI](#) (std::uint32_t enemyId, [ecs::Entity](#) enemy, float deltaTime)
- void [applySineWaveMovement](#) (std::uint32_t enemyId, [ecs::Entity](#) enemy, float deltaTime, float frequency=2.0f, float amplitude=50.0f)
- void [applyAggressiveMovement](#) ([ecs::Entity](#) enemy, float deltaTime, float speed=150.0f)
- void [applyZigzagMovement](#) (std::uint32_t enemyId, [ecs::Entity](#) enemy, float deltaTime)
- void [tryShoot](#) (std::uint32_t enemyId, [ecs::Entity](#) enemy, float deltaTime)
- bool [canShoot](#) ([ecs::Entity](#) enemy) const
- [ecs::Entity](#) [findNearestPlayer](#) (float x, float y) const
- float [getDistanceToNearestPlayer](#) (float x, float y) const
- void [clampToScreen](#) ([ecs::Entity](#) enemy)

Private Attributes

- [ecs::Registry](#) & [m_registry](#)
- [EntityManager](#) & [m_entityManager](#)
- float [m_aggressiveness](#)
- std::mt19937 [m_rng](#)
- std::uniform_real_distribution< float > [m_shootChance](#)
- std::unordered_map< std::uint32_t, float > [m_enemyMovementTimers](#)
- std::unordered_map< std::uint32_t, float > [m_bossSpreadTimers](#)

18.36.1 Detailed Description

Manages enemy behavior, movement patterns, and shooting.
Definition at line 36 of file [EnemyAISystem.hpp](#).

18.36.2 Constructor & Destructor Documentation

18.36.2.1 [EnemyAISystem\(\)](#) [1/3]

[gme::EnemyAISystem::EnemyAISystem](#) (
 [ecs::Registry](#) & registry,
 [EntityManager](#) & entityManager) [explicit]

Definition at line 16 of file [EnemyAISystem.cpp](#).

References [utl::INFO](#), [utl::Logger::log\(\)](#), and [m_rng](#).

Here is the call graph for this function:



18.36.2.2 ~EnemyAISystem()

`gme::EnemyAISystem::~~EnemyAISystem ()` [override], [default]

18.36.2.3 EnemyAISystem() [2/3]

`gme::EnemyAISystem::EnemyAISystem (`
`const EnemyAISystem & )` [delete]

18.36.2.4 EnemyAISystem() [3/3]

`gme::EnemyAISystem::EnemyAISystem (`
`EnemyAISystem && )` [delete]

18.36.3 Member Function Documentation

18.36.3.1 applyAggressiveMovement()

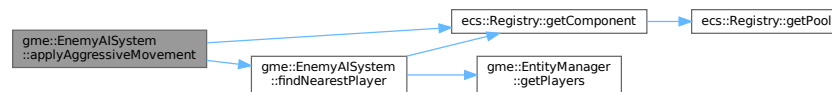
`void gme::EnemyAISystem::applyAggressiveMovement (`
`ecs::Entity enemy,`
`float deltaTime,`
`float speed = 150.0f)` [private]

Definition at line 203 of file [EnemyAISystem.cpp](#).

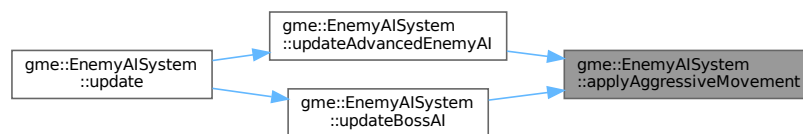
References [findNearestPlayer\(\)](#), [ecs::Registry::getComponent\(\)](#), [ecs::INVALID_ENTITY](#), [m_aggressiveness](#), [m_registry](#), [ecs::Transform::x](#), and [ecs::Transform::y](#).

Referenced by [updateAdvancedEnemyAI\(\)](#), and [updateBossAI\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.36.3.2 applySineWaveMovement()

`void gme::EnemyAISystem::applySineWaveMovement (`
`std::uint32_t enemyId,`
`ecs::Entity enemy,`
`float deltaTime,`
`float frequency = 2.0f,`
`float amplitude = 50.0f)` [private]

Definition at line 182 of file [EnemyAISystem.cpp](#).

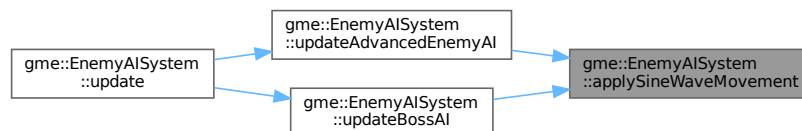
References [ecs::Registry::getComponent\(\)](#), [m_enemyMovementTimers](#), [m_registry](#), [ecs::Transform::x](#), and [ecs::Transform::y](#).

Referenced by [updateAdvancedEnemyAI\(\)](#), and [updateBossAI\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.36.3.3 applyZigzagMovement()

```

void gme::EnemyAISystem::applyZigzagMovement (
    std::uint32_t enemyId,
    ecs::Entity enemy,
    float deltaTime) [private]
  
```

Definition at line 238 of file [EnemyAISystem.cpp](#).

References [ecs::Registry::getComponent\(\)](#), [m_enemyMovementTimers](#), [m_registry](#), [ecs::Transform::x](#), and [ecs::Transform::y](#).

Referenced by [updateBossAI\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.36.3.4 canShoot()

```

bool gme::EnemyAISystem::canShoot (
    ecs::Entity enemy) const [private]
  
```

Definition at line 322 of file [EnemyAISystem.cpp](#).

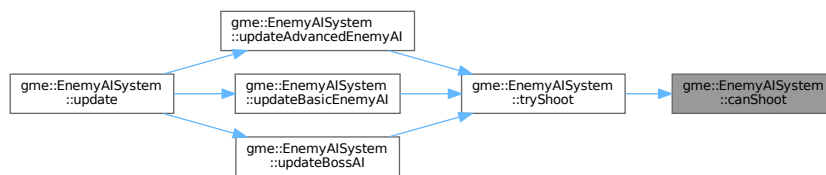
References [ecs::Registry::getComponent\(\)](#), [ecs::Enemy::last_shot_time](#), and [m_registry](#).

Referenced by [tryShoot\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.36.3.5 clampToScreen()

```
void gme::EnemyAISystem::clampToScreen (
    ecs::Entity enemy) [private]
```

Definition at line 383 of file [EnemyAISystem.cpp](#).

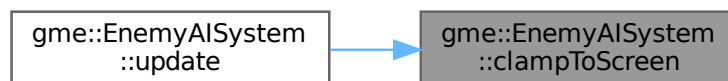
References [ecs::Registry::getComponent\(\)](#), [m_registry](#), [utl::GameConfig::Server::SCREEN_HEIGHT](#), [utl::GameConfig::Server::SCREEN_WIDTH](#), and [ecs::Transform::x](#).

Referenced by [update\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.36.3.6 findNearestPlayer()

```
ecs::Entity gme::EnemyAISystem::findNearestPlayer (
```

```
float x,
float y) const [private]
```

Definition at line 331 of file [EnemyAISystem.cpp](#).

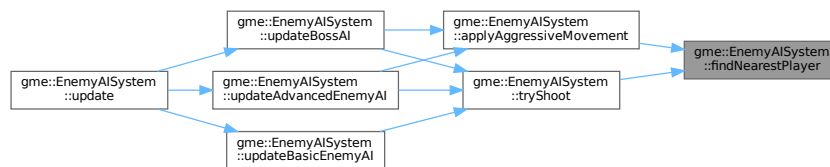
References [ecs::Registry::getComponent\(\)](#), [gme::EntityManager::getPlayers\(\)](#), [ecs::INVALID_ENTITY](#), [m_entityManager](#), [m_registry](#), and [ecs::Transform::x](#).

Referenced by [applyAggressiveMovement\(\)](#), and [tryShoot\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.36.3.7 getAggressiveness()

```
float gme::EnemyAISystem::getAggressiveness () const [inline]
```

Definition at line 51 of file [EnemyAISystem.hpp](#).

References [m_aggressiveness](#).

18.36.3.8 getDistanceToNearestPlayer()

```
float gme::EnemyAISystem::getDistanceToNearestPlayer (
```

```
float x,
float y) const [private]
```

Definition at line 358 of file [EnemyAISystem.cpp](#).

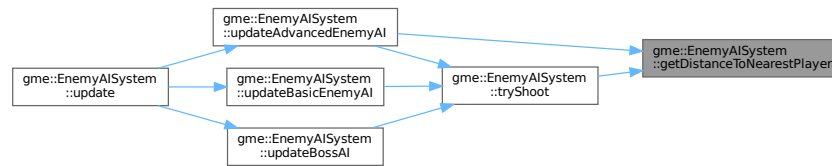
References [ecs::Registry::getComponent\(\)](#), [gme::EntityManager::getPlayers\(\)](#), [m_entityManager](#), [m_registry](#), and [ecs::Transform::x](#).

Referenced by [tryShoot\(\)](#), and [updateAdvancedEnemyAI\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.36.3.9 operator=() [1/2]

[EnemyAISystem](#) & gme::EnemyAISystem::operator= (
 const [EnemyAISystem](#) &) [delete]

18.36.3.10 operator=() [2/2]

[EnemyAISystem](#) & gme::EnemyAISystem::operator= (
 [EnemyAISystem](#) &&) [delete]

18.36.3.11 setAggressiveness()

void gme::EnemyAISystem::setAggressiveness (
 float value) [inline]

Definition at line 50 of file [EnemyAISystem.hpp](#).

References [m_aggressiveness](#).

18.36.3.12 tryShoot()

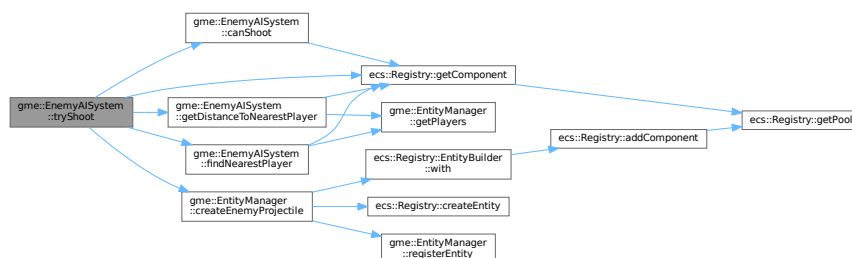
void gme::EnemyAISystem::tryShoot (
 std::uint32_t enemyId,
 [ecs::Entity](#) enemy,
 float deltaTime) [private]

Definition at line 257 of file [EnemyAISystem.cpp](#).

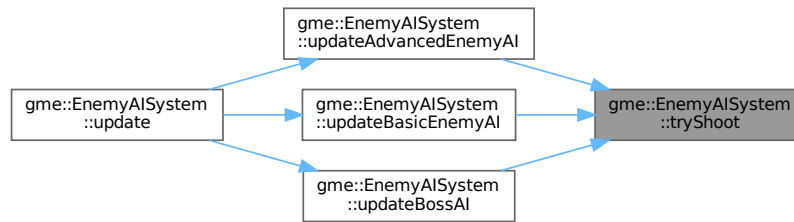
References [canShoot\(\)](#), [gme::EntityManager::createEnemyProjectile\(\)](#), [findNearestPlayer\(\)](#), [ecs::Registry::getComponent\(\)](#), [getDistanceToNearestPlayer\(\)](#), [ecs::INVALID_ENTITY](#), [m_entityManager](#), [m_registry](#), [m_rng](#), [m_shootChance](#), and [ecs::Transform::x](#).

Referenced by [updateAdvancedEnemyAI\(\)](#), [updateBasicEnemyAI\(\)](#), and [updateBossAI\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.36.3.13 update()

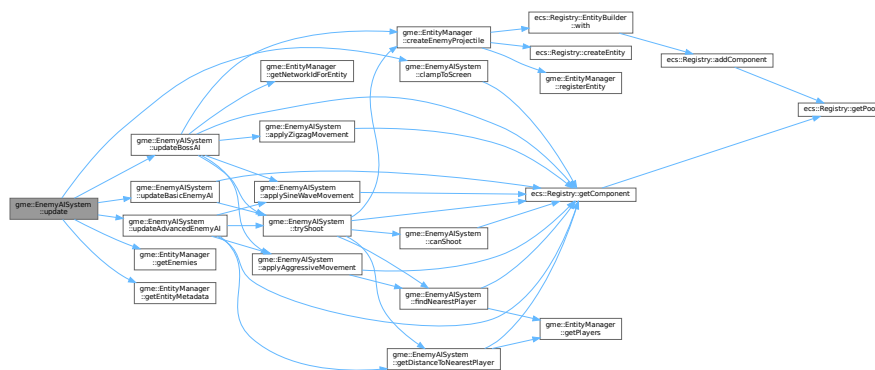
```
void gme::EnemyAISystem::update (
    ecs::Registry & registry,
    float deltaTime) [override], [virtual]
```

Implements [ecs::ISystem](#).

Definition at line 25 of file [EnemyAISystem.cpp](#).

References [gme::BOSS](#), [clampToScreen\(\)](#), [gme::ENEMY_ADVANCED](#), [gme::ENEMY_BASIC](#), [gme::EntityManager::getEnemies\(\)](#), [gme::EntityManager::getEntityMetadata\(\)](#), [m_entityManager](#), [updateAdvancedEnemyAI\(\)](#), [updateBasicEnemyAI\(\)](#), and [updateBossAI\(\)](#).

Here is the call graph for this function:



18.36.3.14 updateAdvancedEnemyAI()

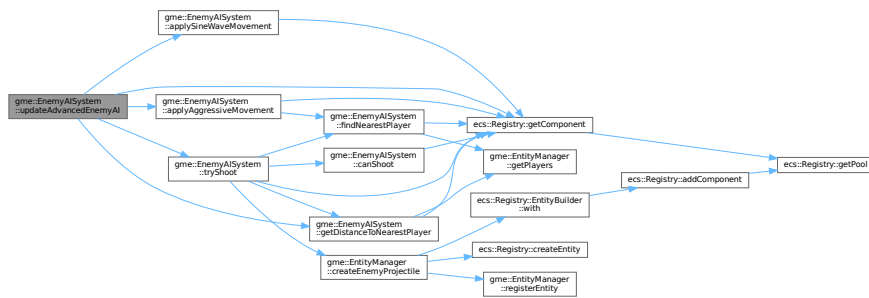
```
void gme::EnemyAISystem::updateAdvancedEnemyAI (
    std::uint32_t enemyId,
    ecs::Entity enemy,
    float deltaTime) [private]
```

Definition at line 78 of file [EnemyAISystem.cpp](#).

References [applyAggressiveMovement\(\)](#), [applySineWaveMovement\(\)](#), [ecs::Registry::getComponent\(\)](#), [getDistanceToNearestPlayer\(\)](#), [m_enemyMovementTimers](#), [m_registry](#), and [tryShoot\(\)](#).

Referenced by [update\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.36.3.15 updateBasicEnemyAI()

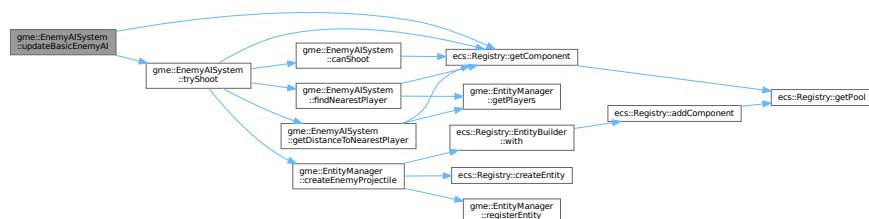
```
void gme::EnemyAISystem::updateBasicEnemyAI (
    std::uint32_t enemyId,
    ecs::Entity enemy,
    float deltaTime) [private]
```

Definition at line 57 of file [EnemyAISystem.cpp](#).

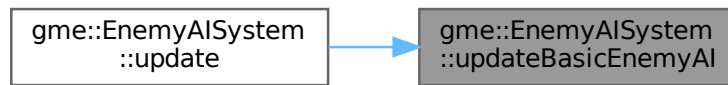
References [ecs::Registry::getComponent\(\)](#), [m_enemyMovementTimers](#), [m_registry](#), [tryShoot\(\)](#), and [ecs::Velocity::x](#).

Referenced by [update\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.36.3.16 updateBossAI()

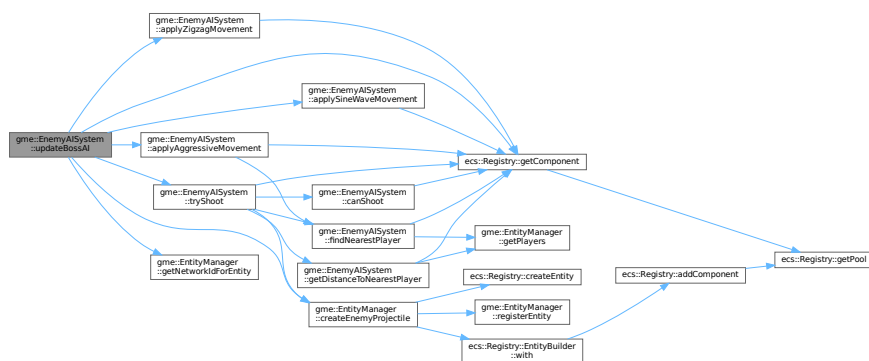
```
void gme::EnemyAISystem::updateBossAI (
    std::uint32_t enemyId,
    ecs::Entity enemy,
    float deltaTime) [private]
```

Definition at line 107 of file `EnemyAISystem.cpp`.

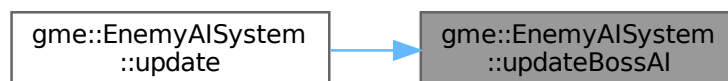
References `applyAggressiveMovement()`, `applySineWaveMovement()`, `applyZigzagMovement()`, `gme::EntityManager::createEntity()`, `ecs::Registry::getComponent()`, `gme::EntityManager::getNetworkIdForEntity()`, `m_bossSpreadTimers`, `m_enemyMovementTimers`, `m_entityManager`, `m_registry`, `tryShoot()`, and `ecs::Transform::x`.

Referenced by `update()`.

Here is the call graph for this function:



Here is the caller graph for this function:



18.36.4 Member Data Documentation

18.36.4.1 m_aggressiveness

```
float gme::EnemyAISystem::m_aggressiveness [private]
```

Definition at line 57 of file [EnemyAISystem.hpp](#).

Referenced by [applyAggressiveMovement\(\)](#), [getAggressiveness\(\)](#), and [setAggressiveness\(\)](#).

18.36.4.2 m_bossSpreadTimers

`std::unordered_map<std::uint32_t, float> gme::EnemyAISystem::m_bossSpreadTimers` [private]

Definition at line 63 of file [EnemyAISystem.hpp](#).

Referenced by [updateBossAI\(\)](#).

18.36.4.3 m_enemyMovementTimers

`std::unordered_map<std::uint32_t, float> gme::EnemyAISystem::m_enemyMovementTimers` [private]

Definition at line 62 of file [EnemyAISystem.hpp](#).

Referenced by [applySineWaveMovement\(\)](#), [applyZigzagMovement\(\)](#), [updateAdvancedEnemyAI\(\)](#), [updateBasicEnemyAI\(\)](#), and [updateBossAI\(\)](#).

18.36.4.4 m_entityManager

`EntityManager& gme::EnemyAISystem::m_entityManager` [private]

Definition at line 55 of file [EnemyAISystem.hpp](#).

Referenced by [findNearestPlayer\(\)](#), [getDistanceToNearestPlayer\(\)](#), [tryShoot\(\)](#), [update\(\)](#), and [updateBossAI\(\)](#).

18.36.4.5 m_registry

`ecs::Registry& gme::EnemyAISystem::m_registry` [private]

Definition at line 54 of file [EnemyAISystem.hpp](#).

Referenced by [applyAggressiveMovement\(\)](#), [applySineWaveMovement\(\)](#), [applyZigzagMovement\(\)](#), [canShoot\(\)](#), [clampToScreen\(\)](#), [findNearestPlayer\(\)](#), [getDistanceToNearestPlayer\(\)](#), [tryShoot\(\)](#), [updateAdvancedEnemyAI\(\)](#), [updateBasicEnemyAI\(\)](#), and [updateBossAI\(\)](#).

18.36.4.6 m_rng

`std::mt19937 gme::EnemyAISystem::m_rng` [private]

Definition at line 58 of file [EnemyAISystem.hpp](#).

Referenced by [EnemyAISystem\(\)](#), and [tryShoot\(\)](#).

18.36.4.7 m_shootChance

`std::uniform_real_distribution<float> gme::EnemyAISystem::m_shootChance` [private]

Definition at line 59 of file [EnemyAISystem.hpp](#).

Referenced by [tryShoot\(\)](#).

The documentation for this class was generated from the following files:

- [/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Server/include/RTypeServer/↵ Systems/EnemyAISystem.hpp](#)
- [/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Server/src/Systems/EnemyAISystem.cpp](#)

18.37 gme::EnemySpawn Struct Reference

Enemy spawn definition for a wave.

`#include <WaveManager.hpp>`

Collaboration diagram for gme::EnemySpawn:

gme::EnemySpawn	
+	type
+	x
+	y
+	health
+	spawnDelay

Public Attributes

- [ServerEntityType](#) type
- float [x](#)
- float [y](#)
- float [health](#)
- float [spawnDelay](#)

18.37.1 Detailed Description

Enemy spawn definition for a wave.

Definition at line 18 of file [WaveManager.hpp](#).

18.37.2 Member Data Documentation

18.37.2.1 health

float gme::EnemySpawn::health

Definition at line 23 of file [WaveManager.hpp](#).

Referenced by [gme::WaveManager::processSpawns\(\)](#).

18.37.2.2 spawnDelay

float gme::EnemySpawn::spawnDelay

Definition at line 24 of file [WaveManager.hpp](#).

Referenced by [gme::WaveManager::processSpawns\(\)](#).

18.37.2.3 type

[ServerEntityType](#) gme::EnemySpawn::type

Definition at line 20 of file [WaveManager.hpp](#).

Referenced by [gme::WaveManager::processSpawns\(\)](#).

18.37.2.4 x

float gme::EnemySpawn::x

Definition at line 21 of file [WaveManager.hpp](#).

Referenced by [gme::WaveManager::processSpawns\(\)](#).

18.37.2.5 y

float gme::EnemySpawn::y

Definition at line 22 of file [WaveManager.hpp](#).

Referenced by [gme::WaveManager::processSpawns\(\)](#).

The documentation for this struct was generated from the following file:

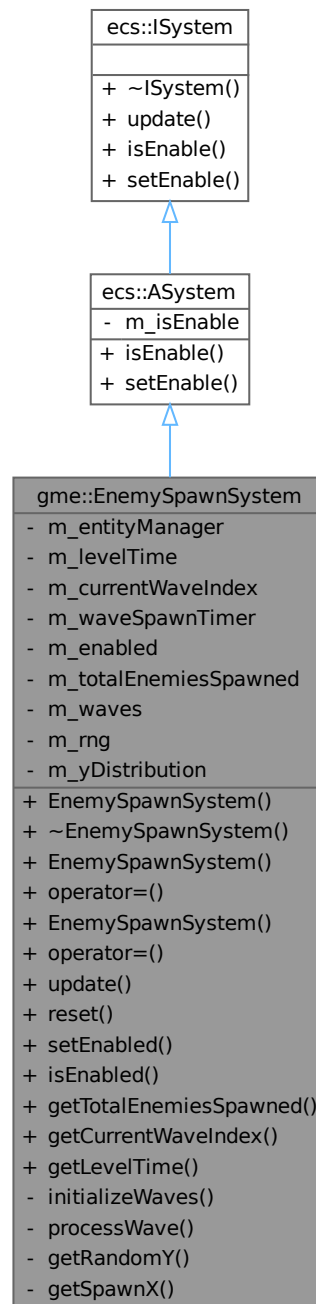
- [/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Server/include/RTypeServer/WaveManager.hpp](#)

18.38 gme::EnemySpawnSystem Class Reference

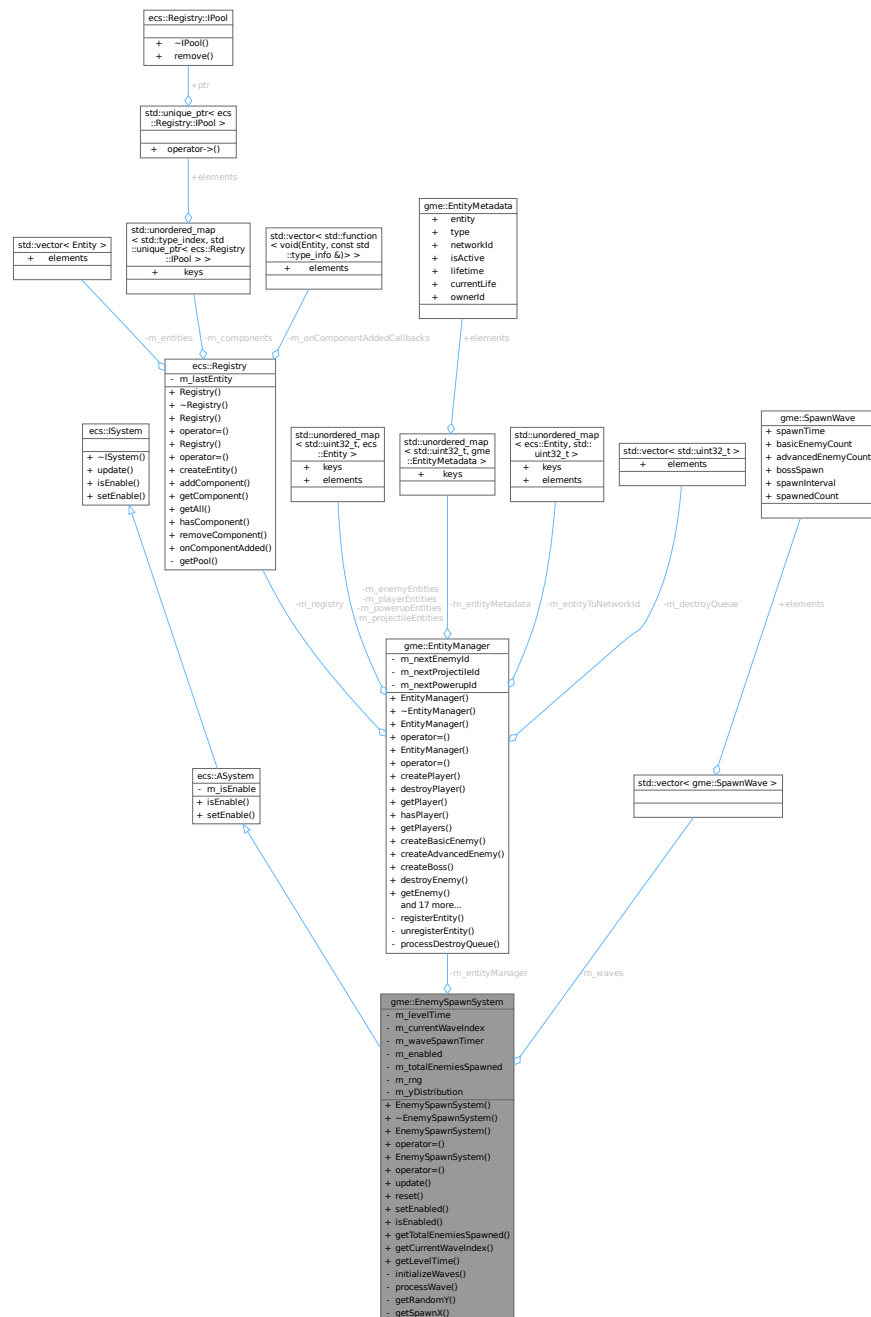
Manages enemy spawning based on level progression.

#include <EnemySpawnSystem.hpp>

Inheritance diagram for gme::EnemySpawnSystem:



Collaboration diagram for `gme::EnemySpawnSystem`:



Public Member Functions

- `EnemySpawnSystem` (`EntityManager` &`entityManager`)
- `~EnemySpawnSystem` () override=default
- `EnemySpawnSystem` (const `EnemySpawnSystem` &)=delete
- `EnemySpawnSystem` & `operator=` (const `EnemySpawnSystem` &)=delete
- `EnemySpawnSystem` (`EnemySpawnSystem` &&)=delete
- `EnemySpawnSystem` & `operator=` (`EnemySpawnSystem` &&)=delete
- void `update` (`ecs::Registry` ®istry, float deltaTime) override
- void `reset` ()
- void `setEnabled` (bool enabled)

- bool [isEnabled](#) () const
- size_t [getTotalEnemiesSpawned](#) () const
- size_t [getCurrentWaveIndex](#) () const
- float [getLevelTime](#) () const

Public Member Functions inherited from [ecs::ASystem](#)

- bool [isEnabled](#) () override
- void [setEnabled](#) (const bool enable) override

Public Member Functions inherited from [ecs::ISystem](#)

- virtual [~ISystem](#) ()=default

Private Member Functions

- void [initializeWaves](#) ()
- void [processWave](#) ([SpawnWave](#) &wave, float deltaTime)
- float [getRandomY](#) ()
- float [getSpawnX](#) () const

Private Attributes

- [EntityManager](#) & [m_entityManager](#)
- float [m_levelTime](#)
- size_t [m_currentWaveIndex](#)
- float [m_waveSpawnTimer](#)
- bool [m_enabled](#)
- size_t [m_totalEnemiesSpawned](#)
- std::vector< [SpawnWave](#) > [m_waves](#)
- std::mt19937 [m_rng](#)
- std::uniform_real_distribution< float > [m_yDistribution](#)

18.38.1 Detailed Description

Manages enemy spawning based on level progression.
Definition at line 34 of file [EnemySpawnSystem.hpp](#).

18.38.2 Constructor & Destructor Documentation

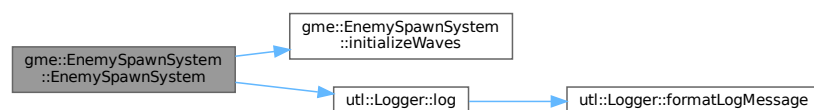
18.38.2.1 EnemySpawnSystem() [1/3]

[gme::EnemySpawnSystem::EnemySpawnSystem](#) (
 [EntityManager](#) & entityManager) [explicit]

Definition at line 13 of file [EnemySpawnSystem.cpp](#).

References [utl::INFO](#), [initializeWaves\(\)](#), [utl::Logger::log\(\)](#), [m_rng](#), and [m_waves](#).

Here is the call graph for this function:



18.38.2.2 ~EnemySpawnSystem()

`gme::EnemySpawnSystem::~~EnemySpawnSystem ()` [override], [default]

18.38.2.3 EnemySpawnSystem() [2/3]

`gme::EnemySpawnSystem::EnemySpawnSystem (`
 const [EnemySpawnSystem](#) &) [delete]

18.38.2.4 EnemySpawnSystem() [3/3]

`gme::EnemySpawnSystem::EnemySpawnSystem (`
 [EnemySpawnSystem](#) &&) [delete]

18.38.3 Member Function Documentation

18.38.3.1 getCurrentWaveIndex()

`size_t gme::EnemySpawnSystem::getCurrentWaveIndex ()` const [inline]

Definition at line 54 of file [EnemySpawnSystem.hpp](#).

References [m_currentWaveIndex](#).

18.38.3.2 getLevelTime()

`float gme::EnemySpawnSystem::getLevelTime ()` const [inline]

Definition at line 55 of file [EnemySpawnSystem.hpp](#).

References [m_levelTime](#).

18.38.3.3 getRandomY()

`float gme::EnemySpawnSystem::getRandomY ()` [private]

Definition at line 197 of file [EnemySpawnSystem.cpp](#).

References [m_rng](#), and [m_yDistribution](#).

Referenced by [processWave\(\)](#).

Here is the caller graph for this function:



18.38.3.4 getSpawnX()

`float gme::EnemySpawnSystem::getSpawnX ()` const [inline], [private]

Definition at line 72 of file [EnemySpawnSystem.hpp](#).

Referenced by [processWave\(\)](#).

Here is the caller graph for this function:



18.38.3.5 getTotalEnemiesSpawned()

size_t gme::EnemySpawnSystem::getTotalEnemiesSpawned () const [inline]

Definition at line 53 of file [EnemySpawnSystem.hpp](#).

References [m_totalEnemiesSpawned](#).

18.38.3.6 initializeWaves()

void gme::EnemySpawnSystem::initializeWaves () [private]

Definition at line 85 of file [EnemySpawnSystem.cpp](#).

References [gme::SpawnWave::advancedEnemyCount](#), [gme::SpawnWave::basicEnemyCount](#), [gme::SpawnWave::bossSpawn](#), [m_waves](#), [gme::SpawnWave::spawnedCount](#), [gme::SpawnWave::spawnInterval](#), and [gme::SpawnWave::spawnTime](#).

Referenced by [EnemySpawnSystem\(\)](#).

Here is the caller graph for this function:



18.38.3.7 isEnabled()

bool gme::EnemySpawnSystem::isEnabled () const [inline]

Definition at line 50 of file [EnemySpawnSystem.hpp](#).

References [m_enabled](#).

18.38.3.8 operator=() [1/2]

[EnemySpawnSystem](#) & gme::EnemySpawnSystem::operator= (const [EnemySpawnSystem](#) &) [delete]

18.38.3.9 operator=() [2/2]

[EnemySpawnSystem](#) & gme::EnemySpawnSystem::operator= ([EnemySpawnSystem](#) &&) [delete]

18.38.3.10 processWave()

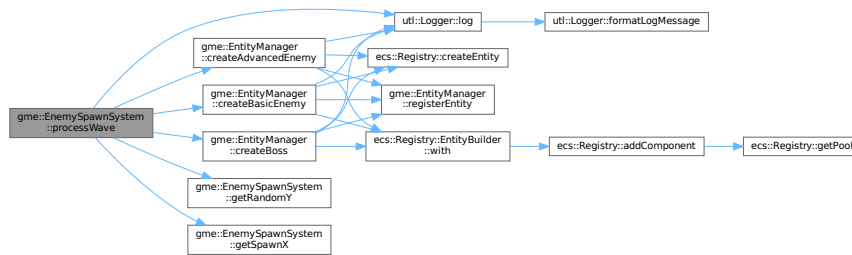
void gme::EnemySpawnSystem::processWave ([SpawnWave](#) & wave, float deltaTime) [private]

Definition at line 140 of file [EnemySpawnSystem.cpp](#).

References [gme::SpawnWave::advancedEnemyCount](#), [gme::SpawnWave::basicEnemyCount](#), [gme::SpawnWave::bossSpawn](#), [gme::EntityManager::createAdvancedEnemy\(\)](#), [gme::EntityManager::createBasicEnemy\(\)](#), [gme::EntityManager::createBossEnemy\(\)](#), [getRandomY\(\)](#), [getSpawnX\(\)](#), [utl::INFO](#), [utl::Logger::log\(\)](#), [m_entityManager](#), [m_totalEnemiesSpawned](#), [m_waveSpawnTimer](#), [gme::SpawnWave::spawnedCount](#), and [gme::SpawnWave::spawnInterval](#).

Referenced by [update\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.38.3.11 reset()

void gme::EnemySpawnSystem::reset ()

Definition at line 68 of file [EnemySpawnSystem.cpp](#).

References [utl::INFO](#), [utl::Logger::log\(\)](#), [m_currentWaveIndex](#), [m_enabled](#), [m_levelTime](#), [m_totalEnemiesSpawned](#), [m_waves](#), and [m_waveSpawnTimer](#).

Here is the call graph for this function:



18.38.3.12 setEnabled()

void gme::EnemySpawnSystem::setEnabled (bool enabled) [inline]

Definition at line 49 of file [EnemySpawnSystem.hpp](#).

References [m_enabled](#).

18.38.3.13 update()

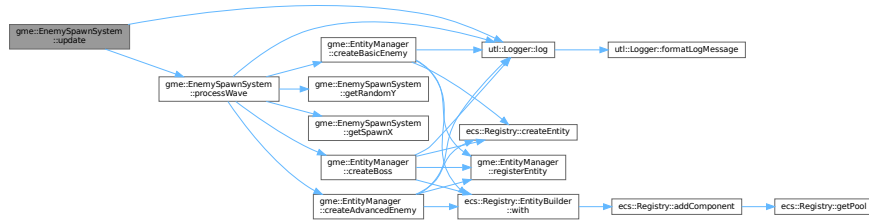
void gme::EnemySpawnSystem::update (ecs::Registry & registry, float deltaTime) [override], [virtual]

Implements [ecs::ISystem](#).

Definition at line 26 of file [EnemySpawnSystem.cpp](#).

References [utl::INFO](#), [utl::Logger::log\(\)](#), [m_currentWaveIndex](#), [m_enabled](#), [m_levelTime](#), [m_waves](#), [m_waveSpawnTimer](#), and [processWave\(\)](#).

Here is the call graph for this function:



18.38.4 Member Data Documentation

18.38.4.1 m_currentWaveIndex

size_t gme::EnemySpawnSystem::m_currentWaveIndex [private]

Definition at line 60 of file [EnemySpawnSystem.hpp](#).

Referenced by [getCurrentWaveIndex\(\)](#), [reset\(\)](#), and [update\(\)](#).

18.38.4.2 m_enabled

bool gme::EnemySpawnSystem::m_enabled [private]

Definition at line 62 of file [EnemySpawnSystem.hpp](#).

Referenced by [isEnabled\(\)](#), [reset\(\)](#), [setEnabled\(\)](#), and [update\(\)](#).

18.38.4.3 m_entityManager

[EntityManager](#)& gme::EnemySpawnSystem::m_entityManager [private]

Definition at line 58 of file [EnemySpawnSystem.hpp](#).

Referenced by [processWave\(\)](#).

18.38.4.4 m_levelTime

float gme::EnemySpawnSystem::m_levelTime [private]

Definition at line 59 of file [EnemySpawnSystem.hpp](#).

Referenced by [getLevelTime\(\)](#), [reset\(\)](#), and [update\(\)](#).

18.38.4.5 m_rng

std::mt19937 gme::EnemySpawnSystem::m_rng [private]

Definition at line 66 of file [EnemySpawnSystem.hpp](#).

Referenced by [EnemySpawnSystem\(\)](#), and [getRandomY\(\)](#).

18.38.4.6 m_totalEnemiesSpawned

size_t gme::EnemySpawnSystem::m_totalEnemiesSpawned [private]

Definition at line 63 of file [EnemySpawnSystem.hpp](#).

Referenced by [getTotalEnemiesSpawned\(\)](#), [processWave\(\)](#), and [reset\(\)](#).

18.38.4.7 m_waves

std::vector<[SpawnWave](#)> gme::EnemySpawnSystem::m_waves [private]

Definition at line 65 of file [EnemySpawnSystem.hpp](#).

Referenced by [EnemySpawnSystem\(\)](#), [initializeWaves\(\)](#), [reset\(\)](#), and [update\(\)](#).

18.38.4.8 m_waveSpawnTimer

float gme::EnemySpawnSystem::m_waveSpawnTimer [private]

Definition at line 61 of file [EnemySpawnSystem.hpp](#).

Referenced by [processWave\(\)](#), [reset\(\)](#), and [update\(\)](#).

18.38.4.9 m_yDistribution

std::uniform_real_distribution<float> gme::EnemySpawnSystem::m_yDistribution [private]

Definition at line 67 of file [EnemySpawnSystem.hpp](#).

Referenced by [getRandomY\(\)](#).

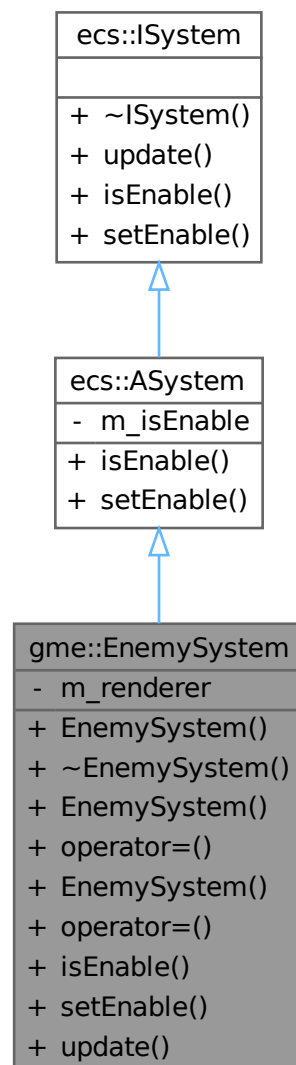
The documentation for this class was generated from the following files:

- /home/masina/Projects/Epitech/rtype/plugins/Game/RType/Server/include/RTypeServer/Systems/[EnemySpawnSystem.hpp](#)
- /home/masina/Projects/Epitech/rtype/plugins/Game/RType/Server/src/Systems/[EnemySpawnSystem.cpp](#)

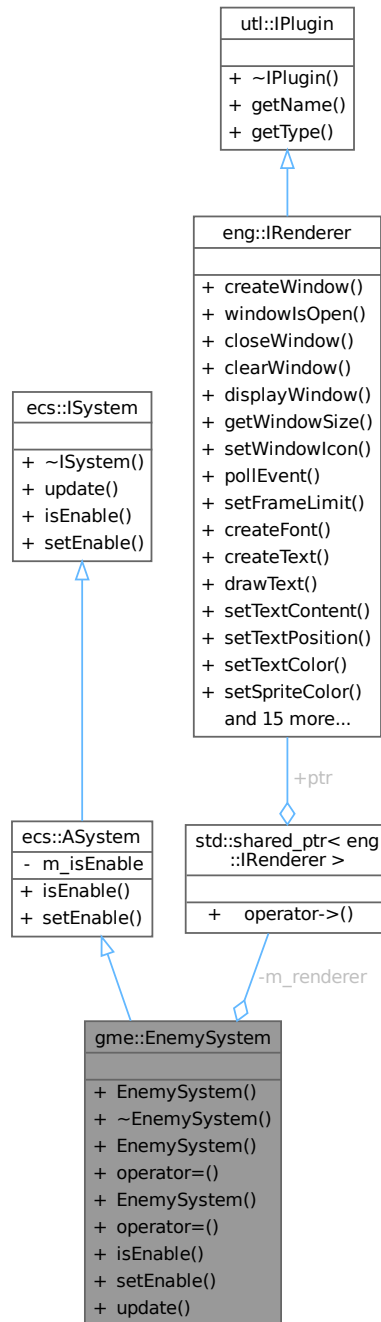
18.39 gme::EnemySystem Class Reference

#include <Enemy.hpp>

Inheritance diagram for gme::EnemySystem:



Collaboration diagram for gme::EnemySystem:



Public Member Functions

- [EnemySystem](#) (const std::shared_ptr< [eng::IRenderer](#) > &renderer)
- [~EnemySystem](#) () override=default
- [EnemySystem](#) (const [EnemySystem](#) &)=delete
- [EnemySystem](#) & operator= (const [EnemySystem](#) &)=delete
- [EnemySystem](#) ([EnemySystem](#) &&)=delete
- [EnemySystem](#) & operator= ([EnemySystem](#) &&)=delete

- bool [isEnabled](#) () override
- void [setEnabled](#) (const bool enable) override
- void [update](#) ([ecs::Registry](#) ®istry, const float dt) override

Public Member Functions inherited from [ecs::ISystem](#)

- virtual [~ISystem](#) ()=default

Private Attributes

- const std::shared_ptr< [eng::IRenderer](#) > & [m_renderer](#)

18.39.1 Detailed Description

Definition at line 22 of file [Enemy.hpp](#).

18.39.2 Constructor & Destructor Documentation

18.39.2.1 EnemySystem() [1/3]

```
gme::EnemySystem::EnemySystem (
    const std::shared_ptr< eng::IRenderer > & renderer)  [inline], [explicit]
```

Definition at line 25 of file [Enemy.hpp](#).

18.39.2.2 ~EnemySystem()

```
gme::EnemySystem::~~EnemySystem ()  [override], [default]
```

18.39.2.3 EnemySystem() [2/3]

```
gme::EnemySystem::EnemySystem (
    const EnemySystem & )  [delete]
```

18.39.2.4 EnemySystem() [3/3]

```
gme::EnemySystem::EnemySystem (
    EnemySystem && )  [delete]
```

18.39.3 Member Function Documentation

18.39.3.1 isEnabled()

```
bool gme::EnemySystem::isEnabled ()  [inline], [override], [virtual]
```

Reimplemented from [ecs::ASystem](#).

Definition at line 33 of file [Enemy.hpp](#).

18.39.3.2 operator=() [1/2]

```
EnemySystem & gme::EnemySystem::operator= (
    const EnemySystem & )  [delete]
```

18.39.3.3 operator=() [2/2]

```
EnemySystem & gme::EnemySystem::operator= (
    EnemySystem && )  [delete]
```

18.39.3.4 setEnabled()

```
void gme::EnemySystem::setEnabled (
    const bool enable)  [inline], [override], [virtual]
```

Reimplemented from [ecs::ASystem](#).

Definition at line 34 of file [Enemy.hpp](#).

18.39.3.5 update()

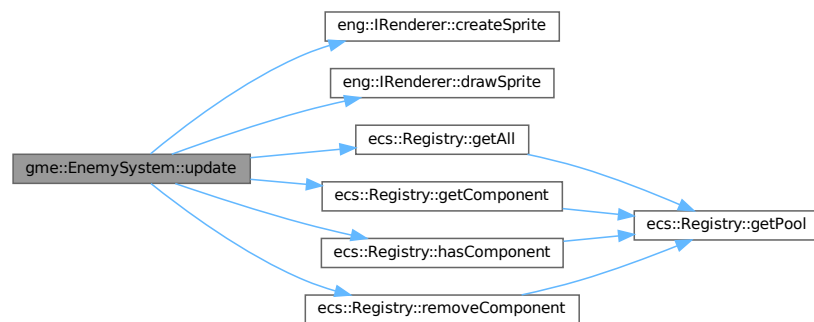
```
void gme::EnemySystem::update (
    ecs::Registry & registry,
    const float dt) [inline], [override], [virtual]
```

Implements [ecs::ISystem](#).

Definition at line 36 of file [Enemy.hpp](#).

References [eng::IRenderer::createSprite\(\)](#), [eng::IRenderer::drawSprite\(\)](#), [ecs::Registry::getAll\(\)](#), [ecs::Registry::getComponent\(\)](#), [ecs::Registry::hasComponent\(\)](#), [ecs::IComponent::id](#), [m_renderer](#), [utl::GameConfig::Screen::REMOVE_MAX_Y](#), [utl::GameConfig::Screen::REMOVE_MIN_Y](#), [utl::GameConfig::Screen::REMOVE_X](#), [ecs::Registry::removeComponent\(\)](#), [utl::GameConfig::Enemy::Easy::SPRITE_WIDTH](#), [ecs::Transform::x](#), and [ecs::Transform::y](#).

Here is the call graph for this function:



18.39.4 Member Data Documentation

18.39.4.1 m_renderer

```
const std::shared_ptr<eng::IRenderer>& gme::EnemySystem::m_renderer [private]
```

Definition at line 125 of file [Enemy.hpp](#).

Referenced by [update\(\)](#).

The documentation for this class was generated from the following file:

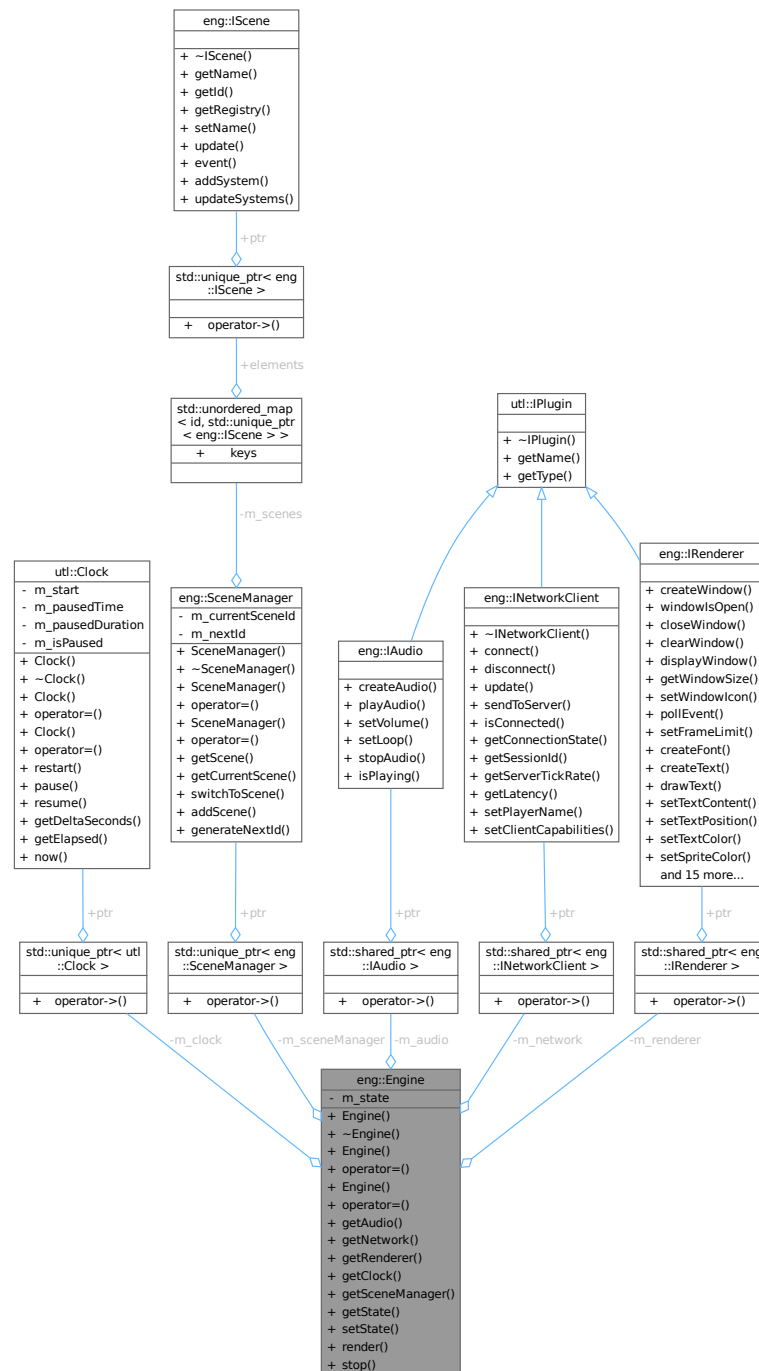
- `/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Solo/include/RTypeClient↔
Solo/Systems/Enemy.hpp`

18.40 eng::Engine Class Reference

Class for the engine.

```
#include <Engine.hpp>
```

Collaboration diagram for eng::Engine:



Public Member Functions

- [Engine](#) (const std::function< std::shared_ptr< [IAudio](#) >()> &audioFactory, const std::function< std::shared_ptr< [INetworkClient](#) >()> &networkFactory, const std::function< std::shared_ptr< [IRenderer](#) >()> &rendererFactory)
- [~Engine](#) ()=default
- [Engine](#) (const [Engine](#) &)=delete
- [Engine](#) & operator= (const [Engine](#) &)=delete
- [Engine](#) ([Engine](#) &&)=delete

- [Engine](#) & [operator=](#) ([Engine](#) &&)=delete
- `std::shared_ptr< IAudio > & getAudio ()`
- `std::shared_ptr< INetworkClient > & getNetwork ()`
- `std::shared_ptr< IRenderer > & getRenderer ()`
- `std::unique_ptr< utl::Clock > & getClock ()`
- `std::unique_ptr< SceneManager > & getSceneManager ()`
- `State getState () const`
- `void setState (const State newState)`
- `void render (const WindowSize &>windowSize, Color clearColor, bool showDebug) const`
- `void stop () const`

Private Attributes

- `State m_state = RUN`
- `std::unique_ptr< utl::Clock > m_clock`
- `std::unique_ptr< SceneManager > m_sceneManager`
- `std::shared_ptr< IAudio > m_audio`
- `std::shared_ptr< INetworkClient > m_network`
- `std::shared_ptr< IRenderer > m_renderer`

18.40.1 Detailed Description

Class for the engine.

Definition at line 33 of file [Engine.hpp](#).

18.40.2 Constructor & Destructor Documentation

18.40.2.1 Engine() [1/3]

```
eng::Engine::Engine (
    const std::function< std::shared_ptr< IAudio >()> & audioFactory,
    const std::function< std::shared_ptr< INetworkClient >()> & networkFactory,
    const std::function< std::shared_ptr< IRenderer >()> & rendererFactory)
```

Definition at line 3 of file [engine.cpp](#).

18.40.2.2 ~Engine()

```
eng::Engine::~Engine () [default]
```

18.40.2.3 Engine() [2/3]

```
eng::Engine::Engine (
    const Engine & ) [delete]
```

18.40.2.4 Engine() [3/3]

```
eng::Engine::Engine (
    Engine && ) [delete]
```

18.40.3 Member Function Documentation

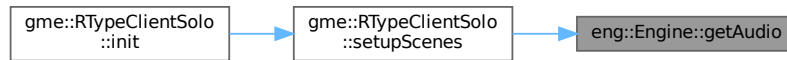
18.40.3.1 getAudio()

```
std::shared_ptr< IAudio > & eng::Engine::getAudio () [inline]
```

Definition at line 47 of file [Engine.hpp](#).

Referenced by [gme::RTypeClientSolo::setupScenes\(\)](#).

Here is the caller graph for this function:



18.40.3.2 getClock()

`std::unique_ptr< utl::Clock > & eng::Engine::getClock ()` [inline]

Definition at line 50 of file [Engine.hpp](#).

18.40.3.3 getNetwork()

`std::shared_ptr< INetworkClient > & eng::Engine::getNetwork ()` [inline]

Definition at line 48 of file [Engine.hpp](#).

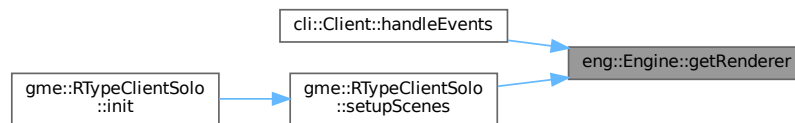
18.40.3.4 getRenderer()

`std::shared_ptr< IRenderer > & eng::Engine::getRenderer ()` [inline]

Definition at line 49 of file [Engine.hpp](#).

Referenced by [cli::Client::handleEvents\(\)](#), and [gme::RTypeClientSolo::setupScenes\(\)](#).

Here is the caller graph for this function:



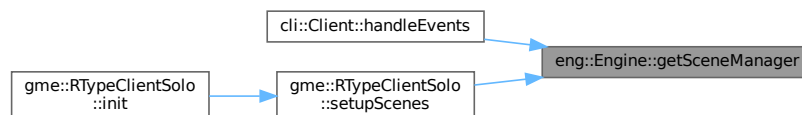
18.40.3.5 getSceneManager()

`std::unique_ptr< SceneManager > & eng::Engine::getSceneManager ()` [inline]

Definition at line 51 of file [Engine.hpp](#).

Referenced by [cli::Client::handleEvents\(\)](#), and [gme::RTypeClientSolo::setupScenes\(\)](#).

Here is the caller graph for this function:



18.40.3.6 getState()

`State eng::Engine::getState () const` [inline]

Definition at line 52 of file [Engine.hpp](#).

18.40.3.7 operator=() [1/2]

[Engine](#) & eng::Engine::operator= (
 const [Engine](#) &) [delete]

18.40.3.8 operator=() [2/2]

[Engine](#) & eng::Engine::operator= (
 [Engine](#) &&) [delete]

18.40.3.9 render()

void eng::Engine::render (
 const [WindowSize](#) & windowSize,
 [Color](#) clearColor,
 bool showDebug) const

Definition at line 11 of file [engine.cpp](#).

18.40.3.10 setState()

void eng::Engine::setState (
 const [State](#) newState) [inline]

Definition at line 54 of file [Engine.hpp](#).

Referenced by [cli::Client::handleEvents\(\)](#).

Here is the caller graph for this function:



18.40.3.11 stop()

void eng::Engine::stop () const [inline]

Definition at line 57 of file [Engine.hpp](#).

18.40.4 Member Data Documentation

18.40.4.1 m_audio

std::shared_ptr<[IAudio](#)> eng::Engine::m_audio [private]

Definition at line 63 of file [Engine.hpp](#).

18.40.4.2 m_clock

std::unique_ptr<[utl::Clock](#)> eng::Engine::m_clock [private]

Definition at line 61 of file [Engine.hpp](#).

18.40.4.3 m_network

std::shared_ptr<[INetworkClient](#)> eng::Engine::m_network [private]

Definition at line 64 of file [Engine.hpp](#).

18.40.4.4 m_renderer

`std::shared_ptr<IRenderer> eng::Engine::m_renderer [private]`

Definition at line 65 of file [Engine.hpp](#).

18.40.4.5 m_sceneManager

`std::unique_ptr<SceneManager> eng::Engine::m_sceneManager [private]`

Definition at line 62 of file [Engine.hpp](#).

18.40.4.6 m_state

`State eng::Engine::m_state = RUN [private]`

Definition at line 60 of file [Engine.hpp](#).

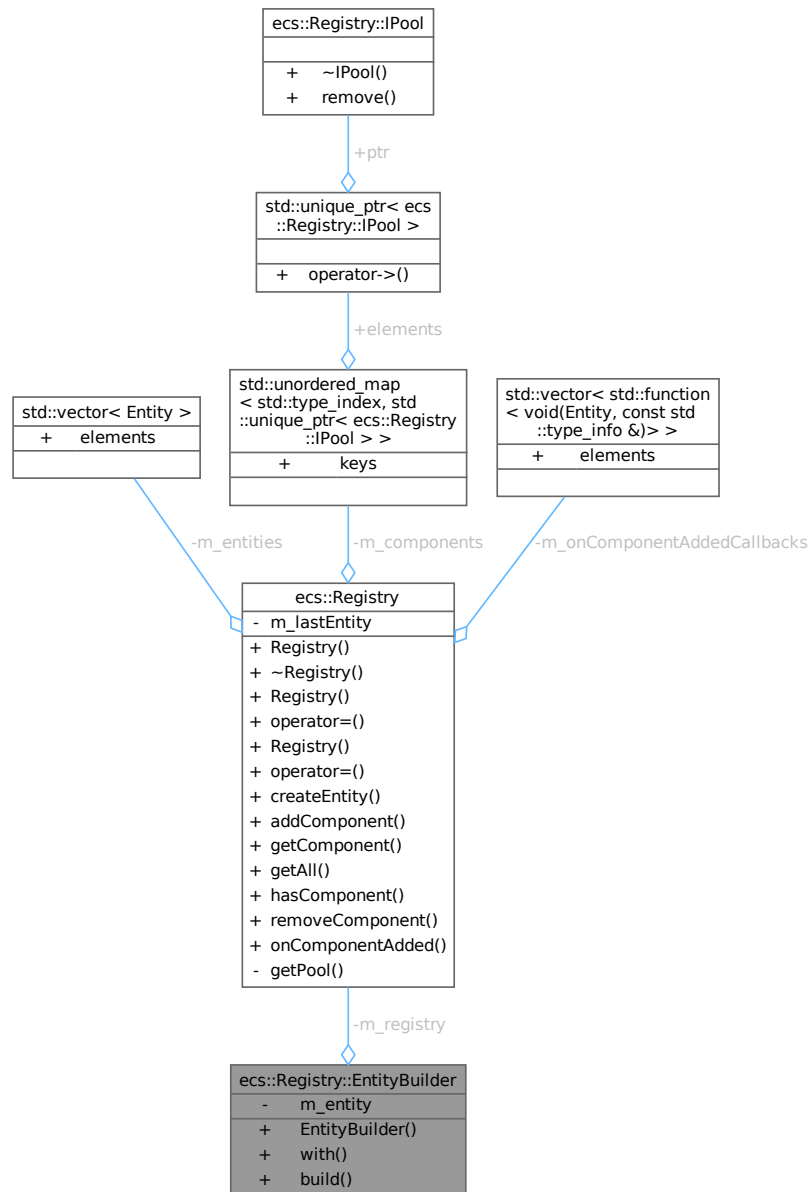
The documentation for this class was generated from the following files:

- [/home/masina/Projects/Epitech/rtype/modules/Engine/include/Engine/Engine.hpp](#)
- [/home/masina/Projects/Epitech/rtype/modules/Engine/src/engine.cpp](#)

18.41 ecs::Registry::EntityBuilder Class Reference

`#include <Registry.hpp>`

Collaboration diagram for ecs::Registry::EntityBuilder:



Public Member Functions

- [EntityBuilder](#) ([Registry](#) ®, [Entity](#) e)
- `template<typename T , typename... Args>`
[EntityBuilder](#) & [with](#) (Args &&...args)
- [Entity](#) [build](#) () const

Private Attributes

- [Registry](#) & [m_registry](#)
- [Entity](#) [m_entity](#)

18.41.1 Detailed Description

Definition at line 35 of file [Registry.hpp](#).

18.41.2 Constructor & Destructor Documentation

18.41.2.1 EntityBuilder()

```
ecs::Registry::EntityBuilder::EntityBuilder (
    Registry & reg,
    Entity e) [inline]
```

Definition at line 38 of file [Registry.hpp](#).

18.41.3 Member Function Documentation

18.41.3.1 build()

```
Entity ecs::Registry::EntityBuilder::build () const [inline]
```

Definition at line 46 of file [Registry.hpp](#).

References [m_entity](#).

18.41.3.2 with()

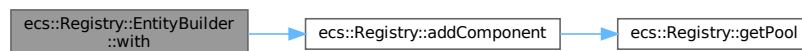
```
template<typename T , typename... Args>
EntityBuilder & ecs::Registry::EntityBuilder::with (
    Args &&... args) [inline]
```

Definition at line 40 of file [Registry.hpp](#).

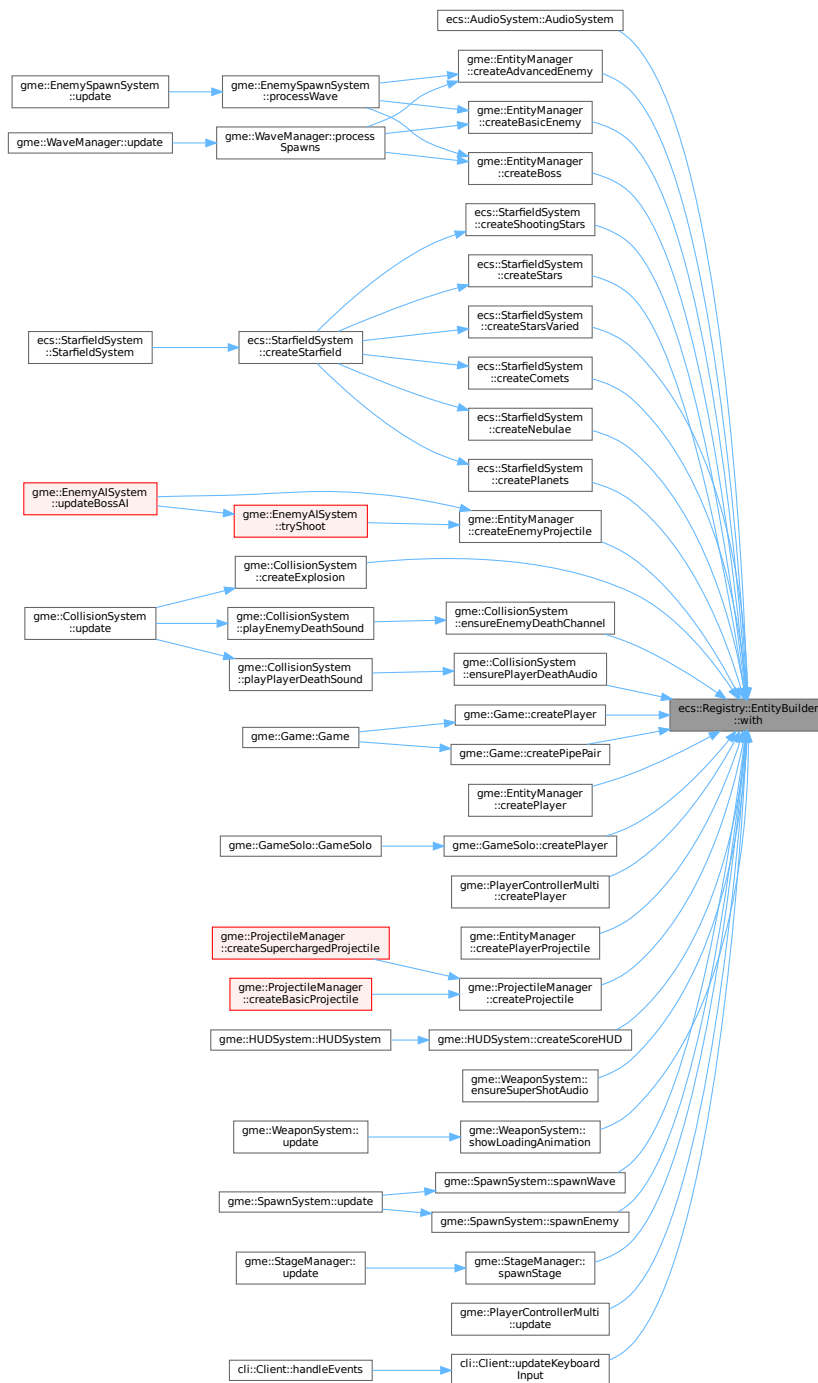
References [ecs::Registry::addComponent\(\)](#), [m_entity](#), and [m_registry](#).

Referenced by [ecs::AudioSystem::AudioSystem\(\)](#), [gme::EntityManager::createAdvancedEnemy\(\)](#), [gme::EntityManager::createBasicEnemy\(\)](#), [gme::EntityManager::createBoss\(\)](#), [ecs::StarfieldSystem::createComets\(\)](#), [gme::EntityManager::createEnemyProjectile\(\)](#), [gme::CollisionSystem::createExplosion\(\)](#), [ecs::StarfieldSystem::createNebula\(\)](#), [gme::Game::createPipePair\(\)](#), [ecs::StarfieldSystem::createPlanets\(\)](#), [gme::EntityManager::createPlayer\(\)](#), [gme::Game::createPlayer\(\)](#), [gme::GameSolo::createPlayer\(\)](#), [gme::PlayerControllerMulti::createPlayer\(\)](#), [gme::EntityManager::createPlayerProjectile\(\)](#), [gme::ProjectileManager::createProjectile\(\)](#), [gme::HUDSystem::createScore\(\)](#), [ecs::StarfieldSystem::createShootingStars\(\)](#), [ecs::StarfieldSystem::createStars\(\)](#), [ecs::StarfieldSystem::createStarsVaried\(\)](#), [gme::CollisionSystem::ensureEnemyDeathChannel\(\)](#), [gme::CollisionSystem::ensurePlayerDeathAudio\(\)](#), [gme::WeaponSystem::ensureSuperShotAudio\(\)](#), [gme::WeaponSystem::showLoadingAnimation\(\)](#), [gme::SpawnSystem::spawnWave\(\)](#), [gme::StageManager::spawnStage\(\)](#), [gme::SpawnSystem::spawnWave\(\)](#), [gme::PlayerControllerMulti::update\(\)](#), and [cli::Client::updateKeyboardInput\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.41.4 Member Data Documentation

18.41.4.1 m_entity

Entity ecs::Registry::EntityBuilder::m_entity [private]

Definition at line 50 of file [Registry.hpp](#).

Referenced by [build\(\)](#), and [with\(\)](#).

18.41.4.2 m_registry

[Registry](#)& ecs::Registry::EntityBuilder::m_registry [private]

Definition at line 49 of file [Registry.hpp](#).

Referenced by [with\(\)](#).

The documentation for this class was generated from the following file:

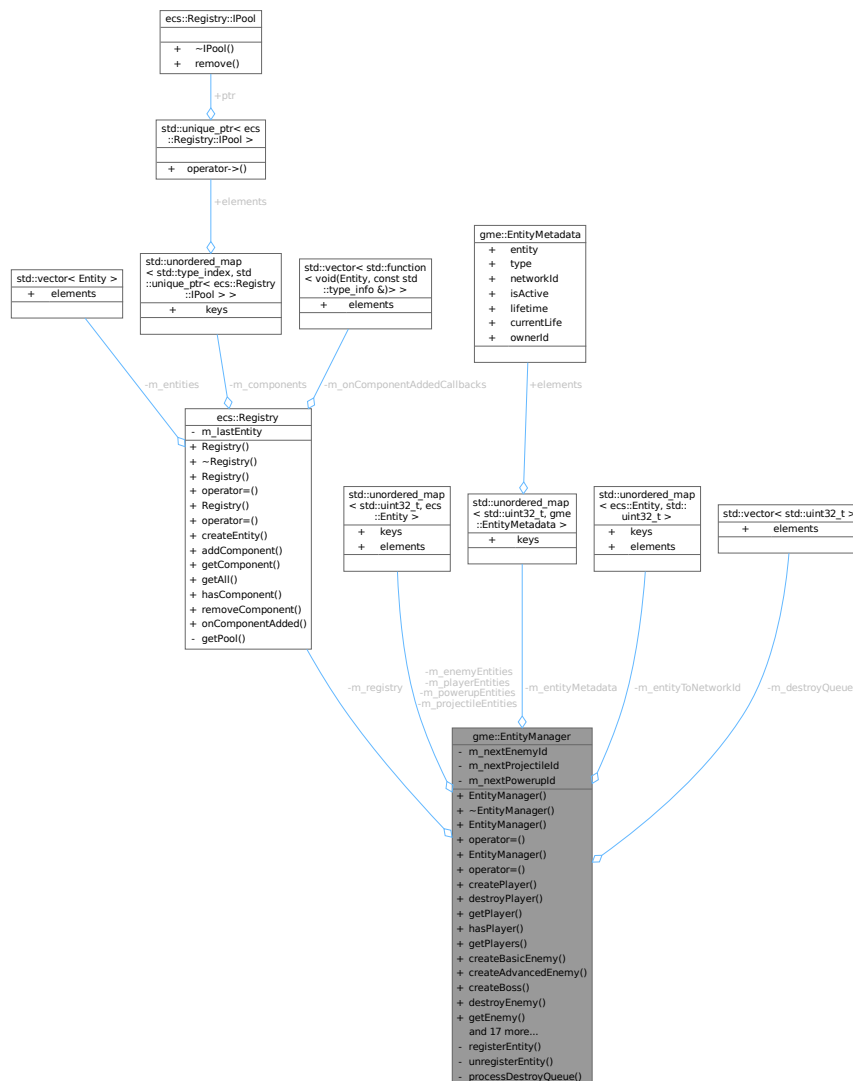
- /home/masina/Projects/Epitech/rtype/modules/ECS/include/ECS/[Registry.hpp](#)

18.42 gme::EntityManager Class Reference

Manages entity lifecycle and provides factory methods.

#include <EntityManager.hpp>

Collaboration diagram for gme::EntityManager:



Public Member Functions

- [EntityManager](#) (ecs::Registry ®istry)
- [~EntityManager](#) ()=default
- [EntityManager](#) (const [EntityManager](#) &)=delete
- [EntityManager](#) & operator= (const [EntityManager](#) &)=delete

- [EntityManager](#) ([EntityManager](#) &&)=delete
- [EntityManager](#) & operator= ([EntityManager](#) &&)=delete
- [ecs::Entity](#) createPlayer (std::uint32_t sessionId, float x, float y)
- void destroyPlayer (std::uint32_t sessionId)
- [ecs::Entity](#) getPlayer (std::uint32_t sessionId)
- bool hasPlayer (std::uint32_t sessionId) const
- const std::unordered_map< std::uint32_t, [ecs::Entity](#) > & getPlayers () const
- [ecs::Entity](#) createBasicEnemy (float x, float y, float health=50.0f)
- [ecs::Entity](#) createAdvancedEnemy (float x, float y, float health=100.0f)
- [ecs::Entity](#) createBoss (float x, float y, float health=1000.0f)
- void destroyEnemy (std::uint32_t enemyId)
- [ecs::Entity](#) getEnemy (std::uint32_t enemyId)
- const std::unordered_map< std::uint32_t, [ecs::Entity](#) > & getEnemies () const
- [ecs::Entity](#) createPlayerProjectile (std::uint32_t playerId, float x, float y, float vx, float vy, bool isSupercharged=false)
- [ecs::Entity](#) createEnemyProjectile (std::uint32_t enemyId, float x, float y, float vx, float vy)
- void destroyProjectile (std::uint32_t projectileId)
- [ecs::Entity](#) getProjectile (std::uint32_t projectileId)
- const std::unordered_map< std::uint32_t, [ecs::Entity](#) > & getProjectiles () const
- [ecs::Entity](#) createPowerup (float x, float y, uint8_t powerupType)
- void destroyPowerup (std::uint32_t powerupId)
- void updateLifetimes (float deltaTime)
- void cleanupDestroyedEntities ()
- [EntityMetadata](#) * getEntityMetadata (std::uint32_t networkId)
- const [EntityMetadata](#) * getEntityMetadata (std::uint32_t networkId) const
- std::uint32_t getNetworkIdForEntity ([ecs::Entity](#) entity) const
- std::vector< [rnp::EntityState](#) > getAllEntityStates () const
- [rnp::EntityType](#) getNetworkEntityType ([ServerEntityType](#) type) const
- void clear ()
- size_t getTotalEntityCount () const

Private Member Functions

- void registerEntity ([ecs::Entity](#) entity, [ServerEntityType](#) type, std::uint32_t networkId, float lifetime=-1.0f, std::uint32_t ownerId=0)
- void unregisterEntity (std::uint32_t networkId)
- void processDestroyQueue ()

Private Attributes

- [ecs::Registry](#) & m_registry
- std::unordered_map< std::uint32_t, [ecs::Entity](#) > m_playerEntities
- std::unordered_map< std::uint32_t, [ecs::Entity](#) > m_enemyEntities
- std::unordered_map< std::uint32_t, [ecs::Entity](#) > m_projectileEntities
- std::unordered_map< std::uint32_t, [ecs::Entity](#) > m_powerupEntities
- std::unordered_map< std::uint32_t, [EntityMetadata](#) > m_entityMetadata
- std::unordered_map< [ecs::Entity](#), std::uint32_t > m_entityToNetworkId
- std::uint32_t m_nextEnemyId = 2000
- std::uint32_t m_nextProjectileId = 1000
- std::uint32_t m_nextPowerupId = 5000
- std::vector< std::uint32_t > m_destroyQueue

18.42.1 Detailed Description

Manages entity lifecycle and provides factory methods.
Definition at line 51 of file [EntityManager.hpp](#).

18.42.2 Constructor & Destructor Documentation

18.42.2.1 EntityManager() [1/3]

`gme::EntityManager::EntityManager (`
 `ecs::Registry & registry) [explicit]`

Definition at line 14 of file [EntityManager.cpp](#).

18.42.2.2 ~EntityManager()

`gme::EntityManager::~~EntityManager () [default]`

18.42.2.3 EntityManager() [2/3]

`gme::EntityManager::EntityManager (`
 `const EntityManager & ) [delete]`

18.42.2.4 EntityManager() [3/3]

`gme::EntityManager::EntityManager (`
 `EntityManager && ) [delete]`

18.42.3 Member Function Documentation

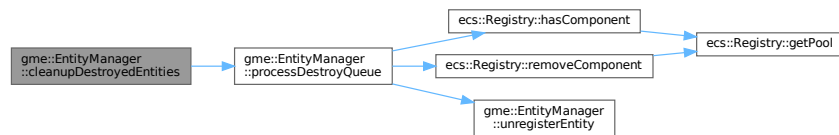
18.42.3.1 cleanupDestroyedEntities()

`void gme::EntityManager::cleanupDestroyedEntities ()`

Definition at line 259 of file [EntityManager.cpp](#).

References [processDestroyQueue\(\)](#).

Here is the call graph for this function:



18.42.3.2 clear()

`void gme::EntityManager::clear ()`

Definition at line 415 of file [EntityManager.cpp](#).

References [ecs::Registry::hasComponent\(\)](#), [m_destroyQueue](#), [m_enemyEntities](#), [m_entityMetadata](#), [m_entityToNetworkId](#), [m_playerEntities](#), [m_powerupEntities](#), [m_projectileEntities](#), [m_registry](#), and [ecs::Registry::removeComponent\(\)](#).

Here is the call graph for this function:



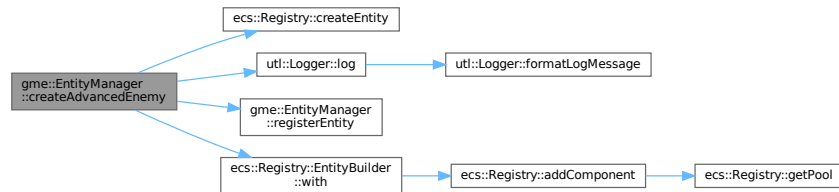
18.42.3.3 createAdvancedEnemy()

```
ecs::Entity gme::EntityManager::createAdvancedEnemy (
    float x,
    float y,
    float health = 100.0f)
```

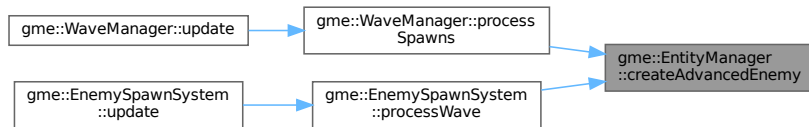
Definition at line 113 of file [EntityManager.cpp](#).

References [ecs::Registry::createEntity\(\)](#), [gme::ENEMY_ADVANCED](#), [utl::INFO](#), [utl::Logger::log\(\)](#), [m_enemyEntities](#), [m_nextEnemyId](#), [m_registry](#), [registerEntity\(\)](#), and [ecs::Registry::EntityBuilder::with\(\)](#).
Referenced by [gme::WaveManager::processSpawns\(\)](#), and [gme::EnemySpawnSystem::processWave\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.42.3.4 createBasicEnemy()

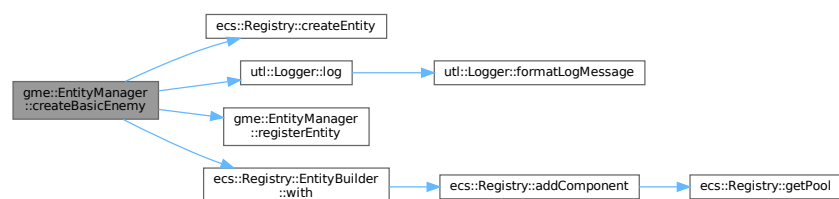
```
ecs::Entity gme::EntityManager::createBasicEnemy (
    float x,
    float y,
    float health = 50.0f)
```

Definition at line 86 of file [EntityManager.cpp](#).

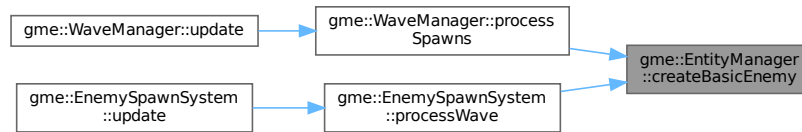
References [ecs::Registry::createEntity\(\)](#), [gme::ENEMY_BASIC](#), [utl::INFO](#), [utl::Logger::log\(\)](#), [m_enemyEntities](#), [m_nextEnemyId](#), [m_registry](#), [registerEntity\(\)](#), and [ecs::Registry::EntityBuilder::with\(\)](#).

Referenced by [gme::WaveManager::processSpawns\(\)](#), and [gme::EnemySpawnSystem::processWave\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.42.3.5 createBoss()

```

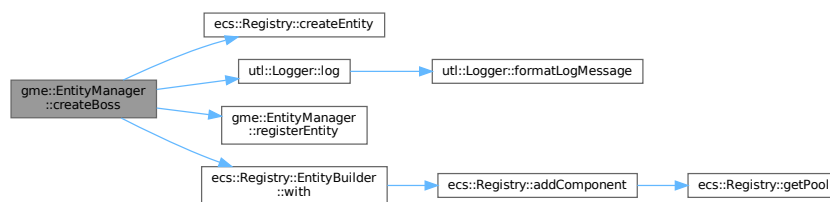
ecs::Entity gme::EntityManager::createBoss (
    float x,
    float y,
    float health = 1000.0f)
  
```

Definition at line 140 of file [EntityManager.cpp](#).

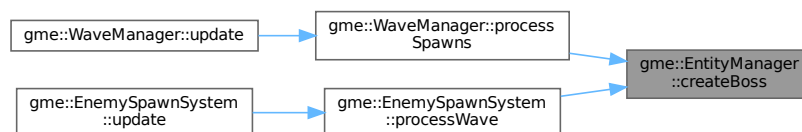
References [gme::BOSS](#), [ecs::Registry::createEntity\(\)](#), [utl::INFO](#), [utl::Logger::log\(\)](#), [m_enemyEntities](#), [m_nextEnemyId](#), [m_registry](#), [registerEntity\(\)](#), and [ecs::Registry::EntityBuilder::with\(\)](#).

Referenced by [gme::WaveManager::processSpawns\(\)](#), and [gme::EnemySpawnSystem::processWave\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



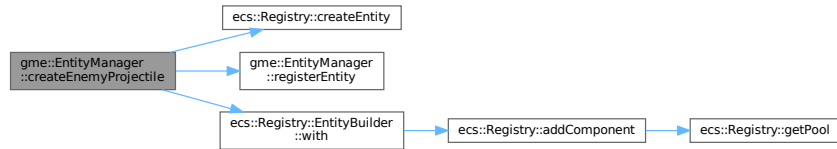
18.42.3.6 createEnemyProjectile()

```

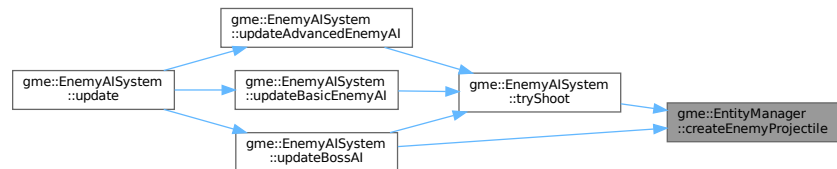
ecs::Entity gme::EntityManager::createEnemyProjectile (
    std::uint32_t enemyId,
    float x,
    float y,
    float vx,
    float vy)
  
```

Definition at line 208 of file [EntityManager.cpp](#).

References [ecs::Projectile::BASIC](#), [ecs::Registry::createEntity\(\)](#), [m_nextProjectileId](#), [m_projectileEntities](#), [m_registry](#), [gme::PROJECTILE_ENEMY](#), [registerEntity\(\)](#), and [ecs::Registry::EntityBuilder::with\(\)](#).
 Referenced by [gme::EnemyAISystem::tryShoot\(\)](#), and [gme::EnemyAISystem::updateBossAI\(\)](#).
 Here is the call graph for this function:



Here is the caller graph for this function:



18.42.3.7 createPlayer()

```

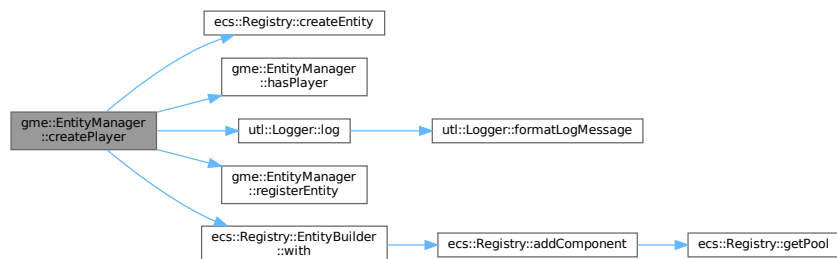
ecs::Entity gme::EntityManager::createPlayer (
    std::uint32_t sessionId,
    float x,
    float y)

```

Definition at line 16 of file [EntityManager.cpp](#).

References [ecs::Registry::createEntity\(\)](#), [hasPlayer\(\)](#), [utl::INFO](#), [utl::Logger::log\(\)](#), [m_playerEntities](#), [m_registry](#), [gme::PLAYER](#), [registerEntity\(\)](#), [utl::GameConfig::Server::SCREEN_HEIGHT](#), [utl::GameConfig::Server::SCREEN_WIDTH](#), [utl::WARNING](#), and [ecs::Registry::EntityBuilder::with\(\)](#).

Here is the call graph for this function:



18.42.3.8 createPlayerProjectile()

```

ecs::Entity gme::EntityManager::createPlayerProjectile (
    std::uint32_t playerId,
    float x,
    float y)

```

```

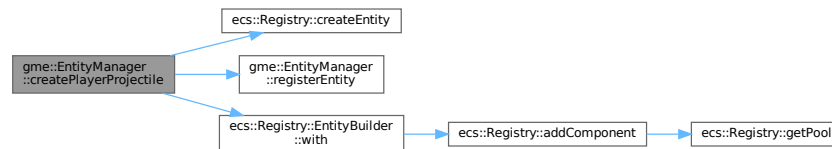
float y,
float vx,
float vy,
bool isSupercharged = false)

```

Definition at line 183 of file [EntityManager.cpp](#).

References [ecs::Projectile::BASIC](#), [ecs::Registry::createEntity\(\)](#), [m_nextProjectileId](#), [m_projectileEntities](#), [m_registry](#), [gme::PROJECTILE_PLAYER](#), [registerEntity\(\)](#), [ecs::Projectile::SUPERCHARGED](#), and [ecs::Registry::EntityBuilder::with\(\)](#).

Here is the call graph for this function:



18.42.3.9 createPowerup()

```

ecs::Entity gme::EntityManager::createPowerup (
    float x,
    float y,
    uint8_t powerupType)

```

18.42.3.10 destroyEnemy()

```

void gme::EntityManager::destroyEnemy (
    std::uint32_t enemyId)

```

Definition at line 167 of file [EntityManager.cpp](#).

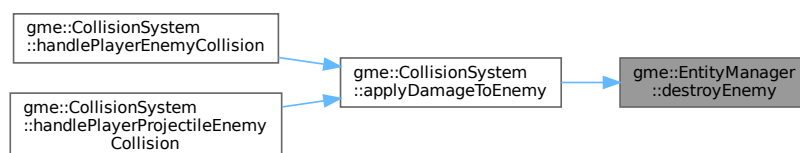
References [getEntityMetadata\(\)](#), and [m_destroyQueue](#).

Referenced by [gme::CollisionSystem::applyDamageToEnemy\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



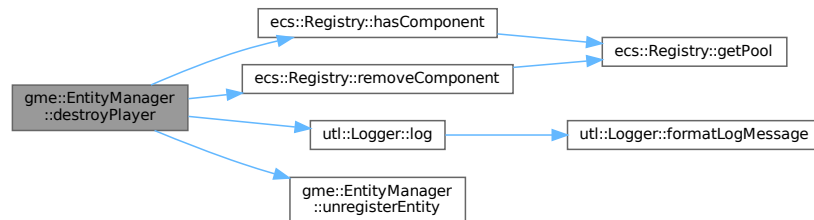
18.42.3.11 destroyPlayer()

```
void gme::EntityManager::destroyPlayer (
    std::uint32_t sessionId)
```

Definition at line 48 of file [EntityManager.cpp](#).

References [ecs::Registry::hasComponent\(\)](#), [utl::INFO](#), [utl::Logger::log\(\)](#), [m_playerEntities](#), [m_registry](#), [ecs::Registry::removeComponent\(\)](#), and [unregisterEntity\(\)](#).

Here is the call graph for this function:



18.42.3.12 destroyPowerup()

```
void gme::EntityManager::destroyPowerup (
    std::uint32_t powerupId)
```

18.42.3.13 destroyProjectile()

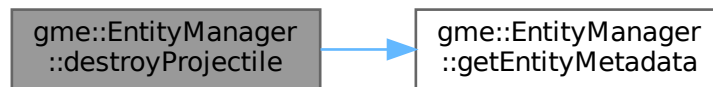
```
void gme::EntityManager::destroyProjectile (
    std::uint32_t projectileId)
```

Definition at line 227 of file [EntityManager.cpp](#).

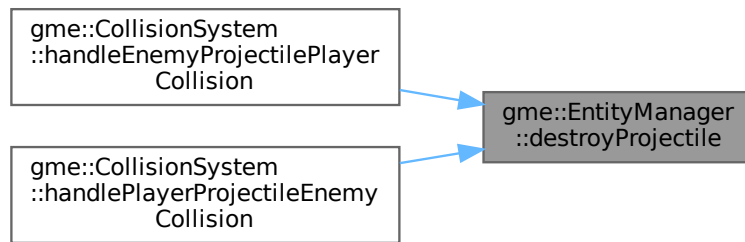
References [getEntityMetadata\(\)](#), and [m_destroyQueue](#).

Referenced by [gme::CollisionSystem::handleEnemyProjectilePlayerCollision\(\)](#), and [gme::CollisionSystem::handlePlayerProjectileCollision\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



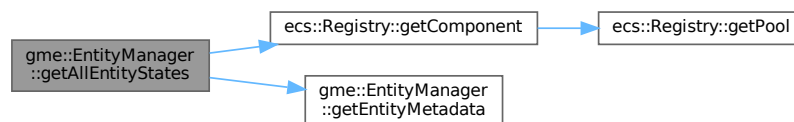
18.42.3.14 getAllEntityStates()

`std::vector< rnp::EntityState > gme::EntityManager::getAllEntityStates () const`

Definition at line 279 of file [EntityManager.cpp](#).

References [gme::BOSS](#), [rnp::BOSS](#), [rnp::ENEMY](#), [gme::ENEMY_ADVANCED](#), [rnp::ENEMY_ADVANCED](#), [rnp::ENEMY_BASIC](#), [rnp::ENEMY_BOSS](#), [ecs::Registry::getComponent\(\)](#), [getEntityMetadata\(\)](#), [rnp::EntityState::healthPercent](#), [rnp::EntityState::id](#), [m_enemyEntities](#), [m_playerEntities](#), [m_projectileEntities](#), [m_registry](#), [rnp::NONE](#), [rnp::PLAYER](#), [rnp::PROJECTILE](#), [gme::PROJECTILE_ENEMY](#), [rnp::PROJECTILE_ENEMY](#), [gme::PROJECTILE_PLAYER](#), [rnp::PROJECTILE_PLAYER](#), [rnp::PROJECTILE_PLAYER_SUPERCHARGED](#), [rnp::EntityState::stateFlags](#), [rnp::EntityState::subtype](#), [ecs::Projectile::SUPERCHARGED](#), [rnp::EntityState::type](#), [rnp::EntityState::vx](#), [rnp::EntityState::vy](#), [ecs::Transform::x](#), [rnp::EntityState::x](#), [ecs::Transform::y](#), and [rnp::EntityState::y](#).

Here is the call graph for this function:



18.42.3.15 getEnemies()

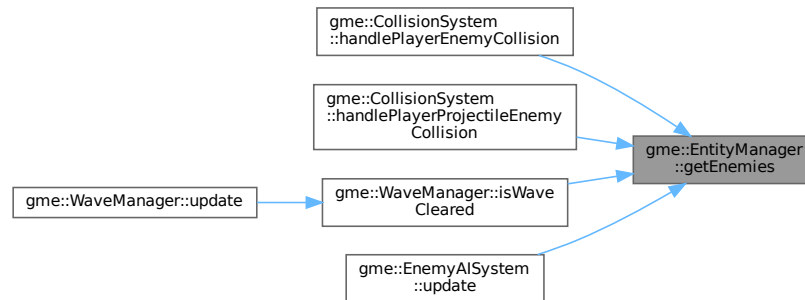
`const std::unordered_map< std::uint32_t, ecs::Entity > & gme::EntityManager::getEnemies () const [inline]`

Definition at line 75 of file [EntityManager.hpp](#).

References [m_enemyEntities](#).

Referenced by [gme::CollisionSystem::handlePlayerEnemyCollision\(\)](#), [gme::CollisionSystem::handlePlayerProjectileEnemyCollision\(\)](#), [gme::WaveManager::isWaveCleared\(\)](#), and [gme::EnemyAISystem::update\(\)](#).

Here is the caller graph for this function:



18.42.3.16 getEnemy()

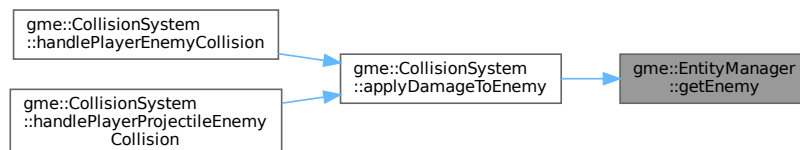
`ecs::Entity` gme::EntityManager::getEnemy (
 std::uint32_t enemyId)

Definition at line 177 of file [EntityManager.cpp](#).

References [ecs::INVALID_ENTITY](#), and [m_enemyEntities](#).

Referenced by [gme::CollisionSystem::applyDamageToEnemy\(\)](#).

Here is the caller graph for this function:



18.42.3.17 getEntityMetadata() [1/2]

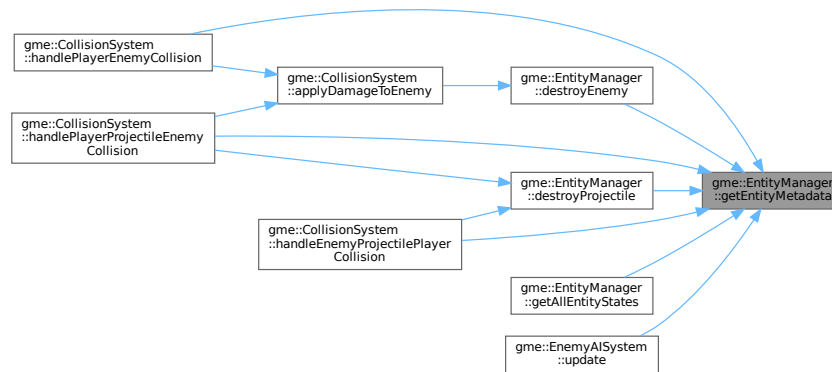
`EntityMetadata *` gme::EntityManager::getEntityMetadata (
 std::uint32_t networkId)

Definition at line 261 of file [EntityManager.cpp](#).

References [m_entityMetadata](#).

Referenced by [destroyEnemy\(\)](#), [destroyProjectile\(\)](#), [getAllEntityStates\(\)](#), [gme::CollisionSystem::handleEnemyProjectileCollision\(\)](#), [gme::CollisionSystem::handlePlayerEnemyCollision\(\)](#), [gme::CollisionSystem::handlePlayerProjectileEnemyCollision\(\)](#), and [gme::EnemyAISystem::update\(\)](#).

Here is the caller graph for this function:



18.42.3.18 getEntityMetadata() [2/2]

```
const EntityMetadata * gme::EntityManager::getEntityMetadata (
    std::uint32_t networkId) const
```

Definition at line 267 of file [EntityManager.cpp](#).

References [m_entityMetadata](#).

18.42.3.19 getNetworkEntityType()

```
rnp::EntityType gme::EntityManager::getNetworkEntityType (
    ServerEntityType type) const
```

Definition at line 396 of file [EntityManager.cpp](#).

References [gme::BOSS](#), [rnp::BOSS](#), [rnp::ENEMY](#), [gme::ENEMY_ADVANCED](#), [gme::ENEMY_BASIC](#), [gme::PLAYER](#), [rnp::PLAYER](#), [rnp::PROJECTILE](#), [gme::PROJECTILE_ENEMY](#), and [gme::PROJECTILE_PLAYER](#).

18.42.3.20 getNetworkIdForEntity()

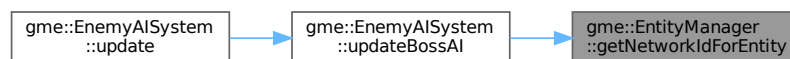
```
std::uint32_t gme::EntityManager::getNetworkIdForEntity (
    ecs::Entity entity) const
```

Definition at line 273 of file [EntityManager.cpp](#).

References [m_entityToNetworkId](#).

Referenced by [gme::EnemyAISystem::updateBossAI\(\)](#).

Here is the caller graph for this function:



18.42.3.21 getPlayer()

```
ecs::Entity gme::EntityManager::getPlayer (
    std::uint32_t sessionId)
```

Definition at line 75 of file [EntityManager.cpp](#).

References [ecs::INVALID_ENTITY](#), and [m_playerEntities](#).

18.42.3.22 getPlayers()

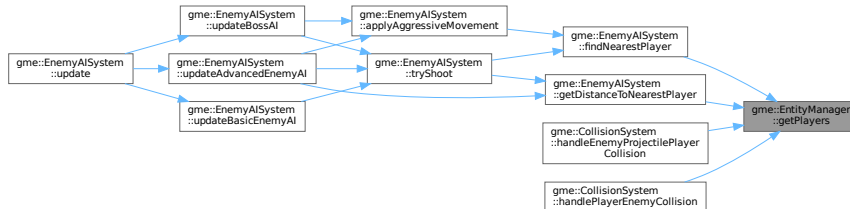
const std::unordered_map< std::uint32_t, [ecs::Entity](#) > & gme::EntityManager::getPlayers () const [inline]

Definition at line 67 of file [EntityManager.hpp](#).

References [m_playerEntities](#).

Referenced by [gme::EnemyAISystem::findNearestPlayer\(\)](#), [gme::EnemyAISystem::getDistanceToNearestPlayer\(\)](#), [gme::CollisionSystem::handleEnemyProjectilePlayerCollision\(\)](#), and [gme::CollisionSystem::handlePlayerEnemyCollision\(\)](#).

Here is the caller graph for this function:



18.42.3.23 getProjectile()

[ecs::Entity](#) gme::EntityManager::getProjectile (
std::uint32_t projectileId)

Definition at line 237 of file [EntityManager.cpp](#).

References [ecs::INVALID_ENTITY](#), and [m_projectileEntities](#).

18.42.3.24 getProjectiles()

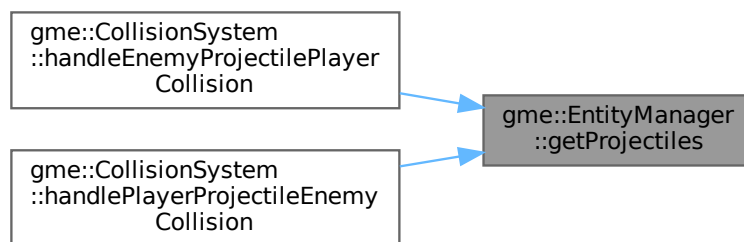
const std::unordered_map< std::uint32_t, [ecs::Entity](#) > & gme::EntityManager::getProjectiles () const [inline]

Definition at line 83 of file [EntityManager.hpp](#).

References [m_projectileEntities](#).

Referenced by [gme::CollisionSystem::handleEnemyProjectilePlayerCollision\(\)](#), and [gme::CollisionSystem::handlePlayerProjectileEnemyCollision\(\)](#).

Here is the caller graph for this function:



18.42.3.25 getTotalEntityCount()

size_t gme::EntityManager::getTotalEntityCount () const

Definition at line 478 of file [EntityManager.cpp](#).

References [m_enemyEntities](#), [m_playerEntities](#), [m_powerupEntities](#), and [m_projectileEntities](#).

18.42.3.26 hasPlayer()

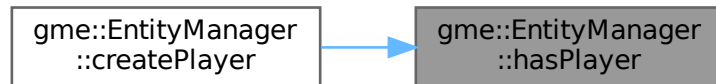
```
bool gme::EntityManager::hasPlayer (
    std::uint32_t sessionId) const
```

Definition at line 81 of file [EntityManager.cpp](#).

References [m_playerEntities](#).

Referenced by [createPlayer\(\)](#).

Here is the caller graph for this function:



18.42.3.27 operator=() [1/2]

```
EntityManager & gme::EntityManager::operator= (
    const EntityManager & ) [delete]
```

18.42.3.28 operator=() [2/2]

```
EntityManager & gme::EntityManager::operator= (
    EntityManager && ) [delete]
```

18.42.3.29 processDestroyQueue()

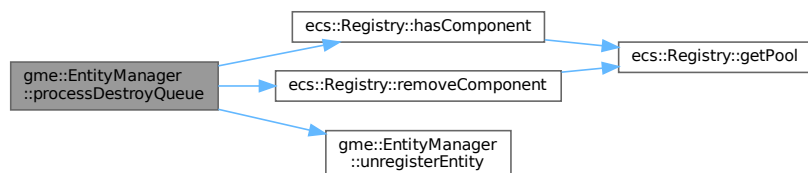
```
void gme::EntityManager::processDestroyQueue () [private]
```

Definition at line 510 of file [EntityManager.cpp](#).

References [ecs::Registry::hasComponent\(\)](#), [m_destroyQueue](#), [m_enemyEntities](#), [m_powerupEntities](#), [m_projectileEntities](#), [m_registry](#), [ecs::Registry::removeComponent\(\)](#), and [unregisterEntity\(\)](#).

Referenced by [cleanupDestroyedEntities\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.42.3.30 registerEntity()

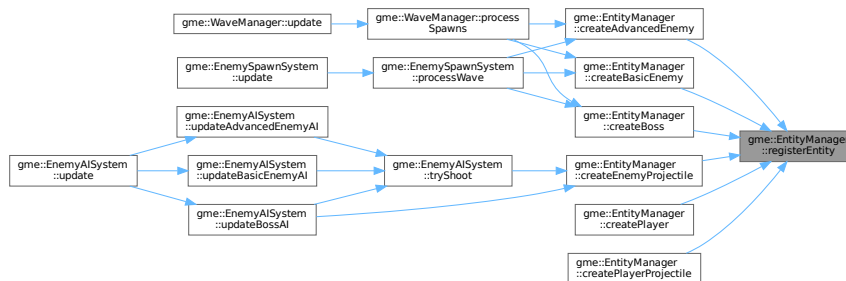
```
void gme::EntityManager::registerEntity (
    ecs::Entity entity,
    ServerEntityType type,
    std::uint32_t networkId,
    float lifetime = -1.0f,
    std::uint32_t ownerId = 0) [private]
```

Definition at line 484 of file [EntityManager.cpp](#).

References [gme::EntityMetadata::currentLife](#), [gme::EntityMetadata::entity](#), [gme::EntityMetadata::isActive](#), [gme::EntityMetadata::lifetime](#), [m_entityMetadata](#), [m_entityToNetworkId](#), [gme::EntityMetadata::networkId](#), [gme::EntityMetadata::ownerId](#), and [gme::EntityMetadata::type](#).

Referenced by [createAdvancedEnemy\(\)](#), [createBasicEnemy\(\)](#), [createBoss\(\)](#), [createEnemyProjectile\(\)](#), [createPlayer\(\)](#), and [createPlayerProjectile\(\)](#).

Here is the caller graph for this function:



18.42.3.31 unregisterEntity()

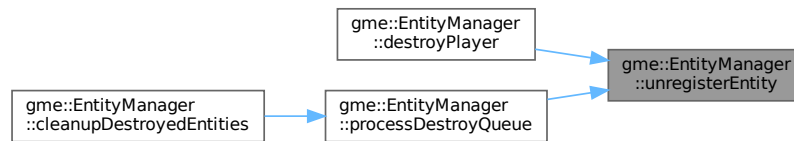
```
void gme::EntityManager::unregisterEntity (
    std::uint32_t networkId) [private]
```

Definition at line 500 of file [EntityManager.cpp](#).

References [m_entityMetadata](#), and [m_entityToNetworkId](#).

Referenced by [destroyPlayer\(\)](#), and [processDestroyQueue\(\)](#).

Here is the caller graph for this function:



18.42.3.32 updateLifetimes()

```
void gme::EntityManager::updateLifetimes (
    float deltaTime)
```

Definition at line 243 of file [EntityManager.cpp](#).

References [m_destroyQueue](#), and [m_entityMetadata](#).

18.42.4 Member Data Documentation

18.42.4.1 m_destroyQueue

```
std::vector<std::uint32_t> gme::EntityManager::m_destroyQueue [private]
```

Definition at line 128 of file [EntityManager.hpp](#).

Referenced by [clear\(\)](#), [destroyEnemy\(\)](#), [destroyProjectile\(\)](#), [processDestroyQueue\(\)](#), and [updateLifetimes\(\)](#).

18.42.4.2 m_enemyEntities

```
std::unordered_map<std::uint32_t, ecs::Entity> gme::EntityManager::m_enemyEntities [private]
```

Definition at line 112 of file [EntityManager.hpp](#).

Referenced by [clear\(\)](#), [createAdvancedEnemy\(\)](#), [createBasicEnemy\(\)](#), [createBoss\(\)](#), [getAllEntityStates\(\)](#), [getEnemies\(\)](#), [getEnemy\(\)](#), [getTotalEntityCount\(\)](#), and [processDestroyQueue\(\)](#).

18.42.4.3 m_entityMetadata

```
std::unordered_map<std::uint32_t, EntityMetadata> gme::EntityManager::m_entityMetadata [private]
```

Definition at line 117 of file [EntityManager.hpp](#).

Referenced by [clear\(\)](#), [getEntityMetadata\(\)](#), [getEntityMetadata\(\)](#), [registerEntity\(\)](#), [unregisterEntity\(\)](#), and [updateLifetimes\(\)](#).

18.42.4.4 m_entityToNetworkId

```
std::unordered_map<ecs::Entity, std::uint32_t> gme::EntityManager::m_entityToNetworkId [private]
```

Definition at line 120 of file [EntityManager.hpp](#).

Referenced by [clear\(\)](#), [getNetworkIdForEntity\(\)](#), [registerEntity\(\)](#), and [unregisterEntity\(\)](#).

18.42.4.5 m_nextEnemyId

```
std::uint32_t gme::EntityManager::m_nextEnemyId = 2000 [private]
```

Definition at line 123 of file [EntityManager.hpp](#).

Referenced by [createAdvancedEnemy\(\)](#), [createBasicEnemy\(\)](#), and [createBoss\(\)](#).

18.42.4.6 m_nextPowerupId

```
std::uint32_t gme::EntityManager::m_nextPowerupId = 5000 [private]
```

Definition at line 125 of file [EntityManager.hpp](#).

18.42.4.7 m_nextProjectileId

std::uint32_t gme::EntityManager::m_nextProjectileId = 1000 [private]

Definition at line 124 of file [EntityManager.hpp](#).

Referenced by [createEnemyProjectile\(\)](#), and [createPlayerProjectile\(\)](#).

18.42.4.8 m_playerEntities

std::unordered_map<std::uint32_t, [ecs::Entity](#)> gme::EntityManager::m_playerEntities [private]

Definition at line 111 of file [EntityManager.hpp](#).

Referenced by [clear\(\)](#), [createPlayer\(\)](#), [destroyPlayer\(\)](#), [getAllEntityStates\(\)](#), [getPlayer\(\)](#), [getPlayers\(\)](#), [getTotalEntityCount\(\)](#), and [hasPlayer\(\)](#).

18.42.4.9 m_powerupEntities

std::unordered_map<std::uint32_t, [ecs::Entity](#)> gme::EntityManager::m_powerupEntities [private]

Definition at line 114 of file [EntityManager.hpp](#).

Referenced by [clear\(\)](#), [getTotalEntityCount\(\)](#), and [processDestroyQueue\(\)](#).

18.42.4.10 m_projectileEntities

std::unordered_map<std::uint32_t, [ecs::Entity](#)> gme::EntityManager::m_projectileEntities [private]

Definition at line 113 of file [EntityManager.hpp](#).

Referenced by [clear\(\)](#), [createEnemyProjectile\(\)](#), [createPlayerProjectile\(\)](#), [getAllEntityStates\(\)](#), [getProjectile\(\)](#), [getProjectiles\(\)](#), [getTotalEntityCount\(\)](#), and [processDestroyQueue\(\)](#).

18.42.4.11 m_registry

[ecs::Registry](#)& gme::EntityManager::m_registry [private]

Definition at line 108 of file [EntityManager.hpp](#).

Referenced by [clear\(\)](#), [createAdvancedEnemy\(\)](#), [createBasicEnemy\(\)](#), [createBoss\(\)](#), [createEnemyProjectile\(\)](#), [createPlayer\(\)](#), [createPlayerProjectile\(\)](#), [destroyPlayer\(\)](#), [getAllEntityStates\(\)](#), and [processDestroyQueue\(\)](#).

The documentation for this class was generated from the following files:

- [/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Server/include/RTypeServer/EntityManager.hpp](#)
- [/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Server/src/EntityManager.cpp](#)

18.43 gme::EntityMetadata Struct Reference

Entity metadata for tracking.

#include <EntityManager.hpp>

Collaboration diagram for gme::EntityMetadata:

gme::EntityMetadata	
+	entity
+	type
+	networkId
+	isActive
+	lifetime
+	currentLife
+	ownerId

Public Attributes

- [ecs::Entity](#) entity
- [ServerEntityType](#) type
- [std::uint32_t](#) networkId
- [bool](#) isActive
- [float](#) lifetime
- [float](#) currentLife
- [std::uint32_t](#) ownerId

18.43.1 Detailed Description

Entity metadata for tracking.

Definition at line 35 of file [EntityManager.hpp](#).

18.43.2 Member Data Documentation

18.43.2.1 currentLife

[float](#) gme::EntityMetadata::currentLife

Definition at line 42 of file [EntityManager.hpp](#).

Referenced by [gme::EntityManager::registerEntity\(\)](#).

18.43.2.2 entity

[ecs::Entity](#) gme::EntityMetadata::entity

Definition at line 37 of file [EntityManager.hpp](#).

Referenced by [gme::EntityManager::registerEntity\(\)](#).

18.43.2.3 isActive

[bool](#) gme::EntityMetadata::isActive

Definition at line 40 of file [EntityManager.hpp](#).

Referenced by [gme::EntityManager::registerEntity\(\)](#).

18.43.2.4 lifetime

float gme::EntityMetadata::lifetime

Definition at line 41 of file [EntityManager.hpp](#).

Referenced by [gme::EntityManager::registerEntity\(\)](#).

18.43.2.5 networkId

std::uint32_t gme::EntityMetadata::networkId

Definition at line 39 of file [EntityManager.hpp](#).

Referenced by [gme::EntityManager::registerEntity\(\)](#).

18.43.2.6 ownerId

std::uint32_t gme::EntityMetadata::ownerId

Definition at line 43 of file [EntityManager.hpp](#).

Referenced by [gme::EntityManager::registerEntity\(\)](#).

18.43.2.7 type

[ServerEntityType](#) gme::EntityMetadata::type

Definition at line 38 of file [EntityManager.hpp](#).

Referenced by [gme::EntityManager::registerEntity\(\)](#).

The documentation for this struct was generated from the following file:

- [/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Server/include/RTypeServer/EntityManager.hpp](#)

18.44 rnp::EntityState Struct Reference

Entity state for WORLD_STATE packet.

#include <Protocol.hpp>

Collaboration diagram for rnp::EntityState:

rnp::EntityState
+ id
+ type
+ subtype
+ x
+ y
+ vx
+ vy
+ healthPercent
+ stateFlags

Public Attributes

- std::uint32_t [id](#)
- std::uint16_t [type](#)

- `std::uint8_t` [subtype](#)
- `float` [x](#)
- `float` [y](#)
- `float` [vx](#)
- `float` [vy](#)
- `std::uint8_t` [healthPercent](#)
- `std::uint8_t` [stateFlags](#)

18.44.1 Detailed Description

Entity state for WORLD_STATE packet.
Definition at line 181 of file [Protocol.hpp](#).

18.44.2 Member Data Documentation

18.44.2.1 healthPercent

`std::uint8_t rnp::EntityState::healthPercent`

Definition at line 188 of file [Protocol.hpp](#).

Referenced by [rnp::Serializer::deserializeEntityState\(\)](#), [gme::EntityManager::getAllEntityStates\(\)](#), and [rnp::Serializer::serializeEntityState\(\)](#).

18.44.2.2 id

`std::uint32_t rnp::EntityState::id`

Definition at line 183 of file [Protocol.hpp](#).

Referenced by [rnp::Serializer::deserializeEntityState\(\)](#), [gme::EntityManager::getAllEntityStates\(\)](#), and [rnp::Serializer::serializeEntityState\(\)](#).

18.44.2.3 stateFlags

`std::uint8_t rnp::EntityState::stateFlags`

Definition at line 189 of file [Protocol.hpp](#).

Referenced by [rnp::Serializer::deserializeEntityState\(\)](#), [gme::EntityManager::getAllEntityStates\(\)](#), and [rnp::Serializer::serializeEntityState\(\)](#).

18.44.2.4 subtype

`std::uint8_t rnp::EntityState::subtype`

Definition at line 185 of file [Protocol.hpp](#).

Referenced by [rnp::Serializer::deserializeEntityState\(\)](#), [gme::EntityManager::getAllEntityStates\(\)](#), and [rnp::Serializer::serializeEntityState\(\)](#).

18.44.2.5 type

`std::uint16_t rnp::EntityState::type`

Definition at line 184 of file [Protocol.hpp](#).

Referenced by [rnp::Serializer::deserializeEntityState\(\)](#), [gme::EntityManager::getAllEntityStates\(\)](#), and [rnp::Serializer::serializeEntityState\(\)](#).

18.44.2.6 vx

`float rnp::EntityState::vx`

Definition at line 187 of file [Protocol.hpp](#).

Referenced by [rnp::Serializer::deserializeEntityState\(\)](#), [gme::EntityManager::getAllEntityStates\(\)](#), and [rnp::Serializer::serializeEntityState\(\)](#).

18.44.2.7 vy

float rnp::EntityState::vy

Definition at line 187 of file [Protocol.hpp](#).

Referenced by [rnp::Serializer::deserializeEntityState\(\)](#), [gme::EntityManager::getAllEntityStates\(\)](#), and [rnp::Serializer::serializeEntityState\(\)](#).

18.44.2.8 x

float rnp::EntityState::x

Definition at line 186 of file [Protocol.hpp](#).

Referenced by [rnp::Serializer::deserializeEntityState\(\)](#), [gme::EntityManager::getAllEntityStates\(\)](#), and [rnp::Serializer::serializeEntityState\(\)](#).

18.44.2.9 y

float rnp::EntityState::y

Definition at line 186 of file [Protocol.hpp](#).

Referenced by [rnp::Serializer::deserializeEntityState\(\)](#), [gme::EntityManager::getAllEntityStates\(\)](#), and [rnp::Serializer::serializeEntityState\(\)](#).

The documentation for this struct was generated from the following file:

- [/home/masina/Projects/Epitech/rtype/modules/Interfaces/include/Interfaces/Protocol/Protocol.hpp](#)

18.45 cli::EnvConfig Struct Reference

`#include <ArgsHandler.hpp>`

Collaboration diagram for cli::EnvConfig:



18.45.1 Detailed Description

Definition at line 36 of file [ArgsHandler.hpp](#).

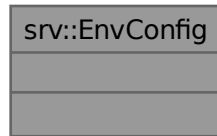
The documentation for this struct was generated from the following file:

- [/home/masina/Projects/Epitech/rtype/client/include/Client/ArgsHandler.hpp](#)

18.46 srv::EnvConfig Struct Reference

`#include <ArgsHandler.hpp>`

Collaboration diagram for `srv::EnvConfig`:



18.46.1 Detailed Description

Definition at line 30 of file [ArgsHandler.hpp](#).

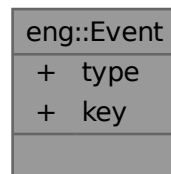
The documentation for this struct was generated from the following file:

- [/home/masina/Projects/Epitech/rtype/server/include/Server/ArgsHandler.hpp](#)

18.47 eng::Event Struct Reference

#include <IRenderer.hpp>

Collaboration diagram for `eng::Event`:



Public Attributes

- [EventType](#) type = [EventType::None](#)
- [Key](#) key = [Key::Unknown](#)

18.47.1 Detailed Description

Definition at line 102 of file [IRenderer.hpp](#).

18.47.2 Member Data Documentation

18.47.2.1 key

[Key](#) `eng::Event::key` = [Key::Unknown](#)

Definition at line 105 of file [IRenderer.hpp](#).

Referenced by [cli::Menu::event\(\)](#), [cli::Settings::event\(\)](#), [gme::ConfigMulti::event\(\)](#), [gme::ConfigSolo::event\(\)](#), [gme::CreateRoomScene::event\(\)](#), [gme::Game::event\(\)](#), [gme::GameSolo::event\(\)](#), [gme::JoinRoomScene::event\(\)](#), [gme::ServerScene::event\(\)](#), [gme::WaitingRoomScene::event\(\)](#), [cli::Client::handleEvents\(\)](#), and [gme::PlayerControllerMulti::event\(\)](#).

18.47.2.2 type

[EventType](#) eng::Event::type = [EventType::None](#)

Definition at line 104 of file [IRenderer.hpp](#).

Referenced by [cli::Menu::event\(\)](#), [cli::Settings::event\(\)](#), [gme::ConfigMulti::event\(\)](#), [gme::ConfigSolo::event\(\)](#), [gme::CreateRoomScene::event\(\)](#), [gme::Game::event\(\)](#), [gme::GameSolo::event\(\)](#), [gme::JoinRoomScene::event\(\)](#), [gme::ServerScene::event\(\)](#), [gme::WaitingRoomScene::event\(\)](#), [cli::Client::handleEvents\(\)](#), and [gme::PlayerControllerMulti::event\(\)](#).

The documentation for this struct was generated from the following file:

- [/home/masina/Projects/Epitech/rtype/modules/Interfaces/include/Interfaces/IRenderer.hpp](#)

18.48 utl::Event Class Reference

[Event](#) structure for inter-component communication.

#include <Event.hpp>

Collaboration diagram for utl::Event:



Public Member Functions

- [Event](#) ()
Default constructor with current timestamp.
- [Event](#) ([EventType](#) t)
Constructor with event type.
- [Event](#) ([EventType](#) t, const std::vector< std::uint8_t > &d)

- Constructor with event type and data.
 - [Event](#) ([EventType](#) t, std::uint32_t src, std::uint32_t tgt=0, [EventPriority](#) prio=[EventPriority::NORMAL](#))
- Constructor with all parameters.
 - std::uint64_t [getAge](#) () const
- Get event age in milliseconds.
 - bool [hasExpired](#) (std::uint64_t maxAgeMs=5000) const
- Check if event has expired.
 - bool [isTargeted](#) () const
- Check if event is targeted to a specific component.
 - bool [isBroadcast](#) () const
- Check if event is a broadcast.
 -

Public Attributes

- [EventType](#) type
- Type of the event.
 - std::uint32_t [sourceId](#) = 0
- ID of the component that sent the event (0 = system)
 - std::uint32_t [targetId](#) = 0
- ID of the target component (0 = broadcast)
 - [EventPriority](#) priority = [EventPriority::NORMAL](#)
- Event priority.
 - std::chrono::steady_clock::time_point [timestamp](#)
- Timestamp when event was created.
 - std::vector< std::uint8_t > [data](#)
- Serialized event data.
 -

18.48.1 Detailed Description

[Event](#) structure for inter-component communication.
 Definition at line 87 of file [Event.hpp](#).

18.48.2 Constructor & Destructor Documentation

18.48.2.1 Event() [1/4]

utl::Event::Event () [inline]
 Default constructor with current timestamp.
 Definition at line 100 of file [Event.hpp](#).

18.48.2.2 Event() [2/4]

utl::Event::Event (
 [EventType](#) t) [inline], [explicit]
 Constructor with event type.

Parameters

t	Event type
---	----------------------------

Definition at line 106 of file [Event.hpp](#).

18.48.2.3 Event() [3/4]

utl::Event::Event (
 [EventType](#) t,
 const std::vector< std::uint8_t > & d) [inline]
 Constructor with event type and data.

Parameters

t	Event type
d	Event data

Definition at line 113 of file [Event.hpp](#).

18.48.2.4 Event() [4/4]

```
utl::Event::Event (
    EventType t,
    std::uint32_t src,
    std::uint32_t tgt = 0,
    EventPriority prio = EventPriority::NORMAL) [inline]
```

Constructor with all parameters.

Parameters

t	Event type
src	Source component ID
tgt	Target component ID (0 for broadcast)
prio	Event priority

Definition at line 125 of file [Event.hpp](#).

18.48.3 Member Function Documentation

18.48.3.1 getAge()

```
std::uint64_t utl::Event::getAge () const [inline]
```

Get event age in milliseconds.

Returns

Age in milliseconds

Definition at line 134 of file [Event.hpp](#).

References [timestamp](#).

Referenced by [hasExpired\(\)](#).

Here is the caller graph for this function:



18.48.3.2 hasExpired()

```
bool utl::Event::hasExpired (
    std::uint64_t maxAgeMs = 5000) const [inline]
```

Check if event has expired.

Parameters

maxAgeMs	Maximum age in milliseconds
----------	-----------------------------

Returns

True if event has expired

Definition at line 146 of file [Event.hpp](#).

References [getAge\(\)](#).

Here is the call graph for this function:



18.48.3.3 isBroadcast()

`bool utl::Event::isBroadcast () const` [inline]

Check if event is a broadcast.

Returns

True if event is broadcast to all components

Definition at line 158 of file [Event.hpp](#).

References [targetId](#).

18.48.3.4 isTargeted()

`bool utl::Event::isTargeted () const` [inline]

Check if event is targeted to a specific component.

Returns

True if event has a specific target

Definition at line 152 of file [Event.hpp](#).

References [targetId](#).

18.48.4 Member Data Documentation

18.48.4.1 data

`std::vector<std::uint8_t> utl::Event::data`

Serialized event data.

Definition at line 95 of file [Event.hpp](#).

Referenced by [gme::RTypeServer::broadcastWorldState\(\)](#), [srv::AsioServer::handleBroadcastEvent\(\)](#), [eng::AsioClient::handleGameStart\(\)](#), [gme::WaitingRoomScene::handleGameStart\(\)](#), [gme::JoinRoomScene::handleLobbyJoinRequest\(\)](#), [gme::JoinRoomScene::handleLobbyListResponse\(\)](#), [gme::WaitingRoomScene::handleLobbyUpdate\(\)](#), [srv::AsioServer::handleSendEntityEventToClients\(\)](#), [srv::AsioServer::handleSendToClientEvent\(\)](#), [srv::AsioServer::handleStartGameRequest\(\)](#), [gme::GameMulti::handleWorldStateUpdate\(\)](#), [gme::RTypeServer::processPlayerAction\(\)](#) and [gme::RTypeServer::processServerStartEvent\(\)](#).

18.48.4.2 priority

[EventPriority](#) utl::Event::priority = [EventPriority::NORMAL](#)

[Event](#) priority.

Definition at line 93 of file [Event.hpp](#).

18.48.4.3 sourceId

std::uint32_t utl::Event::sourceId = 0

ID of the component that sent the event (0 = system)

Definition at line 91 of file [Event.hpp](#).

Referenced by [utl::EventBus::publish\(\)](#).

18.48.4.4 targetId

std::uint32_t utl::Event::targetId = 0

ID of the target component (0 = broadcast)

Definition at line 92 of file [Event.hpp](#).

Referenced by [isBroadcast\(\)](#), [isTargeted\(\)](#), and [utl::EventBus::updateStats\(\)](#).

18.48.4.5 timestamp

std::chrono::steady_clock::time_point utl::Event::timestamp

Timestamp when event was created.

Definition at line 94 of file [Event.hpp](#).

Referenced by [getAge\(\)](#).

18.48.4.6 type

[EventType](#) utl::Event::type

Type of the event.

Definition at line 90 of file [Event.hpp](#).

Referenced by [utl::EventBus::publish\(\)](#).

The documentation for this class was generated from the following file:

- [/home/masina/Projects/Epitech/rtype/modules/Utils/include/Utils/Event.hpp](#)

18.49 utl::EventBus Class Reference

Thread-safe event bus for decoupled component communication.

#include <EventBus.hpp>

- Consume events of specific type.
- `std::vector< Event > consumeForTarget (std::uint32_t targetId, std::uint32_t maxEvents=100)`
 - Consume events targeted to specific component.
- `void subscribe (std::uint32_t componentId, EventType type)`
 - Subscribe component to specific event types.
- `void unsubscribe (std::uint32_t componentId, EventType type)`
 - Unsubscribe component from specific event type.
- `void registerComponent (std::uint32_t componentId, const std::string &name)`
 - Register component name for better debugging.
- `void unregisterComponent (std::uint32_t componentId)`
 - Unregister component.
- `void filterEventType (EventType type)`
 - Add event type to filter (filtered events will be dropped)
- `void unfilterEventType (EventType type)`
 - Remove event type from filter.
- `void setMaxQueueSize (std::uint64_t maxSize)`
 - Set maximum queue size.
- `std::uint64_t getQueueSize () const`
 - Get current queue size.
- `EventStats getStats () const`
 - Get event bus statistics.
- `void clearStats ()`
 - Clear all statistics.
- `void clear ()`
 - Clear all events from queue.
- `void stop ()`
 - Stop the event bus.
- `bool isRunning () const`
 - Check if event bus is running.

Static Public Member Functions

- `static EventBus & getInstance ()`
 - Get singleton instance of [EventBus](#).

Private Member Functions

- `void updateStats (const Event &event, bool isPublish, bool isExpired=false)`
 - Update statistics for an event.

Private Attributes

- `EventQueue m\_eventQueue`
- `PriorityEventQueue m\_priorityQueue`
- `std::unordered_map< EventType, std::unordered_set< std::uint32_t > > m\_subscribers`
- `std::mutex m\_subscribersMutex`
- `EventStats m\_stats`
- `std::mutex m\_statsMutex`
- `std::atomic< bool > m\_running {true}`
- `std::atomic< std::uint64_t > m\_maxQueueSize {10000}`
- `std::unordered_set< EventType > m\_filteredTypes`
- `std::mutex m\_filterMutex`
- `std::unordered_map< std::uint32_t, std::string > m\_componentNames`
- `std::mutex m\_componentMutex`

18.49.1 Detailed Description

Thread-safe event bus for decoupled component communication.
Definition at line 30 of file [EventBus.hpp](#).

18.49.2 Constructor & Destructor Documentation

18.49.2.1 EventBus() [1/3]

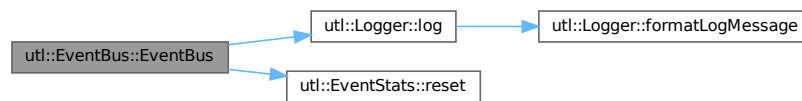
utl::EventBus::EventBus () [inline]

Constructor.

Definition at line 91 of file [EventBus.hpp](#).

References [utl::INFO](#), [utl::Logger::log\(\)](#), [m_stats](#), and [utl::EventStats::reset\(\)](#).

Here is the call graph for this function:



18.49.2.2 ~EventBus()

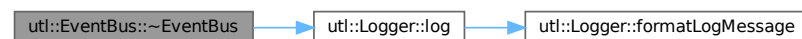
utl::EventBus::~~EventBus () [inline]

Destructor.

Definition at line 100 of file [EventBus.hpp](#).

References [utl::EventBus::EventQueue::cv](#), [utl::INFO](#), [utl::Logger::log\(\)](#), [m_eventQueue](#), and [m_running](#).

Here is the call graph for this function:



18.49.2.3 EventBus() [2/3]

utl::EventBus::EventBus (

const [EventBus](#) &) [delete]

18.49.2.4 EventBus() [3/3]

utl::EventBus::EventBus (

[EventBus](#) &&) [delete]

18.49.3 Member Function Documentation

18.49.3.1 clear()

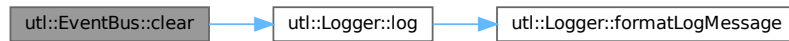
void utl::EventBus::clear () [inline]

Clear all events from queue.

Definition at line 526 of file [EventBus.hpp](#).

References [utl::EventBus::EventQueue::events](#), [utl::INFO](#), [utl::Logger::log\(\)](#), [m_eventQueue](#), and [utl::EventBus::EventQueue::mutex](#).

Here is the call graph for this function:



18.49.3.2 clearStats()

void utl::EventBus::clearStats () [inline]

Clear all statistics.

Definition at line 517 of file [EventBus.hpp](#).

References [m_stats](#), [m_statsMutex](#), and [utl::EventStats::reset\(\)](#).

Here is the call graph for this function:



18.49.3.3 consume()

std::vector< [Event](#) > utl::EventBus::consume (
 std::uint32_t maxEvents = 100) [inline]

Consume events from the bus (non-blocking)

Parameters

maxEvents	Maximum number of events to consume
-----------	-------------------------------------

Returns

Vector of consumed events

Definition at line 271 of file [EventBus.hpp](#).

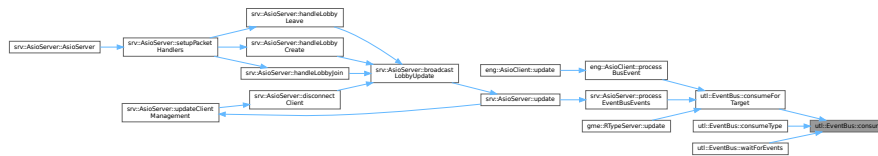
References [utl::EventBus::EventQueue::events](#), [m_eventQueue](#), [utl::EventBus::EventQueue::mutex](#), and [updateStats\(\)](#).

Referenced by [consumeForTarget\(\)](#), [consumeType\(\)](#), and [waitForEvents\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.49.3.4 consumeForTarget()

```
std::vector< Event > utl::EventBus::consumeForTarget (
    std::uint32_t targetId,
    std::uint32_t maxEvents = 100) [inline]
```

Consume events targeted to specific component.

Parameters

targetId	Target component ID
maxEvents	Maximum number of events to consume

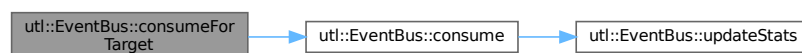
Returns

Vector of events targeted to the component

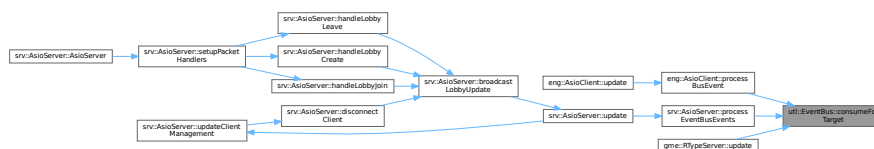
Definition at line 357 of file [EventBus.hpp](#).

References [consume\(\)](#), [utl::EventBus::EventQueue::events](#), [m_eventQueue](#), and [utl::EventBus::EventQueue::mutex](#).
 Referenced by [eng::AsioClient::processBusEvent\(\)](#), [srv::AsioServer::processEventBusEvents\(\)](#), and [gme::RTypeServer::update\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.49.3.5 consumeType()

```
std::vector< Event > utl::EventBus::consumeType (
    EventType type,
    std::uint32_t maxEvents = 100) [inline]
```

Consume events of specific type.

Parameters

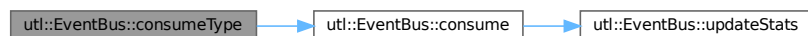
type	Event type to filter
maxEvents	Maximum number of events to consume

Returns

Vector of events of the specified type

Definition at line 329 of file [EventBus.hpp](#).

References [consume\(\)](#), [utl::EventBus::EventQueue::events](#), [m_eventQueue](#), and [utl::EventBus::EventQueue::mutex](#).
Here is the call graph for this function:



18.49.3.6 filterEventType()

void `utl::EventBus::filterEventType` (
 [EventType](#) type) [inline]

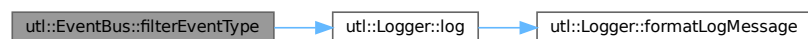
Add event type to filter (filtered events will be dropped)

Parameters

type	Event type to filter
------	--------------------------------------

Definition at line 462 of file [EventBus.hpp](#).

References [utl::INFO](#), [utl::Logger::log\(\)](#), [m_filteredTypes](#), and [m_filterMutex](#).
Here is the call graph for this function:



18.49.3.7 getInstance()

static [EventBus](#) & `utl::EventBus::getInstance` () [inline], [static]

Get singleton instance of [EventBus](#).

Returns

Reference to the singleton [EventBus](#) instance

Definition at line 82 of file [EventBus.hpp](#).

Referenced by [gme::GameMulti::processEventBus\(\)](#), and [gme::GameMulti::setupEventSubscriptions\(\)](#).

Here is the caller graph for this function:

18.49.3.8 `getQueueSize()`

`std::uint64_t utl::EventBus::getQueueSize () const` [inline]

Get current queue size.

Returns

Current number of events in queue

Definition at line 496 of file [EventBus.hpp](#).

References [utl::EventBus::EventQueue::events](#), [m_eventQueue](#), and [utl::EventBus::EventQueue::mutex](#).

Referenced by [getStats\(\)](#).

Here is the caller graph for this function:

18.49.3.9 `getStats()`

[EventStats](#) `utl::EventBus::getStats () const` [inline]

Get event bus statistics.

Returns

Copy of current statistics

Definition at line 506 of file [EventBus.hpp](#).

References [utl::EventStats::currentQueueSize](#), [getQueueSize\(\)](#), [m_stats](#), and [m_statsMutex](#).

Here is the call graph for this function:



18.49.3.10 isRunning()

bool utl::EventBus::isRunning () const [inline]
 Check if event bus is running.

Returns

True if running

Definition at line 550 of file [EventBus.hpp](#).

References [m_running](#).

18.49.3.11 operator=() [1/2]

[EventBus](#) & utl::EventBus::operator= (
 const [EventBus](#) &) [delete]

18.49.3.12 operator=() [2/2]

[EventBus](#) & utl::EventBus::operator= (
 [EventBus](#) &&) [delete]

18.49.3.13 publish() [1/2]

bool utl::EventBus::publish (
 const [Event](#) & event) [inline]
 Publish an event to the bus.

Parameters

event	Event to publish
-------	----------------------------------

Returns

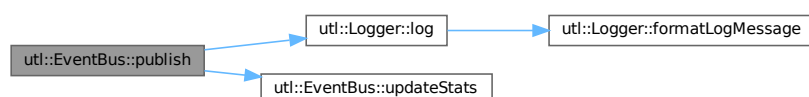
True if event was published successfully

Definition at line 118 of file [EventBus.hpp](#).

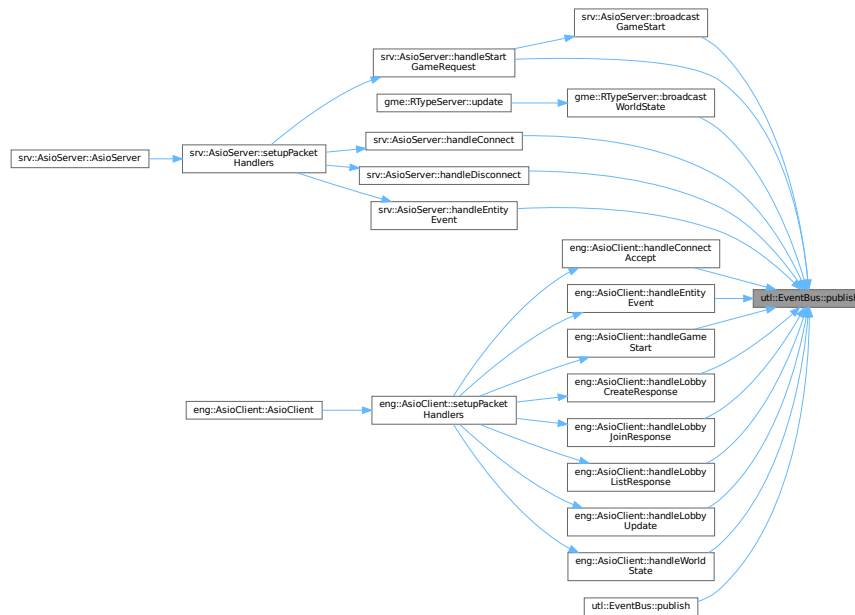
References [utl::EventBus::EventQueue::cv](#), [utl::EventBus::EventQueue::events](#), [utl::Logger::log\(\)](#), [m_eventQueue](#), [m_filteredTypes](#), [m_filterMutex](#), [m_maxQueueSize](#), [m_running](#), [utl::EventBus::EventQueue::mutex](#), [utl::Event::sourceId](#), [utl::EventBus::EventQueue::totalEvents](#), [utl::Event::type](#), [updateStats\(\)](#), and [utl::WARNING](#).

Referenced by [srv::AsioServer::broadcastGameStart\(\)](#), [gme::RTypeServer::broadcastWorldState\(\)](#), [srv::AsioServer::handleConnect\(\)](#), [eng::AsioClient::handleConnectAccept\(\)](#), [srv::AsioServer::handleDisconnect\(\)](#), [eng::AsioClient::handleEntityEvent\(\)](#), [srv::AsioServer::handleEntityEvent\(\)](#), [eng::AsioClient::handleGameStart\(\)](#), [eng::AsioClient::handleLobbyCreateResponse\(\)](#), [eng::AsioClient::handleLobbyJoinResponse\(\)](#), [eng::AsioClient::handleLobbyUpdate\(\)](#), [srv::AsioServer::handleStartGameRequest\(\)](#), [eng::AsioClient::handleWorldState\(\)](#) and [publish\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.49.3.14 publish() [2/2]

```
template<typename T >
bool utl::EventBus::publish (
    EventType type,
    const T & data,
    std::uint32_t sourceId = 0,
    std::uint32_t targetId = 0,
    EventPriority priority = EventPriority::NORMAL) [inline]
```

Publish event with serialized data.

Template Parameters

T	Type of data to serialize
---	---------------------------

Parameters

type	Event type
data	Data to serialize
sourceId	Source component ID
targetId	Target component ID (0 for broadcast)
priority	Event priority

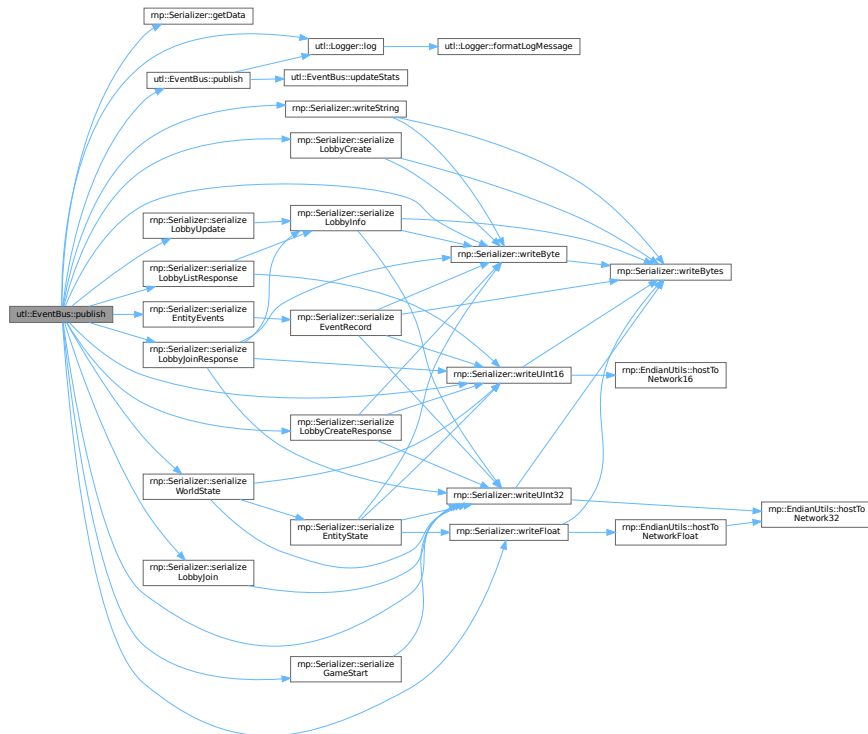
Returns

True if event was published successfully

Definition at line 174 of file [EventBus.hpp](#).

References [rnp::Serializer::getData\(\)](#), [utl::Logger::log\(\)](#), [publish\(\)](#), [rnp::Serializer::serializeEntityEvents\(\)](#), [rnp::Serializer::serializeGameStart\(\)](#), [rnp::Serializer::serializeLobbyCreate\(\)](#), [rnp::Serializer::serializeLobbyCreateResponse\(\)](#)

[rnp::Serializer::serializeLobbyJoin\(\)](#), [rnp::Serializer::serializeLobbyJoinResponse\(\)](#), [rnp::Serializer::serializeLobbyListResp](#)
[rnp::Serializer::serializeLobbyUpdate\(\)](#), [rnp::Serializer::serializeWorldState\(\)](#), [utl::WARNING](#), [rnp::Serializer::writeByte\(\)](#)
[rnp::Serializer::writeFloat\(\)](#), [rnp::Serializer::writeString\(\)](#), [rnp::Serializer::writeUInt16\(\)](#), and [rnp::Serializer::writeUInt32\(\)](#).
 Here is the call graph for this function:



18.49.3.15 registerComponent()

```
void utl::EventBus::registerComponent (
    std::uint32_t componentId,
    const std::string & name) [inline]
```

Register component name for better debugging.

Parameters

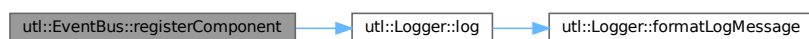
componentId	Component ID
name	Component name

Definition at line 423 of file [EventBus.hpp](#).

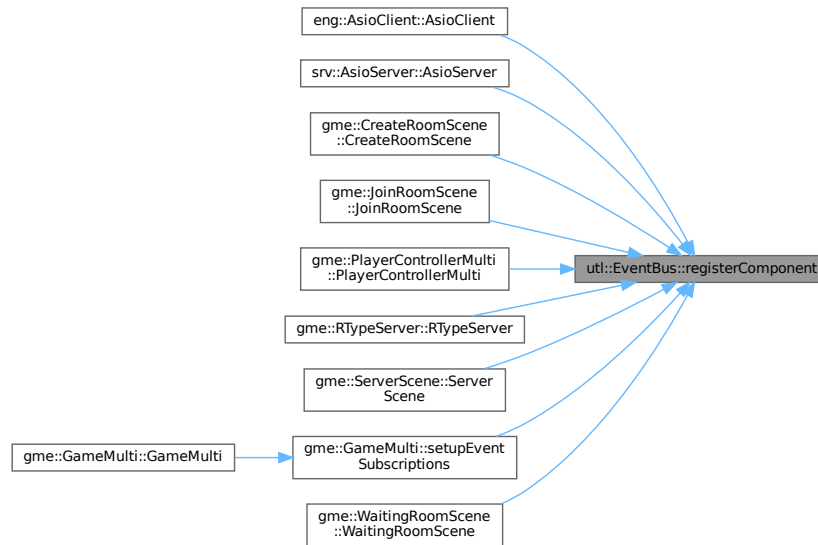
References [utl::INFO](#), [utl::Logger::log\(\)](#), [m_componentMutex](#), and [m_componentNames](#).

Referenced by [eng::AsioClient::AsioClient\(\)](#), [srv::AsioServer::AsioServer\(\)](#), [gme::CreateRoomScene::CreateRoomScene\(\)](#), [gme::JoinRoomScene::JoinRoomScene\(\)](#), [gme::PlayerControllerMulti::PlayerControllerMulti\(\)](#), [gme::RTypeServer::RTypeServer\(\)](#), [gme::ServerScene::ServerScene\(\)](#), [gme::GameMulti::setupEventSubscriptions\(\)](#), and [gme::WaitingRoomScene::WaitingRoomScene\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.49.3.16 setMaxQueueSize()

void utl::EventBus::setMaxQueueSize (
 std::uint64_t maxSize) [inline]
 Set maximum queue size.

Parameters

maxSize	Maximum number of events in queue
---------	-----------------------------------

Definition at line 490 of file [EventBus.hpp](#).

References [m_maxQueueSize](#).

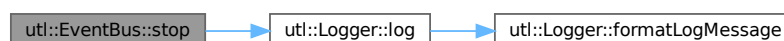
18.49.3.17 stop()

void utl::EventBus::stop () [inline]
 Stop the event bus.

Definition at line 539 of file [EventBus.hpp](#).

References [utl::EventBus::EventQueue::cv](#), [utl::INFO](#), [utl::Logger::log\(\)](#), [m_eventQueue](#), and [m_running](#).

Here is the call graph for this function:



18.49.3.18 subscribe()

void utl::EventBus::subscribe (

```
std::uint32_t componentId,
    EventType type) [inline]
```

Subscribe component to specific event types.

Parameters

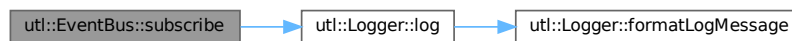
componentId	Component ID
type	Event type to subscribe to

Definition at line 384 of file [EventBus.hpp](#).

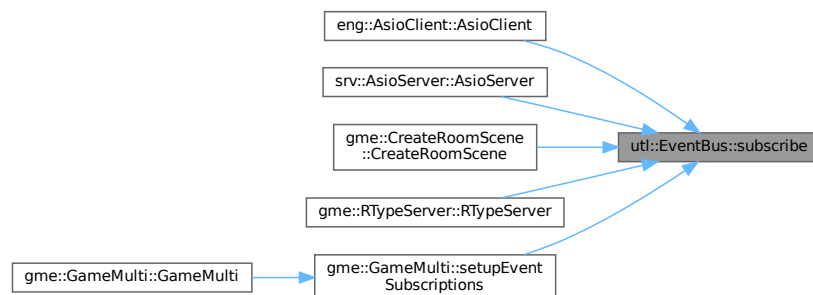
References [utl::Logger::log\(\)](#), [m_subscribers](#), [m_subscribersMutex](#), and [utl::WARNING](#).

Referenced by [eng::AsioClient::AsioClient\(\)](#), [srv::AsioServer::AsioServer\(\)](#), [gme::CreateRoomScene::CreateRoomScene\(\)](#), [gme::RTypeServer::RTypeServer\(\)](#), and [gme::GameMulti::setupEventSubscriptions\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.49.3.19 unfilterEventType()

```
void utl::EventBus::unfilterEventType (
    EventType type) [inline]
```

Remove event type from filter.

Parameters

type	Event type to unfilter
------	------------------------

Definition at line 476 of file [EventBus.hpp](#).

References [utl::INFO](#), [utl::Logger::log\(\)](#), [m_filteredTypes](#), and [m_filterMutex](#).

Here is the call graph for this function:



18.49.3.20 unregisterComponent()

```
void utl::EventBus::unregisterComponent (
    std::uint32_t componentId) [inline]
```

Unregister component.

Parameters

componentId	Component ID
-------------	--------------

Definition at line 436 of file [EventBus.hpp](#).

References [utl::INFO](#), [utl::Logger::log\(\)](#), [m_componentMutex](#), [m_componentNames](#), [m_subscribers](#), and [m_subscribersMutex](#).

Here is the call graph for this function:



18.49.3.21 unsubscribe()

```
void utl::EventBus::unsubscribe (
    std::uint32_t componentId,
    EventType type) [inline]
```

Unsubscribe component from specific event type.

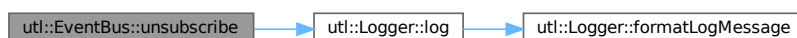
Parameters

componentId	Component ID
type	Event type to unsubscribe from

Definition at line 399 of file [EventBus.hpp](#).

References [utl::Logger::log\(\)](#), [m_subscribers](#), [m_subscribersMutex](#), and [utl::WARNING](#).

Here is the call graph for this function:



18.49.3.22 updateStats()

```
void utl::EventBus::updateStats (
    const Event & event,
    bool isPublish,
    bool isExpired = false) [inline], [private]
```

Update statistics for an event.

Parameters

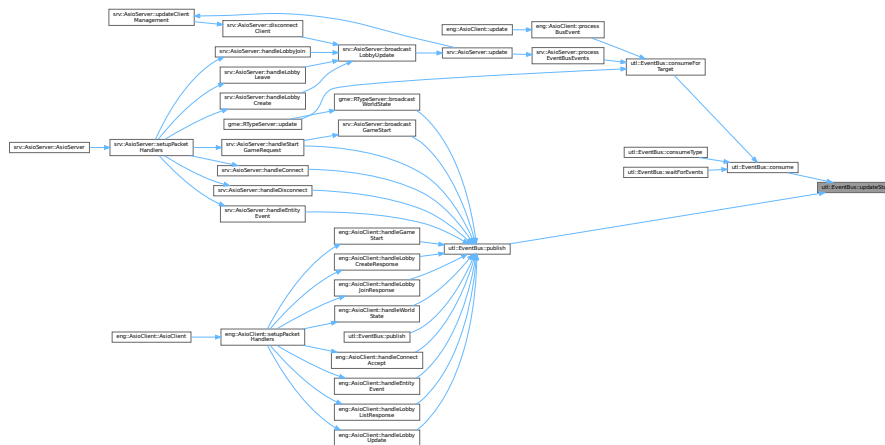
event	Event to update stats for
isPublish	True if this is a publish operation
isExpired	True if event expired

Definition at line 559 of file [EventBus.hpp](#).

References [utl::EventStats::eventTypeCount](#), [m_stats](#), [m_statsMutex](#), [utl::EventStats::priorityCount](#), [utl::EventStats::sourceCount](#), [utl::EventStats::targetCount](#), [utl::Event::targetId](#), [utl::EventStats::totalEventsConsumed](#), [utl::EventStats::totalEventsExpired](#), and [utl::EventStats::totalEventsPublished](#).

Referenced by [consume\(\)](#), and [publish\(\)](#).

Here is the caller graph for this function:



18.49.3.23 waitForEvents()

```
std::vector< Event > utl::EventBus::waitForEvents (
    std::chrono::milliseconds timeout = std::chrono::milliseconds(100),
    std::uint32_t maxEvents = 100) [inline]
```

Consume events with timeout (blocking)

Parameters

timeout	Maximum time to wait for events
maxEvents	Maximum number of events to consume

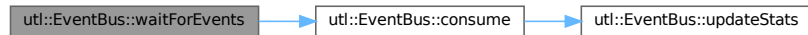
Returns

Vector of consumed events

Definition at line 309 of file [EventBus.hpp](#).

References [consume\(\)](#), [utl::EventBus::EventQueue::cv](#), [m_eventQueue](#), and [utl::EventBus::EventQueue::mutex](#).

Here is the call graph for this function:



18.49.4 Member Data Documentation

18.49.4.1 `m_componentMutex`

`std::mutex utl::EventBus::m_componentMutex` [mutable], [private]

Definition at line 75 of file [EventBus.hpp](#).

Referenced by [registerComponent\(\)](#), and [unregisterComponent\(\)](#).

18.49.4.2 `m_componentNames`

`std::unordered_map<std::uint32_t, std::string> utl::EventBus::m_componentNames` [private]

Definition at line 74 of file [EventBus.hpp](#).

Referenced by [registerComponent\(\)](#), and [unregisterComponent\(\)](#).

18.49.4.3 `m_eventQueue`

[EventQueue](#) `utl::EventBus::m_eventQueue` [private]

Definition at line 60 of file [EventBus.hpp](#).

Referenced by [clear\(\)](#), [consume\(\)](#), [consumeForTarget\(\)](#), [consumeType\(\)](#), [getQueueSize\(\)](#), [publish\(\)](#), [stop\(\)](#), [waitForEvents\(\)](#), and [~EventBus\(\)](#).

18.49.4.4 `m_filteredTypes`

`std::unordered_set<EventType> utl::EventBus::m_filteredTypes` [private]

Definition at line 70 of file [EventBus.hpp](#).

Referenced by [filterEventType\(\)](#), [publish\(\)](#), and [unfilterEventType\(\)](#).

18.49.4.5 `m_filterMutex`

`std::mutex utl::EventBus::m_filterMutex` [mutable], [private]

Definition at line 71 of file [EventBus.hpp](#).

Referenced by [filterEventType\(\)](#), [publish\(\)](#), and [unfilterEventType\(\)](#).

18.49.4.6 `m_maxQueueSize`

`std::atomic<std::uint64_t> utl::EventBus::m_maxQueueSize {10000}` [private]

Definition at line 67 of file [EventBus.hpp](#).

Referenced by [publish\(\)](#), and [setMaxQueueSize\(\)](#).

18.49.4.7 `m_priorityQueue`

[PriorityEventQueue](#) `utl::EventBus::m_priorityQueue` [private]

Definition at line 61 of file [EventBus.hpp](#).

18.49.4.8 `m_running`

`std::atomic<bool> utl::EventBus::m_running {true}` [private]

Definition at line 66 of file [EventBus.hpp](#).

Referenced by [isRunning\(\)](#), [publish\(\)](#), [stop\(\)](#), and [~EventBus\(\)](#).

18.49.4.9 m_stats

[EventStats](#) utl::EventBus::m_stats [private]

Definition at line 64 of file [EventBus.hpp](#).

Referenced by [clearStats\(\)](#), [EventBus\(\)](#), [getStats\(\)](#), and [updateStats\(\)](#).

18.49.4.10 m_statsMutex

std::mutex utl::EventBus::m_statsMutex [mutable], [private]

Definition at line 65 of file [EventBus.hpp](#).

Referenced by [clearStats\(\)](#), [getStats\(\)](#), and [updateStats\(\)](#).

18.49.4.11 m_subscribers

std::unordered_map<[EventType](#), std::unordered_set<std::uint32_t> > utl::EventBus::m_subscribers [private]

Definition at line 62 of file [EventBus.hpp](#).

Referenced by [subscribe\(\)](#), [unregisterComponent\(\)](#), and [unsubscribe\(\)](#).

18.49.4.12 m_subscribersMutex

std::mutex utl::EventBus::m_subscribersMutex [mutable], [private]

Definition at line 63 of file [EventBus.hpp](#).

Referenced by [subscribe\(\)](#), [unregisterComponent\(\)](#), and [unsubscribe\(\)](#).

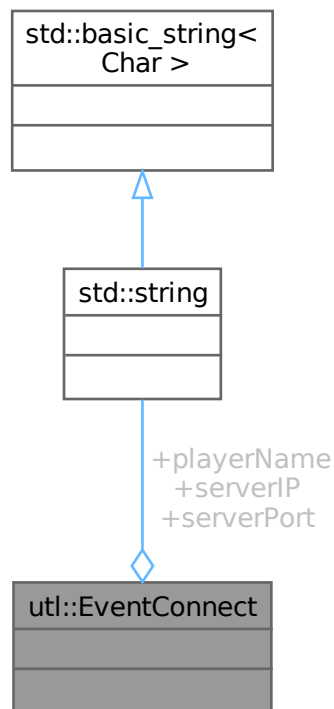
The documentation for this class was generated from the following file:

- [/home/masina/Projects/Epitech/rtype/modules/Utils/include/Utils/EventBus.hpp](#)

18.50 utl::EventConnect Struct Reference

```
#include <Event.hpp>
```

Collaboration diagram for utl::EventConnect:



Public Attributes

- `std::string` [playerName](#)
- `std::string` [serverIP](#)
- `std::string` [serverPort](#)

18.50.1 Detailed Description

Definition at line 77 of file [Event.hpp](#).

18.50.2 Member Data Documentation

18.50.2.1 playerName

`std::string utl::EventConnect::playerName`

Definition at line 79 of file [Event.hpp](#).

18.50.2.2 serverIP

`std::string utl::EventConnect::serverIP`

Definition at line 80 of file [Event.hpp](#).

18.50.2.3 serverPort

`std::string utl::EventConnect::serverPort`

Definition at line 81 of file [Event.hpp](#).

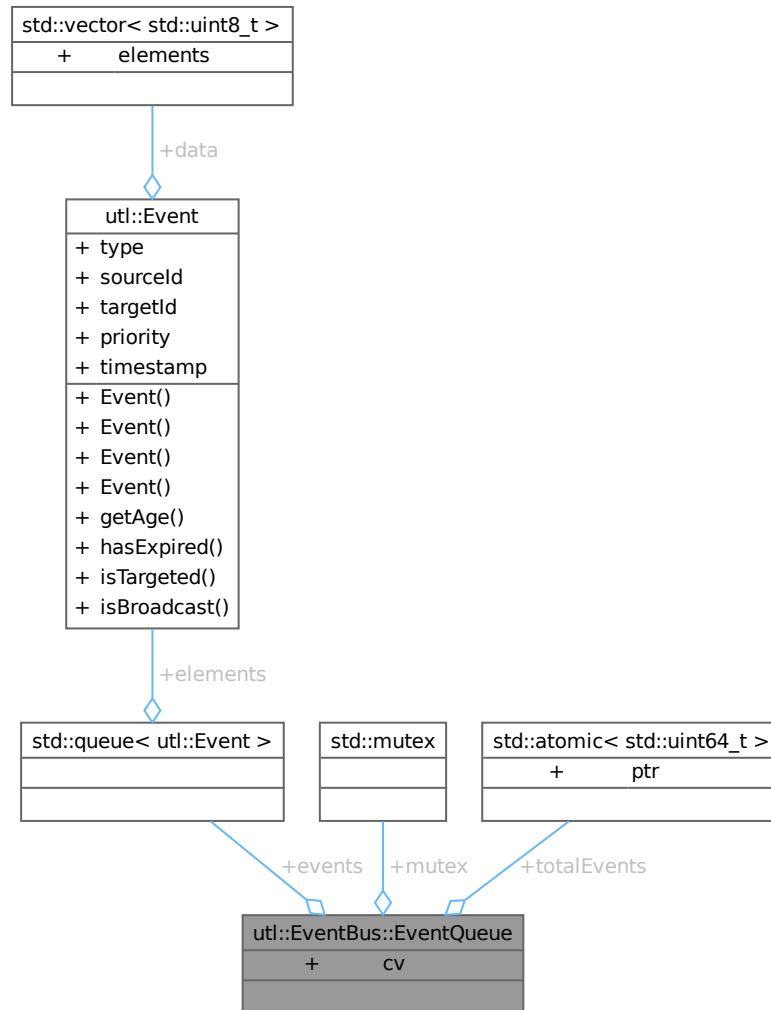
The documentation for this struct was generated from the following file:

- </home/masina/Projects/Epitech/rtype/modules/Utils/include/Utils/Event.hpp>

18.51 utl::EventBus::EventQueue Struct Reference

Internal event queue structure.

Collaboration diagram for utl::EventBus::EventQueue:



Public Attributes

- `std::queue< Event > events`
- `std::mutex mutex`
- `std::condition_variable cv`
- `std::atomic< std::uint64_t > totalEvents {0}`

18.51.1 Detailed Description

Internal event queue structure.

Definition at line 36 of file [EventBus.hpp](#).

18.51.2 Member Data Documentation

18.51.2.1 cv

`std::condition_variable utl::EventBus::EventQueue::cv`

Definition at line 40 of file [EventBus.hpp](#).

Referenced by [utl::EventBus::publish\(\)](#), [utl::EventBus::stop\(\)](#), [utl::EventBus::waitForEvents\(\)](#), and [utl::EventBus::~~EventBus\(\)](#).

18.51.2.2 events

`std::queue<Event> utl::EventBus::EventQueue::events`

Definition at line 38 of file [EventBus.hpp](#).

Referenced by [utl::EventBus::clear\(\)](#), [utl::EventBus::consume\(\)](#), [utl::EventBus::consumeForTarget\(\)](#), [utl::EventBus::consumeType\(\)](#), [utl::EventBus::getQueueSize\(\)](#), and [utl::EventBus::publish\(\)](#).

18.51.2.3 mutex

`std::mutex utl::EventBus::EventQueue::mutex` [mutable]

Definition at line 39 of file [EventBus.hpp](#).

Referenced by [utl::EventBus::clear\(\)](#), [utl::EventBus::consume\(\)](#), [utl::EventBus::consumeForTarget\(\)](#), [utl::EventBus::consumeType\(\)](#), [utl::EventBus::getQueueSize\(\)](#), [utl::EventBus::publish\(\)](#), and [utl::EventBus::waitForEvent](#).

18.51.2.4 totalEvents

`std::atomic<std::uint64_t> utl::EventBus::EventQueue::totalEvents {0}`

Definition at line 41 of file [EventBus.hpp](#).

Referenced by [utl::EventBus::publish\(\)](#).

The documentation for this struct was generated from the following file:

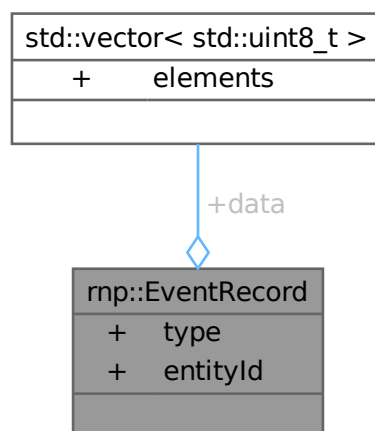
- [/home/masina/Projects/Epitech/rtype/modules/Utils/include/Utils/EventBus.hpp](#)

18.52 rnp::EventRecord Struct Reference

Event record for ENTITY_EVENT packets (TLV format)

`#include <Protocol.hpp>`

Collaboration diagram for `rnp::EventRecord`:



Public Attributes

- [EventType](#) type
- std::uint32_t [entityId](#)
- std::vector< std::uint8_t > [data](#)

18.52.1 Detailed Description

Event record for ENTITY_EVENT packets (TLV format)

Definition at line 115 of file [Protocol.hpp](#).

18.52.2 Member Data Documentation

18.52.2.1 data

std::vector<std::uint8_t> rnp::EventRecord::data

Definition at line 119 of file [Protocol.hpp](#).

Referenced by [rnp::Serializer::deserializeEventRecord\(\)](#), [rnp::HandlerPacket::parseEntityEvents\(\)](#), [gme::PlayerControllerMulti::sendInputToServer\(\)](#), and [rnp::Serializer::serializeEventRecord\(\)](#).

18.52.2.2 entityId

std::uint32_t rnp::EventRecord::entityId

Definition at line 118 of file [Protocol.hpp](#).

Referenced by [gme::PlayerControllerMulti::sendInputToServer\(\)](#), and [rnp::Serializer::serializeEventRecord\(\)](#).

18.52.2.3 type

[EventType](#) rnp::EventRecord::type

Definition at line 117 of file [Protocol.hpp](#).

Referenced by [rnp::Serializer::createDamageEvent\(\)](#), [rnp::Serializer::createDespawnEvent\(\)](#), [rnp::Serializer::createInputEvent\(\)](#), [rnp::Serializer::createScoreEvent\(\)](#), [rnp::Serializer::createSpawnEvent\(\)](#), [rnp::Serializer::deserializeEventRecord\(\)](#), [rnp::HandlerPacket::parseEntityEvents\(\)](#), [gme::PlayerControllerMulti::sendInputToServer\(\)](#), and [rnp::Serializer::serializeEventRecord\(\)](#).

The documentation for this struct was generated from the following file:

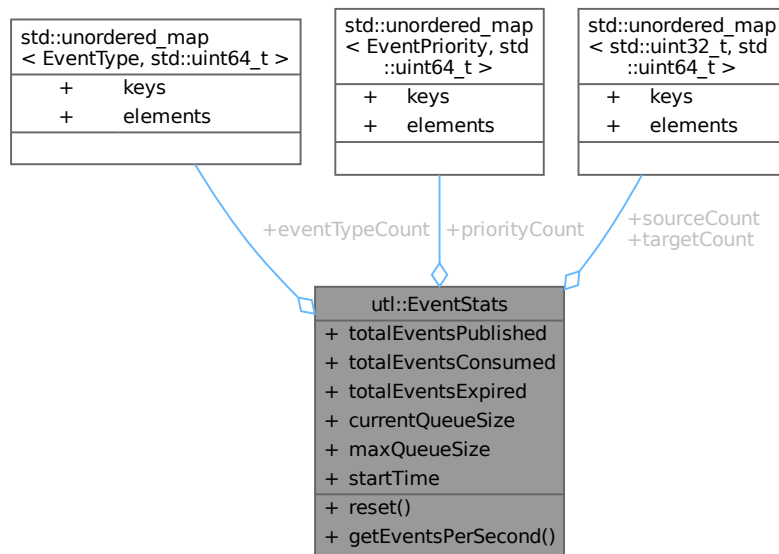
- [/home/masina/Projects/Epitech/rtype/modules/Interfaces/include/Interfaces/Protocol/Protocol.hpp](#)

18.53 utl::EventStats Class Reference

[Event](#) statistics structure.

#include <Event.hpp>

Collaboration diagram for `utl::EventStats`:



Public Member Functions

- void `reset()`
Reset all statistics.
- double `getEventsPerSecond()` const
Get events per second rate.

Public Attributes

- `std::uint64_t totalEventsPublished = 0`
Total events published.
- `std::uint64_t totalEventsConsumed = 0`
Total events consumed.
- `std::uint64_t totalEventsExpired = 0`
Total events that expired.
- `std::uint64_t currentQueueSize = 0`
Current queue size.
- `std::uint64_t maxQueueSize = 0`
Maximum queue size reached.
- `std::chrono::steady_clock::time_point startTime`
Statistics start time.
- `std::unordered_map< EventType, std::uint64_t > eventTypeCount`
Count per event type.
- `std::unordered_map< EventPriority, std::uint64_t > priorityCount`
Count per priority.
- `std::unordered_map< std::uint32_t, std::uint64_t > sourceCount`
Count per source component.
- `std::unordered_map< std::uint32_t, std::uint64_t > targetCount`
Count per target component.

18.53.1 Detailed Description

[Event](#) statistics structure.

Definition at line 164 of file [Event.hpp](#).

18.53.2 Member Function Documentation

18.53.2.1 `getEventsPerSecond()`

`double utl::EventStats::getEventsPerSecond () const` [inline]

Get events per second rate.

Returns

Events per second

Definition at line 199 of file [Event.hpp](#).

References [startTime](#), and [totalEventsPublished](#).

18.53.2.2 `reset()`

`void utl::EventStats::reset ()` [inline]

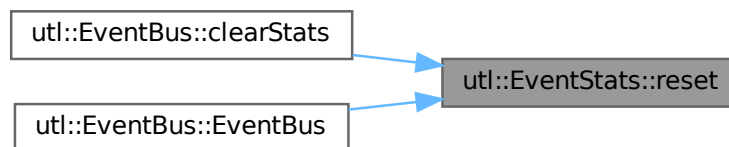
Reset all statistics.

Definition at line 181 of file [Event.hpp](#).

References [currentQueueSize](#), [eventTypeCount](#), [maxQueueSize](#), [priorityCount](#), [sourceCount](#), [startTime](#), [targetCount](#), [totalEventsConsumed](#), [totalEventsExpired](#), and [totalEventsPublished](#).

Referenced by [utl::EventBus::clearStats\(\)](#), and [utl::EventBus::EventBus\(\)](#).

Here is the caller graph for this function:



18.53.3 Member Data Documentation

18.53.3.1 `currentQueueSize`

`std::uint64_t utl::EventStats::currentQueueSize = 0`

Current queue size.

Definition at line 170 of file [Event.hpp](#).

Referenced by [utl::EventBus::getStats\(\)](#), and [reset\(\)](#).

18.53.3.2 `eventTypeCount`

`std::unordered_map<Event Type, std::uint64_t> utl::EventStats::eventTypeCount`

Count per event type.

Definition at line 173 of file [Event.hpp](#).

Referenced by [reset\(\)](#), and [utl::EventBus::updateStats\(\)](#).

18.53.3.3 `maxQueueSize`

`std::uint64_t utl::EventStats::maxQueueSize = 0`

Maximum queue size reached.

Definition at line 171 of file [Event.hpp](#).
 Referenced by [reset\(\)](#).

18.53.3.4 priorityCount

`std::unordered_map<EventPriority, std::uint64_t> utl::EventStats::priorityCount`
 Count per priority.
 Definition at line 174 of file [Event.hpp](#).
 Referenced by [reset\(\)](#), and [utl::EventBus::updateStats\(\)](#).

18.53.3.5 sourceCount

`std::unordered_map<std::uint32_t, std::uint64_t> utl::EventStats::sourceCount`
 Count per source component.
 Definition at line 175 of file [Event.hpp](#).
 Referenced by [reset\(\)](#), and [utl::EventBus::updateStats\(\)](#).

18.53.3.6 startTime

`std::chrono::steady_clock::time_point utl::EventStats::startTime`
 Statistics start time.
 Definition at line 172 of file [Event.hpp](#).
 Referenced by [getEventsPerSecond\(\)](#), and [reset\(\)](#).

18.53.3.7 targetCount

`std::unordered_map<std::uint32_t, std::uint64_t> utl::EventStats::targetCount`
 Count per target component.
 Definition at line 176 of file [Event.hpp](#).
 Referenced by [reset\(\)](#), and [utl::EventBus::updateStats\(\)](#).

18.53.3.8 totalEventsConsumed

`std::uint64_t utl::EventStats::totalEventsConsumed = 0`
 Total events consumed.
 Definition at line 168 of file [Event.hpp](#).
 Referenced by [reset\(\)](#), and [utl::EventBus::updateStats\(\)](#).

18.53.3.9 totalEventsExpired

`std::uint64_t utl::EventStats::totalEventsExpired = 0`
 Total events that expired.
 Definition at line 169 of file [Event.hpp](#).
 Referenced by [reset\(\)](#), and [utl::EventBus::updateStats\(\)](#).

18.53.3.10 totalEventsPublished

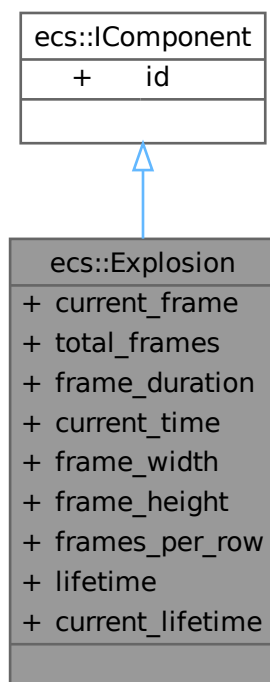
`std::uint64_t utl::EventStats::totalEventsPublished = 0`
 Total events published.
 Definition at line 167 of file [Event.hpp](#).
 Referenced by [getEventsPerSecond\(\)](#), [reset\(\)](#), and [utl::EventBus::updateStats\(\)](#).
 The documentation for this class was generated from the following file:

- [/home/masina/Projects/Epitech/rtype/modules/Utils/include/Utils/Event.hpp](#)

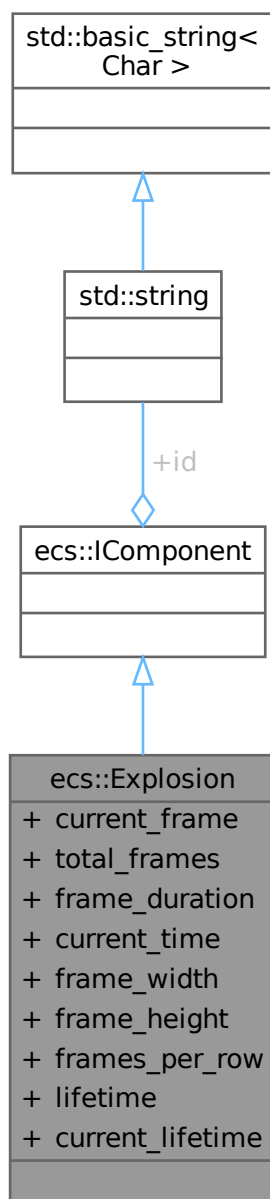
18.54 ecs::Explosion Struct Reference

`#include <Component.hpp>`

Inheritance diagram for ecs::Explosion:



Collaboration diagram for ecs::Explosion:



Public Attributes

- int `current_frame`
- int `total_frames`
- float `frame_duration`
- float `current_time`
- float `frame_width`
- float `frame_height`
- int `frames_per_row`
- float `lifetime`
- float `current_lifetime`

Public Attributes inherited from [ecs::IComponent](#)

- `std::string` [id](#)

18.54.1 Detailed Description

Definition at line 123 of file [Component.hpp](#).

18.54.2 Member Data Documentation

18.54.2.1 `current_frame`

`int ecs::Explosion::current_frame`

Definition at line 125 of file [Component.hpp](#).

18.54.2.2 `current_lifetime`

`float ecs::Explosion::current_lifetime`

Definition at line 133 of file [Component.hpp](#).

18.54.2.3 `current_time`

`float ecs::Explosion::current_time`

Definition at line 128 of file [Component.hpp](#).

18.54.2.4 `frame_duration`

`float ecs::Explosion::frame_duration`

Definition at line 127 of file [Component.hpp](#).

18.54.2.5 `frame_height`

`float ecs::Explosion::frame_height`

Definition at line 130 of file [Component.hpp](#).

18.54.2.6 `frame_width`

`float ecs::Explosion::frame_width`

Definition at line 129 of file [Component.hpp](#).

18.54.2.7 `frames_per_row`

`int ecs::Explosion::frames_per_row`

Definition at line 131 of file [Component.hpp](#).

18.54.2.8 `lifetime`

`float ecs::Explosion::lifetime`

Definition at line 132 of file [Component.hpp](#).

18.54.2.9 `total_frames`

`int ecs::Explosion::total_frames`

Definition at line 126 of file [Component.hpp](#).

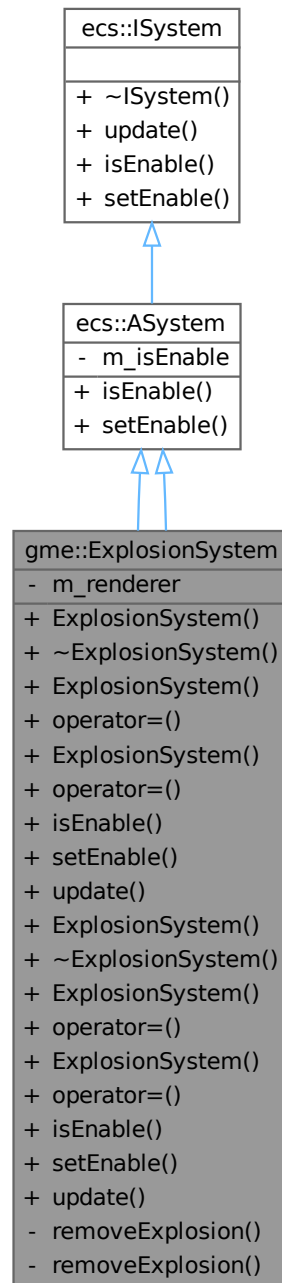
The documentation for this struct was generated from the following file:

- [/home/masina/Projects/Epitech/rtype/modules/ECS/include/ECS/Component.hpp](#)

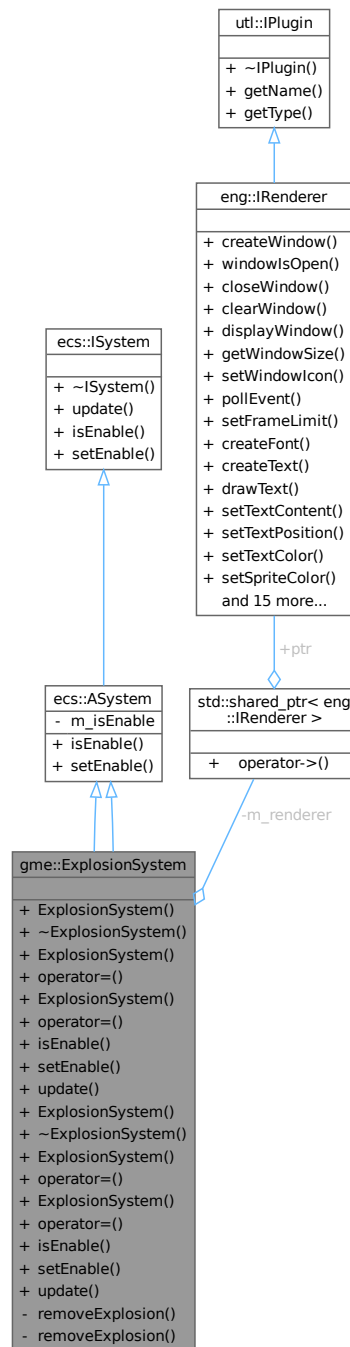
18.55 gme::ExplosionSystem Class Reference

#include <Explosion.hpp>

Inheritance diagram for gme::ExplosionSystem:



Collaboration diagram for gme::ExplosionSystem:



Public Member Functions

- [ExplosionSystem](#) (const std::shared_ptr< [eng::IRenderer](#) > &renderer)
- [~ExplosionSystem](#) () override=default
- [ExplosionSystem](#) (const [ExplosionSystem](#) &)=delete
- [ExplosionSystem](#) & operator= (const [ExplosionSystem](#) &)=delete
- [ExplosionSystem](#) ([ExplosionSystem](#) &&)=delete
- [ExplosionSystem](#) & operator= ([ExplosionSystem](#) &&)=delete

- bool `isEnabled` () override
- void `setEnabled` (bool enable) override
- void `update` (ecs::Registry ®istry, float dt) override
- ExplosionSystem (const std::shared_ptr< eng::IRenderer > &renderer)
- ~ExplosionSystem () override=default
- ExplosionSystem (const ExplosionSystem &)=delete
- ExplosionSystem & operator= (const ExplosionSystem &)=delete
- ExplosionSystem (ExplosionSystem &&)=delete
- ExplosionSystem & operator= (ExplosionSystem &&)=delete
- bool `isEnabled` () override
- void `setEnabled` (bool enable) override
- void `update` (ecs::Registry ®istry, float dt) override

Public Member Functions inherited from `ecs::ISystem`

- virtual `~ISystem` ()=default

Static Private Member Functions

- static void `removeExplosion` (ecs::Registry ®istry, const ecs::Entity entity)
- static void `removeExplosion` (ecs::Registry ®istry, const ecs::Entity entity)

Private Attributes

- const std::shared_ptr< eng::IRenderer > & `m_renderer`

18.55.1 Detailed Description

Definition at line 19 of file `Explosion.hpp`.

18.55.2 Constructor & Destructor Documentation

18.55.2.1 ExplosionSystem() [1/6]

`gme::ExplosionSystem::ExplosionSystem (`
 const std::shared_ptr< eng::IRenderer > & renderer) [inline], [explicit]

Definition at line 22 of file `Explosion.hpp`.

18.55.2.2 ~ExplosionSystem() [1/2]

`gme::ExplosionSystem::~~ExplosionSystem ()` [override], [default]

18.55.2.3 ExplosionSystem() [2/6]

`gme::ExplosionSystem::ExplosionSystem (`
 const ExplosionSystem &) [delete]

18.55.2.4 ExplosionSystem() [3/6]

`gme::ExplosionSystem::ExplosionSystem (`
 ExplosionSystem &&) [delete]

18.55.2.5 ExplosionSystem() [4/6]

`gme::ExplosionSystem::ExplosionSystem (`
 const std::shared_ptr< eng::IRenderer > & renderer) [inline], [explicit]

Definition at line 22 of file `Explosion.hpp`.

18.55.2.6 ~ExplosionSystem() [2/2]

`gme::ExplosionSystem::~~ExplosionSystem ()` [override], [default]

18.55.2.7 ExplosionSystem() [5/6]

`gme::ExplosionSystem::ExplosionSystem (`
 `const ExplosionSystem & )` [delete]

18.55.2.8 ExplosionSystem() [6/6]

`gme::ExplosionSystem::ExplosionSystem (`
 `ExplosionSystem && )` [delete]

18.55.3 Member Function Documentation

18.55.3.1 isEnabled() [1/2]

`bool gme::ExplosionSystem::isEnabled ()` [inline], [override], [virtual]

Reimplemented from [ecs::ASystem](#).

Definition at line 30 of file [Explosion.hpp](#).

18.55.3.2 isEnabled() [2/2]

`bool gme::ExplosionSystem::isEnabled ()` [inline], [override], [virtual]

Reimplemented from [ecs::ASystem](#).

Definition at line 30 of file [Explosion.hpp](#).

18.55.3.3 operator=() [1/4]

`ExplosionSystem & gme::ExplosionSystem::operator= (`
 `const ExplosionSystem & )` [delete]

18.55.3.4 operator=() [2/4]

`ExplosionSystem & gme::ExplosionSystem::operator= (`
 `const ExplosionSystem & )` [delete]

18.55.3.5 operator=() [3/4]

`ExplosionSystem & gme::ExplosionSystem::operator= (`
 `ExplosionSystem && )` [delete]

18.55.3.6 operator=() [4/4]

`ExplosionSystem & gme::ExplosionSystem::operator= (`
 `ExplosionSystem && )` [delete]

18.55.3.7 removeExplosion() [1/2]

`static void gme::ExplosionSystem::removeExplosion (`
 `ecs::Registry & registry,`
 `const ecs::Entity entity)` [inline], [static], [private]

Definition at line 85 of file [Explosion.hpp](#).

References [ecs::Registry::hasComponent\(\)](#), and [ecs::Registry::removeComponent\(\)](#).

Referenced by [update\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.55.3.8 `removeExplosion()` [2/2]

```
static void gme::ExplosionSystem::removeExplosion (
    ecs::Registry & registry,
    const ecs::Entity entity) [inline], [static], [private]
```

Definition at line 85 of file [Explosion.hpp](#).

References [ecs::Registry::hasComponent\(\)](#), and [ecs::Registry::removeComponent\(\)](#).

Here is the call graph for this function:



18.55.3.9 `setEnabled()` [1/2]

```
void gme::ExplosionSystem::setEnabled (
    bool enable) [inline], [override], [virtual]
```

Reimplemented from [ecs::ASystem](#).

Definition at line 31 of file [Explosion.hpp](#).

18.55.3.10 `setEnabled()` [2/2]

```
void gme::ExplosionSystem::setEnabled (
    bool enable) [inline], [override], [virtual]
```

Reimplemented from [ecs::ASystem](#).

Definition at line 31 of file [Explosion.hpp](#).

18.55.3.11 `update()` [1/2]

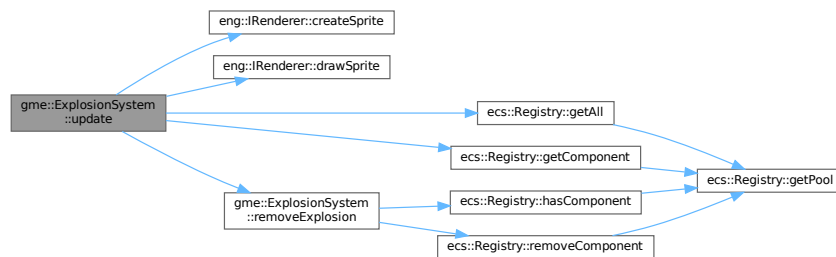
```
void gme::ExplosionSystem::update (
    ecs::Registry & registry,
    float dt) [inline], [override], [virtual]
```

Implements [ecs::ISystem](#).

Definition at line 33 of file [Explosion.hpp](#).

References [eng::IRenderer::createSprite\(\)](#), [eng::IRenderer::drawSprite\(\)](#), [ecs::Registry::getAll\(\)](#), [ecs::Registry::getComponent\(\)](#), [ecs::IComponent::id](#), [m_renderer](#), and [removeExplosion\(\)](#).

Here is the call graph for this function:



18.55.3.12 update() [2/2]

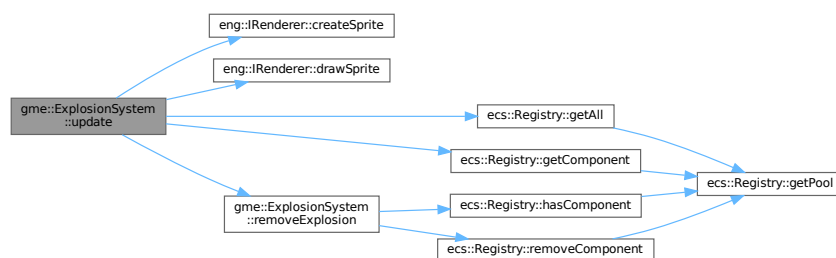
```
void gme::ExplosionSystem::update (
    ecs::Registry & registry,
    float dt) [inline], [override], [virtual]
```

Implements [ecs::ISystem](#).

Definition at line 33 of file [Explosion.hpp](#).

References [eng::IRenderer::createSprite\(\)](#), [eng::IRenderer::drawSprite\(\)](#), [ecs::Registry::getAll\(\)](#), [ecs::Registry::getComponent\(\)](#), [ecs::Registry::getPool\(\)](#), [ecs::IComponent::id](#), [m_renderer](#), and [removeExplosion\(\)](#).

Here is the call graph for this function:



18.55.4 Member Data Documentation

18.55.4.1 m_renderer

```
const std::shared_ptr< eng::IRenderer > & gme::ExplosionSystem::m_renderer [private]
```

Definition at line 83 of file [Explosion.hpp](#).

Referenced by [update\(\)](#).

The documentation for this class was generated from the following files:

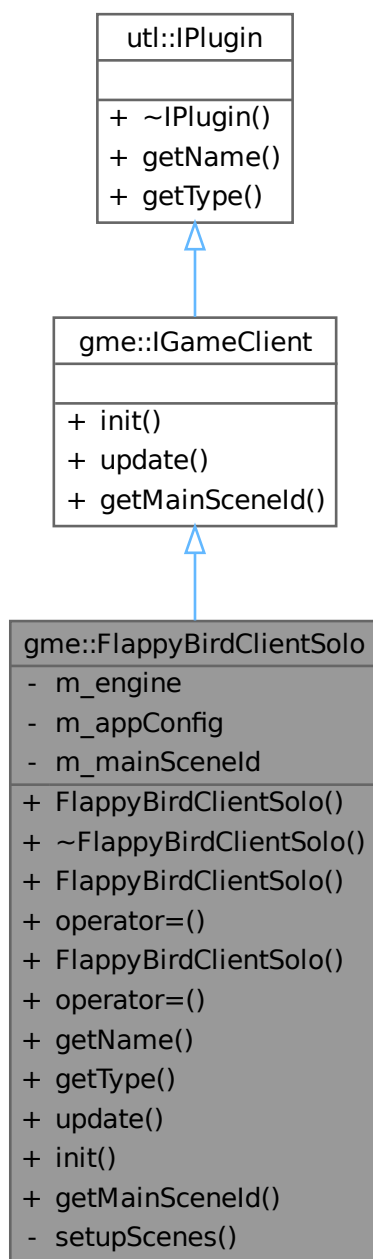
- /home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/include/RTypeClientMulti/Systems/[Explosion.hpp](#)
- /home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Solo/include/RTypeClientSolo/Systems/[Explosion.hpp](#)

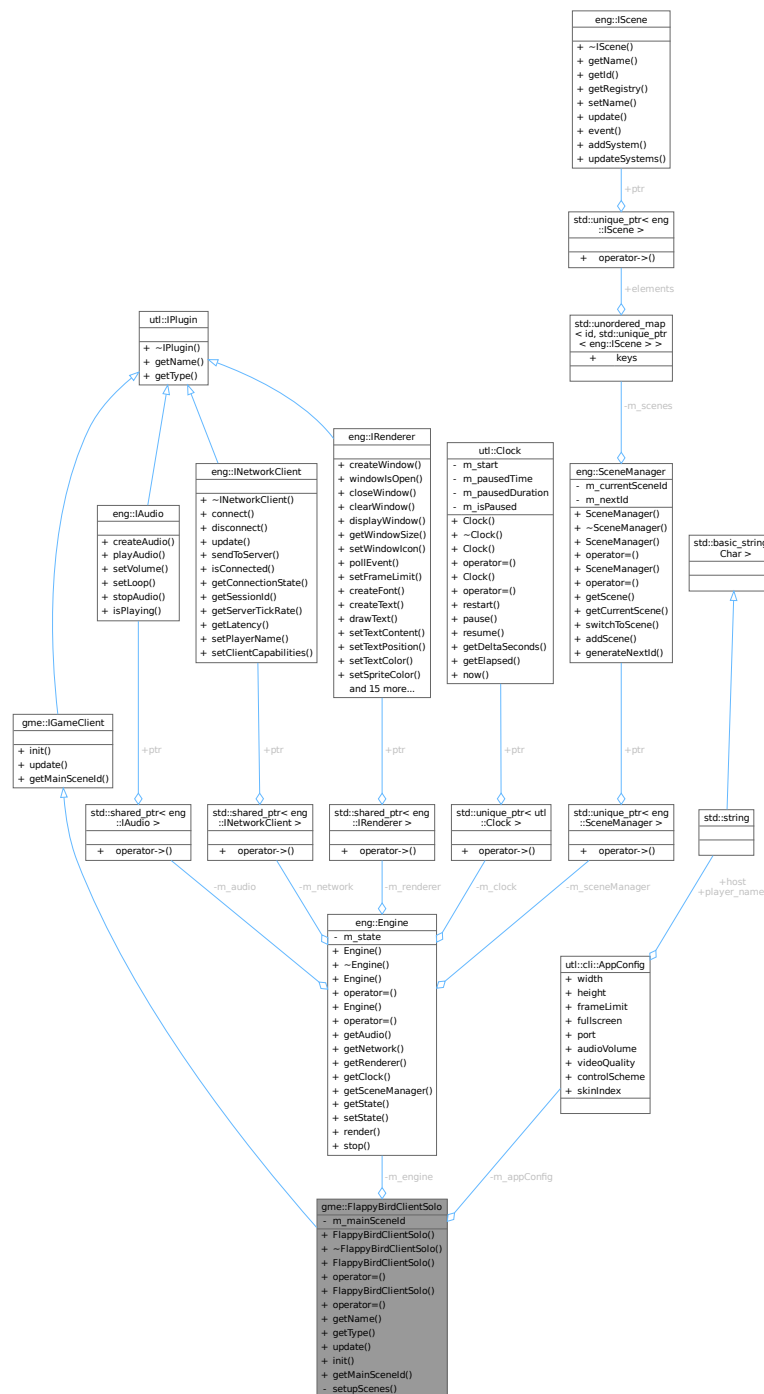
18.56 gme::FlappyBirdClientSolo Class Reference

Class for the [FlappyBirdClientSolo](#) plugin.

```
#include <FlappyBirdClientSolo.hpp>
```

Inheritance diagram for gme::FlappyBirdClientSolo:





- `FlappyBirdClientSolo ()=default`
- `~FlappyBirdClientSolo () override=default`
- `FlappyBirdClientSolo (const FlappyBirdClientSolo &)=delete`
- `FlappyBirdClientSolo & operator= (const FlappyBirdClientSolo &)=delete`
- `FlappyBirdClientSolo (FlappyBirdClientSolo &&)=delete`
- `FlappyBirdClientSolo & operator= (FlappyBirdClientSolo &&)=delete`

- `const std::string getName () const` override
- `utl::PluginType getType () const` override
- `void update (float deltaTime, unsigned int width, unsigned int height)` override
- `void init (eng::Engine &engine, utl::cli::AppConfig &appConfig, bool &showDebug, const eng::id menuSceneId)` override
- `unsigned int getMainSceneId () const` override

Public Member Functions inherited from `utl::IPlugin`

- `virtual ~IPlugin ()=default`

Private Member Functions

- `void setupScenes (bool &showDebug, eng::id menuSceneId)`

Private Attributes

- `eng::Engine * m_engine = nullptr`
- `utl::cli::AppConfig * m_appConfig = nullptr`
- `eng::id m_mainSceneId`

18.56.1 Detailed Description

Class for the `FlappyBirdClientSolo` plugin.

Definition at line 19 of file `FlappyBirdClientSolo.hpp`.

18.56.2 Constructor & Destructor Documentation

18.56.2.1 `FlappyBirdClientSolo()` [1/3]

`gme::FlappyBirdClientSolo::FlappyBirdClientSolo ()` [default]

18.56.2.2 `~FlappyBirdClientSolo()`

`gme::FlappyBirdClientSolo::~~FlappyBirdClientSolo ()` [override], [default]

18.56.2.3 `FlappyBirdClientSolo()` [2/3]

`gme::FlappyBirdClientSolo::FlappyBirdClientSolo (const FlappyBirdClientSolo & )` [delete]

18.56.2.4 `FlappyBirdClientSolo()` [3/3]

`gme::FlappyBirdClientSolo::FlappyBirdClientSolo (FlappyBirdClientSolo && )` [delete]

18.56.3 Member Function Documentation

18.56.3.1 `getMainSceneId()`

`unsigned int gme::FlappyBirdClientSolo::getMainSceneId () const` [inline], [nodiscard], [override], [virtual]

Implements `gme::IGameClient`.

Definition at line 43 of file `FlappyBirdClientSolo.hpp`.

References `m_mainSceneId`.

18.56.3.2 `getName()`

`const std::string gme::FlappyBirdClientSolo::getName () const` [inline], [nodiscard], [override], [virtual]

Implements `utl::IPlugin`.

Definition at line 31 of file `FlappyBirdClientSolo.hpp`.

18.56.3.3 getType()

[utl::PluginType](#) gme::FlappyBirdClientSolo::getType () const [inline], [nodiscard], [override], [virtual]

Implements [utl::IPlugin](#).

Definition at line 32 of file [FlappyBirdClientSolo.hpp](#).

References [utl::GAME_CLIENT](#).

18.56.3.4 init()

```
void gme::FlappyBirdClientSolo::init (
    eng::Engine & engine,
    utl::cli::AppConfig & appConfig,
    bool & showDebug,
    const eng::id menuSceneId) [inline], [override], [virtual]
```

Implements [gme::IGameClient](#).

Definition at line 35 of file [FlappyBirdClientSolo.hpp](#).

References [m_appConfig](#), [m_engine](#), and [setupScenes\(\)](#).

Here is the call graph for this function:



18.56.3.5 operator=() [1/2]

[FlappyBirdClientSolo](#) & gme::FlappyBirdClientSolo::operator= (const [FlappyBirdClientSolo](#) &) [delete]

18.56.3.6 operator=() [2/2]

[FlappyBirdClientSolo](#) & gme::FlappyBirdClientSolo::operator= ([FlappyBirdClientSolo](#) &&) [delete]

18.56.3.7 setupScenes()

```
void gme::FlappyBirdClientSolo::setupScenes (
    bool & showDebug,
    eng::id menuSceneId) [private]
```

Definition at line 7 of file [flappyBirdClientSolo.cpp](#).

Referenced by [init\(\)](#).

Here is the caller graph for this function:



18.56.3.8 update()

```
void gme::FlappyBirdClientSolo::update (
    float deltaTime,
    unsigned int width,
    unsigned int height) [override], [virtual]
```

Implements [gme::IGameClient](#).

Definition at line 5 of file [flappyBirdClientSolo.cpp](#).

18.56.4 Member Data Documentation

18.56.4.1 m_appConfig

```
utl::cli::AppConfig* gme::FlappyBirdClientSolo::m_appConfig = nullptr [private]
```

Definition at line 47 of file [FlappyBirdClientSolo.hpp](#).

Referenced by [init\(\)](#).

18.56.4.2 m_engine

```
eng::Engine* gme::FlappyBirdClientSolo::m_engine = nullptr [private]
```

Definition at line 46 of file [FlappyBirdClientSolo.hpp](#).

Referenced by [init\(\)](#).

18.56.4.3 m_mainSceneId

```
eng::id gme::FlappyBirdClientSolo::m_mainSceneId [private]
```

Definition at line 48 of file [FlappyBirdClientSolo.hpp](#).

Referenced by [getMainSceneId\(\)](#).

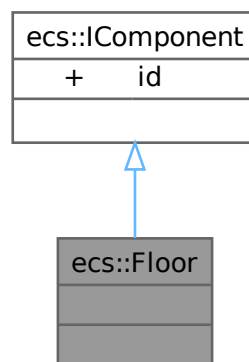
The documentation for this class was generated from the following files:

- [/home/masina/Projects/Epitech/rtype/plugins/Game/FlappyBird/Client/Solo/include/FlappyBirdClientSolo/FlappyBirdClientSolo.hpp](#)
- [/home/masina/Projects/Epitech/rtype/plugins/Game/FlappyBird/Client/Solo/src/flappyBirdClientSolo.cpp](#)

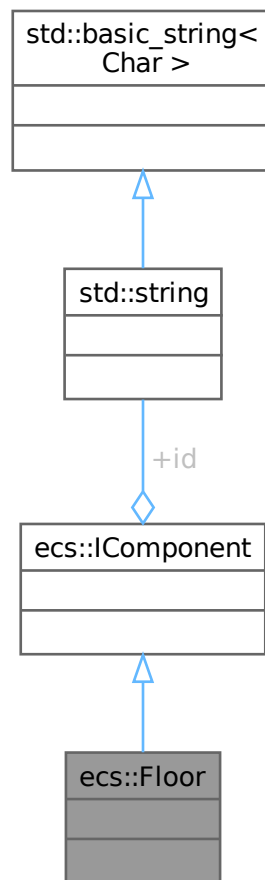
18.57 ecs::Floor Struct Reference

```
#include <Component.hpp>
```

Inheritance diagram for `ecs::Floor`:



Collaboration diagram for `ecs::Floor`:



Additional Inherited Members

Public Attributes inherited from [ecs::IComponent](#)

- `std::string id`

18.57.1 Detailed Description

Definition at line 152 of file [Component.hpp](#).

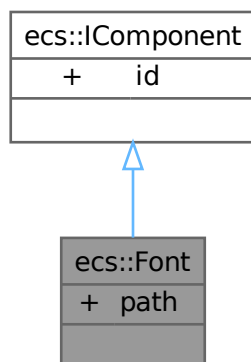
The documentation for this struct was generated from the following file:

- `/home/masina/Projects/Epitech/rtype/modules/ECS/include/ECS/Component.hpp`

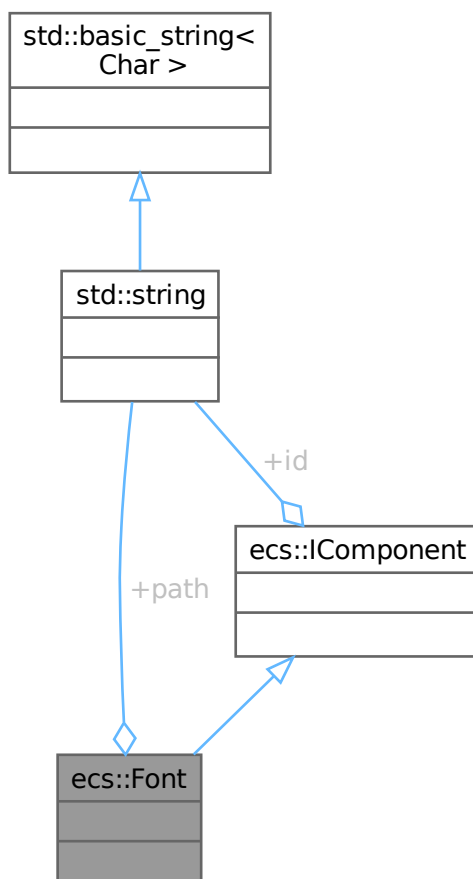
18.58 ecs::Font Struct Reference

`#include <Component.hpp>`

Inheritance diagram for ecs::Font:



Collaboration diagram for ecs::Font:



Public Attributes

- std::string [path](#)

Public Attributes inherited from [ecs::IComponent](#)

- std::string [id](#)

18.58.1 Detailed Description

Definition at line 31 of file [Component.hpp](#).

18.58.2 Member Data Documentation

18.58.2.1 path

std::string [ecs::Font::path](#)

Definition at line 33 of file [Component.hpp](#).

The documentation for this struct was generated from the following file:

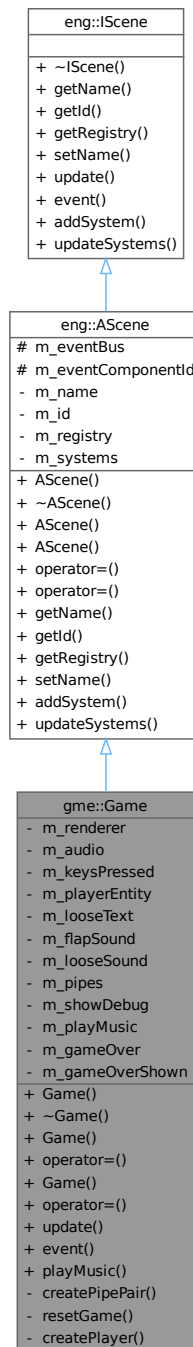
- [/home/masina/Projects/Epitech/rtype/modules/ECS/include/ECS/Component.hpp](#)

18.59 gme::Game Class Reference

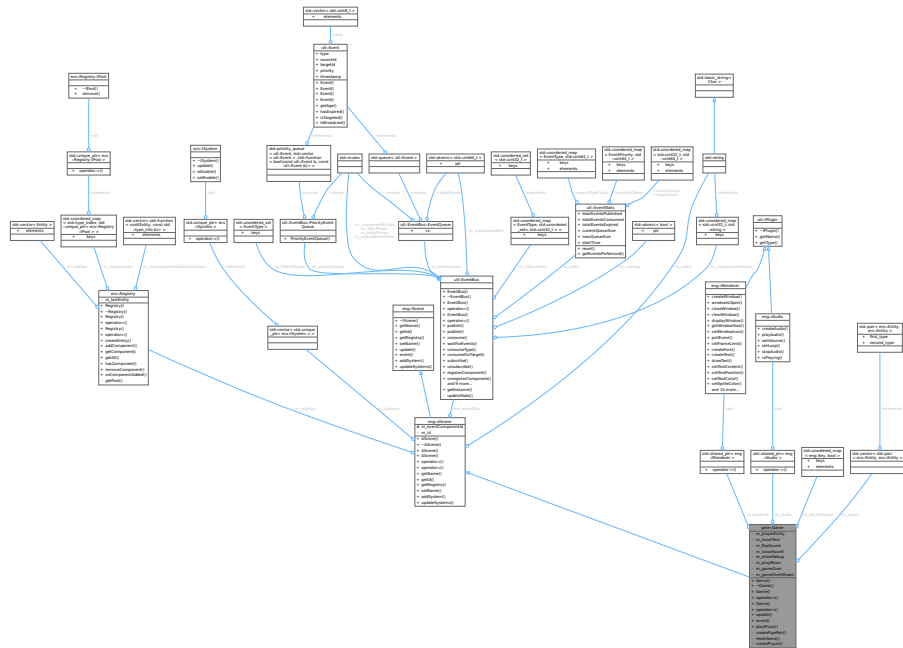
[Game](#) solo scene.

```
#include <Game.hpp>
```

Inheritance diagram for gme::Game:



Collaboration diagram for gme::Game:



Public Member Functions

- [Game](#) ([eng::id](#) assignedId, const std::shared_ptr< [eng::IRenderer](#) > &renderer, const std::shared_ptr< [eng::IAudio](#) > &audio, bool &showDebug)
- [~Game](#) () override=default
- [Game](#) (const [Game](#) &other)=delete
- [Game](#) & operator= (const [Game](#) &other)=delete
- [Game](#) ([Game](#) &&other)=delete
- [Game](#) & operator= ([Game](#) &&other)=delete
- void [update](#) (float dt, const [eng::WindowSize](#) &size) override
- void [event](#) (const [eng::Event](#) &event) override
- bool & [playMusic](#) ()

Public Member Functions inherited from [eng::AScene](#)

- [AScene](#) (const [id](#) assignedId)
- [~AScene](#) () override=default
- [AScene](#) (const [AScene](#) &other)=delete
- [AScene](#) ([AScene](#) &&other)=delete
- [AScene](#) & operator= (const [AScene](#) &other)=delete
- [AScene](#) & operator= ([AScene](#) &&other)=delete
- std::string & [getName](#) () override
- [id](#) [getId](#) () const override
- [ecs::Registry](#) & [getRegistry](#) () override
- void [setName](#) (const std::string &newName) override
- void [addSystem](#) (std::unique_ptr< [ecs::ISystem](#) > system) override
- void [updateSystems](#) (const float dt) override

Public Member Functions inherited from [eng::IScene](#)

- virtual [~IScene](#) ()=default

Private Member Functions

- `std::pair< ecs::Entity, ecs::Entity > createPipePair (ecs::Registry ®istry, float x, float gapY) const`
- `void resetGame ()`

Static Private Member Functions

- `static ecs::Entity createPlayer (ecs::Registry ®istry)`

Private Attributes

- `const std::shared_ptr< eng::IRenderer > & m_renderer`
- `const std::shared_ptr< eng::IAudio > & m_audio`
- `std::unordered_map< eng::Key, bool > m_keysPressed`
- `ecs::Entity m_playerEntity`
- `ecs::Entity m_looseText`
- `ecs::Entity m_flapSound`
- `ecs::Entity m_looseSound`
- `std::vector< std::pair< ecs::Entity, ecs::Entity > > m_pipes`
- `bool & m_showDebug`
- `bool m_playMusic = false`
- `bool m_gameOver = false`
- `bool m_gameOverShown = false`

Additional Inherited Members

Protected Attributes inherited from `eng::AScene`

- `utl::EventBus & m_eventBus = utl::EventBus::getInstance()`
- `std::uint32_t m_eventComponentId = 0`

18.59.1 Detailed Description

`Game` solo scene.

Definition at line 22 of file `Game.hpp`.

18.59.2 Constructor & Destructor Documentation

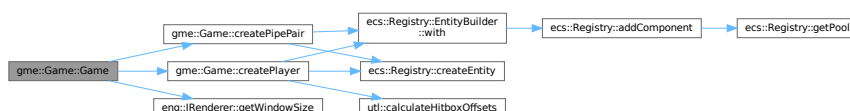
18.59.2.1 `Game()` [1/3]

```
gme::Game::Game (
    eng::id assignedId,
    const std::shared_ptr< eng::IRenderer > & renderer,
    const std::shared_ptr< eng::IAudio > & audio,
    bool & showDebug)
```

Definition at line 40 of file `game.cpp`.

References `eng::Color::a`, `eng::Color::b`, `createPipePair()`, `createPlayer()`, `utl::Config::Color::CYAN_ELECTRIC`, `utl::Path::Font::FONT_S_RTYPE`, `eng::Color::g`, `eng::IRenderer::getWindowSize()`, `ecs::IComponent::id`, `m_flapSound`, `m_looseSound`, `m_looseText`, `m_pipes`, `m_playerEntity`, `m_renderer`, `eng::Color::r`, `ecs::Scale::x`, `ecs::Transform::x`, and `ecs::Transform::y`.

Here is the call graph for this function:



18.59.2.2 ~Game()

`gme::Game::~~Game ()` [override], [default]

18.59.2.3 Game() [2/3]

`gme::Game::Game (`
 const [Game](#) & other) [delete]

18.59.2.4 Game() [3/3]

`gme::Game::Game (`
 [Game](#) && other) [delete]

18.59.3 Member Function Documentation

18.59.3.1 createPipePair()

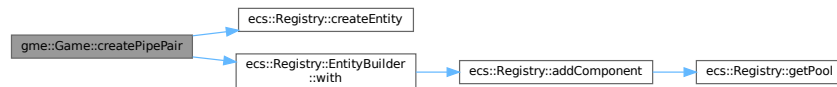
`std::pair< ecs::Entity, ecs::Entity > gme::Game::createPipePair (`
 [ecs::Registry](#) & registry,
 float x,
 float gapY) const [private]

Definition at line 253 of file [game.cpp](#).

References [ecs::Registry::createEntity\(\)](#), and [ecs::Registry::EntityBuilder::with\(\)](#).

Referenced by [Game\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.59.3.2 createPlayer()

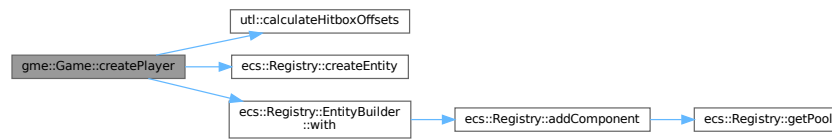
`ecs::Entity gme::Game::createPlayer (`
 [ecs::Registry](#) & registry) [static], [private]

Definition at line 238 of file [game.cpp](#).

References [utl::calculateHitboxOffsets\(\)](#), [ecs::Registry::createEntity\(\)](#), [utl::Path::Texture::TEXTURE_PLAYER](#), and [ecs::Registry::EntityBuilder::with\(\)](#).

Referenced by [Game\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.59.3.3 event()

```
void gme::Game::event (
    const eng::Event & event) [override], [virtual]
```

Implements [eng::IScene](#).

Definition at line 213 of file [game.cpp](#).

References [eng::Event::key](#), [eng::KeyPressed](#), [ecs::Audio::play](#), [eng::R](#), [eng::Space](#), and [eng::Event::type](#).

18.59.3.4 operator=() [1/2]

```
Game & gme::Game::operator= (
    const Game & other) [delete]
```

18.59.3.5 operator=() [2/2]

```
Game & gme::Game::operator= (
    Game && other) [delete]
```

18.59.3.6 playMusic()

```
bool & gme::Game::playMusic () [inline]
```

Definition at line 36 of file [Game.hpp](#).

References [m_playMusic](#).

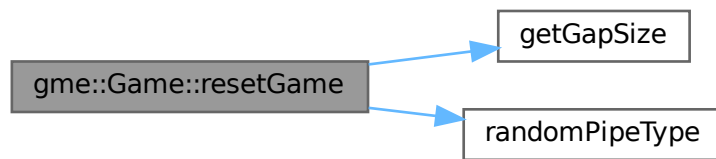
18.59.3.7 resetGame()

```
void gme::Game::resetGame () [private]
```

Definition at line 281 of file [game.cpp](#).

References [getGapSize\(\)](#), [randomPipeType\(\)](#), and [ecs::Transform::x](#).

Here is the call graph for this function:



18.59.3.8 update()

```
void gme::Game::update (
    float dt,
    const eng::WindowSize & size) [override], [virtual]
```

Implements [eng::IScene](#).

Definition at line 141 of file [game.cpp](#).

References [eng::WindowSize::height](#), [eng::WindowSize::width](#), [ecs::Transform::x](#), [ecs::Velocity::x](#), and [ecs::Transform::y](#).

18.59.4 Member Data Documentation

18.59.4.1 m_audio

```
const std::shared_ptr<eng::IAudio>& gme::Game::m_audio [private]
```

Definition at line 44 of file [Game.hpp](#).

18.59.4.2 m_flapSound

```
ecs::Entity gme::Game::m_flapSound [private]
```

Definition at line 48 of file [Game.hpp](#).

Referenced by [Game\(\)](#).

18.59.4.3 m_gameOver

```
bool gme::Game::m_gameOver = false [private]
```

Definition at line 53 of file [Game.hpp](#).

18.59.4.4 m_gameOverShown

```
bool gme::Game::m_gameOverShown = false [private]
```

Definition at line 54 of file [Game.hpp](#).

18.59.4.5 m_keysPressed

```
std::unordered_map<eng::Key, bool> gme::Game::m_keysPressed [private]
```

Definition at line 45 of file [Game.hpp](#).

18.59.4.6 m_looseSound

```
ecs::Entity gme::Game::m_looseSound [private]
```

Definition at line 49 of file [Game.hpp](#).

Referenced by [Game\(\)](#).

18.59.4.7 m_looseText

[ecs::Entity](#) gme::Game::m_looseText [private]

Definition at line 47 of file [Game.hpp](#).

Referenced by [Game\(\)](#).

18.59.4.8 m_pipes

std::vector<std::pair<[ecs::Entity](#), [ecs::Entity](#)> > gme::Game::m_pipes [private]

Definition at line 50 of file [Game.hpp](#).

Referenced by [Game\(\)](#).

18.59.4.9 m_playerEntity

[ecs::Entity](#) gme::Game::m_playerEntity [private]

Definition at line 46 of file [Game.hpp](#).

Referenced by [Game\(\)](#).

18.59.4.10 m_playMusic

bool gme::Game::m_playMusic = false [private]

Definition at line 52 of file [Game.hpp](#).

Referenced by [playMusic\(\)](#).

18.59.4.11 m_renderer

const std::shared_ptr<[eng::IRenderer](#)>& gme::Game::m_renderer [private]

Definition at line 43 of file [Game.hpp](#).

Referenced by [Game\(\)](#).

18.59.4.12 m_showDebug

bool& gme::Game::m_showDebug [private]

Definition at line 51 of file [Game.hpp](#).

The documentation for this class was generated from the following files:

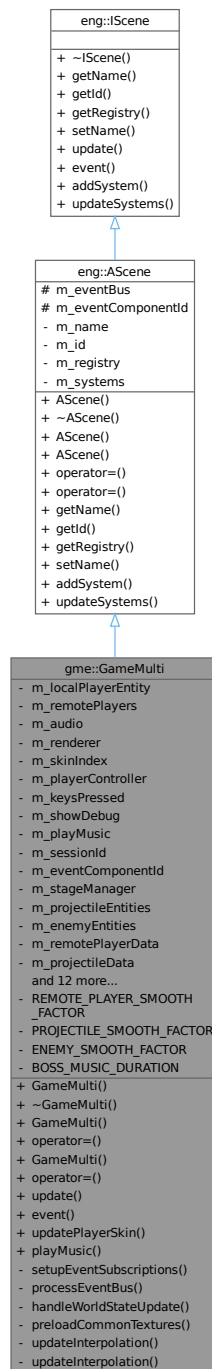
- [/home/masina/Projects/Epitech/rtype/plugins/Game/FlappyBird/Client/Solo/include/FlappyBirdClientSolo/Scenes/\[Game.hpp\]\(#\)](#)
- [/home/masina/Projects/Epitech/rtype/plugins/Game/FlappyBird/Client/Solo/src/scenes/\[game.cpp\]\(#\)](#)

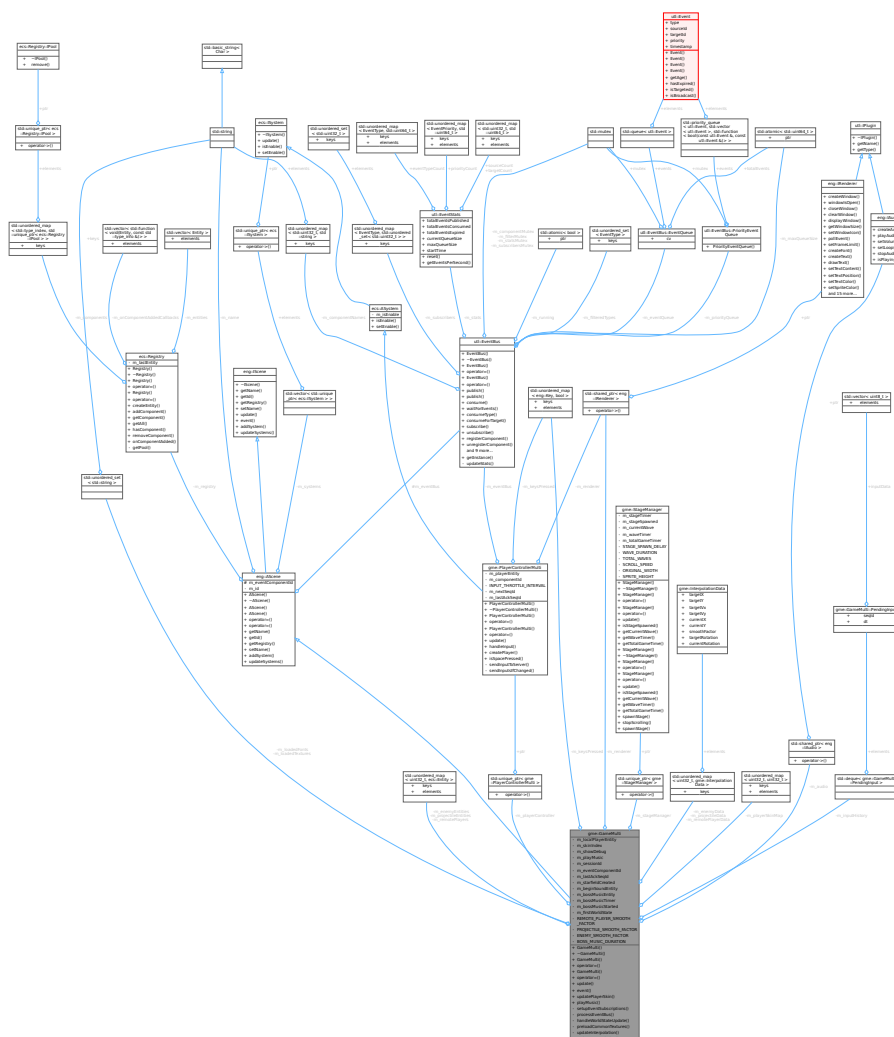
18.60 gme::GameMulti Class Reference

[GameMulti](#) scene.

#include <[GameMulti.hpp](#)>

Inheritance diagram for gme::GameMulti:





- struct PendingInput

- struct PendingInput

- `GameMulti` (`eng::id` assignedId, `const std::shared_ptr< eng::IRenderer >` &renderer, `const std::shared_ptr< eng::IAudio >` &audio, `float` skinIndex, `bool` &showDebug, `uint32_t` sessionId)

- **GameMulti** (eng::id assignedId, const std::shared_ptr< eng::IRenderer > &renderer, const std::shared_ptr< eng::IAudio > &audio, float skinIndex, bool &showDebug, uint32_t sessionId)
- ~**GameMulti** () override=default
- **GameMulti** (const **GameMulti** &other)=delete
- **GameMulti** & operator= (const **GameMulti** &other)=delete
- **GameMulti** (**GameMulti** &&other)=delete
- **GameMulti** & operator= (**GameMulti** &&other)=delete
- void **update** (float dt, const eng::WindowSize &size) override
- void **event** (const eng::Event &event) override
- void **updatePlayerSkin** ()
- bool & **playMusic** ()

- AScene (const id assignedId)

- **AScene** (const **id** assignedId)

- `~AScene ()` override=default
- `AScene (const AScene &other)=delete`
- `AScene (AScene &&other)=delete`
- `AScene & operator= (const AScene &other)=delete`
- `AScene & operator= (AScene &&other)=delete`
- `std::string & getName ()` override
- `id getId ()` const override
- `ecs::Registry & getRegistry ()` override
- `void setName (const std::string &newName)` override
- `void addSystem (std::unique_ptr< ecs::ISystem > system)` override
- `void updateSystems (const float dt)` override

Public Member Functions inherited from `eng::IScene`

- virtual `~IScene ()`=default

Private Member Functions

- `void setupEventSubscriptions ()` const
- `void processEventBus ()`
- `void handleWorldStateUpdate (const utl::Event &event)`
- `void preloadCommonTextures ()`
- `void updateInterpolation (std::unordered_map< uint32_t, InterpolationData > &dataMap, std::unordered_map< uint32_t, ecs::Entity > &entityMap, float smoothFactor, float dt, ecs::Registry ®istry)`
- `void updateInterpolation (std::unordered_map< uint32_t, InterpolationData > &dataMap, std::unordered_map< uint32_t, ecs::Entity > &entityMap, float smoothFactor, float dt, ecs::Registry ®istry)` const

Private Attributes

- `ecs::Entity m_localPlayerEntity`
- `std::unordered_map< uint32_t, ecs::Entity > m_remotePlayers`
- `const std::shared_ptr< eng::IAudio > & m_audio`
- `const std::shared_ptr< eng::IRenderer > & m_renderer`
- `float m_skinIndex`
- `std::unique_ptr< PlayerControllerMulti > m_playerController`
- `std::unordered_map< eng::Key, bool > m_keysPressed`
- `bool & m_showDebug`
- `bool m_playMusic = false`
- `uint32_t m_sessionId`
- `uint32_t m_eventComponentId = 10`
- `std::unique_ptr< StageManager > m_stageManager`
- `std::unordered_map< uint32_t, ecs::Entity > m_projectileEntities`
- `std::unordered_map< uint32_t, ecs::Entity > m_enemyEntities`
- `std::unordered_map< uint32_t, InterpolationData > m_remotePlayerData`
- `std::unordered_map< uint32_t, InterpolationData > m_projectileData`
- `std::unordered_map< uint32_t, InterpolationData > m_enemyData`
- `std::unordered_map< uint32_t, uint32_t > m_playerSkinMap`
- `std::deque< PendingInput > m_inputHistory`
- `uint32_t m_lastAckSeqId = 0`
- `bool m_starfieldCreated = false`
- `ecs::Entity m_beginSoundEntity {}`
- `ecs::Entity m_bossMusicEntity {}`
- `float m_bossMusicTimer = 0.0f`
- `bool m_bossMusicStarted = false`
- `std::unordered_set< std::string > m_loadedTextures`
- `std::unordered_set< std::string > m_loadedFonts`
- `bool m_firstWorldState = true`

Static Private Attributes

- static constexpr float [REMOTE_PLAYER_SMOOTH_FACTOR](#) = 0.25f
- static constexpr float [PROJECTILE_SMOOTH_FACTOR](#) = 0.5f
- static constexpr float [ENEMY_SMOOTH_FACTOR](#) = 0.18f
- static constexpr float [BOSS_MUSIC_DURATION](#) = 40.0f

Additional Inherited Members

Protected Attributes inherited from [eng::AScene](#)

- [utl::EventBus](#) & [m_eventBus](#) = [utl::EventBus::getInstance\(\)](#)
- std::uint32_t [m_eventComponentId](#) = 0

18.60.1 Detailed Description

[GameMulti](#) scene.

Definition at line 39 of file [GameMulti.hpp](#).

18.60.2 Constructor & Destructor Documentation

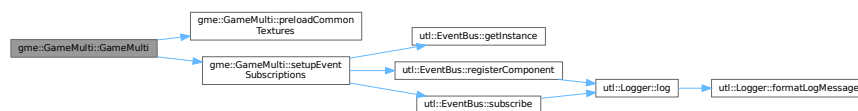
18.60.2.1 GameMulti() [1/3]

```
gme::GameMulti::GameMulti (
    eng::id assignedId,
    const std::shared_ptr< eng::IRenderer > & renderer,
    const std::shared_ptr< eng::IAudio > & audio,
    float skinIndex,
    bool & showDebug,
    uint32_t sessionId)
```

Definition at line 15 of file [gameMulti.cpp](#).

References [utl::Path::Audio::AUDIO_BEGIN](#), [ecs::IComponent::id](#), [m_beginSoundEntity](#), [m_loadedFonts](#), [m_loadedTextures](#), [m_localPlayerEntity](#), [m_playerController](#), [m_playerSkinMap](#), [m_sessionId](#), [m_stageManager](#), [preloadCommonTextures\(\)](#), [setupEventSubscriptions\(\)](#), [utl::GameConfig::Player::SPRITE_HEIGHT](#), [ecs::Scale::x](#), [ecs::Transform::x](#), and [ecs::Transform::y](#).

Here is the call graph for this function:



18.60.2.2 ~GameMulti()

```
gme::GameMulti::~GameMulti () [override], [default]
```

18.60.2.3 GameMulti() [2/3]

```
gme::GameMulti::GameMulti (
    const GameMulti & other) [delete]
```

18.60.2.4 GameMulti() [3/3]

```
gme::GameMulti::GameMulti (
    GameMulti && other) [delete]
```

18.60.3 Member Function Documentation

18.60.3.1 event()

void gme::GameMulti::event (
 const [eng::Event](#) & event) [override], [virtual]

Implements [eng::IScene](#).

Definition at line 673 of file [gameMulti.cpp](#).

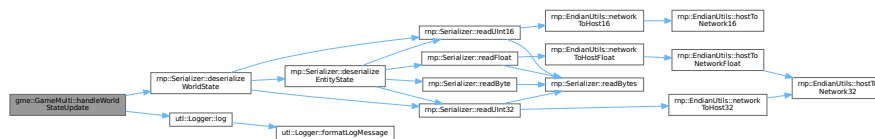
18.60.3.2 handleWorldStateUpdate()

void gme::GameMulti::handleWorldStateUpdate (
 const [utl::Event](#) & event) [private]

Definition at line 205 of file [gameMulti.cpp](#).

References [utl::GameConfig::Explosion::ANIMATION_DURATION](#), [utl::GameConfig::Explosion::ANIMATION_FRAME](#), [rnp::BOSS](#), [utl::Event::data](#), [rnp::Serializer::deserializeWorldState\(\)](#), [rnp::ENEMY](#), [rnp::ENEMY_ADVANCED](#), [rnp::PacketWorldState::entities](#), [utl::GameConfig::Explosion::FRAMES_PER_ROW](#), [utl::INFO](#), [utl::GameConfig::Explosion::LIFETIME](#), [utl::Logger::log\(\)](#), [rnp::PLAYER](#), [rnp::PROJECTILE](#), [rnp::PROJECTILE_ENEMY](#), [rnp::PROJECTILE_PLAYER_SUPERCHARGED](#), [utl::GameConfig::Explosion::SCALE](#), [utl::GameConfig::Player::SCALE](#), [utl::GameConfig::Explosion::SPRITE_HEIGHT](#), [utl::GameConfig::Player::SPRITE_H](#), [utl::GameConfig::Explosion::SPRITE_WIDTH](#), [utl::GameConfig::Player::SPRITE_WIDTH](#), [utl::Path::Texture::TEXTU](#), [utl::Path::Texture::TEXTURE_ENEMY_ADVANCED](#), [utl::Path::Texture::TEXTURE_ENEMY_BASIC](#), [utl::Path::Texture::TEXTURE_ENEMY_PROJECTILE](#), [utl::Path::Texture::TEXTURE_EXPLOSION](#), [utl::Path::Texture::TEXTURE_PLAYER](#), [utl::Path::Texture::TEXTURE_SHOOT](#), [utl::Path::Texture::TEXTURE_SH](#) and [utl::WARNING](#).

Here is the call graph for this function:



18.60.3.3 operator=() [1/2]

[GameMulti](#) & gme::GameMulti::operator= (
 const [GameMulti](#) & other) [delete]

18.60.3.4 operator=() [2/2]

[GameMulti](#) & gme::GameMulti::operator= (
 [GameMulti](#) && other) [delete]

18.60.3.5 playMusic()

bool & gme::GameMulti::playMusic () [inline]
 Definition at line 55 of file [GameMulti.hpp](#).
 References [m_playMusic](#).

18.60.3.6 preloadCommonTextures()

void gme::GameMulti::preloadCommonTextures () [private]
 Definition at line 148 of file [gameMulti.cpp](#).

References [utl::Path::Texture::TEXTURE_BOSS](#), [utl::Path::Texture::TEXTURE_ENEMY_ADVANCED](#), [utl::Path::Texture::TEXTURE_ENEMY_BASIC](#), [utl::Path::Texture::TEXTURE_ENEMY_PROJECTILE](#), [utl::Path::Texture::TEXTURE_EXPLOSION](#), [utl::Path::Texture::TEXTURE_PLAYER](#), [utl::Path::Texture::TEXTURE_SHOOT_CHARGED](#) and [utl::Path::Texture::TEXTURE_SHOOT_CHARGED](#).
 Referenced by [GameMulti\(\)](#).

Here is the caller graph for this function:



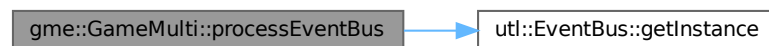
18.60.3.7 processEventBus()

`void gme::GameMulti::processEventBus () [private]`

Definition at line 182 of file [gameMulti.cpp](#).

References [utl::GAME_START](#), [utl::EventBus::getInstance\(\)](#), [utl::PLAYER_INPUT_RECEIVED](#), and [utl::WORLD_STATE_RECEIVED](#).

Here is the call graph for this function:



18.60.3.8 setupEventSubscriptions()

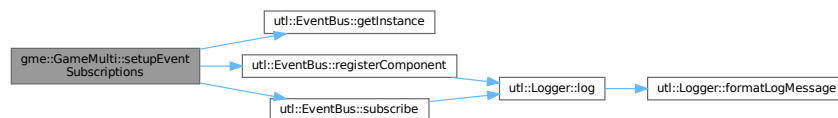
`void gme::GameMulti::setupEventSubscriptions () const [private]`

Definition at line 173 of file [gameMulti.cpp](#).

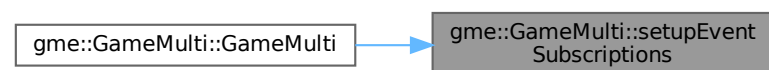
References [utl::GAME_START](#), [utl::EventBus::getInstance\(\)](#), [utl::PLAYER_INPUT_RECEIVED](#), [utl::EventBus::registerComponent\(\)](#), [utl::EventBus::subscribe\(\)](#), and [utl::WORLD_STATE_RECEIVED](#).

Referenced by [GameMulti\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.60.3.9 update()

```
void gme::GameMulti::update (
    float dt,
    const eng::WindowSize & size) [override], [virtual]
```

Implements [eng::IScene](#).

Definition at line 600 of file [gameMulti.cpp](#).

References [utl::Path::Audio::AUDIO_BOSS](#), and [eng::Playing](#).

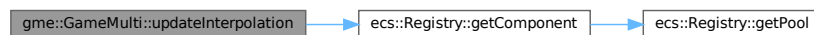
18.60.3.10 updateInterpolation() [1/2]

```
void gme::GameMulti::updateInterpolation (
    std::unordered_map< uint32_t, InterpolationData > & dataMap,
    std::unordered_map< uint32_t, ecs::Entity > & entityMap,
    float smoothFactor,
    float dt,
    ecs::Registry & registry) [private]
```

Definition at line 679 of file [gameMulti.cpp](#).

References [ecs::Registry::getComponent\(\)](#).

Here is the call graph for this function:



18.60.3.11 updateInterpolation() [2/2]

```
void gme::GameMulti::updateInterpolation (
    std::unordered_map< uint32_t, InterpolationData > & dataMap,
    std::unordered_map< uint32_t, ecs::Entity > & entityMap,
    float smoothFactor,
    float dt,
    ecs::Registry & registry) const [private]
```

18.60.3.12 updatePlayerSkin()

```
void gme::GameMulti::updatePlayerSkin ()
```

Definition at line 713 of file [gameMulti.cpp](#).

References [utl::GameConfig::Player::SPRITE_HEIGHT](#).

18.60.4 Member Data Documentation

18.60.4.1 BOSS_MUSIC_DURATION

```
float gme::GameMulti::BOSS_MUSIC_DURATION = 40.0f [static], [constexpr], [private]
```

Definition at line 111 of file [GameMulti.hpp](#).

18.60.4.2 ENEMY_SMOOTH_FACTOR

```
float gme::GameMulti::ENEMY_SMOOTH_FACTOR = 0.18f [static], [constexpr], [private]
```

Definition at line 104 of file [GameMulti.hpp](#).

18.60.4.3 m_audio

```
const std::shared_ptr<eng::IAudio>& gme::GameMulti::m_audio [private]
```

Definition at line 74 of file [GameMulti.hpp](#).

18.60.4.4 m_beginSoundEntity

[ecs::Entity](#) gme::GameMulti::m_beginSoundEntity {} [private]

Definition at line 107 of file [GameMulti.hpp](#).

Referenced by [GameMulti\(\)](#).

18.60.4.5 m_bossMusicEntity

[ecs::Entity](#) gme::GameMulti::m_bossMusicEntity {} [private]

Definition at line 108 of file [GameMulti.hpp](#).

18.60.4.6 m_bossMusicStarted

bool gme::GameMulti::m_bossMusicStarted = false [private]

Definition at line 110 of file [GameMulti.hpp](#).

18.60.4.7 m_bossMusicTimer

float gme::GameMulti::m_bossMusicTimer = 0.0f [private]

Definition at line 109 of file [GameMulti.hpp](#).

18.60.4.8 m_enemyData

std::unordered_map<uint32_t, [InterpolationData](#)> gme::GameMulti::m_enemyData [private]

Definition at line 91 of file [GameMulti.hpp](#).

18.60.4.9 m_enemyEntities

std::unordered_map<uint32_t, [ecs::Entity](#)> gme::GameMulti::m_enemyEntities [private]

Definition at line 87 of file [GameMulti.hpp](#).

18.60.4.10 m_eventComponentId

uint32_t gme::GameMulti::m_eventComponentId = 10 [private]

Definition at line 83 of file [GameMulti.hpp](#).

18.60.4.11 m_firstWorldState

bool gme::GameMulti::m_firstWorldState = true [private]

Definition at line 118 of file [GameMulti.hpp](#).

18.60.4.12 m_inputHistory

std::deque<[PendingInput](#)> gme::GameMulti::m_inputHistory [private]

Definition at line 99 of file [GameMulti.hpp](#).

18.60.4.13 m_keysPressed

std::unordered_map<[eng::Key](#), bool> gme::GameMulti::m_keysPressed [private]

Definition at line 78 of file [GameMulti.hpp](#).

18.60.4.14 m_lastAckSeqId

uint32_t gme::GameMulti::m_lastAckSeqId = 0 [private]

Definition at line 100 of file [GameMulti.hpp](#).

18.60.4.15 m_loadedFonts

std::unordered_set<std::string> gme::GameMulti::m_loadedFonts [private]

Definition at line 115 of file [GameMulti.hpp](#).

Referenced by [GameMulti\(\)](#).

18.60.4.16 m_loadedTextures

std::unordered_set<std::string> gme::GameMulti::m_loadedTextures [private]

Definition at line 114 of file [GameMulti.hpp](#).

Referenced by [GameMulti\(\)](#).

18.60.4.17 m_localPlayerEntity

[ecs::Entity](#) gme::GameMulti::m_localPlayerEntity [private]

Definition at line 71 of file [GameMulti.hpp](#).

Referenced by [GameMulti\(\)](#).

18.60.4.18 m_playerController

std::unique_ptr<[PlayerControllerMulti](#)> gme::GameMulti::m_playerController [private]

Definition at line 77 of file [GameMulti.hpp](#).

Referenced by [GameMulti\(\)](#).

18.60.4.19 m_playerSkinMap

std::unordered_map<uint32_t, uint32_t> gme::GameMulti::m_playerSkinMap [private]

Definition at line 92 of file [GameMulti.hpp](#).

Referenced by [GameMulti\(\)](#).

18.60.4.20 m_playMusic

bool gme::GameMulti::m_playMusic = false [private]

Definition at line 80 of file [GameMulti.hpp](#).

Referenced by [playMusic\(\)](#).

18.60.4.21 m_projectileData

std::unordered_map<uint32_t, [InterpolationData](#)> gme::GameMulti::m_projectileData [private]

Definition at line 90 of file [GameMulti.hpp](#).

18.60.4.22 m_projectileEntities

std::unordered_map<uint32_t, [ecs::Entity](#)> gme::GameMulti::m_projectileEntities [private]

Definition at line 86 of file [GameMulti.hpp](#).

18.60.4.23 m_remotePlayerData

std::unordered_map<uint32_t, [InterpolationData](#)> gme::GameMulti::m_remotePlayerData [private]

Definition at line 89 of file [GameMulti.hpp](#).

18.60.4.24 m_remotePlayers

std::unordered_map<uint32_t, [ecs::Entity](#)> gme::GameMulti::m_remotePlayers [private]

Definition at line 72 of file [GameMulti.hpp](#).

18.60.4.25 m_renderer

const std::shared_ptr<[eng::IRenderer](#)>& gme::GameMulti::m_renderer [private]

Definition at line 75 of file [GameMulti.hpp](#).

18.60.4.26 m_sessionId

uint32_t gme::GameMulti::m_sessionId [private]

Definition at line 82 of file [GameMulti.hpp](#).

Referenced by [GameMulti\(\)](#).

18.60.4.27 m_showDebug

bool& gme::GameMulti::m_showDebug [private]
 Definition at line 79 of file [GameMulti.hpp](#).

18.60.4.28 m_skinIndex

float gme::GameMulti::m_skinIndex [private]
 Definition at line 76 of file [GameMulti.hpp](#).

18.60.4.29 m_stageManager

std::unique_ptr<[StageManager](#)> gme::GameMulti::m_stageManager [private]
 Definition at line 84 of file [GameMulti.hpp](#).
 Referenced by [GameMulti\(\)](#).

18.60.4.30 m_starfieldCreated

bool gme::GameMulti::m_starfieldCreated = false [private]
 Definition at line 106 of file [GameMulti.hpp](#).

18.60.4.31 PROJECTILE_SMOOTH_FACTOR

float gme::GameMulti::PROJECTILE_SMOOTH_FACTOR = 0.5f [static], [constexpr], [private]
 Definition at line 103 of file [GameMulti.hpp](#).

18.60.4.32 REMOTE_PLAYER_SMOOTH_FACTOR

float gme::GameMulti::REMOTE_PLAYER_SMOOTH_FACTOR = 0.25f [static], [constexpr], [private]
 Definition at line 102 of file [GameMulti.hpp](#).

The documentation for this class was generated from the following files:

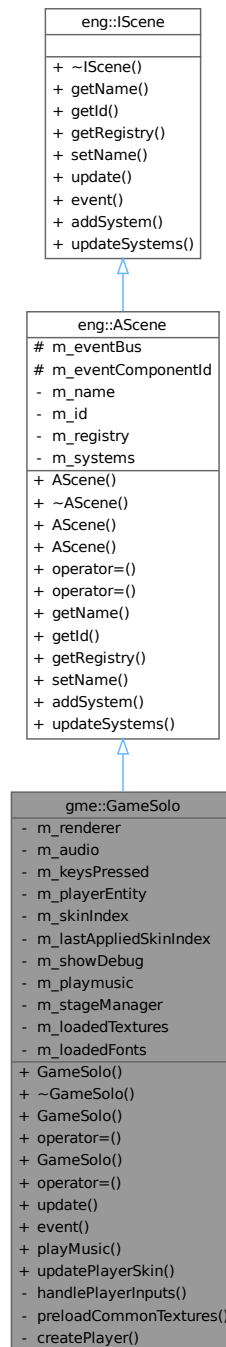
- [/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/include/RType↵ClientMulti/Scenes/\[GameMulti.hpp\]\(#\)](#)
- [/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/src/scenes/\[gameMulti.cpp\]\(#\)](#)

18.61 gme::GameSolo Class Reference

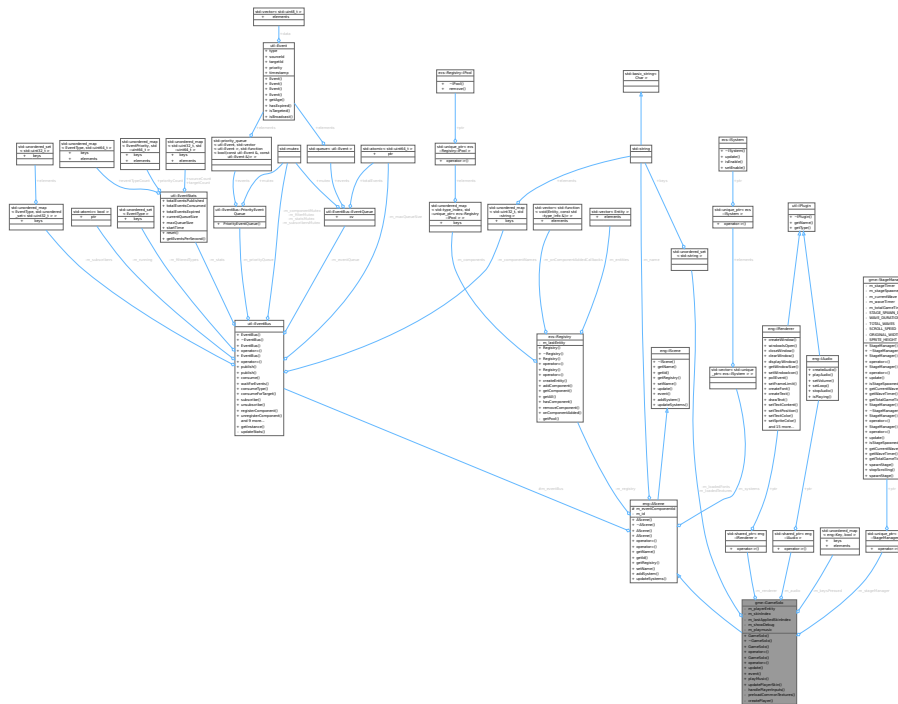
[GameSolo](#) scene.

#include <[GameSolo.hpp](#)>

Inheritance diagram for gme::GameSolo:



Collaboration diagram for `gme::GameSolo`:



Public Member Functions

- `GameSolo` (`eng::id` assignedId, const `std::shared_ptr< eng::IRenderer >` &renderer, const `std::shared_ptr< eng::IAudio >` &audio, int skinIndex, bool &showDebug)
- `~GameSolo` () override=default
- `GameSolo` (const `GameSolo` &other)=delete
- `GameSolo & operator=` (const `GameSolo` &other)=delete
- `GameSolo` (`GameSolo` &&other)=delete
- `GameSolo & operator=` (`GameSolo` &&other)=delete
- void `update` (float dt, const `eng::WindowSize` &size) override
- void `event` (const `eng::Event` &event) override
- bool & `playMusic` ()
- void `updatePlayerSkin` ()

Public Member Functions inherited from `eng::AScene`

- `AScene` (const `id` assignedId)
- `~AScene` () override=default
- `AScene` (const `AScene` &other)=delete
- `AScene` (`AScene` &&other)=delete
- `AScene & operator=` (const `AScene` &other)=delete
- `AScene & operator=` (`AScene` &&other)=delete
- `std::string & getName` () override
- `id getId` () const override
- `ecs::Registry & getRegistry` () override
- void `setName` (const `std::string` &newName) override
- void `addSystem` (`std::unique_ptr< ecs::ISystem >` system) override
- void `updateSystems` (const float dt) override

Public Member Functions inherited from `eng::IScene`

- virtual `~IScene` ()=default

Private Member Functions

- void [handlePlayerInputs](#) (ecs::Registry ®istry, float dt)
- void [preloadCommonTextures](#) ()

Static Private Member Functions

- static [ecs::Entity createPlayer](#) (ecs::Registry ®istry)

Private Attributes

- const std::shared_ptr< [eng::IRenderer](#) > & [m_renderer](#)
- const std::shared_ptr< [eng::IAudio](#) > & [m_audio](#)
- std::unordered_map< [eng::Key](#), bool > [m_keysPressed](#)
- [ecs::Entity](#) [m_playerEntity](#)
- int [m_skinIndex](#)
- int [m_lastAppliedSkinIndex](#) = -1
- bool & [m_showDebug](#)
- bool [m_playmusic](#) = false
- std::unique_ptr< [StageManager](#) > [m_stageManager](#)
- std::unordered_set< std::string > [m_loadedTextures](#)
- std::unordered_set< std::string > [m_loadedFonts](#)

Additional Inherited Members

Protected Attributes inherited from [eng::AScene](#)

- [utl::EventBus](#) & [m_eventBus](#) = [utl::EventBus::getInstance\(\)](#)
- std::uint32_t [m_eventComponentId](#) = 0

18.61.1 Detailed Description

[GameSolo](#) scene.

Definition at line 24 of file [GameSolo.hpp](#).

18.61.2 Constructor & Destructor Documentation

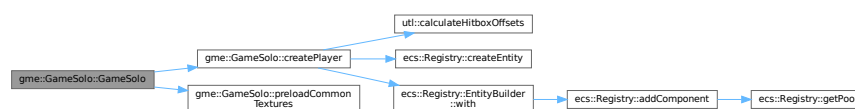
18.61.2.1 [GameSolo\(\)](#) [1/3]

```
gme::GameSolo::GameSolo (
    eng::id assignedId,
    const std::shared_ptr< eng::IRenderer > & renderer,
    const std::shared_ptr< eng::IAudio > & audio,
    int skinIndex,
    bool & showDebug)
```

Definition at line 8 of file [gameSolo.cpp](#).

References [utl::Path::Audio::AUDIO_BEGIN](#), [createPlayer\(\)](#), [ecs::IComponent::id](#), [m_loadedFonts](#), [m_loadedTextures](#), [m_playerEntity](#), [m_stageManager](#), [preloadCommonTextures\(\)](#), [ecs::Scale::x](#), [ecs::Transform::x](#), and [ecs::Transform::y](#).

Here is the call graph for this function:



18.61.2.2 ~GameSolo()

`gme::GameSolo::~~GameSolo()` [override], [default]

18.61.2.3 GameSolo() [2/3]

`gme::GameSolo::GameSolo()` (const [GameSolo](#) & other) [delete]

18.61.2.4 GameSolo() [3/3]

`gme::GameSolo::GameSolo()` ([GameSolo](#) && other) [delete]

18.61.3 Member Function Documentation

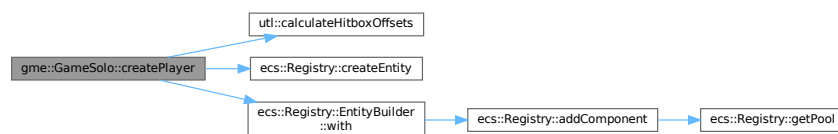
18.61.3.1 createPlayer()

[ecs::Entity](#) `gme::GameSolo::createPlayer()` ([ecs::Registry](#) & registry) [static], [private]

Definition at line 259 of file [gameSolo.cpp](#).

References [utl::calculateHitboxOffsets\(\)](#), [ecs::Registry::createEntity\(\)](#), [utl::GameConfig::Beam::MAX_CHARGE](#), [utl::GameConfig::Hitbox::PLAYER_RADIUS](#), [utl::GameConfig::Player::SCALE](#), [utl::GameConfig::Player::SPRITE_HEIGHT](#), [utl::GameConfig::Player::SPRITE_WIDTH](#), [utl::Path::Texture::TEXTURE_PLAYER](#), and [ecs::Registry::EntityBuilder::with\(\)](#).
Referenced by [GameSolo\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.61.3.2 event()

`void gme::GameSolo::event()` (const [eng::Event](#) & event) [override], [virtual]

Implements [eng::IScene](#).

Definition at line 201 of file [gameSolo.cpp](#).

References [eng::Down](#), [eng::Event::key](#), [eng::KeyPressed](#), [eng::KeyReleased](#), [eng::Left](#), [eng::Right](#), [eng::Event::type](#), and [eng::Up](#).

18.61.3.3 handlePlayerInputs()

`void gme::GameSolo::handlePlayerInputs()` ([ecs::Registry](#) & registry, float dt) [private]

Definition at line 129 of file [gameSolo.cpp](#).

References [utl::GameConfig::Player::DIAGONAL_SPEED_MULTIPLIER](#), [eng::Down](#), [ecs::Registry::getComponent\(\)](#), [eng::Left](#), [eng::Right](#), [utl::GameConfig::Player::SCALE](#), [utl::GameConfig::Player::SPEED](#), [utl::GameConfig::Player::SPRI](#), [utl::GameConfig::Player::SPRITE_WIDTH](#), [eng::Up](#), [ecs::Transform::x](#), and [ecs::Transform::y](#).

Here is the call graph for this function:



18.61.3.4 operator=() [1/2]

[GameSolo](#) & gme::GameSolo::operator= (
 const [GameSolo](#) & other) [delete]

18.61.3.5 operator=() [2/2]

[GameSolo](#) & gme::GameSolo::operator= (
 [GameSolo](#) && other) [delete]

18.61.3.6 playMusic()

bool & gme::GameSolo::playMusic () [inline]

Definition at line 39 of file [GameSolo.hpp](#).

References [m_playmusic](#).

18.61.3.7 preloadCommonTextures()

void gme::GameSolo::preloadCommonTextures () [private]

Definition at line 107 of file [gameSolo.cpp](#).

References [utl::Path::Texture::TEXTURE_PLAYER](#), [utl::Path::Texture::TEXTURE_SHOOT](#), [utl::Path::Texture::TEXTURE_SHOOT_CHARGED](#), [utl::Path::Texture::TEXTURE_STAGE1_CEILING](#), and [utl::Path::Texture::TEXTURE_STAGE1_FLOOR](#).

Referenced by [GameSolo\(\)](#).

Here is the caller graph for this function:



18.61.3.8 update()

void gme::GameSolo::update (
 float dt,
 const [eng::WindowSize](#) & size) [override], [virtual]

Implements [eng::IScene](#).

Definition at line 194 of file [gameSolo.cpp](#).

18.61.3.9 updatePlayerSkin()

void gme::GameSolo::updatePlayerSkin ()

Definition at line 248 of file [gameSolo.cpp](#).

References [utl::GameConfig::Player::SPRITE_HEIGHT](#).

18.61.4 Member Data Documentation

18.61.4.1 m_audio

const std::shared_ptr<[eng::IAudio](#)>& gme::GameSolo::m_audio [private]

Definition at line 48 of file [GameSolo.hpp](#).

18.61.4.2 m_keysPressed

std::unordered_map<[eng::Key](#), bool> gme::GameSolo::m_keysPressed [private]

Definition at line 49 of file [GameSolo.hpp](#).

18.61.4.3 m_lastAppliedSkinIndex

int gme::GameSolo::m_lastAppliedSkinIndex = -1 [private]

Definition at line 52 of file [GameSolo.hpp](#).

18.61.4.4 m_loadedFonts

std::unordered_set<std::string> gme::GameSolo::m_loadedFonts [private]

Definition at line 59 of file [GameSolo.hpp](#).

Referenced by [GameSolo\(\)](#).

18.61.4.5 m_loadedTextures

std::unordered_set<std::string> gme::GameSolo::m_loadedTextures [private]

Definition at line 58 of file [GameSolo.hpp](#).

Referenced by [GameSolo\(\)](#).

18.61.4.6 m_playerEntity

[ecs::Entity](#) gme::GameSolo::m_playerEntity [private]

Definition at line 50 of file [GameSolo.hpp](#).

Referenced by [GameSolo\(\)](#).

18.61.4.7 m_playmusic

bool gme::GameSolo::m_playmusic = false [private]

Definition at line 54 of file [GameSolo.hpp](#).

Referenced by [playMusic\(\)](#).

18.61.4.8 m_renderer

const std::shared_ptr<[eng::IRenderer](#)>& gme::GameSolo::m_renderer [private]

Definition at line 47 of file [GameSolo.hpp](#).

18.61.4.9 m_showDebug

bool& gme::GameSolo::m_showDebug [private]

Definition at line 53 of file [GameSolo.hpp](#).

18.61.4.10 m_skinIndex

int gme::GameSolo::m_skinIndex [private]

Definition at line 51 of file [GameSolo.hpp](#).

18.61.4.11 m_stageManager

std::unique_ptr<[StageManager](#)> gme::GameSolo::m_stageManager [private]

Definition at line 55 of file [GameSolo.hpp](#).

Referenced by [GameSolo\(\)](#).

The documentation for this class was generated from the following files:

- /home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Solo/include/RTypeClient↵
Solo/Scenes/[GameSolo.hpp](#)
- /home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Solo/src/scenes/[gameSolo.cpp](#)

18.62 rnp::HandlerPacket Class Reference

Main packet handler class.

#include <HandlerPacket.hpp>

Collaboration diagram for `mp::HandlerPacket`:



Public Member Functions

- `HandlerPacket ()`=default
Constructor.
- `~HandlerPacket ()`=default
Destructor.
- `HandlerPacket (const HandlerPacket &)=delete`
- `HandlerPacket & operator= (const HandlerPacket &)=delete`

- void [setRateLimitInterval](#) (std::chrono::milliseconds interval)
Set rate limiting interval.
- void [onConnect](#) ([ConnectHandler](#) handler)
Register CONNECT packet handler.
- void [onConnectAccept](#) ([ConnectAcceptHandler](#) handler)
Register CONNECT_ACCEPT packet handler.
- void [onDisconnect](#) ([DisconnectHandler](#) handler)
Register DISCONNECT packet handler.
- void [onWorldState](#) ([WorldStateHandler](#) handler)
Register WORLD_STATE packet handler.
- void [onPing](#) ([PingHandler](#) handler)
Register PING packet handler.
- void [onPong](#) ([PongHandler](#) handler)
Register PONG packet handler.
- void [onError](#) ([ErrorHandler](#) handler)
Register ERROR packet handler.
- void [onEntityEvent](#) ([EntityEventHandler](#) handler)
Register ENTITY_EVENT packet handler.
- void [onLobbyListRequest](#) ([LobbyListRequestHandler](#) handler)
Register LOBBY_LIST_REQUEST packet handler.
- void [onLobbyListResponse](#) ([LobbyListResponseHandler](#) handler)
Register LOBBY_LIST_RESPONSE packet handler.
- void [onLobbyCreate](#) ([LobbyCreateHandler](#) handler)
Register LOBBY_CREATE packet handler.
- void [onLobbyCreateResponse](#) ([LobbyCreateResponseHandler](#) handler)
Register LOBBY_CREATE_RESPONSE packet handler.
- void [onLobbyJoin](#) ([LobbyJoinHandler](#) handler)
Register LOBBY_JOIN packet handler.
- void [onLobbyJoinResponse](#) ([LobbyJoinResponseHandler](#) handler)
Register LOBBY_JOIN_RESPONSE packet handler.
- void [onLobbyLeave](#) ([LobbyLeaveHandler](#) handler)
Register LOBBY_LEAVE packet handler.
- void [onLobbyUpdate](#) ([LobbyUpdateHandler](#) handler)
Register LOBBY_UPDATE packet handler.
- void [onGameStart](#) ([GameStartHandler](#) handler)
Register GAME_START packet handler.
- void [onStartGameRequest](#) ([StartGameRequestHandler](#) handler)
Register START_GAME_REQUEST packet handler.
- [HandlerResult](#) [processPacket](#) (const std::vector< std::uint8_t > &data, const [PacketContext](#) &context)
Process a received packet.
- const [HandlerStats](#) & [getStats](#) () const
Get handler statistics.
- void [resetStats](#) ()
Reset statistics.
- void [clearSession](#) (std::uint32_t sessionId)
Clear rate limiting data for a session.
- void [clearAllSessions](#) ()
Clear all rate limiting data.

Private Member Functions

- bool [isRateLimited](#) (std::uint32_t sessionId, const std::chrono::steady_clock::time_point ¤tTime)
Check rate limiting for a session.
- void [updateStats](#) (PacketType packetType, HandlerResult result, std::size_t bytesReceived)
Update statistics.
- std::vector< [EventRecord](#) > [parseEntityEvents](#) (Serializer &serializer, std::size_t payloadSize)
Parse and handle ENTITY_EVENT packet.

Private Attributes

- [ConnectHandler](#) connectHandler__
- [ConnectAcceptHandler](#) connectAcceptHandler__
- [DisconnectHandler](#) disconnectHandler__
- [WorldStateHandler](#) worldStateHandler__
- [PingHandler](#) pingHandler__
- [PongHandler](#) pongHandler__
- [ErrorHandler](#) errorHandler__
- [EntityEventHandler](#) entityEventHandler__
- [AckHandler](#) ackHandler__
- [LobbyListRequestHandler](#) lobbyListRequestHandler__
- [LobbyListResponseHandler](#) lobbyListResponseHandler__
- [LobbyCreateHandler](#) lobbyCreateHandler__
- [LobbyCreateResponseHandler](#) lobbyCreateResponseHandler__
- [LobbyJoinHandler](#) lobbyJoinHandler__
- [LobbyJoinResponseHandler](#) lobbyJoinResponseHandler__
- [LobbyLeaveHandler](#) lobbyLeaveHandler__
- [LobbyUpdateHandler](#) lobbyUpdateHandler__
- [GameStartHandler](#) gameStartHandler__
- [StartGameRequestHandler](#) startGameRequestHandler__
- [HandlerStats](#) stats__
- std::unordered_map< std::uint32_t, std::chrono::steady_clock::time_point > [lastPacketTime](#)__
- std::chrono::milliseconds [rateLimitInterval](#)__ {10}

18.62.1 Detailed Description

Main packet handler class.

Definition at line 92 of file [HandlerPacket.hpp](#).

18.62.2 Constructor & Destructor Documentation

18.62.2.1 HandlerPacket() [1/2]

rnpp::HandlerPacket::HandlerPacket () [default]
Constructor.

18.62.2.2 ~HandlerPacket()

rnpp::HandlerPacket::~HandlerPacket () [default]
Destructor.

18.62.2.3 HandlerPacket() [2/2]

rnpp::HandlerPacket::HandlerPacket (
const [HandlerPacket](#) &) [delete]

18.62.3 Member Function Documentation

18.62.3.1 clearAllSessions()

void rnp::HandlerPacket::clearAllSessions () [inline]

Clear all rate limiting data.

Definition at line 561 of file [HandlerPacket.hpp](#).

References [lastPacketTime__](#).

18.62.3.2 clearSession()

void rnp::HandlerPacket::clearSession (
 std::uint32_t sessionId) [inline]

Clear rate limiting data for a session.

Parameters

sessionId	Session to clear
-----------	------------------

Definition at line 556 of file [HandlerPacket.hpp](#).

References [lastPacketTime__](#).

Referenced by [srv::AsioServer::disconnectClient\(\)](#).

Here is the caller graph for this function:



18.62.3.3 getStats()

const [HandlerStats](#) & rnp::HandlerPacket::getStats () const [inline]

Get handler statistics.

Returns

Current statistics

Definition at line 545 of file [HandlerPacket.hpp](#).

References [stats__](#).

18.62.3.4 isRateLimited()

bool rnp::HandlerPacket::isRateLimited (
 std::uint32_t sessionId,
 const std::chrono::steady_clock::time_point & currentTime) [inline], [private]

Check rate limiting for a session.

Parameters

sessionId	Session to check
currentTime	Current time

Returns

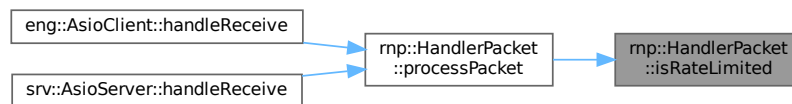
True if packet should be rate limited

Definition at line 131 of file [HandlerPacket.hpp](#).

References [lastPacketTime__](#), and [rateLimitInterval__](#).

Referenced by [processPacket\(\)](#).

Here is the caller graph for this function:

18.62.3.5 `onConnect()`

void `rnp::HandlerPacket::onConnect` (
 [ConnectHandler](#) handler) [inline]

Register CONNECT packet handler.

Parameters

handler	Callback function for CONNECT packets
---------	---------------------------------------

Definition at line 231 of file [HandlerPacket.hpp](#).

References [connectHandler__](#).

Referenced by [srv::AsioServer::setupPacketHandlers\(\)](#).

Here is the caller graph for this function:

18.62.3.6 `onConnectAccept()`

void `rnp::HandlerPacket::onConnectAccept` (
 [ConnectAcceptHandler](#) handler) [inline]

Register CONNECT_ACCEPT packet handler.

Parameters

handler	Callback function for CONNECT_ACCEPT packets
---------	--

Definition at line 237 of file [HandlerPacket.hpp](#).

References [connectAcceptHandler__](#).

Referenced by [eng::AsioClient::setupPacketHandlers\(\)](#).

Here is the caller graph for this function:



18.62.3.7 onDisconnect()

```
void rnp::HandlerPacket::onDisconnect (
    DisconnectHandler handler) [inline]
```

Register DISCONNECT packet handler.

Parameters

handler	Callback function for DISCONNECT packets
---------	--

Definition at line 243 of file [HandlerPacket.hpp](#).

References [disconnectHandler_](#).

Referenced by [eng::AsioClient::setupPacketHandlers\(\)](#), and [srv::AsioServer::setupPacketHandlers\(\)](#).

Here is the caller graph for this function:



18.62.3.8 onEntityEvent()

```
void rnp::HandlerPacket::onEntityEvent (
    EntityEventHandler handler) [inline]
```

Register ENTITY_EVENT packet handler.

Parameters

handler	Callback function for ENTITY_EVENT packets
---------	--

Definition at line 273 of file [HandlerPacket.hpp](#).

References [entityEventHandler_](#).

Referenced by [eng::AsioClient::setupPacketHandlers\(\)](#), and [srv::AsioServer::setupPacketHandlers\(\)](#).

Here is the caller graph for this function:



18.62.3.9 onError()

void rnp::HandlerPacket::onError (
 [ErrorHandler](#) handler) [inline]
 Register ERROR packet handler.

Parameters

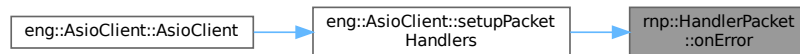
handler	Callback function for ERROR packets
---------	-------------------------------------

Definition at line 267 of file [HandlerPacket.hpp](#).

References [errorHandler_](#).

Referenced by [eng::AsioClient::setupPacketHandlers\(\)](#).

Here is the caller graph for this function:



18.62.3.10 onGameStart()

void rnp::HandlerPacket::onGameStart (
 [GameStartHandler](#) handler) [inline]
 Register GAME_START packet handler.

Parameters

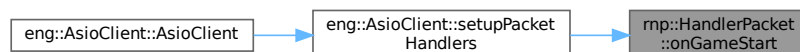
handler	Callback function for GAME_START packets
---------	--

Definition at line 336 of file [HandlerPacket.hpp](#).

References [gameStartHandler_](#).

Referenced by [eng::AsioClient::setupPacketHandlers\(\)](#).

Here is the caller graph for this function:



18.62.3.11 onLobbyCreate()

void rnp::HandlerPacket::onLobbyCreate (
[LobbyCreateHandler](#) handler) [inline]

Register LOBBY_CREATE packet handler.

Parameters

handler	Callback function for LOBBY_CREATE packets
---------	--

Definition at line 294 of file [HandlerPacket.hpp](#).

References [lobbyCreateHandler_](#).

Referenced by [srv::AsioServer::setupPacketHandlers\(\)](#).

Here is the caller graph for this function:



18.62.3.12 onLobbyCreateResponse()

void rnp::HandlerPacket::onLobbyCreateResponse (
[LobbyCreateResponseHandler](#) handler) [inline]

Register LOBBY_CREATE_RESPONSE packet handler.

Parameters

handler	Callback function for LOBBY_CREATE_RESPONSE packets
---------	---

Definition at line 300 of file [HandlerPacket.hpp](#).

References [lobbyCreateResponseHandler_](#).

Referenced by [eng::AsioClient::setupPacketHandlers\(\)](#).

Here is the caller graph for this function:



18.62.3.13 onLobbyJoin()

void rnp::HandlerPacket::onLobbyJoin (
[LobbyJoinHandler](#) handler) [inline]

Register LOBBY_JOIN packet handler.

Parameters

handler	Callback function for LOBBY_JOIN packets
---------	--

Definition at line 309 of file [HandlerPacket.hpp](#).

References [lobbyJoinHandler_](#).

Referenced by [srv::AsioServer::setupPacketHandlers\(\)](#).

Here is the caller graph for this function:



18.62.3.14 onLobbyJoinResponse()

```
void rnp::HandlerPacket::onLobbyJoinResponse (
    LobbyJoinResponseHandler handler) [inline]
```

Register LOBBY_JOIN_RESPONSE packet handler.

Parameters

handler	Callback function for LOBBY_JOIN_RESPONSE packets
---------	---

Definition at line 315 of file [HandlerPacket.hpp](#).

References [lobbyJoinResponseHandler_](#).

Referenced by [eng::AsioClient::setupPacketHandlers\(\)](#).

Here is the caller graph for this function:



18.62.3.15 onLobbyLeave()

```
void rnp::HandlerPacket::onLobbyLeave (
    LobbyLeaveHandler handler) [inline]
```

Register LOBBY_LEAVE packet handler.

Parameters

handler	Callback function for LOBBY_LEAVE packets
---------	---

Definition at line 324 of file [HandlerPacket.hpp](#).

References [lobbyLeaveHandler_](#).

Referenced by [srv::AsioServer::setupPacketHandlers\(\)](#).

Here is the caller graph for this function:



18.62.3.16 onLobbyListRequest()

void rnp::HandlerPacket::onLobbyListRequest (
[LobbyListRequestHandler](#) handler) [inline]
 Register LOBBY_LIST_REQUEST packet handler.

Parameters

handler	Callback function for LOBBY_LIST_REQUEST packets
---------	--

Definition at line 279 of file [HandlerPacket.hpp](#).

References [lobbyListRequestHandler_](#).

Referenced by [srv::AsioServer::setupPacketHandlers\(\)](#).

Here is the caller graph for this function:



18.62.3.17 onLobbyListResponse()

void rnp::HandlerPacket::onLobbyListResponse (
[LobbyListResponseHandler](#) handler) [inline]
 Register LOBBY_LIST_RESPONSE packet handler.

Parameters

handler	Callback function for LOBBY_LIST_RESPONSE packets
---------	---

Definition at line 285 of file [HandlerPacket.hpp](#).

References [lobbyListResponseHandler_](#).

Referenced by [eng::AsioClient::setupPacketHandlers\(\)](#).

Here is the caller graph for this function:



18.62.3.18 onLobbyUpdate()

void rnp::HandlerPacket::onLobbyUpdate (
[LobbyUpdateHandler](#) handler) [inline]
 Register LOBBY_UPDATE packet handler.

Parameters

handler	Callback function for LOBBY_UPDATE packets
---------	--

Definition at line 330 of file [HandlerPacket.hpp](#).

References [lobbyUpdateHandler_](#).

Referenced by [eng::AsioClient::setupPacketHandlers\(\)](#).

Here is the caller graph for this function:



18.62.3.19 onPing()

```
void rnp::HandlerPacket::onPing (
    PingHandler handler) [inline]
```

Register PING packet handler.

Parameters

handler	Callback function for PING packets
---------	------------------------------------

Definition at line 255 of file [HandlerPacket.hpp](#).

References [pingHandler_](#).

Referenced by [eng::AsioClient::setupPacketHandlers\(\)](#), and [srv::AsioServer::setupPacketHandlers\(\)](#).

Here is the caller graph for this function:



18.62.3.20 onPong()

```
void rnp::HandlerPacket::onPong (
    PongHandler handler) [inline]
```

Register PONG packet handler.

Parameters

handler	Callback function for PONG packets
---------	------------------------------------

Definition at line 261 of file [HandlerPacket.hpp](#).

References [pongHandler_](#).

Referenced by [eng::AsioClient::setupPacketHandlers\(\)](#), and [srv::AsioServer::setupPacketHandlers\(\)](#).

Here is the caller graph for this function:



18.62.3.21 onStartGameRequest()

void rnp::HandlerPacket::onStartGameRequest (
[StartGameRequestHandler](#) handler) [inline]
 Register START_GAME_REQUEST packet handler.

Parameters

handler	Callback function for START_GAME_REQUEST packets
---------	--

Definition at line 342 of file [HandlerPacket.hpp](#).

References [startGameRequestHandler_](#).

Referenced by [srv::AsioServer::setupPacketHandlers\(\)](#).

Here is the caller graph for this function:



18.62.3.22 onWorldState()

void rnp::HandlerPacket::onWorldState (
[WorldStateHandler](#) handler) [inline]
 Register WORLD_STATE packet handler.

Parameters

handler	Callback function for WORLD_STATE packets
---------	---

Definition at line 249 of file [HandlerPacket.hpp](#).

References [worldStateHandler_](#).

Referenced by [eng::AsioClient::setupPacketHandlers\(\)](#).

Here is the caller graph for this function:



18.62.3.23 operator=()

[HandlerPacket](#) & rnp::HandlerPacket::operator= (
const [HandlerPacket](#) &) [delete]

18.62.3.24 parseEntityEvents()

std::vector< [EventRecord](#) > rnp::HandlerPacket::parseEntityEvents (
 [Serializer](#) & serializer,
 std::size_t payloadSize) [inline], [private]

Parse and handle ENTITY_EVENT packet.

Parameters

serializer	Serializer containing the packet data
payloadSize	Size of the payload

Returns

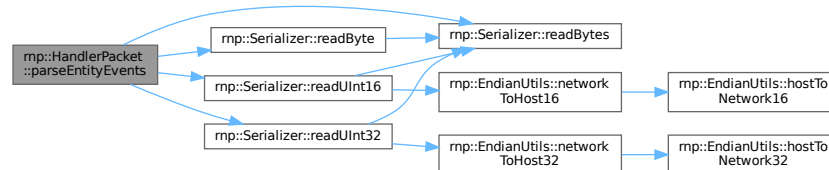
Vector of event records

Definition at line 179 of file [HandlerPacket.hpp](#).

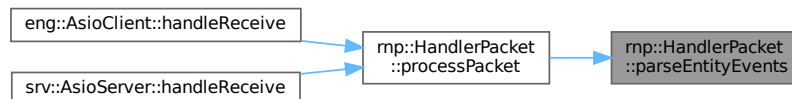
References [rnp::EventRecord::data](#), [rnp::Serializer::readByte\(\)](#), [rnp::Serializer::readBytes\(\)](#), [rnp::Serializer::readUInt16\(\)](#), [rnp::Serializer::readUInt32\(\)](#), and [rnp::EventRecord::type](#).

Referenced by [processPacket\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.62.3.25 processPacket()

[HandlerResult](#) rnp::HandlerPacket::processPacket (
const std::vector< std::uint8_t > & data,
const [PacketContext](#) & context) [inline]

Process a received packet.

Parameters

data	Raw packet data
context	Packet context information

Returns

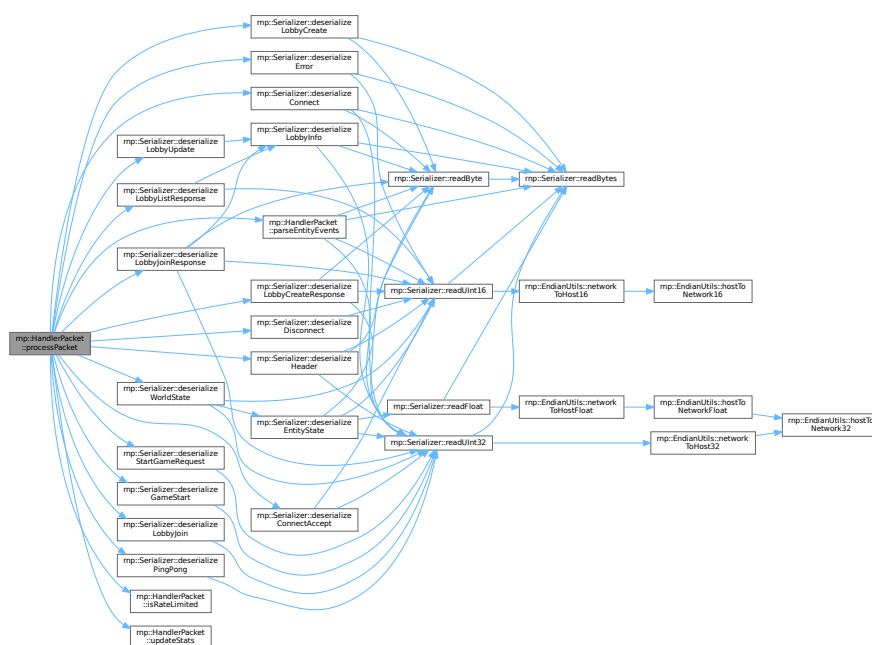
Processing result

Definition at line 350 of file [HandlerPacket.hpp](#).

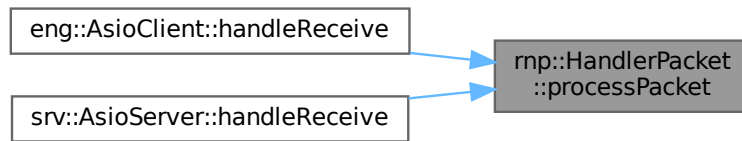
References [rnp::CONNECT](#), [rnp::CONNECT_ACCEPT](#), [connectAcceptHandler_](#), [connectHandler_](#), [rnp::Serializer::deserializeConnect\(\)](#), [rnp::Serializer::deserializeConnectAccept\(\)](#), [rnp::Serializer::deserializeDisconnect\(\)](#), [rnp::Serializer::deserializeError\(\)](#), [rnp::Serializer::deserializeGameStart\(\)](#), [rnp::Serializer::deserializeHeader\(\)](#), [rnp::Serializer::deserializeLobbyCreate\(\)](#), [rnp::Serializer::deserializeLobbyCreateResponse\(\)](#), [rnp::Serializer::deserializeLobbyJoinResponse\(\)](#), [rnp::Serializer::deserializeLobbyListResponse\(\)](#), [rnp::Serializer::deserializeLobbyUpdate\(\)](#), [rnp::Serializer::deserializePingPong\(\)](#), [rnp::Serializer::deserializeStartGameRequest\(\)](#), [rnp::Serializer::deserializeWorldState\(\)](#), [rnp::DISCONNECT](#), [disconnectHandler_](#), [rnp::ENTITY_EVENT](#), [entityEventHandler_](#), [errorHandler_](#), [rnp::GAME_START](#), [gameStartHandler_](#), [rnp::INVALID_PACKET](#), [isRateLimited\(\)](#), [rnp::PacketHeader::length](#), [rnp::LOBBY_CREATE](#), [rnp::LOBBY_CREATE_RESPONSE](#), [rnp::LOBBY_JOIN](#), [rnp::LOBBY_JOIN_RESPONSE](#), [rnp::LOBBY_LEAVE](#), [rnp::LOBBY_LIST_REQUEST](#), [rnp::LOBBY_LIST_RESPONSE](#), [rnp::LOBBY_UPDATE](#), [lobbyCreateHandler_](#), [lobbyCreateResponseHandler_](#), [lobbyJoinHandler_](#), [lobbyJoinResponseHandler_](#), [lobbyLeaveHandler_](#), [lobbyListRequestHandler_](#), [lobbyListResponseHandler_](#), [lobbyUpdateHandler_](#), [rnp::MAX_PAYLOAD](#), [rnp::PACKET_ERROR](#), [parseEntityEvents\(\)](#), [rnp::PING](#), [pingHandler_](#), [rnp::PONG](#), [pongHandler_](#), [rnp::PROCESSING_ERROR](#), [rnp::RATE_LIMITED](#), [rnp::PacketContext::receiveTime](#), [rnp::PacketContext::sessionId](#), [rnp::PacketHeader::sessionId](#), [rnp::START_GAME_REQUEST](#), [startGameRequestHandler_](#), [rnp::PacketHeader::type](#), [rnp::UNSUPPORTED_TYPE](#), [updateStats\(\)](#), [rnp::WORLD_STATE](#), and [worldStateHandler_](#).

Referenced by [eng::AsioClient::handleReceive\(\)](#), and [srv::AsioServer::handleReceive\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.62.3.26 resetStats()

void rnp::HandlerPacket::resetStats () [inline]

Reset statistics.

Definition at line 550 of file [HandlerPacket.hpp](#).

References [stats_](#).

18.62.3.27 setRateLimitInterval()

void rnp::HandlerPacket::setRateLimitInterval (
std::chrono::milliseconds interval) [inline]

Set rate limiting interval.

Parameters

interval	Minimum time between packets per session
----------	--

Definition at line 225 of file [HandlerPacket.hpp](#).

References [rateLimitInterval_](#).

18.62.3.28 updateStats()

void rnp::HandlerPacket::updateStats (
PacketType packetType,
HandlerResult result,
std::size_t bytesReceived) [inline], [private]

Update statistics.

Parameters

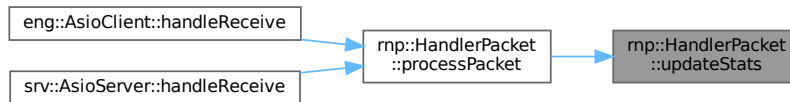
packetType	Type of packet processed
result	Processing result
bytesReceived	Number of bytes in the packet

Definition at line 156 of file [HandlerPacket.hpp](#).

References [rnp::HandlerStats::packetTypeCount](#), [rnp::HandlerStats::resultCount](#), [stats_](#), [rnp::SUCCESS](#), [rnp::HandlerStats::totalBytesReceived](#), [rnp::HandlerStats::totalPacketsDropped](#), [rnp::HandlerStats::totalPacketsProcessed](#) and [rnp::HandlerStats::totalPacketsReceived](#).

Referenced by [processPacket\(\)](#).

Here is the caller graph for this function:



18.62.4 Member Data Documentation

18.62.4.1 ackHandler__

[AckHandler](#) rnp::HandlerPacket::ackHandler__ [private]

Definition at line 104 of file [HandlerPacket.hpp](#).

18.62.4.2 connectAcceptHandler__

[ConnectAcceptHandler](#) rnp::HandlerPacket::connectAcceptHandler__ [private]

Definition at line 97 of file [HandlerPacket.hpp](#).

Referenced by [onConnectAccept\(\)](#), and [processPacket\(\)](#).

18.62.4.3 connectHandler__

[ConnectHandler](#) rnp::HandlerPacket::connectHandler__ [private]

Definition at line 96 of file [HandlerPacket.hpp](#).

Referenced by [onConnect\(\)](#), and [processPacket\(\)](#).

18.62.4.4 disconnectHandler__

[DisconnectHandler](#) rnp::HandlerPacket::disconnectHandler__ [private]

Definition at line 98 of file [HandlerPacket.hpp](#).

Referenced by [onDisconnect\(\)](#), and [processPacket\(\)](#).

18.62.4.5 entityEventHandler__

[EntityEventHandler](#) rnp::HandlerPacket::entityEventHandler__ [private]

Definition at line 103 of file [HandlerPacket.hpp](#).

Referenced by [onEntityEvent\(\)](#), and [processPacket\(\)](#).

18.62.4.6 errorHandler__

[ErrorHandler](#) rnp::HandlerPacket::errorHandler__ [private]

Definition at line 102 of file [HandlerPacket.hpp](#).

Referenced by [onError\(\)](#), and [processPacket\(\)](#).

18.62.4.7 gameStartHandler__

[GameStartHandler](#) rnp::HandlerPacket::gameStartHandler__ [private]

Definition at line 115 of file [HandlerPacket.hpp](#).

Referenced by [onGameStart\(\)](#), and [processPacket\(\)](#).

18.62.4.8 lastPacketTime__

`std::unordered_map<std::uint32_t, std::chrono::steady_clock::time_point>` rnp::HandlerPacket::lastPacketTime__ [private]

Definition at line 122 of file [HandlerPacket.hpp](#).

Referenced by [clearAllSessions\(\)](#), [clearSession\(\)](#), and [isRateLimited\(\)](#).

18.62.4.9 lobbyCreateHandler__

[LobbyCreateHandler](#) rnp::HandlerPacket::lobbyCreateHandler__ [private]

Definition at line 109 of file [HandlerPacket.hpp](#).

Referenced by [onLobbyCreate\(\)](#), and [processPacket\(\)](#).

18.62.4.10 lobbyCreateResponseHandler__

[LobbyCreateResponseHandler](#) rnp::HandlerPacket::lobbyCreateResponseHandler__ [private]

Definition at line 110 of file [HandlerPacket.hpp](#).

Referenced by [onLobbyCreateResponse\(\)](#), and [processPacket\(\)](#).

18.62.4.11 lobbyJoinHandler__

[LobbyJoinHandler](#) rnp::HandlerPacket::lobbyJoinHandler__ [private]

Definition at line 111 of file [HandlerPacket.hpp](#).

Referenced by [onLobbyJoin\(\)](#), and [processPacket\(\)](#).

18.62.4.12 lobbyJoinResponseHandler__

[LobbyJoinResponseHandler](#) rnp::HandlerPacket::lobbyJoinResponseHandler__ [private]

Definition at line 112 of file [HandlerPacket.hpp](#).

Referenced by [onLobbyJoinResponse\(\)](#), and [processPacket\(\)](#).

18.62.4.13 lobbyLeaveHandler__

[LobbyLeaveHandler](#) rnp::HandlerPacket::lobbyLeaveHandler__ [private]

Definition at line 113 of file [HandlerPacket.hpp](#).

Referenced by [onLobbyLeave\(\)](#), and [processPacket\(\)](#).

18.62.4.14 lobbyListRequestHandler__

[LobbyListRequestHandler](#) rnp::HandlerPacket::lobbyListRequestHandler__ [private]

Definition at line 107 of file [HandlerPacket.hpp](#).

Referenced by [onLobbyListRequest\(\)](#), and [processPacket\(\)](#).

18.62.4.15 lobbyListResponseHandler__

[LobbyListResponseHandler](#) rnp::HandlerPacket::lobbyListResponseHandler__ [private]

Definition at line 108 of file [HandlerPacket.hpp](#).

Referenced by [onLobbyListResponse\(\)](#), and [processPacket\(\)](#).

18.62.4.16 lobbyUpdateHandler__

[LobbyUpdateHandler](#) rnp::HandlerPacket::lobbyUpdateHandler__ [private]

Definition at line 114 of file [HandlerPacket.hpp](#).

Referenced by [onLobbyUpdate\(\)](#), and [processPacket\(\)](#).

18.62.4.17 pingHandler__

[PingHandler](#) rnp::HandlerPacket::pingHandler__ [private]

Definition at line 100 of file [HandlerPacket.hpp](#).

Referenced by [onPing\(\)](#), and [processPacket\(\)](#).

18.62.4.18 pongHandler__

[PongHandler](#) rnp::HandlerPacket::pongHandler__ [private]

Definition at line 101 of file [HandlerPacket.hpp](#).

Referenced by [onPong\(\)](#), and [processPacket\(\)](#).

18.62.4.19 rateLimitInterval__

std::chrono::milliseconds rnp::HandlerPacket::rateLimitInterval__ {10} [private]

Definition at line 123 of file [HandlerPacket.hpp](#).

Referenced by [isRateLimited\(\)](#), and [setRateLimitInterval\(\)](#).

18.62.4.20 startGameRequestHandler__

[StartGameRequestHandler](#) rnp::HandlerPacket::startGameRequestHandler__ [private]

Definition at line 116 of file [HandlerPacket.hpp](#).

Referenced by [onStartGameRequest\(\)](#), and [processPacket\(\)](#).

18.62.4.21 stats__

[HandlerStats](#) rnp::HandlerPacket::stats__ [private]

Definition at line 119 of file [HandlerPacket.hpp](#).

Referenced by [getStats\(\)](#), [resetStats\(\)](#), and [updateStats\(\)](#).

18.62.4.22 worldStateHandler__

[WorldStateHandler](#) rnp::HandlerPacket::worldStateHandler__ [private]

Definition at line 99 of file [HandlerPacket.hpp](#).

Referenced by [onWorldState\(\)](#), and [processPacket\(\)](#).

The documentation for this class was generated from the following file:

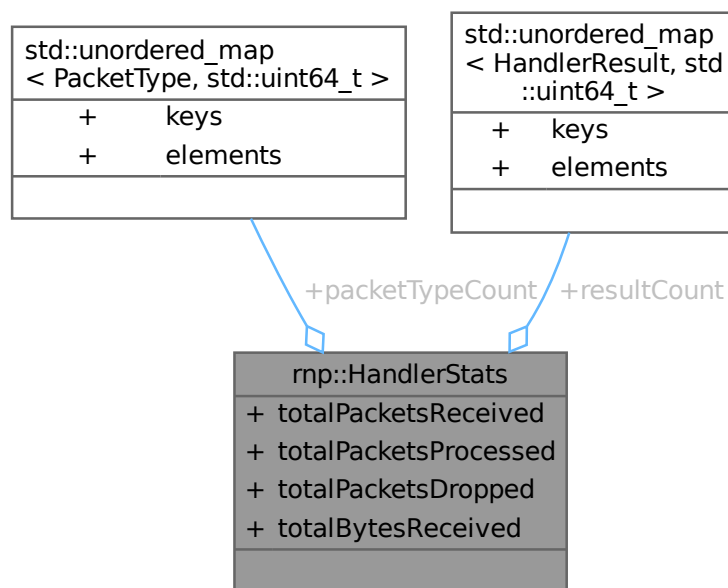
- [/home/masina/Projects/Epitech/rtype/modules/Interfaces/include/Interfaces/Protocol/HandlerPacket.hpp](#)

18.63 rnp::HandlerStats Struct Reference

Statistics for packet handling.

#include <HandlerPacket.hpp>

Collaboration diagram for rnp::HandlerStats:



Public Attributes

- `std::uint64_t totalPacketsReceived = 0`
- `std::uint64_t totalPacketsProcessed = 0`
- `std::uint64_t totalPacketsDropped = 0`
- `std::uint64_t totalBytesReceived = 0`
- `std::unordered_map< PacketType, std::uint64_t > packetTypeCount`
- `std::unordered_map< HandlerResult, std::uint64_t > resultCount`

18.63.1 Detailed Description

Statistics for packet handling.

Definition at line 79 of file [HandlerPacket.hpp](#).

18.63.2 Member Data Documentation

18.63.2.1 packetTypeCount

`std::unordered_map<PacketType, std::uint64_t> rnp::HandlerStats::packetTypeCount`

Definition at line 85 of file [HandlerPacket.hpp](#).

Referenced by [rnp::HandlerPacket::updateStats\(\)](#).

18.63.2.2 resultCount

`std::unordered_map<HandlerResult, std::uint64_t> rnp::HandlerStats::resultCount`

Definition at line 86 of file [HandlerPacket.hpp](#).

Referenced by [rnp::HandlerPacket::updateStats\(\)](#).

18.63.2.3 totalBytesReceived

`std::uint64_t rnp::HandlerStats::totalBytesReceived = 0`

Definition at line 84 of file [HandlerPacket.hpp](#).

Referenced by [rnp::HandlerPacket::updateStats\(\)](#).

18.63.2.4 totalPacketsDropped

`std::uint64_t rnp::HandlerStats::totalPacketsDropped = 0`

Definition at line 83 of file [HandlerPacket.hpp](#).

Referenced by [rnp::HandlerPacket::updateStats\(\)](#).

18.63.2.5 totalPacketsProcessed

`std::uint64_t rnp::HandlerStats::totalPacketsProcessed = 0`

Definition at line 82 of file [HandlerPacket.hpp](#).

Referenced by [rnp::HandlerPacket::updateStats\(\)](#).

18.63.2.6 totalPacketsReceived

`std::uint64_t rnp::HandlerStats::totalPacketsReceived = 0`

Definition at line 81 of file [HandlerPacket.hpp](#).

Referenced by [rnp::HandlerPacket::updateStats\(\)](#).

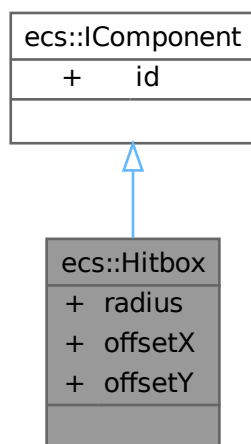
The documentation for this struct was generated from the following file:

- [/home/masina/Projects/Epitech/rtype/modules/Interfaces/include/Interfaces/Protocol/HandlerPacket.hpp](#)

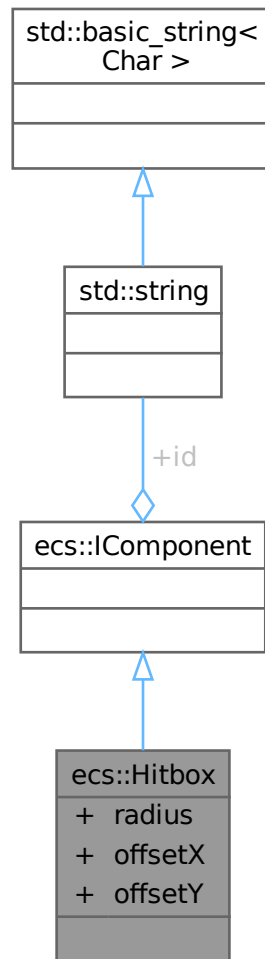
18.64 ecs::Hitbox Struct Reference

`#include <Component.hpp>`

Inheritance diagram for ecs::Hitbox:



Collaboration diagram for `ecs::Hitbox`:



Public Attributes

- float [radius](#)
- float [offsetX](#) = 0.0f
- float [offsetY](#) = 0.0f

Public Attributes inherited from [ecs::IComponent](#)

- `std::string` [id](#)

18.64.1 Detailed Description

Definition at line [136](#) of file [Component.hpp](#).

18.64.2 Member Data Documentation

18.64.2.1 `offsetX`

float `ecs::Hitbox::offsetX` = 0.0f

Definition at line 139 of file [Component.hpp](#).

Referenced by [gme::CollisionSystem::checkCircularCollision\(\)](#).

18.64.2.2 offsetY

float ecs::Hitbox::offsetY = 0.0f

Definition at line 140 of file [Component.hpp](#).

Referenced by [gme::CollisionSystem::checkCircularCollision\(\)](#).

18.64.2.3 radius

float ecs::Hitbox::radius

Definition at line 138 of file [Component.hpp](#).

Referenced by [gme::CollisionSystem::checkCircularCollision\(\)](#).

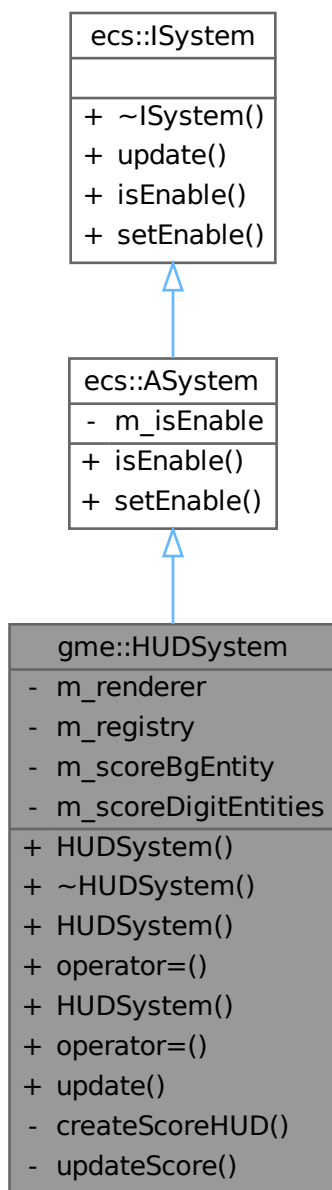
The documentation for this struct was generated from the following file:

- [/home/masina/Projects/Epitech/rtype/modules/ECS/include/ECS/Component.hpp](#)

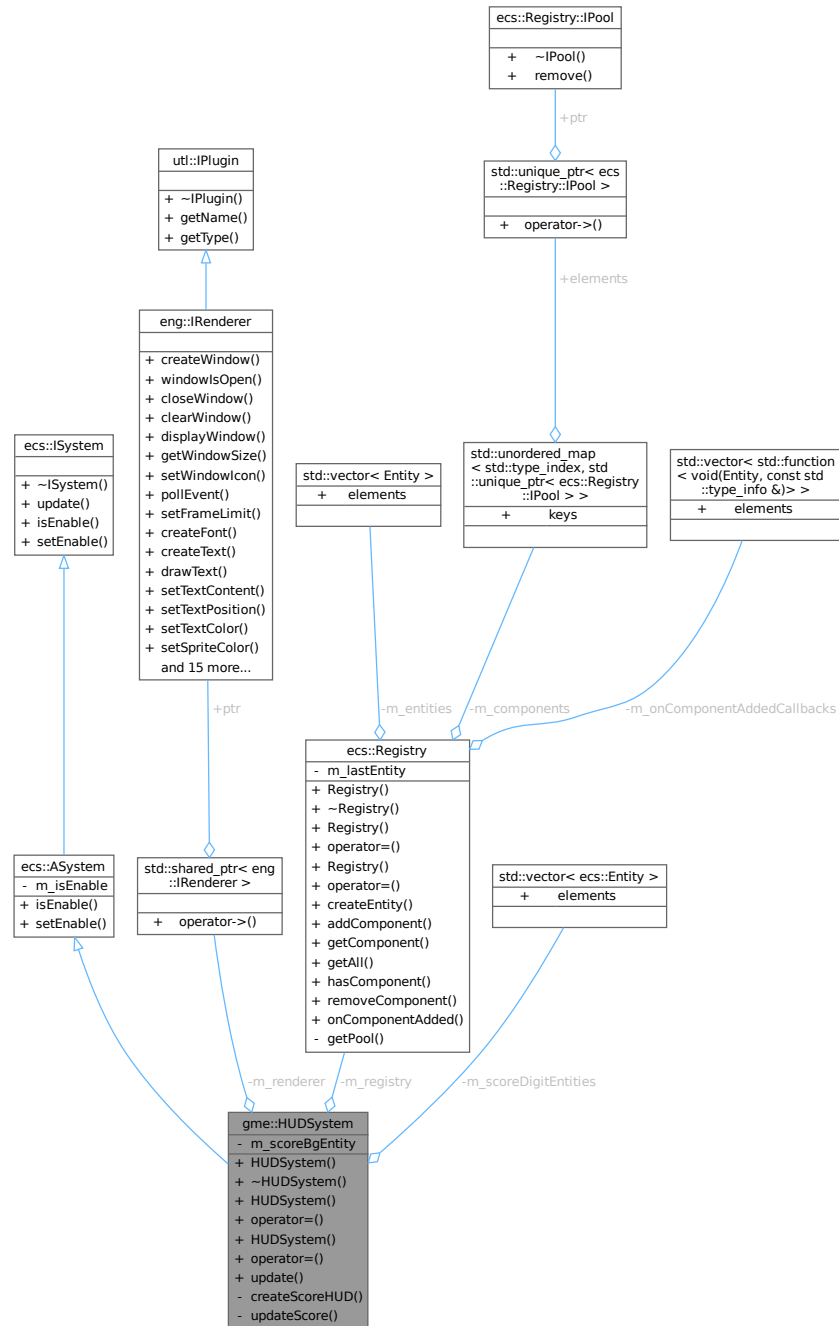
18.65 gme::HUDSystem Class Reference

```
#include <HUD.hpp>
```

Inheritance diagram for gme::HUDSystem:



Collaboration diagram for gme::HUDSystem:



Public Member Functions

- `HUDSystem` (`const std::shared_ptr< eng::IRenderer > &renderer`, `ecs::Registry ®istry`)
- `~HUDSystem` () override=default
- `HUDSystem` (`const HUDSystem &`)=delete
- `HUDSystem & operator=` (`const HUDSystem &`)=delete
- `HUDSystem` (`HUDSystem &&`)=delete
- `HUDSystem & operator=` (`HUDSystem &&`)=delete
- `void update` (`ecs::Registry ®istry`, `float`) override

Public Member Functions inherited from [ecs::ASystem](#)

- bool [isEnabled](#) () override
- void [setEnabled](#) (const bool enable) override

Public Member Functions inherited from [ecs::ISystem](#)

- virtual [~ISystem](#) ()=default

Private Member Functions

- void [createScoreHUD](#) ([ecs::Registry](#) ®istry, float x, float y)
- void [updateScore](#) ([ecs::Registry](#) ®istry, int newScore) const

Private Attributes

- const std::shared_ptr< [eng::IRenderer](#) > & [m_renderer](#)
- [ecs::Registry](#) & [m_registry](#)
- [ecs::Entity](#) [m_scoreBgEntity](#) {}
- std::vector< [ecs::Entity](#) > [m_scoreDigitEntities](#)

18.65.1 Detailed Description

Definition at line 16 of file [HUD.hpp](#).

18.65.2 Constructor & Destructor Documentation

18.65.2.1 HUDSystem() [1/3]

```
gme::HUDSystem::HUDSystem (
    const std::shared_ptr< eng::IRenderer > & renderer,
    ecs::Registry & registry)    [inline], [explicit]
```

Definition at line 19 of file [HUD.hpp](#).

References [createScoreHUD\(\)](#), and [m_registry](#).

Here is the call graph for this function:



18.65.2.2 ~HUDSystem()

```
gme::HUDSystem::~~HUDSystem ()    [override], [default]
```

18.65.2.3 HUDSystem() [2/3]

```
gme::HUDSystem::HUDSystem (
    const HUDSystem & )    [delete]
```

18.65.2.4 HUDSystem() [3/3]

```
gme::HUDSystem::HUDSystem (
    HUDSystem && )    [delete]
```


18.65.3 Member Function Documentation

18.65.3.1 createScoreHUD()

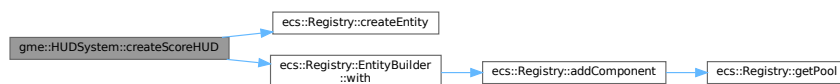
```
void gme::HUDSystem::createScoreHUD (
    ecs::Registry & registry,
    float x,
    float y) [private]
```

Definition at line 7 of file HUD.cpp.

References [ecs::Registry::createEntity\(\)](#), [m_scoreBgEntity](#), [m_scoreDigitEntities](#), [utl::Path::Texture::TEXTURE_SCORE_BACKGROUND](#), [utl::Path::Texture::TEXTURE_SCORE_DIGIT_0](#), and [ecs::Registry::EntityBuilder::with\(\)](#).

Referenced by [HUDSystem\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.65.3.2 operator=() [1/2]

```
HUDSystem & gme::HUDSystem::operator= (
    const HUDSystem & ) [delete]
```

18.65.3.3 operator=() [2/2]

```
HUDSystem & gme::HUDSystem::operator= (
    HUDSystem && ) [delete]
```

18.65.3.4 update()

```
void gme::HUDSystem::update (
    ecs::Registry & registry,
    float ) [override], [virtual]
```

Implements [ecs::ISystem](#).

Definition at line 63 of file HUD.cpp.

References [ecs::Registry::getAll\(\)](#).

Here is the call graph for this function:



18.65.3.5 updateScore()

```
void gme::HUDSystem::updateScore (
    ecs::Registry & registry,
    int newScore) const [private]
```

Definition at line 29 of file [HUD.cpp](#).

References [ecs::Registry::getComponent\(\)](#), [utl::Path::Texture::TEXTURE_SCORE_DIGIT_0](#), [utl::Path::Texture::TEXTURE_SCORE_DIGIT_1](#), [utl::Path::Texture::TEXTURE_SCORE_DIGIT_2](#), [utl::Path::Texture::TEXTURE_SCORE_DIGIT_3](#), [utl::Path::Texture::TEXTURE_SCORE_DIGIT_4](#), [utl::Path::Texture::TEXTURE_SCORE_DIGIT_5](#), [utl::Path::Texture::TEXTURE_SCORE_DIGIT_6](#), [utl::Path::Texture::TEXTURE_SCORE_DIGIT_7](#), [utl::Path::Texture::TEXTURE_SCORE_DIGIT_8](#), [utl::Path::Texture::TEXTURE_SCORE_DIGIT_9](#), and [ecs::Transform::x](#).

Here is the call graph for this function:



18.65.4 Member Data Documentation

18.65.4.1 m_registry

```
ecs::Registry& gme::HUDSystem::m_registry [private]
```

Definition at line 38 of file [HUD.hpp](#).

Referenced by [HUDSystem\(\)](#).

18.65.4.2 m_renderer

```
const std::shared_ptr<eng::IRenderer>& gme::HUDSystem::m_renderer [private]
```

Definition at line 37 of file [HUD.hpp](#).

18.65.4.3 m_scoreBgEntity

```
ecs::Entity gme::HUDSystem::m_scoreBgEntity {} [private]
```

Definition at line 39 of file [HUD.hpp](#).

Referenced by [createScoreHUD\(\)](#).

18.65.4.4 m_scoreDigitEntities

```
std::vector<ecs::Entity> gme::HUDSystem::m_scoreDigitEntities [private]
```

Definition at line 40 of file [HUD.hpp](#).

Referenced by [createScoreHUD\(\)](#).

The documentation for this class was generated from the following files:

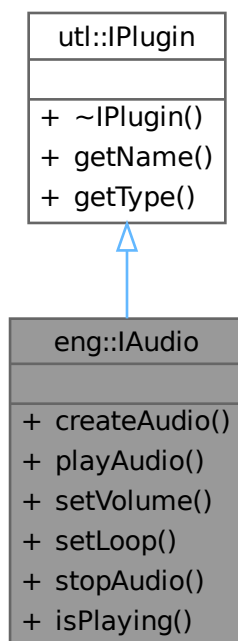
- [/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Solo/include/RTypeClientSolo/Systems/HUD.hpp](#)
- [/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Solo/src/systems/HUD.cpp](#)

18.66 eng::IAudio Interface Reference

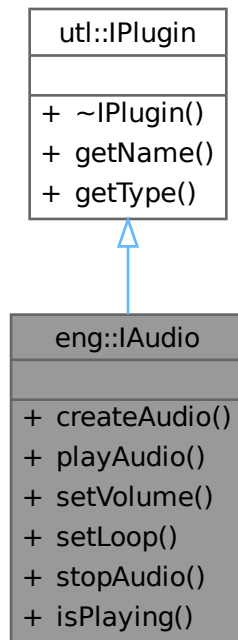
Interface for the audio.

```
#include <IAudio.hpp>
```

Inheritance diagram for eng::IAudio:



Collaboration diagram for `eng::IAudio`:



Public Member Functions

- virtual void `createAudio` (const std::string &path, float volume, bool loop, const std::string &name)=0
- virtual void `playAudio` (const std::string &name)=0
- virtual void `setVolume` (const std::string &name, float volume)=0
- virtual void `setLoop` (const std::string &name, bool loop)=0
- virtual void `stopAudio` (const std::string &name)=0
- virtual `Status` `isPlaying` (const std::string &name)=0

Public Member Functions inherited from `utl::IPlugin`

- virtual `~IPlugin` ()=default
- virtual const std::string `getName` () const =0
- virtual `PluginType` `getType` () const =0

18.66.1 Detailed Description

Interface for the audio.

Definition at line 28 of file `IAudio.hpp`.

18.66.2 Member Function Documentation

18.66.2.1 `createAudio()`

```
virtual void eng::IAudio::createAudio (
    const std::string & path,
    float volume,
```

```
bool loop,  
const std::string & name) [pure virtual]
```

18.66.2.2 isPlaying()

```
virtual Status eng::IAudio::isPlaying (  
    const std::string & name) [pure virtual]
```

Referenced by [ecs::AudioSystem::update\(\)](#).

Here is the caller graph for this function:



18.66.2.3 playAudio()

```
virtual void eng::IAudio::playAudio (  
    const std::string & name) [pure virtual]
```

Referenced by [ecs::AudioSystem::update\(\)](#).

Here is the caller graph for this function:

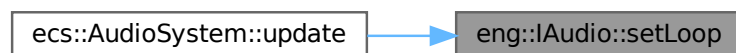


18.66.2.4 setLoop()

```
virtual void eng::IAudio::setLoop (  
    const std::string & name,  
    bool loop) [pure virtual]
```

Referenced by [ecs::AudioSystem::update\(\)](#).

Here is the caller graph for this function:



18.66.2.5 setVolume()

```
virtual void eng::IAudio::setVolume (
    const std::string & name,
    float volume) [pure virtual]
```

Referenced by [ecs::AudioSystem::update\(\)](#).

Here is the caller graph for this function:



18.66.2.6 stopAudio()

```
virtual void eng::IAudio::stopAudio (
    const std::string & name) [pure virtual]
```

Referenced by [ecs::AudioSystem::update\(\)](#).

Here is the caller graph for this function:



The documentation for this interface was generated from the following file:

- `/home/masina/Projects/Epitech/rtype/modules/Interfaces/include/Interfaces/IAudio.hpp`

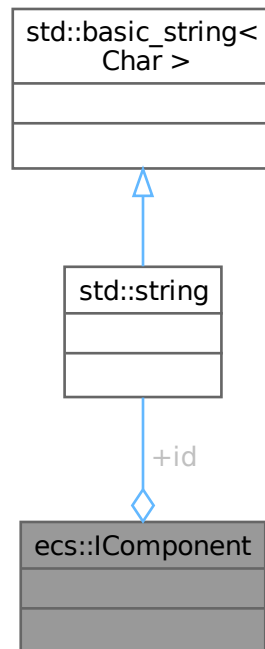
18.67 ecs::IComponent Struct Reference

```
#include <Component.hpp>
```

Inheritance diagram for `ecs::IComponent`:



Collaboration diagram for `ecs::IComponent`:



Public Attributes

- `std::string` [id](#)

18.67.1 Detailed Description

Definition at line 13 of file [Component.hpp](#).

18.67.2 Member Data Documentation

18.67.2.1 id

`std::string` `ecs::IComponent::id`

Definition at line 15 of file [Component.hpp](#).

Referenced by [gme::ConfigMulti::ConfigMulti\(\)](#), [gme::ConfigSolo::ConfigSolo\(\)](#), [gme::CreateRoomScene::CreateRoomScene\(\)](#), [gme::Game::Game\(\)](#), [gme::GameMulti::GameMulti\(\)](#), [gme::GameSolo::GameSolo\(\)](#), [cli::Intro::Intro\(\)](#), [gme::JoinRoomScene::JoinRoomScene\(\)](#), [cli::Menu::Menu\(\)](#), [gme::ServerScene::ServerScene\(\)](#), [cli::Settings::Settings\(\)](#), [ecs::StarfieldSystem::update\(\)](#), [gme::EnemySystem::update\(\)](#), [gme::ExplosionSystem::update\(\)](#), [gme::LoadingAnimationSystem::update\(\)](#), and [gme::WaitingRoomScene::WaitingRoomScene\(\)](#).

The documentation for this struct was generated from the following file:

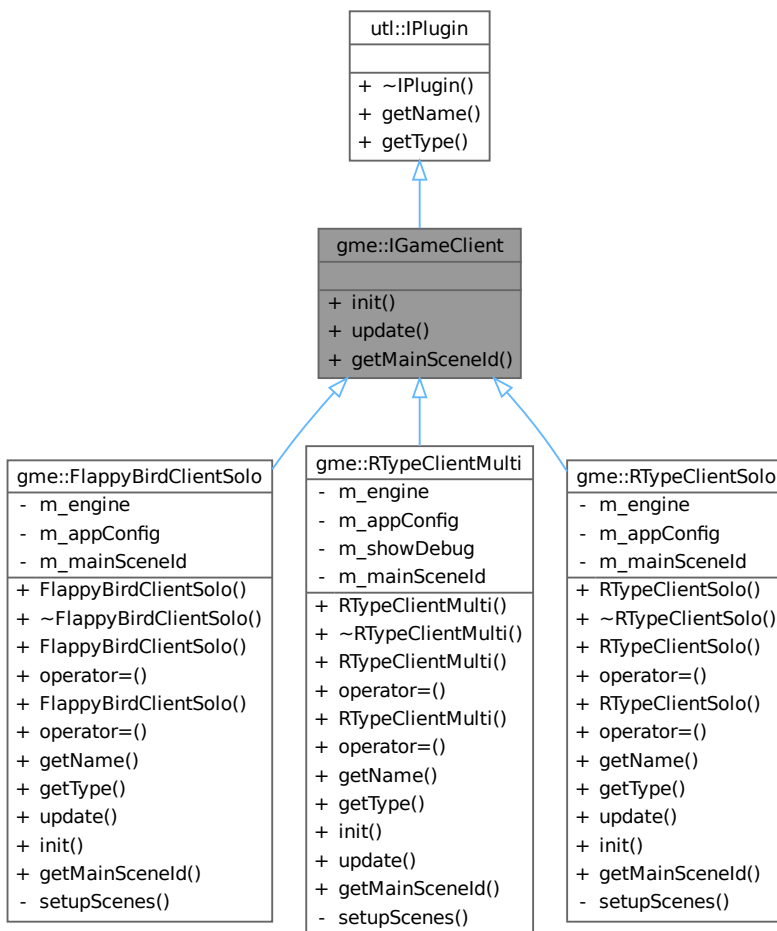
- [/home/masina/Projects/Epitech/rtype/modules/ECS/include/ECS/Component.hpp](#)

18.68 gme::IGameClient Interface Reference

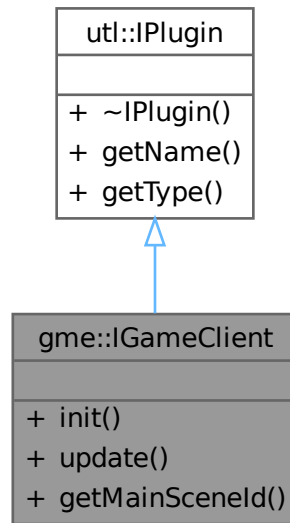
Interface for the games.

```
#include <IGameClient.hpp>
```

Inheritance diagram for gme::IGameClient:



Collaboration diagram for gme::IGameClient:



Public Member Functions

- virtual void `init` ([eng::Engine](#) &engine, [utl::cli::AppConfig](#) &appConfig, bool &showDebug, unsigned int menuSceneId)=0
- virtual void `update` (float deltaTime, unsigned int width, unsigned int height)=0
- virtual unsigned int `getMainSceneId` () const =0

Public Member Functions inherited from [utl::IPlugin](#)

- virtual `~IPlugin` ()=default
- virtual const std::string `getName` () const =0
- virtual `PluginType` `getType` () const =0

18.68.1 Detailed Description

Interface for the games.

Definition at line 22 of file [IGameClient.hpp](#).

18.68.2 Member Function Documentation

18.68.2.1 `getMainSceneId()`

virtual unsigned int `gme::IGameClient::getMainSceneId` () const [pure virtual]

Implemented in [gme::FlappyBirdClientSolo](#), [gme::RTypeClientMulti](#), and [gme::RTypeClientSolo](#).

18.68.2.2 `init()`

virtual void `gme::IGameClient::init` (
[eng::Engine](#) & engine,
[utl::cli::AppConfig](#) & appConfig,
 bool & showDebug,
 unsigned int menuSceneId) [pure virtual]

Implemented in [gme::FlappyBirdClientSolo](#), [gme::RTypeClientMulti](#), and [gme::RTypeClientSolo](#).

18.68.2.3 update()

```
virtual void gme::IGameClient::update (  
    float deltaTime,  
    unsigned int width,  
    unsigned int height) [pure virtual]
```

Implemented in [gme::FlappyBirdClientSolo](#), [gme::RTypeClientMulti](#), and [gme::RTypeClientSolo](#).

The documentation for this interface was generated from the following file:

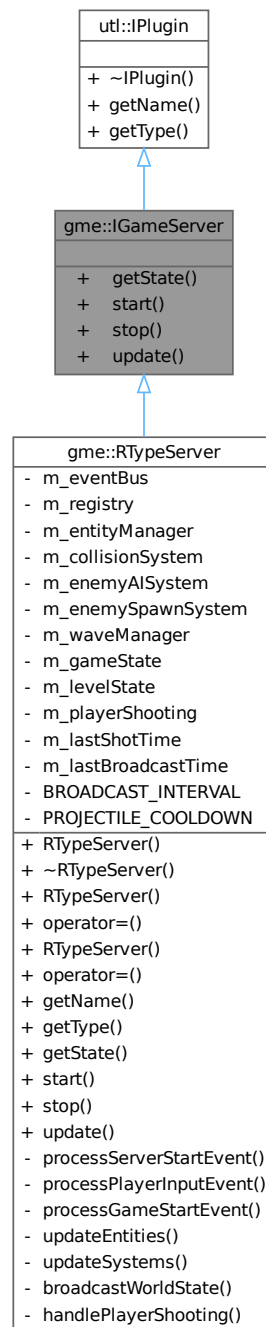
- [/home/masina/Projects/Epitech/rtype/modules/Interfaces/include/Interfaces/IGameClient.hpp](#)

18.69 gme::IGameServer Interface Reference

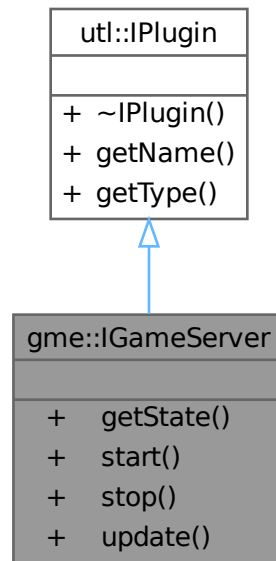
Interface for the games.

```
#include <IGameServer.hpp>
```

Inheritance diagram for gme::IGameServer:



Collaboration diagram for gme::IGameServer:



Public Member Functions

- virtual [State](#) [getState](#) () const =0
- virtual void [start](#) ()=0
- virtual void [stop](#) ()=0
- virtual void [update](#) (float deltaTime)=0

Public Member Functions inherited from [utl::IPlugin](#)

- virtual [~IPlugin](#) ()=default
- virtual const std::string [getName](#) () const =0
- virtual [PluginType](#) [getType](#) () const =0

18.69.1 Detailed Description

Interface for the games.

Definition at line 28 of file [IGameServer.hpp](#).

18.69.2 Member Function Documentation

18.69.2.1 [getState\(\)](#)

virtual [State](#) [gme::IGameServer::getState](#) () const [nodiscard], [pure virtual]

Implemented in [gme::RTypeServer](#).

18.69.2.2 [start\(\)](#)

virtual void [gme::IGameServer::start](#) () [pure virtual]

Implemented in [gme::RTypeServer](#).

18.69.2.3 stop()

virtual void gme::IGameServer::stop () [pure virtual]

Implemented in [gme::RTypeServer](#).

18.69.2.4 update()

virtual void gme::IGameServer::update (
float deltaTime) [pure virtual]

Implemented in [gme::RTypeServer](#).

The documentation for this interface was generated from the following file:

- [/home/masina/Projects/Epitech/rtype/modules/Interfaces/include/Interfaces/IGameServer.hpp](#)

18.70 eng::INetworkClient Interface Reference

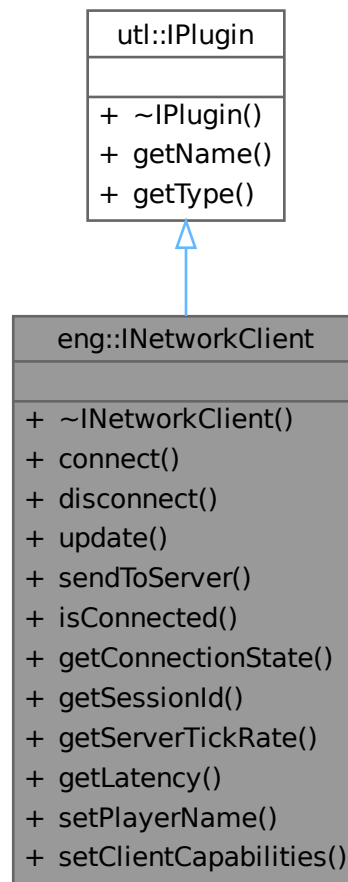
Interface for the client network.

#include <INetworkClient.hpp>

Inheritance diagram for eng::INetworkClient:



Collaboration diagram for eng::INetworkClient:



Public Member Functions

- virtual `~INetworkClient` ()=default
- virtual void `connect` (const std::string &host, uint16_t port)=0
- virtual void `disconnect` ()=0
- virtual void `update` ()=0
- virtual void `sendToServer` (const std::vector< std::uint8_t > &data, bool reliable=false)=0
- virtual bool `isConnected` () const =0
- virtual `ConnectionState` `getConnectionState` () const =0
- virtual std::uint32_t `getSessionId` () const =0
- virtual std::uint16_t `getServerTickRate` () const =0
- virtual std::uint32_t `getLatency` () const =0
- virtual void `setPlayerName` (const std::string &playerName)=0
- virtual void `setClientCapabilities` (std::uint32_t caps)=0

Public Member Functions inherited from `utl::IPlugin`

- virtual `~IPlugin` ()=default
- virtual const std::string `getName` () const =0
- virtual `PluginType` `getType` () const =0

18.70.1 Detailed Description

Interface for the client network.

Definition at line 34 of file [INetworkClient.hpp](#).

18.70.2 Constructor & Destructor Documentation

18.70.2.1 ~INetworkClient()

virtual eng::INetworkClient::~INetworkClient () [virtual], [default]

18.70.3 Member Function Documentation

18.70.3.1 connect()

virtual void eng::INetworkClient::connect (
 const std::string & host,
 uint16_t port) [pure virtual]

18.70.3.2 disconnect()

virtual void eng::INetworkClient::disconnect () [pure virtual]

Implemented in [eng::AsioClient](#).

18.70.3.3 getConnectionState()

virtual [ConnectionState](#) eng::INetworkClient::getConnectionState () const [nodiscard], [pure virtual]

Implemented in [eng::AsioClient](#).

18.70.3.4 getLatency()

virtual std::uint32_t eng::INetworkClient::getLatency () const [nodiscard], [pure virtual]

Implemented in [eng::AsioClient](#).

18.70.3.5 getServerTickRate()

virtual std::uint16_t eng::INetworkClient::getServerTickRate () const [nodiscard], [pure virtual]

Implemented in [eng::AsioClient](#).

18.70.3.6 getSessionId()

virtual std::uint32_t eng::INetworkClient::getSessionId () const [nodiscard], [pure virtual]

Implemented in [eng::AsioClient](#).

18.70.3.7 isConnected()

virtual bool eng::INetworkClient::isConnected () const [nodiscard], [pure virtual]

Implemented in [eng::AsioClient](#).

18.70.3.8 sendToServer()

virtual void eng::INetworkClient::sendToServer (
 const std::vector< std::uint8_t > & data,
 bool reliable = false) [pure virtual]

Implemented in [eng::AsioClient](#).

18.70.3.9 setClientCapabilities()

virtual void eng::INetworkClient::setClientCapabilities (
 std::uint32_t caps) [pure virtual]

Implemented in [eng::AsioClient](#).

18.70.3.10 setPlayerName()

```
virtual void eng::INetworkClient::setPlayerName (  
    const std::string & playerName) [pure virtual]
```

Implemented in [eng::AsioClient](#).

18.70.3.11 update()

```
virtual void eng::INetworkClient::update () [pure virtual]
```

Implemented in [eng::AsioClient](#).

The documentation for this interface was generated from the following file:

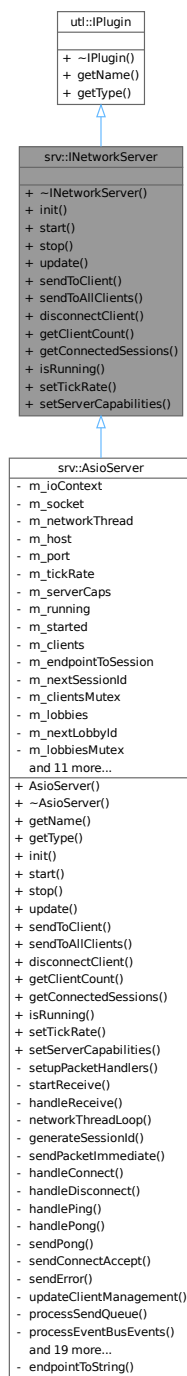
- [/home/masina/Projects/Epitech/rtype/modules/Interfaces/include/Interfaces/INetworkClient.hpp](#)

18.71 srv::INetworkServer Interface Reference

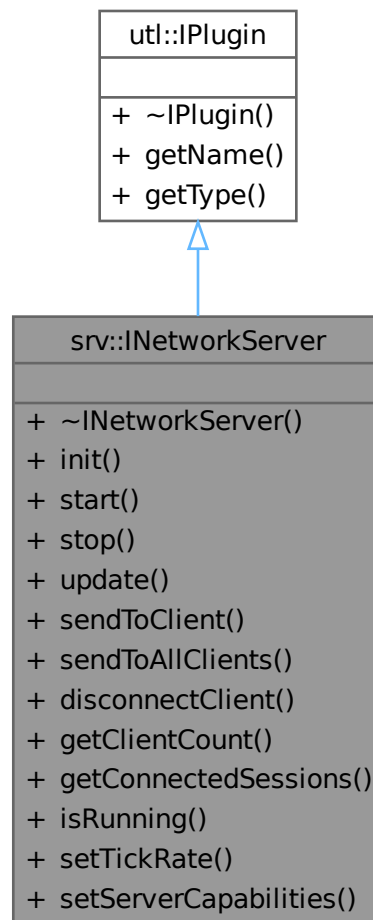
Interface for the server network.

```
#include <INetworkServer.hpp>
```

Inheritance diagram for `srv::INetworkServer`:



Collaboration diagram for srv::INetworkServer:



Public Member Functions

- virtual `~INetworkServer` ()=default
- virtual void `init` (const std::string &host, uint16_t port)=0
- virtual void `start` ()=0
- virtual void `stop` ()=0
- virtual void `update` ()=0
- virtual void `sendToClient` (std::uint32_t sessionId, const std::vector< std::uint8_t > &data, bool reliable=false)=0
- virtual void `sendToAllClients` (const std::vector< std::uint8_t > &data, bool reliable=false)=0
- virtual void `disconnectClient` (std::uint32_t sessionId)=0
- virtual std::size_t `getClientCount` () const =0
- virtual std::vector< std::uint32_t > `getConnectedSessions` () const =0
- virtual bool `isRunning` () const =0
- virtual void `setTickRate` (std::uint16_t tickRate)=0
- virtual void `setServerCapabilities` (std::uint32_t caps)=0

Public Member Functions inherited from [utl::IPlugin](#)

- virtual [~IPlugin](#) ()=default
- virtual const std::string [getName](#) () const =0
- virtual [PluginType](#) [getType](#) () const =0

18.71.1 Detailed Description

Interface for the server network.

Definition at line 27 of file [INetworkServer.hpp](#).

18.71.2 Constructor & Destructor Documentation

18.71.2.1 ~INetworkServer()

virtual srv::INetworkServer::~INetworkServer () [virtual], [default]

18.71.3 Member Function Documentation

18.71.3.1 disconnectClient()

virtual void srv::INetworkServer::disconnectClient (
std::uint32_t sessionId) [pure virtual]

Implemented in [srv::AsioServer](#).

18.71.3.2 getClientCount()

virtual std::size_t srv::INetworkServer::getClientCount () const [nodiscard], [pure virtual]

Implemented in [srv::AsioServer](#).

18.71.3.3 getConnectedSessions()

virtual std::vector< std::uint32_t > srv::INetworkServer::getConnectedSessions () const [nodiscard], [pure virtual]

Implemented in [srv::AsioServer](#).

18.71.3.4 init()

virtual void srv::INetworkServer::init (
const std::string & host,
uint16_t port) [pure virtual]

18.71.3.5 isRunning()

virtual bool srv::INetworkServer::isRunning () const [nodiscard], [pure virtual]

Implemented in [srv::AsioServer](#).

18.71.3.6 sendToAllClients()

virtual void srv::INetworkServer::sendToAllClients (
const std::vector< std::uint8_t > & data,
bool reliable = false) [pure virtual]

Implemented in [srv::AsioServer](#).

18.71.3.7 sendToClient()

virtual void srv::INetworkServer::sendToClient (
std::uint32_t sessionId,
const std::vector< std::uint8_t > & data,
bool reliable = false) [pure virtual]

Implemented in [srv::AsioServer](#).

18.71.3.8 setServerCapabilities()

virtual void srv::INetworkServer::setServerCapabilities (
 std::uint32_t caps) [pure virtual]

Implemented in [srv::AsioServer](#).

18.71.3.9 setTickRate()

virtual void srv::INetworkServer::setTickRate (
 std::uint16_t tickRate) [pure virtual]

Implemented in [srv::AsioServer](#).

18.71.3.10 start()

virtual void srv::INetworkServer::start () [pure virtual]

Implemented in [srv::AsioServer](#).

18.71.3.11 stop()

virtual void srv::INetworkServer::stop () [pure virtual]

Implemented in [srv::AsioServer](#).

18.71.3.12 update()

virtual void srv::INetworkServer::update () [pure virtual]

Implemented in [srv::AsioServer](#).

The documentation for this interface was generated from the following file:

- [/home/masina/Projects/Epitech/rtype/modules/Interfaces/include/Interfaces/INetworkServer.hpp](#)

18.72 gme::InterpolationData Struct Reference

#include <GameMulti.hpp>

Collaboration diagram for gme::InterpolationData:

gme::InterpolationData	
+	targetX
+	targetY
+	targetVx
+	targetVy
+	currentX
+	currentY
+	smoothFactor
+	targetRotation
+	currentRotation

Public Attributes

- float [targetX](#)
- float [targetY](#)
- float [targetVx](#)
- float [targetVy](#)
- float [currentX](#)
- float [currentY](#)
- float [smoothFactor](#)
- float [targetRotation](#)
- float [currentRotation](#)

18.72.1 Detailed Description

Definition at line 24 of file [GameMulti.hpp](#).

18.72.2 Member Data Documentation

18.72.2.1 currentRotation

float gme::InterpolationData::currentRotation

Definition at line 31 of file [GameMulti.hpp](#).

18.72.2.2 currentX

float gme::InterpolationData::currentX

Definition at line 28 of file [GameMulti.hpp](#).

18.72.2.3 currentY

float gme::InterpolationData::currentY

Definition at line 28 of file [GameMulti.hpp](#).

18.72.2.4 smoothFactor

float gme::InterpolationData::smoothFactor

Definition at line 29 of file [GameMulti.hpp](#).

18.72.2.5 targetRotation

float gme::InterpolationData::targetRotation

Definition at line 30 of file [GameMulti.hpp](#).

18.72.2.6 targetVx

float gme::InterpolationData::targetVx

Definition at line 27 of file [GameMulti.hpp](#).

18.72.2.7 targetVy

float gme::InterpolationData::targetVy

Definition at line 27 of file [GameMulti.hpp](#).

18.72.2.8 targetX

float gme::InterpolationData::targetX

Definition at line 26 of file [GameMulti.hpp](#).

18.72.2.9 targetY

float gme::InterpolationData::targetY

Definition at line 26 of file [GameMulti.hpp](#).

The documentation for this struct was generated from the following file:

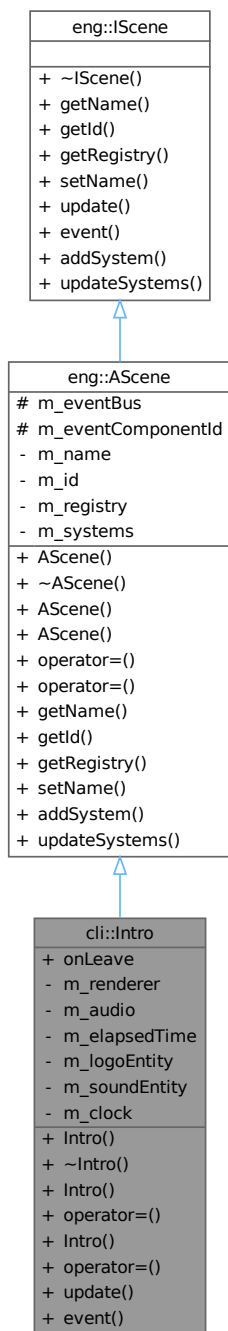
- /home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/include/RType↵
ClientMulti/Scenes/[GameMulti.hpp](#)

18.73 cli::Intro Class Reference

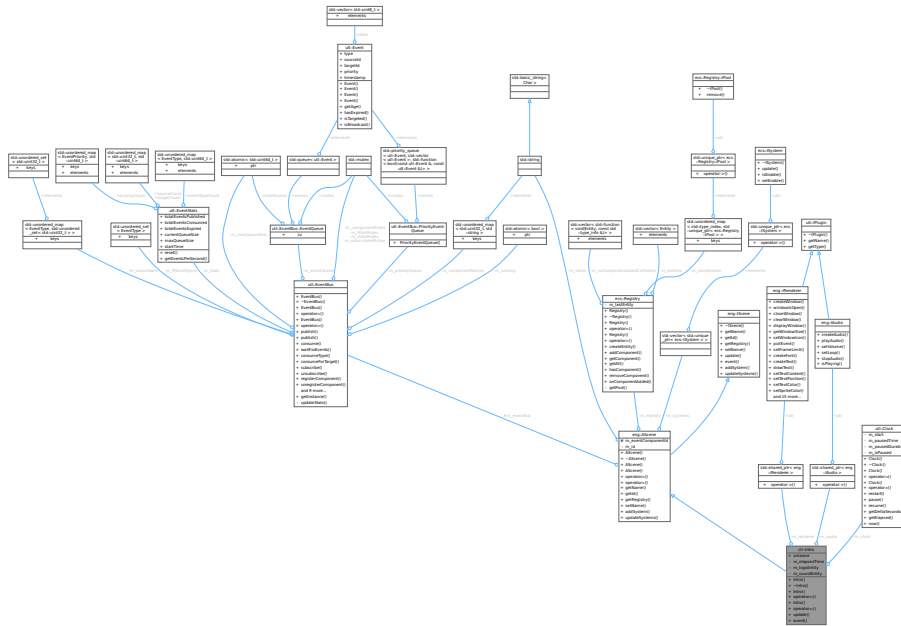
[Intro](#) scene.

```
#include <Intro.hpp>
```

Inheritance diagram for cli::Intro:



Collaboration diagram for cli::Intro:



Public Member Functions

- [Intro](#) ([eng::id](#) assignedId, const std::shared_ptr< [eng::IRenderer](#) > &renderer, const std::shared_ptr< [eng::IAudio](#) > &audio)
- [~Intro](#) () override=default
- [Intro](#) (const [Intro](#) &other)=delete
- [Intro](#) & operator= (const [Intro](#) &other)=delete
- [Intro](#) ([Intro](#) &&other)=delete
- [Intro](#) & operator= ([Intro](#) &&other)=delete
- void [update](#) (float dt, const [eng::WindowSize](#) &size) override
- void [event](#) (const [eng::Event](#) &event) override

Public Member Functions inherited from [eng::AScene](#)

- [AScene](#) (const [id](#) assignedId)
- [~AScene](#) () override=default
- [AScene](#) (const [AScene](#) &other)=delete
- [AScene](#) ([AScene](#) &&other)=delete
- [AScene](#) & operator= (const [AScene](#) &other)=delete
- [AScene](#) & operator= ([AScene](#) &&other)=delete
- std::string & [getName](#) () override
- [id](#) [getId](#) () const override
- [ecs::Registry](#) & [getRegistry](#) () override
- void [setName](#) (const std::string &newName) override
- void [addSystem](#) (std::unique_ptr< [ecs::ISystem](#) > system) override
- void [updateSystems](#) (const float dt) override

Public Member Functions inherited from [eng::IScene](#)

- virtual [~IScene](#) ()=default

Public Attributes

- std::function< void()> [onLeave](#)

Private Attributes

- `std::shared_ptr< eng::IRenderer > m_renderer`
- `std::shared_ptr< eng::IAudio > m_audio`
- `float m_elapsedTime = 0.F`
- `ecs::Entity m_logoEntity = 0`
- `ecs::Entity m_soundEntity = 0`
- `utl::Clock m_clock`

Additional Inherited Members

Protected Attributes inherited from `eng::AScene`

- `utl::EventBus & m_eventBus = utl::EventBus::getInstance()`
- `std::uint32_t m_eventComponentId = 0`

18.73.1 Detailed Description

`Intro` scene.

Definition at line 20 of file `Intro.hpp`.

18.73.2 Constructor & Destructor Documentation

18.73.2.1 `Intro()` [1/3]

```
cli::Intro::Intro (
    eng::id assignedId,
    const std::shared_ptr< eng::IRenderer > & renderer,
    const std::shared_ptr< eng::IAudio > & audio)
```

Definition at line 8 of file `intro.cpp`.

References `eng::Color::a`, `eng::Color::b`, `eng::Color::g`, `utl::Path::Icons::ICON_APP`, `ecs::IComponent::id`, `m_logoEntity`, `m_soundEntity`, `eng::Color::r`, and `utl::Config::Color::WHITE_LOW`.

18.73.2.2 `~Intro()`

```
cli::Intro::~Intro () [override], [default]
```

18.73.2.3 `Intro()` [2/3]

```
cli::Intro::Intro (
    const Intro & other) [delete]
```

18.73.2.4 `Intro()` [3/3]

```
cli::Intro::Intro (
    Intro && other) [delete]
```

18.73.3 Member Function Documentation

18.73.3.1 `event()`

```
void cli::Intro::event (
    const eng::Event & event) [override], [virtual]
```

Implements `eng::IScene`.

Definition at line 100 of file `intro.cpp`.

18.73.3.2 `operator=()` [1/2]

```
Intro & cli::Intro::operator= (
    const Intro & other) [delete]
```

18.73.3.3 operator=() [2/2]

[Intro](#) & cli::Intro::operator= (
[Intro](#) && other) [delete]

18.73.3.4 update()

void cli::Intro::update (
float dt,
const [eng::WindowSize](#) & size) [override], [virtual]

Implements [eng::IScene](#).

Definition at line 69 of file [intro.cpp](#).

References [ecs::Color::a](#).

18.73.4 Member Data Documentation

18.73.4.1 m_audio

std::shared_ptr<[eng::IAudio](#)> cli::Intro::m_audio [private]

Definition at line 39 of file [Intro.hpp](#).

18.73.4.2 m_clock

[utl::Clock](#) cli::Intro::m_clock [private]

Definition at line 44 of file [Intro.hpp](#).

18.73.4.3 m_elapsedTime

float cli::Intro::m_elapsedTime = 0.F [private]

Definition at line 40 of file [Intro.hpp](#).

18.73.4.4 m_logoEntity

[ecs::Entity](#) cli::Intro::m_logoEntity = 0 [private]

Definition at line 42 of file [Intro.hpp](#).

Referenced by [Intro\(\)](#).

18.73.4.5 m_renderer

std::shared_ptr<[eng::IRenderer](#)> cli::Intro::m_renderer [private]

Definition at line 38 of file [Intro.hpp](#).

18.73.4.6 m_soundEntity

[ecs::Entity](#) cli::Intro::m_soundEntity = 0 [private]

Definition at line 43 of file [Intro.hpp](#).

Referenced by [Intro\(\)](#).

18.73.4.7 onLeave

std::function<void()> cli::Intro::onLeave

Definition at line 35 of file [Intro.hpp](#).

The documentation for this class was generated from the following files:

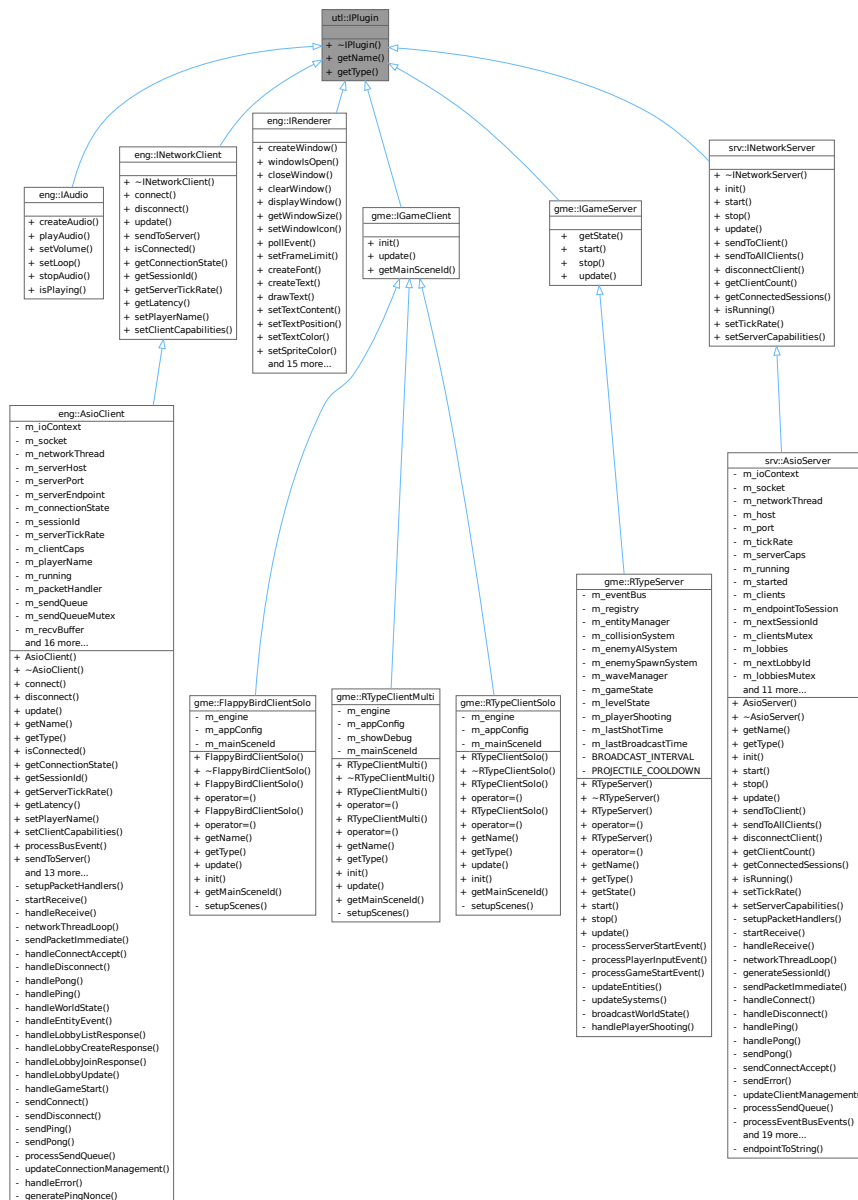
- /home/masina/Projects/Epitech/rtype/client/include/Client/Scenes/[Intro.hpp](#)
- /home/masina/Projects/Epitech/rtype/client/src/scenes/[intro.cpp](#)

18.74 utl::IPlugin Interface Reference

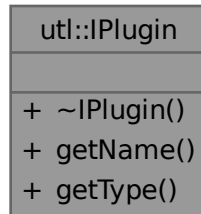
Interface for plugins.

#include <IPlugin.hpp>

Inheritance diagram for `utl::IPlugin`:



Collaboration diagram for utl::IPlugin:



Public Member Functions

- virtual [~IPlugin](#) ()=default
- virtual const std::string [getName](#) () const =0
- virtual [PluginType](#) [getType](#) () const =0

18.74.1 Detailed Description

Interface for plugins.

Definition at line 37 of file [IPlugin.hpp](#).

18.74.2 Constructor & Destructor Documentation

18.74.2.1 ~IPlugin()

virtual utl::IPlugin::~~IPlugin () [virtual], [default]

18.74.3 Member Function Documentation

18.74.3.1 getName()

virtual const std::string utl::IPlugin::getName () const [nodiscard], [pure virtual]

Implemented in [eng::AsioClient](#), [gme::FlappyBirdClientSolo](#), [gme::RTypeClientMulti](#), [gme::RTypeClientSolo](#), [gme::RTypeServer](#), and [srv::AsioServer](#).

18.74.3.2 getType()

virtual [PluginType](#) utl::IPlugin::getType () const [nodiscard], [pure virtual]

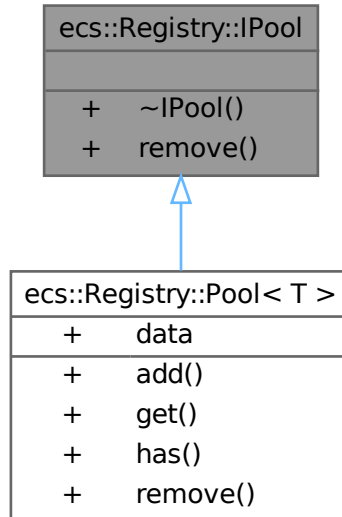
Implemented in [eng::AsioClient](#), [gme::FlappyBirdClientSolo](#), [gme::RTypeClientMulti](#), [gme::RTypeClientSolo](#), [gme::RTypeServer](#), and [srv::AsioServer](#).

The documentation for this interface was generated from the following file:

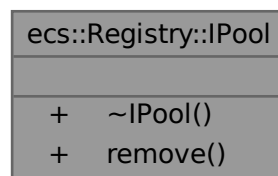
- [/home/masina/Projects/Epitech/rtype/modules/Utils/include/Utils/Interfaces/IPlugin.hpp](#)

18.75 ecs::Registry::IPool Class Reference

Inheritance diagram for ecs::Registry::IPool:



Collaboration diagram for ecs::Registry::IPool:



Public Member Functions

- virtual `~IPool()`=default
- virtual void `remove (Entity e)`=0

18.75.1 Detailed Description

Definition at line 97 of file [Registry.hpp](#).

18.75.2 Constructor & Destructor Documentation

18.75.2.1 ~IPool()

virtual `ecs::Registry::IPool::~~IPool ()` [virtual], [default]

18.75.3 Member Function Documentation

18.75.3.1 remove()

virtual void ecs::Registry::IPool::remove (
[Entity](#) e) [pure virtual]

Implemented in [ecs::Registry::Pool< T >](#).

The documentation for this class was generated from the following file:

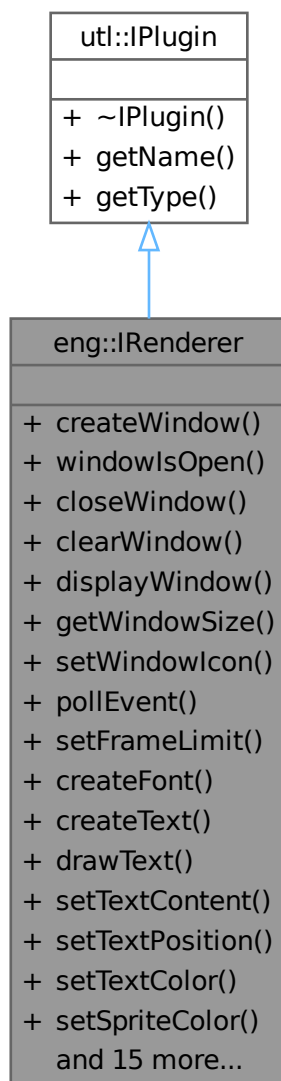
- [/home/masina/Projects/Epitech/rtype/modules/ECS/include/ECS/Registry.hpp](#)

18.76 eng::IRenderer Interface Reference

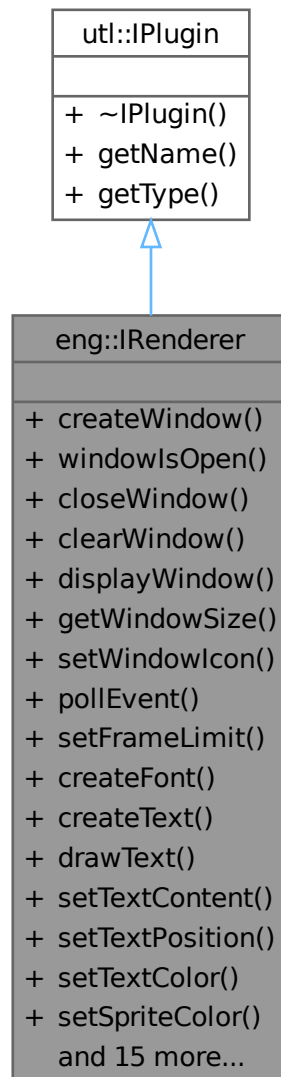
Interface for the renderer.

#include <IRenderer.hpp>

Inheritance diagram for eng::IRenderer:



Collaboration diagram for eng::IRenderer:



Public Member Functions

- virtual void `createWindow` (const std::string &title, unsigned int height, unsigned int width, unsigned int frameLimit, bool fullscreen)=0
- virtual bool `windowIsOpen` () const =0
- virtual void `closeWindow` ()=0
- virtual void `clearWindow` (Color color)=0
- virtual void `displayWindow` ()=0
- virtual WindowSize `getWindowSize` ()=0
- virtual void `setWindowIcon` (const std::string &path)=0
- virtual bool `pollEvent` (Event &event)=0
- virtual void `setFrameLimit` (unsigned int frameLimit)=0
- virtual void `createFont` (const std::string &name, const std::string &path)=0
- virtual void `createText` (Text text)=0

- virtual void [drawText](#) (const std::string &name)=0
- virtual void [setTextContent](#) (const std::string &name, const std::string &content)=0
- virtual void [setTextPosition](#) (const std::string &name, float x, float y)=0
- virtual void [setTextColor](#) (const std::string &name, [Color](#) color)=0
- virtual void [setSpriteColor](#) (const std::string &name, [Color](#) color)=0
- virtual void [createTexture](#) (const std::string &name, const std::string &path)=0
- virtual void [createSprite](#) (const std::string &name, const std::string &textureName, float x, float y, float scale_x=1, float scale_y=1, int fx=0, int fy=0, int fnx=-1, int fny=-1)=0
- virtual void [drawSprite](#) (const std::string &name)=0
- virtual void [setSpritePosition](#) (const std::string &name, float x, float y)=0
- virtual void [setSpriteTexture](#) (const std::string &name, const std::string &path)=0
- virtual void [setSpriteScale](#) (const std::string &name, int x, int y)=0
- virtual void [setSpriteFrame](#) (const std::string &name, int fx, int fy, int fnx, int fny)=0
- virtual void [setSpriteRotation](#) (const std::string &name, float rotation)=0
- virtual void [rotateSprite](#) (const std::string &name, float rotation)=0
- virtual void [setSpriteOrigin](#) (const std::string &name)=0
- virtual void [createCircleShape](#) ([CircleShape](#) circleShape)=0
- virtual void [setCircleShapePosition](#) (const std::string &name, float x, float y)=0
- virtual void [drawCircleShape](#) (const std::string &name)=0
- virtual void [drawPoint](#) (float x, float y, [Color](#) color)=0
- virtual void [renderGui](#) (const [WindowSize](#) &>windowSize)=0

Public Member Functions inherited from [utl::IPlugin](#)

- virtual [~IPlugin](#) ()=default
- virtual const std::string [getName](#) () const =0
- virtual [PluginType](#) [getType](#) () const =0

18.76.1 Detailed Description

Interface for the renderer.

Definition at line 119 of file [IRenderer.hpp](#).

18.76.2 Member Function Documentation

18.76.2.1 clearWindow()

```
virtual void eng::IRenderer::clearWindow (
    Color color) [pure virtual]
```

18.76.2.2 closeWindow()

```
virtual void eng::IRenderer::closeWindow () [pure virtual]
```

18.76.2.3 createCircleShape()

```
virtual void eng::IRenderer::createCircleShape (
    CircleShape circleShape) [pure virtual]
```

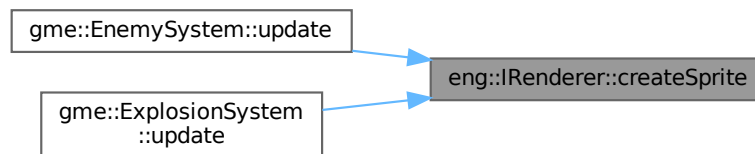
18.76.2.4 createFont()

```
virtual void eng::IRenderer::createFont (
    const std::string & name,
    const std::string & path) [pure virtual]
```

18.76.2.5 createSprite()

```
virtual void eng::IRenderer::createSprite (
    const std::string & name,
    const std::string & textureName,
    float x,
    float y,
    float scale_x = 1,
    float scale_y = 1,
    int fx = 0,
    int fy = 0,
    int fnx = -1,
    int fny = -1) [pure virtual]
```

Referenced by [gme::EnemySystem::update\(\)](#), and [gme::ExplosionSystem::update\(\)](#).
Here is the caller graph for this function:



18.76.2.6 createText()

```
virtual void eng::IRenderer::createText (
    Text text) [pure virtual]
```

18.76.2.7 createTexture()

```
virtual void eng::IRenderer::createTexture (
    const std::string & name,
    const std::string & path) [pure virtual]
```

18.76.2.8 createWindow()

```
virtual void eng::IRenderer::createWindow (
    const std::string & title,
    unsigned int height,
    unsigned int width,
    unsigned int frameLimit,
    bool fullscreen) [pure virtual]
```

18.76.2.9 displayWindow()

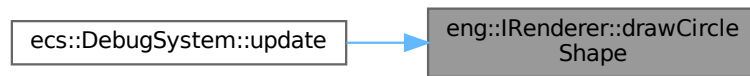
```
virtual void eng::IRenderer::displayWindow () [pure virtual]
```

18.76.2.10 drawCircleShape()

```
virtual void eng::IRenderer::drawCircleShape (
    const std::string & name) [pure virtual]
```

Referenced by [ecs::DebugSystem::update\(\)](#).

Here is the caller graph for this function:

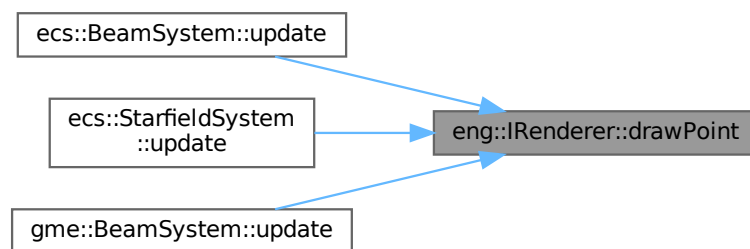


18.76.2.11 drawPoint()

```
virtual void eng::IRenderer::drawPoint (  
    float x,  
    float y,  
    Color color) [pure virtual]
```

Referenced by [ecs::BeamSystem::update\(\)](#), [ecs::StarfieldSystem::update\(\)](#), and [gme::BeamSystem::update\(\)](#).

Here is the caller graph for this function:

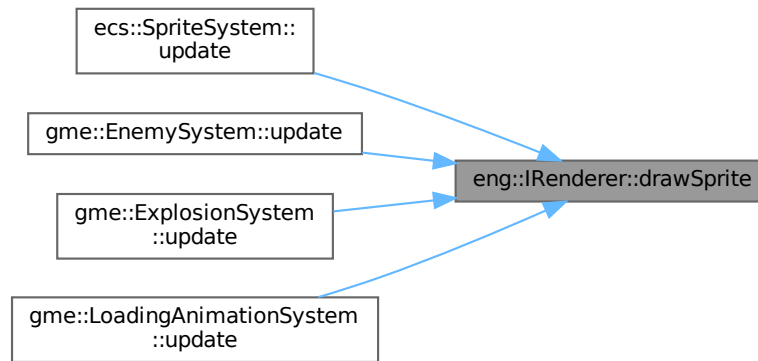


18.76.2.12 drawSprite()

```
virtual void eng::IRenderer::drawSprite (  
    const std::string & name) [pure virtual]
```

Referenced by [ecs::SpriteSystem::update\(\)](#), [gme::EnemySystem::update\(\)](#), [gme::ExplosionSystem::update\(\)](#), and [gme::LoadingAnimationSystem::update\(\)](#).

Here is the caller graph for this function:



18.76.2.13 drawText()

```
virtual void eng::IRenderer::drawText (
    const std::string & name) [pure virtual]
```

Referenced by [ecs::TextSystem::update\(\)](#).

Here is the caller graph for this function:

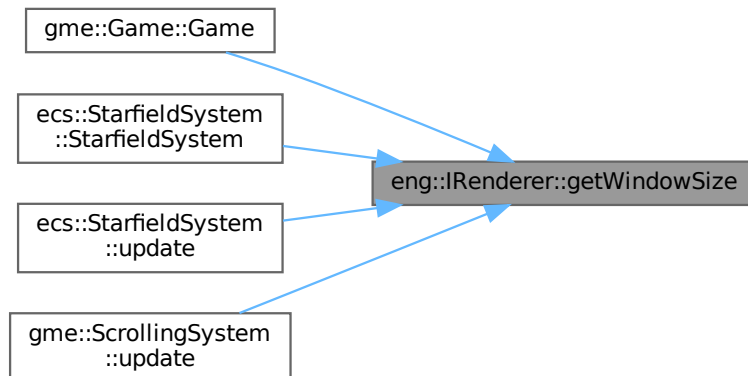


18.76.2.14 getWindowSize()

```
virtual WindowSize eng::IRenderer::getWindowSize () [nodiscard], [pure virtual]
```

Referenced by [gme::Game::Game\(\)](#), [ecs::StarfieldSystem::StarfieldSystem\(\)](#), [ecs::StarfieldSystem::update\(\)](#), and [gme::ScrollingSystem::update\(\)](#).

Here is the caller graph for this function:



18.76.2.15 pollEvent()

```
virtual bool eng::IRenderer::pollEvent (
    Event & event) [nodiscard], [pure virtual]
```

18.76.2.16 renderGui()

```
virtual void eng::IRenderer::renderGui (
    const WindowSize & windowSize) [pure virtual]
```

18.76.2.17 rotateSprite()

```
virtual void eng::IRenderer::rotateSprite (
    const std::string & name,
    float rotation) [pure virtual]
```

18.76.2.18 setCircleShapePosition()

```
virtual void eng::IRenderer::setCircleShapePosition (
    const std::string & name,
    float x,
    float y) [pure virtual]
```

Referenced by [ecs::DebugSystem::update\(\)](#).

Here is the caller graph for this function:



18.76.2.19 setFrameLimit()

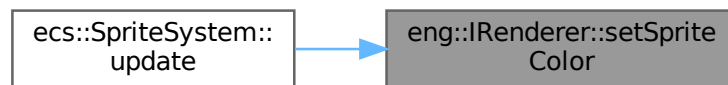
```
virtual void eng::IRenderer::setFrameLimit (
    unsigned int frameLimit) [pure virtual]
```

18.76.2.20 setSpriteColor()

```
virtual void eng::IRenderer::setSpriteColor (
    const std::string & name,
    Color color) [pure virtual]
```

Referenced by [ecs::SpriteSystem::update\(\)](#).

Here is the caller graph for this function:

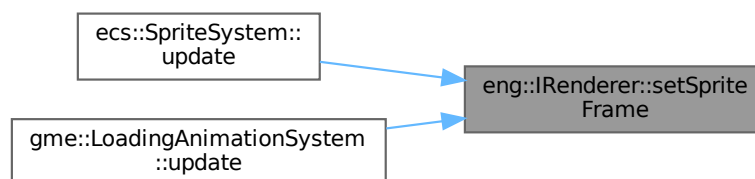


18.76.2.21 setSpriteFrame()

```
virtual void eng::IRenderer::setSpriteFrame (
    const std::string & name,
    int fx,
    int fy,
    int fnx,
    int fny) [pure virtual]
```

Referenced by [ecs::SpriteSystem::update\(\)](#), and [gme::LoadingAnimationSystem::update\(\)](#).

Here is the caller graph for this function:



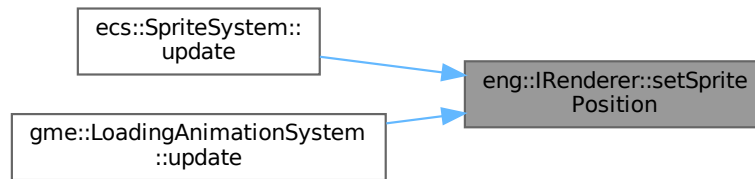
18.76.2.22 setSpriteOrigin()

```
virtual void eng::IRenderer::setSpriteOrigin (
    const std::string & name) [pure virtual]
```

18.76.2.23 setSpritePosition()

```
virtual void eng::IRenderer::setSpritePosition (
    const std::string & name,
    float x,
    float y) [pure virtual]
```

Referenced by [ecs::SpriteSystem::update\(\)](#), and [gme::LoadingAnimationSystem::update\(\)](#).
Here is the caller graph for this function:



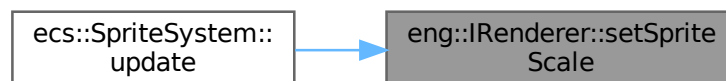
18.76.2.24 setSpriteRotation()

```
virtual void eng::IRenderer::setSpriteRotation (
    const std::string & name,
    float rotation) [pure virtual]
```

18.76.2.25 setSpriteScale()

```
virtual void eng::IRenderer::setSpriteScale (
    const std::string & name,
    int x,
    int y) [pure virtual]
```

Referenced by [ecs::SpriteSystem::update\(\)](#).
Here is the caller graph for this function:

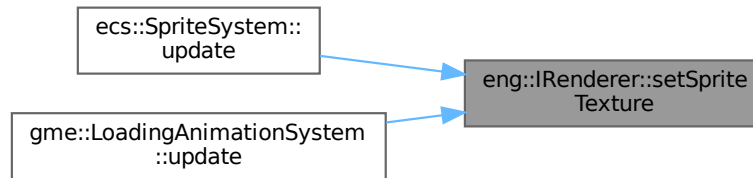


18.76.2.26 setSpriteTexture()

```
virtual void eng::IRenderer::setSpriteTexture (
    const std::string & name,
    const std::string & path) [pure virtual]
```

Referenced by [ecs::SpriteSystem::update\(\)](#), and [gme::LoadingAnimationSystem::update\(\)](#).

Here is the caller graph for this function:



18.76.2.27 setTextColor()

```
virtual void eng::IRenderer::setTextColor (
    const std::string & name,
    Color color) [pure virtual]
```

Referenced by [ecs::TextSystem::update\(\)](#).

Here is the caller graph for this function:



18.76.2.28 setTextContent()

```
virtual void eng::IRenderer::setTextContent (
    const std::string & name,
    const std::string & content) [pure virtual]
```

Referenced by [ecs::TextSystem::update\(\)](#).

Here is the caller graph for this function:

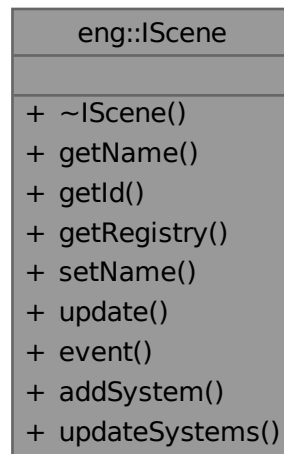


18.76.2.29 setPosition()

```
virtual void eng::IRenderer::setPosition (
    const std::string & name,
    float x,
    float y) [pure virtual]
```

Referenced by [ecs::TextSystem::update\(\)](#).

Collaboration diagram for `eng::IScene`:



Public Member Functions

- virtual `~IScene()`=default
- virtual `std::string & getName()`=0
- virtual `id getId()` const =0
- virtual `ecs::Registry & getRegistry()`=0
- virtual void `setName` (const `std::string &newName`)=0
- virtual void `update` (float dt, const `WindowSize &size`)=0
- virtual void `event` (const `Event &event`)=0
- virtual void `addSystem` (`std::unique_ptr< ecs::ISystem > system`)=0
- virtual void `updateSystems` (float dt)=0

18.77.1 Detailed Description

interface class for scene

Definition at line 27 of file [IScene.hpp](#).

18.77.2 Constructor & Destructor Documentation

18.77.2.1 ~IScene()

virtual `eng::IScene::~IScene()` [virtual], [default]

18.77.3 Member Function Documentation

18.77.3.1 addSystem()

virtual void `eng::IScene::addSystem` (
`std::unique_ptr< ecs::ISystem > system`) [pure virtual]

Implemented in [eng::AScene](#).

18.77.3.2 event()

virtual void eng::IScene::event (
 const [Event](#) & event) [pure virtual]

Implemented in [cli::Intro](#), [cli::Menu](#), [cli::Settings](#), [gme::ConfigMulti](#), [gme::ConfigSolo](#), [gme::CreateRoomScene](#), [gme::Game](#), [gme::GameMulti](#), [gme::GameSolo](#), [gme::JoinRoomScene](#), [gme::ServerScene](#), and [gme::WaitingRoomScene](#).

18.77.3.3 getId()

virtual [id](#) eng::IScene::getId () const [nodiscard], [pure virtual]

Implemented in [eng::AScene](#).

18.77.3.4 getName()

virtual std::string & eng::IScene::getName () [nodiscard], [pure virtual]

Implemented in [eng::AScene](#).

18.77.3.5 getRegistry()

virtual [ecs::Registry](#) & eng::IScene::getRegistry () [nodiscard], [pure virtual]

Implemented in [eng::AScene](#).

18.77.3.6 setName()

virtual void eng::IScene::setName (
 const std::string & newName) [pure virtual]

Implemented in [eng::AScene](#).

18.77.3.7 update()

virtual void eng::IScene::update (
 float dt,
 const [WindowSize](#) & size) [pure virtual]

Implemented in [cli::Intro](#), [cli::Menu](#), [cli::Settings](#), [gme::ConfigMulti](#), [gme::ConfigSolo](#), [gme::CreateRoomScene](#), [gme::Game](#), [gme::GameMulti](#), [gme::GameSolo](#), [gme::JoinRoomScene](#), [gme::ServerScene](#), and [gme::WaitingRoomScene](#).

18.77.3.8 updateSystems()

virtual void eng::IScene::updateSystems (
 float dt) [pure virtual]

Implemented in [eng::AScene](#).

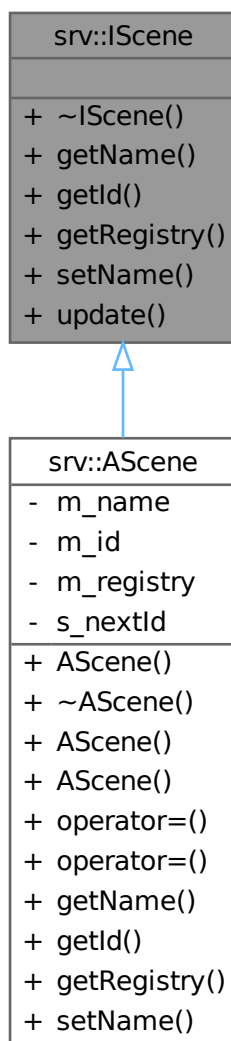
The documentation for this interface was generated from the following file:

- [/home/masina/Projects/Epitech/rtype/modules/Engine/include/Engine/Interfaces/IScene.hpp](#)

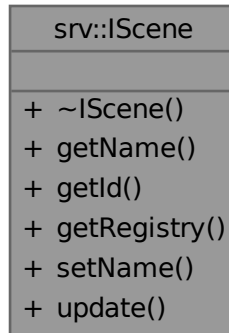
18.78 srv::IScene Class Reference

interface class for scene
 #include <IScene.hpp>

Inheritance diagram for `srv::IScene`:



Collaboration diagram for srv::IScene:



Public Member Functions

- virtual [~IScene](#) ()=default
- virtual std::string & [getName](#) ()=0
- virtual [id](#) [getId](#) () const =0
- virtual [ecs::Registry](#) & [getRegistry](#) ()=0
- virtual void [setName](#) (const std::string &newName)=0
- virtual void [update](#) (float dt)=0

18.78.1 Detailed Description

interface class for scene

Definition at line [23](#) of file [IScene.hpp](#).

18.78.2 Constructor & Destructor Documentation

18.78.2.1 ~IScene()

virtual srv::IScene::~~IScene () [virtual], [default]

18.78.3 Member Function Documentation

18.78.3.1 getId()

virtual [id](#) srv::IScene::getId () const [nodiscard], [pure virtual]

Implemented in [srv::AScene](#).

18.78.3.2 getName()

virtual std::string & srv::IScene::getName () [nodiscard], [pure virtual]

Implemented in [srv::AScene](#).

18.78.3.3 getRegistry()

virtual [ecs::Registry](#) & srv::IScene::getRegistry () [nodiscard], [pure virtual]

Implemented in [srv::AScene](#).

18.78.3.4 setName()

```
virtual void srv::IScene::setName (
    const std::string & newName) [pure virtual]
```

Implemented in [srv::AScene](#).

18.78.3.5 update()

```
virtual void srv::IScene::update (
    float dt) [pure virtual]
```

The documentation for this class was generated from the following file:

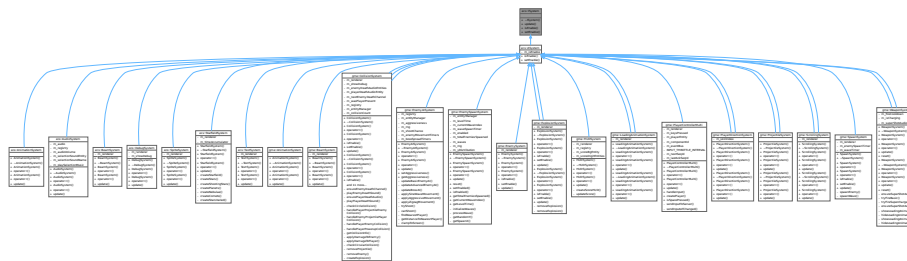
- /home/masina/Projects/Epitech/rtype/server/include/Server/Interfaces/[IScene.hpp](#)

18.79 ecs::ISystem Interface Reference

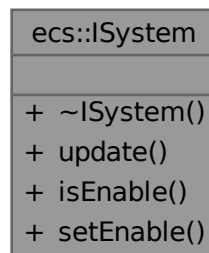
Interface class for system.

```
#include <ISystems.hpp>
```

Inheritance diagram for ecs::ISystem:



Collaboration diagram for ecs::ISystem:



Public Member Functions

- virtual [~ISystem](#) ()=default
- virtual void [update](#) ([Registry](#) ®istry, float dt)=0
- virtual bool [isEnabled](#) ()=0
- virtual void [setEnable](#) (bool enable)=0

18.79.1 Detailed Description

Interface class for system.

Definition at line [19](#) of file [ISystems.hpp](#).

18.79.2 Constructor & Destructor Documentation

18.79.2.1 ~ISystem()

virtual ecs::ISystem::~ISystem () [virtual], [default]

18.79.3 Member Function Documentation

18.79.3.1 isEnabled()

virtual bool ecs::ISystem::isEnabled () [pure virtual]

Implemented in [ecs::ASystem](#), [gme::CollisionSystem](#), [gme::CollisionSystem](#), [gme::EnemySystem](#), [gme::ExplosionSystem](#), [gme::ExplosionSystem](#), and [gme::SpawnSystem](#).

18.79.3.2 setEnable()

virtual void ecs::ISystem::setEnable (bool enable) [pure virtual]

Implemented in [ecs::ASystem](#), [gme::CollisionSystem](#), [gme::CollisionSystem](#), [gme::EnemySystem](#), [gme::ExplosionSystem](#), [gme::ExplosionSystem](#), and [gme::SpawnSystem](#).

18.79.3.3 update()

virtual void ecs::ISystem::update ([Registry](#) & registry, float dt) [pure virtual]

Implemented in [ecs::AnimationSystem](#), [ecs::AudioSystem](#), [ecs::BeamSystem](#), [ecs::DebugSystem](#), [ecs::SpriteSystem](#), [ecs::StarfieldSystem](#), [ecs::TextSystem](#), [gme::AnimationSystem](#), [gme::BeamSystem](#), [gme::CollisionSystem](#), [gme::CollisionSystem](#), [gme::CollisionSystem](#), [gme::EnemyAISystem](#), [gme::EnemySpawnSystem](#), [gme::EnemySystem](#), [gme::ExplosionSystem](#), [gme::ExplosionSystem](#), [gme::HUDSystem](#), [gme::LoadingAnimationSystem](#), [gme::LoadingAnimationSystem](#), [gme::PlayerControllerMulti](#), [gme::PlayerDirectionSystem](#), [gme::PlayerDirectionSystem](#), [gme::ProjectileSystem](#), [gme::ProjectileSystem](#), [gme::ScrollingSystem](#), [gme::ScrollingSystem](#), [gme::SpawnSystem](#), [gme::WeaponSystem](#), and [gme::WeaponSystem](#).

The documentation for this interface was generated from the following file:

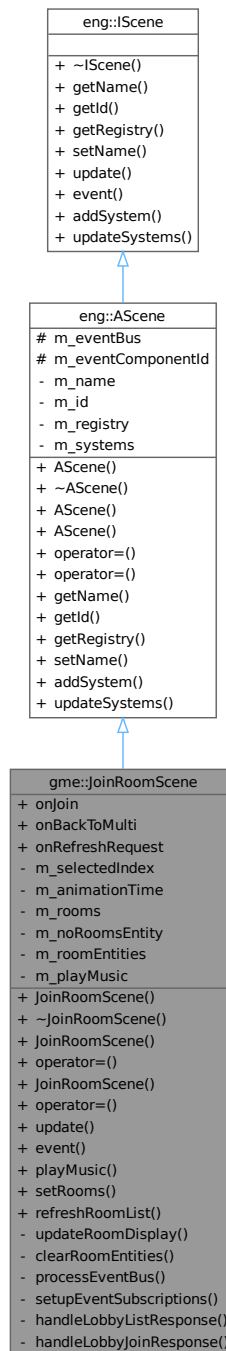
- [/home/masina/Projects/Epitech/rtype/modules/ECS/include/ECS/Interfaces/ISystems.hpp](#)

18.80 gme::JoinRoomScene Class Reference

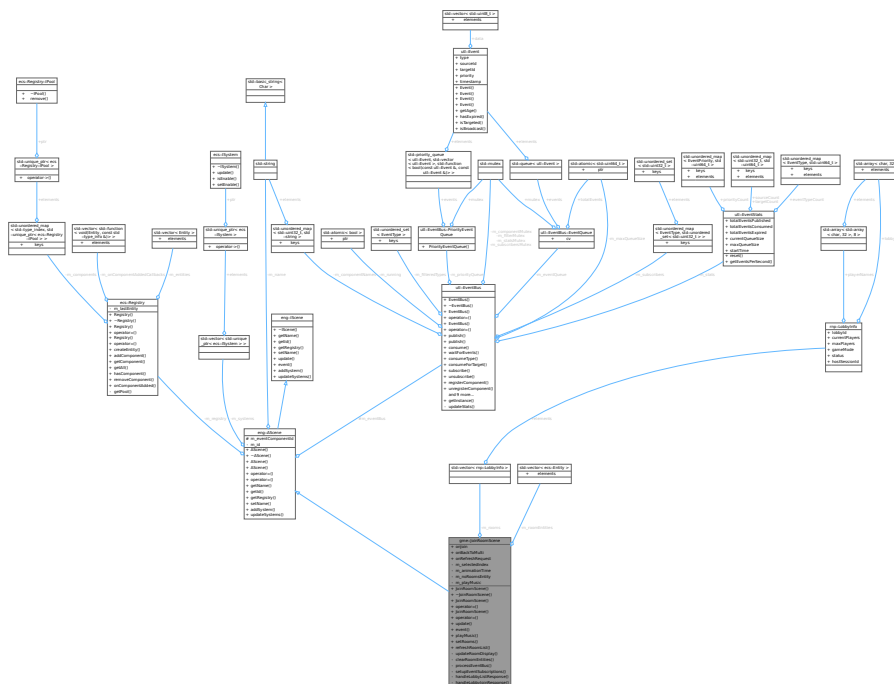
Join room scene.

#include <JoinRoom.hpp>

Inheritance diagram for gme::JoinRoomScene:



Collaboration diagram for gme::JoinRoomScene:



Public Member Functions

- `JoinRoomScene` (`eng::id` assignedId, `const std::shared_ptr< eng::IRenderer > &renderer`)
- `~JoinRoomScene` () override=default
- `JoinRoomScene` (`const JoinRoomScene &other`)=delete
- `JoinRoomScene & operator=` (`const JoinRoomScene &other`)=delete
- `JoinRoomScene` (`JoinRoomScene &&other`)=delete
- `JoinRoomScene & operator=` (`JoinRoomScene &&other`)=delete
- `void update` (`float dt`, `const eng::WindowSize &size`) override
- `void event` (`const eng::Event &event`) override
- `bool & playMusic` ()
- `void setRooms` (`const std::vector< rnp::LobbyInfo > &rooms`)
- `void refreshRoomList` () const

Public Member Functions inherited from [eng::AScene](#)

- `AScene` (const `id` assignedId)
- `~AScene` () override=default
- `AScene` (const `AScene` &other)=delete
- `AScene` (`AScene` &&other)=delete
- `AScene` & `operator=` (const `AScene` &other)=delete
- `AScene` & `operator=` (`AScene` &&other)=delete
- `std::string` & `getName` () override
- `id` `getId` () const override
- `ecs::Registry` & `getRegistry` () override
- void `setName` (const `std::string` &newName) override
- void `addSystem` (`std::unique_ptr`< `ecs::ISystem` > system) override
- void `updateSystems` (const float dt) override

Public Member Functions inherited from `eng::IScene`

- virtual $\sim$ IScene ()=default

Public Attributes

- `std::function< void(int roomId, const rnp::LobbyInfo *lobbyInfo)>` [onJoin](#)
- `std::function< void()>` [onBackToMulti](#)
- `std::function< void()>` [onRefreshRequest](#)

Private Member Functions

- `void` [updateRoomDisplay](#) ()
- `void` [clearRoomEntities](#) ()
- `void` [processEventBus](#) ()
- `void` [setupEventSubscriptions](#) () const
- `void` [handleLobbyListResponse](#) (const [utl::Event](#) &event)
- `void` [handleLobbyJoinResponse](#) (const [utl::Event](#) &event) const

Private Attributes

- `size_t` [m_selectedIndex](#) = 0
- `float` [m_animationTime](#) = 0.0f
- `std::vector< rnp::LobbyInfo >` [m_rooms](#)
- `ecs::Entity` [m_noRoomsEntity](#) = 0
- `std::vector< ecs::Entity >` [m_roomEntities](#)
- `bool` [m_playMusic](#) = false

Additional Inherited Members

Protected Attributes inherited from [eng::AScene](#)

- `utl::EventBus &` [m_eventBus](#) = `utl::EventBus::getInstance\(\)`
- `std::uint32_t` [m_eventComponentId](#) = 0

18.80.1 Detailed Description

Join room scene.

Definition at line 21 of file [JoinRoom.hpp](#).

18.80.2 Constructor & Destructor Documentation

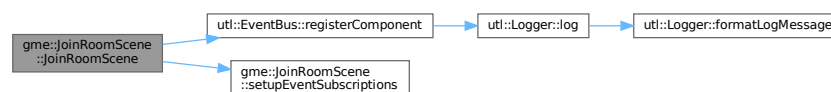
18.80.2.1 [JoinRoomScene\(\)](#) [1/3]

```
gme::JoinRoomScene::JoinRoomScene (
    eng::id assignedId,
    const std::shared_ptr< eng::IRenderer > & renderer)
```

Definition at line 9 of file [joinRoom.cpp](#).

References [eng::Color::a](#), [eng::Color::b](#), [utl::Config::Color::CYAN_ELECTRIC](#), [utl::Path::Font::FONTS_RTYPE](#), [eng::Color::g](#), [utl::Config::Color::GRAY_BLUE_SUBTLE](#), [ecs::IComponent::id](#), [utl::Config::Color::INFO_TEXT_COLOR](#), [eng::AScene::m_eventBus](#), [eng::AScene::m_eventComponentId](#), [m_noRoomsEntity](#), [eng::Color::r](#), [utl::EventBus::registerComponent\(\)](#), and [setupEventSubscriptions\(\)](#).

Here is the call graph for this function:



18.80.2.2 ~JoinRoomScene()

`gme::JoinRoomScene::~~JoinRoomScene ()` [override], [default]

18.80.2.3 JoinRoomScene() [2/3]

`gme::JoinRoomScene::JoinRoomScene (`
 const [JoinRoomScene](#) & other) [delete]

18.80.2.4 JoinRoomScene() [3/3]

`gme::JoinRoomScene::JoinRoomScene (`
 [JoinRoomScene](#) && other) [delete]

18.80.3 Member Function Documentation

18.80.3.1 clearRoomEntities()

`void gme::JoinRoomScene::clearRoomEntities ()` [private]

Definition at line 354 of file [joinRoom.cpp](#).

18.80.3.2 event()

`void gme::JoinRoomScene::event (`
 const [eng::Event](#) & event) [override], [virtual]

Implements [eng::IScene](#).

Definition at line 231 of file [joinRoom.cpp](#).

References [eng::Down](#), [eng::Enter](#), [eng::Escape](#), [eng::Event::key](#), [eng::KeyPressed](#), [eng::KeyReleased](#), [utl::LOBBY_JOIN](#), [rnp::PacketLobbyJoin::lobbyId](#), [utl::NETWORK_CLIENT](#), [eng::Event::type](#), and [eng::Up](#).

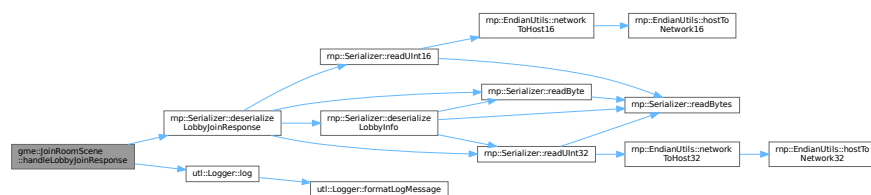
18.80.3.3 handleLobbyJoinResponse()

`void gme::JoinRoomScene::handleLobbyJoinResponse (`
 const [utl::Event](#) & event) const [private]

Definition at line 118 of file [joinRoom.cpp](#).

References [rnp::ALREADY_IN_LOBBY](#), [utl::Event::data](#), [rnp::Serializer::deserializeLobbyJoinResponse\(\)](#), [rnp::LOBBY_FULL](#), [rnp::LOBBY_NOT_FOUND](#), [utl::Logger::log\(\)](#), and [utl::WARNING](#).

Here is the call graph for this function:



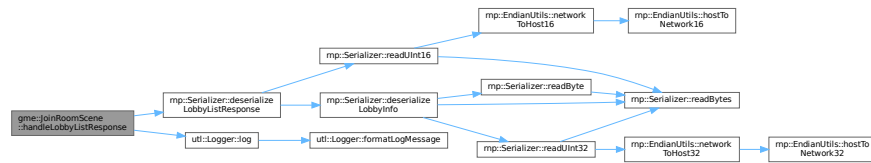
18.80.3.4 handleLobbyListResponse()

`void gme::JoinRoomScene::handleLobbyListResponse (`
 const [utl::Event](#) & event) [private]

Definition at line 85 of file [joinRoom.cpp](#).

References [utl::Event::data](#), [rnp::Serializer::deserializeLobbyListResponse\(\)](#), [utl::INFO](#), [utl::Logger::log\(\)](#), and [utl::WARNING](#).

Here is the call graph for this function:



18.80.3.5 operator=() [1/2]

[JoinRoomScene](#) & gme::JoinRoomScene::operator= (
 const [JoinRoomScene](#) & other) [delete]

18.80.3.6 operator=() [2/2]

[JoinRoomScene](#) & gme::JoinRoomScene::operator= (
 [JoinRoomScene](#) && other) [delete]

18.80.3.7 playMusic()

bool & gme::JoinRoomScene::playMusic () [inline]

Definition at line 35 of file [JoinRoom.hpp](#).

References [m_playMusic](#).

18.80.3.8 processEventBus()

void gme::JoinRoomScene::processEventBus () [private]

Definition at line 381 of file [joinRoom.cpp](#).

References [utl::LOBBY_JOIN_RESPONSE](#), and [utl::LOBBY_LIST_RESPONSE](#).

18.80.3.9 refreshRoomList()

void gme::JoinRoomScene::refreshRoomList () const

Definition at line 294 of file [joinRoom.cpp](#).

References [utl::LOBBY_LIST_REQUEST](#), and [utl::NETWORK_CLIENT](#).

18.80.3.10 setRooms()

void gme::JoinRoomScene::setRooms (
 const std::vector< [rnp::LobbyInfo](#) > & rooms)

Definition at line 288 of file [joinRoom.cpp](#).

18.80.3.11 setupEventSubscriptions()

void gme::JoinRoomScene::setupEventSubscriptions () const [private]

Definition at line 79 of file [joinRoom.cpp](#).

References [utl::LOBBY_JOIN_RESPONSE](#), and [utl::LOBBY_LIST_RESPONSE](#).

Referenced by [JoinRoomScene\(\)](#).

Here is the caller graph for this function:



18.80.3.12 update()

```
void gme::JoinRoomScene::update (
    float dt,
    const eng::WindowSize & size) [override], [virtual]
```

Implements [eng::IScene](#).

Definition at line 162 of file [joinRoom.cpp](#).

References [eng::Color::b](#), [eng::Color::g](#), [utl::Config::Color::GRAY_BLUE_SUBTLE](#), [eng::Color::r](#), and [utl::Config::Color::TEXT_VALUE_COLOR](#).

18.80.3.13 updateRoomDisplay()

```
void gme::JoinRoomScene::updateRoomDisplay () [private]
```

Definition at line 305 of file [joinRoom.cpp](#).

References [eng::Color::a](#), [eng::Color::b](#), [rnp::LobbyInfo::currentPlayers](#), [utl::Path::Font::FONTS_RTYPE](#), [eng::Color::g](#), [utl::INFO](#), [rnp::LobbyInfo::lobbyName](#), [utl::Logger::log\(\)](#), [rnp::LobbyInfo::maxPlayers](#), [eng::Color::r](#), and [utl::Config::Color::TEXT_VALUE_COLOR](#).

Here is the call graph for this function:



18.80.4 Member Data Documentation

18.80.4.1 m_animationTime

```
float gme::JoinRoomScene::m_animationTime = 0.0f [private]
```

Definition at line 46 of file [JoinRoom.hpp](#).

18.80.4.2 m_noRoomsEntity

```
ecs::Entity gme::JoinRoomScene::m_noRoomsEntity = 0 [private]
```

Definition at line 49 of file [JoinRoom.hpp](#).

Referenced by [JoinRoomScene\(\)](#).

18.80.4.3 m_playMusic

```
bool gme::JoinRoomScene::m_playMusic = false [private]
```

Definition at line 51 of file [JoinRoom.hpp](#).

Referenced by [playMusic\(\)](#).

18.80.4.4 m_roomEntities

std::vector<ecs::Entity> gme::JoinRoomScene::m_roomEntities [private]

Definition at line 50 of file [JoinRoom.hpp](#).

18.80.4.5 m_rooms

std::vector<rng::LobbyInfo> gme::JoinRoomScene::m_rooms [private]

Definition at line 47 of file [JoinRoom.hpp](#).

18.80.4.6 m_selectedIndex

size_t gme::JoinRoomScene::m_selectedIndex = 0 [private]

Definition at line 45 of file [JoinRoom.hpp](#).

18.80.4.7 onBackToMulti

std::function<void()> gme::JoinRoomScene::onBackToMulti

Definition at line 41 of file [JoinRoom.hpp](#).

18.80.4.8 onJoin

std::function<void(int roomId, const rng::LobbyInfo *lobbyInfo)> gme::JoinRoomScene::onJoin

Definition at line 40 of file [JoinRoom.hpp](#).

18.80.4.9 onRefreshRequest

std::function<void()> gme::JoinRoomScene::onRefreshRequest

Definition at line 42 of file [JoinRoom.hpp](#).

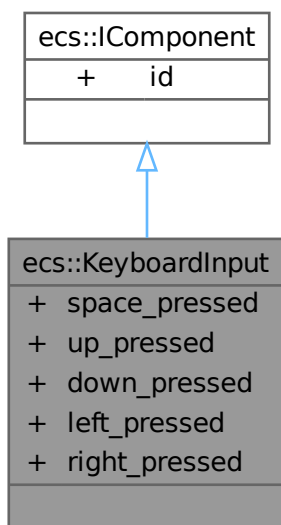
The documentation for this class was generated from the following files:

- /home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/include/RType↵ ClientMulti/Scenes/[JoinRoom.hpp](#)
- /home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/src/scenes/[joinRoom.cpp](#)

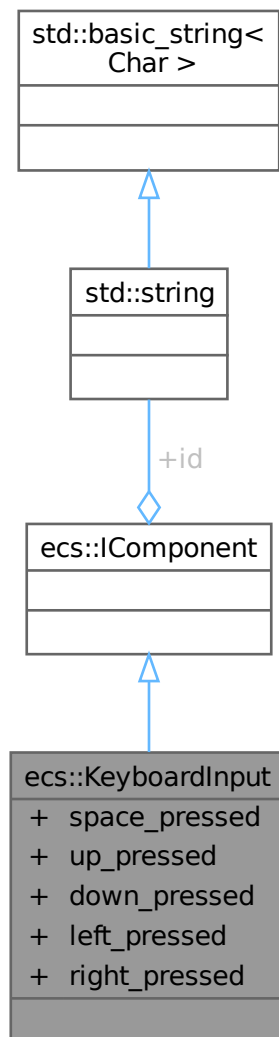
18.81 ecs::KeyboardInput Struct Reference

#include <Component.hpp>

Inheritance diagram for ecs::KeyboardInput:



Collaboration diagram for `ecs::KeyboardInput`:



Public Attributes

- `bool space_pressed {}`
- `bool up_pressed {}`
- `bool down_pressed {}`
- `bool left_pressed {}`
- `bool right_pressed {}`

Public Attributes inherited from `ecs::IComponent`

- `std::string id`

18.81.1 Detailed Description

Definition at line 143 of file `Component.hpp`.

18.81.2 Member Data Documentation

18.81.2.1 down_pressed

bool ecs::KeyboardInput::down_pressed {}

Definition at line 147 of file [Component.hpp](#).

18.81.2.2 left_pressed

bool ecs::KeyboardInput::left_pressed {}

Definition at line 148 of file [Component.hpp](#).

18.81.2.3 right_pressed

bool ecs::KeyboardInput::right_pressed {}

Definition at line 149 of file [Component.hpp](#).

18.81.2.4 space_pressed

bool ecs::KeyboardInput::space_pressed {}

Definition at line 145 of file [Component.hpp](#).

Referenced by [gme::WeaponSystem::update\(\)](#).

18.81.2.5 up_pressed

bool ecs::KeyboardInput::up_pressed {}

Definition at line 146 of file [Component.hpp](#).

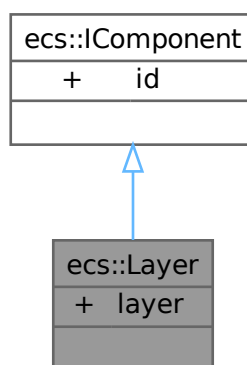
The documentation for this struct was generated from the following file:

- [/home/masina/Projects/Epitech/rtype/modules/ECS/include/ECS/Component.hpp](#)

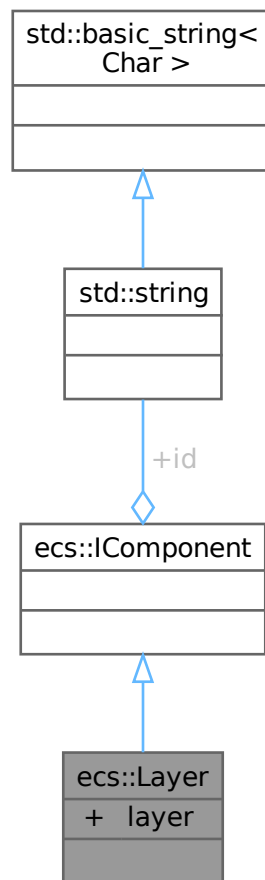
18.82 ecs::Layer Struct Reference

```
#include <Component.hpp>
```

Inheritance diagram for ecs::Layer:



Collaboration diagram for `ecs::Layer`:



Public Attributes

- `int layer`

Public Attributes inherited from `ecs::IComponent`

- `std::string id`

18.82.1 Detailed Description

Definition at line 172 of file [Component.hpp](#).

18.82.2 Member Data Documentation

18.82.2.1 `layer`

`int ecs::Layer::layer`

Definition at line 174 of file [Component.hpp](#).

Referenced by [ecs::SpriteSystem::update\(\)](#).

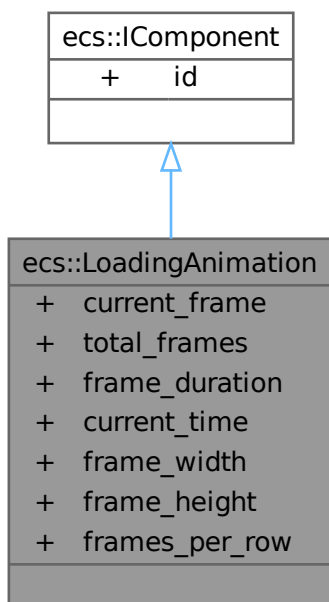
The documentation for this struct was generated from the following file:

- `/home/masina/Projects/Epitech/rtype/modules/ECS/include/ECS/Component.hpp`

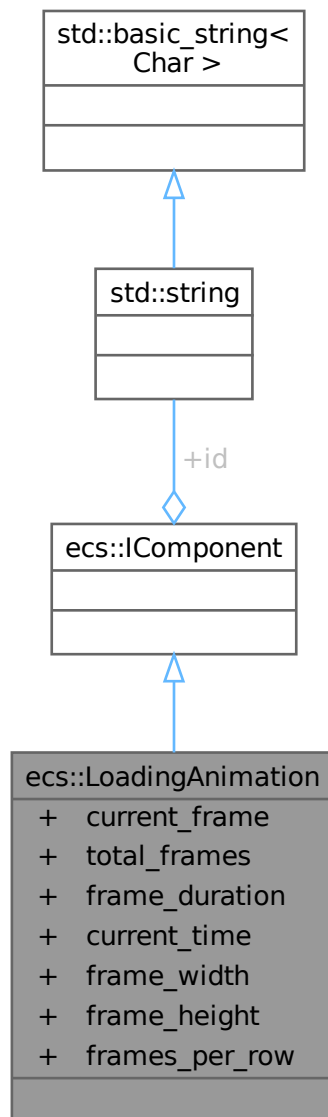
18.83 ecs::LoadingAnimation Struct Reference

#include <Component.hpp>

Inheritance diagram for ecs::LoadingAnimation:



Collaboration diagram for `ecs::LoadingAnimation`:



Public Attributes

- `int` [current_frame](#)
- `int` [total_frames](#)
- `float` [frame_duration](#)
- `float` [current_time](#)
- `float` [frame_width](#)
- `float` [frame_height](#)
- `int` [frames_per_row](#)

Public Attributes inherited from [ecs::IComponent](#)

- `std::string` [id](#)

18.83.1 Detailed Description

Definition at line 102 of file [Component.hpp](#).

18.83.2 Member Data Documentation

18.83.2.1 current_frame

int ecs::LoadingAnimation::current_frame

Definition at line 104 of file [Component.hpp](#).

18.83.2.2 current_time

float ecs::LoadingAnimation::current_time

Definition at line 107 of file [Component.hpp](#).

18.83.2.3 frame_duration

float ecs::LoadingAnimation::frame_duration

Definition at line 106 of file [Component.hpp](#).

18.83.2.4 frame_height

float ecs::LoadingAnimation::frame_height

Definition at line 109 of file [Component.hpp](#).

18.83.2.5 frame_width

float ecs::LoadingAnimation::frame_width

Definition at line 108 of file [Component.hpp](#).

18.83.2.6 frames_per_row

int ecs::LoadingAnimation::frames_per_row

Definition at line 110 of file [Component.hpp](#).

18.83.2.7 total_frames

int ecs::LoadingAnimation::total_frames

Definition at line 105 of file [Component.hpp](#).

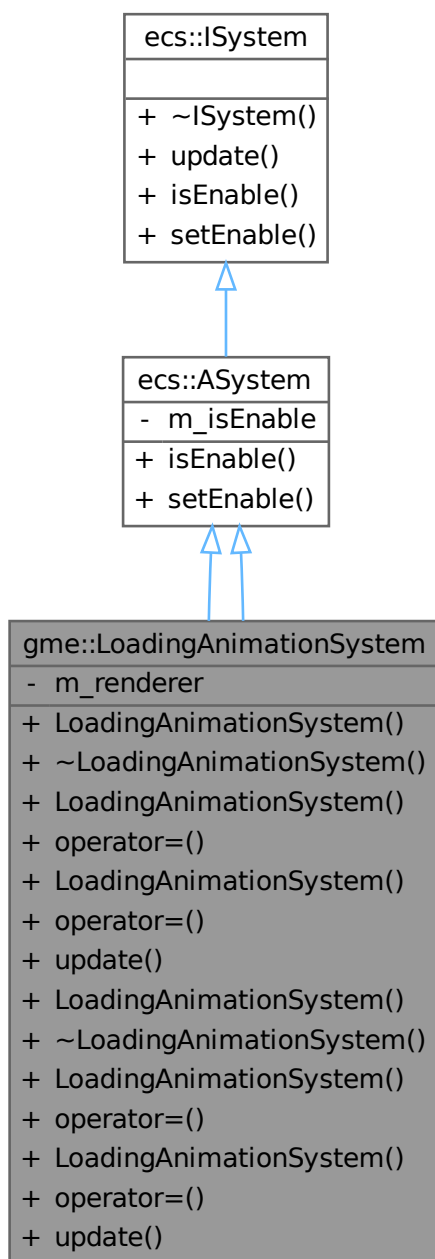
The documentation for this struct was generated from the following file:

- [/home/masina/Projects/Epitech/rtype/modules/ECS/include/ECS/Component.hpp](#)

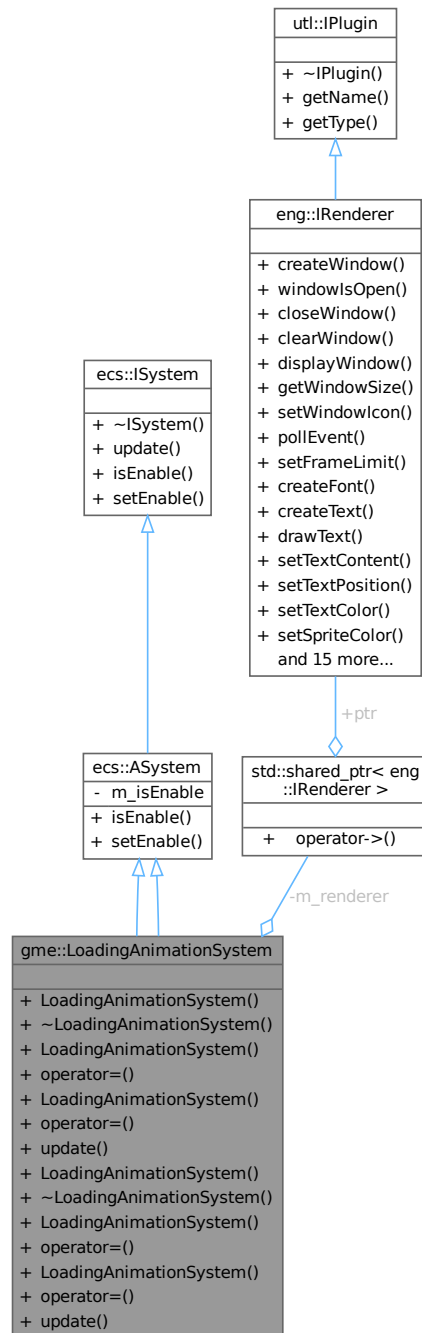
18.84 gme::LoadingAnimationSystem Class Reference

#include <LoadingAnimation.hpp>

Inheritance diagram for gme::LoadingAnimationSystem:



Collaboration diagram for gme::LoadingAnimationSystem:



Public Member Functions

- [LoadingAnimationSystem](#) (const std::shared_ptr< [eng::IRenderer](#) > &renderer)
- [~LoadingAnimationSystem](#) () override=default
- [LoadingAnimationSystem](#) (const [LoadingAnimationSystem](#) &)=delete
- [LoadingAnimationSystem](#) & operator= (const [LoadingAnimationSystem](#) &)=delete
- [LoadingAnimationSystem](#) ([LoadingAnimationSystem](#) &&)=delete
- [LoadingAnimationSystem](#) & operator= (const [LoadingAnimationSystem](#) &&)=delete

- void `update` (`ecs::Registry` ®istry, float dt) override
- `LoadingAnimationSystem` (const std::shared_ptr< `eng::IRenderer` > &renderer)
- `~LoadingAnimationSystem` () override=default
- `LoadingAnimationSystem` (const `LoadingAnimationSystem` &)=delete
- `LoadingAnimationSystem` & operator= (const `LoadingAnimationSystem` &)=delete
- `LoadingAnimationSystem` (`LoadingAnimationSystem` &&)=delete
- `LoadingAnimationSystem` & operator= (`LoadingAnimationSystem` &&)=delete
- void `update` (`ecs::Registry` ®istry, float dt) override

Public Member Functions inherited from `ecs::ASystem`

- bool `isEnabled` () override
- void `setEnabled` (const bool enable) override

Public Member Functions inherited from `ecs::ISystem`

- virtual `~ISystem` ()=default

Private Attributes

- const std::shared_ptr< `eng::IRenderer` > & `m_renderer`

18.84.1 Detailed Description

Definition at line 17 of file `LoadingAnimation.hpp`.

18.84.2 Constructor & Destructor Documentation

18.84.2.1 `LoadingAnimationSystem`() [1/6]

```
gme::LoadingAnimationSystem::LoadingAnimationSystem (
    const std::shared_ptr< eng::IRenderer > & renderer)  [inline], [explicit]
```

Definition at line 14 of file `LoadingAnimation.hpp`.

18.84.2.2 `~LoadingAnimationSystem`() [1/2]

```
gme::LoadingAnimationSystem::~~LoadingAnimationSystem ()  [override], [default]
```

18.84.2.3 `LoadingAnimationSystem`() [2/6]

```
gme::LoadingAnimationSystem::LoadingAnimationSystem (
    const LoadingAnimationSystem & )  [delete]
```

18.84.2.4 `LoadingAnimationSystem`() [3/6]

```
gme::LoadingAnimationSystem::LoadingAnimationSystem (
    LoadingAnimationSystem && )  [delete]
```

18.84.2.5 `LoadingAnimationSystem`() [4/6]

```
gme::LoadingAnimationSystem::LoadingAnimationSystem (
    const std::shared_ptr< eng::IRenderer > & renderer)  [inline], [explicit]
```

Definition at line 20 of file `LoadingAnimation.hpp`.

18.84.2.6 `~LoadingAnimationSystem`() [2/2]

```
gme::LoadingAnimationSystem::~~LoadingAnimationSystem ()  [override], [default]
```

18.84.2.7 `LoadingAnimationSystem`() [5/6]

```
gme::LoadingAnimationSystem::LoadingAnimationSystem (
    const LoadingAnimationSystem & )  [delete]
```


18.84.2.8 LoadingAnimationSystem() [6/6]

`gme::LoadingAnimationSystem::LoadingAnimationSystem (`
 [LoadingAnimationSystem](#) &&) [delete]

18.84.3 Member Function Documentation

18.84.3.1 operator=() [1/4]

[LoadingAnimationSystem](#) & `gme::LoadingAnimationSystem::operator= (`
 const [LoadingAnimationSystem](#) &&) [delete]

18.84.3.2 operator=() [2/4]

[LoadingAnimationSystem](#) & `gme::LoadingAnimationSystem::operator= (`
 const [LoadingAnimationSystem](#) &) [delete]

18.84.3.3 operator=() [3/4]

[LoadingAnimationSystem](#) & `gme::LoadingAnimationSystem::operator= (`
 const [LoadingAnimationSystem](#) &) [delete]

18.84.3.4 operator=() [4/4]

[LoadingAnimationSystem](#) & `gme::LoadingAnimationSystem::operator= (`
 [LoadingAnimationSystem](#) &&) [delete]

18.84.3.5 update() [1/2]

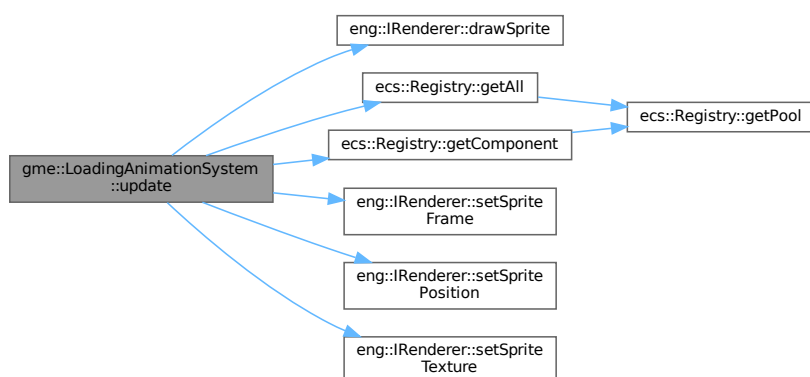
`void gme::LoadingAnimationSystem::update (`
 [ecs::Registry](#) & registry,
 float dt) [inline], [override], [virtual]

Implements [ecs::ISystem](#).

Definition at line 22 of file [LoadingAnimation.hpp](#).

References [eng::IRenderer::drawSprite\(\)](#), [ecs::Registry::getAll\(\)](#), [ecs::Registry::getComponent\(\)](#), [ecs::IComponent::id](#), [m_renderer](#), [eng::IRenderer::setSpriteFrame\(\)](#), [eng::IRenderer::setSpritePosition\(\)](#), and [eng::IRenderer::setSpriteTexture\(\)](#).

Here is the call graph for this function:



18.84.3.6 update() [2/2]

`void gme::LoadingAnimationSystem::update (`

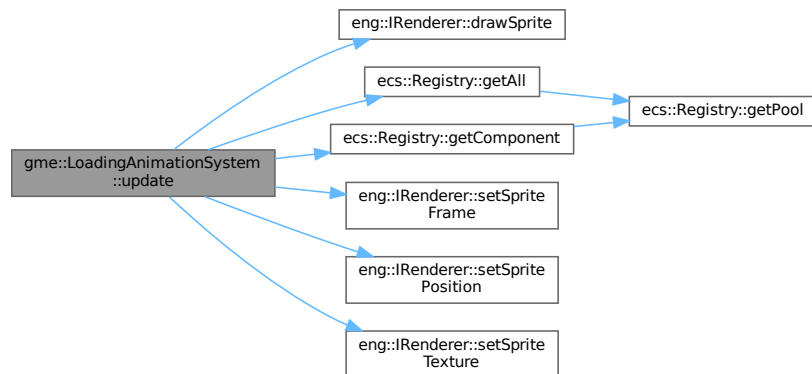
```
ecs::Registry & registry,
float dt) [inline], [override], [virtual]
```

Implements [ecs::ISystem](#).

Definition at line 28 of file [LoadingAnimation.hpp](#).

References [eng::IRenderer::drawSprite\(\)](#), [ecs::Registry::getAll\(\)](#), [ecs::Registry::getComponent\(\)](#), [ecs::IComponent::id](#), [m_renderer](#), [eng::IRenderer::setSpriteFrame\(\)](#), [eng::IRenderer::setSpritePosition\(\)](#), and [eng::IRenderer::setSpriteTexture\(\)](#).

Here is the call graph for this function:



18.84.4 Member Data Documentation

18.84.4.1 m_renderer

```
const std::shared_ptr< eng::IRenderer > & gme::LoadingAnimationSystem::m_renderer [private]
```

Definition at line 66 of file [LoadingAnimation.hpp](#).

Referenced by [update\(\)](#).

The documentation for this class was generated from the following files:

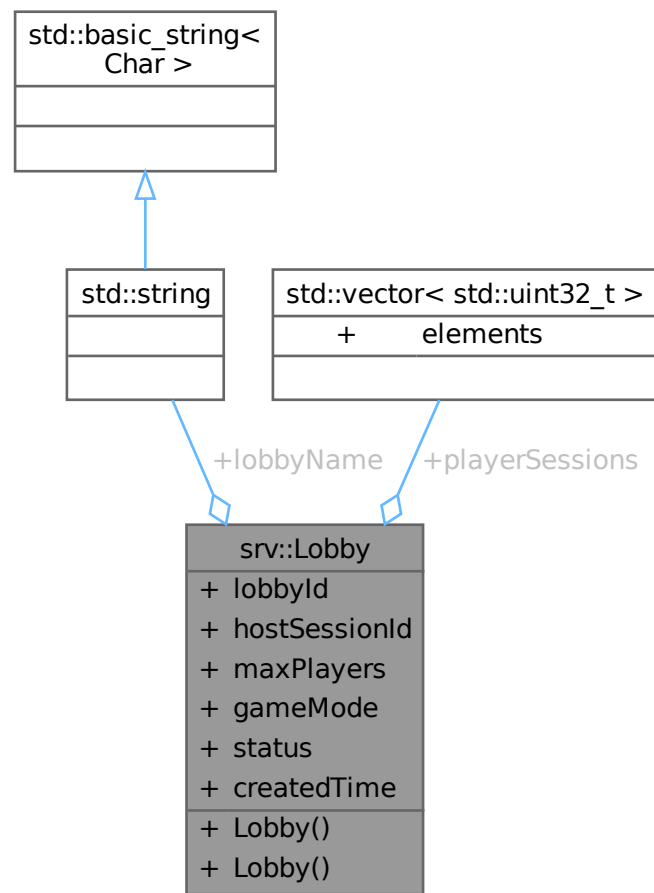
- `/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/include/RTypeClientMulti/Systems/LoadingAnimation.hpp`
- `/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Solo/include/RTypeClientSolo/Systems/LoadingAnimation.hpp`

18.85 srv::Lobby Struct Reference

[Lobby](#) structure.

```
#include <AsioServer.hpp>
```

Collaboration diagram for srv::Lobby:



Public Member Functions

- [Lobby](#) ()
- [Lobby](#) (std::uint32_t id, const std::string &name, std::uint32_t host, std::uint8_t max, std::uint8_t mode)

Public Attributes

- std::uint32_t [lobbyId](#)
- std::string [lobbyName](#)
- std::uint32_t [hostSessionId](#)
- std::vector< std::uint32_t > [playerSessions](#)
- std::uint8_t [maxPlayers](#)
- std::uint8_t [gameMode](#)
- [rnp::LobbyStatus](#) [status](#)
- std::chrono::steady_clock::time_point [createTime](#)

18.85.1 Detailed Description

[Lobby](#) structure.

Definition at line 62 of file [AsioServer.hpp](#).

18.85.2 Constructor & Destructor Documentation

18.85.2.1 Lobby() [1/2]

srv::Lobby::Lobby () [inline]

Definition at line 73 of file [AsioServer.hpp](#).

18.85.2.2 Lobby() [2/2]

```
srv::Lobby::Lobby (  
    std::uint32_t id,  
    const std::string & name,  
    std::uint32_t host,  
    std::uint8_t max,  
    std::uint8_t mode) [inline]
```

Definition at line 78 of file [AsioServer.hpp](#).

18.85.3 Member Data Documentation

18.85.3.1 createdTime

std::chrono::steady_clock::time_point srv::Lobby::createdTime

Definition at line 71 of file [AsioServer.hpp](#).

18.85.3.2 gameMode

std::uint8_t srv::Lobby::gameMode

Definition at line 69 of file [AsioServer.hpp](#).

Referenced by [srv::AsioServer::lobbyToLobbyInfo\(\)](#).

18.85.3.3 hostSessionId

std::uint32_t srv::Lobby::hostSessionId

Definition at line 66 of file [AsioServer.hpp](#).

Referenced by [srv::AsioServer::lobbyToLobbyInfo\(\)](#).

18.85.3.4 lobbyId

std::uint32_t srv::Lobby::lobbyId

Definition at line 64 of file [AsioServer.hpp](#).

Referenced by [srv::AsioServer::lobbyToLobbyInfo\(\)](#).

18.85.3.5 lobbyName

std::string srv::Lobby::lobbyName

Definition at line 65 of file [AsioServer.hpp](#).

Referenced by [srv::AsioServer::lobbyToLobbyInfo\(\)](#).

18.85.3.6 maxPlayers

std::uint8_t srv::Lobby::maxPlayers

Definition at line 68 of file [AsioServer.hpp](#).

Referenced by [srv::AsioServer::joinLobby\(\)](#), and [srv::AsioServer::lobbyToLobbyInfo\(\)](#).

18.85.3.7 playerSessions

std::vector<std::uint32_t> srv::Lobby::playerSessions

Definition at line 67 of file [AsioServer.hpp](#).

Referenced by [srv::AsioServer::createLobby\(\)](#), [srv::AsioServer::joinLobby\(\)](#), and [srv::AsioServer::lobbyToLobbyInfo\(\)](#).

18.85.3.8 status

[rnp::LobbyStatus](#) `srv::Lobby::status`

Definition at line 70 of file [AsioServer.hpp](#).

Referenced by [srv::AsioServer::joinLobby\(\)](#), and [srv::AsioServer::lobbyToLobbyInfo\(\)](#).

The documentation for this struct was generated from the following file:

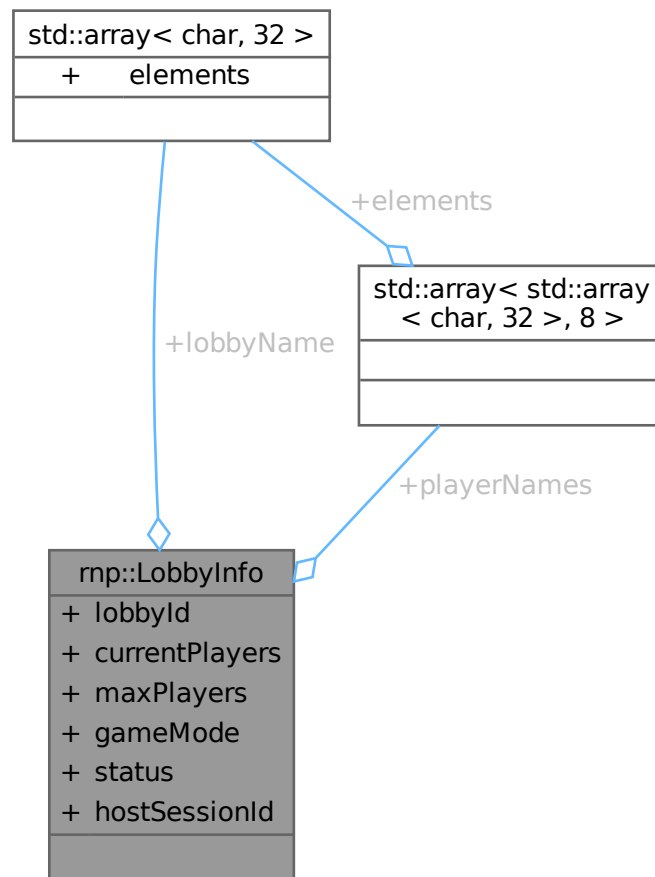
- `/home/masina/Projects/Epitech/rtype/plugins/Network/Asio/Server/include/AsioServer/AsioServer.hpp`

18.86 rnp::LobbyInfo Struct Reference

Lobby information structure.

#include <Protocol.hpp>

Collaboration diagram for `rnp::LobbyInfo`:



Public Attributes

- `std::uint32_t` [lobbyId](#)
- `std::array< char, 32 >` [lobbyName](#)
- `std::uint8_t` [currentPlayers](#)
- `std::uint8_t` [maxPlayers](#)
- `std::uint8_t` [gameMode](#)
- `std::uint8_t` [status](#)

- `std::uint32_t` [hostSessionId](#)
- `std::array< std::array< char, 32 >, 8 >` [playerNames](#)

18.86.1 Detailed Description

Lobby information structure.

Definition at line 234 of file [Protocol.hpp](#).

18.86.2 Member Data Documentation

18.86.2.1 currentPlayers

`std::uint8_t` `rnp::LobbyInfo::currentPlayers`

Definition at line 238 of file [Protocol.hpp](#).

Referenced by [rnp::Serializer::deserializeLobbyInfo\(\)](#), [srv::AsioServer::handleLobbyJoin\(\)](#), [gme::WaitingRoomScene::handleLobbyJoin\(\)](#), [srv::AsioServer::lobbyToLobbyInfo\(\)](#), [srv::AsioServer::sendLobbyList\(\)](#), [rnp::Serializer::serializeLobbyInfo\(\)](#), [gme::WaitingRoomScene::setLobbyInfo\(\)](#), and [gme::JoinRoomScene::updateRoomDisplay\(\)](#).

18.86.2.2 gameMode

`std::uint8_t` `rnp::LobbyInfo::gameMode`

Definition at line 240 of file [Protocol.hpp](#).

Referenced by [rnp::Serializer::deserializeLobbyInfo\(\)](#), [srv::AsioServer::lobbyToLobbyInfo\(\)](#), and [rnp::Serializer::serializeLobbyInfo\(\)](#).

18.86.2.3 hostSessionId

`std::uint32_t` `rnp::LobbyInfo::hostSessionId`

Definition at line 242 of file [Protocol.hpp](#).

Referenced by [rnp::Serializer::deserializeLobbyInfo\(\)](#), [srv::AsioServer::lobbyToLobbyInfo\(\)](#), and [rnp::Serializer::serializeLobbyInfo\(\)](#).

18.86.2.4 lobbyId

`std::uint32_t` `rnp::LobbyInfo::lobbyId`

Definition at line 236 of file [Protocol.hpp](#).

Referenced by [rnp::Serializer::deserializeLobbyInfo\(\)](#), [eng::AsioClient::handleLobbyUpdate\(\)](#), [gme::WaitingRoomScene::handleLobbyUpdate\(\)](#), [srv::AsioServer::lobbyToLobbyInfo\(\)](#), [srv::AsioServer::sendLobbyList\(\)](#), [rnp::Serializer::serializeLobbyInfo\(\)](#), and [gme::WaitingRoomScene::setLobbyInfo\(\)](#).

18.86.2.5 lobbyName

`std::array<char, 32>` `rnp::LobbyInfo::lobbyName`

Definition at line 237 of file [Protocol.hpp](#).

Referenced by [rnp::Serializer::deserializeLobbyInfo\(\)](#), [srv::AsioServer::lobbyToLobbyInfo\(\)](#), [rnp::Serializer::serializeLobbyInfo\(\)](#), and [gme::JoinRoomScene::updateRoomDisplay\(\)](#).

18.86.2.6 maxPlayers

`std::uint8_t` `rnp::LobbyInfo::maxPlayers`

Definition at line 239 of file [Protocol.hpp](#).

Referenced by [rnp::Serializer::deserializeLobbyInfo\(\)](#), [srv::AsioServer::handleLobbyJoin\(\)](#), [gme::WaitingRoomScene::handleLobbyJoin\(\)](#), [srv::AsioServer::lobbyToLobbyInfo\(\)](#), [srv::AsioServer::sendLobbyList\(\)](#), [rnp::Serializer::serializeLobbyInfo\(\)](#), [gme::WaitingRoomScene::setLobbyInfo\(\)](#), and [gme::JoinRoomScene::updateRoomDisplay\(\)](#).

18.86.2.7 playerNames

`std::array<std::array<char, 32>, 8>` `rnp::LobbyInfo::playerNames`

Definition at line 243 of file [Protocol.hpp](#).

Referenced by [rnp::Serializer::deserializeLobbyInfo\(\)](#), [srv::AsioServer::lobbyToLobbyInfo\(\)](#), and [rnp::Serializer::serializeLobbyInfo\(\)](#).

18.86.2.8 status

std::uint8_t rnp::LobbyInfo::status

Definition at line 241 of file [Protocol.hpp](#).

Referenced by [rnp::Serializer::deserializeLobbyInfo\(\)](#), [srv::AsioServer::lobbyToLobbyInfo\(\)](#), and [rnp::Serializer::serializeLobbyInfo\(\)](#).

The documentation for this struct was generated from the following file:

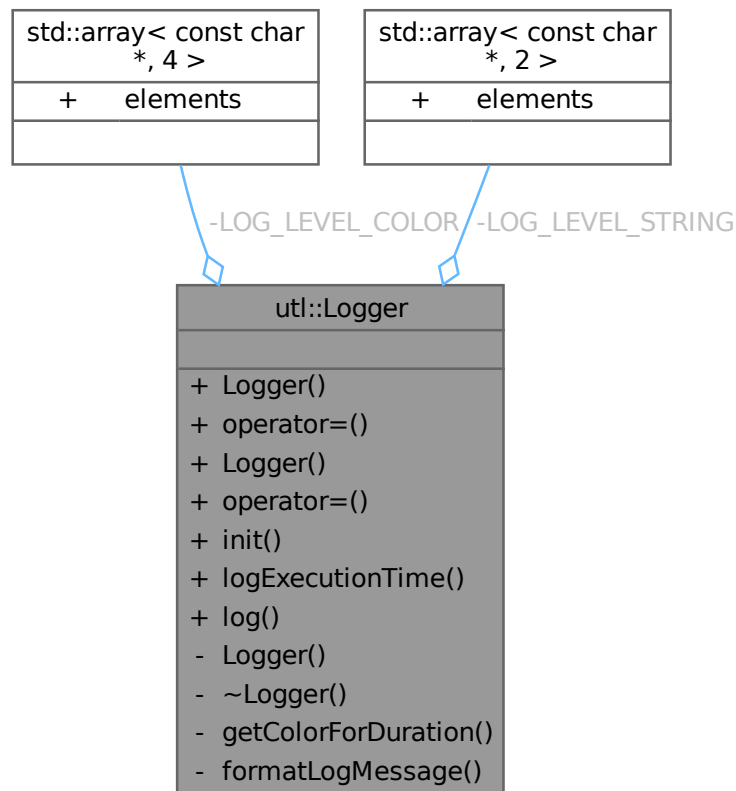
- [/home/masina/Projects/Epitech/rtype/modules/Interfaces/include/Interfaces/Protocol/Protocol.hpp](#)

18.87 utl::Logger Class Reference

Class for logging.

#include <Logger.hpp>

Collaboration diagram for utl::Logger:



Public Member Functions

- [Logger](#) (const [Logger](#) &)=delete
- [Logger](#) & [operator=](#) (const [Logger](#) &)=delete
- [Logger](#) ([Logger](#) &&)=delete
- [Logger](#) & [operator=](#) ([Logger](#) &&)=delete

Static Public Member Functions

- static void [init](#) ()

- `template<typename Func >`
`static void logExecutionTime (const std::string &message, Func &&func)`
- `static void log (const std::string &message, const LogLevel &logLevel)`

Private Types

- `enum ColorIndex : uint8_t { COLOR\_ERROR , COLOR\_INFO , COLOR\_WARNING , COLOR\_RESET }`

Private Member Functions

- `Logger ()=default`
- `~Logger ()=default`

Static Private Member Functions

- `static const char * getColorForDuration (const float duration)`
- `static std::string formatLogMessage (LogLevel level, const std::string &message)`

Static Private Attributes

- `static constexpr std::array< const char *, 4 > LOG\_LEVEL\_COLOR`
- `static constexpr std::array< const char *, 2 > LOG\_LEVEL\_STRING = {"INFO", "WARNING"}`

18.87.1 Detailed Description

Class for logging.

Definition at line 28 of file [Logger.hpp](#).

18.87.2 Member Enumeration Documentation

18.87.2.1 ColorIndex

`enum utl::Logger::ColorIndex : uint8_t [private]`

Enumerator

COLOR_ERROR	
COLOR_INFO	
COLOR_WARNING	
COLOR_RESET	

Definition at line 58 of file [Logger.hpp](#).

18.87.3 Constructor & Destructor Documentation

18.87.3.1 [Logger](#)() [1/3]

`utl::Logger::Logger (`
`const Logger & ) [delete]`

18.87.3.2 [Logger](#)() [2/3]

`utl::Logger::Logger (`
`Logger && ) [delete]`

18.87.3.3 [Logger](#)() [3/3]

`utl::Logger::Logger () [private], [default]`

18.87.3.4 ~Logger()

utl::Logger::~Logger () [private], [default]

18.87.4 Member Function Documentation

18.87.4.1 formatLogMessage()

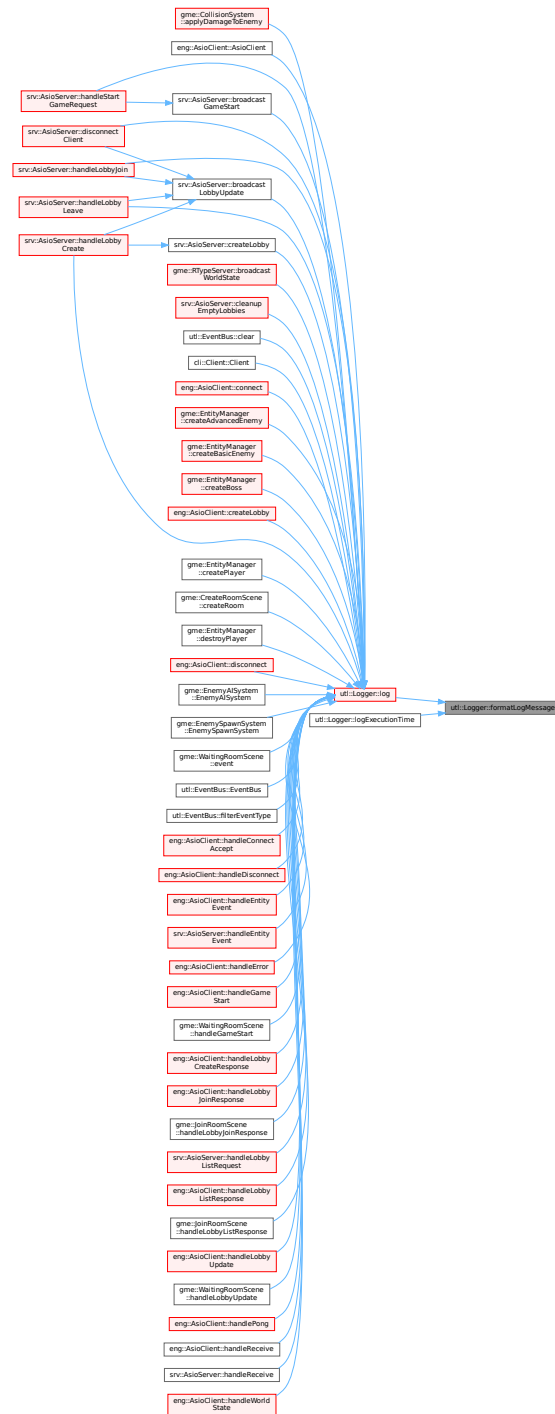
static std::string utl::Logger::formatLogMessage (
 [LogLevel](#) level,
 const std::string & message) [inline], [static], [nodiscard], [private]

Definition at line 85 of file [Logger.hpp](#).

References [LOG_LEVEL_STRING](#).

Referenced by [log\(\)](#), and [logExecutionTime\(\)](#).

Here is the caller graph for this function:



18.87.4.2 getColorForDuration()

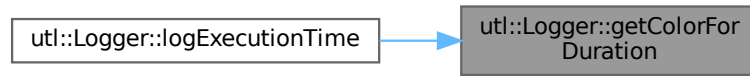
```
static const char * utl::Logger::getColorForDuration (
    const float duration) [inline], [static], [nodiscard], [private]
```

Definition at line 78 of file [Logger.hpp](#).

References [COLOR_ERROR](#), [COLOR_INFO](#), [COLOR_WARNING](#), and [LOG_LEVEL_COLOR](#).

Referenced by [logExecutionTime\(\)](#).

Here is the caller graph for this function:



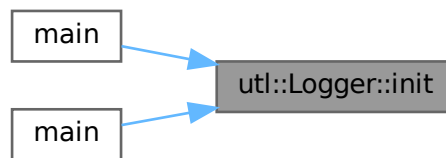
18.87.4.3 init()

`void utl::Logger::init () [static]`

Definition at line 7 of file [logger.cpp](#).

Referenced by [main\(\)](#), and [main\(\)](#).

Here is the caller graph for this function:



18.87.4.4 log()

`static void utl::Logger::log (`
 `const std::string & message,`
 `const LogLevel & logLevel) [inline], [static]`

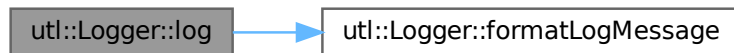
Definition at line 51 of file [Logger.hpp](#).

References [COLOR_INFO](#), [COLOR_RESET](#), [COLOR_WARNING](#), [formatLogMessage\(\)](#), [utl::INFO](#), and [LOG_LEVEL_COLOR](#).

Referenced by [gme::CollisionSystem::applyDamageToEnemy\(\)](#), [eng::AsioClient::AsioClient\(\)](#), [srv::AsioServer::broadcastG](#), [srv::AsioServer::broadcastLobbyUpdate\(\)](#), [gme::RTypeServer::broadcastWorldState\(\)](#), [srv::AsioServer::cleanupEmptyLob](#), [utl::EventBus::clear\(\)](#), [cli::Client::Client\(\)](#), [eng::AsioClient::connect\(\)](#), [gme::EntityManager::createAdvancedEnemy\(\)](#), [gme::EntityManager::createBasicEnemy\(\)](#), [gme::EntityManager::createBoss\(\)](#), [eng::AsioClient::createLobby\(\)](#), [srv::AsioServer::createLobby\(\)](#), [gme::EntityManager::createPlayer\(\)](#), [gme::CreateRoomScene::createRoom\(\)](#), [gme::EntityManager::destroyPlayer\(\)](#), [eng::AsioClient::disconnect\(\)](#), [srv::AsioServer::disconnectClient\(\)](#), [gme::EnemyAISystem::EnemyAISystem\(\)](#), [gme::EnemySpawnSystem::EnemySpawnSystem\(\)](#), [gme::WaitingRoomScene::e](#), [utl::EventBus::EventBus\(\)](#), [utl::EventBus::filterEventType\(\)](#), [eng::AsioClient::handleConnectAccept\(\)](#), [eng::AsioClient::handleDisconnect\(\)](#), [eng::AsioClient::handleEntityEvent\(\)](#), [srv::AsioServer::handleEntityEvent\(\)](#), [eng::AsioClient::handleError\(\)](#), [eng::AsioClient::handleGameStart\(\)](#), [gme::WaitingRoomScene::handleGameStart\(\)](#), [srv::AsioServer::handleLobbyCreate\(\)](#), [eng::AsioClient::handleLobbyCreateResponse\(\)](#), [srv::AsioServer::handleLobbyJoin](#), [eng::AsioClient::handleLobbyJoinResponse\(\)](#), [gme::JoinRoomScene::handleLobbyJoinResponse\(\)](#), [srv::AsioServer::handleLobbyLeave\(\)](#), [srv::AsioServer::handleLobbyListRequest\(\)](#), [eng::AsioClient::handleLobbyListRespo](#), [gme::JoinRoomScene::handleLobbyListResponse\(\)](#), [eng::AsioClient::handleLobbyUpdate\(\)](#), [gme::WaitingRoomScene::han](#), [eng::AsioClient::handlePong\(\)](#), [eng::AsioClient::handleReceive\(\)](#), [srv::AsioServer::handleReceive\(\)](#), [srv::AsioServer::handleStartGameRequest\(\)](#), [eng::AsioClient::handleWorldState\(\)](#), [gme::GameMulti::handleWorldStateUp](#), [eng::AsioClient::joinLobby\(\)](#), [srv::AsioServer::joinLobby\(\)](#), [eng::AsioClient::leaveLobby\(\)](#), [gme::WaitingRoomScene::leavel](#)

[srv::AsioServer::leaveLobby\(\)](#), [utl::PluginLoader::loadPlugin\(\)](#), [main\(\)](#), [main\(\)](#), [eng::AsioClient::networkThreadLoop\(\)](#),
[cli::ArgsHandler::ParseArgs\(\)](#), [srv::ArgsHandler::ParseArgs\(\)](#), [eng::AsioClient::processBusEvent\(\)](#),
[gme::CreateRoomScene::processEventBus\(\)](#), [gme::WaitingRoomScene::processEventBus\(\)](#), [gme::RTypeServer::processGameEvent\(\)](#),
[gme::RTypeServer::processPlayerInputEvent\(\)](#), [eng::AsioClient::processSendQueue\(\)](#), [gme::RTypeServer::processServerStatusEvent\(\)](#),
[gme::EnemySpawnSystem::processWave\(\)](#), [utl::EventBus::publish\(\)](#), [utl::EventBus::publish\(\)](#), [utl::EventBus::registerComponent\(\)](#),
[eng::AsioClient::requestLobbyList\(\)](#), [eng::AsioClient::requestStartGame\(\)](#), [gme::EnemySpawnSystem::reset\(\)](#),
[gme::RTypeServer::RTypeServer\(\)](#), [eng::AsioClient::sendConnect\(\)](#), [eng::AsioClient::sendDisconnect\(\)](#),
[srv::AsioServer::sendLobbyCreateResponse\(\)](#), [srv::AsioServer::sendLobbyJoinResponse\(\)](#), [srv::AsioServer::sendLobbyListResponse\(\)](#),
[eng::AsioClient::sendPacketImmediate\(\)](#), [srv::AsioServer::sendPacketImmediate\(\)](#), [eng::AsioClient::sendToServer\(\)](#),
[srv::Server::Server\(\)](#), [eng::AsioClient::setClientCapabilities\(\)](#), [gme::WaitingRoomScene::setLobbyId\(\)](#),
[gme::WaitingRoomScene::setLobbyInfo\(\)](#), [eng::AsioClient::setPlayerName\(\)](#), [eng::AsioClient::setupPacketHandlers\(\)](#),
[gme::RTypeClientMulti::setupScenes\(\)](#), [gme::RTypeServer::start\(\)](#), [gme::WaitingRoomScene::startGame\(\)](#),
[srv::AsioServer::startReceive\(\)](#), [gme::RTypeServer::stop\(\)](#), [utl::EventBus::stop\(\)](#), [utl::EventBus::subscribe\(\)](#),
[utl::EventBus::unfilterEventType\(\)](#), [utl::EventBus::unregisterComponent\(\)](#), [utl::EventBus::unsubscribe\(\)](#),
[gme::EnemySpawnSystem::update\(\)](#), [eng::AsioClient::updateConnectionManagement\(\)](#), [gme::WaitingRoomScene::updateGameInfo\(\)](#),
[gme::JoinRoomScene::updateRoomDisplay\(\)](#), [eng::AsioClient::~~AsioClient\(\)](#), and [utl::EventBus::~~EventBus\(\)](#).

Here is the call graph for this function:



18.87.4.5 logExecutionTime()

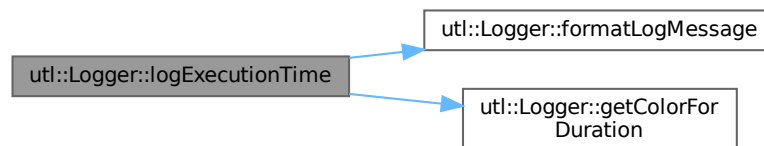
```

template<typename Func >
static void utl::Logger::logExecutionTime (
    const std::string & message,
    Func && func) [inline], [static]
  
```

Definition at line 39 of file [Logger.hpp](#).

References [COLOR_RESET](#), [formatLogMessage\(\)](#), [getColorForDuration\(\)](#), [utl::INFO](#), and [LOG_LEVEL_COLOR](#).

Here is the call graph for this function:



18.87.4.6 operator=() [1/2]

```

Logger & utl::Logger::operator= (
    const Logger & ) [delete]
  
```

18.87.4.7 operator=() [2/2]

```

Logger & utl::Logger::operator= (
  
```

[Logger](#) &&) [delete]

18.87.5 Member Data Documentation

18.87.5.1 LOG_LEVEL_COLOR

std::array<const char *, 4> utl::Logger::LOG_LEVEL_COLOR [static], [constexpr], [private]

Initial value:

```
= {  
    "\033[31m",  
    "\033[32m",  
    "\033[33m",  
    "\033[0m\n"  
}
```

Definition at line 66 of file [Logger.hpp](#).

Referenced by [getColorForDuration\(\)](#), [log\(\)](#), and [logExecutionTime\(\)](#).

18.87.5.2 LOG_LEVEL_STRING

std::array<const char *, 2> utl::Logger::LOG_LEVEL_STRING = {"INFO", "WARNING"} [static], [constexpr], [private]

Definition at line 73 of file [Logger.hpp](#).

Referenced by [formatLogMessage\(\)](#).

The documentation for this class was generated from the following files:

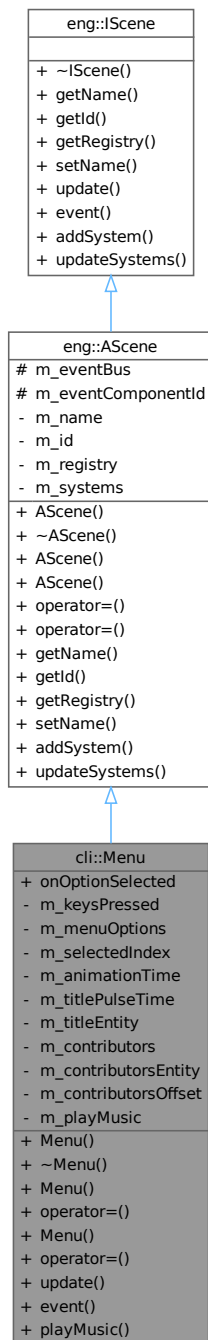
- [/home/masina/Projects/Epitech/rtype/modules/Utils/include/Utils/Logger.hpp](#)
- [/home/masina/Projects/Epitech/rtype/modules/Utils/src/logger.cpp](#)

18.88 cli::Menu Class Reference

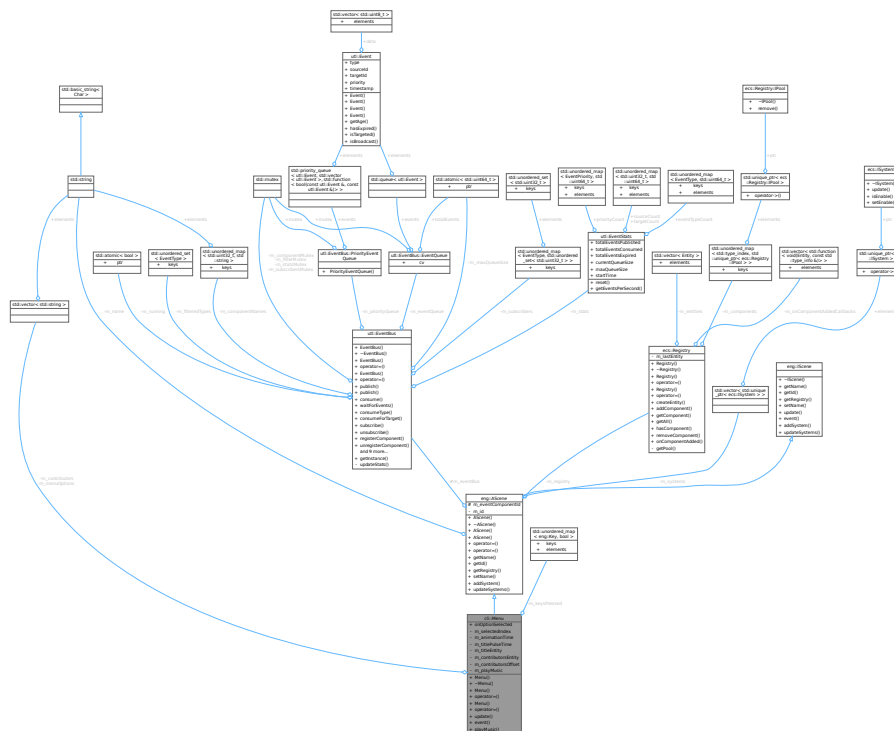
[Menu](#) scene.

```
#include <Menu.hpp>
```

Inheritance diagram for cli::Menu:



Collaboration diagram for cli::Menu:



Public Member Functions

- `Menu` (`eng::id assignedId`, `const std::shared_ptr< eng::IRenderer > &renderer`)
- `~Menu` () override=default
- `Menu` (`const Menu &other`)=delete
- `Menu & operator=` (`const Menu &other`)=delete
- `Menu` (`Menu &&other`)=delete
- `Menu & operator=` (`Menu &&other`)=delete
- `void update` (`float dt`, `const eng::WindowSize &size`) override
- `void event` (`const eng::Event &event`) override
- `bool & playMusic` ()

Public Member Functions inherited from [eng::AScene](#)

- `AScene (const id assignedId)`
- `~AScene ()` override=default
- `AScene (const AScene &other)=delete`
- `AScene (AScene &&other)=delete`
- `AScene & operator= (const AScene &other)=delete`
- `AScene & operator= (AScene &&other)=delete`
- `std::string & getName ()` override
- `id getId ()` const override
- `ecs::Registry & getRegistry ()` override
- `void setName (const std::string &newName)` override
- `void addSystem (std::unique_ptr< ecs::ISystem > system)` override
- `void updateSystems (const float dt)` override

Public Member Functions inherited from [eng::IScene](#)

- virtual $\sim$ IScene ()=default

Public Attributes

- `std::function< void(const std::string &option)>` [onOptionSelected](#)

Private Attributes

- `std::unordered_map< eng::Key, bool >` [m_keysPressed](#)
- `const std::vector< std::string >` [m_menuOptions](#) = {"Solo", "Multi", "Settings"}
- `int` [m_selectedIndex](#) = 0
- `float` [m_animationTime](#) = 0.0f
- `float` [m_titlePulseTime](#) = 0.0f
- `ecs::Entity` [m_titleEntity](#)
- `const std::vector< std::string >` [m_contributors](#) = {"Elliot", "Arthur", "Moana", "Aaron"}
- `ecs::Entity` [m_contributorsEntity](#)
- `float` [m_contributorsOffset](#) = 0.0f
- `bool` [m_playMusic](#) = false

Additional Inherited Members

Protected Attributes inherited from [eng::AScene](#)

- `utl::EventBus &` [m_eventBus](#) = `utl::EventBus::getInstance()`
- `std::uint32_t` [m_eventComponentId](#) = 0

18.88.1 Detailed Description

[Menu](#) scene.

Definition at line 21 of file [Menu.hpp](#).

18.88.2 Constructor & Destructor Documentation

18.88.2.1 [Menu\(\)](#) [1/3]

```
cli::Menu::Menu (
    eng::id assignedId,
    const std::shared_ptr< eng::IRenderer > & renderer)
```

Definition at line 7 of file [menu.cpp](#).

References [eng::Color::a](#), [eng::Color::b](#), [utl::Config::Color::CYAN_ELECTRIC](#), [utl::Path::Font::FONTS_RTYPE](#), [eng::Color::g](#), [utl::Config::Color::GRAY_BLUE_SUBTLE](#), [ecs::IComponent::id](#), [m_contributors](#), [m_contributorsEntity](#), [m_menuOptions](#), [m_selectedIndex](#), [m_titleEntity](#), [eng::Color::r](#), and [ecs::Scale::x](#).

18.88.2.2 [~Menu\(\)](#)

```
cli::Menu::~Menu () [override], [default]
```

18.88.2.3 [Menu\(\)](#) [2/3]

```
cli::Menu::Menu (
    const Menu & other) [delete]
```

18.88.2.4 [Menu\(\)](#) [3/3]

```
cli::Menu::Menu (
    Menu && other) [delete]
```


18.88.3 Member Function Documentation

18.88.3.1 event()

void cli::Menu::event (
 const [eng::Event](#) & event) [override], [virtual]

Implements [eng::IScene](#).

Definition at line 165 of file [menu.cpp](#).

References [eng::Down](#), [eng::Enter](#), [eng::Event::key](#), [eng::KeyPressed](#), [eng::KeyReleased](#), [eng::Left](#), [eng::Right](#), [eng::Space](#), [eng::Event::type](#), and [eng::Up](#).

18.88.3.2 operator=() [1/2]

[Menu](#) & cli::Menu::operator= (
 const [Menu](#) & other) [delete]

18.88.3.3 operator=() [2/2]

[Menu](#) & cli::Menu::operator= (
 [Menu](#) && other) [delete]

18.88.3.4 playMusic()

bool & cli::Menu::playMusic () [inline]

Definition at line 35 of file [Menu.hpp](#).

References [m_playMusic](#).

18.88.3.5 update()

void cli::Menu::update (
 float dt,
 const [eng::WindowSize](#) & size) [override], [virtual]

Implements [eng::IScene](#).

Definition at line 105 of file [menu.cpp](#).

References [eng::Color::b](#), [utl::Config::Color::CYAN_ELECTRIC](#), [eng::Color::g](#), [utl::Config::Color::GRAY_BLUE_SUBT](#), [ecs::Color::r](#), [eng::Color::r](#), and [eng::WindowSize::width](#).

18.88.4 Member Data Documentation

18.88.4.1 m_animationTime

float cli::Menu::m_animationTime = 0.0f [private]

Definition at line 44 of file [Menu.hpp](#).

18.88.4.2 m_contributors

const std::vector<std::string> cli::Menu::m_contributors = {"Elliot", "Arthur", "Moana", "Aaron"} [private]

Definition at line 48 of file [Menu.hpp](#).

Referenced by [Menu\(\)](#).

18.88.4.3 m_contributorsEntity

[ecs::Entity](#) cli::Menu::m_contributorsEntity [private]

Definition at line 49 of file [Menu.hpp](#).

Referenced by [Menu\(\)](#).

18.88.4.4 m_contributorsOffset

float cli::Menu::m_contributorsOffset = 0.0f [private]

Definition at line 50 of file [Menu.hpp](#).

18.88.4.5 m_keysPressed

`std::unordered_map<eng::Key, bool> cli::Menu::m_keysPressed` [private]

Definition at line 40 of file [Menu.hpp](#).

18.88.4.6 m_menuOptions

`const std::vector<std::string> cli::Menu::m_menuOptions = {"Solo", "Multi", "Settings"}` [private]

Definition at line 41 of file [Menu.hpp](#).

Referenced by [Menu\(\)](#).

18.88.4.7 m_playMusic

`bool cli::Menu::m_playMusic = false` [private]

Definition at line 51 of file [Menu.hpp](#).

Referenced by [playMusic\(\)](#).

18.88.4.8 m_selectedIndex

`int cli::Menu::m_selectedIndex = 0` [private]

Definition at line 43 of file [Menu.hpp](#).

Referenced by [Menu\(\)](#).

18.88.4.9 m_titleEntity

`ecs::Entity cli::Menu::m_titleEntity` [private]

Definition at line 46 of file [Menu.hpp](#).

Referenced by [Menu\(\)](#).

18.88.4.10 m_titlePulseTime

`float cli::Menu::m_titlePulseTime = 0.0f` [private]

Definition at line 45 of file [Menu.hpp](#).

18.88.4.11 onOptionSelected

`std::function<void(const std::string &option)> cli::Menu::onOptionSelected`

Definition at line 37 of file [Menu.hpp](#).

The documentation for this class was generated from the following files:

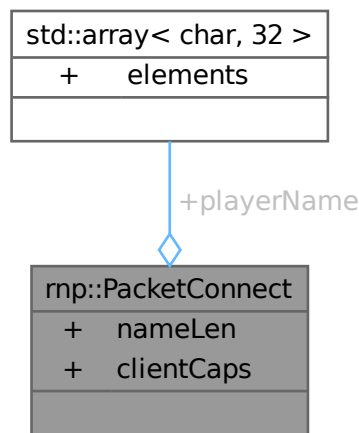
- [/home/masina/Projects/Epitech/rtype/client/include/Client/Scenes/Menu.hpp](#)
- [/home/masina/Projects/Epitech/rtype/client/src/scenes/menu.cpp](#)

18.89 rnp::PacketConnect Struct Reference

CONNECT packet payload.

`#include <Protocol.hpp>`

Collaboration diagram for rnp::PacketConnect:



Public Attributes

- `std::uint8_t` [nameLen](#)
- `std::array< char, 32 >` [playerName](#)
- `std::uint32_t` [clientCaps](#)

18.89.1 Detailed Description

CONNECT packet payload.

Definition at line 136 of file [Protocol.hpp](#).

18.89.2 Member Data Documentation

18.89.2.1 clientCaps

`std::uint32_t` `rnp::PacketConnect::clientCaps`

Definition at line 140 of file [Protocol.hpp](#).

Referenced by [rnp::Serializer::deserializeConnect\(\)](#), [srv::AsioServer::handleConnect\(\)](#), and [rnp::Serializer::serializeConnect\(\)](#).

18.89.2.2 nameLen

`std::uint8_t` `rnp::PacketConnect::nameLen`

Definition at line 138 of file [Protocol.hpp](#).

Referenced by [rnp::Serializer::deserializeConnect\(\)](#), [srv::AsioServer::handleConnect\(\)](#), [eng::AsioClient::sendConnect\(\)](#), and [rnp::Serializer::serializeConnect\(\)](#).

18.89.2.3 playerName

`std::array<char, 32>` `rnp::PacketConnect::playerName`

Definition at line 139 of file [Protocol.hpp](#).

Referenced by [rnp::Serializer::deserializeConnect\(\)](#), [srv::AsioServer::handleConnect\(\)](#), and [rnp::Serializer::serializeConnect\(\)](#).

The documentation for this struct was generated from the following file:

- [/home/masina/Projects/Epitech/rtype/modules/Interfaces/include/Interfaces/Protocol/Protocol.hpp](#)

18.90 rnp::PacketConnectAccept Struct Reference

CONNECT_ACCEPT packet payload.

#include <Protocol.hpp>

Collaboration diagram for rnp::PacketConnectAccept:

rnp::PacketConnectAccept	
+	sessionId
+	tickRateHz
+	mtuPayloadBytes
+	serverCaps

Public Attributes

- std::uint32_t [sessionId](#)
- std::uint16_t [tickRateHz](#)
- std::uint16_t [mtuPayloadBytes](#)
- std::uint32_t [serverCaps](#)

18.90.1 Detailed Description

CONNECT_ACCEPT packet payload.

Definition at line 146 of file [Protocol.hpp](#).

18.90.2 Member Data Documentation

18.90.2.1 mtuPayloadBytes

std::uint16_t rnp::PacketConnectAccept::mtuPayloadBytes

Definition at line 150 of file [Protocol.hpp](#).

Referenced by [rnp::Serializer::deserializeConnectAccept\(\)](#), and [rnp::Serializer::serializeConnectAccept\(\)](#).

18.90.2.2 serverCaps

std::uint32_t rnp::PacketConnectAccept::serverCaps

Definition at line 151 of file [Protocol.hpp](#).

Referenced by [rnp::Serializer::deserializeConnectAccept\(\)](#), and [rnp::Serializer::serializeConnectAccept\(\)](#).

18.90.2.3 sessionId

std::uint32_t rnp::PacketConnectAccept::sessionId

Definition at line 148 of file [Protocol.hpp](#).

Referenced by [rnp::Serializer::deserializeConnectAccept\(\)](#), [eng::AsioClient::handleConnectAccept\(\)](#), [srv::AsioServer::sendConnectAccept\(\)](#), and [rnp::Serializer::serializeConnectAccept\(\)](#).

18.90.2.4 tickRateHz

std::uint16_t rnp::PacketConnectAccept::tickRateHz

Definition at line 149 of file [Protocol.hpp](#).

Referenced by [rnp::Serializer::deserializeConnectAccept\(\)](#), [eng::AsioClient::handleConnectAccept\(\)](#), and [rnp::Serializer::serializeConnectAccept\(\)](#).

The documentation for this struct was generated from the following file:

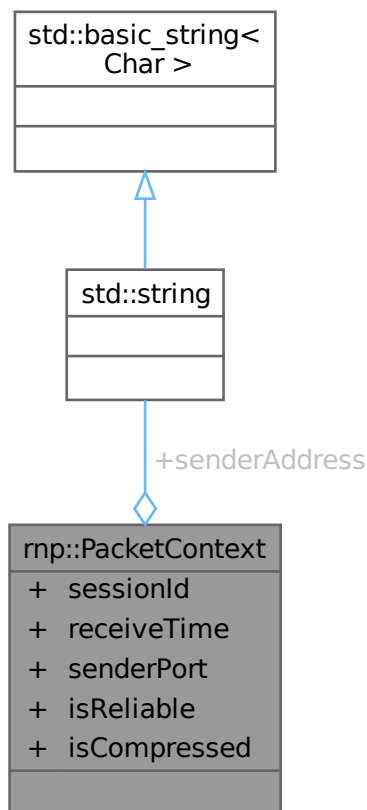
- [/home/masina/Projects/Epitech/rtype/modules/Interfaces/include/Interfaces/Protocol/Protocol.hpp](#)

18.91 rnp::PacketContext Struct Reference

Context information for packet processing.

#include <HandlerPacket.hpp>

Collaboration diagram for rnp::PacketContext:



Public Attributes

- `std::uint32_t` [sessionId](#)
- `std::chrono::steady_clock::time_point` [receiveTime](#)
- `std::string` [senderAddress](#)
- `std::uint16_t` [senderPort](#)
- `bool` [isReliable](#)
- `bool` [isCompressed](#)

18.91.1 Detailed Description

Context information for packet processing.

Definition at line 36 of file [HandlerPacket.hpp](#).

18.91.2 Member Data Documentation

18.91.2.1 isCompressed

bool rnp::PacketContext::isCompressed

Definition at line 43 of file [HandlerPacket.hpp](#).

18.91.2.2 isReliable

bool rnp::PacketContext::isReliable

Definition at line 42 of file [HandlerPacket.hpp](#).

18.91.2.3 receiveTime

std::chrono::steady_clock::time_point rnp::PacketContext::receiveTime

Definition at line 39 of file [HandlerPacket.hpp](#).

Referenced by [srv::AsioServer::handleConnect\(\)](#), [srv::AsioServer::handlePing\(\)](#), [srv::AsioServer::handlePong\(\)](#), [eng::AsioClient::handleReceive\(\)](#), [srv::AsioServer::handleReceive\(\)](#), and [rnp::HandlerPacket::processPacket\(\)](#).

18.91.2.4 senderAddress

std::string rnp::PacketContext::senderAddress

Definition at line 40 of file [HandlerPacket.hpp](#).

Referenced by [eng::AsioClient::handleReceive\(\)](#), and [srv::AsioServer::handleReceive\(\)](#).

18.91.2.5 senderPort

std::uint16_t rnp::PacketContext::senderPort

Definition at line 41 of file [HandlerPacket.hpp](#).

Referenced by [eng::AsioClient::handleReceive\(\)](#), and [srv::AsioServer::handleReceive\(\)](#).

18.91.2.6 sessionId

std::uint32_t rnp::PacketContext::sessionId

Definition at line 38 of file [HandlerPacket.hpp](#).

Referenced by [srv::AsioServer::handleDisconnect\(\)](#), [srv::AsioServer::handleEntityEvent\(\)](#), [eng::AsioClient::handleGameStart\(\)](#), [srv::AsioServer::handleLobbyCreate\(\)](#), [srv::AsioServer::handleLobbyJoin\(\)](#), [srv::AsioServer::handleLobbyLeave\(\)](#), [srv::AsioServer::handleLobbyListRequest\(\)](#), [srv::AsioServer::handlePing\(\)](#), [srv::AsioServer::handlePong\(\)](#), [eng::AsioClient::handleReceive\(\)](#), [srv::AsioServer::handleReceive\(\)](#), [srv::AsioServer::handleStartGameRequest\(\)](#), and [rnp::HandlerPacket::processPacket\(\)](#).

The documentation for this struct was generated from the following file:

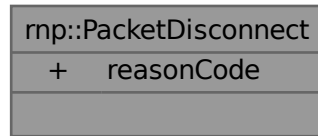
- [/home/masina/Projects/Epitech/rtype/modules/Interfaces/include/Interfaces/Protocol/HandlerPacket.hpp](#)

18.92 rnp::PacketDisconnect Struct Reference

DISCONNECT packet payload.

```
#include <Protocol.hpp>
```

Collaboration diagram for rnp::PacketDisconnect:



Public Attributes

- `std::uint16_t` [reasonCode](#)

18.92.1 Detailed Description

DISCONNECT packet payload.

Definition at line [157](#) of file [Protocol.hpp](#).

18.92.2 Member Data Documentation

18.92.2.1 reasonCode

`std::uint16_t` `rnp::PacketDisconnect::reasonCode`

Definition at line [159](#) of file [Protocol.hpp](#).

Referenced by [rnp::Serializer::deserializeDisconnect\(\)](#), [srv::AsioServer::disconnectClient\(\)](#), [eng::AsioClient::handleDisconnect\(\)](#), [eng::AsioClient::sendDisconnect\(\)](#), [rnp::Serializer::serializeDisconnect\(\)](#), and [srv::AsioServer::stop\(\)](#).

The documentation for this struct was generated from the following file:

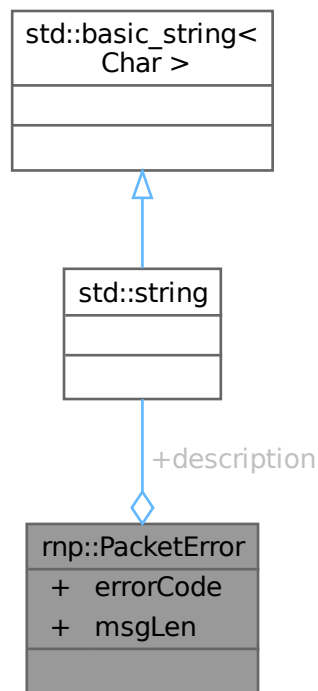
- `/home/masina/Projects/Epitech/rtype/modules/Interfaces/include/Interfaces/Protocol/Protocol.hpp`

18.93 rnp::PacketError Struct Reference

ERROR packet payload.

`#include <Protocol.hpp>`

Collaboration diagram for `rnp::PacketError`:



Public Attributes

- `std::uint16_t` [errorCode](#)
- `std::uint16_t` [msgLen](#)
- `std::string` [description](#)

18.93.1 Detailed Description

ERROR packet payload.

Definition at line 214 of file [Protocol.hpp](#).

18.93.2 Member Data Documentation

18.93.2.1 description

`std::string` `rnp::PacketError::description`

Definition at line 218 of file [Protocol.hpp](#).

Referenced by [rnp::Serializer::deserializeError\(\)](#), [eng::AsioClient::handleError\(\)](#), and [rnp::Serializer::serializeError\(\)](#).

18.93.2.2 errorCode

`std::uint16_t` `rnp::PacketError::errorCode`

Definition at line 216 of file [Protocol.hpp](#).

Referenced by [rnp::Serializer::deserializeError\(\)](#), [eng::AsioClient::handleError\(\)](#), [srv::AsioServer::sendError\(\)](#), and [rnp::Serializer::serializeError\(\)](#).

18.93.2.3 msgLen

std::uint16_t rnp::PacketError::msgLen

Definition at line 217 of file [Protocol.hpp](#).

Referenced by [rnp::Serializer::deserializeError\(\)](#), and [rnp::Serializer::serializeError\(\)](#).

The documentation for this struct was generated from the following file:

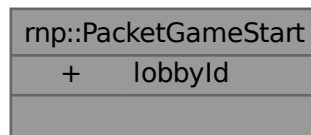
- [/home/masina/Projects/Epitech/rtype/modules/Interfaces/include/Interfaces/Protocol/Protocol.hpp](#)

18.94 rnp::PacketGameStart Struct Reference

GAME_START packet payload.

#include <Protocol.hpp>

Collaboration diagram for rnp::PacketGameStart:



Public Attributes

- std::uint32_t [lobbyId](#)

18.94.1 Detailed Description

GAME_START packet payload.

Definition at line 306 of file [Protocol.hpp](#).

18.94.2 Member Data Documentation

18.94.2.1 lobbyId

std::uint32_t rnp::PacketGameStart::lobbyId

Definition at line 308 of file [Protocol.hpp](#).

Referenced by [srv::AsioServer::broadcastGameStart\(\)](#), [rnp::Serializer::deserializeGameStart\(\)](#), [eng::AsioClient::handleGameStart\(\)](#), [gme::WaitingRoomScene::handleGameStart\(\)](#), and [rnp::Serializer::serializeGameStart\(\)](#).

The documentation for this struct was generated from the following file:

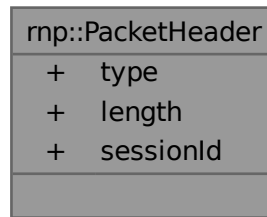
- [/home/masina/Projects/Epitech/rtype/modules/Interfaces/include/Interfaces/Protocol/Protocol.hpp](#)

18.95 rnp::PacketHeader Struct Reference

Packet header according to RNP specification (Big Endian) Total size: 7 bytes (1 + 2 + 4)

#include <Protocol.hpp>

Collaboration diagram for `rnp::PacketHeader`:



Public Attributes

- `std::uint8_t` [type](#)
- `std::uint16_t` [length](#)
- `std::uint32_t` [sessionId](#)

18.95.1 Detailed Description

Packet header according to RNP specification (Big Endian) Total size: 7 bytes (1 + 2 + 4)

Definition at line [126](#) of file [Protocol.hpp](#).

18.95.2 Member Data Documentation

18.95.2.1 length

`std::uint16_t` `rnp::PacketHeader::length`

Definition at line [129](#) of file [Protocol.hpp](#).

Referenced by [srv::AsioServer::broadcastGameStart\(\)](#), [srv::AsioServer::broadcastLobbyUpdate\(\)](#), [gme::RTypeServer::broadcastWorldState\(\)](#), [eng::AsioClient::createLobby\(\)](#), [rnp::Serializer::deserializeHeader\(\)](#), [srv::AsioServer::disconnectClient\(\)](#), [eng::AsioClient::joinLobby\(\)](#), [eng::AsioClient::leaveLobby\(\)](#), [eng::AsioClient::processBusEvent\(\)](#), [rnp::HandlerPacket::processPacket\(\)](#), [eng::AsioClient::requestLobbyList\(\)](#), [eng::AsioClient::requestStartGame\(\)](#), [srv::AsioServer::sendLobbyCreateResponse\(\)](#), [srv::AsioServer::sendLobbyJoinResponse\(\)](#), [srv::AsioServer::sendLobbyList\(\)](#) and [rnp::Serializer::serializeHeader\(\)](#).

18.95.2.2 sessionId

`std::uint32_t` `rnp::PacketHeader::sessionId`

Definition at line [130](#) of file [Protocol.hpp](#).

Referenced by [srv::AsioServer::broadcastGameStart\(\)](#), [srv::AsioServer::broadcastLobbyUpdate\(\)](#), [gme::RTypeServer::broadcastWorldState\(\)](#), [eng::AsioClient::createLobby\(\)](#), [rnp::Serializer::deserializeHeader\(\)](#), [srv::AsioServer::disconnectClient\(\)](#), [eng::AsioClient::handleReceive\(\)](#), [srv::AsioServer::handleReceive\(\)](#), [eng::AsioClient::joinLobby\(\)](#), [eng::AsioClient::leaveLobby\(\)](#), [eng::AsioClient::processBusEvent\(\)](#), [rnp::HandlerPacket::processPacket\(\)](#), [eng::AsioClient::requestLobbyList\(\)](#), [eng::AsioClient::requestStartGame\(\)](#), [srv::AsioServer::sendLobbyCreateResponse\(\)](#), [srv::AsioServer::sendLobbyJoinResponse\(\)](#), [srv::AsioServer::sendLobbyList\(\)](#) and [rnp::Serializer::serializeHeader\(\)](#).

18.95.2.3 type

`std::uint8_t` `rnp::PacketHeader::type`

Definition at line [128](#) of file [Protocol.hpp](#).

Referenced by [srv::AsioServer::broadcastGameStart\(\)](#), [srv::AsioServer::broadcastLobbyUpdate\(\)](#), [gme::RTypeServer::broadcastWorldState\(\)](#), [eng::AsioClient::createLobby\(\)](#), [rnp::Serializer::deserializeHeader\(\)](#), [srv::AsioServer::disconnectClient\(\)](#), [srv::AsioServer::handleReceive\(\)](#), [eng::AsioClient::joinLobby\(\)](#),

[eng::AsioClient::leaveLobby\(\)](#), [eng::AsioClient::processBusEvent\(\)](#), [rnp::HandlerPacket::processPacket\(\)](#), [eng::AsioClient::requestLobbyList\(\)](#), [eng::AsioClient::requestStartGame\(\)](#), [eng::AsioClient::sendConnect\(\)](#), [srv::AsioServer::sendConnectAccept\(\)](#), [eng::AsioClient::sendDisconnect\(\)](#), [srv::AsioServer::sendError\(\)](#), [srv::AsioServer::sendLobbyCreateResponse\(\)](#), [srv::AsioServer::sendLobbyJoinResponse\(\)](#), [srv::AsioServer::sendLobbyList\(\)](#), [eng::AsioClient::sendPing\(\)](#), [eng::AsioClient::sendPong\(\)](#), [srv::AsioServer::sendPong\(\)](#), [rnp::Serializer::serializeHeader\(\)](#), [srv::AsioServer::stop\(\)](#), and [srv::AsioServer::updateClientManagement\(\)](#).

The documentation for this struct was generated from the following file:

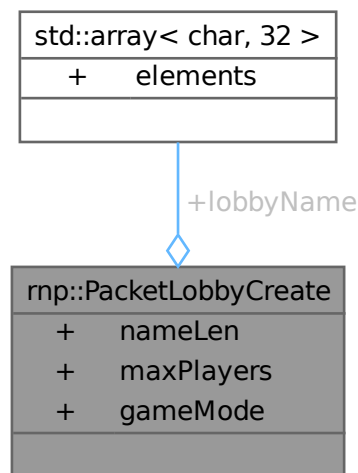
- [/home/masina/Projects/Epitech/rtype/modules/Interfaces/include/Interfaces/Protocol/Protocol.hpp](#)

18.96 rnp::PacketLobbyCreate Struct Reference

LOBBY_CREATE packet payload.

#include <Protocol.hpp>

Collaboration diagram for rnp::PacketLobbyCreate:



Public Attributes

- `std::uint8_t` [nameLen](#)
- `std::array< char, 32 >` [lobbyName](#)
- `std::uint8_t` [maxPlayers](#)
- `std::uint8_t` [gameMode](#)

18.96.1 Detailed Description

LOBBY_CREATE packet payload.

Definition at line 258 of file [Protocol.hpp](#).

18.96.2 Member Data Documentation

18.96.2.1 gameMode

`std::uint8_t` [rnp::PacketLobbyCreate::gameMode](#)

Definition at line 263 of file [Protocol.hpp](#).

Referenced by [eng::AsioClient::createLobby\(\)](#), [rnp::Serializer::deserializeLobbyCreate\(\)](#), [srv::AsioServer::handleLobbyCreate\(\)](#), [eng::AsioClient::processBusEvent\(\)](#), and [rnp::Serializer::serializeLobbyCreate\(\)](#).

18.96.2.2 lobbyName

std::array<char, 32> rnp::PacketLobbyCreate::lobbyName

Definition at line 261 of file [Protocol.hpp](#).

Referenced by [eng::AsioClient::createLobby\(\)](#), [rnp::Serializer::deserializeLobbyCreate\(\)](#), [srv::AsioServer::handleLobbyCreate\(\)](#), [eng::AsioClient::processBusEvent\(\)](#), and [rnp::Serializer::serializeLobbyCreate\(\)](#).

18.96.2.3 maxPlayers

std::uint8_t rnp::PacketLobbyCreate::maxPlayers

Definition at line 262 of file [Protocol.hpp](#).

Referenced by [eng::AsioClient::createLobby\(\)](#), [gme::CreateRoomScene::createRoom\(\)](#), [rnp::Serializer::deserializeLobbyCreate\(\)](#), [srv::AsioServer::handleLobbyCreate\(\)](#), [eng::AsioClient::processBusEvent\(\)](#), and [rnp::Serializer::serializeLobbyCreate\(\)](#).

18.96.2.4 nameLen

std::uint8_t rnp::PacketLobbyCreate::nameLen

Definition at line 260 of file [Protocol.hpp](#).

Referenced by [eng::AsioClient::createLobby\(\)](#), [rnp::Serializer::deserializeLobbyCreate\(\)](#), [srv::AsioServer::handleLobbyCreate\(\)](#), [eng::AsioClient::processBusEvent\(\)](#), and [rnp::Serializer::serializeLobbyCreate\(\)](#).

The documentation for this struct was generated from the following file:

- [/home/masina/Projects/Epitech/rtype/modules/Interfaces/include/Interfaces/Protocol/Protocol.hpp](#)

18.97 rnp::PacketLobbyCreateResponse Struct Reference

LOBBY_CREATE_RESPONSE packet payload.

#include <Protocol.hpp>

Collaboration diagram for rnp::PacketLobbyCreateResponse:

rnp::PacketLobbyCreateResponse	
+	lobbyId
+	success
+	errorCode

Public Attributes

- std::uint32_t [lobbyId](#)
- std::uint8_t [success](#)
- std::uint16_t [errorCode](#)

18.97.1 Detailed Description

LOBBY_CREATE_RESPONSE packet payload.

Definition at line 269 of file [Protocol.hpp](#).

18.97.2 Member Data Documentation

18.97.2.1 errorCode

std::uint16_t rnp::PacketLobbyCreateResponse::errorCode

Definition at line 273 of file [Protocol.hpp](#).

Referenced by [rnp::Serializer::deserializeLobbyCreateResponse\(\)](#), [eng::AsioClient::handleLobbyCreateResponse\(\)](#), [srv::AsioServer::sendLobbyCreateResponse\(\)](#), and [rnp::Serializer::serializeLobbyCreateResponse\(\)](#).

18.97.2.2 lobbyId

std::uint32_t rnp::PacketLobbyCreateResponse::lobbyId

Definition at line 271 of file [Protocol.hpp](#).

Referenced by [rnp::Serializer::deserializeLobbyCreateResponse\(\)](#), [eng::AsioClient::handleLobbyCreateResponse\(\)](#), [srv::AsioServer::sendLobbyCreateResponse\(\)](#), and [rnp::Serializer::serializeLobbyCreateResponse\(\)](#).

18.97.2.3 success

std::uint8_t rnp::PacketLobbyCreateResponse::success

Definition at line 272 of file [Protocol.hpp](#).

Referenced by [rnp::Serializer::deserializeLobbyCreateResponse\(\)](#), [eng::AsioClient::handleLobbyCreateResponse\(\)](#), [srv::AsioServer::sendLobbyCreateResponse\(\)](#), and [rnp::Serializer::serializeLobbyCreateResponse\(\)](#).

The documentation for this struct was generated from the following file:

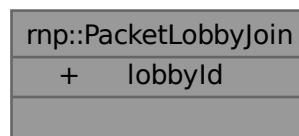
- [/home/masina/Projects/Epitech/rtype/modules/Interfaces/include/Interfaces/Protocol/Protocol.hpp](#)

18.98 rnp::PacketLobbyJoin Struct Reference

LOBBY_JOIN packet payload.

#include <Protocol.hpp>

Collaboration diagram for rnp::PacketLobbyJoin:



Public Attributes

- std::uint32_t [lobbyId](#)

18.98.1 Detailed Description

LOBBY_JOIN packet payload.

Definition at line 279 of file [Protocol.hpp](#).

18.98.2 Member Data Documentation

18.98.2.1 lobbyId

std::uint32_t rnp::PacketLobbyJoin::lobbyId

Definition at line 281 of file [Protocol.hpp](#).

Referenced by [rnp::Serializer::deserializeLobbyJoin\(\)](#), [gme::JoinRoomScene::event\(\)](#), [srv::AsioServer::handleLobbyJoin\(\)](#), [eng::AsioClient::joinLobby\(\)](#), and [rnp::Serializer::serializeLobbyJoin\(\)](#).

The documentation for this struct was generated from the following file:

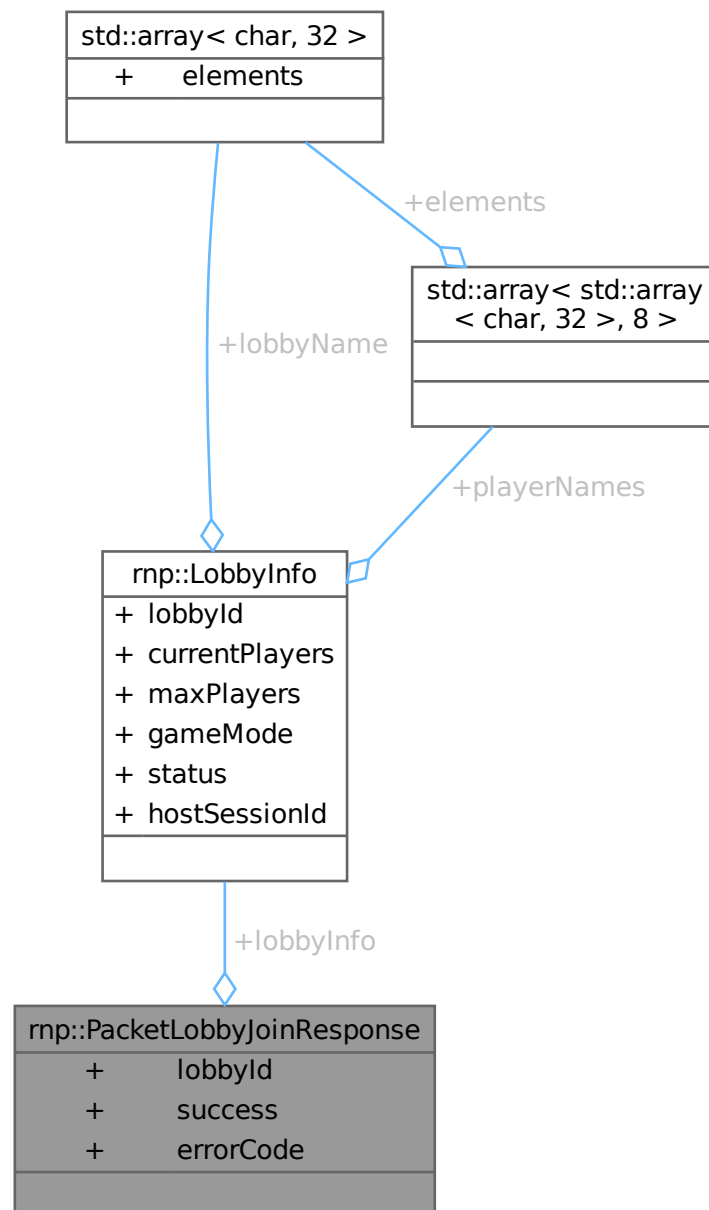
- [/home/masina/Projects/Epitech/rtype/modules/Interfaces/include/Interfaces/Protocol/Protocol.hpp](#)

18.99 rnp::PacketLobbyJoinResponse Struct Reference

LOBBY_JOIN_RESPONSE packet payload.

```
#include <Protocol.hpp>
```

Collaboration diagram for rnp::PacketLobbyJoinResponse:



Public Attributes

- `std::uint32_t lobbyId`
- `std::uint8_t success`
- `std::uint16_t errorCode`
- [LobbyInfo lobbyInfo](#)

18.99.1 Detailed Description

LOBBY_JOIN_RESPONSE packet payload.

Definition at line 287 of file [Protocol.hpp](#).

18.99.2 Member Data Documentation

18.99.2.1 errorCode

`std::uint16_t rnp::PacketLobbyJoinResponse::errorCode`

Definition at line 291 of file [Protocol.hpp](#).

Referenced by [rnp::Serializer::deserializeLobbyJoinResponse\(\)](#), [eng::AsioClient::handleLobbyJoinResponse\(\)](#), [srv::AsioServer::sendLobbyJoinResponse\(\)](#), and [rnp::Serializer::serializeLobbyJoinResponse\(\)](#).

18.99.2.2 lobbyId

`std::uint32_t rnp::PacketLobbyJoinResponse::lobbyId`

Definition at line 289 of file [Protocol.hpp](#).

Referenced by [rnp::Serializer::deserializeLobbyJoinResponse\(\)](#), [eng::AsioClient::handleLobbyJoinResponse\(\)](#), [srv::AsioServer::sendLobbyJoinResponse\(\)](#), and [rnp::Serializer::serializeLobbyJoinResponse\(\)](#).

18.99.2.3 lobbyInfo

[LobbyInfo](#) `rnp::PacketLobbyJoinResponse::lobbyInfo`

Definition at line 292 of file [Protocol.hpp](#).

Referenced by [rnp::Serializer::deserializeLobbyJoinResponse\(\)](#), [eng::AsioClient::handleLobbyJoinResponse\(\)](#), [srv::AsioServer::sendLobbyJoinResponse\(\)](#), and [rnp::Serializer::serializeLobbyJoinResponse\(\)](#).

18.99.2.4 success

`std::uint8_t rnp::PacketLobbyJoinResponse::success`

Definition at line 290 of file [Protocol.hpp](#).

Referenced by [rnp::Serializer::deserializeLobbyJoinResponse\(\)](#), [eng::AsioClient::handleLobbyJoinResponse\(\)](#), [srv::AsioServer::sendLobbyJoinResponse\(\)](#), and [rnp::Serializer::serializeLobbyJoinResponse\(\)](#).

The documentation for this struct was generated from the following file:

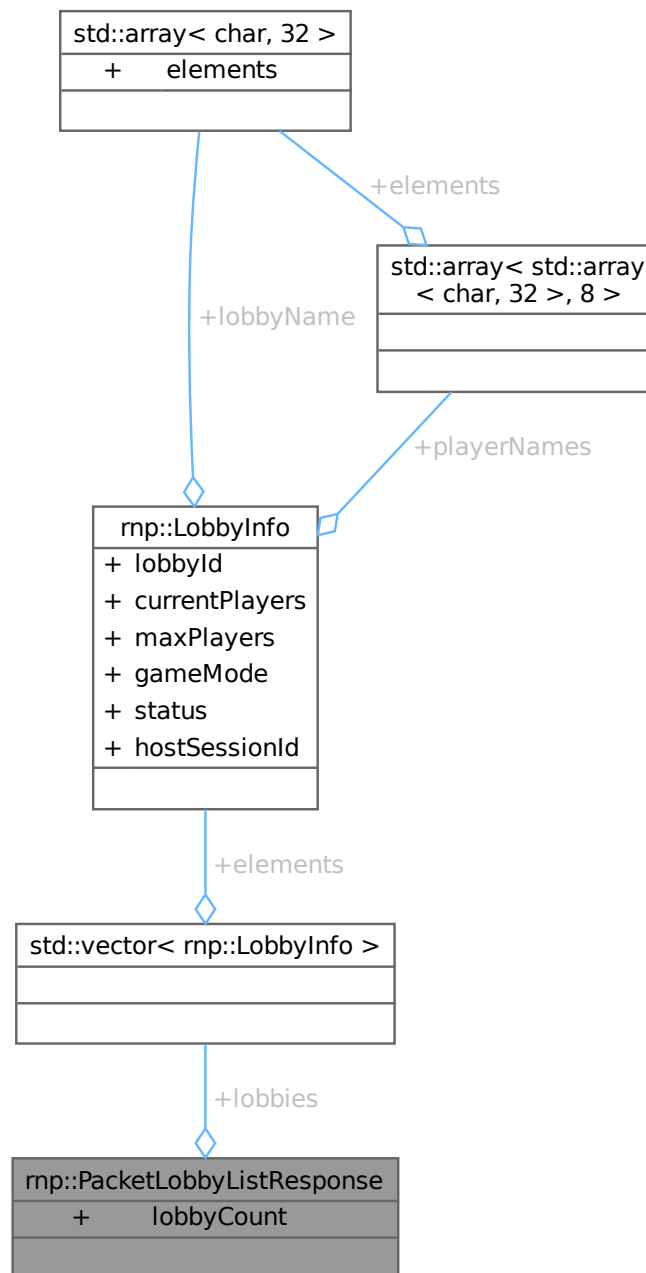
- `/home/masina/Projects/Epitech/rtype/modules/Interfaces/include/Interfaces/Protocol/Protocol.hpp`

18.100 rnp::PacketLobbyListResponse Struct Reference

LOBBY_LIST_RESPONSE packet payload.

`#include <Protocol.hpp>`

Collaboration diagram for `rnmp::PacketLobbyListResponse`:



Public Attributes

- `std::uint16_t lobbyCount`
- `std::vector< LobbyInfo > lobbies`

18.100.1 Detailed Description

LOBBY_LIST_RESPONSE packet payload.

Definition at line 249 of file [Protocol.hpp](#).

18.100.2 Member Data Documentation

18.100.2.1 lobbies

std::vector<[LobbyInfo](#)> rnp::PacketLobbyListResponse::lobbies

Definition at line 252 of file [Protocol.hpp](#).

Referenced by [rnp::Serializer::deserializeLobbyListResponse\(\)](#), [eng::AsioClient::handleLobbyListResponse\(\)](#), [srv::AsioServer::sendLobbyList\(\)](#), and [rnp::Serializer::serializeLobbyListResponse\(\)](#).

18.100.2.2 lobbyCount

std::uint16_t rnp::PacketLobbyListResponse::lobbyCount

Definition at line 251 of file [Protocol.hpp](#).

Referenced by [rnp::Serializer::deserializeLobbyListResponse\(\)](#), [eng::AsioClient::handleLobbyListResponse\(\)](#), [srv::AsioServer::sendLobbyList\(\)](#), and [rnp::Serializer::serializeLobbyListResponse\(\)](#).

The documentation for this struct was generated from the following file:

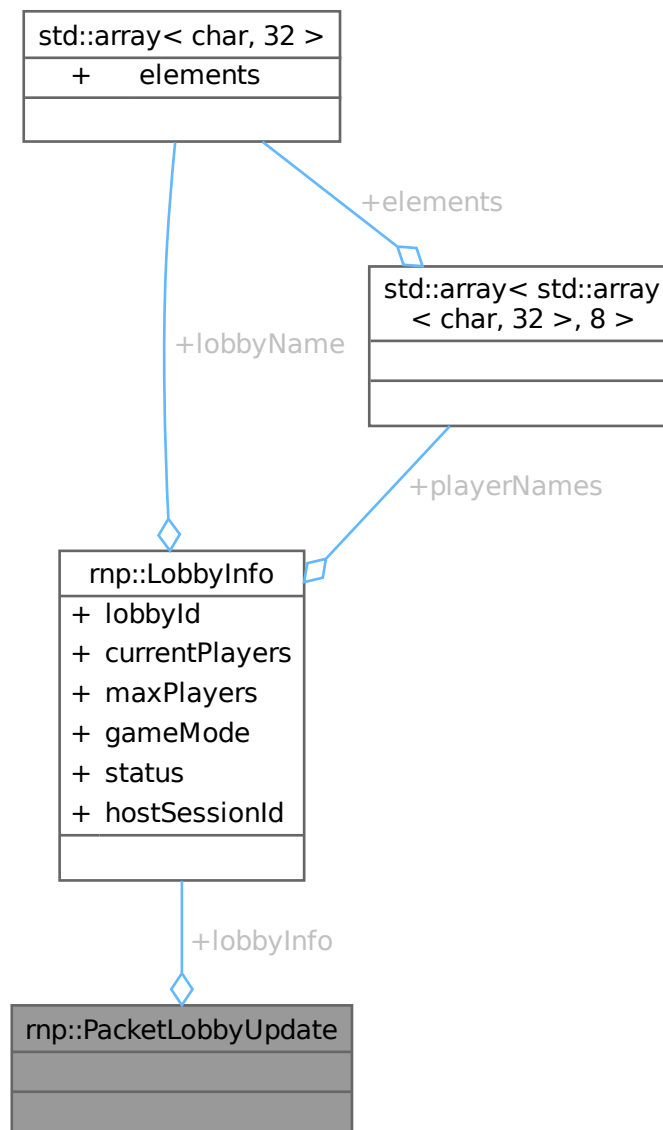
- [/home/masina/Projects/Epitech/rtype/modules/Interfaces/include/Interfaces/Protocol/Protocol.hpp](#)

18.101 rnp::PacketLobbyUpdate Struct Reference

LOBBY_UPDATE packet payload.

#include <Protocol.hpp>

Collaboration diagram for rnp::PacketLobbyUpdate:



Public Attributes

- [LobbyInfo lobbyInfo](#)

18.101.1 Detailed Description

LOBBY_UPDATE packet payload.

Definition at line 298 of file [Protocol.hpp](#).

18.101.2 Member Data Documentation

18.101.2.1 lobbyInfo

[LobbyInfo](#) rnp::PacketLobbyUpdate::lobbyInfo

Definition at line 300 of file [Protocol.hpp](#).

Referenced by [srv::AsioServer::broadcastLobbyUpdate\(\)](#), [rnp::Serializer::deserializeLobbyUpdate\(\)](#), [eng::AsioClient::handleLobbyUpdate\(\)](#), [gme::WaitingRoomScene::handleLobbyUpdate\(\)](#), and [rnp::Serializer::serializeLobbyUpdate\(\)](#).

The documentation for this struct was generated from the following file:

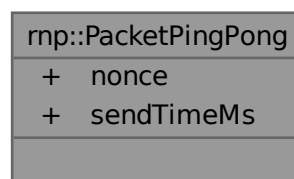
- [/home/masina/Projects/Epitech/rtype/modules/Interfaces/include/Interfaces/Protocol/Protocol.hpp](#)

18.102 rnp::PacketPingPong Struct Reference

PING/PONG packet payload.

#include <Protocol.hpp>

Collaboration diagram for rnp::PacketPingPong:



Public Attributes

- `std::uint32_t` [nonce](#)
- `std::uint32_t` [sendTimeMs](#)

18.102.1 Detailed Description

PING/PONG packet payload.

Definition at line 205 of file [Protocol.hpp](#).

18.102.2 Member Data Documentation

18.102.2.1 nonce

`std::uint32_t` [rnp::PacketPingPong::nonce](#)

Definition at line 207 of file [Protocol.hpp](#).

Referenced by [rnp::Serializer::deserializePingPong\(\)](#), [eng::AsioClient::handlePing\(\)](#), [srv::AsioServer::handlePing\(\)](#), [eng::AsioClient::handlePong\(\)](#), [eng::AsioClient::sendPing\(\)](#), [eng::AsioClient::sendPong\(\)](#), [srv::AsioServer::sendPong\(\)](#), [rnp::Serializer::serializePingPong\(\)](#), and [srv::AsioServer::updateClientManagement\(\)](#).

18.102.2.2 sendTimeMs

`std::uint32_t` [rnp::PacketPingPong::sendTimeMs](#)

Definition at line 208 of file [Protocol.hpp](#).

Referenced by [rnp::Serializer::deserializePingPong\(\)](#), [srv::AsioServer::handlePong\(\)](#), and [rnp::Serializer::serializePingPong\(\)](#).

The documentation for this struct was generated from the following file:

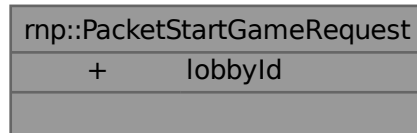
- [/home/masina/Projects/Epitech/rtype/modules/Interfaces/include/Interfaces/Protocol/Protocol.hpp](#)

18.103 rnp::PacketStartGameRequest Struct Reference

START_GAME_REQUEST packet payload (client requests to start game)

```
#include <Protocol.hpp>
```

Collaboration diagram for `rnp::PacketStartGameRequest`:



Public Attributes

- `std::uint32_t` [lobbyId](#)

18.103.1 Detailed Description

START_GAME_REQUEST packet payload (client requests to start game)

Definition at line [314](#) of file [Protocol.hpp](#).

18.103.2 Member Data Documentation

18.103.2.1 lobbyId

`std::uint32_t` `rnp::PacketStartGameRequest::lobbyId`

Definition at line [316](#) of file [Protocol.hpp](#).

Referenced by [rnp::Serializer::deserializeStartGameRequest\(\)](#), [srv::AsioServer::handleStartGameRequest\(\)](#), [eng::AsioClient::requestStartGame\(\)](#), and [rnp::Serializer::serializeStartGameRequest\(\)](#).

The documentation for this struct was generated from the following file:

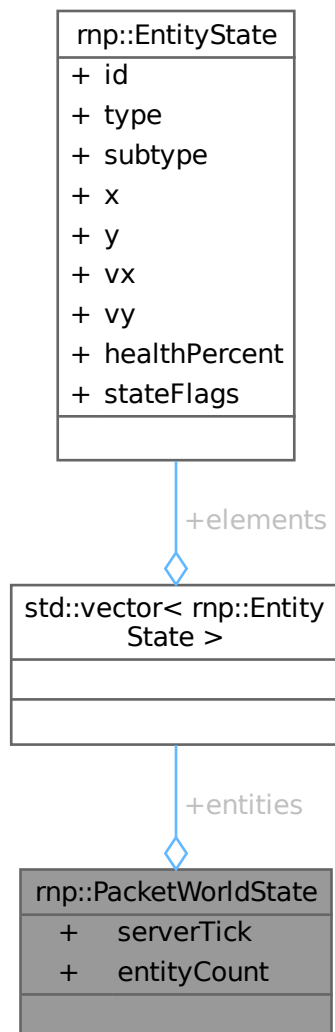
- [/home/masina/Projects/Epitech/rtype/modules/Interfaces/include/Interfaces/Protocol/Protocol.hpp](#)

18.104 rnp::PacketWorldState Struct Reference

WORLD_STATE packet payload.

```
#include <Protocol.hpp>
```

Collaboration diagram for rnp::PacketWorldState:



Public Attributes

- `std::uint32_t` [serverTick](#)
- `std::uint16_t` [entityCount](#)
- `std::vector< EntityState >` [entities](#)

18.104.1 Detailed Description

WORLD_STATE packet payload.

Definition at line 195 of file [Protocol.hpp](#).

18.104.2 Member Data Documentation

18.104.2.1 entities

`std::vector<EntityState>` `rnp::PacketWorldState::entities`

Definition at line 199 of file [Protocol.hpp](#).

Referenced by [gme::RTypeServer::broadcastWorldState\(\)](#), [rnp::Serializer::deserializeWorldState\(\)](#), [gme::GameMulti::handleWorldStateUpdate\(\)](#), and [rnp::Serializer::serializeWorldState\(\)](#).

18.104.2.2 entityCount

`std::uint16_t rnp::PacketWorldState::entityCount`

Definition at line 198 of file [Protocol.hpp](#).

Referenced by [gme::RTypeServer::broadcastWorldState\(\)](#), [rnp::Serializer::deserializeWorldState\(\)](#), [eng::AsioClient::handleWorldState\(\)](#), and [rnp::Serializer::serializeWorldState\(\)](#).

18.104.2.3 serverTick

`std::uint32_t rnp::PacketWorldState::serverTick`

Definition at line 197 of file [Protocol.hpp](#).

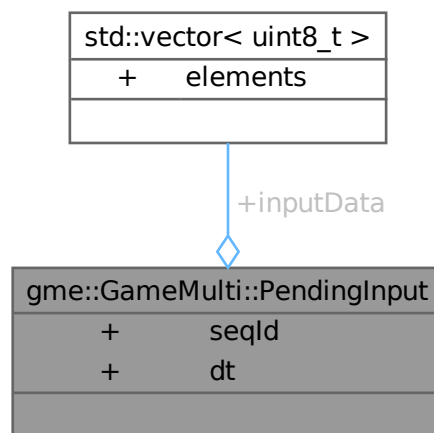
Referenced by [gme::RTypeServer::broadcastWorldState\(\)](#), [rnp::Serializer::deserializeWorldState\(\)](#), [eng::AsioClient::handleWorldState\(\)](#), and [rnp::Serializer::serializeWorldState\(\)](#).

The documentation for this struct was generated from the following file:

- [/home/masina/Projects/Epitech/rtype/modules/Interfaces/include/Interfaces/Protocol/Protocol.hpp](#)

18.105 gme::GameMulti::PendingInput Struct Reference

Collaboration diagram for `gme::GameMulti::PendingInput`:



Public Attributes

- `uint32_t seqId`
- `std::vector< uint8_t > inputData`
- `float dt`

18.105.1 Detailed Description

Definition at line 93 of file [GameMulti.hpp](#).

18.105.2 Member Data Documentation

18.105.2.1 dt

float gme::GameMulti::PendingInput::dt

Definition at line 97 of file [GameMulti.hpp](#).

18.105.2.2 inputData

std::vector<uint8_t> gme::GameMulti::PendingInput::inputData

Definition at line 96 of file [GameMulti.hpp](#).

18.105.2.3 seqId

uint32_t gme::GameMulti::PendingInput::seqId

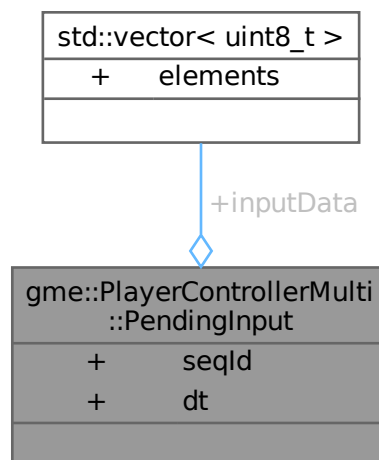
Definition at line 95 of file [GameMulti.hpp](#).

The documentation for this struct was generated from the following file:

- [/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/include/RType↵ ClientMulti/Scenes/GameMulti.hpp](#)

18.106 gme::PlayerControllerMulti::PendingInput Struct Reference

Collaboration diagram for gme::PlayerControllerMulti::PendingInput:



Public Attributes

- uint32_t [seqId](#)
- std::vector< uint8_t > [inputData](#)
- float [dt](#)

18.106.1 Detailed Description

Definition at line 56 of file [PlayerControllerMulti.hpp](#).

18.106.2 Member Data Documentation

18.106.2.1 dt

float gme::PlayerControllerMulti::PendingInput::dt

Definition at line 60 of file [PlayerControllerMulti.hpp](#).

18.106.2.2 inputData

std::vector<uint8_t> gme::PlayerControllerMulti::PendingInput::inputData

Definition at line 59 of file [PlayerControllerMulti.hpp](#).

18.106.2.3 seqId

uint32_t gme::PlayerControllerMulti::PendingInput::seqId

Definition at line 58 of file [PlayerControllerMulti.hpp](#).

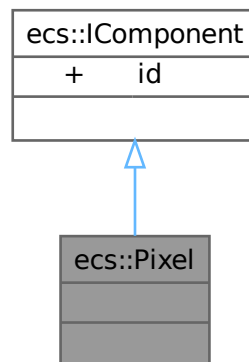
The documentation for this struct was generated from the following file:

- [/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/include/RType↵ ClientMulti/Systems/PlayerControllerMulti.hpp](#)

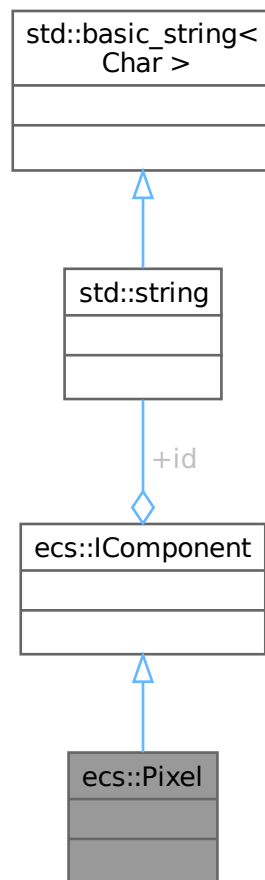
18.107 ecs::Pixel Struct Reference

#include <Component.hpp>

Inheritance diagram for ecs::Pixel:



Collaboration diagram for ecs::Pixel:



Additional Inherited Members

Public Attributes inherited from [ecs::IComponent](#)

- `std::string id`

18.107.1 Detailed Description

Definition at line 40 of file [Component.hpp](#).

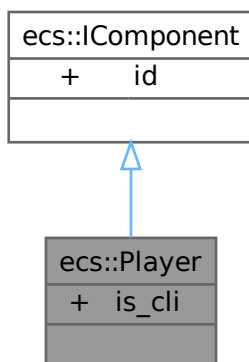
The documentation for this struct was generated from the following file:

- `/home/masina/Projects/Epitech/rtype/modules/ECS/include/ECS/Component.hpp`

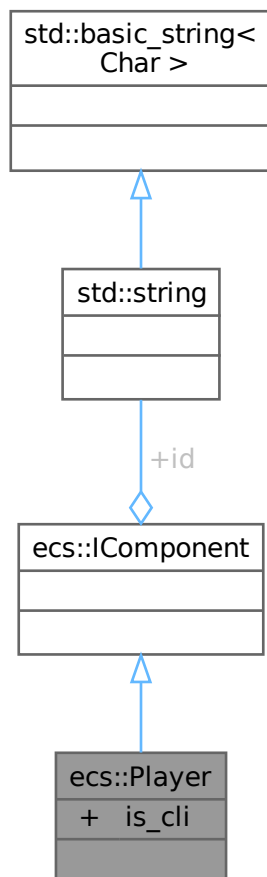
18.108 ecs::Player Struct Reference

```
#include <Component.hpp>
```

Inheritance diagram for ecs::Player:



Collaboration diagram for ecs::Player:



Public Attributes

- bool [is_cli](#) {}

Public Attributes inherited from [ecs::IComponent](#)

- std::string [id](#)

18.108.1 Detailed Description

Definition at line 35 of file [Component.hpp](#).

18.108.2 Member Data Documentation

18.108.2.1 [is_cli](#)

bool [ecs::Player::is_cli](#) {}

Definition at line 37 of file [Component.hpp](#).

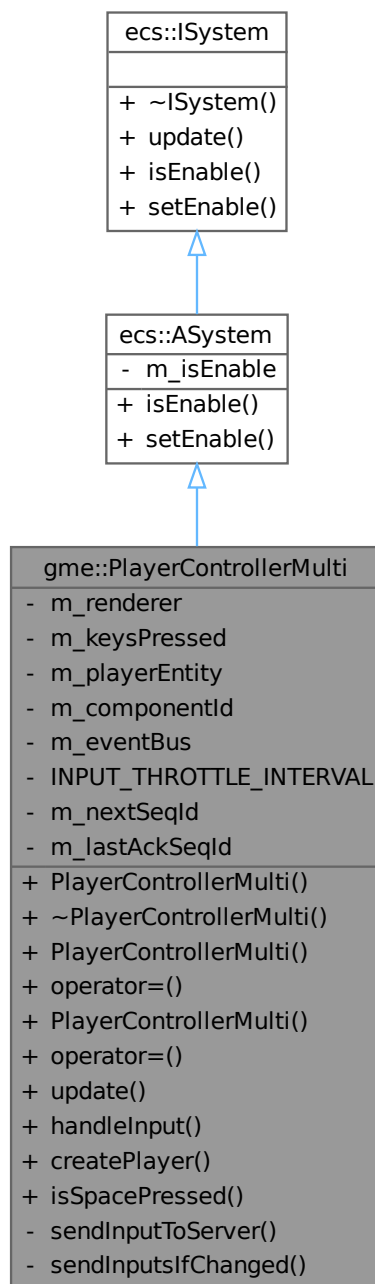
The documentation for this struct was generated from the following file:

- [/home/masina/Projects/Epitech/rtype/modules/ECS/include/ECS/Component.hpp](#)

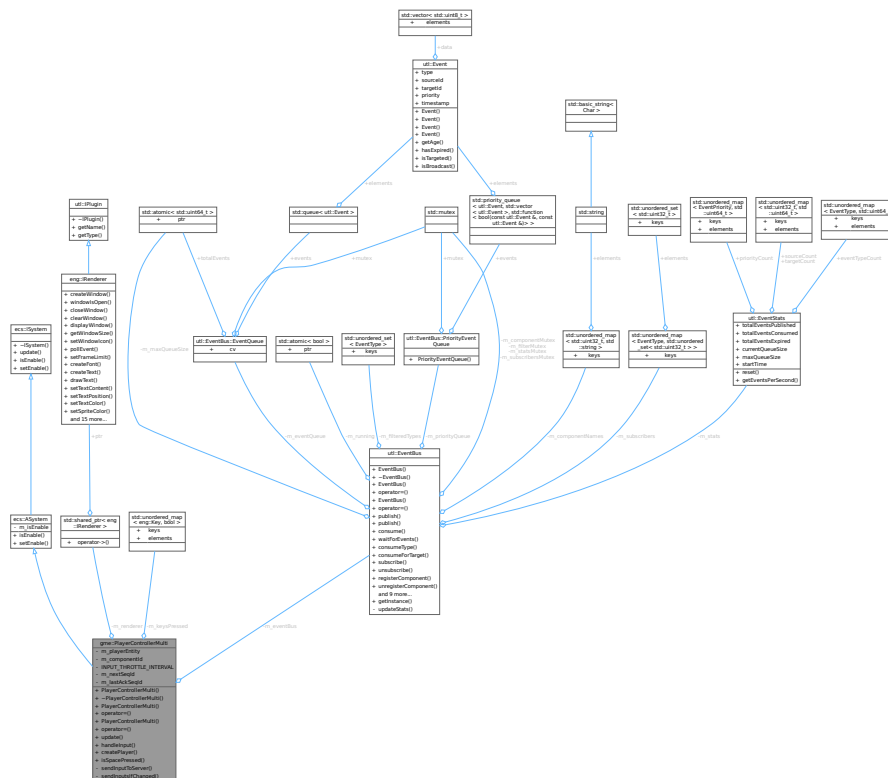
18.109 gme::PlayerControllerMulti Class Reference

`#include <PlayerControllerMulti.hpp>`

Inheritance diagram for gme::PlayerControllerMulti:



Collaboration diagram for gme::PlayerControllerMulti:



Classes

- struct PendingInput

Public Member Functions

- `PlayerControllerMulti` (const std::shared_ptr< [eng::IRenderer](#) > &renderer, uint32_t sessionId)
- `~PlayerControllerMulti` () override=default
- `PlayerControllerMulti` (const `PlayerControllerMulti` &)=delete
- `PlayerControllerMulti` & operator= (const `PlayerControllerMulti` &)=delete
- `PlayerControllerMulti` (`PlayerControllerMulti` &&)=delete
- `PlayerControllerMulti` & operator= (`PlayerControllerMulti` &&)=delete
- void `update` (`ecs::Registry` ®istry, float dt) override
- void `handleInput` (`ecs::Registry` ®istry, const [eng::Event](#) &event)
- `ecs::Entity` `createPlayer` (`ecs::Registry` ®istry, float x, float y)
- bool `isSpacePressed` () const

Public Member Functions inherited from `ecs::ASystem`

- bool `isEnabled` () override
- void `setEnabled` (const bool enable) override

Public Member Functions inherited from `ecs::ISystem`

- virtual `~ISystem ()=default`

Private Member Functions

- void [sendInputToServer](#) (bool up, bool down, bool left, bool right, bool shoot) const
- void [sendInputsIfChanged](#) ()

Private Attributes

- `const std::shared_ptr< eng::IRenderer > & m_renderer`
- `std::unordered_map< eng::Key, bool > m_keysPressed`
- `ecs::Entity m_playerEntity`
- `uint32_t m_componentId`
- `utl::EventBus & m_eventBus`
- `const float INPUT_THROTTLE_INTERVAL = 1.0f / 144.0f`
- `uint32_t m_nextSeqId = 1`
- `uint32_t m_lastAckSeqId = 0`

18.109.1 Detailed Description

Definition at line 19 of file [PlayerControllerMulti.hpp](#).

18.109.2 Constructor & Destructor Documentation

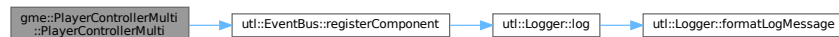
18.109.2.1 PlayerControllerMulti() [1/3]

```
gme::PlayerControllerMulti::PlayerControllerMulti (
    const std::shared_ptr< eng::IRenderer > & renderer,
    uint32_t sessionId) [inline], [explicit]
```

Definition at line 22 of file [PlayerControllerMulti.hpp](#).

References [m_componentId](#), [m_eventBus](#), and [utl::EventBus::registerComponent\(\)](#).

Here is the call graph for this function:



18.109.2.2 ~PlayerControllerMulti()

```
gme::PlayerControllerMulti::~~PlayerControllerMulti () [override], [default]
```

18.109.2.3 PlayerControllerMulti() [2/3]

```
gme::PlayerControllerMulti::PlayerControllerMulti (
    const PlayerControllerMulti & ) [delete]
```

18.109.2.4 PlayerControllerMulti() [3/3]

```
gme::PlayerControllerMulti::PlayerControllerMulti (
    PlayerControllerMulti && ) [delete]
```

18.109.3 Member Function Documentation

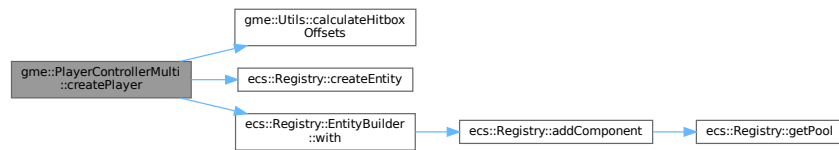
18.109.3.1 createPlayer()

```
ecs::Entity gme::PlayerControllerMulti::createPlayer (
    ecs::Registry & registry,
    float x,
    float y)
```

Definition at line 8 of file [playerControllerMulti.cpp](#).

References [gme::Utils::calculateHitboxOffsets\(\)](#), [ecs::Registry::createEntity\(\)](#), [m_playerEntity](#), [utl::GameConfig::Beam::M](#), [utl::GameConfig::Hitbox::PLAYER_RADIUS](#), [utl::GameConfig::Player::SCALE](#), [utl::GameConfig::Player::SPRITE_HE](#), [utl::GameConfig::Player::SPRITE_WIDTH](#), [utl::Path::Texture::TEXTURE_PLAYER](#), and [ecs::Registry::EntityBuilder:](#)

Here is the call graph for this function:



18.109.3.2 handleInput()

```
void gme::PlayerControllerMulti::handleInput (
    ecs::Registry & registry,
    const eng::Event & event)
```

Definition at line 28 of file [playerControllerMulti.cpp](#).

References [eng::Down](#), [eng::Event::key](#), [eng::KeyPressed](#), [eng::KeyReleased](#), [eng::Left](#), [eng::Right](#), [eng::Space](#), [eng::Event::type](#), and [eng::Up](#).

18.109.3.3 isSpacePressed()

```
bool gme::PlayerControllerMulti::isSpacePressed () const
```

Definition at line 211 of file [playerControllerMulti.cpp](#).

References [eng::Space](#).

18.109.3.4 operator=() [1/2]

```
PlayerControllerMulti & gme::PlayerControllerMulti::operator= (
    const PlayerControllerMulti & ) [delete]
```

18.109.3.5 operator=() [2/2]

```
PlayerControllerMulti & gme::PlayerControllerMulti::operator= (
    PlayerControllerMulti && ) [delete]
```

18.109.3.6 sendInputsIfChanged()

```
void gme::PlayerControllerMulti::sendInputsIfChanged () [private]
```

Definition at line 85 of file [playerControllerMulti.cpp](#).

References [eng::Down](#), [eng::Left](#), [eng::Right](#), [eng::Space](#), and [eng::Up](#).

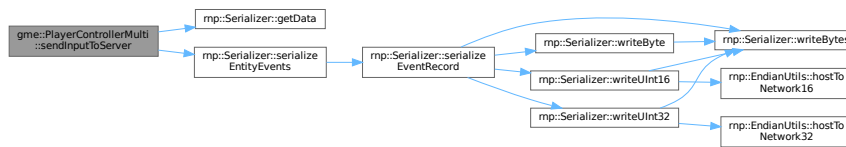
18.109.3.7 sendInputToServer()

```
void gme::PlayerControllerMulti::sendInputToServer (
    bool up,
    bool down,
    bool left,
    bool right,
    bool shoot) const [private]
```

Definition at line 105 of file [playerControllerMulti.cpp](#).

References [rnp::EventRecord::data](#), [rnp::EventRecord::entityId](#), [rnp::Serializer::getData\(\)](#), [rnp::INPUT](#), [utl::NETWORK_CLIENT](#), [utl::SEND_ENTITY_EVENT](#), [rnp::Serializer::serializeEntityEvents\(\)](#), and [rnp::EventRecord::type](#).

Here is the call graph for this function:



18.109.3.8 update()

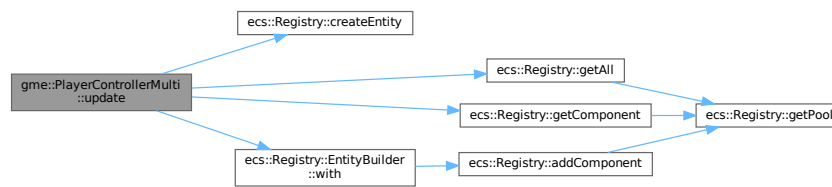
```
void gme::PlayerControllerMulti::update (
    ecs::Registry & registry,
    float dt) [override], [virtual]
```

Implements [ecs::ISystem](#).

Definition at line 130 of file [playerControllerMulti.cpp](#).

References [ecs::Registry::createEntity\(\)](#), [eng::Down](#), [ecs::Registry::getAll\(\)](#), [ecs::Registry::getComponent\(\)](#), [ecs::INVALID_ENTITY](#), [eng::Left](#), [eng::Right](#), [eng::Space](#), [eng::Up](#), and [ecs::Registry::EntityBuilder::with\(\)](#).

Here is the call graph for this function:



18.109.4 Member Data Documentation

18.109.4.1 INPUT_THROTTLE_INTERVAL

```
const float gme::PlayerControllerMulti::INPUT_THROTTLE_INTERVAL = 1.0f / 144.0f [private]
```

Definition at line 51 of file [PlayerControllerMulti.hpp](#).

18.109.4.2 m_componentId

```
uint32_t gme::PlayerControllerMulti::m_componentId [private]
```

Definition at line 48 of file [PlayerControllerMulti.hpp](#).

Referenced by [PlayerControllerMulti\(\)](#).

18.109.4.3 m_eventBus

```
utl::EventBus& gme::PlayerControllerMulti::m_eventBus [private]
```

Definition at line 49 of file [PlayerControllerMulti.hpp](#).

Referenced by [PlayerControllerMulti\(\)](#).

18.109.4.4 m_keysPressed

```
std::unordered_map<eng::Key, bool> gme::PlayerControllerMulti::m_keysPressed [private]
```

Definition at line 46 of file [PlayerControllerMulti.hpp](#).

18.109.4.5 m_lastAckSeqId

```
uint32_t gme::PlayerControllerMulti::m_lastAckSeqId = 0 [private]
```

Definition at line 54 of file [PlayerControllerMulti.hpp](#).

18.109.4.6 m_nextSeqId

uint32_t gme::PlayerControllerMulti::m_nextSeqId = 1 [private]

Definition at line 53 of file [PlayerControllerMulti.hpp](#).

18.109.4.7 m_playerEntity

[ecs::Entity](#) gme::PlayerControllerMulti::m_playerEntity [private]

Definition at line 47 of file [PlayerControllerMulti.hpp](#).

Referenced by [createPlayer\(\)](#).

18.109.4.8 m_renderer

const std::shared_ptr<[eng::IRenderer](#)>& gme::PlayerControllerMulti::m_renderer [private]

Definition at line 45 of file [PlayerControllerMulti.hpp](#).

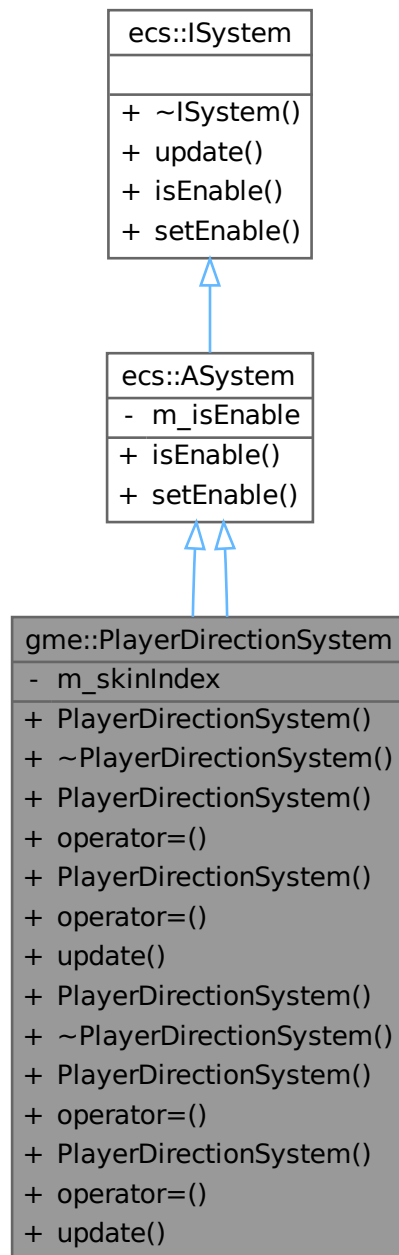
The documentation for this class was generated from the following files:

- [/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/include/RType↵ClientMulti/Systems/PlayerControllerMulti.hpp](#)
- [/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/src/Systems/playerControllerMulti.cp](#)

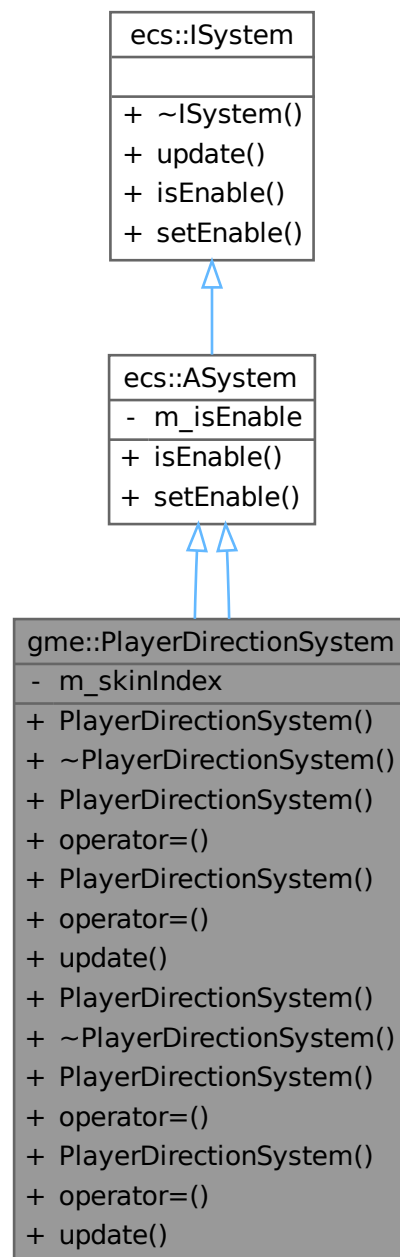
18.110 gme::PlayerDirectionSystem Class Reference

#include <PlayerDirection.hpp>

Inheritance diagram for gme::PlayerDirectionSystem:



Collaboration diagram for gme::PlayerDirectionSystem:



Public Member Functions

- [PlayerDirectionSystem](#) ()=default
- [~PlayerDirectionSystem](#) () override=default
- [PlayerDirectionSystem](#) (const [PlayerDirectionSystem](#) &)=delete
- [PlayerDirectionSystem](#) & operator= (const [PlayerDirectionSystem](#) &)=delete
- [PlayerDirectionSystem](#) ([PlayerDirectionSystem](#) &&)=delete
- [PlayerDirectionSystem](#) & operator= ([PlayerDirectionSystem](#) &&)=delete

- void [update](#) ([ecs::Registry](#) ®istry, float) override
- [PlayerDirectionSystem](#) (const int skinIndex)
- [~PlayerDirectionSystem](#) () override=default
- [PlayerDirectionSystem](#) (const [PlayerDirectionSystem](#) &)=delete
- [PlayerDirectionSystem](#) & operator= (const [PlayerDirectionSystem](#) &)=delete
- [PlayerDirectionSystem](#) ([PlayerDirectionSystem](#) &&)=delete
- [PlayerDirectionSystem](#) & operator= ([PlayerDirectionSystem](#) &&)=delete
- void [update](#) ([ecs::Registry](#) ®istry, float) override

Public Member Functions inherited from [ecs::ASystem](#)

- bool [isEnabled](#) () override
- void [setEnabled](#) (const bool enable) override

Public Member Functions inherited from [ecs::ISystem](#)

- virtual [~ISystem](#) ()=default

Private Attributes

- int [m_skinIndex](#)

18.110.1 Detailed Description

Definition at line 27 of file [PlayerDirection.hpp](#).

18.110.2 Constructor & Destructor Documentation

18.110.2.1 [PlayerDirectionSystem](#)() [1/6]

[gme::PlayerDirectionSystem::PlayerDirectionSystem](#) () [default]

18.110.2.2 [~PlayerDirectionSystem](#)() [1/2]

[gme::PlayerDirectionSystem::~~PlayerDirectionSystem](#) () [override], [default]

18.110.2.3 [PlayerDirectionSystem](#)() [2/6]

[gme::PlayerDirectionSystem::PlayerDirectionSystem](#) (
const [PlayerDirectionSystem](#) &) [delete]

18.110.2.4 [PlayerDirectionSystem](#)() [3/6]

[gme::PlayerDirectionSystem::PlayerDirectionSystem](#) (
[PlayerDirectionSystem](#) &&) [delete]

18.110.2.5 [PlayerDirectionSystem](#)() [4/6]

[gme::PlayerDirectionSystem::PlayerDirectionSystem](#) (
const int skinIndex) [inline], [explicit]

Definition at line 30 of file [PlayerDirection.hpp](#).

18.110.2.6 [~PlayerDirectionSystem](#)() [2/2]

[gme::PlayerDirectionSystem::~~PlayerDirectionSystem](#) () [override], [default]

18.110.2.7 [PlayerDirectionSystem](#)() [5/6]

[gme::PlayerDirectionSystem::PlayerDirectionSystem](#) (
const [PlayerDirectionSystem](#) &) [delete]

18.110.2.8 PlayerDirectionSystem() [6/6]

`gme::PlayerDirectionSystem::PlayerDirectionSystem (`
 [PlayerDirectionSystem](#) &&) [delete]

18.110.3 Member Function Documentation

18.110.3.1 operator=() [1/4]

[PlayerDirectionSystem](#) & `gme::PlayerDirectionSystem::operator= (`
 const [PlayerDirectionSystem](#) &) [delete]

18.110.3.2 operator=() [2/4]

[PlayerDirectionSystem](#) & `gme::PlayerDirectionSystem::operator= (`
 const [PlayerDirectionSystem](#) &) [delete]

18.110.3.3 operator=() [3/4]

[PlayerDirectionSystem](#) & `gme::PlayerDirectionSystem::operator= (`
 [PlayerDirectionSystem](#) &&) [delete]

18.110.3.4 operator=() [4/4]

[PlayerDirectionSystem](#) & `gme::PlayerDirectionSystem::operator= (`
 [PlayerDirectionSystem](#) &&) [delete]

18.110.3.5 update() [1/2]

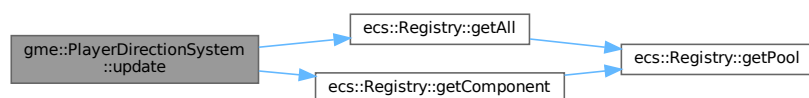
`void gme::PlayerDirectionSystem::update (`
 [ecs::Registry](#) & registry,
 float) [inline], [override], [virtual]

Implements [ecs::ISystem](#).

Definition at line 28 of file [PlayerDirection.hpp](#).

References [utl::GameConfig::Player::FRAMES_PER_ROW](#), [ecs::Registry::getAll\(\)](#), [ecs::Registry::getComponent\(\)](#), [M_PI](#), [utl::GameConfig::Player::SPRITE_HEIGHT](#), and [utl::GameConfig::Player::SPRITE_WIDTH](#).

Here is the call graph for this function:



18.110.3.6 update() [2/2]

`void gme::PlayerDirectionSystem::update (`
 [ecs::Registry](#) & registry,
 float) [inline], [override], [virtual]

Implements [ecs::ISystem](#).

Definition at line 38 of file [PlayerDirection.hpp](#).

References [utl::GameConfig::Player::FRAMES_PER_ROW](#), [ecs::Registry::getAll\(\)](#), [ecs::Registry::getComponent\(\)](#), [M_PI](#), [m_skinIndex](#), [utl::GameConfig::Player::SPRITE_HEIGHT](#), and [utl::GameConfig::Player::SPRITE_WIDTH](#).

Here is the call graph for this function:



18.110.4 Member Data Documentation

18.110.4.1 `m_skinIndex`

`int gme::PlayerDirectionSystem::m_skinIndex` [private]

Definition at line 98 of file [PlayerDirection.hpp](#).

Referenced by [update\(\)](#).

The documentation for this class was generated from the following files:

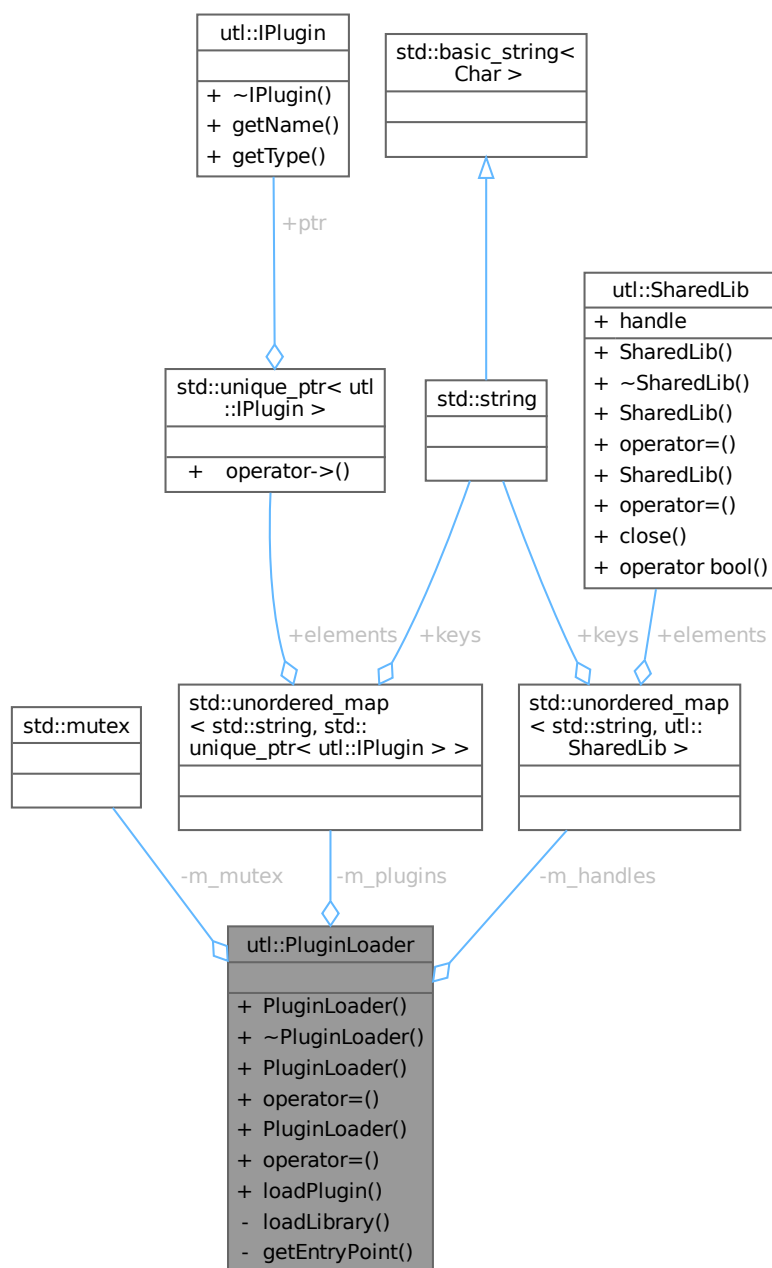
- `/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/include/RTypeClientMulti/Systems/PlayerDirection.hpp`
- `/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Solo/include/RTypeClientSolo/Systems/PlayerDirection.hpp`

18.111 `utl::PluginLoader` Class Reference

Modern, type-safe plugin loader.

`#include <PluginLoader.hpp>`

Collaboration diagram for utl::PluginLoader:



Public Member Functions

- `PluginLoader()`=default
- `~PluginLoader()`=default
- `PluginLoader(const PluginLoader &)=delete`
- `PluginLoader &operator=(const PluginLoader &)=delete`
- `PluginLoader(PluginLoader &&)=delete`
- `PluginLoader &operator=(PluginLoader &&)=delete`
- `template<std::derived_from<IPPlugin> T>`
`std::shared_ptr< T > loadPlugin(const std::string &path)`

Load a plugin of type T.

Private Member Functions

- [SharedLib loadLibrary](#) (const std::string &path)
- [EntryPointFn getEntryPoint](#) ([SharedLib](#) &lib, const std::string &path)

Private Attributes

- std::mutex [m_mutex](#)
- std::unordered_map< std::string, [SharedLib](#) > [m_handles](#)
- std::unordered_map< std::string, std::unique_ptr< [IPlugin](#) > > [m_plugins](#)

18.111.1 Detailed Description

Modern, type-safe plugin loader.

Definition at line 85 of file [PluginLoader.hpp](#).

18.111.2 Constructor & Destructor Documentation

18.111.2.1 PluginLoader() [1/3]

utl::PluginLoader::PluginLoader () [default]

18.111.2.2 ~PluginLoader()

utl::PluginLoader::~~PluginLoader () [default]

18.111.2.3 PluginLoader() [2/3]

utl::PluginLoader::PluginLoader (
const [PluginLoader](#) &) [delete]

18.111.2.4 PluginLoader() [3/3]

utl::PluginLoader::PluginLoader (
[PluginLoader](#) &&) [delete]

18.111.3 Member Function Documentation

18.111.3.1 getEntryPoint()

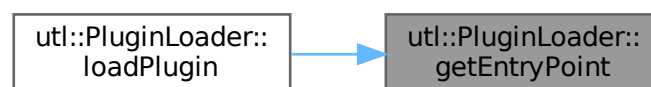
[EntryPointFn](#) utl::PluginLoader::getEntryPoint (
[SharedLib](#) & lib,
const std::string & path) [inline], [private]

Definition at line 176 of file [PluginLoader.hpp](#).

References [utl::SharedLib::handle](#).

Referenced by [loadPlugin\(\)](#).

Here is the caller graph for this function:



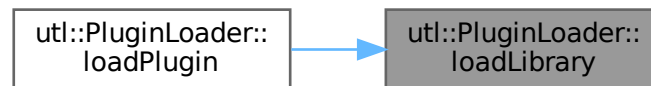
18.111.3.2 loadLibrary()

[SharedLib](#) utl::PluginLoader::loadLibrary (
 const std::string & path) [inline], [private]

Definition at line 137 of file [PluginLoader.hpp](#).

Referenced by [loadPlugin\(\)](#).

Here is the caller graph for this function:



18.111.3.3 loadPlugin()

template<std::derived_from< [IPlugin](#) > T>
 std::shared_ptr< T > utl::PluginLoader::loadPlugin (
 const std::string & path) [inline]

Load a plugin of type T.

Template Parameters

T	Expected plugin interface (must derive from IPlugin)
---	---

Parameters

path	Path to the dynamic library
------	---

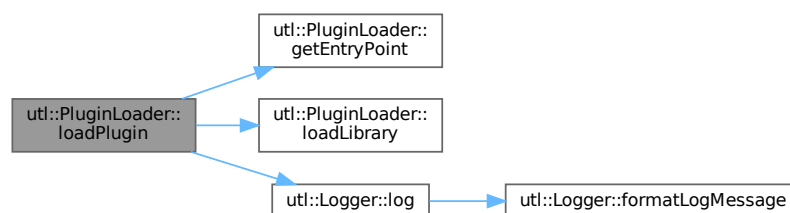
Returns

shared_ptr<T> instance

Definition at line 102 of file [PluginLoader.hpp](#).

References [getEntryPoint\(\)](#), [utl::INFO](#), [loadLibrary\(\)](#), [utl::Logger::log\(\)](#), [m_handles](#), [m_mutex](#), and [m_plugins](#).

Here is the call graph for this function:



18.111.3.4 operator=() [1/2]

[PluginLoader](#) & utl::PluginLoader::operator= (
const [PluginLoader](#) &) [delete]

18.111.3.5 operator=() [2/2]

[PluginLoader](#) & utl::PluginLoader::operator= (
[PluginLoader](#) &&) [delete]

18.111.4 Member Data Documentation

18.111.4.1 m_handles

std::unordered_map<std::string, [SharedLib](#)> utl::PluginLoader::m_handles [private]

Definition at line 134 of file [PluginLoader.hpp](#).

Referenced by [loadPlugin\(\)](#).

18.111.4.2 m_mutex

std::mutex utl::PluginLoader::m_mutex [private]

Definition at line 133 of file [PluginLoader.hpp](#).

Referenced by [loadPlugin\(\)](#).

18.111.4.3 m_plugins

std::unordered_map<std::string, std::unique_ptr<[IPlugin](#)> > utl::PluginLoader::m_plugins [private]

Definition at line 135 of file [PluginLoader.hpp](#).

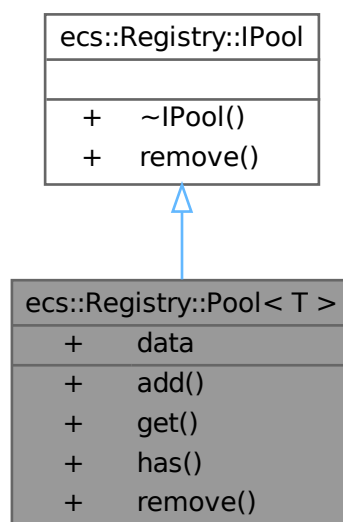
Referenced by [loadPlugin\(\)](#).

The documentation for this class was generated from the following file:

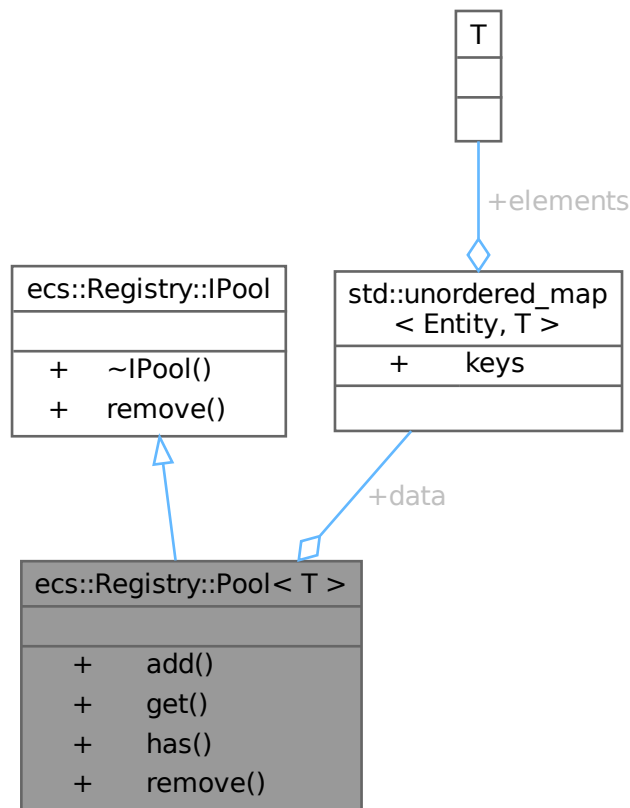
- /home/masina/Projects/Epitech/rtype/modules/Utils/include/Utils/[PluginLoader.hpp](#)

18.112 ecs::Registry::Pool< T > Class Template Reference

Inheritance diagram for `ecs::Registry::Pool< T >`:



Collaboration diagram for `ecs::Registry::Pool< T >`:



Public Member Functions

- `template<typename... Args>`
`T & add (Entity e, Args &&...args)`
- `T * get (Entity e)`
- `bool has (Entity e)`
- `void remove (Entity e)` override

Public Member Functions inherited from `ecs::Registry::IPool`

- `virtual ~IPool ()`=default

Public Attributes

- `std::unordered_map< Entity, T > data`

18.112.1 Detailed Description

`template<typename T>`
`class ecs::Registry::Pool< T >`

Definition at line 104 of file `Registry.hpp`.

18.112.2 Member Function Documentation

18.112.2.1 add()

```
template<typename T >
template<typename... Args>
T & ecs::Registry::Pool< T >::add (
```

```
    Entity e,
    Args &&... args) [inline]
```

Definition at line 109 of file [Registry.hpp](#).

References [ecs::Registry::Pool< T >::data](#).

18.112.2.2 get()

```
template<typename T >
T * ecs::Registry::Pool< T >::get (
    Entity e) [inline]
```

Definition at line 114 of file [Registry.hpp](#).

References [ecs::Registry::Pool< T >::data](#).

18.112.2.3 has()

```
template<typename T >
bool ecs::Registry::Pool< T >::has (
    Entity e) [inline]
```

Definition at line 124 of file [Registry.hpp](#).

References [ecs::Registry::Pool< T >::data](#).

18.112.2.4 remove()

```
template<typename T >
void ecs::Registry::Pool< T >::remove (
    Entity e) [inline], [override], [virtual]
```

Implements [ecs::Registry::IPool](#).

Definition at line 126 of file [Registry.hpp](#).

References [ecs::Registry::Pool< T >::data](#).

18.112.3 Member Data Documentation

18.112.3.1 data

```
template<typename T >
std::unordered_map<Entity, T> ecs::Registry::Pool< T >::data
```

Definition at line 107 of file [Registry.hpp](#).

Referenced by [ecs::Registry::Pool< T >::add\(\)](#), [ecs::Registry::Pool< T >::get\(\)](#), [ecs::Registry::Pool< T >::has\(\)](#), and [ecs::Registry::Pool< T >::remove\(\)](#).

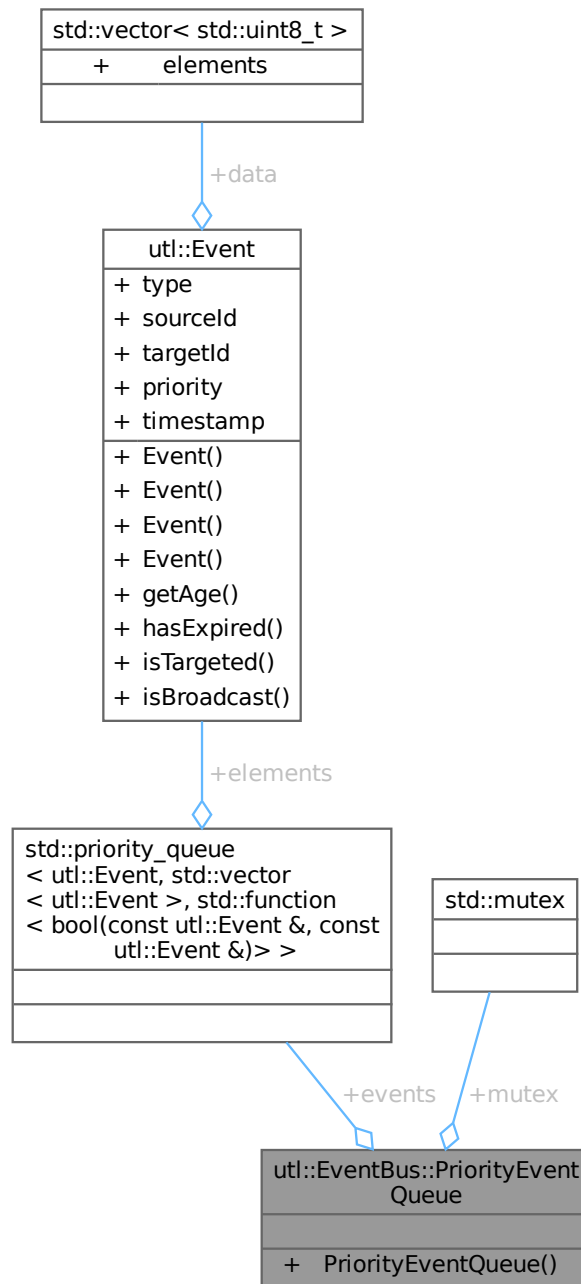
The documentation for this class was generated from the following file:

- [/home/masina/Projects/Epitech/rtype/modules/ECS/include/ECS/Registry.hpp](#)

18.113 utl::EventBus::PriorityEventQueue Struct Reference

Priority queue for events.

Collaboration diagram for utl::EventBus::PriorityEventQueue:



Public Member Functions

- [PriorityEventQueue](#) ()

Public Attributes

- `std::priority_queue< Event, std::vector< Event >, std::function< bool(const Event &, const Event &)> >` `events`
- `std::mutex` `mutex`

18.113.1 Detailed Description

Priority queue for events.

Definition at line 47 of file [EventBus.hpp](#).

18.113.2 Constructor & Destructor Documentation

18.113.2.1 PriorityQueueQueue()

utl::EventBus::PriorityEventQueue::PriorityEventQueue () [inline]

Definition at line 53 of file [EventBus.hpp](#).

18.113.3 Member Data Documentation

18.113.3.1 events

std::priority_queue<[Event](#), std::vector<[Event](#)>, std::function<bool(const [Event](#) &, const [Event](#) &> > utl::EventBus::PriorityEventQueue::events

Definition at line 50 of file [EventBus.hpp](#).

18.113.3.2 mutex

std::mutex utl::EventBus::PriorityEventQueue::mutex [mutable]

Definition at line 51 of file [EventBus.hpp](#).

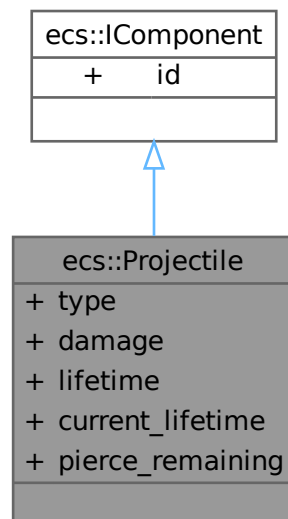
The documentation for this struct was generated from the following file:

- [/home/masina/Projects/Epitech/rtype/modules/Utils/include/Utils/EventBus.hpp](#)

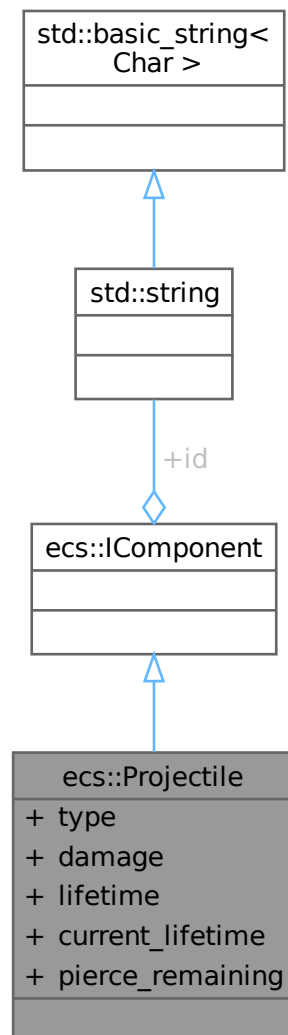
18.114 ecs::Projectile Struct Reference

#include <Component.hpp>

Inheritance diagram for ecs::Projectile:



Collaboration diagram for `ecs::Projectile`:



Public Types

- enum `Type` { `BASIC` , `SUPERCHARGED` }

Public Attributes

- `Type type`
- float `damage`
- float `lifetime`
- float `current_lifetime`
- int `pierce_remaining`

Public Attributes inherited from `ecs::IComponent`

- std::string `id`

18.114.1 Detailed Description

Definition at line 82 of file [Component.hpp](#).

18.114.2 Member Enumeration Documentation

18.114.2.1 Type

enum [ecs::Projectile::Type](#)

Enumerator

BASIC	
SUPERCHARGED	

Definition at line 84 of file [Component.hpp](#).

18.114.3 Member Data Documentation

18.114.3.1 current_lifetime

float [ecs::Projectile::current_lifetime](#)

Definition at line 92 of file [Component.hpp](#).

18.114.3.2 damage

float [ecs::Projectile::damage](#)

Definition at line 90 of file [Component.hpp](#).

18.114.3.3 lifetime

float [ecs::Projectile::lifetime](#)

Definition at line 91 of file [Component.hpp](#).

18.114.3.4 pierce_remaining

int [ecs::Projectile::pierce_remaining](#)

Definition at line 93 of file [Component.hpp](#).

18.114.3.5 type

[Type](#) [ecs::Projectile::type](#)

Definition at line 89 of file [Component.hpp](#).

The documentation for this struct was generated from the following file:

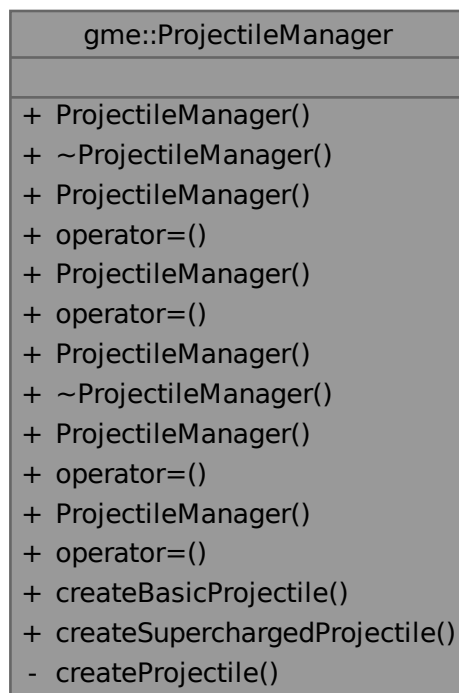
- [/home/masina/Projects/Epitech/rtype/modules/ECS/include/ECS/Component.hpp](#)

18.115 gme::ProjectileManager Class Reference

Handles projectile creation and management.

#include <ProjectileManager.hpp>

Collaboration diagram for gme::ProjectileManager:



Public Member Functions

- [ProjectileManager](#) ()=default
- [~ProjectileManager](#) ()=default
- [ProjectileManager](#) (const [ProjectileManager](#) &)=delete
- [ProjectileManager](#) & operator= (const [ProjectileManager](#) &)=delete
- [ProjectileManager](#) ([ProjectileManager](#) &&)=delete
- [ProjectileManager](#) & operator= ([ProjectileManager](#) &&)=delete
- [ProjectileManager](#) ()=default
- [~ProjectileManager](#) ()=default
- [ProjectileManager](#) (const [ProjectileManager](#) &)=delete
- [ProjectileManager](#) & operator= (const [ProjectileManager](#) &)=delete
- [ProjectileManager](#) ([ProjectileManager](#) &&)=delete
- [ProjectileManager](#) & operator= ([ProjectileManager](#) &&)=delete

Static Public Member Functions

- static [ecs::Entity](#) [createBasicProjectile](#) ([ecs::Registry](#) ®istry, float x, float y, float velocityX, float velocityY)
Create a basic projectile.
- static [ecs::Entity](#) [createSuperchargedProjectile](#) ([ecs::Registry](#) ®istry, float x, float y, float velocityX, float velocityY)
Create a supercharged projectile.

Static Private Member Functions

- static [ecs::Entity](#) [createProjectile](#) ([ecs::Registry](#) ®istry, [ecs::Projectile::Type](#) type, float x, float y, float velocityX, float velocityY)
Create a projectile with given parameters.

18.115.1 Detailed Description

Handles projectile creation and management.

Definition at line 19 of file [ProjectileManager.hpp](#).

18.115.2 Constructor & Destructor Documentation

18.115.2.1 ProjectileManager() [1/6]

[gme::ProjectileManager::ProjectileManager](#) () [default]

18.115.2.2 ~ProjectileManager() [1/2]

[gme::ProjectileManager::~~ProjectileManager](#) () [default]

18.115.2.3 ProjectileManager() [2/6]

[gme::ProjectileManager::ProjectileManager](#) (
const [ProjectileManager](#) &) [delete]

18.115.2.4 ProjectileManager() [3/6]

[gme::ProjectileManager::ProjectileManager](#) (
[ProjectileManager](#) &&) [delete]

18.115.2.5 ProjectileManager() [4/6]

[gme::ProjectileManager::ProjectileManager](#) () [default]

18.115.2.6 ~ProjectileManager() [2/2]

[gme::ProjectileManager::~~ProjectileManager](#) () [default]

18.115.2.7 ProjectileManager() [5/6]

[gme::ProjectileManager::ProjectileManager](#) (
const [ProjectileManager](#) &) [delete]

18.115.2.8 ProjectileManager() [6/6]

[gme::ProjectileManager::ProjectileManager](#) (
[ProjectileManager](#) &&) [delete]

18.115.3 Member Function Documentation

18.115.3.1 createBasicProjectile()

[ecs::Entity](#) [gme::ProjectileManager::createBasicProjectile](#) (
[ecs::Registry](#) & registry,
float x,
float y,
float velocityX,
float velocityY) [static]

Create a basic projectile.

Parameters

registry	The ECS registry
x	X position
y	Y position
velocityX	X velocity
velocityY	Y velocity

Returns

The created entity

Definition at line 8 of file [projectileManager.cpp](#).

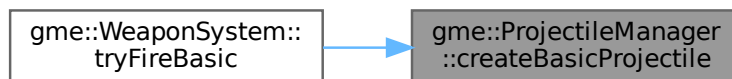
References [ecs::Projectile::BASIC](#), and [createProjectile\(\)](#).

Referenced by [gme::WeaponSystem::tryFireBasic\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.115.3.2 createProjectile()

```

ecs::Entity gme::ProjectileManager::createProjectile (
    ecs::Registry & registry,
    ecs::Projectile::Type type,
    float x,
    float y,
    float velocityX,
    float velocityY) [static], [private]

```

Create a projectile with given parameters.

Parameters

registry	The ECS registry
type	Projectile type
x	X position
y	Y position
velocityX	X velocity

velocityY	Y velocity
-----------	------------

Returns

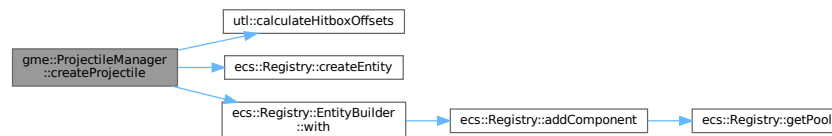
The created entity

Definition at line 20 of file [projectileManager.cpp](#).

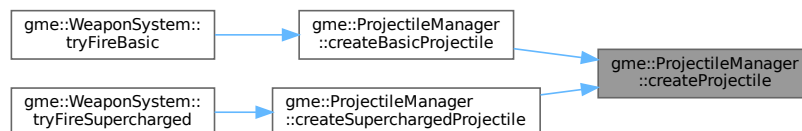
References [utl::GameConfig::Projectile::Supercharged::ANIMATION_DURATION](#), [utl::GameConfig::Projectile::Supercharged::BASIC](#), [utl::GameConfig::Projectile::Supercharged::DAMAGE](#), [utl::GameConfig::Projectile::Supercharged::LIFETIME](#), [utl::GameConfig::Hitbox::PROJECTILE_BASIC_RADIUS](#), [utl::GameConfig::Hitbox::PROJECTILE_SUPERCHARGED_RADIUS](#), [utl::GameConfig::Projectile::Basic::SCALE](#), [utl::GameConfig::Projectile::Supercharged::SCALE](#), [utl::GameConfig::Projectile::Basic::SPRITE_HEIGHT](#), [utl::GameConfig::Projectile::Supercharged::SPRITE_HEIGHT](#), [utl::GameConfig::Projectile::Basic::SPRITE_WIDTH](#), [utl::GameConfig::Projectile::Supercharged::SPRITE_WIDTH](#), [utl::Path::Texture::TEXTURE_SHOOT](#), [utl::Path::Texture::TEXTURE_SHOOT_CHARGED](#), and [ecs::Registry::EntityBuilder::with\(\)](#).

Referenced by [createBasicProjectile\(\)](#), and [createSuperchargedProjectile\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.115.3.3 createSuperchargedProjectile()

```

ecs::Entity gme::ProjectileManager::createSuperchargedProjectile (
    ecs::Registry & registry,
    float x,
    float y,
    float velocityX,
    float velocityY) [static]
  
```

Create a supercharged projectile.

Parameters

registry	The ECS registry
x	X position
y	Y position
velocityX	X velocity

velocityY	Y velocity
-----------	------------

Returns

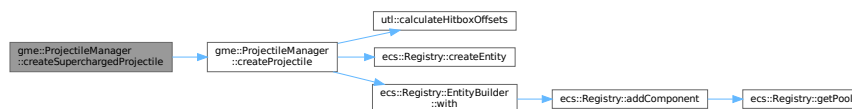
The created entity

Definition at line 14 of file [projectileManager.cpp](#).

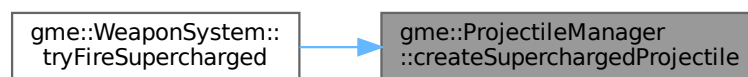
References [createProjectile\(\)](#), and [ecs::Projectile::SUPERCHARGED](#).

Referenced by [gme::WeaponSystem::tryFireSupercharged\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.115.3.4 operator=() [1/4]

[ProjectileManager](#) & gme::ProjectileManager::operator= (const [ProjectileManager](#) &) [delete]

18.115.3.5 operator=() [2/4]

[ProjectileManager](#) & gme::ProjectileManager::operator= (const [ProjectileManager](#) &) [delete]

18.115.3.6 operator=() [3/4]

[ProjectileManager](#) & gme::ProjectileManager::operator= ([ProjectileManager](#) &&) [delete]

18.115.3.7 operator=() [4/4]

[ProjectileManager](#) & gme::ProjectileManager::operator= ([ProjectileManager](#) &&) [delete]

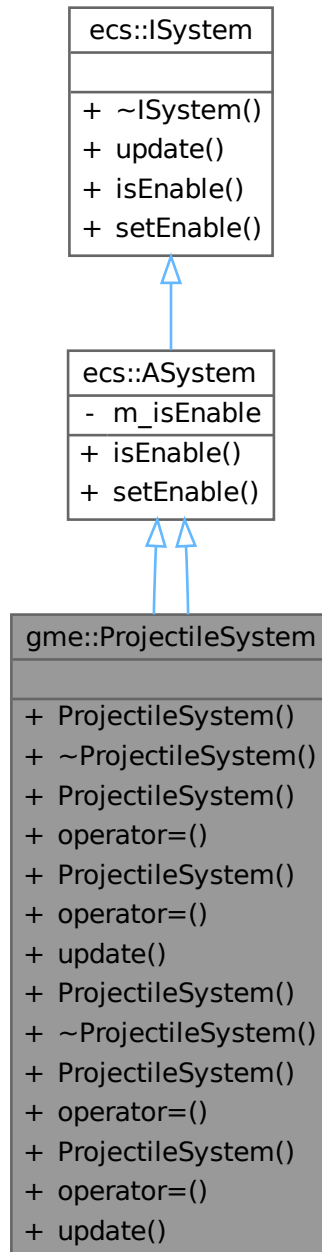
The documentation for this class was generated from the following files:

- /home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/include/RType↔ ClientMulti/Managers/[ProjectileManager.hpp](#)
- /home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Solo/include/RTypeClient↔ Solo/Managers/[ProjectileManager.hpp](#)
- /home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Solo/src/managers/[projectileManager.cpp](#)

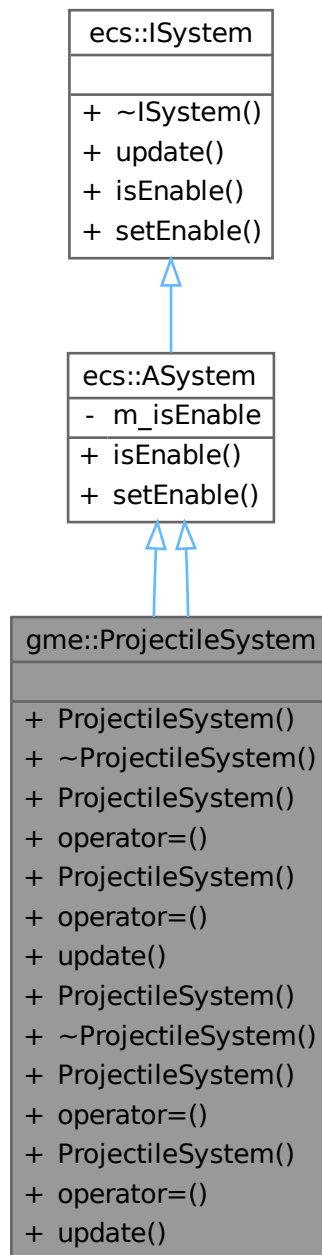
18.116 gme::ProjectileSystem Class Reference

#include <Projectile.hpp>

Inheritance diagram for gme::ProjectileSystem:



Collaboration diagram for gme::ProjectileSystem:



Public Member Functions

- [ProjectileSystem](#) (const std::shared_ptr< [eng::IRenderer](#) > &)
- [~ProjectileSystem](#) () override=default
- [ProjectileSystem](#) (const [ProjectileSystem](#) &)=delete
- [ProjectileSystem](#) & operator= (const [ProjectileSystem](#) &)=delete
- [ProjectileSystem](#) ([ProjectileSystem](#) &&)=delete
- [ProjectileSystem](#) & operator= ([ProjectileSystem](#) &&)=delete

- void `update` (`ecs::Registry` ®istry, float dt) override
- `ProjectileSystem` (const std::shared_ptr< `eng::IRenderer` > &)
- `~ProjectileSystem` () override=default
- `ProjectileSystem` (const `ProjectileSystem` &)=delete
- `ProjectileSystem` & `operator=` (const `ProjectileSystem` &)=delete
- `ProjectileSystem` (`ProjectileSystem` &&)=delete
- `ProjectileSystem` & `operator=` (`ProjectileSystem` &&)=delete
- void `update` (`ecs::Registry` ®istry, float dt) override

Public Member Functions inherited from `ecs::ASystem`

- bool `isEnabled` () override
- void `setEnabled` (const bool enable) override

Public Member Functions inherited from `ecs::ISystem`

- virtual `~ISystem` ()=default

18.116.1 Detailed Description

Definition at line 19 of file `Projectile.hpp`.

18.116.2 Constructor & Destructor Documentation

18.116.2.1 `ProjectileSystem`() [1/6]

```
gme::ProjectileSystem::ProjectileSystem (
    const std::shared_ptr< eng::IRenderer > & ) [inline], [explicit]
```

Definition at line 19 of file `Projectile.hpp`.

18.116.2.2 `~ProjectileSystem`() [1/2]

```
gme::ProjectileSystem::~~ProjectileSystem () [override], [default]
```

18.116.2.3 `ProjectileSystem`() [2/6]

```
gme::ProjectileSystem::ProjectileSystem (
    const ProjectileSystem & ) [delete]
```

18.116.2.4 `ProjectileSystem`() [3/6]

```
gme::ProjectileSystem::ProjectileSystem (
    ProjectileSystem && ) [delete]
```

18.116.2.5 `ProjectileSystem`() [4/6]

```
gme::ProjectileSystem::ProjectileSystem (
    const std::shared_ptr< eng::IRenderer > & ) [inline], [explicit]
```

Definition at line 22 of file `Projectile.hpp`.

18.116.2.6 `~ProjectileSystem`() [2/2]

```
gme::ProjectileSystem::~~ProjectileSystem () [override], [default]
```

18.116.2.7 `ProjectileSystem`() [5/6]

```
gme::ProjectileSystem::ProjectileSystem (
    const ProjectileSystem & ) [delete]
```


18.116.2.8 ProjectileSystem() [6/6]

`gme::ProjectileSystem::ProjectileSystem (`
 [ProjectileSystem](#) &&) [delete]

18.116.3 Member Function Documentation

18.116.3.1 operator=() [1/4]

[ProjectileSystem](#) & `gme::ProjectileSystem::operator= (`
 const [ProjectileSystem](#) &) [delete]

18.116.3.2 operator=() [2/4]

[ProjectileSystem](#) & `gme::ProjectileSystem::operator= (`
 const [ProjectileSystem](#) &) [delete]

18.116.3.3 operator=() [3/4]

[ProjectileSystem](#) & `gme::ProjectileSystem::operator= (`
 [ProjectileSystem](#) &&) [delete]

18.116.3.4 operator=() [4/4]

[ProjectileSystem](#) & `gme::ProjectileSystem::operator= (`
 [ProjectileSystem](#) &&) [delete]

18.116.3.5 update() [1/2]

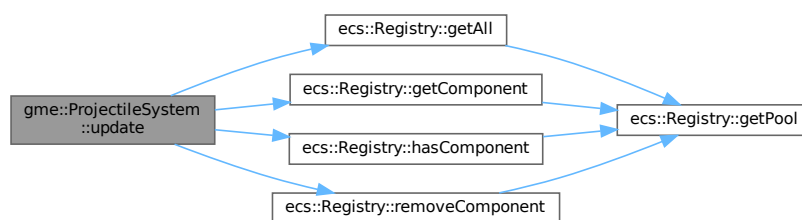
`void gme::ProjectileSystem::update (`
 [ecs::Registry](#) & registry,
 float dt) [override], [virtual]

Implements [ecs::ISystem](#).

Definition at line 4 of file [projectile.cpp](#).

References [ecs::Registry::getAll\(\)](#), [ecs::Registry::getComponent\(\)](#), [ecs::Registry::hasComponent\(\)](#), [ecs::Registry::removeComponent\(\)](#), and [ecs::Transform::x](#).

Here is the call graph for this function:



18.116.3.6 update() [2/2]

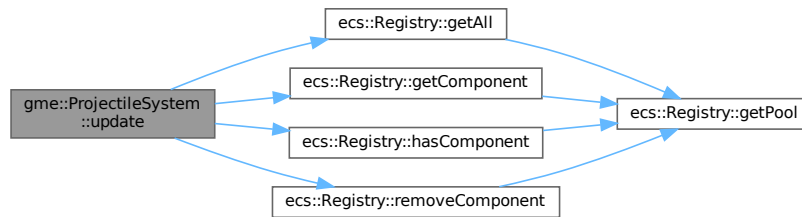
`void gme::ProjectileSystem::update (`
 [ecs::Registry](#) & registry,
 float dt) [inline], [override], [virtual]

Implements [ecs::ISystem](#).

Definition at line 30 of file [Projectile.hpp](#).

References [ecs::Registry::getAll\(\)](#), [ecs::Registry::getComponent\(\)](#), [ecs::Registry::hasComponent\(\)](#), [ecs::Registry::removeComponent\(\)](#), and [ecs::Transform::x](#).

Here is the call graph for this function:



The documentation for this class was generated from the following files:

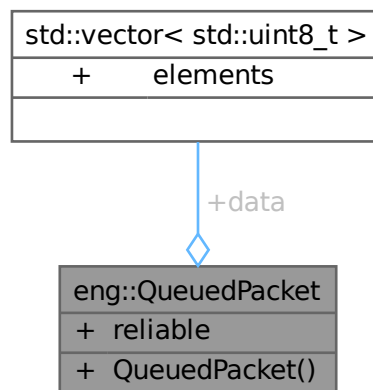
- /home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/include/RTypeClientMulti/Systems/[Projectile.hpp](#)
- /home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Solo/include/RTypeClientSolo/Systems/[Projectile.hpp](#)
- /home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/src/Systems/[projectile.cpp](#)

18.117 eng::QueuedPacket Struct Reference

Queued packet for sending.

```
#include <AsioClient.hpp>
```

Collaboration diagram for eng::QueuedPacket:



Public Member Functions

- [QueuedPacket](#) (const std::vector< std::uint8_t > &d, const bool rel=false)

Public Attributes

- std::vector< std::uint8_t > [data](#)
- bool [reliable](#)

18.117.1 Detailed Description

Queued packet for sending.

Definition at line 31 of file [AsioClient.hpp](#).

18.117.2 Constructor & Destructor Documentation

18.117.2.1 QueuedPacket()

```
eng::QueuedPacket::QueuedPacket (
    const std::vector< std::uint8_t > & d,
    const bool rel = false) [inline], [explicit]
```

Definition at line 36 of file [AsioClient.hpp](#).

18.117.3 Member Data Documentation

18.117.3.1 data

std::vector<std::uint8_t> eng::QueuedPacket::data

Definition at line 33 of file [AsioClient.hpp](#).

Referenced by [eng::AsioClient::processSendQueue\(\)](#).

18.117.3.2 reliable

bool eng::QueuedPacket::reliable

Definition at line 34 of file [AsioClient.hpp](#).

The documentation for this struct was generated from the following file:

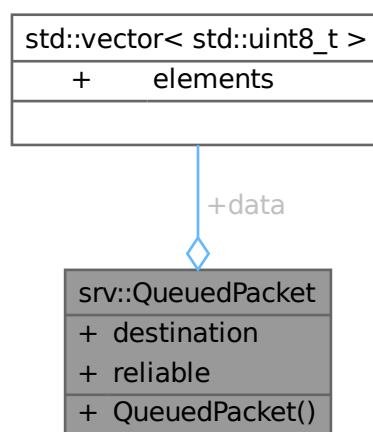
- [/home/masina/Projects/Epitech/rtype/plugins/Network/Asio/Client/include/AsioClient/AsioClient.hpp](#)

18.118 srv::QueuedPacket Struct Reference

Queued packet for sending.

#include <AsioServer.hpp>

Collaboration diagram for srv::QueuedPacket:



Public Member Functions

- [QueuedPacket](#) (const std::vector< std::uint8_t > &d, const asio::ip::udp::endpoint &dest, bool rel=false)

Public Attributes

- std::vector< std::uint8_t > [data](#)
- asio::ip::udp::endpoint [destination](#)
- bool [reliable](#)

18.118.1 Detailed Description

Queued packet for sending.

Definition at line 88 of file [AsioServer.hpp](#).

18.118.2 Constructor & Destructor Documentation

18.118.2.1 QueuedPacket()

```
srv::QueuedPacket::QueuedPacket (
    const std::vector< std::uint8_t > & d,
    const asio::ip::udp::endpoint & dest,
    bool rel = false) [inline]
```

Definition at line 94 of file [AsioServer.hpp](#).

18.118.3 Member Data Documentation

18.118.3.1 data

std::vector<std::uint8_t> srv::QueuedPacket::data

Definition at line 90 of file [AsioServer.hpp](#).

Referenced by [srv::AsioServer::processSendQueue\(\)](#).

18.118.3.2 destination

asio::ip::udp::endpoint srv::QueuedPacket::destination

Definition at line 91 of file [AsioServer.hpp](#).

Referenced by [srv::AsioServer::processSendQueue\(\)](#).

18.118.3.3 reliable

bool srv::QueuedPacket::reliable

Definition at line 92 of file [AsioServer.hpp](#).

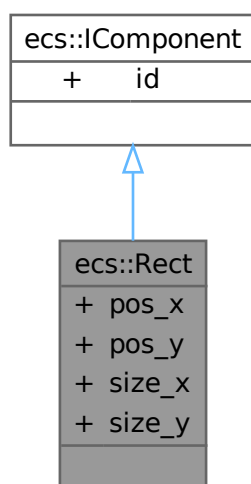
The documentation for this struct was generated from the following file:

- /home/masina/Projects/Epitech/rtype/plugins/Network/Asio/Server/include/AsioServer/[AsioServer.hpp](#)

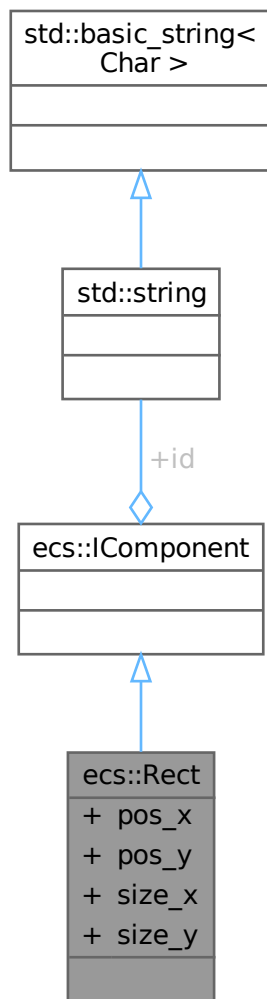
18.119 ecs::Rect Struct Reference

```
#include <Component.hpp>
```

Inheritance diagram for ecs::Rect:



Collaboration diagram for `ecs::Rect`:



Public Attributes

- float `pos_x` {}
- float `pos_y` {}
- int `size_x` {}
- int `size_y` {}

Public Attributes inherited from `ecs::IComponent`

- `std::string id`

18.119.1 Detailed Description

Definition at line 43 of file `Component.hpp`.

18.119.2 Member Data Documentation

18.119.2.1 pos_x

float ecs::Rect::pos_x {}

Definition at line 45 of file [Component.hpp](#).

18.119.2.2 pos_y

float ecs::Rect::pos_y {}

Definition at line 45 of file [Component.hpp](#).

18.119.2.3 size_x

int ecs::Rect::size_x {}

Definition at line 46 of file [Component.hpp](#).

18.119.2.4 size_y

int ecs::Rect::size_y {}

Definition at line 46 of file [Component.hpp](#).

The documentation for this struct was generated from the following file:

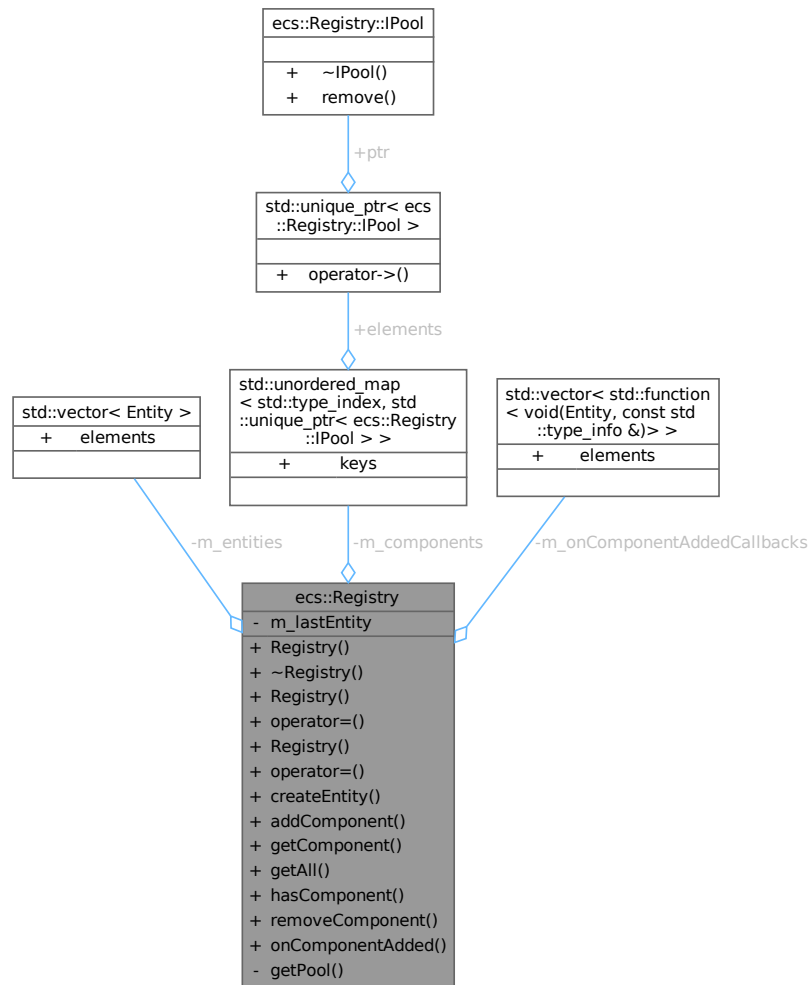
- [/home/masina/Projects/Epitech/rtype/modules/ECS/include/ECS/Component.hpp](#)

18.120 ecs::Registry Class Reference

Class for managing entities and their components.

#include <Registry.hpp>

Collaboration diagram for `ecs::Registry`:



Classes

- class [EntityBuilder](#)
- class [IPool](#)
- class [Pool](#)

Public Member Functions

- [Registry](#) ()=default
- [~Registry](#) ()=default
- [Registry](#) (const [Registry](#) &)=delete
- [Registry](#) & [operator=](#) (const [Registry](#) &)=delete
- [Registry](#) ([Registry](#) &&)=delete
- [Registry](#) & [operator=](#) ([Registry](#) &&)=delete
- [EntityBuilder](#) [createEntity](#) ()
- template<typename T , typename... Args>
T & [addComponent](#) ([Entity](#) e, Args &&...args)
- template<typename T >
T * [getComponent](#) ([Entity](#) e)

- `template<typename T >`
`std::unordered_map< Entity, T > & getAll ()`
- `template<typename T >`
`bool hasComponent (Entity e)`
- `template<typename T >`
`void removeComponent (Entity e)`
- `void onComponentAdded (std::function< void(Entity, const std::type_info &)> cb)`

Private Member Functions

- `template<typename T >`
`Pool< T > & getPool ()`

Private Attributes

- `Entity m\_lastEntity = INVALID\_ENTITY`
- `std::vector< Entity > m\_entities`
- `std::unordered_map< std::type_index, std::unique_ptr< IPool > > m\_components`
- `std::vector< std::function< void(Entity, const std::type_info &)> > > m\_onComponentAddedCallbacks`

18.120.1 Detailed Description

Class for managing entities and their components.

Definition at line 24 of file [Registry.hpp](#).

18.120.2 Constructor & Destructor Documentation

18.120.2.1 Registry() [1/3]

`ecs::Registry::Registry ()` [default]

18.120.2.2 ~Registry()

`ecs::Registry::~~Registry ()` [default]

18.120.2.3 Registry() [2/3]

`ecs::Registry::Registry (`
`const Registry & )` [delete]

18.120.2.4 Registry() [3/3]

`ecs::Registry::Registry (`
`Registry && )` [delete]

18.120.3 Member Function Documentation

18.120.3.1 addComponent()

`template<typename T , typename... Args>`
`T & ecs::Registry::addComponent (`
`Entity e,`
`Args &&... args)` [inline]

Definition at line 60 of file [Registry.hpp](#).

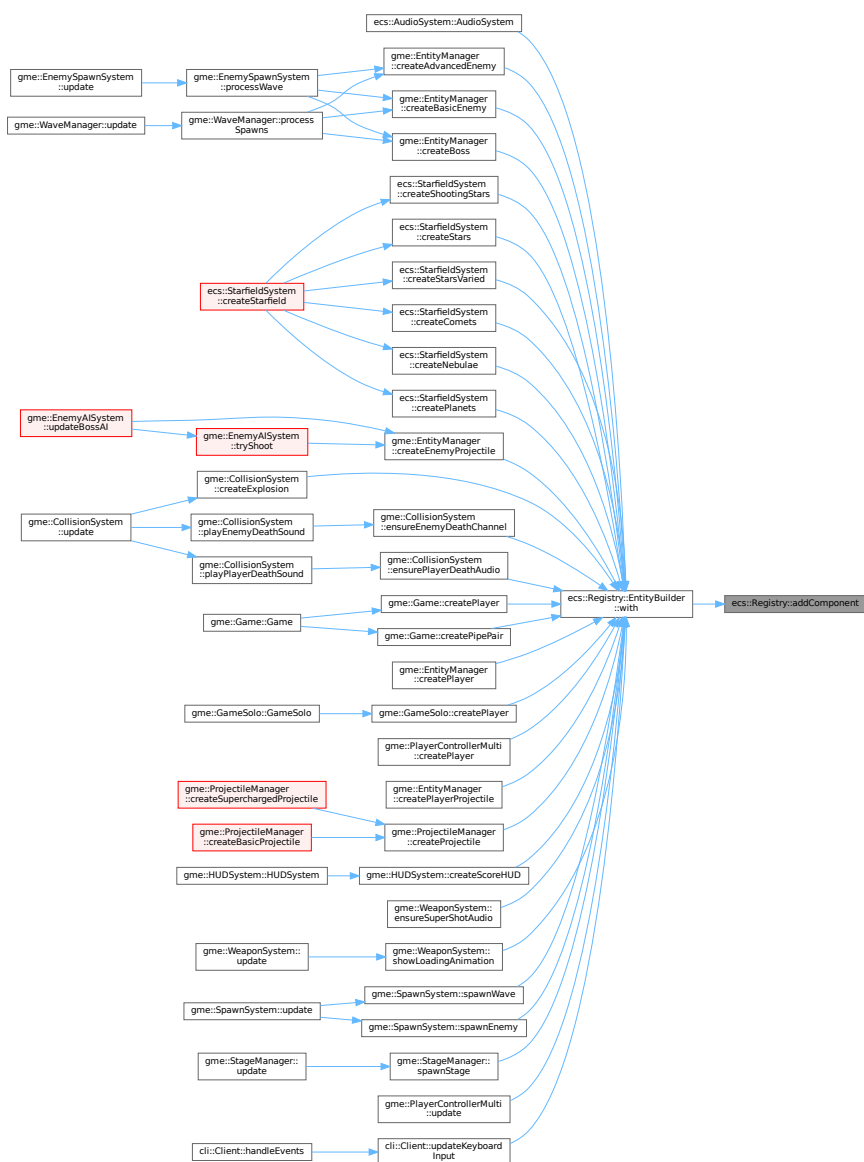
References [getPool\(\)](#), and [m_onComponentAddedCallbacks](#).

Referenced by [ecs::Registry::EntityBuilder::with\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.120.3.2 createEntity()

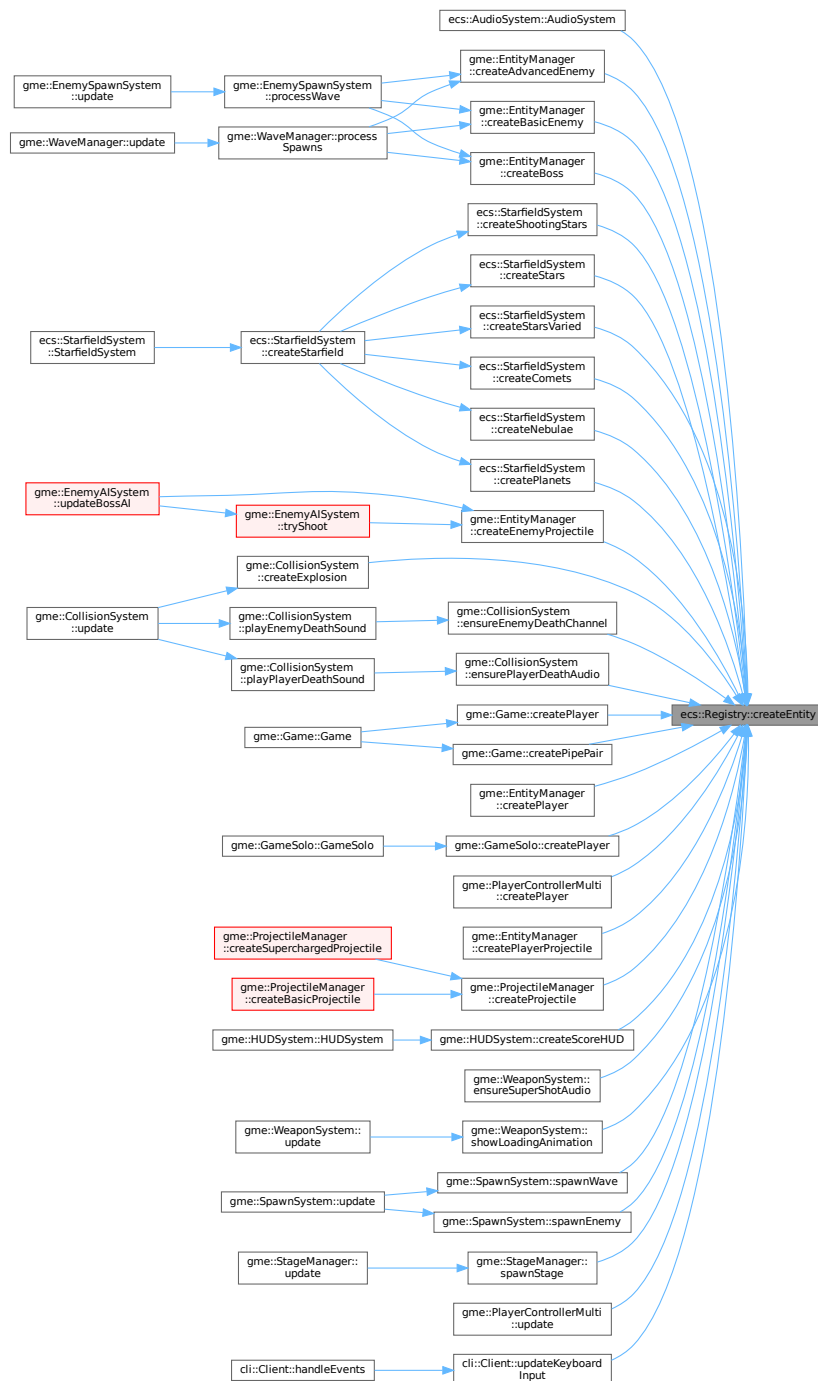
[EntityBuilder](#) `ecs::Registry::createEntity()` [inline]

Definition at line 53 of file [Registry.hpp](#).

References [m_entities](#), and [m_lastEntity](#).

Referenced by [ecs::AudioSystem::AudioSystem\(\)](#), [gme::EntityManager::createAdvancedEnemy\(\)](#), [gme::EntityManager::createBasicEnemy\(\)](#), [gme::EntityManager::createBoss\(\)](#), [ecs::StarfieldSystem::createComets\(\)](#), [gme::EntityManager::createEnemyProjectile\(\)](#), [gme::CollisionSystem::createExplosion\(\)](#), [ecs::StarfieldSystem::createNebula\(\)](#), [gme::Game::createPipePair\(\)](#), [ecs::StarfieldSystem::createPlanets\(\)](#), [gme::EntityManager::createPlayer\(\)](#), [gme::Game::createPlayer\(\)](#), [gme::GameSolo::createPlayer\(\)](#), [gme::PlayerControllerMulti::createPlayer\(\)](#), [gme::EntityManager::createPlayerProjectile\(\)](#), [gme::ProjectileManager::createProjectile\(\)](#), [gme::HUDSystem::createScore\(\)](#), [ecs::StarfieldSystem::createShootingStars\(\)](#), [ecs::StarfieldSystem::createStars\(\)](#), [ecs::StarfieldSystem::createStarsVaried\(\)](#), [gme::CollisionSystem::ensureEnemyDeathChannel\(\)](#), [gme::CollisionSystem::ensurePlayerDeathAudio\(\)](#), [gme::WeaponSystem::ensureSuperShotAudio\(\)](#), [gme::WeaponSystem::showLoadingAnimation\(\)](#), [gme::SpawnSystem::spawnStage\(\)](#), [gme::StageManager::spawnStage\(\)](#), [gme::SpawnSystem::spawnWave\(\)](#), [gme::PlayerControllerMulti::update\(\)](#), and [cli::Client::updateKeyboardInput\(\)](#).

Here is the caller graph for this function:



18.120.3.3 getAll()

```
template<typename T >
```

```
std::unordered_map< Entity, T > & ecs::Registry::getAll () [inline]
```

Definition at line 77 of file [Registry.hpp](#).

References [getPool\(\)](#).

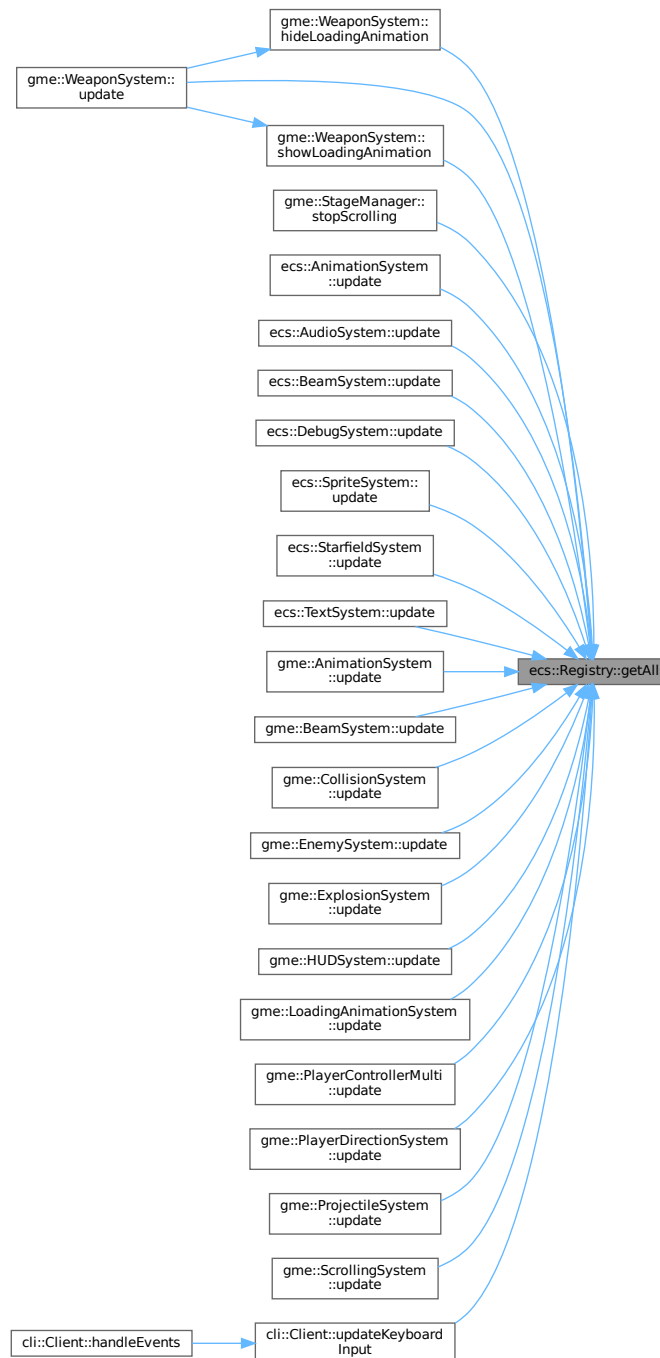
Referenced by [gme::WeaponSystem::hideLoadingAnimation\(\)](#), [gme::WeaponSystem::showLoadingAnimation\(\)](#), [gme::StageManager::stopScrolling\(\)](#), [ecs::AnimationSystem::update\(\)](#), [ecs::AudioSystem::update\(\)](#),

`ecs::BeamSystem::update()`, `ecs::DebugSystem::update()`, `ecs::SpriteSystem::update()`, `ecs::StarfieldSystem::update()`, `ecs::TextSystem::update()`, `gme::AnimationSystem::update()`, `gme::BeamSystem::update()`, `gme::CollisionSystem::update()`, `gme::EnemySystem::update()`, `gme::ExplosionSystem::update()`, `gme::HUDSystem::update()`, `gme::LoadingAnimationSystem::update()`, `gme::PlayerControllerMulti::update()`, `gme::PlayerDirectionSystem::update()`, `gme::ProjectileSystem::update()`, `gme::ScrollingSystem::update()`, `gme::WeaponSystem::update()`, and `cli::Client::updateKeyboardInput()`.

Here is the call graph for this function:



Here is the caller graph for this function:



18.120.3.4 GetComponent()

```
template<typename T >
T * ecs::Registry::GetComponent (
    Entity e) [inline]
```

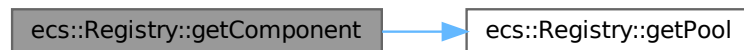
Definition at line 71 of file [Registry.hpp](#).

References [getPool\(\)](#).

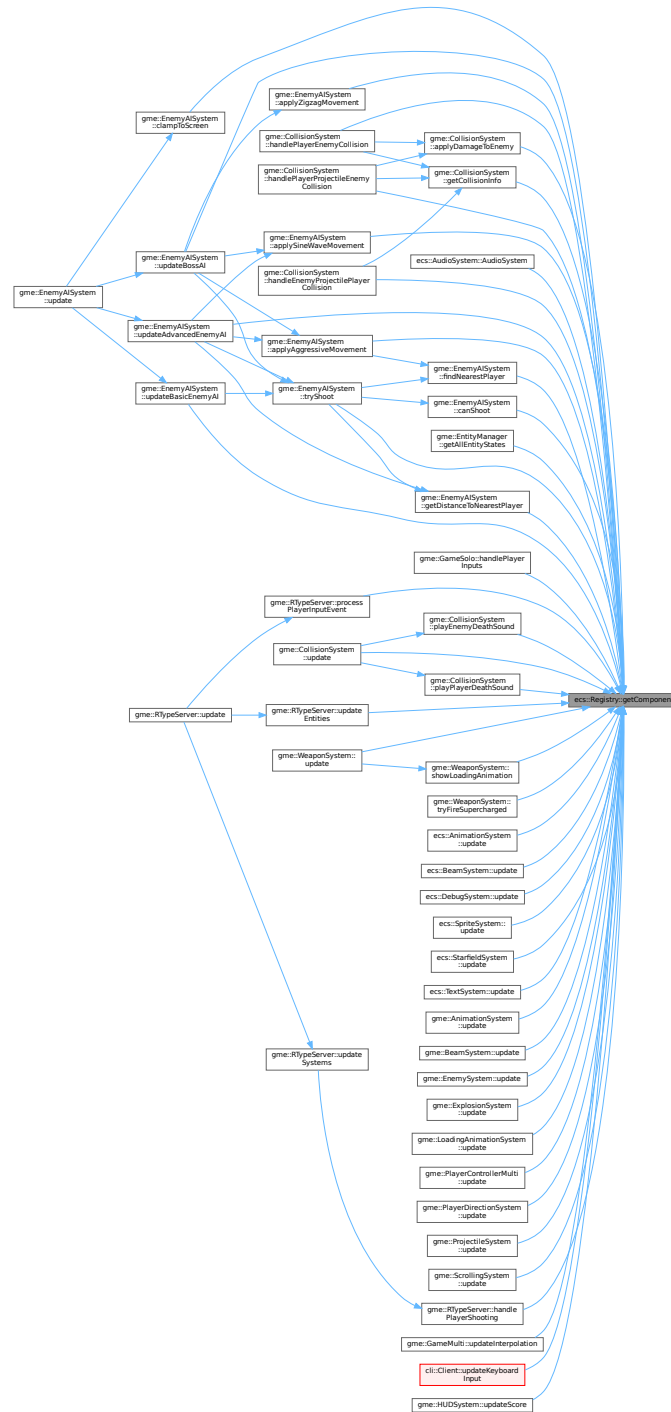
Referenced by [gme::EnemyAISystem::applyAggressiveMovement\(\)](#), [gme::CollisionSystem::applyDamageToEnemy\(\)](#),

[gme::EnemyAISystem::applySineWaveMovement\(\)](#), [gme::EnemyAISystem::applyZigzagMovement\(\)](#),
[ecs::AudioSystem::AudioSystem\(\)](#), [gme::EnemyAISystem::canShoot\(\)](#), [gme::EnemyAISystem::clampToScreen\(\)](#),
[gme::EnemyAISystem::findNearestPlayer\(\)](#), [gme::EntityManager::getAllEntityStates\(\)](#), [gme::CollisionSystem::getCollision](#)
[gme::EnemyAISystem::getDistanceToNearestPlayer\(\)](#), [gme::CollisionSystem::handleEnemyProjectilePlayerCollision\(\)](#),
[gme::CollisionSystem::handlePlayerEnemyCollision\(\)](#), [gme::GameSolo::handlePlayerInputs\(\)](#), [gme::CollisionSystem::hand](#)
[gme::RTypeServer::handlePlayerShooting\(\)](#), [gme::CollisionSystem::playEnemyDeathSound\(\)](#), [gme::CollisionSystem::playE](#)
[gme::RTypeServer::processPlayerInputEvent\(\)](#), [gme::WeaponSystem::showLoadingAnimation\(\)](#), [gme::WeaponSystem::try](#)
[gme::EnemyAISystem::tryShoot\(\)](#), [ecs::AnimationSystem::update\(\)](#), [ecs::BeamSystem::update\(\)](#),
[ecs::DebugSystem::update\(\)](#), [ecs::SpriteSystem::update\(\)](#), [ecs::StarfieldSystem::update\(\)](#), [ecs::TextSystem::update\(\)](#),
[gme::AnimationSystem::update\(\)](#), [gme::BeamSystem::update\(\)](#), [gme::CollisionSystem::update\(\)](#),
[gme::EnemySystem::update\(\)](#), [gme::ExplosionSystem::update\(\)](#), [gme::LoadingAnimationSystem::update\(\)](#),
[gme::PlayerControllerMulti::update\(\)](#), [gme::PlayerDirectionSystem::update\(\)](#), [gme::ProjectileSystem::update\(\)](#),
[gme::ScrollingSystem::update\(\)](#), [gme::WeaponSystem::update\(\)](#), [gme::EnemyAISystem::updateAdvancedEnemyAI\(\)](#),
[gme::EnemyAISystem::updateBasicEnemyAI\(\)](#), [gme::EnemyAISystem::updateBossAI\(\)](#), [gme::RTypeServer::updateEntiti](#)
[gme::GameMulti::updateInterpolation\(\)](#), [cli::Client::updateKeyboardInput\(\)](#), and [gme::HUDSystem::updateScore\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.120.3.5 getPool()

template<typename T >

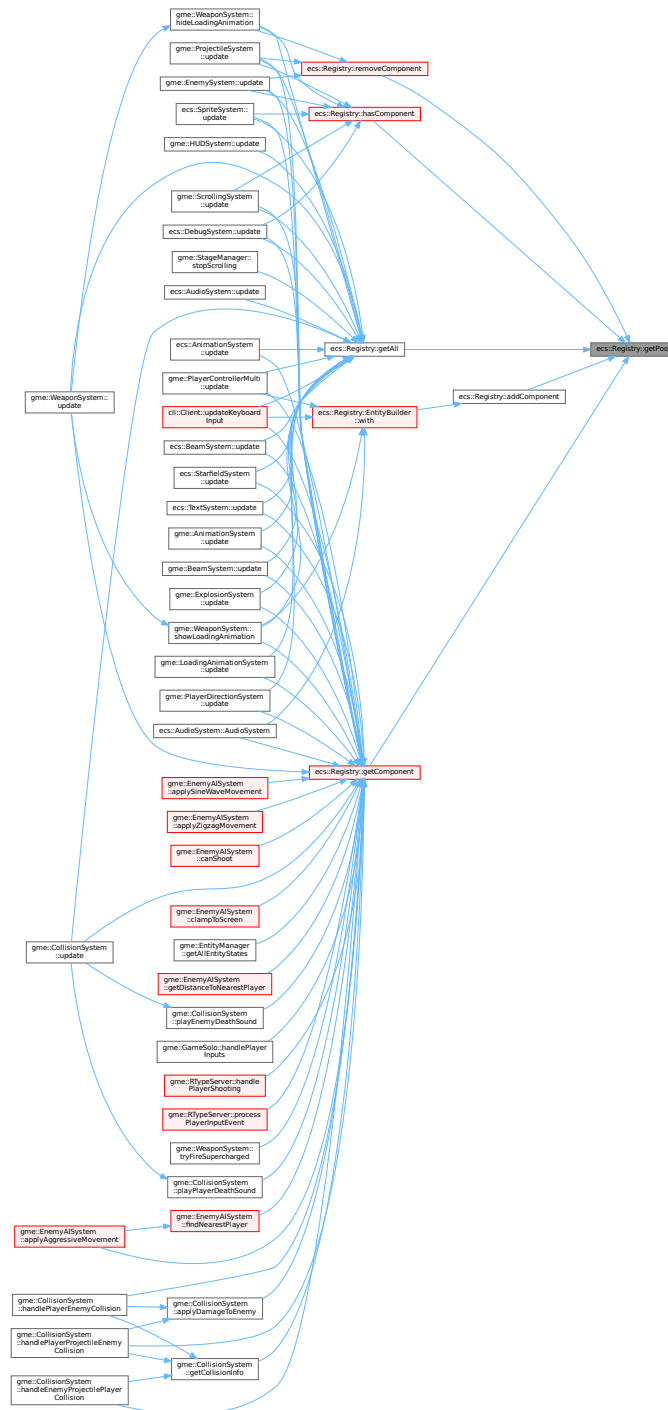
[Pool](#)< T > & ecs::Registry::getPool () [inline], [private]

Definition at line 129 of file [Registry.hpp](#).

References [m_components](#).

Referenced by [addComponent\(\)](#), [getAll\(\)](#), [getComponent\(\)](#), [hasComponent\(\)](#), and [removeComponent\(\)](#).

Here is the caller graph for this function:



18.120.3.6 hasComponent()

```
template<typename T >
```

```
bool ecs::Registry::hasComponent (
    Entity e) [inline]
```

Definition at line 79 of file [Registry.hpp](#).

References [getPool\(\)](#).

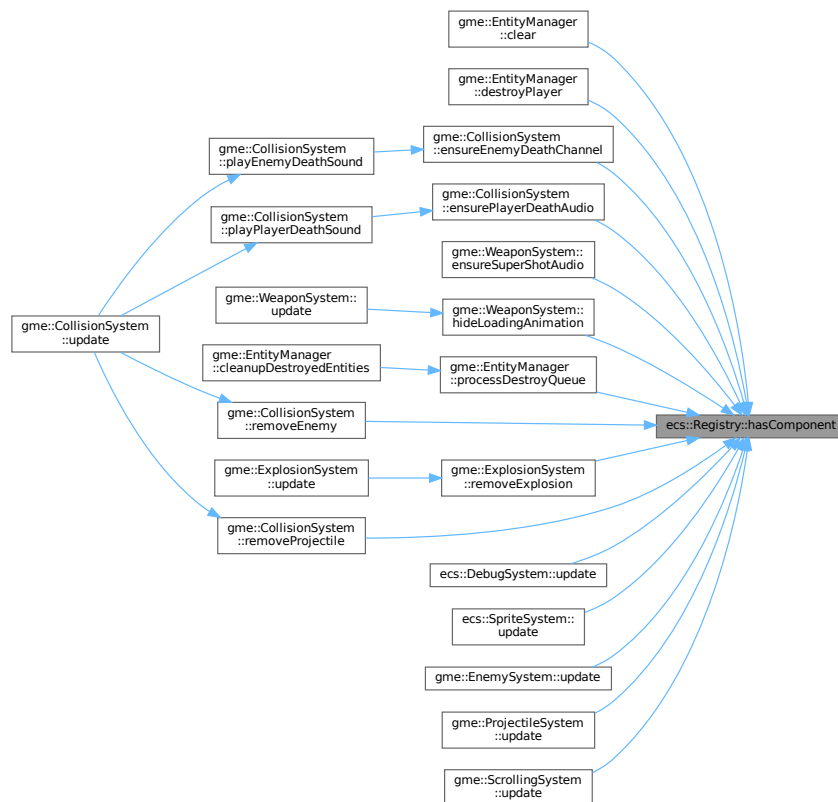
Referenced by [gme::EntityManager::clear\(\)](#), [gme::EntityManager::destroyPlayer\(\)](#), [gme::CollisionSystem::ensureEnemyDe](#)

gme::CollisionSystem::ensurePlayerDeathAudio(), gme::WeaponSystem::ensureSuperShotAudio(),
 gme::WeaponSystem::hideLoadingAnimation(), gme::EntityManager::processDestroyQueue(), gme::CollisionSystem::rem
 gme::ExplosionSystem::removeExplosion(), gme::CollisionSystem::removeProjectile(), ecs::DebugSystem::update(),
 ecs::SpriteSystem::update(), gme::EnemySystem::update(), gme::ProjectileSystem::update(), and
 gme::ScrollingSystem::update().

Here is the call graph for this function:



Here is the caller graph for this function:



18.120.3.7 onComponentAdded()

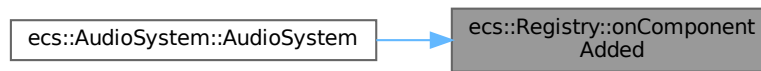
```
void ecs::Registry::onComponentAdded (
    std::function< void(Entity, const std::type_info &)> cb) [inline]
```

Definition at line 91 of file [Registry.hpp](#).

References [m_onComponentAddedCallbacks](#).

Referenced by [ecs::AudioSystem::AudioSystem\(\)](#).

Here is the caller graph for this function:



18.120.3.8 `operator=()` [1/2]

```
Registry & ecs::Registry::operator= (
    const Registry & ) [delete]
```

18.120.3.9 `operator=()` [2/2]

```
Registry & ecs::Registry::operator= (
    Registry && ) [delete]
```

18.120.3.10 `removeComponent()`

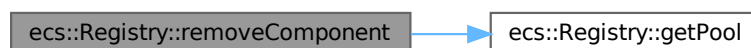
```
template<typename T >
void ecs::Registry::removeComponent (
    Entity e) [inline]
```

Definition at line 85 of file [Registry.hpp](#).

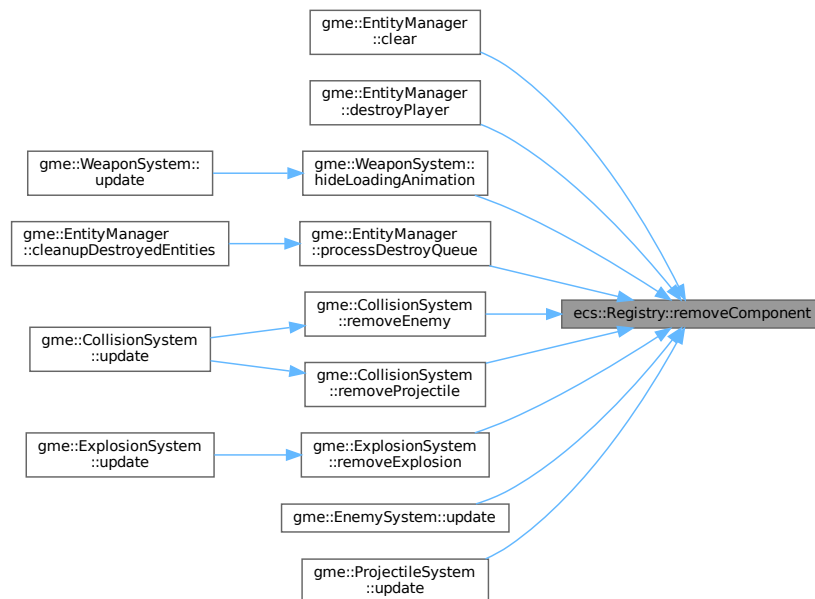
References [getPool\(\)](#).

Referenced by [gme::EntityManager::clear\(\)](#), [gme::EntityManager::destroyPlayer\(\)](#), [gme::WeaponSystem::hideLoadingAnim](#), [gme::EntityManager::processDestroyQueue\(\)](#), [gme::CollisionSystem::removeEnemy\(\)](#), [gme::ExplosionSystem::removeExpl](#), [gme::CollisionSystem::removeProjectile\(\)](#), [gme::EnemySystem::update\(\)](#), and [gme::ProjectileSystem::update\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.120.4 Member Data Documentation

18.120.4.1 m_components

`std::unordered_map<std::type_index, std::unique_ptr<IPool> > ecs::Registry::m_components` [private]

Definition at line 140 of file [Registry.hpp](#).

Referenced by [getPool\(\)](#).

18.120.4.2 m_entities

`std::vector<Entity> ecs::Registry::m_entities` [private]

Definition at line 139 of file [Registry.hpp](#).

Referenced by [createEntity\(\)](#).

18.120.4.3 m_lastEntity

`Entity ecs::Registry::m_lastEntity = INVALID_ENTITY` [private]

Definition at line 138 of file [Registry.hpp](#).

Referenced by [createEntity\(\)](#).

18.120.4.4 m_onComponentAddedCallbacks

`std::vector<std::function<void(Entity, const std::type_info &)> > ecs::Registry::m_onComponentAddedCallbacks` [private]

Definition at line 141 of file [Registry.hpp](#).

Referenced by [addComponent\(\)](#), and [onComponentAdded\(\)](#).

The documentation for this class was generated from the following file:

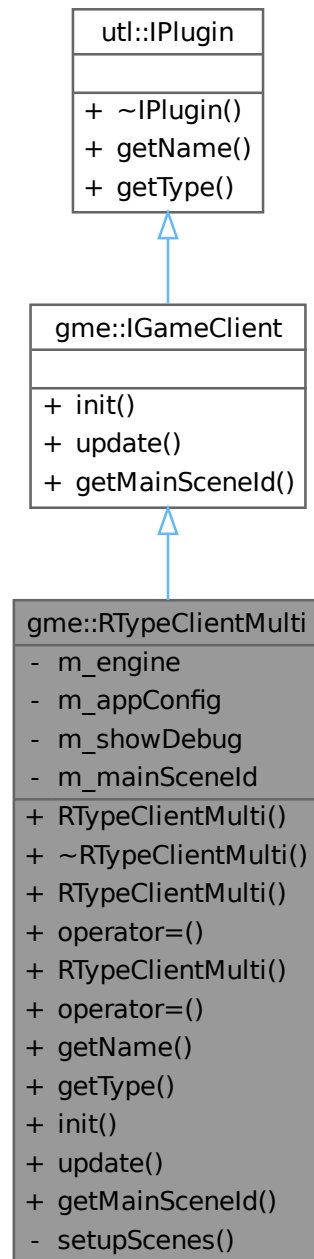
- [/home/masina/Projects/Epitech/rtype/modules/ECS/include/ECS/Registry.hpp](#)

18.121 gme::RTypeClientMulti Class Reference

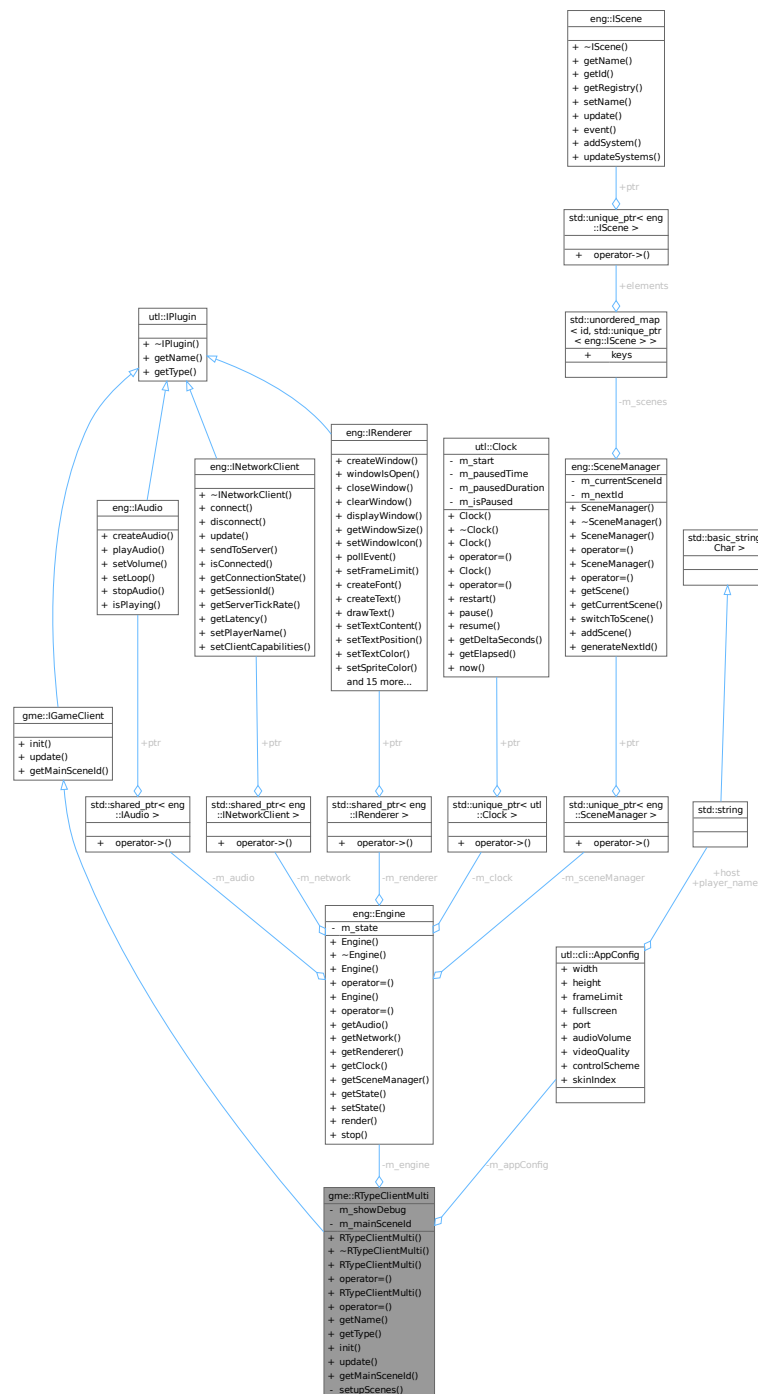
Class for the [RTypeClientMulti](#) plugin.

```
#include <RTypeClientMulti.hpp>
```

Inheritance diagram for gme::RTypeClientMulti:



Collaboration diagram for gme::RTypeClientMulti:



Public Member Functions

- [RTypeClientMulti](#) ()=default
- [~RTypeClientMulti](#) () override=default
- [RTypeClientMulti](#) (const [RTypeClientMulti](#) &)=delete
- [RTypeClientMulti](#) & operator= (const [RTypeClientMulti](#) &)=delete
- [RTypeClientMulti](#) ([RTypeClientMulti](#) &&)=delete
- [RTypeClientMulti](#) & operator= ([RTypeClientMulti](#) &&)=delete

- `const std::string getName () const` override
- `utl::PluginType getType () const` override
- `void init (eng::Engine &engine, utl::cli::AppConfig &appConfig, bool &showDebug, const eng::id menuSceneId)` override
- `void update (float deltaTime, unsigned int width, unsigned int height)` override
- `unsigned int getMainSceneId () const` override

Public Member Functions inherited from `utl::IPlugin`

- `virtual ~IPlugin ()=default`

Private Member Functions

- `void setupScenes (bool &showDebug, eng::id menuSceneId)`

Private Attributes

- `eng::Engine * m_engine = nullptr`
- `utl::cli::AppConfig * m_appConfig = nullptr`
- `bool m_showDebug = false`
- `eng::id m_mainSceneId = 1`

18.121.1 Detailed Description

Class for the `RTypeClientMulti` plugin.

Definition at line 19 of file `RTypeClientMulti.hpp`.

18.121.2 Constructor & Destructor Documentation

18.121.2.1 `RTypeClientMulti()` [1/3]

`gme::RTypeClientMulti::RTypeClientMulti ()` [default]

18.121.2.2 `~RTypeClientMulti()`

`gme::RTypeClientMulti::~~RTypeClientMulti ()` [override], [default]

18.121.2.3 `RTypeClientMulti()` [2/3]

`gme::RTypeClientMulti::RTypeClientMulti (const RTypeClientMulti & )` [delete]

18.121.2.4 `RTypeClientMulti()` [3/3]

`gme::RTypeClientMulti::RTypeClientMulti (RTypeClientMulti && )` [delete]

18.121.3 Member Function Documentation

18.121.3.1 `getMainSceneId()`

`unsigned int gme::RTypeClientMulti::getMainSceneId () const` [inline], [nodiscard], [override], [virtual]

Implements `gme::IGameClient`.

Definition at line 43 of file `RTypeClientMulti.hpp`.

References `m_mainSceneId`.

18.121.3.2 `getName()`

`const std::string gme::RTypeClientMulti::getName () const` [inline], [nodiscard], [override], [virtual]

Implements `utl::IPlugin`.

Definition at line 31 of file `RTypeClientMulti.hpp`.

18.121.3.3 `getType()`

`utl::PluginType` `gme::RTypeClientMulti::getType () const` [inline], [nodiscard], [override], [virtual]

Implements `utl::IPlugin`.

Definition at line 32 of file `RTypeClientMulti.hpp`.

References `utl::GAME_CLIENT`.

18.121.3.4 `init()`

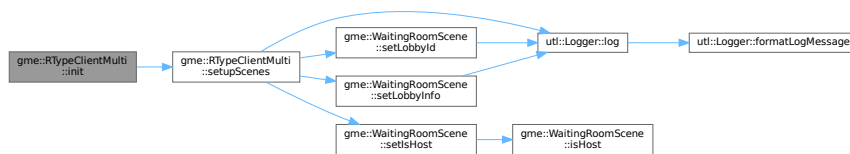
```
void gme::RTypeClientMulti::init (
    eng::Engine & engine,
    utl::cli::AppConfig & appConfig,
    bool & showDebug,
    const eng::id menuSceneId) [inline], [override], [virtual]
```

Implements `gme::IGameClient`.

Definition at line 34 of file `RTypeClientMulti.hpp`.

References `m_appConfig`, `m_engine`, `m_showDebug`, and `setupScenes()`.

Here is the call graph for this function:

18.121.3.5 `operator=()` [1/2]

`RTypeClientMulti` & `gme::RTypeClientMulti::operator= (`
`const RTypeClientMulti & )` [delete]

18.121.3.6 `operator=()` [2/2]

`RTypeClientMulti` & `gme::RTypeClientMulti::operator= (`
`RTypeClientMulti && )` [delete]

18.121.3.7 `setupScenes()`

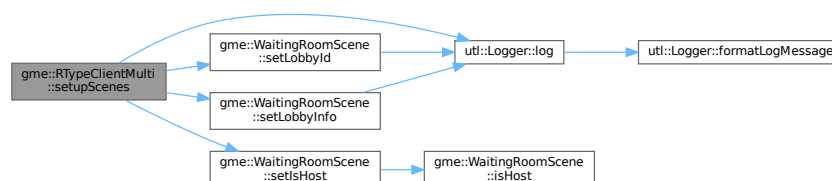
```
void gme::RTypeClientMulti::setupScenes (
    bool & showDebug,
    eng::id menuSceneId) [private]
```

Definition at line 14 of file `rtypeClientMulti.cpp`.

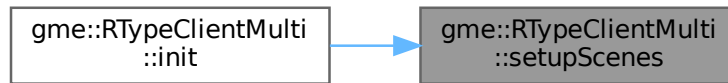
References `utl::INFO`, `utl::Logger::log()`, `gme::WaitingRoomScene::setIsHost()`, `gme::WaitingRoomScene::setLobbyId()`, and `gme::WaitingRoomScene::setLobbyInfo()`.

Referenced by `init()`.

Here is the call graph for this function:



Here is the caller graph for this function:



18.121.3.8 update()

```
void gme::RTypeClientMulti::update (
    float deltaTime,
    unsigned int width,
    unsigned int height) [override], [virtual]
```

Implements [gme::IGameClient](#).

Definition at line 12 of file [rTypeClientMulti.cpp](#).

18.121.4 Member Data Documentation

18.121.4.1 m_appConfig

```
utl::cli::AppConfig* gme::RTypeClientMulti::m_appConfig = nullptr [private]
```

Definition at line 49 of file [RTypeClientMulti.hpp](#).

Referenced by [init\(\)](#).

18.121.4.2 m_engine

```
eng::Engine* gme::RTypeClientMulti::m_engine = nullptr [private]
```

Definition at line 48 of file [RTypeClientMulti.hpp](#).

Referenced by [init\(\)](#).

18.121.4.3 m_mainSceneId

```
eng::id gme::RTypeClientMulti::m_mainSceneId = 1 [private]
```

Definition at line 51 of file [RTypeClientMulti.hpp](#).

Referenced by [getMainSceneId\(\)](#).

18.121.4.4 m_showDebug

```
bool gme::RTypeClientMulti::m_showDebug = false [private]
```

Definition at line 50 of file [RTypeClientMulti.hpp](#).

Referenced by [init\(\)](#).

The documentation for this class was generated from the following files:

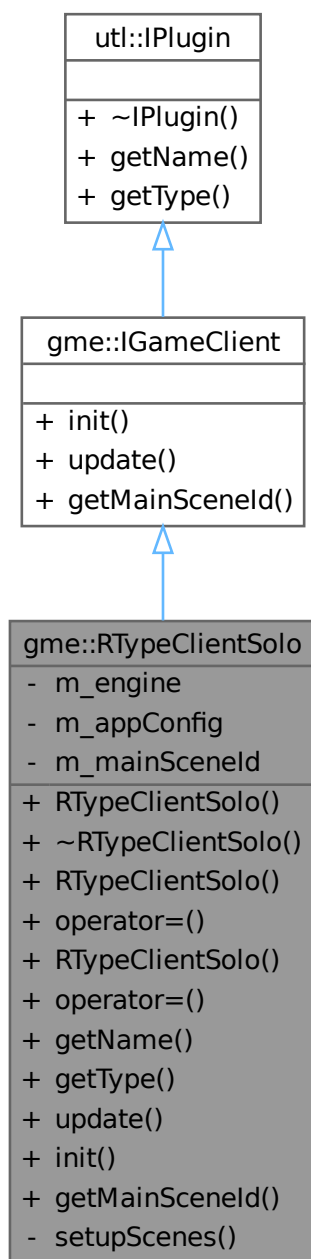
- [/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/include/RTypeClientMulti/RTypeClientMulti.hpp](#)
- [/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/src/rTypeClientMulti.cpp](#)

18.122 gme::RTypeClientSolo Class Reference

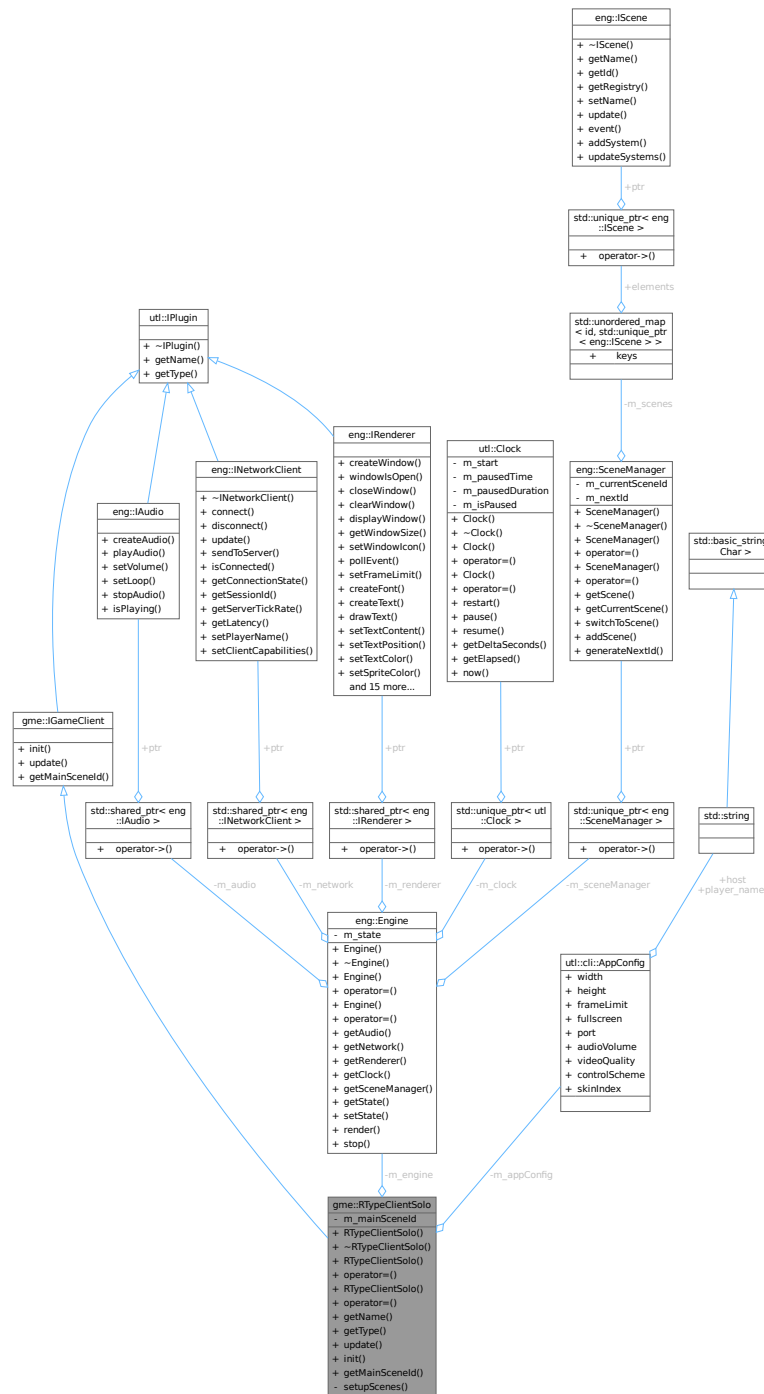
Class for the [RTypeClientSolo](#) plugin.

```
#include <RTypeClientSolo.hpp>
```

Inheritance diagram for gme::RTypeClientSolo:



Collaboration diagram for gme::RTypeClientSolo:



Public Member Functions

- [RTypeClientSolo \(\)](#)=default
- [~RTypeClientSolo \(\)](#) override=default
- [RTypeClientSolo \(const RTypeClientSolo &\)=delete](#)
- [RTypeClientSolo & operator= \(const RTypeClientSolo &\)=delete](#)
- [RTypeClientSolo \(RTypeClientSolo &&\)=delete](#)
- [RTypeClientSolo & operator= \(RTypeClientSolo &&\)=delete](#)

- `const std::string getName () const` override
- `utl::PluginType getType () const` override
- `void update (float deltaTime, unsigned int width, unsigned int height)` override
- `void init (eng::Engine &engine, utl::cli::AppConfig &appConfig, bool &showDebug, const eng::id menuSceneId)` override
- `unsigned int getMainSceneId () const` override

Public Member Functions inherited from [utl::IPlugin](#)

- `virtual ~IPlugin ()=default`

Private Member Functions

- `void setupScenes (bool &showDebug, eng::id menuSceneId)`

Private Attributes

- `eng::Engine * m_engine = nullptr`
- `utl::cli::AppConfig * m_appConfig = nullptr`
- `eng::id m_mainSceneId`

18.122.1 Detailed Description

Class for the [RTypeClientSolo](#) plugin.

Definition at line 19 of file [RTypeClientSolo.hpp](#).

18.122.2 Constructor & Destructor Documentation

18.122.2.1 RTypeClientSolo() [1/3]

`gme::RTypeClientSolo::RTypeClientSolo ()` [default]

18.122.2.2 ~RTypeClientSolo()

`gme::RTypeClientSolo::~~RTypeClientSolo ()` [override], [default]

18.122.2.3 RTypeClientSolo() [2/3]

`gme::RTypeClientSolo::RTypeClientSolo (const RTypeClientSolo & )` [delete]

18.122.2.4 RTypeClientSolo() [3/3]

`gme::RTypeClientSolo::RTypeClientSolo (RTypeClientSolo && )` [delete]

18.122.3 Member Function Documentation

18.122.3.1 getMainSceneId()

`unsigned int gme::RTypeClientSolo::getMainSceneId () const` [inline], [nodiscard], [override], [virtual]

Implements [gme::IGameClient](#).

Definition at line 43 of file [RTypeClientSolo.hpp](#).

References [m_mainSceneId](#).

18.122.3.2 getName()

`const std::string gme::RTypeClientSolo::getName () const` [inline], [nodiscard], [override], [virtual]

Implements [utl::IPlugin](#).

Definition at line 31 of file [RTypeClientSolo.hpp](#).

18.122.3.3 `getType()`

`utl::PluginType` gme::RTypeClientSolo::getType () const [inline], [nodiscard], [override], [virtual]

Implements `utl::IPlugin`.

Definition at line 32 of file `RTypeClientSolo.hpp`.

References `utl::GAME_CLIENT`.

18.122.3.4 `init()`

```
void gme::RTypeClientSolo::init (
    eng::Engine & engine,
    utl::cli::AppConfig & appConfig,
    bool & showDebug,
    const eng::id menuSceneId) [inline], [override], [virtual]
```

Implements `gme::IGameClient`.

Definition at line 35 of file `RTypeClientSolo.hpp`.

References `m_appConfig`, `m_engine`, and `setupScenes()`.

Here is the call graph for this function:

18.122.3.5 `operator=()` [1/2]

`RTypeClientSolo` & gme::RTypeClientSolo::operator= (const `RTypeClientSolo` &) [delete]

18.122.3.6 `operator=()` [2/2]

`RTypeClientSolo` & gme::RTypeClientSolo::operator= (`RTypeClientSolo` &&) [delete]

18.122.3.7 `setupScenes()`

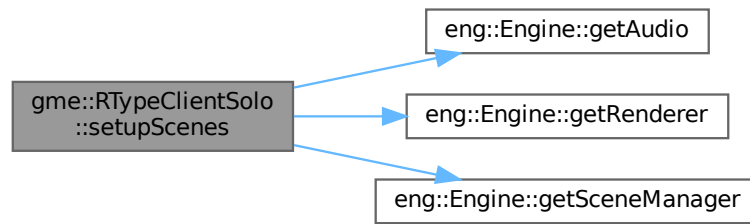
```
void gme::RTypeClientSolo::setupScenes (
    bool & showDebug,
    eng::id menuSceneId) [private]
```

Definition at line 6 of file `rtypeClientSolo.cpp`.

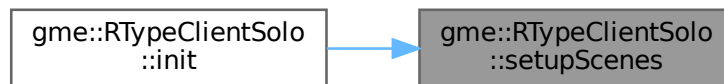
References `utl::cli::AppConfig::audioVolume`, `eng::Engine::getAudio()`, `eng::Engine::getRenderer()`, `eng::Engine::getSceneManager()`, `m_appConfig`, `m_engine`, `m_mainSceneId`, and `utl::cli::AppConfig::skinIndex`.

Referenced by `init()`.

Here is the call graph for this function:



Here is the caller graph for this function:



18.122.3.8 update()

```
void gme::RTypeClientSolo::update (
    float deltaTime,
    unsigned int width,
    unsigned int height) [override], [virtual]
```

Implements [gme::IGameClient](#).

Definition at line 59 of file [rTypeClientSolo.cpp](#).

18.122.4 Member Data Documentation

18.122.4.1 m_appConfig

```
utl::cli::AppConfig* gme::RTypeClientSolo::m_appConfig = nullptr [private]
```

Definition at line 47 of file [RTypeClientSolo.hpp](#).

Referenced by [init\(\)](#), and [setupScenes\(\)](#).

18.122.4.2 m_engine

```
eng::Engine* gme::RTypeClientSolo::m_engine = nullptr [private]
```

Definition at line 46 of file [RTypeClientSolo.hpp](#).

Referenced by [init\(\)](#), and [setupScenes\(\)](#).

18.122.4.3 m_mainSceneId

```
eng::id gme::RTypeClientSolo::m_mainSceneId [private]
```

Definition at line 48 of file [RTypeClientSolo.hpp](#).

Referenced by [getMainSceneId\(\)](#), and [setupScenes\(\)](#).

The documentation for this class was generated from the following files:

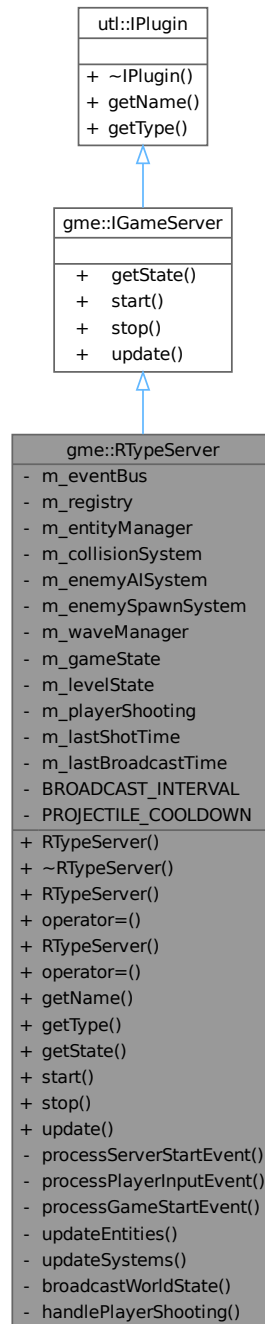
- [/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Solo/include/RTypeClientSolo/RTypeClientSolo.hpp](#)
- [/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Solo/src/rtypeClientSolo.cpp](#)

18.123 gme::RTypeServer Class Reference

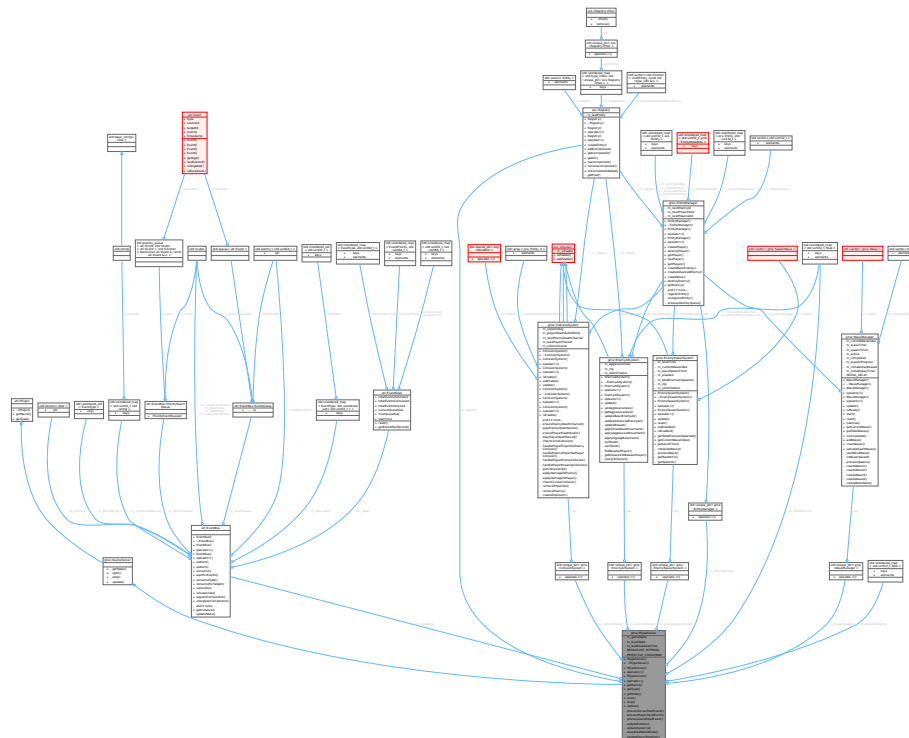
Class for the [RTypeServer](#) plugin.

#include <RTypeServer.hpp>

Inheritance diagram for gme::RTypeServer:



Collaboration diagram for `gme::RTypeServer`:



Public Member Functions

- [RTypeServer](#) ()
- [~RTypeServer](#) () override=default
- [RTypeServer](#) (const [RTypeServer](#) &)=delete
- [RTypeServer](#) & operator= (const [RTypeServer](#) &)=delete
- [RTypeServer](#) ([RTypeServer](#) &&)=delete
- [RTypeServer](#) & operator= ([RTypeServer](#) &&)=delete
- const std::string [getName](#) () const override
- [utl::PluginType](#) [getType](#) () const override
- [State](#) [getState](#) () const override
- void [start](#) () override
- void [stop](#) () override
- void [update](#) (float deltaTime) override

Public Member Functions inherited from [utl::IPlugin](#)

- virtual [~IPlugin](#) ()=default

Private Member Functions

- void [processServerStartEvent](#) (const [utl::Event](#) &event)
- void [processPlayerInputEvent](#) (const [utl::Event](#) &event)
- void [processGameStartEvent](#) (const [utl::Event](#) &event)
- void [updateEntities](#) (float deltaTime)
- void [updateSystems](#) (float deltaTime)
- void [broadcastWorldState](#) ()
- void [handlePlayerShooting](#) (std::uint32_t sessionId, float deltaTime)

Private Attributes

- [utl::EventBus](#) & [m_eventBus](#)
- [ecs::Registry](#) [m_registry](#)
- [std::unique_ptr< EntityManager >](#) [m_entityManager](#)
- [std::unique_ptr< CollisionSystem >](#) [m_collisionSystem](#)
- [std::unique_ptr< EnemyAISystem >](#) [m_enemyAISystem](#)
- [std::unique_ptr< EnemySpawnSystem >](#) [m_enemySpawnSystem](#)
- [std::unique_ptr< WaveManager >](#) [m_waveManager](#)
- [State](#) [m_gameState](#) = [State::PLAYING](#)
- [LevelState](#) [m_levelState](#) = [LevelState::WAITING_FOR_PLAYERS](#)
- [std::unordered_map< std::uint32_t, bool >](#) [m_playerShooting](#)
- [std::unordered_map< std::uint32_t, float >](#) [m_lastShotTime](#)
- [float](#) [m_lastBroadcastTime](#) = 0.0f
- [const float](#) [BROADCAST_INTERVAL](#) = 1.0f / 60.0f
- [const float](#) [PROJECTILE_COOLDOWN](#) = 0.3f

18.123.1 Detailed Description

Class for the [RTypeServer](#) plugin.

Definition at line 39 of file [RTypeServer.hpp](#).

18.123.2 Constructor & Destructor Documentation

18.123.2.1 RTypeServer() [1/3]

[gme::RTypeServer::RTypeServer](#) ()

Definition at line 18 of file [rtypeServer.cpp](#).

References [utl::GAME_LOGIC](#), [utl::GAME_START](#), [utl::INFO](#), [utl::Logger::log\(\)](#), [m_collisionSystem](#), [m_enemyAISystem](#), [m_enemySpawnSystem](#), [m_entityManager](#), [m_eventBus](#), [m_registry](#), [m_waveManager](#), [utl::PLAYER_INPUT_RECEIVED](#), [utl::EventBus::registerComponent\(\)](#), [utl::SERVER_START](#), and [utl::EventBus::subscribe\(\)](#).

Here is the call graph for this function:



18.123.2.2 ~RTypeServer()

[gme::RTypeServer::~~RTypeServer](#) () [override], [default]

18.123.2.3 RTypeServer() [2/3]

[gme::RTypeServer::RTypeServer](#) (
 [const RTypeServer](#) &) [delete]

18.123.2.4 RTypeServer() [3/3]

[gme::RTypeServer::RTypeServer](#) (
 [RTypeServer](#) &&) [delete]

18.123.3 Member Function Documentation

18.123.3.1 broadcastWorldState()

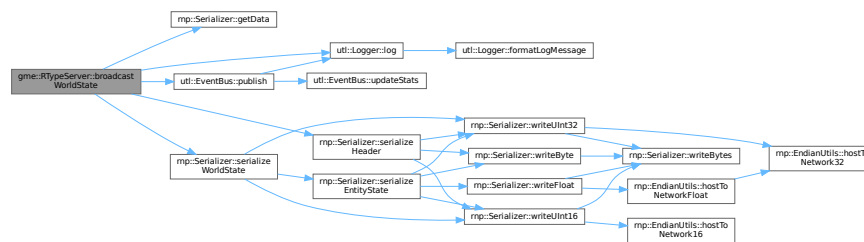
void gme::RTypeServer::broadcastWorldState () [private]

Definition at line 422 of file [rtypeServer.cpp](#).

References [utl::Event::data](#), [rnp::PacketWorldState::entities](#), [rnp::PacketWorldState::entityCount](#), [rnp::Serializer::getData\(\)](#), [rnp::PacketHeader::length](#), [utl::Logger::log\(\)](#), [m_entityManager](#), [m_eventBus](#), [m_lastBroadcastTime](#), [utl::NETWORK_SERVER](#), [utl::EventBus::publish\(\)](#), [utl::SEND_TO_CLIENT](#), [rnp::Serializer::serializeHeader\(\)](#), [rnp::Serializer::serializeWorldState\(\)](#), [rnp::PacketWorldState::serverTick](#), [rnp::PacketHeader::sessionId](#), [rnp::PacketHeader::type](#), [utl::WARNING](#), and [rnp::WORLD_STATE](#).

Referenced by [update\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.123.3.2 getName()

const std::string gme::RTypeServer::getName () const [inline], [nodiscard], [override], [virtual]

Implements [utl::IPlugin](#).

Definition at line 51 of file [RTypeServer.hpp](#).

18.123.3.3 getState()

State gme::RTypeServer::getState () const [inline], [nodiscard], [override], [virtual]

Implements [gme::IGameServer](#).

Definition at line 54 of file [RTypeServer.hpp](#).

References [m_gameState](#).

18.123.3.4 getType()

utl::PluginType gme::RTypeServer::getType () const [inline], [nodiscard], [override], [virtual]

Implements [utl::IPlugin](#).

Definition at line 52 of file [RTypeServer.hpp](#).

References [utl::GAME_SERVER](#).

18.123.3.5 handlePlayerShooting()

```
void gme::RTypeServer::handlePlayerShooting (
    std::uint32_t sessionId,
    float deltaTime) [private]
```

Definition at line 275 of file [rtypeServer.cpp](#).

References [ecs::BeamCharge::current_charge](#), [ecs::Registry::getComponent\(\)](#), [ecs::INVALID_ENTITY](#), [m_entityManager](#), [m_lastShotTime](#), [m_playerShooting](#), [m_registry](#), and [PROJECTILE_COOLDOWN](#).
Referenced by [updateSystems\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.123.3.6 operator=() [1/2]

```
RTypeServer & gme::RTypeServer::operator= (
    const RTypeServer & ) [delete]
```

18.123.3.7 operator=() [2/2]

```
RTypeServer & gme::RTypeServer::operator= (
    RTypeServer && ) [delete]
```

18.123.3.8 processGameStartEvent()

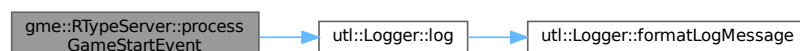
```
void gme::RTypeServer::processGameStartEvent (
    const utl::Event & event) [private]
```

Definition at line 238 of file [rtypeServer.cpp](#).

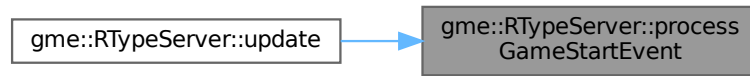
References [gme::IN_PROGRESS](#), [utl::INFO](#), [utl::Logger::log\(\)](#), [m_levelState](#), [m_waveManager](#), and [utl::WARNING](#).

Referenced by [update\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.123.3.9 processPlayerInputEvent()

```
void gme::RTypeServer::processPlayerInputEvent (
    const utl::Event & event) [private]
```

Definition at line 150 of file `rtypeServer.cpp`.

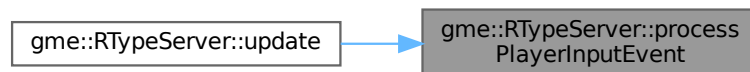
References `utl::Event::data`, `ecs::Registry::getComponent()`, `utl::Logger::log()`, `m_entityManager`, `m_lastShotTime`, `m_playerShooting`, `m_registry`, and `utl::WARNING`.

Referenced by `update()`.

Here is the call graph for this function:



Here is the caller graph for this function:



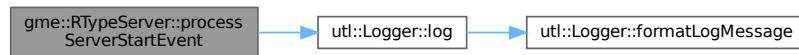
18.123.3.10 processServerStartEvent()

```
void gme::RTypeServer::processServerStartEvent (
    const utl::Event & event) [private]
```

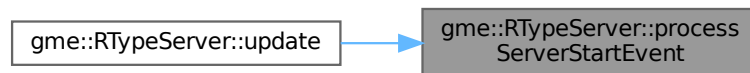
Definition at line 114 of file `rtypeServer.cpp`.

References `utl::Event::data`, `gme::IN_PROGRESS`, `utl::INFO`, `utl::Logger::log()`, `m_enemySpawnSystem`, `m_entityManager`, `m_lastShotTime`, `m_levelState`, `m_playerShooting`, and `gme::WAITING_FOR_PLAYERS`.
Referenced by `update()`.

Here is the call graph for this function:



Here is the caller graph for this function:



18.123.3.11 start()

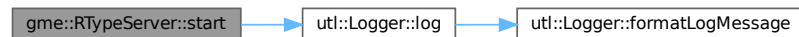
void gme::RTypeServer::start () [override], [virtual]

Implements [gme::IGameServer](#).

Definition at line 43 of file [rtypeServer.cpp](#).

References [utl::INFO](#), [utl::Logger::log\(\)](#), [m_enemySpawnSystem](#), [m_gameState](#), [m_levelState](#), [gme::PLAYING](#), and [gme::WAITING_FOR_PLAYERS](#).

Here is the call graph for this function:



18.123.3.12 stop()

void gme::RTypeServer::stop () [override], [virtual]

Implements [gme::IGameServer](#).

Definition at line 55 of file [rtypeServer.cpp](#).

References [utl::INFO](#), [utl::Logger::log\(\)](#), [gme::LOSE](#), [m_enemySpawnSystem](#), [m_entityManager](#), [m_gameState](#), [m_lastShotTime](#), [m_playerShooting](#), and [m_waveManager](#).

Here is the call graph for this function:



18.123.3.13 update()

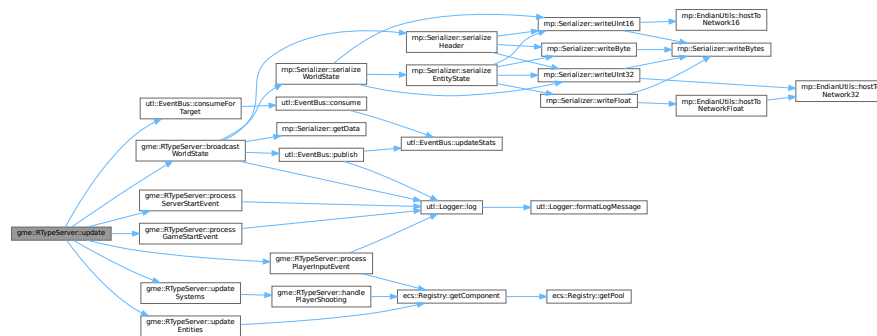
```
void gme::RTypeServer::update (
    float deltaTime) [override], [virtual]
```

Implements [gme::IGameServer](#).

Definition at line 73 of file [rtypeServer.cpp](#).

References [BROADCAST_INTERVAL](#), [broadcastWorldState\(\)](#), [utl::EventBus::consumeForTarget\(\)](#), [utl::GAME_LOGIC](#), [utl::GAME_START](#), [m_entityManager](#), [m_eventBus](#), [m_lastBroadcastTime](#), [utl::PLAYER_INPUT_RECEIVED](#), [processGameStartEvent\(\)](#), [processPlayerInputEvent\(\)](#), [processServerStartEvent\(\)](#), [utl::SERVER_START](#), [updateEntities\(\)](#), and [updateSystems\(\)](#).

Here is the call graph for this function:



18.123.3.14 updateEntities()

```
void gme::RTypeServer::updateEntities (
    float deltaTime) [private]
```

Definition at line 328 of file [rtypeServer.cpp](#).

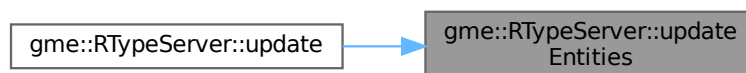
References [ecs::Registry::getComponent\(\)](#), [m_entityManager](#), [m_registry](#), [utl::GameConfig::Server::SCREEN_HEIGHT](#), [utl::GameConfig::Server::SCREEN_WIDTH](#), [utl::GameConfig::Server::WORLD_MARGIN](#), [ecs::Transform::x](#), [ecs::Velocity::x](#), and [ecs::Transform::y](#).

Referenced by [update\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.123.3.15 updateSystems()

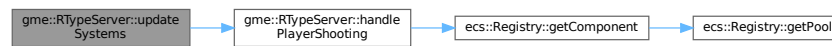
```
void gme::RTypeServer::updateSystems (
    float deltaTime) [private]
```

Definition at line 257 of file [rtypeServer.cpp](#).

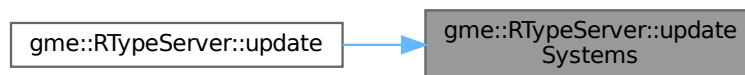
References [handlePlayerShooting\(\)](#), [m_collisionSystem](#), [m_enemyAISystem](#), [m_entityManager](#), [m_registry](#), and [m_waveManager](#).

Referenced by [update\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.123.4 Member Data Documentation

18.123.4.1 BROADCAST_INTERVAL

```
const float gme::RTypeServer::BROADCAST_INTERVAL = 1.0f / 60.0f [private]
```

Definition at line 89 of file [RTypeServer.hpp](#).

Referenced by [update\(\)](#).

18.123.4.2 m_collisionSystem

```
std::unique_ptr<CollisionSystem> gme::RTypeServer::m_collisionSystem [private]
```

Definition at line 76 of file [RTypeServer.hpp](#).

Referenced by [RTypeServer\(\)](#), and [updateSystems\(\)](#).

18.123.4.3 m_enemyAISystem

```
std::unique_ptr<EnemyAISystem> gme::RTypeServer::m_enemyAISystem [private]
```

Definition at line 77 of file [RTypeServer.hpp](#).

Referenced by [RTypeServer\(\)](#), and [updateSystems\(\)](#).

18.123.4.4 m_enemySpawnSystem

```
std::unique_ptr<EnemySpawnSystem> gme::RTypeServer::m_enemySpawnSystem [private]
```

Definition at line 78 of file [RTypeServer.hpp](#).

Referenced by [processServerStartEvent\(\)](#), [RTypeServer\(\)](#), [start\(\)](#), and [stop\(\)](#).

18.123.4.5 m_entityManager

```
std::unique_ptr<EntityManager> gme::RTypeServer::m_entityManager [private]
```

Definition at line 73 of file [RTypeServer.hpp](#).

Referenced by [broadcastWorldState\(\)](#), [handlePlayerShooting\(\)](#), [processPlayerInputEvent\(\)](#), [processServerStartEvent\(\)](#), [RTypeServer\(\)](#), [stop\(\)](#), [update\(\)](#), [updateEntities\(\)](#), and [updateSystems\(\)](#).

18.123.4.6 m_eventBus

[utl::EventBus](#)& gme::RTypeServer::m_eventBus [private]

Definition at line 69 of file [RTypeServer.hpp](#).

Referenced by [broadcastWorldState\(\)](#), [RTypeServer\(\)](#), and [update\(\)](#).

18.123.4.7 m_gameState

[State](#) gme::RTypeServer::m_gameState = [State::PLAYING](#) [private]

Definition at line 81 of file [RTypeServer.hpp](#).

Referenced by [getState\(\)](#), [start\(\)](#), and [stop\(\)](#).

18.123.4.8 m_lastBroadcastTime

float gme::RTypeServer::m_lastBroadcastTime = 0.0f [private]

Definition at line 88 of file [RTypeServer.hpp](#).

Referenced by [broadcastWorldState\(\)](#), and [update\(\)](#).

18.123.4.9 m_lastShotTime

std::unordered_map<std::uint32_t, float> gme::RTypeServer::m_lastShotTime [private]

Definition at line 86 of file [RTypeServer.hpp](#).

Referenced by [handlePlayerShooting\(\)](#), [processPlayerInputEvent\(\)](#), [processServerStartEvent\(\)](#), and [stop\(\)](#).

18.123.4.10 m_levelState

[LevelState](#) gme::RTypeServer::m_levelState = [LevelState::WAITING_FOR_PLAYERS](#) [private]

Definition at line 82 of file [RTypeServer.hpp](#).

Referenced by [processGameStartEvent\(\)](#), [processServerStartEvent\(\)](#), and [start\(\)](#).

18.123.4.11 m_playerShooting

std::unordered_map<std::uint32_t, bool> gme::RTypeServer::m_playerShooting [private]

Definition at line 85 of file [RTypeServer.hpp](#).

Referenced by [handlePlayerShooting\(\)](#), [processPlayerInputEvent\(\)](#), [processServerStartEvent\(\)](#), and [stop\(\)](#).

18.123.4.12 m_registry

[ecs::Registry](#) gme::RTypeServer::m_registry [private]

Definition at line 70 of file [RTypeServer.hpp](#).

Referenced by [handlePlayerShooting\(\)](#), [processPlayerInputEvent\(\)](#), [RTypeServer\(\)](#), [updateEntities\(\)](#), and [updateSystems\(\)](#).

18.123.4.13 m_waveManager

std::unique_ptr<[WaveManager](#)> gme::RTypeServer::m_waveManager [private]

Definition at line 79 of file [RTypeServer.hpp](#).

Referenced by [processGameStartEvent\(\)](#), [RTypeServer\(\)](#), [stop\(\)](#), and [updateSystems\(\)](#).

18.123.4.14 PROJECTILE_COOLDOWN

const float gme::RTypeServer::PROJECTILE_COOLDOWN = 0.3f [private]

Definition at line 90 of file [RTypeServer.hpp](#).

Referenced by [handlePlayerShooting\(\)](#).

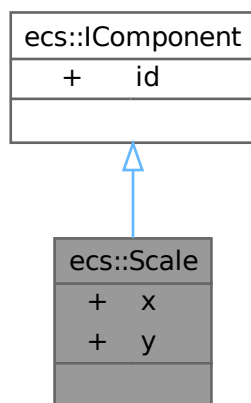
The documentation for this class was generated from the following files:

- [/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Server/include/RTypeServer/RTypeServer.hpp](#)
- [/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Server/src/rtypeServer.cpp](#)

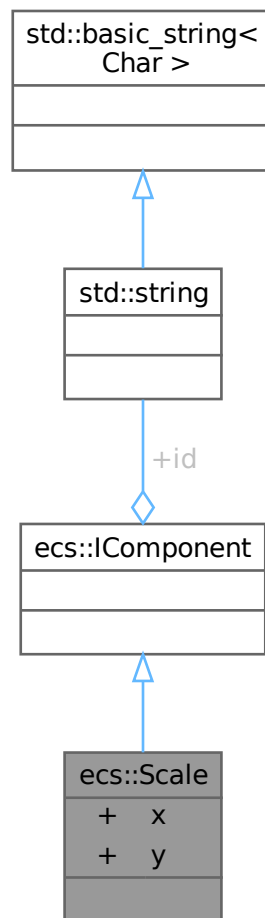
18.124 ecs::Scale Struct Reference

#include <Component.hpp>

Inheritance diagram for ecs::Scale:



Collaboration diagram for `ecs::Scale`:



Public Attributes

- float `x` {}
- float `y` {}

Public Attributes inherited from `ecs::IComponent`

- std::string `id`

18.124.1 Detailed Description

Definition at line 48 of file [Component.hpp](#).

18.124.2 Member Data Documentation

18.124.2.1 `x`

float `ecs::Scale::x` {}

Definition at line 50 of file [Component.hpp](#).

Referenced by [gme::ConfigMulti::ConfigMulti\(\)](#), [gme::ConfigSolo::ConfigSolo\(\)](#), [gme::Game::Game\(\)](#), [gme::GameMulti::GameMulti\(\)](#), [gme::GameSolo::GameSolo\(\)](#), [cli::Menu::Menu\(\)](#), and [cli::Settings::Settings\(\)](#).

18.124.2.2 y

float ecs::Scale::y {}

Definition at line 50 of file [Component.hpp](#).

The documentation for this struct was generated from the following file:

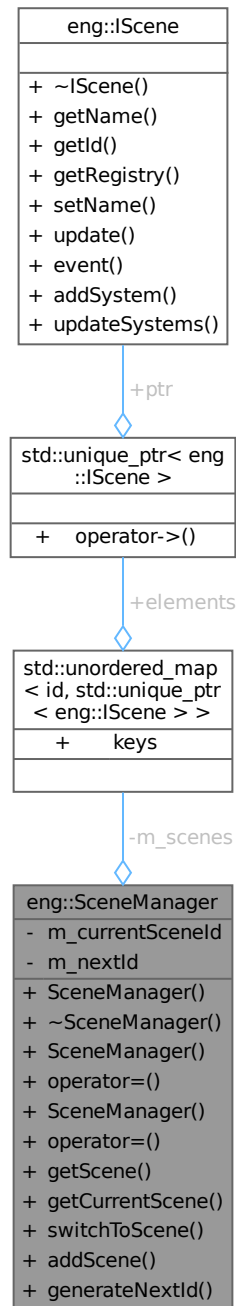
- [/home/masina/Projects/Epitech/rtype/modules/ECS/include/ECS/Component.hpp](#)

18.125 eng::SceneManager Class Reference

Class for managing scenes.

#include <SceneManager.hpp>

Collaboration diagram for eng::SceneManager:



Public Member Functions

- `SceneManager ()=default`
- `~SceneManager ()=default`
- `SceneManager (const SceneManager &)=delete`
- `SceneManager & operator= (const SceneManager &)=delete`
- `SceneManager (SceneManager &&)=delete`
- `SceneManager & operator= (SceneManager &&)=delete`

- `std::unique_ptr< IScene > & getScene (const id sceneId)`
- `std::unique_ptr< IScene > & getCurrentScene ()`
- `void switchToScene (const id sceneId)`
- `void addScene (std::unique_ptr< IScene > scene)`
- `id generateNextId ()`

Private Attributes

- `std::unordered_map< id, std::unique_ptr< IScene > > m\_scenes`
- `id m\_currentSceneId = 1`
- `id m\_nextId = 1`

18.125.1 Detailed Description

Class for managing scenes.

Definition at line 22 of file [SceneManager.hpp](#).

18.125.2 Constructor & Destructor Documentation

18.125.2.1 SceneManager() [1/3]

`eng::SceneManager::SceneManager ()` [default]

18.125.2.2 ~SceneManager()

`eng::SceneManager::~SceneManager ()` [default]

18.125.2.3 SceneManager() [2/3]

`eng::SceneManager::SceneManager (`
 `const SceneManager & )` [delete]

18.125.2.4 SceneManager() [3/3]

`eng::SceneManager::SceneManager (`
 `SceneManager && )` [delete]

18.125.3 Member Function Documentation

18.125.3.1 addScene()

`void eng::SceneManager::addScene (`
 `std::unique_ptr< IScene > scene)` [inline]

Definition at line 37 of file [SceneManager.hpp](#).

References [m_scenes](#).

18.125.3.2 generateNextId()

`id eng::SceneManager::generateNextId ()` [inline]

Definition at line 38 of file [SceneManager.hpp](#).

References [m_nextId](#).

18.125.3.3 getCurrentScene()

`std::unique_ptr< IScene > & eng::SceneManager::getCurrentScene ()` [inline]

Definition at line 35 of file [SceneManager.hpp](#).

References [m_currentSceneId](#), and [m_scenes](#).

18.125.3.4 `getScene()`

```
std::unique_ptr< IScene > & eng::SceneManager::getScene (
    const id sceneId) [inline]
```

Definition at line 34 of file [SceneManager.hpp](#).

References [m_scenes](#).

18.125.3.5 `operator=()` [1/2]

```
SceneManager & eng::SceneManager::operator= (
    const SceneManager & ) [delete]
```

18.125.3.6 `operator=()` [2/2]

```
SceneManager & eng::SceneManager::operator= (
    SceneManager && ) [delete]
```

18.125.3.7 `switchToScene()`

```
void eng::SceneManager::switchToScene (
    const id sceneId) [inline]
```

Definition at line 36 of file [SceneManager.hpp](#).

References [m_currentSceneId](#).

18.125.4 Member Data Documentation

18.125.4.1 `m_currentSceneId`

```
id eng::SceneManager::m_currentSceneId = 1 [private]
```

Definition at line 42 of file [SceneManager.hpp](#).

Referenced by [getCurrentScene\(\)](#), and [switchToScene\(\)](#).

18.125.4.2 `m_nextId`

```
id eng::SceneManager::m_nextId = 1 [private]
```

Definition at line 43 of file [SceneManager.hpp](#).

Referenced by [generateNextId\(\)](#).

18.125.4.3 `m_scenes`

```
std::unordered_map<id, std::unique_ptr<IScene> > eng::SceneManager::m_scenes [private]
```

Definition at line 41 of file [SceneManager.hpp](#).

Referenced by [addScene\(\)](#), [getCurrentScene\(\)](#), and [getScene\(\)](#).

The documentation for this class was generated from the following file:

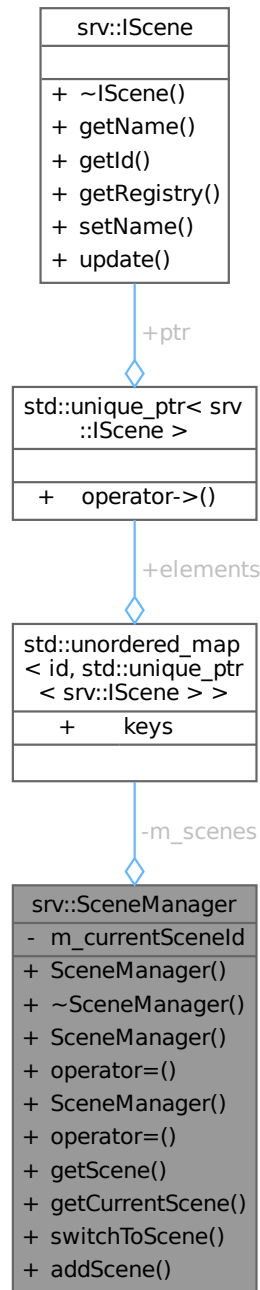
- [/home/masina/Projects/Epitech/rtype/modules/Engine/include/Engine/SceneManager.hpp](#)

18.126 `srv::SceneManager` Class Reference

Class for managing scenes.

```
#include <SceneManager.hpp>
```

Collaboration diagram for srv::SceneManager:



Public Member Functions

- [SceneManager](#) ()=default
- [~SceneManager](#) ()=default
- [SceneManager](#) (const [SceneManager](#) &)=delete
- [SceneManager](#) & operator= (const [SceneManager](#) &)=delete
- [SceneManager](#) ([SceneManager](#) &&)=delete
- [SceneManager](#) & operator= ([SceneManager](#) &&)=delete

- `std::unique_ptr< IScene > & getScene (const id sceneId)`
- `std::unique_ptr< IScene > & getCurrentScene ()`
- `void switchToScene (const id sceneId)`
- `void addScene (std::unique_ptr< IScene > scene)`

Private Attributes

- `std::unordered_map< id, std::unique_ptr< IScene > > m\_scenes`
- `id m\_currentSceneId = 1`

18.126.1 Detailed Description

Class for managing scenes.

Definition at line 22 of file [SceneManager.hpp](#).

18.126.2 Constructor & Destructor Documentation

18.126.2.1 `SceneManager()` [1/3]

`srv::SceneManager::SceneManager ()` [default]

18.126.2.2 `~SceneManager()`

`srv::SceneManager::~SceneManager ()` [default]

18.126.2.3 `SceneManager()` [2/3]

`srv::SceneManager::SceneManager (`
 `const SceneManager & )` [delete]

18.126.2.4 `SceneManager()` [3/3]

`srv::SceneManager::SceneManager (`
 `SceneManager && )` [delete]

18.126.3 Member Function Documentation

18.126.3.1 `addScene()`

`void srv::SceneManager::addScene (`
 `std::unique_ptr< IScene > scene)` [inline]

Definition at line 37 of file [SceneManager.hpp](#).

References [m_scenes](#).

18.126.3.2 `getCurrentScene()`

`std::unique_ptr< IScene > & srv::SceneManager::getCurrentScene ()` [inline]

Definition at line 35 of file [SceneManager.hpp](#).

References [m_currentSceneId](#), and [m_scenes](#).

18.126.3.3 `getScene()`

`std::unique_ptr< IScene > & srv::SceneManager::getScene (`
 `const id sceneId)` [inline]

Definition at line 34 of file [SceneManager.hpp](#).

References [m_scenes](#).

18.126.3.4 `operator=()` [1/2]

`SceneManager & srv::SceneManager::operator= (`
 `const SceneManager & )` [delete]

18.126.3.5 operator=() [2/2]

[SceneManager](#) & srv::SceneManager::operator= (
[SceneManager](#) &&) [delete]

18.126.3.6 switchToScene()

void srv::SceneManager::switchToScene (
const [id](#) sceneId) [inline]

Definition at line 36 of file [SceneManager.hpp](#).

References [m_currentSceneId](#).

18.126.4 Member Data Documentation

18.126.4.1 m_currentSceneId

[id](#) srv::SceneManager::m_currentSceneId = 1 [private]

Definition at line 57 of file [SceneManager.hpp](#).

Referenced by [getCurrentScene\(\)](#), and [switchToScene\(\)](#).

18.126.4.2 m_scenes

std::unordered_map<[id](#), std::unique_ptr<[IScene](#)> > srv::SceneManager::m_scenes [private]

Definition at line 56 of file [SceneManager.hpp](#).

Referenced by [addScene\(\)](#), [getCurrentScene\(\)](#), and [getScene\(\)](#).

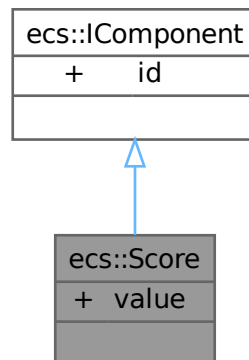
The documentation for this class was generated from the following file:

- /home/masina/Projects/Epitech/rtype/server/include/Server/[SceneManager.hpp](#)

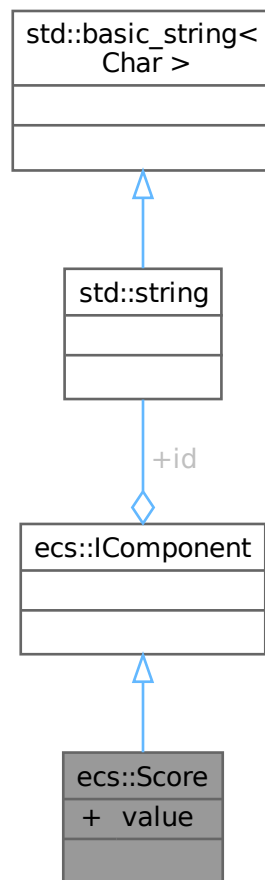
18.127 ecs::Score Struct Reference

#include <Component.hpp>

Inheritance diagram for `ecs::Score`:



Collaboration diagram for `ecs::Score`:



Public Attributes

- int [value](#)

Public Attributes inherited from [ecs::IComponent](#)

- `std::string` [id](#)

18.127.1 Detailed Description

Definition at line [168](#) of file [Component.hpp](#).

18.127.2 Member Data Documentation

18.127.2.1 `value`

`int ecs::Score::value`

Definition at line [170](#) of file [Component.hpp](#).

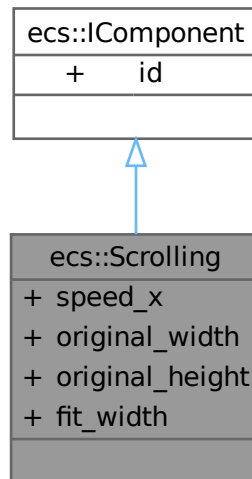
The documentation for this struct was generated from the following file:

- `/home/masina/Projects/Epitech/rtype/modules/ECS/include/ECS/Component.hpp`

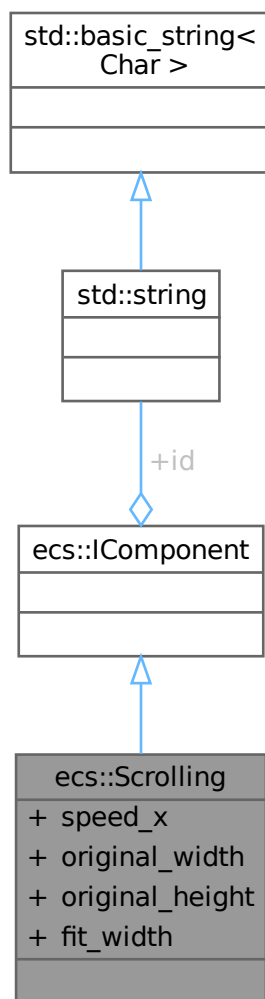
18.128 ecs::Scrolling Struct Reference

#include <Component.hpp>

Inheritance diagram for ecs::Scrolling:



Collaboration diagram for `ecs::Scrolling`:



Public Attributes

- float `speed_x`
- float `original_width`
- float `original_height`
- bool `fit_width`

Public Attributes inherited from `ecs::IComponent`

- `std::string id`

18.128.1 Detailed Description

Definition at line 160 of file `Component.hpp`.

18.128.2 Member Data Documentation

18.128.2.1 fit_width

bool ecs::Scrolling::fit_width

Definition at line 165 of file [Component.hpp](#).

18.128.2.2 original_height

float ecs::Scrolling::original_height

Definition at line 164 of file [Component.hpp](#).

18.128.2.3 original_width

float ecs::Scrolling::original_width

Definition at line 163 of file [Component.hpp](#).

18.128.2.4 speed_x

float ecs::Scrolling::speed_x

Definition at line 162 of file [Component.hpp](#).

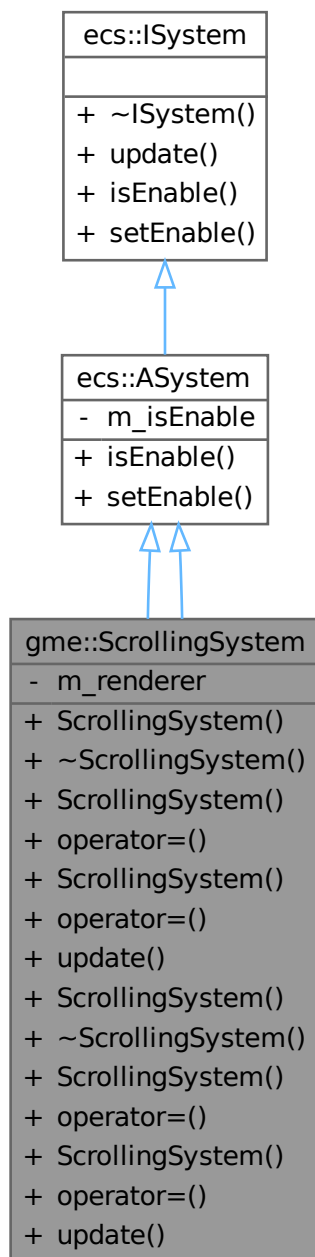
The documentation for this struct was generated from the following file:

- [/home/masina/Projects/Epitech/rtype/modules/ECS/include/ECS/Component.hpp](#)

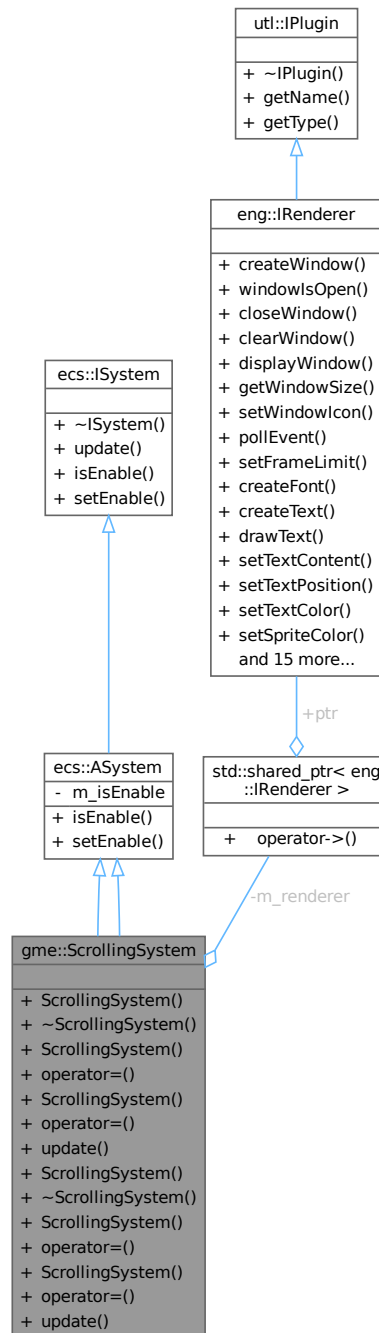
18.129 gme::ScrollingSystem Class Reference

```
#include <Scrolling.hpp>
```

Inheritance diagram for gme::ScrollingSystem:



Collaboration diagram for gme::ScrollingSystem:



Public Member Functions

- `ScrollingSystem` (const `std::shared_ptr< eng::IRenderer >` &renderer)
- `~ScrollingSystem` () override=default
- `ScrollingSystem` (const `ScrollingSystem` &)=delete
- `ScrollingSystem` & `operator=` (const `ScrollingSystem` &)=delete
- `ScrollingSystem` (`ScrollingSystem` &&)=delete
- `ScrollingSystem` & `operator=` (const `ScrollingSystem` &&)=delete

- void `update` (`ecs::Registry` ®istry, float dt) override
- `ScrollingSystem` (const std::shared_ptr< `eng::IRenderer` > &renderer)
- `~ScrollingSystem` () override=default
- `ScrollingSystem` (const `ScrollingSystem` &)=delete
- `ScrollingSystem` & `operator=` (const `ScrollingSystem` &)=delete
- `ScrollingSystem` (`ScrollingSystem` &&)=delete
- `ScrollingSystem` & `operator=` (`ScrollingSystem` &&)=delete
- void `update` (`ecs::Registry` ®istry, float dt) override

Public Member Functions inherited from `ecs::ASystem`

- bool `isEnabled` () override
- void `setEnabled` (const bool enable) override

Public Member Functions inherited from `ecs::ISystem`

- virtual `~ISystem` ()=default

Private Attributes

- const std::shared_ptr< `eng::IRenderer` > & `m_renderer`

18.129.1 Detailed Description

Definition at line 18 of file `Scrolling.hpp`.

18.129.2 Constructor & Destructor Documentation

18.129.2.1 `ScrollingSystem()` [1/6]

```
gme::ScrollingSystem::ScrollingSystem (
    const std::shared_ptr< eng::IRenderer > & renderer)  [inline], [explicit]
```

Definition at line 14 of file `Scrolling.hpp`.

18.129.2.2 `~ScrollingSystem()` [1/2]

```
gme::ScrollingSystem::~~ScrollingSystem ()  [override], [default]
```

18.129.2.3 `ScrollingSystem()` [2/6]

```
gme::ScrollingSystem::ScrollingSystem (
    const ScrollingSystem & )  [delete]
```

18.129.2.4 `ScrollingSystem()` [3/6]

```
gme::ScrollingSystem::ScrollingSystem (
    ScrollingSystem && )  [delete]
```

18.129.2.5 `ScrollingSystem()` [4/6]

```
gme::ScrollingSystem::ScrollingSystem (
    const std::shared_ptr< eng::IRenderer > & renderer)  [inline], [explicit]
```

Definition at line 21 of file `Scrolling.hpp`.

18.129.2.6 `~ScrollingSystem()` [2/2]

```
gme::ScrollingSystem::~~ScrollingSystem ()  [override], [default]
```

18.129.2.7 `ScrollingSystem()` [5/6]

```
gme::ScrollingSystem::ScrollingSystem (
    const ScrollingSystem & )  [delete]
```


18.129.2.8 ScrollingSystem() [6/6]

`gme::ScrollingSystem::ScrollingSystem (`
 [ScrollingSystem](#) &&) [delete]

18.129.3 Member Function Documentation

18.129.3.1 operator=() [1/4]

[ScrollingSystem](#) & gme::ScrollingSystem::operator= (

const [ScrollingSystem](#) &&) [delete]

18.129.3.2 operator=() [2/4]

[ScrollingSystem](#) & gme::ScrollingSystem::operator= (

const [ScrollingSystem](#) &) [delete]

18.129.3.3 operator=() [3/4]

[ScrollingSystem](#) & gme::ScrollingSystem::operator= (

const [ScrollingSystem](#) &) [delete]

18.129.3.4 operator=() [4/4]

[ScrollingSystem](#) & gme::ScrollingSystem::operator= (

[ScrollingSystem](#) &&) [delete]

18.129.3.5 update() [1/2]

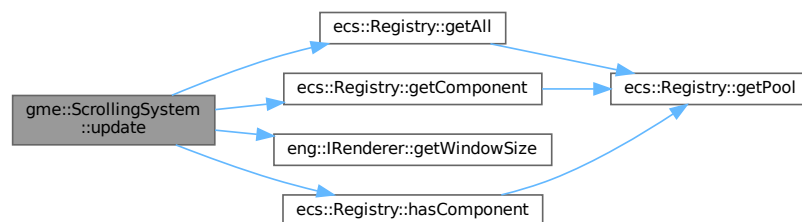
`void gme::ScrollingSystem::update (`
 [ecs::Registry](#) & registry,
 float dt) [inline], [override], [virtual]

Implements [ecs::ISystem](#).

Definition at line 22 of file [Scrolling.hpp](#).

References [utl::GameConfig::Stage::CEILING_OFFSET_Y](#), [utl::GameConfig::Stage::FLOOR_OFFSET_Y](#), [ecs::Registry::getAll\(\)](#), [ecs::Registry::getComponent\(\)](#), [eng::IRenderer::getWindowSize\(\)](#), [ecs::Registry::hasComponent\(\)](#), [m_renderer](#), [ecs::Transform::x](#), and [ecs::Transform::y](#).

Here is the call graph for this function:



18.129.3.6 update() [2/2]

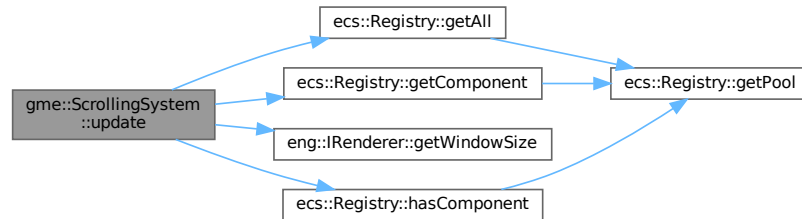
`void gme::ScrollingSystem::update (`
 [ecs::Registry](#) & registry,
 float dt) [inline], [override], [virtual]

Implements [ecs::ISystem](#).

Definition at line 29 of file [Scrolling.hpp](#).

References [utl::GameConfig::Stage::CEILING_OFFSET_Y](#), [utl::GameConfig::Stage::FLOOR_OFFSET_Y](#), [ecs::Registry::getAll\(\)](#), [ecs::Registry::getComponent\(\)](#), [eng::IRenderer::getWindowSize\(\)](#), [ecs::Registry::hasComponent\(\)](#), [m_renderer](#), [ecs::Transform::x](#), and [ecs::Transform::y](#).

Here is the call graph for this function:



18.129.4 Member Data Documentation

18.129.4.1 m_renderer

`const std::shared_ptr< eng::IRenderer > & gme::ScrollingSystem::m_renderer` [private]

Definition at line 68 of file [Scrolling.hpp](#).

Referenced by [update\(\)](#).

The documentation for this class was generated from the following files:

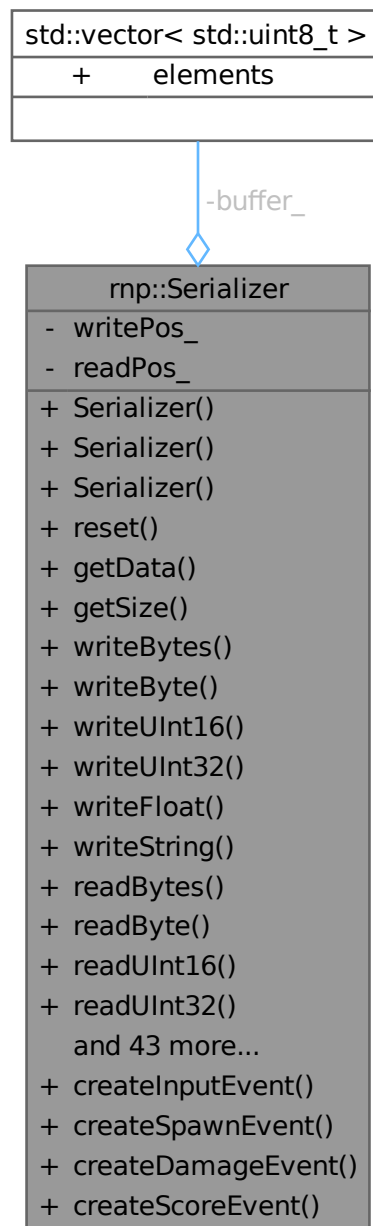
- `/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/include/RTypeClientMulti/Systems/Scrolling.hpp`
- `/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Solo/include/RTypeClientSolo/Systems/Scrolling.hpp`

18.130 rnp::Serializer Class Reference

Binary serializer for RNP protocol packets.

`#include <Serializer.hpp>`

Collaboration diagram for rnp::Serializer:



Public Member Functions

- [Serializer](#) ()
Constructor.
- [Serializer](#) (std::size_t capacity)
Constructor with initial capacity.
- [Serializer](#) (const std::vector< std::uint8_t > &data)
Constructor from existing data.
- void [reset](#) ()

- Reset the serializer for reuse.
- `const std::vector< std::uint8_t > & getData () const`
Get the serialized data.
- `std::size_t getSize () const`
Get the current size of serialized data.
- `void writeBytes (const void *data, std::size_t size)`
Write raw bytes.
- `void writeByte (std::uint8_t value)`
Write a single byte.
- `void writeUInt16 (std::uint16_t value)`
Write a 16-bit integer (network byte order)
- `void writeUInt32 (std::uint32_t value)`
Write a 32-bit integer (network byte order)
- `void writeFloat (float value)`
Write a float (network byte order)
- `void writeString (const std::string &str, std::size_t maxLength)`
Write a string with length prefix.
- `void readBytes (void *data, std::size_t size)`
Read raw bytes.
- `std::uint8_t readByte ()`
Read a single byte.
- `std::uint16_t readUInt16 ()`
Read a 16-bit integer (network byte order)
- `std::uint32_t readUInt32 ()`
Read a 32-bit integer (network byte order)
- `float readFloat ()`
Read a float (network byte order)
- `std::string readString (std::size_t maxLength)`
Read a string with length prefix.
- `void serializeHeader (const PacketHeader &header)`
Serialize packet header.
- `PacketHeader deserializeHeader ()`
Deserialize packet header.
- `void serializeConnect (const PacketConnect &packet)`
Serialize CONNECT packet.
- `PacketConnect deserializeConnect ()`
Deserialize CONNECT packet.
- `void serializeConnectAccept (const PacketConnectAccept &packet)`
Serialize CONNECT_ACCEPT packet.
- `PacketConnectAccept deserializeConnectAccept ()`
Deserialize CONNECT_ACCEPT packet.
- `void serializeDisconnect (const PacketDisconnect &packet)`
Serialize DISCONNECT packet.
- `PacketDisconnect deserializeDisconnect ()`
Deserialize DISCONNECT packet.
- `void serializeEntityState (const EntityState &entity)`
Serialize EntityState.
- `EntityState deserializeEntityState ()`
Deserialize EntityState.
- `void serializeWorldState (const PacketWorldState &packet)`
Serialize WORLD_STATE packet.

- [PacketWorldState deserializeWorldState \(\)](#)
Deserialize WORLD_STATE packet.
- void [serializePingPong](#) (const [PacketPingPong](#) &packet)
Serialize PING/PONG packet.
- [PacketPingPong deserializePingPong \(\)](#)
Deserialize PING/PONG packet.
- void [serializeError](#) (const [PacketError](#) &packet)
Serialize ERROR packet.
- [PacketError deserializeError \(\)](#)
Deserialize ERROR packet.
- void [serializeEventRecord](#) (const [EventRecord](#) &event)
Serialize [EventRecord](#) for ENTITY_EVENT packets.
- [EventRecord deserializeEventRecord \(\)](#)
Deserialize [EventRecord](#) from ENTITY_EVENT packets.
- void [serializeEntityEvents](#) (const std::vector< [EventRecord](#) > &events)
Serialize multiple EventRecords for ENTITY_EVENT packet.
- std::vector< [EventRecord](#) > [deserializeEntityEvents](#) (std::size_t payloadSize)
Deserialize multiple EventRecords from ENTITY_EVENT packet.
- void [serializeInputData](#) (std::uint8_t keys, float mouseX, float mouseY, std::uint8_t mouseButtons, std::uint32_t timestamp)
Serialize input data for INPUT events.
- void [deserializeInputData](#) (std::uint8_t &keys, float &mouseX, float &mouseY, std::uint8_t &mouseButtons, std::uint32_t ×tamp)
Deserialize input data from INPUT events.
- [EventRecord createDespawnEvent](#) (std::uint32_t entityId)
Create an [EventRecord](#) for despawn event.
- void [serializeLobbyInfo](#) (const [LobbyInfo](#) &lobbyInfo)
Serialize [LobbyInfo](#) structure.
- [LobbyInfo deserializeLobbyInfo \(\)](#)
Deserialize [LobbyInfo](#) structure.
- void [serializeLobbyListResponse](#) (const [PacketLobbyListResponse](#) &packet)
Serialize LOBBY_LIST_RESPONSE packet.
- [PacketLobbyListResponse deserializeLobbyListResponse \(\)](#)
Deserialize LOBBY_LIST_RESPONSE packet.
- void [serializeLobbyCreate](#) (const [PacketLobbyCreate](#) &packet)
Serialize LOBBY_CREATE packet.
- [PacketLobbyCreate deserializeLobbyCreate \(\)](#)
Deserialize LOBBY_CREATE packet.
- void [serializeLobbyCreateResponse](#) (const [PacketLobbyCreateResponse](#) &packet)
Serialize LOBBY_CREATE_RESPONSE packet.
- [PacketLobbyCreateResponse deserializeLobbyCreateResponse \(\)](#)
Deserialize LOBBY_CREATE_RESPONSE packet.
- void [serializeLobbyJoin](#) (const [PacketLobbyJoin](#) &packet)
Serialize LOBBY_JOIN packet.
- [PacketLobbyJoin deserializeLobbyJoin \(\)](#)
Deserialize LOBBY_JOIN packet.
- void [serializeLobbyJoinResponse](#) (const [PacketLobbyJoinResponse](#) &packet)
Serialize LOBBY_JOIN_RESPONSE packet.
- [PacketLobbyJoinResponse deserializeLobbyJoinResponse \(\)](#)
Deserialize LOBBY_JOIN_RESPONSE packet.
- void [serializeLobbyUpdate](#) (const [PacketLobbyUpdate](#) &packet)

- Serialize LOBBY_UPDATE packet.
- [PacketLobbyUpdate deserializeLobbyUpdate \(\)](#)
Deserialize LOBBY_UPDATE packet.
- void [serializeGameStart](#) (const [PacketGameStart](#) &packet)
Serialize GAME_START packet.
- [PacketGameStart deserializeGameStart \(\)](#)
Deserialize GAME_START packet.
- void [serializeStartGameRequest](#) (const [PacketStartGameRequest](#) &packet)
Serialize START_GAME_REQUEST packet.
- [PacketStartGameRequest deserializeStartGameRequest \(\)](#)
Deserialize START_GAME_REQUEST packet.

Static Public Member Functions

- static [EventRecord createInputEvent](#) (std::uint32_t entityId, std::uint8_t keys, float mouseX, float mouseY, std::uint8_t mouseButtons, std::uint32_t timestamp)
Create an [EventRecord](#) for input data.
- static [EventRecord createSpawnEvent](#) (std::uint32_t entityId, [EntityType](#) entityType, float x, float y)
Create an [EventRecord](#) for spawn event.
- static [EventRecord createDamageEvent](#) (std::uint32_t entityId, std::uint32_t damage, std::uint32_t sourceId=0)
Create an [EventRecord](#) for damage event.
- static [EventRecord createScoreEvent](#) (std::uint32_t entityId, std::uint32_t score)
Create an [EventRecord](#) for score event.

Private Attributes

- std::vector< std::uint8_t > [buffer_](#)
- std::size_t [writePos_](#)
- std::size_t [readPos_](#)

18.130.1 Detailed Description

Binary serializer for RNP protocol packets.
Definition at line 64 of file [Serializer.hpp](#).

18.130.2 Constructor & Destructor Documentation

18.130.2.1 Serializer() [1/3]

`rnp::Serializer::Serializer () [inline]`

Constructor.

Definition at line 75 of file [Serializer.hpp](#).

References [buffer_](#), and [rnp::MAX_PAYLOAD](#).

18.130.2.2 Serializer() [2/3]

`rnp::Serializer::Serializer (std::size_t capacity) [inline], [explicit]`

Constructor with initial capacity.

Parameters

capacity	Initial buffer capacity
----------	-------------------------

Definition at line 86 of file [Serializer.hpp](#).

References [buffer_](#).

18.130.2.3 Serializer() [3/3]

rnp::Serializer::Serializer (
 const std::vector< std::uint8_t > & data) [inline], [explicit]

Constructor from existing data.

Parameters

data	Existing data to deserialize
------	------------------------------

Definition at line 92 of file [Serializer.hpp](#).

18.130.3 Member Function Documentation

18.130.3.1 createDamageEvent()

static [EventRecord](#) rnp::Serializer::createDamageEvent (
 std::uint32_t entityId,
 std::uint32_t damage,
 std::uint32_t sourceId = 0) [inline], [static]

Create an [EventRecord](#) for damage event.

Parameters

entityId	Entity taking damage
damage	Amount of damage
sourceId	Source entity causing damage

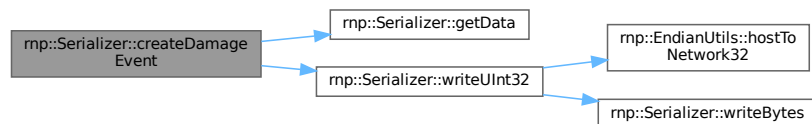
Returns

[EventRecord](#) for damage

Definition at line 651 of file [Serializer.hpp](#).

References [rnp::DAMAGE](#), [getData\(\)](#), [rnp::EventRecord::type](#), and [writeUInt32\(\)](#).

Here is the call graph for this function:



18.130.3.2 createDespawnEvent()

[EventRecord](#) rnp::Serializer::createDespawnEvent (
 std::uint32_t entityId) [inline]

Create an [EventRecord](#) for despawn event.

Parameters

entityId	Entity to despawn
----------	-------------------

Returns

[EventRecord](#) for despawn

Definition at line 690 of file [Serializer.hpp](#).

References [rnp::DESPAWN](#), and [rnp::EventRecord::type](#).

18.130.3.3 createInputEvent()

```
static EventRecord rnp::Serializer::createInputEvent (
    std::uint32_t entityId,
    std::uint8_t keys,
    float mouseX,
    float mouseY,
    std::uint8_t mouseButtons,
    std::uint32_t timestamp) [inline], [static]
```

Create an [EventRecord](#) for input data.

Parameters

entityId	Player entity ID
keys	Keyboard input
mouseX	Mouse X
mouseY	Mouse Y
mouseButtons	Mouse buttons
timestamp	Timestamp

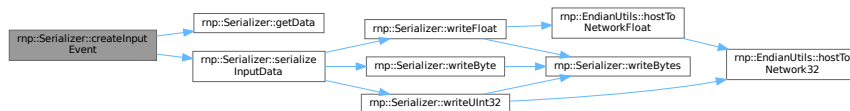
Returns

[EventRecord](#) ready to send

Definition at line 607 of file [Serializer.hpp](#).

References [getData\(\)](#), [rnp::INPUT](#), [serializeInputData\(\)](#), and [rnp::EventRecord::type](#).

Here is the call graph for this function:



18.130.3.4 createScoreEvent()

```
static EventRecord rnp::Serializer::createScoreEvent (
    std::uint32_t entityId,
    std::uint32_t score) [inline], [static]
```

Create an [EventRecord](#) for score event.

Parameters

entityId	Entity gaining score
score	Score amount

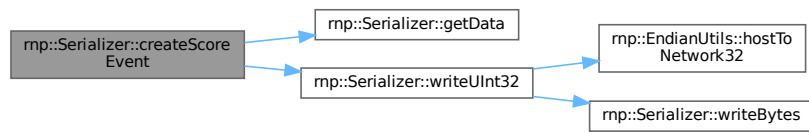
Returns

[EventRecord](#) for score

Definition at line 672 of file [Serializer.hpp](#).

References [getData\(\)](#), [rnp::SCORE](#), [rnp::EventRecord::type](#), and [writeUInt32\(\)](#).

Here is the call graph for this function:



18.130.3.5 createSpawnEvent()

```
static EventRecord rnp::Serializer::createSpawnEvent (
    std::uint32_t entityId,
    EntityType entityType,
    float x,
    float y) [inline], [static]
```

Create an [EventRecord](#) for spawn event.

Parameters

entityId	Entity ID to spawn
entityType	Type of entity
x	X position
y	Y position

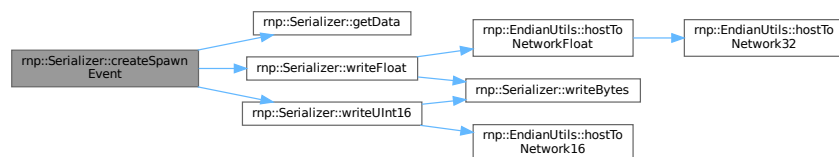
Returns

[EventRecord](#) for spawn

Definition at line 629 of file [Serializer.hpp](#).

References [getData\(\)](#), [rnp::SPAWN](#), [rnp::EventRecord::type](#), [writeFloat\(\)](#), and [writeUInt16\(\)](#).

Here is the call graph for this function:



18.130.3.6 deserializeConnect()

```
PacketConnect rnp::Serializer::deserializeConnect () [inline]
```

Deserialize CONNECT packet.

Returns

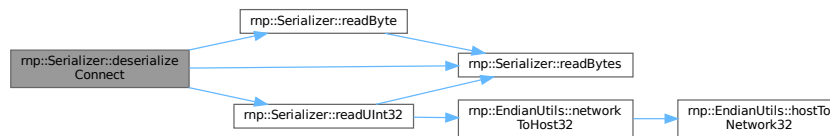
Deserialized CONNECT packet

Definition at line 315 of file [Serializer.hpp](#).

References [rnp::PacketConnect::clientCaps](#), [rnp::PacketConnect::nameLen](#), [rnp::PacketConnect::playerName](#), [readByte\(\)](#), [readBytes\(\)](#), and [readUInt32\(\)](#).

Referenced by [rnp::HandlerPacket::processPacket\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.130.3.7 deserializeConnectAccept()

[PacketConnectAccept](#) [rnp::Serializer::deserializeConnectAccept\(\)](#) [inline]

Deserialize CONNECT_ACCEPT packet.

Returns

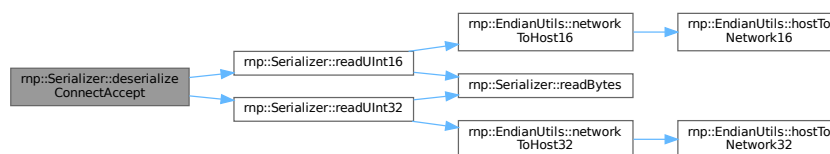
Deserialized CONNECT_ACCEPT packet

Definition at line 340 of file [Serializer.hpp](#).

References [rnp::PacketConnectAccept::mtuPayloadBytes](#), [readUInt16\(\)](#), [readUInt32\(\)](#), [rnp::PacketConnectAccept::serverC](#), [rnp::PacketConnectAccept::sessionId](#), and [rnp::PacketConnectAccept::tickRateHz](#).

Referenced by [rnp::HandlerPacket::processPacket\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.130.3.8 deserializeDisconnect()

[PacketDisconnect](#) `rnp::Serializer::deserializeDisconnect()` [inline]

Deserialize DISCONNECT packet.

Returns

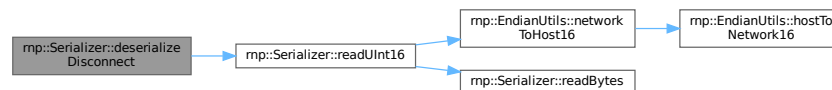
Deserialized DISCONNECT packet

Definition at line 360 of file [Serializer.hpp](#).

References [readUInt16\(\)](#), and [rnp::PacketDisconnect::reasonCode](#).

Referenced by [rnp::HandlerPacket::processPacket\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.130.3.9 deserializeEntityEvents()

`std::vector< EventRecord >` `rnp::Serializer::deserializeEntityEvents(`
 `std::size_t payloadSize)` [inline]

Deserialize multiple EventRecords from ENTITY_EVENT packet.

Parameters

payloadSize	Total size of the payload to deserialize
-------------	--

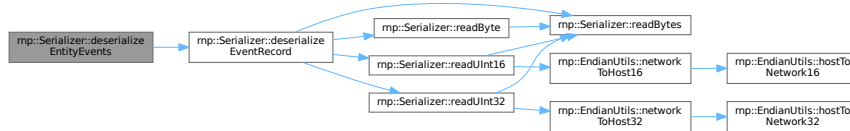
Returns

Vector of deserialized event records

Definition at line 545 of file [Serializer.hpp](#).

References [buffer_](#), [deserializeEventRecord\(\)](#), and [readPos_](#).

Here is the call graph for this function:



18.130.3.10 deserializeEntityState()

[EntityState](#) [mp::Serializer::deserializeEntityState\(\)](#) [inline]

Deserialize [EntityState](#).

Returns

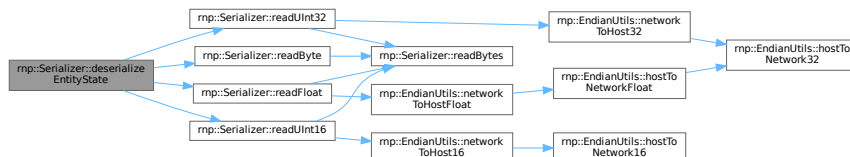
Deserialized entity state

Definition at line 388 of file [Serializer.hpp](#).

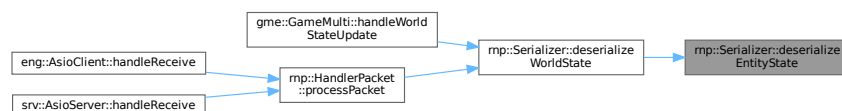
References [mp::EntityState::healthPercent](#), [mp::EntityState::id](#), [readByte\(\)](#), [readFloat\(\)](#), [readUInt16\(\)](#), [readUInt32\(\)](#), [mp::EntityState::stateFlags](#), [mp::EntityState::subtype](#), [mp::EntityState::type](#), [mp::EntityState::vx](#), [mp::EntityState::vy](#), [mp::EntityState::x](#), and [mp::EntityState::y](#).

Referenced by [deserializeWorldState\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.130.3.11 deserializeError()

[PacketError](#) [mp::Serializer::deserializeError\(\)](#) [inline]

Deserialize ERROR packet.

Returns

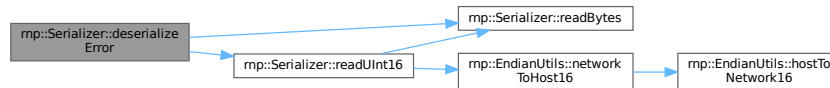
Deserialized ERROR packet

Definition at line 477 of file [Serializer.hpp](#).

References [rnp::PacketError::description](#), [rnp::PacketError::errorCode](#), [rnp::PacketError::msgLen](#), [readBytes\(\)](#), and [readUInt16\(\)](#).

Referenced by [rnp::HandlerPacket::processPacket\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.130.3.12 deserializeEventRecord()

[EventRecord](#) [rnp::Serializer::deserializeEventRecord\(\)](#) [inline]

Deserialize [EventRecord](#) from ENTITY_EVENT packets.

Returns

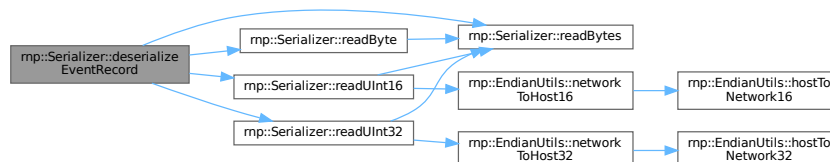
Deserialized event record

Definition at line 512 of file [Serializer.hpp](#).

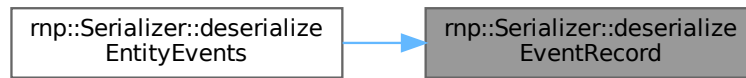
References [rnp::EventRecord::data](#), [readByte\(\)](#), [readBytes\(\)](#), [readUInt16\(\)](#), [readUInt32\(\)](#), and [rnp::EventRecord::type](#).

Referenced by [deserializeEntityEvents\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.130.3.13 deserializeGameStart()

[PacketGameStart](#) `rnp::Serializer::deserializeGameStart ()` [inline]

Deserialize GAME_START packet.

Returns

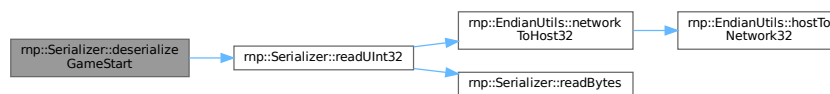
Deserialized packet

Definition at line 901 of file [Serializer.hpp](#).

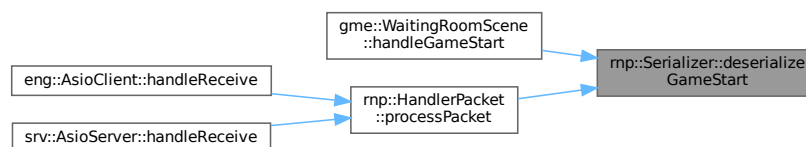
References [rnp::PacketGameStart::lobbyId](#), and [readUInt32\(\)](#).

Referenced by [gme::WaitingRoomScene::handleGameStart\(\)](#), and [rnp::HandlerPacket::processPacket\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.130.3.14 deserializeHeader()

[PacketHeader](#) `rnp::Serializer::deserializeHeader ()` [inline]

Deserialize packet header.

Returns

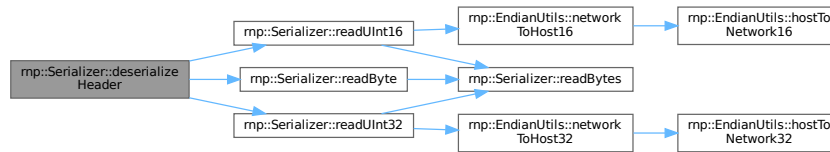
Deserialized header

Definition at line 291 of file [Serializer.hpp](#).

References [rnp::PacketHeader::length](#), [readByte\(\)](#), [readUInt16\(\)](#), [readUInt32\(\)](#), [rnp::PacketHeader::sessionId](#), and [rnp::PacketHeader::type](#).

Referenced by [eng::AsioClient::handleReceive\(\)](#), [srv::AsioServer::handleReceive\(\)](#), and [rnp::HandlerPacket::processPacket\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.130.3.15 deserializeInputData()

```

void rnp::Serializer::deserializeInputData (
    std::uint8_t & keys,
    float & mouseX,
    float & mouseY,
    std::uint8_t & mouseButtons,
    std::uint32_t & timestamp) [inline]
  
```

Deserialize input data from INPUT events.

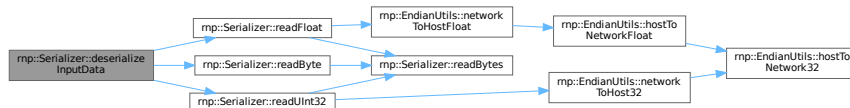
Parameters

keys	Output keyboard input bitmask
mouseX	Output mouse X position
mouseY	Output mouse Y position
mouseButtons	Output mouse button bitmask
timestamp	Output input timestamp

Definition at line 587 of file [Serializer.hpp](#).

References [readByte\(\)](#), [readFloat\(\)](#), and [readUInt32\(\)](#).

Here is the call graph for this function:



18.130.3.16 deserializeLobbyCreate()

```

PacketLobbyCreate rnp::Serializer::deserializeLobbyCreate () [inline]
  
```

Deserialize LOBBY_CREATE packet.

Returns

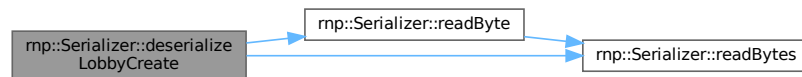
Deserialized packet

Definition at line 791 of file [Serializer.hpp](#).

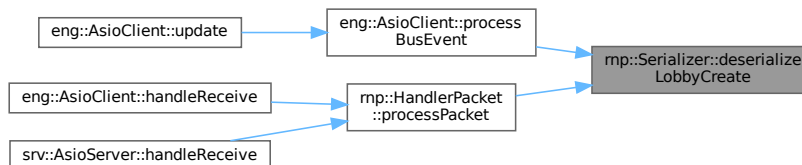
References [rnp::PacketLobbyCreate::gameMode](#), [rnp::PacketLobbyCreate::lobbyName](#), [rnp::PacketLobbyCreate::maxPlay](#), [rnp::PacketLobbyCreate::nameLen](#), [readByte\(\)](#), and [readBytes\(\)](#).

Referenced by [eng::AsioClient::processBusEvent\(\)](#), and [rnp::HandlerPacket::processPacket\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.130.3.17 deserializeLobbyCreateResponse()

[PacketLobbyCreateResponse](#) [rnp::Serializer::deserializeLobbyCreateResponse\(\)](#) [inline]

Deserialize LOBBY_CREATE_RESPONSE packet.

Returns

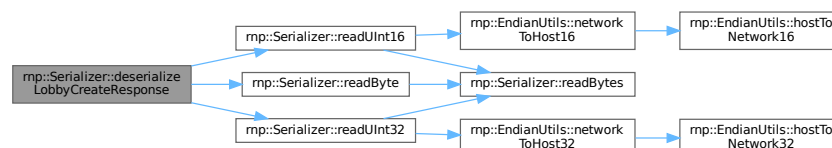
Deserialized packet

Definition at line 816 of file [Serializer.hpp](#).

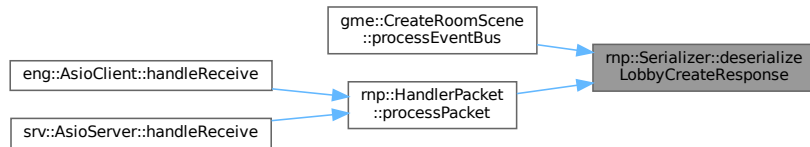
References [rnp::PacketLobbyCreateResponse::errorCode](#), [rnp::PacketLobbyCreateResponse::lobbyId](#), [readByte\(\)](#), [readUInt16\(\)](#), [readUInt32\(\)](#), and [rnp::PacketLobbyCreateResponse::success](#).

Referenced by [gme::CreateRoomScene::processEventBus\(\)](#), and [rnp::HandlerPacket::processPacket\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.130.3.18 deserializeLobbyInfo()

[LobbyInfo](#) `rnp::Serializer::deserializeLobbyInfo()` [inline]

Deserialize [LobbyInfo](#) structure.

Returns

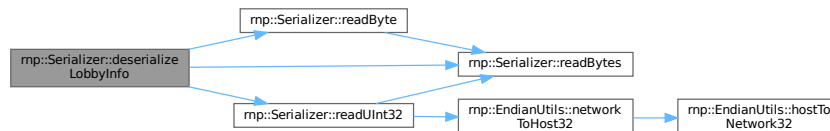
Deserialized lobby info

Definition at line 724 of file [Serializer.hpp](#).

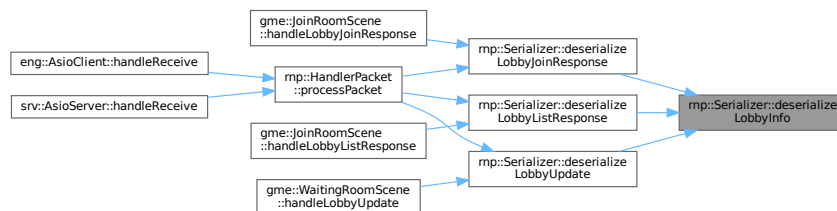
References [rnp::LobbyInfo::currentPlayers](#), [rnp::LobbyInfo::gameMode](#), [rnp::LobbyInfo::hostSessionId](#), [rnp::LobbyInfo::lobbyId](#), [rnp::LobbyInfo::lobbyName](#), [rnp::LobbyInfo::maxPlayers](#), [rnp::LobbyInfo::playerNames](#), [readByte\(\)](#), [readBytes\(\)](#), [readUInt32\(\)](#), and [rnp::LobbyInfo::status](#).

Referenced by [deserializeLobbyJoinResponse\(\)](#), [deserializeLobbyListResponse\(\)](#), and [deserializeLobbyUpdate\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.130.3.19 deserializeLobbyJoin()

[PacketLobbyJoin](#) `rnp::Serializer::deserializeLobbyJoin()` [inline]

Deserialize LOBBY_JOIN packet.

Returns

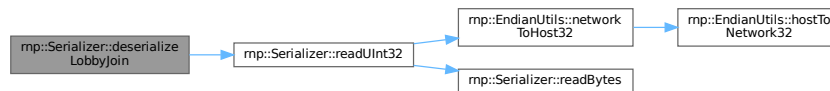
Deserialized packet

Definition at line 835 of file [Serializer.hpp](#).

References [rnp::PacketLobbyJoin::lobbyId](#), and [readUInt32\(\)](#).

Referenced by [rnp::HandlerPacket::processPacket\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.130.3.20 deserializeLobbyJoinResponse()

[PacketLobbyJoinResponse](#) [rnp::Serializer::deserializeLobbyJoinResponse\(\)](#) [inline]

Deserialize LOBBY_JOIN_RESPONSE packet.

Returns

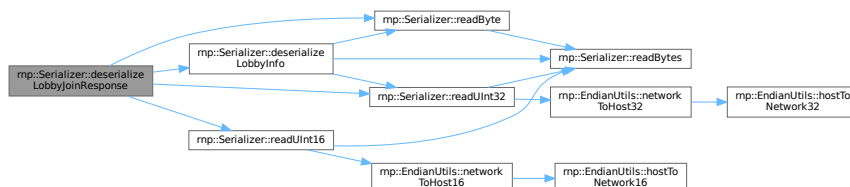
Deserialized packet

Definition at line 861 of file [Serializer.hpp](#).

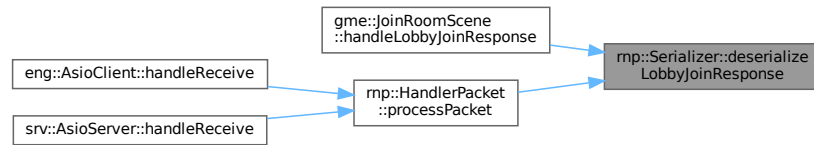
References [deserializeLobbyInfo\(\)](#), [rnp::PacketLobbyJoinResponse::errorCode](#), [rnp::PacketLobbyJoinResponse::lobbyId](#), [rnp::PacketLobbyJoinResponse::lobbyInfo](#), [readByte\(\)](#), [readUInt16\(\)](#), [readUInt32\(\)](#), and [rnp::PacketLobbyJoinResponse::](#)

Referenced by [gme::JoinRoomScene::handleLobbyJoinResponse\(\)](#), and [rnp::HandlerPacket::processPacket\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.130.3.21 deserializeLobbyListResponse()

[PacketLobbyListResponse](#) `rnp::Serializer::deserializeLobbyListResponse ()` [inline]

Deserialize LOBBY_LIST_RESPONSE packet.

Returns

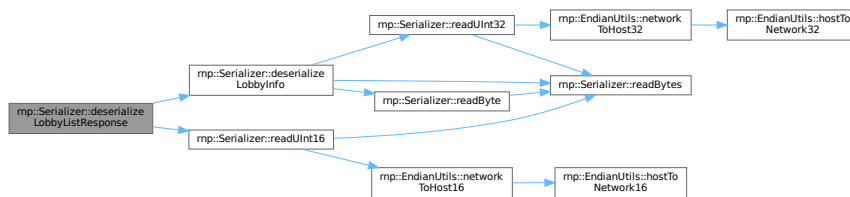
Deserialized packet

Definition at line 761 of file [Serializer.hpp](#).

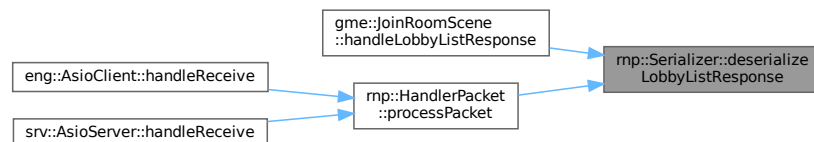
References [deserializeLobbyInfo\(\)](#), [rnp::PacketLobbyListResponse::lobbies](#), [rnp::PacketLobbyListResponse::lobbyCount](#), and [readUInt16\(\)](#).

Referenced by [gme::JoinRoomScene::handleLobbyListResponse\(\)](#), and [rnp::HandlerPacket::processPacket\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.130.3.22 deserializeLobbyUpdate()

[PacketLobbyUpdate](#) `rnp::Serializer::deserializeLobbyUpdate ()` [inline]

Deserialize LOBBY_UPDATE packet.

Returns

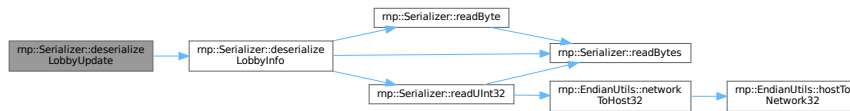
Deserialized packet

Definition at line 884 of file [Serializer.hpp](#).

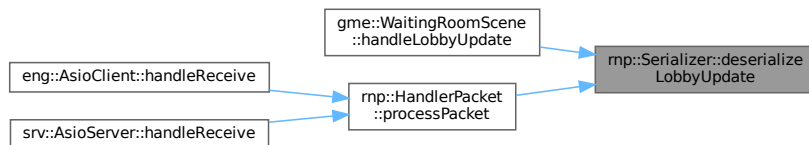
References [deserializeLobbyInfo\(\)](#), and [rnp::PacketLobbyUpdate::lobbyInfo](#).

Referenced by [gme::WaitingRoomScene::handleLobbyUpdate\(\)](#), and [rnp::HandlerPacket::processPacket\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.130.3.23 deserializePingPong()

[PacketPingPong](#) [rnp::Serializer::deserializePingPong\(\)](#) [inline]

Deserialize PING/PONG packet.

Returns

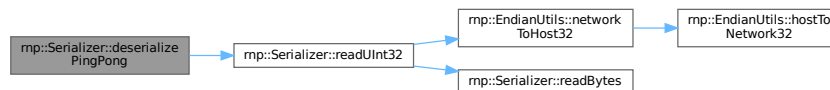
Deserialized PING/PONG packet

Definition at line 451 of file [Serializer.hpp](#).

References [rnp::PacketPingPong::nonce](#), [readUInt32\(\)](#), and [rnp::PacketPingPong::sendTimeMs](#).

Referenced by [rnp::HandlerPacket::processPacket\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.130.3.24 deserializeStartGameRequest()

[PacketStartGameRequest](#) rnp::Serializer::deserializeStartGameRequest () [inline]

Deserialize START_GAME_REQUEST packet.

Returns

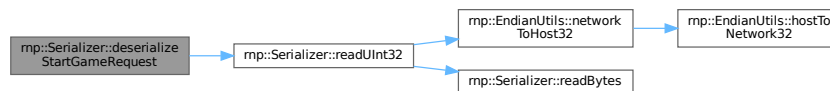
Deserialized packet

Definition at line 918 of file [Serializer.hpp](#).

References [rnp::PacketStartGameRequest::lobbyId](#), and [readUInt32\(\)](#).

Referenced by [rnp::HandlerPacket::processPacket\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.130.3.25 deserializeWorldState()

[PacketWorldState](#) rnp::Serializer::deserializeWorldState () [inline]

Deserialize WORLD_STATE packet.

Returns

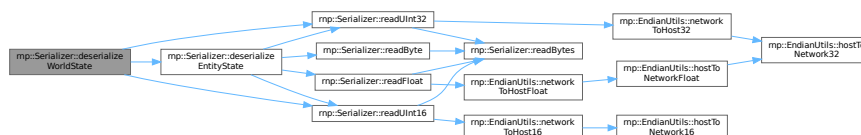
Deserialized WORLD_STATE packet

Definition at line 422 of file [Serializer.hpp](#).

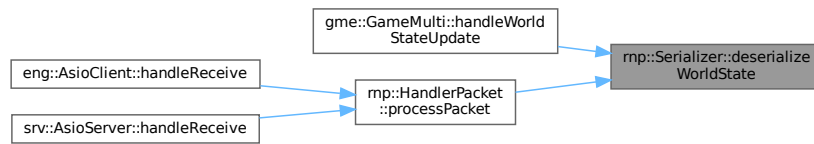
References [deserializeEntityState\(\)](#), [rnp::PacketWorldState::entities](#), [rnp::PacketWorldState::entityCount](#), [readUInt16\(\)](#), [readUInt32\(\)](#), and [rnp::PacketWorldState::serverTick](#).

Referenced by [gme::GameMulti::handleWorldStateUpdate\(\)](#), and [rnp::HandlerPacket::processPacket\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.130.3.26 getData()

```
const std::vector< std::uint8_t > & rnp::Serializer::getData () const [inline]
```

Get the serialized data.

Returns

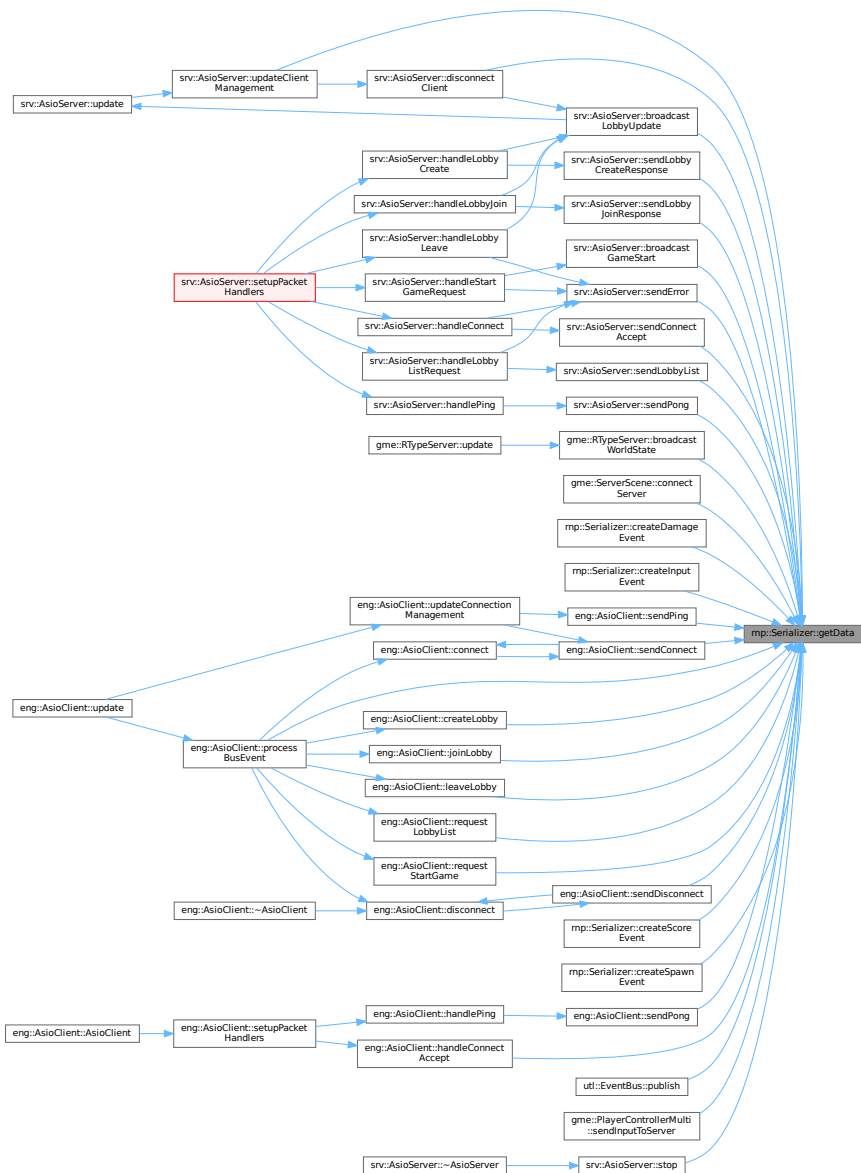
Vector containing the serialized bytes

Definition at line 111 of file [Serializer.hpp](#).

References [buffer_](#).

Referenced by [srv::AsioServer::broadcastGameStart\(\)](#), [srv::AsioServer::broadcastLobbyUpdate\(\)](#), [gme::RTypeServer::broadcastWorldState\(\)](#), [gme::ServerScene::connectServer\(\)](#), [createDamageEvent\(\)](#), [createInputEvent\(\)](#), [eng::AsioClient::createLobby\(\)](#), [createScoreEvent\(\)](#), [createSpawnEvent\(\)](#), [srv::AsioServer::disconnect\(\)](#), [eng::AsioClient::handleConnectAccept\(\)](#), [eng::AsioClient::joinLobby\(\)](#), [eng::AsioClient::leaveLobby\(\)](#), [eng::AsioClient::processBusEvent\(\)](#), [utl::EventBus::publish\(\)](#), [eng::AsioClient::requestLobbyList\(\)](#), [eng::AsioClient::requestStartGame\(\)](#), [eng::AsioClient::sendConnect\(\)](#), [srv::AsioServer::sendConnectAccept\(\)](#), [eng::AsioClient::sendDisconnect\(\)](#), [srv::AsioServer::sendError\(\)](#), [gme::PlayerControllerMulti::sendInputToServer\(\)](#), [srv::AsioServer::sendLobbyCreateResponse\(\)](#), [srv::AsioServer::sendLobbyJoinResponse\(\)](#), [srv::AsioServer::sendLobbyList\(\)](#), [eng::AsioClient::sendPing\(\)](#), [eng::AsioClient::sendPong\(\)](#), [srv::AsioServer::sendPong\(\)](#), [srv::AsioServer::stop\(\)](#), and [srv::AsioServer::updateClientManagement\(\)](#).

Here is the caller graph for this function:



18.130.3.27 getSize()

`std::size_t rnp::Serializer::getSize () const [inline]`
 Get the current size of serialized data.

Returns

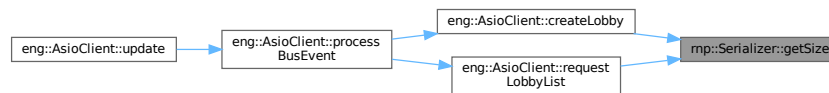
Size in bytes

Definition at line 117 of file [Serializer.hpp](#).

References [writePos_](#).

Referenced by [eng::AsioClient::createLobby\(\)](#), and [eng::AsioClient::requestLobbyList\(\)](#).

Here is the caller graph for this function:



18.130.3.28 readByte()

`std::uint8_t rnp::Serializer::readByte ()` [inline]

Read a single byte.

Returns

Byte value

Definition at line 213 of file [Serializer.hpp](#).

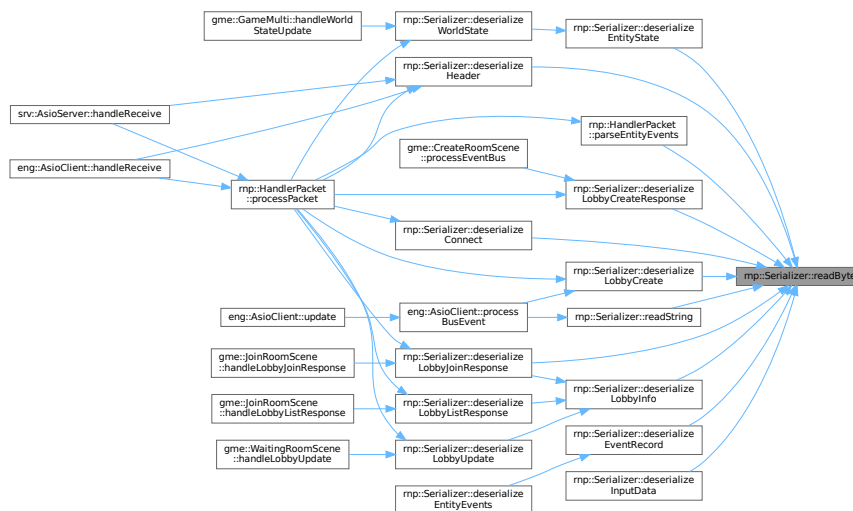
References [readBytes\(\)](#).

Referenced by [deserializeConnect\(\)](#), [deserializeEntityState\(\)](#), [deserializeEventRecord\(\)](#), [deserializeHeader\(\)](#), [deserializeInputData\(\)](#), [deserializeLobbyCreate\(\)](#), [deserializeLobbyCreateResponse\(\)](#), [deserializeLobbyInfo\(\)](#), [deserializeLobbyJoinResponse\(\)](#), [rnp::HandlerPacket::parseEntityEvents\(\)](#), and [readString\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.130.3.29 readBytes()

```
void rnp::Serializer::readBytes (
    void * data,
    std::size_t size) [inline]
```

Read raw bytes.

Parameters

data	Pointer to destination buffer
size	Number of bytes to read

Definition at line 198 of file [Serializer.hpp](#).

References [buffer_](#), and [readPos_](#).

Referenced by [deserializeConnect\(\)](#), [deserializeError\(\)](#), [deserializeEventRecord\(\)](#), [deserializeLobbyCreate\(\)](#), [deserializeLobbyInfo\(\)](#), [rnp::HandlerPacket::parseEntityEvents\(\)](#), [readByte\(\)](#), [readFloat\(\)](#), [readString\(\)](#), [readUInt16\(\)](#), and [readUInt32\(\)](#).

Here is the caller graph for this function:



18.130.3.31 readString()

```
std::string rnp::Serializer::readString (
    std::size_t maxLength) [inline]
```

Read a string with length prefix.

Parameters

maxLength	Maximum expected length
-----------	-------------------------

Returns

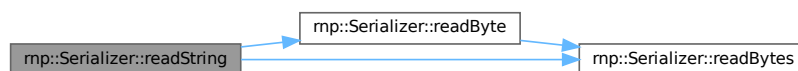
The read string

Definition at line 258 of file [Serializer.hpp](#).

References [readByte\(\)](#), and [readBytes\(\)](#).

Referenced by [eng::AsioClient::processBusEvent\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.130.3.32 readUInt16()

```
std::uint16_t rnp::Serializer::readUInt16 () [inline]
```

Read a 16-bit integer (network byte order)

Returns

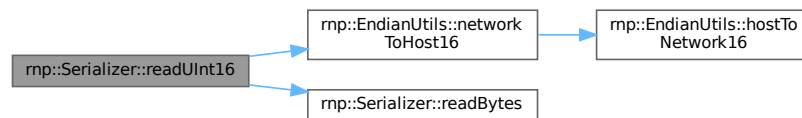
16-bit integer in host byte order

Definition at line 224 of file [Serializer.hpp](#).

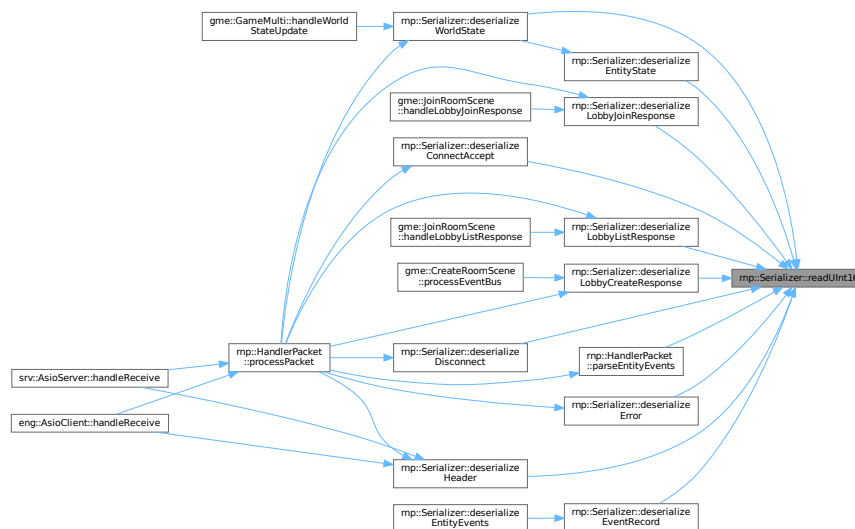
References [rnp::EndianUtils::networkToHost16\(\)](#), and [readBytes\(\)](#).

Referenced by [deserializeConnectAccept\(\)](#), [deserializeDisconnect\(\)](#), [deserializeEntityState\(\)](#), [deserializeError\(\)](#), [deserializeEventRecord\(\)](#), [deserializeHeader\(\)](#), [deserializeLobbyCreateResponse\(\)](#), [deserializeLobbyJoinResponse\(\)](#), [deserializeLobbyListResponse\(\)](#), [deserializeWorldState\(\)](#), and [rnp::HandlerPacket::parseEntityEvents\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.130.3.33 readUInt32()

`std::uint32_t rnp::Serializer::readUInt32 () [inline]`

Read a 32-bit integer (network byte order)

Returns

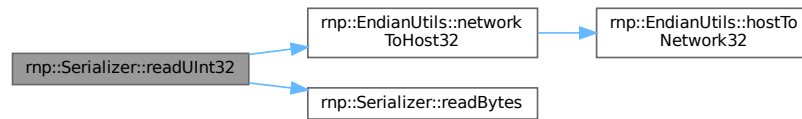
32-bit integer in host byte order

Definition at line 235 of file [Serializer.hpp](#).

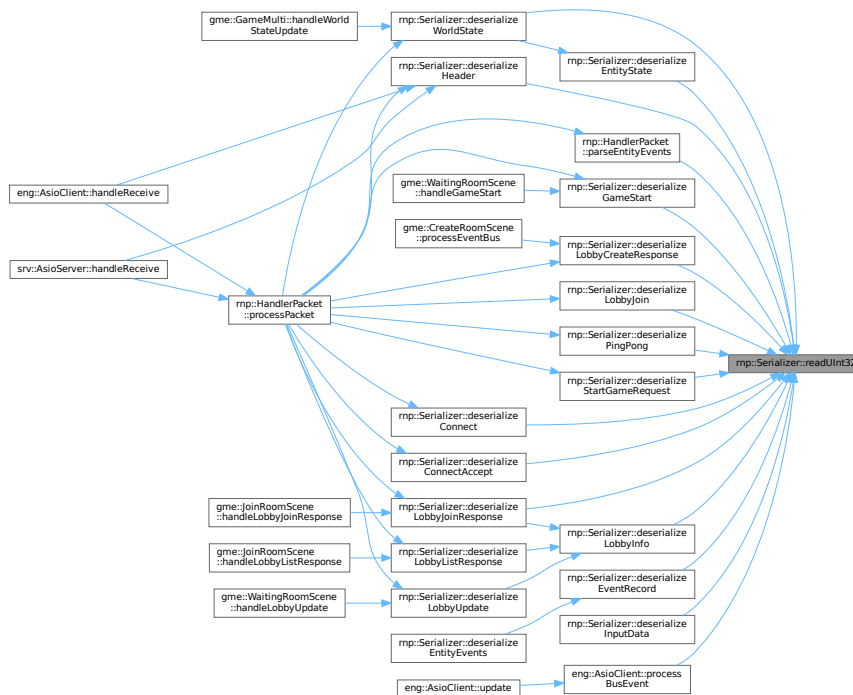
References [rnp::EndianUtils::networkToHost32\(\)](#), and [readBytes\(\)](#).

Referenced by [deserializeConnect\(\)](#), [deserializeConnectAccept\(\)](#), [deserializeEntityState\(\)](#), [deserializeEventRecord\(\)](#), [deserializeGameStart\(\)](#), [deserializeHeader\(\)](#), [deserializeInputData\(\)](#), [deserializeLobbyCreateResponse\(\)](#), [deserializeLobbyInfo\(\)](#), [deserializeLobbyJoin\(\)](#), [deserializeLobbyJoinResponse\(\)](#), [deserializePingPong\(\)](#), [deserializeStartGameRequest\(\)](#), [deserializeWorldState\(\)](#), [rnp::HandlerPacket::parseEntityEvents\(\)](#), and [eng::asioClient::processBusEvent\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.130.3.34 reset()

```
void rnp::Serializer::reset () [inline]
```

Reset the serializer for reuse.

Definition at line 100 of file [Serializer.hpp](#).

References [buffer_](#), [readPos_](#), and [writePos_](#).

18.130.3.35 serializeConnect()

```
void rnp::Serializer::serializeConnect (
    const PacketConnect & packet) [inline]
```

Serialize CONNECT packet.

Parameters

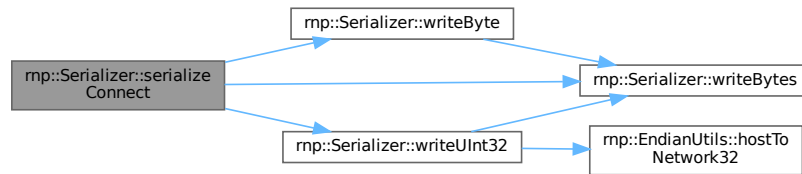
packet	CONNECT packet to serialize
--------	-----------------------------

Definition at line 304 of file [Serializer.hpp](#).

References [rnp::PacketConnect::clientCaps](#), [rnp::PacketConnect::nameLen](#), [rnp::PacketConnect::playerName](#), [writeByte\(\)](#), [writeBytes\(\)](#), and [writeUInt32\(\)](#).

Referenced by [eng::AsioClient::sendConnect\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.130.3.36 serializeConnectAccept()

void `rnp::Serializer::serializeConnectAccept` (
 const [PacketConnectAccept](#) & packet) [inline]
 Serialize CONNECT_ACCEPT packet.

Parameters

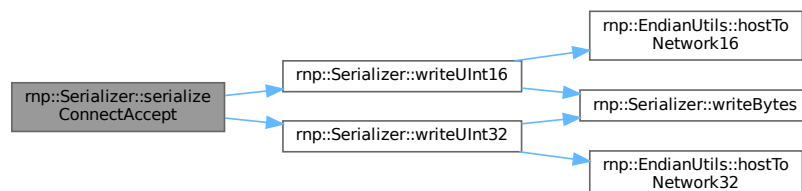
packet	CONNECT_ACCEPT packet to serialize
--------	------------------------------------

Definition at line 328 of file [Serializer.hpp](#).

References [rnp::PacketConnectAccept::mtuPayloadBytes](#), [rnp::PacketConnectAccept::serverCaps](#), [rnp::PacketConnectAccept::sessionId](#), [rnp::PacketConnectAccept::tickRateHz](#), [writeUInt16\(\)](#), and [writeUInt32\(\)](#).

Referenced by [srv::AsioServer::sendConnectAccept\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.130.3.37 serializeDisconnect()

```
void rnp::Serializer::serializeDisconnect (
    const PacketDisconnect & packet) [inline]
Serialize DISCONNECT packet.
```

Parameters

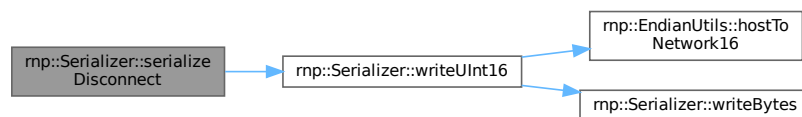
packet	DISCONNECT packet to serialize
--------	--------------------------------

Definition at line 354 of file [Serializer.hpp](#).

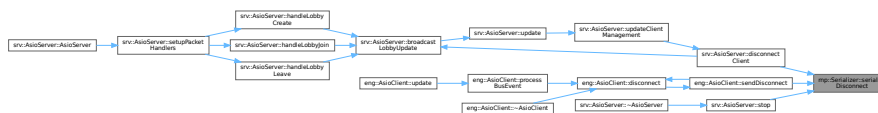
References [rnp::PacketDisconnect::reasonCode](#), and [writeUInt16\(\)](#).

Referenced by [srv::AsioServer::disconnectClient\(\)](#), [eng::AsioClient::sendDisconnect\(\)](#), and [srv::AsioServer::stop\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.130.3.38 serializeEntityEvents()

```
void rnp::Serializer::serializeEntityEvents (
    const std::vector< EventRecord > & events) [inline]
Serialize multiple EventRecords for ENTITY_EVENT packet.
```

Parameters

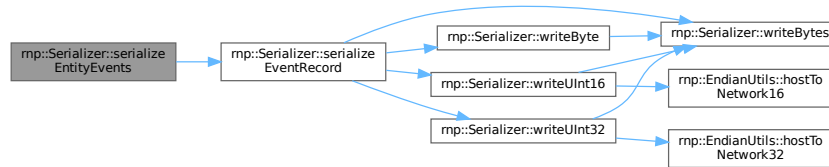
events	Vector of event records to serialize
--------	--------------------------------------

Definition at line 532 of file [Serializer.hpp](#).

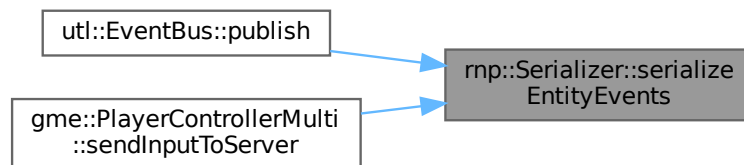
References [serializeEventRecord\(\)](#).

Referenced by [utl::EventBus::publish\(\)](#), and [gme::PlayerControllerMulti::sendInputToServer\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.130.3.39 serializeEntityState()

void rnp::Serializer::serializeEntityState (const [EntityState](#) & entity) [inline]
Serialize [EntityState](#).

Parameters

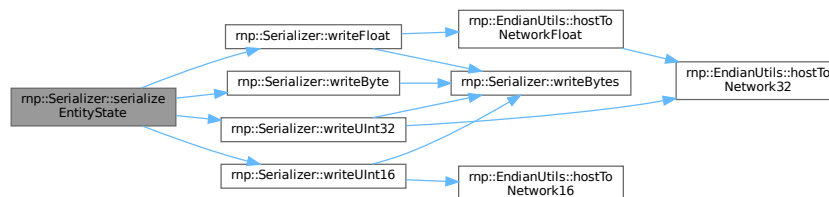
entity	Entity state to serialize
--------	---------------------------

Definition at line 371 of file [Serializer.hpp](#).

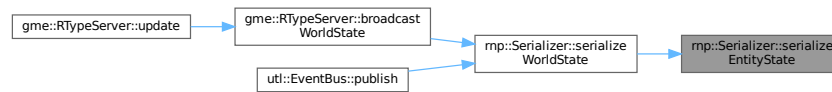
References [rnp::EntityState::healthPercent](#), [rnp::EntityState::id](#), [rnp::EntityState::stateFlags](#), [rnp::EntityState::subtype](#), [rnp::EntityState::type](#), [rnp::EntityState::vx](#), [rnp::EntityState::vy](#), [writeByte\(\)](#), [writeFloat\(\)](#), [writeUInt16\(\)](#), [writeUInt32\(\)](#), [rnp::EntityState::x](#), and [rnp::EntityState::y](#).

Referenced by [serializeWorldState\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.130.3.40 serializeError()

```
void rnp::Serializer::serializeError (
    const PacketError & packet) [inline]
Serialize ERROR packet.
```

Parameters

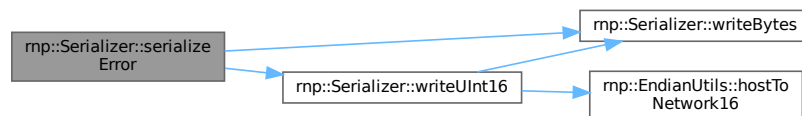
packet	ERROR packet to serialize
--------	---------------------------

Definition at line 463 of file [Serializer.hpp](#).

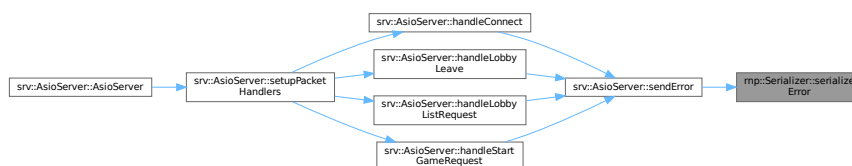
References [rnp::PacketError::description](#), [rnp::PacketError::errorCode](#), [rnp::PacketError::msgLen](#), [writeBytes\(\)](#), and [writeUInt16\(\)](#).

Referenced by [srv::AsioServer::sendError\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.130.3.41 serializeEventRecord()

```
void rnp::Serializer::serializeEventRecord (
    const EventRecord & event) [inline]
Serialize EventRecord for ENTITY_EVENT packets.
```

Parameters

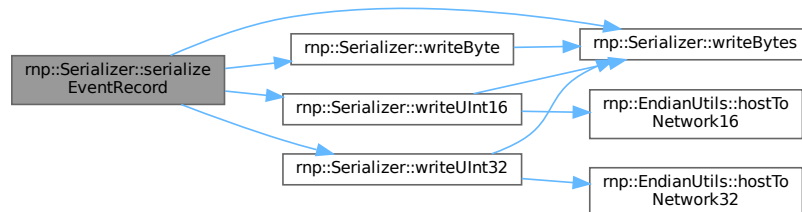
event	Event record to serialize
-------	---------------------------

Definition at line 496 of file [Serializer.hpp](#).

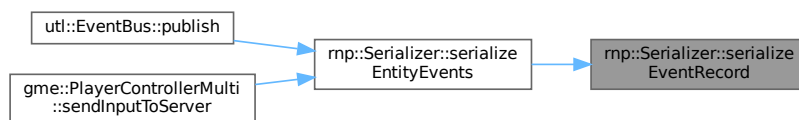
References [rnp::EventRecord::data](#), [rnp::EventRecord::entityId](#), [rnp::EventRecord::type](#), [writeByte\(\)](#), [writeBytes\(\)](#), [writeUInt16\(\)](#), and [writeUInt32\(\)](#).

Referenced by [serializeEntityEvents\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.130.3.42 serializeGameStart()

```
void rnp::Serializer::serializeGameStart (
    const PacketGameStart & packet) [inline]
Serialize GAME_START packet.
```

Parameters

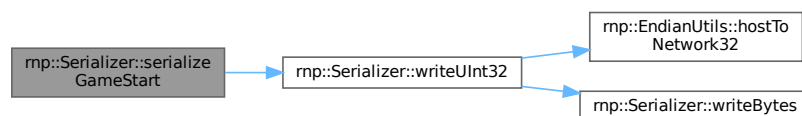
packet	The packet to serialize
--------	-------------------------

Definition at line 895 of file [Serializer.hpp](#).

References [rnp::PacketGameStart::lobbyId](#), and [writeUInt32\(\)](#).

Referenced by [srv::AsioServer::broadcastGameStart\(\)](#), and [utl::EventBus::publish\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.130.3.43 serializeHeader()

```
void rnp::Serializer::serializeHeader (
    const PacketHeader & header) [inline]
Serialize packet header.
```

Parameters

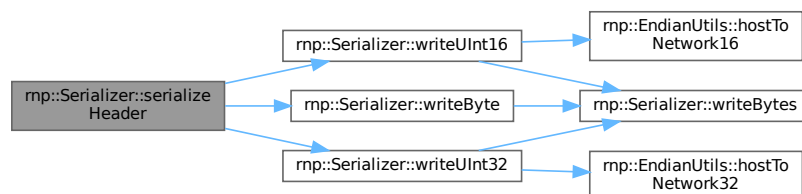
header	Header to serialize
--------	---------------------

Definition at line 280 of file [Serializer.hpp](#).

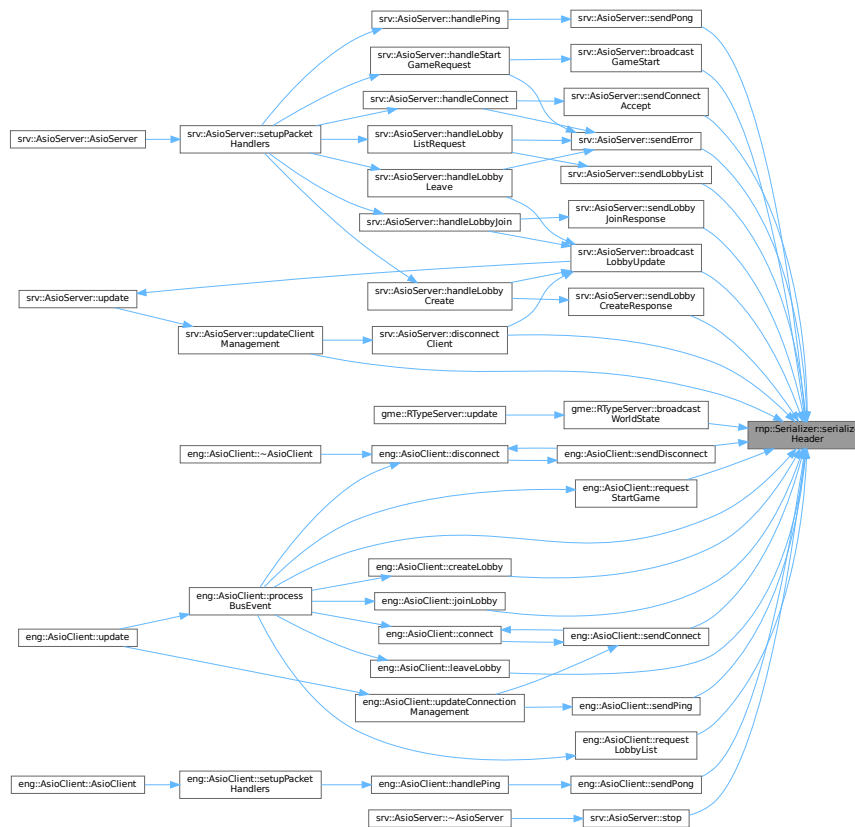
References [rnp::PacketHeader::length](#), [rnp::PacketHeader::sessionId](#), [rnp::PacketHeader::type](#), [writeByte\(\)](#), [writeUInt16\(\)](#), and [writeUInt32\(\)](#).

Referenced by [srv::AsioServer::broadcastGameStart\(\)](#), [srv::AsioServer::broadcastLobbyUpdate\(\)](#), [gme::RTypeServer::broadcastWorldState\(\)](#), [eng::AsioClient::createLobby\(\)](#), [srv::AsioServer::disconnectClient\(\)](#), [eng::AsioClient::joinLobby\(\)](#), [eng::AsioClient::leaveLobby\(\)](#), [eng::AsioClient::processBusEvent\(\)](#), [eng::AsioClient::requestLobbyList\(\)](#), [eng::AsioClient::requestStartGame\(\)](#), [eng::AsioClient::sendConnect\(\)](#), [srv::AsioServer::sendConnectAccept\(\)](#), [eng::AsioClient::sendDisconnect\(\)](#), [srv::AsioServer::sendError\(\)](#), [srv::AsioServer::sendLobbyCreateResponse\(\)](#), [srv::AsioServer::sendLobbyJoinResponse\(\)](#), [srv::AsioServer::sendLobbyList\(\)](#), [eng::AsioClient::sendPing\(\)](#), [eng::AsioClient::sendPong\(\)](#), [srv::AsioServer::sendPong\(\)](#), [srv::AsioServer::stop\(\)](#), and [srv::AsioServer::updateClientManagement\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.130.3.44 serializeInputData()

```
void rnp::Serializer::serializeInputData (
    std::uint8_t keys,
    float mouseX,
    float mouseY,
    std::uint8_t mouseButtons,
    std::uint32_t timestamp) [inline]
```

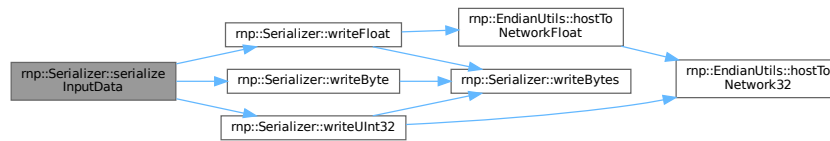
Serialize input data for INPUT events.

Parameters

keys	Keyboard input bitmask
mouseX	Mouse X position
mouseY	Mouse Y position
mouseButtons	Mouse button bitmask
timestamp	Input timestamp

Definition at line 569 of file [Serializer.hpp](#).
References [writeByte\(\)](#), [writeFloat\(\)](#), and [writeUInt32\(\)](#).
Referenced by [createInputEvent\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.130.3.45 serializeLobbyCreate()

void rnp::Serializer::serializeLobbyCreate (
 const [PacketLobbyCreate](#) & packet) [inline]
 Serialize LOBBY_CREATE packet.

Parameters

packet	The packet to serialize
--------	-------------------------

Definition at line 779 of file [Serializer.hpp](#).

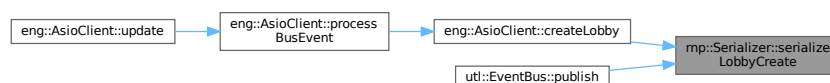
References [rnp::PacketLobbyCreate::gameMode](#), [rnp::PacketLobbyCreate::lobbyName](#), [rnp::PacketLobbyCreate::maxPlay](#), [rnp::PacketLobbyCreate::nameLen](#), [writeByte\(\)](#), and [writeBytes\(\)](#).

Referenced by [eng::AsioClient::createLobby\(\)](#), and [utl::EventBus::publish\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.130.3.46 serializeLobbyCreateResponse()

```
void rnp::Serializer::serializeLobbyCreateResponse (
    const PacketLobbyCreateResponse & packet) [inline]
Serialize LOBBY_CREATE_RESPONSE packet.
```

Parameters

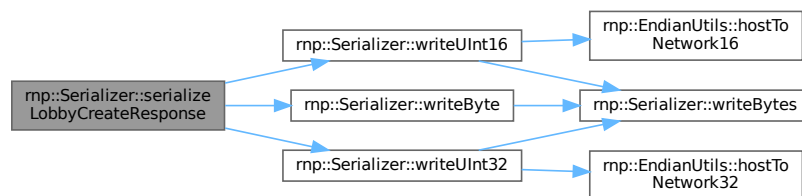
packet	The packet to serialize
--------	-------------------------

Definition at line 805 of file [Serializer.hpp](#).

References [rnp::PacketLobbyCreateResponse::errorCode](#), [rnp::PacketLobbyCreateResponse::lobbyId](#), [rnp::PacketLobbyCreateResponse::success](#), [writeByte\(\)](#), [writeUInt16\(\)](#), and [writeUInt32\(\)](#).

Referenced by [utl::EventBus::publish\(\)](#), and [srv::AsioServer::sendLobbyCreateResponse\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.130.3.47 serializeLobbyInfo()

```
void rnp::Serializer::serializeLobbyInfo (
    const LobbyInfo & lobbyInfo) [inline]
Serialize LobbyInfo structure.
```

Parameters

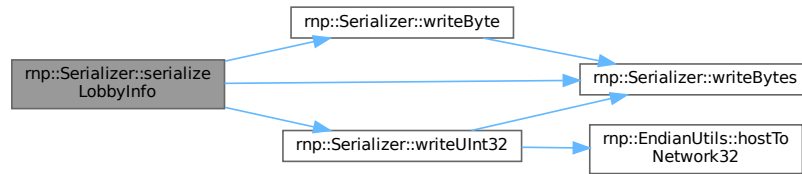
lobbyInfo	The lobby info to serialize
-----------	-----------------------------

Definition at line 702 of file [Serializer.hpp](#).

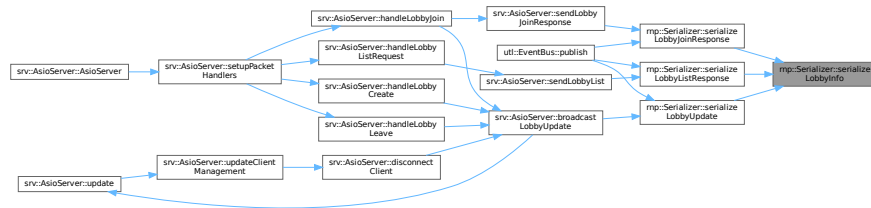
References [rnp::LobbyInfo::currentPlayers](#), [rnp::LobbyInfo::gameMode](#), [rnp::LobbyInfo::hostSessionId](#), [rnp::LobbyInfo::lobbyId](#), [rnp::LobbyInfo::lobbyName](#), [rnp::LobbyInfo::maxPlayers](#), [rnp::LobbyInfo::playerNames](#), [rnp::LobbyInfo::status](#), [writeByte\(\)](#), [writeBytes\(\)](#), and [writeUInt32\(\)](#).

Referenced by [serializeLobbyJoinResponse\(\)](#), [serializeLobbyListResponse\(\)](#), and [serializeLobbyUpdate\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.130.3.48 serializeLobbyJoin()

void rnp::Serializer::serializeLobbyJoin (
 const [PacketLobbyJoin](#) & packet) [inline]
 Serialize LOBBY_JOIN packet.

Parameters

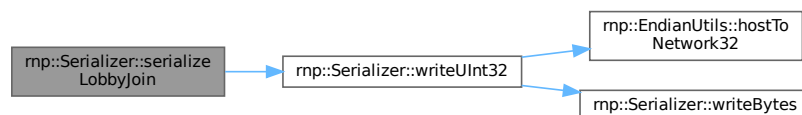
packet	The packet to serialize
--------	-------------------------

Definition at line 829 of file [Serializer.hpp](#).

References [rnp::PacketLobbyJoin::lobbyId](#), and [writeUInt32\(\)](#).

Referenced by [eng::AsioClient::joinLobby\(\)](#), and [utl::EventBus::publish\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.130.3.49 `serializeLobbyJoinResponse()`

```
void rnp::Serializer::serializeLobbyJoinResponse (
    const PacketLobbyJoinResponse & packet) [inline]
Serialize LOBBY_JOIN_RESPONSE packet.
```

Parameters

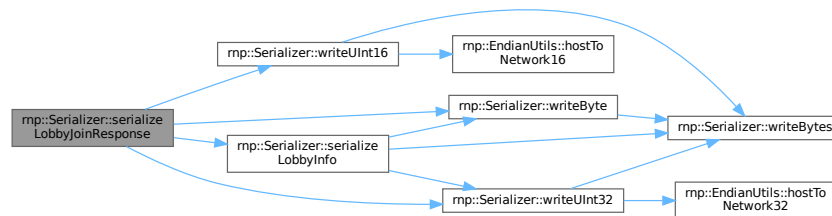
packet	The packet to serialize
--------	-------------------------

Definition at line 846 of file [Serializer.hpp](#).

References [rnp::PacketLobbyJoinResponse::errorCode](#), [rnp::PacketLobbyJoinResponse::lobbyId](#), [rnp::PacketLobbyJoinResponse::serializeLobbyInfo\(\)](#), [rnp::PacketLobbyJoinResponse::success](#), [writeByte\(\)](#), [writeUInt16\(\)](#), and [writeUInt32\(\)](#).

Referenced by [utl::EventBus::publish\(\)](#), and [srv::AsioServer::sendLobbyJoinResponse\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:

18.130.3.50 `serializeLobbyListResponse()`

```
void rnp::Serializer::serializeLobbyListResponse (
    const PacketLobbyListResponse & packet) [inline]
Serialize LOBBY_LIST_RESPONSE packet.
```

Parameters

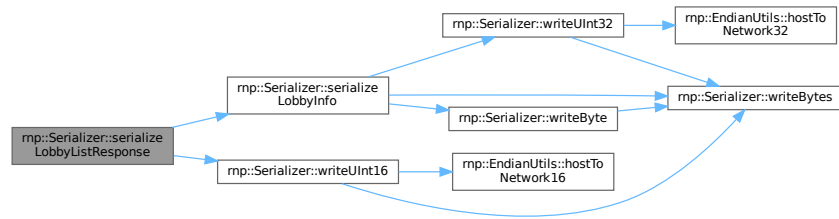
packet	The packet to serialize
--------	-------------------------

Definition at line 748 of file [Serializer.hpp](#).

References [rnp::PacketLobbyListResponse::lobbies](#), [rnp::PacketLobbyListResponse::lobbyCount](#), [serializeLobbyInfo\(\)](#), and [writeUInt16\(\)](#).

Referenced by [utl::EventBus::publish\(\)](#), and [srv::AsioServer::sendLobbyList\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.130.3.51 serializeLobbyUpdate()

```
void rnp::Serializer::serializeLobbyUpdate (
    const PacketLobbyUpdate & packet) [inline]
Serialize LOBBY_UPDATE packet.
```

Parameters

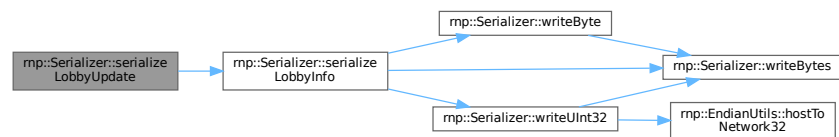
packet	The packet to serialize
--------	-------------------------

Definition at line 878 of file [Serializer.hpp](#).

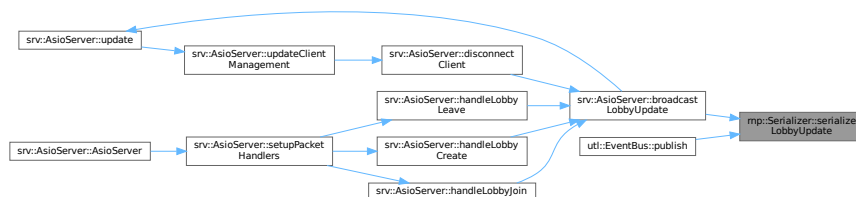
References [rnp::PacketLobbyUpdate::lobbyInfo](#), and [serializeLobbyInfo\(\)](#).

Referenced by [svr::AsioServer::broadcastLobbyUpdate\(\)](#), and [utl::EventBus::publish\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.130.3.52 serializePingPong()

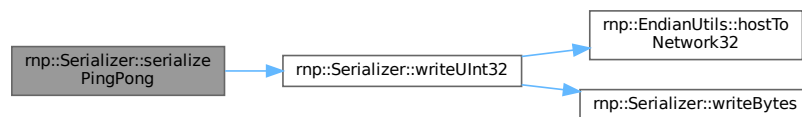
```
void rnp::Serializer::serializePingPong (
    const PacketPingPong & packet) [inline]
Serialize PING/PONG packet.
```

Parameters

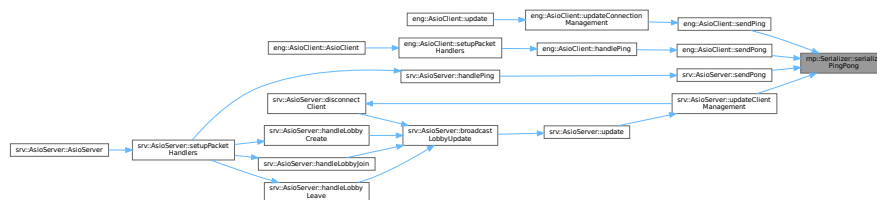
packet	PING/PONG packet to serialize
--------	-------------------------------

Definition at line 441 of file [Serializer.hpp](#).References [rnp::PacketPingPong::nonce](#), [rnp::PacketPingPong::sendTimeMs](#), and [writeUInt32\(\)](#).Referenced by [eng::AsioClient::sendPing\(\)](#), [eng::AsioClient::sendPong\(\)](#), [srv::AsioServer::sendPong\(\)](#), and [srv::AsioServer::updateClientManagement\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.130.3.53 serializeStartGameRequest()

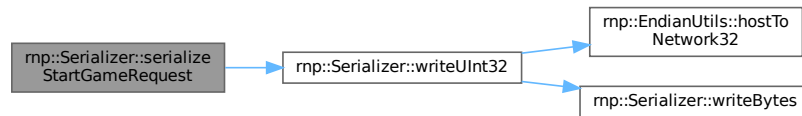
```
void rnp::Serializer::serializeStartGameRequest (
    const PacketStartGameRequest & packet) [inline]
Serialize START_GAME_REQUEST packet.
```

Parameters

packet	The packet to serialize
--------	-------------------------

Definition at line 912 of file [Serializer.hpp](#).References [rnp::PacketStartGameRequest::lobbyId](#), and [writeUInt32\(\)](#).Referenced by [eng::AsioClient::requestStartGame\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.130.3.54 serializeWorldState()

void rnp::Serializer::serializeWorldState (
 const [PacketWorldState](#) & packet) [inline]
 Serialize WORLD_STATE packet.

Parameters

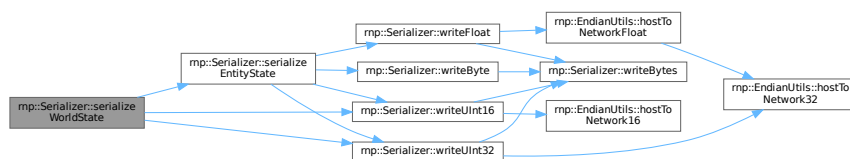
packet	WORLD_STATE packet to serialize
--------	---------------------------------

Definition at line 407 of file [Serializer.hpp](#).

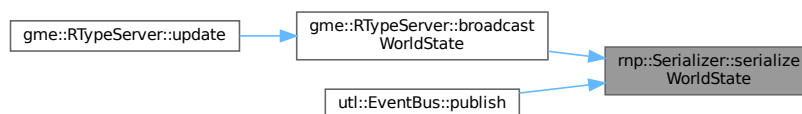
References [rnp::PacketWorldState::entities](#), [rnp::PacketWorldState::entityCount](#), [serializeEntityState\(\)](#), [rnp::PacketWorldState::serverTick](#), [writeUInt16\(\)](#), and [writeUInt32\(\)](#).

Referenced by [gme::RTypeServer::broadcastWorldState\(\)](#), and [utl::EventBus::publish\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.130.3.55 writeByte()

```
void rnp::Serializer::writeByte (
    std::uint8_t value) [inline]
```

Write a single byte.

Parameters

value	Byte value to write
-------	---------------------

Definition at line 142 of file [Serializer.hpp](#).

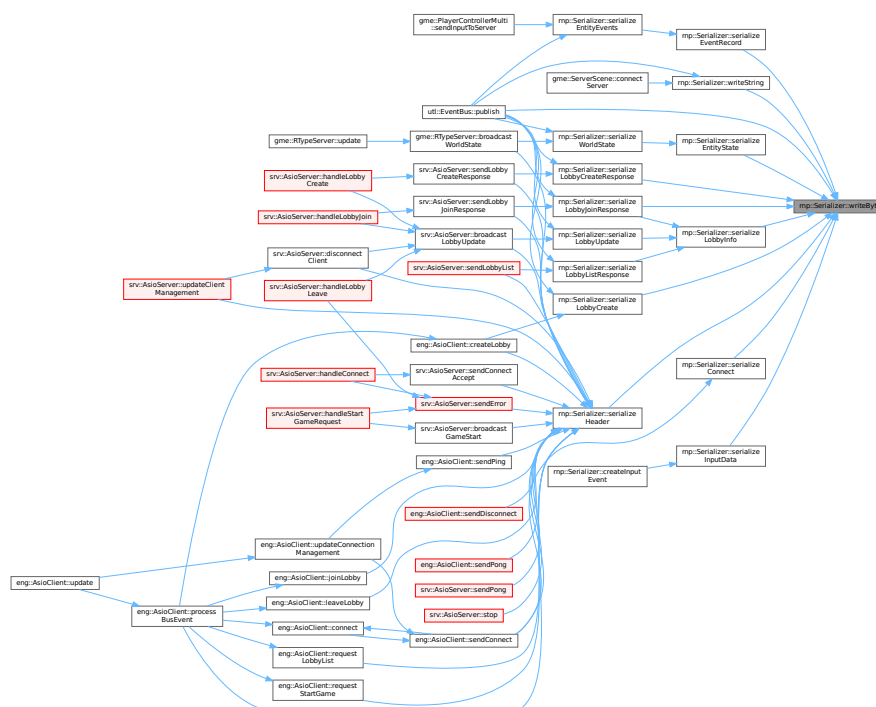
References [writeBytes\(\)](#).

Referenced by [util::EventBus::publish\(\)](#), [serializeConnect\(\)](#), [serializeEntityState\(\)](#), [serializeEventRecord\(\)](#), [serializeHeader\(\)](#), [serializeInputData\(\)](#), [serializeLobbyCreate\(\)](#), [serializeLobbyCreateResponse\(\)](#), [serializeLobbyInfo\(\)](#), [serializeLobbyJoinResponse\(\)](#), and [writeString\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.130.3.56 writeBytes()

```
void rnp::Serializer::writeBytes (
    const void * data,
    std::size_t size) [inline]
```

Write raw bytes.

Parameters

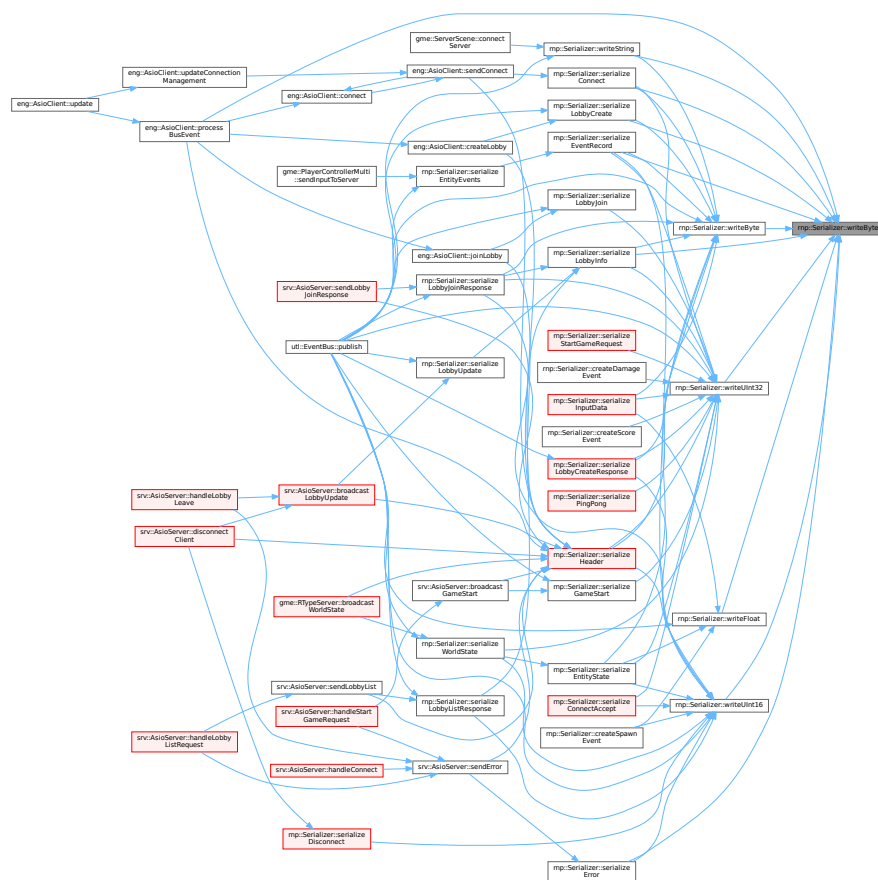
data	Pointer to data
size	Number of bytes to write

Definition at line 124 of file [Serializer.hpp](#).

References [buffer_](#), [rnp::MAX_PAYLOAD](#), and [writePos_](#).

Referenced by [eng::AsioClient::processBusEvent\(\)](#), [serializeConnect\(\)](#), [serializeError\(\)](#), [serializeEventRecord\(\)](#), [serializeLobbyCreate\(\)](#), [serializeLobbyInfo\(\)](#), [writeByte\(\)](#), [writeFloat\(\)](#), [writeString\(\)](#), [writeUInt16\(\)](#), and [writeUInt32\(\)](#).

Here is the caller graph for this function:



18.130.3.57 writeFloat()

```
void rnp::Serializer::writeFloat (
    float value) [inline]
Write a float (network byte order)
```

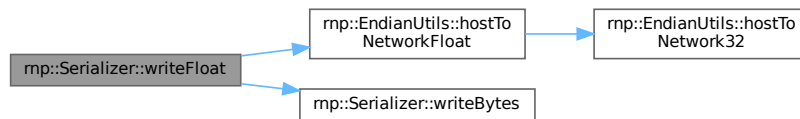
Parameters

value	Float to write
-------	----------------

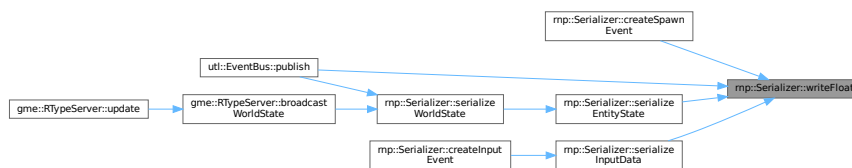
Definition at line 168 of file [Serializer.hpp](#).

References [rnp::EndianUtils::hostToNetworkFloat\(\)](#), and [writeBytes\(\)](#).

Referenced by [createSpawnEvent\(\)](#), [utl::EventBus::publish\(\)](#), [serializeEntityState\(\)](#), and [serializeInputData\(\)](#).
Here is the call graph for this function:



Here is the caller graph for this function:



18.130.3.58 writeString()

```
void rnp::Serializer::writeString (
    const std::string & str,
    std::size_t maxLength) [inline]
```

Write a string with length prefix.

Parameters

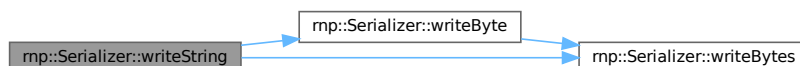
str	String to write
maxLength	Maximum allowed length

Definition at line 179 of file [Serializer.hpp](#).

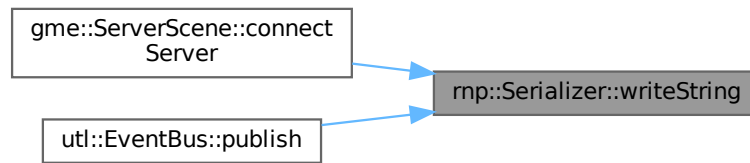
References [writeByte\(\)](#), and [writeBytes\(\)](#).

Referenced by [gme::ServerScene::connectServer\(\)](#), and [utl::EventBus::publish\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.130.3.59 writeUInt16()

```
void rnp::Serializer::writeUInt16 (
    std::uint16_t value) [inline]
```

Write a 16-bit integer (network byte order)

Parameters

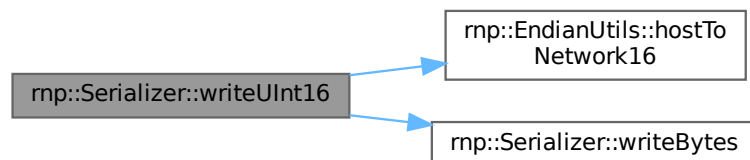
value	16-bit integer to write
-------	-------------------------

Definition at line 148 of file [Serializer.hpp](#).

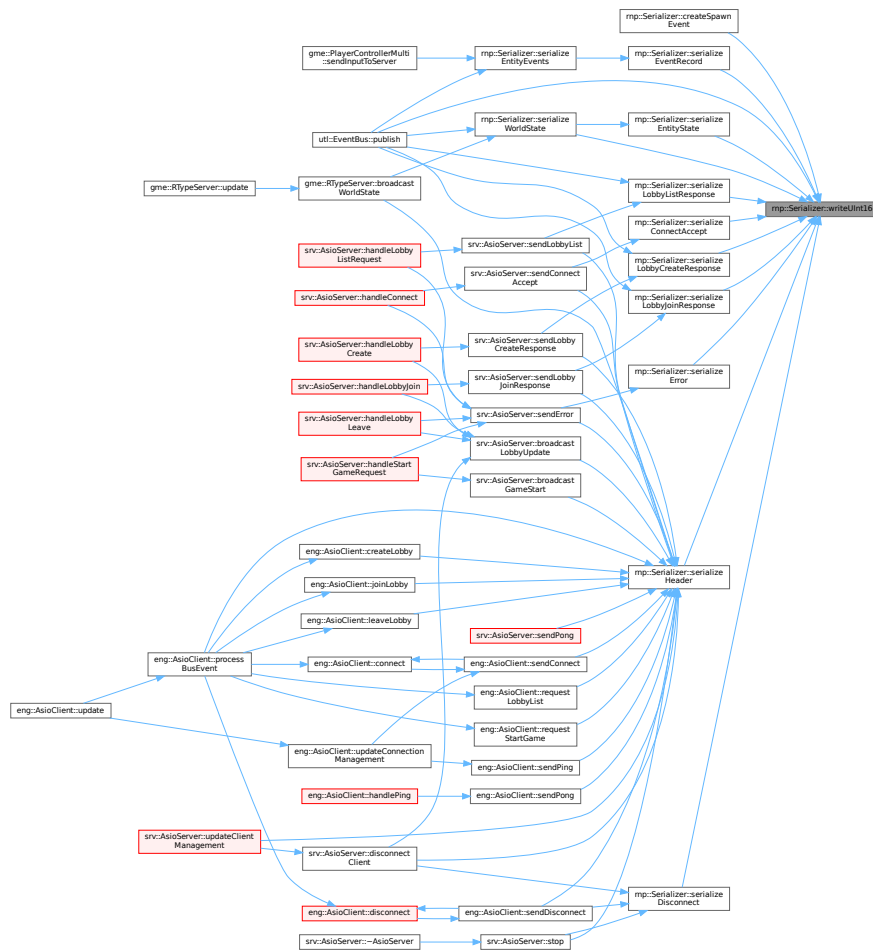
References [rnp::EndianUtils::hostToNetwork16\(\)](#), and [writeBytes\(\)](#).

Referenced by [createSpawnEvent\(\)](#), [utl::EventBus::publish\(\)](#), [serializeConnectAccept\(\)](#), [serializeDisconnect\(\)](#), [serializeEntityState\(\)](#), [serializeError\(\)](#), [serializeEventRecord\(\)](#), [serializeHeader\(\)](#), [serializeLobbyCreateResponse\(\)](#), [serializeLobbyJoinResponse\(\)](#), [serializeLobbyListResponse\(\)](#), and [serializeWorldState\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.130.3.60 writeUInt32()

```
void rnp::Serializer::writeUInt32 (
    std::uint32_t value) [inline]
Write a 32-bit integer (network byte order)
```

Parameters

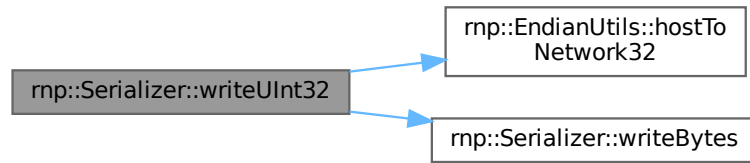
value	32-bit integer to write
-------	-------------------------

Definition at line 158 of file [Serializer.hpp](#).

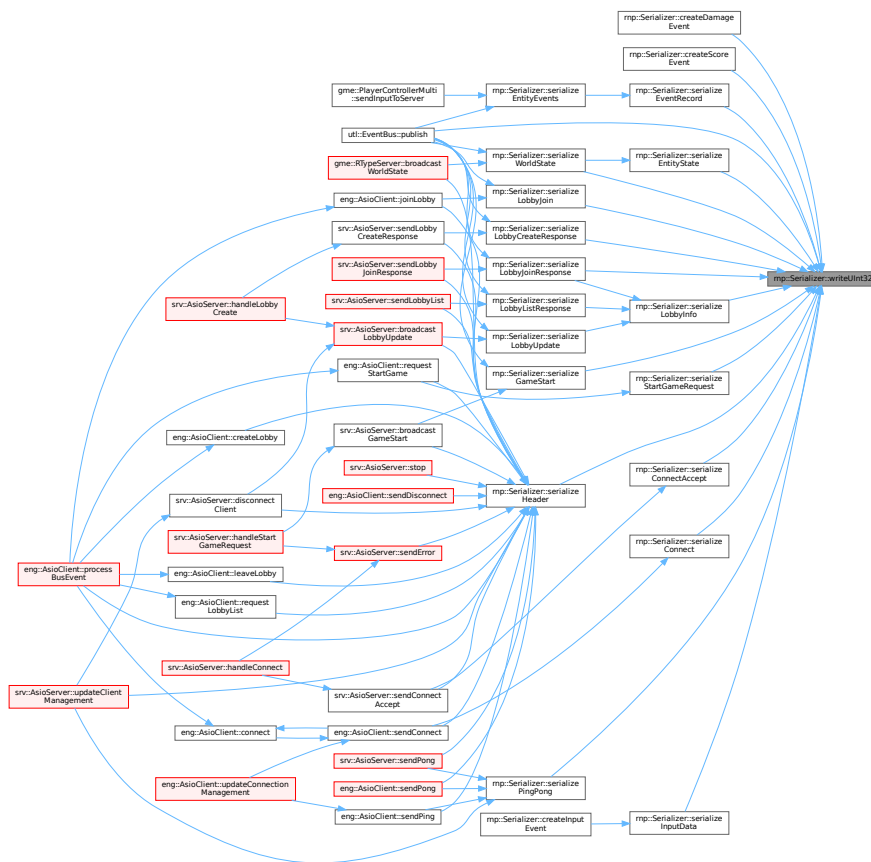
References [rnp::EndianUtils::hostToNetwork32\(\)](#), and [writeBytes\(\)](#).

Referenced by [createDamageEvent\(\)](#), [createScoreEvent\(\)](#), [utl::EventBus::publish\(\)](#), [serializeConnect\(\)](#), [serializeConnectAccept\(\)](#), [serializeEntityState\(\)](#), [serializeEventRecord\(\)](#), [serializeGameStart\(\)](#), [serializeHeader\(\)](#), [serializeInputData\(\)](#), [serializeLobbyCreateResponse\(\)](#), [serializeLobbyInfo\(\)](#), [serializeLobbyJoin\(\)](#), [serializeLobbyJoinResponse\(\)](#), [serializePingPong\(\)](#), [serializeStartGameRequest\(\)](#), and [serializeWorldState\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.130.4 Member Data Documentation

18.130.4.1 buffer_

`std::vector<std::uint8_t> rnp::Serializer::buffer_` [private]

Definition at line 67 of file [Serializer.hpp](#).

Referenced by [deserializeEntityEvents\(\)](#), [getData\(\)](#), [readBytes\(\)](#), [reset\(\)](#), [Serializer\(\)](#), [Serializer\(\)](#), and [writeBytes\(\)](#).

18.130.4.2 readPos_

`std::size_t rnp::Serializer::readPos_` [private]

Definition at line 69 of file [Serializer.hpp](#).

Referenced by [deserializeEntityEvents\(\)](#), [readBytes\(\)](#), and [reset\(\)](#).

18.130.4.3 writePos__

std::size_t rnp::Serializer::writePos__ [private]

Definition at line 68 of file [Serializer.hpp](#).

Referenced by [getSize\(\)](#), [reset\(\)](#), and [writeBytes\(\)](#).

The documentation for this class was generated from the following file:

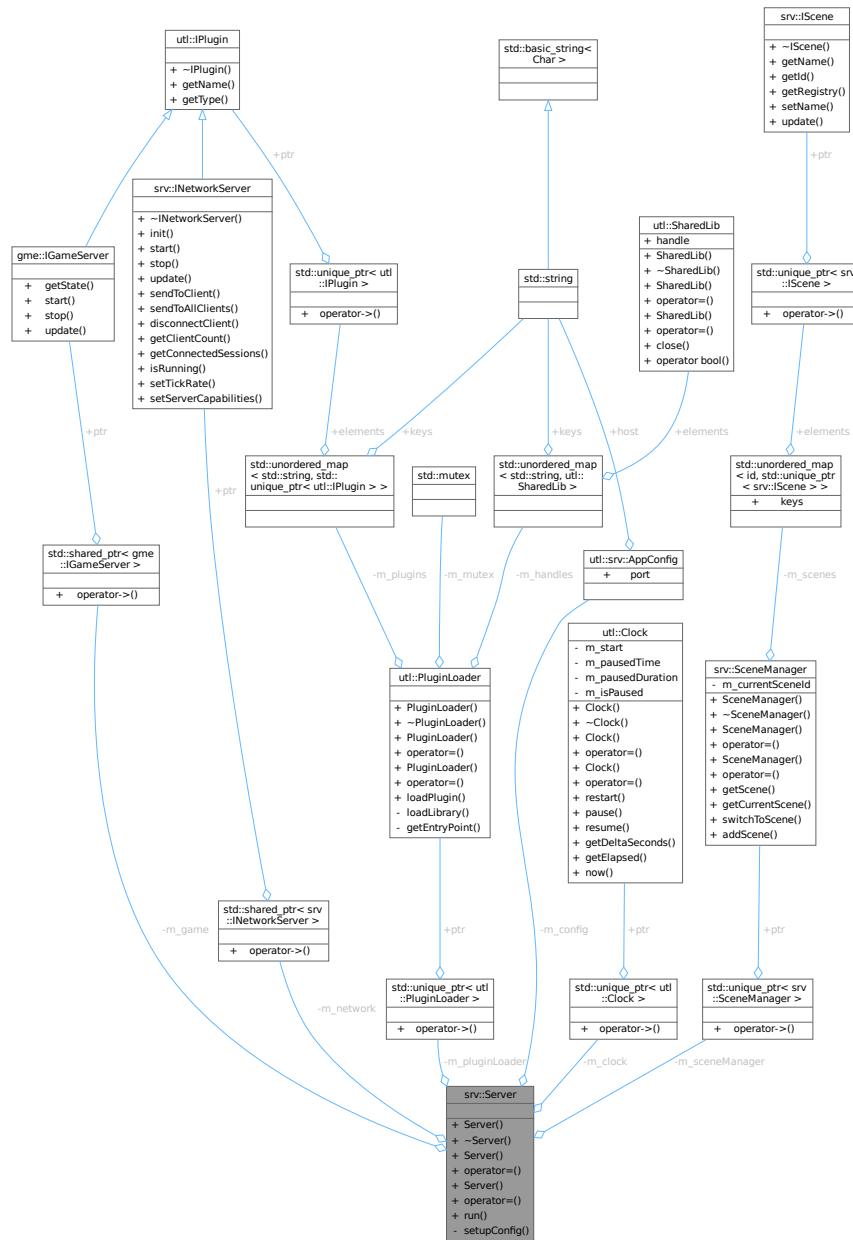
- [/home/masina/Projects/Epitech/rtype/modules/Interfaces/include/Interfaces/Protocol/Serializer.hpp](#)

18.131 srv::Server Class Reference

Class for the server.

#include <Server.hpp>

Collaboration diagram for srv::Server:



Public Member Functions

- [Server](#) (const [ArgsConfig](#) &config)
- [~Server](#) ()=default
- [Server](#) (const [Server](#) &)=delete
- [Server](#) & [operator=](#) (const [Server](#) &)=delete
- [Server](#) ([Server](#) &&)=delete
- [Server](#) & [operator=](#) ([Server](#) &&)=delete
- void [run](#) () const

Static Private Member Functions

- static [utl::srv::AppConfig](#) [setupConfig](#) (const [ArgsConfig](#) &cfg)

Private Attributes

- [utl::srv::AppConfig m_config](#)
- `std::unique_ptr< utl::PluginLoader > m_pluginLoader`
- `std::unique_ptr< utl::Clock > m_clock`
- `std::unique_ptr< SceneManager > m_sceneManager`
- `std::shared_ptr< INetworkServer > m_network`
- `std::shared_ptr< gme::IGameServer > m_game`

18.131.1 Detailed Description

Class for the server.

Definition at line 27 of file [Server.hpp](#).

18.131.2 Constructor & Destructor Documentation

18.131.2.1 `Server()` [1/3]

```
srv::Server::Server (
    const ArgsConfig & config) [explicit]
```

Definition at line 9 of file [server.cpp](#).

References [BUILD_TYPE](#), [GIT_COMMIT_HASH](#), [GIT_TAG](#), [srv::ArgsConfig::host](#), [utl::INFO](#), [utl::Logger::log\(\)](#), [m_config](#), [m_network](#), [srv::ArgsConfig::port](#), [PROJECT_NAME](#), [PROJECT_VERSION](#), and [setupConfig\(\)](#).

Here is the call graph for this function:



18.131.2.2 `~Server()`

```
srv::Server::~Server () [default]
```

18.131.2.3 `Server()` [2/3]

```
srv::Server::Server (
    const Server & ) [delete]
```

18.131.2.4 `Server()` [3/3]

```
srv::Server::Server (
    Server && ) [delete]
```

18.131.3 Member Function Documentation

18.131.3.1 `operator=()` [1/2]

```
Server & srv::Server::operator= (
    const Server & ) [delete]
```

18.131.3.2 `operator=()` [2/2]

```
Server & srv::Server::operator= (
    Server && ) [delete]
```

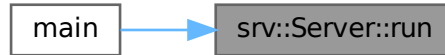
18.131.3.3 run()

void srv::Server::run () const

Definition at line 29 of file [server.cpp](#).

Referenced by [main\(\)](#).

Here is the caller graph for this function:



18.131.3.4 setupConfig()

[utl::srv::AppConfig](#) srv::Server::setupConfig (
 const [ArgsConfig](#) & cfg) [static], [nodiscard], [private]

Definition at line 63 of file [server.cpp](#).

References [srv::ArgsConfig::host](#), [utl::srv::AppConfig::host](#), [srv::ArgsConfig::port](#), and [utl::srv::AppConfig::port](#).

Referenced by [Server\(\)](#).

Here is the caller graph for this function:



18.131.4 Member Data Documentation

18.131.4.1 m_clock

std::unique_ptr<[utl::Clock](#)> srv::Server::m_clock [private]

Definition at line 47 of file [Server.hpp](#).

18.131.4.2 m_config

[utl::srv::AppConfig](#) srv::Server::m_config [private]

Definition at line 44 of file [Server.hpp](#).

Referenced by [Server\(\)](#).

18.131.4.3 m_game

std::shared_ptr<[gme::IGameServer](#)> srv::Server::m_game [private]

Definition at line 51 of file [Server.hpp](#).

18.131.4.4 m_network

std::shared_ptr<[INetworkServer](#)> srv::Server::m_network [private]

Definition at line 50 of file [Server.hpp](#).

Referenced by [Server\(\)](#).

18.131.4.5 m_pluginLoader

std::unique_ptr<[utl::PluginLoader](#)> srv::Server::m_pluginLoader [private]

Definition at line 46 of file [Server.hpp](#).

18.131.4.6 m_sceneManager

std::unique_ptr<[SceneManager](#)> srv::Server::m_sceneManager [private]

Definition at line 48 of file [Server.hpp](#).

The documentation for this class was generated from the following files:

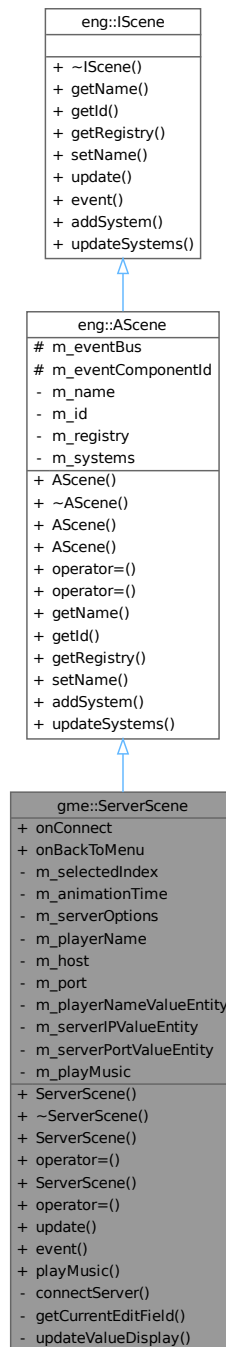
- /home/masina/Projects/Epitech/rtype/server/include/Server/[Server.hpp](#)
- /home/masina/Projects/Epitech/rtype/server/src/[server.cpp](#)

18.132 gme::ServerScene Class Reference

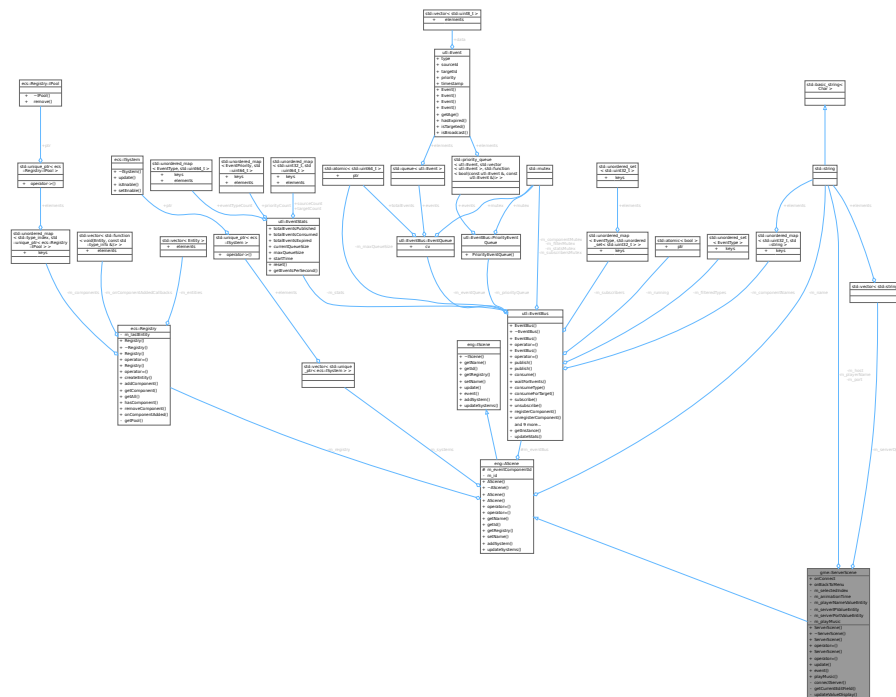
Server configuration scene.

#include <ServerScene.hpp>

Inheritance diagram for gme::ServerScene:



Collaboration diagram for `gme::ServerScene`:



Public Member Functions

- `ServerScene` (`eng::id` assignedId, const std::shared_ptr< `eng::IRenderer` > &renderer, const std::string &host, const std::string &port, const std::string &playerName)
- `~ServerScene` () override=default
- `ServerScene` (const `ServerScene` &other)=delete
- `ServerScene` & operator= (const `ServerScene` &other)=delete
- `ServerScene` (`ServerScene` &&other)=delete
- `ServerScene` & operator= (`ServerScene` &&other)=delete
- void `update` (float dt, const `eng::WindowSize` &size) override
- void `event` (const `eng::Event` &event) override
- bool & `playMusic` ()

Public Member Functions inherited from `eng::AScene`

- `AScene` (const `id` assignedId)
- `~AScene` () override=default
- `AScene` (const `AScene` &other)=delete
- `AScene` (`AScene` &&other)=delete
- `AScene` & operator= (const `AScene` &other)=delete
- `AScene` & operator= (`AScene` &&other)=delete
- std::string & `getName` () override
- `id` `getId` () const override
- `ecs::Registry` & `getRegistry` () override
- void `setName` (const std::string &newName) override
- void `addSystem` (std::unique_ptr< `ecs::ISystem` > system) override
- void `updateSystems` (const float dt) override

Public Member Functions inherited from `eng::IScene`

- virtual `~IScene` ()=default

Public Attributes

- `std::function< void(const std::string &playerName, const std::string &serverIP, const std::string &serverPort)>` [onConnect](#)
- `std::function< void()>` [onBackToMenu](#)

Private Member Functions

- `void` [connectServer](#) (const std::string &playerName, const std::string &serverIP, const std::string &serverPort) const
- `std::string &` [getCurrentEditField](#) ()
- `void` [updateValueDisplay](#) ()

Private Attributes

- `size_t` [m_selectedIndex](#) = 0
- `float` [m_animationTime](#) = 0.0f
- `const std::vector< std::string >` [m_serverOptions](#)
- `std::string` [m_playerName](#)
- `std::string` [m_host](#)
- `std::string` [m_port](#)
- `ecs::Entity` [m_playerNameValueEntity](#) = 0
- `ecs::Entity` [m_serverIPValueEntity](#) = 0
- `ecs::Entity` [m_serverPortValueEntity](#) = 0
- `bool` [m_playMusic](#) = false

Additional Inherited Members

Protected Attributes inherited from [eng::AScene](#)

- `utl::EventBus &` [m_eventBus](#) = `utl::EventBus::getInstance()`
- `std::uint32_t` [m_eventComponentId](#) = 0

18.132.1 Detailed Description

Server configuration scene.

Definition at line 18 of file [ServerScene.hpp](#).

18.132.2 Constructor & Destructor Documentation

18.132.2.1 [ServerScene\(\)](#) [1/3]

```
gme::ServerScene::ServerScene (
    eng::id assignedId,
    const std::shared_ptr< eng::IRenderer > & renderer,
    const std::string & host,
    const std::string & port,
    const std::string & playerName)
```

Definition at line 95 of file [ServerScene.cpp](#).

References [eng::Color::a](#), [eng::Color::b](#), [utl::Config::Color::CYAN_ELECTRIC](#), [utl::Path::Font::FONTS_RTYPE](#), [eng::Color::g](#), [utl::Config::Color::GRAY_BLUE_SUBTLE](#), [ecs::IComponent::id](#), [eng::AScene::m_eventBus](#), [eng::AScene::m_eventComponentId](#), [m_host](#), [m_playerName](#), [m_playerNameValueEntity](#), [m_port](#), [m_serverIPValueEntity](#), [m_serverOptions](#), [m_serverPortValueEntity](#), [eng::Color::r](#), [utl::EventBus::registerComponent\(\)](#), and [utl::Config::Color::TEXT_VALUE_COLOR](#).

Here is the call graph for this function:



18.132.2.2 ~ServerScene()

`gme::ServerScene::~ServerScene ()` [override], [default]

18.132.2.3 ServerScene() [2/3]

`gme::ServerScene::ServerScene (`
 const [ServerScene](#) & other) [delete]

18.132.2.4 ServerScene() [3/3]

`gme::ServerScene::ServerScene (`
 [ServerScene](#) && other) [delete]

18.132.3 Member Function Documentation

18.132.3.1 connectServer()

`void gme::ServerScene::connectServer (`
 const std::string & playerName,
 const std::string & serverIP,
 const std::string & serverPort) const [private]

Definition at line 326 of file [ServerScene.cpp](#).

References [rnp::Serializer::getData\(\)](#), [utl::NETWORK_CLIENT](#), [utl::REQUEST_CONNECT](#), and [rnp::Serializer::writeString\(\)](#).

Here is the call graph for this function:



18.132.3.2 event()

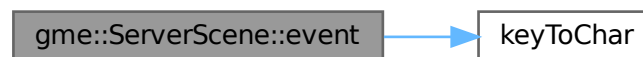
`void gme::ServerScene::event (`
 const [eng::Event](#) & event) [override], [virtual]

Implements [eng::IScene](#).

Definition at line 221 of file [ServerScene.cpp](#).

References [eng::Delete](#), [eng::Down](#), [eng::Enter](#), [eng::Escape](#), [eng::Event::key](#), [eng::KeyPressed](#), [eng::KeyReleased](#), [keyToChar\(\)](#), [eng::Event::type](#), and [eng::Up](#).

Here is the call graph for this function:



18.132.3.3 getCurrentEditField()

`std::string & gme::ServerScene::getCurrentEditField ()` [private]

Definition at line 293 of file [ServerScene.cpp](#).

18.132.3.4 operator=() [1/2]

[ServerScene](#) & gme::ServerScene::operator= (
 const [ServerScene](#) & other) [delete]

18.132.3.5 operator=() [2/2]

[ServerScene](#) & gme::ServerScene::operator= (
 [ServerScene](#) && other) [delete]

18.132.3.6 playMusic()

bool & gme::ServerScene::playMusic () [inline]
 Definition at line 33 of file [ServerScene.hpp](#).
 References [m_playMusic](#).

18.132.3.7 update()

void gme::ServerScene::update (
 float dt,
 const [eng::WindowSize](#) & size) [override], [virtual]

Implements [eng::IScene](#).
 Definition at line 186 of file [ServerScene.cpp](#).
 References [eng::Color::b](#), [eng::Color::g](#), [utl::Config::Color::GRAY_BLUE_SUBTLE](#), and [eng::Color::r](#).

18.132.3.8 updateValueDisplay()

void gme::ServerScene::updateValueDisplay () [private]
 Definition at line 306 of file [ServerScene.cpp](#).

18.132.4 Member Data Documentation

18.132.4.1 m_animationTime

float gme::ServerScene::m_animationTime = 0.0f [private]
 Definition at line 42 of file [ServerScene.hpp](#).

18.132.4.2 m_host

std::string gme::ServerScene::m_host [private]
 Definition at line 50 of file [ServerScene.hpp](#).
 Referenced by [ServerScene\(\)](#).

18.132.4.3 m_playerName

std::string gme::ServerScene::m_playerName [private]
 Definition at line 49 of file [ServerScene.hpp](#).
 Referenced by [ServerScene\(\)](#).

18.132.4.4 m_playerNameValueEntity

[ecs::Entity](#) gme::ServerScene::m_playerNameValueEntity = 0 [private]
 Definition at line 53 of file [ServerScene.hpp](#).
 Referenced by [ServerScene\(\)](#).

18.132.4.5 m_playMusic

bool gme::ServerScene::m_playMusic = false [private]
 Definition at line 56 of file [ServerScene.hpp](#).
 Referenced by [playMusic\(\)](#).

18.132.4.6 m_port

std::string gme::ServerScene::m_port [private]

Definition at line 51 of file [ServerScene.hpp](#).

Referenced by [ServerScene\(\)](#).

18.132.4.7 m_selectedIndex

size_t gme::ServerScene::m_selectedIndex = 0 [private]

Definition at line 41 of file [ServerScene.hpp](#).

18.132.4.8 m_serverIPValueEntity

[ecs::Entity](#) gme::ServerScene::m_serverIPValueEntity = 0 [private]

Definition at line 54 of file [ServerScene.hpp](#).

Referenced by [ServerScene\(\)](#).

18.132.4.9 m_serverOptions

const std::vector<std::string> gme::ServerScene::m_serverOptions [private]

Initial value:

```
= { "Player Name", "Server IP", "Server Port", "Connect",  
    "Back" }
```

Definition at line 43 of file [ServerScene.hpp](#).

Referenced by [ServerScene\(\)](#).

18.132.4.10 m_serverPortValueEntity

[ecs::Entity](#) gme::ServerScene::m_serverPortValueEntity = 0 [private]

Definition at line 55 of file [ServerScene.hpp](#).

Referenced by [ServerScene\(\)](#).

18.132.4.11 onBackToMenu

std::function<void()> gme::ServerScene::onBackToMenu

Definition at line 38 of file [ServerScene.hpp](#).

18.132.4.12 onConnect

std::function<void(const std::string &playerName, const std::string &serverIP, const std::string &serverPort)> gme::ServerScene::onConnect

Definition at line 37 of file [ServerScene.hpp](#).

The documentation for this class was generated from the following files:

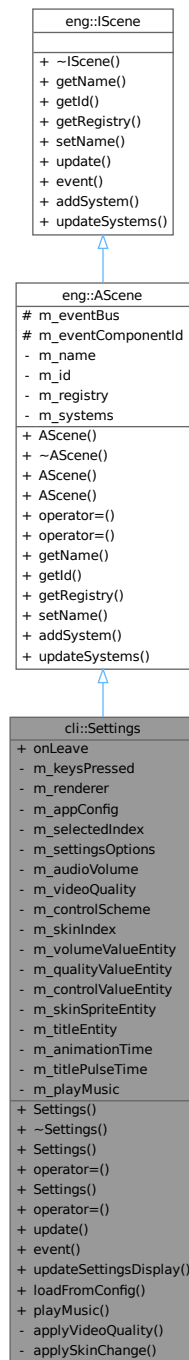
- [/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/include/RTypeClientMulti/Scenes/ServerScene.hpp](#)
- [/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/src/scenes/ServerScene.cpp](#)

18.133 cli::Settings Class Reference

[Settings](#) scene.

```
#include <Settings.hpp>
```

Inheritance diagram for cli::Settings:



Public Member Functions inherited from [eng::IScene](#)

- virtual [~IScene](#) ()=default

Public Attributes

- std::function< void()> [onLeave](#)

Static Private Member Functions

- static void [applyVideoQuality](#) ()
- static void [applySkinChange](#) ()

Private Attributes

- std::unordered_map< [eng::Key](#), bool > [m_keysPressed](#)
- const std::shared_ptr< [eng::IRenderer](#) > & [m_renderer](#)
- [utl::cli::AppConfig](#) & [m_appConfig](#)
- size_t [m_selectedIndex](#) = 0
- const std::vector< std::string > [m_settingsOptions](#)
- float [m_audioVolume](#) = 50.0F
- int [m_videoQuality](#) = 1
- int [m_controlScheme](#) = 2
- int [m_skinIndex](#) = 0
- [ecs::Entity](#) [m_volumeValueEntity](#)
- [ecs::Entity](#) [m_qualityValueEntity](#)
- [ecs::Entity](#) [m_controlValueEntity](#)
- [ecs::Entity](#) [m_skinSpriteEntity](#)
- [ecs::Entity](#) [m_titleEntity](#)
- float [m_animationTime](#) = 0.0f
- float [m_titlePulseTime](#) = 0.0f
- bool [m_playMusic](#) = false

Additional Inherited Members

Protected Attributes inherited from [eng::AScene](#)

- [utl::EventBus](#) & [m_eventBus](#) = [utl::EventBus::getInstance\(\)](#)
- std::uint32_t [m_eventComponentId](#) = 0

18.133.1 Detailed Description

[Settings](#) scene.

Definition at line 22 of file [Settings.hpp](#).

18.133.2 Constructor & Destructor Documentation

18.133.2.1 Settings() [1/3]

```
cli::Settings::Settings (
    eng::id assignedId,
    const std::shared_ptr< eng::IRenderer > & renderer,
    utl::cli::AppConfig & config)
```

Definition at line 9 of file [settings.cpp](#).

References [eng::Color::a](#), [eng::Color::b](#), [utl::Config::Color::CYAN_ELECTRIC](#), [utl::Path::Font::FONTS_RTYPE](#), [eng::Color::g](#), [utl::Config::Color::GRAY_BLUE_SUBTLE](#), [ecs::IComponent::id](#), [utl::Config::Color::INFO_TEXT_COLOR](#), [loadFromConfig\(\)](#), [m_controlValueEntity](#), [m_qualityValueEntity](#), [m_selectedIndex](#), [m_settingsOptions](#), [m_skinSpriteEntity](#), [m_titleEntity](#), [m_volumeValueEntity](#), [eng::Color::r](#), [utl::Config::Color::TEXT_VALUE_COLOR](#), [utl::Config::Color::WHITE](#), and [ecs::Scale::x](#).

Here is the call graph for this function:



18.133.2.2 ~Settings()

cli::Settings::~~Settings () [override], [default]

18.133.2.3 Settings() [2/3]

cli::Settings::Settings (
const [Settings](#) & other) [delete]

18.133.2.4 Settings() [3/3]

cli::Settings::Settings (
[Settings](#) && other) [delete]

18.133.3 Member Function Documentation

18.133.3.1 applySkinChange()

void cli::Settings::applySkinChange () [static], [private]

Definition at line [305](#) of file [settings.cpp](#).

18.133.3.2 applyVideoQuality()

void cli::Settings::applyVideoQuality () [static], [private]

Definition at line [303](#) of file [settings.cpp](#).

18.133.3.3 event()

void cli::Settings::event (
const [eng::Event](#) & event) [override], [virtual]

Implements [eng::IScene](#).

Definition at line [208](#) of file [settings.cpp](#).

References [eng::Down](#), [eng::Enter](#), [eng::Escape](#), [eng::Event::key](#), [eng::KeyPressed](#), [eng::Left](#), [eng::Right](#), [eng::Event::type](#), and [eng::Up](#).

18.133.3.4 loadFromConfig()

void cli::Settings::loadFromConfig ()

Definition at line [292](#) of file [settings.cpp](#).

Referenced by [Settings\(\)](#).

Here is the caller graph for this function:



18.133.3.5 operator=() [1/2]

[Settings](#) & cli::Settings::operator= (const [Settings](#) & other) [delete]

18.133.3.6 operator=() [2/2]

[Settings](#) & cli::Settings::operator= ([Settings](#) && other) [delete]

18.133.3.7 playMusic()

bool & cli::Settings::playMusic () [inline]

Definition at line 37 of file [Settings.hpp](#).

References [m_playMusic](#).

18.133.3.8 update()

void cli::Settings::update (float dt, const [eng::WindowSize](#) & size) [override], [virtual]

Implements [eng::IScene](#).

Definition at line 135 of file [settings.cpp](#).

References [eng::Color::b](#), [utl::Config::Color::CYAN_ELECTRIC](#), [eng::Color::g](#), [utl::Config::Color::GRAY_BLUE_SUBT](#) and [eng::Color::r](#).

18.133.3.9 updateSettingsDisplay()

void cli::Settings::updateSettingsDisplay ()

Definition at line 179 of file [settings.cpp](#).

18.133.4 Member Data Documentation

18.133.4.1 m_animationTime

float cli::Settings::m_animationTime = 0.0f [private]

Definition at line 60 of file [Settings.hpp](#).

18.133.4.2 m_appConfig

[utl::cli::AppConfig](#)& cli::Settings::m_appConfig [private]

Definition at line 47 of file [Settings.hpp](#).

18.133.4.3 m_audioVolume

float cli::Settings::m_audioVolume = 50.0f [private]

Definition at line 51 of file [Settings.hpp](#).

18.133.4.4 m_controlScheme

int cli::Settings::m_controlScheme = 2 [private]

Definition at line 53 of file [Settings.hpp](#).

18.133.4.5 m_controlValueEntity

[ecs::Entity](#) cli::Settings::m_controlValueEntity [private]

Definition at line 57 of file [Settings.hpp](#).

Referenced by [Settings\(\)](#).

18.133.4.6 m_keysPressed

std::unordered_map<[eng::Key](#), bool> cli::Settings::m_keysPressed [private]

Definition at line 45 of file [Settings.hpp](#).

18.133.4.7 m_playMusic

bool cli::Settings::m_playMusic = false [private]

Definition at line 62 of file [Settings.hpp](#).

Referenced by [playMusic\(\)](#).

18.133.4.8 m_qualityValueEntity

[ecs::Entity](#) cli::Settings::m_qualityValueEntity [private]

Definition at line 56 of file [Settings.hpp](#).

Referenced by [Settings\(\)](#).

18.133.4.9 m_renderer

const std::shared_ptr<[eng::IRenderer](#)>& cli::Settings::m_renderer [private]

Definition at line 46 of file [Settings.hpp](#).

18.133.4.10 m_selectedIndex

size_t cli::Settings::m_selectedIndex = 0 [private]

Definition at line 48 of file [Settings.hpp](#).

Referenced by [Settings\(\)](#).

18.133.4.11 m_settingsOptions

const std::vector<std::string> cli::Settings::m_settingsOptions [private]

Initial value:

= { "Audio Volume", "FPS", "Controls", "Skin",
"Back to Menu" }

Definition at line 49 of file [Settings.hpp](#).

Referenced by [Settings\(\)](#).

18.133.4.12 m_skinIndex

int cli::Settings::m_skinIndex = 0 [private]

Definition at line 54 of file [Settings.hpp](#).

18.133.4.13 m_skinSpriteEntity

[ecs::Entity](#) cli::Settings::m_skinSpriteEntity [private]

Definition at line 58 of file [Settings.hpp](#).

Referenced by [Settings\(\)](#).

18.133.4.14 m_titleEntity

[ecs::Entity](#) cli::Settings::m_titleEntity [private]

Definition at line 59 of file [Settings.hpp](#).

Referenced by [Settings\(\)](#).

18.133.4.15 m_titlePulseTime

float cli::Settings::m_titlePulseTime = 0.0f [private]

Definition at line 61 of file [Settings.hpp](#).

18.133.4.16 m_videoQuality

int cli::Settings::m_videoQuality = 1 [private]

Definition at line 52 of file [Settings.hpp](#).

18.133.4.17 m_volumeValueEntity

[ecs::Entity](#) cli::Settings::m_volumeValueEntity [private]

Definition at line 55 of file [Settings.hpp](#).

Referenced by [Settings\(\)](#).

18.133.4.18 onLeave

std::function<void()> cli::Settings::onLeave

Definition at line 39 of file [Settings.hpp](#).

The documentation for this class was generated from the following files:

- [/home/masina/Projects/Epitech/rtype/client/include/Client/Scenes/Settings.hpp](#)
- [/home/masina/Projects/Epitech/rtype/client/src/scenes/settings.cpp](#)

18.134 utl::SharedLib Struct Reference

Handle to a dynamic library with RAII.

#include <PluginLoader.hpp>

Collaboration diagram for utl::SharedLib:

utl::SharedLib
+ handle
+ SharedLib()
+ ~SharedLib()
+ SharedLib()
+ operator=()
+ SharedLib()
+ operator=()
+ close()
+ operator bool()

Public Member Functions

- [SharedLib](#) (const [LibHandle](#) h=nullptr)
- [~SharedLib](#) ()
- [SharedLib](#) (const [SharedLib](#) &)=delete
- [SharedLib](#) & operator= (const [SharedLib](#) &)=delete
- [SharedLib](#) ([SharedLib](#) &&other) noexcept
- [SharedLib](#) & operator= ([SharedLib](#) &&other) noexcept
- void [close](#) ()
- [operator bool](#) () const

Public Attributes

- [LibHandle](#) [handle](#) = nullptr

18.134.1 Detailed Description

Handle to a dynamic library with RAII.

Definition at line 42 of file [PluginLoader.hpp](#).

18.134.2 Constructor & Destructor Documentation

18.134.2.1 SharedLib() [1/3]

```
utl::SharedLib::SharedLib (
    const LibHandle h = nullptr) [inline], [explicit]
```

Definition at line 46 of file [PluginLoader.hpp](#).

18.134.2.2 ~SharedLib()

```
utl::SharedLib::~~SharedLib () [inline]
```

Definition at line 47 of file [PluginLoader.hpp](#).

References [close\(\)](#).

Here is the call graph for this function:



18.134.2.3 SharedLib() [2/3]

```
utl::SharedLib::SharedLib (
    const SharedLib & ) [delete]
```

18.134.2.4 SharedLib() [3/3]

```
utl::SharedLib::SharedLib (
    SharedLib && other) [inline], [noexcept]
```

Definition at line 51 of file [PluginLoader.hpp](#).

18.134.3 Member Function Documentation

18.134.3.1 close()

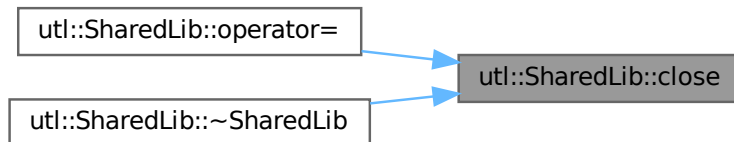
void utl::SharedLib::close () [inline]

Definition at line 63 of file [PluginLoader.hpp](#).

References [handle](#).

Referenced by [operator=\(\)](#), and [~SharedLib\(\)](#).

Here is the caller graph for this function:



18.134.3.2 operator bool()

utl::SharedLib::operator bool () const [inline], [explicit]

Definition at line 75 of file [PluginLoader.hpp](#).

References [handle](#).

18.134.3.3 operator=() [1/2]

[SharedLib](#) & utl::SharedLib::operator= (const [SharedLib](#) &) [delete]

18.134.3.4 operator=() [2/2]

[SharedLib](#) & utl::SharedLib::operator= ([SharedLib](#) && other) [inline], [noexcept]

Definition at line 52 of file [PluginLoader.hpp](#).

References [close\(\)](#), and [handle](#).

Here is the call graph for this function:



18.134.4 Member Data Documentation

18.134.4.1 handle

[LibHandle](#) utl::SharedLib::handle = nullptr

Definition at line 44 of file [PluginLoader.hpp](#).

Referenced by [close\(\)](#), [utl::PluginLoader::getEntryPoint\(\)](#), [operator bool\(\)](#), and [operator=\(\)](#).

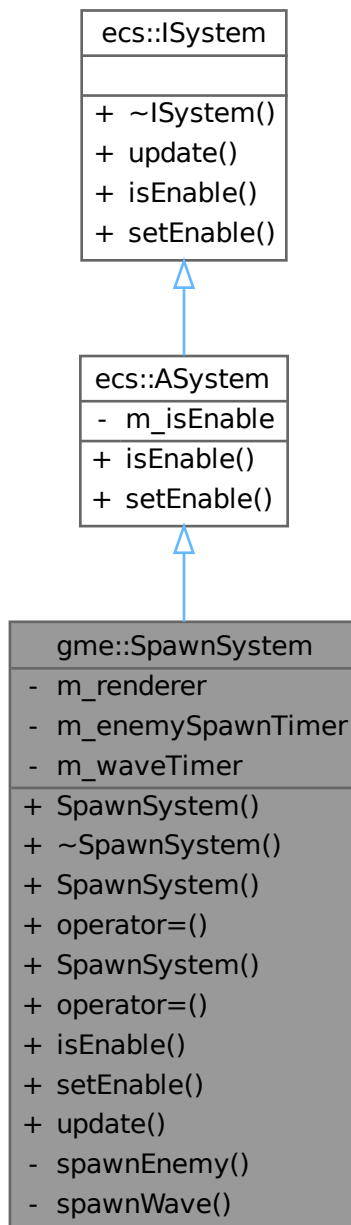
The documentation for this struct was generated from the following file:

- [/home/masina/Projects/Epitech/rtype/modules/Utils/include/Utils/PluginLoader.hpp](#)

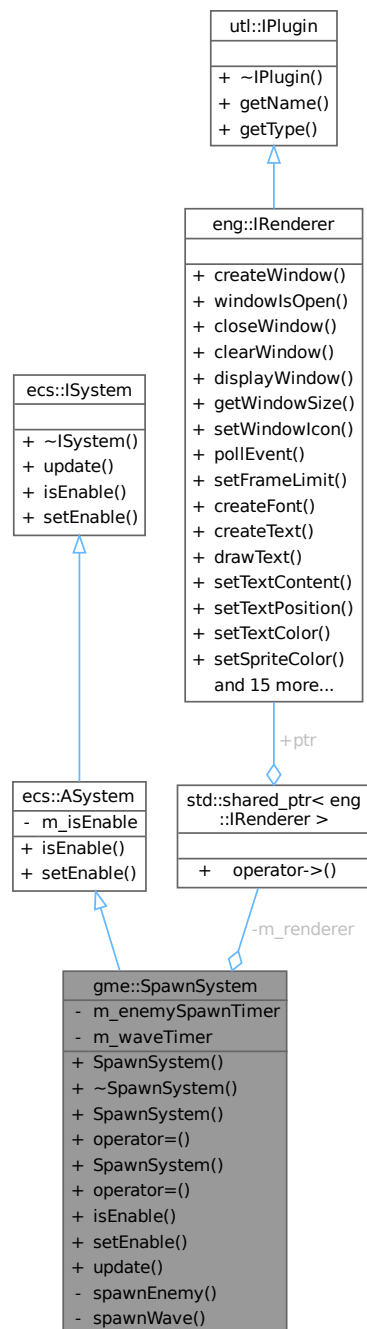
18.135 gme::SpawnSystem Class Reference

#include <Spawn.hpp>

Inheritance diagram for gme::SpawnSystem:



Collaboration diagram for gme::SpawnSystem:



Public Member Functions

- `SpawnSystem` (`const std::shared_ptr< eng::IRenderer > &renderer`)
- `~SpawnSystem` () override=default
- `SpawnSystem` (`const SpawnSystem &`)=delete
- `SpawnSystem & operator=` (`const SpawnSystem &`)=delete
- `SpawnSystem` (`SpawnSystem &&`)=delete
- `SpawnSystem & operator=` (`SpawnSystem &&`)=delete

- bool `isEnabled` () override
- void `setEnabled` (const bool enable) override
- void `update` (ecs::Registry ®istry, float dt) override

Public Member Functions inherited from `ecs::ISystem`

- virtual `~ISystem` ()=default

Static Private Member Functions

- static void `spawnEnemy` (ecs::Registry ®istry)
- static void `spawnWave` (ecs::Registry ®istry)

Private Attributes

- const std::shared_ptr< `eng::IRenderer` > & `m_renderer`
- float `m_enemySpawnTimer` = 0.0F
- float `m_waveTimer` = 0.0F

18.135.1 Detailed Description

Definition at line 20 of file `Spawn.hpp`.

18.135.2 Constructor & Destructor Documentation

18.135.2.1 `SpawnSystem()` [1/3]

```
gme::SpawnSystem::SpawnSystem (
    const std::shared_ptr< eng::IRenderer > & renderer)  [inline], [explicit]
```

Definition at line 23 of file `Spawn.hpp`.

18.135.2.2 `~SpawnSystem()`

```
gme::SpawnSystem::~~SpawnSystem ()  [override], [default]
```

18.135.2.3 `SpawnSystem()` [2/3]

```
gme::SpawnSystem::SpawnSystem (
    const SpawnSystem & )  [delete]
```

18.135.2.4 `SpawnSystem()` [3/3]

```
gme::SpawnSystem::SpawnSystem (
    SpawnSystem && )  [delete]
```

18.135.3 Member Function Documentation

18.135.3.1 `isEnabled()`

```
bool gme::SpawnSystem::isEnabled ()  [inline], [override], [virtual]
```

Reimplemented from `ecs::ASystem`.

Definition at line 31 of file `Spawn.hpp`.

18.135.3.2 `operator=()` [1/2]

```
SpawnSystem & gme::SpawnSystem::operator= (
    const SpawnSystem & )  [delete]
```

18.135.3.3 `operator=()` [2/2]

```
SpawnSystem & gme::SpawnSystem::operator= (
    SpawnSystem && )  [delete]
```


18.135.3.4 setEnable()

void gme::SpawnSystem::setEnable (
 const bool enable) [inline], [override], [virtual]

Reimplemented from [ecs::ASystem](#).

Definition at line 32 of file [Spawn.hpp](#).

18.135.3.5 spawnEnemy()

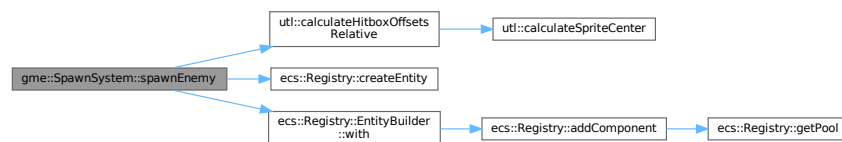
static void gme::SpawnSystem::spawnEnemy (
 [ecs::Registry](#) & registry) [inline], [static], [private]

Definition at line 57 of file [Spawn.hpp](#).

References [utl::GameConfig::Enemy::Easy::ANIMATION_DURATION](#), [utl::GameConfig::Enemy::Easy::ANIMATION_I](#), [utl::calculateHitboxOffsetsRelative\(\)](#), [ecs::Registry::createEntity\(\)](#), [utl::GameConfig::Enemy::Easy::DAMAGE](#), [utl::GameConfig::Hitbox::ENEMY_RADIUS](#), [utl::GameConfig::Enemy::Easy::FRAMES_PER_ROW](#), [utl::GameConfig::Enemy::Easy::HEALTH](#), [utl::GameConfig::Screen::MAX_Y](#), [utl::GameConfig::Screen::MIN_Y](#), [utl::GameConfig::Enemy::Easy::SCALE](#), [utl::GameConfig::Enemy::Easy::SHOOT_COOLDOWN](#), [utl::GameConfig::Screen::SPAWN_X](#), [utl::GameConfig::Enemy::Easy::SPEED](#), [utl::GameConfig::Enemy::Easy::SPRITE](#), [utl::GameConfig::Enemy::Easy::SPRITE_WIDTH](#), [utl::Path::Texture::TEXTURE_ENEMY_EASY](#), and [ecs::Registry::EntityBuilder::with\(\)](#).

Referenced by [update\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.135.3.6 spawnWave()

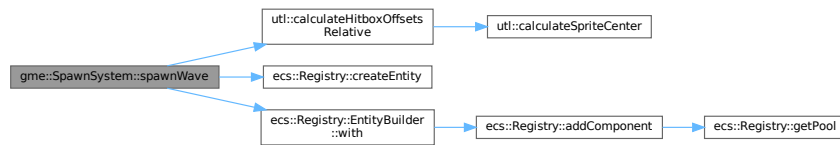
static void gme::SpawnSystem::spawnWave (
 [ecs::Registry](#) & registry) [inline], [static], [private]

Definition at line 90 of file [Spawn.hpp](#).

References [utl::GameConfig::Enemy::Easy::ANIMATION_DURATION](#), [utl::GameConfig::Enemy::Easy::ANIMATION_I](#), [utl::calculateHitboxOffsetsRelative\(\)](#), [ecs::Registry::createEntity\(\)](#), [utl::GameConfig::Enemy::Easy::DAMAGE](#), [utl::GameConfig::Hitbox::ENEMY_RADIUS](#), [utl::GameConfig::Enemy::Easy::FRAMES_PER_ROW](#), [utl::GameConfig::Enemy::Easy::HEALTH](#), [utl::GameConfig::Screen::MAX_Y](#), [utl::GameConfig::Screen::MIN_Y](#), [utl::GameConfig::Enemy::Easy::SCALE](#), [utl::GameConfig::Enemy::Easy::SHOOT_COOLDOWN](#), [utl::GameConfig::Screen::SPAWN_X](#), [utl::GameConfig::Enemy::Easy::SPEED](#), [utl::GameConfig::Enemy::Easy::SPRITE](#), [utl::GameConfig::Enemy::Easy::SPRITE_WIDTH](#), [utl::Path::Texture::TEXTURE_ENEMY_EASY](#), and [ecs::Registry::EntityBuilder::with\(\)](#).

Referenced by [update\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.135.3.7 update()

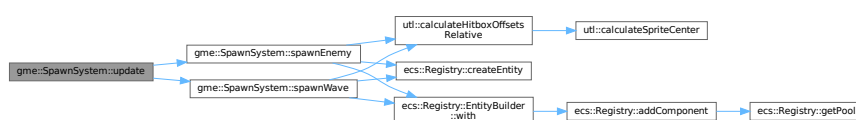
```
void gme::SpawnSystem::update (
    ecs::Registry & registry,
    float dt) [inline], [override], [virtual]
```

Implements [ecs::ISystem](#).

Definition at line 34 of file [Spawn.hpp](#).

References [m_enemySpawnTimer](#), [m_waveTimer](#), [util::GameConfig::Enemy::Easy::SPAWN_RATE](#), [spawnEnemy\(\)](#), and [spawnWave\(\)](#).

Here is the call graph for this function:



18.135.4 Member Data Documentation

18.135.4.1 m_enemySpawnTimer

```
float gme::SpawnSystem::m_enemySpawnTimer = 0.0F [private]
```

Definition at line 54 of file [Spawn.hpp](#).

Referenced by [update\(\)](#).

18.135.4.2 m_renderer

```
const std::shared_ptr<eng::IRenderer>& gme::SpawnSystem::m_renderer [private]
```

Definition at line 53 of file [Spawn.hpp](#).

18.135.4.3 m_waveTimer

```
float gme::SpawnSystem::m_waveTimer = 0.0F [private]
```

Definition at line 55 of file [Spawn.hpp](#).

Referenced by [update\(\)](#).

The documentation for this class was generated from the following file:

- [/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Solo/include/RTypeClient](#)↔
Solo/Systems/[Spawn.hpp](#)

18.136 gme::SpawnWave Struct Reference

Spawn wave configuration.

#include <EnemySpawnSystem.hpp>

Collaboration diagram for gme::SpawnWave:

gme::SpawnWave
+ spawnTime
+ basicEnemyCount
+ advancedEnemyCount
+ bossSpawn
+ spawnInterval
+ spawnedCount

Public Attributes

- float [spawnTime](#)
- int [basicEnemyCount](#)
- int [advancedEnemyCount](#)
- bool [bossSpawn](#)
- float [spawnInterval](#)
- int [spawnedCount](#)

18.136.1 Detailed Description

Spawn wave configuration.

Definition at line 19 of file [EnemySpawnSystem.hpp](#).

18.136.2 Member Data Documentation

18.136.2.1 advancedEnemyCount

int gme::SpawnWave::advancedEnemyCount

Definition at line 23 of file [EnemySpawnSystem.hpp](#).

Referenced by [gme::EnemySpawnSystem::initializeWaves\(\)](#), and [gme::EnemySpawnSystem::processWave\(\)](#).

18.136.2.2 basicEnemyCount

int gme::SpawnWave::basicEnemyCount

Definition at line 22 of file [EnemySpawnSystem.hpp](#).

Referenced by [gme::EnemySpawnSystem::initializeWaves\(\)](#), and [gme::EnemySpawnSystem::processWave\(\)](#).

18.136.2.3 bossSpawn

bool gme::SpawnWave::bossSpawn

Definition at line 24 of file [EnemySpawnSystem.hpp](#).

Referenced by [gme::EnemySpawnSystem::initializeWaves\(\)](#), and [gme::EnemySpawnSystem::processWave\(\)](#).

18.136.2.4 spawnedCount

int gme::SpawnWave::spawnedCount

Definition at line 26 of file [EnemySpawnSystem.hpp](#).

Referenced by [gme::EnemySpawnSystem::initializeWaves\(\)](#), and [gme::EnemySpawnSystem::processWave\(\)](#).

18.136.2.5 spawnInterval

float gme::SpawnWave::spawnInterval

Definition at line 25 of file [EnemySpawnSystem.hpp](#).

Referenced by [gme::EnemySpawnSystem::initializeWaves\(\)](#), and [gme::EnemySpawnSystem::processWave\(\)](#).

18.136.2.6 spawnTime

float gme::SpawnWave::spawnTime

Definition at line 21 of file [EnemySpawnSystem.hpp](#).

Referenced by [gme::EnemySpawnSystem::initializeWaves\(\)](#).

The documentation for this struct was generated from the following file:

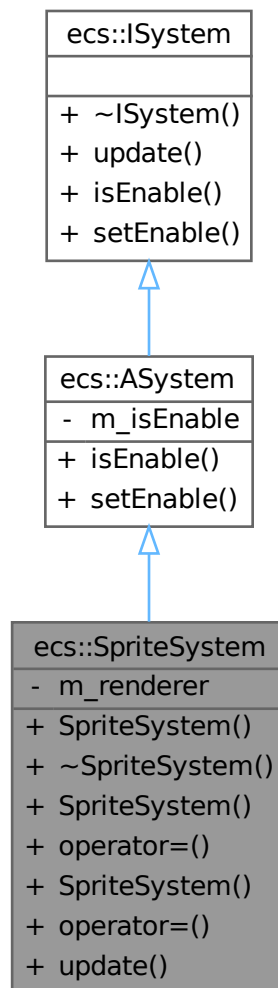
- [/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Server/include/RTypeServer/↵
Systems/EnemySpawnSystem.hpp](#)

18.137 ecs::SpriteSystem Class Reference

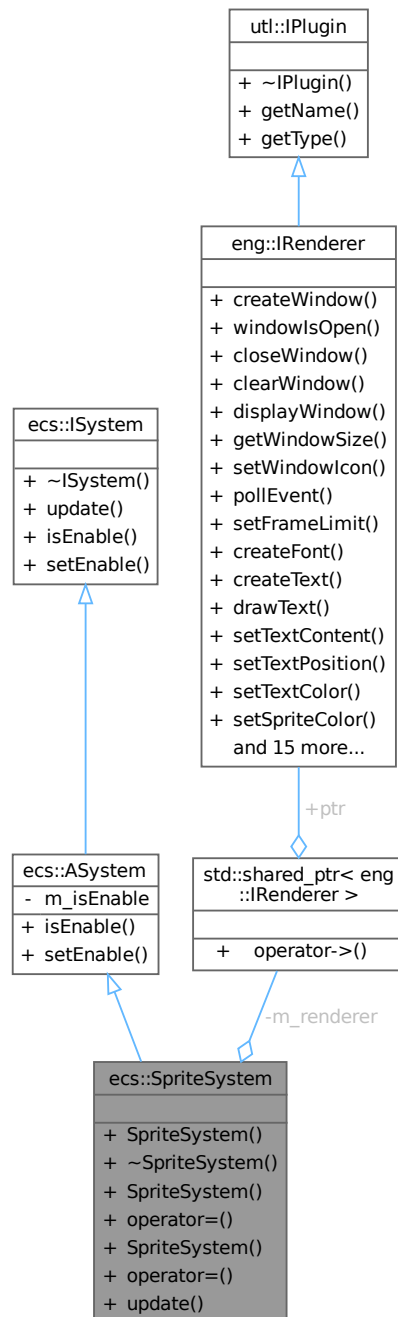
Class for sprite system.

#include <Sprite.hpp>

Inheritance diagram for ecs::SpriteSystem:



Collaboration diagram for `ecs::SpriteSystem`:



Public Member Functions

- `SpriteSystem` (`const std::shared_ptr< eng::IRenderer > &renderer`)
- `~SpriteSystem` () override=default
- `SpriteSystem` (`const SpriteSystem &`)=delete
- `SpriteSystem & operator=` (`const SpriteSystem &`)=delete
- `SpriteSystem` (`SpriteSystem &&`)=delete
- `SpriteSystem & operator=` (`SpriteSystem &&`)=delete
- void `update` (`Registry ®istry`, float) override

Public Member Functions inherited from [ecs::ASystem](#)

- bool [isEnabled](#) () override
- void [setEnabled](#) (const bool enable) override

Public Member Functions inherited from [ecs::ISystem](#)

- virtual [~ISystem](#) ()=default

Private Attributes

- const std::shared_ptr< [eng::IRenderer](#) > & [m_renderer](#)

18.137.1 Detailed Description

Class for sprite system.

Definition at line 26 of file [Sprite.hpp](#).

18.137.2 Constructor & Destructor Documentation

18.137.2.1 [SpriteSystem](#)() [1/3]

[ecs::SpriteSystem::SpriteSystem](#) (
 const std::shared_ptr< [eng::IRenderer](#) > & renderer) [inline], [explicit]

Definition at line 29 of file [Sprite.hpp](#).

18.137.2.2 [~SpriteSystem](#)()

[ecs::SpriteSystem::~SpriteSystem](#) () [override], [default]

18.137.2.3 [SpriteSystem](#)() [2/3]

[ecs::SpriteSystem::SpriteSystem](#) (
 const [SpriteSystem](#) &) [delete]

18.137.2.4 [SpriteSystem](#)() [3/3]

[ecs::SpriteSystem::SpriteSystem](#) (
 [SpriteSystem](#) &&) [delete]

18.137.3 Member Function Documentation

18.137.3.1 [operator=](#)() [1/2]

[SpriteSystem](#) & [ecs::SpriteSystem::operator=](#) (
 const [SpriteSystem](#) &) [delete]

18.137.3.2 [operator=](#)() [2/2]

[SpriteSystem](#) & [ecs::SpriteSystem::operator=](#) (
 [SpriteSystem](#) &&) [delete]

18.137.3.3 [update](#)()

void [ecs::SpriteSystem::update](#) (
 [Registry](#) & registry,
 float) [inline], [override], [virtual]

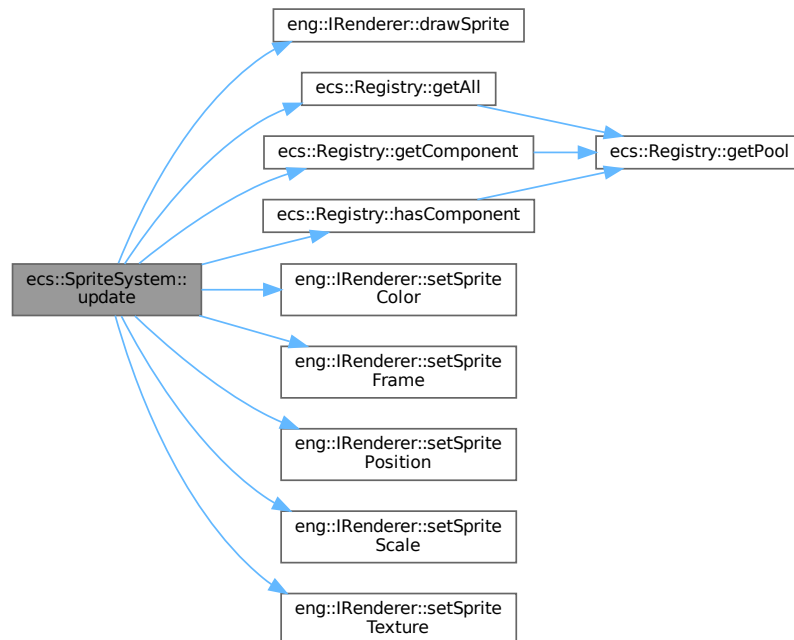
Implements [ecs::ISystem](#).

Definition at line 37 of file [Sprite.hpp](#).

References [eng::IRenderer::drawSprite\(\)](#), [ecs::Registry::getAll\(\)](#), [ecs::Registry::getComponent\(\)](#), [ecs::Registry::hasComponent\(\)](#), [ecs::Layer::layer](#), [m_renderer](#), [eng::IRenderer::setSpriteColor\(\)](#),

[eng::IRenderer::setSpriteFrame\(\)](#), [eng::IRenderer::setSpritePosition\(\)](#), [eng::IRenderer::setSpriteScale\(\)](#), and [eng::IRenderer::setSpriteTexture\(\)](#).

Here is the call graph for this function:



18.137.4 Member Data Documentation

18.137.4.1 m_renderer

`const std::shared_ptr<eng::IRenderer>& ecs::SpriteSystem::m_renderer` [private]

Definition at line 96 of file [Sprite.hpp](#).

Referenced by [update\(\)](#).

The documentation for this class was generated from the following file:

- `/home/masina/Projects/Epitech/rtype/modules/ECS/include/ECS/Systems/Sprite.hpp`

18.138 gme::StageManager Class Reference

```
#include <StageManager.hpp>
```


Collaboration diagram for gme::StageManager:

gme::StageManager
- m_stageTimer - m_stageSpawned - m_currentWave - m_waveTimer - m_totalGameTimer - STAGE_SPAWN_DELAY - WAVE_DURATION - TOTAL_WAVES - SCROLL_SPEED - ORIGINAL_WIDTH - SPRITE_HEIGHT
+ StageManager() + ~StageManager() + StageManager() + operator=() + StageManager() + operator=() + update() + isStageSpawned() + getCurrentWave() + getWaveTimer() + getTotalGameTime() + StageManager() + ~StageManager() + StageManager() + operator=() + StageManager() + operator=() + update() + isStageSpawned() + getCurrentWave() + getWaveTimer() + getTotalGameTime() + spawnStage() + stopScrolling() + spawnStage()

Public Member Functions

- [StageManager](#) ()=default
- [~StageManager](#) ()=default
- [StageManager](#) (const [StageManager](#) &)=delete
- [StageManager](#) & [operator=](#) (const [StageManager](#) &)=delete
- [StageManager](#) ([StageManager](#) &&)=delete
- [StageManager](#) & [operator=](#) (const [StageManager](#) &&)=delete

- void `update` (`ecs::Registry` ®istry, float dt, const `eng::WindowSize` &size)
- bool `isStageSpawned` () const
- int `getCurrentWave` () const
- float `getWaveTimer` () const
- float `getTotalGameTime` () const
- `StageManager` ()=default
- `~StageManager` ()=default
- `StageManager` (const `StageManager` &)=delete
- `StageManager` & `operator=` (const `StageManager` &)=delete
- `StageManager` (`StageManager` &&)=delete
- `StageManager` & `operator=` (`StageManager` &&)=delete
- void `update` (`ecs::Registry` ®istry, float dt, const `eng::WindowSize` &size)
- bool `isStageSpawned` () const
- int `getCurrentWave` () const
- float `getWaveTimer` () const
- float `getTotalGameTime` () const

Static Public Member Functions

- static void `spawnStage` (`ecs::Registry` ®istry, int screenWidth)
- static void `stopScrolling` (`ecs::Registry` ®istry)
- static void `spawnStage` (`ecs::Registry` ®istry, int screenWidth)

Private Attributes

- float `m_stageTimer` = 0.0f
- bool `m_stageSpawned` = false
- int `m_currentWave` = 0
- float `m_waveTimer` = 0.0f
- float `m_totalGameTimer` = 0.0f

Static Private Attributes

- static constexpr float `STAGE_SPAWN_DELAY` = 0.5f
- static constexpr float `WAVE_DURATION` = 30.0f
- static constexpr int `TOTAL_WAVES` = 3
- static constexpr float `SCROLL_SPEED` = -40.0f
- static constexpr float `ORIGINAL_WIDTH` = 2608.0f
- static constexpr float `SPRITE_HEIGHT` = 208.0f

18.138.1 Detailed Description

Definition at line 15 of file `StageManager.hpp`.

18.138.2 Constructor & Destructor Documentation

18.138.2.1 `StageManager()` [1/6]

`gme::StageManager::StageManager` () [default]

18.138.2.2 `~StageManager()` [1/2]

`gme::StageManager::~~StageManager` () [default]

18.138.2.3 `StageManager()` [2/6]

`gme::StageManager::StageManager` (
const `StageManager` &) [delete]

18.138.2.4 StageManager() [3/6]

`gme::StageManager::StageManager (`
 [StageManager](#) &&) [delete]

18.138.2.5 StageManager() [4/6]

`gme::StageManager::StageManager ()` [default]

18.138.2.6 ~StageManager() [2/2]

`gme::StageManager::~~StageManager ()` [default]

18.138.2.7 StageManager() [5/6]

`gme::StageManager::StageManager (`
 const [StageManager](#) &) [delete]

18.138.2.8 StageManager() [6/6]

`gme::StageManager::StageManager (`
 [StageManager](#) &&) [delete]

18.138.3 Member Function Documentation

18.138.3.1 getCurrentWave() [1/2]

`int gme::StageManager::getCurrentWave () const` [inline]
Definition at line 23 of file [StageManager.hpp](#).
References [m_currentWave](#).

18.138.3.2 getCurrentWave() [2/2]

`int gme::StageManager::getCurrentWave () const` [inline]
Definition at line 29 of file [StageManager.hpp](#).
References [m_currentWave](#).

18.138.3.3 getTotalGameTime() [1/2]

`float gme::StageManager::getTotalGameTime () const` [inline]
Definition at line 25 of file [StageManager.hpp](#).
References [m_totalGameTimer](#).

18.138.3.4 getTotalGameTime() [2/2]

`float gme::StageManager::getTotalGameTime () const` [inline]
Definition at line 31 of file [StageManager.hpp](#).
References [m_totalGameTimer](#).

18.138.3.5 getWaveTimer() [1/2]

`float gme::StageManager::getWaveTimer () const` [inline]
Definition at line 24 of file [StageManager.hpp](#).
References [m_waveTimer](#).

18.138.3.6 getWaveTimer() [2/2]

`float gme::StageManager::getWaveTimer () const` [inline]
Definition at line 30 of file [StageManager.hpp](#).
References [m_waveTimer](#).

18.138.3.7 isStageSpawned() [1/2]

bool gme::StageManager::isStageSpawned () const [inline]

Definition at line 22 of file [StageManager.hpp](#).

References [m_stageSpawned](#).

18.138.3.8 isStageSpawned() [2/2]

bool gme::StageManager::isStageSpawned () const [inline]

Definition at line 28 of file [StageManager.hpp](#).

References [m_stageSpawned](#).

18.138.3.9 operator=() [1/4]

[StageManager](#) & gme::StageManager::operator= (
const [StageManager](#) &&) [delete]

18.138.3.10 operator=() [2/4]

[StageManager](#) & gme::StageManager::operator= (
const [StageManager](#) &) [delete]

18.138.3.11 operator=() [3/4]

[StageManager](#) & gme::StageManager::operator= (
const [StageManager](#) &) [delete]

18.138.3.12 operator=() [4/4]

[StageManager](#) & gme::StageManager::operator= (
[StageManager](#) &&) [delete]

18.138.3.13 spawnStage() [1/2]

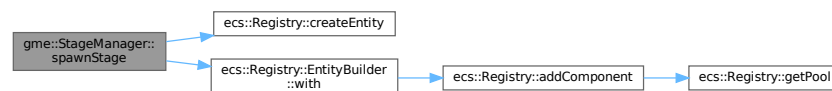
void gme::StageManager::spawnStage (
[ecs::Registry](#) & registry,
int screenWidth) [static]

Definition at line 43 of file [stageManager.cpp](#).

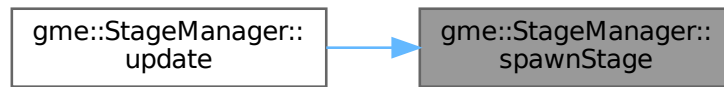
References [ecs::Registry::createEntity\(\)](#), [utl::Path::Texture::TEXTURE_STAGE1_CEILING](#), [utl::Path::Texture::TEXTURE_STAGE1_FLOOR](#), and [ecs::Registry::EntityBuilder::with\(\)](#).

Referenced by [update\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.138.3.14 spawnStage() [2/2]

```
static void gme::StageManager::spawnStage (
    ecs::Registry & registry,
    int screenWidth) [static]
```

18.138.3.15 stopScrolling()

```
void gme::StageManager::stopScrolling (
    ecs::Registry & registry) [static]
```

Definition at line 35 of file [stageManager.cpp](#).

References [ecs::Registry::getAll\(\)](#).

Here is the call graph for this function:



18.138.3.16 update() [1/2]

```
void gme::StageManager::update (
    ecs::Registry & registry,
    float dt,
    const eng::WindowSize & size)
```

Definition at line 7 of file [stageManager.cpp](#).

References [m_currentWave](#), [m_stageSpawned](#), [m_stageTimer](#), [m_totalGameTimer](#), [m_waveTimer](#), [spawnStage\(\)](#), [STAGE_SPAWN_DELAY](#), [TOTAL_WAVES](#), [WAVE_DURATION](#), and [eng::WindowSize::width](#).

Here is the call graph for this function:



18.138.3.17 update() [2/2]

```
void gme::StageManager::update (
    ecs::Registry & registry,
```

```
float dt,
const eng::WindowSize & size)
```

18.138.4 Member Data Documentation

18.138.4.1 `m_currentWave`

int `gme::StageManager::m_currentWave` = 0 [private]
Definition at line 34 of file [StageManager.hpp](#).
Referenced by [getCurrentWave\(\)](#), and [update\(\)](#).

18.138.4.2 `m_stageSpawned`

bool `gme::StageManager::m_stageSpawned` = false [private]
Definition at line 31 of file [StageManager.hpp](#).
Referenced by [isStageSpawned\(\)](#), and [update\(\)](#).

18.138.4.3 `m_stageTimer`

float `gme::StageManager::m_stageTimer` = 0.0f [private]
Definition at line 30 of file [StageManager.hpp](#).
Referenced by [update\(\)](#).

18.138.4.4 `m_totalGameTimer`

float `gme::StageManager::m_totalGameTimer` = 0.0f [private]
Definition at line 36 of file [StageManager.hpp](#).
Referenced by [getTotalGameTime\(\)](#), and [update\(\)](#).

18.138.4.5 `m_waveTimer`

float `gme::StageManager::m_waveTimer` = 0.0f [private]
Definition at line 35 of file [StageManager.hpp](#).
Referenced by [getWaveTimer\(\)](#), and [update\(\)](#).

18.138.4.6 `ORIGINAL_WIDTH`

static constexpr float `gme::StageManager::ORIGINAL_WIDTH` = 2608.0f [static], [constexpr], [private]
Definition at line 42 of file [StageManager.hpp](#).

18.138.4.7 `SCROLL_SPEED`

static constexpr float `gme::StageManager::SCROLL_SPEED` = -40.0f [static], [constexpr], [private]
Definition at line 41 of file [StageManager.hpp](#).

18.138.4.8 `SPRITE_HEIGHT`

static constexpr float `gme::StageManager::SPRITE_HEIGHT` = 208.0f [static], [constexpr], [private]
Definition at line 43 of file [StageManager.hpp](#).

18.138.4.9 `STAGE_SPAWN_DELAY`

static constexpr float `gme::StageManager::STAGE_SPAWN_DELAY` = 0.5f [static], [constexpr], [private]
Definition at line 38 of file [StageManager.hpp](#).
Referenced by [update\(\)](#).

18.138.4.10 `TOTAL_WAVES`

static constexpr int `gme::StageManager::TOTAL_WAVES` = 3 [static], [constexpr], [private]
Definition at line 40 of file [StageManager.hpp](#).
Referenced by [update\(\)](#).

18.138.4.11 WAVE_DURATION

static constexpr float gme::StageManager::WAVE_DURATION = 30.0f [static], [constexpr], [private]

Definition at line 39 of file [StageManager.hpp](#).

Referenced by [update\(\)](#).

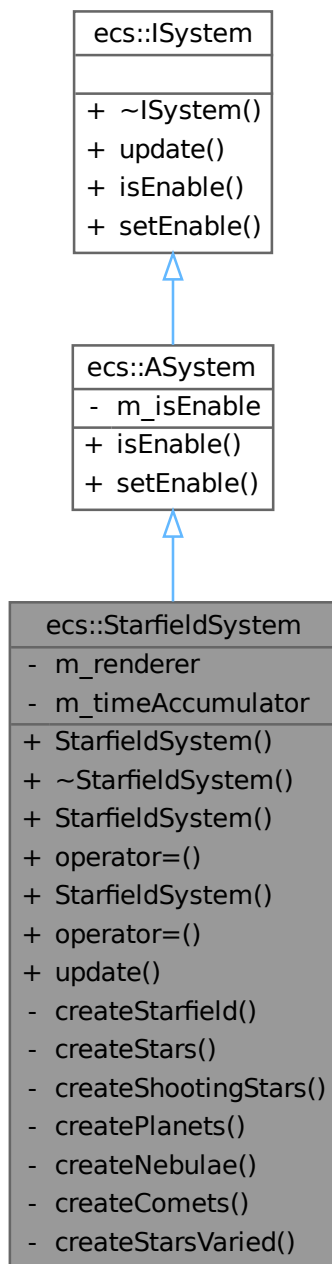
The documentation for this class was generated from the following files:

- /home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/include/RType↵
ClientMulti/Managers/[StageManager.hpp](#)
- /home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Solo/include/RTypeClient↵
Solo/Managers/[StageManager.hpp](#)
- /home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/src/managers/[stageManager.cpp](#)
- /home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Solo/src/managers/[stageManager.cpp](#)

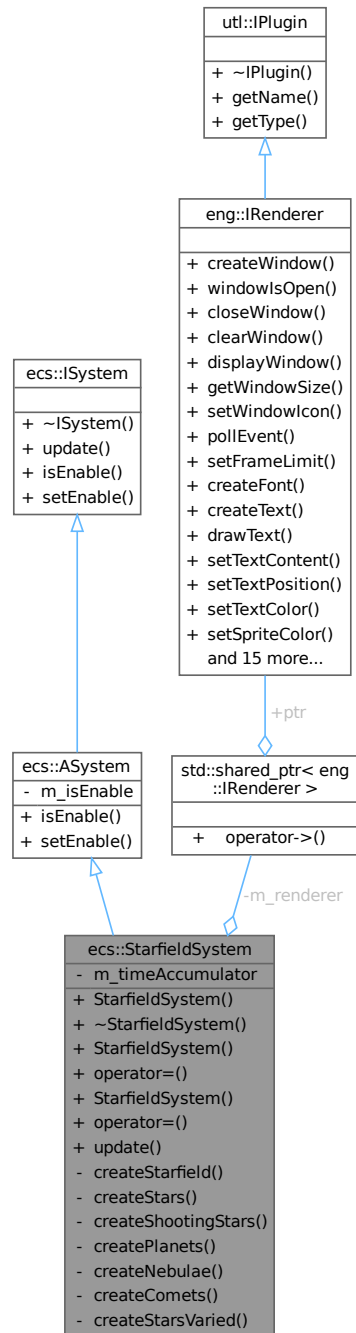
18.139 ecs::StarfieldSystem Class Reference

#include <Starfield.hpp>

Inheritance diagram for ecs::StarfieldSystem:



Collaboration diagram for ecs::StarfieldSystem:



Public Member Functions

- [StarfieldSystem](#) (const std::shared_ptr< [eng::IRenderer](#) > &renderer, [Registry](#) ®istry)
- [~StarfieldSystem](#) () override=default
- [StarfieldSystem](#) (const [StarfieldSystem](#) &)=delete
- [StarfieldSystem](#) & operator= (const [StarfieldSystem](#) &)=delete
- [StarfieldSystem](#) ([StarfieldSystem](#) &&)=delete
- [StarfieldSystem](#) & operator= ([StarfieldSystem](#) &&)=delete
- void [update](#) ([Registry](#) ®istry, const float dt) override

Public Member Functions inherited from [ecs::ASystem](#)

- bool [isEnabled](#) () override
- void [setEnabled](#) (const bool enable) override

Public Member Functions inherited from [ecs::ISystem](#)

- virtual [~ISystem](#) ()=default

Static Private Member Functions

- static void [createStarfield](#) ([Registry](#) ®istry, const [eng::WindowSize](#) &>windowSize)
- static void [createStars](#) ([Registry](#) ®istry, const int count, const unsigned int screenWidth, const unsigned int screenHeight, const [eng::Color](#) &color, float velocity, const std::string &id)
- static void [createShootingStars](#) ([Registry](#) ®istry, const int count, const unsigned int screenWidth, const unsigned int screenHeight)
- static void [createPlanets](#) ([Registry](#) ®istry, const int count, const unsigned int screenWidth, const unsigned int screenHeight)
- static void [createNebulae](#) ([Registry](#) ®istry, const int count, const unsigned int screenWidth, const unsigned int screenHeight)
- static void [createComets](#) ([Registry](#) ®istry, const int count, const unsigned int screenWidth, const unsigned int screenHeight)
- static void [createStarsVaried](#) ([Registry](#) ®istry, const int count, const unsigned int screenWidth, const unsigned int screenHeight, float velocity, const std::string &baseId)

Private Attributes

- const std::shared_ptr< [eng::IRenderer](#) > & [m_renderer](#)
- float [m_timeAccumulator](#) = 0.0f

18.139.1 Detailed Description

Definition at line 21 of file [Starfield.hpp](#).

18.139.2 Constructor & Destructor Documentation

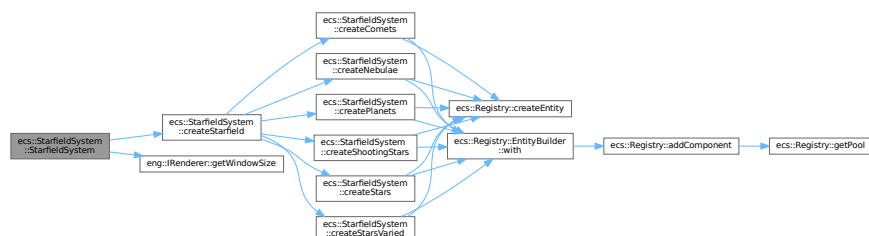
18.139.2.1 StarfieldSystem() [1/3]

```
ecs::StarfieldSystem::StarfieldSystem (
    const std::shared_ptr< eng::IRenderer > & renderer,
    Registry & registry) [inline], [explicit]
```

Definition at line 24 of file [Starfield.hpp](#).

References [createStarfield\(\)](#), [eng::IRenderer::getWindowSize\(\)](#), and [m_renderer](#).

Here is the call graph for this function:



18.139.2.2 ~StarfieldSystem()

```
ecs::StarfieldSystem::~StarfieldSystem () [override], [default]
```

18.139.2.3 StarfieldSystem() [2/3]

```
ecs::StarfieldSystem::StarfieldSystem (
    const StarfieldSystem & ) [delete]
```

18.139.2.4 StarfieldSystem() [3/3]

```
ecs::StarfieldSystem::StarfieldSystem (
    StarfieldSystem && ) [delete]
```

18.139.3 Member Function Documentation

18.139.3.1 createComets()

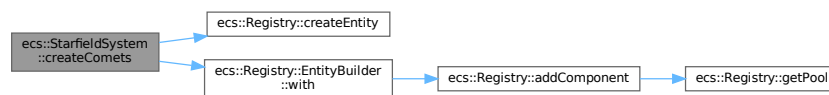
```
static void ecs::StarfieldSystem::createComets (
    Registry & registry,
    const int count,
    const unsigned int screenWidth,
    const unsigned int screenHeight) [inline], [static], [private]
```

Definition at line 188 of file [Starfield.hpp](#).

References [eng::Color::a](#), [eng::Color::b](#), [ecs::Registry::createEntity\(\)](#), [eng::Color::g](#), [utl::Config::Color::GREEN](#), [eng::Color::r](#), and [ecs::Registry::EntityBuilder::with\(\)](#).

Referenced by [createStarfield\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.139.3.2 createNebulae()

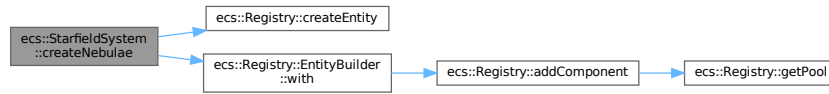
```
static void ecs::StarfieldSystem::createNebulae (
    Registry & registry,
    const int count,
    const unsigned int screenWidth,
    const unsigned int screenHeight) [inline], [static], [private]
```

Definition at line 172 of file [Starfield.hpp](#).

References [eng::Color::a](#), [eng::Color::b](#), [utl::Config::Color::BLUE_SECOND](#), [ecs::Registry::createEntity\(\)](#), [eng::Color::g](#), [eng::Color::r](#), and [ecs::Registry::EntityBuilder::with\(\)](#).

Referenced by [createStarfield\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.139.3.3 createPlanets()

```
static void ecs::StarfieldSystem::createPlanets (
    Registry & registry,
    const int count,
    const unsigned int screenWidth,
    const unsigned int screenHeight) [inline], [static], [private]
```

Definition at line 157 of file [Starfield.hpp](#).

References [eng::Color::a](#), [eng::Color::b](#), [ecs::Registry::createEntity\(\)](#), [eng::Color::g](#), [utl::Config::Color::PURPLE](#), [eng::Color::r](#), and [ecs::Registry::EntityBuilder::with\(\)](#).

Referenced by [createStarfield\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.139.3.4 createShootingStars()

```
static void ecs::StarfieldSystem::createShootingStars (
    Registry & registry,
    const int count,
```

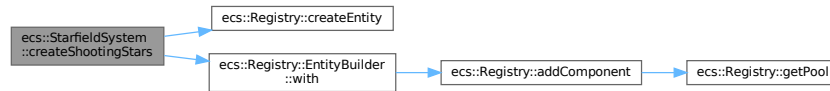
```
const unsigned int screenWidth,
const unsigned int screenHeight) [inline], [static], [private]
```

Definition at line 142 of file [Starfield.hpp](#).

References [eng::Color::a](#), [eng::Color::b](#), [ecs::Registry::createEntity\(\)](#), [eng::Color::g](#), [utl::Config::Color::GREEN](#), [eng::Color::r](#), and [ecs::Registry::EntityBuilder::with\(\)](#).

Referenced by [createStarfield\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.139.3.5 createStarfield()

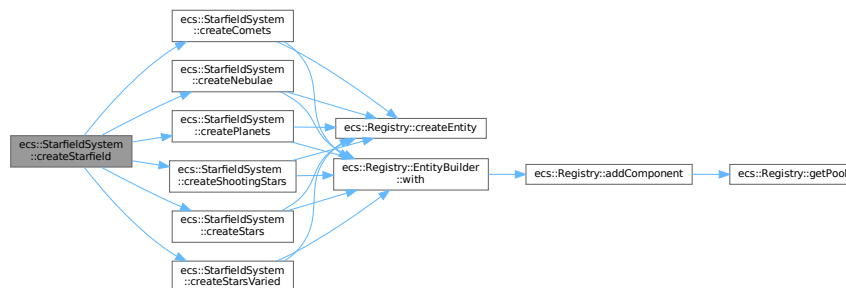
```
static void ecs::StarfieldSystem::createStarfield (
    Registry & registry,
    const eng::WindowSize & windowSize) [inline], [static], [private]
```

Definition at line 112 of file [Starfield.hpp](#).

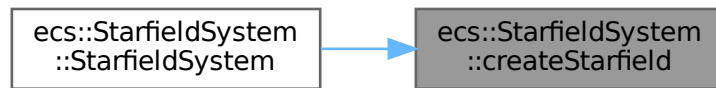
References [utl::Config::Color::BLUE](#), [createComets\(\)](#), [createNebulae\(\)](#), [createPlanets\(\)](#), [createShootingStars\(\)](#), [createStars\(\)](#), [createStarsVaried\(\)](#), [eng::WindowSize::height](#), [utl::Config::Color::WHITE_TRANS](#), [eng::WindowSize::width](#), and [utl::Config::Color::YELLOW](#).

Referenced by [StarfieldSystem\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.139.3.6 createStars()

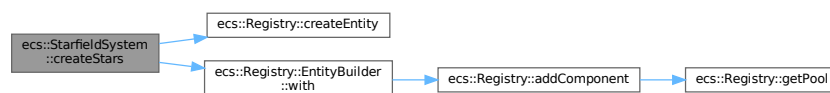
```
static void ecs::StarfieldSystem::createStars (
    Registry & registry,
    const int count,
    const unsigned int screenWidth,
    const unsigned int screenHeight,
    const eng::Color & color,
    float velocity,
    const std::string & id) [inline], [static], [private]
```

Definition at line 127 of file [Starfield.hpp](#).

References [eng::Color::a](#), [eng::Color::b](#), [ecs::Registry::createEntity\(\)](#), [eng::Color::g](#), [eng::Color::r](#), and [ecs::Registry::EntityBuilder::with\(\)](#).

Referenced by [createStarfield\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.139.3.7 createStarsVaried()

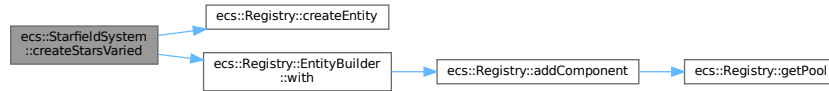
```
static void ecs::StarfieldSystem::createStarsVaried (
    Registry & registry,
    const int count,
    const unsigned int screenWidth,
    const unsigned int screenHeight,
    float velocity,
    const std::string & baseId) [inline], [static], [private]
```

Definition at line 204 of file [Starfield.hpp](#).

References [ecs::Registry::createEntity\(\)](#), and [ecs::Registry::EntityBuilder::with\(\)](#).

Referenced by [createStarfield\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.139.3.8 operator=() [1/2]

[StarfieldSystem](#) & [ecs::StarfieldSystem::operator=](#) (
const [StarfieldSystem](#) &) [delete]

18.139.3.9 operator=() [2/2]

[StarfieldSystem](#) & [ecs::StarfieldSystem::operator=](#) (
[StarfieldSystem](#) &&) [delete]

18.139.3.10 update()

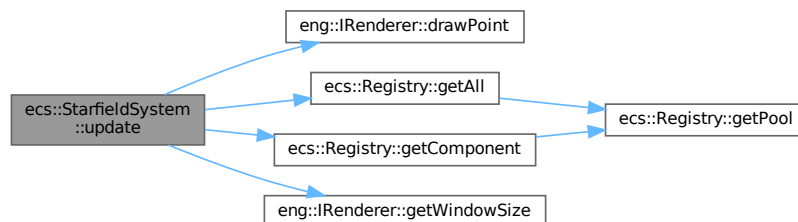
void [ecs::StarfieldSystem::update](#) (
 [Registry](#) & registry,
 const float dt) [inline], [override], [virtual]

Implements [ecs::ISystem](#).

Definition at line 36 of file [Starfield.hpp](#).

References [eng::IRenderer::drawPoint\(\)](#), [ecs::Registry::getAll\(\)](#), [ecs::Registry::getComponent\(\)](#), [eng::IRenderer::getWindowSize\(\)](#), [ecs::IComponent::id](#), [m_renderer](#), [m_timeAccumulator](#), and [ecs::Velocity::x](#).

Here is the call graph for this function:



18.139.4 Member Data Documentation

18.139.4.1 m_renderer

const std::shared_ptr<eng::IRenderer>& ecs::StarfieldSystem::m_renderer [private]

Definition at line 109 of file [Starfield.hpp](#).

Referenced by [StarfieldSystem\(\)](#), and [update\(\)](#).

18.139.4.2 m_timeAccumulator

float ecs::StarfieldSystem::m_timeAccumulator = 0.0f [private]

Definition at line 110 of file [Starfield.hpp](#).

Referenced by [update\(\)](#).

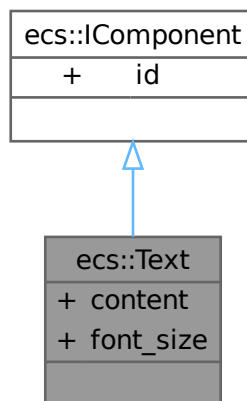
The documentation for this class was generated from the following file:

- [/home/masina/Projects/Epitech/rtype/modules/ECS/include/ECS/Systems/Starfield.hpp](#)

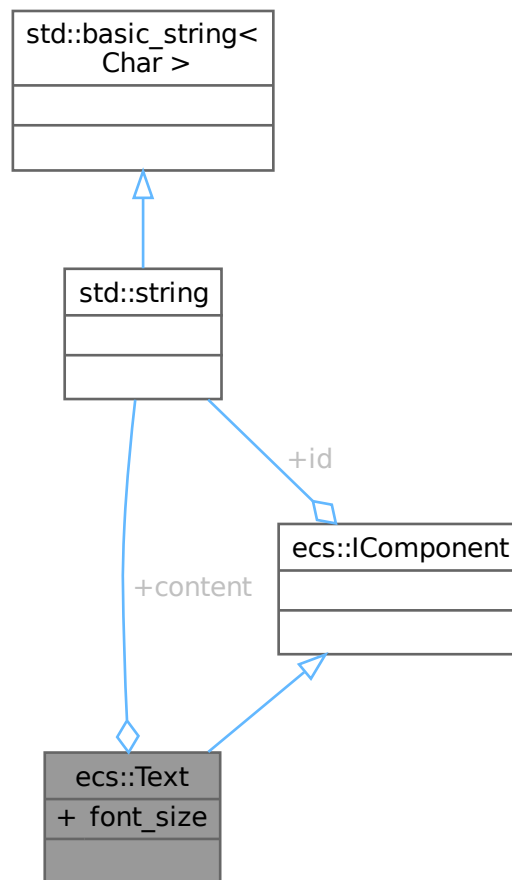
18.140 ecs::Text Struct Reference

#include <Component.hpp>

Inheritance diagram for ecs::Text:



Collaboration diagram for `ecs::Text`:



Public Attributes

- `std::string` [content](#)
- `unsigned int` [font_size](#)

Public Attributes inherited from [ecs::IComponent](#)

- `std::string` [id](#)

18.140.1 Detailed Description

Definition at line 52 of file [Component.hpp](#).

18.140.2 Member Data Documentation

18.140.2.1 content

`std::string` `ecs::Text::content`

Definition at line 54 of file [Component.hpp](#).

Referenced by [gme::WaitingRoomScene::update\(\)](#).

18.140.2.2 font_size

unsigned int ecs::Text::font_size

Definition at line 55 of file [Component.hpp](#).

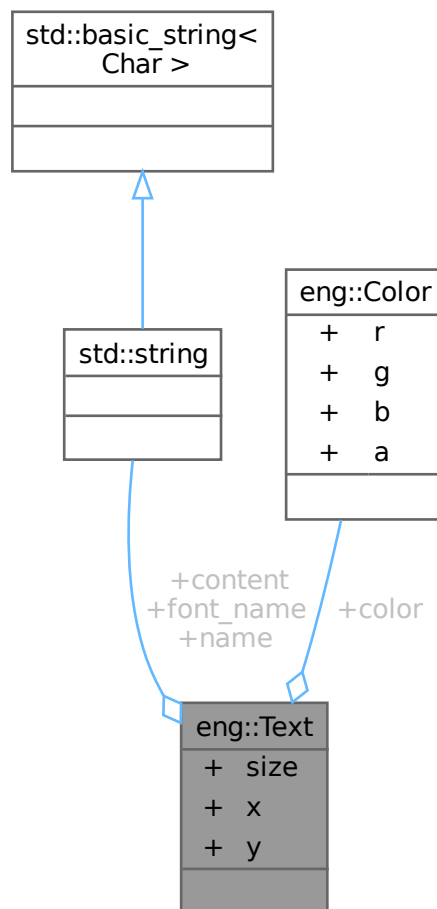
The documentation for this struct was generated from the following file:

- [/home/masina/Projects/Epitech/rtype/modules/ECS/include/ECS/Component.hpp](#)

18.141 eng::Text Struct Reference

#include <IRenderer.hpp>

Collaboration diagram for eng::Text:



Public Attributes

- `std::string` [font_name](#)
- [Color](#) [color](#)
- `std::string` [content](#)
- unsigned int [size](#)
- float [x](#)
- float [y](#)
- `std::string` [name](#)

18.141.1 Detailed Description

Definition at line 22 of file [IRenderer.hpp](#).

18.141.2 Member Data Documentation

18.141.2.1 color

[Color](#) eng::Text::color

Definition at line 25 of file [IRenderer.hpp](#).

18.141.2.2 content

std::string eng::Text::content

Definition at line 26 of file [IRenderer.hpp](#).

18.141.2.3 font_name

std::string eng::Text::font_name

Definition at line 24 of file [IRenderer.hpp](#).

18.141.2.4 name

std::string eng::Text::name

Definition at line 30 of file [IRenderer.hpp](#).

18.141.2.5 size

unsigned int eng::Text::size

Definition at line 27 of file [IRenderer.hpp](#).

18.141.2.6 x

float eng::Text::x

Definition at line 28 of file [IRenderer.hpp](#).

18.141.2.7 y

float eng::Text::y

Definition at line 29 of file [IRenderer.hpp](#).

The documentation for this struct was generated from the following file:

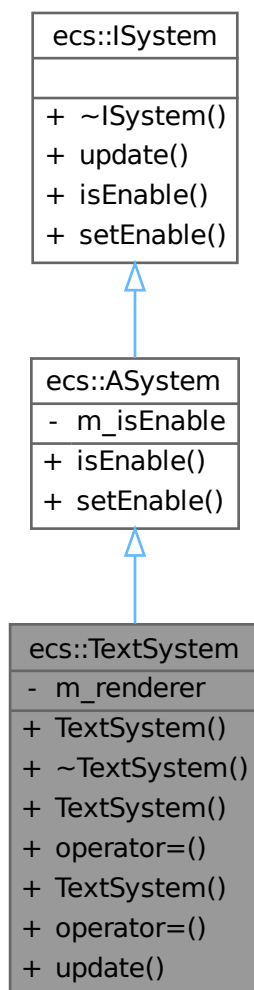
- [/home/masina/Projects/Epitech/rtype/modules/Interfaces/include/Interfaces/IRenderer.hpp](#)

18.142 ecs::TextSystem Class Reference

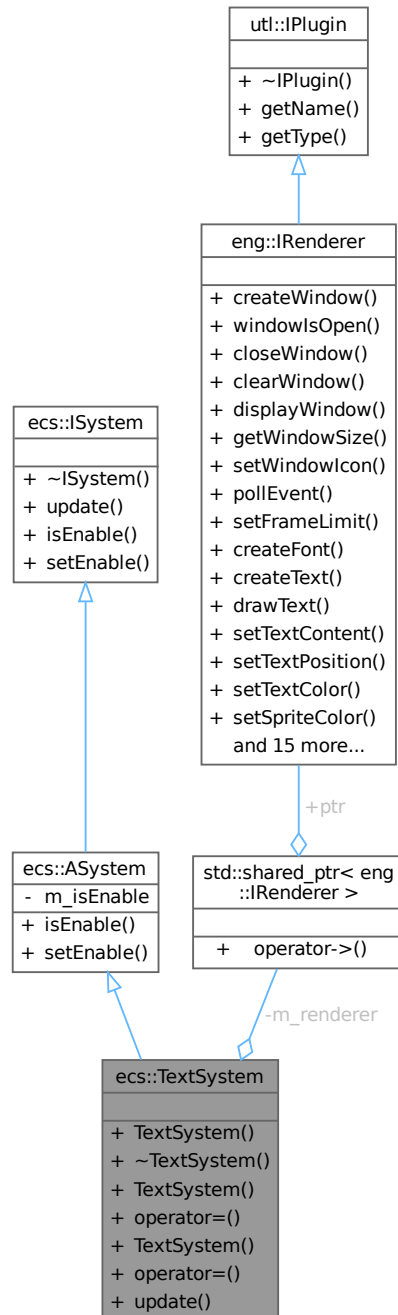
Class for text system.

```
#include <Text.hpp>
```

Inheritance diagram for ecs::TextSystem:



Collaboration diagram for ecs::TextSystem:



Public Member Functions

- `TextSystem` (const std::shared_ptr< eng::IRenderer > &renderer)
- `~TextSystem` () override=default
- `TextSystem` (const `TextSystem` &)=delete
- `TextSystem` & operator= (const `TextSystem` &)=delete
- `TextSystem` (`TextSystem` &&)=delete
- `TextSystem` & operator= (`TextSystem` &&)=delete
- void `update` (`Registry` ®istry, float) override

Public Member Functions inherited from [ecs::ASystem](#)

- bool [isEnabled](#) () override
- void [setEnabled](#) (const bool enable) override

Public Member Functions inherited from [ecs::ISystem](#)

- virtual [~ISystem](#) ()=default

Private Attributes

- const std::shared_ptr< [eng::IRenderer](#) > & [m_renderer](#)

18.142.1 Detailed Description

Class for text system.

Definition at line 21 of file [Text.hpp](#).

18.142.2 Constructor & Destructor Documentation

18.142.2.1 TextSystem() [1/3]

```
ecs::TextSystem::TextSystem (
    const std::shared_ptr< eng::IRenderer > & renderer) [inline], [explicit]
```

Definition at line 24 of file [Text.hpp](#).

18.142.2.2 ~TextSystem()

```
ecs::TextSystem::~TextSystem () [override], [default]
```

18.142.2.3 TextSystem() [2/3]

```
ecs::TextSystem::TextSystem (
    const TextSystem & ) [delete]
```

18.142.2.4 TextSystem() [3/3]

```
ecs::TextSystem::TextSystem (
    TextSystem && ) [delete]
```

18.142.3 Member Function Documentation

18.142.3.1 operator=() [1/2]

```
TextSystem & ecs::TextSystem::operator= (
    const TextSystem & ) [delete]
```

18.142.3.2 operator=() [2/2]

```
TextSystem & ecs::TextSystem::operator= (
    TextSystem && ) [delete]
```

18.142.3.3 update()

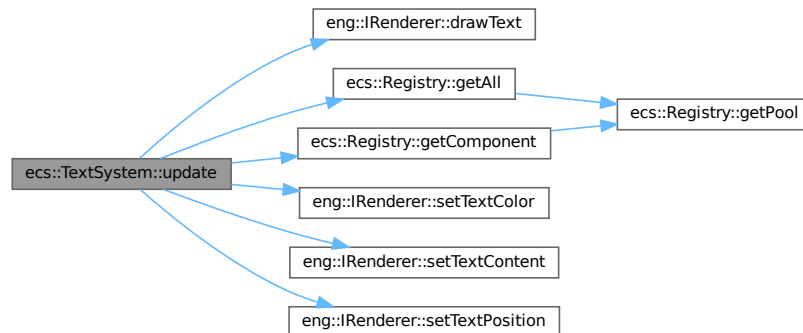
```
void ecs::TextSystem::update (
    Registry & registry,
    float ) [inline], [override], [virtual]
```

Implements [ecs::ISystem](#).

Definition at line 32 of file [Text.hpp](#).

References [eng::IRenderer::drawText\(\)](#), [ecs::Registry::getAll\(\)](#), [ecs::Registry::getComponent\(\)](#), [m_renderer](#), [eng::IRenderer::setTextColor\(\)](#), [eng::IRenderer::setTextContent\(\)](#), and [eng::IRenderer::setTextPosition\(\)](#).

Here is the call graph for this function:



18.142.4 Member Data Documentation

18.142.4.1 m_renderer

`const std::shared_ptr<eng::IRenderer>& ecs::TextSystem::m_renderer` [private]

Definition at line 56 of file [Text.hpp](#).

Referenced by [update\(\)](#).

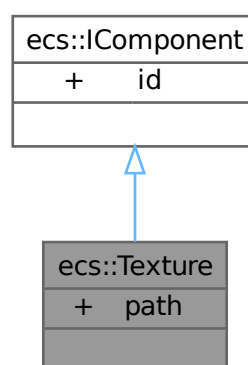
The documentation for this class was generated from the following file:

- [/home/masina/Projects/Epitech/rtype/modules/ECS/include/ECS/Systems/Text.hpp](#)

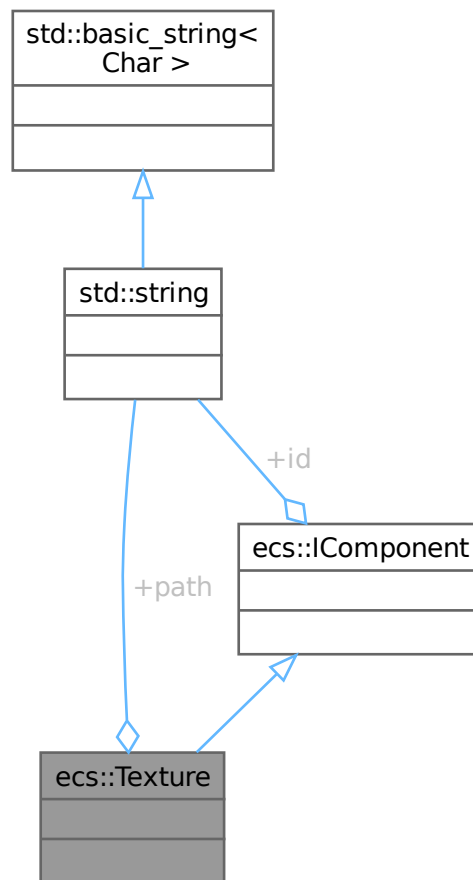
18.143 ecs::Texture Struct Reference

`#include <Component.hpp>`

Inheritance diagram for `ecs::Texture`:



Collaboration diagram for `ecs::Texture`:



Public Attributes

- `std::string` [path](#)

Public Attributes inherited from `ecs::IComponent`

- `std::string` [id](#)

18.143.1 Detailed Description

Definition at line 57 of file [Component.hpp](#).

18.143.2 Member Data Documentation

18.143.2.1 `path`

`std::string` `ecs::Texture::path`

Definition at line 59 of file [Component.hpp](#).

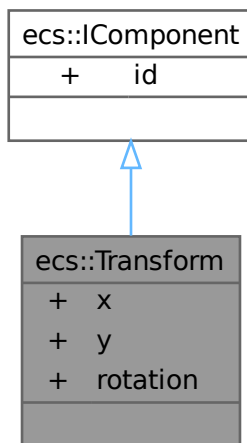
The documentation for this struct was generated from the following file:

- `/home/masina/Projects/Epitech/rtype/modules/ECS/include/ECS/Component.hpp`

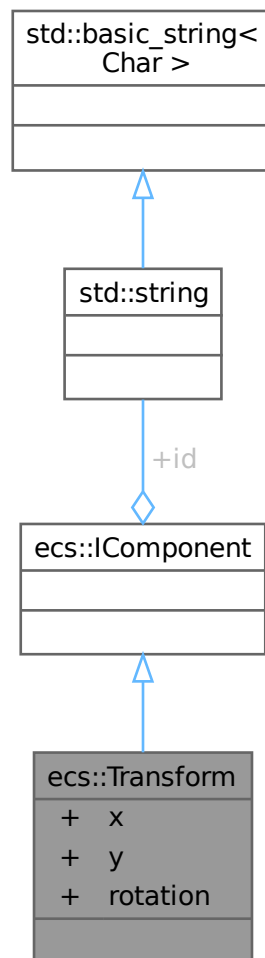
18.144 ecs::Transform Struct Reference

#include <Component.hpp>

Inheritance diagram for ecs::Transform:



Collaboration diagram for `ecs::Transform`:



Public Attributes

- float `x` {}
- float `y` {}
- float `rotation` {}

Public Attributes inherited from `ecs::IComponent`

- `std::string id`

18.144.1 Detailed Description

Definition at line 63 of file [Component.hpp](#).

18.144.2 Member Data Documentation

18.144.2.1 rotation

float `ecs::Transform::rotation` {}

Definition at line 66 of file [Component.hpp](#).

18.144.2.2 x

```
float ecs::Transform::x {}
```

Definition at line 65 of file [Component.hpp](#).

Referenced by [gme::EnemyAISystem::applyAggressiveMovement\(\)](#), [gme::EnemyAISystem::applySineWaveMovement\(\)](#), [gme::EnemyAISystem::applyZigzagMovement\(\)](#), [gme::CollisionSystem::checkCircularCollision\(\)](#), [gme::EnemyAISystem::c](#), [gme::EnemyAISystem::findNearestPlayer\(\)](#), [gme::Game::Game\(\)](#), [gme::GameMulti::GameMulti\(\)](#), [gme::GameSolo::GameSolo\(\)](#), [gme::EntityManager::getAllEntityStates\(\)](#), [gme::EnemyAISystem::getDistanceToNearestPl](#), [gme::GameSolo::handlePlayerInputs\(\)](#), [gme::Game::resetGame\(\)](#), [gme::WeaponSystem::showLoadingAnimation\(\)](#), [gme::EnemyAISystem::tryShoot\(\)](#), [ecs::BeamSystem::update\(\)](#), [gme::BeamSystem::update\(\)](#), [gme::EnemySystem::update](#), [gme::Game::update\(\)](#), [gme::ProjectileSystem::update\(\)](#), [gme::ScrollingSystem::update\(\)](#), [gme::EnemyAISystem::updateBo](#), [gme::RTypeServer::updateEntities\(\)](#), and [gme::HUDSystem::updateScore\(\)](#).

18.144.2.3 y

```
float ecs::Transform::y {}
```

Definition at line 65 of file [Component.hpp](#).

Referenced by [gme::EnemyAISystem::applyAggressiveMovement\(\)](#), [gme::EnemyAISystem::applySineWaveMovement\(\)](#), [gme::EnemyAISystem::applyZigzagMovement\(\)](#), [gme::CollisionSystem::checkCircularCollision\(\)](#), [gme::Game::Game\(\)](#), [gme::GameMulti::GameMulti\(\)](#), [gme::GameSolo::GameSolo\(\)](#), [gme::EntityManager::getAllEntityStates\(\)](#), [gme::CollisionSystem::getCollisionInfo\(\)](#), [gme::GameSolo::handlePlayerInputs\(\)](#), [gme::WeaponSystem::showLoadingAnim](#), [ecs::DebugSystem::update\(\)](#), [gme::CollisionSystem::update\(\)](#), [gme::EnemySystem::update\(\)](#), [gme::Game::update\(\)](#), [gme::ScrollingSystem::update\(\)](#), and [gme::RTypeServer::updateEntities\(\)](#).

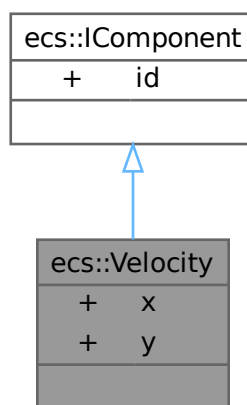
The documentation for this struct was generated from the following file:

- [/home/masina/Projects/Epitech/rtype/modules/ECS/include/ECS/Component.hpp](#)

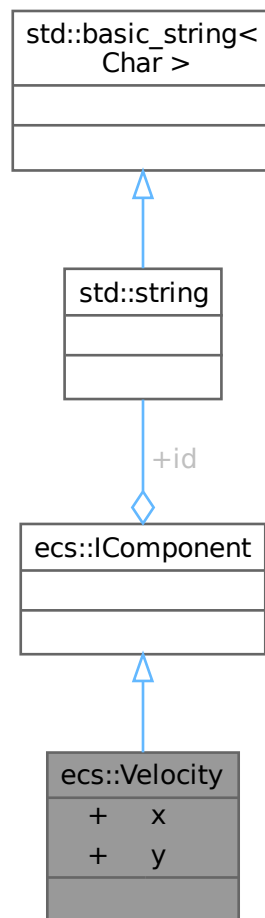
18.145 ecs::Velocity Struct Reference

```
#include <Component.hpp>
```

Inheritance diagram for `ecs::Velocity`:



Collaboration diagram for `ecs::Velocity`:



Public Attributes

- float `x` {}
- float `y` {}

Public Attributes inherited from `ecs::IComponent`

- `std::string id`

18.145.1 Detailed Description

Definition at line 68 of file [Component.hpp](#).

18.145.2 Member Data Documentation

18.145.2.1 `x`

float `ecs::Velocity::x` {}

Definition at line 70 of file [Component.hpp](#).

Referenced by [ecs::StarfieldSystem::update\(\)](#), [gme::Game::update\(\)](#), [gme::EnemyAISystem::updateBasicEnemyAI\(\)](#), and [gme::RTypeServer::updateEntities\(\)](#).

18.145.2.2 y

float ecs::Velocity::y {}

Definition at line 70 of file [Component.hpp](#).

Referenced by [gme::CollisionSystem::update\(\)](#).

The documentation for this struct was generated from the following file:

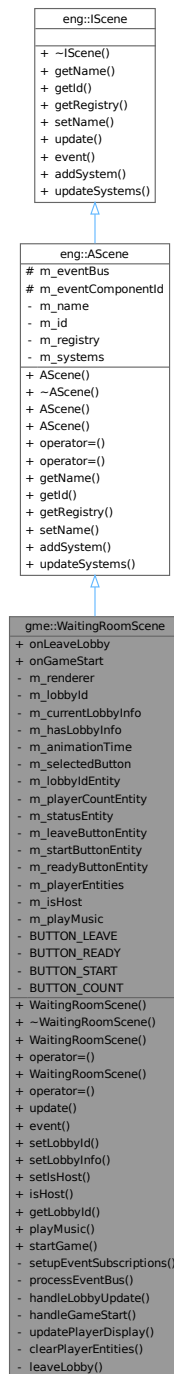
- [/home/masina/Projects/Epitech/rtype/modules/ECS/include/ECS/Component.hpp](#)

18.146 gme::WaitingRoomScene Class Reference

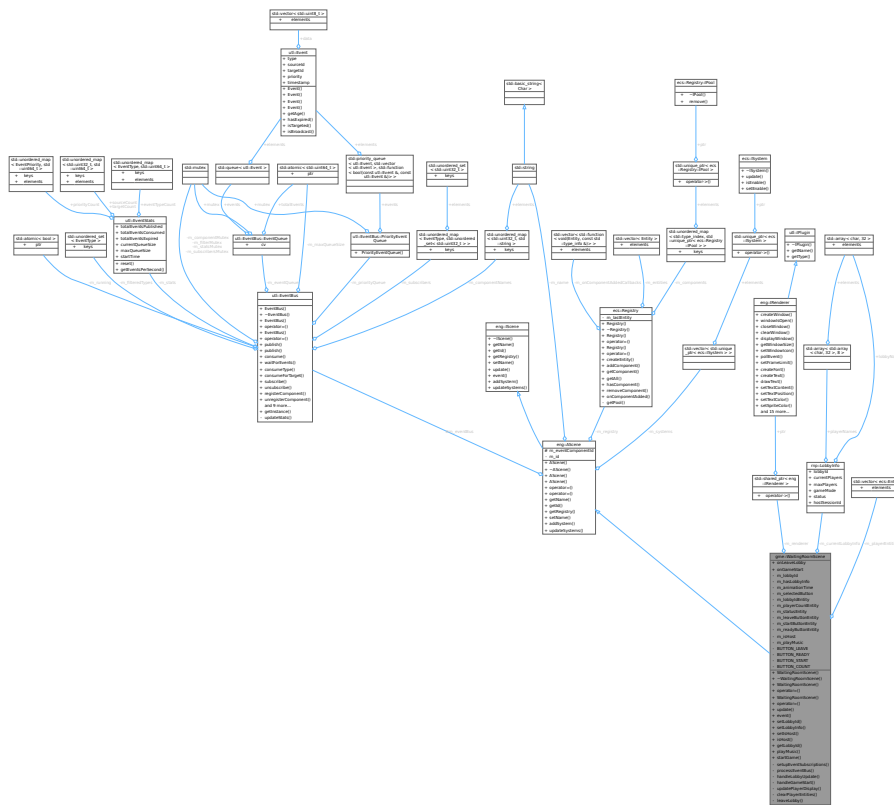
Waiting room scene where players wait before game starts.

#include <WaitingRoom.hpp>

Inheritance diagram for gme::WaitingRoomScene:



Collaboration diagram for gme::WaitingRoomScene:



Public Member Functions

- `WaitingRoomScene (eng::id assignedId, const std::shared_ptr< eng::IRenderer > &renderer)`
- `~WaitingRoomScene ()` override=default
- `WaitingRoomScene (const WaitingRoomScene &other)=delete`
- `WaitingRoomScene & operator= (const WaitingRoomScene &other)=delete`
- `WaitingRoomScene (WaitingRoomScene &&other)=delete`
- `WaitingRoomScene & operator= (WaitingRoomScene &&other)=delete`
- `void update (float dt, const eng::WindowSize &size)` override
- `void event (const eng::Event &event)` override
- `void setLobbyId (std::uint32_t lobbyId)`
- `void setLobbyInfo (const rnp::LobbyInfo &lobbyInfo)`
- `void setIsHost (const bool isHost)`
- `bool isHost ()` const
- `std::uint32_t getLobbyId ()` const
- `bool & playMusic ()`
- `void startGame ()`

Public Member Functions inherited from `eng::AScene`

- `AScene` (const `id` assignedId)
- `~AScene` () override=default
- `AScene` (const `AScene` &other)=delete
- `AScene` (`AScene` &&other)=delete
- `AScene` & `operator=` (const `AScene` &other)=delete
- `AScene` & `operator=` (`AScene` &&other)=delete
- `std::string` & `getName` () override

- [id getId \(\)](#) const override
- [ecs::Registry & getRegistry \(\)](#) override
- void [setName](#) (const std::string &newName) override
- void [addSystem](#) (std::unique_ptr< [ecs::ISystem](#) > system) override
- void [updateSystems](#) (const float dt) override

Public Member Functions inherited from [eng::IScene](#)

- virtual [~IScene \(\)](#)=default

Public Attributes

- std::function< void()> [onLeaveLobby](#)
- std::function< void()> [onGameStart](#)

Private Member Functions

- void [setupEventSubscriptions \(\)](#) const
- void [processEventBus \(\)](#)
- void [handleLobbyUpdate](#) (const [utl::Event](#) &event)
- void [handleGameStart](#) (const [utl::Event](#) &event) const
- void [updatePlayerDisplay \(\)](#)
- void [clearPlayerEntities \(\)](#)
- void [leaveLobby \(\)](#) const

Private Attributes

- const std::shared_ptr< [eng::IRenderer](#) > & [m_renderer](#)
- std::uint32_t [m_lobbyId](#) = 0
- [rnp::LobbyInfo](#) [m_currentLobbyInfo](#) {}
- bool [m_hasLobbyInfo](#) = false
- float [m_animationTime](#) = 0.0f
- int [m_selectedButton](#) = 0
- [ecs::Entity](#) [m_lobbyIdEntity](#) = 0
- [ecs::Entity](#) [m_playerCountEntity](#) = 0
- [ecs::Entity](#) [m_statusEntity](#) = 0
- [ecs::Entity](#) [m_leaveButtonEntity](#) = 0
- [ecs::Entity](#) [m_startButtonEntity](#) = 0
- [ecs::Entity](#) [m_readyButtonEntity](#) = 0
- std::vector< [ecs::Entity](#) > [m_playerEntities](#)
- bool [m_isHost](#) = false
- bool [m_playMusic](#) = false

Static Private Attributes

- static constexpr int [BUTTON_LEAVE](#) = 0
- static constexpr int [BUTTON_READY](#) = 1
- static constexpr int [BUTTON_START](#) = 2
- static constexpr int [BUTTON_COUNT](#) = 3

Additional Inherited Members

Protected Attributes inherited from [eng::AScene](#)

- [utl::EventBus](#) & [m_eventBus](#) = [utl::EventBus::getInstance\(\)](#)
- std::uint32_t [m_eventComponentId](#) = 0

18.146.1 Detailed Description

Waiting room scene where players wait before game starts.

Definition at line 22 of file [WaitingRoom.hpp](#).

18.146.2 Constructor & Destructor Documentation

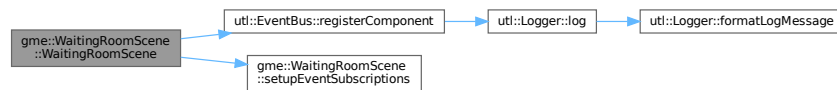
18.146.2.1 WaitingRoomScene() [1/3]

```
gme::WaitingRoomScene::WaitingRoomScene (
    eng::id assignedId,
    const std::shared_ptr< eng::IRenderer > & renderer)
```

Definition at line 9 of file [waitingRoom.cpp](#).

References [eng::Color::a](#), [eng::Color::b](#), [utl::Config::Color::CYAN_ELECTRIC](#), [utl::Path::Font::FONTS_RTYPE](#), [eng::Color::g](#), [utl::Config::Color::GRAY_BLUE_SUBTLE](#), [ecs::IComponent::id](#), [utl::Config::Color::INFO_TEXT_COLOR](#), [eng::AScene::m_eventBus](#), [eng::AScene::m_eventComponentId](#), [m_leaveButtonEntity](#), [m_lobbyIdEntity](#), [m_playerCountEntity](#), [m_readyButtonEntity](#), [m_startButtonEntity](#), [m_statusEntity](#), [eng::Color::r](#), [utl::EventBus::registerComponent\(\)](#), [setupEventSubscriptions\(\)](#), and [utl::Config::Color::TEXT_VALUE_COLOR](#).

Here is the call graph for this function:



18.146.2.2 ~WaitingRoomScene()

```
gme::WaitingRoomScene::~~WaitingRoomScene () [override], [default]
```

18.146.2.3 WaitingRoomScene() [2/3]

```
gme::WaitingRoomScene::WaitingRoomScene (
    const WaitingRoomScene & other) [delete]
```

18.146.2.4 WaitingRoomScene() [3/3]

```
gme::WaitingRoomScene::WaitingRoomScene (
    WaitingRoomScene && other) [delete]
```

18.146.3 Member Function Documentation

18.146.3.1 clearPlayerEntities()

```
void gme::WaitingRoomScene::clearPlayerEntities () [private]
```

Definition at line 376 of file [waitingRoom.cpp](#).

18.146.3.2 event()

```
void gme::WaitingRoomScene::event (
    const eng::Event & event) [override], [virtual]
```

Implements [eng::IScene](#).

Definition at line 231 of file [waitingRoom.cpp](#).

References [eng::Down](#), [eng::Enter](#), [eng::Escape](#), [utl::INFO](#), [eng::Event::key](#), [eng::KeyPressed](#), [utl::Logger::log\(\)](#), [eng::Space](#), [eng::Event::type](#), and [eng::Up](#).

Here is the call graph for this function:



18.146.3.3 getLobbyId()

`std::uint32_t gme::WaitingRoomScene::getLobbyId () const [inline]`

Definition at line 40 of file [WaitingRoom.hpp](#).

References [m_lobbyId](#).

18.146.3.4 handleGameStart()

`void gme::WaitingRoomScene::handleGameStart (const utl::Event & event) const [private]`

Definition at line 142 of file [waitingRoom.cpp](#).

References [utl::Event::data](#), [rnp::Serializer::deserializeGameStart\(\)](#), [utl::INFO](#), [rnp::PacketGameStart::lobbyId](#), [utl::Logger::log\(\)](#), and [utl::WARNING](#).

Here is the call graph for this function:



18.146.3.5 handleLobbyUpdate()

`void gme::WaitingRoomScene::handleLobbyUpdate (const utl::Event & event) [private]`

Definition at line 118 of file [waitingRoom.cpp](#).

References [rnp::LobbyInfo::currentPlayers](#), [utl::Event::data](#), [rnp::Serializer::deserializeLobbyUpdate\(\)](#), [utl::INFO](#), [rnp::LobbyInfo::lobbyId](#), [rnp::PacketLobbyUpdate::lobbyInfo](#), [utl::Logger::log\(\)](#), [rnp::LobbyInfo::maxPlayers](#), and [utl::WARNING](#).

Here is the call graph for this function:



18.146.3.6 isHost()

`bool gme::WaitingRoomScene::isHost () const [inline], [nodiscard]`

Definition at line 39 of file [WaitingRoom.hpp](#).

References [m_isHost](#).

Referenced by [setIsHost\(\)](#).

Here is the caller graph for this function:



18.146.3.7 leaveLobby()

void gme::WaitingRoomScene::leaveLobby () const [private]

Definition at line 401 of file [waitingRoom.cpp](#).

References [utl::INFO](#), [utl::LOBBY_LEAVE](#), [utl::Logger::log\(\)](#), and [utl::NETWORK_CLIENT](#).

Here is the call graph for this function:



18.146.3.8 operator=() [1/2]

[WaitingRoomScene](#) & gme::WaitingRoomScene::operator= (const [WaitingRoomScene](#) & other) [delete]

18.146.3.9 operator=() [2/2]

[WaitingRoomScene](#) & gme::WaitingRoomScene::operator= ([WaitingRoomScene](#) && other) [delete]

18.146.3.10 playMusic()

bool & gme::WaitingRoomScene::playMusic () [inline]

Definition at line 42 of file [WaitingRoom.hpp](#).

References [m_playMusic](#).

18.146.3.11 processEventBus()

void gme::WaitingRoomScene::processEventBus () [private]

Definition at line 431 of file [waitingRoom.cpp](#).

References [utl::GAME_START](#), [utl::INFO](#), [utl::LOBBY_UPDATE](#), and [utl::Logger::log\(\)](#).

Here is the call graph for this function:



18.146.3.12 setIsHost()

```
void gme::WaitingRoomScene::setIsHost (
    const bool isHost) [inline]
```

Definition at line 38 of file [WaitingRoom.hpp](#).

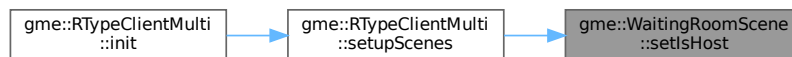
References [isHost\(\)](#), and [m_isHost](#).

Referenced by [gme::RTypeClientMulti::setupScenes\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.146.3.13 setLobbyId()

```
void gme::WaitingRoomScene::setLobbyId (
    std::uint32_t lobbyId)
```

Definition at line 284 of file [waitingRoom.cpp](#).

References [utl::INFO](#), and [utl::Logger::log\(\)](#).

Referenced by [gme::RTypeClientMulti::setupScenes\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.146.3.14 setLobbyInfo()

```
void gme::WaitingRoomScene::setLobbyInfo (
    const rnp::LobbyInfo & lobbyInfo)
```

Definition at line 295 of file [waitingRoom.cpp](#).

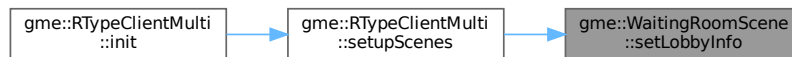
References [rnp::LobbyInfo::currentPlayers](#), [utl::INFO](#), [rnp::LobbyInfo::lobbyId](#), [utl::Logger::log\(\)](#), and [rnp::LobbyInfo::maxPlayers](#).

Referenced by [gme::RTypeClientMulti::setupScenes\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.146.3.15 setupEventSubscriptions()

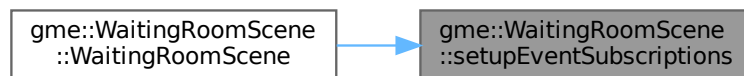
```
void gme::WaitingRoomScene::setupEventSubscriptions () const [private]
```

Definition at line 112 of file [waitingRoom.cpp](#).

References [utl::GAME_START](#), and [utl::LOBBY_UPDATE](#).

Referenced by [WaitingRoomScene\(\)](#).

Here is the caller graph for this function:



18.146.3.16 startGame()

```
void gme::WaitingRoomScene::startGame ()
```

Definition at line 414 of file [waitingRoom.cpp](#).

References [utl::INFO](#), [utl::LOBBY_CREATE](#), [utl::Logger::log\(\)](#), [utl::NETWORK_CLIENT](#), and [utl::WARNING](#).

Here is the call graph for this function:



18.146.3.17 update()

```
void gme::WaitingRoomScene::update (
    float dt,
    const eng::WindowSize & size) [override], [virtual]
```

Implements [eng::IScene](#).

Definition at line 164 of file [waitingRoom.cpp](#).

References [ecs::Color::b](#), [eng::Color::b](#), [ecs::Text::content](#), [utl::Config::Color::CYAN_ELECTRIC](#), [ecs::Color::g](#), [eng::Color::g](#), [utl::Config::Color::GRAY_BLUE_SUBTLE](#), [ecs::Color::r](#), and [eng::Color::r](#).

18.146.3.18 updatePlayerDisplay()

```
void gme::WaitingRoomScene::updatePlayerDisplay () [private]
```

Definition at line 306 of file [waitingRoom.cpp](#).

References [utl::Path::Font::FONT_S_RTYPE](#), [utl::Config::Color::GREEN_READY](#), [utl::INFO](#), [utl::Config::Color::INFO_TEXT_COLOR](#), and [utl::Logger::log\(\)](#).

Here is the call graph for this function:



18.146.4 Member Data Documentation

18.146.4.1 BUTTON_COUNT

```
int gme::WaitingRoomScene::BUTTON_COUNT = 3 [static], [constexpr], [private]
```

Definition at line 60 of file [WaitingRoom.hpp](#).

18.146.4.2 BUTTON_LEAVE

```
int gme::WaitingRoomScene::BUTTON_LEAVE = 0 [static], [constexpr], [private]
```

Definition at line 57 of file [WaitingRoom.hpp](#).

18.146.4.3 BUTTON_READY

```
int gme::WaitingRoomScene::BUTTON_READY = 1 [static], [constexpr], [private]
```

Definition at line 58 of file [WaitingRoom.hpp](#).

18.146.4.4 BUTTON_START

```
int gme::WaitingRoomScene::BUTTON_START = 2 [static], [constexpr], [private]
```

Definition at line 59 of file [WaitingRoom.hpp](#).

18.146.4.5 m_animationTime

float gme::WaitingRoomScene::m_animationTime = 0.0f [private]

Definition at line 54 of file [WaitingRoom.hpp](#).

18.146.4.6 m_currentLobbyInfo

[nnp::LobbyInfo](#) gme::WaitingRoomScene::m_currentLobbyInfo {} [private]

Definition at line 52 of file [WaitingRoom.hpp](#).

18.146.4.7 m_hasLobbyInfo

bool gme::WaitingRoomScene::m_hasLobbyInfo = false [private]

Definition at line 53 of file [WaitingRoom.hpp](#).

18.146.4.8 m_isHost

bool gme::WaitingRoomScene::m_isHost = false [private]

Definition at line 71 of file [WaitingRoom.hpp](#).

Referenced by [isHost\(\)](#), and [setIsHost\(\)](#).

18.146.4.9 m_leaveButtonEntity

[ecs::Entity](#) gme::WaitingRoomScene::m_leaveButtonEntity = 0 [private]

Definition at line 66 of file [WaitingRoom.hpp](#).

Referenced by [WaitingRoomScene\(\)](#).

18.146.4.10 m_lobbyId

std::uint32_t gme::WaitingRoomScene::m_lobbyId = 0 [private]

Definition at line 51 of file [WaitingRoom.hpp](#).

Referenced by [getLobbyId\(\)](#).

18.146.4.11 m_lobbyIdEntity

[ecs::Entity](#) gme::WaitingRoomScene::m_lobbyIdEntity = 0 [private]

Definition at line 63 of file [WaitingRoom.hpp](#).

Referenced by [WaitingRoomScene\(\)](#).

18.146.4.12 m_playerCountEntity

[ecs::Entity](#) gme::WaitingRoomScene::m_playerCountEntity = 0 [private]

Definition at line 64 of file [WaitingRoom.hpp](#).

Referenced by [WaitingRoomScene\(\)](#).

18.146.4.13 m_playerEntities

std::vector<[ecs::Entity](#)> gme::WaitingRoomScene::m_playerEntities [private]

Definition at line 69 of file [WaitingRoom.hpp](#).

18.146.4.14 m_playMusic

bool gme::WaitingRoomScene::m_playMusic = false [private]

Definition at line 72 of file [WaitingRoom.hpp](#).

Referenced by [playMusic\(\)](#).

18.146.4.15 m_readyButtonEntity

[ecs::Entity](#) gme::WaitingRoomScene::m_readyButtonEntity = 0 [private]

Definition at line 68 of file [WaitingRoom.hpp](#).

Referenced by [WaitingRoomScene\(\)](#).

18.146.4.16 m_renderer

const std::shared_ptr<[eng::IRenderer](#)>& gme::WaitingRoomScene::m_renderer [private]
 Definition at line 49 of file [WaitingRoom.hpp](#).

18.146.4.17 m_selectedButton

int gme::WaitingRoomScene::m_selectedButton = 0 [private]
 Definition at line 56 of file [WaitingRoom.hpp](#).

18.146.4.18 m_startButtonEntity

[ecs::Entity](#) gme::WaitingRoomScene::m_startButtonEntity = 0 [private]
 Definition at line 67 of file [WaitingRoom.hpp](#).
 Referenced by [WaitingRoomScene\(\)](#).

18.146.4.19 m_statusEntity

[ecs::Entity](#) gme::WaitingRoomScene::m_statusEntity = 0 [private]
 Definition at line 65 of file [WaitingRoom.hpp](#).
 Referenced by [WaitingRoomScene\(\)](#).

18.146.4.20 onGameStart

std::function<void()> gme::WaitingRoomScene::onGameStart
 Definition at line 45 of file [WaitingRoom.hpp](#).

18.146.4.21 onLeaveLobby

std::function<void()> gme::WaitingRoomScene::onLeaveLobby
 Definition at line 44 of file [WaitingRoom.hpp](#).

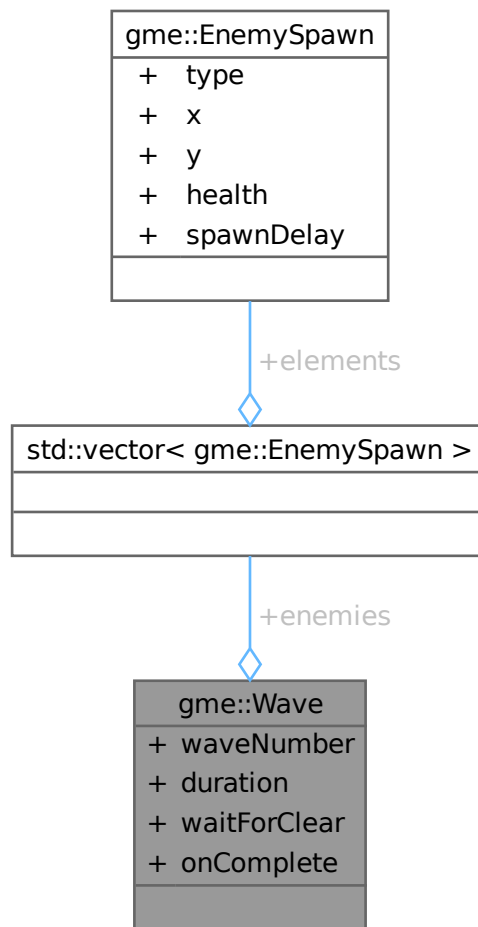
The documentation for this class was generated from the following files:

- /home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/include/RType↵
 ClientMulti/Scenes/[WaitingRoom.hpp](#)
- /home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/src/scenes/[waitingRoom.cpp](#)

18.147 gme::Wave Struct Reference

[Wave](#) definition containing multiple enemy spawns.
 #include <WaveManager.hpp>

Collaboration diagram for gme::Wave:



Public Attributes

- int [waveNumber](#)
- std::vector< [EnemySpawn](#) > [enemies](#)
- float [duration](#)
- bool [waitForClear](#)
- std::function< void()> [onComplete](#)

18.147.1 Detailed Description

[Wave](#) definition containing multiple enemy spawns.
Definition at line 30 of file [WaveManager.hpp](#).

18.147.2 Member Data Documentation

18.147.2.1 duration

float gme::Wave::duration

Definition at line 34 of file [WaveManager.hpp](#).

Referenced by [gme::WaveManager::createBossWave\(\)](#), [gme::WaveManager::createWave1\(\)](#), [gme::WaveManager::createWave2\(\)](#), [gme::WaveManager::createWave3\(\)](#), [gme::WaveManager::createWave4\(\)](#), and [gme::WaveManager::update\(\)](#).

18.147.2.2 enemies

`std::vector<EnemySpawn> gme::Wave::enemies`

Definition at line 33 of file [WaveManager.hpp](#).

Referenced by [gme::WaveManager::createBossWave\(\)](#), [gme::WaveManager::createWave1\(\)](#), [gme::WaveManager::createWave2\(\)](#), [gme::WaveManager::createWave3\(\)](#), [gme::WaveManager::createWave4\(\)](#), [gme::WaveManager::processSpawns\(\)](#), and [gme::WaveManager::startNextWave\(\)](#).

18.147.2.3 onComplete

`std::function<void()> gme::Wave::onComplete`

Definition at line 36 of file [WaveManager.hpp](#).

Referenced by [gme::WaveManager::update\(\)](#).

18.147.2.4 waitForClear

`bool gme::Wave::waitForClear`

Definition at line 35 of file [WaveManager.hpp](#).

Referenced by [gme::WaveManager::createBossWave\(\)](#), [gme::WaveManager::createWave1\(\)](#), [gme::WaveManager::createWave2\(\)](#), [gme::WaveManager::createWave3\(\)](#), [gme::WaveManager::createWave4\(\)](#), and [gme::WaveManager::update\(\)](#).

18.147.2.5 waveNumber

`int gme::Wave::waveNumber`

Definition at line 32 of file [WaveManager.hpp](#).

Referenced by [gme::WaveManager::createBossWave\(\)](#), [gme::WaveManager::createWave1\(\)](#), [gme::WaveManager::createWave2\(\)](#), [gme::WaveManager::createWave3\(\)](#), and [gme::WaveManager::createWave4\(\)](#).

The documentation for this struct was generated from the following file:

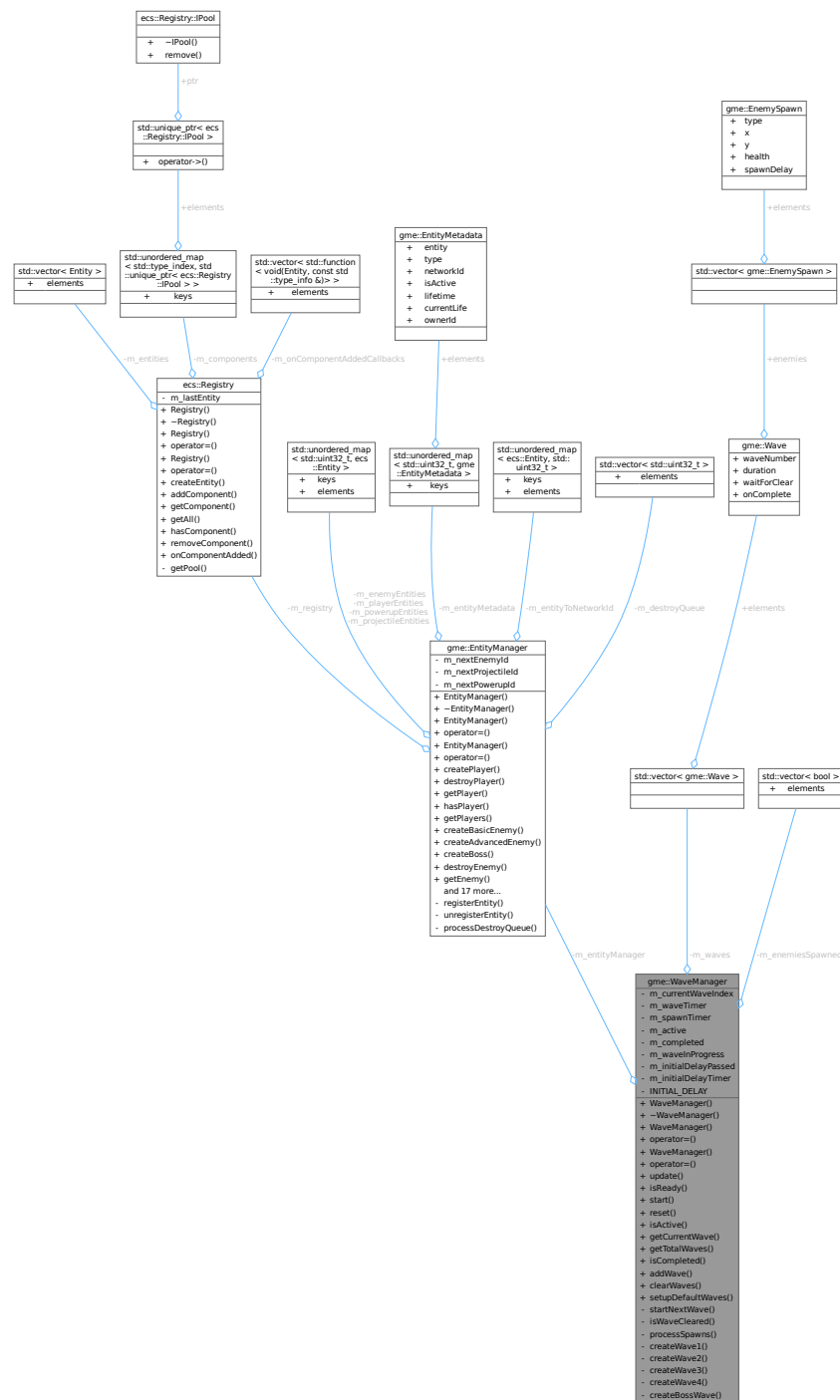
- [/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Server/include/RTypeServer/WaveManager.hpp](#)

18.148 gme::WaveManager Class Reference

Manages wave-based enemy spawning with configurable patterns.

`#include <WaveManager.hpp>`

Collaboration diagram for gme::WaveManager:



Public Member Functions

- [WaveManager](#) ([EntityManager](#) &entityManager)
- [~WaveManager](#) ()=default
- [WaveManager](#) (const [WaveManager](#) &)=delete
- [WaveManager](#) & operator= (const [WaveManager](#) &)=delete
- [WaveManager](#) ([WaveManager](#) &&)=delete
- [WaveManager](#) & operator= ([WaveManager](#) &&)=delete

- void `update` (`ecs::Registry` ®istry, float dt, int screenWidth)
 - Update wave manager logic.
- bool `isReady` () const
 - Check if initial delay has passed and waves can start.
- void `start` ()
 - Start the wave system.
- void `reset` ()
 - Stop and reset the wave system.
- bool `isActive` () const
 - Check if wave system is active.
- int `getCurrentWave` () const
 - Get current wave number.
- size_t `getTotalWaves` () const
 - Get total number of waves.
- bool `isCompleted` () const
 - Check if all waves are completed.
- void `addWave` (const `Wave` &wave)
 - Add a custom wave.
- void `clearWaves` ()
 - Clear all waves.
- void `setupDefaultWaves` ()
 - Initialize with default wave patterns.

Private Member Functions

- void `startNextWave` ()
 - Start the next wave.
- bool `isWaveCleared` () const
 - Check if current wave is cleared.
- void `processSpawns` (float dt, int screenWidth)
 - Process enemy spawns for current wave.
- void `createWave1` ()
 - Create default wave patterns (5 waves: 4x30s + boss 60s)
- void `createWave2` ()
- void `createWave3` ()
- void `createWave4` ()
- void `createBossWave` ()

Private Attributes

- `EntityManager` & `m_entityManager`
- std::vector< `Wave` > `m_waves`
- int `m_currentWaveIndex`
- float `m_waveTimer`
- float `m_spawnTimer`
- bool `m_active`
- bool `m_completed`
- bool `m_waveInProgress`
- bool `m_initialDelayPassed`
- float `m_initialDelayTimer`
- std::vector< bool > `m_enemiesSpawned`

Static Private Attributes

- static constexpr float [INITIAL_DELAY](#) = 0.5f

18.148.1 Detailed Description

Manages wave-based enemy spawning with configurable patterns.

Definition at line 44 of file [WaveManager.hpp](#).

18.148.2 Constructor & Destructor Documentation

18.148.2.1 WaveManager() [1/3]

`gme::WaveManager::WaveManager (
 EntityManager & entityManager)` [explicit]

Definition at line 13 of file [WaveManager.cpp](#).

18.148.2.2 ~WaveManager()

`gme::WaveManager::~WaveManager ()` [default]

18.148.2.3 WaveManager() [2/3]

`gme::WaveManager::WaveManager (
 const WaveManager &)` [delete]

18.148.2.4 WaveManager() [3/3]

`gme::WaveManager::WaveManager (
 WaveManager &&)` [delete]

18.148.3 Member Function Documentation

18.148.3.1 addWave()

`void gme::WaveManager::addWave (
 const Wave & wave)`

Add a custom wave.

Definition at line 134 of file [WaveManager.cpp](#).

References [m_waves](#).

18.148.3.2 clearWaves()

`void gme::WaveManager::clearWaves ()`

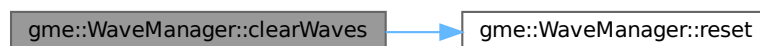
Clear all waves.

Definition at line 136 of file [WaveManager.cpp](#).

References [m_waves](#), and [reset\(\)](#).

Referenced by [setupDefaultWaves\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.148.3.3 createBossWave()

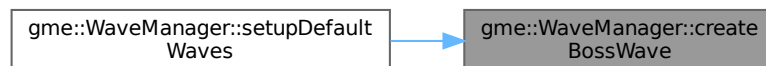
void gme::WaveManager::createBossWave () [private]

Definition at line 352 of file [WaveManager.cpp](#).

References [gme::BOSS](#), [gme::Wave::duration](#), [gme::Wave::enemies](#), [gme::ENEMY_ADVANCED](#), [gme::ENEMY_BASIC](#), [m_waves](#), [gme::Wave::waitForClear](#), and [gme::Wave::waveNumber](#).

Referenced by [setupDefaultWaves\(\)](#).

Here is the caller graph for this function:



18.148.3.4 createWave1()

void gme::WaveManager::createWave1 () [private]

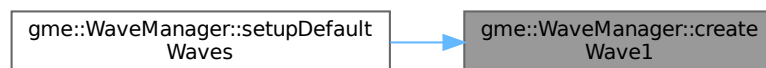
Create default wave patterns (5 waves: 4x30s + boss 60s)

Definition at line 241 of file [WaveManager.cpp](#).

References [gme::Wave::duration](#), [gme::Wave::enemies](#), [gme::ENEMY_BASIC](#), [m_waves](#), [gme::Wave::waitForClear](#), and [gme::Wave::waveNumber](#).

Referenced by [setupDefaultWaves\(\)](#).

Here is the caller graph for this function:



18.148.3.5 createWave2()

void gme::WaveManager::createWave2 () [private]

Definition at line 265 of file [WaveManager.cpp](#).

References [gme::Wave::duration](#), [gme::Wave::enemies](#), [gme::ENEMY_ADVANCED](#), [gme::ENEMY_BASIC](#), [m_waves](#), [gme::Wave::waitForClear](#), and [gme::Wave::waveNumber](#).

Referenced by [setupDefaultWaves\(\)](#).

Here is the caller graph for this function:



18.148.3.6 createWave3()

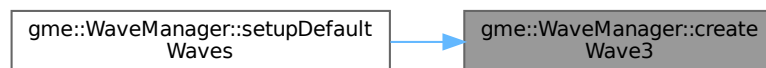
void gme::WaveManager::createWave3 () [private]

Definition at line 291 of file [WaveManager.cpp](#).

References [gme::Wave::duration](#), [gme::Wave::enemies](#), [gme::ENEMY_ADVANCED](#), [gme::ENEMY_BASIC](#), [m_waves](#), [gme::Wave::waitForClear](#), and [gme::Wave::waveNumber](#).

Referenced by [setupDefaultWaves\(\)](#).

Here is the caller graph for this function:



18.148.3.7 createWave4()

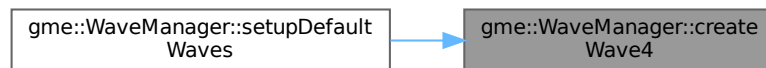
void gme::WaveManager::createWave4 () [private]

Definition at line 319 of file [WaveManager.cpp](#).

References [gme::Wave::duration](#), [gme::Wave::enemies](#), [gme::ENEMY_ADVANCED](#), [gme::ENEMY_BASIC](#), [m_waves](#), [gme::Wave::waitForClear](#), and [gme::Wave::waveNumber](#).

Referenced by [setupDefaultWaves\(\)](#).

Here is the caller graph for this function:



18.148.3.8 getCurrentWave()

int gme::WaveManager::getCurrentWave () const [inline]

Get current wave number.

Definition at line 74 of file [WaveManager.hpp](#).

References [m_currentWaveIndex](#).

18.148.3.9 getTotalWaves()

size_t gme::WaveManager::getTotalWaves () const [inline]

Get total number of waves.

Definition at line 77 of file [WaveManager.hpp](#).

References [m_waves](#).

18.148.3.10 isActive()

bool gme::WaveManager::isActive () const [inline]

Check if wave system is active.

Definition at line 71 of file [WaveManager.hpp](#).

References [m_active](#).

18.148.3.11 isCompleted()

bool gme::WaveManager::isCompleted () const [inline]

Check if all waves are completed.

Definition at line 80 of file [WaveManager.hpp](#).

References [m_completed](#).

18.148.3.12 isReady()

bool gme::WaveManager::isReady () const [inline]

Check if initial delay has passed and waves can start.

Definition at line 62 of file [WaveManager.hpp](#).

References [m_initialDelayPassed](#).

18.148.3.13 isWaveCleared()

bool gme::WaveManager::isWaveCleared () const [private]

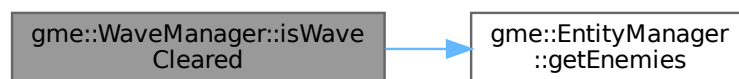
Check if current wave is cleared.

Definition at line 176 of file [WaveManager.cpp](#).

References [gme::EntityManager::getEnemies\(\)](#), and [m_entityManager](#).

Referenced by [update\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.148.3.14 operator=() [1/2]

[WaveManager](#) & gme::WaveManager::operator= (
 const [WaveManager](#) &) [delete]

18.148.3.15 operator=() [2/2]

[WaveManager](#) & gme::WaveManager::operator= (
 [WaveManager](#) &&) [delete]

18.148.3.16 processSpawns()

```
void gme::WaveManager::processSpawns (
    float dt,
    int screenWidth) [private]
```

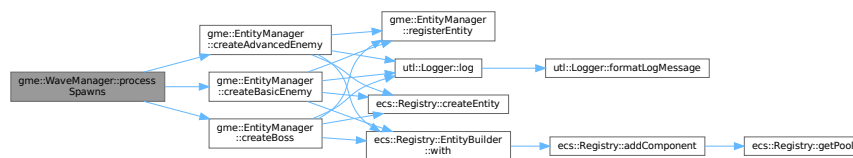
Process enemy spawns for current wave.

Definition at line 183 of file [WaveManager.cpp](#).

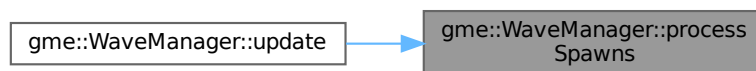
References [gme::BOSS](#), [gme::EntityManager::createAdvancedEnemy\(\)](#), [gme::EntityManager::createBasicEnemy\(\)](#), [gme::EntityManager::createBoss\(\)](#), [gme::Wave::enemies](#), [gme::ENEMY_ADVANCED](#), [gme::ENEMY_BASIC](#), [gme::EnemySpawn::health](#), [m_currentWaveIndex](#), [m_enemiesSpawned](#), [m_entityManager](#), [m_spawnTimer](#), [m_waves](#), [gme::EnemySpawn::spawnDelay](#), [gme::EnemySpawn::type](#), [gme::EnemySpawn::x](#), and [gme::EnemySpawn::y](#).

Referenced by [update\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.148.3.17 reset()

```
void gme::WaveManager::reset ()
```

Stop and reset the wave system.

Definition at line 120 of file [WaveManager.cpp](#).

References [m_active](#), [m_completed](#), [m_currentWaveIndex](#), [m_enemiesSpawned](#), [m_initialDelayPassed](#), [m_initialDelayTimer](#), [m_spawnTimer](#), [m_waveInProgress](#), and [m_waveTimer](#).

Referenced by [clearWaves\(\)](#).

Here is the caller graph for this function:



18.148.3.18 `setupDefaultWaves()`

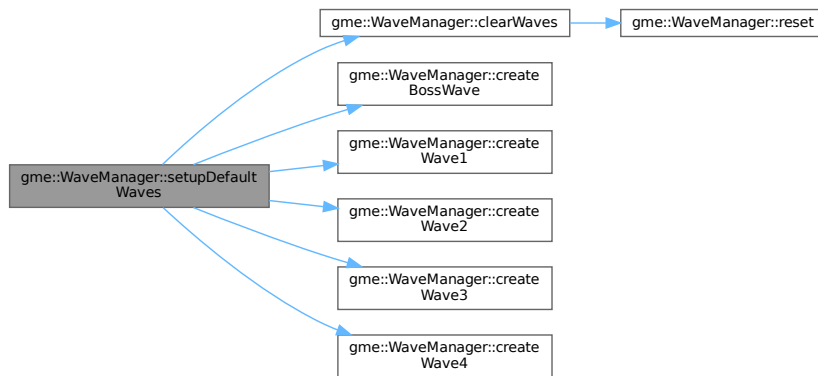
```
void gme::WaveManager::setupDefaultWaves ()
```

Initialize with default wave patterns.

Definition at line 142 of file [WaveManager.cpp](#).

References [clearWaves\(\)](#), [createBossWave\(\)](#), [createWave1\(\)](#), [createWave2\(\)](#), [createWave3\(\)](#), [createWave4\(\)](#), and [m_waves](#).

Here is the call graph for this function:



18.148.3.19 `start()`

```
void gme::WaveManager::start ()
```

Start the wave system.

Definition at line 106 of file [WaveManager.cpp](#).

References [INITIAL_DELAY](#), [m_active](#), [m_completed](#), [m_currentWaveIndex](#), [m_initialDelayPassed](#), [m_initialDelayTimer](#), [m_spawnTimer](#), [m_waveInProgress](#), [m_waves](#), and [m_waveTimer](#).

18.148.3.20 `startNextWave()`

```
void gme::WaveManager::startNextWave () [private]
```

Start the next wave.

Definition at line 153 of file [WaveManager.cpp](#).

References [gme::Wave::enemies](#), [m_completed](#), [m_currentWaveIndex](#), [m_enemiesSpawned](#), [m_spawnTimer](#), [m_waveInProgress](#), [m_waves](#), and [m_waveTimer](#).

Referenced by [update\(\)](#).

Here is the caller graph for this function:



18.148.3.21 update()

```
void gme::WaveManager::update (
    ecs::Registry & registry,
    float dt,
    int screenWidth)
```

Update wave manager logic.

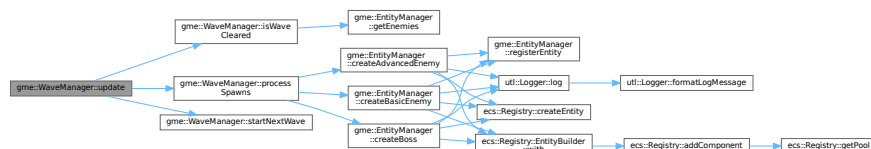
Parameters

registry	The ECS registry
dt	Delta time in seconds
screenWidth	Screen width for spawn positioning

Definition at line 20 of file [WaveManager.cpp](#).

References [gme::Wave::duration](#), [INITIAL_DELAY](#), [isWaveCleared\(\)](#), [m_active](#), [m_completed](#), [m_currentWaveIndex](#), [m_initialDelayPassed](#), [m_initialDelayTimer](#), [m_spawnTimer](#), [m_waveInProgress](#), [m_waves](#), [m_waveTimer](#), [gme::Wave::onComplete](#), [processSpawns\(\)](#), [startNextWave\(\)](#), and [gme::Wave::waitForClear](#).

Here is the call graph for this function:



18.148.4 Member Data Documentation

18.148.4.1 INITIAL_DELAY

```
float gme::WaveManager::INITIAL_DELAY = 0.5f [static], [constexpr], [private]
```

Definition at line 104 of file [WaveManager.hpp](#).

Referenced by [start\(\)](#), and [update\(\)](#).

18.148.4.2 m_active

```
bool gme::WaveManager::m_active [private]
```

Definition at line 97 of file [WaveManager.hpp](#).

Referenced by [isActive\(\)](#), [reset\(\)](#), [start\(\)](#), and [update\(\)](#).

18.148.4.3 m_completed

```
bool gme::WaveManager::m_completed [private]
```

Definition at line 98 of file [WaveManager.hpp](#).

Referenced by [isCompleted\(\)](#), [reset\(\)](#), [start\(\)](#), [startNextWave\(\)](#), and [update\(\)](#).

18.148.4.4 m_currentWaveIndex

int gme::WaveManager::m_currentWaveIndex [private]

Definition at line 94 of file [WaveManager.hpp](#).

Referenced by [getCurrentWave\(\)](#), [processSpawns\(\)](#), [reset\(\)](#), [start\(\)](#), [startNextWave\(\)](#), and [update\(\)](#).

18.148.4.5 m_enemiesSpawned

std::vector<bool> gme::WaveManager::m_enemiesSpawned [private]

Definition at line 102 of file [WaveManager.hpp](#).

Referenced by [processSpawns\(\)](#), [reset\(\)](#), and [startNextWave\(\)](#).

18.148.4.6 m_entityManager

[EntityManager&](#) gme::WaveManager::m_entityManager [private]

Definition at line 92 of file [WaveManager.hpp](#).

Referenced by [isWaveCleared\(\)](#), and [processSpawns\(\)](#).

18.148.4.7 m_initialDelayPassed

bool gme::WaveManager::m_initialDelayPassed [private]

Definition at line 100 of file [WaveManager.hpp](#).

Referenced by [isReady\(\)](#), [reset\(\)](#), [start\(\)](#), and [update\(\)](#).

18.148.4.8 m_initialDelayTimer

float gme::WaveManager::m_initialDelayTimer [private]

Definition at line 101 of file [WaveManager.hpp](#).

Referenced by [reset\(\)](#), [start\(\)](#), and [update\(\)](#).

18.148.4.9 m_spawnTimer

float gme::WaveManager::m_spawnTimer [private]

Definition at line 96 of file [WaveManager.hpp](#).

Referenced by [processSpawns\(\)](#), [reset\(\)](#), [start\(\)](#), [startNextWave\(\)](#), and [update\(\)](#).

18.148.4.10 m_waveInProgress

bool gme::WaveManager::m_waveInProgress [private]

Definition at line 99 of file [WaveManager.hpp](#).

Referenced by [reset\(\)](#), [start\(\)](#), [startNextWave\(\)](#), and [update\(\)](#).

18.148.4.11 m_waves

std::vector<[Wave](#)> gme::WaveManager::m_waves [private]

Definition at line 93 of file [WaveManager.hpp](#).

Referenced by [addWave\(\)](#), [clearWaves\(\)](#), [createBossWave\(\)](#), [createWave1\(\)](#), [createWave2\(\)](#), [createWave3\(\)](#), [createWave4\(\)](#), [getTotalWaves\(\)](#), [processSpawns\(\)](#), [setupDefaultWaves\(\)](#), [start\(\)](#), [startNextWave\(\)](#), and [update\(\)](#).

18.148.4.12 m_waveTimer

float gme::WaveManager::m_waveTimer [private]

Definition at line 95 of file [WaveManager.hpp](#).

Referenced by [reset\(\)](#), [start\(\)](#), [startNextWave\(\)](#), and [update\(\)](#).

The documentation for this class was generated from the following files:

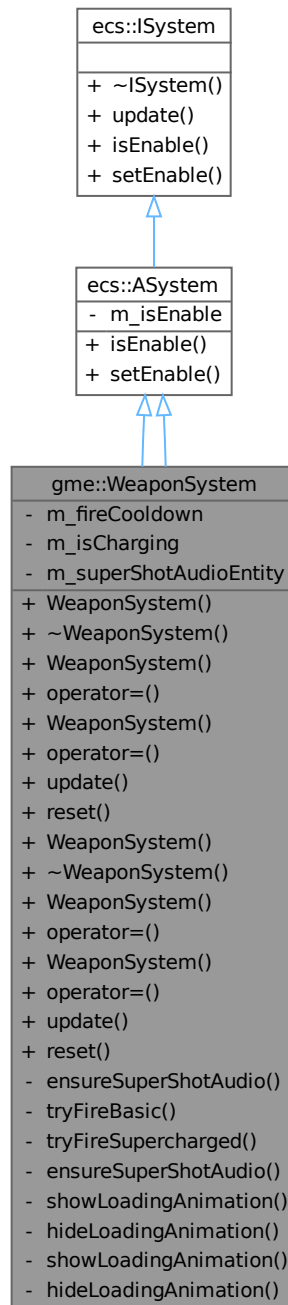
- [/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Server/include/RTypeServer/WaveManager.hpp](#)
- [/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Server/src/WaveManager.cpp](#)

18.149 gme::WeaponSystem Class Reference

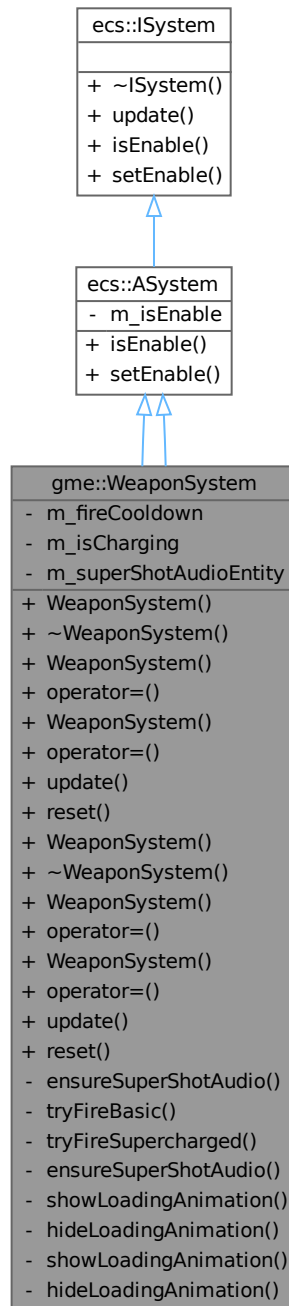
Manages weapon firing and charging (client-side visual feedback only)

#include <Weapon.hpp>

Inheritance diagram for gme::WeaponSystem:



Collaboration diagram for gme::WeaponSystem:



Public Member Functions

- `WeaponSystem` (`const std::shared_ptr< eng::IRenderer > &`)
- `~WeaponSystem` () override=default
- `WeaponSystem` (`const WeaponSystem &`)=delete
- `WeaponSystem & operator=` (`const WeaponSystem &`)=delete
- `WeaponSystem` (`WeaponSystem &&`)=delete
- `WeaponSystem & operator=` (`WeaponSystem &&`)=delete

- void [update](#) ([ecs::Registry](#) ®istry, float dt) override
Update weapon system (handles charging animation)
- void [reset](#) ()
Reset weapon state.
- [WeaponSystem](#) (const std::shared_ptr< [eng::IRenderer](#) > &)
- [~WeaponSystem](#) () override=default
- [WeaponSystem](#) (const [WeaponSystem](#) &)=delete
- [WeaponSystem](#) & operator= (const [WeaponSystem](#) &)=delete
- [WeaponSystem](#) ([WeaponSystem](#) &&)=delete
- [WeaponSystem](#) & operator= ([WeaponSystem](#) &&)=delete
- void [update](#) ([ecs::Registry](#) ®istry, float dt) override
Update weapon system.
- void [reset](#) ()
Reset weapon state.

Public Member Functions inherited from [ecs::ASystem](#)

- bool [isEnabled](#) () override
- void [setEnabled](#) (const bool enable) override

Public Member Functions inherited from [ecs::ISystem](#)

- virtual [~ISystem](#) ()=default

Private Member Functions

- void [ensureSuperShotAudio](#) ([ecs::Registry](#) ®istry)
- bool [tryFireBasic](#) ([ecs::Registry](#) ®istry, float x, float y)
Try to fire basic projectile.
- bool [tryFireSupercharged](#) ([ecs::Registry](#) ®istry, float x, float y)
Try to fire supercharged projectile.
- void [ensureSuperShotAudio](#) ([ecs::Registry](#) ®istry)

Static Private Member Functions

- static void [showLoadingAnimation](#) ([ecs::Registry](#) ®istry, [ecs::Entity](#) playerEntity, const [ecs::Transform](#) *playerTransform)
Show loading animation in front of the player.
- static void [hideLoadingAnimation](#) ([ecs::Registry](#) ®istry, [ecs::Entity](#) playerEntity)
Hide loading animation.
- static void [showLoadingAnimation](#) ([ecs::Registry](#) ®istry, [ecs::Entity](#) playerEntity, const [ecs::Transform](#) *playerTransform)
Show loading animation in front of the player.
- static void [hideLoadingAnimation](#) ([ecs::Registry](#) ®istry, [ecs::Entity](#) playerEntity)
Hide loading animation.

Private Attributes

- float [m_fireCooldown](#) = 0.0f
- bool [m_isCharging](#) = false
- [ecs::Entity](#) [m_superShotAudioEntity](#) = [ecs::INVALID_ENTITY](#)

18.149.1 Detailed Description

Manages weapon firing and charging (client-side visual feedback only)

Manages weapon firing and charging.

Definition at line 21 of file [Weapon.hpp](#).

18.149.2 Constructor & Destructor Documentation

18.149.2.1 WeaponSystem() [1/6]

`gme::WeaponSystem::WeaponSystem (`
 `const std::shared_ptr< eng::IRenderer > & )` [inline], [explicit]

Definition at line 24 of file [Weapon.hpp](#).

18.149.2.2 ~WeaponSystem() [1/2]

`gme::WeaponSystem::~~WeaponSystem ()` [override], [default]

18.149.2.3 WeaponSystem() [2/6]

`gme::WeaponSystem::WeaponSystem (`
 `const WeaponSystem & )` [delete]

18.149.2.4 WeaponSystem() [3/6]

`gme::WeaponSystem::WeaponSystem (`
 `WeaponSystem && )` [delete]

18.149.2.5 WeaponSystem() [4/6]

`gme::WeaponSystem::WeaponSystem (`
 `const std::shared_ptr< eng::IRenderer > & )` [inline], [explicit]

Definition at line 24 of file [Weapon.hpp](#).

18.149.2.6 ~WeaponSystem() [2/2]

`gme::WeaponSystem::~~WeaponSystem ()` [override], [default]

18.149.2.7 WeaponSystem() [5/6]

`gme::WeaponSystem::WeaponSystem (`
 `const WeaponSystem & )` [delete]

18.149.2.8 WeaponSystem() [6/6]

`gme::WeaponSystem::WeaponSystem (`
 `WeaponSystem && )` [delete]

18.149.3 Member Function Documentation

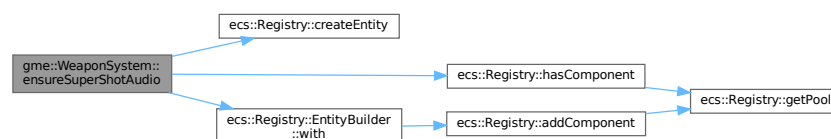
18.149.3.1 ensureSuperShotAudio() [1/2]

`void gme::WeaponSystem::ensureSuperShotAudio (`
 `ecs::Registry & registry)` [private]

Definition at line 126 of file [weapon.cpp](#).

References [utl::Path::Audio::AUDIO_SUPERCHARGED_SHOT](#), [ecs::Registry::createEntity\(\)](#), [ecs::Registry::hasComponent\(\)](#), [ecs::INVALID_ENTITY](#), and [ecs::Registry::EntityBuilder::with\(\)](#).

Here is the call graph for this function:



18.149.3.2 ensureSuperShotAudio() [2/2]

void gme::WeaponSystem::ensureSuperShotAudio (
[ecs::Registry](#) & registry) [private]

18.149.3.3 hideLoadingAnimation() [1/2]

void gme::WeaponSystem::hideLoadingAnimation (
[ecs::Registry](#) & registry,
[ecs::Entity](#) playerEntity) [static], [private]

Hide loading animation.

Parameters

registry	The ECS registry
playerEntity	The player entity

Definition at line 91 of file [weapon.cpp](#).

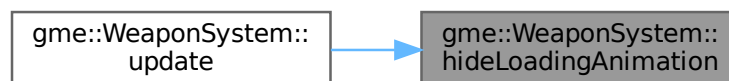
References [ecs::Registry::getAll\(\)](#), [ecs::Registry::hasComponent\(\)](#), and [ecs::Registry::removeComponent\(\)](#).

Referenced by [update\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.149.3.4 hideLoadingAnimation() [2/2]

static void gme::WeaponSystem::hideLoadingAnimation (
[ecs::Registry](#) & registry,
[ecs::Entity](#) playerEntity) [static], [private]

Hide loading animation.

Parameters

registry	The ECS registry
playerEntity	The player entity

18.149.3.5 operator=() [1/4]

[WeaponSystem](#) & gme::WeaponSystem::operator= (const [WeaponSystem](#) &) [delete]

18.149.3.6 operator=() [2/4]

[WeaponSystem](#) & gme::WeaponSystem::operator= (const [WeaponSystem](#) &) [delete]

18.149.3.7 operator=() [3/4]

[WeaponSystem](#) & gme::WeaponSystem::operator= ([WeaponSystem](#) &&) [delete]

18.149.3.8 operator=() [4/4]

[WeaponSystem](#) & gme::WeaponSystem::operator= ([WeaponSystem](#) &&) [delete]

18.149.3.9 reset() [1/2]

void gme::WeaponSystem::reset () [inline]

Reset weapon state.

Definition at line 42 of file [Weapon.hpp](#).

References [m_fireCooldown](#), and [m_isCharging](#).

18.149.3.10 reset() [2/2]

void gme::WeaponSystem::reset () [inline]

Reset weapon state.

Definition at line 42 of file [Weapon.hpp](#).

References [m_fireCooldown](#), and [m_isCharging](#).

18.149.3.11 showLoadingAnimation() [1/2]

void gme::WeaponSystem::showLoadingAnimation ([ecs::Registry](#) & registry, [ecs::Entity](#) playerEntity, const [ecs::Transform](#) * playerTransform) [static], [private]

Show loading animation in front of the player.

Parameters

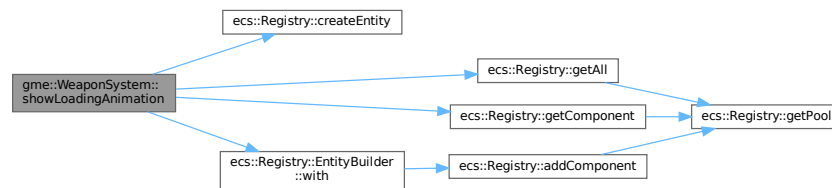
registry	The ECS registry
playerEntity	The player entity
playerTransform	The player transform

Definition at line 62 of file [weapon.cpp](#).

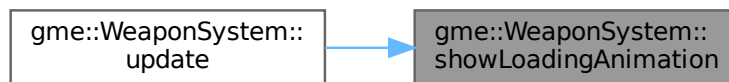
References [utl::GameConfig::LoadingAnimation::ANIMATION_DURATION](#), [utl::GameConfig::LoadingAnimation::ANIMATION_DURATION](#), [ecs::Registry::createEntity\(\)](#), [ecs::Registry::getAll\(\)](#), [ecs::Registry::getComponent\(\)](#), [utl::GameConfig::LoadingAnimation::ANIMATION_DURATION](#), [utl::GameConfig::LoadingAnimation::OFFSET_Y](#), [utl::GameConfig::LoadingAnimation::SPRITE_HEIGHT](#), [utl::GameConfig::LoadingAnimation::SPRITE_WIDTH](#), [utl::Path::Texture::TEXTURE_SHOOT_LOADING](#), [ecs::Registry::EntityBuilder::with\(\)](#), [ecs::Transform::x](#), and [ecs::Transform::y](#).

Referenced by [update\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



18.149.3.12 showLoadingAnimation() [2/2]

```
static void gme::WeaponSystem::showLoadingAnimation (
    ecs::Registry & registry,
    ecs::Entity playerEntity,
    const ecs::Transform * playerTransform) [static], [private]
```

Show loading animation in front of the player.

Parameters

registry	The ECS registry
playerEntity	The player entity
playerTransform	The player transform

18.149.3.13 tryFireBasic()

```
bool gme::WeaponSystem::tryFireBasic (
    ecs::Registry & registry,
    float x,
    float y) [private]
```

Try to fire basic projectile.

Parameters

registry	The ECS registry
x	X position
y	Y position

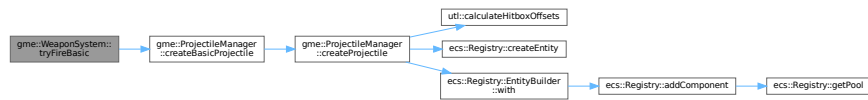
Returns

True if fired successfully

Definition at line 76 of file [weapon.cpp](#).

References [gme::ProjectileManager::createBasicProjectile\(\)](#), [utl::GameConfig::Projectile::Basic::FIRE_COOLDOWN](#), and [utl::GameConfig::Projectile::Basic::SPEED](#).

Here is the call graph for this function:



18.149.3.14 tryFireSupercharged()

```

bool gme::WeaponSystem::tryFireSupercharged (
    ecs::Registry & registry,
    float x,
    float y) [private]
  
```

Try to fire supercharged projectile.

Parameters

registry	The ECS registry
x	X position
y	Y position

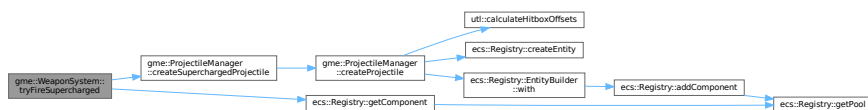
Returns

True if fired successfully

Definition at line 86 of file [weapon.cpp](#).

References [gme::ProjectileManager::createSuperchargedProjectile\(\)](#), [ecs::Registry::getComponent\(\)](#), [ecs::INVALID_ENTITY](#), and [utl::GameConfig::Projectile::Supercharged::SPEED](#).

Here is the call graph for this function:



18.149.3.15 update() [1/2]

```

void gme::WeaponSystem::update (
    ecs::Registry & registry,
    float dt) [override], [virtual]
  
```

Update weapon system (handles charging animation)

Parameters

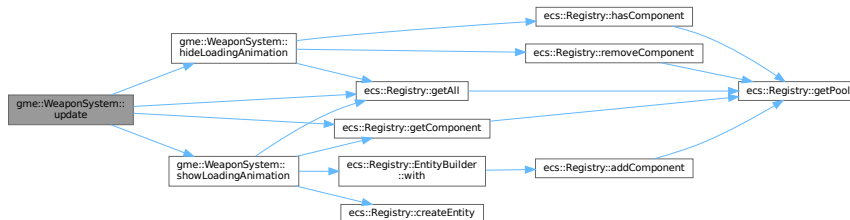
registry	The ECS registry
dt	Delta time

Implements [ecs::ISystem](#).

Definition at line 8 of file [weapon.cpp](#).

References [utl::GameConfig::Beam::CHARGE_RATE](#), [ecs::Registry::getAll\(\)](#), [ecs::Registry::getComponent\(\)](#), [hideLoadingAnimation\(\)](#), [m_fireCooldown](#), [m_isCharging](#), [showLoadingAnimation\(\)](#), and [ecs::KeyboardInput::space_pressed](#).

Here is the call graph for this function:



18.149.3.16 update() [2/2]

```
void gme::WeaponSystem::update (
    ecs::Registry & registry,
    float dt) [override], [virtual]
```

Update weapon system.

Parameters

registry	The ECS registry
dt	Delta time

Implements [ecs::ISystem](#).

18.149.4 Member Data Documentation

18.149.4.1 m_fireCooldown

```
float gme::WeaponSystem::m_fireCooldown = 0.0f [private]
```

Definition at line 49 of file [Weapon.hpp](#).

Referenced by [reset\(\)](#), and [update\(\)](#).

18.149.4.2 m_isCharging

```
bool gme::WeaponSystem::m_isCharging = false [private]
```

Definition at line 50 of file [Weapon.hpp](#).

Referenced by [reset\(\)](#), and [update\(\)](#).

18.149.4.3 m_superShotAudioEntity

```
ecs::Entity gme::WeaponSystem::m_superShotAudioEntity = ecs::INVALID\_ENTITY [private]
```

Definition at line 70 of file [Weapon.hpp](#).

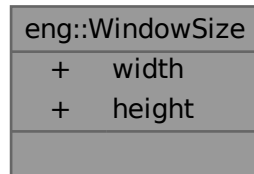
The documentation for this class was generated from the following files:

- /home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/include/RTypeClientMulti/Systems/[Weapon.hpp](#)
- /home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Solo/include/RTypeClientSolo/Systems/[Weapon.hpp](#)
- /home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/src/Systems/[weapon.cpp](#)
- /home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Solo/src/systems/[weapon.cpp](#)

18.150 eng::WindowSize Struct Reference

#include <IRenderer.hpp>

Collaboration diagram for eng::WindowSize:



Public Attributes

- unsigned int [width](#)
- unsigned int [height](#)

18.150.1 Detailed Description

Definition at line [108](#) of file [IRenderer.hpp](#).

18.150.2 Member Data Documentation

18.150.2.1 height

unsigned int eng::WindowSize::height

Definition at line [111](#) of file [IRenderer.hpp](#).

Referenced by [ecs::StarfieldSystem::createStarfield\(\)](#), and [gme::Game::update\(\)](#).

18.150.2.2 width

unsigned int eng::WindowSize::width

Definition at line [110](#) of file [IRenderer.hpp](#).

Referenced by [ecs::StarfieldSystem::createStarfield\(\)](#), [cli::Menu::update\(\)](#), [gme::Game::update\(\)](#), and [gme::StageManager::update\(\)](#).

The documentation for this struct was generated from the following file:

- [/home/masina/Projects/Epitech/rtype/modules/Interfaces/include/Interfaces/IRenderer.hpp](#)

Chapter 19

File Documentation

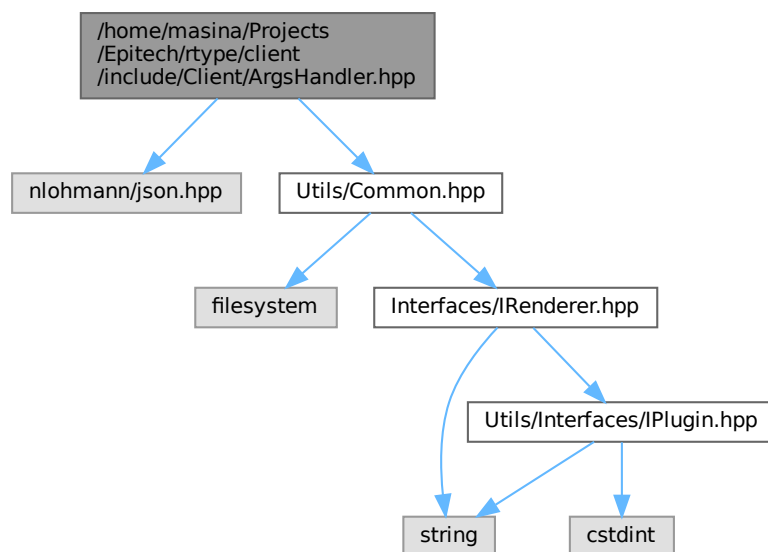
19.1 /home/masina/Projects/Epitech/rtype/client/include/Client/ArgsHandler.hpp File Reference

This file contains the ArgsHandler class declaration.

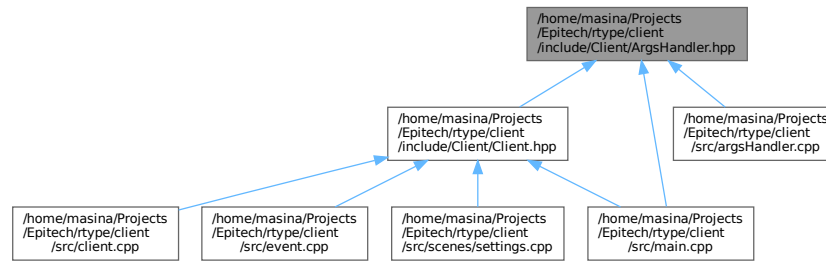
```
#include <nlohmann/json.hpp>
```

```
#include "Utils/Common.hpp"
```

Include dependency graph for ArgsHandler.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- struct [cli::ArgsConfig](#)
- struct [cli::EnvConfig](#)
- class [cli::ArgsHandler](#)

Class to handle command line arguments.

Namespaces

- namespace [cli](#)

Typedefs

- using [cli::json](#) = [nlohmann::json](#)

19.1.1 Detailed Description

This file contains the ArgsHandler class declaration.

Definition in file [ArgsHandler.hpp](#).

19.2 ArgsHandler.hpp

[Go to the documentation of this file.](#)

```

00001 ///
00002 /// @file ArgsHandler.hpp
00003 /// @brief This file contains the ArgsHandler class declaration
00004 /// @namespace cli
00005 ///
00006
00007 #pragma once
00008
00009 #include <nlohmann/json.hpp>
00010
00011 #include "Utils/Common.hpp"
00012
00013 namespace cli
00014 {
00015
00016     using json = nlohmann::json;
00017
00018     struct ArgsConfig
00019     {
00020         bool exit = false;
00021         unsigned int width = utl::Config::Window::DEFAULT_WINDOW_WIDTH;
00022         unsigned int height = utl::Config::Window::DEFAULT_WINDOW_HEIGHT;
00023         unsigned int frameLimit = utl::Config::Window::DEFAULT_WINDOW_FRAME_LIMIT;
00024         bool fullscreen = utl::Config::Window::DEFAULT_WINDOW_FULLSCREEN;
00025         std::string host = utl::Config::Network::DEFAULT_NETWORK_HOST;
00026         unsigned int port = utl::Config::Network::DEFAULT_NETWORK_PORT;
00027         std::string player_name = utl::Config::Player::DEFAULT_PLAYER_NAME;
00028         std::string audio_lib_path = utl::Path::Plugin::PLUGIN_AUDIO_SFML.string();
00029         std::string network_lib_path = utl::Path::Plugin::PLUGIN_NETWORK_ASIO_CLIENT.string();
00030         std::string renderer_lib_path = utl::Path::Plugin::PLUGIN_RENDERER_SFML.string();
00031     };
00032
00033 }
  
```



```

00031     std::string game_solo_lib_path = utl::Path::Plugin::PLUGIN_GAME_SOLO.string();
00032     std::string game_multi_lib_path = utl::Path::Plugin::PLUGIN_GAME_MULTI.string();
00033
00034     static ArgsConfig fromFile(const std::string &path);
00035 }; // struct Config
00036 struct EnvConfig
00037 {
00038 };
00039
00040 ///
00041 /// @class ArgsHandler
00042 /// @brief Class to handle command line arguments
00043 /// @namespace cli
00044 ///
00045 class ArgsHandler
00046 {
00047
00048     public:
00049         ArgsHandler() = default;
00050         ~ArgsHandler() = default;
00051
00052         ArgsHandler(const ArgsHandler &) = delete;
00053         ArgsHandler &operator=(const ArgsHandler &) = delete;
00054         ArgsHandler(ArgsHandler &&) = delete;
00055         ArgsHandler &operator=(ArgsHandler &&) = delete;
00056
00057         static ArgsConfig ParseArgs(int argc, const char *const argv[]);
00058         static EnvConfig ParseEnv(const char *const env[]);
00059
00060     private:
00061 }; // class ArgsHandler
00062 } // namespace cli

```

19.3 /home/masina/Projects/Epitech/rtype/server/include/Server/ArgsHandler.hpp File Reference

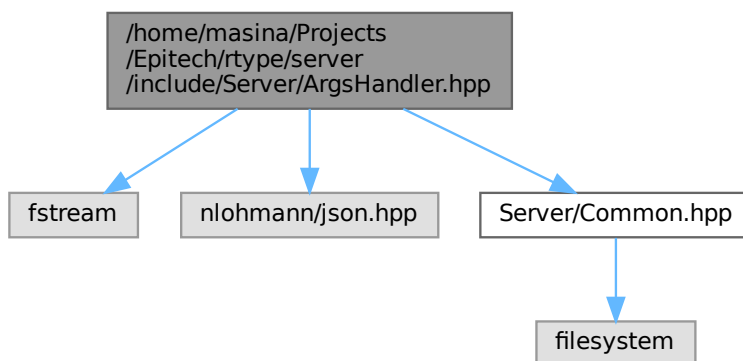
This file contains the ArgsHandler class declaration.

```
#include <fstream>
```

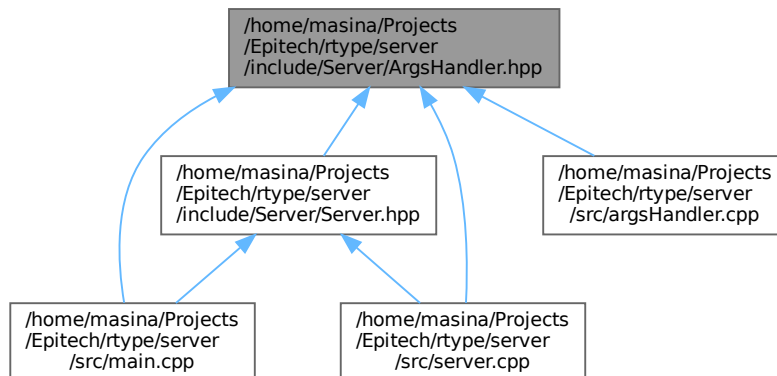
```
#include <nlohmann/json.hpp>
```

```
#include "Server/Common.hpp"
```

Include dependency graph for ArgsHandler.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- struct [srv::ArgsConfig](#)
- struct [srv::EnvConfig](#)
- class [srv::ArgsHandler](#)

Class to handle command line arguments.

Namespaces

- namespace [srv](#)

Typedefs

- using [srv::json](#) = [nlohmann::json](#)

19.3.1 Detailed Description

This file contains the ArgsHandler class declaration.
Definition in file [ArgsHandler.hpp](#).

19.4 ArgsHandler.hpp

[Go to the documentation of this file.](#)

```

00001 ///  

00002 ///  

00003 ///  

00004 ///  

00005 ///  

00006 ///  

00007 #pragma once  

00008  

00009 #include <fstream>  

00010  

00011 #include <nlohmann/json.hpp>  

00012  

00013 #include "Server/Common.hpp"  

00014  

00015 namespace srv  

00016 {  

00017  

00018     using json = nlohmann::json;  

00019  

00020     struct ArgsConfig  

00021     {  

00022         bool exit = false;  

00023         std::string host = Config::Network::DEFAULT_NETWORK_HOST;  


```

```

00024     uint16_t port = Config::Network::DEFAULT_NETWORK_PORT;
00025     std::string network_lib_path = Path::Plugin::PLUGINS_NETWORK_ASIO_SERVER.string();
00026     std::string game_lib_path = Path::Plugin::PLUGINS_GAME_RTYPE_SERVER.string();
00027
00028     static ArgsConfig fromFile(const std::string &path);
00029 }; // struct Config
00030 struct EnvConfig
00031 {
00032 };
00033
00034 ///
00035 /// @class ArgsHandler
00036 /// @brief Class to handle command line arguments
00037 /// @namespace srv
00038 ///
00039 class ArgsHandler
00040 {
00041
00042 public:
00043     ArgsHandler() = default;
00044     ~ArgsHandler() = default;
00045
00046     ArgsHandler(const ArgsHandler &) = delete;
00047     ArgsHandler &operator=(const ArgsHandler &) = delete;
00048     ArgsHandler(ArgsHandler &&) = delete;
00049     ArgsHandler &operator=(ArgsHandler &&) = delete;
00050
00051     static ArgsConfig ParseArgs(int argc, const char *const argv[]);
00052     static EnvConfig ParseEnv(const char *const env[]);
00053
00054 private:
00055 }; // class ArgsHandler
00056
00057 } // namespace srv

```

19.5 /home/masina/Projects/Epitech/rtype/client/include/Client/Client.hpp File Reference

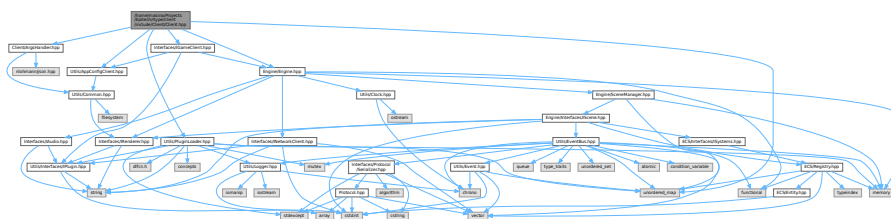
This file contains the Client class declaration.

```

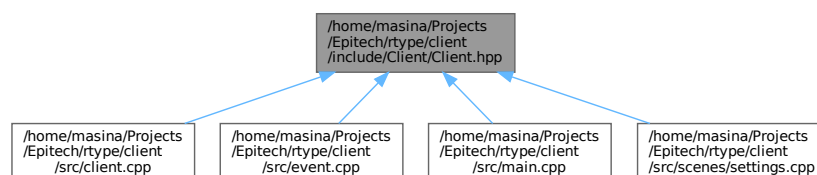
#include <unordered_map>
#include "Client/ArgsHandler.hpp"
#include "Engine/Engine.hpp"
#include "Interfaces/IGameClient.hpp"
#include "Utils/AppConfigClient.hpp"
#include "Utils/PluginLoader.hpp"

```

Include dependency graph for Client.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [cli::Client](#)
Class for the client.

Namespaces

- namespace [cli](#)

19.5.1 Detailed Description

This file contains the Client class declaration.

Definition in file [Client.hpp](#).

19.6 Client.hpp

[Go to the documentation of this file.](#)

```

00001 ///  

00002 ///  

00003 ///  

00004 ///  

00005 ///  

00006 ///  

00007 #pragma once  

00008  

00009 #include <unordered_map>  

00010  

00011 #include "Client/ArgsHandler.hpp"  

00012 #include "Engine/Engine.hpp"  

00013 #include "Interfaces/IGameClient.hpp"  

00014 #include "Utils/AppConfigClient.hpp"  

00015 #include "Utils/PluginLoader.hpp"  

00016  

00017 namespace cli  

00018 {  

00019  

00020     ///  

00021     ///  

00022     ///  

00023     ///  

00024     ///  

00025     class Client  

00026     {  

00027  

00028     public:  

00029         explicit Client(const ArgsConfig &cfg);  

00030         ~Client() = default;  

00031  

00032         Client(const Client &) = delete;  

00033         Client &operator=(const Client &) = delete;  

00034         Client(Client &&) = delete;  

00035         Client &operator=(Client &&) = delete;  

00036  

00037         void run();  

00038         void stop() const;  

00039  

00040     private:  

00041         void handleEvents(eng::Event &event);  

00042         void updateKeyboardInput(ecs::Registry &registry);  

00043         static utl::cli::AppConfig setupConfig(const ArgsConfig &cfg);  

00044         void setupScenes();  

00045  

00046         std::unique_ptr<utl::PluginLoader> m_pluginLoader;  

00047         std::unique_ptr<eng::Engine> m_engine;  

00048         std::shared_ptr<gme::IGameClient> m_gameSolo;  

00049         std::shared_ptr<gme::IGameClient> m_gameMulti;  

00050         std::unordered_map<eng::Key, bool> m_keysPressed;  

00051  

00052         utl::cli::AppConfig m_config;  

00053         bool m_showDebug = false;  

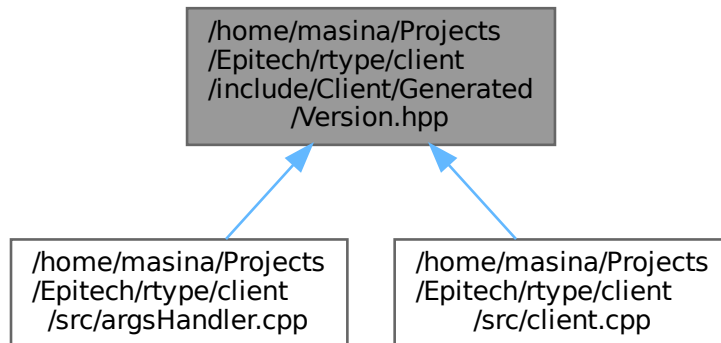
00054     }; // class Client  

00055 } // namespace cli

```

19.7 /home/masina/Projects/Epitech/rtype/client/include/Client/Generated/Version.hpp File Reference

This graph shows which files directly or indirectly include this file:



Macros

- `#define PROJECT_NAME "r-type_client"`
- `#define PROJECT_VERSION "0.0.0"`
- `#define PROJECT_VERSION_MAJOR "0"`
- `#define PROJECT_VERSION_MINOR "0"`
- `#define PROJECT_VERSION_PATCH "0"`
- `#define GIT_COMMIT_HASH "971d5ea"`
- `#define GIT_TAG "971d5eac"`
- `#define BUILD_TYPE "Release"`

19.7.1 Macro Definition Documentation

19.7.1.1 BUILD_TYPE

```
#define BUILD_TYPE "Release"
```

Definition at line 15 of file [Version.hpp](#).

Referenced by [cli::Client::Client\(\)](#), and [srv::Server::Server\(\)](#).

19.7.1.2 GIT_COMMIT_HASH

```
#define GIT_COMMIT_HASH "971d5ea"
```

Definition at line 13 of file [Version.hpp](#).

Referenced by [cli::Client::Client\(\)](#), and [srv::Server::Server\(\)](#).

19.7.1.3 GIT_TAG

```
#define GIT_TAG "971d5eac"
```

Definition at line 14 of file [Version.hpp](#).

Referenced by [cli::Client::Client\(\)](#), and [srv::Server::Server\(\)](#).

19.7.1.4 PROJECT_NAME

```
#define PROJECT_NAME "r-type_client"
```

Definition at line 7 of file [Version.hpp](#).

Referenced by [cli::Client::Client\(\)](#), and [srv::Server::Server\(\)](#).

19.7.1.5 PROJECT_VERSION

`#define PROJECT_VERSION "0.0.0"`

Definition at line 8 of file [Version.hpp](#).

Referenced by [cli::Client::Client\(\)](#), and [srv::Server::Server\(\)](#).

19.7.1.6 PROJECT_VERSION_MAJOR

`#define PROJECT_VERSION_MAJOR "0"`

Definition at line 9 of file [Version.hpp](#).

19.7.1.7 PROJECT_VERSION_MINOR

`#define PROJECT_VERSION_MINOR "0"`

Definition at line 10 of file [Version.hpp](#).

19.7.1.8 PROJECT_VERSION_PATCH

`#define PROJECT_VERSION_PATCH "0"`

Definition at line 11 of file [Version.hpp](#).

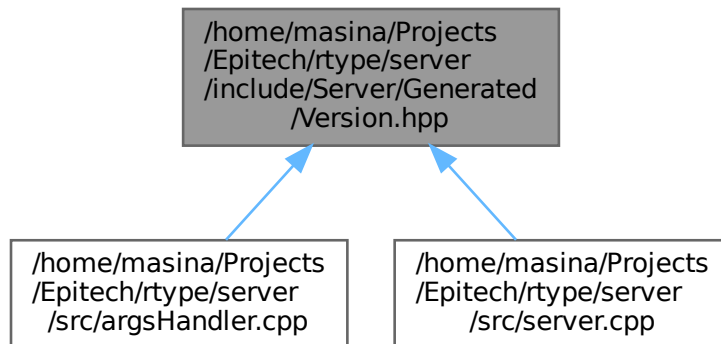
19.8 Version.hpp

[Go to the documentation of this file.](#)

```
00001 #pragma once
00002
00003 //
=====
00004 // DO NOT EDIT THIS FILE MANUALLY. IT IS GENERATED BY CMAKE DURING THE BUILD PROCESS.
00005 //
=====
00006
00007 #define PROJECT_NAME "r-type_client"
00008 #define PROJECT_VERSION "0.0.0"
00009 #define PROJECT_VERSION_MAJOR "0"
00010 #define PROJECT_VERSION_MINOR "0"
00011 #define PROJECT_VERSION_PATCH "0"
00012
00013 #define GIT_COMMIT_HASH "971d5ea"
00014 #define GIT_TAG "971d5eac"
00015 #define BUILD_TYPE "Release"
```

19.9 /home/masina/Projects/Epitech/rtype/server/include/Server/Generated/Version.hpp File Reference

This graph shows which files directly or indirectly include this file:



Macros

- `#define PROJECT_NAME "r-type_server"`
- `#define PROJECT_VERSION "0.0.0"`
- `#define PROJECT_VERSION_MAJOR "0"`
- `#define PROJECT_VERSION_MINOR "0"`
- `#define PROJECT_VERSION_PATCH "0"`
- `#define GIT_COMMIT_HASH "971d5ea"`
- `#define GIT_TAG "971d5eac"`
- `#define BUILD_TYPE "Release"`

19.9.1 Macro Definition Documentation

19.9.1.1 BUILD_TYPE

```
#define BUILD_TYPE "Release"
```

Definition at line 15 of file [Version.hpp](#).

19.9.1.2 GIT_COMMIT_HASH

```
#define GIT_COMMIT_HASH "971d5ea"
```

Definition at line 13 of file [Version.hpp](#).

19.9.1.3 GIT_TAG

```
#define GIT_TAG "971d5eac"
```

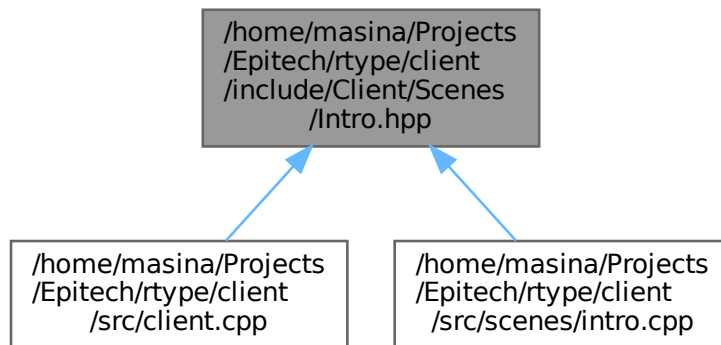
Definition at line 14 of file [Version.hpp](#).

19.9.1.4 PROJECT_NAME

```
#define PROJECT_NAME "r-type_server"
```

Definition at line 7 of file [Version.hpp](#).

This graph shows which files directly or indirectly include this file:



Classes

- class `cli::Intro`
Intro scene.

Namespaces

- namespace `cli`

19.11.1 Detailed Description

This file contains the intro scene.

Definition in file [Intro.hpp](#).

19.12 Intro.hpp

[Go to the documentation of this file.](#)

```

00001 ///  

00002 ///  

00003 ///  

00004 ///  

00005 ///  

00006 ///  

00007 #pragma once  

00008  

00009 #include "Engine/Interfaces/IScene.hpp"  

00010 #include "Interfaces/IAudio.hpp"  

00011 #include "Utils/Clock.hpp"  

00012  

00013 namespace cli  

00014 {  

00015     ///  

00016     ///  

00017     ///  

00018     ///  

00019     ///  

00020     class Intro final : public eng::AScene  

00021     {  

00022     public:  

00023         Intro(eng::id assignedId, const std::shared_ptr<eng::IRenderer> &renderer,  

00024             const std::shared_ptr<eng::IAudio> &audio);  

00025         ~Intro() override = default;  

00026  

00027         Intro(const Intro &other) = delete;  

00028         Intro &operator=(const Intro &other) = delete;  

00029         Intro(Intro &&other) = delete;  

00030         Intro &operator=(Intro &&other) = delete;  


```


Namespaces

- namespace [cli](#)

19.13.1 Detailed Description

This file contains the menu scene.

Definition in file [Menu.hpp](#).

19.14 Menu.hpp

[Go to the documentation of this file.](#)

```
00001 ///  
00002 ///  
00003 ///  
00004 ///  
00005 ///  
00006 ///  
00007 #pragma once  
00008  
00009 #include <unordered_map>  
00010 #include <vector>  
00011  
00012 #include "Engine/Interfaces/IScene.hpp"  
00013  
00014 namespace cli  
00015 {  
00016     ///  
00017     ///  
00018     ///  
00019     ///  
00020     ///  
00021     class Menu final : public eng::AScene  
00022     {  
00023     public:  
00024         Menu(eng::id assignedId, const std::shared_ptr<eng::IRenderer> &renderer);  
00025         ~Menu() override = default;  
00026  
00027         Menu(const Menu &other) = delete;  
00028         Menu &operator=(const Menu &other) = delete;  
00029         Menu(Menu &&other) = delete;  
00030         Menu &operator=(Menu &&other) = delete;  
00031  
00032         void update(float dt, const eng::WindowSize &size) override;  
00033         void event(const eng::Event &event) override;  
00034  
00035         bool &playMusic() { return m_playMusic; }  
00036  
00037         std::function<void(const std::string &option)> onOptionSelected;  
00038  
00039     private:  
00040         std::unordered_map<eng::Key, bool> m_keysPressed;  
00041         const std::vector<std::string> m_menuOptions = {"Solo", "Multi", "Settings"};  
00042  
00043         int m_selectedIndex = 0;  
00044         float m_animationTime = 0.0f;  
00045         float m_titlePulseTime = 0.0f;  
00046         ecs::Entity m_titleEntity;  
00047  
00048         const std::vector<std::string> m_contributors = {"Elliot", "Arthur", "Moana", "Aaron"};  
00049         ecs::Entity m_contributorsEntity;  
00050         float m_contributorsOffset = 0.0f;  
00051         bool m_playMusic = false;  
00052     }; // class Menu  
00053 } // namespace cli
```

19.15 /home/masina/Projects/Epitech/rtype/client/include/Client/Scenes/Settings.hpp File Reference

This file contains the settings scene.

```
#include <unordered_map>
```

```
#include "Engine/Interfaces/IScene.hpp"
```



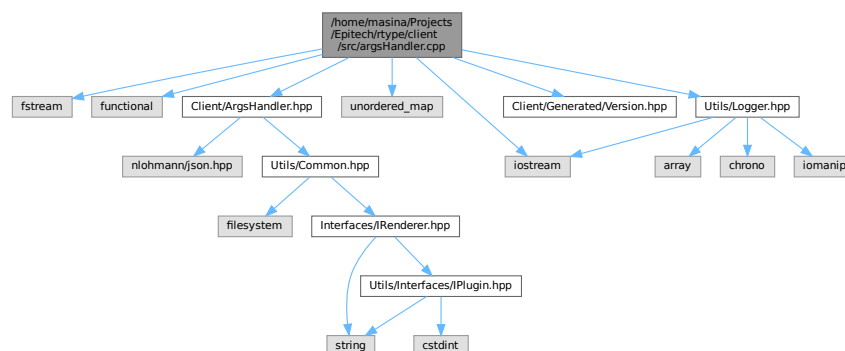
```

00015     struct AppConfig;
00016
00017     ///
00018     /// @class Settings
00019     /// @brief Settings scene
00020     /// @namespace cli
00021     ///
00022     class Settings final : public eng::AScene
00023     {
00024     public:
00025         Settings(eng::id assignedId, const std::shared_ptr<eng::IRenderer> &renderer, utl::cli::AppConfig &config);
00026         ~Settings() override = default;
00027
00028         Settings(const Settings &other) = delete;
00029         Settings &operator=(const Settings &other) = delete;
00030         Settings(Settings &&other) = delete;
00031         Settings &operator=(Settings &&other) = delete;
00032
00033         void update(float dt, const eng::WindowSize &size) override;
00034         void event(const eng::Event &event) override;
00035         void updateSettingsDisplay();
00036         void loadFromConfig();
00037         bool &playMusic() { return m_playMusic; }
00038
00039         std::function<void()> onLeave;
00040
00041     private:
00042         static void applyVideoQuality();
00043         static void applySkinChange();
00044
00045         std::unordered_map<eng::Key, bool> m_keysPressed;
00046         const std::shared_ptr<eng::IRenderer> &m_renderer;
00047         utl::cli::AppConfig &m_appConfig;
00048         size_t m_selectedIndex = 0;
00049         const std::vector<std::string> m_settingsOptions = {"Audio Volume", "FPS", "Controls", "Skin",
00050                                                         "Back to Menu"};
00051         float m_audioVolume = 50.0F;
00052         int m_videoQuality = 1;
00053         int m_controlScheme = 2;
00054         int m_skinIndex = 0;
00055         ecs::Entity m_volumeValueEntity;
00056         ecs::Entity m_qualityValueEntity;
00057         ecs::Entity m_controlValueEntity;
00058         ecs::Entity m_skinSpriteEntity;
00059         ecs::Entity m_titleEntity;
00060         float m_animationTime = 0.0f;
00061         float m_titlePulseTime = 0.0f;
00062         bool m_playMusic = false;
00063     }; // class Settings
00064 } // namespace cli

```

- 19.17 [/home/masina/Projects/Epitech/rtype/client/README.md](#) File Reference
- 19.18 [/home/masina/Projects/Epitech/rtype/plugins/Audio/↵](#)
README.md File Reference
- 19.19 [/home/masina/Projects/Epitech/rtype/plugins/Game/↵](#)
README.md File Reference
- 19.20 [/home/masina/Projects/Epitech/rtype/plugins/Network/↵](#)
README.md File Reference
- 19.21 [/home/masina/Projects/Epitech/rtype/plugins/README.md](#) File Reference
- 19.22 [/home/masina/Projects/Epitech/rtype/plugins/Renderer/↵](#)
README.md File Reference
- 19.23 [/home/masina/Projects/Epitech/rtype/README.md](#) File Reference
- 19.24 [/home/masina/Projects/Epitech/rtype/server/README.md](#) File Reference
- 19.25 [/home/masina/Projects/Epitech/rtype/client/src/args↵](#)
Handler.cpp File Reference

```
#include <fstream>
#include <functional>
#include <iostream>
#include <unordered_map>
#include "Client/ArgsHandler.hpp"
#include "Client/Generated/Version.hpp"
#include "Utils/Logger.hpp"
Include dependency graph for argsHandler.cpp:
```



Macros

- `#define APP_EXTENSION ""`

Variables

- static constexpr std::string_view `HELP_MESSAGE`
- static constexpr std::string_view `VERSION_MESSAGE`

19.25.1 Macro Definition Documentation

19.25.1.1 APP_EXTENSION

`#define APP_EXTENSION ""`

Definition at line 9 of file `argsHandler.cpp`.

19.25.2 Variable Documentation

19.25.2.1 HELP_MESSAGE

std::string_view `HELP_MESSAGE` [static], [constexpr]

Initial value:

```
= "Usage: " PROJECT_NAME APP_EXTENSION " [options]\n\n"
    "Options:\n"
    "\t--help, -h      Show this help message\n"
    "\t--version, -v     Show version information\n"
    "\t--config, -c      Specify path to config file\n"
```

Definition at line 16 of file `argsHandler.cpp`.

Referenced by `cli::ArgsHandler::ParseArgs()`, and `srv::ArgsHandler::ParseArgs()`.

19.25.2.2 VERSION_MESSAGE

std::string_view `VERSION_MESSAGE` [static], [constexpr]

Initial value:

```
= PROJECT_NAME " version " PROJECT_VERSION "\n"
    "Build type: " BUILD_TYPE "\n"
    "Git tag: " GIT_TAG "\n"
    "Git commit hash: " GIT_COMMIT_HASH "\n"
```

Definition at line 21 of file `argsHandler.cpp`.

Referenced by `cli::ArgsHandler::ParseArgs()`, and `srv::ArgsHandler::ParseArgs()`.

19.26 argsHandler.cpp

[Go to the documentation of this file.](#)

```
00001 #include <fstream>
00002 #include <functional>
00003 #include <iostream>
00004 #include <unordered_map>
00005
00006 #ifdef _WIN32
00007 #define APP_EXTENSION ".exe"
00008 #else
00009 #define APP_EXTENSION ""
00010 #endif
00011
00012 #include "Client/ArgsHandler.hpp"
00013 #include "Client/Generated/Version.hpp"
00014 #include "Utils/Logger.hpp"
00015
00016 static constexpr std::string_view HELP_MESSAGE = "Usage: " PROJECT_NAME APP_EXTENSION " [options]\n\n"
00017     "Options:\n"
00018     "\t--help, -h      Show this help message\n"
00019     "\t--version, -v     Show version information\n"
00020     "\t--config, -c      Specify path to config file\n";
00021 static constexpr std::string_view VERSION_MESSAGE = PROJECT_NAME " version " PROJECT_VERSION "\n"
00022     "Build type: " BUILD_TYPE "\n"
00023     "Git tag: " GIT_TAG "\n"
00024     "Git commit hash: " GIT_COMMIT_HASH "\n";
00025
00026 cli::ArgsConfig cli::ArgsConfig::fromFile(const std::string &path)
00027 {
00028     ArgsConfig cfg;
00029     std::ifstream file(path);
```

```

00030     if (!file.is_open())
00031     {
00032         throw std::runtime_error("Cannot open config file: " + path);
00033     }
00034
00035     json j;
00036     file » j;
00037
00038     if (j.contains("window"))
00039     {
00040         const auto &w = j["window"];
00041         if (w.contains("width"))
00042         {
00043             cfg.width = w["width"];
00044         }
00045         if (w.contains("height"))
00046         {
00047             cfg.height = w["height"];
00048         }
00049         if (w.contains("frame_limit"))
00050         {
00051             cfg.frameLimit = w["frame_limit"];
00052         }
00053         if (w.contains("fullscreen"))
00054         {
00055             cfg.fullscreen = w["fullscreen"];
00056         }
00057         const auto &p = j["plugins"];
00058         if (p.contains("audio"))
00059         {
00060             cfg.audio_lib_path = p["audio"];
00061         }
00062         if (p.contains("network"))
00063         {
00064             cfg.network_lib_path = p["network"];
00065         }
00066         if (p.contains("renderer"))
00067         {
00068             cfg.renderer_lib_path = p["renderer"];
00069         }
00070         if (p.contains("game_solo"))
00071         {
00072             cfg.game_solo_lib_path = p["game_solo"];
00073         }
00074         if (p.contains("game_multi"))
00075         {
00076             cfg.game_multi_lib_path = p["game_multi"];
00077         }
00078         const auto &c = j["client"];
00079         if (c.contains("host"))
00080         {
00081             cfg.host = c["host"];
00082         }
00083         if (c.contains("port"))
00084         {
00085             cfg.port = c["port"];
00086         }
00087         if (c.contains("player_name"))
00088         {
00089             cfg.player_name = c["player_name"];
00090         }
00091     }
00092     return cfg;
00093 }
00094
00095 cli::ArgsConfig cli::ArgsHandler::ParseArgs(const int argc, const char *const argv[])
00096 {
00097     if (argc <= 1)
00098     {
00099         return {};
00100     }
00101
00102     using ArgHandler = std::function<void(const char *arg)>;
00103     std::unordered_map<std::string_view, ArgHandler> handlers;
00104     ArgsConfig config{};
00105     for (const auto *const opt : {"-h", "--help"})
00106     {
00107         handlers[opt] = [&config](const char *)
00108         {
00109             std::cout « HELP_MESSAGE;
00110             config.exit = true;
00111         };
00112     }
00113     for (const auto *const opt : {"-v", "--version"})
00114     {
00115         handlers[opt] = [&config](const char *)
00116     {

```



```

00117         std::cout << VERSION_MESSAGE;
00118         config.exit = true;
00119     };
00120 }
00121
00122 for (const auto *const opt : {"-c", "--config"})
00123 {
00124     handlers[opt] = [&config](const char *arg)
00125     {
00126         if (!arg)
00127         {
00128             throw std::runtime_error("Missing config file argument");
00129         }
00130         config = ArgsConfig::fromFile(arg);
00131         utl::Logger::log("Loaded config from file: " + std::string(arg), utl::LogLevel::INFO);
00132         std::cout << "\tWidth: " << config.width << '\n'
00133                 << "\tHeight: " << config.height << '\n'
00134                 << "\tFrameLimit: " << config.frameLimit << "\n"
00135                 << "\tFullscreen: " << (config.fullscreen ? "true" : "false") << '\n';
00136     };
00137 }
00138
00139 const std::string_view key = argv[1];
00140 const char *argValue = (argc > 2) ? argv[2] : nullptr;
00141
00142 if (const auto it = handlers.find(key); it != handlers.end())
00143 {
00144     it->second(argValue);
00145     return config;
00146 }
00147
00148 throw std::runtime_error("Unknown argument: " + std::string(key));
00149 }
00150
00151 cli::EnvConfig cli::ArgsHandler::ParseEnv(const char *const env[])
00152 {
00153     (void)env; // Currently unused
00154     return {};
00155 }

```

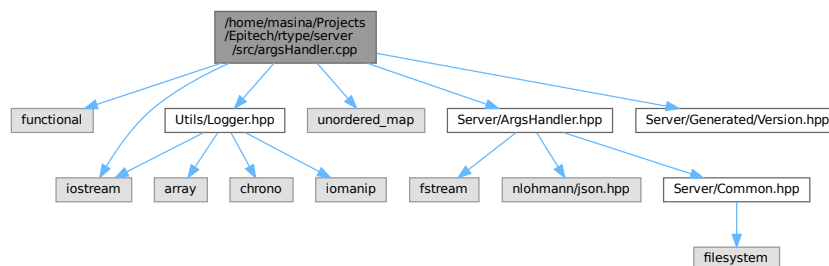
19.27 /home/masina/Projects/Epitech/rtype/server/src/argsHandler.cpp File Reference

```

#include <functional>
#include <iostream>
#include <unordered_map>
#include "Server/ArgsHandler.hpp"
#include "Server/Generated/Version.hpp"
#include "Utils/Logger.hpp"

```

Include dependency graph for argsHandler.cpp:



Macros

- `#define APP_EXTENSION ""`

Variables

- static constexpr std::string_view [HELP_MESSAGE](#)
- static constexpr std::string_view [VERSION_MESSAGE](#)

19.27.1 Macro Definition Documentation

19.27.1.1 APP_EXTENSION

```
#define APP_EXTENSION ""
```

Definition at line 9 of file [argsHandler.cpp](#).

19.27.2 Variable Documentation

19.27.2.1 HELP_MESSAGE

```
std::string_view HELP_MESSAGE [static], [constexpr]
```

Initial value:

```
= "Usage: " PROJECT_NAME APP_EXTENSION " [options]\n\n"
    "Options:\n"
    "\t--help, -h      Show this help message\n"
    "\t--version, -v     Show version information\n"
    "\t--config, -c      Specify path to config file\n"
```

Definition at line 16 of file [argsHandler.cpp](#).

19.27.2.2 VERSION_MESSAGE

```
std::string_view VERSION_MESSAGE [static], [constexpr]
```

Initial value:

```
= PROJECT_NAME " version " PROJECT_VERSION "\n"
    "Build type: " BUILD_TYPE "\n"
    "Git tag: " GIT_TAG "\n"
    "Git commit hash: " GIT_COMMIT_HASH "\n"
```

Definition at line 21 of file [argsHandler.cpp](#).

19.28 argsHandler.cpp

[Go to the documentation of this file.](#)

```
00001 #include <functional>
00002 #include <iostream>
00003 #include <unordered_map>
00004
00005 #ifdef _WIN32
00006 #include <windows.h>
00007 #define APP_EXTENSION ".exe"
00008 #else
00009 #define APP_EXTENSION ""
00010 #endif
00011
00012 #include "Server/ArgsHandler.hpp"
00013 #include "Server/Generated/Version.hpp"
00014 #include "Utils/Logger.hpp"
00015
00016 static constexpr std::string_view HELP_MESSAGE = "Usage: " PROJECT_NAME APP_EXTENSION " [options]\n\n"
00017     "Options:\n"
00018     "\t--help, -h      Show this help message\n"
00019     "\t--version, -v     Show version information\n"
00020     "\t--config, -c      Specify path to config file\n";
00021 static constexpr std::string_view VERSION_MESSAGE = PROJECT_NAME " version " PROJECT_VERSION "\n"
00022     "Build type: " BUILD_TYPE "\n"
00023     "Git tag: " GIT_TAG "\n"
00024     "Git commit hash: " GIT_COMMIT_HASH "\n";
00025
00026 srv::ArgsConfig srv::ArgsConfig::fromFile(const std::string &path)
00027 {
00028     ArgsConfig cfg;
00029     std::ifstream file(path);
00030     if (!file.is_open())
00031     {
00032         throw std::runtime_error("Cannot open config file: " + path);
00033     }
00034
00035     json j;
00036     file » j;
00037 }
```

```

00038     if (j.contains("host"))
00039     {
00040         cfg.host = j["host"];
00041     }
00042     if (j.contains("port"))
00043     {
00044         cfg.port = j["port"];
00045     }
00046     if (const auto &p = j["plugins"]; p.contains("network"))
00047     {
00048         cfg.network_lib_path = p["network"];
00049     }
00050     return cfg;
00051 }
00052
00053 srv::ArgsConfig srv::ArgsHandler::ParseArgs(const int argc, const char *const argv[])
00054 {
00055     if (argc <= 1)
00056     {
00057         return {};
00058     }
00059
00060     using ArgHandler = std::function<void(const char *arg)>;
00061     std::unordered_map<std::string_view, ArgHandler> handlers;
00062     ArgsConfig config{};
00063     for (const auto *const opt : {"-h", "--help"})
00064     {
00065         handlers[opt] = [&config](const char *)
00066         {
00067             std::cout << HELP_MESSAGE;
00068             config.exit = true;
00069         };
00070     }
00071     for (const auto *const opt : {"-v", "--version"})
00072     {
00073         handlers[opt] = [&config](const char *)
00074         {
00075             std::cout << VERSION_MESSAGE;
00076             config.exit = true;
00077         };
00078     }
00079     for (const auto *const opt : {"-c", "--config"})
00080     {
00081         handlers[opt] = [&config](const char *arg)
00082         {
00083             if (!arg)
00084             {
00085                 throw std::runtime_error("Missing config file argument");
00086             }
00087             config = ArgsConfig::fromFile(arg);
00088             utl::Logger::log("Loaded config from file: " + std::string(arg), utl::LogLevel::INFO);
00089             std::cout << "\tHost: " << config.host << '\n' << "\tPort: " << config.port << '\n';
00090         };
00091     }
00092
00093     const std::string_view key = argv[1];
00094     const char *argValue = (argc > 2) ? argv[2] : nullptr;
00095
00096     if (const auto it = handlers.find(key); it != handlers.end())
00097     {
00098         it->second(argValue);
00099         return config;
00100     }
00101
00102     throw std::runtime_error("Unknown argument: " + std::string(key));
00103 }
00104
00105
00106 srv::EnvConfig srv::ArgsHandler::ParseEnv(const char *const env[]) { return {}; }

```

19.29 /home/masina/Projects/Epitech/rtype/client/src/client.cpp File Reference

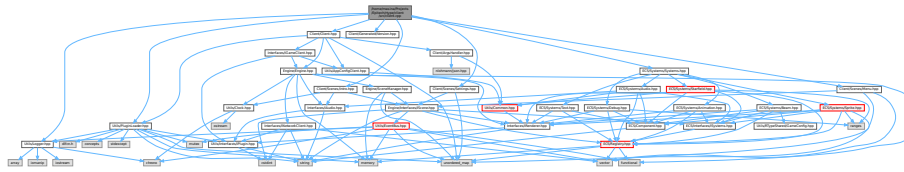
```

#include "Client/Client.hpp"
#include "Client/Generated/Version.hpp"
#include "Client/Scenes/Intro.hpp"
#include "Client/Scenes/Menu.hpp"
#include "Client/Scenes/Settings.hpp"
#include "ECS/Systems/Systems.hpp"
#include "Utils/Logger.hpp"

```

```
#include "Utils/PluginLoader.hpp"
```

Include dependency graph for client.cpp:



19.30 client.cpp

[Go to the documentation of this file.](#)

```
00001 #include "Client/Client.hpp"
00002 #include "Client/Generated/Version.hpp"
00003 #include "Client/Scenes/Intro.hpp"
00004 #include "Client/Scenes/Menu.hpp"
00005 #include "Client/Scenes/Settings.hpp"
00006 #include "ECS/Systems/Systems.hpp"
00007 #include "Utils/Logger.hpp"
00008 #include "Utils/PluginLoader.hpp"
00009
00010 utl::cli::AppConfig cli::Client::setupConfig(const ArgsConfig &cfg)
00011 {
00012     utl::cli::AppConfig appConfig;
00013
00014     appConfig.frameLimit = cfg.frameLimit;
00015     appConfig.fullscreen = cfg.fullscreen;
00016     appConfig.height = cfg.height;
00017     appConfig.width = cfg.width;
00018     appConfig.host = cfg.host;
00019     appConfig.port = cfg.port;
00020     appConfig.player_name = cfg.player_name;
00021
00022     appConfig.audioVolume = utl::Config::Audio::DEFAULT_AUDIO_VOLUME;
00023     appConfig.videoQuality = utl::Config::Game::DEFAULT_VIDEO_QUALITY;
00024     appConfig.controlScheme = utl::Config::Game::DEFAULT_CONTROL_SCHEME;
00025     appConfig.skinIndex = utl::Config::Game::DEFAULT_SKIN_INDEX;
00026
00027     return appConfig;
00028 }
00029
00030 cli::Client::Client(const ArgsConfig &cfg)
00031 {
00032     utl::Logger::log("PROJECT INFO:", utl::LogLevel::INFO);
00033     std::cout << "\tName: " PROJECT_NAME "\n"
00034               << "\tVersion: " PROJECT_VERSION "\n"
00035               << "\tBuild type: " BUILD_TYPE "\n"
00036               << "\tGit tag: " GIT_TAG "\n"
00037               << "\tGit commit hash: " GIT_COMMIT_HASH "\n";
00038
00039     m_config = setupConfig(cfg);
00040     m_pluginLoader = std::make_unique<utl::PluginLoader>();
00041     m_engine = std::make_unique<eng::Engine>(
00042         [this, cfg]()
00043         {
00044             return m_pluginLoader->loadPlugin<eng::IAudio>(
00045                 !cfg.audio_lib_path.empty() ? cfg.audio_lib_path : utl::Path::Plugin::PLUGIN_AUDIO_SFML.string());
00046         },
00047         [this, cfg]()
00048         {
00049             return m_pluginLoader->loadPlugin<eng::INetworkClient>(
00050                 !cfg.network_lib_path.empty() ? cfg.network_lib_path
00051                 : utl::Path::Plugin::PLUGIN_NETWORK_ASIO_CLIENT.string());
00052         },
00053         [this, cfg]()
00054         {
00055             return m_pluginLoader->loadPlugin<eng::IRenderer>(!cfg.renderer_lib_path.empty()
00056                 ? cfg.renderer_lib_path
00057                 : utl::Path::Plugin::PLUGIN_RENDERER_SFML.string());
00058         });
00059     m_gameSolo = m_pluginLoader->loadPlugin<gme::IGameClient>(
00060         !cfg.game_solo_lib_path.empty() ? cfg.game_solo_lib_path : utl::Path::Plugin::PLUGIN_GAME_SOLO.string());
00061     m_gameMulti = m_pluginLoader->loadPlugin<gme::IGameClient>(
00062         !cfg.game_multi_lib_path.empty() ? cfg.game_multi_lib_path :
00063         utl::Path::Plugin::PLUGIN_GAME_MULTI.string());
00064     m_engine->getRenderer()->createWindow("R-Type Client", m_config.height, m_config.width, m_config.frameLimit,
00065         m_config.fullscreen);
00066     m_engine->getRenderer()->setWindowIcon(utl::Path::Texture::ICON_PATH);
00067 }
```

```

00066 }
00067
00068 void cli::Client::run()
00069 {
00070     setupScenes();
00071
00072     eng::Event event;
00073     while (m_engine->getState() == eng::State::RUN)
00074     {
00075         handleEvents(event);
00076         m_engine->render(m_engine->getRenderer()->getWindowSize(), utl::Config::Color::DARK, m_showDebug);
00077         m_engine->getNetwork()->update();
00078         m_gameSolo->update(m_engine->getClock()->getDeltaSeconds(),
00079             m_engine->getRenderer()->getWindowSize().width,
00080             m_engine->getRenderer()->getWindowSize().height);
00081     }
00082 }
00083 void cli::Client::stop() const
00084 {
00085     m_engine->getNetwork()->disconnect();
00086     m_engine->stop();
00087 }
00088
00089 void cli::Client::setupScenes()
00090 {
00091     bool f = false;
00092     auto menuId = m_engine->getSceneManager()->generateNextId();
00093     auto menu = std::make_unique<Menu>(menuId, m_engine->getRenderer());
00094     menu->addSystem(std::make_unique<ecs::AudioSystem>(m_engine->getAudio(), m_config.audioVolume,
00095         menu->getRegistry(),
00096         menu->playMusic()));
00097     menu->addSystem(std::make_unique<ecs::SpriteSystem>(m_engine->getRenderer()));
00098     menu->addSystem(std::make_unique<ecs::TextSystem>(m_engine->getRenderer()));
00099     menu->addSystem(std::make_unique<ecs::DebugSystem>(m_engine->getRenderer(), m_showDebug));
00100
00101     auto introId = m_engine->getSceneManager()->generateNextId();
00102     auto intro = std::make_unique<Intro>(introId, m_engine->getRenderer(), m_engine->getAudio());
00103     intro->addSystem(std::make_unique<ecs::AudioSystem>(m_engine->getAudio(), m_config.audioVolume,
00104         intro->getRegistry(), menu->playMusic()));
00105     intro->addSystem(std::make_unique<ecs::SpriteSystem>(m_engine->getRenderer()));
00106     intro->addSystem(std::make_unique<ecs::TextSystem>(m_engine->getRenderer()));
00107     intro->addSystem(std::make_unique<ecs::DebugSystem>(m_engine->getRenderer(), m_showDebug));
00108     intro->onLeave = [this, menuId]() { m_engine->getSceneManager()->switchToScene(menuId); };
00109
00110     m_gameSolo->init(*m_engine, m_config, m_showDebug, menuId);
00111     m_gameMulti->init(*m_engine, m_config, m_showDebug, menuId);
00112
00113     auto settingsId = m_engine->getSceneManager()->generateNextId();
00114     auto settings = std::make_unique<Settings>(settingsId, m_engine->getRenderer(), m_config);
00115     settings->addSystem(std::make_unique<ecs::AudioSystem>(m_engine->getAudio(), m_config.audioVolume,
00116         settings->getRegistry(), settings->playMusic()));
00117     settings->addSystem(std::make_unique<ecs::StarfieldSystem>(m_engine->getRenderer(), settings->getRegistry()));
00118     settings->addSystem(std::make_unique<ecs::SpriteSystem>(m_engine->getRenderer()));
00119     settings->addSystem(std::make_unique<ecs::TextSystem>(m_engine->getRenderer()));
00120     settings->addSystem(std::make_unique<ecs::DebugSystem>(m_engine->getRenderer(), m_showDebug));
00121     const auto configSoloId = m_gameSolo->getMainSceneId();
00122     const auto serverSceneId = m_gameMulti->getMainSceneId();
00123     menu->onOptionSelected = [this, configSoloId, serverSceneId, settingsId](const std::string &option)
00124     {
00125         {
00126             if (option == "Solo")
00127             {
00128                 m_engine->getSceneManager()->switchToScene(configSoloId);
00129             }
00130             else if (option == "Multi")
00131             {
00132                 m_engine->getSceneManager()->switchToScene(serverSceneId);
00133             }
00134             else if (option == "Settings")
00135             {
00136                 m_engine->getSceneManager()->switchToScene(settingsId);
00137             }
00138         };
00139         settings->onLeave = [this, menuId]() { m_engine->getSceneManager()->switchToScene(menuId); };
00140
00141         m_engine->getSceneManager()->addScene(std::move(menu));
00142         m_engine->getSceneManager()->addScene(std::move(intro));
00143         m_engine->getSceneManager()->addScene(std::move(settings));
00144         m_engine->getSceneManager()->switchToScene(introId);
00145     }

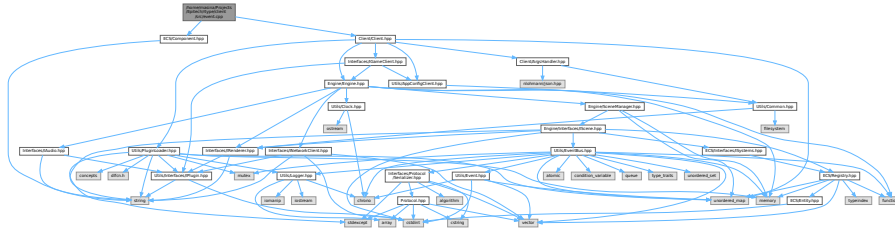
```

19.31 /home/masina/Projects/Epitech/rtype/client/src/event.cpp File Reference

#include "Client/Client.hpp"

#include "ECS/Component.hpp"

Include dependency graph for event.cpp:



19.32 event.cpp

[Go to the documentation of this file.](#)

```
00001 #include "Client/Client.hpp"
00002 #include "ECS/Component.hpp"
00003
00004 void cli::Client::handleEvents(eng::Event &event)
00005 {
00006     const auto &scene = m_engine->getSceneManager()->getCurrentScene();
00007
00008     while (m_engine->getRenderer()->pollEvent(event))
00009     {
00010         scene->event(event);
00011
00012         switch (event.type)
00013         {
00014             case eng::EventType::Closed:
00015                 m_engine->setState(eng::State::STOP);
00016                 break;
00017
00018             case eng::EventType::KeyPressed:
00019                 m_keysPressed[event.key] = true;
00020                 if (event.key == eng::Key::LControl)
00021                 {
00022                     m_showDebug = !m_showDebug;
00023                 }
00024                 break;
00025
00026             case eng::EventType::KeyReleased:
00027                 m_keysPressed[event.key] = false;
00028                 break;
00029
00030             default:
00031                 break;
00032         }
00033     }
00034
00035     updateKeyboardInput(scene->getRegistry());
00036 }
00037
00038 void cli::Client::updateKeyboardInput(ecs::Registry &registry)
00039 {
00040     const auto keyboardEntities = registry.getAll<ecs::KeyboardInput>();
00041     ecs::Entity keyboardEntity = 0;
00042
00043     if (keyboardEntities.empty())
00044     {
00045         keyboardEntity = registry.createEntity().with<ecs::KeyboardInput>("keyboard_input").build();
00046     }
00047     else
00048     {
00049         keyboardEntity = keyboardEntities.begin()->first;
00050     }
00051     if (auto *keyboardInput = registry.getComponent<ecs::KeyboardInput>(keyboardEntity); keyboardInput != nullptr)
00052     {
00053         keyboardInput->space_pressed = m_keysPressed[eng::Key::Space];
00054         keyboardInput->up_pressed = m_keysPressed[eng::Key::Up];
00055         keyboardInput->down_pressed = m_keysPressed[eng::Key::Down];
00056         keyboardInput->left_pressed = m_keysPressed[eng::Key::Left];
00057     }
00058 }
```

```

00057     keyboardInput->right_pressed = m_keysPressed[eng::Key::Right];
00058 }
00059 }

```

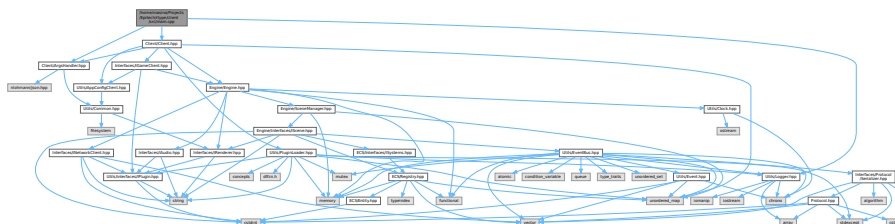
19.33 /home/masina/Projects/Epitech/rtype/client/src/main.cpp File Reference

```

#include "Client/ArgsHandler.hpp"
#include "Client/Client.hpp"
#include "Utils/Logger.hpp"

```

Include dependency graph for main.cpp:



Functions

- `int main (const int argc, const char *const *argv, const char *const *env)`

19.33.1 Function Documentation

19.33.1.1 main()

```

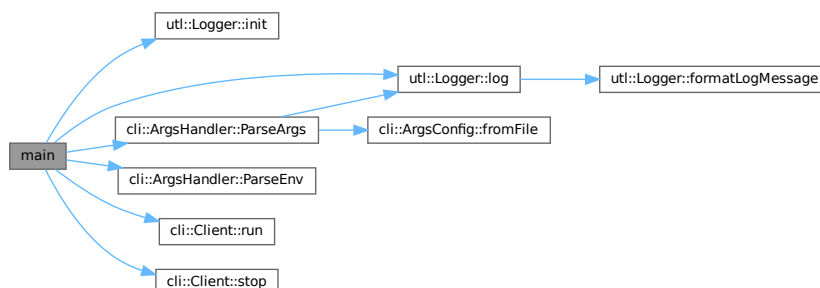
int main (
    const int argc,
    const char *const * argv,
    const char *const * env)

```

Definition at line 5 of file [main.cpp](#).

References [cli::ArgsConfig::exit](#), [utl::Logger::init\(\)](#), [utl::Logger::log\(\)](#), [cli::ArgsHandler::ParseArgs\(\)](#), [cli::ArgsHandler::ParseEnv\(\)](#), [cli::Client::run\(\)](#), [cli::Client::stop\(\)](#), and [utl::WARNING](#).

Here is the call graph for this function:



19.34 main.cpp

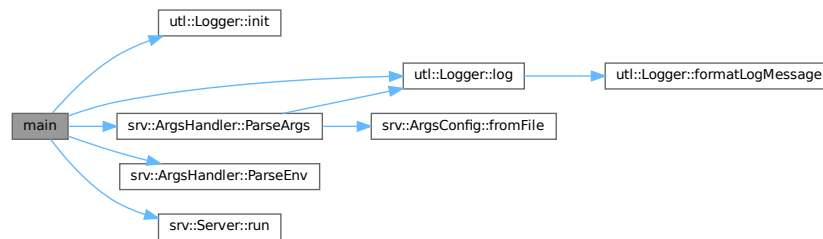
[Go to the documentation of this file.](#)

```

00001 #include "Client/ArgsHandler.hpp"
00002 #include "Client/Client.hpp"
00003 #include "Utils/Logger.hpp"

```


Here is the call graph for this function:



19.36 main.cpp

[Go to the documentation of this file.](#)

```

00001 #include "Server/ArgsHandler.hpp"
00002 #include "Server/Server.hpp"
00003 #include "Utils/Logger.hpp"
00004
00005 int main(const int argc, const char *const argv[], const char *const env[])
00006 {
00007     utl::Logger::init();
00008     try
00009     {
00010         const srv::ArgsConfig argsConf = srv::ArgsHandler::ParseArgs(argc, argv);
00011         const srv::EnvConfig envConf = srv::ArgsHandler::ParseEnv(env);
00012         if (argsConf.exit)
00013         {
00014             return EXIT_SUCCESS;
00015         }
00016         const srv::Server server(argsConf);
00017         server.run();
00018     }
00019     catch (const std::exception &e)
00020     {
00021         utl::Logger::log(std::string("Exception: ") + e.what(), utl::LogLevel::WARNING);
00022         return EXIT_FAILURE;
00023     }
00024     catch (...)
00025     {
00026         utl::Logger::log("Unknown exception", utl::LogLevel::WARNING);
00027         return EXIT_FAILURE;
00028     }
00029     return EXIT_SUCCESS;
00030 }

```

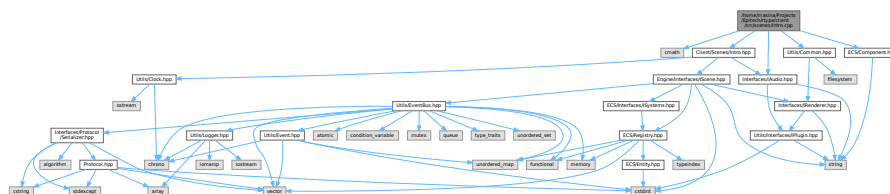
19.37 /home/masina/Projects/↵ Epitech/rtype/client/src/scenes/intro.cpp File Reference

```

#include <cmath>
#include "Client/Scenes/Intro.hpp"
#include "ECS/Component.hpp"
#include "Interfaces/IAudio.hpp"
#include "Utils/Common.hpp"

```

Include dependency graph for intro.cpp:



19.38 intro.cpp

[Go to the documentation of this file.](#)

```

00001 #include <cmath>
00002
00003 #include "Client/Scenes/Intro.hpp"
00004 #include "ECS/Component.hpp"
00005 #include "Interfaces/IAudio.hpp"
00006 #include "Utils/Common.hpp"
00007
00008 cli::Intro::Intro(const eng::id assignedId, const std::shared_ptr<eng::IRenderer> &renderer,
00009                  const std::shared_ptr<eng::IAudio> &audio)
00010 : AScene(assignedId), m_renderer(renderer), m_audio(audio)
00011 {
00012     auto &registry = AScene::getRegistry();
00013
00014     registry.onComponentAdded(
00015         [&renderer, &registry, &audio](const ecs::Entity e, const std::type_info &type)
00016         {
00017             const auto *audioComp = registry.getComponent<ecs::Audio>(e);
00018             const auto *colorComp = registry.getComponent<ecs::Color>(e);
00019             const auto *scaleComp = registry.getComponent<ecs::Scale>(e);
00020             const auto *textureComp = registry.getComponent<ecs::Texture>(e);
00021             const auto *transform = registry.getComponent<ecs::Transform>(e);
00022             const auto *rectComp = registry.getComponent<ecs::Rect>(e);
00023
00024             if (type == typeid(ecs::Texture))
00025             {
00026                 const float scale_x = scaleComp ? scaleComp->x : 1.F;
00027                 const float scale_y = scaleComp ? scaleComp->y : 1.F;
00028
00029                 renderer->createTexture(textureComp->id, textureComp->path);
00030
00031                 if (transform && textureComp)
00032                 {
00033                     if (rectComp)
00034                     {
00035                         renderer->createSprite(textureComp->id + std::to_string(e), textureComp->id, transform->x,
00036                                                 transform->y, scale_x, scale_y, static_cast<int>(rectComp->pos_x),
00037                                                 static_cast<int>(rectComp->pos_y), rectComp->size_x, rectComp->size_y);
00038                     }
00039                     else
00040                     {
00041                         renderer->createSprite(textureComp->id + std::to_string(e), textureComp->id, transform->x,
00042                                                 transform->y);
00043                     }
00044                 }
00045             }
00046             else if (type == typeid(ecs::Audio))
00047             {
00048                 if (audioComp)
00049                 {
00050                     audio->createAudio(audioComp->path, audioComp->volume, audioComp->loop,
00051                                         audioComp->id + std::to_string(e));
00052                 }
00053             }
00054         });
00055
00056     m_soundEntity =
00057         registry.createEntity().with<ecs::Audio>("game_begin", "assets/audio/elephant.mp3", 20.0F, false, true).build();
00058
00059     auto [width, height] = renderer->getWindowSize();
00060     m_logoEntity = registry.createEntity()
00061         .with<ecs::Transform>("logo_transform", width * 0.37F, height * 0.2F)
00062         .with<ecs::Scale>("logo_scale", 1.5F, 1.5F)
00063         .with<ecs::Color>("logo_color", utl::Config::Color::WHITE_LOW.r, utl::Config::Color::WHITE_LOW.g,
00064                          utl::Config::Color::WHITE_LOW.b, utl::Config::Color::WHITE_LOW.a)
00065         .with<ecs::Texture>("logo_text", utl::Path::Icons::ICON_APP)
00066         .build();
00067 }
00068
00069 void cli::Intro::update(const float dt, const eng::WindowSize &size)
00070 {
00071     auto &reg = getRegistry();
00072
00073     auto *color = reg.getComponent<ecs::Color>(m_logoEntity);
00074     m_elapsedTime += dt;
00075
00076     if (color == nullptr)
00077     {
00078         return;
00079     }
00080
00081     if (m_elapsedTime < 2.0F)
00082     {
00083         color->a = static_cast<uint8_t>(std::min(255.0F, (m_elapsedTime / 2.0F) * 255.0F));
00084     }
00085 }

```

```

00084     }
00085     else if (m_elapsedTime < 6.0F)
00086     {
00087         color->a = 255;
00088     }
00089     else if (m_elapsedTime < 8.0f)
00090     {
00091         const float t = (m_elapsedTime - 6.0F) / 2.0F;
00092         color->a = static_cast<uint8_t>(255.0F * (1.0F - t));
00093     }
00094     else
00095     {
00096         onLeave();
00097     }
00098 }
00099
00100 void cli::Intro::event(const eng::Event &event) {}

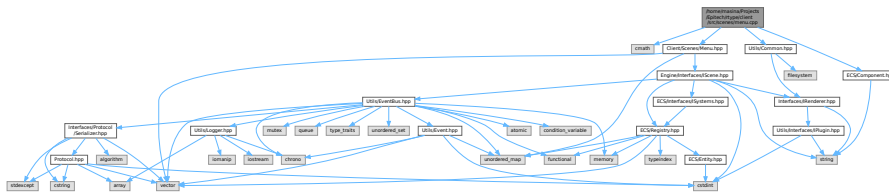
```

19.39 /home/masina/Projects/↵ Epitech/rtype/client/src/scenes/menu.cpp File Reference

```

#include <cmath>
#include "Client/Scenes/Menu.hpp"
#include "ECS/Component.hpp"
#include "Utils/Common.hpp"
Include dependency graph for menu.cpp:

```



19.40 menu.cpp

[Go to the documentation of this file.](#)

```

00001 #include <cmath>
00002
00003 #include "Client/Scenes/Menu.hpp"
00004 #include "ECS/Component.hpp"
00005 #include "Utils/Common.hpp"
00006
00007 cli::Menu::Menu(const eng::id assignedId, const std::shared_ptr<eng::IRenderer> &renderer) : AScene(assignedId)
00008 {
00009     auto &registry = AScene::getRegistry();
00010
00011     registry.onComponentAdded(
00012         [&renderer, &registry](const ecs::Entity e, const std::type_info &type)
00013         {
00014             const auto *colorComp = registry.getComponent<ecs::Color>(e);
00015             const auto *fontComp = registry.getComponent<ecs::Font>(e);
00016             const auto *rectComp = registry.getComponent<ecs::Rect>(e);
00017             const auto *scaleComp = registry.getComponent<ecs::Scale>(e);
00018             const auto *textComp = registry.getComponent<ecs::Text>(e);
00019             const auto *textureComp = registry.getComponent<ecs::Texture>(e);
00020             const auto *transform = registry.getComponent<ecs::Transform>(e);
00021
00022             if (type == typeid(ecs::Text))
00023             {
00024                 if (textComp && transform && fontComp)
00025                 {
00026                     renderer->createFont(fontComp->id, fontComp->path);
00027                     renderer->createText(
00028                         {font_name = fontComp->id,
00029                          .color = {r = colorComp->r, .g = colorComp->g, .b = colorComp->b, .a = colorComp->a},
00029                          .content = textComp->content,
00030                          .size = textComp->font_size,
00031                          .x = transform->x,
00032                          .y = transform->y,
00033                          .name = textComp->id});
00034                 }
00035             }
00036         }
00037     );
00038 }

```

```

00036     }
00037     else if (type == typeid(ecs::Texture))
00038     {
00039         const float scale_x = scaleComp ? scaleComp->x : 1.F;
00040         const float scale_y = scaleComp ? scaleComp->y : 1.F;
00041
00042         renderer->createTexture(textureComp->id, textureComp->path);
00043
00044         if (transform && textureComp)
00045         {
00046             if (rectComp)
00047             {
00048                 renderer->createSprite(textureComp->id + std::to_string(e), textureComp->id, transform->x,
00049                                     transform->y, scale_x, scale_y, static_cast<int>(rectComp->pos_x),
00050                                     static_cast<int>(rectComp->pos_y), rectComp->size_x, rectComp->size_y);
00051             }
00052             else
00053             {
00054                 renderer->createSprite(textureComp->id + std::to_string(e), textureComp->id, transform->x,
00055                                     transform->y);
00056             }
00057         }
00058     }
00059 });
00060
00061 m_titleEntity =
00062     registry.createEntity()
00063     .with<ecs::Font>("main_font", utl::Path::Font::FONTS_RTYPE)
00064     .with<ecs::Transform>("transform_title", 100.F, 60.F, 0.F)
00065     .with<ecs::Color>("color_title", utl::Config::Color::CYAN_ELECTRIC.r,
00066                     utl::Config::Color::CYAN_ELECTRIC.g,
00067                     utl::Config::Color::CYAN_ELECTRIC.b, utl::Config::Color::CYAN_ELECTRIC.a)
00068     .with<ecs::Text>("id", std::string("RTYPE"), 72U)
00069     .build();
00070
00071 for (size_t i = 0; i < m_menuOptions.size(); ++i)
00072 {
00073     registry.createEntity()
00074     .with<ecs::Font>("main_font", utl::Path::Font::FONTS_RTYPE)
00075     .with<ecs::Transform>("transform_menu", 100.F, 200.F + i * 60.F, 0.F)
00076     .with<ecs::Color>("color_menu", utl::Config::Color::GRAY_BLUE_SUBTLE.r,
00077                     utl::Config::Color::GRAY_BLUE_SUBTLE.g, utl::Config::Color::GRAY_BLUE_SUBTLE.b,
00078                     utl::Config::Color::GRAY_BLUE_SUBTLE.a)
00079     .with<ecs::Text>("menu_" + m_menuOptions[i], m_menuOptions[i], 32U)
00080     .build();
00081 }
00082
00083 std::string contributorsText = "Contributors ";
00084 for (size_t i = 0; i < m_contributors.size(); ++i)
00085 {
00086     contributorsText += m_contributors[i];
00087     if (i < m_contributors.size() - 1)
00088     {
00089         contributorsText += " ";
00090     }
00091 }
00092 m_contributorsEntity =
00093     registry.createEntity()
00094     .with<ecs::Font>("main_font", utl::Path::Font::FONTS_RTYPE)
00095     .with<ecs::Transform>("transform_contributors", renderer->getWindowSize().width * 0.9F,
00096                         renderer->getWindowSize().height * 0.9F)
00097     .with<ecs::Color>("color_contributors", utl::Config::Color::GRAY_BLUE_SUBTLE.r,
00098                     utl::Config::Color::GRAY_BLUE_SUBTLE.g, utl::Config::Color::GRAY_BLUE_SUBTLE.b,
00099                     utl::Config::Color::GRAY_BLUE_SUBTLE.a)
00100     .with<ecs::Text>("contributors_text", contributorsText, 24U)
00101     .build();
00102
00103 m_selectedIndex = 2;
00104 }
00105 void cli::Menu::update(const float dt, const eng::WindowSize &size)
00106 {
00107     auto &reg = getRegistry();
00108
00109     auto &transforms = reg.getAll<ecs::Transform>();
00110     auto &colors = reg.getAll<ecs::Color>();
00111     m_animationTime += dt;
00112     m_titlePulseTime += dt;
00113
00114     if (auto *titleColor = reg.getComponent<ecs::Color>(m_titleEntity))
00115     {
00116         const float pulse = (std::sin(m_titlePulseTime * 1.2f) + 1.0f) * 0.5f;
00117         titleColor->r = static_cast<uint8_t>(utl::Config::Color::CYAN_ELECTRIC.r * (0.8f + pulse * 0.2f));
00118         titleColor->g = static_cast<uint8_t>(utl::Config::Color::CYAN_ELECTRIC.g * (0.8f + pulse * 0.2f));
00119         titleColor->b = static_cast<uint8_t>(utl::Config::Color::CYAN_ELECTRIC.b * (0.9f + pulse * 0.1f));
00120     }
00121 }

```

```

00122     if (auto *titleTransform = reg.getComponent<ecs::Transform>(m_titleEntity))
00123     {
00124         titleTransform->y = 60.0f + std::sin(m_titlePulseTime * 0.8f) * 2.0f;
00125     }
00126
00127     auto &texts = reg.getAll<ecs::Text>();
00128     size_t i = 0;
00129     for (auto &[entity, text] : texts)
00130     {
00131         if (text.content == "Solo" || text.content == "Multi" || text.content == "Settings")
00132         {
00133             auto &color = colors.at(entity);
00134
00135             if (i == m_selectedIndex)
00136             {
00137                 const float glowIntensity = std::sin(m_animationTime * 2.5f);
00138                 color.r = 0U;
00139                 color.g = static_cast<unsigned char>(191U + (glowIntensity * 50));
00140                 color.b = 255U;
00141             }
00142             else
00143             {
00144                 color.r = utl::Config::Color::GRAY_BLUE_SUBTLE.r;
00145                 color.g = utl::Config::Color::GRAY_BLUE_SUBTLE.g;
00146                 color.b = utl::Config::Color::GRAY_BLUE_SUBTLE.b;
00147             }
00148
00149             i++;
00150         }
00151     }
00152     m_contributorsOffset += dt * 50.0f;
00153     if (auto *contributorsTransform = reg.getComponent<ecs::Transform>(m_contributorsEntity))
00154     {
00155         contributorsTransform->x = (size.width * 0.9f) - m_contributorsOffset;
00156
00157         if (contributorsTransform->x < -(size.width * 0.9f))
00158         {
00159             m_contributorsOffset = 0.0f;
00160             contributorsTransform->x = size.width * 0.9f;
00161         }
00162     }
00163 }
00164
00165 void cli::Menu::event(const eng::Event &event)
00166 {
00167     switch (event.type)
00168     {
00169     case eng::EventType::KeyPressed:
00170     {
00171         const int previousIndex = m_selectedIndex;
00172         bool handledNavigation = false;
00173
00174         if (event.key == eng::Key::Up)
00175         {
00176             handledNavigation = true;
00177             if (m_selectedIndex == 2)
00178             {
00179                 m_selectedIndex = 0;
00180             }
00181             else
00182             {
00183                 m_selectedIndex++;
00184             }
00185         }
00186         else if (event.key == eng::Key::Down)
00187         {
00188             handledNavigation = true;
00189             if (m_selectedIndex == 0)
00190             {
00191                 m_selectedIndex = 2;
00192             }
00193             else
00194             {
00195                 m_selectedIndex--;
00196             }
00197         }
00198         else if (event.key == eng::Key::Enter)
00199         {
00200             const std::string &selectedOption =
00201                 m_menuOptions[static_cast<int>(m_menuOptions.size() - 1 - m_selectedIndex)];
00202             if (onOptionSelected)
00203             {
00204                 onOptionSelected(selectedOption);
00205             }
00206         }
00207
00208         if (handledNavigation && m_selectedIndex != previousIndex)

```

```

00209     {
00210         m_playMusic = true;
00211     }
00212     break;
00213 }
00214
00215 case eng::EventType::KeyReleased:
00216     if (event.key == eng::Key::Up)
00217     {
00218         m_keysPressed[eng::Key::Up] = false;
00219     }
00220     if (event.key == eng::Key::Down)
00221     {
00222         m_keysPressed[eng::Key::Down] = false;
00223     }
00224     if (event.key == eng::Key::Left)
00225     {
00226         m_keysPressed[eng::Key::Left] = false;
00227     }
00228     if (event.key == eng::Key::Right)
00229     {
00230         m_keysPressed[eng::Key::Right] = false;
00231     }
00232     if (event.key == eng::Key::Space)
00233     {
00234         m_keysPressed[eng::Key::Space] = false;
00235     }
00236     break;
00237
00238 default:
00239     break;
00240 }
00241 }

```

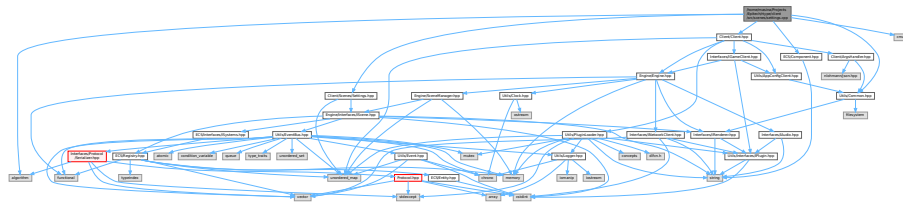
19.41 /home/masina/Projects/↵ Epitech/rtype/client/src/scenes/settings.cpp File Reference

```

#include <algorithm>
#include <cmath>
#include "Client/Client.hpp"
#include "Client/Scenes/Settings.hpp"
#include "ECS/Component.hpp"
#include "Utils/Common.hpp"

```

Include dependency graph for settings.cpp:



19.42 settings.cpp

[Go to the documentation of this file.](#)

```

00001 #include <algorithm>
00002 #include <cmath>
00003
00004 #include "Client/Client.hpp"
00005 #include "Client/Scenes/Settings.hpp"
00006 #include "ECS/Component.hpp"
00007 #include "Utils/Common.hpp"
00008
00009 cli::Settings::Settings(const eng::id assignedId, const std::shared_ptr<eng::IRenderer> &renderer,
00010                        utl::cli::AppConfig &config)
00011 : AScene(assignedId), m_renderer(renderer), m_appConfig(config)
00012 {
00013     auto &registry = AScene::getRegistry();
00014
00015     registry.onComponentAdded(
00016         [&renderer, &registry](const ecs::Entity e, const std::type_info &type)

```

```

00017     {
00018         const auto *colorComp = registry.getComponent<ecs::Color>(e);
00019         const auto *fontComp = registry.getComponent<ecs::Font>(e);
00020         const auto *rectComp = registry.getComponent<ecs::Rect>(e);
00021         const auto *scaleComp = registry.getComponent<ecs::Scale>(e);
00022         const auto *textComp = registry.getComponent<ecs::Text>(e);
00023         const auto *textureComp = registry.getComponent<ecs::Texture>(e);
00024         const auto *transform = registry.getComponent<ecs::Transform>(e);
00025
00026         if (type == typeid(ecs::Text))
00027         {
00028             if (textComp && transform && fontComp)
00029             {
00030                 renderer->createFont(fontComp->id, fontComp->path);
00031                 renderer->createText(
00032                     {font_name = fontComp->id,
00033                      .color = {r = colorComp->r, .g = colorComp->g, .b = colorComp->b, .a = colorComp->a},
00034                      .content = textComp->content,
00035                      .size = textComp->font_size,
00036                      .x = transform->x,
00037                      .y = transform->y,
00038                      .name = textComp->id});
00039             }
00040         }
00041         else if (type == typeid(ecs::Texture))
00042         {
00043             const float scale_x = scaleComp ? scaleComp->x : 1.F;
00044             const float scale_y = scaleComp ? scaleComp->y : 1.F;
00045
00046             renderer->createTexture(textureComp->id, textureComp->path);
00047
00048             if (transform && textureComp)
00049             {
00050                 if (rectComp)
00051                 {
00052                     renderer->createSprite(textureComp->id + std::to_string(e), textureComp->id, transform->x,
00053                                             transform->y, scale_x, scale_y, static_cast<int>(rectComp->pos_x),
00054                                             static_cast<int>(rectComp->pos_y), rectComp->size_x, rectComp->size_y);
00055                 }
00056                 else
00057                 {
00058                     renderer->createSprite(textureComp->id + std::to_string(e), textureComp->id, transform->x,
00059                                             transform->y);
00060                 }
00061             }
00062         }
00063     });
00064
00065     m_titleEntity =
00066         registry.createEntity()
00067             .with<ecs::Font>("main_font", utl::Path::Font::FONTS_RTYPE)
00068             .with<ecs::Transform>("transform_title", 100.F, 60.F, 0.F)
00069             .with<ecs::Color>("color_title", utl::Config::Color::CYAN_ELECTRIC.r,
00070                             utl::Config::Color::CYAN_ELECTRIC.g,
00071                             utl::Config::Color::CYAN_ELECTRIC.b, utl::Config::Color::CYAN_ELECTRIC.a)
00072             .with<ecs::Text>("title", std::string("SETTINGS"), 72U)
00073             .build();
00074     for (size_t i = 0; i < m_settingsOptions.size(); ++i)
00075     {
00076         registry.createEntity()
00077             .with<ecs::Font>("main_font", utl::Path::Font::FONTS_RTYPE)
00078             .with<ecs::Transform>("transform_setting_" + std::to_string(i), 100.F, 200.F + i * 50.F, 0.F)
00079             .with<ecs::Color>("color_setting_" + std::to_string(i), utl::Config::Color::GRAY_BLUE_SUBTLE.r,
00080                             utl::Config::Color::GRAY_BLUE_SUBTLE.g, utl::Config::Color::GRAY_BLUE_SUBTLE.b,
00081                             utl::Config::Color::GRAY_BLUE_SUBTLE.a)
00082             .with<ecs::Text>("setting_" + m_settingsOptions[i], m_settingsOptions[i], 32U)
00083             .build();
00084     }
00085     m_volumeValueEntity =
00086         registry.createEntity()
00087             .with<ecs::Font>("main_font", utl::Path::Font::FONTS_RTYPE)
00088             .with<ecs::Transform>("transform_volume_value", 580.F, 200.F, 0.F)
00089             .with<ecs::Color>("color_volume_value", utl::Config::Color::TEXT_VALUE_COLOR.r,
00090                             utl::Config::Color::TEXT_VALUE_COLOR.g, utl::Config::Color::TEXT_VALUE_COLOR.b,
00091                             utl::Config::Color::TEXT_VALUE_COLOR.a)
00092             .with<ecs::Text>("volume_value", std::string("50"), 24U)
00093             .build();
00094     m_qualityValueEntity =
00095         registry.createEntity()
00096             .with<ecs::Font>("main_font", utl::Path::Font::FONTS_RTYPE)
00097             .with<ecs::Transform>("transform_quality_value", 580.F, 250.F, 0.F)
00098             .with<ecs::Color>("color_quality_value", utl::Config::Color::TEXT_VALUE_COLOR.r,
00099                             utl::Config::Color::TEXT_VALUE_COLOR.g, utl::Config::Color::TEXT_VALUE_COLOR.b,
00100                             utl::Config::Color::TEXT_VALUE_COLOR.a)
00101             .with<ecs::Text>("quality_value", std::string("Medium"), 24U)
00102             .build();

```

```

00103 m_controlValueEntity =
00104     registry.createEntity()
00105     .with<ecs::Font>("main_font", utl::Path::Font::FONTS_RTYPE)
00106     .with<ecs::Transform>("transform_control_value", 580.F, 300.F, 0.F)
00107     .with<ecs::Color>("color_control_value", utl::Config::Color::TEXT_VALUE_COLOR.r,
00108         utl::Config::Color::TEXT_VALUE_COLOR.g, utl::Config::Color::TEXT_VALUE_COLOR.b,
00109         utl::Config::Color::TEXT_VALUE_COLOR.a)
00110     .with<ecs::Text>("control_value", std::string("WASD"), 24U)
00111     .build();
00112
00113 m_skinSpriteEntity =
00114     registry.createEntity()
00115     .with<ecs::Transform>("transform_skin_sprite", 580.F, 345.F, 0.F)
00116     .with<ecs::Scale>("scale_skin_sprite", 2.0f, 2.0f)
00117     .with<ecs::Color>("color_skin_sprite", utl::Config::Color::WHITE.r, utl::Config::Color::WHITE.g,
00118         utl::Config::Color::WHITE.b, utl::Config::Color::WHITE.a)
00119     .with<ecs::Rect>("rect_skin_sprite", 0.0f, 0.0f, 33, 17)
00120     .with<ecs::Texture>("skin_sprite", "assets/sprites/r-typesheet42.gif")
00121     .build();
00122
00123 registry.createEntity()
00124     .with<ecs::Font>("main_font", utl::Path::Font::FONTS_RTYPE)
00125     .with<ecs::Transform>("transform_instruction", 80.F, 480.F, 0.F)
00126     .with<ecs::Color>("color_instruction", utl::Config::Color::INFO_TEXT_COLOR.r,
00127         utl::Config::Color::INFO_TEXT_COLOR.g, utl::Config::Color::INFO_TEXT_COLOR.b,
00128         utl::Config::Color::INFO_TEXT_COLOR.a)
00129     .with<ecs::Text>("instruction", std::string("UP/DOWN navigate, LEFT/RIGHT change, ESC back"), 16U)
00130     .build();
00131
00132 m_selectedIndex = 0;
00133 loadFromConfig();
00134 }
00135 void cli::Settings::update(const float dt, const eng::WindowSize & /*size*/)
00136 {
00137     auto &reg = getRegistry();
00138     auto &colors = reg.getAll<ecs::Color>();
00139     auto &texts = reg.getAll<ecs::Text>();
00140
00141     m_animationTime += dt;
00142     m_titlePulseTime += dt;
00143     for (auto &[entity, text] : texts)
00144     {
00145         for (size_t i = 0; i < m_settingsOptions.size(); ++i)
00146         {
00147             if (text.id == "setting_" + m_settingsOptions[i])
00148             {
00149                 auto &color = colors.at(entity);
00150
00151                 if (i == m_selectedIndex)
00152                 {
00153                     float glowIntensity = std::sin(m_animationTime * 2.5f);
00154                     color.r = 0U;
00155                     color.g = static_cast<unsigned char>(191U + glowIntensity * 50);
00156                     color.b = 255U;
00157                 }
00158                 else
00159                 {
00160                     color.r = utl::Config::Color::GRAY_BLUE_SUBTLE.r;
00161                     color.g = utl::Config::Color::GRAY_BLUE_SUBTLE.g;
00162                     color.b = utl::Config::Color::GRAY_BLUE_SUBTLE.b;
00163                 }
00164                 break;
00165             }
00166         }
00167     }
00168     if (auto *titleColor = reg.getComponent<ecs::Color>(m_titleEntity))
00169     {
00170         const float pulsation = std::sin(m_titlePulseTime * 2.0f) * 0.4f + 0.6f;
00171         titleColor->r = static_cast<unsigned char>(utl::Config::Color::CYAN_ELECTRIC.r * pulsation);
00172         titleColor->g = static_cast<unsigned char>(utl::Config::Color::CYAN_ELECTRIC.g * pulsation);
00173         titleColor->b = static_cast<unsigned char>(utl::Config::Color::CYAN_ELECTRIC.b * pulsation);
00174     }
00175     updateSettingsDisplay();
00176 }
00177
00178 void cli::Settings::updateSettingsDisplay()
00179 {
00180     auto &registry = getRegistry();
00181
00182     if (auto *volumeValueText = registry.getComponent<ecs::Text>(m_volumeValueEntity))
00183     {
00184         volumeValueText->content = std::to_string(m_audioVolume);
00185     }
00186     if (auto *qualityValueText = registry.getComponent<ecs::Text>(m_qualityValueEntity))
00187     {
00188         const std::vector<std::string> fpsOptions = {"60 FPS", "144 FPS", "240 FPS"};
00189     }

```



```

00190     qualityValueText->content = fpsOptions[static_cast<size_t>(m_videoQuality)];
00191 }
00192 if (auto *controlValueText = registry.getComponent<ecs::Text>(m_controlValueEntity))
00193 {
00194     const std::vector<std::string> controlSchemes = {"WASD", "ZQSD", "Arrows"};
00195     controlValueText->content = controlSchemes[static_cast<size_t>(m_controlScheme)];
00196 }
00197 if (auto *skinRect = registry.getComponent<ecs::Rect>(m_skinSpriteEntity))
00198 {
00199     const std::vector shipLines = {0.0f, 17.0f, 34.0f, 51.0f, 68.0f};
00200
00201     skinRect->pos_y = shipLines[static_cast<size_t>(m_skinIndex)];
00202     skinRect->pos_x = 0.0f;
00203     skinRect->size_x = 33U;
00204     skinRect->size_y = 17U;
00205 }
00206 }
00207
00208 void cli::Settings::event(const eng::Event &event)
00209 {
00210     switch (event.type)
00211     {
00212     case eng::EventType::KeyPressed:
00213         if (event.key == eng::Key::Escape)
00214         {
00215             onLeave();
00216         }
00217         else if (event.key == eng::Key::Up)
00218         {
00219             m_selectedIndex = (m_selectedIndex == 0) ? m_settingsOptions.size() - 1 : m_selectedIndex - 1;
00220             m_playMusic = true;
00221         }
00222         else if (event.key == eng::Key::Down)
00223         {
00224             m_selectedIndex = (m_selectedIndex == m_settingsOptions.size() - 1) ? 0 : m_selectedIndex + 1;
00225             m_playMusic = true;
00226         }
00227         else if (event.key == eng::Key::Enter)
00228         {
00229             if (const std::string &selectedOption = m_settingsOptions[m_selectedIndex];
00230                 selectedOption == "Back to Menu")
00231             {
00232                 onLeave();
00233             }
00234         }
00235         else if (event.key == eng::Key::Left || event.key == eng::Key::Right)
00236         {
00237
00238             if (const std::string &selectedOption = m_settingsOptions[m_selectedIndex];
00239                 selectedOption == "Audio Volume")
00240             {
00241                 const float newVolume = m_audioVolume + ((event.key == eng::Key::Right) ? 0.01F : -0.01F);
00242                 m_audioVolume = (std::max)(0.0F, (std::min)(10.0F, newVolume));
00243                 m_appConfig.audioVolume = m_audioVolume;
00244             }
00245             else if (selectedOption == "FPS")
00246             {
00247                 if (event.key == eng::Key::Left)
00248                 {
00249                     m_videoQuality = (m_videoQuality == 0) ? 2 : m_videoQuality - 1;
00250                 }
00251                 else
00252                 {
00253                     m_videoQuality = (m_videoQuality == 2) ? 0 : m_videoQuality + 1;
00254                 }
00255                 m_appConfig.videoQuality = m_videoQuality;
00256                 applyVideoQuality();
00257             }
00258             else if (selectedOption == "Controls")
00259             {
00260                 if (event.key == eng::Key::Left)
00261                 {
00262                     m_controlScheme = (m_controlScheme == 0) ? 2 : m_controlScheme - 1;
00263                 }
00264                 else
00265                 {
00266                     m_controlScheme = (m_controlScheme == 2) ? 0 : m_controlScheme + 1;
00267                 }
00268                 m_appConfig.controlScheme = m_controlScheme;
00269             }
00270             else if (selectedOption == "Skin")
00271             {
00272                 if (event.key == eng::Key::Left)
00273                 {
00274                     m_skinIndex = (m_skinIndex == 0) ? 4 : m_skinIndex - 1;
00275                 }
00276                 else

```

```

00277         {
00278             m_skinIndex = (m_skinIndex == 4) ? 0 : m_skinIndex + 1;
00279         }
00280         m_appConfig.skinIndex = m_skinIndex;
00281
00282         applySkinChange();
00283     }
00284     updateSettingsDisplay();
00285 }
00286 break;
00287 default:
00288     break;
00289 }
00290 }
00291
00292 void cli::Settings::loadFromConfig()
00293 {
00294     m_audioVolume = m_appConfig.audioVolume;
00295     m_videoQuality = m_appConfig.videoQuality;
00296     m_controlScheme = m_appConfig.controlScheme;
00297     m_skinIndex = m_appConfig.skinIndex;
00298
00299     updateSettingsDisplay();
00300     applyVideoQuality();
00301 }
00302
00303 void cli::Settings::applyVideoQuality() {}
00304
00305 void cli::Settings::applySkinChange() {}

```

19.43 [/home/masina/Projects/Epitech/rtype/CONTRIBUTING.md](#)
File Reference

19.44 [/home/masina/Projects/↵
Epitech/rtype/documentation/comparative_study.md](#) File
Reference

19.45 [/home/masina/Projects/Epitech/rtype/documentation/network↵
_protocol_specification.md](#) File Reference

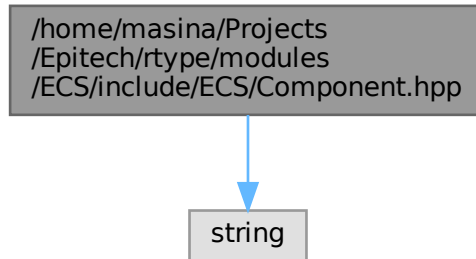
19.46 [/home/masina/Projects/Epitech/rtype/LICENSE.md](#) File
Reference

19.47 [/home/masina/Projects/Epitech/rtype/modules/ECS/include/↵
ECS/Component.hpp](#) File Reference

This file contains the component definitions.

```
#include <string>
```

Include dependency graph for Component.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- struct `ecs::IComponent`
- struct `ecs::Audio`
- struct `ecs::Color`
- struct `ecs::Font`
- struct `ecs::Player`
- struct `ecs::Pixel`
- struct `ecs::Rect`
- struct `ecs::Scale`
- struct `ecs::Text`
- struct `ecs::Texture`
- struct `ecs::Transform`
- struct `ecs::Velocity`
- struct `ecs::Animation`
- struct `ecs::Projectile`
- struct `ecs::BeamCharge`
- struct `ecs::LoadingAnimation`
- struct `ecs::Enemy`
- struct `ecs::Explosion`
- struct `ecs::Hitbox`
- struct `ecs::KeyboardInput`
- struct `ecs::Floor`
- struct `ecs::Ceiling`
- struct `ecs::Scrolling`
- struct `ecs::Score`
- struct `ecs::Layer`

Namespaces

- namespace `ecs`

19.47.1 Detailed Description

This file contains the component definitions.
Definition in file [Component.hpp](#).

19.48 Component.hpp

[Go to the documentation of this file.](#)

```

00001 ///  

00002 ///  

00003 ///  

00004 ///  

00005 ///  

00006 ///  

00007 #pragma once  

00008  

00009 #include <string>  

00010  

00011 namespace ecs  

00012 {  

00013     struct IComponent  

00014     {  

00015         std::string id;  

00016     };  

00017     struct Audio final : IComponent  

00018     {  

00019         std::string path;  

00020         float volume;  

00021         bool loop;  

00022         bool play;  

00023     };  

00024     struct Color final : IComponent  

00025     {  

00026         unsigned char r{};  

00027         unsigned char g{};  

00028         unsigned char b{};  

00029         unsigned char a{};  

00030     };  

00031     struct Font final : IComponent  

00032     {  

00033         std::string path;  

00034     };  

00035     struct Player final : IComponent  

00036     {  

00037         bool is_cli{};  

00038         // bool is_alive{};  

00039     };  

00040     struct Pixel final : IComponent  

00041     {  

00042     };  

00043     struct Rect final : IComponent  

00044     { // TODO(bobis33): remove, only used for texture actually  

00045         float pos_x{}, pos_y{};  

00046         int size_x{}, size_y{};  

00047     };  

00048     struct Scale final : IComponent  

00049     {  

00050         float x{}, y{};  

00051     };  

00052     struct Text final : IComponent  

00053     {  

00054         std::string content;  

00055         unsigned int font_size;  

00056     };  

00057     struct Texture final : IComponent  

00058     {  

00059         std::string path;  

00060         // float rect_pos_x{}, rect_pos_y{};  

00061         // int rect_size_x{}, rect_size_y{};  

00062     };  

00063     struct Transform final : IComponent  

00064     {  

00065         float x{}, y{};  

00066         float rotation{};  

00067     };  

00068     struct Velocity final : IComponent  

00069     {  

00070         float x{}, y{};  

00071     };  

00072     struct Animation final : IComponent  

00073     {  

00074         int current_frame{};  

00075         int total_frames{};  

00076         float frame_duration{};

```

```

00077     float current_time{};
00078     int frame_width{};
00079     int frame_height{};
00080     int frames_per_row{};
00081 };
00082 struct Projectile final : IComponent
00083 {
00084     enum Type
00085     {
00086         BASIC,
00087         SUPERCHARGED
00088     };
00089     Type type;
00090     float damage;
00091     float lifetime;
00092     float current_lifetime;
00093     int pierce_remaining; // number of enemies it can still pass through (>=1)
00094 };
00095
00096 struct BeamCharge final : IComponent
00097 {
00098     float current_charge;
00099     float max_charge;
00100 };
00101
00102 struct LoadingAnimation final : IComponent
00103 {
00104     int current_frame;
00105     int total_frames;
00106     float frame_duration;
00107     float current_time;
00108     float frame_width;
00109     float frame_height;
00110     int frames_per_row;
00111 };
00112
00113 struct Enemy final : IComponent
00114 {
00115     float health;
00116     float max_health;
00117     float damage;
00118     float speed;
00119     float last_shot_time;
00120     float shoot_cooldown;
00121 };
00122
00123 struct Explosion final : IComponent
00124 {
00125     int current_frame;
00126     int total_frames;
00127     float frame_duration;
00128     float current_time;
00129     float frame_width;
00130     float frame_height;
00131     int frames_per_row;
00132     float lifetime;
00133     float current_lifetime;
00134 };
00135
00136 struct Hitbox final : IComponent
00137 {
00138     float radius;
00139     float offsetX = 0.0f;
00140     float offsetY = 0.0f;
00141 };
00142
00143 struct KeyboardInput final : IComponent
00144 {
00145     bool space_pressed{};
00146     bool up_pressed{};
00147     bool down_pressed{};
00148     bool left_pressed{};
00149     bool right_pressed{};
00150 };
00151
00152 struct Floor final : IComponent
00153 {
00154 };
00155
00156 struct Ceiling final : IComponent
00157 {
00158 };
00159
00160 struct Scrolling final : IComponent
00161 {
00162     float speed_x; // Negative to scroll left
00163     float original_width; // Unscaled texture width in pixels

```

```

00164         float original_height; // Unscaled texture height in pixels
00165         bool fit_width; // If true, scale X to fit window width
00166     };
00167
00168     struct Score final : IComponent
00169     {
00170         int value;
00171     };
00172     struct Layer final : IComponent
00173     {
00174         int layer;
00175     };
00176
00177 } // namespace ecs

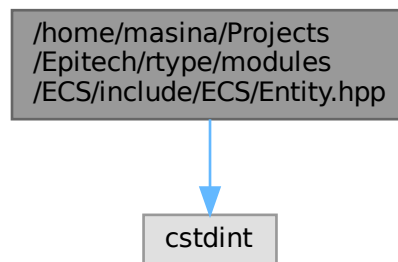
```

19.49 /home/masina/Projects/Epitech/rtype/modules/ECS/include/ECS/Entity.hpp File Reference

This file contains the entity definitions.

#include <cstdint>

Include dependency graph for Entity.hpp:



This graph shows which files directly or indirectly include this file:



Namespaces

- namespace `ecs`

Typedefs

- using `ecs::Entity` = `std::uint32_t`

Variables

- constexpr `Entity ecs::INVALID_ENTITY` = 0

19.49.1 Detailed Description

This file contains the entity definitions.

Definition in file [Entity.hpp](#).

19.50 Entity.hpp

[Go to the documentation of this file.](#)

```

00001 ///
00002 /// @file Entity.hpp
00003 /// @brief This file contains the entity definitions
00004 /// @namespace ecs
00005 ///
00006
00007 #pragma once
00008
00009 #include <cstdint>
00010
00011 namespace ecs
00012 {
00013     using Entity = std::uint32_t;
00014     constexpr Entity INVALID_ENTITY = 0;
00015 } // namespace ecs

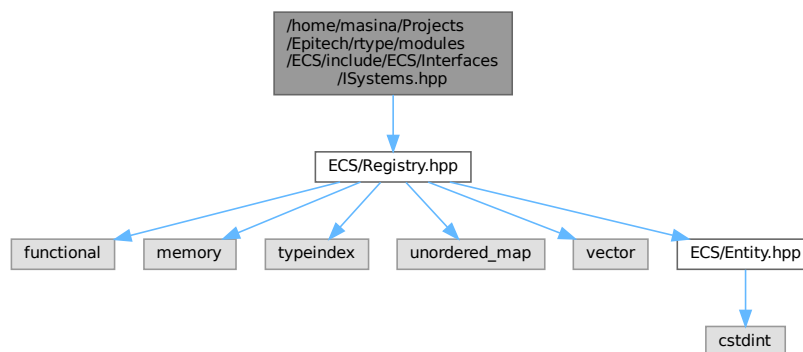
```

19.51 /home/masina/Projects/Epitech/rtype/modules/ECS/include/↵ ECS/Interfaces/ISystems.hpp File Reference

This file contains the interface for systems.

#include "ECS/Registry.hpp"

Include dependency graph for ISystems.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- interface `ecs::ISystem`
Interface class for system.
- class `ecs::ASystem`
Abstract class for system.

Namespaces

- namespace `ecs`

19.51.1 Detailed Description

This file contains the interface for systems.
Definition in file [ISystems.hpp](#).

19.52 ISystems.hpp

[Go to the documentation of this file.](#)

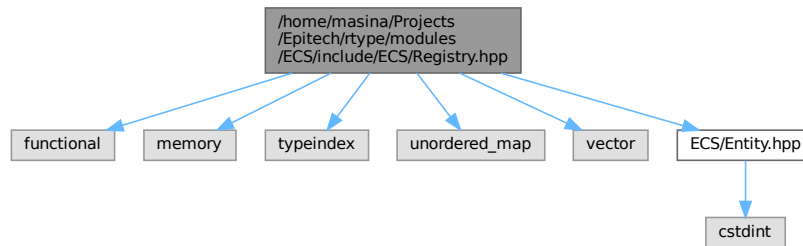
```
00001 ///  
00002 ///  
00003 ///  
00004 ///  
00005 ///  
00006 ///  
00007 #pragma once  
00008  
00009 #include "ECS/Registry.hpp"  
00010  
00011 namespace ecs  
00012 {  
00013  
00014     ///  
00015     ///  
00016     ///  
00017     ///  
00018     ///  
00019     class ISystem  
00020     {  
00021     public:  
00022         virtual ~ISystem() = default;  
00023         virtual void update(Registry &registry, float dt) = 0;  
00024         virtual bool isEnabled() = 0;  
00025         virtual void setEnable(bool enable) = 0;  
00026     };  
00027  
00028     ///  
00029     ///  
00030     ///  
00031     ///  
00032     ///  
00033     class ASystem : public ISystem  
00034     {  
00035     public:  
00036         bool isEnabled() override { return m_isEnable; }  
00037         void setEnable(const bool enable) override { m_isEnable = enable; }  
00038  
00039     private:  
00040         bool m_isEnable = true;  
00041     };  
00042  
00043 } // namespace ecs
```

19.53 /home/masina/Projects/Epitech/rtype/modules/ECS/include/↵ ECS/Registry.hpp File Reference

This file contains the Registry class declaration.

```
#include <functional>  
#include <memory>  
#include <typeindex>  
#include <unordered_map>  
#include <vector>  
#include "ECS/Entity.hpp"
```


Include dependency graph for Registry.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class `ecs::Registry`
Class for managing entities and their components.
- class `ecs::Registry::EntityBuilder`
- class `ecs::Registry::IPool`
- class `ecs::Registry::Pool< T >`

Namespaces

- namespace `ecs`

19.53.1 Detailed Description

This file contains the Registry class declaration.

Definition in file [Registry.hpp](#).

19.54 Registry.hpp

[Go to the documentation of this file.](#)

```

00001 ///  

00002 ///  

00003 ///  

00004 ///  

00005 ///  

00006 ///  

00007 #pragma once  

00008  

00009 #include <functional>  

00010 #include <memory>  

00011 #include <typeindex>  

00012 #include <unordered_map>  

00013 #include <vector>  

00014  

00015 #include "ECS/Entity.hpp"  

00016  

00017 namespace ecs  

00018 {  

00019     ///  

00020     ///  

00021     ///  

00022     ///  

00023     ///  

00024     class Registry  

00025     {

```

```

00026     public:
00027         Registry() = default;
00028         ~Registry() = default;
00029
00030         Registry(const Registry &) = delete;
00031         Registry &operator=(const Registry &) = delete;
00032         Registry(Registry &&) = delete;
00033         Registry &operator=(Registry &&) = delete;
00034
00035     class EntityBuilder
00036     {
00037     public:
00038         EntityBuilder(Registry &reg, Entity e) : m_registry(reg), m_entity(e) {}
00039
00040         template <typename T, typename... Args> EntityBuilder &with(Args &&...args)
00041         {
00042             m_registry.addComponent<T>(m_entity, std::forward<Args>(args)...);
00043             return *this;
00044         }
00045
00046         Entity build() const { return m_entity; }
00047
00048     private:
00049         Registry &m_registry;
00050         Entity m_entity;
00051     };
00052
00053     EntityBuilder createEntity()
00054     {
00055         const Entity entity = ++m_lastEntity;
00056         m_entities.push_back(entity);
00057         return EntityBuilder(*this, entity);
00058     }
00059
00060     template <typename T, typename... Args> T &addComponent(Entity e, Args &&...args)
00061     {
00062         auto &pool = getPool<T>();
00063         T &comp = pool.add(e, std::forward<Args>(args)...);
00064         for (auto &cb : m_onComponentAddedCallbacks)
00065         {
00066             cb(e, typeid(T));
00067         }
00068         return comp;
00069     }
00070
00071     template <typename T> T *getComponent(Entity e)
00072     {
00073         auto &pool = getPool<T>();
00074         return pool.get(e);
00075     }
00076
00077     template <typename T> std::unordered_map<Entity, T> &getAll() { return getPool<T>().data; }
00078
00079     template <typename T> bool hasComponent(Entity e)
00080     {
00081         auto &pool = getPool<T>();
00082         return pool.has(e);
00083     }
00084
00085     template <typename T> void removeComponent(Entity e)
00086     {
00087         auto &pool = getPool<T>();
00088         pool.remove(e);
00089     }
00090
00091     void onComponentAdded(std::function<void(Entity, const std::type_info &)> cb)
00092     {
00093         m_onComponentAddedCallbacks.push_back(std::move(cb));
00094     }
00095
00096 private:
00097     class IPool
00098     {
00099     public:
00100         virtual ~IPool() = default;
00101         virtual void remove(Entity e) = 0;
00102     };
00103
00104     template <typename T> class Pool final : public IPool
00105     {
00106     public:
00107         std::unordered_map<Entity, T> data;
00108
00109         template <typename... Args> T &add(Entity e, Args &&...args)
00110         {
00111             return data.emplace(e, T{std::forward<Args>(args)...}).first->second;
00112         }

```

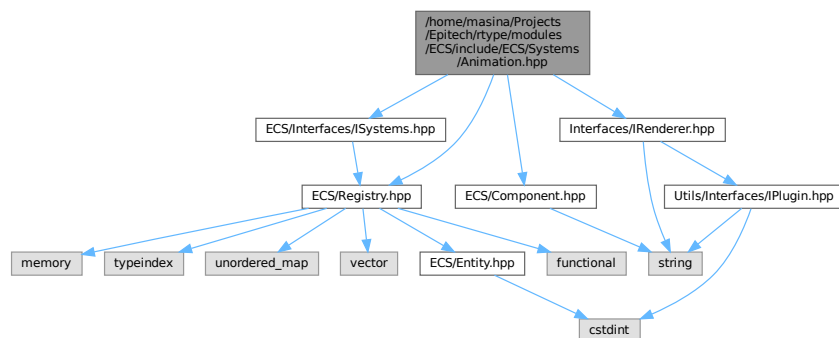
```

00113
00114     T *get(Entity e)
00115     {
00116         auto it = data.find(e);
00117         if (it != data.end())
00118         {
00119             return &it->second;
00120         }
00121         return nullptr;
00122     }
00123
00124     bool has(Entity e) { return data.contains(e); }
00125
00126     void remove(Entity e) override { data.erase(e); }
00127 };
00128
00129 template <typename T> Pool<T> &getPool()
00130 {
00131     const std::type_index ti(typeid(T));
00132     if (!m_components.contains(ti))
00133     {
00134         m_components[ti] = std::make_unique<Pool<T>>();
00135     }
00136     return *static_cast<Pool<T>>*>(m_components[ti].get());
00137 }
00138 Entity m_lastEntity = INVALID_ENTITY;
00139 std::vector<Entity> m_entities;
00140 std::unordered_map<std::type_index, std::unique_ptr<IPool>> m_components;
00141 std::vector<std::function<void(Entity, const std::type_info &)>> m_onComponentAddedCallbacks;
00142
00143 }; // class Registry
00144
00145 } // namespace ecs

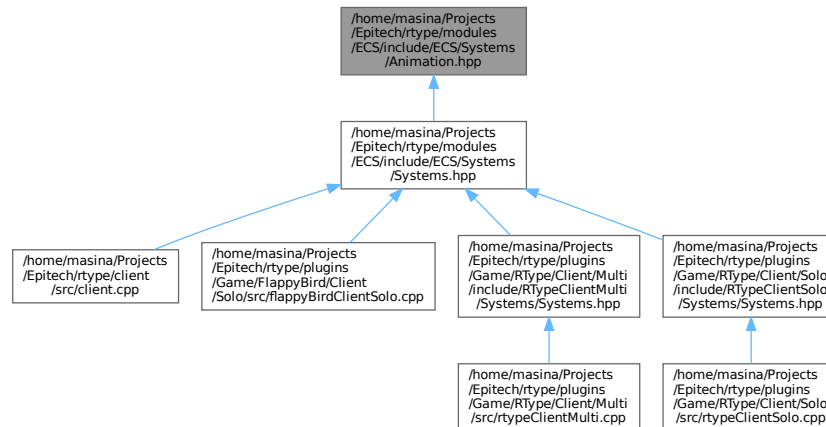
```

19.55 /home/masina/Projects/Epitech/rtype/modules/ECS/include/ECS/Systems/Animation.hpp File Reference

#include "ECS/Component.hpp"
#include "ECS/Interfaces/ISystems.hpp"
#include "ECS/Registry.hpp"
#include "Interfaces/IRenderer.hpp"
Include dependency graph for Animation.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [ecs::AnimationSystem](#)

Namespaces

- namespace [ecs](#)

19.56 Animation.hpp

[Go to the documentation of this file.](#)

```

00001 ///
00002 /// @file Systems.hpp
00003 /// @brief This file contains the system definitions
00004 /// @namespace ecs
00005 ///
00006
00007 #pragma once
00008
00009 #include "ECS/Component.hpp"
00010 #include "ECS/Interfaces/ISystems.hpp"
00011 #include "ECS/Registry.hpp"
00012 #include "Interfaces/IRenderer.hpp"
00013
00014 namespace ecs
00015 {
00016
00017     class AnimationSystem final : public ASystem
00018     {
00019     public:
00020         explicit AnimationSystem(const std::shared_ptr<eng::IRenderer> & /* renderer */) {}
00021         ~AnimationSystem() override = default;
00022
00023         AnimationSystem(const AnimationSystem &) = delete;
00024         AnimationSystem &operator=(const AnimationSystem &) = delete;
00025         AnimationSystem(AnimationSystem &&) = delete;
00026         AnimationSystem &operator=(AnimationSystem &&) = delete;
00027
00028         void update(Registry &registry, const float dt) override
00029         {
00030             for (auto &[entity, animation] : registry.getAll<ecs::Animation>())
00031             {
00032                 animation.current_time += dt;
00033
00034                 if (animation.current_time >= animation.frame_duration)
00035                 {
00036                     animation.current_time = 0.0f;
00037                     animation.current_frame = (animation.current_frame + 1) % animation.total_frames;
00038                 }
00039
00040                 if (auto *rect = registry.getComponent<ecs::Rect>(entity))

```

```

00041     {
00042         const int frame_x =
00043             (animation.current_frame % animation.frames_per_row) * animation.frame_width;
00044         if (const int frame_y =
00045             (animation.current_frame / animation.frames_per_row) * animation.frame_height;
00046             rect->pos_x != static_cast<float>(frame_x) || rect->pos_y != static_cast<float>(frame_y))
00047         {
00048             rect->pos_x = static_cast<float>(frame_x);
00049             rect->pos_y = static_cast<float>(frame_y);
00050             rect->size_x = animation.frame_width;
00051             rect->size_y = animation.frame_height;
00052         }
00053     }
00054 }
00055 }
00056
00057 }; // class AnimationSystem
00058 } // namespace ecs

```

19.57 /home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/include/RTypeClientMulti/Systems/Animation.hpp File Reference

Animation system for multiplayer.

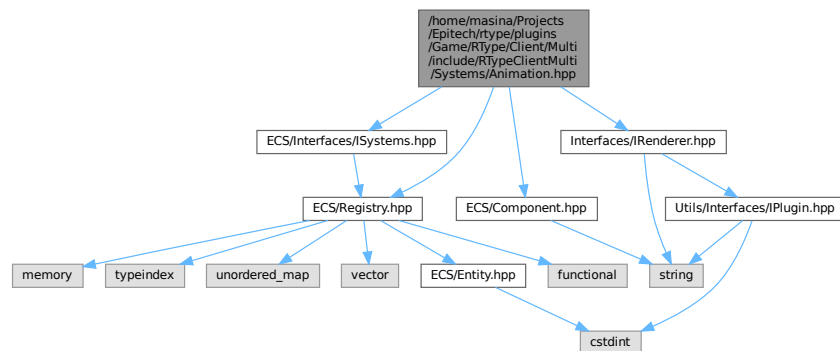
#include "ECS/Component.hpp"

#include "ECS/Interfaces/ISystems.hpp"

#include "ECS/Registry.hpp"

#include "Interfaces/IRenderer.hpp"

Include dependency graph for Animation.hpp:



Classes

- class [gme::AnimationSystem](#)

Namespaces

- namespace [gme](#)

19.57.1 Detailed Description

Animation system for multiplayer.

Definition in file [Animation.hpp](#).

19.58 Animation.hpp

[Go to the documentation of this file.](#)

00001 ///

```

00002 /// @file Animation.hpp
00003 /// @brief Animation system for multiplayer
00004 /// @namespace gme
00005 ///
00006
00007 #pragma once
00008
00009 #include "ECS/Component.hpp"
00010 #include "ECS/Interfaces/ISystems.hpp"
00011 #include "ECS/Registry.hpp"
00012 #include "Interfaces/IRenderer.hpp"
00013
00014 namespace gme
00015 {
00016
00017     class AnimationSystem final : public ecs::ASystem
00018     {
00019     public:
00020         explicit AnimationSystem(const std::shared_ptr<eng::IRenderer> & /* renderer */) {}
00021         ~AnimationSystem() override = default;
00022
00023         AnimationSystem(const AnimationSystem &) = delete;
00024         AnimationSystem &operator=(const AnimationSystem &) = delete;
00025         AnimationSystem(AnimationSystem &&) = delete;
00026         AnimationSystem &operator=(const AnimationSystem &&) = delete;
00027
00028         void update(ecs::Registry &registry, float dt) override
00029         {
00030             for (auto &[entity, animation] : registry.getAll<ecs::Animation>())
00031             {
00032                 animation.current_time += dt;
00033
00034                 if (animation.current_time >= animation.frame_duration)
00035                 {
00036                     animation.current_time = 0.0f;
00037                     animation.current_frame = (animation.current_frame + 1) % animation.total_frames;
00038                 }
00039
00040                 if (auto *rect = registry.getComponent<ecs::Rect>(entity))
00041                 {
00042                     const int frame_x =
00043                         (animation.current_frame % animation.frames_per_row) * animation.frame_width;
00044                     if (int frame_y = (animation.current_frame / animation.frames_per_row) * animation.frame_height;
00045                         rect->pos_x != static_cast<float>(frame_x) || rect->pos_y != static_cast<float>(frame_y))
00046                     {
00047                         rect->pos_x = static_cast<float>(frame_x);
00048                         rect->pos_y = static_cast<float>(frame_y);
00049                         rect->size_x = animation.frame_width;
00050                         rect->size_y = animation.frame_height;
00051                     }
00052                 }
00053             }
00054         }
00055     }; // class AnimationSystem
00056 } // namespace gme
00057
00058

```

19.59 /home/masina/Projects/Epitech/rtype/modules/ECS/include/↵ ECS/Systems/Audio.hpp File Reference

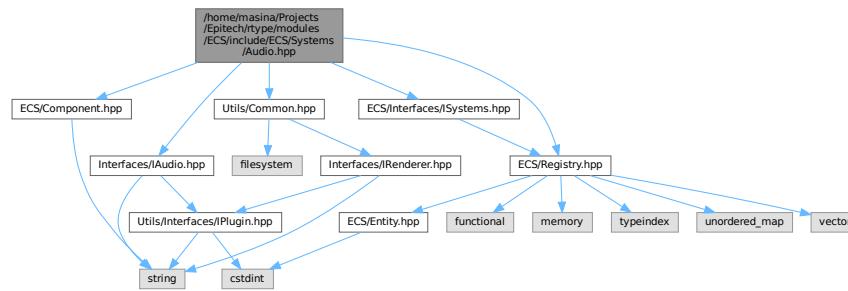
This file contains the audio system definition.

```

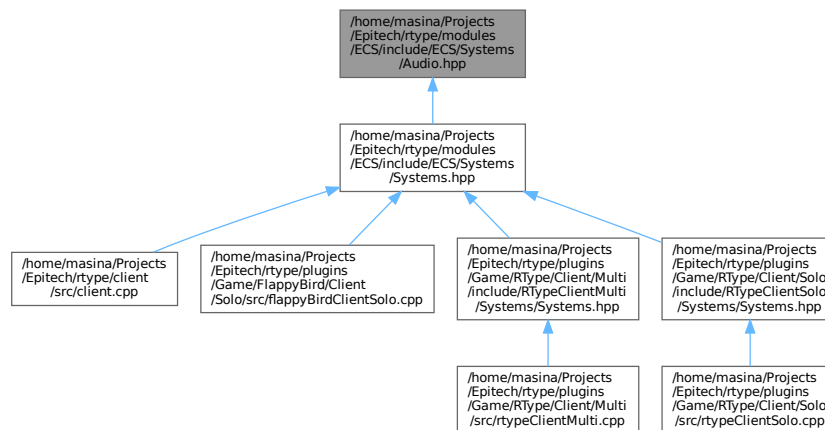
#include "ECS/Component.hpp"
#include "ECS/Interfaces/ISystems.hpp"
#include "ECS/Registry.hpp"
#include "Interfaces/IAudio.hpp"
#include "Utils/Common.hpp"

```

Include dependency graph for Audio.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class `ecs::AudioSystem`
Class for audio system.

Namespaces

- namespace `ecs`

19.59.1 Detailed Description

This file contains the audio system definition.
Definition in file [Audio.hpp](#).

19.60 Audio.hpp

[Go to the documentation of this file.](#)

```
00001 ///  
00002 ///  
00003 ///  
00004 ///  
00005 ///  
00006 ///  
00007 #pragma once  
00008
```

```

00009 #include "ECS/Component.hpp"
00010 #include "ECS/Interfaces/ISystems.hpp"
00011 #include "ECS/Registry.hpp"
00012 #include "Interfaces/IAudio.hpp"
00013 #include "Utils/Common.hpp"
00014
00015 namespace ecs
00016 {
00017
00018     ///
00019     /// @class AudioSystem
00020     /// @brief Class for audio system
00021     /// @namespace ecs
00022     ///
00023     class AudioSystem final : public ASystem
00024     {
00025     public:
00026         explicit AudioSystem(const std::shared_ptr<eng::IAudio> &audio, const float &audioVolume,
00027                             Registry &registry, bool &playSelectionMusic)
00028             : m_audio(audio), m_registry(registry), m_audioVolume(audioVolume),
00029               m_playSelectionMusic(playSelectionMusic)
00030         {
00031
00032             registry.onComponentAdded(
00033                 [&audio, &registry](const Entity e, const std::type_info &type)
00034                 {
00035                     const auto *audioComp = registry.getComponent<Audio>(e);
00036
00037                     if (type == typeid(Audio))
00038                     {
00039                         if (audioComp)
00040                         {
00041                             audio->createAudio(audioComp->path, audioComp->volume, audioComp->loop,
00042                                                 audioComp->id + std::to_string(e));
00043                         }
00044                     }
00045                 });
00046             m_selectionSoundEntity =
00047                 registry.createEntity()
00048                     .with<Audio>("menu_input", utl::Path::Audio::AUDIO_INPUT, 10.F, false, false)
00049                     .build();
00050             m_selectionSoundName = "menu_input" + std::to_string(m_selectionSoundEntity);
00051         }
00052         ~AudioSystem() override = default;
00053
00054         AudioSystem(const AudioSystem &) = delete;
00055         AudioSystem &operator=(const AudioSystem &) = delete;
00056         AudioSystem(AudioSystem &&) = delete;
00057         AudioSystem &operator=(AudioSystem &&) = delete;
00058
00059         void update(Registry &registry, float /* dt */) override
00060         {
00061             for (auto &[entity, audio] : registry.getAll<Audio>())
00062             {
00063                 std::string audioName = audio.id + std::to_string(entity);
00064                 m_audio->setVolume(audioName, audio.volume * m_audioVolume);
00065                 m_audio->setLoop(audioName, audio.loop);
00066
00067                 const auto status = m_audio->isPlaying(audioName);
00068                 if (audio.play)
00069                 {
00070                     if (status != eng::Status::Playing)
00071                     {
00072                         m_audio->playAudio(audioName);
00073                         if (!audio.loop)
00074                         {
00075                             audio.play = false;
00076                         }
00077                     }
00078                 }
00079                 else if (audio.loop)
00080                 {
00081                     if (status != eng::Status::Stopped)
00082                     {
00083                         m_audio->stopAudio(audioName);
00084                     }
00085                 }
00086             }
00087             if (m_playSelectionMusic)
00088             {
00089                 if (m_selectionSoundName.empty())
00090                     return;
00091
00092                 m_audio->stopAudio(m_selectionSoundName);
00093                 m_audio->playAudio(m_selectionSoundName);
00094                 m_playSelectionMusic = false;
00095             }
00096         }
00097     };
00098 }

```



```

00096     }
00097
00098     private:
00099         const std::shared_ptr<eng::IAudio> &m_audio;
00100         Registry &m_registry;
00101         const float &m_audioVolume;
00102
00103         Entity m_selectionSoundEntity{};
00104         std::string m_selectionSoundName;
00105         bool &m_playSelectionMusic;
00106     }; // class AudioSystem
00107 } // namespace ecs

```

19.61 /home/masina/Projects/Epitech/rtype/modules/ECS/include/ECS/Systems/Beam.hpp File Reference

This file contains the beam system definitions.

#include "ECS/Component.hpp"

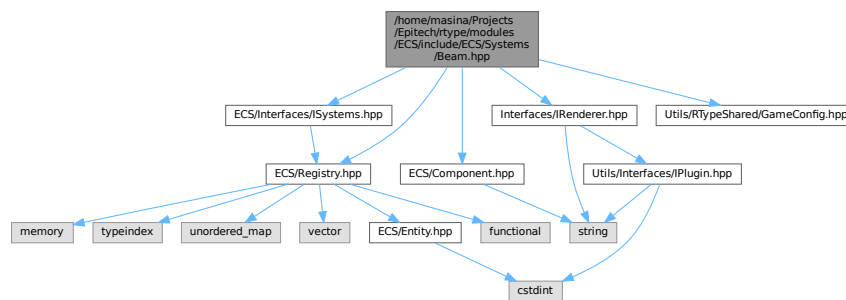
#include "ECS/Interfaces/ISystems.hpp"

#include "ECS/Registry.hpp"

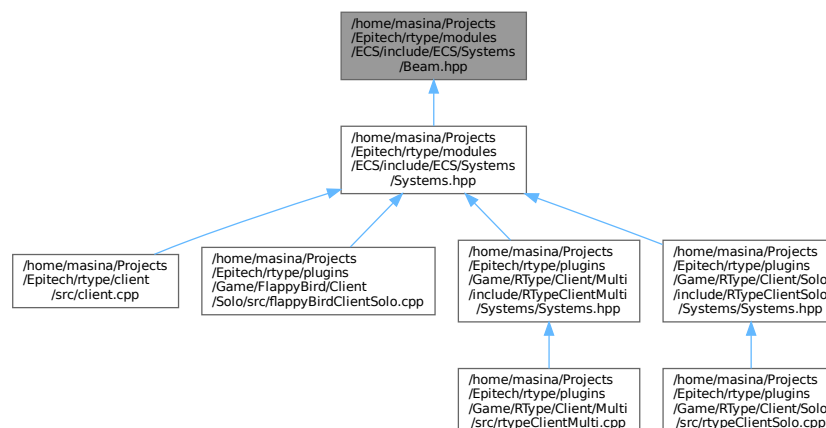
#include "Interfaces/IRenderer.hpp"

#include "Utils/RTypeShared/GameConfig.hpp"

Include dependency graph for Beam.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class `ecs::BeamSystem`

Namespaces

- namespace [ecs](#)

19.61.1 Detailed Description

This file contains the beam system definitions.

Definition in file [Beam.hpp](#).

19.62 Beam.hpp

[Go to the documentation of this file.](#)

```

00001 ///  

00002 ///  

00003 ///  

00004 ///  

00005 ///  

00006 ///  

00007 #pragma once  

00008  

00009 #include "ECS/Component.hpp"  

00010 #include "ECS/Interfaces/ISystems.hpp"  

00011 #include "ECS/Registry.hpp"  

00012 #include "Interfaces/IRenderer.hpp"  

00013 #include "Utils/RTypeShared/GameConfig.hpp"  

00014  

00015 namespace ecs  

00016 {  

00017  

00018     class BeamSystem final : public ASystem  

00019     {  

00020     public:  

00021         explicit BeamSystem(const std::shared_ptr<eng::IRenderer> &renderer) : m_renderer(renderer) {}  

00022         ~BeamSystem() override = default;  

00023  

00024         BeamSystem(const BeamSystem &) = delete;  

00025         BeamSystem &operator=(const BeamSystem &) = delete;  

00026         BeamSystem(BeamSystem &&) = delete;  

00027         BeamSystem &operator=(BeamSystem &&) = delete;  

00028  

00029         void update(Registry &registry, float /* dt */) override  

00030         {  

00031             for (auto &[entity, beamCharge] : registry.getAll<BeamCharge>())  

00032             {  

00033                 if (const auto *player = registry.getComponent<Player>(entity); player == nullptr)  

00034                 {  

00035                     continue;  

00036                 }  

00037  

00038                 const auto *transform = registry.getComponent<Transform>(entity);  

00039                 if (transform == nullptr)  

00040                 {  

00041                     continue;  

00042                 }  

00043  

00044                 const float barX = transform->x + (utl::GameConfig::Player::SPRITE_WIDTH / 2.0f) -  

00045                     (utl::GameConfig::Beam::BAR_WIDTH / 2.0f);  

00046                 const float barY = transform->y - utl::GameConfig::Beam::BAR_HEIGHT - 10.0f;  

00047  

00048                 for (int x = 0; x < static_cast<int>(utl::GameConfig::Beam::BAR_WIDTH); x += 3)  

00049                 {  

00050                     for (int y = 0; y < static_cast<int>(utl::GameConfig::Beam::BAR_HEIGHT); y += 3)  

00051                     {  

00052                         m_renderer->drawPoint(barX + x, barY + y, {.r = 30, .g = 30, .b = 30, .a = 200});  

00053                     }  

00054                 }  

00055  

00056                 const float chargeRatio = beamCharge.current_charge / beamCharge.max_charge;  

00057                 const float chargeWidth = utl::GameConfig::Beam::BAR_WIDTH * chargeRatio;  

00058  

00059                 const float thresholdX = barX + (utl::GameConfig::Beam::BAR_WIDTH * 0.5f);  

00060                 for (int y = 0; y < static_cast<int>(utl::GameConfig::Beam::BAR_HEIGHT); y += 2)  

00061                 {  

00062                     m_renderer->drawPoint(thresholdX, barY + y, {.r = 255, .g = 255, .b = 255, .a = 150});  

00063                 }  

00064  

00065                 if (chargeWidth > 0)  

00066                 {  

00067                     for (int x = 0; x < static_cast<int>(chargeWidth); x += 3)  

00068                     {  

00069                         for (int y = 0; y < static_cast<int>(utl::GameConfig::Beam::BAR_HEIGHT); y += 3)  

00070                         {

```

```

00071         eng::Color chargeColor{};
00072         if (chargeRatio < 0.5f)
00073         {
00074             chargeColor = {.r = 255, .g = 100, .b = 0, .a = 255};
00075         }
00076         else if (chargeRatio < 0.8f)
00077         {
00078             chargeColor = {.r = 255, .g = 200, .b = 0, .a = 255};
00079         }
00080         else
00081         {
00082             chargeColor = {.r = 0, .g = 255, .b = 0, .a = 255};
00083         }
00084
00085         m_renderer->drawPoint(barX + x, barY + y, chargeColor);
00086     }
00087 }
00088 }
00089 }
00090 }
00091
00092 private:
00093     const std::shared_ptr<eng::IRenderer> &m_renderer;
00094 }; // class BeamSystem
00095 } // namespace ecs

```

19.63 /home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/include/RTypeClientMulti/Systems/Beam.hpp File Reference

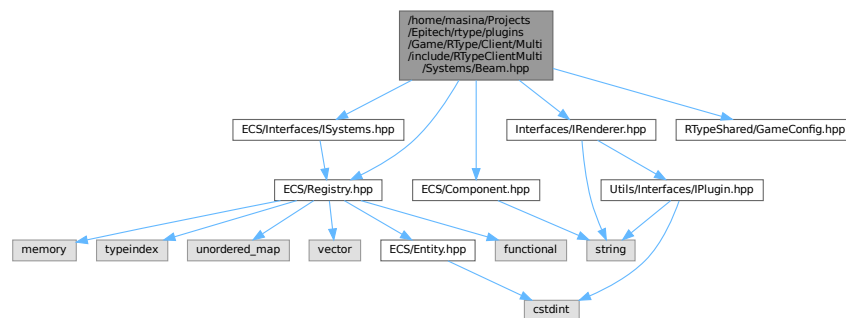
Beam system for multiplayer.

```

#include "ECS/Component.hpp"
#include "ECS/Interfaces/ISystems.hpp"
#include "ECS/Registry.hpp"
#include "Interfaces/IRenderer.hpp"
#include "RTypeShared/GameConfig.hpp"

```

Include dependency graph for Beam.hpp:



Classes

- class [gme::BeamSystem](#)

Namespaces

- namespace [gme](#)

19.63.1 Detailed Description

Beam system for multiplayer.

Definition in file [Beam.hpp](#).

19.64 Beam.hpp

[Go to the documentation of this file.](#)

```

00001 ///  

00002 ///  

00003 ///  

00004 ///  

00005 ///  

00006 ///  

00007 #pragma once  

00008  

00009 #include "ECS/Component.hpp"  

00010 #include "ECS/Interfaces/ISystems.hpp"  

00011 #include "ECS/Registry.hpp"  

00012 #include "Interfaces/IRenderer.hpp"  

00013 #include "RTypeShared/GameConfig.hpp"  

00014  

00015 namespace gme  

00016 {  

00017  

00018     class BeamSystem final : public ecs::ASystem  

00019     {  

00020     public:  

00021         explicit BeamSystem(const std::shared_ptr<eng::IRenderer> &renderer) : m_renderer(renderer) {}  

00022         ~BeamSystem() override = default;  

00023  

00024         BeamSystem(const BeamSystem &) = delete;  

00025         BeamSystem &operator=(const BeamSystem &) = delete;  

00026         BeamSystem(BeamSystem &&) = delete;  

00027         BeamSystem &operator=(BeamSystem &&) = delete;  

00028  

00029         void update(ecs::Registry &registry, float /* dt */ override  

00030         {  

00031             for (auto &[entity, beamCharge] : registry.getAll<ecs::BeamCharge>())  

00032             {  

00033                 const auto *player = registry.getComponent<ecs::Player>(entity);  

00034                 if (player == nullptr || !player->is_cli)  

00035                 {  

00036                     continue;  

00037                 }  

00038  

00039                 const auto *transform = registry.getComponent<ecs::Transform>(entity);  

00040                 if (transform == nullptr)  

00041                 {  

00042                     continue;  

00043                 }  

00044  

00045                 const float barX =  

00046                 transform->x + (GameConfig::Player::SPRITE_WIDTH / 2.0f) - (GameConfig::Beam::BAR_WIDTH /  

00047                 2.0f);  

00048  

00049                 const float barY = transform->y - GameConfig::Beam::BAR_HEIGHT - 10.0f;  

00050  

00051                 for (int x = 0; x < static_cast<int>(GameConfig::Beam::BAR_WIDTH); x += 3)  

00052                 {  

00053                     for (int y = 0; y < static_cast<int>(GameConfig::Beam::BAR_HEIGHT); y += 3)  

00054                     {  

00055                         m_renderer->drawPoint(barX + x, barY + y, {.r = 30, .g = 30, .b = 30, .a = 200});  

00056                     }  

00057  

00058                     const float chargeRatio = beamCharge.current_charge / beamCharge.max_charge;  

00059                     const float chargeWidth = GameConfig::Beam::BAR_WIDTH * chargeRatio;  

00060  

00061                     const float thresholdX = barX + (GameConfig::Beam::BAR_WIDTH * 0.5f);  

00062                     for (int y = 0; y < static_cast<int>(GameConfig::Beam::BAR_HEIGHT); y += 2)  

00063                     {  

00064                         m_renderer->drawPoint(thresholdX, barY + y, {.r = 255, .g = 255, .b = 255, .a = 150});  

00065                     }  

00066  

00067                     if (chargeWidth > 0)  

00068                     {  

00069                         for (int x = 0; x < static_cast<int>(chargeWidth); x += 3)  

00070                         {  

00071                             for (int y = 0; y < static_cast<int>(GameConfig::Beam::BAR_HEIGHT); y += 3)  

00072                             {  

00073                                 eng::Color chargeColor{};  

00074                                 if (chargeRatio < 0.5f)  

00075                                 {  

00076                                     chargeColor = {.r = 255, .g = 100, .b = 0, .a = 255};  

00077                                 }  

00078                                 else if (chargeRatio < 0.8f)  

00079                                 {  

00080                                     chargeColor = {.r = 255, .g = 200, .b = 0, .a = 255};  

00081                                 }  

00082                                 else  

00083                                 {  


```

```

00083         chargeColor = { .r = 0, .g = 255, .b = 0, .a = 255 };
00084     }
00085
00086     m_renderer->drawPoint(barX + x, barY + y, chargeColor);
00087 }
00088 }
00089 }
00090 }
00091 }
00092
00093 private:
00094     const std::shared_ptr<eng::IRenderer> &m_renderer;
00095 }; // class BeamSystem
00096 } // namespace gme

```

19.65 /home/masina/Projects/Epitech/rtype/modules/ECS/include/ECS/Systems/Debug.hpp File Reference

This file contains the debug system definition.

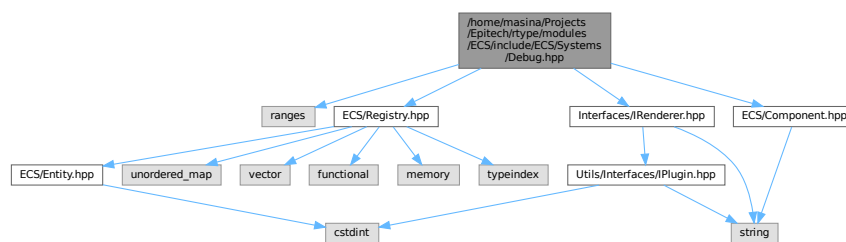
```
#include <ranges>
```

```
#include "ECS/Component.hpp"
```

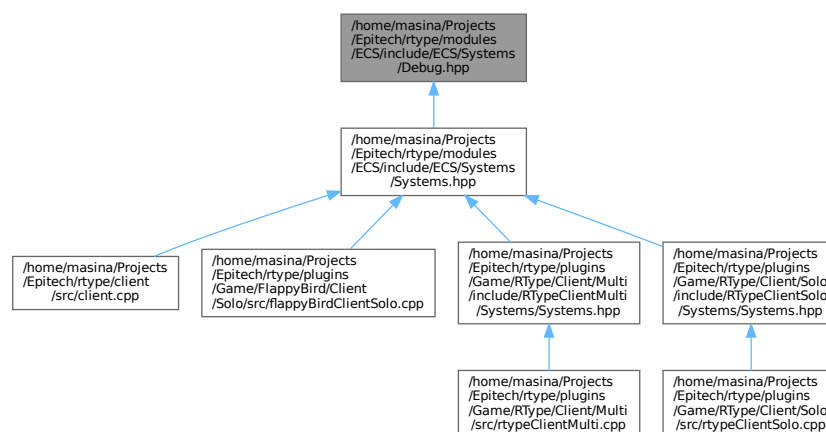
```
#include "ECS/Registry.hpp"
```

```
#include "Interfaces/IRenderer.hpp"
```

Include dependency graph for Debug.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [ecs::DebugSystem](#)
Class for debug system.

Namespaces

- namespace [ecs](#)

19.65.1 Detailed Description

This file contains the debug system definition.

Definition in file [Debug.hpp](#).

19.66 Debug.hpp

[Go to the documentation of this file.](#)

```

00001 ///  

00002 ///  

00003 ///  

00004 ///  

00005 ///  

00006 ///  

00007 #pragma once  

00008  

00009 #include <ranges>  

00010  

00011 #include "ECS/Component.hpp"  

00012 #include "ECS/Registry.hpp"  

00013 #include "Interfaces/IRenderer.hpp"  

00014  

00015 namespace ecs  

00016 {  

00017  

00018     ///  

00019     ///  

00020     ///  

00021     ///  

00022     ///  

00023     class DebugSystem final : public ASystem  

00024     {  

00025     public:  

00026         explicit DebugSystem(const std::shared_ptr<eng::IRenderer> &renderer, bool &showDebug)  

00027             : m_renderer(renderer), m_showDebug(showDebug)  

00028         {  

00029         }  

00030  

00031         ~DebugSystem() override = default;  

00032  

00033         DebugSystem(const DebugSystem &) = delete;  

00034         DebugSystem &operator=(const DebugSystem &) = delete;  

00035         DebugSystem(DebugSystem &&) = delete;  

00036         DebugSystem &operator=(DebugSystem &&) = delete;  

00037  

00038         void update(Registry &registry, float dt) override  

00039         {  

00040  

00041             const auto &circleShapes = registry.getAll<Hitbox>();  

00042             const auto &transforms = registry.getAll<Transform>();  

00043             for (const auto &key : circleShapes | std::views::keys)  

00044             {  

00045                 if (!registry.hasComponent<Hitbox>(key) || !registry.hasComponent<Transform>(key))  

00046                 {  

00047                     continue;  

00048                 }  

00049  

00050                 const auto *transform = registry.getComponent<Transform>(key);  

00051                 const auto *hitbox = registry.getComponent<Hitbox>(key);  

00052  

00053                 if ((transform == nullptr) || (hitbox == nullptr))  

00054                 {  

00055                     continue;  

00056                 }  

00057                 const float hitboxX = transform->x + hitbox->offsetX - hitbox->radius;  

00058                 const float hitboxY = transform->y + hitbox->offsetY - hitbox->radius;  

00059                 m_renderer->setCircleShapePosition("hitbox_" + std::to_string(key), hitboxX, hitboxY);  

00060                 if (m_showDebug)  

00061                 {  

00062                     m_renderer->drawCircleShape("hitbox_" + std::to_string(key));  

00063                 }  

00064             }  

00065         }  

00066  

00067     private:  

00068         const std::shared_ptr<eng::IRenderer> &m_renderer;  

00069         bool &m_showDebug;  

00070

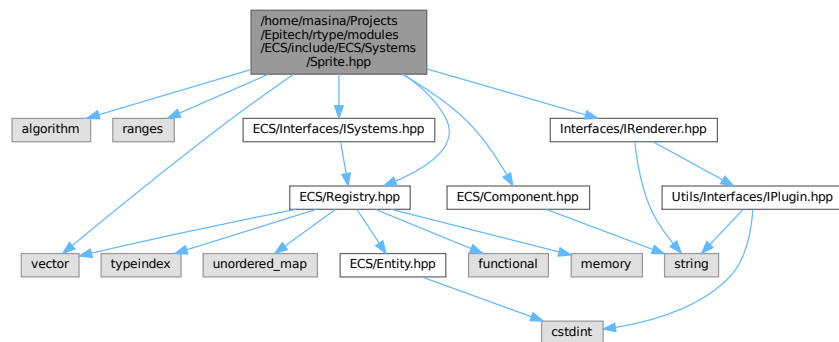
```

```
00071     }; // class DebugSystem
00072 } // namespace ecs
```

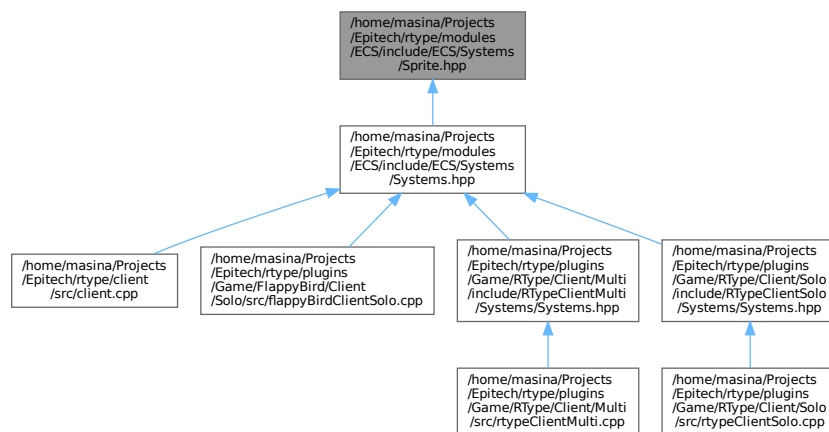
19.67 /home/masina/Projects/Epitech/rtype/modules/ECS/include/ECS/Systems/Sprite.hpp File Reference

This file contains the sprite system definition.

```
#include <algorithm>
#include <ranges>
#include <vector>
#include "ECS/Component.hpp"
#include "ECS/Interfaces/ISystems.hpp"
#include "ECS/Registry.hpp"
#include "Interfaces/IRenderer.hpp"
Include dependency graph for Sprite.hpp:
```



This graph shows which files directly or indirectly include this file:



Classes

- class `ecs::SpriteSystem`
Class for sprite system.

Namespaces

- namespace [ecs](#)

19.67.1 Detailed Description

This file contains the sprite system definition.

Definition in file [Sprite.hpp](#).

19.68 Sprite.hpp

[Go to the documentation of this file.](#)

```

00001 ///  

00002 ///  

00003 ///  

00004 ///  

00005 ///  

00006 ///  

00007 #pragma once  

00008  

00009 #include <algorithm>  

00010 #include <ranges>  

00011 #include <vector>  

00012  

00013 #include "ECS/Component.hpp"  

00014 #include "ECS/Interfaces/ISystems.hpp"  

00015 #include "ECS/Registry.hpp"  

00016 #include "Interfaces/IRenderer.hpp"  

00017  

00018 namespace ecs  

00019 {  

00020  

00021 ///  

00022 ///  

00023 ///  

00024 ///  

00025 ///  

00026 class SpriteSystem final : public ASystem  

00027 {  

00028     public:  

00029     explicit SpriteSystem(const std::shared_ptr<eng::IRenderer> &renderer) : m_renderer(renderer) {}  

00030     ~SpriteSystem() override = default;  

00031  

00032     SpriteSystem(const SpriteSystem &) = delete;  

00033     SpriteSystem &operator=(const SpriteSystem &) = delete;  

00034     SpriteSystem(SpriteSystem &&) = delete;  

00035     SpriteSystem &operator=(SpriteSystem &&) = delete;  

00036  

00037     void update(Registry &registry, float /* dt */) override  

00038     {  

00039         std::vector<std::pair<Entity, int>> spritesWithLayers;  

00040         spritesWithLayers.reserve(  

00041             registry.getAll<Texture>().size()); // Pré-allocation pour éviter réallocations  

00042  

00043         for (auto &pair : registry.getAll<Texture>())  

00044         {  

00045             const auto entity = pair.first;  

00046             const auto *layer = registry.GetComponent<Layer>(entity);  

00047             int layerValue = (layer != nullptr) ? layer->layer : 0;  

00048             spritesWithLayers.emplace_back(entity, layerValue);  

00049         }  

00050         std::sort(spritesWithLayers.begin(), spritesWithLayers.end(),  

00051             [](const auto &a, const auto &b) { return a.second < b.second; });  

00052         std::string spriteName;  

00053         spriteName.reserve(64); // Taille typique attendue  

00054  

00055         for (const auto &pair : spritesWithLayers)  

00056         {  

00057             const auto entity = pair.first;  

00058             const auto *sprite = registry.GetComponent<Texture>(entity);  

00059             if (!sprite)  

00060                 continue;  

00061  

00062             const auto *transform = registry.GetComponent<Transform>(entity);  

00063             const auto *rect = registry.GetComponent<Rect>(entity);  

00064             const auto *scale = registry.GetComponent<Scale>(entity);  

00065             const auto *color = registry.GetComponent<Color>(entity);  

00066             const bool hasScrolling = registry.GetComponent<Scrolling>(entity);  

00067  

00068             const float x = (transform != nullptr) ? transform->x : 0.F;  

00069             const float y = (transform != nullptr) ? transform->y : 0.F;  

00070

```



```

00071         spriteName.clear();
00072         spriteName = sprite->id;
00073         spriteName += std::to_string(entity);
00074
00075         m_renderer->setSpriteTexture(spriteName, sprite->path);
00076         m_renderer->setSpritePosition(spriteName, x, y);
00077         if ((scale != nullptr) && !hasScrolling)
00078         {
00079             m_renderer->setSpriteScale(spriteName, static_cast<int>(scale->x), static_cast<int>(scale->y));
00080         }
00081         if (rect != nullptr)
00082         {
00083             m_renderer->setSpriteFrame(spriteName, static_cast<int>(rect->pos_x),
00084                                     static_cast<int>(rect->pos_y), rect->size_x, rect->size_y);
00085         }
00086         if (color != nullptr)
00087         {
00088             m_renderer->setSpriteColor(sprite->id + std::to_string(entity),
00089                                     {.r = color->r, .g = color->g, .b = color->b, .a = color->a});
00090         }
00091         m_renderer->drawSprite(spriteName);
00092     }
00093 }
00094
00095 private:
00096     const std::shared_ptr<eng::IRenderer> &m_renderer;
00097 }; // class SpriteSystem
00098 } // namespace ecs

```

19.69 /home/masina/Projects/Epitech/rtype/modules/ECS/include/ECS/Systems/Starfield.hpp File Reference

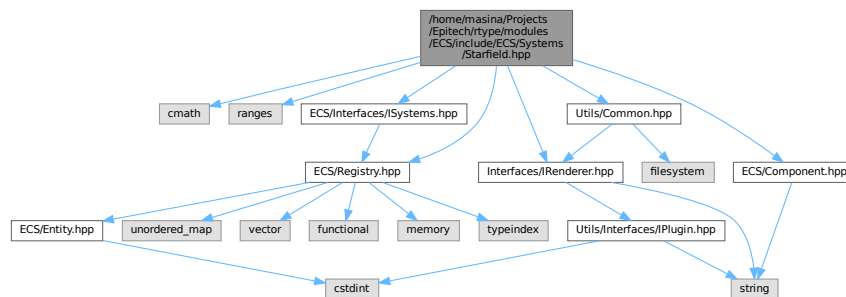
Starfield System for managing background stars and visual effects.

```

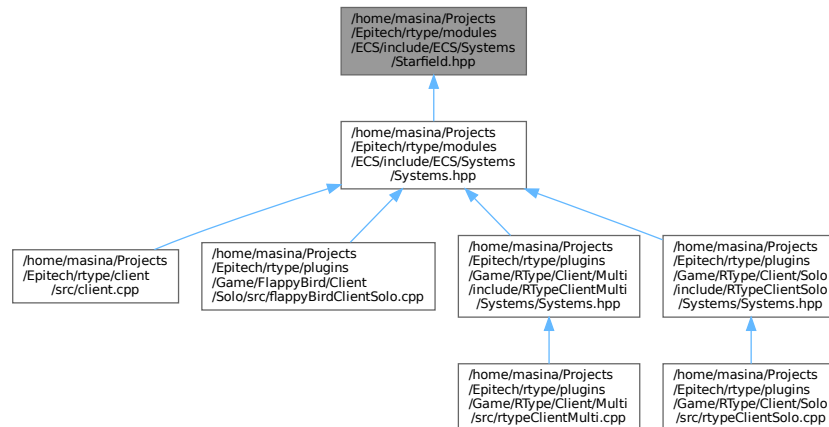
#include <cmath>
#include <ranges>
#include "ECS/Component.hpp"
#include "ECS/Interfaces/ISystems.hpp"
#include "ECS/Registry.hpp"
#include "Interfaces/IRenderer.hpp"
#include "Utils/Common.hpp"

```

Include dependency graph for Starfield.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class `ecs::StarfieldSystem`

Namespaces

- namespace `ecs`

19.69.1 Detailed Description

Starfield System for managing background stars and visual effects.

Definition in file [Starfield.hpp](#).

19.70 Starfield.hpp

[Go to the documentation of this file.](#)

```

00001 ///
00002 /// @file Starfield.hpp
00003 /// @brief Starfield System for managing background stars and visual effects
00004 /// @namespace ecs
00005 ///
00006
00007 #pragma once
00008
00009 #include <cmath>
00010 #include <ranges>
00011
00012 #include "ECS/Component.hpp"
00013 #include "ECS/Interfaces/ISystems.hpp"
00014 #include "ECS/Registry.hpp"
00015 #include "Interfaces/IRenderer.hpp"
00016 #include "Utils/Common.hpp"
00017
00018 namespace ecs
00019 {
00020
00021     class StarfieldSystem final : public ASystem
00022     {
00023     public:
00024         explicit StarfieldSystem(const std::shared_ptr<eng::IRenderer> &renderer, Registry &registry)
00025             : m_renderer(renderer)
00026         {
00027             createStarfield(registry, m_renderer->getWindowSize());
00028         }
00029         ~StarfieldSystem() override = default;
00030
00031         StarfieldSystem(const StarfieldSystem &) = delete;
00032         StarfieldSystem &operator=(const StarfieldSystem &) = delete;
00033         StarfieldSystem(StarfieldSystem &&) = delete;

```

```

00034     StarfieldSystem &operator=(StarfieldSystem &&) = delete;
00035
00036     void update(Registry &registry, const float dt) override
00037     {
00038         auto [width, height] = m_renderer->getWindowSize();
00039         const float screenWidth = static_cast<float>(width);
00040         const float screenHeight = static_cast<float>(height);
00041
00042         m_timeAccumulator += dt;
00043
00044         eng::Color cachedColor{};
00045         bool isStar = false;
00046         bool isNear = false;
00047         bool isMid = false;
00048
00049         for (auto &pair : registry.getAll<Pixel>())
00050         {
00051             const auto entity = pair.first;
00052             auto *transform = registry.getComponent<Transform>(entity);
00053             if (!transform)
00054                 continue;
00055
00056             const auto *velocity = registry.getComponent<Velocity>(entity);
00057             if (velocity)
00058             {
00059                 transform->x += velocity->x * dt;
00060                 transform->y += velocity->y * dt;
00061
00062                 if (transform->x < -10.0f || transform->x > screenWidth + 10.0f || transform->y < -10.0f ||
00063                     transform->y > screenHeight + 10.0f)
00064                 {
00065                     transform->x = screenWidth + std::rand() % 200;
00066                     transform->y = static_cast<float>(std::rand() % static_cast<int>(screenHeight));
00067                 }
00068             }
00069
00070             const auto *color = registry.getComponent<Color>(entity);
00071             if (!color)
00072                 continue;
00073
00074             const Pixel *pixel = registry.getComponent<Pixel>(entity);
00075             if (!pixel)
00076                 continue;
00077
00078             const std::string &pixelId = pixel->id;
00079             isStar = (pixelId == "star_far" || pixelId == "star_mid" || pixelId == "star_near");
00080             isNear = (pixelId == "star_near");
00081             isMid = (pixelId == "star_mid");
00082
00083             unsigned char finalAlpha = color->a;
00084             if (isStar)
00085             {
00086                 float starPhase = (transform->x * 0.1f + transform->y * 0.15f + m_timeAccumulator * 0.8f);
00087                 float twinkle = (std::sin(starPhase) + 1.0f) * 0.5f;
00088                 finalAlpha = static_cast<unsigned char>(color->a * (0.5f + twinkle * 0.5f));
00089             }
00090
00091             cachedColor = {.r = color->r, .g = color->g, .b = color->b, .a = finalAlpha};
00092
00093             m_renderer->drawPoint(transform->x, transform->y, cachedColor);
00094
00095             if (isNear)
00096             {
00097                 cachedColor.a = static_cast<unsigned char>(finalAlpha * 0.8f);
00098                 m_renderer->drawPoint(transform->x + 0.8f, transform->y, cachedColor);
00099             }
00100             else if (isMid)
00101             {
00102                 cachedColor.a = static_cast<unsigned char>(finalAlpha * 0.7f);
00103                 m_renderer->drawPoint(transform->x + 0.5f, transform->y, cachedColor);
00104             }
00105         }
00106     }
00107
00108 private:
00109     const std::shared_ptr<eng::IRenderer> &m_renderer;
00110     float m_timeAccumulator = 0.0f;
00111
00112     static void createStarfield(Registry &registry, const eng::WindowSize &windowSize)
00113     {
00114         createStars(registry, 60, windowSize.width, windowSize.height, utl::Config::Color::WHITE_TRANS, -20.0f,
00115                     "star_far");
00116         createStars(registry, 35, windowSize.width, windowSize.height, utl::Config::Color::BLUE, -40.0f,
00117                     "star_mid");
00118         createStars(registry, 25, windowSize.width, windowSize.height, utl::Config::Color::YELLOW, -80.0f,
00119                     "star_near");
00120         createStarsVaried(registry, 15, windowSize.width, windowSize.height, -25.0f, "star_far");

```

```

00121         createStarsVaried(registry, 12, windowSize.width, windowSize.height, -50.0f, "star_mid");
00122         createShootingStars(registry, 8, windowSize.width, windowSize.height);
00123         createPlanets(registry, 4, windowSize.width, windowSize.height);
00124         createNebulae(registry, 3, windowSize.width, windowSize.height);
00125         createComets(registry, 6, windowSize.width, windowSize.height);
00126     }
00127     static void createStars(Registry &registry, const int count, const unsigned int screenWidth,
00128                             const unsigned int screenHeight, const eng::Color &color, float velocity,
00129                             const std::string &id)
00130     {
00131         for (int i = 0; i < count; ++i)
00132         {
00133             registry.createEntity()
00134                 .with<Pixel>(id)
00135                 .with<Transform>(id + "_transform", static_cast<float>(std::rand() % screenWidth),
00136                                 static_cast<float>(std::rand() % screenHeight), 0.0f)
00137                 .with<Color>(id + "_color", color.r, color.g, color.b, color.a)
00138                 .with<Velocity>(id + "_vel", velocity, 0.0f)
00139                 .build();
00140         }
00141     }
00142     static void createShootingStars(Registry &registry, const int count, const unsigned int screenWidth,
00143                                     const unsigned int screenHeight)
00144     {
00145         for (int i = 0; i < count; ++i)
00146         {
00147             registry.createEntity()
00148                 .with<Pixel>("star_shooting")
00149                 .with<Transform>("star_shooting_transform", static_cast<float>(std::rand() % screenWidth),
00150                                 static_cast<float>(std::rand() % screenHeight), 0.0f)
00151                 .with<Color>("star_shooting_color", utl::Config::Color::GREEN.r, utl::Config::Color::GREEN.g,
00152                             utl::Config::Color::GREEN.b, utl::Config::Color::GREEN.a)
00153                 .with<Velocity>("star_shooting_vel", -120.0f, static_cast<float>((std::rand() % 20) - 10))
00154                 .build();
00155         }
00156     }
00157     static void createPlanets(Registry &registry, const int count, const unsigned int screenWidth,
00158                             const unsigned int screenHeight)
00159     {
00160         for (int i = 0; i < count; ++i)
00161         {
00162             registry.createEntity()
00163                 .with<Pixel>("planet_far")
00164                 .with<Transform>("planet_far_transform", static_cast<float>(std::rand() % screenWidth),
00165                                 static_cast<float>(std::rand() % screenHeight), 0.0f)
00166                 .with<Color>("planet_far_color", utl::Config::Color::PURPLE.r, utl::Config::Color::PURPLE.g,
00167                             utl::Config::Color::PURPLE.b, utl::Config::Color::PURPLE.a)
00168                 .with<Velocity>("planet_far_vel", -5.0f, 0.0f)
00169                 .build();
00170         }
00171     }
00172     static void createNebulae(Registry &registry, const int count, const unsigned int screenWidth,
00173                             const unsigned int screenHeight)
00174     {
00175         for (int i = 0; i < count; ++i)
00176         {
00177             registry.createEntity()
00178                 .with<Pixel>("nebula")
00179                 .with<Transform>("nebula_transform", static_cast<float>(std::rand() % screenWidth),
00180                                 static_cast<float>(std::rand() % screenHeight), 0.0f)
00181                 .with<Color>("nebula_color", utl::Config::Color::BLUE_SECOND.r,
00182                             utl::Config::Color::BLUE_SECOND.g, utl::Config::Color::BLUE_SECOND.b,
00183                             utl::Config::Color::BLUE_SECOND.a)
00184                 .with<Velocity>("nebula_vel", -8.0f, 0.0f)
00185                 .build();
00186         }
00187     }
00188     static void createComets(Registry &registry, const int count, const unsigned int screenWidth,
00189                             const unsigned int screenHeight)
00190     {
00191         for (int i = 0; i < count; ++i)
00192         {
00193             registry.createEntity()
00194                 .with<Pixel>("comet")
00195                 .with<Transform>("comet_transform", static_cast<float>(std::rand() % screenWidth),
00196                                 static_cast<float>(std::rand() % screenHeight), 0.0f)
00197                 .with<Color>("comet_color", utl::Config::Color::GREEN.r, utl::Config::Color::GREEN.g,
00198                             utl::Config::Color::GREEN.b, utl::Config::Color::GREEN.a)
00199                 .with<Velocity>("comet_vel", -60.0f, static_cast<float>((std::rand() % 40) - 20))
00200                 .build();
00201         }
00202     }
00203     static void createStarsVaried(Registry &registry, const int count, const unsigned int screenWidth,
00204                                     const unsigned int screenHeight, float velocity, const std::string &baseId)
00205     {
00206         for (int i = 0; i < count; ++i)
00207     
```

```

00208     {
00209         int colorType = std::rand() % 4;
00210         unsigned char r, g, b, a;
00211
00212         switch (colorType)
00213         {
00214             case 0: // Blanc froid
00215                 r = 200;
00216                 g = 220;
00217                 b = 255;
00218                 a = 180;
00219                 break;
00220             case 1: // Cyan
00221                 r = 150;
00222                 g = 230;
00223                 b = 255;
00224                 a = 200;
00225                 break;
00226             case 2: // Bleu doux
00227                 r = 100;
00228                 g = 150;
00229                 b = 255;
00230                 a = 160;
00231                 break;
00232             default: // Jaune pâle
00233                 r = 255;
00234                 g = 250;
00235                 b = 200;
00236                 a = 170;
00237                 break;
00238         }
00239
00240         registry.createEntity()
00241             .with<Pixel>(baseId)
00242             .with<Transform>(baseId + "_transform", static_cast<float>(std::rand() % screenWidth),
00243                 static_cast<float>(std::rand() % screenHeight), 0.0f)
00244             .with<Color>(baseId + "_color", r, g, b, a)
00245             .with<Velocity>(baseId + "_vel", velocity, 0.0f)
00246             .build();
00247     }
00248 }
00249 }; // class StarfieldSystem
00250 } // namespace ecs

```

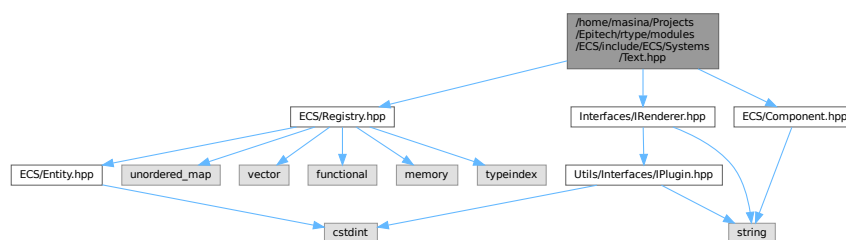
19.71 /home/masina/Projects/Epitech/rtype/modules/ECS/include/ECS/Systems/Text.hpp File Reference

#include "ECS/Component.hpp"

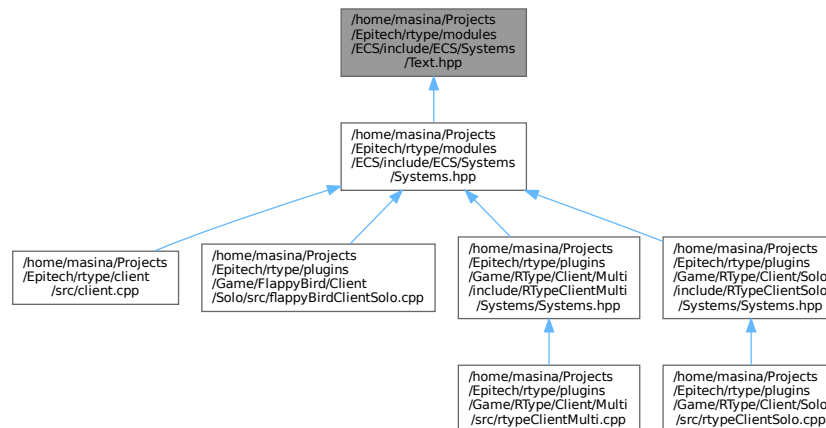
#include "ECS/Registry.hpp"

#include "Interfaces/IRenderer.hpp"

Include dependency graph for Text.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [ecs::TextSystem](#)
Class for text system.

Namespaces

- namespace [ecs](#)

19.72 Text.hpp

[Go to the documentation of this file.](#)

```

00001 ///
00002 /// @file Systems.hpp
00003 /// @brief This file contains the TextSystem definitions
00004 /// @namespace ecs
00005 ///
00006
00007 #pragma once
00008
00009 #include "ECS/Component.hpp"
00010 #include "ECS/Registry.hpp"
00011 #include "Interfaces/IRenderer.hpp"
00012
00013 namespace ecs
00014 {
00015
00016     ///
00017     /// @class TextSystem
00018     /// @brief Class for text system
00019     /// @namespace ecs
00020     ///
00021     class TextSystem final : public ASystem
00022     {
00023     public:
00024         explicit TextSystem(const std::shared_ptr<eng::IRenderer> &renderer) : m_renderer(renderer) {}
00025         ~TextSystem() override = default;
00026
00027         TextSystem(const TextSystem &) = delete;
00028         TextSystem &operator=(const TextSystem &) = delete;
00029         TextSystem(TextSystem &&) = delete;
00030         TextSystem &operator=(TextSystem &&) = delete;
00031
00032         void update(Registry &registry, float /* dt */) override
00033         {
00034
00035             for (auto &[entity, text] : registry.getAll<Text>())
00036             {
00037                 const auto *transform = registry.getComponent<Transform>(entity);
00038                 const auto *color = registry.getComponent<Color>(entity);
00039

```

```

00040         const float x = (transform != nullptr) ? transform->x : 0.F;
00041         const float y = (transform != nullptr) ? transform->y : 0.F;
00042
00043         const std::uint8_t r = color ? color->r : 255u;
00044         const std::uint8_t g = color ? color->g : 255u;
00045         const std::uint8_t b = color ? color->b : 255u;
00046         const std::uint8_t a = color ? color->a : 255u;
00047
00048         m_renderer->setTextContent(text.id, text.content);
00049         m_renderer->setTextPosition(text.id, x, y);
00050         m_renderer->setTextColor(text.id, {.r = r, .g = g, .b = b, .a = a});
00051         m_renderer->drawText(text.id);
00052     }
00053 }
00054
00055 private:
00056     const std::shared_ptr<eng::IRenderer> &m_renderer;
00057 }; // class TextSystem
00058 } // namespace ecs

```

19.73 /home/masina/Projects/Epitech/rtype/modules/Engine/include/Engine/Engine.hpp File Reference

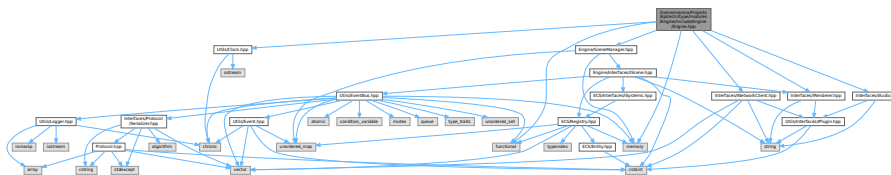
This file contains the Engine class declaration.

```

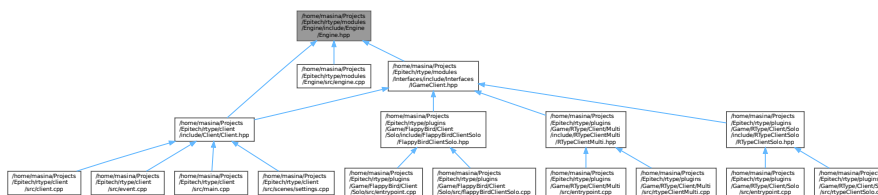
#include <functional>
#include <memory>
#include "Engine/SceneManager.hpp"
#include "Interfaces/IAudio.hpp"
#include "Interfaces/INetworkClient.hpp"
#include "Interfaces/IRenderer.hpp"
#include "Utils/Clock.hpp"

```

Include dependency graph for Engine.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [eng::Engine](#)
Class for the engine.

Namespaces

- namespace [eng](#)

Enumerations

- enum [eng::State](#) : unsigned char { [eng::STOP](#) = 0 , [eng::RUN](#) = 1 , [eng::DEFAULT](#) = 2 }

19.73.1 Detailed Description

This file contains the Engine class declaration.
Definition in file [Engine.hpp](#).

19.74 Engine.hpp

[Go to the documentation of this file.](#)

```

00001 ///  

00002 ///  

00003 ///  

00004 ///  

00005 ///  

00006 ///  

00007 #pragma once  

00008  

00009 #include <functional>  

00010 #include <memory>  

00011  

00012 #include "Engine/SceneManager.hpp"  

00013 #include "Interfaces/IAudio.hpp"  

00014 #include "Interfaces/INetworkClient.hpp"  

00015 #include "Interfaces/IRenderer.hpp"  

00016 #include "Utils/Clock.hpp"  

00017  

00018 namespace eng  

00019 {  

00020  

00021     enum State : unsigned char  

00022     {  

00023         STOP = 0,  

00024         RUN = 1,  

00025         DEFAULT = 2,  

00026     };  

00027  

00028 ///  

00029 ///  

00030 ///  

00031 ///  

00032 ///  

00033 class PLUGIN_EXPORT Engine  

00034 {  

00035  

00036     public:  

00037         Engine(const std::function<std::shared_ptr<IAudio>()> &audioFactory,  

00038               const std::function<std::shared_ptr<INetworkClient>()> &networkFactory,  

00039               const std::function<std::shared_ptr<IRenderer>()> &rendererFactory);  

00040         ~Engine() = default;  

00041  

00042         Engine(const Engine &) = delete;  

00043         Engine &operator=(const Engine &) = delete;  

00044         Engine(Engine &&) = delete;  

00045         Engine &operator=(Engine &&) = delete;  

00046  

00047         std::shared_ptr<IAudio> &getAudio() { return m_audio; }  

00048         std::shared_ptr<INetworkClient> &getNetwork() { return m_network; }  

00049         std::shared_ptr<IRenderer> &getRenderer() { return m_renderer; }  

00050         std::unique_ptr<util::Clock> &getClock() { return m_clock; }  

00051         std::unique_ptr<SceneManager> &getSceneManager() { return m_sceneManager; }  

00052         State getState() const { return m_state; }  

00053  

00054         void setState(const State newState) { m_state = newState; }  

00055  

00056         void render(const WindowSize &windowSize, Color clearColor, bool showDebug) const;  

00057         void stop() const { m_renderer->closeWindow(); }  

00058  

00059     private:  

00060         State m_state = RUN;  

00061         std::unique_ptr<util::Clock> m_clock;  

00062         std::unique_ptr<SceneManager> m_sceneManager;  

00063         std::shared_ptr<IAudio> m_audio;  

00064         std::shared_ptr<INetworkClient> m_network;  

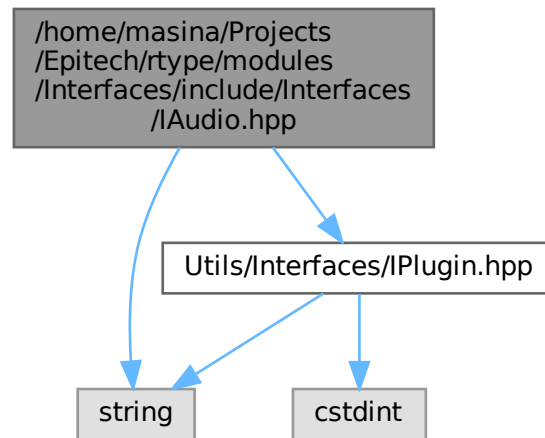
00065         std::shared_ptr<IRenderer> m_renderer;  

00066     }; // class Engine  

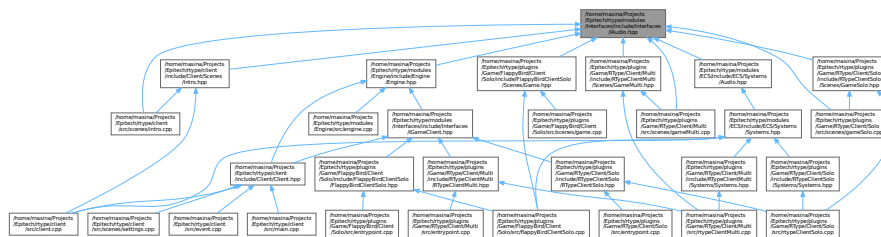
00067 } // namespace eng

```


Include dependency graph for IAudio.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- interface [eng::IAudio](#)
Interface for the audio.

Namespaces

- namespace [eng](#)

Enumerations

- enum class [eng::Status](#) { [eng::Stopped](#) , [eng::Paused](#) , [eng::Playing](#) }

19.77.1 Detailed Description

This file contains the Audio interface.
Definition in file [IAudio.hpp](#).

19.78 IAudio.hpp

[Go to the documentation of this file.](#)

00001 ///

```

00002 /// @file IAudio.hpp
00003 /// @brief This file contains the Audio interface
00004 /// @namespace eng
00005 ///
00006
00007 #pragma once
00008
00009 #include <string>
00010
00011 #include "Utils/Interfaces/IPlugin.hpp"
00012
00013 namespace eng
00014 {
00015
00016     enum class Status
00017     {
00018         Stopped,
00019         Paused,
00020         Playing
00021     };
00022
00023 ///
00024 /// @interface IAudio
00025 /// @brief Interface for the audio
00026 /// @namespace eng
00027 ///
00028 class IAudio : public utl::IPlugin
00029 {
00030     public:
00031         virtual void createAudio(const std::string &path, float volume, bool loop, const std::string &name) = 0;
00032         virtual void playAudio(const std::string &name) = 0;
00033
00034         virtual void setVolume(const std::string &name, float volume) = 0;
00035         virtual void setLoop(const std::string &name, bool loop) = 0;
00036         virtual void stopAudio(const std::string &name) = 0;
00037         virtual Status isPlaying(const std::string &name) = 0;
00038
00039     private:
00040 }; // class IAudio
00041 } // namespace eng

```

19.79 /home/masina/Projects/Epitech/rtype/modules/↵ Interfaces/include/Interfaces/IGameClient.hpp File Reference

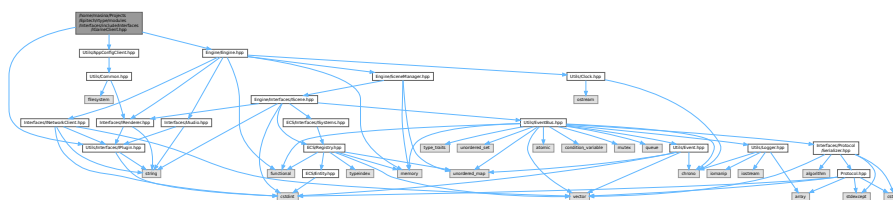
This file contains the Game interface.

#include "Utils/AppConfigClient.hpp"

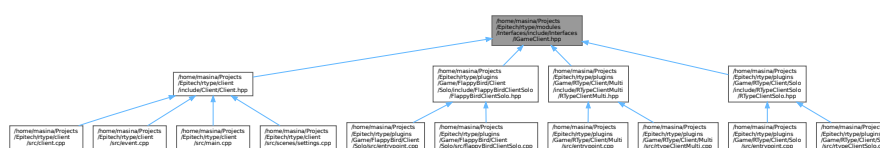
#include "Engine/Engine.hpp"

#include "Utils/Interfaces/IPlugin.hpp"

Include dependency graph for IGameClient.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- interface [gme::IGameClient](#)
Interface for the games.

Namespaces

- namespace [gme](#)

19.79.1 Detailed Description

This file contains the Game interface.
Definition in file [IGameClient.hpp](#).

19.80 IGameClient.hpp

[Go to the documentation of this file.](#)

```

00001 ///  

00002 ///  

00003 ///  

00004 ///  

00005 ///  

00006 ///  

00007 #pragma once  

00008  

00009 #include "Utils/AppConfigClient.hpp"  

00010  

00011 #include "Engine/Engine.hpp"  

00012 #include "Utils/Interfaces/IPlugin.hpp"  

00013  

00014 namespace gme  

00015 {  

00016  

00017     ///  

00018     ///  

00019     ///  

00020     ///  

00021     ///  

00022     class IGameClient : public utl::IPlugin  

00023     {  

00024     public:  

00025         virtual void init(eng::Engine &engine, utl::cli::AppConfig &appConfig, bool &showDebug,  

00026             unsigned int menuSceneId) = 0;  

00027         virtual void update(float deltaTime, unsigned int width, unsigned int height) = 0;  

00028         virtual unsigned int getMainSceneId() const = 0;  

00029  

00030     private:  

00031     }; // class IGameClient  

00032 } // namespace gme

```

19.81 /home/masina/Projects/Epitech/rtype/modules/ Interfaces/include/Interfaces/IGameServer.hpp File Reference

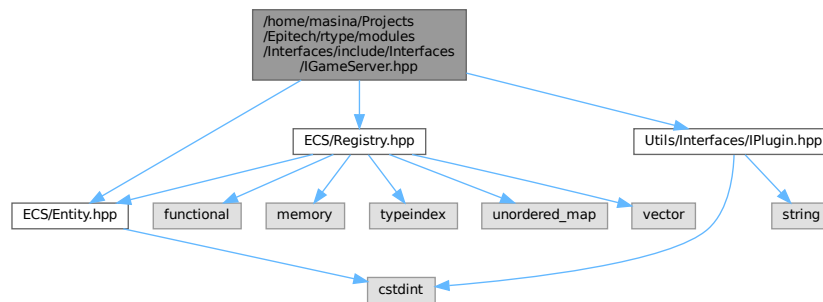
This file contains the Game interface.

```

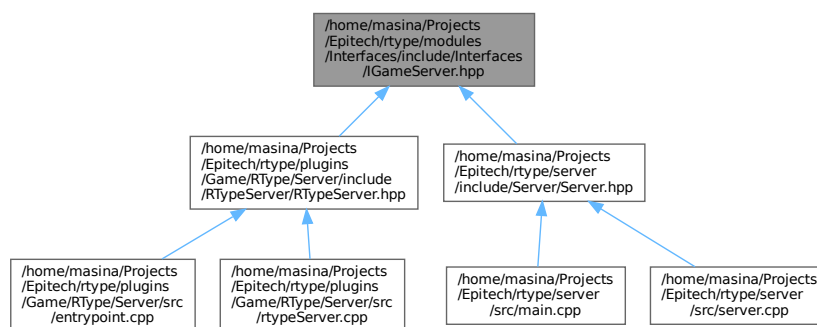
#include "ECS/Entity.hpp"
#include "ECS/Registry.hpp"
#include "Utils/Interfaces/IPlugin.hpp"

```

Include dependency graph for IGameServer.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- interface `gme::IGameServer`
Interface for the games.

Namespaces

- namespace `gme`

Enumerations

- enum class `gme::State` : `uint8_t` { `gme::PLAYING` = 0 , `gme::WIN` = 1 , `gme::LOSE` = 2 }

19.81.1 Detailed Description

This file contains the Game interface.
Definition in file [IGameServer.hpp](#).

19.82 IGameServer.hpp

[Go to the documentation of this file.](#)

```

00001 ///
00002 /// @file IGameServer.hpp
00003 /// @brief This file contains the Game interface
00004 /// @namespace gme
00005 ///

```

```

00006
00007 #pragma once
00008
00009 #include "ECS/Entity.hpp"
00010 #include "ECS/Registry.hpp"
00011 #include "Utils/Interfaces/IPlugin.hpp"
00012
00013 namespace gme
00014 {
00015
00016     enum class State : uint8_t
00017     {
00018         PLAYING = 0,
00019         WIN = 1,
00020         LOSE = 2,
00021     };
00022
00023     ///
00024     /// @interface IGameServer
00025     /// @brief Interface for the games
00026     /// @namespace gme
00027     ///
00028     class IGameServer : public utl::IPlugin
00029     {
00030     public:
00031         [[nodiscard]] virtual State getState() const = 0;
00032
00033         virtual void start() = 0;
00034         virtual void stop() = 0;
00035         virtual void update(float deltaTime) = 0;
00036
00037     private:
00038     }; // class IGameServer
00039 } // namespace gme

```

19.83 /home/masina/Projects/Epitech/rtype/modules/ Interfaces/include/Interfaces/INetworkClient.hpp File Reference

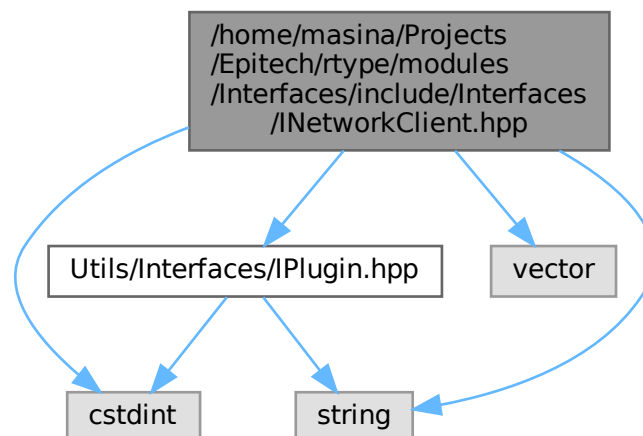
This file contains the client network interface.

```

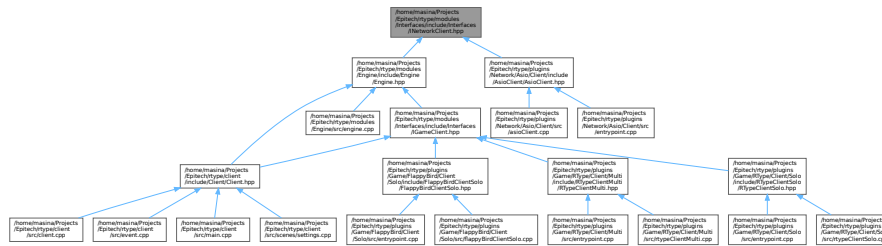
#include <cstdint>
#include <string>
#include <vector>
#include "Utils/Interfaces/IPlugin.hpp"

```

Include dependency graph for INetworkClient.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- interface [eng::INetworkClient](#)
Interface for the client network.

Namespaces

- namespace [eng](#)

Enumerations

- enum class [eng::ConnectionState](#) { [eng::DISCONNECTED](#) , [eng::CONNECTING](#) , [eng::CONNECTED](#) , [eng::DISCONNECTING](#) }
Connection state enumeration.

19.83.1 Detailed Description

This file contains the client network interface.

Definition in file [INetworkClient.hpp](#).

19.84 INetworkClient.hpp

[Go to the documentation of this file.](#)

```
00001 ///  
00002 ///  
00003 ///  
00004 ///  
00005 ///  
00006 ///  
00007 #pragma once  
00008 ///  
00009 #include <cstdint>  
00010 #include <string>  
00011 #include <vector>  
00012 ///  
00013 #include "Utils/Interfaces/IPlugin.hpp"  
00014 ///  
00015 namespace eng  
00016 {  
00017 ///  
00018 ///  
00019 ///  
00020 ///  
00021 enum class ConnectionState  
00022 {  
00023     DISCONNECTED,  
00024     CONNECTING,  
00025     CONNECTED,  
00026     DISCONNECTING  
00027 };  
00028 ///  
00029 ///  
00030 ///  
00031 ///  
00032 ///  
00033 ///  
00034 class INetworkClient : public util::IPlugin
```

```

00035 {
00036     public:
00037         virtual ~INetworkClient() = default;
00038
00039         // Connection management
00040         virtual void connect(const std::string &host, uint16_t port) = 0;
00041         virtual void disconnect() = 0;
00042
00043         // Packet handling
00044         virtual void update() = 0;
00045
00046         virtual void sendToServer(const std::vector<std::uint8_t> &data, bool reliable = false) = 0;
00047
00048         // Connection status
00049         [[nodiscard]] virtual bool isConnected() const = 0;
00050         [[nodiscard]] virtual ConnectionState getConnectionState() const = 0;
00051         [[nodiscard]] virtual std::uint32_t getSessionId() const = 0;
00052         [[nodiscard]] virtual std::uint16_t getServerTickRate() const = 0;
00053         [[nodiscard]] virtual std::uint32_t getLatency() const = 0;
00054
00055         // Client configuration
00056         virtual void setPlayerName(const std::string &playerName) = 0;
00057         virtual void setClientCapabilities(std::uint32_t caps) = 0;
00058
00059     private:
00060 }; // class INetworkClient
00061 } // namespace eng

```

19.85 /home/masina/Projects/Epitech/rtype/modules/↵ Interfaces/include/Interfaces/INetworkServer.hpp File Reference

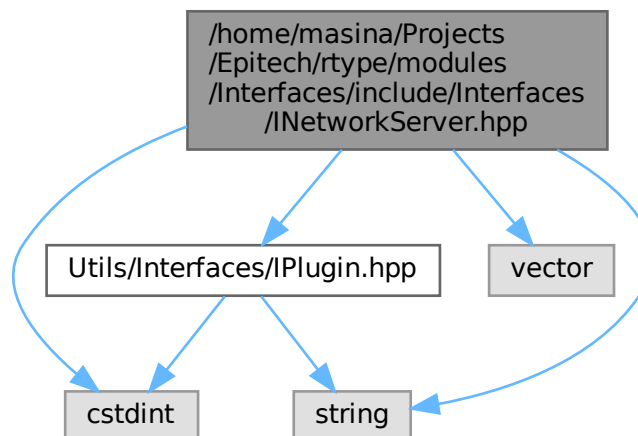
This file contains the server network interface.

```

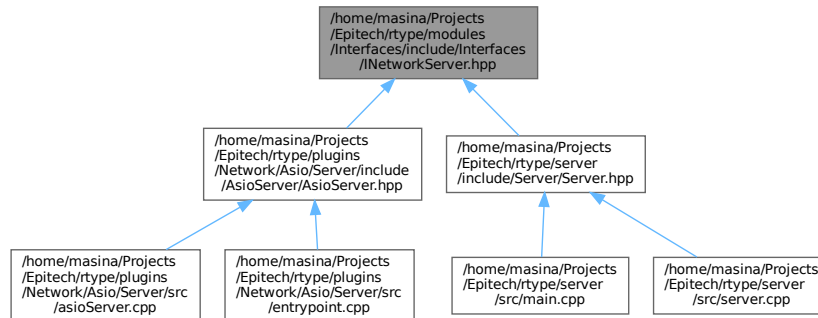
#include <stdint>
#include <string>
#include <vector>
#include "Utils/Interfaces/IPlugin.hpp"

```

Include dependency graph for INetworkServer.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- interface [srv::INetworkServer](#)
Interface for the server network.

Namespaces

- namespace [srv](#)

Variables

- constexpr size_t [srv::MAX_CLIENTS](#) = 16
- constexpr size_t [srv::MAX_IP_LENGTH](#) = 8
- constexpr size_t [srv::MAX_LEN_RECV_BUFFER](#) = 1024

19.85.1 Detailed Description

This file contains the server network interface.
Definition in file [INetworkServer.hpp](#).

19.86 INetworkServer.hpp

[Go to the documentation of this file.](#)

```

00001 ///  

00002 ///  

00003 ///  

00004 ///  

00005 ///  

00006 ///  

00007 #pragma once  

00008  

00009 #include <stdint>  

00010 #include <string>  

00011 #include <vector>  

00012  

00013 #include "Utils/Interfaces/IPlugin.hpp"  

00014  

00015 namespace srv  

00016 {  

00017  

00018     constexpr size_t MAX_CLIENTS = 16;  

00019     constexpr size_t MAX_IP_LENGTH = 8;  

00020     constexpr size_t MAX_LEN_RECV_BUFFER = 1024;  

00021  

00022     ///  

00023     ///  

00024     ///  

00025     ///  

00026     ///  

00027     class INetworkServer : public utl::IPlugin
  
```

```

00028 {
00029     public:
00030         virtual ~INetworkServer() = default;
00031
00032         // Server lifecycle
00033         virtual void init(const std::string &host, uint16_t port) = 0;
00034         virtual void start() = 0;
00035         virtual void stop() = 0;
00036         virtual void update() = 0;
00037
00038         // Client management
00039         virtual void sendToClient(std::uint32_t sessionId, const std::vector<std::uint8_t> &data,
00040                                 bool reliable = false) = 0;
00041         virtual void sendToAllClients(const std::vector<std::uint8_t> &data, bool reliable = false) = 0;
00042         virtual void disconnectClient(std::uint32_t sessionId) = 0;
00043
00044         // Server information
00045         [[nodiscard]] virtual std::size_t getClientCount() const = 0;
00046         [[nodiscard]] virtual std::vector<std::uint32_t> getConnectedSessions() const = 0;
00047         [[nodiscard]] virtual bool isRunning() const = 0;
00048
00049         // Configuration
00050         virtual void setTickRate(std::uint16_t tickRate) = 0;
00051         virtual void setServerCapabilities(std::uint32_t caps) = 0;
00052
00053     private:
00054 }; // class INetworkServer
00055 } // namespace srv

```

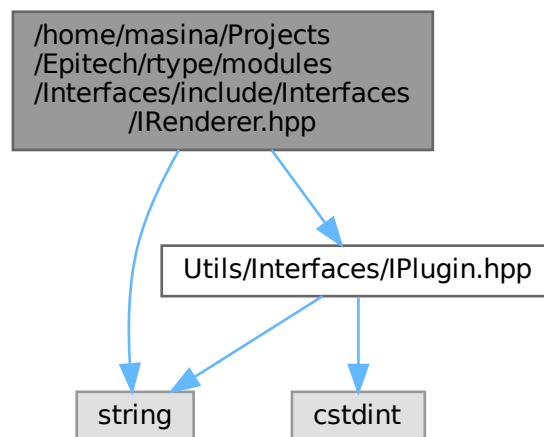
19.87 /home/masina/Projects/Epitech/rtype/modules/ Interfaces/include/Interfaces/IRenderer.hpp File Reference

This file contains the IRenderer class declaration.

```
#include <string>
```

```
#include "Utils/Interfaces/IPlugin.hpp"
```

Include dependency graph for IRenderer.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- struct [eng::Color](#)
- struct [eng::Text](#)
- struct [eng::CircleShape](#)
- struct [eng::Event](#)
- struct [eng::WindowSize](#)
- interface [eng::IRenderer](#)

Interface for the renderer.

Namespaces

- namespace [eng](#)

Enumerations

- enum class [eng::Key](#) {
[eng::Unknown](#) , [eng::Escape](#) , [eng::Enter](#) , [eng::Space](#) ,
[eng::Up](#) , [eng::Down](#) , [eng::Left](#) , [eng::Right](#) ,
[eng::Delete](#) , [eng::Dot](#) , [eng::A](#) , [eng::B](#) ,
[eng::C](#) , [eng::D](#) , [eng::E](#) , [eng::F](#) ,
[eng::G](#) , [eng::H](#) , [eng::I](#) , [eng::J](#) ,
[eng::K](#) , [eng::L](#) , [eng::M](#) , [eng::N](#) ,
[eng::O](#) , [eng::P](#) , [eng::Q](#) , [eng::R](#) ,
[eng::S](#) , [eng::T](#) , [eng::U](#) , [eng::V](#) ,
[eng::W](#) , [eng::X](#) , [eng::Y](#) , [eng::Z](#) ,
[eng::Num0](#) , [eng::Num1](#) , [eng::Num2](#) , [eng::Num3](#) ,
[eng::Num4](#) , [eng::Num5](#) , [eng::Num6](#) , [eng::Num7](#) ,
[eng::Num8](#) , [eng::Num9](#) , [eng::LControl](#) }
- enum class [eng::EventType](#) { [eng::Closed](#) , [eng::KeyPressed](#) , [eng::KeyReleased](#) , [eng::None](#) }

19.87.1 Detailed Description

This file contains the IRenderer class declaration.

Definition in file [IRenderer.hpp](#).

19.88 IRenderer.hpp

[Go to the documentation of this file.](#)

```

00001 ///  

00002 ///  

00003 ///  

00004 ///  

00005 ///  

00006 ///  

00007 #pragma once  

00008  

00009 #include <string>  

00010  

00011 #include "Utils/Interfaces/IPPlugin.hpp"  

00012  

00013 namespace eng  

00014 {  

00015     struct Color  

00016     {  

00017         unsigned char r;  

00018         unsigned char g;  

00019         unsigned char b;  

00020         unsigned char a;  

00021     };  

00022     struct Text  

00023     {  

00024         std::string font_name;  

00025         Color color;  

00026         std::string content;  

00027         unsigned int size;  

00028         float x;  

00029         float y;  


```

```

00030         std::string name;
00031     };
00032
00033     struct CircleShape
00034     {
00035         std::string name;
00036         float radius;
00037         Color color;
00038         float x;
00039         float y;
00040         float outline_thickness;
00041         Color outline_color;
00042     };
00043
00044     enum class Key
00045     {
00046         Unknown,
00047         Escape,
00048         Enter,
00049         Space,
00050         Up,
00051         Down,
00052         Left,
00053         Right,
00054         Delete,
00055         Dot,
00056         A,
00057         B,
00058         C,
00059         D,
00060         E,
00061         F,
00062         G,
00063         H,
00064         I,
00065         J,
00066         K,
00067         L,
00068         M,
00069         N,
00070         O,
00071         P,
00072         Q,
00073         R,
00074         S,
00075         T,
00076         U,
00077         V,
00078         W,
00079         X,
00080         Y,
00081         Z,
00082         Num0,
00083         Num1,
00084         Num2,
00085         Num3,
00086         Num4,
00087         Num5,
00088         Num6,
00089         Num7,
00090         Num8,
00091         Num9,
00092         LControl,
00093     };
00094     enum class EventType
00095     {
00096         Closed,
00097         KeyPressed,
00098         KeyReleased,
00099         None
00100     };
00101
00102     struct Event
00103     {
00104         EventType type = EventType::None;
00105         Key key = Key::Unknown;
00106     };
00107
00108     struct WindowSize
00109     {
00110         unsigned int width;
00111         unsigned int height;
00112     };
00113
00114     ///
00115     /// @interface IRenderer
00116     /// @brief Interface for the renderer

```

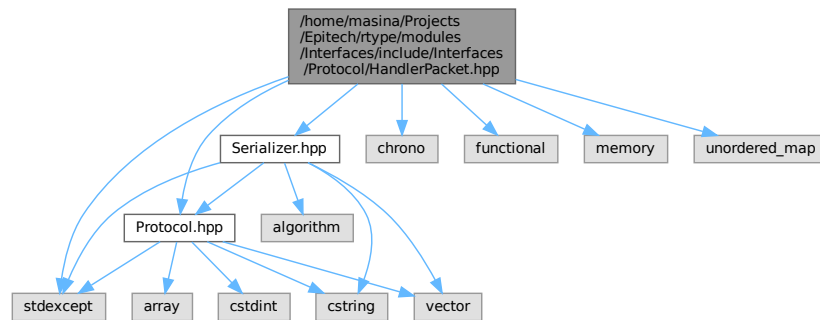
```
00117  /// @namespace eng
00118  ///
00119  class IRenderer : public utl::IPlugin
00120  {
00121
00122      public:
00123          virtual void createWindow(const std::string &title, unsigned int height, unsigned int width,
00124                                  unsigned int frameLimit, bool fullscreen) = 0;
00125          [[nodiscard]] virtual bool windowIsOpen() const = 0;
00126          virtual void closeWindow() = 0;
00127          virtual void clearWindow(Color color) = 0;
00128          virtual void displayWindow() = 0;
00129          [[nodiscard]] virtual WindowSize getWindowSize() = 0;
00130          virtual void setWindowIcon(const std::string &path) = 0;
00131
00132          [[nodiscard]] virtual bool pollEvent(Event &event) = 0;
00133          virtual void setFrameLimit(unsigned int frameLimit) = 0;
00134
00135          virtual void createFont(const std::string &name, const std::string &path) = 0;
00136          virtual void createText(Text text) = 0;
00137          virtual void drawText(const std::string &name) = 0;
00138          virtual void setTextContent(const std::string &name, const std::string &content) = 0;
00139          virtual void setTextPosition(const std::string &name, float x, float y) = 0;
00140          virtual void setTextColor(const std::string &name, Color color) = 0;
00141          virtual void setSpriteColor(const std::string &name, Color color) = 0;
00142
00143          virtual void createTexture(const std::string &name, const std::string &path) = 0;
00144          virtual void createSprite(const std::string &name, const std::string &textureName, float x, float y,
00145                                  float scale_x = 1, float scale_y = 1, int fx = 0, int fy = 0, int fnx = -1,
00146                                  int fny = -1) = 0;
00147          virtual void drawSprite(const std::string &name) = 0;
00148          virtual void setSpritePosition(const std::string &name, float x, float y) = 0;
00149          virtual void setSpriteTexture(const std::string &name, const std::string &path) = 0;
00150          virtual void setSpriteScale(const std::string &name, int x, int y) = 0;
00151          virtual void setSpriteFrame(const std::string &name, int fx, int fy, int fnx, int fny) = 0;
00152          virtual void setSpriteRotation(const std::string &name, float rotation) = 0;
00153          virtual void rotateSprite(const std::string &name, float rotation) = 0;
00154          virtual void setSpriteOrigin(const std::string &name) = 0;
00155
00156          virtual void createCircleShape(CircleShape circleShape) = 0;
00157          virtual void setCircleShapePosition(const std::string &name, float x, float y) = 0;
00158          virtual void drawCircleShape(const std::string &name) = 0;
00159
00160          virtual void drawPoint(float x, float y, Color color) = 0;
00161          virtual void renderGui(const WindowSize &>windowSize) = 0;
00162
00163      private:
00164  }; // class IRenderer
00165 } // namespace eng
```

19.89 /home/masina/Projects/Epitech/rtype/modules/↵ Interfaces/include/Interfaces/Protocol/HandlerPacket.hpp File Reference

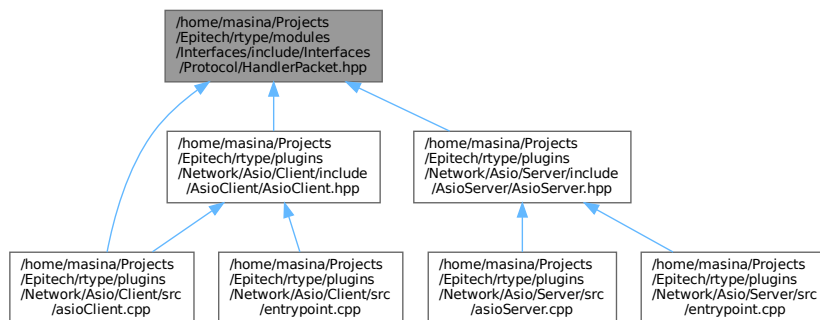
Network packet handler for RNP protocol.

```
#include "Protocol.hpp"
#include "Serializer.hpp"
#include <chrono>
#include <functional>
#include <memory>
#include <stdexcept>
#include <unordered_map>
```

Include dependency graph for HandlerPacket.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- struct [rnp::PacketContext](#)
Context information for packet processing.
- struct [rnp::HandlerStats](#)
Statistics for packet handling.
- class [rnp::HandlerPacket](#)
Main packet handler class.

Namespaces

- namespace [rnp](#)

Typedefs

- using [rnp::ConnectHandler](#) = std::function<[HandlerResult](#)(const [PacketConnect](#) &, const [PacketContext](#) &)>
- using [rnp::ConnectAcceptHandler](#) = std::function<[HandlerResult](#)(const [PacketConnectAccept](#) &, const [PacketContext](#) &)>
- using [rnp::DisconnectHandler](#) = std::function<[HandlerResult](#)(const [PacketDisconnect](#) &, const [PacketContext](#) &)>
- using [rnp::WorldStateHandler](#) = std::function<[HandlerResult](#)(const [PacketWorldState](#) &, const [PacketContext](#) &)>

- using `rnp::PingHandler` = `std::function<HandlerResult(const PacketPingPong &, const PacketContext &>`
- using `rnp::PongHandler` = `std::function<HandlerResult(const PacketPingPong &, const PacketContext &>`
- using `rnp::ErrorHandler` = `std::function<HandlerResult(const PacketError &, const PacketContext &>`
- using `rnp::EntityEventHandler` = `std::function<HandlerResult(const std::vector<EventRecord> &, const PacketContext &>`
- using `rnp::AckHandler` = `std::function<HandlerResult(std::uint32_t sequenceId, const PacketContext &>`
- using `rnp::LobbyListRequestHandler` = `std::function<HandlerResult(const PacketContext &>`
- using `rnp::LobbyListResponseHandler`
- using `rnp::LobbyCreateHandler` = `std::function<HandlerResult(const PacketLobbyCreate &, const PacketContext &>`
- using `rnp::LobbyCreateResponseHandler`
- using `rnp::LobbyJoinHandler` = `std::function<HandlerResult(const PacketLobbyJoin &, const PacketContext &>`
- using `rnp::LobbyJoinResponseHandler`
- using `rnp::LobbyLeaveHandler` = `std::function<HandlerResult(const PacketContext &>`
- using `rnp::LobbyUpdateHandler` = `std::function<HandlerResult(const PacketLobbyUpdate &, const PacketContext &>`
- using `rnp::GameStartHandler` = `std::function<HandlerResult(const PacketGameStart &, const PacketContext &>`
- using `rnp::StartGameRequestHandler` = `std::function<HandlerResult(const PacketStartGameRequest &, const PacketContext &>`

Enumerations

- enum class `rnp::HandlerResult` {
`rnp::SUCCESS` , `rnp::INVALID_PACKET` , `rnp::UNSUPPORTED_TYPE` , `rnp::PROCESSING_ERROR`
, `rnp::SESSION_INVALID` , `rnp::RATE_LIMITED` }
Packet processing result.

19.89.1 Detailed Description

Network packet handler for RNP protocol.

Definition in file [HandlerPacket.hpp](#).

19.90 HandlerPacket.hpp

[Go to the documentation of this file.](#)

```
00001 ///  
00002 ///  
00003 ///  
00004 ///  
00005 ///  
00006 ///  
00007 #pragma once  
00008  
00009 #include "Protocol.hpp"  
00010 #include "Serializer.hpp"  
00011 #include <chrono>  
00012 #include <functional>  
00013 #include <memory>  
00014 #include <stdexcept>  
00015 #include <unordered_map>  
00016  
00017 namespace rnp  
00018 {  
00019  
00020     ///  
00021     ///  
00022     ///  
00023     enum class HandlerResult
```

```

00024 {
00025     SUCCESS,
00026     INVALID_PACKET,
00027     UNSUPPORTED_TYPE,
00028     PROCESSING_ERROR,
00029     SESSION_INVALID,
00030     RATE_LIMITED
00031 };
00032
00033 ///
00034 /// @brief Context information for packet processing
00035 ///
00036 struct PacketContext
00037 {
00038     std::uint32_t sessionId;
00039     std::chrono::steady_clock::time_point receiveTime;
00040     std::string senderAddress;
00041     std::uint16_t senderPort;
00042     bool isReliable;
00043     bool isCompressed;
00044 };
00045
00046 ///
00047 /// @brief Forward declarations for callback types
00048 ///
00049 class HandlerPacket;
00050
00051 using ConnectHandler = std::function<HandlerResult(const PacketConnect &, const PacketContext &)>;
00052 using ConnectAcceptHandler = std::function<HandlerResult(const PacketConnectAccept &, const PacketContext &)>;
00053 using DisconnectHandler = std::function<HandlerResult(const PacketDisconnect &, const PacketContext &)>;
00054 using WorldStateHandler = std::function<HandlerResult(const PacketWorldState &, const PacketContext &)>;
00055 using PingHandler = std::function<HandlerResult(const PacketPingPong &, const PacketContext &)>;
00056 using PongHandler = std::function<HandlerResult(const PacketPingPong &, const PacketContext &)>;
00057 using ErrorHandler = std::function<HandlerResult(const PacketError &, const PacketContext &)>;
00058 using EntityEventHandler = std::function<HandlerResult(const std::vector<EventRecord> &, const PacketContext
&)>;
00059 using AckHandler = std::function<HandlerResult(std::uint32_t sequenceId, const PacketContext &)>;
00060
00061 // Lobby system handlers
00062 using LobbyListRequestHandler = std::function<HandlerResult(const PacketContext &)>;
00063 using LobbyListResponseHandler =
00064     std::function<HandlerResult(const PacketLobbyListResponse &, const PacketContext &)>;
00065 using LobbyCreateHandler = std::function<HandlerResult(const PacketLobbyCreate &, const PacketContext &)>;
00066 using LobbyCreateResponseHandler =
00067     std::function<HandlerResult(const PacketLobbyCreateResponse &, const PacketContext &)>;
00068 using LobbyJoinHandler = std::function<HandlerResult(const PacketLobbyJoin &, const PacketContext &)>;
00069 using LobbyJoinResponseHandler =
00070     std::function<HandlerResult(const PacketLobbyJoinResponse &, const PacketContext &)>;
00071 using LobbyLeaveHandler = std::function<HandlerResult(const PacketContext &)>;
00072 using LobbyUpdateHandler = std::function<HandlerResult(const PacketLobbyUpdate &, const PacketContext &)>;
00073 using GameStartHandler = std::function<HandlerResult(const PacketGameStart &, const PacketContext &)>;
00074 using StartGameRequestHandler = std::function<HandlerResult(const PacketStartGameRequest &, const
PacketContext &)>;
00075
00076 ///
00077 /// @brief Statistics for packet handling
00078 ///
00079 struct HandlerStats
00080 {
00081     std::uint64_t totalPacketsReceived = 0;
00082     std::uint64_t totalPacketsProcessed = 0;
00083     std::uint64_t totalPacketsDropped = 0;
00084     std::uint64_t totalBytesReceived = 0;
00085     std::unordered_map<PacketType, std::uint64_t> packetTypeCount;
00086     std::unordered_map<HandlerResult, std::uint64_t> resultCount;
00087 };
00088
00089 ///
00090 /// @brief Main packet handler class
00091 ///
00092 class HandlerPacket
00093 {
00094     private:
00095         // Handler callbacks
00096         ConnectHandler connectHandler_;
00097         ConnectAcceptHandler connectAcceptHandler_;
00098         DisconnectHandler disconnectHandler_;
00099         WorldStateHandler worldStateHandler_;
00100         PingHandler pingHandler_;
00101         PongHandler pongHandler_;
00102         ErrorHandler errorHandler_;
00103         EntityEventHandler entityEventHandler_;
00104         AckHandler ackHandler_;
00105
00106         // Lobby system handlers
00107         LobbyListRequestHandler lobbyListRequestHandler_;
00108         LobbyListResponseHandler lobbyListResponseHandler_;

```



```

00109     LobbyCreateHandler lobbyCreateHandler_;
00110     LobbyCreateResponseHandler lobbyCreateResponseHandler_;
00111     LobbyJoinHandler lobbyJoinHandler_;
00112     LobbyJoinResponseHandler lobbyJoinResponseHandler_;
00113     LobbyLeaveHandler lobbyLeaveHandler_;
00114     LobbyUpdateHandler lobbyUpdateHandler_;
00115     GameStartHandler gameStartHandler_;
00116     StartGameRequestHandler startGameRequestHandler_;
00117
00118     // Statistics
00119     HandlerStats stats_;
00120
00121     // Rate limiting (simple token bucket per session)
00122     std::unordered_map<std::uint32_t, std::chrono::steady_clock::time_point> lastPacketTime_;
00123     std::chrono::milliseconds rateLimitInterval_{10}; // 10ms minimum between packets
00124
00125     ///
00126     /// @brief Check rate limiting for a session
00127     /// @param sessionId Session to check
00128     /// @param currentTime Current time
00129     /// @return True if packet should be rate limited
00130     ///
00131     bool isRateLimited(std::uint32_t sessionId, const std::chrono::steady_clock::time_point &currentTime)
00132     {
00133         auto it = lastPacketTime_.find(sessionId);
00134         if (it == lastPacketTime_.end())
00135         {
00136             lastPacketTime_[sessionId] = currentTime;
00137             return false;
00138         }
00139
00140         auto timeSinceLastPacket = currentTime - it->second;
00141         if (timeSinceLastPacket < rateLimitInterval_)
00142         {
00143             return true;
00144         }
00145
00146         it->second = currentTime;
00147         return false;
00148     }
00149
00150     ///
00151     /// @brief Update statistics
00152     /// @param packetType Type of packet processed
00153     /// @param result Processing result
00154     /// @param bytesReceived Number of bytes in the packet
00155     ///
00156     void updateStats(PacketType packetType, HandlerResult result, std::size_t bytesReceived)
00157     {
00158         stats_.totalPacketsReceived++;
00159         stats_.totalBytesReceived += bytesReceived;
00160         stats_.packetTypeCount[packetType]++;
00161         stats_.resultCount[result]++;
00162
00163         if (result == HandlerResult::SUCCESS)
00164         {
00165             stats_.totalPacketsProcessed++;
00166         }
00167         else
00168         {
00169             stats_.totalPacketsDropped++;
00170         }
00171     }
00172
00173     ///
00174     /// @brief Parse and handle ENTITY_EVENT packet
00175     /// @param serializer Serializer containing the packet data
00176     /// @param payloadSize Size of the payload
00177     /// @return Vector of event records
00178     ///
00179     std::vector<EventRecord> parseEntityEvents(Serializer &serializer, std::size_t payloadSize)
00180     {
00181         std::vector<EventRecord> events;
00182         std::size_t bytesRead = 0;
00183
00184         while (bytesRead < payloadSize)
00185         {
00186             EventRecord event;
00187             event.type = static_cast<EventType>(serializer.readByte());
00188             event.entityId = serializer.readUInt32();
00189
00190             std::uint16_t dataLength = serializer.readUInt16();
00191             bytesRead += 7; // 1 + 4 + 2 bytes for type, entityId, dataLength
00192
00193             if (dataLength > 0)
00194             {
00195                 event.data.resize(dataLength);

```

```

00196         serializer.readBytes(event.data.data(), dataLength);
00197         bytesRead += dataLength;
00198     }
00199
00200     events.push_back(std::move(event));
00201 }
00202
00203     return events;
00204 }
00205
00206 public:
00207     ///
00208     /// @brief Constructor
00209     ///
00210     HandlerPacket() = default;
00211
00212     ///
00213     /// @brief Destructor
00214     ///
00215     ~HandlerPacket() = default;
00216
00217     // Delete copy constructor and assignment operator
00218     HandlerPacket(const HandlerPacket &) = delete;
00219     HandlerPacket &operator=(const HandlerPacket &) = delete;
00220
00221     ///
00222     /// @brief Set rate limiting interval
00223     /// @param interval Minimum time between packets per session
00224     ///
00225     void setRateLimitInterval(std::chrono::milliseconds interval) { rateLimitInterval_ = interval; }
00226
00227     ///
00228     /// @brief Register CONNECT packet handler
00229     /// @param handler Callback function for CONNECT packets
00230     ///
00231     void onConnect(ConnectHandler handler) { connectHandler_ = std::move(handler); }
00232
00233     ///
00234     /// @brief Register CONNECT_ACCEPT packet handler
00235     /// @param handler Callback function for CONNECT_ACCEPT packets
00236     ///
00237     void onConnectAccept(ConnectAcceptHandler handler) { connectAcceptHandler_ = std::move(handler); }
00238
00239     ///
00240     /// @brief Register DISCONNECT packet handler
00241     /// @param handler Callback function for DISCONNECT packets
00242     ///
00243     void onDisconnect(DisconnectHandler handler) { disconnectHandler_ = std::move(handler); }
00244
00245     ///
00246     /// @brief Register WORLD_STATE packet handler
00247     /// @param handler Callback function for WORLD_STATE packets
00248     ///
00249     void onWorldState(WorldStateHandler handler) { worldStateHandler_ = std::move(handler); }
00250
00251     ///
00252     /// @brief Register PING packet handler
00253     /// @param handler Callback function for PING packets
00254     ///
00255     void onPing(PingHandler handler) { pingHandler_ = std::move(handler); }
00256
00257     ///
00258     /// @brief Register PONG packet handler
00259     /// @param handler Callback function for PONG packets
00260     ///
00261     void onPong(PongHandler handler) { pongHandler_ = std::move(handler); }
00262
00263     ///
00264     /// @brief Register ERROR packet handler
00265     /// @param handler Callback function for ERROR packets
00266     ///
00267     void onError(ErrorHandler handler) { errorHandler_ = std::move(handler); }
00268
00269     ///
00270     /// @brief Register ENTITY_EVENT packet handler
00271     /// @param handler Callback function for ENTITY_EVENT packets
00272     ///
00273     void onEntityEvent(EntityEventHandler handler) { entityEventHandler_ = std::move(handler); }
00274
00275     ///
00276     /// @brief Register LOBBY_LIST_REQUEST packet handler
00277     /// @param handler Callback function for LOBBY_LIST_REQUEST packets
00278     ///
00279     void onLobbyListRequest(LobbyListRequestHandler handler) { lobbyListRequestHandler_ = std::move(handler); }
00280
00281     ///
00282     /// @brief Register LOBBY_LIST_RESPONSE packet handler

```

```

00283     /// @param handler Callback function for LOBBY_LIST_RESPONSE packets
00284     ///
00285     void onLobbyListResponse(LobbyListResponseHandler handler)
00286     {
00287         lobbyListResponseHandler_ = std::move(handler);
00288     }
00289     ///
00290     /// @brief Register LOBBY_CREATE packet handler
00291     /// @param handler Callback function for LOBBY_CREATE packets
00292     ///
00293     void onLobbyCreate(LobbyCreateHandler handler) { lobbyCreateHandler_ = std::move(handler); }
00294     ///
00295     /// @brief Register LOBBY_CREATE_RESPONSE packet handler
00296     /// @param handler Callback function for LOBBY_CREATE_RESPONSE packets
00297     ///
00298     void onLobbyCreateResponse(LobbyCreateResponseHandler handler)
00299     {
00300         lobbyCreateResponseHandler_ = std::move(handler);
00301     }
00302     ///
00303     /// @brief Register LOBBY_JOIN packet handler
00304     /// @param handler Callback function for LOBBY_JOIN packets
00305     ///
00306     void onLobbyJoin(LobbyJoinHandler handler) { lobbyJoinHandler_ = std::move(handler); }
00307     ///
00308     /// @brief Register LOBBY_JOIN_RESPONSE packet handler
00309     /// @param handler Callback function for LOBBY_JOIN_RESPONSE packets
00310     ///
00311     void onLobbyJoinResponse(LobbyJoinResponseHandler handler)
00312     {
00313         lobbyJoinResponseHandler_ = std::move(handler);
00314     }
00315     ///
00316     /// @brief Register LOBBY_LEAVE packet handler
00317     /// @param handler Callback function for LOBBY_LEAVE packets
00318     ///
00319     void onLobbyLeave(LobbyLeaveHandler handler) { lobbyLeaveHandler_ = std::move(handler); }
00320     ///
00321     /// @brief Register LOBBY_UPDATE packet handler
00322     /// @param handler Callback function for LOBBY_UPDATE packets
00323     ///
00324     void onLobbyUpdate(LobbyUpdateHandler handler) { lobbyUpdateHandler_ = std::move(handler); }
00325     ///
00326     /// @brief Register GAME_START packet handler
00327     /// @param handler Callback function for GAME_START packets
00328     ///
00329     void onGameStart(GameStartHandler handler) { gameStartHandler_ = std::move(handler); }
00330     ///
00331     /// @brief Register START_GAME_REQUEST packet handler
00332     /// @param handler Callback function for START_GAME_REQUEST packets
00333     ///
00334     void onStartGameRequest(StartGameRequestHandler handler) { startGameRequestHandler_ =
std::move(handler); }
00335     ///
00336     /// @brief Process a received packet
00337     /// @param data Raw packet data
00338     /// @param context Packet context information
00339     /// @return Processing result
00340     ///
00341     HandlerResult processPacket(const std::vector<std::uint8_t> &data, const PacketContext &context)
00342     {
00343         // PacketHeader serialized size: 1 (type) + 2 (length) + 4 (sessionId) = 7 bytes
00344         constexpr std::size_t PACKET_HEADER_SIZE = 7;
00345         if (data.size() < PACKET_HEADER_SIZE)
00346         {
00347             updateStats(static_cast<PacketType>(0), HandlerResult::INVALID_PACKET, data.size());
00348             return HandlerResult::INVALID_PACKET;
00349         }
00350         // Check rate limiting
00351         if (isRateLimited(context.sessionId, context.receiveTime))
00352         {
00353             updateStats(static_cast<PacketType>(0), HandlerResult::RATE_LIMITED, data.size());
00354             return HandlerResult::RATE_LIMITED;
00355         }
00356         try
00357         {

```

```

00369     Serializer serializer(data);
00370     PacketHeader header = serializer.deserializeHeader();
00371
00372     // Validate header
00373     if (header.length > MAX_PAYLOAD || header.sessionId != context.sessionId)
00374     {
00375         updateStats(static_cast<PacketType>(header.type), HandlerResult::INVALID_PACKET, data.size());
00376         return HandlerResult::INVALID_PACKET;
00377     }
00378
00379     PacketType packetType = static_cast<PacketType>(header.type);
00380     HandlerResult result = HandlerResult::UNSUPPORTED_TYPE;
00381
00382     switch (packetType)
00383     {
00384     case PacketType::CONNECT:
00385         if (connectHandler_)
00386         {
00387             auto packet = serializer.deserializeConnect();
00388             result = connectHandler_(packet, context);
00389         }
00390         break;
00391
00392     case PacketType::CONNECT_ACCEPT:
00393         if (connectAcceptHandler_)
00394         {
00395             auto packet = serializer.deserializeConnectAccept();
00396             result = connectAcceptHandler_(packet, context);
00397         }
00398         break;
00399
00400     case PacketType::DISCONNECT:
00401         if (disconnectHandler_)
00402         {
00403             auto packet = serializer.deserializeDisconnect();
00404             result = disconnectHandler_(packet, context);
00405         }
00406         break;
00407
00408     case PacketType::WORLD_STATE:
00409         if (worldStateHandler_)
00410         {
00411             auto packet = serializer.deserializeWorldState();
00412             result = worldStateHandler_(packet, context);
00413         }
00414         break;
00415
00416     case PacketType::PING:
00417         if (pingHandler_)
00418         {
00419             auto packet = serializer.deserializePingPong();
00420             result = pingHandler_(packet, context);
00421         }
00422         break;
00423
00424     case PacketType::PONG:
00425         if (pongHandler_)
00426         {
00427             auto packet = serializer.deserializePingPong();
00428             result = pongHandler_(packet, context);
00429         }
00430         break;
00431
00432     case PacketType::PACKET_ERROR:
00433         if (errorHandler_)
00434         {
00435             auto packet = serializer.deserializeError();
00436             result = errorHandler_(packet, context);
00437         }
00438         break;
00439
00440     case PacketType::ENTITY_EVENT:
00441         if (entityEventHandler_)
00442         {
00443             auto events = parseEntityEvents(serializer, header.length);
00444             result = entityEventHandler_(events, context);
00445         }
00446         break;
00447
00448     case PacketType::LOBBY_LIST_REQUEST:
00449         if (lobbyListRequestHandler_)
00450         {
00451             result = lobbyListRequestHandler_(context);
00452         }
00453         break;
00454
00455     case PacketType::LOBBY_LIST_RESPONSE:

```

```

00456         if (lobbyListResponseHandler_)
00457         {
00458             auto packet = serializer.deserializeLobbyListResponse();
00459             result = lobbyListResponseHandler_(packet, context);
00460         }
00461         break;
00462
00463     case PacketType::LOBBY_CREATE:
00464     {
00465         if (lobbyCreateHandler_)
00466         {
00467             auto packet = serializer.deserializeLobbyCreate();
00468             result = lobbyCreateHandler_(packet, context);
00469         }
00470         break;
00471
00472     case PacketType::LOBBY_CREATE_RESPONSE:
00473     {
00474         if (lobbyCreateResponseHandler_)
00475         {
00476             auto packet = serializer.deserializeLobbyCreateResponse();
00477             result = lobbyCreateResponseHandler_(packet, context);
00478         }
00479         break;
00480
00481     case PacketType::LOBBY_JOIN:
00482     {
00483         if (lobbyJoinHandler_)
00484         {
00485             auto packet = serializer.deserializeLobbyJoin();
00486             result = lobbyJoinHandler_(packet, context);
00487         }
00488         break;
00489
00490     case PacketType::LOBBY_JOIN_RESPONSE:
00491     {
00492         if (lobbyJoinResponseHandler_)
00493         {
00494             auto packet = serializer.deserializeLobbyJoinResponse();
00495             result = lobbyJoinResponseHandler_(packet, context);
00496         }
00497         break;
00498
00499     case PacketType::LOBBY_LEAVE:
00500     {
00501         if (lobbyLeaveHandler_)
00502         {
00503             result = lobbyLeaveHandler_(context);
00504         }
00505         break;
00506
00507     case PacketType::LOBBY_UPDATE:
00508     {
00509         if (lobbyUpdateHandler_)
00510         {
00511             auto packet = serializer.deserializeLobbyUpdate();
00512             result = lobbyUpdateHandler_(packet, context);
00513         }
00514         break;
00515
00516     case PacketType::GAME_START:
00517     {
00518         if (gameStartHandler_)
00519         {
00520             auto packet = serializer.deserializeGameStart();
00521             result = gameStartHandler_(packet, context);
00522         }
00523         break;
00524
00525     case PacketType::START_GAME_REQUEST:
00526     {
00527         if (startGameRequestHandler_)
00528         {
00529             auto packet = serializer.deserializeStartGameRequest();
00530             result = startGameRequestHandler_(packet, context);
00531         }
00532         break;
00533
00534     default:
00535         result = HandlerResult::UNSUPPORTED_TYPE;
00536         break;
00537     }
00538
00539     updateStats(packetType, result, data.size());
00540     return result;
00541 }
00542 catch (const std::exception &)
00543 {
00544     updateStats(static_cast<PacketType>(0), HandlerResult::PROCESSING_ERROR, data.size());
00545     return HandlerResult::PROCESSING_ERROR;
00546 }
00547 }
00548
00549 ///
00550 /// @brief Get handler statistics

```

```

00543     /// @return Current statistics
00544     ///
00545     const HandlerStats &getStats() const { return stats_; }
00546
00547     ///
00548     /// @brief Reset statistics
00549     ///
00550     void resetStats() { stats_ = HandlerStats{}; }
00551
00552     ///
00553     /// @brief Clear rate limiting data for a session
00554     /// @param sessionId Session to clear
00555     ///
00556     void clearSession(std::uint32_t sessionId) { lastPacketTime_.erase(sessionId); }
00557
00558     ///
00559     /// @brief Clear all rate limiting data
00560     ///
00561     void clearAllSessions() { lastPacketTime_.clear(); }
00562 };
00563
00564 } // namespace rnp

```

19.91 /home/masina/Projects/Epitech/rtype/modules/↵ Interfaces/include/Interfaces/Protocol/Protocol.hpp File Reference

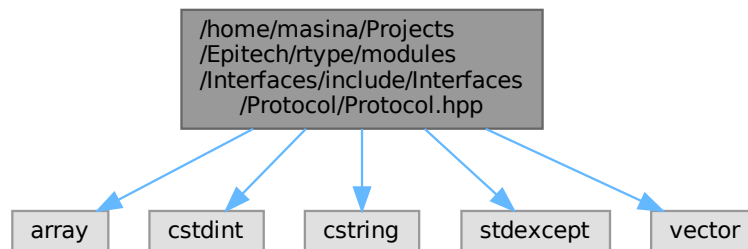
This file contains the network protocol.

```

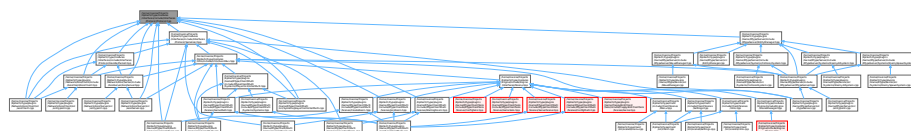
#include <array>
#include <cstdint>
#include <cstring>
#include <stdexcept>
#include <vector>

```

Include dependency graph for Protocol.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- struct [rnp::EventRecord](#)
Event record for ENTITY_EVENT packets (TLV format)
- struct [rnp::PacketHeader](#)

Packet header according to RNP specification (Big Endian) Total size: 7 bytes (1 + 2 + 4)

- struct [rnp::PacketConnect](#)
CONNECT packet payload.
- struct [rnp::PacketConnectAccept](#)
CONNECT_ACCEPT packet payload.
- struct [rnp::PacketDisconnect](#)
DISCONNECT packet payload.
- struct [rnp::EntityState](#)
Entity state for WORLD_STATE packet.
- struct [rnp::PacketWorldState](#)
WORLD_STATE packet payload.
- struct [rnp::PacketPingPong](#)
PING/PONG packet payload.
- struct [rnp::PacketError](#)
ERROR packet payload.
- struct [rnp::LobbyInfo](#)
Lobby information structure.
- struct [rnp::PacketLobbyListResponse](#)
LOBBY_LIST_RESPONSE packet payload.
- struct [rnp::PacketLobbyCreate](#)
LOBBY_CREATE packet payload.
- struct [rnp::PacketLobbyCreateResponse](#)
LOBBY_CREATE_RESPONSE packet payload.
- struct [rnp::PacketLobbyJoin](#)
LOBBY_JOIN packet payload.
- struct [rnp::PacketLobbyJoinResponse](#)
LOBBY_JOIN_RESPONSE packet payload.
- struct [rnp::PacketLobbyUpdate](#)
LOBBY_UPDATE packet payload.
- struct [rnp::PacketGameStart](#)
GAME_START packet payload.
- struct [rnp::PacketStartGameRequest](#)
START_GAME_REQUEST packet payload (client requests to start game)

Namespaces

- namespace [rnp](#)

Enumerations

- enum class [rnp::PacketType](#) : std::uint8_t {
[rnp::CONNECT](#) = 0x01 , [rnp::DISCONNECT](#) = 0x02 , [rnp::WORLD_STATE](#) = 0x03 , [rnp::PING](#) = 0x04 ,
[rnp::PONG](#) = 0x05 , [rnp::PACKET_ERROR](#) = 0x06 , [rnp::ENTITY_EVENT](#) = 0x07 ,
[rnp::CONNECT_ACCEPT](#) = 0x08 ,
[rnp::LOBBY_LIST_REQUEST](#) = 0x09 , [rnp::LOBBY_LIST_RESPONSE](#) = 0x0A ,
[rnp::LOBBY_CREATE](#) = 0x0B , [rnp::LOBBY_CREATE_RESPONSE](#) = 0x0C ,
[rnp::LOBBY_JOIN](#) = 0x0D , [rnp::LOBBY_JOIN_RESPONSE](#) = 0x0E , [rnp::LOBBY_LEAVE](#) = 0x0F ,
[rnp::LOBBY_UPDATE](#) = 0x10 ,
[rnp::GAME_START](#) = 0x11 , [rnp::START_GAME_REQUEST](#) = 0x12 }
 Packet types according to RNP specification.
- enum class [rnp::PacketFlags](#) : std::uint16_t { [rnp::NONE](#) = 0x0000 , [rnp::RELIABLE](#) = 0x0001 ,
[rnp::COMPRESSED](#) = 0x0002 }


```

00023 enum class PacketType : std::uint8_t
00024 {
00025     CONNECT = 0x01,
00026     DISCONNECT = 0x02,
00027     WORLD_STATE = 0x03,
00028     PING = 0x04,
00029     PONG = 0x05,
00030     PACKET_ERROR = 0x06,
00031     ENTITY_EVENT = 0x07,
00032     CONNECT_ACCEPT = 0x08,
00033     // Lobby system packets
00034     LOBBY_LIST_REQUEST = 0x09,
00035     LOBBY_LIST_RESPONSE = 0x0A,
00036     LOBBY_CREATE = 0x0B,
00037     LOBBY_CREATE_RESPONSE = 0x0C,
00038     LOBBY_JOIN = 0x0D,
00039     LOBBY_JOIN_RESPONSE = 0x0E,
00040     LOBBY_LEAVE = 0x0F,
00041     LOBBY_UPDATE = 0x10,
00042     GAME_START = 0x11,
00043     START_GAME_REQUEST = 0x12,
00044 };
00045
00046 ///
00047 /// @brief Packet flags for reliability and fragmentation
00048 ///
00049 enum class PacketFlags : std::uint16_t
00050 {
00051     NONE = 0x0000,
00052     RELIABLE = 0x0001,
00053     COMPRESSED = 0x0002
00054 };
00055
00056 ///
00057 /// @brief Disconnect reason codes
00058 ///
00059 enum class DisconnectReason : std::uint16_t
00060 {
00061     UNSPECIFIED = 0,
00062     CLIENT_REQUEST = 1,
00063     TIMEOUT = 2,
00064     PROTOCOL_ERROR = 3,
00065     SERVER_SHUTDOWN = 4,
00066     SERVER_FULL = 5,
00067     BANNED = 6
00068 };
00069
00070 ///
00071 /// @brief Error codes
00072 ///
00073 enum class ErrorCode : std::uint16_t
00074 {
00075     NONE = 0,
00076     INVALID_PAYLOAD = 1,
00077     UNAUTHORIZED_SESSION = 2,
00078     RATE_LIMITED = 3,
00079     INTERNAL_ERROR = 4,
00080     LOBBY_NOT_FOUND = 5,
00081     LOBBY_FULL = 6,
00082     ALREADY_IN_LOBBY = 7,
00083     NOT_IN_LOBBY = 8,
00084     NOT_LOBBY_HOST = 9
00085 };
00086
00087 ///
00088 /// @brief Event types for ENTITY_EVENT packets
00089 ///
00090 enum class EventType : std::uint8_t
00091 {
00092     SPAWN = 0x01,
00093     DESPAWN = 0x02,
00094     DAMAGE = 0x03,
00095     SCORE = 0x04,
00096     POWERUP = 0x05,
00097     INPUT = 0x06,
00098     CUSTOM = 0xFF
00099 };
00100
00101 ///
00102 /// @brief Entity types for world state
00103 ///
00104 enum class EntityType : std::uint16_t
00105 {
00106     PLAYER = 0x01,
00107     ENEMY = 0x02,
00108     PROJECTILE = 0x03,
00109     BOSS = 0x04,

```

```

00110     };
00111
00112     ///
00113     /// @brief Event record for ENTITY_EVENT packets (TLV format)
00114     ///
00115     struct EventRecord
00116     {
00117         EventType type;
00118         std::uint32_t entityId;
00119         std::vector<std::uint8_t> data;
00120     };
00121
00122     ///
00123     /// @brief Packet header according to RNP specification (Big Endian)
00124     /// Total size: 7 bytes (1 + 2 + 4)
00125     ///
00126     struct PacketHeader
00127     {
00128         std::uint8_t type;
00129         std::uint16_t length;
00130         std::uint32_t sessionId;
00131     };
00132
00133     ///
00134     /// @brief CONNECT packet payload
00135     ///
00136     struct PacketConnect
00137     {
00138         std::uint8_t nameLen;
00139         std::array<char, 32> playerName;
00140         std::uint32_t clientCaps;
00141     };
00142
00143     ///
00144     /// @brief CONNECT_ACCEPT packet payload
00145     ///
00146     struct PacketConnectAccept
00147     {
00148         std::uint32_t sessionId;
00149         std::uint16_t tickRateHz;
00150         std::uint16_t mtuPayloadBytes;
00151         std::uint32_t serverCaps;
00152     };
00153
00154     ///
00155     /// @brief DISCONNECT packet payload
00156     ///
00157     struct PacketDisconnect
00158     {
00159         std::uint16_t reasonCode; // DisconnectReason
00160     };
00161
00162     ///
00163     /// @brief Entity subtype enumeration
00164     ///
00165     enum class EntitySubtype : std::uint8_t
00166     {
00167         NONE = 0,
00168         // Enemy subtypes
00169         ENEMY_BASIC = 1,
00170         ENEMY_ADVANCED = 2,
00171         ENEMY_BOSS = 3,
00172         // Projectile subtypes
00173         PROJECTILE_PLAYER = 10,
00174         PROJECTILE_PLAYER_SUPERCHARGED = 11,
00175         PROJECTILE_ENEMY = 12,
00176     };
00177
00178     ///
00179     /// @brief Entity state for WORLD_STATE packet
00180     ///
00181     struct EntityState
00182     {
00183         std::uint32_t id;
00184         std::uint16_t type; // EntityType
00185         std::uint8_t subtype; // EntitySubtype - specific variant of entity
00186         float x, y;
00187         float vx, vy;
00188         std::uint8_t healthPercent; // Health as percentage (0-100), 255 = no health bar
00189         std::uint8_t stateFlags; // Additional flags (beam charge for players, etc.)
00190     };
00191
00192     ///
00193     /// @brief WORLD_STATE packet payload
00194     ///
00195     struct PacketWorldState
00196     {

```

```

00197         std::uint32_t serverTick;
00198         std::uint16_t entityCount;
00199         std::vector<EntityState> entities;
00200     };
00201
00202     ///
00203     /// @brief PING/PONG packet payload
00204     ///
00205     struct PacketPingPong
00206     {
00207         std::uint32_t nonce;
00208         std::uint32_t sendTimeMs;
00209     };
00210
00211     ///
00212     /// @brief ERROR packet payload
00213     ///
00214     struct PacketError
00215     {
00216         std::uint16_t errorCode; // ErrorCode
00217         std::uint16_t msgLen;
00218         std::string description;
00219     };
00220
00221     ///
00222     /// @brief Lobby status enumeration
00223     ///
00224     enum class LobbyStatus : std::uint8_t
00225     {
00226         WAITING = 0,
00227         IN_GAME = 1,
00228         FINISHED = 2
00229     };
00230
00231     ///
00232     /// @brief Lobby information structure
00233     ///
00234     struct LobbyInfo
00235     {
00236         std::uint32_t lobbyId;
00237         std::array<char, 32> lobbyName;
00238         std::uint8_t currentPlayers;
00239         std::uint8_t maxPlayers;
00240         std::uint8_t gameMode;
00241         std::uint8_t status; // LobbyStatus
00242         std::uint32_t hostSessionId;
00243         std::array<std::array<char, 32>, 8> playerNames; // Max 8 players
00244     };
00245
00246     ///
00247     /// @brief LOBBY_LIST_RESPONSE packet payload
00248     ///
00249     struct PacketLobbyListResponse
00250     {
00251         std::uint16_t lobbyCount;
00252         std::vector<LobbyInfo> lobbies;
00253     };
00254
00255     ///
00256     /// @brief LOBBY_CREATE packet payload
00257     ///
00258     struct PacketLobbyCreate
00259     {
00260         std::uint8_t nameLen;
00261         std::array<char, 32> lobbyName;
00262         std::uint8_t maxPlayers;
00263         std::uint8_t gameMode;
00264     };
00265
00266     ///
00267     /// @brief LOBBY_CREATE_RESPONSE packet payload
00268     ///
00269     struct PacketLobbyCreateResponse
00270     {
00271         std::uint32_t lobbyId;
00272         std::uint8_t success; // 0=failure, 1=success
00273         std::uint16_t errorCode; // ErrorCode if success=0
00274     };
00275
00276     ///
00277     /// @brief LOBBY_JOIN packet payload
00278     ///
00279     struct PacketLobbyJoin
00280     {
00281         std::uint32_t lobbyId;
00282     };
00283

```

```

00284    ///
00285    /// @brief LOBBY_JOIN_RESPONSE packet payload
00286    ///
00287    struct PacketLobbyJoinResponse
00288    {
00289        std::uint32_t lobbyId;
00290        std::uint8_t success; // 0=failure, 1=success
00291        std::uint16_t errorCode; // ErrorCode if success=0
00292        LobbyInfo lobbyInfo; // Current lobby state if success=1
00293    };
00294
00295    ///
00296    /// @brief LOBBY_UPDATE packet payload
00297    ///
00298    struct PacketLobbyUpdate
00299    {
00300        LobbyInfo lobbyInfo;
00301    };
00302
00303    ///
00304    /// @brief GAME_START packet payload
00305    ///
00306    struct PacketGameStart
00307    {
00308        std::uint32_t lobbyId;
00309    };
00310
00311    ///
00312    /// @brief START_GAME_REQUEST packet payload (client requests to start game)
00313    ///
00314    struct PacketStartGameRequest
00315    {
00316        std::uint32_t lobbyId;
00317    };
00318
00319 } // namespace rnp

```

19.93 /home/masina/Projects/Epitech/rtype/modules/ Interfaces/include/Interfaces/Protocol/Serializer.hpp File Reference

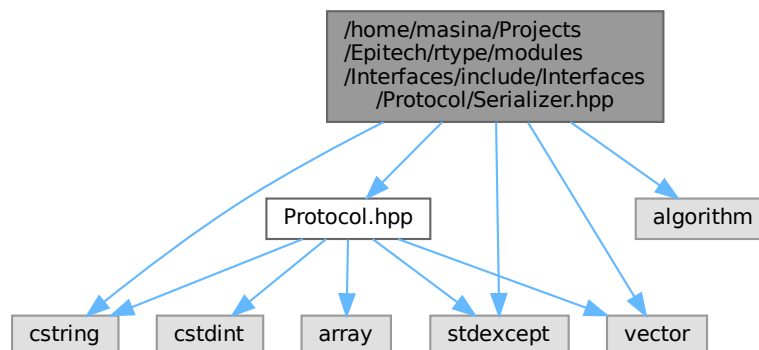
Network packet serializer for RNP protocol.

```

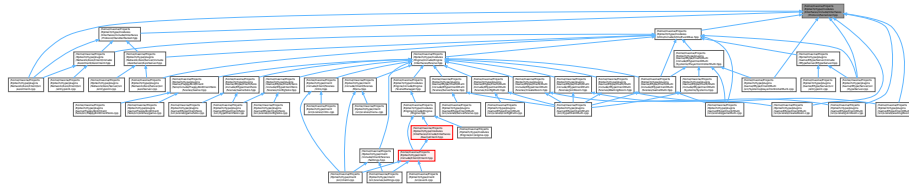
#include "Protocol.hpp"
#include <algorithm>
#include <cstring>
#include <stdexcept>
#include <vector>

```

Include dependency graph for Serializer.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [rnp::EndianUtils](#)
Utility class for endianness conversion.
- class [rnp::Serializer](#)
Binary serializer for RNP protocol packets.

Namespaces

- namespace [rnp](#)

19.93.1 Detailed Description

Network packet serializer for RNP protocol.

Definition in file [Serializer.hpp](#).

19.94 Serializer.hpp

[Go to the documentation of this file.](#)

```

00001 ///  

00002 ///  

00003 ///  

00004 ///  

00005 ///  

00006 ///  

00007 #pragma once  

00008 ///  

00009 #include "Protocol.hpp"  

00010 #include <algorithm>  

00011 #include <cstring>  

00012 #include <stdexcept>  

00013 #include <vector>  

00014 ///  

00015 namespace rnp  

00016 {  

00017     ///  

00018     ///  

00019     ///  

00020     ///  

00021     class EndianUtils  

00022     {  

00023     public:  

00024         static std::uint16_t hostToNetwork16(std::uint16_t hostValue)  

00025         {  

00026             return ((hostValue & 0xFF00) » 8) | ((hostValue & 0x00FF) « 8);  

00027         }  

00028         static std::uint32_t hostToNetwork32(std::uint32_t hostValue)  

00029         {  

00030             return ((hostValue & 0xFF000000) » 24) | ((hostValue & 0x00FF0000) » 8) |  

00031                 ((hostValue & 0x0000FF00) « 8) | ((hostValue & 0x000000FF) « 24);  

00032         }  

00033         static float hostToNetworkFloat(float hostValue)  

00034         {  

00035             std::uint32_t intValue;  

00036             std::memcpy(&intValue, &hostValue, sizeof(float));  

00037             intValue = hostToNetwork32(intValue);  

00038             float result;  

00039             std::memcpy(&result, &intValue, sizeof(float));  

00040             return result;  

00041         }  

00042     }  

00043 }

```

```

00044
00045     static std::uint16_t networkToHost16(std::uint16_t networkValue)
00046     {
00047         return hostToNetwork16(networkValue); // Same operation for 16-bit
00048     }
00049
00050     static std::uint32_t networkToHost32(std::uint32_t networkValue)
00051     {
00052         return hostToNetwork32(networkValue); // Same operation for 32-bit
00053     }
00054
00055     static float networkToHostFloat(float networkValue)
00056     {
00057         return hostToNetworkFloat(networkValue); // Same operation for float
00058     }
00059 };
00060
00061 ///
00062 /// @brief Binary serializer for RNP protocol packets
00063 ///
00064 class Serializer
00065 {
00066 private:
00067     std::vector<std::uint8_t> buffer_;
00068     std::size_t writePos_;
00069     std::size_t readPos_;
00070
00071 public:
00072     ///
00073     /// @brief Constructor
00074     ///
00075     Serializer() : writePos_(0), readPos_(0)
00076     {
00077         // PacketHeader serialized size: 1 (type) + 2 (length) + 4 (sessionId) = 7 bytes
00078         constexpr std::size_t PACKET_HEADER_SIZE = 7;
00079         buffer_.reserve(MAX_PAYLOAD + PACKET_HEADER_SIZE);
00080     }
00081
00082     ///
00083     /// @brief Constructor with initial capacity
00084     /// @param capacity Initial buffer capacity
00085     ///
00086     explicit Serializer(std::size_t capacity) : writePos_(0), readPos_(0) { buffer_.reserve(capacity); }
00087
00088     ///
00089     /// @brief Constructor from existing data
00090     /// @param data Existing data to deserialize
00091     ///
00092     explicit Serializer(const std::vector<std::uint8_t> &data)
00093     : buffer_(data), writePos_(data.size()), readPos_(0)
00094     {
00095     }
00096
00097     ///
00098     /// @brief Reset the serializer for reuse
00099     ///
00100     void reset()
00101     {
00102         buffer_.clear();
00103         writePos_ = 0;
00104         readPos_ = 0;
00105     }
00106
00107     ///
00108     /// @brief Get the serialized data
00109     /// @return Vector containing the serialized bytes
00110     ///
00111     const std::vector<std::uint8_t> &getData() const { return buffer_; }
00112
00113     ///
00114     /// @brief Get the current size of serialized data
00115     /// @return Size in bytes
00116     ///
00117     std::size_t getSize() const { return writePos_; }
00118
00119     ///
00120     /// @brief Write raw bytes
00121     /// @param data Pointer to data
00122     /// @param size Number of bytes to write
00123     ///
00124     void writeBytes(const void *data, std::size_t size)
00125     {
00126         // PacketHeader serialized size: 1 (type) + 2 (length) + 4 (sessionId) = 7 bytes
00127         constexpr std::size_t PACKET_HEADER_SIZE = 7;
00128         if (writePos_ + size > MAX_PAYLOAD + PACKET_HEADER_SIZE)
00129         {
00130             throw std::runtime_error("Serializer buffer overflow");

```

```

00131     }
00132
00133     buffer_.resize(writePos_ + size);
00134     std::memcpy(buffer_.data() + writePos_, data, size);
00135     writePos_ += size;
00136 }
00137
00138 ///
00139 /// @brief Write a single byte
00140 /// @param value Byte value to write
00141 ///
00142 void writeByte(std::uint8_t value) { writeBytes(&value, sizeof(value)); }
00143
00144 ///
00145 /// @brief Write a 16-bit integer (network byte order)
00146 /// @param value 16-bit integer to write
00147 ///
00148 void writeUInt16(std::uint16_t value)
00149 {
00150     std::uint16_t networkValue = EndianUtils::hostToNetwork16(value);
00151     writeBytes(&networkValue, sizeof(networkValue));
00152 }
00153
00154 ///
00155 /// @brief Write a 32-bit integer (network byte order)
00156 /// @param value 32-bit integer to write
00157 ///
00158 void writeUInt32(std::uint32_t value)
00159 {
00160     std::uint32_t networkValue = EndianUtils::hostToNetwork32(value);
00161     writeBytes(&networkValue, sizeof(networkValue));
00162 }
00163
00164 ///
00165 /// @brief Write a float (network byte order)
00166 /// @param value Float to write
00167 ///
00168 void writeFloat(float value)
00169 {
00170     float networkValue = EndianUtils::hostToNetworkFloat(value);
00171     writeBytes(&networkValue, sizeof(networkValue));
00172 }
00173
00174 ///
00175 /// @brief Write a string with length prefix
00176 /// @param str String to write
00177 /// @param maxLength Maximum allowed length
00178 ///
00179 void writeString(const std::string &str, std::size_t maxLength)
00180 {
00181     if (str.length() > maxLength)
00182     {
00183         throw std::runtime_error("String too long for serialization");
00184     }
00185
00186     writeByte(static_cast<std::uint8_t>(str.length()));
00187     if (!str.empty())
00188     {
00189         writeBytes(str.data(), str.length());
00190     }
00191 }
00192
00193 ///
00194 /// @brief Read raw bytes
00195 /// @param data Pointer to destination buffer
00196 /// @param size Number of bytes to read
00197 ///
00198 void readBytes(void *data, std::size_t size)
00199 {
00200     if (readPos_ + size > buffer_.size())
00201     {
00202         throw std::runtime_error("Serializer buffer underflow");
00203     }
00204
00205     std::memcpy(data, buffer_.data() + readPos_, size);
00206     readPos_ += size;
00207 }
00208
00209 ///
00210 /// @brief Read a single byte
00211 /// @return Byte value
00212 ///
00213 std::uint8_t readByte()
00214 {
00215     std::uint8_t value;
00216     readBytes(&value, sizeof(value));
00217     return value;

```

```

00218     }
00219
00220     ///
00221     /// @brief Read a 16-bit integer (network byte order)
00222     /// @return 16-bit integer in host byte order
00223     ///
00224     std::uint16_t readUInt16()
00225     {
00226         std::uint16_t networkValue;
00227         readBytes(&networkValue, sizeof(networkValue));
00228         return EndianUtils::networkToHost16(networkValue);
00229     }
00230
00231     ///
00232     /// @brief Read a 32-bit integer (network byte order)
00233     /// @return 32-bit integer in host byte order
00234     ///
00235     std::uint32_t readUInt32()
00236     {
00237         std::uint32_t networkValue;
00238         readBytes(&networkValue, sizeof(networkValue));
00239         return EndianUtils::networkToHost32(networkValue);
00240     }
00241
00242     ///
00243     /// @brief Read a float (network byte order)
00244     /// @return Float in host byte order
00245     ///
00246     float readFloat()
00247     {
00248         float networkValue;
00249         readBytes(&networkValue, sizeof(networkValue));
00250         return EndianUtils::networkToHostFloat(networkValue);
00251     }
00252
00253     ///
00254     /// @brief Read a string with length prefix
00255     /// @param maxLength Maximum expected length
00256     /// @return The read string
00257     ///
00258     std::string readString(std::size_t maxLength)
00259     {
00260         std::uint8_t length = readByte();
00261         if (length > maxLength)
00262         {
00263             throw std::runtime_error("String length exceeds maximum");
00264         }
00265
00266         if (length == 0)
00267         {
00268             return std::string();
00269         }
00270
00271         std::string result(length, '\0');
00272         readBytes(result.data(), length);
00273         return result;
00274     }
00275
00276     ///
00277     /// @brief Serialize packet header
00278     /// @param header Header to serialize
00279     ///
00280     void serializeHeader(const PacketHeader &header)
00281     {
00282         writeByte(header.type);
00283         writeUInt16(header.length);
00284         writeUInt32(header.sessionId);
00285     }
00286
00287     ///
00288     /// @brief Deserialize packet header
00289     /// @return Deserialized header
00290     ///
00291     PacketHeader deserializeHeader()
00292     {
00293         PacketHeader header;
00294         header.type = readByte();
00295         header.length = readUInt16();
00296         header.sessionId = readUInt32();
00297         return header;
00298     }
00299
00300     ///
00301     /// @brief Serialize CONNECT packet
00302     /// @param packet CONNECT packet to serialize
00303     ///
00304     void serializeConnect(const PacketConnect &packet)

```



```

00305     {
00306         writeByte(packet.nameLen);
00307         writeBytes(packet.playerName.data(), packet.playerName.size());
00308         writeUInt32(packet.clientCaps);
00309     }
00310
00311     ///
00312     /// @brief Deserialize CONNECT packet
00313     /// @return Deserialized CONNECT packet
00314     ///
00315     PacketConnect deserializeConnect()
00316     {
00317         PacketConnect packet;
00318         packet.nameLen = readByte();
00319         readBytes(packet.playerName.data(), packet.playerName.size());
00320         packet.clientCaps = readUInt32();
00321         return packet;
00322     }
00323
00324     ///
00325     /// @brief Serialize CONNECT_ACCEPT packet
00326     /// @param packet CONNECT_ACCEPT packet to serialize
00327     ///
00328     void serializeConnectAccept(const PacketConnectAccept &packet)
00329     {
00330         writeUInt32(packet.sessionId);
00331         writeUInt16(packet.tickRateHz);
00332         writeUInt16(packet.mtuPayloadBytes);
00333         writeUInt32(packet.serverCaps);
00334     }
00335
00336     ///
00337     /// @brief Deserialize CONNECT_ACCEPT packet
00338     /// @return Deserialized CONNECT_ACCEPT packet
00339     ///
00340     PacketConnectAccept deserializeConnectAccept()
00341     {
00342         PacketConnectAccept packet;
00343         packet.sessionId = readUInt32();
00344         packet.tickRateHz = readUInt16();
00345         packet.mtuPayloadBytes = readUInt16();
00346         packet.serverCaps = readUInt32();
00347         return packet;
00348     }
00349
00350     ///
00351     /// @brief Serialize DISCONNECT packet
00352     /// @param packet DISCONNECT packet to serialize
00353     ///
00354     void serializeDisconnect(const PacketDisconnect &packet) { writeUInt16(packet.reasonCode); }
00355
00356     ///
00357     /// @brief Deserialize DISCONNECT packet
00358     /// @return Deserialized DISCONNECT packet
00359     ///
00360     PacketDisconnect deserializeDisconnect()
00361     {
00362         PacketDisconnect packet;
00363         packet.reasonCode = readUInt16();
00364         return packet;
00365     }
00366
00367     ///
00368     /// @brief Serialize EntityState
00369     /// @param entity Entity state to serialize
00370     ///
00371     void serializeEntityState(const EntityState &entity)
00372     {
00373         writeUInt32(entity.id);
00374         writeUInt16(entity.type);
00375         writeByte(entity.subtype);
00376         writeFloat(entity.x);
00377         writeFloat(entity.y);
00378         writeFloat(entity.vx);
00379         writeFloat(entity.vy);
00380         writeByte(entity.healthPercent);
00381         writeByte(entity.stateFlags);
00382     }
00383
00384     ///
00385     /// @brief Deserialize EntityState
00386     /// @return Deserialized entity state
00387     ///
00388     EntityState deserializeEntityState()
00389     {
00390         EntityState entity;
00391         entity.id = readUInt32();

```

```

00392     entity.type = readUInt16();
00393     entity.subtype = readByte();
00394     entity.x = readFloat();
00395     entity.y = readFloat();
00396     entity.vx = readFloat();
00397     entity.vy = readFloat();
00398     entity.healthPercent = readByte();
00399     entity.stateFlags = readByte();
00400     return entity;
00401 }
00402
00403 ///
00404 /// @brief Serialize WORLD_STATE packet
00405 /// @param packet WORLD_STATE packet to serialize
00406 ///
00407 void serializeWorldState(const PacketWorldState &packet)
00408 {
00409     writeUInt32(packet.serverTick);
00410     writeUInt16(packet.entityCount);
00411
00412     for (const auto &entity : packet.entities)
00413     {
00414         serializeEntityState(entity);
00415     }
00416 }
00417
00418 ///
00419 /// @brief Deserialize WORLD_STATE packet
00420 /// @return Deserialized WORLD_STATE packet
00421 ///
00422 PacketWorldState deserializeWorldState()
00423 {
00424     PacketWorldState packet;
00425     packet.serverTick = readUInt32();
00426     packet.entityCount = readUInt16();
00427
00428     packet.entities.reserve(packet.entityCount);
00429     for (std::uint16_t i = 0; i < packet.entityCount; ++i)
00430     {
00431         packet.entities.push_back(deserializeEntityState());
00432     }
00433
00434     return packet;
00435 }
00436
00437 ///
00438 /// @brief Serialize PING/PONG packet
00439 /// @param packet PING/PONG packet to serialize
00440 ///
00441 void serializePingPong(const PacketPingPong &packet)
00442 {
00443     writeUInt32(packet.nonce);
00444     writeUInt32(packet.sendTimeMs);
00445 }
00446
00447 ///
00448 /// @brief Deserialize PING/PONG packet
00449 /// @return Deserialized PING/PONG packet
00450 ///
00451 PacketPingPong deserializePingPong()
00452 {
00453     PacketPingPong packet;
00454     packet.nonce = readUInt32();
00455     packet.sendTimeMs = readUInt32();
00456     return packet;
00457 }
00458
00459 ///
00460 /// @brief Serialize ERROR packet
00461 /// @param packet ERROR packet to serialize
00462 ///
00463 void serializeError(const PacketError &packet)
00464 {
00465     writeUInt16(packet.errorCode);
00466     writeUInt16(packet.msgLen);
00467     if (!packet.description.empty())
00468     {
00469         writeBytes(packet.description.data(), packet.description.length());
00470     }
00471 }
00472
00473 ///
00474 /// @brief Deserialize ERROR packet
00475 /// @return Deserialized ERROR packet
00476 ///
00477 PacketError deserializeError()
00478 {

```

```

00479     PacketError packet;
00480     packet.errorCode = readUInt16();
00481     packet.msgLen = readUInt16();
00482
00483     if (packet.msgLen > 0)
00484     {
00485         packet.description.resize(packet.msgLen);
00486         readBytes(packet.description.data(), packet.msgLen);
00487     }
00488
00489     return packet;
00490 }
00491
00492 ///
00493 /// @brief Serialize EventRecord for ENTITY_EVENT packets
00494 /// @param event Event record to serialize
00495 ///
00496 void serializeEventRecord(const EventRecord &event)
00497 {
00498     writeByte(static_cast<std::uint8_t>(event.type));
00499     writeUInt32(event.entityId);
00500     writeUInt16(static_cast<std::uint16_t>(event.data.size()));
00501
00502     if (!event.data.empty())
00503     {
00504         writeBytes(event.data.data(), event.data.size());
00505     }
00506 }
00507
00508 ///
00509 /// @brief Deserialize EventRecord from ENTITY_EVENT packets
00510 /// @return Deserialized event record
00511 ///
00512 EventRecord deserializeEventRecord()
00513 {
00514     EventRecord event;
00515     event.type = static_cast<EventType>(readByte());
00516     event.entityId = readUInt32();
00517
00518     std::uint16_t dataLength = readUInt16();
00519     if (dataLength > 0)
00520     {
00521         event.data.resize(dataLength);
00522         readBytes(event.data.data(), dataLength);
00523     }
00524
00525     return event;
00526 }
00527
00528 ///
00529 /// @brief Serialize multiple EventRecords for ENTITY_EVENT packet
00530 /// @param events Vector of event records to serialize
00531 ///
00532 void serializeEntityEvents(const std::vector<EventRecord> &events)
00533 {
00534     for (const auto &event : events)
00535     {
00536         serializeEventRecord(event);
00537     }
00538 }
00539
00540 ///
00541 /// @brief Deserialize multiple EventRecords from ENTITY_EVENT packet
00542 /// @param payloadSize Total size of the payload to deserialize
00543 /// @return Vector of deserialized event records
00544 ///
00545 std::vector<EventRecord> deserializeEntityEvents(std::size_t payloadSize)
00546 {
00547     std::vector<EventRecord> events;
00548     std::size_t bytesRead = 0;
00549
00550     while (bytesRead < payloadSize && readPos_ < buffer_.size())
00551     {
00552         std::size_t startPos = readPos_;
00553         EventRecord event = deserializeEventRecord();
00554         bytesRead += (readPos_ - startPos);
00555         events.push_back(event);
00556     }
00557
00558     return events;
00559 }
00560
00561 ///
00562 /// @brief Serialize input data for INPUT events
00563 /// @param keys Keyboard input bitmask
00564 /// @param mouseX Mouse X position
00565 /// @param mouseY Mouse Y position

```

```

00566     /// @param mouseButtons Mouse button bitmask
00567     /// @param timestamp Input timestamp
00568     ///
00569 void serializeInputData(std::uint8_t keys, float mouseX, float mouseY, std::uint8_t mouseButtons,
00570                        std::uint32_t timestamp)
00571 {
00572     writeByte(keys);
00573     writeFloat(mouseX);
00574     writeFloat(mouseY);
00575     writeByte(mouseButtons);
00576     writeUInt32(timestamp);
00577 }
00578
00579     ///
00580     /// @brief Deserialize input data from INPUT events
00581     /// @param keys Output keyboard input bitmask
00582     /// @param mouseX Output mouse X position
00583     /// @param mouseY Output mouse Y position
00584     /// @param mouseButtons Output mouse button bitmask
00585     /// @param timestamp Output input timestamp
00586     ///
00587 void deserializeInputData(std::uint8_t &keys, float &mouseX, float &mouseY, std::uint8_t &mouseButtons,
00588                          std::uint32_t &timestamp)
00589 {
00590     keys = readByte();
00591     mouseX = readFloat();
00592     mouseY = readFloat();
00593     mouseButtons = readByte();
00594     timestamp = readUInt32();
00595 }
00596
00597     ///
00598     /// @brief Create an EventRecord for input data
00599     /// @param entityId Player entity ID
00600     /// @param keys Keyboard input
00601     /// @param mouseX Mouse X
00602     /// @param mouseY Mouse Y
00603     /// @param mouseButtons Mouse buttons
00604     /// @param timestamp Timestamp
00605     /// @return EventRecord ready to send
00606     ///
00607 static EventRecord createInputEvent(std::uint32_t entityId, std::uint8_t keys, float mouseX, float mouseY,
00608                                    std::uint8_t mouseButtons, std::uint32_t timestamp)
00609 {
00610     Serializer inputSerializer;
00611     inputSerializer.serializeInputData(keys, mouseX, mouseY, mouseButtons, timestamp);
00612
00613     EventRecord event;
00614     event.type = EventType::INPUT;
00615     event.entityId = entityId;
00616     event.data = inputSerializer.getData();
00617
00618     return event;
00619 }
00620
00621     ///
00622     /// @brief Create an EventRecord for spawn event
00623     /// @param entityId Entity ID to spawn
00624     /// @param entityType Type of entity
00625     /// @param x X position
00626     /// @param y Y position
00627     /// @return EventRecord for spawn
00628     ///
00629 static EventRecord createSpawnEvent(std::uint32_t entityId, EntityType entityType, float x, float y)
00630 {
00631     Serializer spawnSerializer;
00632     spawnSerializer.writeUInt16(static_cast<std::uint16_t>(entityType));
00633     spawnSerializer.writeFloat(x);
00634     spawnSerializer.writeFloat(y);
00635
00636     EventRecord event;
00637     event.type = EventType::SPAWN;
00638     event.entityId = entityId;
00639     event.data = spawnSerializer.getData();
00640
00641     return event;
00642 }
00643
00644     ///
00645     /// @brief Create an EventRecord for damage event
00646     /// @param entityId Entity taking damage
00647     /// @param damage Amount of damage
00648     /// @param sourceId Source entity causing damage
00649     /// @return EventRecord for damage
00650     ///
00651 static EventRecord createDamageEvent(std::uint32_t entityId, std::uint32_t damage,
00652                                     std::uint32_t sourceId = 0)

```

```

00653     {
00654         Serializer damageSerializer;
00655         damageSerializer.writeUInt32(damage);
00656         damageSerializer.writeUInt32(sourceId);
00657
00658         EventRecord event;
00659         event.type = EventType::DAMAGE;
00660         event.entityId = entityId;
00661         event.data = damageSerializer.getData();
00662
00663         return event;
00664     }
00665
00666     ///
00667     /// @brief Create an EventRecord for score event
00668     /// @param entityId Entity gaining score
00669     /// @param score Score amount
00670     /// @return EventRecord for score
00671     ///
00672     static EventRecord createScoreEvent(std::uint32_t entityId, std::uint32_t score)
00673     {
00674         Serializer scoreSerializer;
00675         scoreSerializer.writeUInt32(score);
00676
00677         EventRecord event;
00678         event.type = EventType::SCORE;
00679         event.entityId = entityId;
00680         event.data = scoreSerializer.getData();
00681
00682         return event;
00683     }
00684
00685     ///
00686     /// @brief Create an EventRecord for despawn event
00687     /// @param entityId Entity to despawn
00688     /// @return EventRecord for despawn
00689     ///
00690     EventRecord createDespawnEvent(std::uint32_t entityId)
00691     {
00692         EventRecord event;
00693         event.type = EventType::DESPAWN;
00694         event.entityId = entityId;
00695         return event;
00696     }
00697
00698     ///
00699     /// @brief Serialize LobbyInfo structure
00700     /// @param lobbyInfo The lobby info to serialize
00701     ///
00702     void serializeLobbyInfo(const LobbyInfo &lobbyInfo)
00703     {
00704         writeUInt32(lobbyInfo.lobbyId);
00705         writeBytes(reinterpret_cast<const std::uint8_t *>(lobbyInfo.lobbyName.data()),
00706             lobbyInfo.lobbyName.size());
00707         writeByte(lobbyInfo.currentPlayers);
00708         writeByte(lobbyInfo.maxPlayers);
00709         writeByte(lobbyInfo.gameMode);
00710         writeByte(lobbyInfo.status);
00711         writeUInt32(lobbyInfo.hostSessionId);
00712
00713         // Serialize player names array
00714         for (const auto &playerName : lobbyInfo.playerNames)
00715         {
00716             writeBytes(reinterpret_cast<const std::uint8_t *>(playerName.data()), playerName.size());
00717         }
00718     }
00719
00720     ///
00721     /// @brief Deserialize LobbyInfo structure
00722     /// @return Deserialized lobby info
00723     ///
00724     LobbyInfo deserializeLobbyInfo()
00725     {
00726         LobbyInfo lobbyInfo;
00727         lobbyInfo.lobbyId = readUInt32();
00728         readBytes(reinterpret_cast<std::uint8_t *>(lobbyInfo.lobbyName.data()), lobbyInfo.lobbyName.size());
00729         lobbyInfo.currentPlayers = readByte();
00730         lobbyInfo.maxPlayers = readByte();
00731         lobbyInfo.gameMode = readByte();
00732         lobbyInfo.status = readByte();
00733         lobbyInfo.hostSessionId = readUInt32();
00734
00735         // Deserialize player names array
00736         for (auto &playerName : lobbyInfo.playerNames)
00737         {
00738             readBytes(reinterpret_cast<std::uint8_t *>(playerName.data()), playerName.size());
00739         }

```

```

00740
00741     return lobbyInfo;
00742 }
00743
00744 ///
00745 /// @brief Serialize LOBBY_LIST_RESPONSE packet
00746 /// @param packet The packet to serialize
00747 ///
00748 void serializeLobbyListResponse(const PacketLobbyListResponse &packet)
00749 {
00750     writeUInt16(packet.lobbyCount);
00751     for (const auto &lobby : packet.lobbies)
00752     {
00753         serializeLobbyInfo(lobby);
00754     }
00755 }
00756
00757 ///
00758 /// @brief Deserialize LOBBY_LIST_RESPONSE packet
00759 /// @return Deserialized packet
00760 ///
00761 PacketLobbyListResponse deserializeLobbyListResponse()
00762 {
00763     PacketLobbyListResponse packet;
00764     packet.lobbyCount = readUInt16();
00765     packet.lobbies.reserve(packet.lobbyCount);
00766
00767     for (std::uint16_t i = 0; i < packet.lobbyCount; ++i)
00768     {
00769         packet.lobbies.push_back(deserializeLobbyInfo());
00770     }
00771
00772     return packet;
00773 }
00774
00775 ///
00776 /// @brief Serialize LOBBY_CREATE packet
00777 /// @param packet The packet to serialize
00778 ///
00779 void serializeLobbyCreate(const PacketLobbyCreate &packet)
00780 {
00781     writeByte(packet.nameLen);
00782     writeBytes(reinterpret_cast<const std::uint8_t *>(packet.lobbyName.data()), packet.lobbyName.size());
00783     writeByte(packet.maxPlayers);
00784     writeByte(packet.gameMode);
00785 }
00786
00787 ///
00788 /// @brief Deserialize LOBBY_CREATE packet
00789 /// @return Deserialized packet
00790 ///
00791 PacketLobbyCreate deserializeLobbyCreate()
00792 {
00793     PacketLobbyCreate packet;
00794     packet.nameLen = readByte();
00795     readBytes(reinterpret_cast<std::uint8_t *>(packet.lobbyName.data()), packet.lobbyName.size());
00796     packet.maxPlayers = readByte();
00797     packet.gameMode = readByte();
00798     return packet;
00799 }
00800
00801 ///
00802 /// @brief Serialize LOBBY_CREATE_RESPONSE packet
00803 /// @param packet The packet to serialize
00804 ///
00805 void serializeLobbyCreateResponse(const PacketLobbyCreateResponse &packet)
00806 {
00807     writeUInt32(packet.lobbyId);
00808     writeByte(packet.success);
00809     writeUInt16(packet.errorCode);
00810 }
00811
00812 ///
00813 /// @brief Deserialize LOBBY_CREATE_RESPONSE packet
00814 /// @return Deserialized packet
00815 ///
00816 PacketLobbyCreateResponse deserializeLobbyCreateResponse()
00817 {
00818     PacketLobbyCreateResponse packet;
00819     packet.lobbyId = readUInt32();
00820     packet.success = readByte();
00821     packet.errorCode = readUInt16();
00822     return packet;
00823 }
00824
00825 ///
00826 /// @brief Serialize LOBBY_JOIN packet

```

```

00827     /// @param packet The packet to serialize
00828     ///
00829 void serializeLobbyJoin(const PacketLobbyJoin &packet) { writeUInt32(packet.lobbyId); }
00830
00831     ///
00832     /// @brief Deserialize LOBBY_JOIN packet
00833     /// @return Deserialized packet
00834     ///
00835 PacketLobbyJoin deserializeLobbyJoin()
00836 {
00837     PacketLobbyJoin packet;
00838     packet.lobbyId = readUInt32();
00839     return packet;
00840 }
00841
00842     ///
00843     /// @brief Serialize LOBBY_JOIN_RESPONSE packet
00844     /// @param packet The packet to serialize
00845     ///
00846 void serializeLobbyJoinResponse(const PacketLobbyJoinResponse &packet)
00847 {
00848     writeUInt32(packet.lobbyId);
00849     writeByte(packet.success);
00850     writeUInt16(packet.errorCode);
00851     if (packet.success == 1)
00852     {
00853         serializeLobbyInfo(packet.lobbyInfo);
00854     }
00855 }
00856
00857     ///
00858     /// @brief Deserialize LOBBY_JOIN_RESPONSE packet
00859     /// @return Deserialized packet
00860     ///
00861 PacketLobbyJoinResponse deserializeLobbyJoinResponse()
00862 {
00863     PacketLobbyJoinResponse packet;
00864     packet.lobbyId = readUInt32();
00865     packet.success = readByte();
00866     packet.errorCode = readUInt16();
00867     if (packet.success == 1)
00868     {
00869         packet.lobbyInfo = deserializeLobbyInfo();
00870     }
00871     return packet;
00872 }
00873
00874     ///
00875     /// @brief Serialize LOBBY_UPDATE packet
00876     /// @param packet The packet to serialize
00877     ///
00878 void serializeLobbyUpdate(const PacketLobbyUpdate &packet) { serializeLobbyInfo(packet.lobbyInfo); }
00879
00880     ///
00881     /// @brief Deserialize LOBBY_UPDATE packet
00882     /// @return Deserialized packet
00883     ///
00884 PacketLobbyUpdate deserializeLobbyUpdate()
00885 {
00886     PacketLobbyUpdate packet;
00887     packet.lobbyInfo = deserializeLobbyInfo();
00888     return packet;
00889 }
00890
00891     ///
00892     /// @brief Serialize GAME_START packet
00893     /// @param packet The packet to serialize
00894     ///
00895 void serializeGameStart(const PacketGameStart &packet) { writeUInt32(packet.lobbyId); }
00896
00897     ///
00898     /// @brief Deserialize GAME_START packet
00899     /// @return Deserialized packet
00900     ///
00901 PacketGameStart deserializeGameStart()
00902 {
00903     PacketGameStart packet;
00904     packet.lobbyId = readUInt32();
00905     return packet;
00906 }
00907
00908     ///
00909     /// @brief Serialize START_GAME_REQUEST packet
00910     /// @param packet The packet to serialize
00911     ///
00912 void serializeStartGameRequest(const PacketStartGameRequest &packet) { writeUInt32(packet.lobbyId); }
00913

```


Classes

- struct [utl::cli::AppConfig](#)

Namespaces

- namespace [utl](#)
- namespace [utl::cli](#)

19.95.1 Detailed Description

This file contains the app config structure for client.

Definition in file [AppConfigClient.hpp](#).

19.96 AppConfigClient.hpp

[Go to the documentation of this file.](#)

```

00001 ///
00002 /// @file AppConfigClient.hpp
00003 /// @brief This file contains the app config structure for client
00004 /// @namespace utl
00005 ///
00006
00007 #pragma once
00008
00009 #include "Utils/Common.hpp"
00010
00011 namespace utl
00012 {
00013     namespace cli
00014     {
00015         struct AppConfig
00016         {
00017             unsigned int width = Config::Window::DEFAULT_WINDOW_WIDTH;
00018             unsigned int height = Config::Window::DEFAULT_WINDOW_HEIGHT;
00019             unsigned int frameLimit = Config::Window::DEFAULT_WINDOW_FRAME_LIMIT;
00020             bool fullscreen = Config::Window::DEFAULT_WINDOW_FULLSCREEN;
00021             std::string host = Config::Network::DEFAULT_NETWORK_HOST;
00022             unsigned int port = Config::Network::DEFAULT_NETWORK_PORT;
00023             std::string player_name = Config::Player::DEFAULT_PLAYER_NAME;
00024             float audioVolume = Config::Audio::DEFAULT_AUDIO_VOLUME;
00025             int videoQuality = Config::Game::DEFAULT_VIDEO_QUALITY;
00026             int controlScheme = Config::Game::DEFAULT_CONTROL_SCHEME;
00027             int skinIndex = Config::Game::DEFAULT_SKIN_INDEX;
00028         };
00029     } // namespace cli
00030 } // namespace utl

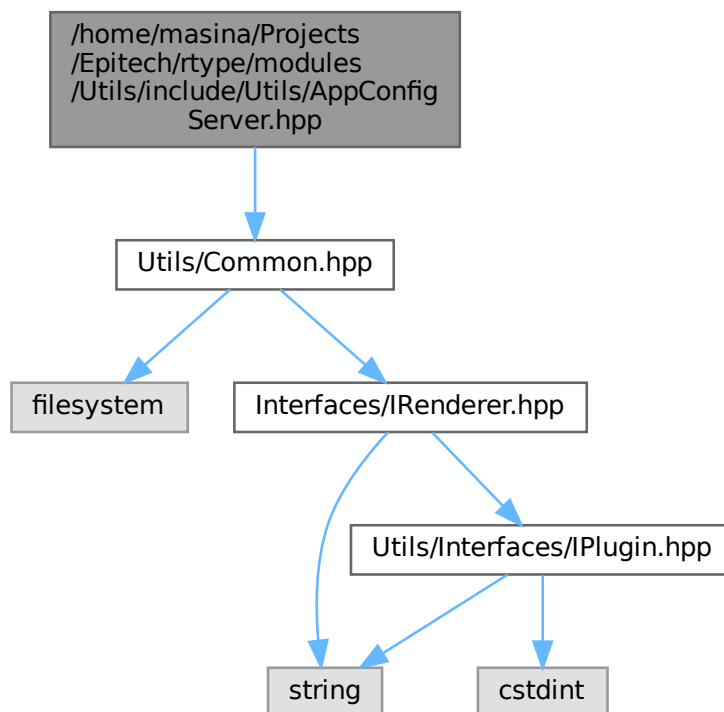
```

19.97 /home/masina/Projects/Epitech/rtype/modules/Utils/include/↵ Utils/AppConfigServer.hpp File Reference

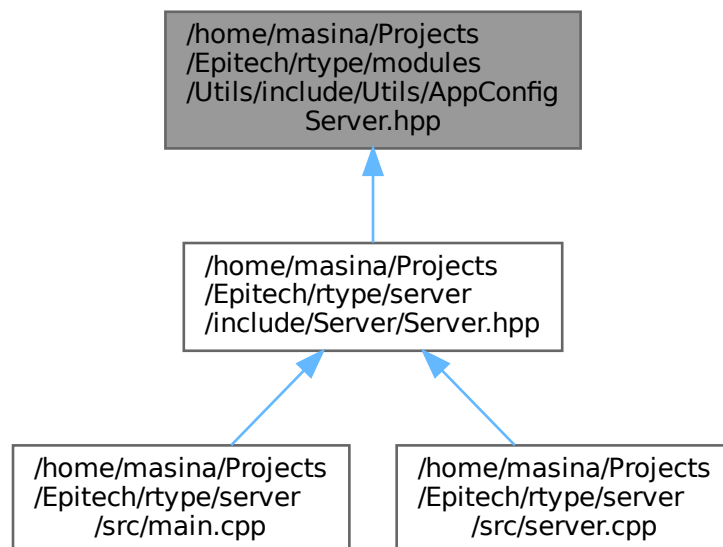
This file contains the app config structure for server.

```
#include "Utils/Common.hpp"
```

Include dependency graph for AppConfigServer.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- struct `utl::srv::AppConfig`

Namespaces

- namespace `utl`
- namespace `utl::srv`

19.97.1 Detailed Description

This file contains the app config structure for server.

Definition in file `AppConfigServer.hpp`.

19.98 AppConfigServer.hpp

[Go to the documentation of this file.](#)

```

00001 ///  

00002 ///  

00003 ///  

00004 ///  

00005 ///  

00006 ///  

00007 #pragma once  

00008 ///  

00009 #include "Utils/Common.hpp"  

00010 ///  

00011 namespace utl  

00012 {  

00013     namespace srv  

00014     {  

00015         struct AppConfig  

00016         {  

00017             std::string host;  

00018             uint16_t port;  

00019         };  

00020     } // namespace srv  

00021 } // namespace utl

```

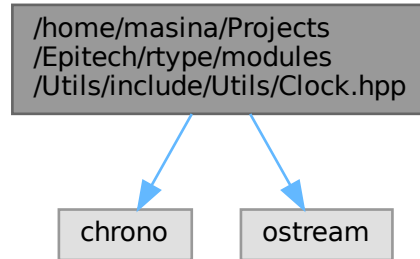
19.99 /home/masina/Projects/Epitech/rtype/modules/Utils/include/Utils/Clock.hpp File Reference

This file contains the Clock class.

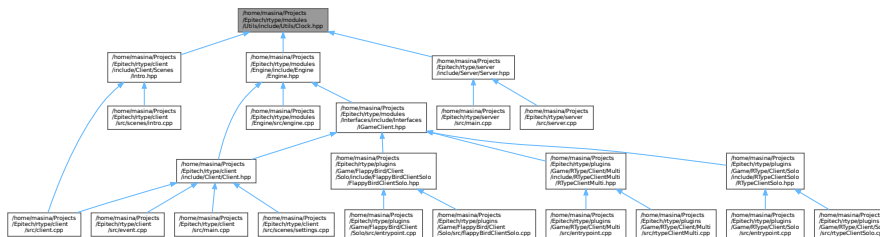
```
#include <chrono>
```

```
#include <ostream>
```

Include dependency graph for Clock.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class `utl::Clock`
Class for clock.

Namespaces

- namespace `utl`

19.99.1 Detailed Description

This file contains the Clock class.

Definition in file [Clock.hpp](#).

19.100 Clock.hpp

[Go to the documentation of this file.](#)

```

00001 ///
00002 /// @file Clock.hpp
00003 /// @brief This file contains the Clock class
00004 /// @namespace utl
00005 ///

```

```

00006
00007 #pragma once
00008
00009 #include <chrono>
00010 #include <ostream>
00011
00012 namespace utl
00013 {
00014
00015     ///
00016     /// @class Clock
00017     /// @brief Class for clock
00018     /// @namespace utl
00019     ///
00020     class Clock
00021     {
00022
00023     public:
00024         using TimePoint = std::chrono::time_point<std::chrono::high_resolution_clock>;
00025
00026         explicit Clock(const bool startNow = true) : m_start{startNow ? now() : TimePoint()}, m_pausedDuration{0} {}
00027         ~Clock() = default;
00028
00029         Clock(const Clock &) = delete;
00030         Clock &operator=(const Clock &) = delete;
00031         Clock(Clock &&) = delete;
00032         Clock &operator=(Clock &&) = delete;
00033
00034         friend std::ostream &operator<<(std::ostream &os, const Clock &clock)
00035         {
00036             os << "Elapsed time: " << clock.getDeltaSeconds() << " seconds";
00037             return os;
00038         }
00039
00040         static TimePoint now() { return std::chrono::high_resolution_clock::now(); }
00041         void restart()
00042         {
00043             m_start = now();
00044             m_pausedDuration = Duration(0);
00045             m_isPaused = false;
00046         }
00047         void pause()
00048         {
00049             if (!m_isPaused)
00050             {
00051                 m_pausedTime = now();
00052                 m_isPaused = true;
00053             }
00054         }
00055         void resume()
00056         {
00057             if (m_isPaused)
00058             {
00059                 m_pausedDuration += now() - m_pausedTime;
00060                 m_isPaused = false;
00061             }
00062         }
00063         [[nodiscard]] float getDeltaSeconds() const
00064         {
00065             if (m_isPaused)
00066             {
00067                 return std::chrono::duration<float>(m_pausedTime - m_start - m_pausedDuration).count();
00068             }
00069             return std::chrono::duration<float>(now() - m_start - m_pausedDuration).count();
00070         }
00071
00072         template <typename Duration = std::chrono::seconds> [[nodiscard]] auto getElapsed() const
00073         {
00074             return std::chrono::duration_cast<Duration>(now() - m_start - m_pausedDuration);
00075         }
00076
00077     private:
00078         using Duration = std::chrono::high_resolution_clock::duration;
00079
00080         TimePoint m_start;
00081         TimePoint m_pausedTime;
00082         Duration m_pausedDuration;
00083         bool m_isPaused{false};
00084     }; // class Clock
00085 } // namespace utl

```

19.101 /home/masina/Projects/Epitech/rtype/modules/Utl/include/Utl/Event.hpp File Reference

Event structures and types for event-driven communication.

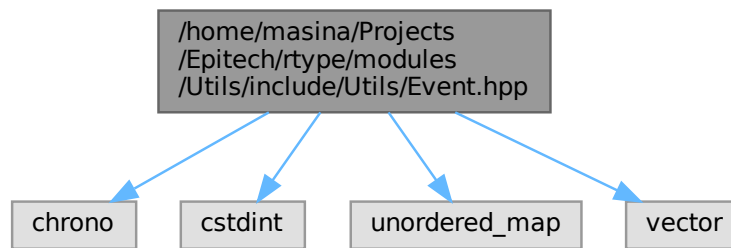
```
#include <chrono>
```

```
#include <cstdint>
```

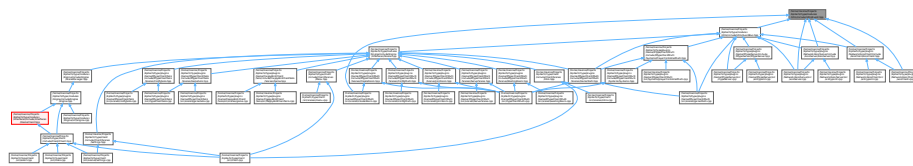
```
#include <unordered_map>
```

```
#include <vector>
```

Include dependency graph for Event.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- struct [utl::EventConnect](#)
- class [utl::Event](#)
[Event](#) structure for inter-component communication.
- class [utl::EventStats](#)
[Event](#) statistics structure.

Namespaces

- namespace [utl](#)

Enumerations

- enum class [utl::EventType](#) : `std::uint32_t` {
[utl::WORLD_STATE_RECEIVED](#) = 0x1000 , [utl::ENTITY_EVENTS_RECEIVED](#) = 0x1001 ,
[utl::PLAYER_CONNECTED](#) = 0x1002 , [utl::PLAYER_DISCONNECTED](#) = 0x1003 ,
[utl::PLAYER_INPUT_RECEIVED](#) = 0x1010 , [utl::ENTITY_EVENT_RECEIVED](#) = 0x1011 ,
[utl::CONNECTION_ACCEPTED](#) = 0x1020 , [utl::CONNECTION_REJECTED](#) = 0x1021 ,
[utl::SERVER_DISCONNECTED](#) = 0x1022 , [utl::SEND_PLAYER_INPUT](#) = 0x2000 ,
[utl::SEND_ENTITY_EVENT](#) = 0x2001 , [utl::SEND_ENTITY_EVENTS](#) = 0x2002 ,
[utl::BROADCAST_WORLD_STATE](#) = 0x2003 , [utl::BROADCAST_TO_CLIENTS](#) = 0x2004 ,
[utl::SEND_TO_CLIENT](#) = 0x2010 , [utl::DISCONNECT_CLIENT](#) = 0x2011 ,

```

utl::REQUEST_CONNECT = 0x2020 , utl::REQUEST_DISCONNECT = 0x2021 , utl::SERVER_START
= 0x3000 , utl::SERVER_STOP = 0x3001 ,
utl::SERVER_STATUS_UPDATE = 0x3002 , utl::LOBBY_LIST_REQUEST = 0x4000 ,
utl::LOBBY_LIST_RESPONSE = 0x4001 , utl::LOBBY_CREATE = 0x4010 ,
utl::LOBBY_CREATE_RESPONSE = 0x4011 , utl::LOBBY_JOIN = 0x4020 , utl::LOBBY_JOIN_RESPONSE
= 0x4021 , utl::LOBBY_LEAVE = 0x4030 ,
utl::LOBBY_UPDATE = 0x4040 , utl::GAME_START = 0x4050 }

```

Event types for inter-component communication.

- enum class `utl::EventPriority` : `std::uint8_t` { `utl::LOW` = 0 , `utl::NORMAL` = 1 , `utl::HIGH` = 2 , `utl::CRITICAL` = 3 }

Event priority levels.

Variables

- static constexpr `std::uint32_t` `utl::NETWORK_CLIENT` = 1
- static constexpr `std::uint32_t` `utl::NETWORK_SERVER` = 2
- static constexpr `std::uint32_t` `utl::GAME_LOGIC` = 3
- static constexpr `std::uint32_t` `utl::RENDERING_ENGINE` = 4

19.101.1 Detailed Description

Event structures and types for event-driven communication.

Definition in file [Event.hpp](#).

19.102 Event.hpp

[Go to the documentation of this file.](#)

```

00001 ///
00002 /// @file Event.hpp
00003 /// @brief Event structures and types for event-driven communication
00004 /// @namespace utl
00005 ///
00006
00007 #pragma once
00008
00009 #include <chrono>
00010 #include <cstdint>
00011 #include <unordered_map>
00012 #include <vector>
00013
00014 namespace utl
00015 {
00016
00017     static constexpr std::uint32_t NETWORK_CLIENT = 1;
00018     static constexpr std::uint32_t NETWORK_SERVER = 2;
00019     static constexpr std::uint32_t GAME_LOGIC = 3;
00020     static constexpr std::uint32_t RENDERING_ENGINE = 4;
00021
00022     ///
00023     /// @brief Event types for inter-component communication
00024     ///
00025     enum class EventType : std::uint32_t
00026     {
00027         // Network → Game events (0x1000 - 0x1FFF)
00028         WORLD_STATE_RECEIVED = 0x1000,
00029         ENTITY_EVENTS_RECEIVED = 0x1001,
00030         PLAYER_CONNECTED = 0x1002,
00031         PLAYER_DISCONNECTED = 0x1003,
00032         PLAYER_INPUT_RECEIVED = 0x1010,
00033         ENTITY_EVENT_RECEIVED = 0x1011,
00034         CONNECTION_ACCEPTED = 0x1020,
00035         CONNECTION_REJECTED = 0x1021,
00036         SERVER_DISCONNECTED = 0x1022,
00037
00038         // Game → Network events (0x2000 - 0x2FFF)
00039         SEND_PLAYER_INPUT = 0x2000,
00040         SEND_ENTITY_EVENT = 0x2001,
00041         SEND_ENTITY_EVENTS = 0x2002,
00042         BROADCAST_WORLD_STATE = 0x2003,
00043         BROADCAST_TO_CLIENTS = 0x2004,
00044         SEND_TO_CLIENT = 0x2010,
00045         DISCONNECT_CLIENT = 0x2011,
00046         REQUEST_CONNECT = 0x2020,

```

```

00047     REQUEST_DISCONNECT = 0x2021,
00048
00049     // Server Management events (0x3000 - 0x3FFF)
00050     SERVER_START = 0x3000,
00051     SERVER_STOP = 0x3001,
00052     SERVER_STATUS_UPDATE = 0x3002,
00053
00054     // Lobby Management events (0x4000 - 0x4FFF)
00055     LOBBY_LIST_REQUEST = 0x4000,
00056     LOBBY_LIST_RESPONSE = 0x4001,
00057     LOBBY_CREATE = 0x4010,
00058     LOBBY_CREATE_RESPONSE = 0x4011,
00059     LOBBY_JOIN = 0x4020,
00060     LOBBY_JOIN_RESPONSE = 0x4021,
00061     LOBBY_LEAVE = 0x4030,
00062     LOBBY_UPDATE = 0x4040,
00063     GAME_START = 0x4050,
00064 };
00065
00066 ///
00067 /// @brief Event priority levels
00068 ///
00069 enum class EventPriority : std::uint8_t
00070 {
00071     LOW = 0,
00072     NORMAL = 1,
00073     HIGH = 2,
00074     CRITICAL = 3
00075 };
00076
00077 struct EventConnect
00078 {
00079     std::string playerName;
00080     std::string serverIP;
00081     std::string serverPort;
00082 };
00083
00084 ///
00085 /// @brief Event structure for inter-component communication
00086 ///
00087 class Event
00088 {
00089 public:
00090     EventType type; //< Type of the event
00091     std::uint32_t sourceId = 0; //< ID of the component that sent the event (0 = system)
00092     std::uint32_t targetId = 0; //< ID of the target component (0 = broadcast)
00093     EventPriority priority = EventPriority::NORMAL; //< Event priority
00094     std::chrono::steady_clock::time_point timestamp; //< Timestamp when event was created
00095     std::vector<std::uint8_t> data; //< Serialized event data
00096
00097     ///
00098     /// @brief Default constructor with current timestamp
00099     ///
00100     Event() : type(), timestamp(std::chrono::steady_clock::now()) {}
00101
00102     ///
00103     /// @brief Constructor with event type
00104     /// @param t Event type
00105     ///
00106     explicit Event(EventType t) : type(t), timestamp(std::chrono::steady_clock::now()) {}
00107
00108     ///
00109     /// @brief Constructor with event type and data
00110     /// @param t Event type
00111     /// @param d Event data
00112     ///
00113     Event(EventType t, const std::vector<std::uint8_t> &d)
00114         : type(t), timestamp(std::chrono::steady_clock::now()), data(d)
00115     {
00116     }
00117
00118     ///
00119     /// @brief Constructor with all parameters
00120     /// @param t Event type
00121     /// @param src Source component ID
00122     /// @param tgt Target component ID (0 for broadcast)
00123     /// @param prio Event priority
00124     ///
00125     Event(EventType t, std::uint32_t src, std::uint32_t tgt = 0, EventPriority prio = EventPriority::NORMAL)
00126         : type(t), sourceId(src), targetId(tgt), priority(prio), timestamp(std::chrono::steady_clock::now())
00127     {
00128     }
00129
00130     ///
00131     /// @brief Get event age in milliseconds
00132     /// @return Age in milliseconds
00133     ///

```



```

00134     std::uint64_t getAge() const
00135     {
00136         auto now = std::chrono::steady_clock::now();
00137         return static_cast<std::uint64_t> (
00138             std::chrono::duration_cast<std::chrono::milliseconds>(now - timestamp).count());
00139     }
00140
00141     ///
00142     /// @brief Check if event has expired
00143     /// @param maxAgeMs Maximum age in milliseconds
00144     /// @return True if event has expired
00145     ///
00146     bool hasExpired(std::uint64_t maxAgeMs = 5000) const { return getAge() > maxAgeMs; }
00147
00148     ///
00149     /// @brief Check if event is targeted to a specific component
00150     /// @return True if event has a specific target
00151     ///
00152     bool isTargeted() const { return targetId != 0; }
00153
00154     ///
00155     /// @brief Check if event is a broadcast
00156     /// @return True if event is broadcast to all components
00157     ///
00158     bool isBroadcast() const { return targetId == 0; }
00159 };
00160
00161 ///
00162 /// @brief Event statistics structure
00163 ///
00164 class EventStats
00165 {
00166 public:
00167     std::uint64_t totalEventsPublished = 0;           ///< Total events published
00168     std::uint64_t totalEventsConsumed = 0;           ///< Total events consumed
00169     std::uint64_t totalEventsExpired = 0;            ///< Total events that expired
00170     std::uint64_t currentQueueSize = 0;              ///< Current queue size
00171     std::uint64_t maxQueueSize = 0;                  ///< Maximum queue size reached
00172     std::chrono::steady_clock::time_point startTime; ///< Statistics start time
00173     std::unordered_map<EventType, std::uint64_t> eventTypeCount; ///< Count per event type
00174     std::unordered_map<EventPriority, std::uint64_t> priorityCount; ///< Count per priority
00175     std::unordered_map<std::uint32_t, std::uint64_t> sourceCount; ///< Count per source component
00176     std::unordered_map<std::uint32_t, std::uint64_t> targetCount; ///< Count per target component
00177
00178     ///
00179     /// @brief Reset all statistics
00180     ///
00181     void reset()
00182     {
00183         totalEventsPublished = 0;
00184         totalEventsConsumed = 0;
00185         totalEventsExpired = 0;
00186         currentQueueSize = 0;
00187         maxQueueSize = 0;
00188         startTime = std::chrono::steady_clock::now();
00189         eventTypeCount.clear();
00190         priorityCount.clear();
00191         sourceCount.clear();
00192         targetCount.clear();
00193     }
00194
00195     ///
00196     /// @brief Get events per second rate
00197     /// @return Events per second
00198     ///
00199     double getEventsPerSecond() const
00200     {
00201         auto elapsed =
00202             std::chrono::duration_cast<std::chrono::seconds>(std::chrono::steady_clock::now() - startTime);
00203         if (elapsed.count() == 0)
00204             return 0.0;
00205         return static_cast<double>(totalEventsPublished) / elapsed.count();
00206     }
00207 };
00208
00209 } // namespace utl

```

19.103 /home/masina/Projects/Epitech/rtype/modules/Utils/include/Utils/EventBus.hpp File Reference

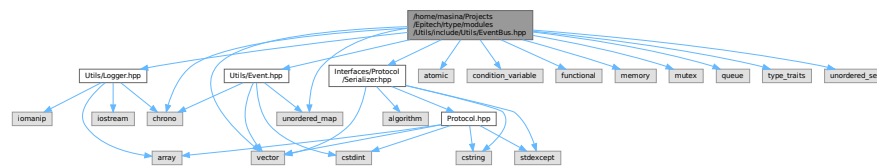
Thread-safe event bus implementation for inter-component communication.

```

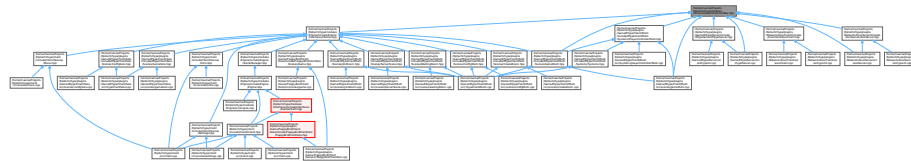
#include "Interfaces/Protocol/Serializer.hpp"
#include "Utils/Event.hpp"
#include "Utils/Logger.hpp"
#include <atomic>
#include <chrono>
#include <condition_variable>
#include <functional>
#include <memory>
#include <mutex>
#include <queue>
#include <type_traits>
#include <unordered_map>
#include <unordered_set>
#include <vector>

```

Include dependency graph for EventBus.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [utl::EventBus](#)
Thread-safe event bus for decoupled component communication.
- struct [utl::EventBus::EventQueue](#)
Internal event queue structure.
- struct [utl::EventBus::PriorityEventQueue](#)
Priority queue for events.

Namespaces

- namespace [utl](#)

19.103.1 Detailed Description

Thread-safe event bus implementation for inter-component communication.
Definition in file [EventBus.hpp](#).

19.104 EventBus.hpp

[Go to the documentation of this file.](#)

```

00001 ///
00002 /// @file EventBus.hpp
00003 /// @brief Thread-safe event bus implementation for inter-component communication
00004 /// @namespace utl

```

```

00005 ///
00006
00007 #pragma once
00008
00009 #include "Interfaces/Protocol/Serializer.hpp"
00010 #include "Utils/Event.hpp"
00011 #include "Utils/Logger.hpp"
00012 #include <atomic>
00013 #include <chrono>
00014 #include <condition_variable>
00015 #include <functional>
00016 #include <memory>
00017 #include <mutex>
00018 #include <queue>
00019 #include <type_traits>
00020 #include <unordered_map>
00021 #include <unordered_set>
00022 #include <vector>
00023
00024 namespace utl
00025 {
00026
00027     ///
00028     /// @brief Thread-safe event bus for decoupled component communication
00029     ///
00030     class EventBus
00031     {
00032     private:
00033         ///
00034         /// @brief Internal event queue structure
00035         ///
00036         struct EventQueue
00037         {
00038             std::queue<Event> events;
00039             mutable std::mutex mutex;
00040             std::condition_variable cv;
00041             std::atomic<std::uint64_t> totalEvents{0};
00042         };
00043
00044         ///
00045         /// @brief Priority queue for events
00046         ///
00047         struct PriorityEventQueue
00048         {
00049             std::priority_queue<Event, std::vector<Event>, std::function<bool(const Event &, const Event &)>>
00050                 events;
00051             mutable std::mutex mutex;
00052
00053             PriorityEventQueue()
00054                 : events([](const Event &a, const Event &b)
00055                     { return static_cast<int>(a.priority) < static_cast<int>(b.priority); })
00056             {
00057             }
00058         };
00059
00060         EventQueue m_eventQueue;
00061         PriorityEventQueue m_priorityQueue;
00062         std::unordered_map<EventType, std::unordered_set<std::uint32_t> m_subscribers;
00063         mutable std::mutex m_subscribersMutex;
00064         EventStats m_stats;
00065         mutable std::mutex m_statsMutex;
00066         std::atomic<bool> m_running{true};
00067         std::atomic<std::uint64_t> m_maxQueueSize{10000};
00068
00069         // Event filtering
00070         std::unordered_set<EventType> m_filteredTypes;
00071         mutable std::mutex m_filterMutex;
00072
00073         // Component tracking
00074         std::unordered_map<std::uint32_t, std::string> m_componentNames;
00075         mutable std::mutex m_componentMutex;
00076
00077     public:
00078         ///
00079         /// @brief Get singleton instance of EventBus
00080         /// @return Reference to the singleton EventBus instance
00081         ///
00082         static EventBus &getInstance()
00083         {
00084             static EventBus instance;
00085             return instance;
00086         }
00087
00088         ///
00089         /// @brief Constructor
00090         ///
00091         EventBus()

```

```

00092     {
00093         m_stats.reset();
00094         utl::Logger::log("EventBus: Initialized", utl::LogLevel::INFO);
00095     }
00096
00097     ///
00098     /// @brief Destructor
00099     ///
00100     ~EventBus()
00101     {
00102         m_running.store(false);
00103         m_eventQueue.cv.notify_all();
00104         utl::Logger::log("EventBus: Destroyed", utl::LogLevel::INFO);
00105     }
00106
00107     // Delete copy constructor and assignment operator
00108     EventBus(const EventBus &) = delete;
00109     EventBus &operator=(const EventBus &) = delete;
00110     EventBus(EventBus &&) = delete;
00111     EventBus &operator=(EventBus &&) = delete;
00112
00113     ///
00114     /// @brief Publish an event to the bus
00115     /// @param event Event to publish
00116     /// @return True if event was published successfully
00117     ///
00118     bool publish(const Event &event)
00119     {
00120         if (!m_running.load())
00121         {
00122             return false;
00123         }
00124
00125         // Check if event type is filtered
00126         {
00127             std::lock_guard<std::mutex> lock(m_filterMutex);
00128             if (m_filteredTypes.find(event.type) != m_filteredTypes.end())
00129             {
00130                 return false; // Event type is filtered out
00131             }
00132         }
00133
00134         // Check queue size limit
00135         {
00136             std::lock_guard<std::mutex> lock(m_eventQueue.mutex);
00137             if (m_eventQueue.events.size() >= m_maxQueueSize.load())
00138             {
00139                 utl::Logger::log("EventBus: Queue full, dropping event type " +
00140                     std::to_string(static_cast<std::uint32_t>(event.type)) +
00141                     " from source " +
00142                     std::to_string(event.sourceId),
00143                     utl::LogLevel::WARNING);
00144                 return false;
00145             }
00146             m_eventQueue.events.push(event);
00147             m_eventQueue.totalEvents++;
00148         }
00149
00150         // Notify waiting consumers
00151         m_eventQueue.cv.notify_all();
00152
00153         // Update statistics
00154         updateStats(event, true);
00155
00156         utl::Logger::log("EventBus: Published event type " +
00157             std::to_string(static_cast<std::uint32_t>(event.type)) + " from source " +
00158             std::to_string(event.sourceId),
00159             utl::LogLevel::WARNING);
00160
00161         return true;
00162     }
00163
00164     ///
00165     /// @brief Publish event with serialized data
00166     /// @tparam T Type of data to serialize
00167     /// @param type Event type
00168     /// @param data Data to serialize
00169     /// @param sourceId Source component ID
00170     /// @param targetId Target component ID (0 for broadcast)
00171     /// @param priority Event priority
00172     /// @return True if event was published successfully
00173     ///
00174     template <typename T>
00175     bool publish(EventType type, const T &data, std::uint32_t sourceId = 0, std::uint32_t targetId = 0,
00176         EventPriority priority = EventPriority::NORMAL)
00177     {
00178         Event event(type, sourceId, targetId, priority);

```

```

00179 // Serialize data using existing Serializer
00180 try
00181 {
00182     rnp::Serializer serializer;
00183     if constexpr (std::is_same_v<T, rnp::PacketWorldState>)
00184     {
00185         serializer.serializeWorldState(data);
00186     }
00187     else if constexpr (std::is_same_v<T, std::vector<rnp::EventRecord>)
00188     {
00189         serializer.serializeEntityEvents(data);
00190     }
00191     else if constexpr (std::is_same_v<T, rnp::PacketLobbyCreate>)
00192     {
00193         serializer.serializeLobbyCreate(data);
00194     }
00195     else if constexpr (std::is_same_v<T, rnp::PacketLobbyCreateResponse>)
00196     {
00197         serializer.serializeLobbyCreateResponse(data);
00198     }
00199     else if constexpr (std::is_same_v<T, rnp::PacketLobbyListResponse>)
00200     {
00201         serializer.serializeLobbyListResponse(data);
00202     }
00203     else if constexpr (std::is_same_v<T, rnp::PacketLobbyJoin>)
00204     {
00205         serializer.serializeLobbyJoin(data);
00206     }
00207     else if constexpr (std::is_same_v<T, rnp::PacketLobbyJoinResponse>)
00208     {
00209         serializer.serializeLobbyJoinResponse(data);
00210     }
00211     else if constexpr (std::is_same_v<T, rnp::PacketLobbyUpdate>)
00212     {
00213         serializer.serializeLobbyUpdate(data);
00214     }
00215     else if constexpr (std::is_same_v<T, rnp::PacketGameStart>)
00216     {
00217         serializer.serializeGameStart(data);
00218     }
00219     else if constexpr (std::is_same_v<T, std::string>)
00220     {
00221         serializer.writeString(data);
00222     }
00223     else if constexpr (std::is_arithmetic_v<T>)
00224     {
00225         if constexpr (sizeof(T) == 1)
00226         {
00227             serializer.writeByte(static_cast<std::uint8_t>(data));
00228         }
00229         else if constexpr (sizeof(T) == 2)
00230         {
00231             serializer.writeUInt16(static_cast<std::uint16_t>(data));
00232         }
00233         else if constexpr (sizeof(T) == 4)
00234         {
00235             if constexpr (std::is_floating_point_v<T>)
00236             {
00237                 serializer.writeFloat(static_cast<float>(data));
00238             }
00239             else
00240             {
00241                 serializer.writeUInt32(static_cast<std::uint32_t>(data));
00242             }
00243         }
00244     }
00245     else
00246     {
00247         // For raw byte vectors, use directly
00248         static_assert(std::is_same_v<T, std::vector<std::uint8_t>,
00249             "Unsupported data type for EventBus serialization");
00250         event.data = data;
00251         return publish(event);
00252     }
00253     event.data = serializer.getData();
00254     return publish(event);
00255 }
00256 catch (const std::exception &e)
00257 {
00258     {
00259         utl::Logger::log("EventBus: Failed to serialize data for event type " +
00260             std::to_string(static_cast<std::uint32_t>(type)) + " - " + e.what(),
00261             utl::LogLevel::WARNING);
00262         return false;
00263     }
00264 }
00265

```

```

00266     ///
00267     /// @brief Consume events from the bus (non-blocking)
00268     /// @param maxEvents Maximum number of events to consume
00269     /// @return Vector of consumed events
00270     ///
00271     std::vector<Event> consume(std::uint32_t maxEvents = 100)
00272     {
00273         std::vector<Event> events;
00274
00275         std::lock_guard<std::mutex> lock(m_eventQueue.mutex);
00276
00277         std::uint32_t count = 0;
00278         while (!m_eventQueue.events.empty() && count < maxEvents)
00279         {
00280             Event event = m_eventQueue.events.front();
00281             m_eventQueue.events.pop();
00282
00283             // Check if event has expired
00284             if (event.hasExpired())
00285             {
00286                 updateStats(event, false, true);
00287                 continue;
00288             }
00289
00290             events.push_back(event);
00291             count++;
00292         }
00293
00294         // Update statistics
00295         for (const auto &event : events)
00296         {
00297             updateStats(event, false);
00298         }
00299
00300         return events;
00301     }
00302
00303     ///
00304     /// @brief Consume events with timeout (blocking)
00305     /// @param timeout Maximum time to wait for events
00306     /// @param maxEvents Maximum number of events to consume
00307     /// @return Vector of consumed events
00308     ///
00309     std::vector<Event> waitForEvents(std::chrono::milliseconds timeout = std::chrono::milliseconds(100),
00310                                     std::uint32_t maxEvents = 100)
00311     {
00312         std::unique_lock<std::mutex> lock(m_eventQueue.mutex);
00313
00314         // Wait for events or timeout
00315         if (m_eventQueue.cv.wait_for(lock, timeout, [this] { return !m_eventQueue.events.empty(); }))
00316         {
00317             return consume(maxEvents);
00318         }
00319
00320         return {}; // Timeout occurred
00321     }
00322
00323     ///
00324     /// @brief Consume events of specific type
00325     /// @param type Event type to filter
00326     /// @param maxEvents Maximum number of events to consume
00327     /// @return Vector of events of the specified type
00328     ///
00329     std::vector<Event> consumeType(EventType type, std::uint32_t maxEvents = 100)
00330     {
00331         std::vector<Event> allEvents = consume(maxEvents);
00332         std::vector<Event> filteredEvents;
00333
00334         for (const auto &event : allEvents)
00335         {
00336             if (event.type == type)
00337             {
00338                 filteredEvents.push_back(event);
00339             }
00340             else
00341             {
00342                 // Put other events back in queue
00343                 std::lock_guard<std::mutex> lock(m_eventQueue.mutex);
00344                 m_eventQueue.events.push(event);
00345             }
00346         }
00347
00348         return filteredEvents;
00349     }
00350
00351     ///
00352     /// @brief Consume events targeted to specific component

```

```

00353     /// @param targetId Target component ID
00354     /// @param maxEvents Maximum number of events to consume
00355     /// @return Vector of events targeted to the component
00356     ///
00357     std::vector<Event> consumeForTarget(std::uint32_t targetId, std::uint32_t maxEvents = 100)
00358     {
00359         std::vector<Event> allEvents = consume(maxEvents);
00360         std::vector<Event> filteredEvents;
00361
00362         for (const auto &event : allEvents)
00363         {
00364             if (event.targetId == targetId || event.isBroadcast())
00365             {
00366                 filteredEvents.push_back(event);
00367             }
00368             else
00369             {
00370                 // Put other events back in queue
00371                 std::lock_guard<std::mutex> lock(m_eventQueue.mutex);
00372                 m_eventQueue.events.push(event);
00373             }
00374         }
00375
00376         return filteredEvents;
00377     }
00378
00379     ///
00380     /// @brief Subscribe component to specific event types
00381     /// @param componentId Component ID
00382     /// @param type Event type to subscribe to
00383     ///
00384     void subscribe(std::uint32_t componentId, EventType type)
00385     {
00386         std::lock_guard<std::mutex> lock(m_subscribersMutex);
00387         m_subscribers[type].insert(componentId);
00388
00389         utl::Logger::log("EventBus: Component " + std::to_string(componentId) + " subscribed to event type " +
00390             std::to_string(static_cast<std::uint32_t>(type)),
00391             utl::LogLevel::WARNING);
00392     }
00393
00394     ///
00395     /// @brief Unsubscribe component from specific event type
00396     /// @param componentId Component ID
00397     /// @param type Event type to unsubscribe from
00398     ///
00399     void unsubscribe(std::uint32_t componentId, EventType type)
00400     {
00401         std::lock_guard<std::mutex> lock(m_subscribersMutex);
00402         auto it = m_subscribers.find(type);
00403         if (it != m_subscribers.end())
00404         {
00405             it->second.erase(componentId);
00406             if (it->second.empty())
00407             {
00408                 m_subscribers.erase(it);
00409             }
00410         }
00411
00412         utl::Logger::log("EventBus: Component " + std::to_string(componentId) +
00413             " unsubscribed from event type " +
00414             std::to_string(static_cast<std::uint32_t>(type)),
00415             utl::LogLevel::WARNING);
00416     }
00417
00418     ///
00419     /// @brief Register component name for better debugging
00420     /// @param componentId Component ID
00421     /// @param name Component name
00422     ///
00423     void registerComponent(std::uint32_t componentId, const std::string &name)
00424     {
00425         std::lock_guard<std::mutex> lock(m_componentMutex);
00426         m_componentNames[componentId] = name;
00427
00428         utl::Logger::log("EventBus: Registered component '" + name + "' with ID " + std::to_string(componentId),
00429             utl::LogLevel::INFO);
00430     }
00431
00432     ///
00433     /// @brief Unregister component
00434     /// @param componentId Component ID
00435     ///
00436     void unregisterComponent(std::uint32_t componentId)
00437     {
00438         {
00439             std::lock_guard<std::mutex> lock(m_componentMutex);

```

```

00440         auto it = m_componentNames.find(componentId);
00441         if (it != m_componentNames.end())
00442         {
00443             utl::Logger::log("EventBus: Unregistered component '" + it->second +
00444                             "' (ID: " + std::to_string(componentId) + ")",
00445                             utl::LogLevel::INFO);
00446             m_componentNames.erase(it);
00447         }
00448     }
00449
00450     /// Remove from all subscriptions
00451     std::lock_guard<std::mutex> lock(m_subscribersMutex);
00452     for (auto &pair : m_subscribers)
00453     {
00454         pair.second.erase(componentId);
00455     }
00456 }
00457
00458 ///
00459 /// @brief Add event type to filter (filtered events will be dropped)
00460 /// @param type Event type to filter
00461 ///
00462 void filterEventType(EventType type)
00463 {
00464     std::lock_guard<std::mutex> lock(m_filterMutex);
00465     m_filteredTypes.insert(type);
00466
00467     utl::Logger::log("EventBus: Added filter for event type " +
00468                     std::to_string(static_cast<std::uint32_t>(type)),
00469                     utl::LogLevel::INFO);
00470 }
00471
00472 ///
00473 /// @brief Remove event type from filter
00474 /// @param type Event type to unfilter
00475 ///
00476 void unfilterEventType(EventType type)
00477 {
00478     std::lock_guard<std::mutex> lock(m_filterMutex);
00479     m_filteredTypes.erase(type);
00480
00481     utl::Logger::log("EventBus: Removed filter for event type " +
00482                     std::to_string(static_cast<std::uint32_t>(type)),
00483                     utl::LogLevel::INFO);
00484 }
00485
00486 ///
00487 /// @brief Set maximum queue size
00488 /// @param maxSize Maximum number of events in queue
00489 ///
00490 void setMaxQueueSize(std::uint64_t maxSize) { m_maxQueueSize.store(maxSize); }
00491
00492 ///
00493 /// @brief Get current queue size
00494 /// @return Current number of events in queue
00495 ///
00496 std::uint64_t getQueueSize() const
00497 {
00498     std::lock_guard<std::mutex> lock(m_eventQueue.mutex);
00499     return m_eventQueue.events.size();
00500 }
00501
00502 ///
00503 /// @brief Get event bus statistics
00504 /// @return Copy of current statistics
00505 ///
00506 EventStats getStats() const
00507 {
00508     std::lock_guard<std::mutex> lock(m_statsMutex);
00509     EventStats stats = m_stats;
00510     stats.currentQueueSize = getQueueSize();
00511     return stats;
00512 }
00513
00514 ///
00515 /// @brief Clear all statistics
00516 ///
00517 void clearStats()
00518 {
00519     std::lock_guard<std::mutex> lock(m_statsMutex);
00520     m_stats.reset();
00521 }
00522
00523 ///
00524 /// @brief Clear all events from queue
00525 ///
00526 void clear()

```



```

00527     {
00528         std::lock_guard<std::mutex> lock(m_eventQueue.mutex);
00529         while (!m_eventQueue.events.empty())
00530         {
00531             m_eventQueue.events.pop();
00532         }
00533         utl::Logger::log("EventBus: Cleared all events from queue", utl::LogLevel::INFO);
00534     }
00535
00536     ///
00537     /// @brief Stop the event bus
00538     ///
00539     void stop()
00540     {
00541         m_running.store(false);
00542         m_eventQueue.cv.notify_all();
00543         utl::Logger::log("EventBus: Stopped", utl::LogLevel::INFO);
00544     }
00545
00546     ///
00547     /// @brief Check if event bus is running
00548     /// @return True if running
00549     ///
00550     bool isRunning() const { return m_running.load(); }
00551
00552 private:
00553     ///
00554     /// @brief Update statistics for an event
00555     /// @param event Event to update stats for
00556     /// @param isPublish True if this is a publish operation
00557     /// @param isExpired True if event expired
00558     ///
00559     void updateStats(const Event &event, bool isPublish, bool isExpired = false)
00560     {
00561         std::lock_guard<std::mutex> lock(m_statsMutex);
00562
00563         if (isPublish)
00564         {
00565             m_stats.totalEventsPublished++;
00566         }
00567         else
00568         {
00569             m_stats.totalEventsConsumed++;
00570         }
00571
00572         if (isExpired)
00573         {
00574             m_stats.totalEventsExpired++;
00575         }
00576
00577         m_stats.eventTypeCount[event.type]++;
00578         m_stats.priorityCount[event.priority]++;
00579         m_stats.sourceCount[event.sourceId]++;
00580
00581         if (event.targetId != 0)
00582         {
00583             m_stats.targetCount[event.targetId]++;
00584         }
00585
00586         // Note: We don't update max queue size here to avoid deadlock
00587         // (getQueueSize() would try to lock m_eventQueue.mutex which may already be locked)
00588     }
00589 };
00590
00591 } // namespace utl

```

19.105 /home/masina/Projects/Epitech/rtype/modules/Utils/include/Utils/HitboxUtils.hpp File Reference

Utility functions for hitbox calculations.

Namespaces

- namespace [utl](#)

Functions

- `std::pair< float, float > utl::calculateSpriteCenter` (const float x, const float y, const float spriteWidth, const float spriteHeight, const float scale)

Calculate the center position of a sprite.

- `std::pair< float, float > utl::calculateHitboxOffsets` (const float spriteWidth, const float spriteHeight, const float scale)

Calculate hitbox offsets to center the hitbox on the sprite.

- `std::pair< float, float > utl::calculateHitboxOffsetsRelative` (const float x, const float y, const float spriteWidth, const float spriteHeight, const float scale)

Calculate hitbox offsets relative to sprite position.

19.105.1 Detailed Description

Utility functions for hitbox calculations.

Definition in file [HitboxUtils.hpp](#).

19.106 HitboxUtils.hpp

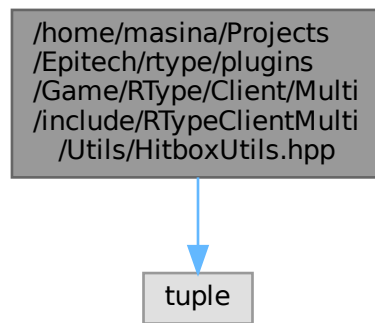
[Go to the documentation of this file.](#)

```
00001 ///  
00002 ///  
00003 ///  
00004 ///  
00005 ///  
00006 ///  
00007 #pragma once  
00008  
00009 namespace utl  
00010 {  
00011     ///  
00012     ///  
00013     ///  
00014     ///  
00015     ///  
00016     ///  
00017     ///  
00018     ///  
00019     ///  
00020     inline std::pair<float, float> calculateSpriteCenter(const float x, const float y, const float spriteWidth,  
00021                                                         const float spriteHeight, const float scale)  
00022     {  
00023         return {x + (spriteWidth * scale) / 2.0f, y + (spriteHeight * scale) / 2.0f};  
00024     }  
00025  
00026     ///  
00027     ///  
00028     ///  
00029     ///  
00030     ///  
00031     ///  
00032     ///  
00033     inline std::pair<float, float> calculateHitboxOffsets(const float spriteWidth, const float spriteHeight,  
00034                                                         const float scale)  
00035     {  
00036         return {(spriteWidth * scale) / 2.0f, (spriteHeight * scale) / 2.0f};  
00037     }  
00038  
00039     ///  
00040     ///  
00041     ///  
00042     ///  
00043     ///  
00044     ///  
00045     ///  
00046     ///  
00047     ///  
00048     inline std::pair<float, float> calculateHitboxOffsetsRelative(const float x, const float y, const float spriteWidth,  
00049                                                         const float spriteHeight, const float scale)  
00050     {  
00051         auto [centerX, centerY] = calculateSpriteCenter(x, y, spriteWidth, spriteHeight, scale);  
00052         return {centerX - x, centerY - y};  
00053     }  
00054 } // namespace utl
```

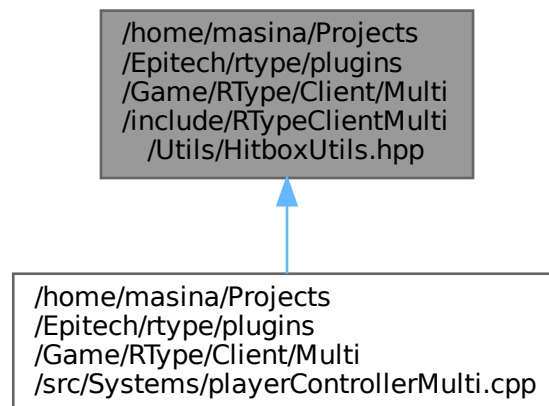
19.107 /home/masina/Projects/Epitech/rtype/plugins/Game/RType/↵ Client/Multi/include/RTypeClientMulti/Utils/HitboxUtils.hpp File Reference

#include <tuple>

Include dependency graph for HitboxUtils.hpp:



This graph shows which files directly or indirectly include this file:



Namespaces

- namespace [gme](#)
- namespace [gme::Utils](#)

Functions

- `std::tuple< float, float > gme::Utils::calculateHitboxOffsets` (float spriteWidth, float spriteHeight, float scale)

19.108 HitboxUtils.hpp

[Go to the documentation of this file.](#)

```

00001 #pragma once
00002
00003 #include <tuple>
00004
00005 namespace gme
00006 {
00007     namespace Utils
00008     {
00009         inline std::tuple<float, float> calculateHitboxOffsets(float spriteWidth, float spriteHeight, float scale)
00010         {
00011             const float halfWidth = (spriteWidth * scale) / 2.0f;
00012             const float halfHeight = (spriteHeight * scale) / 2.0f;
00013             return {halfWidth, halfHeight};
00014         }
00015     } // namespace Utils
00016 } // namespace gme

```

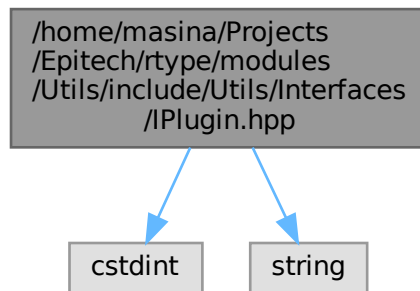
19.109 /home/masina/Projects/Epitech/rtype/modules/Utils/include/Utils/Interfaces/IPlugin.hpp File Reference

This file contains the plugin interface.

```
#include <cstdint>
```

```
#include <string>
```

Include dependency graph for IPlugin.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- interface [utl::IPlugin](#)
Interface for plugins.

Namespaces

- namespace [utl](#)

Macros

- `#define PLUGIN_EXPORT`

Enumerations

- enum class `utl::PluginType` : `uint8_t` {
`utl::AUDIO` = 0 , `utl::NETWORK_CLIENT` = 1 , `utl::NETWORK_SERVER` = 1 ,
`utl::RENDERER` = 2 ,
`utl::GAME_CLIENT` = 3 , `utl::GAME_SERVER` = 4 , `utl::UNKNOWN` = 255 }

19.109.1 Detailed Description

This file contains the plugin interface.

Definition in file [IPlugin.hpp](#).

19.109.2 Macro Definition Documentation

19.109.2.1 PLUGIN_EXPORT

`#define PLUGIN_EXPORT`

Definition at line 15 of file [IPlugin.hpp](#).

19.110 IPlugin.hpp

[Go to the documentation of this file.](#)

```

00001 ///  

00002 ///  

00003 ///  

00004 ///  

00005 ///  

00006 ///  

00007 #pragma once  

00008  

00009 #include <cstdint>  

00010 #include <string>  

00011  

00012 #if defined(_WIN32) || defined(_WIN64)  

00013 #define PLUGIN_EXPORT __declspec(dllexport)  

00014 #else  

00015 #define PLUGIN_EXPORT  

00016 #endif  

00017  

00018 namespace utl  

00019 {  

00020  

00021     enum class PluginType : uint8_t  

00022     {  

00023         AUDIO = 0,  

00024         NETWORK_CLIENT = 1,  

00025         NETWORK_SERVER = 1,  

00026         RENDERER = 2,  

00027         GAME_CLIENT = 3,  

00028         GAME_SERVER = 4,  

00029         UNKNOWN = 255  

00030     };  

00031  

00032     ///  

00033     ///  

00034     ///  

00035     ///  

00036     ///  

00037     class IPlugin  

00038     {  

00039     public:  

00040         virtual ~IPlugin() = default;  

00041         [[nodiscard]] virtual const std::string getName() const = 0;  

00042         [[nodiscard]] virtual PluginType getType() const = 0;  

00043     }; // interface IPlugin  

00044 } // namespace utl

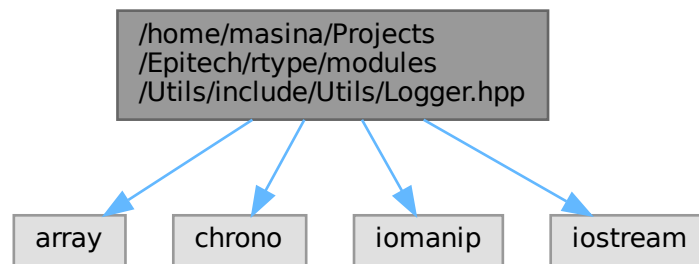
```

19.111 /home/masina/Projects/Epitech/rtype/modules/Utils/include/Utils/Logger.hpp File Reference

This file contains the Logger class.

```
#include <array>
#include <chrono>
#include <iomanip>
#include <iostream>
```

Include dependency graph for Logger.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class `utl::Logger`
Class for logging.

Namespaces

- namespace `utl`

Enumerations

- enum class `utl::LogLevel` : `uint8_t` { `utl::INFO` , `utl::WARNING` }

19.111.1 Detailed Description

This file contains the Logger class.

Definition in file `Logger.hpp`.

19.112 Logger.hpp

[Go to the documentation of this file.](#)

```
00001 ///
00002 /// @file Logger.hpp
00003 /// @brief This file contains the Logger class
00004 /// @namespace utl
00005 ///
00006
```

```

00007 #pragma once
00008
00009 #include <array>
00010 #include <chrono>
00011 #include <iomanip>
00012 #include <iostream>
00013
00014 namespace utl
00015 {
00016
00017     enum class LogLevel : uint8_t
00018     {
00019         INFO,
00020         WARNING
00021     };
00022
00023     ///
00024     /// @class Logger
00025     /// @brief Class for logging
00026     /// @namespace utl
00027     ///
00028     class Logger
00029     {
00030
00031     public:
00032         Logger(const Logger &) = delete;
00033         Logger &operator=(const Logger &) = delete;
00034         Logger(Logger &&) = delete;
00035         Logger &operator=(Logger &&) = delete;
00036
00037         static void init();
00038
00039         template <typename Func> static void logExecutionTime(const std::string &message, Func &&func)
00040         {
00041             const auto start = std::chrono::high_resolution_clock::now();
00042             func();
00043             const auto end = std::chrono::high_resolution_clock::now();
00044             const auto duration = std::chrono::duration<float, std::milli>(end - start).count();
00045
00046             std::cout << getColorForDuration(duration)
00047                 << formatLogMessage(LogLevel::INFO, message + " took " + std::to_string(duration) + " ms")
00048                 << LOG_LEVEL_COLOR[COLOR_RESET];
00049         }
00050
00051         static void log(const std::string &message, const LogLevel &logLevel)
00052         {
00053             std::cout << (logLevel == LogLevel::INFO ? LOG_LEVEL_COLOR[COLOR_INFO] :
LOG_LEVEL_COLOR[COLOR_WARNING])
00054                 << formatLogMessage(logLevel, message) << LOG_LEVEL_COLOR[COLOR_RESET];
00055         }
00056
00057     private:
00058         enum ColorIndex : uint8_t
00059         {
00060             COLOR_ERROR,
00061             COLOR_INFO,
00062             COLOR_WARNING,
00063             COLOR_RESET
00064         };
00065
00066         static constexpr std::array<const char *, 4> LOG_LEVEL_COLOR = {
00067             "\033[31m", // ERROR/slow execution
00068             "\033[32m", // INFO/fast execution
00069             "\033[33m", // WARNING/medium execution
00070             "\033[0m\n" // RESET + newline
00071         };
00072
00073         static constexpr std::array<const char *, 2> LOG_LEVEL_STRING = {"INFO", "WARNING"};
00074
00075         Logger() = default;
00076         ~Logger() = default;
00077
00078         [[nodiscard]] static const char *getColorForDuration(const float duration)
00079         {
00080             return duration < 20.0F
00081                 ? LOG_LEVEL_COLOR[COLOR_INFO]
00082                 : (duration < 90.0F ? LOG_LEVEL_COLOR[COLOR_WARNING] :
LOG_LEVEL_COLOR[COLOR_ERROR]);
00083         }
00084
00085         [[nodiscard]] static std::string formatLogMessage(LogLevel level, const std::string &message)
00086         {
00087             const auto inTime = std::chrono::system_clock::to_time_t(std::chrono::system_clock::now());
00088             std::ostringstream ss;
00089             ss << "[" << std::put_time(std::localtime(&inTime), "%Y-%m-%d %X") << " ";
00090             ss << "[" << LOG_LEVEL_STRING[static_cast<uint8_t>(level)] << " ] " << message;
00091             return ss.str();

```

```

00092     }
00093
00094 }; // class Logger
00095
00096 } // namespace utl

```

19.113 /home/masina/Projects/Epitech/rtype/modules/Utils/include/Utils/PluginLoader.hpp File Reference

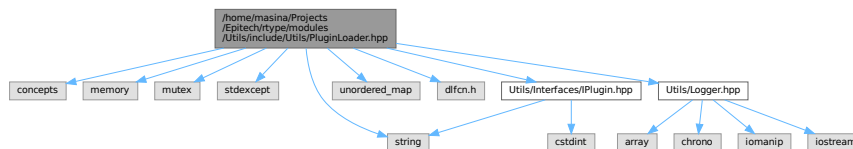
Modern, cross-platform plugin loader.

```

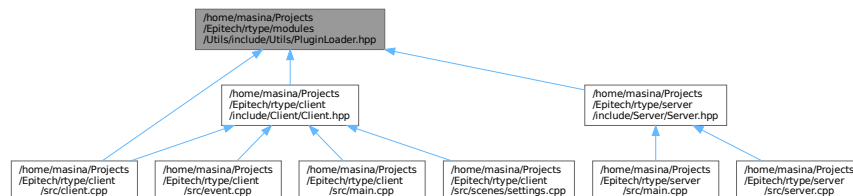
#include <concepts>
#include <memory>
#include <mutex>
#include <stdexcept>
#include <string>
#include <unordered_map>
#include <dlfcn.h>
#include "Utils/Interfaces/IPlugin.hpp"
#include "Utils/Logger.hpp"

```

Include dependency graph for PluginLoader.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- struct [utl::SharedLib](#)
Handle to a dynamic library with RAII.
- class [utl::PluginLoader](#)
Modern, type-safe plugin loader.

Namespaces

- namespace [utl](#)

Macros

- `#define PLUGINS\_EXTENSION ".so"`

Typedefs

- using [utl::LibHandle](#)
- using [utl::EntryPointFn](#) = [IPlugin](#) *(*)()

19.113.1 Detailed Description

Modern, cross-platform plugin loader.

Definition in file [PluginLoader.hpp](#).

19.113.2 Macro Definition Documentation

19.113.2.1 PLUGINS_EXTENSION

```
#define PLUGINS_EXTENSION ".so"
```

Definition at line 24 of file [PluginLoader.hpp](#).

19.114 PluginLoader.hpp

[Go to the documentation of this file.](#)

```
00001 ///  
00002 ///  
00003 ///  
00004 ///  
00005 ///  
00006 ///  
00007 #pragma once  
00008  
00009 #include <concepts>  
00010 #include <memory>  
00011 #include <mutex>  
00012 #include <stdexcept>  
00013 #include <string>  
00014 #include <unordered_map>  
00015  
00016 #ifdef _WIN32  
00017 #include <windows.h>  
00018 #define PLUGINS_EXTENSION ".dll"  
00019 #else  
00020 #include <dlfcn.h>  
00021 #if __APPLE__  
00022 #define PLUGINS_EXTENSION ".dylib"  
00023 #else  
00024 #define PLUGINS_EXTENSION ".so"  
00025 #endif  
00026 #endif  
00027  
00028 #include "Utils/Interfaces/IPlugin.hpp"  
00029 #include "Utils/Logger.hpp"  
00030  
00031 namespace utl  
00032 {  
00033  
00034     using LibHandle =  
00035     #ifdef _WIN32  
00036         HMODULE;  
00037     #else  
00038         void *;  
00039     #endif  
00040  
00041     ///  
00042     struct SharedLib  
00043     {  
00044         LibHandle handle = nullptr;  
00045  
00046         explicit SharedLib(const LibHandle h = nullptr) : handle(h) {}  
00047         ~SharedLib() { close(); }  
00048  
00049         SharedLib(const SharedLib &) = delete;  
00050         SharedLib &operator=(const SharedLib &) = delete;  
00051         SharedLib(SharedLib &&other) noexcept : handle(other.handle) { other.handle = nullptr; }  
00052         SharedLib &operator=(SharedLib &&other) noexcept  
00053         {  
00054             if (this != &other)  
00055             {  
00056                 close();  
00057                 handle = other.handle;  
00058                 other.handle = nullptr;  
00059             }  
00060         }  
00061     }  
00062 }
```

```

00060         return *this;
00061     }
00062
00063     void close()
00064     {
00065         if (!handle)
00066             return;
00067 #ifdef _WIN32
00068         FreeLibrary(handle);
00069 #else
00070         dlclose(handle);
00071 #endif
00072         handle = nullptr;
00073     }
00074
00075     explicit operator bool() const { return handle != nullptr; }
00076 };
00077
00078 using EntryPointFn = IPlugin *(*)();
00079
00080 ///
00081 /// @class PluginLoader
00082 /// @brief Modern, type-safe plugin loader
00083 /// @namespace utl
00084 ///
00085 class PluginLoader
00086 {
00087 public:
00088     PluginLoader() = default;
00089     ~PluginLoader() = default;
00090
00091     PluginLoader(const PluginLoader &) = delete;
00092     PluginLoader &operator=(const PluginLoader &) = delete;
00093     PluginLoader(PluginLoader &&) = delete;
00094     PluginLoader &operator=(PluginLoader &&) = delete;
00095
00096     ///
00097     /// Load a plugin of type T
00098     /// @tparam T Expected plugin interface (must derive from IPlugin)
00099     /// @param path Path to the dynamic library
00100     /// @return shared_ptr<T> instance
00101     ///
00102     template <std::derived_from<IPlugin> T> std::shared_ptr<T> loadPlugin(const std::string &path)
00103     {
00104         std::scoped_lock lock(m_mutex);
00105
00106         if (m_plugins.contains(path))
00107             throw std::runtime_error("Plugin already loaded: " + path);
00108
00109         SharedLib lib = loadLibrary(path);
00110         const EntryPointFn entry = getEntryPoint(lib, path);
00111
00112         std::unique_ptr<IPlugin> plugin(entry());
00113         if (!plugin)
00114             throw std::runtime_error("EntryPoint failed: " + path);
00115
00116         T *typed = dynamic_cast<T *>(plugin.get());
00117         if (!typed)
00118             throw std::runtime_error("Plugin type mismatch: " + path);
00119
00120         auto [it, inserted] = m_plugins.emplace(path, std::move(plugin));
00121         if (!inserted)
00122             throw std::runtime_error("Failed to store plugin: " + path);
00123
00124         m_handles[path] = std::move(lib);
00125
00126         Logger::log("Plugin loaded:\t name: " + it->second->getName() + "\t path: " + path, LogLevel::INFO);
00127
00128         std::shared_ptr<IPlugin> baseShared(it->second.get(), [](IPlugin *) {});
00129         return std::shared_ptr<T>(baseShared, typed);
00130     }
00131
00132 private:
00133     std::mutex m_mutex;
00134     std::unordered_map<std::string, SharedLib> m_handles;
00135     std::unordered_map<std::string, std::unique_ptr<IPlugin> > m_plugins;
00136
00137     SharedLib loadLibrary(const std::string &path)
00138     {
00139
00140 #ifdef _WIN32
00141         const LibHandle handle = LoadLibraryA(path.c_str());
00142         if (!handle)
00143         {
00144             const DWORD errorCode = GetLastError();
00145             LPVOID errorMsg;
00146             FormatMessageA(FORMAT_MESSAGE_ALLOCATE_BUFFER |

```

```

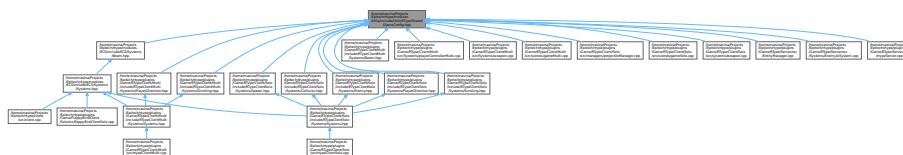
00147     FORMAT_MESSAGE_FROM_SYSTEM |
00148         FORMAT_MESSAGE_IGNORE_INSERTS,
00149     nullptr, errorCode, MAKELANGID(LANG_NEUTRAL, SUBLANG_DEFAULT),
00150     reinterpret_cast<LPSTR>(&errorMsg), 0, nullptr);
00151     std::cerr << "[ERROR] Failed to load plugin: " << path << "\n[WinError " << errorCode << " ] "
00152     << (errorMsg ? static_cast<LPSTR>(errorMsg) : "Unknown error") << std::endl;
00153
00154     if (errorMsg)
00155         LocalFree(errorMsg);
00156
00157     throw std::runtime_error("Cannot load library: " + path);
00158 }
00159 #else
00160     dLError();
00161     const LibHandle handle = dlopen(path.c_str(), RTLD_LAZY);
00162     if (!handle)
00163     {
00164         const char *error = dLError();
00165         std::string msg = "Cannot load library: " + path;
00166         if (error)
00167         {
00168             msg += " (" + std::string(error) + ")";
00169         }
00170         throw std::runtime_error(msg);
00171     }
00172 #endif
00173     return SharedLib(handle);
00174 }
00175
00176 EntryPointFn getEntryPoint(SharedLib &lib, const std::string &path)
00177 {
00178     EntryPointFn entry =
00179 #ifdef _WIN32
00180         reinterpret_cast<EntryPointFn>(GetProcAddress(lib.handle, "entryPoint"));
00181 #else
00182         reinterpret_cast<EntryPointFn>(dlsym(lib.handle, "entryPoint"));
00183 #endif
00184     if (!entry)
00185         throw std::runtime_error("EntryPoint not found in plugin: " + path);
00186     return entry;
00187 }
00188 };
00189
00190 } // namespace utl

```

19.115 /home/masina/Projects/Epitech/rtype/modules/Utils/include/Utils/RTypeShared/GameConfig.hpp File Reference

Configuration constants for the multiplayer game.

This graph shows which files directly or indirectly include this file:



Namespaces

- namespace `gme`
- namespace `utl`
- namespace `utl::GameConfig`
- namespace `utl::GameConfig::Screen`
- namespace `utl::GameConfig::Player`
- namespace `utl::GameConfig::Projectile`
- namespace `utl::GameConfig::Projectile::Basic`
- namespace `utl::GameConfig::Projectile::Supercharged`
- namespace `utl::GameConfig::Animation`
- namespace `utl::GameConfig::Beam`

- namespace [utl::GameConfig::LoadingAnimation](#)
- namespace [utl::GameConfig::Stage](#)
- namespace [utl::GameConfig::Enemy](#)
- namespace [utl::GameConfig::Enemy::Easy](#)
- namespace [utl::GameConfig::Explosion](#)
- namespace [utl::GameConfig::Hitbox](#)
- namespace [utl::GameConfig::Server](#)
- namespace [utl::GameConfig::Server::Enemy](#)
- namespace [utl::GameConfig::Server::Enemy::Basic](#)
- namespace [utl::GameConfig::Server::Enemy::Advanced](#)
- namespace [utl::GameConfig::Server::Enemy::Boss](#)
- namespace [utl::GameConfig::Server::Enemy::AI](#)
- namespace [utl::GameConfig::Server::Projectile](#)
- namespace [utl::GameConfig::Server::Network](#)

Variables

- constexpr float [utl::GameConfig::Screen::SPAWN_X](#) = 1950.0f
- constexpr float [utl::GameConfig::Screen::MIN_Y](#) = 50.0f
- constexpr float [utl::GameConfig::Screen::MAX_Y](#) = 1030.0f
- constexpr float [utl::GameConfig::Screen::REMOVE_X](#) = -100.0f
- constexpr float [utl::GameConfig::Screen::REMOVE_MIN_Y](#) = -50.0f
- constexpr float [utl::GameConfig::Screen::REMOVE_MAX_Y](#) = 1130.0f
- constexpr float [utl::GameConfig::Player::SPEED](#) = 500.0f
- constexpr float [utl::GameConfig::Player::DIAGONAL_SPEED_MULTIPLIER](#) = 0.707f
- constexpr float [utl::GameConfig::Player::SPRITE_WIDTH](#) = 33.0f
- constexpr float [utl::GameConfig::Player::SPRITE_HEIGHT](#) = 17.0f
- constexpr float [utl::GameConfig::Player::WIDTH](#) = 66.0f
- constexpr float [utl::GameConfig::Player::HEIGHT](#) = 17.0f
- constexpr float [utl::GameConfig::Player::SCALE](#) = 2.0f
- constexpr int [utl::GameConfig::Player::FRAMES_PER_ROW](#) = 5
- constexpr float [utl::GameConfig::Projectile::Basic::SPEED](#) = 800.0f
- constexpr float [utl::GameConfig::Projectile::Basic::DAMAGE](#) = 10.0f
- constexpr float [utl::GameConfig::Projectile::Basic::LIFETIME](#) = 3.0f
- constexpr float [utl::GameConfig::Projectile::Basic::SCALE](#) = 1.0f
- constexpr float [utl::GameConfig::Projectile::Basic::FIRE_COOLDOWN](#) = 0.3f
- constexpr float [utl::GameConfig::Projectile::Basic::SPRITE_WIDTH](#) = 16.0f
- constexpr float [utl::GameConfig::Projectile::Basic::SPRITE_HEIGHT](#) = 8.0f
- constexpr float [utl::GameConfig::Projectile::Supercharged::SPEED](#) = 1200.0f
- constexpr float [utl::GameConfig::Projectile::Supercharged::DAMAGE](#) = 25.0f
- constexpr float [utl::GameConfig::Projectile::Supercharged::LIFETIME](#) = 5.0f
- constexpr float [utl::GameConfig::Projectile::Supercharged::SCALE](#) = 1.5f
- constexpr float [utl::GameConfig::Projectile::Supercharged::FIRE_COOLDOWN](#) = 0.2f
- constexpr float [utl::GameConfig::Projectile::Supercharged::CHARGE_TIME](#) = 0.5f
- constexpr float [utl::GameConfig::Projectile::Supercharged::SPRITE_WIDTH](#) = 29.0f
- constexpr float [utl::GameConfig::Projectile::Supercharged::SPRITE_HEIGHT](#) = 24.0f
- constexpr int [utl::GameConfig::Projectile::Supercharged::ANIMATION_FRAMES](#) = 4
- constexpr float [utl::GameConfig::Projectile::Supercharged::ANIMATION_DURATION](#) = 0.15f
- constexpr float [utl::GameConfig::Animation::FRAME_DURATION](#) = 0.1f
- constexpr float [utl::GameConfig::Beam::MAX_CHARGE](#) = 1.0f
- constexpr float [utl::GameConfig::Beam::CHARGE_RATE](#) = 1.0f
- constexpr float [utl::GameConfig::Beam::BAR_WIDTH](#) = 120.0f
- constexpr float [utl::GameConfig::Beam::BAR_HEIGHT](#) = 12.0f
- constexpr float [utl::GameConfig::Beam::BAR_X](#) = 10.0f
- constexpr float [utl::GameConfig::Beam::BAR_Y](#) = 10.0f

-
- constexpr float [utl::GameConfig::LoadingAnimation::SPRITE_WIDTH](#) = 29.0f
 - constexpr float [utl::GameConfig::LoadingAnimation::SPRITE_HEIGHT](#) = 24.0f
 - constexpr int [utl::GameConfig::LoadingAnimation::ANIMATION_FRAMES](#) = 4
 - constexpr float [utl::GameConfig::LoadingAnimation::ANIMATION_DURATION](#) = 0.15f
 - constexpr float [utl::GameConfig::LoadingAnimation::OFFSET_X](#) = 60.0f
 - constexpr float [utl::GameConfig::LoadingAnimation::OFFSET_Y](#) = 6.0f
 - constexpr float [utl::GameConfig::Stage::FLOOR_OFFSET_Y](#) = 16.0f
 - constexpr float [utl::GameConfig::Stage::CEILING_OFFSET_Y](#) = -1.0f
 - constexpr float [utl::GameConfig::Enemy::Easy::HEALTH](#) = 1.0f
 - constexpr float [utl::GameConfig::Enemy::Easy::DAMAGE](#) = 5.0f
 - constexpr float [utl::GameConfig::Enemy::Easy::SPEED](#) = 80.0f
 - constexpr float [utl::GameConfig::Enemy::Easy::SPRITE_WIDTH](#) = 32.0f
 - constexpr float [utl::GameConfig::Enemy::Easy::SPRITE_HEIGHT](#) = 32.0f
 - constexpr float [utl::GameConfig::Enemy::Easy::SCALE](#) = 2.0f
 - constexpr float [utl::GameConfig::Enemy::Easy::SHOOT_COOLDOWN](#) = 2.0f
 - constexpr float [utl::GameConfig::Enemy::Easy::SPAWN_RATE](#) = 2.0f
 - constexpr int [utl::GameConfig::Enemy::Easy::ANIMATION_FRAMES](#) = 4
 - constexpr float [utl::GameConfig::Enemy::Easy::ANIMATION_DURATION](#) = 0.5f
 - constexpr int [utl::GameConfig::Enemy::Easy::FRAMES_PER_ROW](#) = 4
 - constexpr float [utl::GameConfig::Explosion::SPRITE_WIDTH](#) = 32.0f
 - constexpr float [utl::GameConfig::Explosion::SPRITE_HEIGHT](#) = 32.0f
 - constexpr int [utl::GameConfig::Explosion::ANIMATION_FRAMES](#) = 4
 - constexpr float [utl::GameConfig::Explosion::ANIMATION_DURATION](#) = 0.1f
 - constexpr int [utl::GameConfig::Explosion::FRAMES_PER_ROW](#) = 4
 - constexpr float [utl::GameConfig::Explosion::LIFETIME](#) = 0.4f
 - constexpr float [utl::GameConfig::Explosion::SCALE](#) = 2.0f
 - constexpr float [utl::GameConfig::Hitbox::PLAYER_RADIUS](#) = 20.0f
 - constexpr float [utl::GameConfig::Hitbox::ENEMY_RADIUS](#) = 15.0f
 - constexpr float [utl::GameConfig::Hitbox::PROJECTILE_BASIC_RADIUS](#) = 5.0f
 - constexpr float [utl::GameConfig::Hitbox::PROJECTILE_SUPERCHARGED_RADIUS](#) = 8.0f
 - constexpr float [utl::GameConfig::Server::SCREEN_WIDTH](#) = 1920.0f
 - constexpr float [utl::GameConfig::Server::SCREEN_HEIGHT](#) = 1080.0f
 - constexpr float [utl::GameConfig::Server::WORLD_MARGIN](#) = 200.0f
 - constexpr float [utl::GameConfig::Server::Enemy::Basic::HEALTH](#) = 50.0f
 - constexpr float [utl::GameConfig::Server::Enemy::Basic::DAMAGE](#) = 10.0f
 - constexpr float [utl::GameConfig::Server::Enemy::Basic::SPEED](#) = 200.0f
 - constexpr float [utl::GameConfig::Server::Enemy::Basic::SHOOT_COOLDOWN](#) = 2.0f
 - constexpr float [utl::GameConfig::Server::Enemy::Basic::HITBOX_RADIUS](#) = 15.0f
 - constexpr float [utl::GameConfig::Server::Enemy::Advanced::HEALTH](#) = 100.0f
 - constexpr float [utl::GameConfig::Server::Enemy::Advanced::DAMAGE](#) = 15.0f
 - constexpr float [utl::GameConfig::Server::Enemy::Advanced::SPEED](#) = 150.0f
 - constexpr float [utl::GameConfig::Server::Enemy::Advanced::SHOOT_COOLDOWN](#) = 1.5f
 - constexpr float [utl::GameConfig::Server::Enemy::Advanced::HITBOX_RADIUS](#) = 15.0f
 - constexpr float [utl::GameConfig::Server::Enemy::Advanced::SINE_FREQUENCY](#) = 1.5f
 - constexpr float [utl::GameConfig::Server::Enemy::Advanced::SINE_AMPLITUDE](#) = 100.0f
 - constexpr float [utl::GameConfig::Server::Enemy::Boss::HEALTH](#) = 1000.0f
 - constexpr float [utl::GameConfig::Server::Enemy::Boss::DAMAGE](#) = 50.0f
 - constexpr float [utl::GameConfig::Server::Enemy::Boss::HITBOX_RADIUS](#) = 50.0f
 - constexpr float [utl::GameConfig::Server::Enemy::Boss::RAM_DAMAGE](#) = 20.0f
 - constexpr float [utl::GameConfig::Server::Enemy::Boss::PHASE_2_THRESHOLD](#) = 0.66f
 - constexpr float [utl::GameConfig::Server::Enemy::Boss::PHASE_3_THRESHOLD](#) = 0.33f
 - constexpr float [utl::GameConfig::Server::Enemy::Boss::PHASE1_SPEED](#) = 30.0f
 - constexpr float [utl::GameConfig::Server::Enemy::Boss::PHASE1_COOLDOWN](#) = 1.5f
 - constexpr float [utl::GameConfig::Server::Enemy::Boss::PHASE1_SINE_FREQUENCY](#) = 0.5f
 - constexpr float [utl::GameConfig::Server::Enemy::Boss::PHASE1_SINE_AMPLITUDE](#) = 150.0f
-

- constexpr float [utl::GameConfig::Server::Enemy::Boss::PHASE2_COOLDOWN](#) = 1.0f
- constexpr float [utl::GameConfig::Server::Enemy::Boss::PHASE3_SPEED](#) = 50.0f
- constexpr float [utl::GameConfig::Server::Enemy::Boss::PHASE3_COOLDOWN](#) = 0.5f
- constexpr float [utl::GameConfig::Server::Enemy::Boss::PHASE3_AGGRESSION](#) = 80.0f
- constexpr float [utl::GameConfig::Server::Enemy::Boss::SPREAD_INTERVAL](#) = 1.0f
- constexpr float [utl::GameConfig::Server::Enemy::Boss::SPREAD_ANGLE_DEGREES](#) = 15.0f
- constexpr float [utl::GameConfig::Server::Enemy::Boss::SPREAD_SPEED](#) = 300.0f
- constexpr float [utl::GameConfig::Server::Enemy::AI::SHOOT_CHANCE](#) = 0.3f
- constexpr float [utl::GameConfig::Server::Enemy::AI::MAX_SHOOT_DISTANCE](#) = 1500.0f
- constexpr float [utl::GameConfig::Server::Enemy::AI::AGGRESSIVE_ACTIVATION_DISTANCE](#) = 400.0f
- constexpr float [utl::GameConfig::Server::Enemy::AI::ZIGZAG_PERIOD](#) = 1.5f
- constexpr float [utl::GameConfig::Server::Enemy::AI::ZIGZAG_SPEED_X](#) = 150.0f
- constexpr float [utl::GameConfig::Server::Enemy::AI::ZIGZAG_SPEED_Y](#) = 100.0f
- constexpr float [utl::GameConfig::Server::Projectile::ENEMY_SPEED](#) = 500.0f
- constexpr float [utl::GameConfig::Server::Projectile::ENEMY_DAMAGE](#) = 15.0f
- constexpr float [utl::GameConfig::Server::Projectile::ENEMY_LIFETIME](#) = 10.0f
- constexpr float [utl::GameConfig::Server::Network::BROADCAST_RATE](#) = 60.0f
- constexpr float [utl::GameConfig::Server::Network::BROADCAST_INTERVAL](#) = 1.0f / [BROADCAST_RATE](#)

19.115.1 Detailed Description

Configuration constants for the multiplayer game.
Definition in file [GameConfig.hpp](#).

19.116 GameConfig.hpp

[Go to the documentation of this file.](#)

```
00001 ///
00002 /// @file GameConfig.hpp
00003 /// @brief Configuration constants for the multiplayer game
00004 /// @namespace gme
00005 ///
00006
00007 #pragma once
00008
00009 namespace utl
00010 {
00011     namespace GameConfig
00012     {
00013         namespace Screen
00014         {
00015             inline constexpr float SPAWN_X = 1950.0f;
00016             inline constexpr float MIN_Y = 50.0f;
00017             inline constexpr float MAX_Y = 1030.0f;
00018             inline constexpr float REMOVE_X = -100.0f;
00019             inline constexpr float REMOVE_MIN_Y = -50.0f;
00020             inline constexpr float REMOVE_MAX_Y = 1130.0f;
00021         } // namespace Screen
00022         namespace Player
00023         {
00024             inline constexpr float SPEED = 500.0f;
00025             inline constexpr float DIAGONAL_SPEED_MULTIPLIER = 0.707f;
00026             inline constexpr float SPRITE_WIDTH = 33.0f;
00027             inline constexpr float SPRITE_HEIGHT = 17.0f;
00028             inline constexpr float WIDTH = 66.0f;
00029             inline constexpr float HEIGHT = 17.0f;
00030             inline constexpr float SCALE = 2.0f;
00031             inline constexpr int FRAMES_PER_ROW = 5;
00032         } // namespace Player
00033         namespace Projectile
00034         {
00035             namespace Basic
00036             {
00037                 inline constexpr float SPEED = 800.0f;
00038                 inline constexpr float DAMAGE = 10.0f;
00039                 inline constexpr float LIFETIME = 3.0f;
00040                 inline constexpr float SCALE = 1.0f;
00041                 inline constexpr float FIRE_COOLDOWN = 0.3f;
00042                 inline constexpr float SPRITE_WIDTH = 16.0f;
00043                 inline constexpr float SPRITE_HEIGHT = 8.0f;
```



```

00044     } // namespace Basic
00045
00046     namespace Supercharged
00047     {
00048         inline constexpr float SPEED = 1200.0f;
00049         inline constexpr float DAMAGE = 25.0f;
00050         inline constexpr float LIFETIME = 5.0f;
00051         inline constexpr float SCALE = 1.5f;
00052         inline constexpr float FIRE_COOLDOWN = 0.2f;
00053         inline constexpr float CHARGE_TIME = 0.5f;
00054         inline constexpr float SPRITE_WIDTH = 29.0f;
00055         inline constexpr float SPRITE_HEIGHT = 24.0f;
00056         inline constexpr int ANIMATION_FRAMES = 4;
00057         inline constexpr float ANIMATION_DURATION = 0.15f;
00058     } // namespace Supercharged
00059 } // namespace Projectile
00060 namespace Animation
00061 {
00062     inline constexpr float FRAME_DURATION = 0.1f;
00063 }
00064 namespace Beam
00065 {
00066     inline constexpr float MAX_CHARGE = 1.0f;
00067     inline constexpr float CHARGE_RATE = 1.0f;
00068     inline constexpr float BAR_WIDTH = 120.0f;
00069     inline constexpr float BAR_HEIGHT = 12.0f;
00070     inline constexpr float BAR_X = 10.0f;
00071     inline constexpr float BAR_Y = 10.0f;
00072 } // namespace Beam
00073 namespace LoadingAnimation
00074 {
00075     inline constexpr float SPRITE_WIDTH = 29.0f;
00076     inline constexpr float SPRITE_HEIGHT = 24.0f;
00077     inline constexpr int ANIMATION_FRAMES = 4;
00078     inline constexpr float ANIMATION_DURATION = 0.15f;
00079     inline constexpr float OFFSET_X = 60.0f;
00080     inline constexpr float OFFSET_Y = 6.0f;
00081 } // namespace LoadingAnimation
00082 namespace Stage
00083 {
00084     inline constexpr float FLOOR_OFFSET_Y = 16.0f;
00085     inline constexpr float CEILING_OFFSET_Y = -1.0f;
00086 } // namespace Stage
00087 namespace Enemy
00088 {
00089     namespace Easy
00090     {
00091         inline constexpr float HEALTH = 1.0f;
00092         inline constexpr float DAMAGE = 5.0f;
00093         inline constexpr float SPEED = 80.0f;
00094         inline constexpr float SPRITE_WIDTH = 32.0f;
00095         inline constexpr float SPRITE_HEIGHT = 32.0f;
00096         inline constexpr float SCALE = 2.0f;
00097         inline constexpr float SHOOT_COOLDOWN = 2.0f;
00098         inline constexpr float SPAWN_RATE = 2.0f;
00099         inline constexpr int ANIMATION_FRAMES = 4;
00100         inline constexpr float ANIMATION_DURATION = 0.5f;
00101         inline constexpr int FRAMES_PER_ROW = 4;
00102     } // namespace Easy
00103 } // namespace Enemy
00104 namespace Explosion
00105 {
00106     inline constexpr float SPRITE_WIDTH = 32.0f;
00107     inline constexpr float SPRITE_HEIGHT = 32.0f;
00108     inline constexpr int ANIMATION_FRAMES = 4;
00109     inline constexpr float ANIMATION_DURATION = 0.1f;
00110     inline constexpr int FRAMES_PER_ROW = 4;
00111     inline constexpr float LIFETIME = 0.4f;
00112     inline constexpr float SCALE = 2.0f;
00113 } // namespace Explosion
00114 namespace Hitbox
00115 {
00116     inline constexpr float PLAYER_RADIUS = 20.0f;
00117     inline constexpr float ENEMY_RADIUS = 15.0f;
00118     inline constexpr float PROJECTILE_BASIC_RADIUS = 5.0f;
00119     inline constexpr float PROJECTILE_SUPERCHARGED_RADIUS = 8.0f;
00120 } // namespace Hitbox
00121 namespace Server
00122 {
00123     inline constexpr float SCREEN_WIDTH = 1920.0f;
00124     inline constexpr float SCREEN_HEIGHT = 1080.0f;
00125     inline constexpr float WORLD_MARGIN = 200.0f;
00126
00127     namespace Enemy
00128     {
00129         namespace Basic
00130         {

```

```

00131         inline constexpr float HEALTH = 50.0f;
00132         inline constexpr float DAMAGE = 10.0f;
00133         inline constexpr float SPEED = 200.0f;
00134         inline constexpr float SHOOT_COOLDOWN = 2.0f;
00135         inline constexpr float HITBOX_RADIUS = 15.0f;
00136     } // namespace Basic
00137
00138     namespace Advanced
00139     {
00140         inline constexpr float HEALTH = 100.0f;
00141         inline constexpr float DAMAGE = 15.0f;
00142         inline constexpr float SPEED = 150.0f;
00143         inline constexpr float SHOOT_COOLDOWN = 1.5f;
00144         inline constexpr float HITBOX_RADIUS = 15.0f;
00145         inline constexpr float SINE_FREQUENCY = 1.5f;
00146         inline constexpr float SINE_AMPLITUDE = 100.0f;
00147     } // namespace Advanced
00148
00149     namespace Boss
00150     {
00151         inline constexpr float HEALTH = 1000.0f;
00152         inline constexpr float DAMAGE = 50.0f;
00153         inline constexpr float HITBOX_RADIUS = 50.0f;
00154         inline constexpr float RAM_DAMAGE = 20.0f;
00155         inline constexpr float PHASE_2_THRESHOLD = 0.66f;
00156         inline constexpr float PHASE_3_THRESHOLD = 0.33f;
00157         inline constexpr float PHASE1_SPEED = 30.0f;
00158         inline constexpr float PHASE1_COOLDOWN = 1.5f;
00159         inline constexpr float PHASE1_SINE_FREQUENCY = 0.5f;
00160         inline constexpr float PHASE1_SINE_AMPLITUDE = 150.0f;
00161         inline constexpr float PHASE2_COOLDOWN = 1.0f;
00162         inline constexpr float PHASE3_SPEED = 50.0f;
00163         inline constexpr float PHASE3_COOLDOWN = 0.5f;
00164         inline constexpr float PHASE3_AGGRESSION = 80.0f;
00165         inline constexpr float SPREAD_INTERVAL = 1.0f;
00166         inline constexpr float SPREAD_ANGLE_DEGREES = 15.0f;
00167         inline constexpr float SPREAD_SPEED = 300.0f;
00168     } // namespace Boss
00169
00170     namespace AI
00171     {
00172         inline constexpr float SHOOT_CHANCE = 0.3f;
00173         inline constexpr float MAX_SHOOT_DISTANCE = 1500.0f;
00174         inline constexpr float AGGRESSIVE_ACTIVATION_DISTANCE = 400.0f;
00175         inline constexpr float ZIGZAG_PERIOD = 1.5f;
00176         inline constexpr float ZIGZAG_SPEED_X = 150.0f;
00177         inline constexpr float ZIGZAG_SPEED_Y = 100.0f;
00178     } // namespace AI
00179 } // namespace Enemy
00180
00181     namespace Projectile
00182     {
00183         inline constexpr float ENEMY_SPEED = 500.0f;
00184         inline constexpr float ENEMY_DAMAGE = 15.0f;
00185         inline constexpr float ENEMY_LIFETIME = 10.0f;
00186     } // namespace Projectile
00187
00188     namespace Network
00189     {
00190         inline constexpr float BROADCAST_RATE = 60.0f;
00191         inline constexpr float BROADCAST_INTERVAL = 1.0f / BROADCAST_RATE;
00192     } // namespace Network
00193 } // namespace Server
00194 } // namespace GameConfig
00195 } // namespace utl

```

19.117 /home/masina/Projects/Epitech/rtype/plugins/Game/RType/↵ Client/Multi/include/RTypeClientMulti/GameConfig.hpp File Reference

Configuration constants for the game.

Namespaces

- namespace [cli](#)
- namespace [cli::GameConfig](#)
- namespace [cli::GameConfig::Screen](#)
- namespace [cli::GameConfig::Player](#)

- namespace [cli::GameConfig::Projectile](#)
- namespace [cli::GameConfig::Projectile::Basic](#)
- namespace [cli::GameConfig::Projectile::Supercharged](#)
- namespace [cli::GameConfig::Animation](#)
- namespace [cli::GameConfig::Beam](#)
- namespace [cli::GameConfig::LoadingAnimation](#)
- namespace [cli::GameConfig::Stage](#)
- namespace [cli::GameConfig::Enemy](#)
- namespace [cli::GameConfig::Enemy::Easy](#)
- namespace [cli::GameConfig::Explosion](#)
- namespace [cli::GameConfig::Hitbox](#)

Variables

- constexpr float [cli::GameConfig::Screen::SPAWN_X](#) = 1950.0f
- constexpr float [cli::GameConfig::Screen::MIN_Y](#) = 50.0f
- constexpr float [cli::GameConfig::Screen::MAX_Y](#) = 1030.0f
- constexpr float [cli::GameConfig::Screen::REMOVE_X](#) = -100.0f
- constexpr float [cli::GameConfig::Screen::REMOVE_MIN_Y](#) = -50.0f
- constexpr float [cli::GameConfig::Screen::REMOVE_MAX_Y](#) = 1130.0f
- constexpr float [cli::GameConfig::Player::SPEED](#) = 500.0f
- constexpr float [cli::GameConfig::Player::DIAGONAL_SPEED_MULTIPLIER](#) = 0.707f
- constexpr float [cli::GameConfig::Player::SPRITE_WIDTH](#) = 33.0f
- constexpr float [cli::GameConfig::Player::SPRITE_HEIGHT](#) = 17.0f
- constexpr float [cli::GameConfig::Player::SCALE](#) = 2.0f
- constexpr int [cli::GameConfig::Player::FRAMES_PER_ROW](#) = 5
- constexpr int [cli::GameConfig::Player::TOTAL_FRAMES](#) = 5
- constexpr float [cli::GameConfig::Projectile::Basic::SPEED](#) = 800.0f
- constexpr float [cli::GameConfig::Projectile::Basic::DAMAGE](#) = 10.0f
- constexpr float [cli::GameConfig::Projectile::Basic::LIFETIME](#) = 3.0f
- constexpr float [cli::GameConfig::Projectile::Basic::SCALE](#) = 1.0f
- constexpr float [cli::GameConfig::Projectile::Basic::FIRE_COOLDOWN](#) = 0.3f
- constexpr float [cli::GameConfig::Projectile::Basic::SPRITE_WIDTH](#) = 16.0f
- constexpr float [cli::GameConfig::Projectile::Basic::SPRITE_HEIGHT](#) = 8.0f
- constexpr float [cli::GameConfig::Projectile::Supercharged::SPEED](#) = 1200.0f
- constexpr float [cli::GameConfig::Projectile::Supercharged::DAMAGE](#) = 25.0f
- constexpr float [cli::GameConfig::Projectile::Supercharged::LIFETIME](#) = 5.0f
- constexpr float [cli::GameConfig::Projectile::Supercharged::SCALE](#) = 1.5f
- constexpr float [cli::GameConfig::Projectile::Supercharged::FIRE_COOLDOWN](#) = 0.2f
- constexpr float [cli::GameConfig::Projectile::Supercharged::CHARGE_TIME](#) = 0.5f
- constexpr float [cli::GameConfig::Projectile::Supercharged::SPRITE_WIDTH](#) = 29.0f
- constexpr float [cli::GameConfig::Projectile::Supercharged::SPRITE_HEIGHT](#) = 24.0f
- constexpr int [cli::GameConfig::Projectile::Supercharged::ANIMATION_FRAMES](#) = 4
- constexpr float [cli::GameConfig::Projectile::Supercharged::ANIMATION_DURATION](#) = 0.15f
- constexpr float [cli::GameConfig::Animation::FRAME_DURATION](#) = 0.1f
- constexpr float [cli::GameConfig::Beam::MAX_CHARGE](#) = 1.0f
- constexpr float [cli::GameConfig::Beam::CHARGE_RATE](#) = 1.0f
- constexpr float [cli::GameConfig::Beam::BAR_WIDTH](#) = 120.0f
- constexpr float [cli::GameConfig::Beam::BAR_HEIGHT](#) = 12.0f
- constexpr float [cli::GameConfig::Beam::BAR_X](#) = 10.0f
- constexpr float [cli::GameConfig::Beam::BAR_Y](#) = 10.0f
- constexpr float [cli::GameConfig::LoadingAnimation::SPRITE_WIDTH](#) = 29.0f
- constexpr float [cli::GameConfig::LoadingAnimation::SPRITE_HEIGHT](#) = 24.0f
- constexpr int [cli::GameConfig::LoadingAnimation::ANIMATION_FRAMES](#) = 4
- constexpr float [cli::GameConfig::LoadingAnimation::ANIMATION_DURATION](#) = 0.15f

- constexpr float cli::GameConfig::LoadingAnimation::OFFSET_X = 60.0f
- constexpr float cli::GameConfig::LoadingAnimation::OFFSET_Y = 6.0f
- constexpr float cli::GameConfig::Stage::FLOOR_OFFSET_Y = 16.0f
- constexpr float cli::GameConfig::Stage::CEILING_OFFSET_Y = -1.0f
- constexpr float cli::GameConfig::Enemy::Easy::HEALTH = 1.0f
- constexpr float cli::GameConfig::Enemy::Easy::DAMAGE = 5.0f
- constexpr float cli::GameConfig::Enemy::Easy::SPEED = 80.0f
- constexpr float cli::GameConfig::Enemy::Easy::SPRITE_WIDTH = 32.0f
- constexpr float cli::GameConfig::Enemy::Easy::SPRITE_HEIGHT = 32.0f
- constexpr float cli::GameConfig::Enemy::Easy::SCALE = 2.0f
- constexpr float cli::GameConfig::Enemy::Easy::SHOOT_COOLDOWN = 2.0f
- constexpr float cli::GameConfig::Enemy::Easy::SPAWN_RATE = 2.0f
- constexpr int cli::GameConfig::Enemy::Easy::ANIMATION_FRAMES = 4
- constexpr float cli::GameConfig::Enemy::Easy::ANIMATION_DURATION = 0.5f
- constexpr int cli::GameConfig::Enemy::Easy::FRAMES_PER_ROW = 4
- constexpr float cli::GameConfig::Explosion::SPRITE_WIDTH = 32.0f
- constexpr float cli::GameConfig::Explosion::SPRITE_HEIGHT = 32.0f
- constexpr int cli::GameConfig::Explosion::ANIMATION_FRAMES = 4
- constexpr float cli::GameConfig::Explosion::ANIMATION_DURATION = 0.1f
- constexpr int cli::GameConfig::Explosion::FRAMES_PER_ROW = 4
- constexpr float cli::GameConfig::Explosion::LIFETIME = 0.4f
- constexpr float cli::GameConfig::Explosion::SCALE = 2.0f
- constexpr float cli::GameConfig::Hitbox::PLAYER_RADIUS = 20.0f
- constexpr float cli::GameConfig::Hitbox::ENEMY_RADIUS = 15.0f
- constexpr float cli::GameConfig::Hitbox::PROJECTILE_BASIC_RADIUS = 5.0f
- constexpr float cli::GameConfig::Hitbox::PROJECTILE_SUPERCHARGED_RADIUS = 8.0f

19.117.1 Detailed Description

Configuration constants for the game.

Definition in file [GameConfig.hpp](#).

19.118 GameConfig.hpp

[Go to the documentation of this file.](#)

```

00001 ///
00002 /// @file GameConfig.hpp
00003 /// @brief Configuration constants for the game
00004 /// @namespace cli
00005 ///
00006
00007 #pragma once
00008
00009 namespace cli
00010 {
00011     namespace GameConfig
00012     {
00013         namespace Screen
00014         {
00015             inline constexpr float SPAWN_X = 1950.0f;
00016             inline constexpr float MIN_Y = 50.0f;
00017             inline constexpr float MAX_Y = 1030.0f;
00018             inline constexpr float REMOVE_X = -100.0f;
00019             inline constexpr float REMOVE_MIN_Y = -50.0f;
00020             inline constexpr float REMOVE_MAX_Y = 1130.0f;
00021         } // namespace Screen
00022         namespace Player
00023         {
00024             inline constexpr float SPEED = 500.0f;
00025             inline constexpr float DIAGONAL_SPEED_MULTIPLIER = 0.707f;
00026             inline constexpr float SPRITE_WIDTH = 33.0f;
00027             inline constexpr float SPRITE_HEIGHT = 17.0f;
00028             inline constexpr float SCALE = 2.0f;
00029             inline constexpr int FRAMES_PER_ROW = 5;
00030             inline constexpr int TOTAL_FRAMES = 5;
00031         } // namespace Player
00032         namespace Projectile

```

```

00033     {
00034         namespace Basic
00035         {
00036             inline constexpr float SPEED = 800.0f;
00037             inline constexpr float DAMAGE = 10.0f;
00038             inline constexpr float LIFETIME = 3.0f;
00039             inline constexpr float SCALE = 1.0f;
00040             inline constexpr float FIRE_COOLDOWN = 0.3f;
00041             inline constexpr float SPRITE_WIDTH = 16.0f;
00042             inline constexpr float SPRITE_HEIGHT = 8.0f;
00043         } // namespace Basic
00044
00045         namespace Supercharged
00046         {
00047             inline constexpr float SPEED = 1200.0f;
00048             inline constexpr float DAMAGE = 25.0f;
00049             inline constexpr float LIFETIME = 5.0f;
00050             inline constexpr float SCALE = 1.5f;
00051             inline constexpr float FIRE_COOLDOWN = 0.2f;
00052             inline constexpr float CHARGE_TIME = 0.5f;
00053             inline constexpr float SPRITE_WIDTH = 29.0f;
00054             inline constexpr float SPRITE_HEIGHT = 24.0f;
00055             inline constexpr int ANIMATION_FRAMES = 4;
00056             inline constexpr float ANIMATION_DURATION = 0.15f;
00057         } // namespace Supercharged
00058     } // namespace Projectile
00059     namespace Animation
00060     {
00061         inline constexpr float FRAME_DURATION = 0.1f;
00062     }
00063     namespace Beam
00064     {
00065         inline constexpr float MAX_CHARGE = 1.0f;
00066         inline constexpr float CHARGE_RATE = 1.0f;
00067         inline constexpr float BAR_WIDTH = 120.0f;
00068         inline constexpr float BAR_HEIGHT = 12.0f;
00069         inline constexpr float BAR_X = 10.0f;
00070         inline constexpr float BAR_Y = 10.0f;
00071     } // namespace Beam
00072     namespace LoadingAnimation
00073     {
00074         inline constexpr float SPRITE_WIDTH = 29.0f;
00075         inline constexpr float SPRITE_HEIGHT = 24.0f;
00076         inline constexpr int ANIMATION_FRAMES = 4;
00077         inline constexpr float ANIMATION_DURATION = 0.15f;
00078         inline constexpr float OFFSET_X = 60.0f;
00079         inline constexpr float OFFSET_Y = 6.0f;
00080     } // namespace LoadingAnimation
00081     namespace Stage
00082     {
00083         inline constexpr float FLOOR_OFFSET_Y = 16.0f; // remonte le sol par rapport au bas
00084         inline constexpr float CEILING_OFFSET_Y = -1.0f; // descend le plafond depuis le haut
00085     } // namespace Stage
00086     namespace Enemy
00087     {
00088         namespace Easy
00089         {
00090             inline constexpr float HEALTH = 1.0f;
00091             inline constexpr float DAMAGE = 5.0f;
00092             inline constexpr float SPEED = 80.0f;
00093             inline constexpr float SPRITE_WIDTH = 32.0f;
00094             inline constexpr float SPRITE_HEIGHT = 32.0f;
00095             inline constexpr float SCALE = 2.0f;
00096             inline constexpr float SHOOT_COOLDOWN = 2.0f;
00097             inline constexpr float SPAWN_RATE = 2.0f;
00098             inline constexpr int ANIMATION_FRAMES = 4;
00099             inline constexpr float ANIMATION_DURATION = 0.5f;
00100             inline constexpr int FRAMES_PER_ROW = 4;
00101         } // namespace Easy
00102     } // namespace Enemy
00103     namespace Explosion
00104     {
00105         inline constexpr float SPRITE_WIDTH = 32.0f;
00106         inline constexpr float SPRITE_HEIGHT = 32.0f;
00107         inline constexpr int ANIMATION_FRAMES = 4;
00108         inline constexpr float ANIMATION_DURATION = 0.1f;
00109         inline constexpr int FRAMES_PER_ROW = 4;
00110         inline constexpr float LIFETIME = 0.4f;
00111         inline constexpr float SCALE = 2.0f;
00112     } // namespace Explosion
00113     namespace Hitbox
00114     {
00115         inline constexpr float PLAYER_RADIUS = 20.0f;
00116         inline constexpr float ENEMY_RADIUS = 15.0f;
00117         inline constexpr float PROJECTILE_BASIC_RADIUS = 5.0f;
00118         inline constexpr float PROJECTILE_SUPERCHARGED_RADIUS = 8.0f;
00119     } // namespace Hitbox

```

```
00120 } // namespace GameConfig
00121 } // namespace cli
```

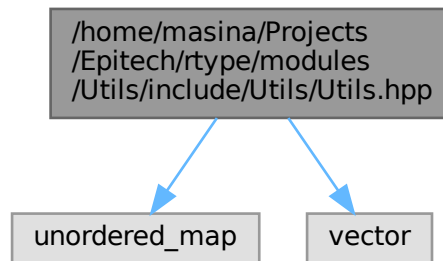
19.119 /home/masina/Projects/Epitech/rtype/modules/Utils/include/↵ Utils/Utils.hpp File Reference

This file contains utility functions.

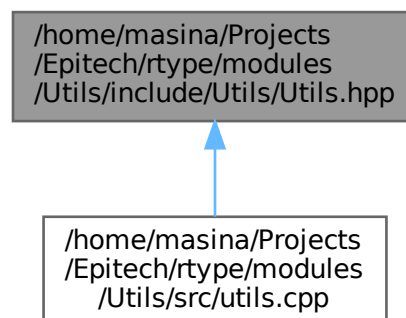
```
#include <unordered_map>
```

```
#include <vector>
```

Include dependency graph for Utils.hpp:



This graph shows which files directly or indirectly include this file:



Namespaces

- namespace [utl](#)

Functions

- `std::vector< char >` [utl::readFile](#) (const std::string &filename)
- `std::unordered_map< std::string, std::string >` [utl::getEnvMap](#) (const char *const *env)

19.119.1 Detailed Description

This file contains utility functions.

Definition in file [Utils.hpp](#).

19.120 Utils.hpp

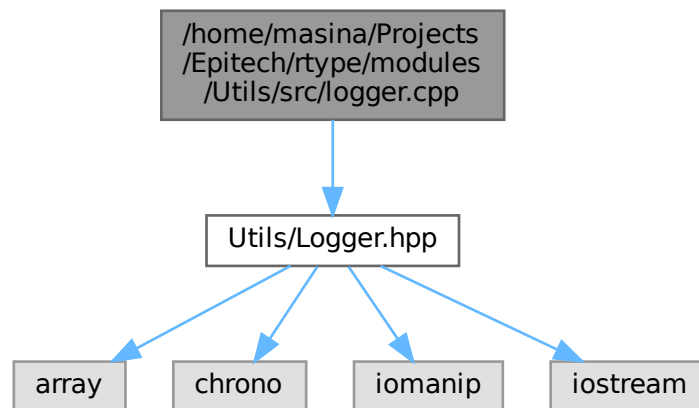
[Go to the documentation of this file.](#)

```
00001 ///  
00002 ///  
00003 ///  
00004 ///  
00005 ///  
00006 ///  
00007 #pragma once  
00008  
00009 #include <unordered_map>  
00010 #include <vector>  
00011  
00012 namespace utl  
00013 {  
00014  
00015     [[nodiscard]] std::vector<char> readFile(const std::string &filename);  
00016     [[nodiscard]] std::unordered_map<std::string, std::string> getEnvMap(const char *const *env);  
00017  
00018 } // namespace utl
```

19.121 /home/masina/Projects/Epitech/rtype/modules/ Utils/src/logger.cpp File Reference

#include "Utils/Logger.hpp"

Include dependency graph for logger.cpp:



19.122 logger.cpp

[Go to the documentation of this file.](#)

```
00001 #ifdef _WIN32  
00002 #include <windows.h>  
00003 #endif  
00004  
00005 #include "Utils/Logger.hpp"  
00006  
00007 void utl::Logger::init()  
00008 {
```

```

00009 #ifdef _WIN32
00010     const HANDLE hOut = GetStdHandle(STD_OUTPUT_HANDLE);
00011     DWORD dwMode = 0;
00012     if (hOut != INVALID_HANDLE_VALUE && GetConsoleMode(hOut, &dwMode))
00013     {
00014         SetConsoleMode(hOut, dwMode | ENABLE_VIRTUAL_TERMINAL_PROCESSING);
00015     }
00016 #endif
00017 }

```

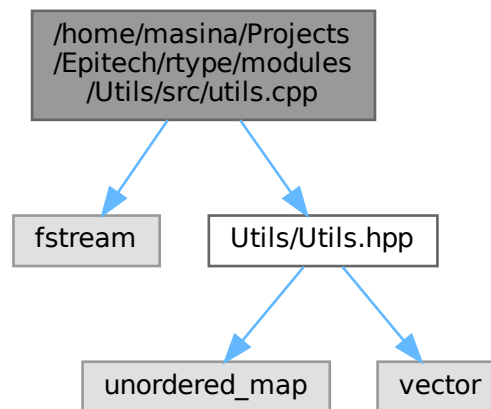
19.123 /home/masina/Projects/Epitech/rtype/modules/↵ Utils/src/Utils.cpp File Reference

```

#include <fstream>
#include "Utils/Utils.hpp"

```

Include dependency graph for utils.cpp:



19.124 utils.cpp

[Go to the documentation of this file.](#)

```

00001 #include <fstream>
00002
00003 #ifdef _WIN32
00004 #include <windows.h>
00005 #endif
00006
00007 #include "Utils/Utils.hpp"
00008
00009 std::vector<char> utl::readFile(const std::string &filename)
00010 {
00011     std::ifstream file(filename, std::ios::binary | std::ios::ate);
00012     if (!file.is_open())
00013     {
00014         throw std::runtime_error("failed to open file " + filename);
00015     }
00016     const long int fileSize = file.tellg();
00017     if (fileSize <= 0)
00018     {
00019         throw std::runtime_error("file " + filename + " is empty");
00020     }
00021     std::vector<char> buffer(static_cast<long unsigned int>(fileSize));
00022     file.seekg(0, std::ios::beg);
00023     if (!file.read(buffer.data(), fileSize))
00024     {
00025         throw std::runtime_error("failed to read file " + filename);
00026     }
00027     return buffer;

```

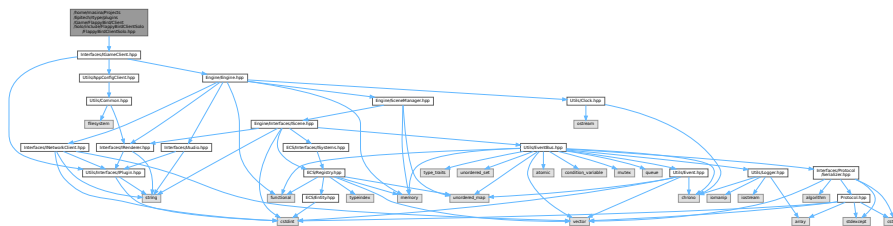
```
00028 }
00029
00030 std::unordered_map<std::string, std::string> utl::getEnvMap(const char *const *env)
00031 {
00032     std::unordered_map<std::string, std::string> cpyEnv;
00033
00034     #ifdef __WIN32
00035         LPCH envStrings = GetEnvironmentStringsA();
00036         if (!envStrings)
00037         {
00038             return cpyEnv;
00039         }
00040
00041         for (LPCH var = envStrings; *var; var += std::strlen(var) + 1)
00042         {
00043             std::string entry(var);
00044             if (const auto pos = entry.find('='); pos != std::string::npos)
00045             {
00046                 cpyEnv.emplace(entry.substr(0, pos), entry.substr(pos + 1));
00047             }
00048         }
00049
00050         FreeEnvironmentStringsA(envStrings);
00051     #else
00052         for (const char *const *current = env; (current != nullptr) && (*current != nullptr); ++current)
00053         {
00054             std::string entry(*current);
00055             if (const auto pos = entry.find('='); pos != std::string::npos)
00056             {
00057                 cpyEnv.emplace(entry.substr(0, pos), entry.substr(pos + 1));
00058             }
00059         }
00060     #endif
00061
00062     return cpyEnv;
00063 }
```

19.125 /home/masina/Projects/Epitech/rtype/plugins/Game/Flappy↵ Bird/Client/Solo/include/FlappyBirdClientSolo/FlappyBird↵ ClientSolo.hpp File Reference

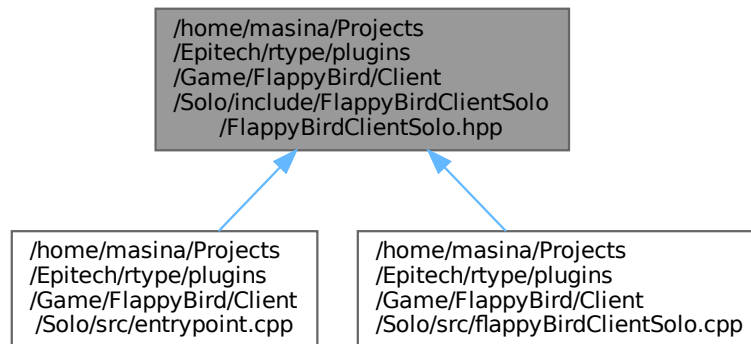
This file contains the FlappyBirdClientSolo class declaration.

#include "Interfaces/IGameClient.hpp"

Include dependency graph for FlappyBirdClientSolo.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [gme::FlappyBirdClientSolo](#)
Class for the [FlappyBirdClientSolo](#) plugin.

Namespaces

- namespace [gme](#)

19.125.1 Detailed Description

This file contains the [FlappyBirdClientSolo](#) class declaration.

Definition in file [FlappyBirdClientSolo.hpp](#).

19.126 FlappyBirdClientSolo.hpp

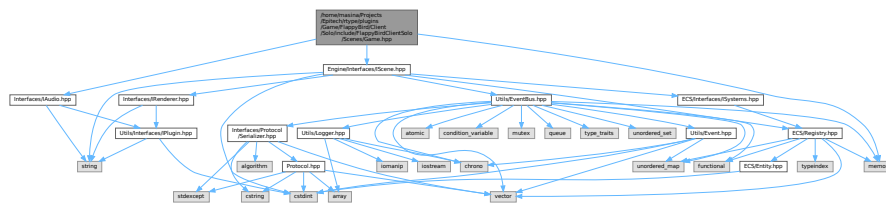
[Go to the documentation of this file.](#)

```

00001 ///
00002 /// @file FlappyBirdClientSolo.hpp
00003 /// @brief This file contains the FlappyBirdClientSolo class declaration
00004 /// @namespace gme
00005 ///
00006
00007 #pragma once
00008
00009 #include "Interfaces/IGameClient.hpp"
00010
00011 namespace gme
00012 {
00013
00014     ///
00015     /// @class FlappyBirdClientSolo
00016     /// @brief Class for the FlappyBirdClientSolo plugin
00017     /// @namespace gme
00018     ///
00019     class FlappyBirdClientSolo final : public IGameClient
00020     {
00021
00022     public:
00023         FlappyBirdClientSolo() = default;
00024         ~FlappyBirdClientSolo() override = default;
00025
00026         FlappyBirdClientSolo(const FlappyBirdClientSolo &) = delete;
00027         FlappyBirdClientSolo &operator=(const FlappyBirdClientSolo &) = delete;
00028         FlappyBirdClientSolo(FlappyBirdClientSolo &&) = delete;
00029         FlappyBirdClientSolo &operator=(FlappyBirdClientSolo &&) = delete;
00030
00031         [[nodiscard]] const std::string getName() const override { return "RType_Client_Solo"; }
00032         [[nodiscard]] utl::PluginType getType() const override { return utl::PluginType::GAME_CLIENT; }
  
```


19.127 /home/masina/Projects/Epitech/rtype/plugins/Game/FlappyBird/Client/Solo/include/FlappyBirdClientSolo/Scenes/Game.hpp File Reference

```
#include <memory>
#include "Engine/Interfaces/IScene.hpp"
#include "Interfaces/IAudio.hpp"
Include dependency graph for Game.hpp:
```



```
graph BT; A["/home/masina/Projects/Epitech/rtype/plugins/Game/FlappyBird/Client/Solo/src/flappyBirdClientSolo.cpp"] --> C["/home/masina/Projects/Epitech/rtype/plugins/Game/FlappyBird/Client/Solo/include/FlappyBirdClientSolo/Scenes/Game.hpp"]; B["/home/masina/Projects/Epitech/rtype/plugins/Game/FlappyBird/Client/Solo/src/scenes/game.cpp"] --> C;
```

The diagram illustrates the relationship between source files and a header file. At the bottom, there are two source files:

- `/home/masina/Projects/Epitech/rtype/plugins/Game/FlappyBird/Client/Solo/src/flappyBirdClientSolo.cpp`
- `/home/masina/Projects/Epitech/rtype/plugins/Game/FlappyBird/Client/Solo/src/scenes/game.cpp`

Both source files have arrows pointing to a single header file at the top:

- `/home/masina/Projects/Epitech/rtype/plugins/Game/FlappyBird/Client/Solo/include/FlappyBirdClientSolo/Scenes/Game.hpp`

- class gme::Game

[Game](#) solo scene.

Namespaces

- namespace [gme](#)

19.127.1 Detailed Description

This file contains the solo Game scene.

Definition in file [Game.hpp](#).

19.128 Game.hpp

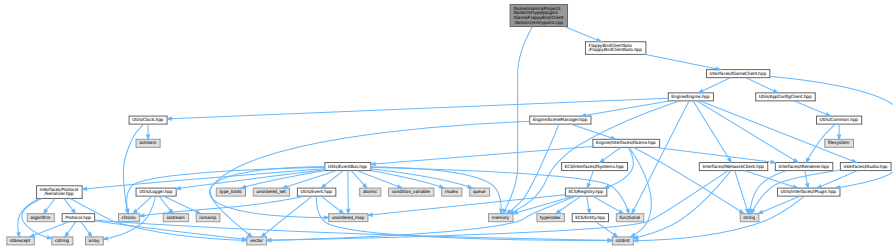
[Go to the documentation of this file.](#)

```
00001 ///  
00002 ///  
00003 ///  
00004 ///  
00005 ///  
00006 ///  
00007 #pragma once  
00008 ///  
00009 #include <memory>  
00010 ///  
00011 #include "Engine/Interfaces/IScene.hpp"  
00012 #include "Interfaces/IAudio.hpp"  
00013 ///  
00014 namespace gme  
00015 {  
00016     ///  
00017     ///  
00018     ///  
00019     ///  
00020     ///  
00021     ///  
00022     class Game final : public eng::AScene  
00023     {  
00024     public:  
00025         Game(eng::id assignedId, const std::shared_ptr<eng::IRenderer> &renderer,  
00026             const std::shared_ptr<eng::IAudio> &audio, bool &showDebug);  
00027         ~Game() override = default;  
00028         Game(const Game &other) = delete;  
00029         Game &operator=(const Game &other) = delete;  
00030         Game(Game &&other) = delete;  
00031         Game &operator=(Game &&other) = delete;  
00032         void update(float dt, const eng::WindowSize &size) override;  
00033         void event(const eng::Event &event) override;  
00034         bool &playMusic() { return m_playMusic; }  
00035     private:  
00036         static ecs::Entity createPlayer(ecs::Registry &registry);  
00037         std::pair<ecs::Entity, ecs::Entity> createPipePair(ecs::Registry &registry, float x, float gapY) const;  
00038         void resetGame();  
00039         const std::shared_ptr<eng::IRenderer> &m_renderer;  
00040         const std::shared_ptr<eng::IAudio> &m_audio;  
00041         std::unordered_map<eng::Key, bool> m_keysPressed;  
00042         ecs::Entity m_playerEntity;  
00043         ecs::Entity m_looseText;  
00044         ecs::Entity m_flapSound;  
00045         ecs::Entity m_looseSound;  
00046         std::vector<std::pair<ecs::Entity, ecs::Entity>> m_pipes;  
00047         bool &m_showDebug;  
00048         bool m_playMusic = false;  
00049         bool m_gameOver = false;  
00050         bool m_gameOverShown = false;  
00051     }; // class Game  
00052 } // namespace gme
```

19.129 /home/masina/Projects/Epitech/rtype/plugins/Game/FlappyBird/Client/Solo/src/entrypoint.cpp File Reference

```
#include <memory>  
#include "FlappyBirdClientSolo/FlappyBirdClientSolo.hpp"
```

Include dependency graph for entrypt.cpp:



Functions

- `PLUGIN_EXPORT gme::IGameClient * entrypt ()`

19.129.1 Function Documentation

19.129.1.1 entrypt()

`PLUGIN_EXPORT gme::IGameClient * entrypt ()`

Definition at line 7 of file `entrypt.cpp`.

19.130 entrypt.cpp

[Go to the documentation of this file.](#)

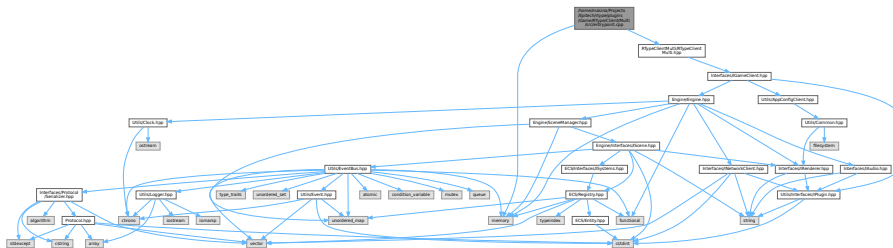
```
00001 #include <memory>
00002
00003 #include "FlappyBirdClientSolo/FlappyBirdClientSolo.hpp"
00004
00005 extern "C"
00006 {
00007     PLUGIN_EXPORT gme::IGameClient *entrypt() { return
std::make_unique<gme::FlappyBirdClientSolo>().release(); }
00008 }
```

19.131 /home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/src/entrypt.cpp File Reference

`#include <memory>`

`#include "RTypeClientMulti/RTypeClientMulti.hpp"`

Include dependency graph for entrypt.cpp:



Functions

- `PLUGIN_EXPORT gme::IGameClient * entrypt ()`

19.135 /home/masina/Projects/Epitech/rtype/plugins/Game/RType/↵
Server/src/entrypoint.cpp File Reference

```
#include <memory>
#include "RTypeServer/RTypeServer.hpp"
Include dependency graph for entrypt.cpp:
```



Functions

- `PLUGIN_EXPORT gme::IGameServer * entryPoint ()`

19.135.1 Function Documentation

19.135.1.1 entryPoint()

PLUGIN_EXPORT gme::IGameServer * entryPoint ()
Definition at line 7 of file entrypt.cpp.

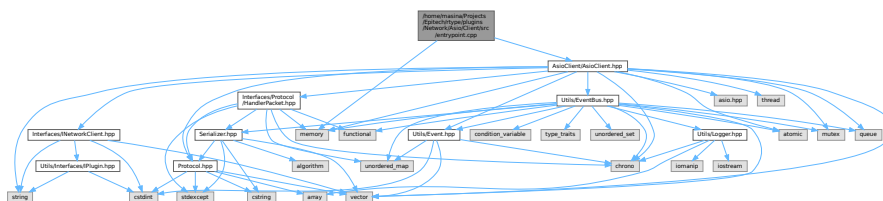
19.136 entrypt.cpp

[Go to the documentation of this file.](#)

```
00001 #include <memory>
00002
00003 #include "RTypeServer/RTypeServer.hpp"
00004
00005 extern "C"
00006 {
00007     PLUGIN_EXPORT gme::IGameServer *entryPoint() { return std::make_unique<gme::RTypeServer>().release(); }
00008 }
```

19.137 /home/masina/Projects/Epitech/rtype/plugins/Network/Asio/↵
Client/src/entrypoint.cpp File Reference

```
#include <memory>
#include "AsioClient/AsioClient.hpp"
Include dependency graph for entypoint.cpp:
```

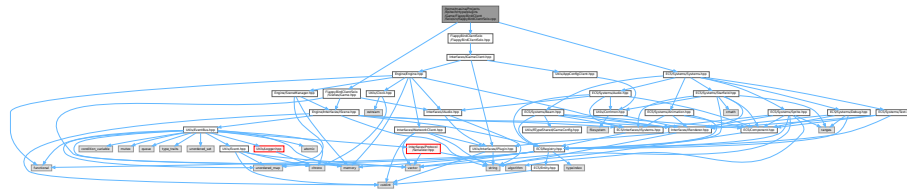


Functions

- `PLUGIN_EXPORT eng::INetworkClient * entryPoint ()`

19.141 /home/masina/Projects/Epitech/rtype/plugins/Game/FlappyBird/Client/Solo/src/flappyBirdClientSolo.cpp File Reference

```
#include "FlappyBirdClientSolo/FlappyBirdClientSolo.hpp"
#include "ECS/Systems/Systems.hpp"
#include "FlappyBirdClientSolo/Scenes/Game.hpp"
Include dependency graph for flappyBirdClientSolo.cpp:
```



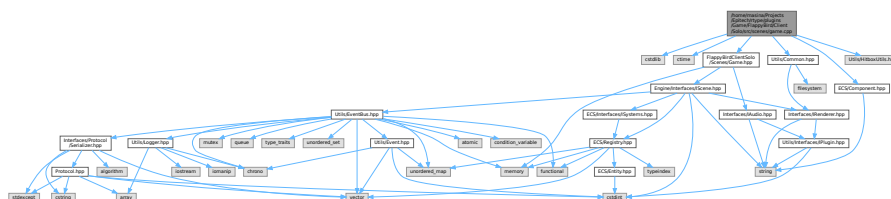
19.142 flappyBirdClientSolo.cpp

[Go to the documentation of this file.](#)

```
00001 #include "FlappyBirdClientSolo/FlappyBirdClientSolo.hpp"
00002 #include "ECS/Systems/Systems.hpp"
00003 #include "FlappyBirdClientSolo/Scenes/Game.hpp"
00004
00005 void gme::FlappyBirdClientSolo::update(float deltaTime, unsigned int width, unsigned int height) {}
00006
00007 void gme::FlappyBirdClientSolo::setupScenes(bool &showDebug, eng::id menuSceneId)
00008 {
00009     auto gameSoloId = m_engine->getSceneManager()->generateNextId();
00010     auto gameSolo = std::make_unique<Game>(gameSoloId, m_engine->getRenderer(), m_engine->getAudio(),
00011     showDebug);
00012     gameSolo->addSystem(std::make_unique<ecs::AudioSystem>(m_engine->getAudio(), m_appConfig->audioVolume,
00013     gameSolo->getRegistry(), gameSolo->playMusic()));
00014     gameSolo->addSystem(std::make_unique<ecs::StarfieldSystem>(m_engine->getRenderer(), gameSolo->getRegistry()));
00015     gameSolo->addSystem(std::make_unique<ecs::SpriteSystem>(m_engine->getRenderer()));
00016     gameSolo->addSystem(std::make_unique<ecs::TextSystem>(m_engine->getRenderer()));
00017     m_mainSceneId = gameSoloId;
00018
00019     m_engine->getSceneManager()->addScene(std::move(gameSolo));
00020 }
```

19.143 /home/masina/Projects/Epitech/rtype/plugins/Game/FlappyBird/Client/Solo/src/scenes/game.cpp File Reference

```
#include <cstdlib>
#include <ctime>
#include "ECS/Component.hpp"
#include "FlappyBirdClientSolo/Scenes/Game.hpp"
#include "Utils/Common.hpp"
#include "Utils/HitboxUtils.hpp"
Include dependency graph for game.cpp:
```



Enumerations

- enum class [PipeType](#) { [Small](#) , [Medium](#) , [Large](#) }

Functions

- [PipeType](#) [randomPipeType](#) ()
- float [getGapSize](#) (const [PipeType](#) type)

19.143.1 Enumeration Type Documentation

19.143.1.1 PipeType

enum class [PipeType](#) [strong]

Enumerator

Small	
Medium	
Large	

Definition at line 9 of file [game.cpp](#).

19.143.2 Function Documentation

19.143.2.1 getGapSize()

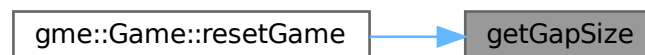
float [getGapSize](#) (
 const [PipeType](#) type)

Definition at line 26 of file [game.cpp](#).

References [Large](#), [Medium](#), and [Small](#).

Referenced by [gme::Game::resetGame\(\)](#).

Here is the caller graph for this function:



19.143.2.2 randomPipeType()

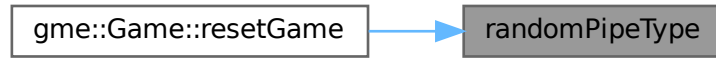
[PipeType](#) [randomPipeType](#) ()

Definition at line 16 of file [game.cpp](#).

References [Large](#), [Medium](#), and [Small](#).

Referenced by [gme::Game::resetGame\(\)](#).

Here is the caller graph for this function:



19.144 game.cpp

[Go to the documentation of this file.](#)

```

00001 #include <cstdlib>
00002 #include <ctime>
00003
00004 #include "ECS/Component.hpp"
00005 #include "FlappyBirdClientSolo/Scenes/Game.hpp"
00006 #include "Utils/Common.hpp"
00007 #include "Utils/HitboxUtils.hpp"
00008
00009 enum class PipeType
00010 {
00011     Small,
00012     Medium,
00013     Large
00014 };
00015
00016 PipeType randomPipeType()
00017 {
00018     const int r = rand() % 3;
00019     if (r == 0)
00020         return PipeType::Small;
00021     if (r == 1)
00022         return PipeType::Medium;
00023     return PipeType::Large;
00024 }
00025
00026 float getGapSize(const PipeType type)
00027 {
00028     switch (type)
00029     {
00030     case PipeType::Small:
00031         return 140.f;
00032     case PipeType::Medium:
00033         return 180.f;
00034     case PipeType::Large:
00035         return 220.f;
00036     }
00037     return 180.f;
00038 }
00039
00040 gme::Game::Game(const eng::id assignedId, const std::shared_ptr<eng::IRenderer> &renderer,
00041                 const std::shared_ptr<eng::IAudio> &audio, bool &showDebug)
00042 : AScene(assignedId, m_renderer(renderer), m_audio(audio), m_showDebug(showDebug))
00043 {
00044     auto &registry = AScene::getRegistry();
00045
00046     registry.onComponentAdded(
00047         [&renderer, &audio, &registry](const ecs::Entity e, const std::type_info &type)
00048         {
00049             const auto *audioComp = registry.getComponent<ecs::Audio>(e);
00050             const auto *colorComp = registry.getComponent<ecs::Color>(e);
00051             const auto *fontComp = registry.getComponent<ecs::Font>(e);
00052             const auto *rectComp = registry.getComponent<ecs::Rect>(e);
00053             const auto *scaleComp = registry.getComponent<ecs::Scale>(e);
00054             const auto *textComp = registry.getComponent<ecs::Text>(e);
00055             const auto *textureComp = registry.getComponent<ecs::Texture>(e);
00056             const auto *transform = registry.getComponent<ecs::Transform>(e);
00057             const auto *hitBox = registry.getComponent<ecs::Hitbox>(e);
00058
00059             if (hitBox && transform)
00060             {
00061                 renderer->createCircleShape({.name = "hitbox_" + std::to_string(e),
00062                                             .radius = hitBox->radius,
00063                                             .color = {.r = 255, .g = 0, .b = 0, .a = 100},
00064                                             .x = transform->x + hitBox->offsetX,
  
```

```

00065         .y = transform->y + hitBox->offsetY,
00066         .outline_thickness = 1.0f,
00067         .outline_color = {.r = 255, .g = 0, .b = 0, .a = 200});
00068     }
00069
00070     if (type == typeid(ecs::Text))
00071     {
00072         if (textComp && transform && fontComp)
00073         {
00074             renderer->createFont(fontComp->id, fontComp->path);
00075             renderer->createText(
00076                 {.font_name = fontComp->id,
00077                  .color = {.r = colorComp->r, .g = colorComp->g, .b = colorComp->b, .a = colorComp->a},
00078                  .content = textComp->content,
00079                  .size = textComp->font_size,
00080                  .x = transform->x,
00081                  .y = transform->y,
00082                  .name = textComp->id});
00083         }
00084     }
00085     else if (type == typeid(ecs::Texture))
00086     {
00087         const float scale_x = scaleComp ? scaleComp->x : 1.F;
00088         const float scale_y = scaleComp ? scaleComp->y : 1.F;
00089
00090         renderer->createTexture(textureComp->id, textureComp->path);
00091
00092         if (transform && textureComp)
00093         {
00094             if (rectComp)
00095             {
00096                 renderer->createSprite(textureComp->id + std::to_string(e), textureComp->id, transform->x,
00097                                         transform->y, scale_x, scale_y, static_cast<int>(rectComp->pos_x),
00098                                         static_cast<int>(rectComp->pos_y), rectComp->size_x, rectComp->size_y);
00099             }
00100             else
00101             {
00102                 renderer->createSprite(textureComp->id + std::to_string(e), textureComp->id, transform->x,
00103                                         transform->y);
00104             }
00105         }
00106     }
00107     else if (type == typeid(ecs::Audio))
00108     {
00109         if (audioComp)
00110         {
00111             audio->createAudio(audioComp->path, audioComp->volume, audioComp->loop,
00112                               audioComp->id + std::to_string(e));
00113         }
00114     }
00115 });
00116
00117 m_playerEntity = createPlayer(registry);
00118
00119 auto [width, height] = m_renderer->getWindowSize();
00120 m_pipes.emplace_back(createPipePair(registry, width + 200.f, height / 2.f));
00121 m_pipes.emplace_back(createPipePair(registry, width + 600.f, height / 2.f + 50.f));
00122 m_pipes.emplace_back(createPipePair(registry, width + 1000.f, height / 2.f - 50.f));
00123 m_looseText =
00124     registry.createEntity()
00125     .with<ecs::Font>("main_font", utl::Path::Font::FONTS_RTYPE)
00126     .with<ecs::Transform>("transform_loose", 0.0F, height / 2.0F, 0.F)
00127     .with<ecs::Color>("color_loose", utl::Config::Color::CYAN_ELECTRIC.r,
00128                      utl::Config::Color::CYAN_ELECTRIC.g,
00129                      utl::Config::Color::CYAN_ELECTRIC.b, utl::Config::Color::CYAN_ELECTRIC.a)
00130     .with<ecs::Text>("id_loose", std::string("You loose, press R to restart"), 24U)
00131     .build();
00132 m_looseSound = registry.createEntity()
00133     .with<ecs::Audio>("audio" + std::to_string(m_looseSound), "assets/audio/flappy-die.ogg", 10.0F,
00134                      false, false)
00135     .build();
00136 m_flapSound = registry.createEntity()
00137     .with<ecs::Audio>("audio" + std::to_string(m_flapSound), "assets/audio/flappy-flap.ogg", 10.0F,
00138                      false, false)
00139     .build();
00140
00141 void gme::Game::update(const float dt, const eng::WindowSize &size)
00142 {
00143     auto &registry = AScene::getRegistry();
00144     auto *transformLooseText = registry.getComponent<ecs::Transform>(m_looseText);
00145     auto *soundLoose = registry.getComponent<ecs::Audio>(m_looseSound);
00146
00147     transformLooseText->x = 90000.0F;
00148
00149     if (m_gameOver)
00150     {

```

```

00151     transformLooseText->x = 100.0f;
00152     if (!m_gameOverShown)
00153     {
00154         soundLoose->play = true;
00155         m_gameOverShown = true;
00156     }
00157     return;
00158 }
00159
00160 auto *transform = registry.getComponent<ecs::Transform>(m_playerEntity);
00161 auto *velocity = registry.getComponent<ecs::Velocity>(m_playerEntity);
00162
00163 if (transform && velocity)
00164 {
00165     constexpr float gravity = 900.f;
00166     velocity->y += gravity * dt;
00167     transform->y += velocity->y * dt;
00168
00169     if (transform->y > size.height - 50)
00170     {
00171         transform->y = size.height - 50;
00172         velocity->y = 0.f;
00173         m_gameOver = true;
00174     }
00175     if (transform->y < 0)
00176     {
00177         transform->y = 0;
00178         velocity->y = 0.f;
00179     }
00180
00181     m_renderer->setSpritePosition("player_texture" + std::to_string(m_playerEntity), transform->x, transform->y);
00182 }
00183
00184 for (auto &[topPipe, bottomPipe] : m_pipes)
00185 {
00186     auto *topTransform = registry.getComponent<ecs::Transform>(topPipe);
00187     auto *botTransform = registry.getComponent<ecs::Transform>(bottomPipe);
00188     const auto *topVel = registry.getComponent<ecs::Velocity>(topPipe);
00189
00190     if (const auto *botVel = registry.getComponent<ecs::Velocity>(bottomPipe);
00191         topTransform && botTransform && topVel && botVel)
00192     {
00193         topTransform->x += topVel->x * dt;
00194         botTransform->x += botVel->x * dt;
00195
00196         if (topTransform->x < -150)
00197         {
00198             topTransform->x = size.width + 200.f;
00199             botTransform->x = size.width + 200.f;
00200
00201             topTransform->y = 0.0f;
00202             botTransform->y = size.height - 320.f;
00203         }
00204
00205         m_renderer->setSpritePosition("pipe_top_texture" + std::to_string(topPipe), topTransform->x,
00206                                     topTransform->y);
00207         m_renderer->setSpritePosition("pipe_bottom_texture" + std::to_string(bottomPipe), botTransform->x,
00208                                     botTransform->y);
00209     }
00210 }
00211 }
00212
00213 void gme::Game::event(const eng::Event &event)
00214 {
00215     auto &registry = AScene::getRegistry();
00216     auto *soundFlap = registry.getComponent<ecs::Audio>(m_flapSound);
00217
00218     switch (event.type)
00219     {
00220     case eng::EventType::KeyPressed:
00221         if (event.key == eng::Key::Space && !m_gameOver)
00222         {
00223             soundFlap->play = true;
00224             if (auto *vel = registry.getComponent<ecs::Velocity>(m_playerEntity))
00225                 vel->y = -300.f;
00226         }
00227     else if (event.key == eng::Key::R && m_gameOver)
00228     {
00229         resetGame();
00230     }
00231     break;
00232
00233     default:
00234         break;
00235     }
00236 }
00237

```

```

00238 ecs::Entity gme::Game::createPlayer(ecs::Registry &registry)
00239 {
00240     auto [offsetX, offsetY] = utl::calculateHitboxOffsets(33.0F, 17.0F, 2.0F);
00241
00242     return registry.createEntity()
00243         .with<ecs::Transform>("player_transform", 200.0F, 200.0F, 0.F)
00244         .with<ecs::Velocity>("player_velocity", 0.F, 0.F)
00245         .with<ecs::Rect>("player_rect", 0.F, 0.F, 33, 17)
00246         .with<ecs::Scale>("player_scale", 2.0F, 2.0F)
00247         .with<ecs::Texture>("player_texture", utl::Path::Texture::TEXTURE_PLAYER)
00248         .with<ecs::Player>("player", true)
00249         .with<ecs::Hitbox>("player_hitbox", 20.0F, offsetX, offsetY)
00250         .build();
00251 }
00252
00253 std::pair<ecs::Entity, ecs::Entity> gme::Game::createPipePair(ecs::Registry &registry, float x, float /*gapY*/) const
00254 {
00255     auto [width, height] = m_renderer->getWindowSize();
00256     const float scale = width / 1920.f;
00257
00258     auto topPipe = registry.createEntity()
00259         .with<ecs::Transform>("pipe_top_transform", x, 0.f, 0.f)
00260         .with<ecs::Velocity>("pipe_top_velocity", -200.f * scale, 0.f)
00261         .with<ecs::Scale>("pipe_top_scale", scale * 3.f, scale * 3.f)
00262         .with<ecs::Texture>("pipe_top_texture", "assets/sprites/flappybird-pipe-xl.png")
00263         .build();
00264
00265     m_renderer->setSpriteRotation("pipe_top_texture" + std::to_string(topPipe), -90.f);
00266     m_renderer->setSpriteOrigin("pipe_top_texture" + std::to_string(topPipe));
00267
00268     auto bottomPipe = registry.createEntity()
00269         .with<ecs::Transform>("pipe_bottom_transform", x, static_cast<float>(height), 0.f)
00270         .with<ecs::Velocity>("pipe_bottom_velocity", -200.f * scale, 0.f)
00271         .with<ecs::Scale>("pipe_bottom_scale", scale * 3.f, scale * 3.f)
00272         .with<ecs::Texture>("pipe_bottom_texture", "assets/sprites/flappybird-pipe-xl.png")
00273         .build();
00274
00275     m_renderer->setSpriteRotation("pipe_bottom_texture" + std::to_string(bottomPipe), 90.f);
00276     m_renderer->setSpriteOrigin("pipe_bottom_texture" + std::to_string(bottomPipe));
00277
00278     return {topPipe, bottomPipe};
00279 }
00280
00281 void gme::Game::resetGame()
00282 {
00283     auto &registry = AScene::getRegistry();
00284     auto *playerTransform = registry.getComponent<ecs::Transform>(m_playerEntity);
00285
00286     if (auto *playerVel = registry.getComponent<ecs::Velocity>(m_playerEntity);
00287         (playerTransform != nullptr) && playerVel)
00288     {
00289         playerTransform->x = 200.f;
00290         playerTransform->y = 200.f;
00291         playerVel->y = 0.f;
00292     }
00293
00294     auto [width, height] = m_renderer->getWindowSize();
00295     float startX = width + 200.f;
00296
00297     for (auto &[topPipe, bottomPipe] : m_pipes)
00298     {
00299         auto *topTransform = registry.getComponent<ecs::Transform>(topPipe);
00300         if (auto *botTransform = registry.getComponent<ecs::Transform>(bottomPipe); topTransform && botTransform)
00301         {
00302             topTransform->x = startX;
00303             botTransform->x = startX;
00304             const PipeType type = randomPipeType();
00305             const float gapSize = getGapSize(type);
00306             const float gapY = static_cast<float>(100 + rand() % (height - 100 - static_cast<int>(gapSize)));
00307
00308             topTransform->y = gapY - 320.f;
00309             botTransform->y = gapY + gapSize;
00310         }
00311         startX += 400.f;
00312     }
00313
00314     m_gameOver = false;
00315     m_gameOverShown = false;
00316 }

```

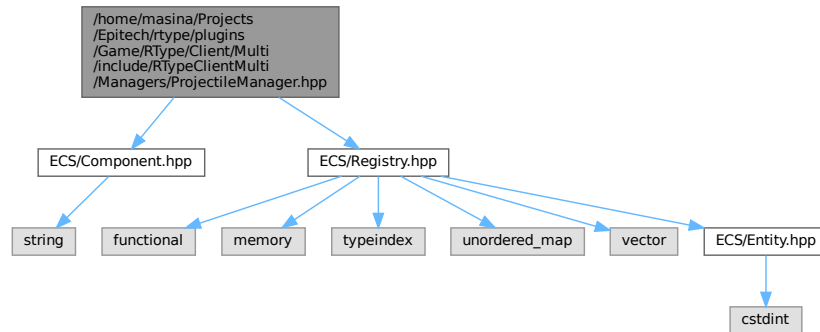
19.145 /home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/include/RTypeClientMulti/Managers/ProjectileManager.hpp File Reference

Manages projectile creation and configuration for multiplayer.

#include "ECS/Component.hpp"

#include "ECS/Registry.hpp"

Include dependency graph for ProjectileManager.hpp:



Classes

- class [gme::ProjectileManager](#)
Handles projectile creation and management.

Namespaces

- namespace [gme](#)

19.145.1 Detailed Description

Manages projectile creation and configuration for multiplayer.

Definition in file [ProjectileManager.hpp](#).

19.146 ProjectileManager.hpp

[Go to the documentation of this file.](#)

```

00001 ///
00002 /// @file ProjectileManager.hpp
00003 /// @brief Manages projectile creation and configuration for multiplayer
00004 /// @namespace gme
00005 ///
00006
00007 #pragma once
00008
00009 #include "ECS/Component.hpp"
00010 #include "ECS/Registry.hpp"
00011
00012 namespace gme
00013 {
00014     ///
00015     /// @class ProjectileManager
00016     /// @brief Handles projectile creation and management
00017     /// @namespace gme
00018     ///
00019     class ProjectileManager
00020     {
00021     public:
00022         ProjectileManager() = default;
00023         ~ProjectileManager() = default;
    
```

```

00024
00025     ProjectileManager(const ProjectileManager &) = delete;
00026     ProjectileManager &operator=(const ProjectileManager &) = delete;
00027     ProjectileManager(ProjectileManager &&) = delete;
00028     ProjectileManager &operator=(ProjectileManager &&) = delete;
00029 }; // class ProjectileManager
00030 } // namespace gme

```

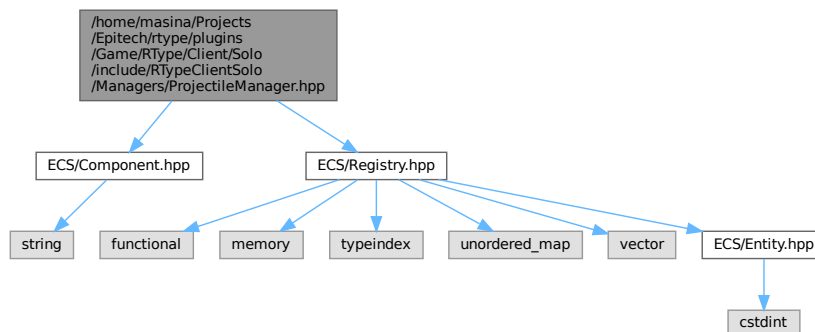
19.147 /home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Solo/include/RTypeClientSolo/Managers/ProjectileManager.hpp File Reference

Manages projectile creation and configuration.

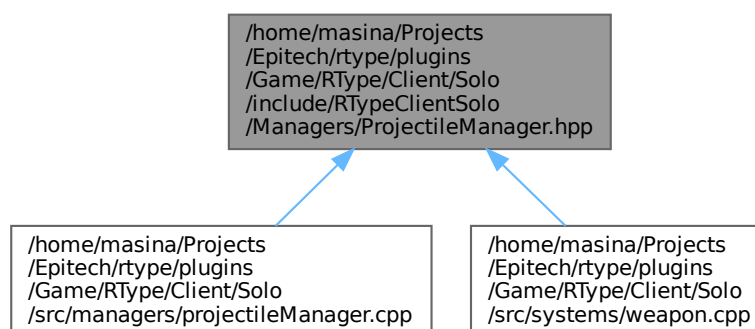
#include "ECS/Component.hpp"

#include "ECS/Registry.hpp"

Include dependency graph for ProjectileManager.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class `gme::ProjectileManager`
Handles projectile creation and management.

Namespaces

- namespace `gme`

19.147.1 Detailed Description

Manages projectile creation and configuration.

Definition in file [ProjectileManager.hpp](#).

19.148 ProjectileManager.hpp

[Go to the documentation of this file.](#)

```

00001 ///  

00002 ///  

00003 ///  

00004 ///  

00005 ///  

00006 ///  

00007 #pragma once  

00008  

00009 #include "ECS/Component.hpp"  

00010 #include "ECS/Registry.hpp"  

00011  

00012 namespace gme  

00013 {  

00014     ///  

00015     ///  

00016     ///  

00017     ///  

00018     ///  

00019     class ProjectileManager  

00020     {  

00021     public:  

00022         ProjectileManager() = default;  

00023         ~ProjectileManager() = default;  

00024  

00025         ProjectileManager(const ProjectileManager &) = delete;  

00026         ProjectileManager &operator=(const ProjectileManager &) = delete;  

00027         ProjectileManager(ProjectileManager &&) = delete;  

00028         ProjectileManager &operator=(ProjectileManager &&) = delete;  

00029  

00030         ///  

00031         ///  

00032         ///  

00033         ///  

00034         ///  

00035         ///  

00036         ///  

00037         ///  

00038         ///  

00039         static ecs::Entity createBasicProjectile(ecs::Registry &registry, float x, float y, float velocityX,  

00040                                             float velocityY);  

00041  

00042         ///  

00043         ///  

00044         ///  

00045         ///  

00046         ///  

00047         ///  

00048         ///  

00049         ///  

00050         ///  

00051         static ecs::Entity createSuperchargedProjectile(ecs::Registry &registry, float x, float y, float velocityX,  

00052                                             float velocityY);  

00053  

00054     private:  

00055         ///  

00056         ///  

00057         ///  

00058         ///  

00059         ///  

00060         ///  

00061         ///  

00062         ///  

00063         ///  

00064         ///  

00065         static ecs::Entity createProjectile(ecs::Registry &registry, ecs::Projectile::Type type, float x, float y,  

00066                                             float velocityX, float velocityY);  

00067     }; // class ProjectileManager  

00068 } // namespace gme

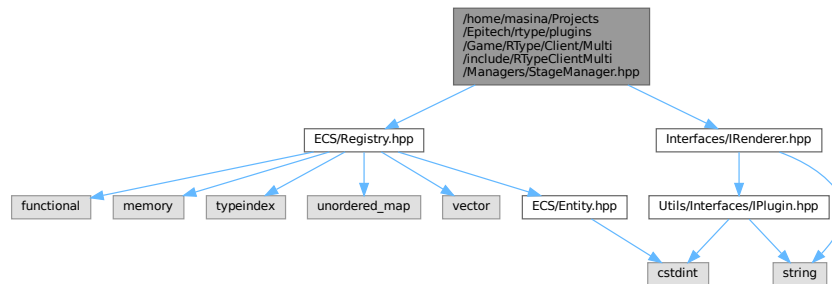
```

19.149 /home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/include/RTypeClientMulti/Managers/StageManager.hpp File Reference

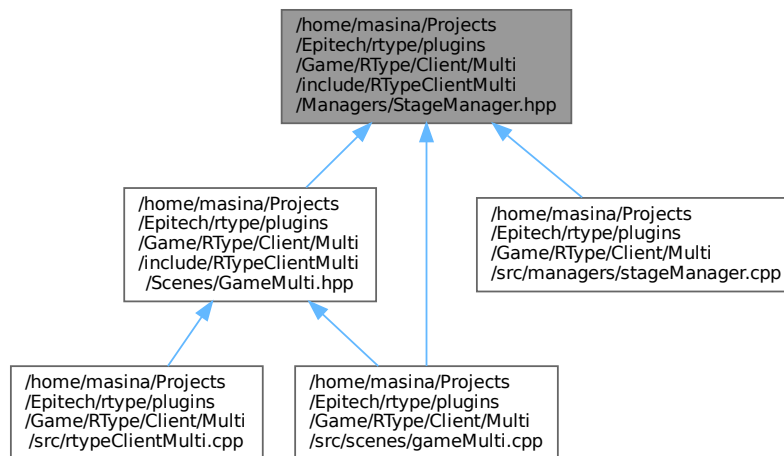
```
#include "ECS/Registry.hpp"
```

```
#include "Interfaces/IRenderer.hpp"
```

Include dependency graph for StageManager.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [gme::StageManager](#)

Namespaces

- namespace [gme](#)

19.150 StageManager.hpp

[Go to the documentation of this file.](#)

```
00001 #pragma once
```

```
00002
```

```
00003 #include "ECS/Registry.hpp"
```



```

00004 #include "Interfaces/IRenderer.hpp"
00005
00006 namespace gme
00007 {
00008
00009     class StageManager
00010     {
00011     public:
00012         StageManager() = default;
00013         ~StageManager() = default;
00014
00015         StageManager(const StageManager &) = delete;
00016         StageManager &operator=(const StageManager &) = delete;
00017         StageManager(StageManager &&) = delete;
00018         StageManager &operator=(const StageManager &&) = delete;
00019
00020         void update(ecs::Registry &registry, float dt, const eng::WindowSize &size);
00021         static void spawnStage(ecs::Registry &registry, int screenWidth);
00022         bool isStageSpawned() const { return m_stageSpawned; }
00023         int getCurrentWave() const { return m_currentWave; }
00024         float getWaveTimer() const { return m_waveTimer; }
00025         float getTotalGameTime() const { return m_totalGameTimer; }
00026
00027         static void stopScrolling(ecs::Registry &registry);
00028
00029     private:
00030         float m_stageTimer = 0.0f;
00031         bool m_stageSpawned = false;
00032
00033         // Wave tracking (synchronized with server)
00034         int m_currentWave = 0;
00035         float m_waveTimer = 0.0f;
00036         float m_totalGameTimer = 0.0f;
00037
00038         static constexpr float STAGE_SPAWN_DELAY = 0.5f;
00039         static constexpr float WAVE_DURATION = 30.0f;
00040         static constexpr int TOTAL_WAVES = 3;
00041         static constexpr float SCROLL_SPEED = -40.0f;
00042         static constexpr float ORIGINAL_WIDTH = 2608.0f;
00043         static constexpr float SPRITE_HEIGHT = 208.0f;
00044     };
00045 } // namespace gme

```

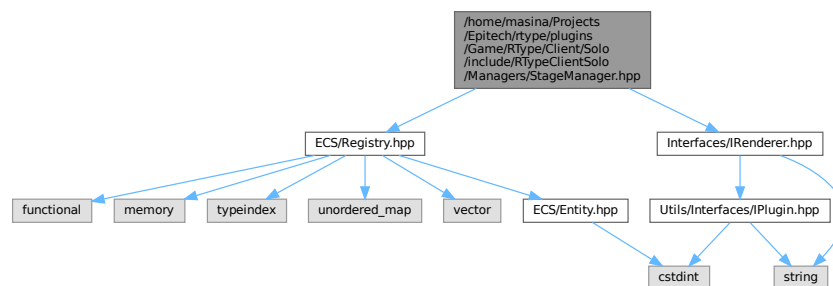
19.151 /home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Solo/include/RTypeClientSolo/Managers/StageManager.hpp File Reference

Stage Manager for handling floor and ceiling spawning.

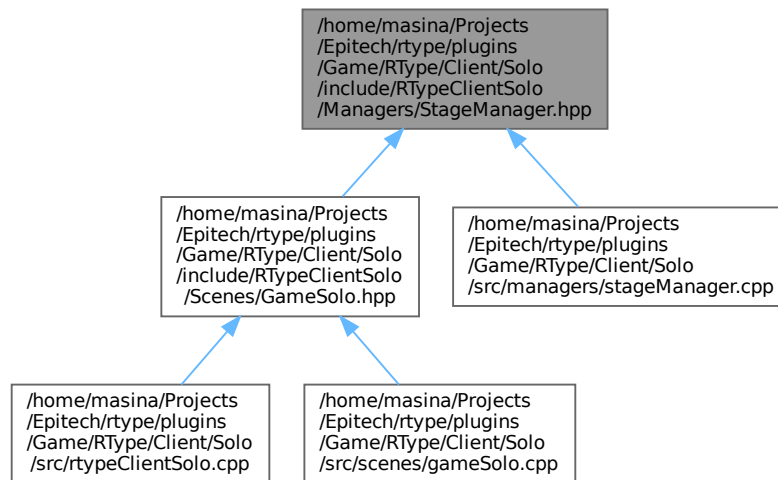
#include "ECS/Registry.hpp"

#include "Interfaces/IRenderer.hpp"

Include dependency graph for StageManager.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [gme::StageManager](#)

Namespaces

- namespace [gme](#)

19.151.1 Detailed Description

Stage Manager for handling floor and ceiling spawning.

Definition in file [StageManager.hpp](#).

19.152 StageManager.hpp

[Go to the documentation of this file.](#)

```

00001 ///  

00002 ///  

00003 ///  

00004 ///  

00005 ///  

00006 ///  

00007 #pragma once  

00008  

00009 #include "ECS/Registry.hpp"  

00010 #include "Interfaces/IRenderer.hpp"  

00011  

00012 namespace gme  

00013 {  

00014  

00015     class StageManager  

00016     {  

00017     public:  

00018         StageManager() = default;  

00019         ~StageManager() = default;  

00020  

00021         StageManager(const StageManager &) = delete;  

00022         StageManager &operator=(const StageManager &) = delete;  

00023         StageManager(StageManager &&) = delete;  

00024         StageManager &operator=(StageManager &&) = delete;  

00025  

00026         void update(ecs::Registry &registry, float dt, const eng::WindowSize &size);  

00027         static void spawnStage(ecs::Registry &registry, int screenWidth);  

00028         bool isStageSpawned() const { return m_stageSpawned; }
  
```

```

00029 int getCurrentWave() const { return m_currentWave; }
00030 float getWaveTimer() const { return m_waveTimer; }
00031 float getTotalGameTime() const { return m_totalGameTimer; }
00032
00033 private:
00034     float m_stageTimer = 0.0f;
00035     bool m_stageSpawned = false;
00036
00037     // Wave tracking (synchronized with server)
00038     int m_currentWave = 0;
00039     float m_waveTimer = 0.0f;
00040     float m_totalGameTimer = 0.0f;
00041
00042     static constexpr float STAGE_SPAWN_DELAY = 0.5f;
00043     static constexpr float WAVE_DURATION = 30.0f;
00044     static constexpr int TOTAL_WAVES = 3;
00045     static constexpr float SCROLL_SPEED = -40.0f;
00046     static constexpr float ORIGINAL_WIDTH = 2608.0f;
00047     static constexpr float SPRITE_HEIGHT = 208.0f;
00048 }; // class StageManager
00049
00050 } // namespace gme

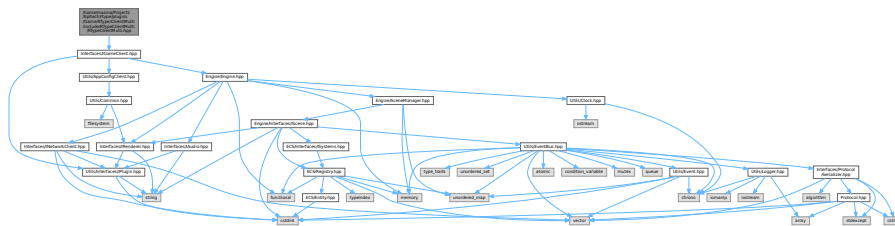
```

19.153 /home/masina/Projects/Epitech/rtype/plugins/Game/RType/↵
Client/Multi/include/RTypeClientMulti/RTypeClientMulti.hpp
File Reference

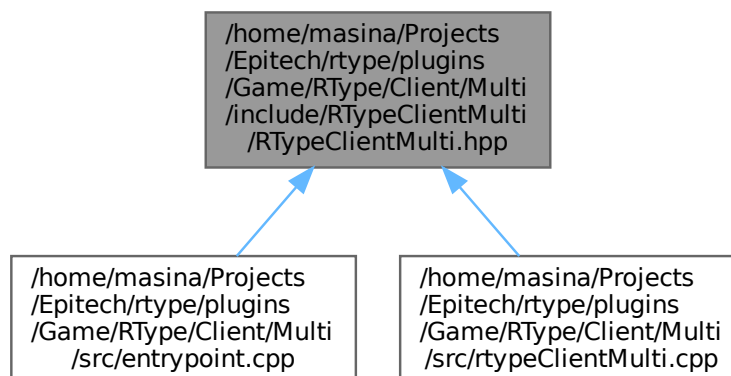
This file contains the RTypeClientMulti class declaration.

```
#include "Interfaces/IGameClient.hpp"
```

Include dependency graph for RTypeClientMulti.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [gme::RTypeClientMulti](#)
Class for the [RTypeClientMulti](#) plugin.

Namespaces

- namespace [gme](#)

19.153.1 Detailed Description

This file contains the [RTypeClientMulti](#) class declaration.

Definition in file [RTypeClientMulti.hpp](#).

19.154 [RTypeClientMulti.hpp](#)

[Go to the documentation of this file.](#)

```

00001 ///  

00002 ///  

00003 ///  

00004 ///  

00005 ///  

00006 ///  

00007 #pragma once  

00008  

00009 #include "Interfaces/IGameClient.hpp"  

00010  

00011 namespace gme  

00012 {  

00013  

00014     ///  

00015     ///  

00016     ///  

00017     ///  

00018     ///  

00019     class RTypeClientMulti final : public IGameClient  

00020     {  

00021  

00022     public:  

00023         RTypeClientMulti() = default;  

00024         ~RTypeClientMulti() override = default;  

00025  

00026         RTypeClientMulti(const RTypeClientMulti &) = delete;  

00027         RTypeClientMulti &operator=(const RTypeClientMulti &) = delete;  

00028         RTypeClientMulti(RTypeClientMulti &&) = delete;  

00029         RTypeClientMulti &operator=(RTypeClientMulti &&) = delete;  

00030  

00031         [[nodiscard]] const std::string getName() const override { return "RType_Client_Multi"; }  

00032         [[nodiscard]] utl::PluginType getType() const override { return utl::PluginType::GAME_CLIENT; }  

00033  

00034         void init(eng::Engine &engine, utl::cli::AppConfig &appConfig, bool &showDebug,  

00035                 const eng::id menuSceneId) override  

00036         {  

00037             m_engine = &engine;  

00038             m_appConfig = &appConfig;  

00039             m_showDebug = showDebug;  

00040             setupScenes(showDebug, menuSceneId);  

00041         }  

00042         void update(float deltaTime, unsigned int width, unsigned int height) override;  

00043         [[nodiscard]] unsigned int getMainSceneId() const override { return m_mainSceneId; }  

00044  

00045     private:  

00046         void setupScenes(bool &showDebug, eng::id menuSceneId);  

00047  

00048         eng::Engine *m_engine = nullptr;  

00049         utl::cli::AppConfig *m_appConfig = nullptr;  

00050         bool m_showDebug = false;  

00051         eng::id m_mainSceneId = 1;  

00052  

00053     }; // class RTypeClientMulti  

00054  

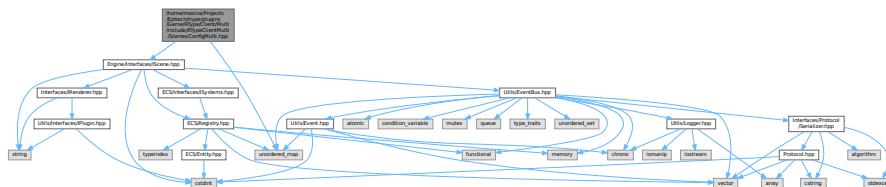
00055 } // namespace gme

```

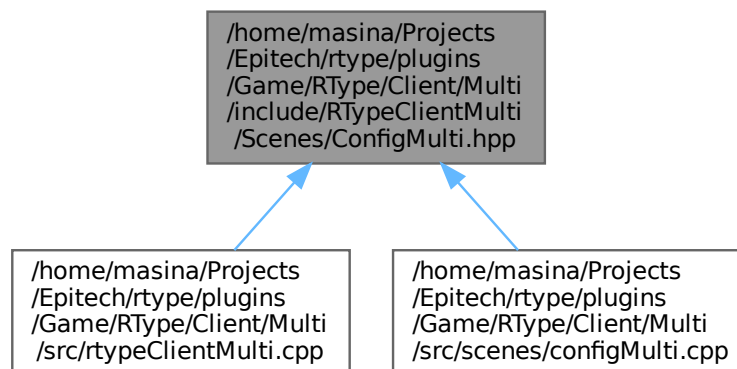
This file contains the multiplayer configuration scene.

```
#include "Engine/Interfaces/IScene.hpp"
```

Include dependency graph for ConfigMulti.hpp:



This graph shows which files directly or indirectly include this file:



- class gme::ConfigMulti
ConfigMulti scene.

- namespace `gme`

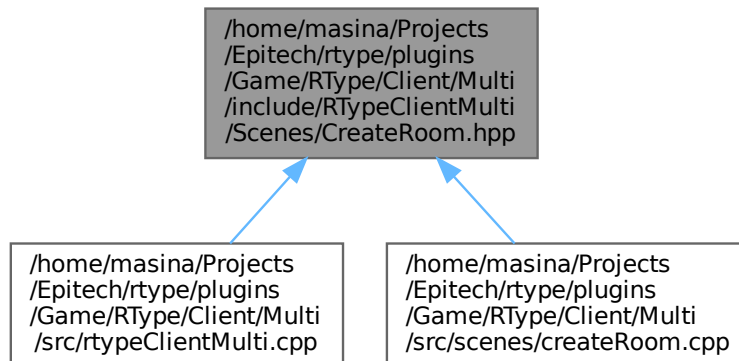
This file contains the multiplayer configuration scene.

Definition in file [ConfigMulti.hpp](#).

[Go to the documentation of this file.](#)

```
00001 ///
00002 /// @file ConfigMulti.hpp
```


This graph shows which files directly or indirectly include this file:



Classes

- class [gme::CreateRoomScene](#)
Create room scene.

Namespaces

- namespace [gme](#)

19.157.1 Detailed Description

This file contains the create room scene.

Definition in file [CreateRoom.hpp](#).

19.158 CreateRoom.hpp

[Go to the documentation of this file.](#)

```

00001 ///  

00002 ///  

00003 ///  

00004 ///  

00005 ///  

00006 ///  

00007 #pragma once  

00008  

00009 #include "Engine/Interfaces/IScene.hpp"  

00010  

00011 namespace gme  

00012 {  

00013     ///  

00014     ///  

00015     ///  

00016     ///  

00017     ///  

00018     class CreateRoomScene final : public eng::AScene  

00019     {  

00020     public:  

00021         CreateRoomScene(eng::id assignedId, const std::shared_ptr<eng::IRenderer> &renderer);  

00022         ~CreateRoomScene() override = default;  

00023  

00024         CreateRoomScene(const CreateRoomScene &other) = delete;  

00025         CreateRoomScene &operator=(const CreateRoomScene &other) = delete;  

00026         CreateRoomScene(CreateRoomScene &&other) = delete;  

00027         CreateRoomScene &operator=(CreateRoomScene &&other) = delete;  

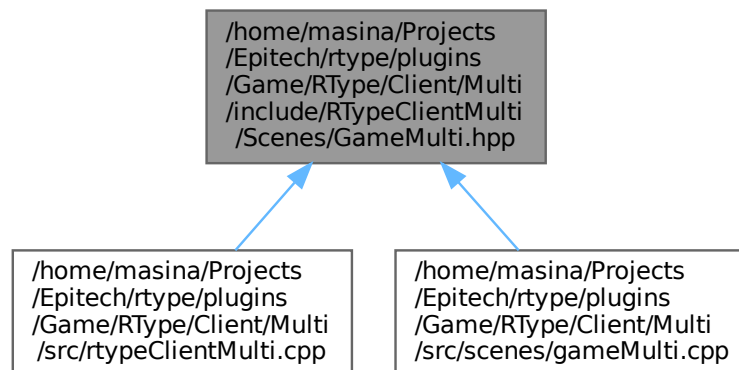
00028  

00029         void update(float dt, const eng::WindowSize &size) override;  

00030         void event(const eng::Event &event) override;  


```


This graph shows which files directly or indirectly include this file:



Classes

- struct `gme::InterpolationData`
- class `gme::GameMulti`
 `GameMulti` scene.
- struct `gme::GameMulti::PendingInput`

Namespaces

- namespace `gme`

19.159.1 Detailed Description

This file contains the multiplayer Game scene.

Definition in file [GameMulti.hpp](#).

19.160 GameMulti.hpp

[Go to the documentation of this file.](#)

```

00001 ///
00002 /// @file GameMulti.hpp
00003 /// @brief This file contains the multiplayer Game scene
00004 /// @namespace gme
00005 ///
00006
00007 #pragma once
00008
00009 #include <deque>
00010 #include <memory>
00011 #include <unordered_map>
00012 #include <unordered_set>
00013 #include <vector>
00014
00015 #include "Engine/Interfaces/IScene.hpp"
00016 #include "Interfaces/IAudio.hpp"
00017 #include "RTypeClientMulti/Managers/StageManager.hpp"
00018 #include "Utils/EventBus.hpp"
00019 #include "RTypeClientMulti/Systems/PlayerControllerMulti.hpp"
00020
00021
00022 namespace gme
00023 {
00024     struct InterpolationData
00025     {
00026         float targetX, targetY;

```

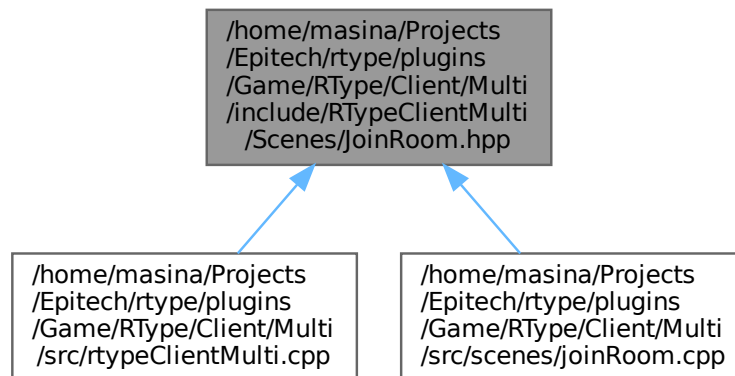
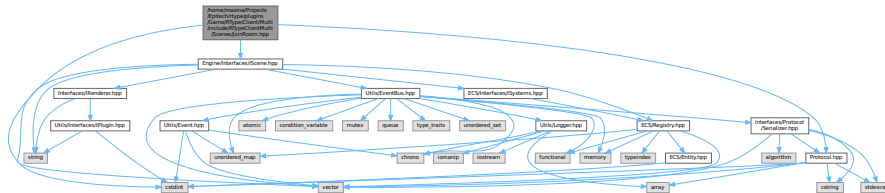
```

00027     float targetVx, targetVy;
00028     float currentX, currentY;
00029     float smoothFactor;
00030     float targetRotation;
00031     float currentRotation;
00032 };
00033
00034 ///
00035 /// @class GameMulti
00036 /// @brief GameMulti scene
00037 /// @namespace gme
00038 ///
00039 class GameMulti final : public eng::AScene
00040 {
00041     public:
00042         GameMulti(eng::id assignedId, const std::shared_ptr<eng::IRenderer> &renderer,
00043                 const std::shared_ptr<eng::IAudio> &audio, float skinIndex, bool &showDebug, uint32_t sessionId);
00044         ~GameMulti() override = default;
00045
00046         GameMulti(const GameMulti &other) = delete;
00047         GameMulti &operator=(const GameMulti &other) = delete;
00048         GameMulti(GameMulti &&other) = delete;
00049         GameMulti &operator=(GameMulti &&other) = delete;
00050
00051         void update(float dt, const eng::WindowSize &size) override;
00052         void event(const eng::Event &event) override;
00053         void updatePlayerSkin();
00054
00055         bool &playMusic() { return m_playMusic; }
00056
00057     private:
00058         void setupEventSubscriptions() const;
00059         void processEventBus();
00060         void handleWorldStateUpdate(const utl::Event &event);
00061         void preloadCommonTextures();
00062
00063         void updateInterpolation(std::unordered_map<uint32_t, InterpolationData> &dataMap,
00064                                 std::unordered_map<uint32_t, ecs::Entity> &entityMap, float smoothFactor, float dt,
00065                                 ecs::Registry &registry);
00066
00067         void updateInterpolation(std::unordered_map<uint32_t, InterpolationData> &dataMap,
00068                                 std::unordered_map<uint32_t, ecs::Entity> &entityMap, float smoothFactor, float dt,
00069                                 ecs::Registry &registry) const;
00070
00071         ecs::Entity m_localPlayerEntity;
00072         std::unordered_map<uint32_t, ecs::Entity> m_remotePlayers;
00073
00074         const std::shared_ptr<eng::IAudio> &m_audio;
00075         const std::shared_ptr<eng::IRenderer> &m_renderer;
00076         float m_skinIndex;
00077         std::unique_ptr<PlayerControllerMulti> m_playerController;
00078         std::unordered_map<eng::Key, bool> m_keysPressed;
00079         bool &m_showDebug;
00080         bool m_playMusic = false;
00081
00082         uint32_t m_sessionId;
00083         uint32_t m_eventComponentId = 10;
00084         std::unique_ptr<StageManager> m_stageManager;
00085
00086         std::unordered_map<uint32_t, ecs::Entity> m_projectileEntities;
00087         std::unordered_map<uint32_t, ecs::Entity> m_enemyEntities;
00088
00089         std::unordered_map<uint32_t, InterpolationData> m_remotePlayerData;
00090         std::unordered_map<uint32_t, InterpolationData> m_projectileData;
00091         std::unordered_map<uint32_t, InterpolationData> m_enemyData;
00092         std::unordered_map<uint32_t, uint32_t> m_playerSkinMap;
00093         struct PendingInput
00094         {
00095             uint32_t seqId;
00096             std::vector<uint8_t> inputData;
00097             float dt;
00098         };
00099         std::deque<PendingInput> m_inputHistory;
00100         uint32_t m_lastAckSeqId = 0;
00101
00102         static constexpr float REMOTE_PLAYER_SMOOTH_FACTOR = 0.25f;
00103         static constexpr float PROJECTILE_SMOOTH_FACTOR = 0.5f;
00104         static constexpr float ENEMY_SMOOTH_FACTOR = 0.18f;
00105
00106         bool m_starfieldCreated = false;
00107         ecs::Entity m_beginSoundEntity{};
00108         ecs::Entity m_bossMusicEntity{};
00109         float m_bossMusicTimer = 0.0f;
00110         bool m_bossMusicStarted = false;
00111         static constexpr float BOSS_MUSIC_DURATION = 40.0f;
00112
00113         // Cache to avoid recreating textures/fonts

```

```
00114         std::unordered_set<std::string> m_loadedTextures;
00115         std::unordered_set<std::string> m_loadedFonts;
00116
00117         // Track if this is the first world state (for player skin initialization)
00118         bool m_firstWorldState = true;
00119     }; // class GameMulti
00120 } // namespace gme
```

This file contains the join room scene.



- class `gme::JoinRoomScene`
Join room scene.

- namespace `gme`

19.161.1 Detailed Description

This file contains the join room scene.
Definition in file [JoinRoom.hpp](#).

19.162 JoinRoom.hpp

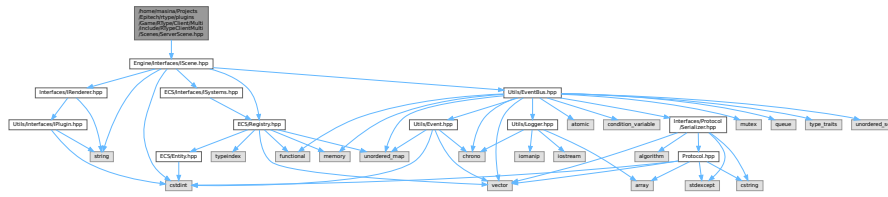
[Go to the documentation of this file.](#)

```
00001 ///  
00002 ///  
00003 ///  
00004 ///  
00005 ///  
00006 ///  
00007 #pragma once  
00008 ///  
00009 #include <vector>  
00010 ///  
00011 #include "Engine/Interfaces/IScene.hpp"  
00012 #include "Interfaces/Protocol/Protocol.hpp"  
00013 ///  
00014 namespace gme  
00015 {  
00016     ///  
00017     ///  
00018     ///  
00019     ///  
00020     ///  
00021     class JoinRoomScene final : public eng::AScene  
00022     {  
00023     public:  
00024         JoinRoomScene(eng::id assignedId, const std::shared_ptr<eng::IRenderer> &renderer);  
00025         ~JoinRoomScene() override = default;  
00026         ///  
00027         JoinRoomScene(const JoinRoomScene &other) = delete;  
00028         JoinRoomScene &operator=(const JoinRoomScene &other) = delete;  
00029         JoinRoomScene(JoinRoomScene &&other) = delete;  
00030         JoinRoomScene &operator=(JoinRoomScene &&other) = delete;  
00031         ///  
00032         void update(float dt, const eng::WindowSize &size) override;  
00033         void event(const eng::Event &event) override;  
00034         ///  
00035         bool &playMusic() { return m_playMusic; }  
00036         ///  
00037         void setRooms(const std::vector<rng::LobbyInfo> &rooms);  
00038         void refreshRoomList() const;  
00039         ///  
00040         std::function<void(int roomId, const rng::LobbyInfo *lobbyInfo)> onJoin;  
00041         std::function<void()> onBackToMulti;  
00042         std::function<void()> onRefreshRequest;  
00043         ///  
00044     private:  
00045         size_t m_selectedIndex = 0;  
00046         float m_animationTime = 0.0f;  
00047         std::vector<rng::LobbyInfo> m_rooms;  
00048         ///  
00049         ecs::Entity m_noRoomsEntity = 0;  
00050         std::vector<ecs::Entity> m_roomEntities;  
00051         bool m_playMusic = false;  
00052         void updateRoomDisplay();  
00053         void clearRoomEntities();  
00054         void processEventBus();  
00055         void setupEventSubscriptions() const;  
00056         void handleLobbyListResponse(const utl::Event &event);  
00057         void handleLobbyJoinResponse(const utl::Event &event) const;  
00058     }; // class JoinRoomScene  
00059 } // namespace gme
```

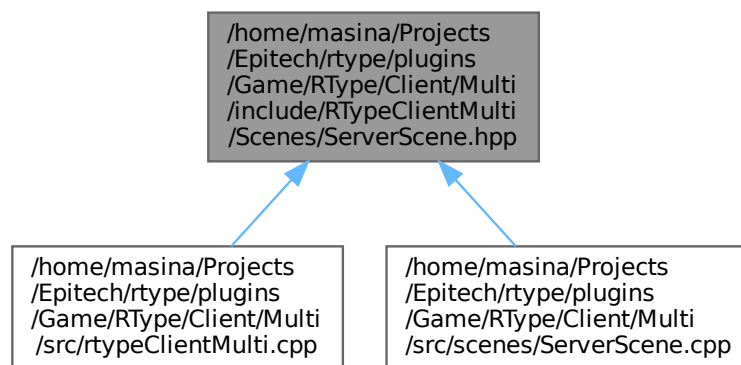
19.163 /home/masina/Projects/Epitech/rtype/plugins/Game/RType/↵ Client/Multi/include/RTypeClientMulti/Scenes/Server↵ Scene.hpp File Reference

This file contains the server configuration scene.

#include "Engine/Interfaces/IScene.hpp"
 Include dependency graph for ServerScene.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class `gme::ServerScene`
 Server configuration scene.

Namespaces

- namespace `gme`

19.163.1 Detailed Description

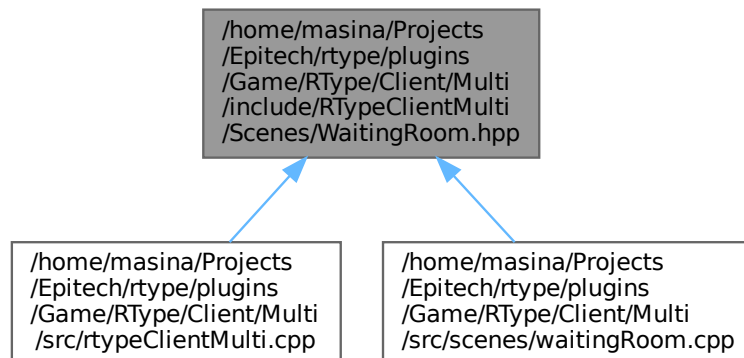
This file contains the server configuration scene.
 Definition in file [ServerScene.hpp](#).

19.164 ServerScene.hpp

[Go to the documentation of this file.](#)

```
00001 ///  
00002 ///  
00003 ///  
00004 ///  
00005 ///  
00006 ///  
00007 #pragma once  
00008  
00009 #include "Engine/Interfaces/IScene.hpp"  
00010  
00011 namespace gme  
00012 {
```


This graph shows which files directly or indirectly include this file:



Classes

- class [gme::WaitingRoomScene](#)
Waiting room scene where players wait before game starts.

Namespaces

- namespace [gme](#)

19.165.1 Detailed Description

This file contains the waiting room scene.
Definition in file [WaitingRoom.hpp](#).

19.166 WaitingRoom.hpp

[Go to the documentation of this file.](#)

```

00001 ///  

00002 ///  

00003 ///  

00004 ///  

00005 ///  

00006 ///  

00007 #pragma once  

00008  

00009 #include <functional>  

00010 #include <vector>  

00011  

00012 #include "Engine/Interfaces/IScene.hpp"  

00013 #include "Interfaces/Protocol/Protocol.hpp"  

00014  

00015 namespace gme  

00016 {  

00017     ///  

00018     ///  

00019     ///  

00020     ///  

00021     ///  

00022     class WaitingRoomScene final : public eng::AScene  

00023     {  

00024     public:  

00025         WaitingRoomScene(eng::id assignedId, const std::shared_ptr<eng::IRenderer> &renderer);  

00026         ~WaitingRoomScene() override = default;  

00027  

00028         WaitingRoomScene(const WaitingRoomScene &other) = delete;  

00029         WaitingRoomScene &operator=(const WaitingRoomScene &other) = delete;  

00030         WaitingRoomScene(WaitingRoomScene &&other) = delete;  


```

```

00031     WaitingRoomScene &operator=(WaitingRoomScene &&other) = delete;
00032
00033     void update(float dt, const eng::WindowSize &size) override;
00034     void event(const eng::Event &event) override;
00035
00036     void setLobbyId(std::uint32_t lobbyId);
00037     void setLobbyInfo(const rnp::LobbyInfo &lobbyInfo);
00038     void setIsHost(const bool isHost) { m_isHost = isHost; }
00039     [[nodiscard]] bool isHost() const { return m_isHost; }
00040     std::uint32_t getLobbyId() const { return m_lobbyId; };
00041
00042     bool &playMusic() { return m_playMusic; }
00043
00044     std::function<void()> onLeaveLobby;
00045     std::function<void()> onGameStart;
00046     void startGame();
00047
00048 private:
00049     const std::shared_ptr<eng::IRenderer> &m_renderer;
00050
00051     std::uint32_t m_lobbyId = 0;
00052     rnp::LobbyInfo m_currentLobbyInfo{};
00053     bool m_hasLobbyInfo = false;
00054     float m_animationTime = 0.0f;
00055
00056     int m_selectedButton = 0;
00057     static constexpr int BUTTON_LEAVE = 0;
00058     static constexpr int BUTTON_READY = 1;
00059     static constexpr int BUTTON_START = 2;
00060     static constexpr int BUTTON_COUNT = 3;
00061
00062     // UI Entities
00063     ecs::Entity m_lobbyIdEntity = 0;
00064     ecs::Entity m_playerCountEntity = 0;
00065     ecs::Entity m_statusEntity = 0;
00066     ecs::Entity m_leaveButtonEntity = 0;
00067     ecs::Entity m_startButtonEntity = 0;
00068     ecs::Entity m_readyButtonEntity = 0;
00069     std::vector<ecs::Entity> m_playerEntities;
00070
00071     bool m_isHost = false;
00072     bool m_playMusic = false;
00073     void setupEventSubscriptions() const;
00074     void processEventBus();
00075     void handleLobbyUpdate(const utl::Event &event);
00076     void handleGameStart(const utl::Event &event) const;
00077     void updatePlayerDisplay();
00078     void clearPlayerEntities();
00079     void leaveLobby() const;
00080
00081 }; // class WaitingRoomScene
00082 } // namespace gme

```

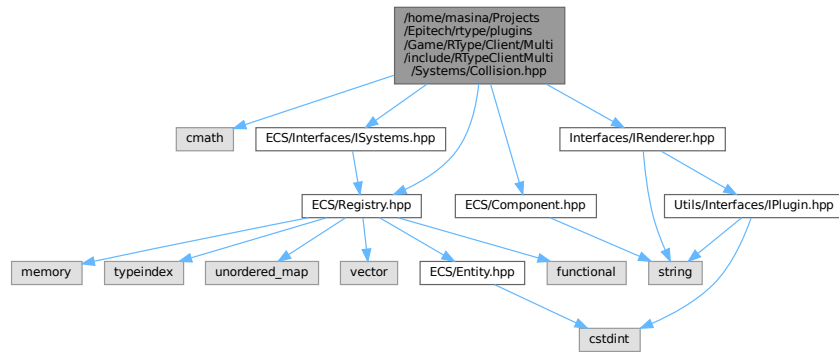
19.167 /home/masina/Projects/Epitech/rtype/plugins/Game/RType/↵ Client/Multi/include/RTypeClientMulti/Systems/Collision.hpp File Reference

```

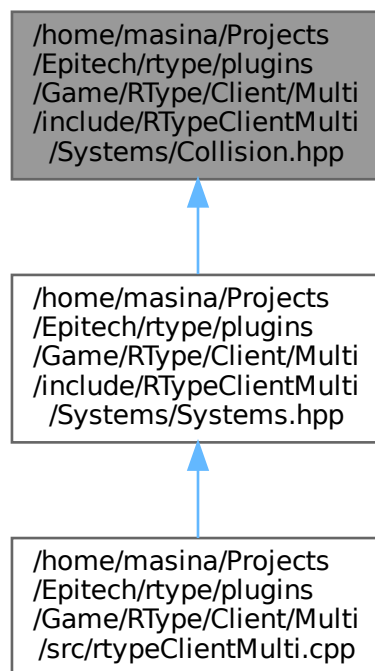
#include <cmath>
#include "ECS/Component.hpp"
#include "ECS/Interfaces/ISystems.hpp"
#include "ECS/Registry.hpp"
#include "Interfaces/IRenderer.hpp"

```


Include dependency graph for Collision.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [gme::CollisionSystem](#)
Handles collision detection and resolution on server.

Namespaces

- namespace [gme](#)

19.168 Collision.hpp

[Go to the documentation of this file.](#)

```

00001 #pragma once
00002
00003 #include <cmath>
00004
00005 #include "ECS/Component.hpp"
00006 #include "ECS/Interfaces/ISystems.hpp"
00007 #include "ECS/Registry.hpp"
00008 #include "Interfaces/IRenderer.hpp"
00009
00010 namespace gme
00011 {
00012     class CollisionSystem final : public ecs::ASystem
00013     {
00014     public:
00015         explicit CollisionSystem(const std::shared_ptr<eng::IRenderer> &renderer, bool &showDebug)
00016             : m_renderer(renderer), m_showDebug(showDebug)
00017         {
00018         }
00019         ~CollisionSystem() override = default;
00020
00021         CollisionSystem(const CollisionSystem &) = delete;
00022         CollisionSystem &operator=(const CollisionSystem &) = delete;
00023         CollisionSystem(CollisionSystem &&) = delete;
00024         CollisionSystem &operator=(const CollisionSystem &&) = delete;
00025
00026         bool isEnabled() override { return true; }
00027         void setEnable(bool enable) override { (void)enable; }
00028
00029         void update(ecs::Registry &registry, float dt) override
00030         {
00031             std::optional<float> ceilingBottomY;
00032             std::optional<float> floorTopY;
00033
00034             for (auto &pair : registry.getAll<ecs::Ceiling>())
00035             {
00036                 const auto entity = pair.first;
00037                 const auto *t = registry.getComponent<ecs::Transform>(entity);
00038                 const auto *s = registry.getComponent<ecs::Scale>(entity);
00039                 const auto *scroll = registry.getComponent<ecs::Scrolling>(entity);
00040                 if ((t == nullptr) || (scroll == nullptr))
00041                 {
00042                     continue;
00043                 }
00044                 const float scaledHeight = (s ? s->y : 1.0f) * scroll->original_height;
00045                 ceilingBottomY = t->y + scaledHeight;
00046                 break; // Un seul suffit
00047             }
00048
00049             for (auto &pair : registry.getAll<ecs::Floor>())
00050             {
00051                 const auto entity = pair.first;
00052                 const auto *t = registry.getComponent<ecs::Transform>(entity);
00053                 if (t == nullptr)
00054                 {
00055                     continue;
00056                 }
00057                 floorTopY = t->y;
00058                 break; // Un seul suffit
00059             }
00060
00061             if (ceilingBottomY.has_value() || floorTopY.has_value())
00062             {
00063                 // Optimisation: itération directe
00064                 for (auto &pair : registry.getAll<ecs::Player>())
00065                 {
00066                     const auto playerEntity = pair.first;
00067                     auto *t = registry.getComponent<ecs::Transform>(playerEntity);
00068                     auto *hb = registry.getComponent<ecs::Hitbox>(playerEntity);
00069                     auto *vel = registry.getComponent<ecs::Velocity>(playerEntity);
00070                     if (!t || !hb)
00071                     {
00072                         continue;
00073                     }
00074
00075                     float hitboxY = t->y + hb->offsetY;
00076
00077                     if (ceilingBottomY.has_value() && (hitboxY - hb->radius < ceilingBottomY.value()))
00078                     {
00079                         t->y = ceilingBottomY.value() + hb->radius - hb->offsetY;
00080                         if (vel != nullptr)
00081                         {
00082                             vel->y = std::max(0.0f, vel->y);
00083                         }
00084                     }
00085                 }
00086             }
00087         }
00088     };
00089 }

```

```

00084         }
00085
00086         if (floorTopY.has_value() && (hitboxY + hb->radius > floorTopY.value()))
00087         {
00088             t->y = floorTopY.value() - hb->radius - hb->offsetY;
00089             if (vel != nullptr)
00090             {
00091                 vel->y = std::min(0.0f, vel->y);
00092             }
00093         }
00094     }
00095 }
00096
00097 private:
00098     const std::shared_ptr<eng::IRenderer> &m_renderer;
00099     bool &m_showDebug;
00100 };
00101 } // namespace gme
00102

```

19.169 /home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Solo/include/RTypeClientSolo/Systems/Collision.hpp File Reference

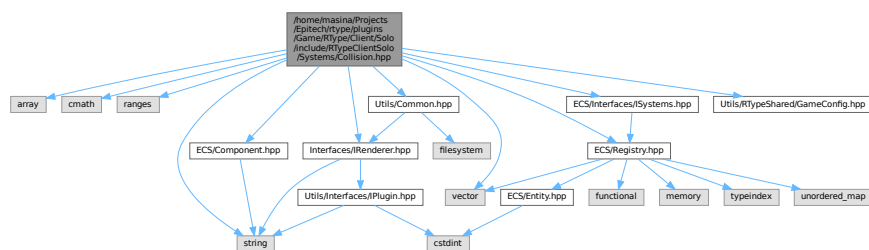
This file contains the collision system definitions.

```

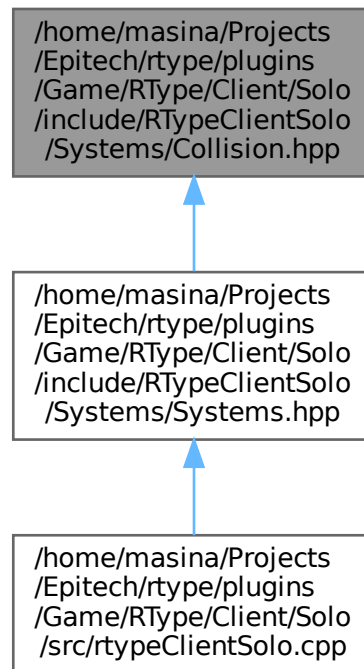
#include <array>
#include <cmath>
#include <ranges>
#include <string>
#include <vector>
#include "ECS/Component.hpp"
#include "ECS/Interfaces/ISystems.hpp"
#include "ECS/Registry.hpp"
#include "Interfaces/IRenderer.hpp"
#include "Utils/Common.hpp"
#include "Utils/RTypeShared/GameConfig.hpp"

```

Include dependency graph for Collision.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [gme::CollisionSystem](#)
Handles collision detection and resolution on server.

Namespaces

- namespace [gme](#)

19.169.1 Detailed Description

This file contains the collision system definitions.

Definition in file [Collision.hpp](#).

19.170 Collision.hpp

[Go to the documentation of this file.](#)

```

00001 ///  

00002 ///  

00003 ///  

00004 ///  

00005 ///  

00006 ///  

00007 #pragma once  

00008  

00009 #include <array>  

00010 #include <cmath>  

00011 #include <ranges>  

00012 #include <string>  

00013 #include <vector>  

00014  

00015 #include "ECS/Component.hpp"
  
```

```

00016 #include "ECS/Interfaces/ISystems.hpp"
00017 #include "ECS/Registry.hpp"
00018 #include "Interfaces/IRenderer.hpp"
00019 #include "Utils/Common.hpp"
00020 #include "Utils/RTypeShared/GameConfig.hpp"
00021
00022 namespace gme
00023 {
00024
00025     class CollisionSystem final : public ecs::ASystem
00026     {
00027     public:
00028         explicit CollisionSystem(const std::shared_ptr<eng::IRenderer> &renderer, bool &showDebug)
00029             : m_renderer(renderer), m_showDebug(showDebug)
00030         {
00031             m_enemyDeathAudioEntities.fill(ecs::INVALID_ENTITY);
00032         }
00033         ~CollisionSystem() override = default;
00034
00035         CollisionSystem(const CollisionSystem &) = delete;
00036         CollisionSystem &operator=(const CollisionSystem &) = delete;
00037         CollisionSystem(CollisionSystem &&) = delete;
00038         CollisionSystem &operator=(CollisionSystem &&) = delete;
00039
00040         bool isEnabled() override { return true; }
00041         void setEnable(const bool enable) override { (void)enable; }
00042
00043         void update(ecs::Registry &registry, float dt) override
00044         {
00045             const bool hasPlayer = !registry.getAll<ecs::Player>().empty();
00046
00047             std::optional<float> ceilingBottomY;
00048             std::optional<float> floorTopY;
00049             for (const auto &entity : registry.getAll<ecs::Ceiling>() | std::views::keys)
00050             {
00051                 const auto *t = registry.getComponent<ecs::Transform>(entity);
00052                 const auto *s = registry.getComponent<ecs::Scale>(entity);
00053                 const auto *scroll = registry.getComponent<ecs::Scrolling>(entity);
00054                 if ((t == nullptr) || (scroll == nullptr))
00055                 {
00056                     continue;
00057                 }
00058                 const float scaledHeight = (s ? s->y : 1.0f) * scroll->original_height;
00059                 ceilingBottomY = t->y + scaledHeight;
00060                 break;
00061             }
00062             for (const auto &entity : registry.getAll<ecs::Floor>() | std::views::keys)
00063             {
00064                 const auto *t = registry.getComponent<ecs::Transform>(entity);
00065                 if (t == nullptr)
00066                 {
00067                     continue;
00068                 }
00069                 floorTopY = t->y;
00070                 break;
00071             }
00072
00073             std::vector<ecs::Entity> projectilesToRemove;
00074             std::vector<ecs::Entity> enemiesToRemove;
00075
00076             for (auto &[projectileEntity, projectile] : registry.getAll<ecs::Projectile>())
00077             {
00078                 const auto *projectileTransform = registry.getComponent<ecs::Transform>(projectileEntity);
00079                 const auto *projectileHitbox = registry.getComponent<ecs::Hitbox>(projectileEntity);
00080                 if ((projectileTransform == nullptr) || (projectileHitbox == nullptr))
00081                 {
00082                     continue;
00083                 }
00084
00085                 for (auto &[enemyEntity, enemy] : registry.getAll<ecs::Enemy>())
00086                 {
00087                     const auto *enemyTransform = registry.getComponent<ecs::Transform>(enemyEntity);
00088                     const auto *enemyHitbox = registry.getComponent<ecs::Hitbox>(enemyEntity);
00089                     if ((enemyTransform == nullptr) || (enemyHitbox == nullptr))
00090                     {
00091                         continue;
00092                     }
00093
00094                     if (checkCircularCollision(*projectileTransform, *projectileHitbox, *enemyTransform,
00095                                             *enemyHitbox))
00096                     {
00097                         enemy.health -= projectile.damage;
00098                         if (projectile.type == ecs::Projectile::SUPERCHARGED && projectile.pierce_remaining > 1)
00099                         {
00100                             projectile.pierce_remaining -= 1;
00101                         }
00102                         else

```

```

00103         {
00104             projectilesToRemove.push_back(projectileEntity);
00105         }
00106
00107         if (enemy.health <= 0.0f)
00108         {
00109             createExplosion(registry, enemyTransform->x, enemyTransform->y);
00110             enemiesToRemove.push_back(enemyEntity);
00111             for (auto &score : registry.getAll<ecs::Score>() | std::views::values)
00112             {
00113                 score.value += 100;
00114                 break;
00115             }
00116             playEnemyDeathSound(registry);
00117         }
00118         break;
00119     }
00120 }
00121 }
00122
00123 for (const ecs::Entity entity : projectilesToRemove)
00124 {
00125     removeProjectile(registry, entity);
00126 }
00127 for (const ecs::Entity entity : enemiesToRemove)
00128 {
00129     removeEnemy(registry, entity);
00130 }
00131 if (ceilingBottomY.has_value() || floorTopY.has_value())
00132 {
00133     for (const auto &playerEntity : registry.getAll<ecs::Player>() | std::views::keys)
00134     {
00135         auto *t = registry.getComponent<ecs::Transform>(playerEntity);
00136         auto *hb = registry.getComponent<ecs::Hitbox>(playerEntity);
00137         auto *vel = registry.getComponent<ecs::Velocity>(playerEntity);
00138         if ((t == nullptr) || (hb == nullptr))
00139         {
00140             continue;
00141         }
00142         const float hitboxY = t->y + hb->offsetY;
00143         if (ceilingBottomY.has_value() && (hitboxY - hb->radius < ceilingBottomY.value()))
00144         {
00145             t->y = ceilingBottomY.value() + hb->radius - hb->offsetY;
00146             if (vel != nullptr)
00147             {
00148                 vel->y = std::max(0.0f, vel->y);
00149             }
00150         }
00151         if (floorTopY.has_value() && (hitboxY + hb->radius > floorTopY.value()))
00152         {
00153             t->y = floorTopY.value() - hb->radius - hb->offsetY;
00154             if (vel != nullptr)
00155             {
00156                 vel->y = std::min(0.0f, vel->y);
00157             }
00158         }
00159     }
00160     for (const auto &enemyEntity : registry.getAll<ecs::Enemy>() | std::views::keys)
00161     {
00162         auto *t = registry.getComponent<ecs::Transform>(enemyEntity);
00163         auto *hb = registry.getComponent<ecs::Hitbox>(enemyEntity);
00164         auto *vel = registry.getComponent<ecs::Velocity>(enemyEntity);
00165         if ((t == nullptr) || (hb == nullptr))
00166         {
00167             continue;
00168         }
00169         const float hitboxY = t->y + hb->offsetY;
00170         if (ceilingBottomY.has_value() && (hitboxY - hb->radius < ceilingBottomY.value()))
00171         {
00172             t->y = ceilingBottomY.value() + hb->radius - hb->offsetY;
00173             if (vel != nullptr)
00174             {
00175                 vel->y = std::max(0.0f, vel->y);
00176             }
00177         }
00178         if (floorTopY.has_value() && (hitboxY + hb->radius > floorTopY.value()))
00179         {
00180             t->y = floorTopY.value() - hb->radius - hb->offsetY;
00181             if (vel != nullptr)
00182             {
00183                 vel->y = std::min(0.0f, vel->y);
00184             }
00185         }
00186     }
00187 }
00188 if (m_wasPlayerPresent && !hasPlayer)
00189 {

```

```

00190         playPlayerDeathSound(registry);
00191     }
00192     m_wasPlayerPresent = hasPlayer;
00193 }
00194
00195 private:
00196     const std::shared_ptr<eng::IRenderer> &m_renderer;
00197     std::array<ecs::Entity, 4> m_enemyDeathAudioEntities{};
00198     ecs::Entity m_playerDeathAudioEntity = ecs::INVALID_ENTITY;
00199     std::size_t m_nextEnemyDeathChannel = 0;
00200     bool m_wasPlayerPresent = false;
00201     bool &m_showDebug;
00202
00203     static bool checkCircularCollision(const ecs::Transform &transform1, const ecs::Hitbox &hitbox1,
00204                                       const ecs::Transform &transform2, const ecs::Hitbox &hitbox2)
00205     {
00206         const float x1 = transform1.x + hitbox1.offsetX;
00207         const float y1 = transform1.y + hitbox1.offsetY;
00208         const float x2 = transform2.x + hitbox2.offsetX;
00209         const float y2 = transform2.y + hitbox2.offsetY;
00210
00211         const float dx = x1 - x2;
00212         const float dy = y1 - y2;
00213         const float distance = std::sqrt(dx * dx + dy * dy);
00214         const float combinedRadius = hitbox1.radius + hitbox2.radius;
00215
00216         return distance < combinedRadius;
00217     }
00218
00219     static void removeProjectile(ecs::Registry &registry, ecs::Entity entity)
00220     {
00221         if (registry.hasComponent<ecs::Projectile>(entity)) {
00222             registry.removeComponent<ecs::Projectile>(entity);
00223         }
00224         if (registry.hasComponent<ecs::Transform>(entity)) {
00225             registry.removeComponent<ecs::Transform>(entity);
00226         }
00227         if (registry.hasComponent<ecs::Velocity>(entity)) {
00228             registry.removeComponent<ecs::Velocity>(entity);
00229         }
00230         if (registry.hasComponent<ecs::Rect>(entity)) {
00231             registry.removeComponent<ecs::Rect>(entity);
00232         }
00233         if (registry.hasComponent<ecs::Texture>(entity)) {
00234             registry.removeComponent<ecs::Texture>(entity);
00235         }
00236         if (registry.hasComponent<ecs::Scale>(entity)) {
00237             registry.removeComponent<ecs::Scale>(entity);
00238         }
00239         if (registry.hasComponent<ecs::Animation>(entity)) {
00240             registry.removeComponent<ecs::Animation>(entity);
00241         }
00242         if (registry.hasComponent<ecs::Hitbox>(entity)) {
00243             registry.removeComponent<ecs::Hitbox>(entity);
00244         }
00245     }
00246
00247     static void removeEnemy(ecs::Registry &registry, ecs::Entity entity)
00248     {
00249         if (registry.hasComponent<ecs::Enemy>(entity))
00250         {
00251             registry.removeComponent<ecs::Enemy>(entity);
00252         }
00253         if (registry.hasComponent<ecs::Transform>(entity)) {
00254             registry.removeComponent<ecs::Transform>(entity);
00255         }
00256         if (registry.hasComponent<ecs::Velocity>(entity)) {
00257             registry.removeComponent<ecs::Velocity>(entity);
00258         }
00259         if (registry.hasComponent<ecs::Rect>(entity)) {
00260             registry.removeComponent<ecs::Rect>(entity);
00261         }
00262         if (registry.hasComponent<ecs::Texture>(entity)) {
00263             registry.removeComponent<ecs::Texture>(entity);
00264         }
00265         if (registry.hasComponent<ecs::Scale>(entity)) {
00266             registry.removeComponent<ecs::Scale>(entity);
00267         }
00268         if (registry.hasComponent<ecs::Animation>(entity)) {
00269             registry.removeComponent<ecs::Animation>(entity);
00270         }
00271         if (registry.hasComponent<ecs::Hitbox>(entity)) {
00272             registry.removeComponent<ecs::Hitbox>(entity);
00273         }
00274         if (registry.hasComponent<ecs::Projectile>(entity)) {
00275             registry.removeComponent<ecs::Projectile>(entity);
00276         }

```

```

00277     }
00278
00279     static void createExplosion(ecs::Registry &registry, float x, float y)
00280     {
00281         registry.createEntity()
00282             .with<ecs::Transform>("explosion_transform", x, y, 0.0f)
00283             .with<ecs::Rect>("explosion_rect", 0.0f, 0.0f,
00284                 static_cast<int>(utl::GameConfig::Explosion::SPRITE_WIDTH),
00285                 static_cast<int>(utl::GameConfig::Explosion::SPRITE_HEIGHT))
00286             .with<ecs::Scale>("explosion_scale", utl::GameConfig::Explosion::SCALE,
00287                 utl::GameConfig::Explosion::SCALE)
00288             .with<ecs::Texture>("explosion_texture", utl::Path::Texture::TEXTURE_EXPLOSION)
00289             .with<ecs::Explosion>("explosion", 0, utl::GameConfig::Explosion::ANIMATION_FRAMES,
00290                 utl::GameConfig::Explosion::ANIMATION_DURATION, 0.0f,
00291                 utl::GameConfig::Explosion::SPRITE_WIDTH,
00292                 utl::GameConfig::Explosion::SPRITE_HEIGHT, utl::GameConfig::Explosion::FRAMES_PER_ROW,
00293                 utl::GameConfig::Explosion::LIFETIME, 0.0f)
00294             .build();
00295     }
00296
00297     void ensureEnemyDeathChannel(ecs::Registry &registry, const std::size_t channelIndex)
00298     {
00299         if (channelIndex >= m_enemyDeathAudioEntities.size())
00300         {
00301             return;
00302         }
00303
00304         ecs::Entity &entity = m_enemyDeathAudioEntities[channelIndex];
00305         if (entity != ecs::INVALID_ENTITY && registry.hasComponent<ecs::Audio>(entity))
00306         {
00307             return;
00308         }
00309
00310         entity = registry.createEntity()
00311             .with<ecs::Audio>("enemy_death_" + std::to_string(channelIndex),
00312                 utl::Path::Audio::AUDIO_DEATH_ENEMIES, 1.5f, false, false)
00313             .build();
00314     }
00315
00316     void playEnemyDeathSound(ecs::Registry &registry)
00317     {
00318         ensureEnemyDeathChannel(registry, m_nextEnemyDeathChannel);
00319         const ecs::Entity entity = m_enemyDeathAudioEntities[m_nextEnemyDeathChannel];
00320         m_nextEnemyDeathChannel = (m_nextEnemyDeathChannel + 1) % m_enemyDeathAudioEntities.size();
00321
00322         if (entity == ecs::INVALID_ENTITY)
00323         {
00324             return;
00325         }
00326
00327         if (auto *audio = registry.getComponent<ecs::Audio>(entity))
00328         {
00329             audio->play = true;
00330         }
00331     }
00332
00333     void ensurePlayerDeathAudio(ecs::Registry &registry)
00334     {
00335         if (m_playerDeathAudioEntity != ecs::INVALID_ENTITY &&
00336             registry.hasComponent<ecs::Audio>(m_playerDeathAudioEntity))
00337         {
00338             return;
00339         }
00340
00341         m_playerDeathAudioEntity =
00342             registry.createEntity()
00343                 .with<ecs::Audio>("player_death", utl::Path::Audio::AUDIO_DEATH_ALLIES, 1.5f, false, false)
00344                 .build();
00345     }
00346
00347     void playPlayerDeathSound(ecs::Registry &registry)
00348     {
00349         ensurePlayerDeathAudio(registry);
00350         if (m_playerDeathAudioEntity == ecs::INVALID_ENTITY)
00351         {
00352             return;
00353         }
00354
00355         if (auto *audio = registry.getComponent<ecs::Audio>(m_playerDeathAudioEntity))
00356         {
00357             audio->play = true;
00358         }
00359     }
00360 }; // class CollisionSystem
00361 } // namespace gme

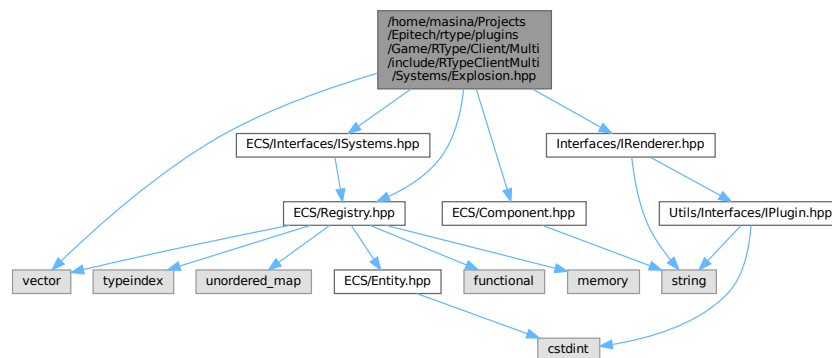
```


19.171 /home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/include/RTypeClientMulti/Systems/Explosion.hpp File Reference

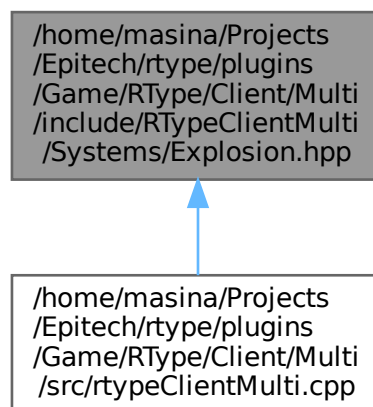
This file contains the explosion system definitions.

```
#include <vector>
#include "ECS/Component.hpp"
#include "ECS/Interfaces/ISystems.hpp"
#include "ECS/Registry.hpp"
#include "Interfaces/IRenderer.hpp"
```

Include dependency graph for Explosion.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [gme::ExplosionSystem](#)

Namespaces

- namespace [gme](#)

19.171.1 Detailed Description

This file contains the explosion system definitions.
Definition in file [Explosion.hpp](#).

19.172 Explosion.hpp

[Go to the documentation of this file.](#)

```

00001 ///  

00002 ///  

00003 ///  

00004 ///  

00005 ///  

00006 ///  

00007 #pragma once  

00008  

00009 #include <vector>  

00010  

00011 #include "ECS/Component.hpp"  

00012 #include "ECS/Interfaces/ISystems.hpp"  

00013 #include "ECS/Registry.hpp"  

00014 #include "Interfaces/IRenderer.hpp"  

00015  

00016 namespace gme  

00017 {  

00018  

00019     class ExplosionSystem final : public ecs::ASystem  

00020     {  

00021     public:  

00022         explicit ExplosionSystem(const std::shared_ptr<eng::IRenderer> &renderer) : m_renderer(renderer) {}  

00023         ~ExplosionSystem() override = default;  

00024  

00025         ExplosionSystem(const ExplosionSystem &) = delete;  

00026         ExplosionSystem &operator=(const ExplosionSystem &) = delete;  

00027         ExplosionSystem(ExplosionSystem &&) = delete;  

00028         ExplosionSystem &operator=(ExplosionSystem &&) = delete;  

00029  

00030         bool isEnabled() override { return true; }  

00031         void setEnable(bool enable) override { (void)enable; }  

00032  

00033         void update(ecs::Registry &registry, float dt) override  

00034         {  

00035             std::vector<ecs::Entity> explosionsToRemove;  

00036  

00037             for (auto &[entity, explosion] : registry.getAll<ecs::Explosion>())  

00038             {  

00039                 auto *transform = registry.GetComponent<ecs::Transform>(entity);  

00040                 auto *rect = registry.GetComponent<ecs::Rect>(entity);  

00041                 auto *texture = registry.GetComponent<ecs::Texture>(entity);  

00042                 auto *scale = registry.GetComponent<ecs::Scale>(entity);  

00043  

00044                 if ((transform == nullptr) || (rect == nullptr) || (texture == nullptr) || (scale == nullptr))  

00045                 {  

00046                     continue;  

00047                 }  

00048  

00049                 explosion.current_time += dt;  

00050                 if (explosion.current_time >= explosion.frame_duration)  

00051                 {  

00052                     explosion.current_time = 0.0f;  

00053                     explosion.current_frame = (explosion.current_frame + 1) % explosion.total_frames;  

00054  

00055                     int frame_x = (explosion.current_frame % explosion.frames_per_row) *  

00056                         static_cast<int>(explosion.frame_width);  

00057                     int frame_y = (explosion.current_frame / explosion.frames_per_row) *  

00058                         static_cast<int>(explosion.frame_height);  

00059  

00060                     rect->pos_x = static_cast<float>(frame_x);  

00061                     rect->pos_y = static_cast<float>(frame_y);  

00062                 }  

00063  

00064                 m_renderer->createSprite(texture->id + std::to_string(entity), texture->path, transform->x,  

00065                     transform->y, scale->x, scale->y, static_cast<int>(rect->pos_x),  

00066                     static_cast<int>(rect->pos_y), rect->size_x, rect->size_y);  

00067                 m_renderer->drawSprite(texture->id + std::to_string(entity));  

00068  

00069                 explosion.current_lifetime += dt;  

00070                 if (explosion.current_lifetime >= explosion.lifetime)  

00071                 {  

00072                     explosionsToRemove.push_back(entity);  

00073                 }  

00074             }  

00075  

00076             for (const ecs::Entity entity : explosionsToRemove)
```

```

00077     {
00078         removeExplosion(registry, entity);
00079     }
00080 }
00081
00082 private:
00083     const std::shared_ptr<eng::IRenderer> &m_renderer;
00084
00085     static void removeExplosion(ecs::Registry &registry, const ecs::Entity entity)
00086     {
00087         if (registry.hasComponent<ecs::Explosion>(entity))
00088             registry.removeComponent<ecs::Explosion>(entity);
00089         if (registry.hasComponent<ecs::Transform>(entity))
00090             registry.removeComponent<ecs::Transform>(entity);
00091         if (registry.hasComponent<ecs::Rect>(entity))
00092             registry.removeComponent<ecs::Rect>(entity);
00093         if (registry.hasComponent<ecs::Texture>(entity))
00094             registry.removeComponent<ecs::Texture>(entity);
00095         if (registry.hasComponent<ecs::Scale>(entity))
00096             registry.removeComponent<ecs::Scale>(entity);
00097     }
00098 }; // class ExplosionSystem
00099 } // namespace gme

```

19.173 /home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Solo/include/RTypeClientSolo/Systems/Explosion.hpp File Reference

This file contains the explosion system definitions.

```
#include <vector>
```

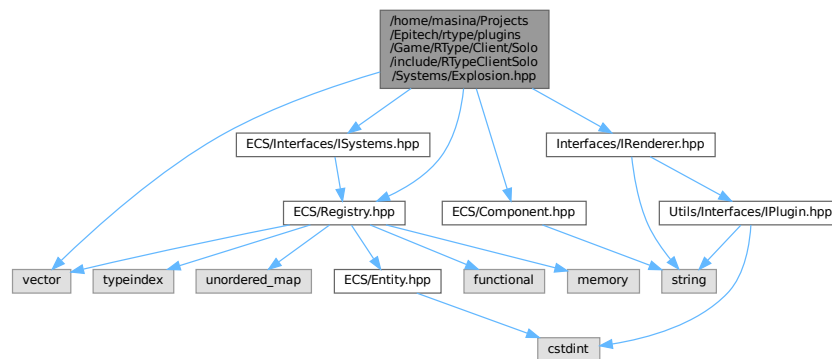
```
#include "ECS/Component.hpp"
```

```
#include "ECS/Interfaces/ISystems.hpp"
```

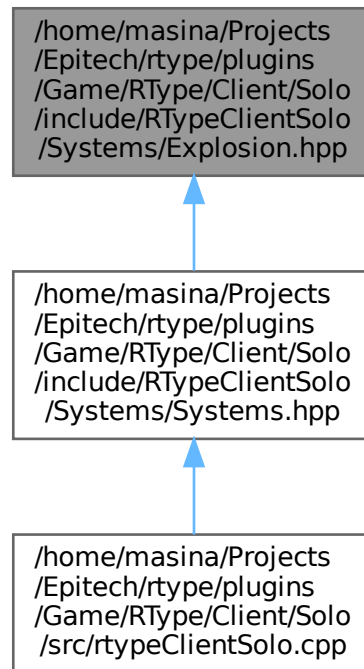
```
#include "ECS/Registry.hpp"
```

```
#include "Interfaces/IRenderer.hpp"
```

Include dependency graph for Explosion.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [gme::ExplosionSystem](#)

Namespaces

- namespace [gme](#)

19.173.1 Detailed Description

This file contains the explosion system definitions.

Definition in file [Explosion.hpp](#).

19.174 Explosion.hpp

[Go to the documentation of this file.](#)

```

00001 ///  

00002 ///  

00003 ///  

00004 ///  

00005 ///  

00006 ///  

00007 #pragma once  

00008  

00009 #include <vector>  

00010  

00011 #include "ECS/Component.hpp"  

00012 #include "ECS/Interfaces/ISystems.hpp"  

00013 #include "ECS/Registry.hpp"  

00014 #include "Interfaces/IRenderer.hpp"  

00015  

00016 namespace gme  

00017 {
  
```

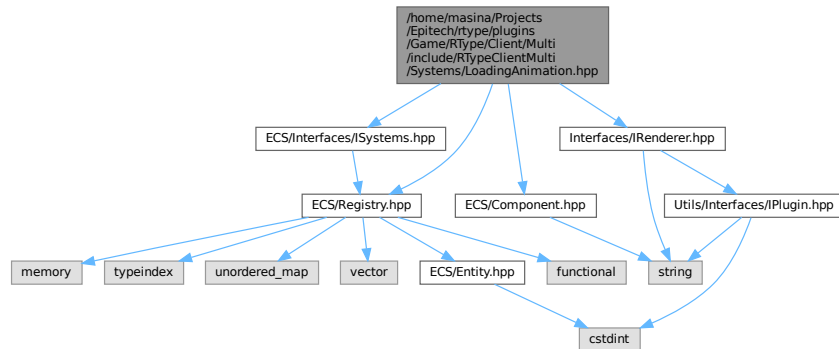
```

00018
00019 class ExplosionSystem final : public ecs::ASystem
00020 {
00021     public:
00022         explicit ExplosionSystem(const std::shared_ptr<eng::IRenderer> &renderer) : m_renderer(renderer) {}
00023         ~ExplosionSystem() override = default;
00024
00025         ExplosionSystem(const ExplosionSystem &) = delete;
00026         ExplosionSystem &operator=(const ExplosionSystem &) = delete;
00027         ExplosionSystem(ExplosionSystem &&) = delete;
00028         ExplosionSystem &operator=(ExplosionSystem &&) = delete;
00029
00030         bool isEnabled() override { return true; }
00031         void setEnable(bool enable) override { (void)enable; }
00032
00033         void update(ecs::Registry &registry, float dt) override
00034         {
00035             std::vector<ecs::Entity> explosionsToRemove;
00036
00037             for (auto &[entity, explosion] : registry.getAll<ecs::Explosion>())
00038             {
00039                 auto *transform = registry.getComponent<ecs::Transform>(entity);
00040                 auto *rect = registry.getComponent<ecs::Rect>(entity);
00041                 auto *texture = registry.getComponent<ecs::Texture>(entity);
00042                 auto *scale = registry.getComponent<ecs::Scale>(entity);
00043
00044                 if ((transform == nullptr) || (rect == nullptr) || (texture == nullptr) || (scale == nullptr))
00045                 {
00046                     continue;
00047                 }
00048
00049                 explosion.current_time += dt;
00050                 if (explosion.current_time >= explosion.frame_duration)
00051                 {
00052                     explosion.current_time = 0.0f;
00053                     explosion.current_frame = (explosion.current_frame + 1) % explosion.total_frames;
00054
00055                     int frame_x = (explosion.current_frame % explosion.frames_per_row) *
00056                         static_cast<int>(explosion.frame_width);
00057                     int frame_y = (explosion.current_frame / explosion.frames_per_row) *
00058                         static_cast<int>(explosion.frame_height);
00059
00060                     rect->pos_x = static_cast<float>(frame_x);
00061                     rect->pos_y = static_cast<float>(frame_y);
00062                 }
00063
00064                 m_renderer->createSprite(texture->id + std::to_string(entity), texture->path, transform->x,
00065                                         transform->y, scale->x, scale->y, static_cast<int>(rect->pos_x),
00066                                         static_cast<int>(rect->pos_y), rect->size_x, rect->size_y);
00067                 m_renderer->drawSprite(texture->id + std::to_string(entity));
00068
00069                 explosion.current_lifetime += dt;
00070                 if (explosion.current_lifetime >= explosion.lifetime)
00071                 {
00072                     explosionsToRemove.push_back(entity);
00073                 }
00074             }
00075
00076             for (const ecs::Entity entity : explosionsToRemove)
00077             {
00078                 removeExplosion(registry, entity);
00079             }
00080         }
00081
00082     private:
00083         const std::shared_ptr<eng::IRenderer> &m_renderer;
00084
00085         static void removeExplosion(ecs::Registry &registry, const ecs::Entity entity)
00086         {
00087             if (registry.hasComponent<ecs::Explosion>(entity))
00088                 registry.removeComponent<ecs::Explosion>(entity);
00089             if (registry.hasComponent<ecs::Transform>(entity))
00090                 registry.removeComponent<ecs::Transform>(entity);
00091             if (registry.hasComponent<ecs::Rect>(entity))
00092                 registry.removeComponent<ecs::Rect>(entity);
00093             if (registry.hasComponent<ecs::Texture>(entity))
00094                 registry.removeComponent<ecs::Texture>(entity);
00095             if (registry.hasComponent<ecs::Scale>(entity))
00096                 registry.removeComponent<ecs::Scale>(entity);
00097         }
00098     }; // class ExplosionSystem
00099 } // namespace gme

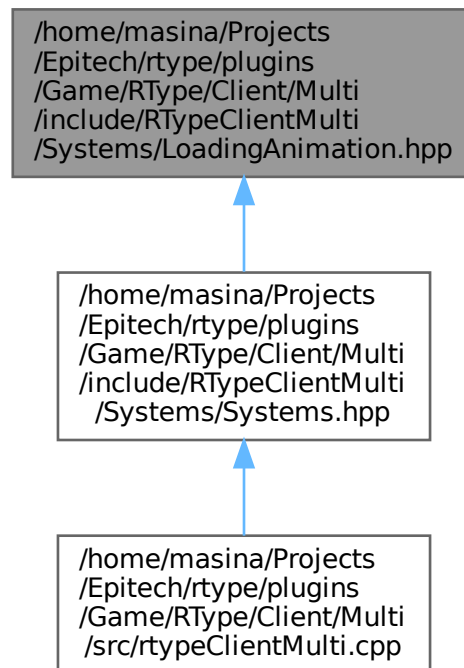
```

19.175 /home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/include/RTypeClientMulti/Systems/LoadingAnimation.hpp File Reference

```
#include "ECS/Component.hpp"
#include "ECS/Interfaces/ISystems.hpp"
#include "ECS/Registry.hpp"
#include "Interfaces/IRenderer.hpp"
Include dependency graph for LoadingAnimation.hpp:
```



This graph shows which files directly or indirectly include this file:



Classes

- class [gme::LoadingAnimationSystem](#)

Namespaces

- namespace [gme](#)

19.176 LoadingAnimation.hpp

[Go to the documentation of this file.](#)

```

00001 #pragma once
00002
00003 #include "ECS/Component.hpp"
00004 #include "ECS/Interfaces/ISystems.hpp"
00005 #include "ECS/Registry.hpp"
00006 #include "Interfaces/IRenderer.hpp"
00007
00008 namespace gme
00009 {
00010
00011     class LoadingAnimationSystem final : public ecs::ASystem
00012     {
00013     public:
00014         explicit LoadingAnimationSystem(const std::shared_ptr<eng::IRenderer> &renderer) : m_renderer(renderer) {}
00015         ~LoadingAnimationSystem() override = default;
00016
00017         LoadingAnimationSystem(const LoadingAnimationSystem &) = delete;
00018         LoadingAnimationSystem &operator=(const LoadingAnimationSystem &) = delete;
00019         LoadingAnimationSystem(LoadingAnimationSystem &&) = delete;
00020         LoadingAnimationSystem &operator=(const LoadingAnimationSystem &&) = delete;
00021
00022         void update(ecs::Registry &registry, float dt) override
00023         {
00024             std::string spriteName;
00025             spriteName.reserve(64);
00026
00027             for (auto &[entity, animation] : registry.getAll<ecs::LoadingAnimation>())
00028             {
00029                 const auto *transform = registry.getComponent<ecs::Transform>(entity);
00030                 auto *rect = registry.getComponent<ecs::Rect>(entity);
00031                 const auto *texture = registry.getComponent<ecs::Texture>(entity);
00032
00033                 if (!transform || !rect || !texture)
00034                 {
00035                     continue;
00036                 }
00037
00038                 animation.current_time += dt;
00039                 if (animation.current_time >= animation.frame_duration)
00040                 {
00041                     animation.current_time = 0.0f;
00042                     animation.current_frame = (animation.current_frame + 1) % animation.total_frames;
00043
00044                     const int frame_x = (animation.current_frame % animation.frames_per_row) *
00045                         static_cast<int>(animation.frame_width);
00046                     const int frame_y = (animation.current_frame / animation.frames_per_row) *
00047                         static_cast<int>(animation.frame_height);
00048
00049                     rect->pos_x = static_cast<float>(frame_x);
00050                     rect->pos_y = static_cast<float>(frame_y);
00051                 }
00052
00053                 spriteName.clear();
00054                 spriteName = texture->id;
00055                 spriteName += std::to_string(entity);
00056
00057                 m_renderer->setSpriteTexture(spriteName, texture->path);
00058                 m_renderer->setSpritePosition(spriteName, transform->x, transform->y);
00059                 m_renderer->setSpriteFrame(spriteName, static_cast<int>(rect->pos_x), static_cast<int>(rect->pos_y),
00060                     rect->size_x, rect->size_y);
00061                 m_renderer->drawSprite(spriteName);
00062             }
00063         }
00064
00065     private:
00066         const std::shared_ptr<eng::IRenderer> &m_renderer;
00067     };
00068 } // namespace gme

```

19.177 /home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Solo/include/RTypeClientSolo/Systems/LoadingAnimation.hpp File Reference

This file contains the loading animation system definitions.

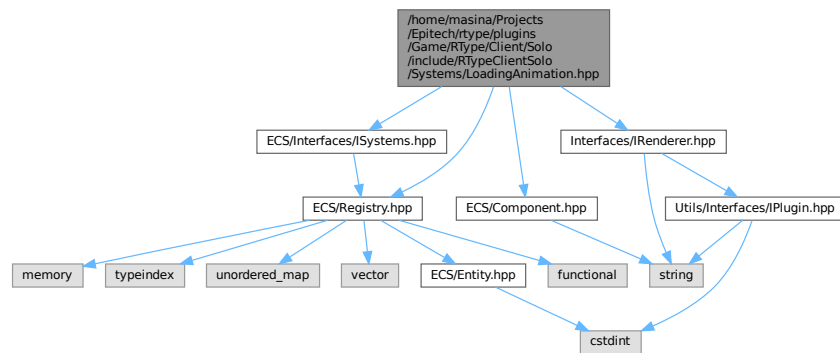
```
#include "ECS/Component.hpp"
```

```
#include "ECS/Interfaces/ISystems.hpp"
```

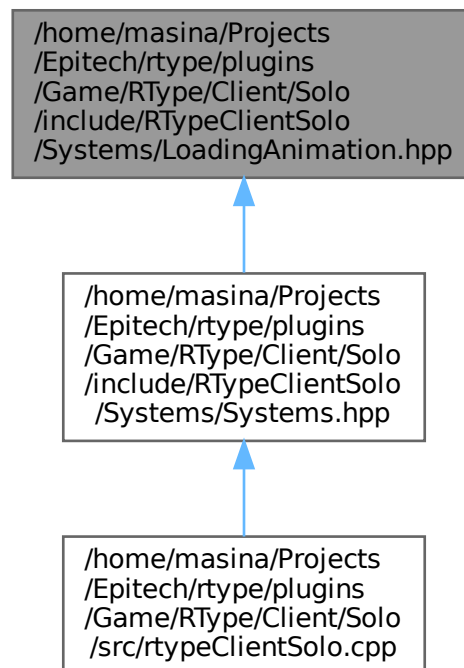
```
#include "ECS/Registry.hpp"
```

```
#include "Interfaces/IRenderer.hpp"
```

Include dependency graph for LoadingAnimation.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [gme::LoadingAnimationSystem](#)

Namespaces

- namespace [gme](#)

19.177.1 Detailed Description

This file contains the loading animation system definitions.

Definition in file [LoadingAnimation.hpp](#).

19.178 LoadingAnimation.hpp

[Go to the documentation of this file.](#)

```

00001 ///
00002 /// @file LoadingAnimation.hpp
00003 /// @brief This file contains the loading animation system definitions
00004 /// @namespace gme
00005 ///
00006
00007 #pragma once
00008
00009 #include "ECS/Component.hpp"
00010 #include "ECS/Interfaces/ISystems.hpp"
00011 #include "ECS/Registry.hpp"
00012 #include "Interfaces/IRenderer.hpp"
00013
00014 namespace gme
00015 {
00016
00017     class LoadingAnimationSystem final : public ecs::ASystem
00018     {
00019     public:
00020         explicit LoadingAnimationSystem(const std::shared_ptr<eng::IRenderer> &renderer) : m_renderer(renderer) {}
00021         ~LoadingAnimationSystem() override = default;
00022
00023         LoadingAnimationSystem(const LoadingAnimationSystem &) = delete;
00024         LoadingAnimationSystem &operator=(const LoadingAnimationSystem &) = delete;
00025         LoadingAnimationSystem(LoadingAnimationSystem &&) = delete;
00026         LoadingAnimationSystem &operator=(LoadingAnimationSystem &&) = delete;
00027
00028         void update(ecs::Registry &registry, float dt) override
00029         {
00030             for (auto &[entity, animation] : registry.getAll<ecs::LoadingAnimation>())
00031             {
00032                 const auto *transform = registry.getComponent<ecs::Transform>(entity);
00033                 auto *rect = registry.getComponent<ecs::Rect>(entity);
00034                 const auto *texture = registry.getComponent<ecs::Texture>(entity);
00035
00036                 if (!transform || !rect || !texture)
00037                 {
00038                     continue;
00039                 }
00040
00041                 animation.current_time += dt;
00042                 if (animation.current_time >= animation.frame_duration)
00043                 {
00044                     animation.current_time = 0.0f;
00045                     animation.current_frame = (animation.current_frame + 1) % animation.total_frames;
00046
00047                     int frame_x = (animation.current_frame % animation.frames_per_row) *
00048                         static_cast<int>(animation.frame_width);
00049                     int frame_y = (animation.current_frame / animation.frames_per_row) *
00050                         static_cast<int>(animation.frame_height);
00051
00052                     rect->pos_x = static_cast<float>(frame_x);
00053                     rect->pos_y = static_cast<float>(frame_y);
00054                 }
00055
00056                 m_renderer->setSpriteTexture(texture->id + std::to_string(entity), texture->path);
00057                 m_renderer->setSpritePosition(texture->id + std::to_string(entity), transform->x, transform->y);
00058                 m_renderer->setSpriteFrame(texture->id + std::to_string(entity), static_cast<int>(rect->pos_x),
00059                     static_cast<int>(rect->pos_y), rect->size_x, rect->size_y);
00060                 m_renderer->drawSprite(texture->id + std::to_string(entity));
00061             }
00062         }
00063
00064     private:

```

```

00065     const std::shared_ptr<eng::IRenderer> &m_renderer;
00066 }; // class LoadingAnimationSystem
00067
00068 } // namespace gme

```

19.179 /home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/include/RTypeClientMulti/Systems/PlayerControllerMulti.hpp File Reference

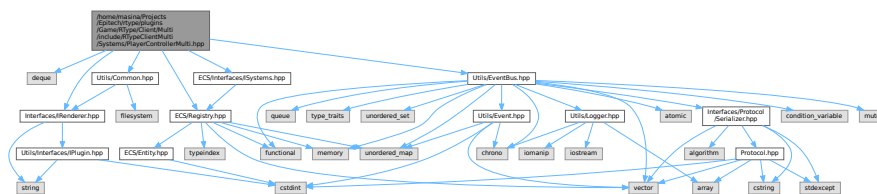
Player Controller System for multiplayer (sends inputs to server)

```

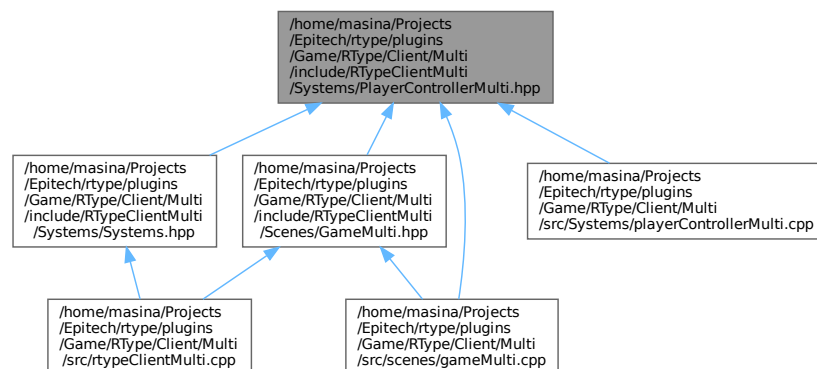
#include <deque>
#include "ECS/Interfaces/ISystems.hpp"
#include "ECS/Registry.hpp"
#include "Interfaces/IRenderer.hpp"
#include "Utils/Common.hpp"
#include "Utils/EventBus.hpp"

```

Include dependency graph for PlayerControllerMulti.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [gme::PlayerControllerMulti](#)
- struct [gme::PlayerControllerMulti::PendingInput](#)

Namespaces

- namespace [gme](#)

19.179.1 Detailed Description

Player Controller System for multiplayer (sends inputs to server)

Definition in file [PlayerControllerMulti.hpp](#).

19.180 PlayerControllerMulti.hpp

[Go to the documentation of this file.](#)

```

00001 ///  

00002 ///  

00003 ///  

00004 ///  

00005 ///  

00006 ///  

00007 #pragma once  

00008 ///  

00009 #include <deque>  

00010 ///  

00011 #include "ECS/Interfaces/ISystems.hpp"  

00012 #include "ECS/Registry.hpp"  

00013 #include "Interfaces/IRenderer.hpp"  

00014 #include "Utils/Common.hpp"  

00015 #include "Utils/EventBus.hpp"  

00016 ///  

00017 namespace gme  

00018 {  

00019     class PlayerControllerMulti final : public ecs::ASystem  

00020     {  

00021     public:  

00022         explicit PlayerControllerMulti(const std::shared_ptr<eng::IRenderer> &renderer, uint32_t sessionId)  

00023             : m_renderer(renderer), m_playerEntity(0), m_eventBus(utl::EventBus::getInstance())  

00024         {  

00025             m_componentId = 11;  

00026             m_eventBus.registerComponent(m_componentId, "PlayerControllerMulti");  

00027         }  

00028         ~PlayerControllerMulti() override = default;  

00029         PlayerControllerMulti(const PlayerControllerMulti &) = delete;  

00030         PlayerControllerMulti &operator=(const PlayerControllerMulti &) = delete;  

00031         PlayerControllerMulti(PlayerControllerMulti &&) = delete;  

00032         PlayerControllerMulti &operator=(PlayerControllerMulti &&) = delete;  

00033         void update(ecs::Registry &registry, float dt) override;  

00034         void handleInput(ecs::Registry &registry, const eng::Event &event);  

00035         ecs::Entity createPlayer(ecs::Registry &registry, float x, float y);  

00036         bool isSpacePressed() const;  

00037     private:  

00038         void sendInputToServer(bool up, bool down, bool left, bool right, bool shoot) const;  

00039         void sendInputsIfChanged();  

00040         const std::shared_ptr<eng::IRenderer> &m_renderer;  

00041         std::unordered_map<eng::Key, bool> m_keysPressed;  

00042         ecs::Entity m_playerEntity;  

00043         uint32_t m_componentId;  

00044         utl::EventBus &m_eventBus;  

00045         const float INPUT_THROTTLE_INTERVAL = 1.0f / 144.0f; // Send inputs at 144 Hz max  

00046         uint32_t m_nextSeqId = 1;  

00047         uint32_t m_lastAckSeqId = 0;  

00048         struct PendingInput  

00049         {  

00050             uint32_t seqId;  

00051             std::vector<uint8_t> inputData;  

00052             float dt;  

00053         };  

00054     }; // class PlayerControllerMulti  

00055 } // namespace gme

```

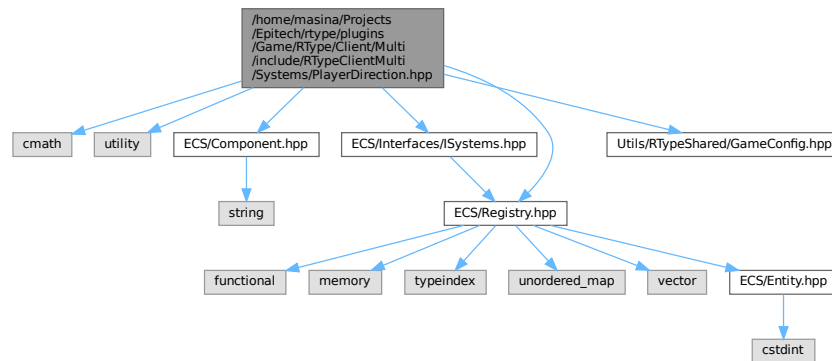
19.181 /home/masina/Projects/Epitech/rtype/plugins/Game/RType/↵ Client/Multi/include/RTypeClientMulti/Systems/Player↵ Direction.hpp File Reference

```

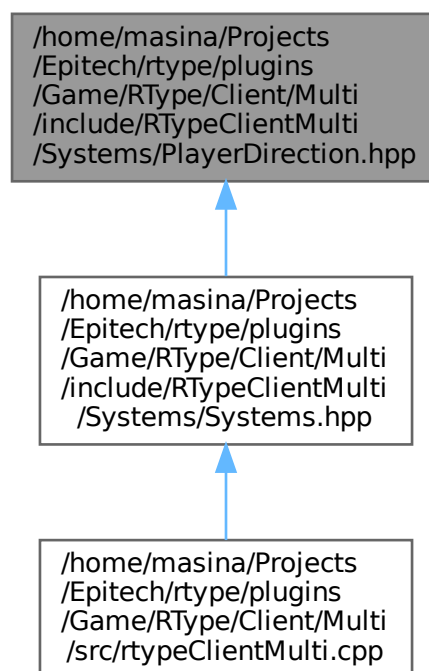
#include <cmath>
#include <utility>
#include "ECS/Component.hpp"
#include "ECS/Interfaces/ISystems.hpp"
#include "ECS/Registry.hpp"
#include "Utils/RTypeShared/GameConfig.hpp"

```

Include dependency graph for `PlayerDirection.hpp`:



This graph shows which files directly or indirectly include this file:



Classes

- class [gme::PlayerDirectionSystem](#)

Namespaces

- namespace [gme](#)

Macros

- `#define M_PI 3.14159265358979323846`

19.181.1 Macro Definition Documentation

19.181.1.1 M_PI

`#define M_PI 3.14159265358979323846`

Definition at line 12 of file [PlayerDirection.hpp](#).

Referenced by [gme::PlayerDirectionSystem::update\(\)](#).

19.182 PlayerDirection.hpp

[Go to the documentation of this file.](#)

```

00001 #pragma once
00002
00003 #include <cmath>
00004 #include <utility>
00005
00006 #include "ECS/Component.hpp"
00007 #include "ECS/Interfaces/ISystems.hpp"
00008 #include "ECS/Registry.hpp"
00009 #include "Utils/RTypeShared/GameConfig.hpp"
00010
00011 #ifndef M_PI
00012 #define M_PI 3.14159265358979323846
00013 #endif
00014
00015 namespace gme
00016 {
00017     class PlayerDirectionSystem final : public ecs::ASystem
00018     {
00019     public:
00020         PlayerDirectionSystem() = default;
00021         ~PlayerDirectionSystem() override = default;
00022
00023         PlayerDirectionSystem(const PlayerDirectionSystem &) = delete;
00024         PlayerDirectionSystem &operator=(const PlayerDirectionSystem &) = delete;
00025         PlayerDirectionSystem(PlayerDirectionSystem &&) = delete;
00026         PlayerDirectionSystem &operator=(PlayerDirectionSystem &&) = delete;
00027
00028         void update(ecs::Registry &registry, float /* dt */ override)
00029         {
00030             for (auto &pair : registry.getAll<ecs::Player>())
00031             {
00032                 const auto entity = pair.first;
00033                 const auto *velocity = registry.getComponent<ecs::Velocity>(entity);
00034
00035                 if (auto *rect = registry.getComponent<ecs::Rect>(entity);
00036                     (velocity != nullptr) && (rect != nullptr))
00037                 {
00038                     int frame = 0;
00039                     float angle = std::atan2(velocity->y, velocity->x);
00040
00041                     const bool isIdle = (std::abs(velocity->x) < 0.1f && std::abs(velocity->y) < 0.1f);
00042                     if (isIdle)
00043                     {
00044                         frame = 0;
00045                     }
00046                     else
00047                     {
00048                         if (angle < 0)
00049                         {
00050                             angle += 2.0f * static_cast<float>(M_PI);
00051                         }
00052                         if (angle >= 0 && angle < M_PI / 4)
00053                         {
00054                             frame = 0; // droite
00055                         }
00056                         else if (angle >= M_PI / 4 && angle < 3 * M_PI / 4)
00057                         {
00058                             frame = 1; // bas
00059                         }
00060                         else if (angle >= 3 * M_PI / 4 && angle < 5 * M_PI / 4)
00061                         {
00062                             frame = 2; // gauche
00063                         }
00064                         else if (angle >= 5 * M_PI / 4 && angle < 7 * M_PI / 4)
00065                         {
00066                             frame = 3; // haut

```

```

00067         }
00068         else
00069         {
00070             frame = 4; // wrap
00071         }
00072     }
00073
00074     const int frame_x = (frame % utl::GameConfig::Player::FRAMES_PER_ROW) *
00075         static_cast<int>(utl::GameConfig::Player::SPRITE_WIDTH);
00076     int frame_y = (frame / utl::GameConfig::Player::FRAMES_PER_ROW) *
00077         static_cast<int>(utl::GameConfig::Player::SPRITE_HEIGHT);
00078
00079     const int current_row = static_cast<int>(
00080         rect->pos_y / static_cast<float>(static_cast<int>(utl::GameConfig::Player::SPRITE_HEIGHT)));
00081     const int skin_offset = current_row * static_cast<int>(utl::GameConfig::Player::SPRITE_HEIGHT);
00082     frame_y = skin_offset + frame_y;
00083
00084     rect->pos_x = static_cast<float>(frame_x);
00085     rect->pos_y = static_cast<float>(frame_y);
00086     rect->size_x = static_cast<int>(utl::GameConfig::Player::SPRITE_WIDTH);
00087     rect->size_y = static_cast<int>(utl::GameConfig::Player::SPRITE_HEIGHT);
00088 }
00089 }
00090 }
00091 };
00092 } // namespace gme

```

19.183 /home/masina/Projects/Epitech/rtype/plugins/Game/RType/↵ Client/Solo/include/RTypeClientSolo/Systems/Player↵ Direction.hpp File Reference

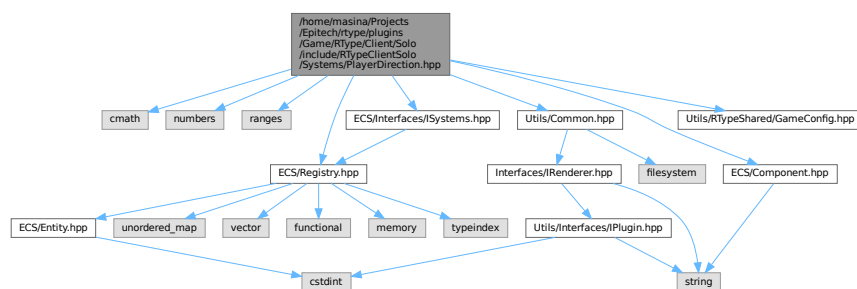
This file contains the player direction system definitions.

```

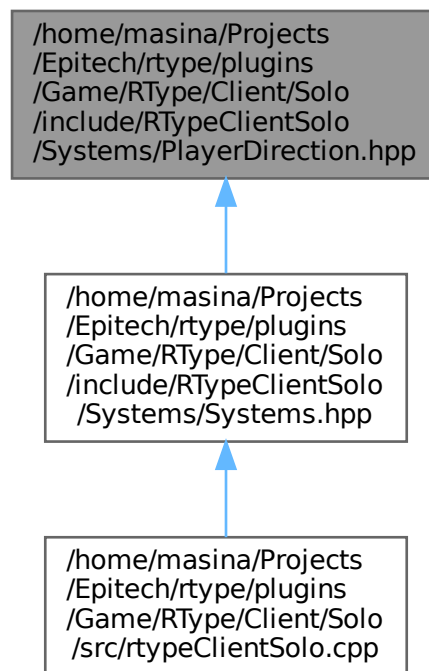
#include <cmath>
#include <numbers>
#include <ranges>
#include "ECS/Component.hpp"
#include "ECS/Interfaces/ISystems.hpp"
#include "ECS/Registry.hpp"
#include "Utils/Common.hpp"
#include "Utils/RTypeShared/GameConfig.hpp"

```

Include dependency graph for PlayerDirection.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [gme::PlayerDirectionSystem](#)

Namespaces

- namespace [gme](#)

Macros

- `#define` [M_PI](#) 3.14159265358979323846

19.183.1 Detailed Description

This file contains the player direction system definitions.
Definition in file [PlayerDirection.hpp](#).

19.183.2 Macro Definition Documentation

19.183.2.1 M_PI

`#define M_PI 3.14159265358979323846`

Definition at line 20 of file [PlayerDirection.hpp](#).

19.184 PlayerDirection.hpp

[Go to the documentation of this file.](#)

00001 ///

```

00002 /// @file PlayerDirection.hpp
00003 /// @brief This file contains the player direction system definitions
00004 /// @namespace gme
00005 ///
00006
00007 #pragma once
00008
00009 #include <cmath>
00010 #include <numbers>
00011 #include <ranges>
00012
00013 #include "ECS/Component.hpp"
00014 #include "ECS/Interfaces/ISystems.hpp"
00015 #include "ECS/Registry.hpp"
00016 #include "Utils/Common.hpp"
00017 #include "Utils/RTypeShared/GameConfig.hpp"
00018
00019 #ifndef M_PI
00020 #define M_PI 3.14159265358979323846
00021 #endif
00022
00023 namespace gme
00024 {
00025     struct AppConfig;
00026
00027     class PlayerDirectionSystem final : public ecs::ASystem
00028     {
00029     public:
00030         explicit PlayerDirectionSystem(const int skinIndex) : m_skinIndex(skinIndex) {}
00031         ~PlayerDirectionSystem() override = default;
00032
00033         PlayerDirectionSystem(const PlayerDirectionSystem &) = delete;
00034         PlayerDirectionSystem &operator=(const PlayerDirectionSystem &) = delete;
00035         PlayerDirectionSystem(PlayerDirectionSystem &&) = delete;
00036         PlayerDirectionSystem &operator=(PlayerDirectionSystem &&) = delete;
00037
00038         void update(ecs::Registry &registry, float /* dt */) override
00039         {
00040             for (const auto &entity : registry.getAll<ecs::Player>() | std::views::keys)
00041             {
00042                 const auto *velocity = registry.getComponent<ecs::Velocity>(entity);
00043
00044                 if (auto *rect = registry.getComponent<ecs::Rect>(entity);
00045                     (velocity != nullptr) && (rect != nullptr))
00046                 {
00047                     int frame = 0;
00048                     float angle = std::atan2(velocity->y, velocity->x);
00049                     if (std::abs(velocity->x) < 0.1f && std::abs(velocity->y) < 0.1f)
00050                     {
00051                         frame = 0;
00052                     }
00053                     else
00054                     {
00055                         if (angle < 0)
00056                         {
00057                             angle += 2.0f * std::numbers::pi_v<float>;
00058                         }
00059                         if (angle >= 0 && angle < M_PI / 4)
00060                         {
00061                             frame = 0;
00062                         }
00063                         else if (angle >= M_PI / 4 && angle < 3 * M_PI / 4)
00064                         {
00065                             frame = 1;
00066                         }
00067                         else if (angle >= 3 * M_PI / 4 && angle < 5 * M_PI / 4)
00068                         {
00069                             frame = 2;
00070                         }
00071                         else if (angle >= 5 * M_PI / 4 && angle < 7 * M_PI / 4)
00072                         {
00073                             frame = 3;
00074                         }
00075                         else
00076                         {
00077                             frame = 4;
00078                         }
00079                     }
00080                     int frame_width = static_cast<int>(utl::GameConfig::Player::SPRITE_WIDTH);
00081                     int frame_height = static_cast<int>(utl::GameConfig::Player::SPRITE_HEIGHT);
00082                     int frames_per_row = utl::GameConfig::Player::FRAMES_PER_ROW;
00083                     int frame_x = (frame % frames_per_row) * frame_width;
00084                     int frame_y = (frame / frames_per_row) * frame_height;
00085
00086                     const int skin_offset = m_skinIndex * static_cast<int>(utl::GameConfig::Player::SPRITE_HEIGHT);
00087                     frame_y += skin_offset;
00088

```



```

00089         rect->pos_x = static_cast<float>(frame_x);
00090         rect->pos_y = static_cast<float>(frame_y);
00091         rect->size_x = frame_width;
00092         rect->size_y = frame_height;
00093     }
00094 }
00095 }
00096
00097 private:
00098     int m_skinIndex;
00099 }; // class PlayerDirectionSystem
00100 } // namespace gme

```

19.185 /home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/include/RTypeClientMulti/Systems/Projectile.hpp File Reference

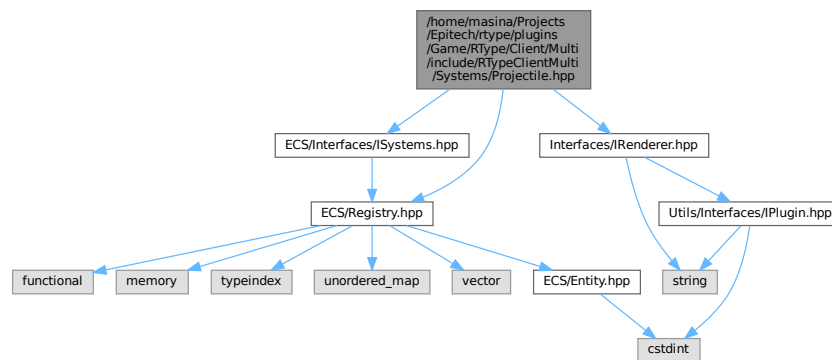
Projectile system for multiplayer.

```
#include "ECS/Interfaces/ISystems.hpp"
```

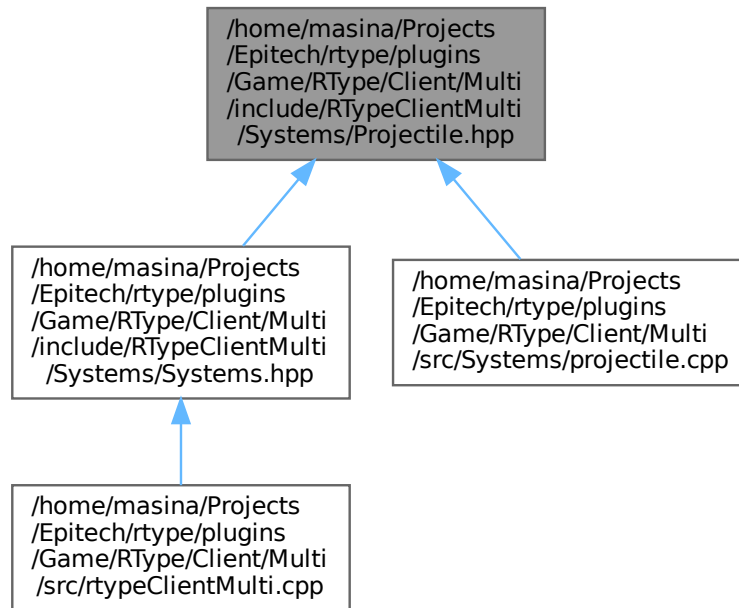
```
#include "ECS/Registry.hpp"
```

```
#include "Interfaces/IRenderer.hpp"
```

Include dependency graph for Projectile.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [gme::ProjectileSystem](#)

Namespaces

- namespace [gme](#)

19.185.1 Detailed Description

Projectile system for multiplayer.

Definition in file [Projectile.hpp](#).

19.186 Projectile.hpp

[Go to the documentation of this file.](#)

```

00001 ///  

00002 ///  

00003 ///  

00004 ///  

00005 ///  

00006 ///  

00007 #pragma once  

00008  

00009 #include "ECS/Interfaces/ISystems.hpp"  

00010 #include "ECS/Registry.hpp"  

00011 #include "Interfaces/IRenderer.hpp"  

00012  

00013 namespace gme  

00014 {  

00015  

00016     class ProjectileSystem final : public ecs::ASystem  

00017     {  

00018     public:  

00019         explicit ProjectileSystem(const std::shared_ptr<eng::IRenderer> & /* renderer */) {}  

00020         ~ProjectileSystem() override = default;  


```

```

00021
00022     ProjectileSystem(const ProjectileSystem &) = delete;
00023     ProjectileSystem &operator=(const ProjectileSystem &) = delete;
00024     ProjectileSystem(ProjectileSystem &&) = delete;
00025     ProjectileSystem &operator=(ProjectileSystem &&) = delete;
00026
00027     void update(ecs::Registry &registry, float dt) override;
00028
00029 }; // class ProjectileSystem
00030 } // namespace gme

```

19.187 /home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Solo/include/RTypeClientSolo/Systems/Projectile.hpp File Reference

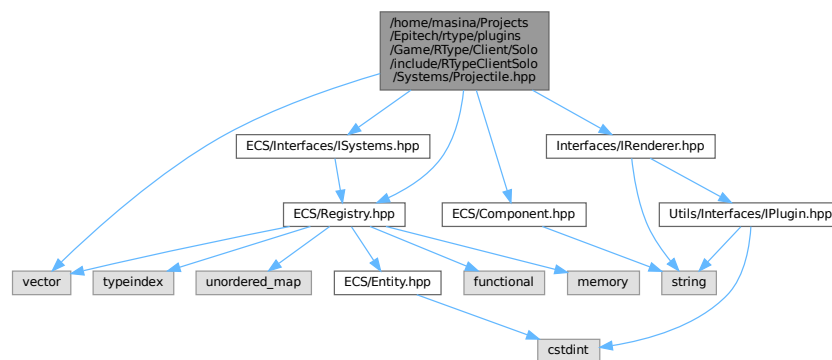
This file contains the projectile system definitions.

```

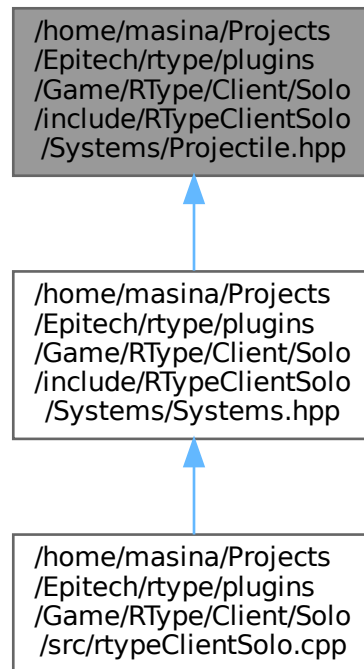
#include <vector>
#include "ECS/Component.hpp"
#include "ECS/Interfaces/ISystems.hpp"
#include "ECS/Registry.hpp"
#include "Interfaces/IRenderer.hpp"

```

Include dependency graph for Projectile.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [gme::ProjectileSystem](#)

Namespaces

- namespace [gme](#)

19.187.1 Detailed Description

This file contains the projectile system definitions.

Definition in file [Projectile.hpp](#).

19.188 Projectile.hpp

[Go to the documentation of this file.](#)

```

00001 ///  

00002 ///  

00003 ///  

00004 ///  

00005 ///  

00006 ///  

00007 #pragma once  

00008  

00009 #include <vector>  

00010  

00011 #include "ECS/Component.hpp"  

00012 #include "ECS/Interfaces/ISystems.hpp"  

00013 #include "ECS/Registry.hpp"  

00014 #include "Interfaces/IRenderer.hpp"  

00015  

00016 namespace gme  

00017 {
  
```

```

00018
00019 class ProjectileSystem final : public ecs::ASystem
00020 {
00021     public:
00022         explicit ProjectileSystem(const std::shared_ptr<eng::IRenderer> & /* renderer */) {}
00023         ~ProjectileSystem() override = default;
00024
00025         ProjectileSystem(const ProjectileSystem &) = delete;
00026         ProjectileSystem &operator=(const ProjectileSystem &) = delete;
00027         ProjectileSystem(ProjectileSystem &&) = delete;
00028         ProjectileSystem &operator=(ProjectileSystem &&) = delete;
00029
00030         void update(ecs::Registry &registry, float dt) override
00031         {
00032             std::vector<ecs::Entity> entitiesToRemove;
00033
00034             for (auto &[entity, projectile] : registry.getAll<ecs::Projectile>())
00035             {
00036                 projectile.current_lifetime += dt;
00037                 if (projectile.current_lifetime >= projectile.lifetime)
00038                 {
00039                     entitiesToRemove.push_back(entity);
00040                     continue;
00041                 }
00042                 auto *transform = registry.getComponent<ecs::Transform>(entity);
00043
00044                 if (auto *velocity = registry.getComponent<ecs::Velocity>(entity);
00045                     (transform != nullptr) && (velocity != nullptr))
00046                 {
00047                     transform->x += velocity->x * dt;
00048                     transform->y += velocity->y * dt;
00049                 }
00050             }
00051
00052             for (const auto &entity : entitiesToRemove)
00053             {
00054                 if (registry.hasComponent<ecs::Projectile>(entity))
00055                     registry.removeComponent<ecs::Projectile>(entity);
00056                 if (registry.hasComponent<ecs::Transform>(entity))
00057                     registry.removeComponent<ecs::Transform>(entity);
00058                 if (registry.hasComponent<ecs::Velocity>(entity))
00059                     registry.removeComponent<ecs::Velocity>(entity);
00060                 if (registry.hasComponent<ecs::Rect>(entity))
00061                     registry.removeComponent<ecs::Rect>(entity);
00062                 if (registry.hasComponent<ecs::Scale>(entity))
00063                     registry.removeComponent<ecs::Scale>(entity);
00064                 if (registry.hasComponent<ecs::Texture>(entity))
00065                     registry.removeComponent<ecs::Texture>(entity);
00066                 if (registry.hasComponent<ecs::Animation>(entity))
00067                     registry.removeComponent<ecs::Animation>(entity);
00068             }
00069         }
00070
00071     }; // class ProjectileSystem
00072 } // namespace gme

```

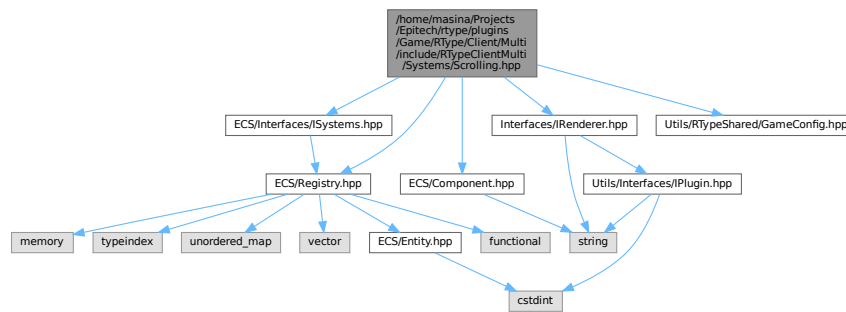
19.189 /home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/include/RTypeClientMulti/Systems/Scrolling.hpp File Reference

```

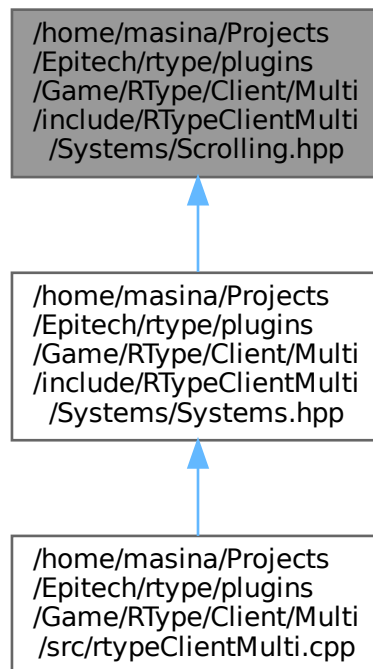
#include "ECS/Component.hpp"
#include "ECS/Interfaces/ISystems.hpp"
#include "ECS/Registry.hpp"
#include "Interfaces/IRenderer.hpp"
#include "Utils/RTypeShared/GameConfig.hpp"

```

Include dependency graph for Scrolling.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [gme::ScrollingSystem](#)

Namespaces

- namespace [gme](#)

19.190 Scrolling.hpp

[Go to the documentation of this file.](#)

```

00001 #pragma once
00002
00003 #include "ECS/Component.hpp"
00004 #include "ECS/Interfaces/ISystems.hpp"
00005 #include "ECS/Registry.hpp"
00006 #include "Interfaces/IRenderer.hpp"
00007 #include "Utils/RTypeShared/GameConfig.hpp"
00008
00009 namespace gme
00010 {
00011     class ScrollingSystem final : public ecs::ASystem
00012     {
00013     public:
00014         explicit ScrollingSystem(const std::shared_ptr<eng::IRenderer> &renderer) : m_renderer(renderer) {}
00015         ~ScrollingSystem() override = default;
00016
00017         ScrollingSystem(const ScrollingSystem &) = delete;
00018         ScrollingSystem &operator=(const ScrollingSystem &) = delete;
00019         ScrollingSystem(ScrollingSystem &&) = delete;
00020         ScrollingSystem &operator=(const ScrollingSystem &&) = delete;
00021
00022         void update(ecs::Registry &registry, float dt) override
00023         {
00024             const auto [width, height] = m_renderer->getWindowSize();
00025
00026             for (auto &[entity, scrolling] : registry.getAll<ecs::Scrolling>())
00027             {
00028                 auto *transform = registry.getComponent<ecs::Transform>(entity);
00029                 auto *scale = registry.getComponent<ecs::Scale>(entity);
00030                 const bool isFloor = registry.hasComponent<ecs::Floor>(entity);
00031                 const bool isCeiling = registry.hasComponent<ecs::Ceiling>(entity);
00032
00033                 if (transform == nullptr)
00034                 {
00035                     continue;
00036                 }
00037
00038                 if (scrolling.fit_width && (scale != nullptr))
00039                 {
00040                     const auto targetWidth = static_cast<float>(width);
00041                     const float scaleX = targetWidth / std::max(1.0f, scrolling.original_width);
00042                     scale->x = scaleX;
00043                     scale->y = scaleX;
00044                 }
00045
00046                 const float scaledHeight = ((scale != nullptr) ? scale->y : 1.0f) * scrolling.original_height;
00047                 if (isCeiling)
00048                 {
00049                     transform->y = utl::GameConfig::Stage::CEILING_OFFSET_Y;
00050                 }
00051                 else if (isFloor)
00052                 {
00053                     transform->y =
00054                         static_cast<float>(height) - scaledHeight - utl::GameConfig::Stage::FLOOR_OFFSET_Y;
00055                 }
00056
00057                 transform->x += scrolling.speed_x * dt;
00058
00059                 if (const float scaledWidth = ((scale != nullptr) ? scale->x : 1.0f) * scrolling.original_width;
00060                     transform->x + scaledWidth < 0.0f)
00061                 {
00062                     transform->x += scaledWidth + static_cast<float>(width);
00063                 }
00064             }
00065         }
00066
00067     private:
00068         const std::shared_ptr<eng::IRenderer> &m_renderer;
00069     };
00070 } // namespace gme

```

19.191 /home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Solo/include/RTypeClientSolo/Systems/Scrolling.hpp File Reference

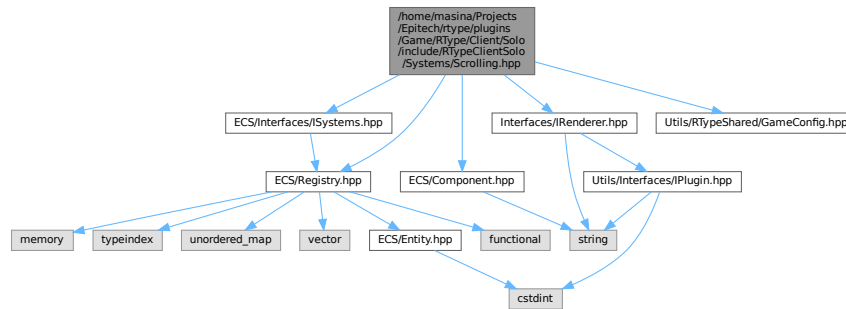
Horizontal scrolling system for large floor/ceiling sprites.

```

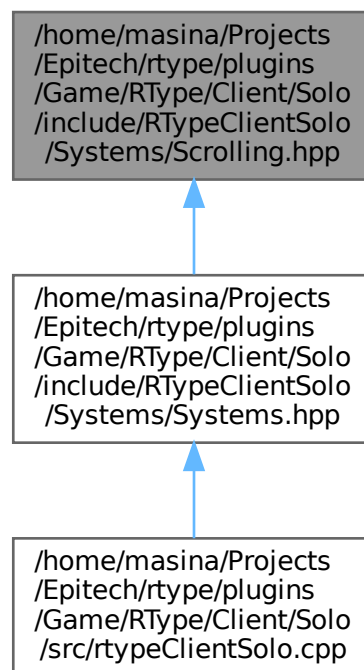
#include "ECS/Component.hpp"
#include "ECS/Interfaces/ISystems.hpp"
#include "ECS/Registry.hpp"
#include "Interfaces/IRenderer.hpp"

```

#include "Utils/RTypeShared/GameConfig.hpp"
 Include dependency graph for Scrolling.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [gme::ScrollingSystem](#)

Namespaces

- namespace [gme](#)

19.191.1 Detailed Description

Horizontal scrolling system for large floor/ceiling sprites.

Definition in file [Scrolling.hpp](#).

19.192 Scrolling.hpp

[Go to the documentation of this file.](#)

```

00001 ///
00002 /// @file Scrolling.hpp
00003 /// @brief Horizontal scrolling system for large floor/ceiling sprites
00004 /// @namespace gme
00005 ///
00006
00007 #pragma once
00008
00009 #include "ECS/Component.hpp"
00010 #include "ECS/Interfaces/ISystems.hpp"
00011 #include "ECS/Registry.hpp"
00012 #include "Interfaces/IRenderer.hpp"
00013 #include "Utils/RTypeShared/GameConfig.hpp"
00014
00015 namespace gme
00016 {
00017
00018     class ScrollingSystem final : public ecs::ASystem
00019     {
00020     public:
00021         explicit ScrollingSystem(const std::shared_ptr<eng::IRenderer> &renderer) : m_renderer(renderer) {}
00022         ~ScrollingSystem() override = default;
00023
00024         ScrollingSystem(const ScrollingSystem &) = delete;
00025         ScrollingSystem &operator=(const ScrollingSystem &) = delete;
00026         ScrollingSystem(ScrollingSystem &&) = delete;
00027         ScrollingSystem &operator=(ScrollingSystem &&) = delete;
00028
00029         void update(ecs::Registry &registry, float dt) override
00030         {
00031             const auto [width, height] = m_renderer->getWindowSize();
00032
00033             for (auto &[entity, scrolling] : registry.getAll<ecs::Scrolling>())
00034             {
00035                 auto *transform = registry.getComponent<ecs::Transform>(entity);
00036                 auto *scale = registry.getComponent<ecs::Scale>(entity);
00037                 const bool isFloor = registry.hasComponent<ecs::Floor>(entity);
00038                 const bool isCeiling = registry.hasComponent<ecs::Ceiling>(entity);
00039
00040                 if (transform == nullptr)
00041                 {
00042                     continue;
00043                 }
00044
00045                 if (scrolling.fit_width && (scale != nullptr))
00046                 {
00047                     const auto targetWidth = static_cast<float>(width);
00048                     const float scaleX = targetWidth / std::max(1.0f, scrolling.original_width);
00049                     scale->x = scaleX;
00050                     scale->y = scaleX;
00051                 }
00052
00053                 const float scaledHeight = ((scale != nullptr) ? scale->y : 1.0f) * scrolling.original_height;
00054                 if (isCeiling)
00055                 {
00056                     transform->y = utl::GameConfig::Stage::CEILING_OFFSET_Y;
00057                 }
00058                 else if (isFloor)
00059                 {
00060                     transform->y =
00061                         static_cast<float>(height) - scaledHeight - utl::GameConfig::Stage::FLOOR_OFFSET_Y;
00062                 }
00063
00064                 transform->x += scrolling.speed_x * dt;
00065
00066                 if (const float scaledWidth = ((scale != nullptr) ? scale->x : 1.0f) * scrolling.original_width;
00067                     transform->x + scaledWidth < 0.0f)
00068                 {
00069                     transform->x += scaledWidth + static_cast<float>(width);
00070                 }
00071             }
00072         }
00073
00074     private:
00075         const std::shared_ptr<eng::IRenderer> &m_renderer;
00076     }; // class ScrollingSystem
00077 } // namespace gme

```

19.193 /home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/include/RTypeClientMulti/Systems/Weapon.hpp ↩ File Reference

Handles weapon firing logic for multiplayer.

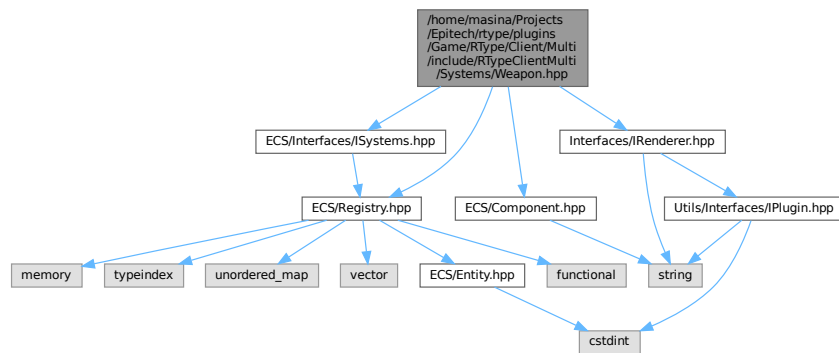
```
#include "ECS/Component.hpp"
```

```
#include "ECS/Interfaces/ISystems.hpp"
```

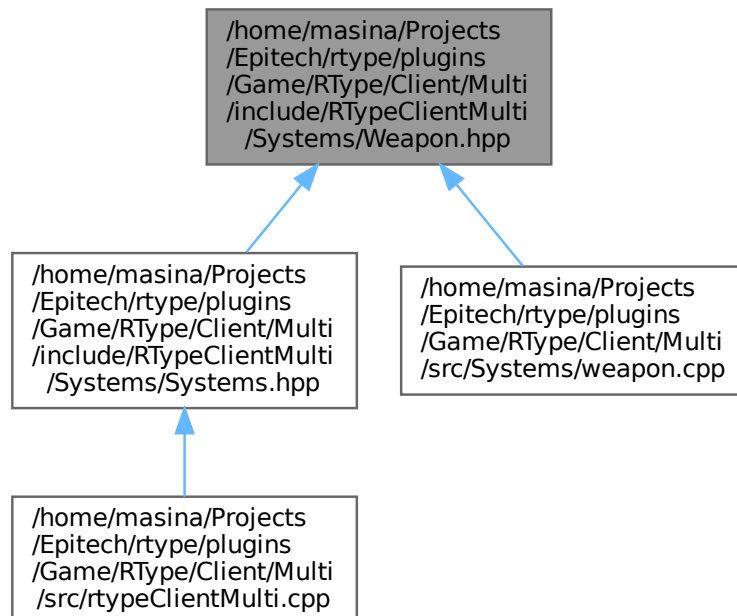
```
#include "ECS/Registry.hpp"
```

```
#include "Interfaces/IRenderer.hpp"
```

Include dependency graph for Weapon.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [gme::WeaponSystem](#)
Manages weapon firing and charging (client-side visual feedback only)

Namespaces

- namespace [gme](#)

19.193.1 Detailed Description

Handles weapon firing logic for multiplayer.

Definition in file [Weapon.hpp](#).

19.194 Weapon.hpp

[Go to the documentation of this file.](#)

```

00001 ///  

00002 ///  

00003 ///  

00004 ///  

00005 ///  

00006 ///  

00007 #pragma once  

00008  

00009 #include "ECS/Component.hpp"  

00010 #include "ECS/Interfaces/ISystems.hpp"  

00011 #include "ECS/Registry.hpp"  

00012 #include "Interfaces/IRenderer.hpp"  

00013  

00014 namespace gme  

00015 {  

00016     ///  

00017     ///  

00018     ///  

00019     ///  

00020     ///  

00021     class WeaponSystem final : public ecs::ASystem  

00022     {  

00023     public:  

00024         explicit WeaponSystem(const std::shared_ptr<eng::IRenderer> & /* renderer */) {}  

00025         ~WeaponSystem() override = default;  

00026  

00027         WeaponSystem(const WeaponSystem &) = delete;  

00028         WeaponSystem &operator=(const WeaponSystem &) = delete;  

00029         WeaponSystem(WeaponSystem &&) = delete;  

00030         WeaponSystem &operator=(WeaponSystem &&) = delete;  

00031  

00032         ///  

00033         ///  

00034         ///  

00035         ///  

00036         ///  

00037         void update(ecs::Registry &registry, float dt) override;  

00038  

00039         ///  

00040         ///  

00041         ///  

00042         void reset()  

00043         {  

00044             m_fireCooldown = 0.0f;  

00045             m_isCharging = false;  

00046         }  

00047  

00048     private:  

00049         float m_fireCooldown = 0.0f;  

00050         bool m_isCharging = false;  

00051  

00052         ///  

00053         ///  

00054         ///  

00055         ///  

00056         ///  

00057         ///  

00058         static void showLoadingAnimation(ecs::Registry &registry, ecs::Entity playerEntity,  

00059                                         const ecs::Transform *playerTransform);  

00060  

00061         ///  

00062         ///  


```

```

00063     /// @param registry The ECS registry
00064     /// @param playerEntity The player entity
00065     ///
00066     static void hideLoadingAnimation(ecs::Registry &registry, ecs::Entity playerEntity);
00067
00068     void ensureSuperShotAudio(ecs::Registry &registry);
00069
00070     ecs::Entity m_superShotAudioEntity = ecs::INVALID_ENTITY;
00071 }; // class WeaponSystem
00072 } // namespace gme

```

19.195 /home/masina/Projects/Epitech/rtype/plugins/Game/RType/↵ Client/Solo/include/RTypeClientSolo/Systems/Weapon.hpp File Reference

Handles weapon firing logic.

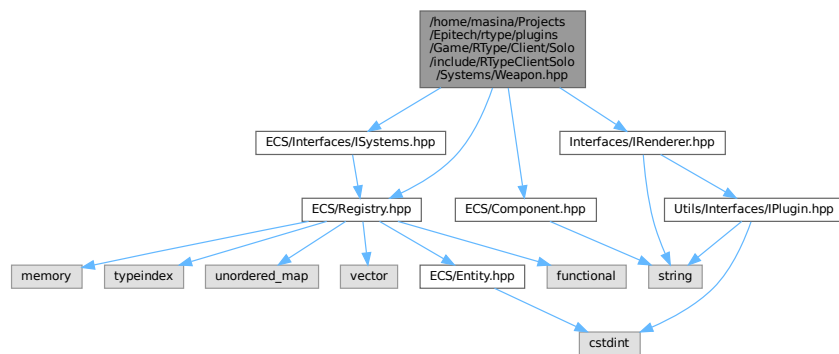
#include "ECS/Component.hpp"

#include "ECS/Interfaces/ISystems.hpp"

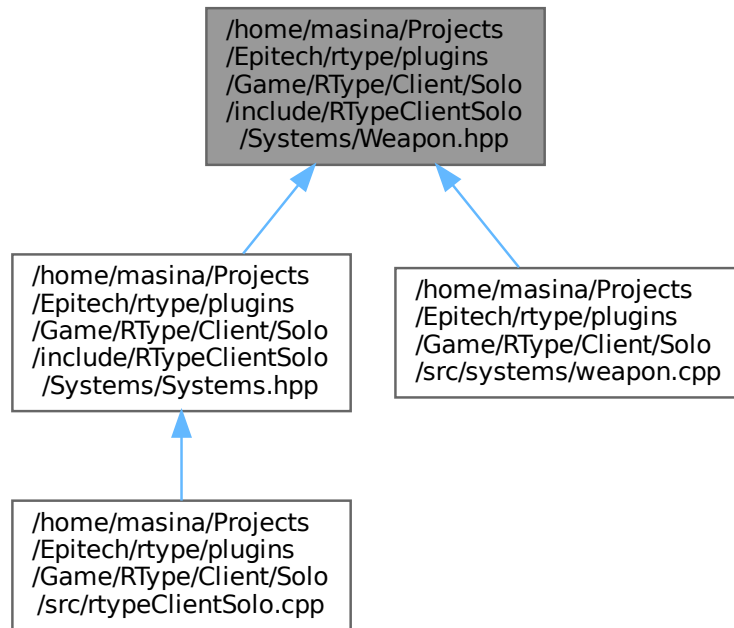
#include "ECS/Registry.hpp"

#include "Interfaces/IRenderer.hpp"

Include dependency graph for Weapon.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class `gme::WeaponSystem`
Manages weapon firing and charging (client-side visual feedback only)

Namespaces

- namespace `gme`

19.195.1 Detailed Description

Handles weapon firing logic.

Definition in file [Weapon.hpp](#).

19.196 Weapon.hpp

[Go to the documentation of this file.](#)

```

00001 ///  

00002 ///  

00003 ///  

00004 ///  

00005 ///  

00006  

00007 #pragma once  

00008  

00009 #include "ECS/Component.hpp"  

00010 #include "ECS/Interfaces/ISystems.hpp"  

00011 #include "ECS/Registry.hpp"  

00012 #include "Interfaces/IRenderer.hpp"  

00013  

00014 namespace gme  

00015 {  

00016     ///  

00017     ///  


```

```

00018  /// @brief Manages weapon firing and charging
00019  /// @namespace gme
00020  ///
00021  class WeaponSystem final : public ecs::ASystem
00022  {
00023  public:
00024      explicit WeaponSystem(const std::shared_ptr<eng::IRenderer> & /* renderer */) {}
00025      ~WeaponSystem() override = default;
00026
00027      WeaponSystem(const WeaponSystem &) = delete;
00028      WeaponSystem &operator=(const WeaponSystem &) = delete;
00029      WeaponSystem(WeaponSystem &&) = delete;
00030      WeaponSystem &operator=(WeaponSystem &&) = delete;
00031
00032      ///
00033      /// @brief Update weapon system
00034      /// @param registry The ECS registry
00035      /// @param dt Delta time
00036      ///
00037      void update(ecs::Registry &registry, float dt) override;
00038
00039      ///
00040      /// @brief Reset weapon state
00041      ///
00042      void reset()
00043      {
00044          m_fireCooldown = 0.0f;
00045          m_isCharging = false;
00046      }
00047
00048  private:
00049      float m_fireCooldown = 0.0f;
00050      bool m_isCharging = false;
00051
00052      ///
00053      /// @brief Try to fire basic projectile
00054      /// @param registry The ECS registry
00055      /// @param x X position
00056      /// @param y Y position
00057      /// @return True if fired successfully
00058      ///
00059      bool tryFireBasic(ecs::Registry &registry, float x, float y);
00060
00061      ///
00062      /// @brief Try to fire supercharged projectile
00063      /// @param registry The ECS registry
00064      /// @param x X position
00065      /// @param y Y position
00066      /// @return True if fired successfully
00067      ///
00068      bool tryFireSupercharged(ecs::Registry &registry, float x, float y);
00069
00070      ///
00071      /// @brief Show loading animation in front of the player
00072      /// @param registry The ECS registry
00073      /// @param playerEntity The player entity
00074      /// @param playerTransform The player transform
00075      ///
00076      static void showLoadingAnimation(ecs::Registry &registry, ecs::Entity playerEntity,
00077                                      const ecs::Transform *playerTransform);
00078
00079      ///
00080      /// @brief Hide loading animation
00081      /// @param registry The ECS registry
00082      /// @param playerEntity The player entity
00083      ///
00084      static void hideLoadingAnimation(ecs::Registry &registry, ecs::Entity playerEntity);
00085
00086      void ensureSuperShotAudio(ecs::Registry &registry);
00087
00088      ecs::Entity m_superShotAudioEntity = ecs::INVALID_ENTITY;
00089  }; // class WeaponSystem
00090 } // namespace gme

```

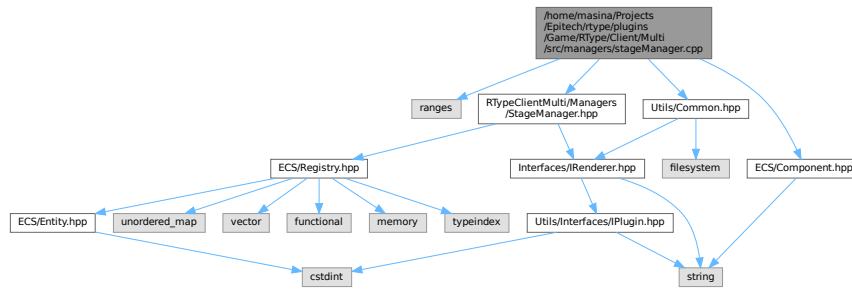
19.197 /home/masina/Projects/Epitech/rtype/plugins/Game/RType/↵ Client/Multi/src/managers/stageManager.cpp File Reference

```

#include <ranges>
#include "ECS/Component.hpp"
#include "RTypeClientMulti/Managers/StageManager.hpp"
#include "Utils/Common.hpp"

```

Include dependency graph for stageManager.cpp:



19.198 stageManager.cpp

[Go to the documentation of this file.](#)

```

00001 #include <ranges>
00002
00003 #include "ECS/Component.hpp"
00004 #include "RTypeClientMulti/Managers/StageManager.hpp"
00005 #include "Utils/Common.hpp"
00006
00007 void gme::StageManager::update(ecs::Registry &registry, const float dt, const eng::WindowSize &size)
00008 {
00009     m_stageTimer += dt;
00010
00011     // Spawn stage after initial delay
00012     if (!m_stageSpawned && m_stageTimer >= STAGE_SPAWN_DELAY)
00013     {
00014         spawnStage(registry, size.width);
00015         m_stageSpawned = true;
00016         m_currentWave = 1; // Start wave 1
00017         m_waveTimer = 0.0f;
00018     }
00019
00020     // Track wave progression (synchronized with server WaveManager)
00021     if (m_stageSpawned && m_currentWave <= TOTAL_WAVES)
00022     {
00023         m_totalGameTimer += dt;
00024         m_waveTimer += dt;
00025
00026         // Check if current wave duration is complete
00027         if (m_waveTimer >= WAVE_DURATION && m_currentWave < TOTAL_WAVES)
00028         {
00029             m_currentWave++;
00030             m_waveTimer = 0.0f;
00031         }
00032     }
00033 }
00034
00035 void gme::StageManager::stopScrolling(ecs::Registry &registry)
00036 {
00037     for (auto &scrolling : registry.getAll<ecs::Scrolling>() | std::views::values)
00038     {
00039         scrolling.speed_x = 0.0f;
00040     }
00041 }
00042
00043 void gme::StageManager::spawnStage(ecs::Registry &registry, const int screenWidth)
00044 {
00045     (void)registry.createEntity()
00046         .with<ecs::Transform>("floor1_transform", 0.F, 0.F, 0.F)
00047         .with<ecs::Scale>("floor1_scale", 1.F, 1.F)
00048         .with<ecs::Texture>("floor1_texture", utl::Path::Texture::TEXTURE_STAGE1_FLOOR)
00049         .with<ecs::Scrolling>("floor1_scroll", SCROLL_SPEED, ORIGINAL_WIDTH, SPRITE_HEIGHT, true)
00050         .with<ecs::Floor>("floor1_tag")
00051         .build();
00052
00053     (void)registry.createEntity()
00054         .with<ecs::Transform>("floor2_transform", static_cast<float>(screenWidth), 0.F, 0.F)
00055         .with<ecs::Scale>("floor2_scale", 1.F, 1.F)
00056         .with<ecs::Texture>("floor2_texture", utl::Path::Texture::TEXTURE_STAGE1_FLOOR)
00057         .with<ecs::Scrolling>("floor2_scroll", SCROLL_SPEED, ORIGINAL_WIDTH, SPRITE_HEIGHT, true)
00058         .with<ecs::Floor>("floor2_tag")
00059         .build();
00060 }

```

```

00061 (void)registry.createEntity()
00062 .with<ecs::Transform>("cei1_transform", 0.F, 0.F, 0.F)
00063 .with<ecs::Scale>("cei1_scale", 1.F, 1.F)
00064 .with<ecs::Texture>("cei1_texture", utl::Path::Texture::TEXTURE_STAGE1_CEILING)
00065 .with<ecs::Scrolling>("cei1_scroll", SCROLL_SPEED, ORIGINAL_WIDTH, SPRITE_HEIGHT, true)
00066 .with<ecs::Ceiling>("cei1_tag")
00067 .build();
00068
00069 (void)registry.createEntity()
00070 .with<ecs::Transform>("cei2_transform", static_cast<float>(screenWidth), 0.F, 0.F)
00071 .with<ecs::Scale>("cei2_scale", 1.F, 1.F)
00072 .with<ecs::Texture>("cei2_texture", utl::Path::Texture::TEXTURE_STAGE1_CEILING)
00073 .with<ecs::Scrolling>("cei2_scroll", SCROLL_SPEED, ORIGINAL_WIDTH, SPRITE_HEIGHT, true)
00074 .with<ecs::Ceiling>("cei2_tag")
00075 .build();
00076 }

```

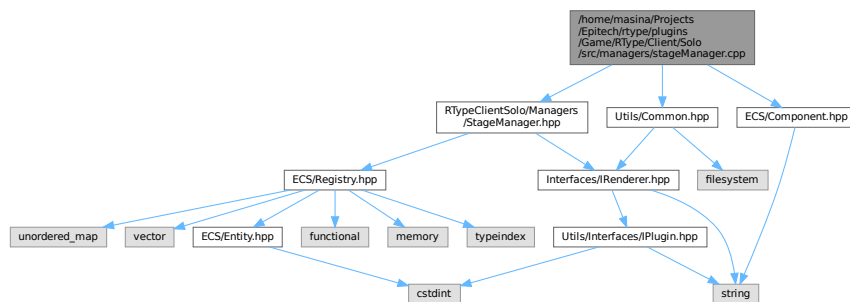
19.199 /home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Solo/src/managers/stageManager.cpp File Reference

```

#include "RTypeClientSolo/Managers/StageManager.hpp"
#include "ECS/Component.hpp"
#include "Utils/Common.hpp"

```

Include dependency graph for stageManager.cpp:



19.200 stageManager.cpp

[Go to the documentation of this file.](#)

```

00001 #include "RTypeClientSolo/Managers/StageManager.hpp"
00002 #include "ECS/Component.hpp"
00003 #include "Utils/Common.hpp"
00004
00005 void gme::StageManager::update(ecs::Registry &registry, const float dt, const eng::WindowSize &size)
00006 {
00007     m_stageTimer += dt;
00008
00009     // Spawn stage after initial delay
00010     if (!m_stageSpawned && m_stageTimer >= STAGE_SPAWN_DELAY)
00011     {
00012         spawnStage(registry, size.width);
00013         m_stageSpawned = true;
00014         m_currentWave = 1; // Start wave 1
00015         m_waveTimer = 0.0f;
00016     }
00017
00018     // Track wave progression (synchronized with server WaveManager)
00019     if (m_stageSpawned && m_currentWave <= TOTAL_WAVES)
00020     {
00021         m_totalGameTimer += dt;
00022         m_waveTimer += dt;
00023
00024         // Check if current wave duration is complete
00025         if (m_waveTimer >= WAVE_DURATION && m_currentWave < TOTAL_WAVES)
00026         {
00027             m_currentWave++;
00028             m_waveTimer = 0.0f;
00029         }
00030     }

```



```

00031 }
00032
00033 void gme::StageManager::spawnStage(ecs::Registry &registry, const int screenWidth)
00034 {
00035     auto floor1 = registry.createEntity()
00036         .with<ecs::Transform>("floor1_transform", 0.F, 0.F, 0.F)
00037         .with<ecs::Scale>("floor1_scale", 1.F, 1.F)
00038         .with<ecs::Texture>("floor1_texture", utl::Path::Texture::TEXTURE_STAGE1_FLOOR)
00039         .with<ecs::Scrolling>("floor1_scroll", SCROLL_SPEED, ORIGINAL_WIDTH, SPRITE_HEIGHT, true)
00040         .with<ecs::Floor>("floor1_tag")
00041         .build();
00042
00043     auto floor2 = registry.createEntity()
00044         .with<ecs::Transform>("floor2_transform", static_cast<float>(screenWidth), 0.F, 0.F)
00045         .with<ecs::Scale>("floor2_scale", 1.F, 1.F)
00046         .with<ecs::Texture>("floor2_texture", utl::Path::Texture::TEXTURE_STAGE1_FLOOR)
00047         .with<ecs::Scrolling>("floor2_scroll", SCROLL_SPEED, ORIGINAL_WIDTH, SPRITE_HEIGHT, true)
00048         .with<ecs::Floor>("floor2_tag")
00049         .build();
00050
00051     auto ceil1 = registry.createEntity()
00052         .with<ecs::Transform>("ceil1_transform", 0.F, 0.F, 0.F)
00053         .with<ecs::Scale>("ceil1_scale", 1.F, 1.F)
00054         .with<ecs::Texture>("ceil1_texture", utl::Path::Texture::TEXTURE_STAGE1_CEILING)
00055         .with<ecs::Scrolling>("ceil1_scroll", SCROLL_SPEED, ORIGINAL_WIDTH, SPRITE_HEIGHT, true)
00056         .with<ecs::Ceiling>("ceil1_tag")
00057         .build();
00058
00059     auto ceil2 = registry.createEntity()
00060         .with<ecs::Transform>("ceil2_transform", static_cast<float>(screenWidth), 0.F, 0.F)
00061         .with<ecs::Scale>("ceil2_scale", 1.F, 1.F)
00062         .with<ecs::Texture>("ceil2_texture", utl::Path::Texture::TEXTURE_STAGE1_CEILING)
00063         .with<ecs::Scrolling>("ceil2_scroll", SCROLL_SPEED, ORIGINAL_WIDTH, SPRITE_HEIGHT, true)
00064         .with<ecs::Ceiling>("ceil2_tag")
00065         .build();
00066 }

```

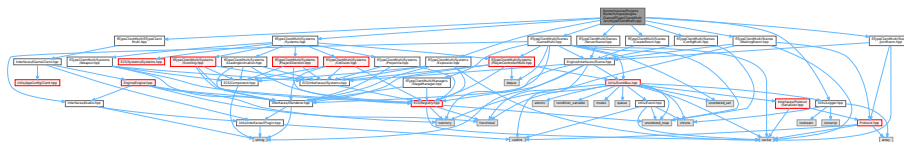
19.201 /home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/src/rtypeClientMulti.cpp File Reference

```

#include "RTypeClientMulti/RTypeClientMulti.hpp"
#include "RTypeClientMulti/Scenes/ConfigMulti.hpp"
#include "RTypeClientMulti/Scenes/CreateRoom.hpp"
#include "RTypeClientMulti/Scenes/GameMulti.hpp"
#include "RTypeClientMulti/Scenes/JoinRoom.hpp"
#include "RTypeClientMulti/Scenes/ServerScene.hpp"
#include "RTypeClientMulti/Scenes/WaitingRoom.hpp"
#include "RTypeClientMulti/Systems/Systems.hpp"
#include "RTypeClientMulti/Systems/Explosion.hpp"
#include "Utils/Logger.hpp"

```

Include dependency graph for rtypeClientMulti.cpp:



19.202 rtypeClientMulti.cpp

[Go to the documentation of this file.](#)

```

00001 #include "RTypeClientMulti/RTypeClientMulti.hpp"
00002 #include "RTypeClientMulti/Scenes/ConfigMulti.hpp"
00003 #include "RTypeClientMulti/Scenes/CreateRoom.hpp"
00004 #include "RTypeClientMulti/Scenes/GameMulti.hpp"
00005 #include "RTypeClientMulti/Scenes/JoinRoom.hpp"
00006 #include "RTypeClientMulti/Scenes/ServerScene.hpp"
00007 #include "RTypeClientMulti/Scenes/WaitingRoom.hpp"
00008 #include "RTypeClientMulti/Systems/Systems.hpp"
00009 #include "RTypeClientMulti/Systems/Explosion.hpp"
00010 #include "Utils/Logger.hpp"

```

```

00011
00012 void gme::RTypeClientMulti::update(float deltaTime, unsigned int width, unsigned int height) {}
00013
00014 void gme::RTypeClientMulti::setupScenes(bool &showDebug, eng::id menuSceneId)
00015 {
00016     auto serverSceneId = m_engine->getSceneManager()->generateNextId();
00017     auto serverScene = std::make_unique<ServerScene>(serverSceneId, m_engine->getRenderer(), m_appConfig->host,
00018                                                     std::to_string(m_appConfig->port), m_appConfig->player_name);
00019     m_mainSceneId = serverSceneId;
00020     serverScene->addSystem(std::make_unique<ecs::AudioSystem>(m_engine->getAudio(), m_appConfig->audioVolume,
00021                                                             serverScene->getRegistry(), serverScene->playMusic()));
00022     serverScene->addSystem(std::make_unique<ecs::StarfieldSystem>(m_engine->getRenderer(),
serverScene->getRegistry()));
00023     serverScene->addSystem(std::make_unique<ecs::SpriteSystem>(m_engine->getRenderer()));
00024     serverScene->addSystem(std::make_unique<ecs::TextSystem>(m_engine->getRenderer()));
00025     serverScene->addSystem(std::make_unique<ecs::DebugSystem>(m_engine->getRenderer(), m_showDebug));
00026     auto configMultiId = m_engine->getSceneManager()->generateNextId();
00027     auto configMulti = std::make_unique<ConfigMulti>(configMultiId, m_engine->getRenderer());
00028     configMulti->addSystem(std::make_unique<ecs::AudioSystem>(m_engine->getAudio(), m_appConfig->audioVolume,
00029                                                             configMulti->getRegistry(), configMulti->playMusic()));
00030     configMulti->addSystem(std::make_unique<ecs::StarfieldSystem>(m_engine->getRenderer(),
configMulti->getRegistry()));
00031     configMulti->addSystem(std::make_unique<ecs::SpriteSystem>(m_engine->getRenderer()));
00032     configMulti->addSystem(std::make_unique<ecs::TextSystem>(m_engine->getRenderer()));
00033     configMulti->addSystem(std::make_unique<ecs::DebugSystem>(m_engine->getRenderer(), m_showDebug));
00034     auto createRoomId = m_engine->getSceneManager()->generateNextId();
00035     auto createRoomScene = std::make_unique<CreateRoomScene>(createRoomId, m_engine->getRenderer());
00036     createRoomScene->addSystem(std::make_unique<ecs::AudioSystem>(
00037         m_engine->getAudio(), m_appConfig->audioVolume, createRoomScene->getRegistry(),
createRoomScene->playMusic()));
00038     createRoomScene->addSystem(
00039         std::make_unique<ecs::StarfieldSystem>(m_engine->getRenderer(), createRoomScene->getRegistry()));
00040     createRoomScene->addSystem(std::make_unique<ecs::SpriteSystem>(m_engine->getRenderer()));
00041     createRoomScene->addSystem(std::make_unique<ecs::TextSystem>(m_engine->getRenderer()));
00042     createRoomScene->addSystem(std::make_unique<ecs::DebugSystem>(m_engine->getRenderer(), m_showDebug));
00043     auto joinRoomId = m_engine->getSceneManager()->generateNextId();
00044     auto joinRoomScene = std::make_unique<JoinRoomScene>(joinRoomId, m_engine->getRenderer());
00045     joinRoomScene->addSystem(std::make_unique<ecs::AudioSystem>(
00046         m_engine->getAudio(), m_appConfig->audioVolume, joinRoomScene->getRegistry(),
joinRoomScene->playMusic()));
00047     joinRoomScene->addSystem(
00048         std::make_unique<ecs::StarfieldSystem>(m_engine->getRenderer(), joinRoomScene->getRegistry()));
00049     joinRoomScene->addSystem(std::make_unique<ecs::SpriteSystem>(m_engine->getRenderer()));
00050     joinRoomScene->addSystem(std::make_unique<ecs::TextSystem>(m_engine->getRenderer()));
00051     joinRoomScene->addSystem(std::make_unique<ecs::DebugSystem>(m_engine->getRenderer(), m_showDebug));
00052     auto waitingRoomId = m_engine->getSceneManager()->generateNextId();
00053     auto waitingRoomScene = std::make_unique<WaitingRoomScene>(waitingRoomId, m_engine->getRenderer());
00054     waitingRoomScene->addSystem(std::make_unique<ecs::AudioSystem>(m_engine->getAudio(),
m_appConfig->audioVolume,
00055                                                             waitingRoomScene->getRegistry(),
00056                                                             waitingRoomScene->playMusic()));
00057     waitingRoomScene->addSystem(
00058         std::make_unique<ecs::StarfieldSystem>(m_engine->getRenderer(), waitingRoomScene->getRegistry()));
00059     waitingRoomScene->addSystem(std::make_unique<ecs::SpriteSystem>(m_engine->getRenderer()));
00060     waitingRoomScene->addSystem(std::make_unique<ecs::TextSystem>(m_engine->getRenderer()));
00061     waitingRoomScene->addSystem(std::make_unique<ecs::DebugSystem>(m_engine->getRenderer(), m_showDebug));
00062     WaitingRoomScene *waitingRoomScenePtr = waitingRoomScene.get();
00063
00064     serverScene->onConnect =
00065         [this, configMultiId](const std::string &playerName, const std::string &serverIP, const std::string &serverPort)
00066         { m_engine->getSceneManager()->switchToScene(configMultiId); };
00067     serverScene->onBackToMenu = [this, menuSceneId]() { m_engine->getSceneManager()->switchToScene(menuSceneId);
};
00068
00069     configMulti->onOptionSelected = [this, menuSceneId, createRoomId, joinRoomId](const std::string &option)
00070     {
00071         if (option == "Create room")
00072         {
00073             m_engine->getSceneManager()->switchToScene(createRoomId);
00074         }
00075         else if (option == "Join room")
00076         {
00077             m_engine->getSceneManager()->switchToScene(joinRoomId);
00078         }
00079         else if (option == "Go back to menu")
00080         {
00081             m_engine->getSceneManager()->switchToScene(menuSceneId);
00082         }
00083     };
00084
00085     createRoomScene->onCreate = [this](const std::string &roomName, int maxPlayers)
00086     {
00087         // Room creation is handled through event bus
00088         // Response will be processed in CreateRoomScene::processEventBus()
00089         // We stay on the create room scene until we get a response
00090     };
00091     createRoomScene->onRoomCreated =

```

```

00092     [this, waitingRoomId, waitingRoomScenePtr](const int lobbyId, const rnp::LobbyInfo *lobbyInfo)
00093     {
00094         utl::Logger::log("RTypeClientMulti: Room created with ID " + std::to_string(lobbyId), utl::LogLevel::INFO);
00095         // Transition to waiting room after successful room creation
00096         waitingRoomScenePtr->setLobbyId(static_cast<std::uint32_t>(lobbyId));
00097         waitingRoomScenePtr->setIsHost(true); // Creator is the host
00098         if (lobbyInfo)
00099         {
00100             waitingRoomScenePtr->setLobbyInfo(*lobbyInfo);
00101         }
00102         m_engine->getSceneManager()->switchToScene(waitingRoomId);
00103     };
00104     createRoomScene->onBackToMulti = [this, configMultiId]()
00105     { m_engine->getSceneManager()->switchToScene(configMultiId); };
00106
00107     joinRoomScene->onJoin = [this, waitingRoomId, waitingRoomScenePtr](int roomId, const rnp::LobbyInfo *lobbyInfo)
00108     {
00109         utl::Logger::log("RTypeClientMulti: Joined room with ID " + std::to_string(roomId), utl::LogLevel::INFO);
00110         waitingRoomScenePtr->setLobbyId(static_cast<std::uint32_t>(roomId));
00111         waitingRoomScenePtr->setIsHost(false);
00112         if (lobbyInfo)
00113         {
00114             waitingRoomScenePtr->setLobbyInfo(*lobbyInfo);
00115         }
00116         m_engine->getSceneManager()->switchToScene(waitingRoomId);
00117     };
00118     joinRoomScene->onBackToMulti = [this, configMultiId]()
00119     { m_engine->getSceneManager()->switchToScene(configMultiId); };
00120     joinRoomScene->onRefreshRequest = []()
00121     {
00122         // Room list refresh is handled through event bus
00123         // JoinRoomScene::refreshRoomList() publishes LOBBY_LIST_REQUEST
00124         // Response is processed in JoinRoomScene::handleLobbyListResponse()
00125     };
00126
00127     waitingRoomScene->onLeaveLobby = [this, configMultiId]()
00128     { m_engine->getSceneManager()->switchToScene(configMultiId); };
00129     waitingRoomScene->onGameStart = [this]()
00130     {
00131         utl::Logger::log("RTypeClientMulti: Game starting!", utl::LogLevel::INFO);
00132
00133         uint32_t sessionId = 0;
00134         if (m_engine && m_engine->getNetwork())
00135         {
00136             sessionId = m_engine->getNetwork()->getSessionId();
00137             utl::Logger::log("RTypeClientMulti: Using sessionId " + std::to_string(sessionId), utl::LogLevel::INFO);
00138         }
00139         auto gameMultiId = m_engine->getSceneManager()->generateNextId();
00140         auto gameMulti = std::make_unique<GameMulti>(gameMultiId, m_engine->getRenderer(), m_engine->getAudio(),
00141             m_appConfig->skinIndex, m_showDebug, sessionId);
00142         gameMulti->addSystem(std::make_unique<ecs::AudioSystem>(m_engine->getAudio(),
00143             m_appConfig->audioVolume,
00144             gameMulti->getRegistry(), gameMulti->playMusic()));
00145         gameMulti->addSystem(std::make_unique<ExplosionSystem>(m_engine->getRenderer()));
00146         gameMulti->addSystem(std::make_unique<ecs::SpriteSystem>(m_engine->getRenderer()));
00147         gameMulti->addSystem(std::make_unique<ecs::TextSystem>(m_engine->getRenderer()));
00148         gameMulti->addSystem(std::make_unique<ecs::DebugSystem>(m_engine->getRenderer(), m_showDebug));
00149         gameMulti->addSystem(std::make_unique<ecs::StarfieldSystem>(m_engine->getRenderer(),
00150             gameMulti->getRegistry()));
00151         gameMulti->addSystem(std::make_unique<ecs::AnimationSystem>(m_engine->getRenderer()));
00152         gameMulti->addSystem(std::make_unique<gme::LoadingAnimationSystem>(m_engine->getRenderer()));
00153         gameMulti->addSystem(std::make_unique<gme::ScrollingSystem>(m_engine->getRenderer()));
00154         gameMulti->addSystem(std::make_unique<gme::CollisionSystem>(m_engine->getRenderer(), m_showDebug));
00155         gameMulti->addSystem(std::make_unique<ecs::BeamSystem>(m_engine->getRenderer()));
00156         gameMulti->addSystem(std::make_unique<gme::ProjectileSystem>(m_engine->getRenderer()));
00157         gameMulti->addSystem(std::make_unique<gme::WeaponSystem>(m_engine->getRenderer()));
00158         gameMulti->addSystem(std::make_unique<gme::PlayerDirectionSystem>());
00159
00160         m_engine->getSceneManager()->addScene(std::move(gameMulti));
00161         m_engine->getSceneManager()->switchToScene(gameMultiId);
00162     };
00163
00164     m_engine->getSceneManager()->addScene(std::move(serverScene));
00165     m_engine->getSceneManager()->addScene(std::move(configMulti));
00166     m_engine->getSceneManager()->addScene(std::move(createRoomScene));
00167     m_engine->getSceneManager()->addScene(std::move(joinRoomScene));
00168     m_engine->getSceneManager()->addScene(std::move(waitingRoomScene));
00169 }

```

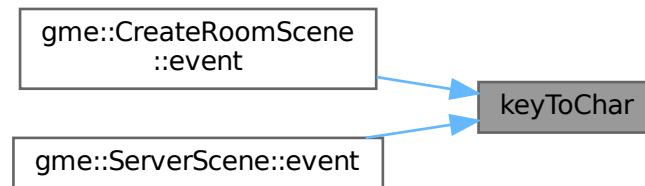


```

00054         {
00055             renderer->createSprite(textureComp->id + std::to_string(e), textureComp->id, transform->x,
00056                                     transform->y);
00057         }
00058     }
00059 }
00060 });
00061
00062 m_titleEntity =
00063     registry.createEntity()
00064         .with<ecs::Font>("main_font", utl::Path::Font::FONTS_RTYPE)
00065         .with<ecs::Transform>("transform_title", 100.F, 60.F, 0.F)
00066         .with<ecs::Color>("color_title", utl::Config::Color::CYAN_ELECTRIC.r,
utl::Config::Color::CYAN_ELECTRIC.g,
00067             utl::Config::Color::CYAN_ELECTRIC.b, utl::Config::Color::CYAN_ELECTRIC.a)
00068         .with<ecs::Text>("title", std::string("MULTIPLAYER"), 72U)
00069         .build();
00070
00071 for (size_t i = 0; i < m_menuOptions.size(); ++i)
00072 {
00073     registry.createEntity()
00074         .with<ecs::Font>("main_font", utl::Path::Font::FONTS_RTYPE)
00075         .with<ecs::Transform>("transform_menu_" + std::to_string(i), 100.F, 200.F + i * 60.F, 0.F)
00076         .with<ecs::Color>("color_menu_" + std::to_string(i), utl::Config::Color::GRAY_BLUE_SUBTLE.r,
utl::Config::Color::GRAY_BLUE_SUBTLE.g, utl::Config::Color::GRAY_BLUE_SUBTLE.b,
00077             utl::Config::Color::GRAY_BLUE_SUBTLE.a)
00078         .with<ecs::Text>("menu_" + m_menuOptions[i], m_menuOptions[i], 40U)
00079         .build();
00080 }
00081
00082
00083 m_selectedIndex = 2;
00084 }
00085
00086 void gme::ConfigMulti::update(const float dt, const eng::WindowSize & /*size*/)
00087 {
00088     auto &reg = getRegistry();
00089
00090     auto &colors = reg.getAll<ecs::Color>();
00091     auto &texts = reg.getAll<ecs::Text>();
00092
00093     m_animationTime += dt;
00094     m_titlePulseTime += dt;
00095
00096     int i = 0;
00097     for (auto &[entity, text] : texts)
00098     {
00099         if (text.content == "Create room" || text.content == "Join room" || text.content == "Go back to menu")
00100         {
00101             auto &color = colors.at(entity);
00102
00103             if (i == m_selectedIndex)
00104             {
00105                 const float glowIntensity = std::sin(m_animationTime * 2.5f);
00106                 color.r = 0U;
00107                 color.g = static_cast<unsigned char>(191U + glowIntensity * 50);
00108                 color.b = 255U;
00109             }
00110             else
00111             {
00112                 color.r = utl::Config::Color::GRAY_BLUE_SUBTLE.r;
00113                 color.g = utl::Config::Color::GRAY_BLUE_SUBTLE.g;
00114                 color.b = utl::Config::Color::GRAY_BLUE_SUBTLE.b;
00115             }
00116             i++;
00117         }
00118     }
00119     if (auto *titleColor = reg.getComponent<ecs::Color>(m_titleEntity))
00120     {
00121         const float pulsation = (std::sin(m_titlePulseTime * 2.0f) * 0.4f) + 0.6f;
00122         titleColor->r = static_cast<unsigned char>(utl::Config::Color::CYAN_ELECTRIC.r * pulsation);
00123         titleColor->g = static_cast<unsigned char>(utl::Config::Color::CYAN_ELECTRIC.g * pulsation);
00124         titleColor->b = static_cast<unsigned char>(utl::Config::Color::CYAN_ELECTRIC.b * pulsation);
00125     }
00126 }
00127
00128 void gme::ConfigMulti::event(const eng::Event &event)
00129 {
00130     switch (event.type)
00131     {
00132     case eng::EventType::KeyPressed:
00133         if (event.key == eng::Key::Up)
00134         {
00135             m_selectedIndex = (m_selectedIndex == 2) ? 0 : m_selectedIndex + 1;
00136             m_playMusic = true;
00137         }
00138         else if (event.key == eng::Key::Down)
00139         {

```


Here is the caller graph for this function:



19.206 createRoom.cpp

[Go to the documentation of this file.](#)

```

00001 #include <cmath>
00002
00003 #include "Interfaces/Protocol/Protocol.hpp"
00004 #include "Interfaces/Protocol/Serializer.hpp"
00005 #include "Utils/Event.hpp"
00006 #include "Utils/Logger.hpp"
00007
00008 #include "ECS/Component.hpp"
00009 #include "RTypeClientMulti/Scenes/CreateRoom.hpp"
00010 #include "Utils/Common.hpp"
00011
00012 static char keyToChar(const eng::Key key, bool shift = false)
00013 {
00014     switch (key)
00015     {
00016     case eng::Key::Q:
00017         return shift ? 'A' : 'a';
00018     case eng::Key::W:
00019         return shift ? 'Z' : 'z';
00020     case eng::Key::E:
00021         return shift ? 'E' : 'e';
00022     case eng::Key::R:
00023         return shift ? 'R' : 'r';
00024     case eng::Key::T:
00025         return shift ? 'T' : 't';
00026     case eng::Key::Y:
00027         return shift ? 'Y' : 'y';
00028     case eng::Key::U:
00029         return shift ? 'U' : 'u';
00030     case eng::Key::I:
00031         return shift ? 'I' : 'i';
00032     case eng::Key::O:
00033         return shift ? 'O' : 'o';
00034     case eng::Key::P:
00035         return shift ? 'P' : 'p';
00036
00037     case eng::Key::A:
00038         return shift ? 'Q' : 'q';
00039     case eng::Key::S:
00040         return shift ? 'S' : 's';
00041     case eng::Key::D:
00042         return shift ? 'D' : 'd';
00043     case eng::Key::F:
00044         return shift ? 'F' : 'f';
00045     case eng::Key::G:
00046         return shift ? 'G' : 'g';
00047     case eng::Key::H:
00048         return shift ? 'H' : 'h';
00049     case eng::Key::J:
00050         return shift ? 'J' : 'j';
00051     case eng::Key::K:
00052         return shift ? 'K' : 'k';
00053     case eng::Key::M:
00054         return shift ? 'M' : 'm';
00055     case eng::Key::L:
00056         return shift ? 'L' : 'l';
00057
00058     case eng::Key::Z:

```



```

00059         return shift ? 'W' : 'w';
00060     case eng::Key::X:
00061         return shift ? 'X' : 'x';
00062     case eng::Key::C:
00063         return shift ? 'C' : 'c';
00064     case eng::Key::V:
00065         return shift ? 'V' : 'v';
00066     case eng::Key::B:
00067         return shift ? 'B' : 'b';
00068     case eng::Key::N:
00069         return shift ? 'N' : 'n';
00070
00071     case eng::Key::Num0:
00072         return '0';
00073     case eng::Key::Num1:
00074         return '1';
00075     case eng::Key::Num2:
00076         return '2';
00077     case eng::Key::Num3:
00078         return '3';
00079     case eng::Key::Num4:
00080         return '4';
00081     case eng::Key::Num5:
00082         return '5';
00083     case eng::Key::Num6:
00084         return '6';
00085     case eng::Key::Num7:
00086         return '7';
00087     case eng::Key::Num8:
00088         return '8';
00089     case eng::Key::Num9:
00090         return '9';
00091     case eng::Key::Space:
00092         return ' ';
00093     default:
00094         return '\\0';
00095 }
00096 }
00097
00098 gme::CreateRoomScene::CreateRoomScene(const eng::id assignedId, const std::shared_ptr<eng::IRenderer> &renderer)
00099 : AScene(assignedId)
00100 {
00101     auto &registry = AScene::getRegistry();
00102
00103     registry.onComponentAdded(
00104         [&renderer, &registry](const ecs::Entity e, const std::type_info &type)
00105         {
00106             const auto *colorComp = registry.getComponent<ecs::Color>(e);
00107             const auto *fontComp = registry.getComponent<ecs::Font>(e);
00108             const auto *textComp = registry.getComponent<ecs::Text>(e);
00109             const auto *transform = registry.getComponent<ecs::Transform>(e);
00110
00111             if (type == typeid(ecs::Text))
00112             {
00113                 if (textComp && transform && fontComp)
00114                 {
00115                     renderer->createFont(fontComp->id, fontComp->path);
00116                     renderer->createText(
00117                         {font_name = fontComp->id,
00118                          .color = {.r = colorComp->r, .g = colorComp->g, .b = colorComp->b, .a = colorComp->a},
00119                          .content = textComp->content,
00120                          .size = textComp->font_size,
00121                          .x = transform->x,
00122                          .y = transform->y,
00123                          .name = textComp->id});
00124                 }
00125             }
00126         });
00127
00128     m_titleEntity =
00129         registry.createEntity()
00130             .with<ecs::Font>("main_font", utl::Path::Font::FONTS_RTYPE)
00131             .with<ecs::Transform>("transform_title", 100.F, 60.F, 0.F)
00132             .with<ecs::Color>("color_title", utl::Config::Color::CYAN_ELECTRIC.r,
00133                             utl::Config::Color::CYAN_ELECTRIC.b, utl::Config::Color::CYAN_ELECTRIC.a)
00134             .with<ecs::Text>("title", std::string("CREATE ROOM"), 72U)
00135             .build();
00136
00137     for (size_t i = 0; i < m_options.size(); ++i)
00138     {
00139         registry.createEntity()
00140             .with<ecs::Font>("main_font", utl::Path::Font::FONTS_RTYPE)
00141             .with<ecs::Transform>("transform_option_" + std::to_string(i), 100.F, 200.F + i * 50.F, 0.F)
00142             .with<ecs::Color>("color_option_" + std::to_string(i), utl::Config::Color::GRAY_BLUE_SUBTLE.r,
00143                             utl::Config::Color::GRAY_BLUE_SUBTLE.g, utl::Config::Color::GRAY_BLUE_SUBTLE.b,
00144                             utl::Config::Color::GRAY_BLUE_SUBTLE.a)

```



```

00145         .with<ecs::Text>("option_" + m_options[i], m_options[i], 32U)
00146         .build();
00147     }
00148
00149     m_roomNameValueEntity =
00150     registry.createEntity()
00151     .with<ecs::Font>("main_font", utl::Path::Font::FONTS_RTYPE)
00152     .with<ecs::Transform>("transform_room_name_value", 580.F, 200.F, 0.F)
00153     .with<ecs::Color>("color_room_name_value", utl::Config::Color::TEXT_VALUE_COLOR.r,
00154         utl::Config::Color::TEXT_VALUE_COLOR.g, utl::Config::Color::TEXT_VALUE_COLOR.b,
00155         utl::Config::Color::TEXT_VALUE_COLOR.a)
00156     .with<ecs::Text>("room_name_value", m_roomName, 24U)
00157     .build();
00158
00159     m_maxPlayersValueEntity =
00160     registry.createEntity()
00161     .with<ecs::Font>("main_font", utl::Path::Font::FONTS_RTYPE)
00162     .with<ecs::Transform>("transform_max_players_value", 580.F, 250.F, 0.F)
00163     .with<ecs::Color>("color_max_players_value", utl::Config::Color::TEXT_VALUE_COLOR.r,
00164         utl::Config::Color::TEXT_VALUE_COLOR.g, utl::Config::Color::TEXT_VALUE_COLOR.b,
00165         utl::Config::Color::TEXT_VALUE_COLOR.a)
00166     .with<ecs::Text>("max_players_value", std::to_string(m_maxPlayers), 24U)
00167     .build();
00168     m_eventComponentId = 6;
00169     m_eventBus.registerComponent(m_eventComponentId, "Room_Manager");
00170     m_eventBus.subscribe(m_eventComponentId, utl::EventType::LOBBY_CREATE_RESPONSE);
00171 }
00172
00173 void gme::CreateRoomScene::update(const float dt, const eng::WindowSize & /*size*/)
00174 {
00175     auto &reg = getRegistry();
00176     auto &colors = reg.getAll<ecs::Color>();
00177     auto &texts = reg.getAll<ecs::Text>();
00178
00179     m_animationTime += dt;
00180
00181     processEventBus();
00182
00183     for (auto &[entity, text] : texts)
00184     {
00185         for (size_t i = 0; i < m_options.size(); ++i)
00186         {
00187             if (text.id == "option_" + m_options[i])
00188             {
00189                 auto &color = colors.at(entity);
00190
00191                 if (i == m_selectedIndex)
00192                 {
00193                     const float glowIntensity = std::sin(m_animationTime * 2.5F);
00194                     color.r = 0U;
00195                     color.g = static_cast<unsigned char>(191U + (glowIntensity * 50));
00196                     color.b = 255U;
00197                 }
00198                 else
00199                 {
00200                     color.r = utl::Config::Color::GRAY_BLUE_SUBTLE.r;
00201                     color.g = utl::Config::Color::GRAY_BLUE_SUBTLE.g;
00202                     color.b = utl::Config::Color::GRAY_BLUE_SUBTLE.b;
00203                 }
00204                 break;
00205             }
00206         }
00207     }
00208 }
00209
00210 void gme::CreateRoomScene::event(const eng::Event &event)
00211 {
00212     switch (event.type)
00213     {
00214     case eng::EventType::KeyPressed:
00215         if (event.key == eng::Key::Escape)
00216         {
00217             if (onBackToMulti)
00218             {
00219                 onBackToMulti();
00220             }
00221         }
00222         else if (event.key == eng::Key::Up)
00223         {
00224             m_playMusic = true;
00225             m_selectedIndex = (m_selectedIndex == 0) ? m_options.size() - 1 : m_selectedIndex - 1;
00226         }
00227         else if (event.key == eng::Key::Down)
00228         {
00229             m_playMusic = true;
00230             m_selectedIndex = (m_selectedIndex == m_options.size() - 1) ? 0 : m_selectedIndex + 1;
00231         }
00232     }

```

```

00232         else if (event.key == eng::Key::Enter)
00233         {
00234             if (m_selectedIndex == 2 && onCreate)
00235             {
00236                 createRoom();
00237                 onCreate(m_roomName, m_maxPlayers);
00238             }
00239             else if (m_selectedIndex == 3 && onBackToMulti)
00240             {
00241                 onBackToMulti();
00242             }
00243         }
00244         else if (event.key == eng::Key::Left || event.key == eng::Key::Right)
00245         {
00246             if (m_selectedIndex == 1)
00247             {
00248                 if (event.key == eng::Key::Left && m_maxPlayers > 2)
00249                 {
00250                     m_maxPlayers--;
00251                     updateValueDisplay();
00252                 }
00253                 else if (event.key == eng::Key::Right && m_maxPlayers < 4)
00254                 {
00255                     m_maxPlayers++;
00256                     updateValueDisplay();
00257                 }
00258             }
00259         }
00260         else if (event.key == eng::Key::Delete)
00261         {
00262             if (m_selectedIndex == 0)
00263             {
00264                 if (!m_roomName.empty())
00265                 {
00266                     m_roomName.pop_back();
00267                     updateValueDisplay();
00268                 }
00269             }
00270         }
00271         else
00272         {
00273             if (m_selectedIndex == 0)
00274             {
00275                 const char inputChar = keyToChar(event.key);
00276
00277                 if (inputChar != '\0')
00278                 {
00279                     if (m_roomName.length() < 20)
00280                     {
00281                         m_roomName += inputChar;
00282                         updateValueDisplay();
00283                     }
00284                 }
00285             }
00286         }
00287         break;
00288     case eng::EventType::KeyReleased:
00289         break;
00290     default:
00291         break;
00292 }
00293 }
00294
00295 void gme::CreateRoomScene::updateValueDisplay()
00296 {
00297     auto &reg = getRegistry();
00298
00299     if (auto *roomNameText = reg.getComponent<ecs::Text>(m_roomNameValueEntity))
00300     {
00301         roomNameText->content = m_roomName;
00302     }
00303
00304     if (auto *maxPlayersText = reg.getComponent<ecs::Text>(m_maxPlayersValueEntity))
00305     {
00306         maxPlayersText->content = std::to_string(m_maxPlayers);
00307     }
00308 }
00309
00310 void gme::CreateRoomScene::createRoom() const
00311 {
00312     utl::Logger::log("CreateRoomScene: Creating room '" + m_roomName + "' with " + std::to_string(m_maxPlayers) +
00313                     " max players",
00314                     utl::LogLevel::INFO);
00315
00316     rnp::PacketLobbyCreate packet{};
00317     packet.maxPlayers = static_cast<std::uint8_t>(m_maxPlayers);
00318     packet.gameMode = 0;

```

19.207

/home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/src/scenes/gameMulti.cpp	
File Reference	1011

```

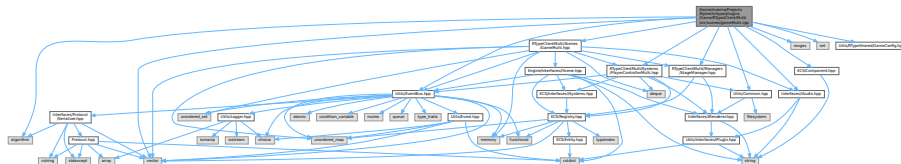
00319 packet.nameLen = static_cast<uint8_t>(m_roomName.size());
00320 for (size_t i = 0; i < m_roomName.size(); ++i)
00321 {
00322     packet.lobbyName[i] = m_roomName[i];
00323 }
00324
00325 utl::Logger::log("CreateRoomScene: Publishing LOBBY_CREATE event to component " +
00326                 std::to_string(utl::NETWORK_CLIENT),
00327                 utl::LogLevel::INFO);
00328 m_eventBus.publish(utl::EventType::LOBBY_CREATE, packet, m_eventComponentId, utl::NETWORK_CLIENT);
00329 utl::Logger::log("CreateRoomScene: LOBBY_CREATE event published successfully", utl::LogLevel::INFO);
00330 }
00331
00332 void gme::CreateRoomScene::processEventBus() const
00333 {
00334     for (const auto events = m_eventBus.consumeForTarget(m_eventComponentId); const auto &event : events)
00335     {
00336         if (event.type == utl::EventType::LOBBY_CREATE_RESPONSE)
00337         {
00338             rnp::Serializer serializer(event.data);
00339             if (const auto [lobbyId, success, errorCode] = serializer.deserializeLobbyCreateResponse(); success != 0U)
00340             {
00341                 utl::Logger::log("CreateRoomScene: Lobby created successfully with ID " + std::to_string(lobbyId),
00342                                 utl::LogLevel::INFO);
00343
00344                 if (onRoomCreated)
00345                 {
00346                     onRoomCreated(static_cast<int>(lobbyId), nullptr);
00347                 }
00348             }
00349             else
00350             {
00351                 utl::Logger::log("CreateRoomScene: Failed to create lobby, error code " + std::to_string(errorCode),
00352                                 utl::LogLevel::WARNING);
00353             }
00354         }
00355     }
00356 }

```

19.207 /home/masina/Projects/Epitech/rtype/plugins/Game/RType/↵
Client/Multi/src/scenes/gameMulti.cpp File Reference

```
#include <algorithm>
#include <ranges>
#include <set>
#include <vector>
#include "ECS/Component.hpp"
#include "Interfaces/IAudio.hpp"
#include "RTypeClientMulti/Scenes/GameMulti.hpp"
#include "RTypeClientMulti/Managers/StageManager.hpp"
#include "RTypeClientMulti/Systems/PlayerControllerMulti.hpp"
#include "Utils/Common.hpp"
#include "Utils/EventBus.hpp"
#include "Utils/RTypeShared/GameConfig.hpp"
```

Include dependency graph for gameMulti.cpp:



19.208 gameMulti.cpp

Go to the documentation of this file.

```
00001 #include <algorithm>
00002 #include <ranges>
00003 #include <set>
```

```

00004 #include <vector>
00005
00006 #include "ECS/Component.hpp"
00007 #include "Interfaces/IAudio.hpp"
00008 #include "RTypeClientMulti/Scenes/GameMulti.hpp"
00009 #include "RTypeClientMulti/Managers/StageManager.hpp"
00010 #include "RTypeClientMulti/Systems/PlayerControllerMulti.hpp"
00011 #include "Utils/Common.hpp"
00012 #include "Utils/EventBus.hpp"
00013 #include "Utils/RTypeShared/GameConfig.hpp"
00014
00015 gme::GameMulti::GameMulti(const eng::id assignedId, const std::shared_ptr<eng::IRenderer> &renderer,
00016                          const std::shared_ptr<eng::IAudio> &audio, const float skinIndex, bool &showDebug,
00017                          const uint32_t sessionId)
00018 : AScene(assignedId), m_audio(audio), m_renderer(renderer), m_skinIndex(skinIndex), m_showDebug(showDebug),
00019   m_sessionId(sessionId)
00020 {
00021     auto &registry = AScene::getRegistry();
00022
00023     registry.onComponentAdded(
00024         [this, &renderer, &audio, &registry](const ecs::Entity e, const std::type_info &type)
00025         {
00026             const auto *audioComp = registry.getComponent<ecs::Audio>(e);
00027             const auto *colorComp = registry.getComponent<ecs::Color>(e);
00028             const auto *fontComp = registry.getComponent<ecs::Font>(e);
00029             const auto *rectComp = registry.getComponent<ecs::Rect>(e);
00030             const auto *scaleComp = registry.getComponent<ecs::Scale>(e);
00031             const auto *textComp = registry.getComponent<ecs::Text>(e);
00032             const auto *textureComp = registry.getComponent<ecs::Texture>(e);
00033             const auto *transform = registry.getComponent<ecs::Transform>(e);
00034             const auto *hitBox = registry.getComponent<ecs::Hitbox>(e);
00035
00036             if (hitBox && transform)
00037             {
00038                 renderer->createCircleShape({.name = "hitbox_" + std::to_string(e),
00039                                             .radius = hitBox->radius,
00040                                             .color = {.r = 255, .g = 0, .b = 0, .a = 100},
00041                                             .x = transform->x + hitBox->offsetX,
00042                                             .y = transform->y + hitBox->offsetY,
00043                                             .outline_thickness = 1.0f,
00044                                             .outline_color = {.r = 255, .g = 0, .b = 0, .a = 200}});
00045             }
00046             if (type == typeid(ecs::Text))
00047             {
00048                 if (textComp && transform && fontComp)
00049                 {
00050                     // Only create font if not already loaded (cache)
00051                     if (this->m_loadedFonts.find(fontComp->id) == this->m_loadedFonts.end())
00052                     {
00053                         renderer->createFont(fontComp->id, fontComp->path);
00054                         this->m_loadedFonts.insert(fontComp->id);
00055                     }
00056                     renderer->createText(
00057                         {.font_name = fontComp->id,
00058                         .color = {.r = colorComp->r, .g = colorComp->g, .b = colorComp->b, .a = colorComp->a},
00059                         .content = textComp->content,
00060                         .size = textComp->font_size,
00061                         .x = transform->x,
00062                         .y = transform->y,
00063                         .name = textComp->id});
00064                 }
00065             }
00066             else if (type == typeid(ecs::Texture))
00067             {
00068                 const float scale_x = scaleComp ? scaleComp->x : 1.F;
00069                 const float scale_y = scaleComp ? scaleComp->y : 1.F;
00070
00071                 // Only create texture if not already loaded (cache)
00072                 if (this->m_loadedTextures.find(textureComp->id) == this->m_loadedTextures.end())
00073                 {
00074                     renderer->createTexture(textureComp->id, textureComp->path);
00075                     this->m_loadedTextures.insert(textureComp->id);
00076                 }
00077
00078                 if (transform && textureComp)
00079                 {
00080                     if (rectComp)
00081                     {
00082                         renderer->createSprite(textureComp->id + std::to_string(e), textureComp->id, transform->x,
00083                                                transform->y, scale_x, scale_y, static_cast<int>(rectComp->pos_x),
00084                                                static_cast<int>(rectComp->pos_y), rectComp->size_x, rectComp->size_y);
00085                     }
00086                     else
00087                     {
00088                         renderer->createSprite(textureComp->id + std::to_string(e), textureComp->id, transform->x,
00089                                                transform->y);
00090                     }
00091                 }
00092             }
00093         }
00094     );
00095 }

```

```

00091     }
00092   }
00093   else if (type == typeid(ecs::Audio))
00094   {
00095     if (audioComp)
00096     {
00097       audio->createAudio(audioComp->path, audioComp->volume, audioComp->loop,
00098         audioComp->id + std::to_string(e));
00099     }
00100   }
00101   });
00102
00103   registry.createEntity().with<ecs::Score>("score", 0).build();
00104
00105   m_playerController = std::make_unique<PlayerControllerMulti>(renderer, m_sessionId);
00106
00107   m_localPlayerEntity = m_playerController->createPlayer(registry, 200.F, 100.F);
00108
00109   if (auto *playerRect = registry.getComponent<ecs::Rect>(m_localPlayerEntity))
00110   {
00111     uint32_t skinIndex = 0;
00112     if (m_playerSkinMap.find(m_sessionId) != m_playerSkinMap.end())
00113     {
00114       skinIndex = m_playerSkinMap[m_sessionId];
00115     }
00116     float skinPosY = static_cast<float>(skinIndex) * utl::GameConfig::Player::SPRITE_HEIGHT;
00117     playerRect->pos_y = skinPosY;
00118   }
00119
00120   if (auto *playerRect = registry.getComponent<ecs::Rect>(m_localPlayerEntity))
00121   {
00122     uint32_t skinIndex = 0;
00123     if (m_playerSkinMap.find(m_sessionId) != m_playerSkinMap.end())
00124     {
00125       skinIndex = m_playerSkinMap[m_sessionId];
00126     }
00127     float skinPosY = static_cast<float>(skinIndex) * utl::GameConfig::Player::SPRITE_HEIGHT;
00128     playerRect->pos_y = skinPosY;
00129   }
00130
00131   auto beginSoundEntity = registry.createEntity()
00132     .with<ecs::Audio>("game_begin", utl::Path::Audio::AUDIO_BEGIN, 1.0F, false, false)
00133     .build();
00134   if (auto *audioComp = registry.getComponent<ecs::Audio>(beginSoundEntity))
00135   {
00136     audioComp->play = true;
00137   }
00138   m_beginSoundEntity = beginSoundEntity;
00139
00140   m_stageManager = std::make_unique<StageManager>();
00141
00142   // Preload all common textures to avoid lag spikes during gameplay
00143   preloadCommonTextures();
00144
00145   setupEventSubscriptions();
00146 }
00147
00148 void gme::GameMulti::preloadCommonTextures()
00149 {
00150   // Preload all textures that will be used during gameplay
00151   // This prevents lag when entities are first created from world state
00152
00153   std::vector<std::pair<std::string, std::string>> texturesToLoad = {
00154     {"player", utl::Path::Texture::TEXTURE_PLAYER},
00155     {"enemy_basic", utl::Path::Texture::TEXTURE_ENEMY_BASIC},
00156     {"enemy_advanced", utl::Path::Texture::TEXTURE_ENEMY_ADVANCED},
00157     {"boss", utl::Path::Texture::TEXTURE_BOSS},
00158     {"shoot", utl::Path::Texture::TEXTURE_SHOOT},
00159     {"shoot_charged", utl::Path::Texture::TEXTURE_SHOOT_CHARGED},
00160     {"enemy_projectile", utl::Path::Texture::TEXTURE_ENEMY_PROJECTILE},
00161     {"explosion", utl::Path::Texture::TEXTURE_EXPLOSION}};
00162
00163   for (const auto &[id, path] : texturesToLoad)
00164   {
00165     if (m_loadedTextures.find(id) == m_loadedTextures.end())
00166     {
00167       m_renderer->createTexture(id, path);
00168       m_loadedTextures.insert(id);
00169     }
00170   }
00171 }
00172
00173 void gme::GameMulti::setupEventSubscriptions() const
00174 {
00175   utl::EventBus &eventBus = utl::EventBus::getInstance();
00176   eventBus.registerComponent(m_eventComponentId, "GameMulti");
00177   eventBus.subscribe(m_eventComponentId, utl::EventType::GAME_START);

```

```

00178     eventBus.subscribe(m_eventComponentId, utl::EventType::PLAYER_INPUT_RECEIVED);
00179     eventBus.subscribe(m_eventComponentId, utl::EventType::WORLD_STATE_RECEIVED);
00180 }
00181
00182 void gme::GameMulti::processEventBus()
00183 {
00184     auto &eventBus = utl::EventBus::getInstance();
00185     for (const std::vector<utl::Event> events = eventBus.consumeForTarget(m_eventComponentId);
00186         const auto &event : events)
00187     {
00188         switch (event.type)
00189         {
00190             case utl::EventType::GAME_START:
00191             {
00192                 break;
00193             }
00194             case utl::EventType::PLAYER_INPUT_RECEIVED:
00195                 break;
00196             case utl::EventType::WORLD_STATE_RECEIVED:
00197                 handleWorldStateUpdate(event);
00198                 break;
00199             default:
00200                 break;
00201         }
00202     }
00203 }
00204
00205 void gme::GameMulti::handleWorldStateUpdate(const utl::Event &event)
00206 {
00207     try
00208     {
00209         rnp::Serializer deserializer(event.data);
00210         rnp::PacketWorldState worldState = deserializer.deserializeWorldState();
00211
00212         auto &registry = getRegistry();
00213
00214         // Track entities present in this world state
00215         std::set<uint32_t> currentEnemyIds;
00216         std::set<uint32_t> currentProjectileIds;
00217
00218         if (m_firstWorldState)
00219         {
00220             uint32_t playerId = 0;
00221             std::vector<uint32_t> playerIds;
00222
00223             playerIds.push_back(m_sessionId);
00224             for (const auto &entityState : worldState.entities)
00225             {
00226                 if (entityState.type == static_cast<std::uint16_t>(rnp::EntityType::PLAYER))
00227                 {
00228                     if (entityState.id != m_sessionId)
00229                     {
00230                         playerIds.push_back(entityState.id);
00231                     }
00232                 }
00233             }
00234
00235             for (uint32_t playerId : playerIds)
00236             {
00237                 m_playerSkinMap[playerId] = playerId;
00238                 playerId++;
00239             }
00240
00241             m_firstWorldState = false;
00242         }
00243
00244         // First pass: collect IDs of entities in the world state
00245         for (const auto &entityState : worldState.entities)
00246         {
00247             if (entityState.type == static_cast<std::uint16_t>(rnp::EntityType::ENEMY) ||
00248                 entityState.type == static_cast<std::uint16_t>(rnp::EntityType::BOSS))
00249             {
00250                 currentEnemyIds.insert(entityState.id);
00251             }
00252             else if (entityState.type == static_cast<std::uint16_t>(rnp::EntityType::PROJECTILE))
00253             {
00254                 currentProjectileIds.insert(entityState.id);
00255             }
00256         }
00257
00258         // Second pass: process entities
00259         for (const auto &entityState : worldState.entities)
00260         {
00261             if (entityState.id == m_sessionId)
00262             {
00263                 if (auto *transform = registry.getComponent<ecs::Transform>(m_localPlayerEntity))
00264                     {

```

```

00265         float deltaX = std::abs(transform->x - entityState.x);
00266         float deltaY = std::abs(transform->y - entityState.y);
00267         const float CORRECTION_THRESHOLD = 5.0f;
00268
00269         if (deltaX > CORRECTION_THRESHOLD || deltaY > CORRECTION_THRESHOLD)
00270         {
00271             float t = 0.2f;
00272             transform->x = transform->x + t * (entityState.x - transform->x);
00273             transform->y = transform->y + t * (entityState.y - transform->y);
00274         }
00275     }
00276     if (auto *velocity = registry.getComponent<ecs::Velocity>(m_localPlayerEntity))
00277     {
00278         velocity->x = entityState.vx;
00279         velocity->y = entityState.vy;
00280     }
00281
00282     // Synchronize beam charge from server
00283     if (auto *beamCharge = registry.getComponent<ecs::BeamCharge>(m_localPlayerEntity))
00284     {
00285         // Always accept server charge value
00286         beamCharge->current_charge = static_cast<float>(entityState.stateFlags) / 255.0f;
00287     }
00288 }
00289 else if (entityState.type == static_cast<std::uint16_t>(rnp::EntityType::PLAYER))
00290 {
00291     if (m_remotePlayers.find(entityState.id) == m_remotePlayers.end())
00292     {
00293         uint32_t skinIndex = 0;
00294         if (m_playerSkinMap.find(entityState.id) != m_playerSkinMap.end())
00295         {
00296             skinIndex = m_playerSkinMap[entityState.id];
00297         }
00298         float skinPosY = static_cast<float>(skinIndex) * utl::GameConfig::Player::SPRITE_HEIGHT;
00299
00300         ecs::Entity remotePlayer =
00301             registry.createEntity()
00302                 .with<ecs::Transform>("remote_player_" + std::to_string(entityState.id), entityState.x,
00303                                     entityState.y, 0.F)
00304                 .with<ecs::Velocity>("remote_velocity_" + std::to_string(entityState.id), entityState.vx,
00305                                     entityState.vy)
00306                 .with<ecs::Rect>("remote_rect_" + std::to_string(entityState.id), 0.F, skinPosY,
00307                                static_cast<int>(utl::GameConfig::Player::SPRITE_WIDTH),
00308                                static_cast<int>(utl::GameConfig::Player::SPRITE_HEIGHT))
00309                 .with<ecs::Scale>("remote_scale_" + std::to_string(entityState.id),
00310                                utl::GameConfig::Player::SCALE, utl::GameConfig::Player::SCALE)
00311                 .with<ecs::Texture>("remote_texture_" + std::to_string(entityState.id),
00312                                utl::Path::Texture::TEXTURE_PLAYER)
00313                 .with<ecs::Player>("remote_player_comp_" + std::to_string(entityState.id), false)
00314                 .build();
00315         m_remotePlayers[entityState.id] = remotePlayer;
00316
00317         m_remotePlayerData[entityState.id] = { .targetX = entityState.x,
00318                                                .targetY = entityState.y,
00319                                                .targetVx = entityState.vx,
00320                                                .targetVy = entityState.vy,
00321                                                .currentX = entityState.x,
00322                                                .currentY = entityState.y,
00323                                                .smoothFactor = REMOTE_PLAYER_SMOOTH_FACTOR,
00324                                                .targetRotation = 0.0f,
00325                                                .currentRotation = 0.0f};
00326     }
00327     else
00328     {
00329         m_remotePlayerData[entityState.id].targetX = entityState.x;
00330         m_remotePlayerData[entityState.id].targetY = entityState.y;
00331         m_remotePlayerData[entityState.id].targetVx = entityState.vx;
00332         m_remotePlayerData[entityState.id].targetVy = entityState.vy;
00333     }
00334 }
00335 else if (entityState.type == static_cast<std::uint16_t>(rnp::EntityType::PROJECTILE))
00336 {
00337
00338     if (m_projectileEntities.find(entityState.id) == m_projectileEntities.end())
00339     {
00340         // Use subtype field to determine projectile type
00341         bool isEnemyProjectile =
00342             (entityState.subtype == static_cast<std::uint8_t>(rnp::EntitySubtype::PROJECTILE_ENEMY));
00343         bool isSupercharged =
00344             (entityState.subtype ==
00345              static_cast<std::uint8_t>(rnp::EntitySubtype::PROJECTILE_PLAYER_SUPERCHARGED));
00346
00347         std::string texturePath;
00348         float width, height, scale;
00349
00350         if (isEnemyProjectile)
00351         {

```



```

00352         // Enemy projectile
00353         texturePath = utl::Path::Texture::TEXTURE_ENEMY_PROJECTILE;
00354         width = 16.0f;
00355         height = 16.0f;
00356         scale = 1.0f;
00357     }
00358     else
00359     {
00360         // Player projectile
00361         texturePath = isSupercharged ? utl::Path::Texture::TEXTURE_SHOOT_CHARGED
00362             : utl::Path::Texture::TEXTURE_SHOOT;
00363         width = isSupercharged ? 29.0f : 20.0f;
00364         height = isSupercharged ? 24.0f : 10.0f;
00365         scale = isSupercharged ? 1.5f : 1.0f;
00366     }
00367
00368     auto entityBuilder =
00369         registry.createEntity()
00370             .with<ecs::Transform>("projectile_" + std::to_string(entityState.id), entityState.x,
00371                 entityState.y, 0.F)
00372             .with<ecs::Velocity>("projectile_velocity_" + std::to_string(entityState.id),
00373                 entityState.vx, entityState.vy)
00374             .with<ecs::Rect>("projectile_rect_" + std::to_string(entityState.id), 0.F, 0.F, width,
00375                 height)
00376             .with<ecs::Scale>("projectile_scale_" + std::to_string(entityState.id), scale, scale)
00377             .with<ecs::Texture>("projectile_texture_" + std::to_string(entityState.id), texturePath);
00378
00379     if (isSupercharged && !isEnemyProjectile)
00380     {
00381         entityBuilder.with<ecs::Animation>("projectile_animation_" + std::to_string(entityState.id), 0,
00382             4, 0.15f, 0.0f, 29, 24, 4);
00383     }
00384
00385     ecs::Entity projectile = entityBuilder.build();
00386     m_projectileEntities[entityState.id] = projectile;
00387
00388     // Don't play sound for every projectile to avoid audio spam and lag
00389     // Sound should be played on player action locally, not on world state sync
00390 }
00391 else
00392 {
00393     // Update existing projectile position and velocity
00394     // Server handles cleanup of off-screen projectiles
00395     if (auto *transform = registry.getComponent<ecs::Transform>(m_projectileEntities[entityState.id]))
00396     {
00397         transform->x = entityState.x;
00398         transform->y = entityState.y;
00399     }
00400     if (auto *velocity = registry.getComponent<ecs::Velocity>(m_projectileEntities[entityState.id]))
00401     {
00402         velocity->x = entityState.vx;
00403         velocity->y = entityState.vy;
00404     }
00405 }
00406 }
00407 else if (entityState.type == static_cast<std::uint16_t>(rnp::EntityType::ENEMY))
00408 {
00409     if (m_enemyEntities.find(entityState.id) == m_enemyEntities.end())
00410     {
00411         // Use subtype field to determine enemy type
00412         std::string texturePath;
00413         float width, height, scale;
00414         int animFrames = 4;
00415
00416         if (entityState.subtype == static_cast<std::uint8_t>(rnp::EntitySubtype::ENEMY_ADVANCED))
00417         {
00418             texturePath = utl::Path::Texture::TEXTURE_ENEMY_ADVANCED;
00419             width = 33.0f;
00420             height = 36.0f;
00421             scale = 2.0f;
00422         }
00423         else // ENEMY_BASIC
00424         {
00425             texturePath = utl::Path::Texture::TEXTURE_ENEMY_BASIC;
00426             width = 32.0f;
00427             height = 32.0f;
00428             scale = 2.0f;
00429         }
00430
00431         ecs::Entity enemy =
00432             registry.createEntity()
00433                 .with<ecs::Transform>("enemy_" + std::to_string(entityState.id), entityState.x,
00434                     entityState.y, 0.F)
00435                 .with<ecs::Velocity>("enemy_velocity_" + std::to_string(entityState.id), entityState.vx,
00436                     entityState.vy)
00437                 .with<ecs::Rect>("enemy_rect_" + std::to_string(entityState.id), 0.F, 0.F, width, height)
00438                 .with<ecs::Scale>("enemy_scale_" + std::to_string(entityState.id), scale, scale)

```



```

00439         .with<ecs::Texture>("enemy_texture_" + std::to_string(entityState.id), texturePath)
00440         .with<ecs::Animation>("enemy_animation_" + std::to_string(entityState.id), 0, animFrames,
00441             0.5f, 0.0f, static_cast<int>(width), static_cast<int>(height),
00442             animFrames)
00443         .build();
00444
00445     m_enemyEntities[entityState.id] = enemy;
00446
00447     m_enemyData[entityState.id] = { .targetX = entityState.x,
00448         .targetY = entityState.y,
00449         .targetVx = entityState.vx,
00450         .targetVy = entityState.vy,
00451         .currentX = entityState.x,
00452         .currentY = entityState.y,
00453         .smoothFactor = ENEMY_SMOOTH_FACTOR,
00454         .targetRotation = 0.0f,
00455         .currentRotation = 0.0f};
00456     }
00457     else
00458     {
00459         m_enemyData[entityState.id].targetX = entityState.x;
00460         m_enemyData[entityState.id].targetY = entityState.y;
00461         m_enemyData[entityState.id].targetVx = entityState.vx;
00462         m_enemyData[entityState.id].targetVy = entityState.vy;
00463     }
00464 }
00465 else if (entityState.type == static_cast<std::uint16_t>(rnp::EntityType::BOSS))
00466 {
00467     if (m_enemyEntities.find(entityState.id) == m_enemyEntities.end())
00468     {
00469         // Boss entity
00470         std::string texturePath = utl::Path::Texture::TEXTURE_BOSS;
00471         float width = 64.0f;
00472         float height = 64.0f;
00473         float scale = 3.0f;
00474
00475         ecs::Entity boss =
00476             registry.createEntity()
00477             .with<ecs::Transform>("boss_" + std::to_string(entityState.id), entityState.x,
00478                 entityState.y, 0.F)
00479             .with<ecs::Velocity>("boss_velocity_" + std::to_string(entityState.id), entityState.vx,
00480                 entityState.vy)
00481             .with<ecs::Rect>("boss_rect_" + std::to_string(entityState.id), 0.F, 0.F, width, height)
00482             .with<ecs::Scale>("boss_scale_" + std::to_string(entityState.id), scale, scale)
00483             .with<ecs::Texture>("boss_texture_" + std::to_string(entityState.id), texturePath)
00484             .with<ecs::Animation>("boss_animation_" + std::to_string(entityState.id), 0, 2, 0.3f, 0.0f,
00485                 static_cast<int>(width), static_cast<int>(height), 2)
00486             .build();
00487
00488         m_enemyEntities[entityState.id] = boss;
00489
00490         m_enemyData[entityState.id] = { .targetX = entityState.x,
00491             .targetY = entityState.y,
00492             .targetVx = entityState.vx,
00493             .targetVy = entityState.vy,
00494             .currentX = entityState.x,
00495             .currentY = entityState.y,
00496             .smoothFactor = ENEMY_SMOOTH_FACTOR,
00497             .targetRotation = 0.0f,
00498             .currentRotation = 0.0f};
00499
00500         utl::Logger::log("GameMulti: Boss spawned with ID " + std::to_string(entityState.id),
00501             utl::LogLevel::INFO);
00502     }
00503     else
00504     {
00505         m_enemyData[entityState.id].targetX = entityState.x;
00506         m_enemyData[entityState.id].targetY = entityState.y;
00507         m_enemyData[entityState.id].targetVx = entityState.vx;
00508         m_enemyData[entityState.id].targetVy = entityState.vy;
00509     }
00510 }
00511 }
00512
00513 // Clean up entities that are no longer in the world state
00514 // Remove enemies that disappeared (create explosion before removal)
00515 std::vector<uint32_t> enemiesToRemove;
00516 for (const auto &[enemyId, enemyEntity] : m_enemyEntities)
00517 {
00518     if (currentEnemyIds.find(enemyId) == currentEnemyIds.end())
00519     {
00520         enemiesToRemove.push_back(enemyId);
00521     }
00522 }
00523
00524 for (uint32_t enemyId : enemiesToRemove)
00525 {

```

```

00526     ecs::Entity enemyEntity = m_enemyEntities[enemyId];
00527
00528     // Create explosion at enemy position before removing
00529     if (auto *transform = registry.getComponent<ecs::Transform>(enemyEntity))
00530     {
00531         registry.createEntity()
00532             .with<ecs::Transform>("explosion_transform", transform->x, transform->y, 0.0f)
00533             .with<ecs::Rect>("explosion_rect", 0.0f, 0.0f,
00534                 static_cast<int>(utl::GameConfig::Explosion::SPRITE_WIDTH),
00535                 static_cast<int>(utl::GameConfig::Explosion::SPRITE_HEIGHT))
00536             .with<ecs::Scale>("explosion_scale", utl::GameConfig::Explosion::SCALE,
utl::GameConfig::Explosion::SCALE)
00537             .with<ecs::Texture>("explosion_texture", utl::Path::Texture::TEXTURE_EXPLOSION)
00538             .with<ecs::Explosion>("explosion", 0, utl::GameConfig::Explosion::ANIMATION_FRAMES,
00539                 utl::GameConfig::Explosion::ANIMATION_DURATION, 0.0f,
00540                 utl::GameConfig::Explosion::SPRITE_WIDTH,
utl::GameConfig::Explosion::SPRITE_HEIGHT,
00541                 utl::GameConfig::Explosion::FRAMES_PER_ROW,
utl::GameConfig::Explosion::LIFETIME, 0.0f)
00542             .build();
00543     }
00544
00545     // Remove all components
00546     if (registry.hasComponent<ecs::Transform>(enemyEntity))
00547         registry.removeComponent<ecs::Transform>(enemyEntity);
00548     if (registry.hasComponent<ecs::Velocity>(enemyEntity))
00549         registry.removeComponent<ecs::Velocity>(enemyEntity);
00550     if (registry.hasComponent<ecs::Rect>(enemyEntity))
00551         registry.removeComponent<ecs::Rect>(enemyEntity);
00552     if (registry.hasComponent<ecs::Scale>(enemyEntity))
00553         registry.removeComponent<ecs::Scale>(enemyEntity);
00554     if (registry.hasComponent<ecs::Texture>(enemyEntity))
00555         registry.removeComponent<ecs::Texture>(enemyEntity);
00556     if (registry.hasComponent<ecs::Animation>(enemyEntity))
00557         registry.removeComponent<ecs::Animation>(enemyEntity);
00558
00559     m_enemyEntities.erase(enemyId);
00560     m_enemyData.erase(enemyId);
00561 }
00562
00563 // Remove projectiles that disappeared
00564 std::vector<uint32_t> projectilesToRemove;
00565 for (const auto &projectId : m_projectileEntities | std::views::keys)
00566 {
00567     if (!currentProjectileIds.contains(projectId))
00568     {
00569         projectilesToRemove.push_back(projectId);
00570     }
00571 }
00572
00573 for (uint32_t projectId : projectilesToRemove)
00574 {
00575     ecs::Entity projectileEntity = m_projectileEntities[projectId];
00576
00577     // Remove all components
00578     if (registry.hasComponent<ecs::Transform>(projectileEntity))
00579         registry.removeComponent<ecs::Transform>(projectileEntity);
00580     if (registry.hasComponent<ecs::Velocity>(projectileEntity))
00581         registry.removeComponent<ecs::Velocity>(projectileEntity);
00582     if (registry.hasComponent<ecs::Rect>(projectileEntity))
00583         registry.removeComponent<ecs::Rect>(projectileEntity);
00584     if (registry.hasComponent<ecs::Scale>(projectileEntity))
00585         registry.removeComponent<ecs::Scale>(projectileEntity);
00586     if (registry.hasComponent<ecs::Texture>(projectileEntity))
00587         registry.removeComponent<ecs::Texture>(projectileEntity);
00588     if (registry.hasComponent<ecs::Animation>(projectileEntity))
00589         registry.removeComponent<ecs::Animation>(projectileEntity);
00590
00591     m_projectileEntities.erase(projectId);
00592 }
00593 }
00594 catch (const std::exception &e)
00595 {
00596     utl::Logger::log("GameMulti: Error handling world state: " + std::string(e.what()), utl::LogLevel::WARNING);
00597 }
00598 }
00599
00600 void gme::GameMulti::update(const float dt, const eng::WindowSize &size)
00601 {
00602     auto &reg = getRegistry();
00603
00604     if (m_playerController)
00605     {
00606         m_playerController->update(reg, dt);
00607     }
00608     for (const auto &audios = reg.getAll<ecs::Audio>(); const auto &audio : audios | std::views::values)
00609     {

```

```

00610         if (!audio.play && audio.loop && (m_audio->isPlaying(audio.id) == eng::Status::Playing))
00611         {
00612             m_audio->stopAudio(audio.id);
00613         }
00614     }
00615     if (m_beginSoundEntity != ecs::Entity{} && m_stageManager && !m_bossMusicStarted)
00616     {
00617         thread_local std::string beginNameCache;
00618         beginNameCache.clear();
00619         beginNameCache = "game_begin";
00620         beginNameCache += std::to_string(m_beginSoundEntity);
00621
00622         if (m_audio->isPlaying(beginNameCache) != eng::Status::Playing)
00623         {
00624             if (auto *beginAudio = reg.getComponent<ecs::Audio>(m_beginSoundEntity))
00625             {
00626                 beginAudio->play = false;
00627             }
00628             m_stageManager->stopScrolling(reg);
00629             m_bossMusicEntity = reg.createEntity()
00630                 .with<ecs::Audio>("boss_music", utl::Path::Audio::AUDIO_BOSS, 1.0f, true, false)
00631                 .build();
00632             if (auto *bossAudio = reg.getComponent<ecs::Audio>(m_bossMusicEntity))
00633             {
00634                 bossAudio->loop = true;
00635                 bossAudio->play = true;
00636             }
00637             m_bossMusicStarted = true;
00638             m_bossMusicTimer = 0.0f;
00639             m_beginSoundEntity = {};
00640         }
00641     }
00642     if (m_bossMusicStarted)
00643     {
00644         m_bossMusicTimer += dt;
00645         if (m_bossMusicTimer >= BOSS_MUSIC_DURATION)
00646         {
00647             if (auto *bossAudio = reg.getComponent<ecs::Audio>(m_bossMusicEntity))
00648             {
00649                 bossAudio->loop = false;
00650                 bossAudio->play = false;
00651             }
00652             m_bossMusicStarted = false;
00653         }
00654         else
00655         {
00656             if (auto *bossAudio = reg.getComponent<ecs::Audio>(m_bossMusicEntity))
00657             {
00658                 bossAudio->loop = true;
00659                 bossAudio->play = true;
00660             }
00661         }
00662     }
00663
00664     updateInterpolation(m_remotePlayerData, m_remotePlayers, REMOTE_PLAYER_SMOOTH_FACTOR, dt, reg);
00665     updateInterpolation(m_projectileData, m_projectileEntities, PROJECTILE_SMOOTH_FACTOR, dt, reg);
00666     updateInterpolation(m_enemyData, m_enemyEntities, ENEMY_SMOOTH_FACTOR, dt, reg);
00667
00668     m_stageManager->update(reg, dt, size);
00669
00670     processEventBus();
00671 }
00672
00673 void gme::GameMulti::event(const eng::Event &event)
00674 {
00675     auto &reg = getRegistry();
00676     m_playerController->handleInput(reg, event);
00677 }
00678
00679 void gme::GameMulti::updateInterpolation(std::unordered_map<uint32_t, InterpolationData> &dataMap,
00680                                         std::unordered_map<uint32_t, ecs::Entity> &entityMap, float smoothFactor,
00681                                         float dt, ecs::Registry &registry) {
00682     for (auto &[entityId, interpData] : dataMap)
00683     {
00684         if (&dataMap == &m_remotePlayerData && entityId == m_sessionId) {
00685             continue;
00686         }
00687
00688         if (entityMap.contains(entityId))
00689         {
00690             const ecs::Entity entity = entityMap[entityId];
00691             if (auto *transform = registry.getComponent<ecs::Transform>(entity))
00692             {
00693                 interpData.currentX += (interpData.targetX - interpData.currentX) * interpData.smoothFactor;
00694                 interpData.currentY += (interpData.targetY - interpData.currentY) * interpData.smoothFactor;
00695
00696                 transform->x = interpData.currentX;

```



```

00022         if (type == typeid(ecs::Text))
00023         {
00024             if (textComp && transform && fontComp)
00025             {
00026                 renderer->createFont(fontComp->id, fontComp->path);
00027                 renderer->createText(
00028                     { .font_name = fontComp->id,
00029                       .color = { .r = colorComp->r, .g = colorComp->g, .b = colorComp->b, .a = colorComp->a },
00030                       .content = textComp->content,
00031                       .size = textComp->font_size,
00032                       .x = transform->x,
00033                       .y = transform->y,
00034                       .name = textComp->id });
00035             }
00036         }
00037     });
00038
00039     registry.createEntity()
00040         .with<ecs::Font>("main_font", utl::Path::Font::FONTS_RTYPE)
00041         .with<ecs::Transform>("transform_title", 100.F, 60.F, 0.F)
00042         .with<ecs::Color>("color_title", utl::Config::Color::CYAN_ELECTRIC.r, utl::Config::Color::CYAN_ELECTRIC.g,
00043             utl::Config::Color::CYAN_ELECTRIC.b, utl::Config::Color::CYAN_ELECTRIC.a)
00044         .with<ecs::Text>("title", std::string("JOIN ROOM"), 72U)
00045         .build();
00046
00047     m_noRoomsEntity =
00048         registry.createEntity()
00049             .with<ecs::Font>("main_font", utl::Path::Font::FONTS_RTYPE)
00050             .with<ecs::Transform>("transform_no_rooms", 100.F, 200.F, 0.F)
00051             .with<ecs::Color>("color_no_rooms", utl::Config::Color::INFO_TEXT_COLOR.r,
00052                 utl::Config::Color::INFO_TEXT_COLOR.g, utl::Config::Color::INFO_TEXT_COLOR.b,
00053                 utl::Config::Color::INFO_TEXT_COLOR.a)
00054             .with<ecs::Text>("no_rooms_text", std::string("No rooms available"), 32U)
00055             .build();
00056
00057     registry.createEntity()
00058         .with<ecs::Font>("main_font", utl::Path::Font::FONTS_RTYPE)
00059         .with<ecs::Transform>("transform_refresh", 100.F, 400.F, 0.F)
00060         .with<ecs::Color>("color_refresh", utl::Config::Color::GRAY_BLUE_SUBTLE.r,
00061             utl::Config::Color::GRAY_BLUE_SUBTLE.g, utl::Config::Color::GRAY_BLUE_SUBTLE.b,
00062             utl::Config::Color::GRAY_BLUE_SUBTLE.a)
00063         .with<ecs::Text>("refresh_text", std::string("Refresh"), 32U)
00064         .build();
00065
00066     registry.createEntity()
00067         .with<ecs::Font>("main_font", utl::Path::Font::FONTS_RTYPE)
00068         .with<ecs::Transform>("transform_back", 100.F, 450.F, 0.F)
00069         .with<ecs::Color>("color_back", utl::Config::Color::GRAY_BLUE_SUBTLE.r,
00070             utl::Config::Color::GRAY_BLUE_SUBTLE.g, utl::Config::Color::GRAY_BLUE_SUBTLE.b, utl::Config::Color::GRAY_BLUE_SUBTLE.a)
00071         .with<ecs::Text>("back_text", std::string("Back"), 32U)
00072         .build();
00073
00074     m_eventComponentId = 7;
00075     m_eventBus.registerComponent(m_eventComponentId, "Join_Room_Scene");
00076     setupEventSubscriptions();
00077 }
00078
00079 void gme::JoinRoomScene::setupEventSubscriptions() const
00080 {
00081     m_eventBus.subscribe(m_eventComponentId, utl::EventType::LOBBY_LIST_RESPONSE);
00082     m_eventBus.subscribe(m_eventComponentId, utl::EventType::LOBBY_JOIN_RESPONSE);
00083 }
00084
00085 void gme::JoinRoomScene::handleLobbyListResponse(const utl::Event &event)
00086 {
00087     try
00088     {
00089         rnp::Serializer serializer(event.data);
00090         auto [lobbyCount, lobbies] = serializer.deserializeLobbyListResponse();
00091
00092         utl::Logger::log("JoinRoomScene: Received lobby list with " + std::to_string(lobbyCount) + " lobbies",
00093             utl::LogLevel::INFO);
00094
00095         setRooms(lobbies);
00096
00097         for (const auto &lobby : lobbies)
00098         {
00099             size_t nameLen = 0;
00100             for (size_t i = 0; i < lobby.lobbyName.size() && lobby.lobbyName[i] != '\0'; ++i)
00101             {
00102                 nameLen = i + 1;
00103             }
00104             std::string lobbyName(lobby.lobbyName.data(), nameLen);
00105             utl::Logger::log("JoinRoomScene: Found lobby '" + lobbyName + "' (ID: " + std::to_string(lobby.lobbyId) +
00106                 ") with " + std::to_string(lobby.currentPlayers) + "/" +
00107                 std::to_string(lobby.maxPlayers) + " players",

```

```

00108         utl::LogLevel::INFO);
00109     }
00110 }
00111 catch (const std::exception &e)
00112 {
00113     utl::Logger::log("JoinRoomScene: Failed to handle lobby list response - " + std::string(e.what()),
00114         utl::LogLevel::WARNING);
00115 }
00116 }
00117
00118 void gme::JoinRoomScene::handleLobbyJoinResponse(const utl::Event &event) const
00119 {
00120     try
00121     {
00122         rnp::Serializer deserializer(event.data);
00123         auto [lobbyId, success, errorCode, lobbyInfo] = deserializer.deserializeLobbyJoinResponse();
00124
00125         if (success == 1)
00126         {
00127             utl::Logger::log("JoinRoomScene: Successfully joined lobby " + std::to_string(lobbyId),
00128                 utl::LogLevel::WARNING);
00129             if (onJoin)
00130             {
00131                 onJoin(static_cast<int>(lobbyId), &lobbyInfo);
00132             }
00133         }
00134         else
00135         {
00136             std::string errorMsg;
00137             switch (static_cast<rnp::ErrorCode>(errorCode))
00138             {
00139                 case rnp::ErrorCode::LOBBY_NOT_FOUND:
00140                     errorMsg = "Lobby not found";
00141                     break;
00142                 case rnp::ErrorCode::LOBBY_FULL:
00143                     errorMsg = "Lobby is full";
00144                     break;
00145                 case rnp::ErrorCode::ALREADY_IN_LOBBY:
00146                     errorMsg = "Already in a lobby";
00147                     break;
00148                 default:
00149                     errorMsg = "Failed to join lobby";
00150                     break;
00151             }
00152             utl::Logger::log("JoinRoomScene: " + errorMsg, utl::LogLevel::WARNING);
00153         }
00154     }
00155     catch (const std::exception &e)
00156     {
00157         utl::Logger::log("JoinRoomScene: Failed to handle lobby join response - " + std::string(e.what()),
00158             utl::LogLevel::WARNING);
00159     }
00160 }
00161
00162 void gme::JoinRoomScene::update(const float dt, const eng::WindowSize & /*size*/)
00163 {
00164     auto &reg = getRegistry();
00165     auto &colors = reg.getAll<ecs::Color>();
00166     auto &texts = reg.getAll<ecs::Text>();
00167     auto &audios = reg.getAll<ecs::Audio>();
00168
00169     m_animationTime += dt;
00170
00171     for (auto &[entity, text] : texts)
00172     {
00173         if (text.id == "refresh_text")
00174         {
00175             auto &color = colors.at(entity);
00176             if (m_selectedIndex == m_rooms.size())
00177             {
00178                 const float glowIntensity = std::sin(m_animationTime * 2.5F);
00179                 color.r = 0U;
00180                 color.g = static_cast<unsigned char>(191U + (glowIntensity * 50));
00181                 color.b = 255U;
00182             }
00183             else
00184             {
00185                 color.r = utl::Config::Color::GRAY_BLUE_SUBTLE.r;
00186                 color.g = utl::Config::Color::GRAY_BLUE_SUBTLE.g;
00187                 color.b = utl::Config::Color::GRAY_BLUE_SUBTLE.b;
00188             }
00189         }
00190         else if (text.id == "back_text")
00191         {
00192             auto &color = colors.at(entity);
00193             if (m_selectedIndex == m_rooms.size() + 1)
00194         {

```

```

00195         const float glowIntensity = std::sin(m_animationTime * 2.5F);
00196         color.r = 0U;
00197         color.g = static_cast<unsigned char>(191U + (glowIntensity * 50));
00198         color.b = 255U;
00199     }
00200     else
00201     {
00202         color.r = utl::Config::Color::GRAY_BLUE_SUBTLE.r;
00203         color.g = utl::Config::Color::GRAY_BLUE_SUBTLE.g;
00204         color.b = utl::Config::Color::GRAY_BLUE_SUBTLE.b;
00205     }
00206 }
00207 }
00208
00209 for (size_t i = 0; i < m_roomEntities.size(); ++i)
00210 {
00211     if (auto *color = reg.getComponent<ecs::Color>(m_roomEntities[i]))
00212     {
00213         if (i == m_selectedIndex)
00214         {
00215             float glowIntensity = std::sin(m_animationTime * 2.5F);
00216             color->r = 0U;
00217             color->g = static_cast<unsigned char>(191U + (glowIntensity * 50));
00218             color->b = 255U;
00219         }
00220         else
00221         {
00222             color->r = utl::Config::Color::TEXT_VALUE_COLOR.r;
00223             color->g = utl::Config::Color::TEXT_VALUE_COLOR.g;
00224             color->b = utl::Config::Color::TEXT_VALUE_COLOR.b;
00225         }
00226     }
00227 }
00228 processEventBus();
00229 }
00230
00231 void gme::JoinRoomScene::event(const eng::Event &event)
00232 {
00233     const size_t totalOptions = m_rooms.size() + 2;
00234
00235     switch (event.type)
00236     {
00237     case eng::EventType::KeyPressed:
00238         if (event.key == eng::Key::Escape)
00239         {
00240             if (onBackToMulti)
00241             {
00242                 onBackToMulti();
00243             }
00244         }
00245         else if (event.key == eng::Key::Up)
00246         {
00247             m_playMusic = true;
00248             if (totalOptions > 0)
00249             {
00250                 m_selectedIndex = (m_selectedIndex == 0) ? totalOptions - 1 : m_selectedIndex - 1;
00251             }
00252         }
00253         else if (event.key == eng::Key::Down)
00254         {
00255             m_playMusic = true;
00256             if (totalOptions > 0)
00257             {
00258                 m_selectedIndex = (m_selectedIndex == totalOptions - 1) ? 0 : m_selectedIndex + 1;
00259             }
00260         }
00261         else if (event.key == eng::Key::Enter)
00262         {
00263             if (m_selectedIndex < m_rooms.size())
00264             {
00265                 rnp::PacketLobbyJoin joinPacket{};
00266                 joinPacket.lobbyId = m_rooms[m_selectedIndex].lobbyId;
00267
00268                 m_eventBus.publish(utl::EventType::LOBBY_JOIN, joinPacket, m_eventComponentId,
utl::NETWORK_CLIENT);
00269             }
00270             else if (m_selectedIndex == m_rooms.size() && onRefreshRequest)
00271             {
00272                 onRefreshRequest();
00273                 refreshRoomList();
00274             }
00275             else if (m_selectedIndex == m_rooms.size() + 1 && onBackToMulti)
00276             {
00277                 onBackToMulti();
00278             }
00279         }
00280         break;

```



```

00281         case eng::EventType::KeyReleased:
00282             break;
00283         default:
00284             break;
00285     }
00286 }
00287
00288 void gme::JoinRoomScene::setRooms(const std::vector<rnpl::LobbyInfo> &rooms)
00289 {
00290     m_rooms = rooms;
00291     updateRoomDisplay();
00292 }
00293
00294 void gme::JoinRoomScene::refreshRoomList() const
00295 {
00296     m_eventBus.publish(utl::EventType::LOBBY_LIST_REQUEST, std::vector<std::uint8_t>(), m_eventComponentId,
00297         utl::NETWORK_CLIENT);
00298
00299     if (onRefreshRequest)
00300     {
00301         onRefreshRequest();
00302     }
00303 }
00304
00305 void gme::JoinRoomScene::updateRoomDisplay()
00306 {
00307     auto &registry = getRegistry();
00308
00309     utl::Logger::log("JoinRoomScene: Updating room display with " + std::to_string(m_rooms.size()) + " rooms",
00310         utl::LogLevel::INFO);
00311
00312     clearRoomEntities();
00313
00314     if (auto *noRoomsText = registry.getComponent<ecs::Text>(m_noRoomsEntity))
00315     {
00316         noRoomsText->content = m_rooms.empty() ? "No rooms available" : "";
00317     }
00318
00319     for (size_t i = 0; i < m_rooms.size(); ++i)
00320     {
00321         const rnpl::LobbyInfo &lobby = m_rooms[i];
00322
00323         size_t nameLen = 0;
00324         for (size_t j = 0; j < lobby.lobbyName.size() && lobby.lobbyName[j] != '\0'; ++j)
00325         {
00326             nameLen = j + 1;
00327         }
00328         std::string lobbyName(lobby.lobbyName.data(), nameLen);
00329
00330         std::string roomText =
00331             lobbyName + " " + std::to_string(lobby.currentPlayers) + "/" + std::to_string(lobby.maxPlayers);
00332
00333         utl::Logger::log("JoinRoomScene: Creating entity for room: " + roomText, utl::LogLevel::INFO);
00334
00335         ecs::Entity roomEntity =
00336             registry.createEntity()
00337                 .with<ecs::Font>("main_font", utl::Path::Font::FONTS_RTYPE)
00338                 .with<ecs::Transform>("transform_room_" + std::to_string(i), 100.F, 200.F + i * 40.F, 0.F)
00339                 .with<ecs::Color>("color_room_" + std::to_string(i), utl::Config::Color::TEXT_VALUE_COLOR.r,
00340                     utl::Config::Color::TEXT_VALUE_COLOR.g, utl::Config::Color::TEXT_VALUE_COLOR.b,
00341                     utl::Config::Color::TEXT_VALUE_COLOR.a)
00342                 .with<ecs::Text>("room_" + std::to_string(i), roomText, 28U)
00343                 .build();
00344
00345         m_roomEntities.push_back(roomEntity);
00346     }
00347
00348     if (m_selectedIndex >= m_rooms.size() + 2)
00349     {
00350         m_selectedIndex = 0;
00351     }
00352 }
00353
00354 void gme::JoinRoomScene::clearRoomEntities()
00355 {
00356     auto &registry = getRegistry();
00357
00358     for (const ecs::Entity entity : m_roomEntities)
00359     {
00360         if (registry.getComponent<ecs::Text>(entity) != nullptr)
00361         {
00362             registry.removeComponent<ecs::Text>(entity);
00363         }
00364         if (registry.getComponent<ecs::Color>(entity) != nullptr)
00365         {
00366             registry.removeComponent<ecs::Color>(entity);
00367         }
00368     }

```



```

00368     if (registry.getComponent<ecs::Transform>(entity) != nullptr)
00369     {
00370         registry.removeComponent<ecs::Transform>(entity);
00371     }
00372     if (registry.getComponent<ecs::Font>(entity) != nullptr)
00373     {
00374         registry.removeComponent<ecs::Font>(entity);
00375     }
00376 }
00377
00378 m_roomEntities.clear();
00379 }
00380
00381 void gme::JoinRoomScene::processEventBus()
00382 {
00383     for (const std::vector<utl::Event> events = m_eventBus.consumeForTarget(m_eventComponentId);
00384          const auto &event : events)
00385     {
00386         switch (event.type)
00387         {
00388             case utl::EventType::LOBBY_LIST_RESPONSE:
00389                 handleLobbyListResponse(event);
00390                 break;
00391             case utl::EventType::LOBBY_JOIN_RESPONSE:
00392                 handleLobbyJoinResponse(event);
00393                 break;
00394             default:
00395                 break;
00396         }
00397     }
00398 }

```

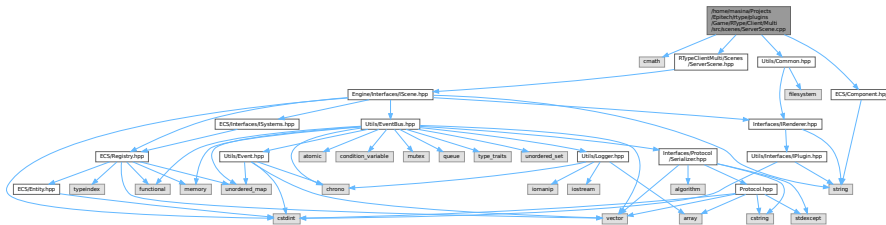
19.211 /home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/src/scenes/ServerScene.cpp File Reference

```

#include <cmath>
#include "ECS/Component.hpp"
#include "RTypeClientMulti/Scenes/ServerScene.hpp"
#include "Utils/Common.hpp"

```

Include dependency graph for ServerScene.cpp:



Functions

- static char [keyToChar](#) (const [eng::Key](#) key, bool shift=false)

19.211.1 Function Documentation

19.211.1.1 keyToChar()

static char keyToChar (
 const [eng::Key](#) key,
 bool shift = false) [static]

Definition at line 7 of file [ServerScene.cpp](#).

References [eng::A](#), [eng::B](#), [eng::C](#), [eng::D](#), [eng::Dot](#), [eng::E](#), [eng::F](#), [eng::G](#), [eng::H](#), [eng::I](#), [eng::J](#), [eng::K](#), [eng::L](#), [eng::M](#), [eng::N](#), [eng::Num0](#), [eng::Num1](#), [eng::Num2](#), [eng::Num3](#), [eng::Num4](#), [eng::Num5](#), [eng::Num6](#), [eng::Num7](#), [eng::Num8](#), [eng::Num9](#), [eng::O](#), [eng::P](#), [eng::Q](#), [eng::R](#), [eng::S](#), [eng::Space](#), [eng::T](#), [eng::U](#), [eng::V](#), [eng::W](#), [eng::X](#), [eng::Y](#), and [eng::Z](#).

19.212 ServerScene.cpp

[Go to the documentation of this file.](#)

```

00001 #include <cmath>
00002
00003 #include "ECS/Component.hpp"
00004 #include "RTYPEClientMulti/Scenes/ServerScene.hpp"
00005 #include "Utils/Common.hpp"
00006
00007 static char keyToChar(const eng::Key key, bool shift = false)
00008 {
00009     switch (key)
00010     {
00011         case eng::Key::Q:
00012             return shift ? 'A' : 'a';
00013         case eng::Key::W:
00014             return shift ? 'Z' : 'z';
00015         case eng::Key::E:
00016             return shift ? 'E' : 'e';
00017         case eng::Key::R:
00018             return shift ? 'R' : 'r';
00019         case eng::Key::T:
00020             return shift ? 'T' : 't';
00021         case eng::Key::Y:
00022             return shift ? 'Y' : 'y';
00023         case eng::Key::U:
00024             return shift ? 'U' : 'u';
00025         case eng::Key::I:
00026             return shift ? 'I' : 'i';
00027         case eng::Key::O:
00028             return shift ? 'O' : 'o';
00029         case eng::Key::P:
00030             return shift ? 'P' : 'p';
00031
00032         case eng::Key::A:
00033             return shift ? 'Q' : 'q';
00034         case eng::Key::S:
00035             return shift ? 'S' : 's';
00036         case eng::Key::D:
00037             return shift ? 'D' : 'd';
00038         case eng::Key::F:
00039             return shift ? 'F' : 'f';
00040         case eng::Key::G:
00041             return shift ? 'G' : 'g';
00042         case eng::Key::H:
00043             return shift ? 'H' : 'h';
00044         case eng::Key::J:
00045             return shift ? 'J' : 'j';
00046         case eng::Key::K:
00047             return shift ? 'K' : 'k';
00048         case eng::Key::M:
00049             return shift ? 'M' : 'm';
00050         case eng::Key::L:
00051             return shift ? 'L' : 'l';
00052
00053         case eng::Key::Z:
00054             return shift ? 'W' : 'w';
00055         case eng::Key::X:
00056             return shift ? 'X' : 'x';
00057         case eng::Key::C:
00058             return shift ? 'C' : 'c';
00059         case eng::Key::V:
00060             return shift ? 'V' : 'v';
00061         case eng::Key::B:
00062             return shift ? 'B' : 'b';
00063         case eng::Key::N:
00064             return shift ? 'N' : 'n';
00065
00066         case eng::Key::Num0:
00067             return '0';
00068         case eng::Key::Num1:
00069             return '1';
00070         case eng::Key::Num2:
00071             return '2';
00072         case eng::Key::Num3:
00073             return '3';
00074         case eng::Key::Num4:
00075             return '4';
00076         case eng::Key::Num5:
00077             return '5';
00078         case eng::Key::Num6:
00079             return '6';
00080         case eng::Key::Num7:
00081             return '7';
00082         case eng::Key::Num8:
00083             return '8';

```

```

00084     case eng::Key::Num9:
00085         return '9';
00086     case eng::Key::Space:
00087         return ' ';
00088     case eng::Key::Dot:
00089         return '.';
00090     default:
00091         return '\0';
00092     }
00093 }
00094
00095 gme::ServerScene::ServerScene(const eng::id assignedId, const std::shared_ptr<eng::IRenderer> &renderer,
00096                               const std::string &host, const std::string &port, const std::string &playerName)
00097 : AScene(assignedId), m_playerName(playerName), m_host(host), m_port(port)
00098 {
00099     auto &registry = AScene::getRegistry();
00100
00101     registry.onComponentAdded(
00102         [&renderer, &registry](const ecs::Entity e, const std::type_info &type)
00103         {
00104             const auto *colorComp = registry.getComponent<ecs::Color>(e);
00105             const auto *fontComp = registry.getComponent<ecs::Font>(e);
00106             const auto *textComp = registry.getComponent<ecs::Text>(e);
00107             const auto *transform = registry.getComponent<ecs::Transform>(e);
00108
00109             if (type == typeid(ecs::Text))
00110             {
00111                 if (textComp && transform && fontComp)
00112                 {
00113                     renderer->createFont(fontComp->id, fontComp->path);
00114                     renderer->createText(
00115                         {font_name = fontComp->id,
00116                          .color = {.r = colorComp->r, .g = colorComp->g, .b = colorComp->b, .a = colorComp->a},
00117                          .content = textComp->content,
00118                          .size = textComp->font_size,
00119                          .x = transform->x,
00120                          .y = transform->y,
00121                          .name = textComp->id});
00122                 }
00123             }
00124         });
00125     registry.createEntity()
00126         .with<ecs::Font>("main_font", utl::Path::Font::FONTS_RTYPE)
00127         .with<ecs::Transform>("transform_title", 100.F, 60.F, 0.F)
00128         .with<ecs::Color>("color_title", utl::Config::Color::CYAN_ELECTRIC.r, utl::Config::Color::CYAN_ELECTRIC.g,
00129                          utl::Config::Color::CYAN_ELECTRIC.b, utl::Config::Color::CYAN_ELECTRIC.a)
00130         .with<ecs::Text>("title", std::string("SERVER"), 72U)
00131         .build();
00132     for (size_t i = 0; i < m_serverOptions.size(); ++i)
00133     {
00134         float yPosition = 0.0F;
00135         if (i < 3)
00136         {
00137             yPosition = 200.F + i * 50.F;
00138         }
00139         else
00140         {
00141             yPosition = 200.F + i * 50.F + 30.F;
00142         }
00143         registry.createEntity()
00144             .with<ecs::Font>("main_font", utl::Path::Font::FONTS_RTYPE)
00145             .with<ecs::Transform>("transform_option_" + std::to_string(i), 100.F, yPosition, 0.F)
00146             .with<ecs::Color>("color_option_" + std::to_string(i), utl::Config::Color::GRAY_BLUE_SUBTLE.r,
00147                              utl::Config::Color::GRAY_BLUE_SUBTLE.g, utl::Config::Color::GRAY_BLUE_SUBTLE.b,
00148                              utl::Config::Color::GRAY_BLUE_SUBTLE.a)
00149             .with<ecs::Text>("option_" + m_serverOptions[i], m_serverOptions[i], 32U)
00150             .build();
00151     }
00152     m_playerNameValueEntity =
00153         registry.createEntity()
00154             .with<ecs::Font>("main_font", utl::Path::Font::FONTS_RTYPE)
00155             .with<ecs::Transform>("transform_player_name_value", 580.F, 200.F, 0.F)
00156             .with<ecs::Color>("color_player_name_value", utl::Config::Color::TEXT_VALUE_COLOR.r,
00157                              utl::Config::Color::TEXT_VALUE_COLOR.g, utl::Config::Color::TEXT_VALUE_COLOR.b,
00158                              utl::Config::Color::TEXT_VALUE_COLOR.a)
00159             .with<ecs::Text>("player_name_value", m_playerName, 24U)
00160             .build();
00161     m_serverIPValueEntity =
00162         registry.createEntity()
00163             .with<ecs::Font>("main_font", utl::Path::Font::FONTS_RTYPE)
00164             .with<ecs::Transform>("transform_server_ip_value", 580.F, 250.F, 0.F)
00165             .with<ecs::Color>("color_server_ip_value", utl::Config::Color::TEXT_VALUE_COLOR.r,
00166                              utl::Config::Color::TEXT_VALUE_COLOR.g, utl::Config::Color::TEXT_VALUE_COLOR.b,
00167                              utl::Config::Color::TEXT_VALUE_COLOR.a)
00168             .with<ecs::Text>("server_ip_value", m_host, 24U)
00169             .build();
00170

```

```

00171
00172     m_serverPortValueEntity =
00173         registry.createEntity()
00174         .with<ecs::Font>("main_font", utl::Path::Font::FONTS_RTYPE)
00175         .with<ecs::Transform>("transform_server_port_value", 580.F, 300.F, 0.F)
00176         .with<ecs::Color>("color_server_port_value", utl::Config::Color::TEXT_VALUE_COLOR.r,
00177             utl::Config::Color::TEXT_VALUE_COLOR.g, utl::Config::Color::TEXT_VALUE_COLOR.b,
00178             utl::Config::Color::TEXT_VALUE_COLOR.a)
00179         .with<ecs::Text>("server_port_value", m_port, 24U)
00180         .build();
00181
00182     m_eventComponentId = 5;
00183     m_eventBus.registerComponent(m_eventComponentId, "ServerConnect");
00184 }
00185
00186 void gme::ServerScene::update(const float dt, const eng::WindowSize & /*size*/)
00187 {
00188     auto &reg = getRegistry();
00189     auto &colors = reg.getAll<ecs::Color>();
00190     auto &texts = reg.getAll<ecs::Text>();
00191
00192     m_animationTime += dt;
00193
00194     for (auto &[entity, text] : texts)
00195     {
00196         for (size_t i = 0; i < m_serverOptions.size(); ++i)
00197         {
00198             if (text.id == "option_" + m_serverOptions[i])
00199             {
00200                 auto &color = colors.at(entity);
00201
00202                 if (i == m_selectedIndex)
00203                 {
00204                     const float glowIntensity = std::sin(m_animationTime * 2.5f);
00205                     color.r = 0U;
00206                     color.g = static_cast<unsigned char>(191U + glowIntensity * 50);
00207                     color.b = 255U;
00208                 }
00209                 else
00210                 {
00211                     color.r = utl::Config::Color::GRAY_BLUE_SUBTLE.r;
00212                     color.g = utl::Config::Color::GRAY_BLUE_SUBTLE.g;
00213                     color.b = utl::Config::Color::GRAY_BLUE_SUBTLE.b;
00214                 }
00215                 break;
00216             }
00217         }
00218     }
00219 }
00220
00221 void gme::ServerScene::event(const eng::Event &event)
00222 {
00223     switch (event.type)
00224     {
00225     case eng::EventType::KeyPressed:
00226         if (event.key == eng::Key::Escape)
00227         {
00228             if (onBackToMenu)
00229             {
00230                 onBackToMenu();
00231             }
00232         }
00233     else if (event.key == eng::Key::Up)
00234     {
00235         m_playMusic = true;
00236         m_selectedIndex = (m_selectedIndex == 0) ? m_serverOptions.size() - 1 : m_selectedIndex - 1;
00237     }
00238     else if (event.key == eng::Key::Down)
00239     {
00240         m_playMusic = true;
00241         m_selectedIndex = (m_selectedIndex == m_serverOptions.size() - 1) ? 0 : m_selectedIndex + 1;
00242     }
00243     else if (event.key == eng::Key::Enter)
00244     {
00245         if (m_selectedIndex == 3 && onConnect)
00246         {
00247             connectServer(m_playerName, m_host, m_port);
00248             onConnect(m_playerName, m_host, m_port);
00249         }
00250         else if (m_selectedIndex == 4 && onBackToMenu)
00251         {
00252             onBackToMenu();
00253         }
00254     }
00255     else if (event.key == eng::Key::Delete)
00256     {
00257         if (m_selectedIndex < 3)

```

```

00258         {
00259             if (std::string &currentField = getCurrentEditField(); !currentField.empty())
00260             {
00261                 currentField.pop_back();
00262                 updateValueDisplay();
00263             }
00264         }
00265     }
00266     else
00267     {
00268         if (m_selectedIndex < 3)
00269         {
00270             if (const char c = keyToChar(event.key); c != '\0')
00271             {
00272                 std::string &currentField = getCurrentEditField();
00273                 if (c == ' ' && m_selectedIndex != 0)
00274                 {
00275                     return;
00276                 }
00277                 if (currentField.length() < 20)
00278                 {
00279                     currentField += c;
00280                     updateValueDisplay();
00281                 }
00282             }
00283         }
00284     }
00285     break;
00286     case eng::EventType::KeyReleased:
00287     break;
00288     default:
00289     break;
00290 }
00291 }
00292
00293 std::string &gme::ServerScene::getCurrentEditField()
00294 {
00295     if (m_selectedIndex == 0)
00296     {
00297         return m_playerName;
00298     }
00299     if (m_selectedIndex == 1)
00300     {
00301         return m_host;
00302     }
00303     return m_port;
00304 }
00305
00306 void gme::ServerScene::updateValueDisplay()
00307 {
00308     auto &reg = getRegistry();
00309
00310     if (auto *playerNameText = reg.getComponent<ecs::Text>(m_playerNameValueEntity))
00311     {
00312         playerNameText->content = m_playerName;
00313     }
00314
00315     if (auto *serverIPText = reg.getComponent<ecs::Text>(m_serverIPValueEntity))
00316     {
00317         serverIPText->content = m_host;
00318     }
00319
00320     if (auto *serverPortText = reg.getComponent<ecs::Text>(m_serverPortValueEntity))
00321     {
00322         serverPortText->content = m_port;
00323     }
00324 }
00325
00326 void gme::ServerScene::connectServer(const std::string &playerName, const std::string &serverIP,
00327                                     const std::string &serverPort) const
00328 {
00329     rnp::Serializer serializer;
00330     serializer.writeString(playerName, 32);
00331     serializer.writeString(serverIP, 15);
00332     serializer.writeString(serverPort, 5);
00333     const auto data = serializer.getData();
00334
00335     m_eventBus.publish(utl::EventType::REQUEST_CONNECT, data, m_eventComponentId,
00336                       utl::NETWORK_CLIENT);
00337 }

```



```

00052         .with<ecs::Color>("color_lobby_id", utl::Config::Color::TEXT_VALUE_COLOR.r,
00053             utl::Config::Color::TEXT_VALUE_COLOR.g, utl::Config::Color::TEXT_VALUE_COLOR.b,
00054             utl::Config::Color::TEXT_VALUE_COLOR.a)
00055         .with<ecs::Text>("lobby_id_text", std::string("Lobby ID: ..."), 36U)
00056         .build();
00057
00058     m_playerCountEntity =
00059         registry.createEntity()
00060         .with<ecs::Font>("main_font", utl::Path::Font::FONTS_RTYPE)
00061         .with<ecs::Transform>("transform_player_count", 100.F, 200.F, 0.F)
00062         .with<ecs::Color>("color_player_count", utl::Config::Color::TEXT_VALUE_COLOR.r,
00063             utl::Config::Color::TEXT_VALUE_COLOR.g, utl::Config::Color::TEXT_VALUE_COLOR.b,
00064             utl::Config::Color::TEXT_VALUE_COLOR.a)
00065         .with<ecs::Text>("player_count_text", std::string("Players: 0/4"), 36U)
00066         .build();
00067
00068     m_statusEntity = registry.createEntity()
00069         .with<ecs::Font>("main_font", utl::Path::Font::FONTS_RTYPE)
00070         .with<ecs::Transform>("transform_status", 100.F, 250.F, 0.F)
00071         .with<ecs::Color>("color_status", utl::Config::Color::INFO_TEXT_COLOR.r,
00072             utl::Config::Color::INFO_TEXT_COLOR.g, utl::Config::Color::INFO_TEXT_COLOR.b,
00073             utl::Config::Color::INFO_TEXT_COLOR.a)
00074         .with<ecs::Text>("status_text", std::string("Waiting for players..."), 32U)
00075         .build();
00076
00077     m_leaveButtonEntity =
00078         registry.createEntity()
00079         .with<ecs::Font>("main_font", utl::Path::Font::FONTS_RTYPE)
00080         .with<ecs::Transform>("transform_leave", 100.F, 500.F, 0.F)
00081         .with<ecs::Color>("color_leave", utl::Config::Color::GRAY_BLUE_SUBTLE.r,
00082             utl::Config::Color::GRAY_BLUE_SUBTLE.g, utl::Config::Color::GRAY_BLUE_SUBTLE.b,
00083             utl::Config::Color::GRAY_BLUE_SUBTLE.a)
00084         .with<ecs::Text>("leave_text", std::string("> Leave Lobby"), 32U)
00085         .build();
00086
00087     m_readyButtonEntity =
00088         registry.createEntity()
00089         .with<ecs::Font>("main_font", utl::Path::Font::FONTS_RTYPE)
00090         .with<ecs::Transform>("transform_ready", 100.F, 450.F, 0.F)
00091         .with<ecs::Color>("color_ready", utl::Config::Color::GRAY_BLUE_SUBTLE.r,
00092             utl::Config::Color::GRAY_BLUE_SUBTLE.g, utl::Config::Color::GRAY_BLUE_SUBTLE.b,
00093             utl::Config::Color::GRAY_BLUE_SUBTLE.a)
00094         .with<ecs::Text>("ready_text", std::string(" Ready"), 32U)
00095         .build();
00096
00097     m_startButtonEntity =
00098         registry.createEntity()
00099         .with<ecs::Font>("main_font", utl::Path::Font::FONTS_RTYPE)
00100         .with<ecs::Transform>("transform_start", 100.F, 550.F, 0.F)
00101         .with<ecs::Color>("color_start", utl::Config::Color::GRAY_BLUE_SUBTLE.r,
00102             utl::Config::Color::GRAY_BLUE_SUBTLE.g, utl::Config::Color::GRAY_BLUE_SUBTLE.b,
00103             utl::Config::Color::GRAY_BLUE_SUBTLE.a)
00104         .with<ecs::Text>("start_text", std::string("> Start Game"), 32U)
00105         .build();
00106
00107     m_eventComponentId = 8;
00108     m_eventBus.registerComponent(m_eventComponentId, "Waiting_Room_Scene");
00109     setupEventSubscriptions();
00110 }
00111
00112 void gme::WaitingRoomScene::setupEventSubscriptions() const
00113 {
00114     m_eventBus.subscribe(m_eventComponentId, utl::EventType::LOBBY_UPDATE);
00115     m_eventBus.subscribe(m_eventComponentId, utl::EventType::GAME_START);
00116 }
00117
00118 void gme::WaitingRoomScene::handleLobbyUpdate(const utl::Event &event)
00119 {
00120     try
00121     {
00122         rnp::Serializer deserializer(event.data);
00123         rnp::PacketLobbyUpdate packet = deserializer.deserializeLobbyUpdate();
00124
00125         utl::Logger::log("WaitingRoomScene: Received lobby update for lobby " +
00126             std::to_string(packet.lobbyInfo.lobbyId) +
00127             " - Players: " + std::to_string(packet.lobbyInfo.currentPlayers) + "/" +
00128             std::to_string(packet.lobbyInfo.maxPlayers),
00129             utl::LogLevel::INFO);
00130
00131         m_currentLobbyInfo = packet.lobbyInfo;
00132         m_hasLobbyInfo = true;
00133         updatePlayerDisplay();
00134     }
00135     catch (const std::exception &e)
00136     {
00137         utl::Logger::log("WaitingRoomScene: Error handling lobby update: " + std::string(e.what()),
00138             utl::LogLevel::WARNING);

```

```

00139     }
00140 }
00141
00142 void gme::WaitingRoomScene::handleGameStart(const utl::Event &event) const
00143 {
00144     try
00145     {
00146         rnp::Serializer deserializer(event.data);
00147         rnp::PacketGameStart packet = deserializer.deserializeGameStart();
00148
00149         utl::Logger::log("WaitingRoomScene: Game starting for lobby " + std::to_string(packet.lobbyId),
00150             utl::LogLevel::INFO);
00151
00152         if (onGameStart)
00153         {
00154             onGameStart();
00155         }
00156     }
00157     catch (const std::exception &e)
00158     {
00159         utl::Logger::log("WaitingRoomScene: Error handling game start: " + std::string(e.what()),
00160             utl::LogLevel::WARNING);
00161     }
00162 }
00163
00164 void gme::WaitingRoomScene::update(float dt, const eng::WindowSize & /*size*/)
00165 {
00166     processEventBus();
00167
00168     m_animationTime += dt;
00169     auto &registry = AScene::getRegistry();
00170
00171     // Update button selection visual
00172     auto *leaveColor = registry.getComponent<ecs::Color>(m_leaveButtonEntity);
00173     auto *readyColor = registry.getComponent<ecs::Color>(m_readyButtonEntity);
00174     auto *startColor = registry.getComponent<ecs::Color>(m_startButtonEntity);
00175     auto *leaveText = registry.getComponent<ecs::Text>(m_leaveButtonEntity);
00176     auto *readyText = registry.getComponent<ecs::Text>(m_readyButtonEntity);
00177     auto *startText = registry.getComponent<ecs::Text>(m_startButtonEntity);
00178
00179     if ((leaveColor != nullptr) && (readyColor != nullptr) && (leaveText != nullptr) && (readyText != nullptr))
00180     {
00181         leaveColor->r = utl::Config::Color::GRAY_BLUE_SUBTLE.r;
00182         leaveColor->g = utl::Config::Color::GRAY_BLUE_SUBTLE.g;
00183         leaveColor->b = utl::Config::Color::GRAY_BLUE_SUBTLE.b;
00184         leaveText->content = " Leave Lobby";
00185
00186         readyColor->r = utl::Config::Color::GRAY_BLUE_SUBTLE.r;
00187         readyColor->g = utl::Config::Color::GRAY_BLUE_SUBTLE.g;
00188         readyColor->b = utl::Config::Color::GRAY_BLUE_SUBTLE.b;
00189         readyText->content = m_isHost ? "> Ready" : " Ready";
00190
00191         if (m_isHost && (startColor != nullptr) && (startText != nullptr))
00192         {
00193             startColor->r = utl::Config::Color::GRAY_BLUE_SUBTLE.r;
00194             startColor->g = utl::Config::Color::GRAY_BLUE_SUBTLE.g;
00195             startColor->b = utl::Config::Color::GRAY_BLUE_SUBTLE.b;
00196             startText->content = " Start Game";
00197         }
00198     }
00199     if (m_selectedButton == BUTTON_LEAVE)
00200     {
00201         leaveColor->r = utl::Config::Color::CYAN_ELECTRIC.r;
00202         leaveColor->g = utl::Config::Color::CYAN_ELECTRIC.g;
00203         leaveColor->b = utl::Config::Color::CYAN_ELECTRIC.b;
00204         leaveText->content = "> Leave Lobby";
00205     }
00206     else if (m_selectedButton == BUTTON_READY)
00207     {
00208         readyColor->r = utl::Config::Color::CYAN_ELECTRIC.r;
00209         readyColor->g = utl::Config::Color::CYAN_ELECTRIC.g;
00210         readyColor->b = utl::Config::Color::CYAN_ELECTRIC.b;
00211         readyText->content = "> Ready";
00212     }
00213     else if (m_selectedButton == BUTTON_START && m_isHost && (startColor != nullptr) && (startText !=
00214         nullptr))
00215     {
00216         startColor->r = utl::Config::Color::CYAN_ELECTRIC.r;
00217         startColor->g = utl::Config::Color::CYAN_ELECTRIC.g;
00218         startColor->b = utl::Config::Color::CYAN_ELECTRIC.b;
00219         startText->content = "> Start Game";
00220     }
00221     if (m_hasLobbyInfo)
00222     {
00223         if (auto *statusColor = registry.getComponent<ecs::Color>(m_statusEntity))
00224         {

```



```

00225         const float pulse = (std::sin(m_animationTime * 2.0F) + 1.0F) / 2.0F;
00226         statusColor->a = static_cast<std::uint8_t>(150U + (pulse * 105.0F));
00227     }
00228 }
00229 }
00230
00231 void gme::WaitingRoomScene::event(const eng::Event &event)
00232 {
00233     switch (event.type)
00234     {
00235         case eng::EventType::KeyPressed:
00236             if (event.key == eng::Key::Up)
00237             {
00238                 if (m_isHost)
00239                 {
00240                     m_selectedButton = (m_selectedButton - 1 + BUTTON_COUNT) % BUTTON_COUNT;
00241                 }
00242                 else
00243                 {
00244                     m_selectedButton = (m_selectedButton - 1 + 2) % 2;
00245                 }
00246             }
00247             else if (event.key == eng::Key::Down)
00248             {
00249                 if (m_isHost)
00250                 {
00251                     m_selectedButton = (m_selectedButton + 1) % BUTTON_COUNT;
00252                 }
00253                 else
00254                 {
00255                     m_selectedButton = (m_selectedButton + 1) % 2;
00256                 }
00257             }
00258             else if (event.key == eng::Key::Enter || event.key == eng::Key::Space)
00259             {
00260                 if (m_selectedButton == BUTTON_LEAVE)
00261                 {
00262                     leaveLobby();
00263                 }
00264                 else if (m_selectedButton == BUTTON_READY)
00265                 {
00266                     utl::Logger::log("WaitingRoomScene: Player ready", utl::LogLevel::INFO);
00267                     // TODO: Send ready status to server
00268                 }
00269                 else if (m_selectedButton == BUTTON_START && m_isHost)
00270                 {
00271                     startGame();
00272                 }
00273             }
00274             else if (event.key == eng::Key::Escape)
00275             {
00276                 leaveLobby();
00277             }
00278             break;
00279         default:
00280             break;
00281     }
00282 }
00283
00284 void gme::WaitingRoomScene::setLobbyId(std::uint32_t lobbyId)
00285 {
00286     m_lobbyId = lobbyId;
00287     auto &registry = AScene::getRegistry();
00288     if (auto *lobbyIdText = registry.getComponent<ecs::Text>(m_lobbyIdEntity))
00289     {
00290         lobbyIdText->content = "Lobby ID: " + std::to_string(lobbyId);
00291     }
00292     utl::Logger::log("WaitingRoomScene: Set lobby ID to " + std::to_string(lobbyId), utl::LogLevel::INFO);
00293 }
00294
00295 void gme::WaitingRoomScene::setLobbyInfo(const rnp::LobbyInfo &lobbyInfo)
00296 {
00297     m_currentLobbyInfo = lobbyInfo;
00298     m_hasLobbyInfo = true;
00299     m_lobbyId = lobbyInfo.lobbyId;
00300     updatePlayerDisplay();
00301     utl::Logger::log("WaitingRoomScene: Lobby info set - Players: " + std::to_string(lobbyInfo.currentPlayers) + "/" +
00302                     std::to_string(lobbyInfo.maxPlayers),
00303                     utl::LogLevel::INFO);
00304 }
00305
00306 void gme::WaitingRoomScene::updatePlayerDisplay()
00307 {
00308     if (!m_hasLobbyInfo)
00309     {
00310         return;
00311     }

```

```

00312
00313     auto &registry = AScene::getRegistry();
00314
00315     if (auto *lobbyIdText = registry.getComponent<ecs::Text>(m_lobbyIdEntity))
00316     {
00317         lobbyIdText->content = "Lobby ID: " + std::to_string(m_currentLobbyInfo.lobbyId);
00318     }
00319
00320     if (auto *playerCountText = registry.getComponent<ecs::Text>(m_playerCountEntity))
00321     {
00322         playerCountText->content = "Players: " + std::to_string(m_currentLobbyInfo.currentPlayers) + "/" +
00323             std::to_string(m_currentLobbyInfo.maxPlayers);
00324     }
00325
00326     // Update status
00327     if (auto *statusText = registry.getComponent<ecs::Text>(m_statusEntity))
00328     {
00329         if (m_currentLobbyInfo.currentPlayers >= m_currentLobbyInfo.maxPlayers)
00330         {
00331             statusText->content = "Lobby is full! Waiting for host to start...";
00332         }
00333         else
00334         {
00335             statusText->content = "Waiting for " +
00336                 std::to_string(m_currentLobbyInfo.maxPlayers - m_currentLobbyInfo.currentPlayers) +
00337                 " more player(s)...";
00338         }
00339     }
00340
00341     clearPlayerEntities();
00342
00343     for (std::uint8_t i = 0; i < m_currentLobbyInfo.maxPlayers; ++i)
00344     {
00345         std::string playerText;
00346         eng::Color playerColor{};
00347
00348         if (i < m_currentLobbyInfo.currentPlayers)
00349         {
00350             playerText = "Player " + std::to_string(i + 1) + " - Connected";
00351             playerColor = utl::Config::Color::GREEN_READY;
00352         }
00353         else
00354         {
00355             playerText = "Player " + std::to_string(i + 1) + " - Waiting...";
00356             playerColor = utl::Config::Color::INFO_TEXT_COLOR;
00357         }
00358
00359         ecs::Entity playerEntity =
00360             registry.createEntity()
00361                 .with<ecs::Font>("main_font", utl::Path::Font::FONTS_RTYPE)
00362                 .with<ecs::Transform>("transform_player_" + std::to_string(i), 120.F, 320.0F + (i * 40.0F), 0.F)
00363                 .with<ecs::Color>("color_player_" + std::to_string(i), playerColor.r, playerColor.g, playerColor.b,
00364                     playerColor.a)
00365                 .with<ecs::Text>("player_" + std::to_string(i), playerText, 28U)
00366                 .build();
00367
00368         m_playerEntities.push_back(playerEntity);
00369     }
00370
00371     utl::Logger::log("WaitingRoomScene: Updated player display - " + std::to_string(m_currentLobbyInfo.currentPlayers) +
00372         "/" + std::to_string(m_currentLobbyInfo.maxPlayers) + " players",
00373         utl::LogLevel::INFO);
00374 }
00375
00376 void gme::WaitingRoomScene::clearPlayerEntities()
00377 {
00378     auto &registry = AScene::getRegistry();
00379     for (const auto &entity : m_playerEntities)
00380     {
00381         if (registry.getComponent<ecs::Text>(entity) != nullptr)
00382         {
00383             registry.removeComponent<ecs::Text>(entity);
00384         }
00385         if (registry.getComponent<ecs::Color>(entity) != nullptr)
00386         {
00387             registry.removeComponent<ecs::Color>(entity);
00388         }
00389         if (registry.getComponent<ecs::Transform>(entity) != nullptr)
00390         {
00391             registry.removeComponent<ecs::Transform>(entity);
00392         }
00393         if (registry.getComponent<ecs::Font>(entity) != nullptr)
00394         {
00395             registry.removeComponent<ecs::Font>(entity);
00396         }
00397     }
00398     m_playerEntities.clear();

```

```
00399 }
00400
00401 void gme::WaitingRoomScene::leaveLobby() const
00402 {
00403     utl::Logger::log("WaitingRoomScene: Leaving lobby " + std::to_string(m_lobbyId), utl::LogLevel::INFO);
00404
00405     m_eventBus.publish(utl::EventType::LOBBY_LEAVE, std::vector<std::uint8_t>(), m_eventComponentId,
00406                       utl::NETWORK_CLIENT);
00407
00408     if (onLeaveLobby)
00409     {
00410         onLeaveLobby();
00411     }
00412 }
00413
00414 void gme::WaitingRoomScene::startGame()
00415 {
00416     utl::Logger::log("WaitingRoomScene: Starting game for lobby " + std::to_string(m_lobbyId), utl::LogLevel::INFO);
00417
00418     if (!m_isHost)
00419     {
00420         utl::Logger::log("WaitingRoomScene: Only host can start game", utl::LogLevel::WARNING);
00421         return;
00422     }
00423
00424     std::vector<std::uint8_t> packetData;
00425     packetData.resize(sizeof(std::uint32_t));
00426     std::memcpy(packetData.data(), &m_lobbyId, sizeof(std::uint32_t));
00427
00428     m_eventBus.publish(utl::EventType::LOBBY_CREATE, packetData, m_eventComponentId,
00429                       utl::NETWORK_CLIENT);
00430 }
00431 void gme::WaitingRoomScene::processEventBus()
00432 {
00433     std::vector<utl::Event> events = m_eventBus.consumeForTarget(m_eventComponentId);
00434     for (const auto &event : events)
00435     {
00436         switch (event.type)
00437         {
00438             case utl::EventType::LOBBY_UPDATE:
00439                 handleLobbyUpdate(event);
00440                 break;
00441             case utl::EventType::GAME_START:
00442                 utl::Logger::log("WaitingRoomScene: Received GAME_START event", utl::LogLevel::INFO);
00443                 if (onGameStart)
00444                 {
00445                     onGameStart();
00446                 }
00447                 break;
00448             default:
00449                 break;
00450         }
00451     }
00452 }
```

19.215 /home/masina/Projects/Epitech/rtype/plugins/Game/RType/↵ Client/Multi/src/Systems/playerControllerMulti.cpp File Reference

```
#include "RTypeClientMulti/Systems/PlayerControllerMulti.hpp"
#include "ECS/Component.hpp"
#include "Interfaces/Protocol/Protocol.hpp"
#include "Interfaces/Protocol/Serializer.hpp"
#include "RTypeClientMulti/Utils/HitboxUtils.hpp"
#include "Utils/RTypeShared/GameConfig.hpp"
```



```

00067     }
00068     if (event.key == eng::Key::Right)
00069     {
00070         m_keysPressed[eng::Key::Right] = false;
00071     }
00072     if (event.key == eng::Key::Space)
00073     {
00074         m_keysPressed[eng::Key::Space] = false;
00075     }
00076     break;
00077
00078     default:
00079         break;
00080 }
00081
00082 sendInputsIfChanged();
00083 }
00084
00085 void gme::PlayerControllerMulti::sendInputsIfChanged()
00086 {
00087     // Send inputs immediately for better responsiveness
00088     // No throttling for shoot input
00089
00090     auto checkKey = [this](const eng::Key key) -> bool
00091     {
00092         auto it = m_keysPressed.find(key);
00093         return it != m_keysPressed.end() && it->second;
00094     };
00095
00096     const bool up = checkKey(eng::Key::Up);
00097     const bool down = checkKey(eng::Key::Down);
00098     const bool left = checkKey(eng::Key::Left);
00099     const bool right = checkKey(eng::Key::Right);
00100     const bool shoot = checkKey(eng::Key::Space);
00101
00102     sendInputToServer(up, down, left, right, shoot);
00103 }
00104
00105 void gme::PlayerControllerMulti::sendInputToServer(bool up, bool down, bool left, bool right, bool shoot) const
00106 {
00107     std::vector<std::uint8_t> inputData;
00108     inputData.push_back(up ? 1 : 0);
00109     inputData.push_back(down ? 1 : 0);
00110     inputData.push_back(left ? 1 : 0);
00111     inputData.push_back(right ? 1 : 0);
00112     inputData.push_back(shoot ? 1 : 0);
00113     // Create EventRecord
00114     rnp::EventRecord eventRecord;
00115     eventRecord.entityId = 0;
00116     eventRecord.type = rnp::EventType::INPUT;
00117     eventRecord.data = inputData;
00118
00119     // Serialize and send
00120     std::vector<rnp::EventRecord> events;
00121     events.push_back(eventRecord);
00122
00123     rnp::Serializer serializer;
00124     serializer.serializeEntityEvents(events);
00125
00126     // Publish to event bus to send to server
00127     m_eventBus.publish(utl::EventType::SEND_ENTITY_EVENT, serializer.getData(), m_componentId,
utl::NETWORK_CLIENT);
00128 }
00129
00130 void gme::PlayerControllerMulti::update(ecs::Registry &registry, float dt)
00131 {
00132     const auto keyboardEntities = registry.getAll<ecs::KeyboardInput>();
00133     ecs::Entity keyboardEntity = ecs::INVALID_ENTITY;
00134     if (!keyboardEntities.empty())
00135     {
00136         keyboardEntity = keyboardEntities.begin()->first;
00137     }
00138     else
00139     {
00140         keyboardEntity = registry.createEntity().with<ecs::KeyboardInput>("keyboard_input").build();
00141     }
00142
00143     if (auto *keyboardInput = registry.getComponent<ecs::KeyboardInput>(keyboardEntity))
00144     {
00145         auto checkKey = [this](const eng::Key key) -> bool
00146         {
00147             auto it = m_keysPressed.find(key);
00148             return it != m_keysPressed.end() && it->second;
00149         };
00150
00151         keyboardInput->space_pressed = checkKey(eng::Key::Space);
00152         keyboardInput->up_pressed = checkKey(eng::Key::Up);

```

```

00153     keyboardInput->down_pressed = checkKey(eng::Key::Down);
00154     keyboardInput->left_pressed = checkKey(eng::Key::Left);
00155     keyboardInput->right_pressed = checkKey(eng::Key::Right);
00156 }
00157
00158 // Client-side prediction: Apply inputs immediately to local player
00159 if (auto *transform = registry.getComponent<ecs::Transform>(m_playerEntity))
00160 {
00161     if (auto *velocity = registry.getComponent<ecs::Velocity>(m_playerEntity))
00162     {
00163         // Calculate velocity based on current key states
00164         constexpr float SPEED = 500.0f;
00165         velocity->x = 0.0f;
00166         velocity->y = 0.0f;
00167
00168         auto checkKey = [this](const eng::Key key) -> bool
00169         {
00170             auto it = m_keysPressed.find(key);
00171             return it != m_keysPressed.end() && it->second;
00172         };
00173
00174         const bool up = checkKey(eng::Key::Up);
00175         const bool down = checkKey(eng::Key::Down);
00176         const bool left = checkKey(eng::Key::Left);
00177         const bool right = checkKey(eng::Key::Right);
00178
00179         if (up)
00180         {
00181             velocity->y = -SPEED;
00182         }
00183         if (down)
00184         {
00185             velocity->y = SPEED;
00186         }
00187         if (left)
00188         {
00189             velocity->x = -SPEED;
00190         }
00191         if (right)
00192         {
00193             velocity->x = SPEED;
00194         }
00195
00196         if (velocity->x != 0.0f && velocity->y != 0.0f)
00197         {
00198             velocity->x *= 0.707f;
00199             velocity->y *= 0.707f;
00200         }
00201
00202         transform->x += velocity->x * dt;
00203         transform->y += velocity->y * dt;
00204
00205         transform->x = std::max(0.0f, std::min(1920.0f, transform->x));
00206         transform->y = std::max(0.0f, std::min(1080.0f, transform->y));
00207     }
00208 }
00209 }
00210
00211 bool gme::PlayerControllerMulti::isSpacePressed() const
00212 {
00213     auto it = m_keysPressed.find(eng::Key::Space);
00214     return it != m_keysPressed.end() && it->second;
00215 }

```

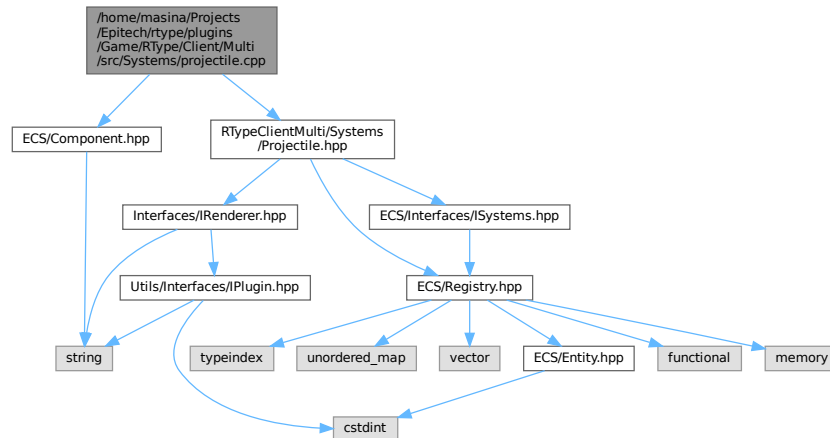
19.217 /home/masina/Projects/Epitech/rtype/plugins/Game/RType/↵ Client/Multi/src/Systems/projectile.cpp File Reference

```

#include "RTypeClientMulti/Systems/Projectile.hpp"
#include "ECS/Component.hpp"

```

Include dependency graph for projectile.cpp:



19.218 projectile.cpp

[Go to the documentation of this file.](#)

```

00001 #include "RTypeClientMulti/Systems/Projectile.hpp"
00002 #include "ECS/Component.hpp"
00003
00004 void gme::ProjectileSystem::update(ecs::Registry &registry, float dt)
00005 {
00006     std::vector<ecs::Entity> entitiesToRemove;
00007
00008     for (auto &[entity, projectile] : registry.getAll<ecs::Projectile>())
00009     {
00010         projectile.current_lifetime += dt;
00011         if (projectile.current_lifetime >= projectile.lifetime)
00012         {
00013             entitiesToRemove.push_back(entity);
00014             continue;
00015         }
00016         auto *transform = registry.getComponent<ecs::Transform>(entity);
00017
00018         if (const auto *velocity = registry.getComponent<ecs::Velocity>(entity);
00019             (transform != nullptr) && (velocity != nullptr))
00020         {
00021             transform->x += velocity->x * dt;
00022             transform->y += velocity->y * dt;
00023         }
00024     }
00025
00026     for (const auto &entity : entitiesToRemove)
00027     {
00028         if (registry.hasComponent<ecs::Projectile>(entity))
00029         {
00030             registry.removeComponent<ecs::Projectile>(entity);
00031         }
00032         if (registry.hasComponent<ecs::Transform>(entity))
00033         {
00034             registry.removeComponent<ecs::Transform>(entity);
00035         }
00036         if (registry.hasComponent<ecs::Velocity>(entity))
00037         {
00038             registry.removeComponent<ecs::Velocity>(entity);
00039         }
00040         if (registry.hasComponent<ecs::Rect>(entity))
00041         {
00042             registry.removeComponent<ecs::Rect>(entity);
00043         }
00044         if (registry.hasComponent<ecs::Scale>(entity))
00045         {
00046             registry.removeComponent<ecs::Scale>(entity);
00047         }
00048         if (registry.hasComponent<ecs::Texture>(entity))
00049         {
00050             registry.removeComponent<ecs::Texture>(entity);
00051         }
00052     }
00053 }

```

```

00052     if (registry.hasComponent<ecs::Animation>(entity))
00053     {
00054         registry.removeComponent<ecs::Animation>(entity);
00055     }
00056 }
00057 }

```

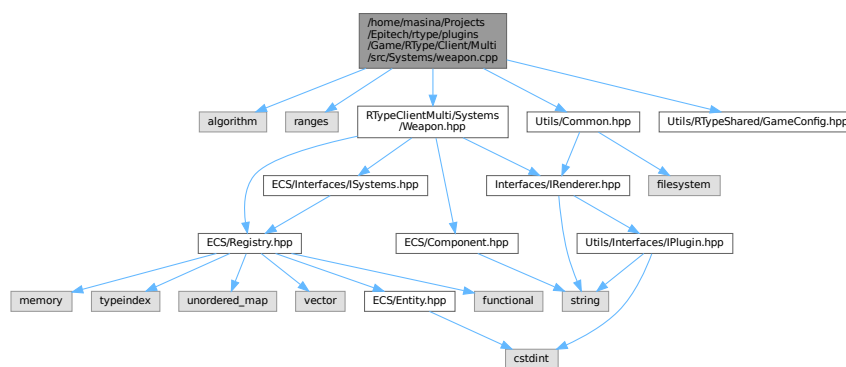
19.219 /home/masina/Projects/Epitech/rtype/plugins/Game/RType/↵ Client/Multi/src/Systems/weapon.cpp File Reference

```

#include <algorithm>
#include <ranges>
#include "RTypeClientMulti/Systems/Weapon.hpp"
#include "Utils/Common.hpp"
#include "Utils/RTypeShared/GameConfig.hpp"

```

Include dependency graph for weapon.cpp:



19.220 weapon.cpp

[Go to the documentation of this file.](#)

```

00001 #include <algorithm>
00002 #include <ranges>
00003
00004 #include "RTypeClientMulti/Systems/Weapon.hpp"
00005 #include "Utils/Common.hpp"
00006 #include "Utils/RTypeShared/GameConfig.hpp"
00007
00008 void gme::WeaponSystem::update(ecs::Registry &registry, const float dt)
00009 {
00010
00011     if (m_fireCooldown > 0.0f)
00012     {
00013         m_fireCooldown -= dt;
00014     }
00015     auto playerEntities = registry.getAll<ecs::Player>();
00016     if (playerEntities.empty())
00017     {
00018         return;
00019     }
00020     auto &[playerEntity, player] = *playerEntities.begin();
00021     const auto *transform = registry.getComponent<ecs::Transform>(playerEntity);
00022     auto *beamCharge = registry.getComponent<ecs::BeamCharge>(playerEntity);
00023     if ((transform == nullptr) || (beamCharge == nullptr))
00024     {
00025         return;
00026     }
00027     auto keyboardEntities = registry.getAll<ecs::KeyboardInput>();
00028     bool spacePressed = false;
00029     if (!keyboardEntities.empty())
00030     {
00031         auto &[keyboardEntity, keyboardInput] = *keyboardEntities.begin();
00032         spacePressed = keyboardInput.space_pressed;
00033     }
00034 }

```



```

00035     if (spacePressed)
00036     {
00037         if (!m_isCharging)
00038         {
00039             m_isCharging = true;
00040         }
00041         beamCharge->current_charge += utl::GameConfig::Beam::CHARGE_RATE * dt;
00042         beamCharge->current_charge = std::min(beamCharge->current_charge, beamCharge->max_charge);
00043         if (beamCharge->current_charge < beamCharge->max_charge)
00044         {
00045             showLoadingAnimation(registry, playerEntity, transform);
00046         }
00047         else
00048         {
00049             hideLoadingAnimation(registry, playerEntity);
00050         }
00051         return;
00052     }
00053     if (m_isCharging)
00054     {
00055         m_isCharging = false;
00056         hideLoadingAnimation(registry, playerEntity);
00057         beamCharge->current_charge = 0.0f;
00058     }
00059 }
00060 }
00061
00062 void gme::WeaponSystem::showLoadingAnimation(ecs::Registry &registry, ecs::Entity /* playerEntity */,
00063                                             const ecs::Transform *playerTransform)
00064 {
00065     for (auto loadingEntities = registry.getAll<ecs::LoadingAnimation>();
00066          const auto &entity : loadingEntities | std::views::keys)
00067     {
00068         if (auto *loadingTransform = registry.getComponent<ecs::Transform>(entity))
00069         {
00070             loadingTransform->x = playerTransform->x + utl::GameConfig::LoadingAnimation::OFFSET_X;
00071             loadingTransform->y = playerTransform->y + utl::GameConfig::LoadingAnimation::OFFSET_Y;
00072             return;
00073         }
00074     }
00075     (void)registry.createEntity()
00076         .with<ecs::Transform>("loading_transform", playerTransform->x +
00077         utl::GameConfig::LoadingAnimation::OFFSET_X,
00078         playerTransform->y + utl::GameConfig::LoadingAnimation::OFFSET_Y, 0.0f)
00079         .with<ecs::Rect>("loading_rect", 0.0f, 0.0f,
00080         static_cast<int>(utl::GameConfig::LoadingAnimation::SPRITE_WIDTH),
00081         static_cast<int>(utl::GameConfig::LoadingAnimation::SPRITE_HEIGHT))
00082         .with<ecs::Scale>("loading_scale", 1.0f, 1.0f)
00083         .with<ecs::Texture>("loading_texture", utl::Path::Texture::TEXTURE_SHOOT_LOADING)
00084         .with<ecs::LoadingAnimation>("loading_animation", 0,
00085         utl::GameConfig::LoadingAnimation::ANIMATION_FRAMES,
00086         utl::GameConfig::LoadingAnimation::ANIMATION_DURATION, 0.0f,
00087         utl::GameConfig::LoadingAnimation::SPRITE_WIDTH,
00088         utl::GameConfig::LoadingAnimation::SPRITE_HEIGHT,
00089         utl::GameConfig::LoadingAnimation::ANIMATION_FRAMES)
00090         .build();
00091 }
00092 void gme::WeaponSystem::hideLoadingAnimation(ecs::Registry &registry, ecs::Entity /* playerEntity */)
00093 {
00094     auto loadingEntities = registry.getAll<ecs::LoadingAnimation>();
00095     std::vector<ecs::Entity> toRemove;
00096     for (const auto &entity : loadingEntities | std::views::keys)
00097     {
00098         toRemove.push_back(entity);
00099     }
00100     for (const auto entity : toRemove)
00101     {
00102         if (registry.hasComponent<ecs::Transform>(entity))
00103         {
00104             registry.removeComponent<ecs::Transform>(entity);
00105         }
00106         if (registry.hasComponent<ecs::Rect>(entity))
00107         {
00108             registry.removeComponent<ecs::Rect>(entity);
00109         }
00110         if (registry.hasComponent<ecs::Scale>(entity))
00111         {
00112             registry.removeComponent<ecs::Scale>(entity);
00113         }
00114         if (registry.hasComponent<ecs::Texture>(entity))
00115         {
00116             registry.removeComponent<ecs::Texture>(entity);
00117         }
00118     }

```

```

00119         if (registry.hasComponent<ecs::LoadingAnimation>(entity))
00120         {
00121             registry.removeComponent<ecs::LoadingAnimation>(entity);
00122         }
00123     }
00124 }
00125
00126 void gme::WeaponSystem::ensureSuperShotAudio(ecs::Registry &registry)
00127 {
00128     if (m_superShotAudioEntity != ecs::INVALID_ENTITY &&
00129         registry.hasComponent<ecs::Audio>(m_superShotAudioEntity))
00130     {
00131         return;
00132     }
00133     m_superShotAudioEntity =
00134         registry.createEntity()
00135             .with<ecs::Audio>("player_super_shot", utl::Path::Audio::AUDIO_SUPERCHARGED_SHOT, 2.0f, false, false)
00136             .build();
00137 }

```

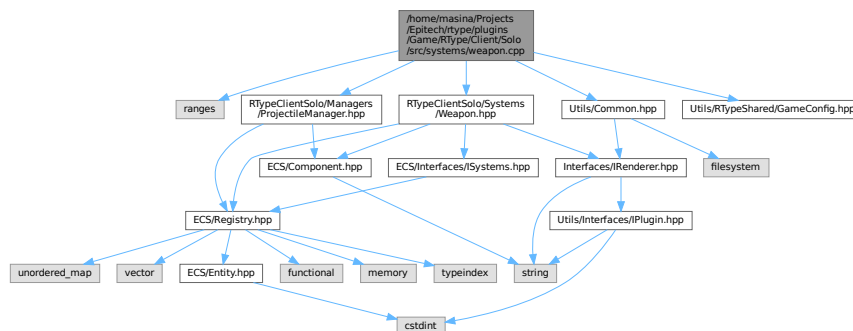
19.221 /home/masina/Projects/Epitech/rtype/plugins/Game/RType/↵ Client/Solo/src/systems/weapon.cpp File Reference

```

#include <ranges>
#include "RTypeClientSolo/Managers/ProjectileManager.hpp"
#include "RTypeClientSolo/Systems/Weapon.hpp"
#include "Utils/Common.hpp"
#include "Utils/RTypeShared/GameConfig.hpp"

```

Include dependency graph for weapon.cpp:



19.222 weapon.cpp

[Go to the documentation of this file.](#)

```

00001 #include <ranges>
00002
00003 #include "RTypeClientSolo/Managers/ProjectileManager.hpp"
00004 #include "RTypeClientSolo/Systems/Weapon.hpp"
00005 #include "Utils/Common.hpp"
00006 #include "Utils/RTypeShared/GameConfig.hpp"
00007
00008 void gme::WeaponSystem::update(ecs::Registry &registry, const float dt)
00009 {
00010     if (m_fireCooldown > 0.0f)
00011     {
00012         m_fireCooldown -= dt;
00013     }
00014     auto playerEntities = registry.getAll<ecs::Player>();
00015     if (playerEntities.empty())
00016         return;
00017     auto &[playerEntity, player] = *playerEntities.begin();
00018     const auto *transform = registry.getComponent<ecs::Transform>(playerEntity);
00019     auto *beamCharge = registry.getComponent<ecs::BeamCharge>(playerEntity);
00020     if (!transform || !beamCharge)
00021         return;
00022     auto keyboardEntities = registry.getAll<ecs::KeyboardInput>();

```

```

00023     bool spacePressed = false;
00024     if (!keyboardEntities.empty())
00025     {
00026         auto &[keyboardEntity, keyboardInput] = *keyboardEntities.begin();
00027         spacePressed = keyboardInput.space_pressed;
00028     }
00029     float projectileX = transform->x + utl::GameConfig::Player::SPRITE_WIDTH;
00030     float projectileY = transform->y + utl::GameConfig::Player::SPRITE_HEIGHT / 2.0f;
00031     if (spacePressed)
00032     {
00033         if (!m_isCharging)
00034         {
00035             m_isCharging = true;
00036         }
00037         beamCharge->current_charge += utl::GameConfig::Beam::CHARGE_RATE * dt;
00038         if (beamCharge->current_charge > beamCharge->max_charge)
00039             beamCharge->current_charge = beamCharge->max_charge;
00040         if (beamCharge->current_charge < beamCharge->max_charge)
00041         {
00042             showLoadingAnimation(registry, playerEntity, transform);
00043         }
00044         else
00045         {
00046             hideLoadingAnimation(registry, playerEntity);
00047         }
00048         return;
00049     }
00050     else
00051     {
00052         if (m_isCharging)
00053         {
00054             m_isCharging = false;
00055             hideLoadingAnimation(registry, playerEntity);
00056             if (m_fireCooldown <= 0.0f)
00057             {
00058                 if (const float chargeThreshold = beamCharge->max_charge * 0.5f;
00059                     beamCharge->current_charge >= chargeThreshold)
00060                 {
00061                     if (tryFireSupercharged(registry, projectileX, projectileY))
00062                     {
00063                         beamCharge->current_charge = 0.0f;
00064                         m_fireCooldown = utl::GameConfig::Projectile::Supercharged::FIRE_COOLDOWN;
00065                     }
00066                     else
00067                     {
00068                         tryFireBasic(registry, projectileX, projectileY);
00069                     }
00070                 }
00071             }
00072         }
00073     }
00074 }
00075
00076 bool gme::WeaponSystem::tryFireBasic(ecs::Registry &registry, float x, float y)
00077 {
00078     if (m_fireCooldown > 0.0f)
00079         return false;
00080
00081     ProjectileManager::createBasicProjectile(registry, x, y, utl::GameConfig::Projectile::Basic::SPEED, 0.0f);
00082     m_fireCooldown = utl::GameConfig::Projectile::Basic::FIRE_COOLDOWN;
00083     return true;
00084 }
00085
00086 bool gme::WeaponSystem::tryFireSupercharged(ecs::Registry &registry, float x, float y)
00087 {
00088     ProjectileManager::createSuperchargedProjectile(registry, x, y, utl::GameConfig::Projectile::Supercharged::SPEED,
00089                                                     0.0f);
00090     ensureSuperShotAudio(registry);
00091     if (m_superShotAudioEntity != ecs::INVALID_ENTITY)
00092     {
00093         if (auto *audio = registry.getComponent<ecs::Audio>(m_superShotAudioEntity))
00094         {
00095             audio->play = true;
00096         }
00097     }
00098     return true;
00099 }
00100
00101 void gme::WeaponSystem::showLoadingAnimation(ecs::Registry &registry, ecs::Entity playerEntity,
00102                                              const ecs::Transform *playerTransform)
00103 {
00104     for (auto loadingEntities = registry.getAll<ecs::LoadingAnimation>();
00105          const auto &entity : loadingEntities | std::views::keys)
00106     {
00107         if (auto *loadingTransform = registry.getComponent<ecs::Transform>(entity))
00108         {
00109             loadingTransform->x = playerTransform->x + utl::GameConfig::LoadingAnimation::OFFSET_X;

```

```

00110         loadingTransform->y = playerTransform->y + utl::GameConfig::LoadingAnimation::OFFSET_Y;
00111         return;
00112     }
00113 }
00114
00115 auto loadingEntity =
00116     registry.createEntity()
00117     .with<ecs::Transform>("loading_transform", playerTransform->x +
00118         utl::GameConfig::LoadingAnimation::OFFSET_X,
00119         playerTransform->y + utl::GameConfig::LoadingAnimation::OFFSET_Y, 0.0f)
00120     .with<ecs::Rect>("loading_rect", 0.0f, 0.0f,
00121         static_cast<int>(utl::GameConfig::LoadingAnimation::SPRITE_WIDTH),
00122         static_cast<int>(utl::GameConfig::LoadingAnimation::SPRITE_HEIGHT))
00123     .with<ecs::Scale>("loading_scale", 1.0f, 1.0f)
00124     .with<ecs::Texture>("loading_texture", utl::Path::Texture::TEXTURE_SHOOT_LOADING)
00125     .with<ecs::LoadingAnimation>("loading_animation", 0,
00126         utl::GameConfig::LoadingAnimation::ANIMATION_FRAMES,
00127         utl::GameConfig::LoadingAnimation::ANIMATION_DURATION, 0.0f,
00128         utl::GameConfig::LoadingAnimation::SPRITE_WIDTH,
00129         utl::GameConfig::LoadingAnimation::SPRITE_HEIGHT,
00130         utl::GameConfig::LoadingAnimation::ANIMATION_FRAMES)
00131     .build();
00132 void game::WeaponSystem::hideLoadingAnimation(ecs::Registry &registry, ecs::Entity playerEntity)
00133 {
00134     auto loadingEntities = registry.getAll<ecs::LoadingAnimation>();
00135     std::vector<ecs::Entity> toRemove;
00136     for (const auto &entity : loadingEntities | std::views::keys)
00137     {
00138         toRemove.push_back(entity);
00139     }
00140     for (const auto entity : toRemove)
00141     {
00142         if (registry.hasComponent<ecs::Transform>(entity))
00143         {
00144             registry.removeComponent<ecs::Transform>(entity);
00145         }
00146         if (registry.hasComponent<ecs::Rect>(entity))
00147         {
00148             registry.removeComponent<ecs::Rect>(entity);
00149         }
00150         if (registry.hasComponent<ecs::Scale>(entity))
00151         {
00152             registry.removeComponent<ecs::Scale>(entity);
00153         }
00154         if (registry.hasComponent<ecs::Texture>(entity))
00155         {
00156             registry.removeComponent<ecs::Texture>(entity);
00157         }
00158         if (registry.hasComponent<ecs::LoadingAnimation>(entity))
00159         {
00160             registry.removeComponent<ecs::LoadingAnimation>(entity);
00161         }
00162     }
00163 }
00164 void game::WeaponSystem::ensureSuperShotAudio(ecs::Registry &registry)
00165 {
00166     if (m_superShotAudioEntity != ecs::INVALID_ENTITY &&
00167         registry.hasComponent<ecs::Audio>(m_superShotAudioEntity))
00168     {
00169         return;
00170     }
00171     m_superShotAudioEntity =
00172         registry.createEntity()
00173         .with<ecs::Audio>("player_super_shot", utl::Path::Audio::AUDIO_SUPERCHARGED_SHOT, 2.0f, false, false)
00174         .build();
00175 }

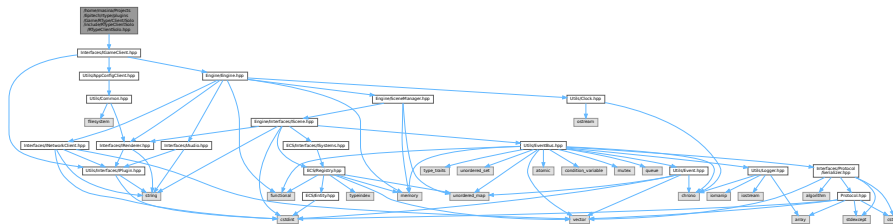
```

19.223 /home/masina/Projects/Epitech/rtype/plugins/Game/RType/↵ Client/Solo/include/RTypeClientSolo/RTypeClientSolo.hpp File Reference

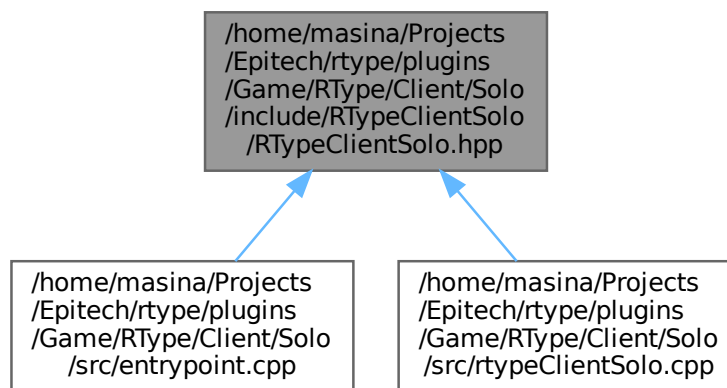
This file contains the RTypeClientSolo class declaration.

```
#include "Interfaces/IGameClient.hpp"
```

Include dependency graph for RTypeClientSolo.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [gme::RTypeClientSolo](#)
Class for the [RTypeClientSolo](#) plugin.

Namespaces

- namespace [gme](#)

19.223.1 Detailed Description

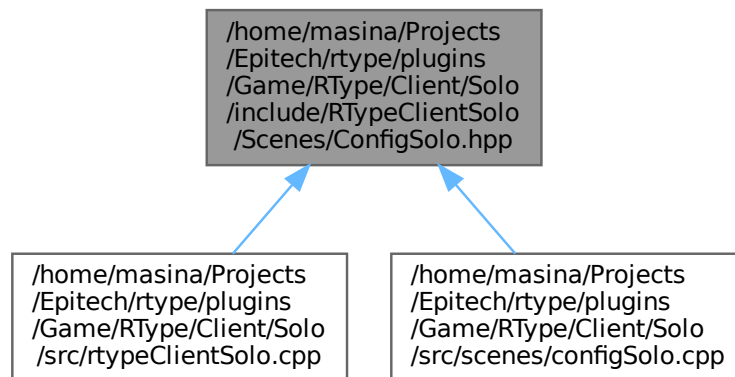
This file contains the RTypeClientSolo class declaration.
Definition in file [RTypeClientSolo.hpp](#).

19.224 RTypeClientSolo.hpp

[Go to the documentation of this file.](#)

```
00001 ///  
00002 ///  
00003 ///  
00004 ///  
00005 ///  
00006 ///  
00007 #pragma once  
00008  
00009 #include "Interfaces/IGameClient.hpp"
```


This graph shows which files directly or indirectly include this file:



Classes

- class `gme::ConfigSolo`
`ConfigSolo` scene.

Namespaces

- namespace `gme`

19.225.1 Detailed Description

This file contains the solo configuration scene.
 Definition in file [ConfigSolo.hpp](#).

19.226 ConfigSolo.hpp

[Go to the documentation of this file.](#)

```

00001 ///
00002 /// @file ConfigSolo.hpp
00003 /// @brief This file contains the solo configuration scene
00004 /// @namespace gme
00005 ///
00006
00007 #pragma once
00008
00009 #include <unordered_map>
00010
00011 #include "Engine/Interfaces/IScene.hpp"
00012
00013 namespace gme
00014 {
00015     ///
00016     /// @class ConfigSolo
00017     /// @brief ConfigSolo scene
00018     /// @namespace gme
00019     ///
00020     class ConfigSolo final : public eng::AScene
00021     {
00022     public:
00023         ConfigSolo(eng::id assignedId, const std::shared_ptr<eng::IRenderer> &renderer);
00024         ~ConfigSolo() override = default;
00025
00026         ConfigSolo(const ConfigSolo &other) = delete;
00027         ConfigSolo &operator=(const ConfigSolo &other) = delete;
00028         ConfigSolo(ConfigSolo &&other) = delete;
00029         ConfigSolo &operator=(ConfigSolo &&other) = delete;
  
```

```

00030
00031 void update(float dt, const eng::WindowSize &size) override;
00032 void event(const eng::Event &event) override;
00033 bool &playMusic() { return m_playMusic; }
00034
00035 std::function<void(const std::string &option)> onOptionSelected;
00036
00037 private:
00038     std::unordered_map<eng::Key, bool> m_keysPressed;
00039     ecs::Entity m_fpsEntity;
00040     ecs::Entity m_titleEntity;
00041     const std::vector<std::string> m_menuOptions = {"Level easy", "Level medium", "Go back to menu"};
00042
00043     int m_selectedIndex = 0;
00044     float m_animationTime = 0.0f;
00045     float m_titlePulseTime = 0.0f;
00046     bool m_playMusic = false;
00047 }; // class ConfigSolo
00048 } // namespace gme

```

19.227 /home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Solo/include/RTypeClientSolo/Scenes/GameSolo.hpp File Reference

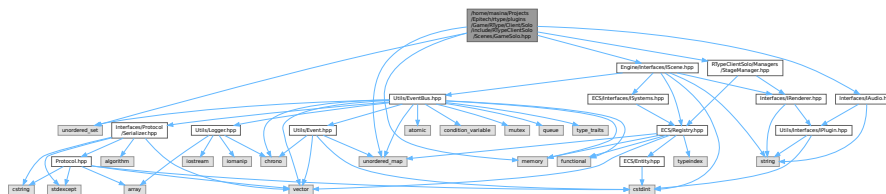
This file contains the solo Game scene.

```

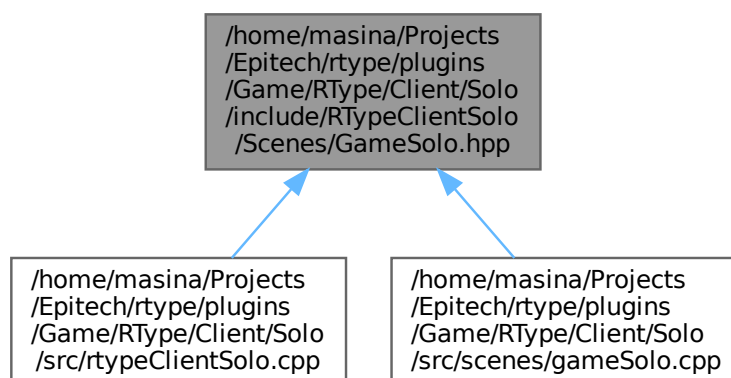
#include <memory>
#include <unordered_map>
#include <unordered_set>
#include "Engine/Interfaces/IScene.hpp"
#include "Interfaces/IAudio.hpp"
#include "RTypeClientSolo/Managers/StageManager.hpp"

```

Include dependency graph for GameSolo.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [gme::GameSolo](#)
 [GameSolo](#) scene.

Namespaces

- namespace [gme](#)

19.227.1 Detailed Description

This file contains the solo Game scene.

Definition in file [GameSolo.hpp](#).

19.228 GameSolo.hpp

[Go to the documentation of this file.](#)

```

00001 ///  

00002 ///  

00003 ///  

00004 ///  

00005 ///  

00006 ///  

00007 #pragma once  

00008  

00009 #include <memory>  

00010 #include <unordered_map>  

00011 #include <unordered_set>  

00012  

00013 #include "Engine/Interfaces/IScene.hpp"  

00014 #include "Interfaces/IAudio.hpp"  

00015 #include "RTypeClientSolo/Managers/StageManager.hpp"  

00016  

00017 namespace gme  

00018 {  

00019     ///  

00020     ///  

00021     ///  

00022     ///  

00023     ///  

00024     class GameSolo final : public eng::AScene  

00025     {  

00026     public:  

00027         GameSolo(eng::id assignedId, const std::shared_ptr<eng::IRenderer> &renderer,  

00028                 const std::shared_ptr<eng::IAudio> &audio, int skinIndex, bool &showDebug);  

00029         ~GameSolo() override = default;  

00030  

00031         GameSolo(const GameSolo &other) = delete;  

00032         GameSolo &operator=(const GameSolo &other) = delete;  

00033         GameSolo(GameSolo &&other) = delete;  

00034         GameSolo &operator=(GameSolo &&other) = delete;  

00035  

00036         void update(float dt, const eng::WindowSize &size) override;  

00037         void event(const eng::Event &event) override;  

00038  

00039         bool &playMusic() { return m_playmusic; }  

00040         void updatePlayerSkin();  

00041  

00042     private:  

00043         void handlePlayerInputs(ecs::Registry &registry, float dt);  

00044         void preloadCommonTextures();  

00045         static ecs::Entity createPlayer(ecs::Registry &registry);  

00046  

00047         const std::shared_ptr<eng::IRenderer> &m_renderer;  

00048         const std::shared_ptr<eng::IAudio> &m_audio;  

00049         std::unordered_map<eng::Key, bool> m_keysPressed;  

00050         ecs::Entity m_playerEntity;  

00051         int m_skinIndex;  

00052         int m_lastAppliedSkinIndex = -1;  

00053         bool &m_showDebug;  

00054         bool m_playmusic = false;  

00055         std::unique_ptr<StageManager> m_stageManager;  

00056  

00057         // Cache to avoid recreating textures/fonts  

00058         std::unordered_set<std::string> m_loadedTextures;  

00059         std::unordered_set<std::string> m_loadedFonts;  

00060     }; // class GameSolo  

00061 } // namespace gme

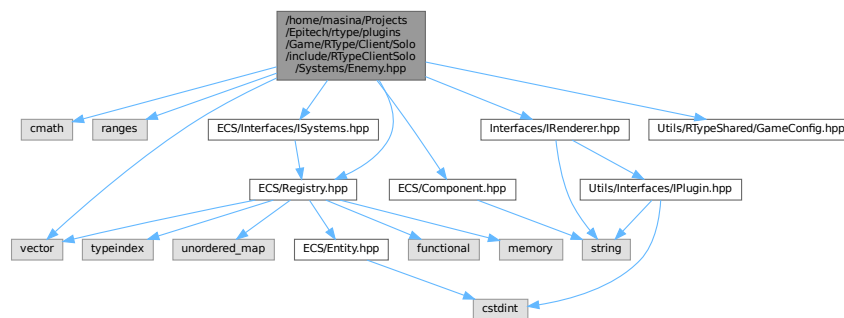
```

19.229 /home/masina/Projects/Epitech/rtype/plugins/Game/RType/↵ Client/Solo/include/RTypeClientSolo/Systems/Enemy.hpp File Reference

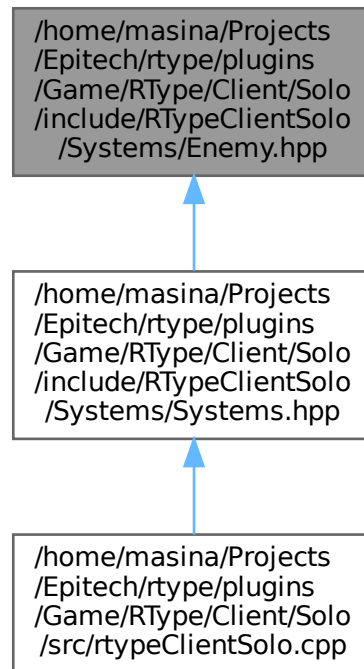
This file contains the enemy system definitions.

```
#include <cmath>
#include <ranges>
#include <vector>
#include "ECS/Component.hpp"
#include "ECS/Interfaces/ISystems.hpp"
#include "ECS/Registry.hpp"
#include "Interfaces/IRenderer.hpp"
#include "Utils/RTypeShared/GameConfig.hpp"
```

Include dependency graph for Enemy.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [gme::EnemySystem](#)

Namespaces

- namespace [gme](#)

19.229.1 Detailed Description

This file contains the enemy system definitions.

Definition in file [Enemy.hpp](#).

19.230 Enemy.hpp

[Go to the documentation of this file.](#)

```

00001 ///
00002 /// @file Enemy.hpp
00003 /// @brief This file contains the enemy system definitions
00004 /// @namespace gme
00005 ///
00006
00007 #pragma once
00008
00009 #include <cmath>
00010 #include <ranges>
00011 #include <vector>
00012
00013 #include "ECS/Component.hpp"
00014 #include "ECS/Interfaces/ISystems.hpp"
00015 #include "ECS/Registry.hpp"
00016 #include "Interfaces/IRenderer.hpp"
00017 #include "Utils/RTypeShared/GameConfig.hpp"

```

```

00018
00019 namespace gme
00020 {
00021
00022     class EnemySystem final : public ecs::ASystem
00023     {
00024     public:
00025         explicit EnemySystem(const std::shared_ptr<eng::IRenderer> &renderer) : m_renderer(renderer) {}
00026         ~EnemySystem() override = default;
00027
00028         EnemySystem(const EnemySystem &) = delete;
00029         EnemySystem &operator=(const EnemySystem &) = delete;
00030         EnemySystem(EnemySystem &&) = delete;
00031         EnemySystem &operator=(EnemySystem &&) = delete;
00032
00033         bool isEnabled() override { return true; }
00034         void setEnable(const bool enable) override { (void)enable; }
00035
00036         void update(ecs::Registry &registry, const float dt) override
00037         {
00038             std::vector<ecs::Entity> enemiesToRemove;
00039
00040             for (const auto &entity : registry.getAll<ecs::Enemy>() | std::views::keys)
00041             {
00042                 auto *transform = registry.getComponent<ecs::Transform>(entity);
00043                 const auto *velocity = registry.getComponent<ecs::Velocity>(entity);
00044                 auto *rect = registry.getComponent<ecs::Rect>(entity);
00045                 const auto *texture = registry.getComponent<ecs::Texture>(entity);
00046                 const auto *scale = registry.getComponent<ecs::Scale>(entity);
00047                 auto *animation = registry.getComponent<ecs::Animation>(entity);
00048
00049                 if ((transform == nullptr) || (velocity == nullptr) || (rect == nullptr) || (texture == nullptr) ||
00050                     (scale == nullptr))
00051                 {
00052                     continue;
00053                 }
00054
00055                 transform->x += velocity->x * dt;
00056                 transform->y += velocity->y * dt;
00057
00058                 if (animation != nullptr)
00059                 {
00060                     animation->current_time += dt;
00061                     if (animation->current_time >= animation->frame_duration)
00062                     {
00063                         animation->current_time = 0.0F;
00064                         animation->current_frame = (animation->current_frame + 1) % animation->total_frames;
00065
00066                         const int frame_x =
00067                             animation->current_frame * static_cast<int>(utl::GameConfig::Enemy::Easy::SPRITE_WIDTH);
00068                         rect->pos_x = static_cast<float>(frame_x);
00069                         rect->pos_y = static_cast<float>(0);
00070                     }
00071                 }
00072
00073                 m_renderer->createSprite(texture->id + std::to_string(entity), texture->path,
00074                     std::round(transform->x), std::round(transform->y), scale->x, scale->y,
00075                     static_cast<int>(rect->pos_x), static_cast<int>(rect->pos_y), rect->size_x,
00076                     rect->size_y);
00077                 m_renderer->drawSprite(texture->id + std::to_string(entity));
00078
00079                 if (transform->x < utl::GameConfig::Screen::REMOVE_X ||
00080                     transform->y < utl::GameConfig::Screen::REMOVE_MIN_Y ||
00081                     transform->y > utl::GameConfig::Screen::REMOVE_MAX_Y)
00082                 {
00083                     enemiesToRemove.push_back(entity);
00084                 }
00085             }
00086
00087             for (const ecs::Entity entity : enemiesToRemove)
00088             {
00089                 if (registry.hasComponent<ecs::Enemy>(entity))
00090                 {
00091                     registry.removeComponent<ecs::Enemy>(entity);
00092                 }
00093                 if (registry.hasComponent<ecs::Transform>(entity))
00094                 {
00095                     registry.removeComponent<ecs::Transform>(entity);
00096                 }
00097                 if (registry.hasComponent<ecs::Velocity>(entity))
00098                 {
00099                     registry.removeComponent<ecs::Velocity>(entity);
00100                 }
00101                 if (registry.hasComponent<ecs::Rect>(entity))
00102                 {
00103                     registry.removeComponent<ecs::Rect>(entity);
00104                 }
00105             }
00106         }
00107     }
00108 }

```

```

00105         if (registry.hasComponent<ecs::Texture>(entity))
00106         {
00107             registry.removeComponent<ecs::Texture>(entity);
00108         }
00109         if (registry.hasComponent<ecs::Scale>(entity))
00110         {
00111             registry.removeComponent<ecs::Scale>(entity);
00112         }
00113         if (registry.hasComponent<ecs::Animation>(entity))
00114         {
00115             registry.removeComponent<ecs::Animation>(entity);
00116         }
00117         if (registry.hasComponent<ecs::Hitbox>(entity))
00118         {
00119             registry.removeComponent<ecs::Hitbox>(entity);
00120         }
00121     }
00122 }
00123
00124 private:
00125     const std::shared_ptr<eng::IRenderer> &m_renderer;
00126 }; // class EnemySystem
00127
00128 } // namespace gme

```

19.231 /home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Solo/include/RTypeClientSolo/Systems/HUD.hpp File Reference

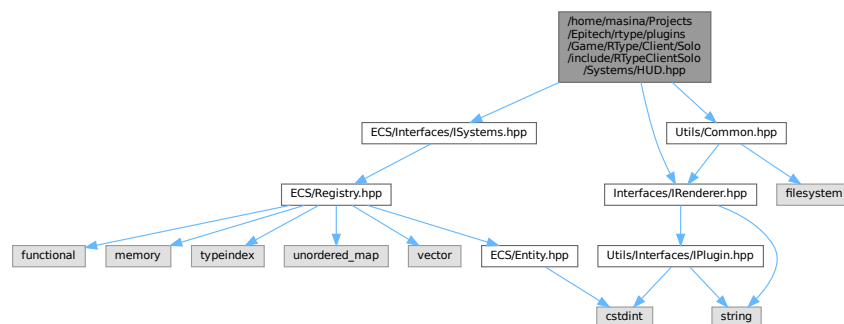
HUD System for managing score display and UI elements.

#include "ECS/Interfaces/ISystems.hpp"

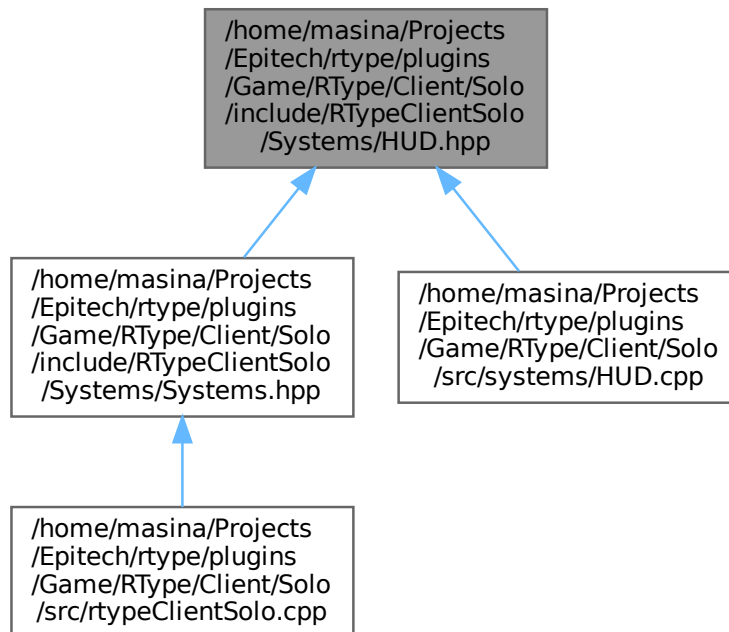
#include "Interfaces/IRenderer.hpp"

#include "Utils/Common.hpp"

Include dependency graph for HUD.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [gme::HUDSystem](#)

Namespaces

- namespace [gme](#)

19.231.1 Detailed Description

HUD System for managing score display and UI elements.

Definition in file [HUD.hpp](#).

19.232 HUD.hpp

[Go to the documentation of this file.](#)

```

00001 ///  

00002 ///  

00003 ///  

00004 ///  

00005 ///  

00006 ///  

00007 #pragma once  

00008  

00009 #include "ECS/Interfaces/ISystems.hpp"  

00010 #include "Interfaces/IRenderer.hpp"  

00011 #include "Utils/Common.hpp"  

00012  

00013 namespace gme  

00014 {  

00015  

00016     class HUDSystem final : public ecs::ASystem  

00017     {  

00018     public:  

00019         explicit HUDSystem(const std::shared_ptr<eng::IRenderer> &renderer, ecs::Registry &registry)
  
```

```

00020         : m_renderer(renderer), m_registry(registry)
00021     {
00022         createScoreHUD(m_registry, 10.0F, 10.0F);
00023     }
00024     ~HUDSystem() override = default;
00025
00026     HUDSystem(const HUDSystem &) = delete;
00027     HUDSystem &operator=(const HUDSystem &) = delete;
00028     HUDSystem(HUDSystem &&) = delete;
00029     HUDSystem &operator=(HUDSystem &&) = delete;
00030
00031     void update(ecs::Registry &registry, float /* dt */) override;
00032
00033 private:
00034     void createScoreHUD(ecs::Registry &registry, float x, float y);
00035     void updateScore(ecs::Registry &registry, int newScore) const;
00036
00037     const std::shared_ptr<eng::IRenderer> &m_renderer;
00038     ecs::Registry &m_registry;
00039     ecs::Entity m_scoreBgEntity{};
00040     std::vector<ecs::Entity> m_scoreDigitEntities;
00041 }; // class HUDSystem
00042
00043 } // namespace gme

```

19.233 /home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Solo/include/RTypeClientSolo/Systems/Spawn.hpp File Reference

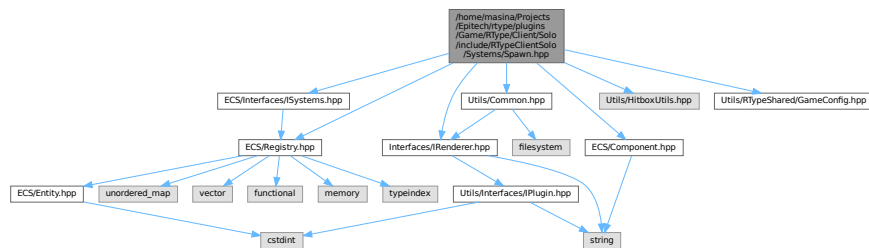
This file contains the spawn system definitions.

```

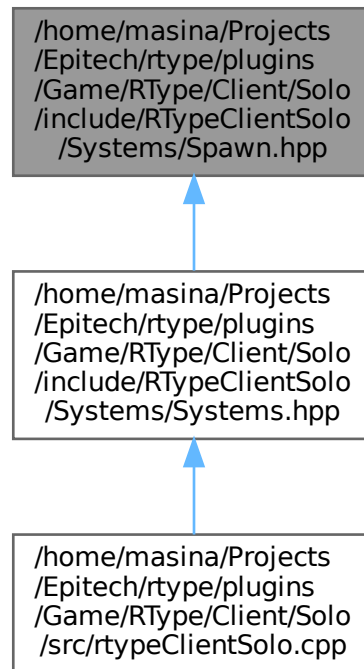
#include "ECS/Component.hpp"
#include "ECS/Interfaces/ISystems.hpp"
#include "ECS/Registry.hpp"
#include "Interfaces/IRenderer.hpp"
#include "Utils/Common.hpp"
#include "Utils/HitboxUtils.hpp"
#include "Utils/RTypeShared/GameConfig.hpp"

```

Include dependency graph for Spawn.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [gme::SpawnSystem](#)

Namespaces

- namespace [gme](#)

19.233.1 Detailed Description

This file contains the spawn system definitions.

Definition in file [Spawn.hpp](#).

19.234 Spawn.hpp

[Go to the documentation of this file.](#)

```

00001 ///
00002 /// @file Spawn.hpp
00003 /// @brief This file contains the spawn system definitions
00004 /// @namespace gme
00005 ///
00006
00007 #pragma once
00008
00009 #include "ECS/Component.hpp"
00010 #include "ECS/Interfaces/ISystems.hpp"
00011 #include "ECS/Registry.hpp"
00012 #include "Interfaces/IRenderer.hpp"
00013 #include "Utils/Common.hpp"
00014 #include "Utils/HitboxUtils.hpp"
00015 #include "Utils/RTypeShared/GameConfig.hpp"
00016
00017 namespace gme

```



```

00018 {
00019
00020     class SpawnSystem final : public ecs::ASystem
00021     {
00022     public:
00023         explicit SpawnSystem(const std::shared_ptr<eng::IRenderer> &renderer) : m_renderer(renderer) {}
00024         ~SpawnSystem() override = default;
00025
00026         SpawnSystem(const SpawnSystem &) = delete;
00027         SpawnSystem &operator=(const SpawnSystem &) = delete;
00028         SpawnSystem(SpawnSystem &&) = delete;
00029         SpawnSystem &operator=(SpawnSystem &&) = delete;
00030
00031         bool isEnabled() override { return true; }
00032         void setEnable(const bool enable) override { (void)enable; }
00033
00034         void update(ecs::Registry &registry, float dt) override
00035         {
00036             m_enemySpawnTimer += dt;
00037             m_waveTimer += dt;
00038
00039             if (m_enemySpawnTimer >= utl::GameConfig::Enemy::Easy::SPAWN_RATE)
00040             {
00041                 spawnEnemy(registry);
00042                 m_enemySpawnTimer = 0.0f;
00043             }
00044
00045             if (m_waveTimer >= 10.0f)
00046             {
00047                 spawnWave(registry);
00048                 m_waveTimer = 0.0f;
00049             }
00050         }
00051
00052     private:
00053         const std::shared_ptr<eng::IRenderer> &m_renderer;
00054         float m_enemySpawnTimer = 0.0f;
00055         float m_waveTimer = 0.0f;
00056
00057         static void spawnEnemy(ecs::Registry &registry)
00058         {
00059             float x = utl::GameConfig::Screen::SPAWN_X;
00060             float y =
00061                 utl::GameConfig::Screen::MIN_Y +
00062                 (std::rand() % static_cast<int>(utl::GameConfig::Screen::MAX_Y - utl::GameConfig::Screen::MIN_Y));
00063
00064             auto [offsetX, offsetY] = utl::calculateHitboxOffsetsRelative(
00065                 x, y, utl::GameConfig::Enemy::Easy::SPRITE_WIDTH, utl::GameConfig::Enemy::Easy::SPRITE_HEIGHT,
00066                 utl::GameConfig::Enemy::Easy::SCALE);
00067
00068             registry.createEntity()
00069                 .with<ecs::Transform>("enemy_transform", x, y, 0.0f)
00070                 .with<ecs::Velocity>("enemy_velocity", -utl::GameConfig::Enemy::Easy::SPEED, 0.0f)
00071                 .with<ecs::Rect>("enemy_rect", 0.0f, 0.0f,
00072                     static_cast<int>(utl::GameConfig::Enemy::Easy::SPRITE_WIDTH),
00073                     static_cast<int>(utl::GameConfig::Enemy::Easy::SPRITE_HEIGHT))
00074                 .with<ecs::Scale>("enemy_scale", utl::GameConfig::Enemy::Easy::SCALE,
00075                     utl::GameConfig::Enemy::Easy::SCALE)
00076                 .with<ecs::Texture>("enemy_texture", utl::Path::Texture::TEXTURE_ENEMY_EASY)
00077                 .with<ecs::Animation>("enemy_animation", 0, utl::GameConfig::Enemy::Easy::ANIMATION_FRAMES,
00078                     utl::GameConfig::Enemy::Easy::ANIMATION_DURATION, 0.0f,
00079                     static_cast<int>(utl::GameConfig::Enemy::Easy::SPRITE_WIDTH),
00080                     static_cast<int>(utl::GameConfig::Enemy::Easy::SPRITE_HEIGHT),
00081                     static_cast<int>(utl::GameConfig::Enemy::Easy::FRAMES_PER_ROW))
00082                 .with<ecs::Enemy>("enemy", utl::GameConfig::Enemy::Easy::HEALTH,
00083                     utl::GameConfig::Enemy::Easy::HEALTH, utl::GameConfig::Enemy::Easy::DAMAGE,
00084                     utl::GameConfig::Enemy::Easy::SPEED, 0.0f,
00085                     utl::GameConfig::Enemy::Easy::SHOOT_COOLDOWN)
00086                 .with<ecs::Hitbox>("enemy_hitbox", utl::GameConfig::Hitbox::ENEMY_RADIUS, offsetX, offsetY)
00087                 .build();
00088         }
00089
00090         static void spawnWave(ecs::Registry &registry)
00091         {
00092             const int waveSize = 5 + (std::rand() % 4);
00093
00094             for (int i = 0; i < waveSize; ++i)
00095             {
00096                 float x = utl::GameConfig::Screen::SPAWN_X + (i * 100.0f);
00097                 float y = utl::GameConfig::Screen::MIN_Y +
00098                     (std::rand() %
00099                     static_cast<int>(utl::GameConfig::Screen::MAX_Y - utl::GameConfig::Screen::MIN_Y));
00100                 auto [offsetX, offsetY] = utl::calculateHitboxOffsetsRelative(
00101                     x, y, utl::GameConfig::Enemy::Easy::SPRITE_WIDTH,
00102                     utl::GameConfig::Enemy::Easy::SPRITE_HEIGHT,
00103                     utl::GameConfig::Enemy::Easy::SCALE);

```

```

00104 registry.createEntity()
00105 .with<ecs::Transform>("enemy_wave_transform", x, y, 0.0f)
00106 .with<ecs::Velocity>("enemy_wave_velocity", -utl::GameConfig::Enemy::Easy::SPEED, 0.0f)
00107 .with<ecs::Rect>("enemy_wave_rect", 0.0f, 0.0f,
00108     static_cast<int>(utl::GameConfig::Enemy::Easy::SPRITE_WIDTH),
00109     static_cast<int>(utl::GameConfig::Enemy::Easy::SPRITE_HEIGHT))
00110 .with<ecs::Scale>("enemy_wave_scale", utl::GameConfig::Enemy::Easy::SCALE,
00111     utl::GameConfig::Enemy::Easy::SCALE)
00112 .with<ecs::Texture>("enemy_wave_texture", utl::Path::Texture::TEXTURE_ENEMY_EASY)
00113 .with<ecs::Animation>("enemy_wave_animation", 0,
00114     static_cast<int>(utl::GameConfig::Enemy::Easy::ANIMATION_FRAMES),
00115     utl::GameConfig::Enemy::Easy::ANIMATION_DURATION, 0.0f,
00116     static_cast<int>(utl::GameConfig::Enemy::Easy::SPRITE_WIDTH),
00117     static_cast<int>(utl::GameConfig::Enemy::Easy::SPRITE_HEIGHT),
00118     static_cast<int>(utl::GameConfig::Enemy::Easy::FRAMES_PER_ROW))
00119 .with<ecs::Enemy>("enemy_wave", utl::GameConfig::Enemy::Easy::HEALTH,
00120     utl::GameConfig::Enemy::Easy::HEALTH, utl::GameConfig::Enemy::Easy::DAMAGE,
00121     utl::GameConfig::Enemy::Easy::SPEED, 0.0f,
00122     utl::GameConfig::Enemy::Easy::SHOOT_COOLDOWN)
00123 .with<ecs::Hitbox>("enemy_wave_hitbox", utl::GameConfig::Hitbox::ENEMY_RADIUS, offsetX, offsetY)
00124 .build();
00125 }
00126 }
00127 }; // class SpawnSystem
00128 } // namespace gme

```

19.235 /home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Solo/src/managers/projectileManager.cpp File Reference

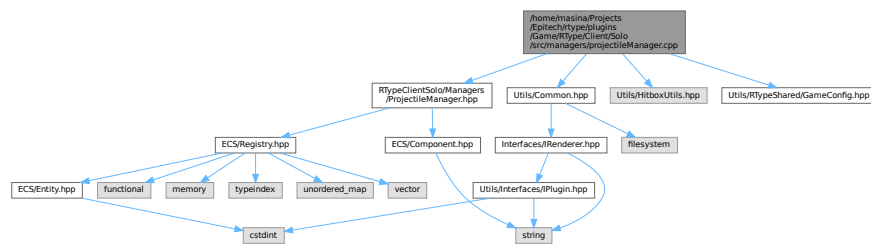
#include "RTypeClientSolo/Managers/ProjectileManager.hpp"

#include "Utils/Common.hpp"

#include "Utils/HitboxUtils.hpp"

#include "Utils/RTypeShared/GameConfig.hpp"

Include dependency graph for projectileManager.cpp:



Namespaces

- namespace [gme](#)

19.236 projectileManager.cpp

[Go to the documentation of this file.](#)

```

00001 #include "RTypeClientSolo/Managers/ProjectileManager.hpp"
00002 #include "Utils/Common.hpp"
00003 #include "Utils/HitboxUtils.hpp"
00004 #include "Utils/RTypeShared/GameConfig.hpp"
00005
00006 namespace gme
00007 {
00008     ecs::Entity ProjectileManager::createBasicProjectile(ecs::Registry &registry, const float x, const float y,
00009                                                         const float velocityX, const float velocityY)
00010     {
00011         return createProjectile(registry, ecs::Projectile::BASIC, x, y, velocityX, velocityY);
00012     }
00013
00014     ecs::Entity ProjectileManager::createSuperchargedProjectile(ecs::Registry &registry, const float x, const float y,
00015                                                         const float velocityX, const float velocityY)
00016     {
00017         return createProjectile(registry, ecs::Projectile::SUPERCHARGED, x, y, velocityX, velocityY);
00018     }

```

```

00019
00020     ecs::Entity ProjectileManager::createProjectile(ecs::Registry &registry, ecs::Projectile::Type type, float x,
00021                                                    float y, float velocityX, float velocityY)
00022     {
00023         if (type == ecs::Projectile::BASIC)
00024         {
00025             auto [offsetX, offsetY] = utl::calculateHitboxOffsets(utl::GameConfig::Projectile::Basic::SPRITE_WIDTH,
00026                                                                utl::GameConfig::Projectile::Basic::SPRITE_HEIGHT,
00027                                                                utl::GameConfig::Projectile::Basic::SCALE);
00028
00029             return registry.createEntity()
00030                 .with<ecs::Transform>("projectile_transform", x, y, 0.F)
00031                 .with<ecs::Velocity>("projectile_velocity", velocityX, velocityY)
00032                 .with<ecs::Rect>("projectile_rect", 0.F, 0.F,
00033                                static_cast<int>(utl::GameConfig::Projectile::Basic::SPRITE_WIDTH),
00034                                static_cast<int>(utl::GameConfig::Projectile::Basic::SPRITE_HEIGHT))
00035                 .with<ecs::Scale>("projectile_scale", utl::GameConfig::Projectile::Basic::SCALE,
00036                                utl::GameConfig::Projectile::Basic::SCALE)
00037                 .with<ecs::Texture>("projectile_texture", utl::Path::Texture::TEXTURE_SHOOT)
00038                 .with<ecs::Projectile>("projectile", type, utl::GameConfig::Projectile::Basic::DAMAGE,
00039                                utl::GameConfig::Projectile::Basic::LIFETIME, 0.0f, 1)
00040                 .with<ecs::Hitbox>("projectile_hitbox", utl::GameConfig::Hitbox::PROJECTILE_BASIC_RADIUS, offsetX,
00041                                offsetY)
00042                 .build();
00043         }
00044
00045         auto [offsetX, offsetY] = utl::calculateHitboxOffsets(utl::GameConfig::Projectile::Supercharged::SPRITE_WIDTH,
00046                                                                utl::GameConfig::Projectile::Supercharged::SPRITE_HEIGHT,
00047                                                                utl::GameConfig::Projectile::Supercharged::SCALE);
00048
00049         return registry.createEntity()
00050             .with<ecs::Transform>("projectile_transform", x, y, 0.F)
00051             .with<ecs::Velocity>("projectile_velocity", velocityX, velocityY)
00052             .with<ecs::Rect>("projectile_rect", 0.F, 0.F,
00053                            static_cast<int>(utl::GameConfig::Projectile::Supercharged::SPRITE_WIDTH),
00054                            static_cast<int>(utl::GameConfig::Projectile::Supercharged::SPRITE_HEIGHT))
00055             .with<ecs::Scale>("projectile_scale", utl::GameConfig::Projectile::Supercharged::SCALE,
00056                            utl::GameConfig::Projectile::Supercharged::SCALE)
00057             .with<ecs::Texture>("projectile_texture", utl::Path::Texture::TEXTURE_SHOOT_CHARGED)
00058             .with<ecs::Projectile>("projectile", type, utl::GameConfig::Projectile::Supercharged::DAMAGE,
00059                            utl::GameConfig::Projectile::Supercharged::LIFETIME, 0.0f, 3)
00060             .with<ecs::Animation>("projectile_animation", 0,
00061                            utl::GameConfig::Projectile::Supercharged::ANIMATION_FRAMES,
00062                            utl::GameConfig::Projectile::Supercharged::ANIMATION_DURATION, 0.0f,
00063                            static_cast<int>(utl::GameConfig::Projectile::Supercharged::SPRITE_WIDTH),
00064                            static_cast<int>(utl::GameConfig::Projectile::Supercharged::SPRITE_HEIGHT),
00065                            utl::GameConfig::Projectile::Supercharged::ANIMATION_FRAMES)
00066             .with<ecs::Hitbox>("projectile_hitbox", utl::GameConfig::Hitbox::PROJECTILE_SUPERCHARGED_RADIUS,
00067                            offsetX, offsetY)
00068             .build();
00069     }
00070 } // namespace gme

```

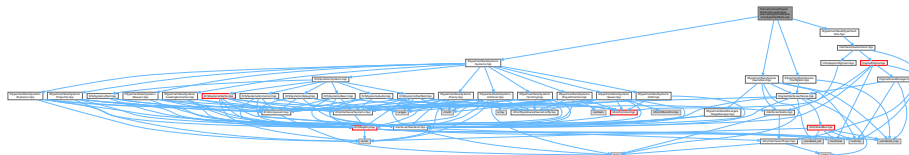
19.237 /home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Solo/src/rtypeClientSolo.cpp File Reference

```

#include "RTypeClientSolo/RTypeClientSolo.hpp"
#include "RTypeClientSolo/Scenes/ConfigSolo.hpp"
#include "RTypeClientSolo/Scenes/GameSolo.hpp"
#include "RTypeClientSolo/Systems/Systems.hpp"

```

Include dependency graph for rtypeClientSolo.cpp:



19.238 rtypeClientSolo.cpp

[Go to the documentation of this file.](#)

```
00001 #include "RTypeClientSolo/RTypeClientSolo.hpp"
```

```

00002 #include "RTypeClientSolo/Scenes/ConfigSolo.hpp"
00003 #include "RTypeClientSolo/Scenes/GameSolo.hpp"
00004 #include "RTypeClientSolo/Systems/Systems.hpp"
00005
00006 void gme::RTypeClientSolo::setupScenes(bool &showDebug, eng::id menuSceneId)
00007 {
00008     auto configSoloId = m_engine->getSceneManager()->generateNextId();
00009     auto configSolo = std::make_unique<ConfigSolo>(configSoloId, m_engine->getRenderrer());
00010     configSolo->addSystem(std::make_unique<ecs::AudioSystem>(m_engine->getAudio(), m_appConfig->audioVolume,
00011         configSolo->getRegistry(), configSolo->playMusic()));
00012     configSolo->addSystem(std::make_unique<ecs::DebugSystem>(m_engine->getRenderrer(), showDebug));
00013     configSolo->addSystem(std::make_unique<ecs::SpriteSystem>(m_engine->getRenderrer()));
00014     configSolo->addSystem(std::make_unique<ecs::TextSystem>(m_engine->getRenderrer()));
00015     configSolo->addSystem(std::make_unique<ecs::StarfieldSystem>(m_engine->getRenderrer(),
00016         configSolo->getRegistry()));
00017     auto gameSoloId = m_engine->getSceneManager()->generateNextId();
00018     auto gameSolo = std::make_unique<GameSolo>(gameSoloId, m_engine->getRenderrer(), m_engine->getAudio(),
00019         m_appConfig->skinIndex, showDebug);
00020     gameSolo->addSystem(std::make_unique<ecs::AudioSystem>(m_engine->getAudio(), m_appConfig->audioVolume,
00021         gameSolo->getRegistry(), gameSolo->playMusic()));
00022     gameSolo->addSystem(std::make_unique<ecs::SpriteSystem>(m_engine->getRenderrer()));
00023     gameSolo->addSystem(std::make_unique<ecs::TextSystem>(m_engine->getRenderrer()));
00024     gameSolo->addSystem(std::make_unique<ecs::AnimationSystem>(m_engine->getRenderrer()));
00025     gameSolo->addSystem(std::make_unique<ecs::BeamSystem>(m_engine->getRenderrer()));
00026     gameSolo->addSystem(std::make_unique<CollisionSystem>(m_engine->getRenderrer(), showDebug));
00027     gameSolo->addSystem(std::make_unique<EnemySystem>(m_engine->getRenderrer()));
00028     gameSolo->addSystem(std::make_unique<ExplosionSystem>(m_engine->getRenderrer()));
00029     gameSolo->addSystem(std::make_unique<LoadingAnimationSystem>(m_engine->getRenderrer()));
00030     gameSolo->addSystem(std::make_unique<PlayerDirectionSystem>(m_appConfig->skinIndex));
00031     gameSolo->addSystem(std::make_unique<ProjectileSystem>(m_engine->getRenderrer()));
00032     gameSolo->addSystem(std::make_unique<ScrollingSystem>(m_engine->getRenderrer()));
00033     gameSolo->addSystem(std::make_unique<WeaponSystem>(m_engine->getRenderrer()));
00034     gameSolo->addSystem(std::make_unique<SpawnSystem>(m_engine->getRenderrer()));
00035     gameSolo->addSystem(std::make_unique<ecs::DebugSystem>(m_engine->getRenderrer(), showDebug));
00036     gameSolo->addSystem(std::make_unique<HUDSystem>(m_engine->getRenderrer(), gameSolo->getRegistry()));
00037     gameSolo->addSystem(std::make_unique<ecs::StarfieldSystem>(m_engine->getRenderrer(), gameSolo->getRegistry()));
00038     m_mainSceneId = configSoloId;
00039     configSolo->onOptionSelected = [this, gameSoloId, menuSceneId](const std::string &option)
00040     {
00041         if (option == "Level easy")
00042         {
00043             m_engine->getSceneManager()->switchToScene(gameSoloId);
00044         }
00045         else if (option == "Level medium")
00046         {
00047             m_engine->getSceneManager()->switchToScene(gameSoloId);
00048         }
00049         else if (option == "Go back to menu")
00050         {
00051             m_engine->getSceneManager()->switchToScene(menuSceneId);
00052         }
00053     };
00054     m_engine->getSceneManager()->addScene(std::move(configSolo));
00055     m_engine->getSceneManager()->addScene(std::move(gameSolo));
00056 }
00057
00058 void gme::RTypeClientSolo::update(float deltaTime, unsigned int width, unsigned int height) {}

```

19.239 /home/masina/Projects/Epitech/rtype/plugins/Game/RType/↵ Client/Solo/src/scenes/configSolo.cpp File Reference

```

#include <cmath>
#include "ECS/Component.hpp"
#include "RTypeClientSolo/Scenes/ConfigSolo.hpp"
#include "Utils/Common.hpp"

```

Go to the documentation of this file.

Generated by Doxygen

```

00066         .with<ecs::Color>("color_title", utl::Config::Color::CYAN_ELECTRIC.r,
utl::Config::Color::CYAN_ELECTRIC.g,
00067         utl::Config::Color::CYAN_ELECTRIC.b, utl::Config::Color::CYAN_ELECTRIC.a)
00068         .with<ecs::Text>("id", std::string("SOLO"), 80U)
00069         .build();
00070
00071     for (size_t i = 0; i < m_menuOptions.size(); ++i)
00072     {
00073         registry.createEntity()
00074         .with<ecs::Font>("main_font", utl::Path::Font::FONTS_RTYPE)
00075         .with<ecs::Transform>("transform_menu", 100.F, 200.F + i * 60.F, 0.F)
00076         .with<ecs::Color>("color_menu", utl::Config::Color::WHITE.r, utl::Config::Color::WHITE.g,
00077         utl::Config::Color::WHITE.b, utl::Config::Color::WHITE.a)
00078         .with<ecs::Text>("menu_" + m_menuOptions[i], m_menuOptions[i], 40U)
00079         .build();
00080     }
00081     m_selectedIndex = 2;
00082 }
00083
00084 void gme::ConfigSolo::update(const float dt, const eng::WindowSize & /*size*/)
00085 {
00086     auto &reg = getRegistry();
00087
00088     auto &colors = reg.getAll<ecs::Color>();
00089     auto &texts = reg.getAll<ecs::Text>();
00090
00091     m_animationTime += dt;
00092     m_titlePulseTime += dt;
00093
00094     if (auto *titleColor = reg.getComponent<ecs::Color>(m_titleEntity))
00095     {
00096         const float pulse = (std::sin(m_titlePulseTime * 1.2f) + 1.0f) * 0.5f;
00097         titleColor->r = static_cast<uint8_t>(utl::Config::Color::CYAN_ELECTRIC.r * (0.8f + pulse * 0.2f));
00098         titleColor->g = static_cast<uint8_t>(utl::Config::Color::CYAN_ELECTRIC.g * (0.8f + pulse * 0.2f));
00099         titleColor->b = static_cast<uint8_t>(utl::Config::Color::CYAN_ELECTRIC.b * (0.9f + pulse * 0.1f));
00100     }
00101     if (auto *titleTransform = reg.getComponent<ecs::Transform>(m_titleEntity))
00102     {
00103         titleTransform->y = 60.0f + std::sin(m_titlePulseTime * 0.8f) * 2.0f;
00104     }
00105     int i = 0;
00106     for (auto &[entity, text] : texts)
00107     {
00108         if (text.content == "Level easy" || text.content == "Level medium" || text.content == "Go back to menu")
00109         {
00110             auto &color = colors.at(entity);
00111
00112             if (i == m_selectedIndex)
00113             {
00114                 const float glowIntensity = std::sin(m_animationTime * 2.5f);
00115                 color.r = 0U;
00116                 color.g = static_cast<unsigned char>(191U + (glowIntensity * 50));
00117                 color.b = 255U;
00118             }
00119             else
00120             {
00121                 color.r = utl::Config::Color::GRAY_BLUE_SUBTLE.r;
00122                 color.g = utl::Config::Color::GRAY_BLUE_SUBTLE.g;
00123                 color.b = utl::Config::Color::GRAY_BLUE_SUBTLE.b;
00124             }
00125
00126             i++;
00127         }
00128     }
00129     if (auto *titleColor = reg.getComponent<ecs::Color>(m_titleEntity))
00130     {
00131         const float pulsation = std::sin(m_titlePulseTime * 2.0f) * 0.4f + 0.6f;
00132         titleColor->r = static_cast<unsigned char>(utl::Config::Color::CYAN_ELECTRIC.r * pulsation);
00133         titleColor->g = static_cast<unsigned char>(utl::Config::Color::CYAN_ELECTRIC.g * pulsation);
00134         titleColor->b = static_cast<unsigned char>(utl::Config::Color::CYAN_ELECTRIC.b * pulsation);
00135     }
00136     if (auto *fpsText = reg.getComponent<ecs::Text>(m_fpsEntity))
00137     {
00138         fpsText->content = "FPS: " + std::to_string(static_cast<int>(1 / dt));
00139     }
00140 }
00141
00142 void gme::ConfigSolo::event(const eng::Event &event)
00143 {
00144     switch (event.type)
00145     {
00146     case eng::EventType::KeyPressed:
00147         if (event.key == eng::Key::Up)
00148         {
00149             m_playMusic = true;
00150             if (m_selectedIndex == 2)
00151                 {

```

```

00152         m_selectedIndex = 0;
00153     }
00154     else
00155     {
00156         m_selectedIndex++;
00157     }
00158 }
00159 else if (event.key == eng::Key::Down)
00160 {
00161     m_playMusic = true;
00162     if (m_selectedIndex == 0)
00163     {
00164         m_selectedIndex = 2;
00165     }
00166     else
00167     {
00168         m_selectedIndex--;
00169     }
00170 }
00171 else if (event.key == eng::Key::Enter)
00172 {
00173     const std::string &selectedOption =
00174         m_menuOptions[static_cast<int>(m_menuOptions.size()) - 1 - m_selectedIndex];
00175     if (onOptionSelected)
00176     {
00177         onOptionSelected(selectedOption);
00178     }
00179 }
00180 break;
00181
00182 case eng::EventType::KeyReleased:
00183     if (event.key == eng::Key::Up)
00184     {
00185         m_keysPressed[eng::Key::Up] = false;
00186     }
00187     if (event.key == eng::Key::Down)
00188     {
00189         m_keysPressed[eng::Key::Down] = false;
00190     }
00191     if (event.key == eng::Key::Left)
00192     {
00193         m_keysPressed[eng::Key::Left] = false;
00194     }
00195     if (event.key == eng::Key::Right)
00196     {
00197         m_keysPressed[eng::Key::Right] = false;
00198     }
00199     if (event.key == eng::Key::Space)
00200     {
00201         m_keysPressed[eng::Key::Space] = false;
00202     }
00203     break;
00204
00205 default:
00206     break;
00207 }
00208 }
```

19.241 /home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Solo/src/scenes/gameSolo.cpp File Reference ↵

```

#include "RTypeClientSolo/Scenes/GameSolo.hpp"
#include "ECS/Component.hpp"
#include "Interfaces/IAudio.hpp"
#include "Utils/Common.hpp"
#include "Utils/HitboxUtils.hpp"
#include "Utils/RTypeShared/GameConfig.hpp"
```



```

00067     }
00068
00069     if (transform && textureComp)
00070     {
00071         if (rectComp)
00072         {
00073             renderer->createSprite(textureComp->id + std::to_string(e), textureComp->id, transform->x,
00074                                     transform->y, scale_x, scale_y, static_cast<int>(rectComp->pos_x),
00075                                     static_cast<int>(rectComp->pos_y), rectComp->size_x, rectComp->size_y);
00076         }
00077         else
00078         {
00079             renderer->createSprite(textureComp->id + std::to_string(e), textureComp->id, transform->x,
00080                                     transform->y);
00081         }
00082     }
00083 }
00084 else if (type == typeid(ecs::Audio))
00085 {
00086     if (audioComp)
00087     {
00088         audio->createAudio(audioComp->path, audioComp->volume, audioComp->loop,
00089                             audioComp->id + std::to_string(e));
00090     }
00091 }
00092 });
00093
00094 registry.createEntity().with<ecs::Score>("score", 0).build();
00095
00096 m_playerEntity = createPlayer(registry);
00097 m_stageManager = std::make_unique<StageManager>();
00098
00099 // Preload all common textures to avoid lag spikes during gameplay
00100 preloadCommonTextures();
00101
00102 const auto beginSoundEntity = registry.createEntity()
00103     .with<ecs::Audio>("game_begin", utl::Path::Audio::AUDIO_BEGIN, 10.0f, true, true)
00104     .build();
00105 }
00106
00107 void gme::GameSolo::preloadCommonTextures()
00108 {
00109     // Preload all textures that will be used during gameplay
00110     // This prevents lag when entities are first created
00111
00112     std::vector<std::pair<std::string, std::string>> texturesToLoad = {
00113         {"player", utl::Path::Texture::TEXTURE_PLAYER},
00114         {"stage1_floor", utl::Path::Texture::TEXTURE_STAGE1_FLOOR},
00115         {"stage1_ceiling", utl::Path::Texture::TEXTURE_STAGE1_CEILING},
00116         {"shoot", utl::Path::Texture::TEXTURE_SHOOT},
00117         {"shoot_charged", utl::Path::Texture::TEXTURE_SHOOT_CHARGED}};
00118
00119     for (const auto &[id, path] : texturesToLoad)
00120     {
00121         if (m_loadedTextures.find(id) == m_loadedTextures.end())
00122         {
00123             m_renderer->createTexture(id, path);
00124             m_loadedTextures.insert(id);
00125         }
00126     }
00127 }
00128
00129 void gme::GameSolo::handlePlayerInputs(ecs::Registry &registry, const float dt)
00130 {
00131     auto *playerTransform = registry.getComponent<ecs::Transform>(m_playerEntity);
00132     auto *playerVelocity = registry.getComponent<ecs::Velocity>(m_playerEntity);
00133
00134     if ((playerTransform == nullptr) || (playerVelocity == nullptr)) {
00135         return;
00136     }
00137
00138     constexpr float diagonal_speed =
00139         utl::GameConfig::Player::SPEED * utl::GameConfig::Player::DIAGONAL_SPEED_MULTIPLIER;
00140
00141     playerVelocity->x = 0.0f;
00142     playerVelocity->y = 0.0f;
00143
00144     const bool up = m_keysPressed[eng::Key::Up];
00145     const bool down = m_keysPressed[eng::Key::Down];
00146     const bool left = m_keysPressed[eng::Key::Left];
00147     if (const bool right = m_keysPressed[eng::Key::Right]; up && right)
00148     {
00149         playerVelocity->x = diagonal_speed;
00150         playerVelocity->y = -diagonal_speed;
00151     }
00152     else if (up && left)
00153     {

```

```

00154     playerVelocity->x = -diagonal_speed;
00155     playerVelocity->y = -diagonal_speed;
00156 }
00157 else if (down && right)
00158 {
00159     playerVelocity->x = diagonal_speed;
00160     playerVelocity->y = diagonal_speed;
00161 }
00162 else if (down && left)
00163 {
00164     playerVelocity->x = -diagonal_speed;
00165     playerVelocity->y = diagonal_speed;
00166 }
00167 else
00168 {
00169     if (up)
00170         playerVelocity->y = -utl::GameConfig::Player::SPEED;
00171     if (down)
00172         playerVelocity->y = utl::GameConfig::Player::SPEED;
00173     if (left)
00174         playerVelocity->x = -utl::GameConfig::Player::SPEED;
00175     if (right)
00176         playerVelocity->x = utl::GameConfig::Player::SPEED;
00177 }
00178
00179 playerTransform->x += playerVelocity->x * dt;
00180 playerTransform->y += playerVelocity->y * dt;
00181 playerTransform->x = std::max(playerTransform->x, 0.F);
00182 playerTransform->y = std::max(playerTransform->y, 0.F);
00183
00184 auto [width, height] = m_renderer->getWindowSize();
00185 const float maxX =
00186     static_cast<float>(width) - utl::GameConfig::Player::SPRITE_WIDTH * utl::GameConfig::Player::SCALE;
00187 const float maxY =
00188     static_cast<float>(height) - utl::GameConfig::Player::SPRITE_HEIGHT * utl::GameConfig::Player::SCALE;
00189
00190 playerTransform->x = std::min(playerTransform->x, maxX);
00191 playerTransform->y = std::min(playerTransform->y, maxY);
00192 }
00193
00194 void gme::GameSolo::update(const float dt, const eng::WindowSize &size)
00195 {
00196     auto &reg = getRegistry();
00197     handlePlayerInputs(reg, dt);
00198     m_stageManager->update(reg, dt, size);
00199 }
00200
00201 void gme::GameSolo::event(const eng::Event &event)
00202 {
00203     switch (event.type)
00204     {
00205     case eng::EventType::KeyPressed:
00206         if (event.key == eng::Key::Down)
00207         {
00208             m_keysPressed[eng::Key::Down] = true;
00209         }
00210         if (event.key == eng::Key::Up)
00211         {
00212             m_keysPressed[eng::Key::Up] = true;
00213         }
00214         if (event.key == eng::Key::Left)
00215         {
00216             m_keysPressed[eng::Key::Left] = true;
00217         }
00218         if (event.key == eng::Key::Right)
00219         {
00220             m_keysPressed[eng::Key::Right] = true;
00221         }
00222         break;
00223
00224     case eng::EventType::KeyReleased:
00225         if (event.key == eng::Key::Down)
00226         {
00227             m_keysPressed[eng::Key::Down] = false;
00228         }
00229         if (event.key == eng::Key::Up)
00230         {
00231             m_keysPressed[eng::Key::Up] = false;
00232         }
00233         if (event.key == eng::Key::Left)
00234         {
00235             m_keysPressed[eng::Key::Left] = false;
00236         }
00237         if (event.key == eng::Key::Right)
00238         {
00239             m_keysPressed[eng::Key::Right] = false;
00240         }
00241     }

```

```

00241     break;
00242
00243     default:
00244         break;
00245     }
00246 }
00247
00248 void gme::GameSolo::updatePlayerSkin()
00249 {
00250     auto &registry = getRegistry();
00251
00252     if (auto *playerRect = registry.getComponent<ecs::Rect>(m_playerEntity); playerRect != nullptr)
00253     {
00254         const float skinPosY = m_skinIndex * utl::GameConfig::Player::SPRITE_HEIGHT;
00255         playerRect->pos_y = skinPosY;
00256     }
00257 }
00258
00259 ecs::Entity gme::GameSolo::createPlayer(ecs::Registry &registry)
00260 {
00261     auto [offsetX, offsetY] = utl::calculateHitboxOffsets(
00262         utl::GameConfig::Player::SPRITE_WIDTH, utl::GameConfig::Player::SPRITE_HEIGHT,
00263         utl::GameConfig::Player::SCALE);
00264
00265     return registry.createEntity()
00266         .with<ecs::Transform>("player_transform", 200.0F, 100.0F, 0.F)
00267         .with<ecs::Velocity>("player_velocity", 0.F, 0.F)
00268         .with<ecs::Rect>("player_rect", 0.F, 0.F, static_cast<int>(utl::GameConfig::Player::SPRITE_WIDTH),
00269             static_cast<int>(utl::GameConfig::Player::SPRITE_HEIGHT))
00270         .with<ecs::Scale>("player_scale", utl::GameConfig::Player::SCALE, utl::GameConfig::Player::SCALE)
00271         .with<ecs::Texture>("player_texture", utl::Path::Texture::TEXTURE_PLAYER)
00272         .with<ecs::Player>("player", true)
00273         .with<ecs::BeamCharge>("beam_charge", 0.0f, utl::GameConfig::Beam::MAX_CHARGE)
00274         .with<ecs::Hitbox>("player_hitbox", utl::GameConfig::Hitbox::PLAYER_RADIUS, offsetX, offsetY)
00275         .build();
00276 }

```

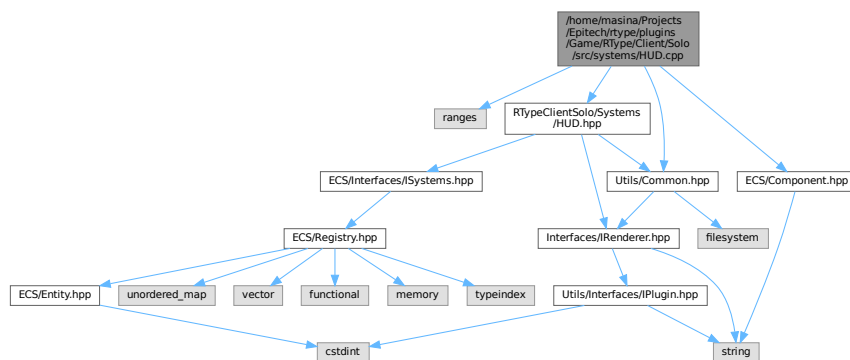
19.243 /home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Solo/src/systems/HUD.cpp File Reference

```

#include <ranges>
#include "ECS/Component.hpp"
#include "RTypeClientSolo/Systems/HUD.hpp"
#include "Utils/Common.hpp"

```

Include dependency graph for HUD.cpp:



19.244 HUD.cpp

[Go to the documentation of this file.](#)

```

00001 #include <ranges>
00002
00003 #include "ECS/Component.hpp"
00004 #include "RTypeClientSolo/Systems/HUD.hpp"
00005 #include "Utils/Common.hpp"
00006

```

```

00007 void gme::HUDSystem::createScoreHUD(ecs::Registry &registry, float x, float y)
00008 {
00009     m_scoreBgEntity = registry.createEntity()
00010         .with<ecs::Transform>("score_bg_transform", x, y, 0.0f)
00011         .with<ecs::Scale>("score_bg_scale", 3.0f, 2.0f)
00012         .with<ecs::Texture>("score_bg_texture", utl::Path::Texture::TEXTURE_SCORE_COUNTER_BG)
00013         .with<ecs::Layer>("score_bg_layer", 1)
00014         .build();
00015
00016     m_scoreDigitEntities.clear();
00017     for (int i = 0; i < 6; ++i)
00018     {
00019         auto digitEntity = registry.createEntity()
00020             .with<ecs::Transform>("score_digit_transform", x + 140.0f + i * 20.0f, y + 4.0f, 0.0f)
00021             .with<ecs::Scale>("score_digit_scale", 1.0f, 1.0f)
00022             .with<ecs::Texture>("score_digit_texture", utl::Path::Texture::TEXTURE_SCORE_DIGIT_0)
00023             .with<ecs::Layer>("score_digit_layer", 2)
00024             .build();
00025         m_scoreDigitEntities.push_back(digitEntity);
00026     }
00027 }
00028
00029 void gme::HUDSystem::updateScore(ecs::Registry &registry, int newScore) const
00030 {
00031     int digits[6];
00032     for (int i = 5; i >= 0; --i)
00033     {
00034         digits[i] = newScore % 10;
00035         newScore /= 10;
00036     }
00037
00038     static const char *digitPaths[10] = {
00039         utl::Path::Texture::TEXTURE_SCORE_DIGIT_0, utl::Path::Texture::TEXTURE_SCORE_DIGIT_1,
00040         utl::Path::Texture::TEXTURE_SCORE_DIGIT_2, utl::Path::Texture::TEXTURE_SCORE_DIGIT_3,
00041         utl::Path::Texture::TEXTURE_SCORE_DIGIT_4, utl::Path::Texture::TEXTURE_SCORE_DIGIT_5,
00042         utl::Path::Texture::TEXTURE_SCORE_DIGIT_6, utl::Path::Texture::TEXTURE_SCORE_DIGIT_7,
00043         utl::Path::Texture::TEXTURE_SCORE_DIGIT_8, utl::Path::Texture::TEXTURE_SCORE_DIGIT_9};
00044
00045     for (int i = 0; i < 6 && i < m_scoreDigitEntities.size(); ++i)
00046     {
00047         if (auto *digitTexture = registry.getComponent<ecs::Texture>(m_scoreDigitEntities[i]))
00048         {
00049             digitTexture->path = digitPaths[digits[i]];
00050         }
00051
00052         auto *digitTransform = registry.getComponent<ecs::Transform>(m_scoreDigitEntities[i]);
00053         if (auto *digitScale = registry.getComponent<ecs::Scale>(m_scoreDigitEntities[i]); (digitTransform != nullptr) &&
            (digitScale != nullptr))
00054         {
00055             digitTransform->x = 10.0f + 130.0f + i * 20.0f;
00056             digitTransform->y = 10.0f + 4.0f;
00057             digitScale->x = 1.5f;
00058             digitScale->y = 1.5f;
00059         }
00060     }
00061 }
00062
00063 void gme::HUDSystem::update(ecs::Registry &registry, float /* dt */)
00064 {
00065     int currentScore = 0;
00066     for (const auto &score : registry.getAll<ecs::Score>() | std::views::values)
00067     {
00068         currentScore = score.value;
00069         break;
00070     }
00071
00072     updateScore(registry, currentScore);
00073 }

```

19.245 /home/masina/Projects/Epitech/rtype/plugins/Game/RType/↵ Server/include/RTypeServer/EntityManager.hpp File Reference

Centralized entity management for RTypeServer.

```

#include "ECS/Component.hpp"
#include "ECS/Entity.hpp"
#include "ECS/Registry.hpp"
#include "Interfaces/Protocol/Protocol.hpp"
#include <memory>
#include <unordered_map>
#include <vector>

```



```

00006 #pragma once
00007
00008 #include "ECS/Component.hpp"
00009 #include "ECS/Entity.hpp"
00010 #include "ECS/Registry.hpp"
00011 #include "Interfaces/Protocol/Protocol.hpp"
00012 #include <memory>
00013 #include <unordered_map>
00014 #include <vector>
00015
00016 namespace gme
00017 {
00018     ///
00019     /// @brief Entity type enumeration for server-side management
00020     ///
00021     enum class ServerEntityType : uint8_t
00022     {
00023         PLAYER = 0,
00024         ENEMY_BASIC = 1,
00025         ENEMY_ADVANCED = 2,
00026         BOSS = 3,
00027         PROJECTILE_PLAYER = 4,
00028         PROJECTILE_ENEMY = 5,
00029         POWERUP = 6
00030     };
00031
00032     ///
00033     /// @brief Entity metadata for tracking
00034     ///
00035     struct EntityMetadata
00036     {
00037         ecs::Entity entity;
00038         ServerEntityType type;
00039         std::uint32_t networkId; // ID used in network packets
00040         bool isActive;
00041         float lifetime; // -1 for infinite
00042         float currentLife; // Current lifetime counter
00043         std::uint32_t ownerId; // For projectiles, the entity that spawned it
00044     };
00045
00046     ///
00047     /// @class EntityManager
00048     /// @brief Manages entity lifecycle and provides factory methods
00049     /// @namespace gme
00050     ///
00051     class EntityManager
00052     {
00053     public:
00054         explicit EntityManager(ecs::Registry &registry);
00055         ~EntityManager() = default;
00056
00057         EntityManager(const EntityManager &) = delete;
00058         EntityManager &operator=(const EntityManager &) = delete;
00059         EntityManager(EntityManager &&) = delete;
00060         EntityManager &operator=(EntityManager &&) = delete;
00061
00062         // ===== Player Management =====
00063         ecs::Entity createPlayer(std::uint32_t sessionId, float x, float y);
00064         void destroyPlayer(std::uint32_t sessionId);
00065         ecs::Entity getPlayer(std::uint32_t sessionId);
00066         bool hasPlayer(std::uint32_t sessionId) const;
00067         const std::unordered_map<std::uint32_t, ecs::Entity> &getPlayers() const { return m_playerEntities; }
00068
00069         // ===== Enemy Management =====
00070         ecs::Entity createBasicEnemy(float x, float y, float health = 50.0f);
00071         ecs::Entity createAdvancedEnemy(float x, float y, float health = 100.0f);
00072         ecs::Entity createBoss(float x, float y, float health = 1000.0f);
00073         void destroyEnemy(std::uint32_t enemyId);
00074         ecs::Entity getEnemy(std::uint32_t enemyId);
00075         const std::unordered_map<std::uint32_t, ecs::Entity> &getEnemies() const { return m_enemyEntities; }
00076
00077         // ===== Projectile Management =====
00078         ecs::Entity createPlayerProjectile(std::uint32_t playerId, float x, float y, float vx, float vy,
00079             bool isSupercharged = false);
00080         ecs::Entity createEnemyProjectile(std::uint32_t enemyId, float x, float y, float vx, float vy);
00081         void destroyProjectile(std::uint32_t projectileId);
00082         ecs::Entity getProjectile(std::uint32_t projectileId);
00083         const std::unordered_map<std::uint32_t, ecs::Entity> &getProjectiles() const
00084         {
00085             return m_projectileEntities;
00086         }
00087
00088         // ===== Powerup Management =====
00089         ecs::Entity createPowerup(float x, float y, uint8_t powerupType);
00090         void destroyPowerup(std::uint32_t powerupId);
00091
00092         // ===== Generic Entity Management =====

```

```

00093 void updateLifetimes(float deltaTime);
00094 void cleanupDestroyedEntities();
00095 EntityMetadata *getEntityMetadata(std::uint32_t networkId);
00096 const EntityMetadata *getEntityMetadata(std::uint32_t networkId) const;
00097 std::uint32_t getNetworkIdForEntity(ecs::Entity entity) const;
00098
00099 // ===== Network Sync =====
00100 std::vector<rnps::EntityState> getAllEntityStates() const;
00101 rnps::EntityType getNetworkEntityType(ServerEntityType type) const;
00102
00103 // ===== Utility =====
00104 void clear();
00105 size_t getTotalEntityCount() const;
00106
00107 private:
00108     ecs::Registry &m_registry;
00109
00110     // Entity containers (networkId -> Entity)
00111     std::unordered_map<std::uint32_t, ecs::Entity> m_playerEntities;
00112     std::unordered_map<std::uint32_t, ecs::Entity> m_enemyEntities;
00113     std::unordered_map<std::uint32_t, ecs::Entity> m_projectileEntities;
00114     std::unordered_map<std::uint32_t, ecs::Entity> m_powerupEntities;
00115
00116     // Metadata tracking (networkId -> Metadata)
00117     std::unordered_map<std::uint32_t, EntityMetadata> m_entityMetadata;
00118
00119     // Reverse lookup (Entity -> networkId)
00120     std::unordered_map<ecs::Entity, std::uint32_t> m_entityToNetworkId;
00121
00122     // ID generators
00123     std::uint32_t m_nextEnemyId = 2000;
00124     std::uint32_t m_nextProjectileId = 1000;
00125     std::uint32_t m_nextPowerupId = 5000;
00126
00127     // Destruction queue (to avoid modifying containers during iteration)
00128     std::vector<std::uint32_t> m_destroyQueue;
00129
00130     // Helper methods
00131     void registerEntity(ecs::Entity entity, ServerEntityType type, std::uint32_t networkId,
00132         float lifetime = -1.0f, std::uint32_t ownerId = 0);
00133     void unregisterEntity(std::uint32_t networkId);
00134     void processDestroyQueue();
00135 };
00136
00137 } // namespace gme

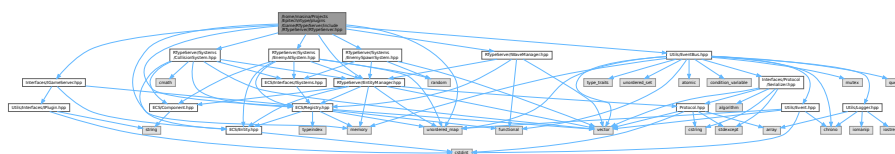
```

19.247 /home/masina/Projects/Epitech/rtype/plugins/Game/RType/↵
Server/include/RTypeServer/RTypeServer.hpp File Reference

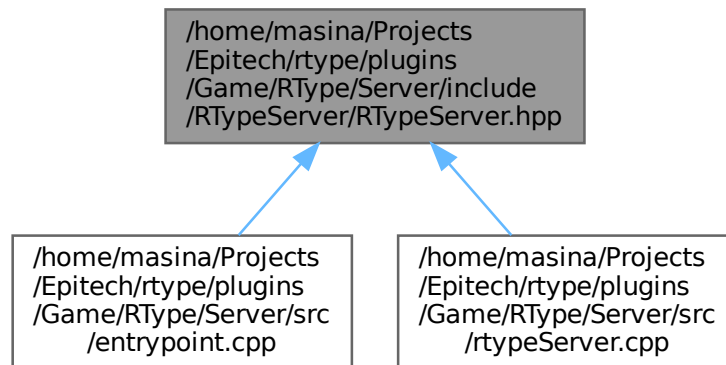
This file contains the `RTypeServer` class declaration.

```
#include <memory>
#include <unordered_map>
#include "ECS/Registry.hpp"
#include "Interfaces/IGameServer.hpp"
#include "Utils/EventBus.hpp"
#include "RTypeServer/EntityManager.hpp"
#include "RTypeServer/Systems/CollisionSystem.hpp"
#include "RTypeServer/Systems/EnemyAISystem.hpp"
#include "RTypeServer/Systems/EnemySpawnSystem.hpp"
#include "RTypeServer/WaveManager.hpp"
```

Include dependency graph for RTypeServer.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [gme::RTypeServer](#)
Class for the [RTypeServer](#) plugin.

Namespaces

- namespace [gme](#)

Enumerations

- enum class [gme::LevelState](#) : uint8_t { [gme::WAITING_FOR_PLAYERS](#) = 0 , [gme::IN_PROGRESS](#) = 1 , [gme::COMPLETED](#) = 2 , [gme::LOOSE](#) = 3 }

19.247.1 Detailed Description

This file contains the [RTypeServer](#) class declaration.
Definition in file [RTypeServer.hpp](#).

19.248 RTypeServer.hpp

[Go to the documentation of this file.](#)

```

00001 ///  

00002 ///  

00003 ///  

00004 ///  

00005 ///  

00006 ///  

00007 #pragma once  

00008  

00009 #include <memory>  

00010 #include <unordered_map>  

00011  

00012 #include "ECS/Registry.hpp"  

00013 #include "Interfaces/IGameServer.hpp"  

00014 #include "Utils/EventBus.hpp"  

00015  

00016 // New includes for entity management  

00017 #include "RTypeServer/EntityManager.hpp"  

00018 #include "RTypeServer/Systems/CollisionSystem.hpp"  

00019 #include "RTypeServer/Systems/EnemyAISystem.hpp"  

00020 #include "RTypeServer/Systems/EnemySpawnSystem.hpp"  

00021 #include "RTypeServer/WaveManager.hpp"  

00022

```



```

00023 namespace gme
00024 {
00025
00026     enum class LevelState : uint8_t
00027     {
00028         WAITING_FOR_PLAYERS = 0,
00029         IN_PROGRESS = 1,
00030         COMPLETED = 2,
00031         LOOSE = 3,
00032     };
00033
00034     ///
00035     /// @class RTypeServer
00036     /// @brief Class for the RTypeServer plugin
00037     /// @namespace gme
00038     ///
00039     class RTypeServer final : public IGameServer
00040     {
00041
00042     public:
00043         RTypeServer();
00044         ~RTypeServer() override = default;
00045
00046         RTypeServer(const RTypeServer &) = delete;
00047         RTypeServer &operator=(const RTypeServer &) = delete;
00048         RTypeServer(RTypeServer &&) = delete;
00049         RTypeServer &operator=(RTypeServer &&) = delete;
00050
00051         [[nodiscard]] const std::string getName() const override { return "RType_Server"; }
00052         [[nodiscard]] utl::PluginType getType() const override { return utl::PluginType::GAME_SERVER; }
00053
00054         [[nodiscard]] State getState() const override { return m_gameState; }
00055
00056         void start() override;
00057         void stop() override;
00058         void update(float deltaTime) override;
00059
00060     private:
00061         void processServerStartEvent(const utl::Event &event);
00062         void processPlayerInputEvent(const utl::Event &event);
00063         void processGameStartEvent(const utl::Event &event);
00064         void updateEntities(float deltaTime);
00065         void updateSystems(float deltaTime);
00066         void broadcastWorldState();
00067         void handlePlayerShooting(std::uint32_t sessionId, float deltaTime);
00068
00069         utl::EventBus &m_eventBus;
00070         ecs::Registry m_registry;
00071
00072         // NEW: Entity management
00073         std::unique_ptr<EntityManager> m_entityManager;
00074
00075         // NEW: Game systems
00076         std::unique_ptr<CollisionSystem> m_collisionSystem;
00077         std::unique_ptr<EnemyAISystem> m_enemyAISystem;
00078         std::unique_ptr<EnemySpawnSystem> m_enemySpawnSystem;
00079         std::unique_ptr<WaveManager> m_waveManager;
00080
00081         State m_gameState = State::PLAYING;
00082         LevelState m_levelState = LevelState::WAITING_FOR_PLAYERS;
00083
00084         // Player input tracking
00085         std::unordered_map<std::uint32_t, bool> m_playerShooting;
00086         std::unordered_map<std::uint32_t, float> m_lastShotTime;
00087
00088         float m_lastBroadcastTime = 0.0f;
00089         const float BROADCAST_INTERVAL = 1.0f / 60.0f; // 60 Hz
00090         const float PROJECTILE_COOLDOWN = 0.3f;
00091
00092     }; // class RTypeServer
00093 } // namespace gme

```

19.249 /home/masina/Projects/Epitech/rtype/plugins/Game/RType/↵ Server/include/RTypeServer/Systems/CollisionSystem.hpp File Reference

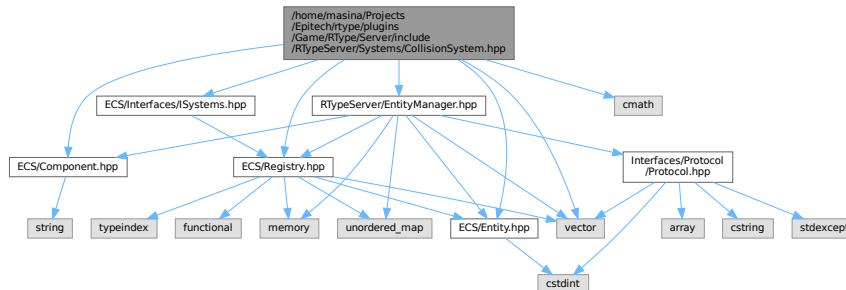
Server-side collision detection system.

#include "ECS/Component.hpp"

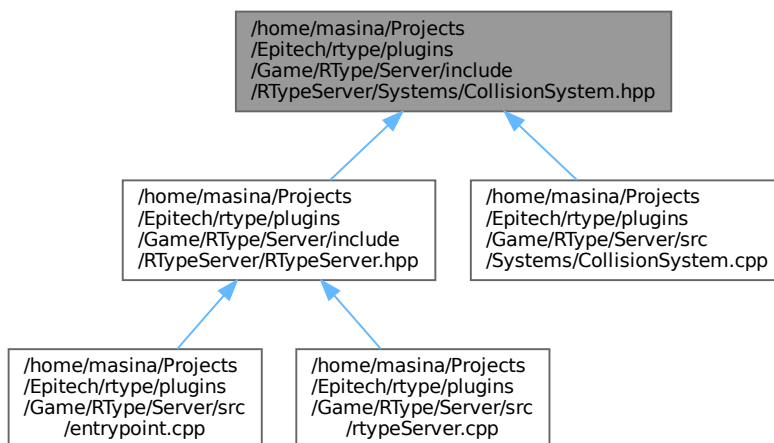
#include "ECS/Entity.hpp"

#include "ECS/Interfaces/ISystems.hpp"

```
#include "ECS/Registry.hpp"
#include "RTypeServer/EntityManager.hpp"
#include <cmath>
#include <vector>
Include dependency graph for CollisionSystem.hpp:
```



This graph shows which files directly or indirectly include this file:



Classes

- struct [gme::CollisionPair](#)
Collision pair for tracking collisions.
- class [gme::CollisionSystem](#)
Handles collision detection and resolution on server.

Namespaces

- namespace [gme](#)

19.249.1 Detailed Description

Server-side collision detection system.
Definition in file [CollisionSystem.hpp](#).

19.250 CollisionSystem.hpp

[Go to the documentation of this file.](#)

```

00001 ///  

00002 ///  

00003 ///  

00004 ///  

00005 ///  

00006 ///  

00007 #pragma once  

00008  

00009 #include "ECS/Component.hpp"  

00010 #include "ECS/Entity.hpp"  

00011 #include "ECS/Interfaces/ISystems.hpp"  

00012 #include "ECS/Registry.hpp"  

00013 #include "RTypeServer/EntityManager.hpp"  

00014 #include <cmath>  

00015 #include <vector>  

00016  

00017 namespace gme  

00018 {  

00019     ///  

00020     ///  

00021     ///  

00022     struct CollisionPair  

00023     {  

00024         std::uint32_t entityId1;  

00025         std::uint32_t entityId2;  

00026         float overlapDistance;  

00027     };  

00028  

00029     ///  

00030     ///  

00031     ///  

00032     ///  

00033     ///  

00034     class CollisionSystem final : public ecs::ASystem  

00035     {  

00036     public:  

00037         explicit CollisionSystem(ecs::Registry &registry, EntityManager &entityManager)  

00038             : m_registry(registry), m_entityManager(entityManager)  

00039         {  

00040         }  

00041         ~CollisionSystem() override = default;  

00042  

00043         CollisionSystem(const CollisionSystem &) = delete;  

00044         CollisionSystem &operator=(const CollisionSystem &) = delete;  

00045         CollisionSystem(CollisionSystem &&) = delete;  

00046         CollisionSystem &operator=(CollisionSystem &&) = delete;  

00047  

00048         void update(ecs::Registry &registry, float deltaTime) override;  

00049  

00050         ///  

00051         size_t getCollisionCount() const { return m_collisionCount; }  

00052         void resetCollisionCount() { m_collisionCount = 0; }  

00053     private:  

00054         ecs::Registry &m_registry;  

00055         EntityManager &m_entityManager;  

00056         size_t m_collisionCount = 0;  

00057  

00058         ///  

00059         ///  

00060         bool checkCircleCollision(float x1, float y1, float r1, float x2, float y2, float r2,  

00061             float *overlapDist = nullptr) const;  

00062  

00063         ///  

00064         void handlePlayerProjectileEnemyCollision();  

00065         void handleEnemyProjectilePlayerCollision();  

00066         void handlePlayerEnemyCollision();  

00067         void handlePlayerPowerupCollision();  

00068  

00069         ///  

00070         bool getCollisionInfo(ecs::Entity entity, float &x, float &y, float &radius) const;  

00071  

00072         ///  

00073         void applyDamageToEnemy(std::uint32_t enemyId, float damage);  

00074         void applyDamageToPlayer(std::uint32_t playerId, float damage);  

00075     };  

00076  

00077     inline bool CollisionSystem::checkCircleCollision(float x1, float y1, float r1, float x2, float y2, float r2,  

00078         float *overlapDist) const  

00079     {  

00080         float dx = x2 - x1;  

00081         float dy = y2 - y1;  

00082         float distance = std::sqrt(dx * dx + dy * dy);  

00083         float radiusSum = r1 + r2;  


```

```

00084
00085     if (overlapDist)
00086     {
00087         *overlapDist = radiusSum - distance;
00088     }
00089
00090     return distance < radiusSum;
00091 }
00092
00093 inline bool CollisionSystem::getCollisionInfo(ecs::Entity entity, float &x, float &y, float &radius) const
00094 {
00095     auto *transform = m_registry.getComponent<ecs::Transform>(entity);
00096     auto *hitbox = m_registry.getComponent<ecs::Hitbox>(entity);
00097
00098     if (!transform || !hitbox)
00099         return false;
00100
00101     x = transform->x + hitbox->offsetX;
00102     y = transform->y + hitbox->offsetY;
00103     radius = hitbox->radius;
00104     return true;
00105 }
00106
00107 } // namespace gme

```

19.251 /home/masina/Projects/Epitech/rtype/plugins/Game/RType/↵ Server/include/RTypeServer/Systems/EnemyAISystem.hpp File Reference

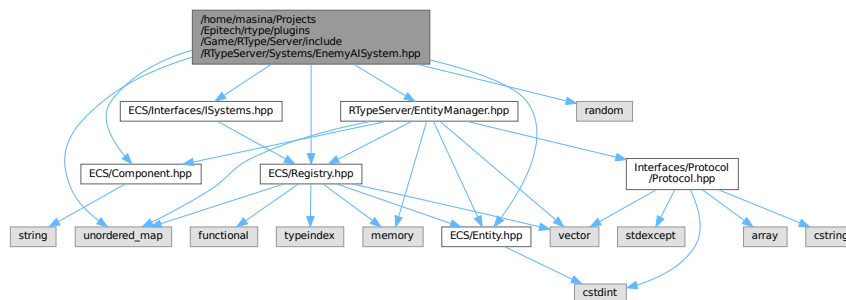
Enemy AI and behavior system for server.

```

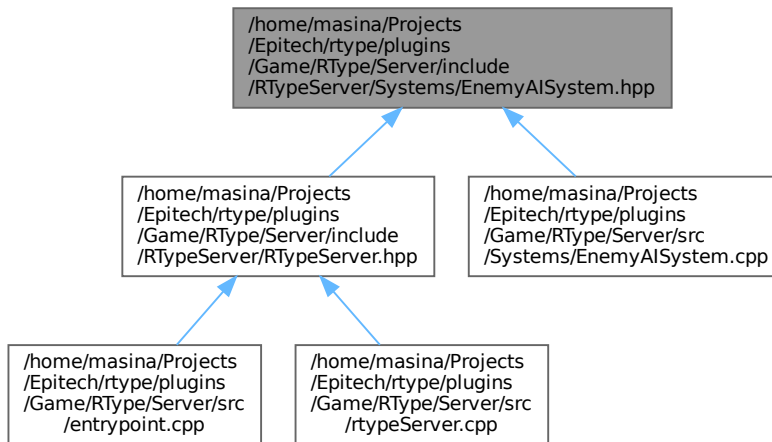
#include "ECS/Component.hpp"
#include "ECS/Entity.hpp"
#include "ECS/Interfaces/ISystems.hpp"
#include "ECS/Registry.hpp"
#include "RTypeServer/EntityManager.hpp"
#include <random>
#include <unordered_map>

```

Include dependency graph for EnemyAISystem.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [gme::EnemyAISystem](#)
Manages enemy behavior, movement patterns, and shooting.

Namespaces

- namespace [gme](#)

Enumerations

- enum class [gme::EnemyBehavior](#) : `uint8_t` {
[gme::STRAIGHT](#) = 0 , [gme::SINE_WAVE](#) = 1 , [gme::AGGRESSIVE](#) = 2 , [gme::STATIONARY](#) = 3 ,
[gme::ZIGZAG](#) = 4 }
 Enemy behavior pattern types.

19.251.1 Detailed Description

Enemy AI and behavior system for server.

Definition in file [EnemyAISystem.hpp](#).

19.252 EnemyAISystem.hpp

[Go to the documentation of this file.](#)

```

00001 ///  

00002 ///  

00003 ///  

00004 ///  

00005 ///  

00006 ///  

00007 #pragma once  

00008  

00009 #include "ECS/Component.hpp"  

00010 #include "ECS/Entity.hpp"  

00011 #include "ECS/Interfaces/ISystems.hpp"  

00012 #include "ECS/Registry.hpp"  

00013 #include "RTypeServer/EntityManager.hpp"  

00014 #include <random>  

00015 #include <unordered_map>  

00016
  
```

```

00017 namespace gme
00018 {
00019     ///
00020     /// @brief Enemy behavior pattern types
00021     ///
00022     enum class EnemyBehavior : uint8_t
00023     {
00024         STRAIGHT = 0, // Move straight left
00025         SINE_WAVE = 1, // Sine wave pattern
00026         AGGRESSIVE = 2, // Follow player
00027         STATIONARY = 3, // Stay in place and shoot
00028         ZIGZAG = 4 // Zigzag pattern
00029     };
00030
00031     ///
00032     /// @class EnemyAISystem
00033     /// @brief Manages enemy behavior, movement patterns, and shooting
00034     /// @namespace gme
00035     ///
00036     class EnemyAISystem final : public ecs::ASystem
00037     {
00038     public:
00039         explicit EnemyAISystem(ecs::Registry &registry, EntityManager &entityManager);
00040         ~EnemyAISystem() override = default;
00041
00042         EnemyAISystem(const EnemyAISystem &) = delete;
00043         EnemyAISystem &operator=(const EnemyAISystem &) = delete;
00044         EnemyAISystem(EnemyAISystem &&) = delete;
00045         EnemyAISystem &operator=(EnemyAISystem &&) = delete;
00046
00047         void update(ecs::Registry &registry, float deltaTime) override;
00048
00049         // Configuration
00050         void setAggressiveness(float value) { m_aggressiveness = value; }
00051         float getAggressiveness() const { return m_aggressiveness; }
00052
00053     private:
00054         ecs::Registry &m_registry;
00055         EntityManager &m_entityManager;
00056
00057         float m_aggressiveness; // How aggressively enemies pursue players
00058         std::mt19937 m_rng;
00059         std::uniform_real_distribution<float> m_shootChance;
00060
00061         // Per-entity timing data to avoid static variable synchronization
00062         std::unordered_map<std::uint32_t, float> m_enemyMovementTimers;
00063         std::unordered_map<std::uint32_t, float> m_bossSpreadTimers;
00064
00065         // AI behaviors for different enemy types
00066         void updateBasicEnemyAI(std::uint32_t enemyId, ecs::Entity enemy, float deltaTime);
00067         void updateAdvancedEnemyAI(std::uint32_t enemyId, ecs::Entity enemy, float deltaTime);
00068         void updateBossAI(std::uint32_t enemyId, ecs::Entity enemy, float deltaTime);
00069
00070         // Movement patterns (now takes enemyId for per-entity timing)
00071         void applySineWaveMovement(std::uint32_t enemyId, ecs::Entity enemy, float deltaTime,
00072                                     float frequency = 2.0f, float amplitude = 50.0f);
00073         void applyAggressiveMovement(ecs::Entity enemy, float deltaTime, float speed = 150.0f);
00074         void applyZigzagMovement(std::uint32_t enemyId, ecs::Entity enemy, float deltaTime);
00075
00076         // Shooting logic
00077         void tryShoot(std::uint32_t enemyId, ecs::Entity enemy, float deltaTime);
00078         bool canShoot(ecs::Entity enemy) const;
00079
00080         // Utility
00081         ecs::Entity findNearestPlayer(float x, float y) const;
00082         float getDistanceToNearestPlayer(float x, float y) const;
00083         void clampToScreen(ecs::Entity enemy);
00084     };
00085
00086 } // namespace gme

```

19.253 /home/masina/Projects/Epitech/rtype/plugins/Game/RType/↵ Server/include/RTypeServer/Systems/EnemySpawnSystem.hpp File Reference

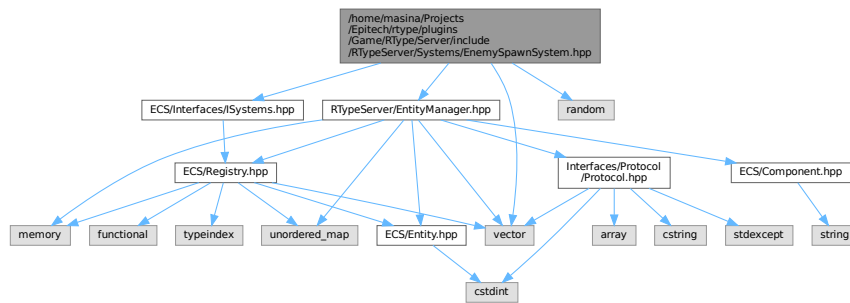
Enemy spawning system for server.

```

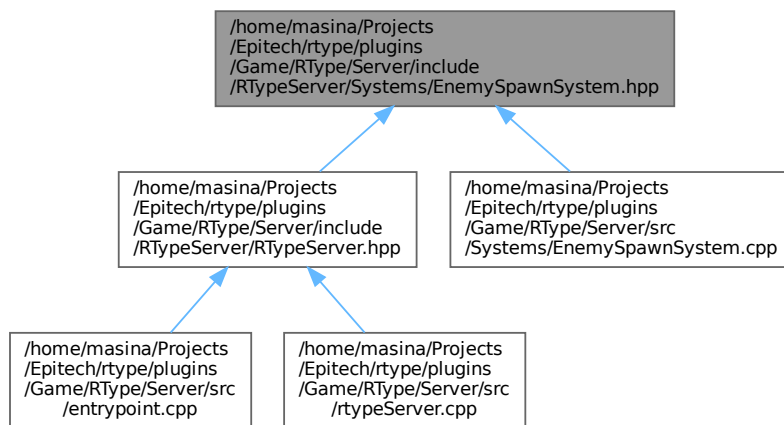
#include "ECS/Interfaces/ISystems.hpp"
#include "RTypeServer/EntityManager.hpp"
#include <random>
#include <vector>

```

Include dependency graph for EnemySpawnSystem.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- struct [gme::SpawnWave](#)
Spawn wave configuration.
- class [gme::EnemySpawnSystem](#)
Manages enemy spawning based on level progression.

Namespaces

- namespace [gme](#)

19.253.1 Detailed Description

Enemy spawning system for server.

Definition in file [EnemySpawnSystem.hpp](#).

19.254 EnemySpawnSystem.hpp

[Go to the documentation of this file.](#)

```
00001 ///
00002 /// @file EnemySpawnSystem.hpp
```

```

00003 /// @brief Enemy spawning system for server
00004 /// @namespace gme
00005 ///
00006
00007 #pragma once
00008
00009 #include "ECS/Interfaces/ISystems.hpp"
00010 #include "RTypeServer/EntityManager.hpp"
00011 #include <random>
00012 #include <vector>
00013
00014 namespace gme
00015 {
00016     ///
00017     /// @brief Spawn wave configuration
00018     ///
00019     struct SpawnWave
00020     {
00021         float spawnTime; // Time from level start
00022         int basicEnemyCount;
00023         int advancedEnemyCount;
00024         bool bossSpawn;
00025         float spawnInterval; // Time between enemy spawns in wave
00026         int spawnedCount; // Track how many have spawned
00027     };
00028
00029     ///
00030     /// @class EnemySpawnSystem
00031     /// @brief Manages enemy spawning based on level progression
00032     /// @namespace gme
00033     ///
00034     class EnemySpawnSystem final : public ecs::ASystem
00035     {
00036     public:
00037         explicit EnemySpawnSystem(EntityManager &entityManager);
00038         ~EnemySpawnSystem() override = default;
00039
00040         EnemySpawnSystem(const EnemySpawnSystem &) = delete;
00041         EnemySpawnSystem &operator=(const EnemySpawnSystem &) = delete;
00042         EnemySpawnSystem(EnemySpawnSystem &&) = delete;
00043         EnemySpawnSystem &operator=(EnemySpawnSystem &&) = delete;
00044
00045         void update(ecs::Registry &registry, float deltaTime) override;
00046         void reset();
00047
00048         // Configuration
00049         void setEnabled(bool enabled) { m_enabled = enabled; }
00050         bool isEnabled() const { return m_enabled; }
00051
00052         // Statistics
00053         size_t getTotalEnemiesSpawned() const { return m_totalEnemiesSpawned; }
00054         size_t getCurrentWaveIndex() const { return m_currentWaveIndex; }
00055         float getLevelTime() const { return m_levelTime; }
00056
00057     private:
00058         EntityManager &m_entityManager;
00059         float m_levelTime;
00060         size_t m_currentWaveIndex;
00061         float m_waveSpawnTimer;
00062         bool m_enabled;
00063         size_t m_totalEnemiesSpawned;
00064
00065         std::vector<SpawnWave> m_waves;
00066         std::mt19937 m_rng;
00067         std::uniform_real_distribution<float> m_yDistribution;
00068
00069         void initializeWaves();
00070         void processWave(SpawnWave &wave, float deltaTime);
00071         float getRandomY();
00072         float getSpawnX() const { return 2000.0f; } // Right side of screen
00073     };
00074
00075 } // namespace gme

```

19.255 /home/masina/Projects/Epitech/rtype/plugins/Game/RType/↵ Server/include/RTypeServer/WaveManager.hpp File Reference

Wave-based enemy spawning system for RTypeServer.

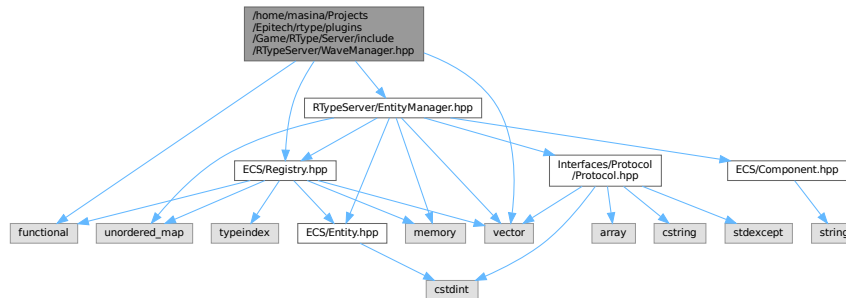
```

#include "ECS/Registry.hpp"
#include "RTypeServer/EntityManager.hpp"
#include <functional>

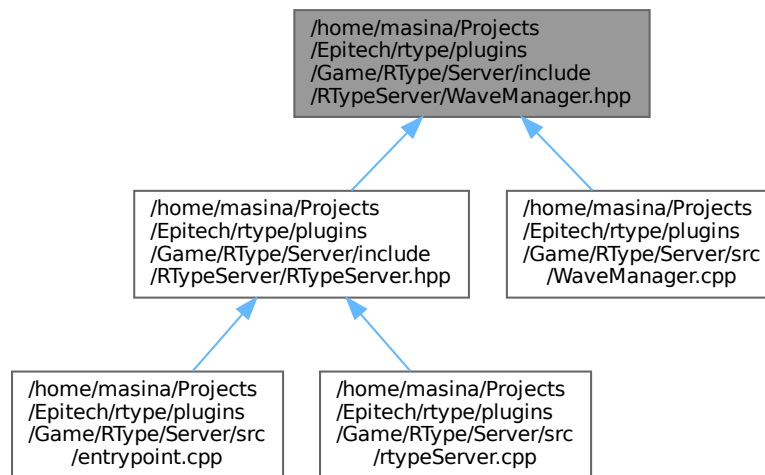
```


#include <vector>

Include dependency graph for WaveManager.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- struct [gme::EnemySpawn](#)
Enemy spawn definition for a wave.
- struct [gme::Wave](#)
[Wave](#) definition containing multiple enemy spawns.
- class [gme::WaveManager](#)
Manages wave-based enemy spawning with configurable patterns.

Namespaces

- namespace [gme](#)

19.255.1 Detailed Description

Wave-based enemy spawning system for RTypeServer.

Definition in file [WaveManager.hpp](#).

19.256 WaveManager.hpp

[Go to the documentation of this file.](#)

```

00001 /// @file WaveManager.hpp
00002 /// @brief Wave-based enemy spawning system for RTypeServer
00003 /// @namespace gme
00004 ///
00005
00006 #pragma once
00007
00008 #include "ECS/Registry.hpp"
00009 #include "RTypeServer/EntityManager.hpp"
00010 #include <functional>
00011 #include <vector>
00012
00013 namespace gme
00014 {
00015     ///
00016     /// @brief Enemy spawn definition for a wave
00017     ///
00018     struct EnemySpawn
00019     {
00020         ServerEntityType type;
00021         float x;           // Spawn X position
00022         float y;           // Spawn Y position
00023         float health;      // Enemy health
00024         float spawnDelay;  // Delay before spawning this enemy (in seconds from wave start)
00025     };
00026
00027     ///
00028     /// @brief Wave definition containing multiple enemy spawns
00029     ///
00030     struct Wave
00031     {
00032         int waveNumber;
00033         std::vector<EnemySpawn> enemies;
00034         float duration;      // How long until next wave if not cleared
00035         bool waitForClear;    // If true, wait until all enemies are dead
00036         std::function<void()> onComplete; // Optional callback when wave completes
00037     };
00038
00039     ///
00040     /// @class WaveManager
00041     /// @brief Manages wave-based enemy spawning with configurable patterns
00042     /// @namespace gme
00043     ///
00044     class WaveManager
00045     {
00046     public:
00047         explicit WaveManager(EntityManager &entityManager);
00048         ~WaveManager() = default;
00049
00050         WaveManager(const WaveManager &) = delete;
00051         WaveManager &operator=(const WaveManager &) = delete;
00052         WaveManager(WaveManager &&) = delete;
00053         WaveManager &operator=(WaveManager &&) = delete;
00054
00055         /// @brief Update wave manager logic
00056         /// @param registry The ECS registry
00057         /// @param dt Delta time in seconds
00058         /// @param screenWidth Screen width for spawn positioning
00059         void update(ecs::Registry &registry, float dt, int screenWidth);
00060
00061         /// @brief Check if initial delay has passed and waves can start
00062         bool isReady() const { return m_initialDelayPassed; }
00063
00064         /// @brief Start the wave system
00065         void start();
00066
00067         /// @brief Stop and reset the wave system
00068         void reset();
00069
00070         /// @brief Check if wave system is active
00071         bool isActive() const { return m_active; }
00072
00073         /// @brief Get current wave number
00074         int getCurrentWave() const { return m_currentWaveIndex; }
00075
00076         /// @brief Get total number of waves
00077         size_t getTotalWaves() const { return m_waves.size(); }
00078
00079         /// @brief Check if all waves are completed
00080         bool isCompleted() const { return m_completed; }
00081
00082         /// @brief Add a custom wave
00083         void addWave(const Wave &wave);

```

```

00084
00085     /// @brief Clear all waves
00086     void clearWaves();
00087
00088     /// @brief Initialize with default wave patterns
00089     void setupDefaultWaves();
00090
00091 private:
00092     EntityManager &m_entityManager;
00093     std::vector<Wave> m_waves;
00094     int m_currentWaveIndex;
00095     float m_waveTimer;
00096     float m_spawnTimer;
00097     bool m_active;
00098     bool m_completed;
00099     bool m_waveInProgress;
00100     bool m_initialDelayPassed;
00101     float m_initialDelayTimer;
00102     std::vector<bool> m_enemiesSpawned; // Track which enemies in current wave have spawned
00103
00104     static constexpr float INITIAL_DELAY = 0.5f; // Sync with client STAGE_SPAWN_DELAY
00105
00106     /// @brief Start the next wave
00107     void startNextWave();
00108
00109     /// @brief Check if current wave is cleared
00110     bool isWaveCleared() const;
00111
00112     /// @brief Process enemy spawns for current wave
00113     void processSpawns(float dt, int screenWidth);
00114
00115     /// @brief Create default wave patterns (5 waves: 4x30s + boss 60s)
00116     void createWave1();
00117     void createWave2();
00118     void createWave3();
00119     void createWave4();
00120     void createBossWave();
00121 };
00122
00123 } // namespace gme

```

19.257 /home/masina/Projects/Epitech/rtype/plugins/Game/RType/Server/src/EntityManager.cpp File Reference ↩

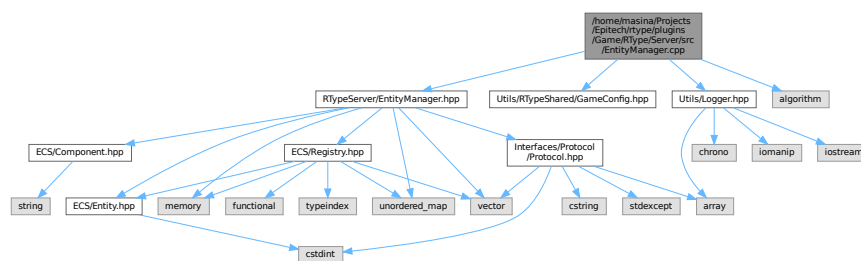
Implementation of EntityManager for RTypeServer.

```

#include "RTypeServer/EntityManager.hpp"
#include "Utils/RTypeShared/GameConfig.hpp"
#include "Utils/Logger.hpp"
#include <algorithm>

```

Include dependency graph for EntityManager.cpp:



Namespaces

- namespace [gme](#)

19.257.1 Detailed Description

Implementation of EntityManager for RTypeServer.

Definition in file [EntityManager.cpp](#).

19.258 EntityManager.cpp

[Go to the documentation of this file.](#)

```
00001 ///  
00002 ///  
00003 ///  
00004 ///  
00005 ///  
00006  
00007 #include "RTypeServer/EntityManager.hpp"  
00008 #include "Utils/RTypeShared/GameConfig.hpp"  
00009 #include "Utils/Logger.hpp"  
00010 #include <algorithm>  
00011  
00012 namespace gme  
00013 {  
00014     EntityManager::EntityManager(ecs::Registry &registry) : m_registry(registry) {}  
00015  
00016     ecs::Entity EntityManager::createPlayer(std::uint32_t sessionId, float x, float y)  
00017     {  
00018         if (hasPlayer(sessionId))  
00019         {  
00020             utl::Logger::log("EntityManager: Player with sessionId " + std::to_string(sessionId) + " already exists",  
00021                 utl::LogLevel::WARNING);  
00022             return m_playerEntities[sessionId];  
00023         }  
00024  
00025         // Validate spawn position  
00026         x = std::max(0.0f, std::min(x, utl::GameConfig::Server::SCREEN_WIDTH));  
00027         y = std::max(0.0f, std::min(y, utl::GameConfig::Server::SCREEN_HEIGHT));  
00028  
00029         ecs::Entity playerEntity =  
00030             m_registry.createEntity()  
00031             .with<ecs::Transform>("player_transform_" + std::to_string(sessionId), x, y, 0.0f)  
00032             .with<ecs::Velocity>("player_velocity_" + std::to_string(sessionId), 0.0f, 0.0f)  
00033             .with<ecs::Player>("player_" + std::to_string(sessionId), true)  
00034             .with<ecs::BeamCharge>("beam_charge_" + std::to_string(sessionId), 0.0f, 1.0f)  
00035             .with<ecs::Hitbox>("player_hitbox_" + std::to_string(sessionId), 10.0f, 0.0f, 0.0f)  
00036             .build();  
00037  
00038         m_playerEntities[sessionId] = playerEntity;  
00039         registerEntity(playerEntity, ServerEntityType::PLAYER, sessionId, -1.0f);  
00040  
00041         utl::Logger::log("EntityManager: Created player entity for sessionId " + std::to_string(sessionId) +  
00042             " at position (" + std::to_string(x) + ", " + std::to_string(y) + ")",  
00043             utl::LogLevel::INFO);  
00044  
00045         return playerEntity;  
00046     }  
00047  
00048     void EntityManager::destroyPlayer(std::uint32_t sessionId)  
00049     {  
00050         auto it = m_playerEntities.find(sessionId);  
00051         if (it != m_playerEntities.end())  
00052         {  
00053             ecs::Entity entity = it->second;  
00054  
00055             // Remove components  
00056             if (m_registry.hasComponent<ecs::Transform>(entity))  
00057                 m_registry.removeComponent<ecs::Transform>(entity);  
00058             if (m_registry.hasComponent<ecs::Velocity>(entity))  
00059                 m_registry.removeComponent<ecs::Velocity>(entity);  
00060             if (m_registry.hasComponent<ecs::Player>(entity))  
00061                 m_registry.removeComponent<ecs::Player>(entity);  
00062             if (m_registry.hasComponent<ecs::BeamCharge>(entity))  
00063                 m_registry.removeComponent<ecs::BeamCharge>(entity);  
00064             if (m_registry.hasComponent<ecs::Hitbox>(entity))  
00065                 m_registry.removeComponent<ecs::Hitbox>(entity);  
00066  
00067             unregisterEntity(sessionId);  
00068             m_playerEntities.erase(it);  
00069  
00070             utl::Logger::log("EntityManager: Destroyed player entity for sessionId " + std::to_string(sessionId),  
00071                 utl::LogLevel::INFO);  
00072         }  
00073     }  
00074  
00075     ecs::Entity EntityManager::getPlayer(std::uint32_t sessionId)  
00076     {  
00077         auto it = m_playerEntities.find(sessionId);  
00078         return (it != m_playerEntities.end()) ? it->second : ecs::INVALID_ENTITY;  
00079     }  
00080  
00081     bool EntityManager::hasPlayer(std::uint32_t sessionId) const  
00082     {  
00083         return m_playerEntities.find(sessionId) != m_playerEntities.end();  
00084     }  
00085 }
```

```

00084     }
00085
00086     ecs::Entity EntityManager::createBasicEnemy(float x, float y, float health)
00087     {
00088         // Validate parameters
00089         if (health <= 0.0f)
00090             health = 50.0f;
00091
00092         std::uint32_t enemyId = m_nextEnemyId++;
00093
00094         ecs::Entity enemyEntity =
00095             m_registry.createEntity()
00096                 .with<ecs::Transform>("enemy_transform_" + std::to_string(enemyId), x, y, 0.0f)
00097                 .with<ecs::Velocity>("enemy_velocity_" + std::to_string(enemyId), -200.0f, 0.0f) // Move left
00098                 .with<ecs::Enemy>("enemy_" + std::to_string(enemyId), health, health, 10.0f, 200.0f, 0.0f, 2.0f)
00099                 .with<ecs::Hitbox>("enemy_hitbox_" + std::to_string(enemyId), 30.0f, 0.0f,
00100                     0.0f) // Increased from 15 to 30 for better collision
00101                 .build();
00102
00103         m_enemyEntities[enemyId] = enemyEntity;
00104         registerEntity(enemyEntity, ServerEntityType::ENEMY_BASIC, enemyId, -1.0f);
00105
00106         utl::Logger::log("EntityManager: Created basic enemy " + std::to_string(enemyId) + " at position (" +
00107             std::to_string(x) + ", " + std::to_string(y) + ")",
00108             utl::LogLevel::INFO);
00109
00110         return enemyEntity;
00111     }
00112
00113     ecs::Entity EntityManager::createAdvancedEnemy(float x, float y, float health)
00114     {
00115         // Validate parameters
00116         if (health <= 0.0f)
00117             health = 100.0f;
00118
00119         std::uint32_t enemyId = m_nextEnemyId++;
00120
00121         ecs::Entity enemyEntity =
00122             m_registry.createEntity()
00123                 .with<ecs::Transform>("enemy_transform_" + std::to_string(enemyId), x, y, 0.0f)
00124                 .with<ecs::Velocity>("enemy_velocity_" + std::to_string(enemyId), -150.0f, 0.0f)
00125                 .with<ecs::Enemy>("enemy_" + std::to_string(enemyId), health, health, 20.0f, 150.0f, 0.0f, 1.5f)
00126                 .with<ecs::Hitbox>("enemy_hitbox_" + std::to_string(enemyId), 35.0f, 0.0f,
00127                     0.0f) // Increased from 20 to 35 for better collision
00128                 .build();
00129
00130         m_enemyEntities[enemyId] = enemyEntity;
00131         registerEntity(enemyEntity, ServerEntityType::ENEMY_ADVANCED, enemyId, -1.0f);
00132
00133         utl::Logger::log("EntityManager: Created advanced enemy " + std::to_string(enemyId) + " at position (" +
00134             std::to_string(x) + ", " + std::to_string(y) + ")",
00135             utl::LogLevel::INFO);
00136
00137         return enemyEntity;
00138     }
00139
00140     ecs::Entity EntityManager::createBoss(float x, float y, float health)
00141     {
00142         // Validate parameters
00143         if (health <= 0.0f)
00144             health = 1000.0f;
00145
00146         std::uint32_t enemyId = m_nextEnemyId++;
00147
00148         ecs::Entity bossEntity =
00149             m_registry.createEntity()
00150                 .with<ecs::Transform>("boss_transform_" + std::to_string(enemyId), x, y, 0.0f)
00151                 .with<ecs::Velocity>("boss_velocity_" + std::to_string(enemyId), -50.0f, 0.0f)
00152                 .with<ecs::Enemy>("boss_" + std::to_string(enemyId), health, health, 50.0f, 50.0f, 0.0f, 0.5f)
00153                 .with<ecs::Hitbox>("boss_hitbox_" + std::to_string(enemyId), 90.0f, 0.0f,
00154                     0.0f) // Increased from 50 to 90 for better collision with large sprite
00155                 .build();
00156
00157         m_enemyEntities[enemyId] = bossEntity;
00158         registerEntity(bossEntity, ServerEntityType::BOSS, enemyId, -1.0f);
00159
00160         utl::Logger::log("EntityManager: Created boss " + std::to_string(enemyId) + " at position (" +
00161             std::to_string(x) + ", " + std::to_string(y) + ")",
00162             utl::LogLevel::INFO);
00163
00164         return bossEntity;
00165     }
00166
00167     void EntityManager::destroyEnemy(std::uint32_t enemyId)
00168     {
00169         auto *metadata = getEntityMetadata(enemyId);
00170         if (metadata)

```

```

00171     {
00172         metadata->isActive = false;
00173     }
00174     m_destroyQueue.push_back(enemyId);
00175 }
00176
00177 ecs::Entity EntityManager::getEnemy(std::uint32_t enemyId)
00178 {
00179     auto it = m_enemyEntities.find(enemyId);
00180     return (it != m_enemyEntities.end()) ? it->second : ecs::INVALID_ENTITY;
00181 }
00182
00183 ecs::Entity EntityManager::createPlayerProjectile(std::uint32_t playerId, float x, float y, float vx, float vy,
00184                                                  bool isSupercharged)
00185 {
00186     std::uint32_t projectileId = m_nextProjectileId++;
00187
00188     ecs::Projectile::Type projType = isSupercharged ? ecs::Projectile::SUPERCHARGED : ecs::Projectile::BASIC;
00189     float damage = isSupercharged ? 50.0f : 25.0f;
00190     int pierce = isSupercharged ? 3 : 1;
00191     float hitboxRadius = isSupercharged ? 8.0f : 5.0f; // Match GameConfig::Hitbox values
00192
00193     ecs::Entity projectileEntity =
00194         m_registry.createEntity()
00195             .with<ecs::Transform>("projectile_transform_" + std::to_string(projectileId), x, y, 0.0f)
00196             .with<ecs::Velocity>("projectile_velocity_" + std::to_string(projectileId), vx, vy)
00197             .with<ecs::Projectile>("projectile_" + std::to_string(projectileId), projType, damage, 5.0f, 0.0f,
00198                                 pierce)
00199             .with<ecs::Hitbox>("projectile_hitbox_" + std::to_string(projectileId), hitboxRadius, 0.0f, 0.0f)
00200             .build();
00201
00202     m_projectileEntities[projectileId] = projectileEntity;
00203     registerEntity(projectileEntity, ServerEntityType::PROJECTILE_PLAYER, projectileId, 5.0f, playerId);
00204
00205     return projectileEntity;
00206 }
00207
00208 ecs::Entity EntityManager::createEnemyProjectile(std::uint32_t enemyId, float x, float y, float vx, float vy)
00209 {
00210     std::uint32_t projectileId = m_nextProjectileId++;
00211
00212     ecs::Entity projectileEntity =
00213         m_registry.createEntity()
00214             .with<ecs::Transform>("enemy_projectile_transform_" + std::to_string(projectileId), x, y, 0.0f)
00215             .with<ecs::Velocity>("enemy_projectile_velocity_" + std::to_string(projectileId), vx, vy)
00216             .with<ecs::Projectile>("enemy_projectile_" + std::to_string(projectileId), ecs::Projectile::BASIC,
00217                                 15.0f, 10.0f, 0.0f, 1)
00218             .with<ecs::Hitbox>("enemy_projectile_hitbox_" + std::to_string(projectileId), 5.0f, 0.0f, 0.0f)
00219             .build();
00220
00221     m_projectileEntities[projectileId] = projectileEntity;
00222     registerEntity(projectileEntity, ServerEntityType::PROJECTILE_ENEMY, projectileId, 10.0f, enemyId);
00223
00224     return projectileEntity;
00225 }
00226
00227 void EntityManager::destroyProjectile(std::uint32_t projectileId)
00228 {
00229     auto *metadata = getEntityMetadata(projectileId);
00230     if (metadata)
00231     {
00232         metadata->isActive = false;
00233     }
00234     m_destroyQueue.push_back(projectileId);
00235 }
00236
00237 ecs::Entity EntityManager::getProjectile(std::uint32_t projectileId)
00238 {
00239     auto it = m_projectileEntities.find(projectileId);
00240     return (it != m_projectileEntities.end()) ? it->second : ecs::INVALID_ENTITY;
00241 }
00242
00243 void EntityManager::updateLifetimes(float deltaTime)
00244 {
00245     for (auto &[networkId, metadata] : m_entityMetadata)
00246     {
00247         if (metadata.lifetime > 0.0f && metadata.isActive)
00248         {
00249             metadata.currentLife += deltaTime;
00250             if (metadata.currentLife >= metadata.lifetime)
00251             {
00252                 // Mark for destruction
00253                 m_destroyQueue.push_back(networkId);
00254             }
00255         }
00256     }
00257 }

```

```

00258
00259 void EntityManager::cleanupDestroyedEntities() { processDestroyQueue(); }
00260
00261 EntityMetadata *EntityManager::getEntityMetadata(std::uint32_t networkId)
00262 {
00263     auto it = m_entityMetadata.find(networkId);
00264     return (it != m_entityMetadata.end()) ? &it->second : nullptr;
00265 }
00266
00267 const EntityMetadata *EntityManager::getEntityMetadata(std::uint32_t networkId) const
00268 {
00269     auto it = m_entityMetadata.find(networkId);
00270     return (it != m_entityMetadata.end()) ? &it->second : nullptr;
00271 }
00272
00273 std::uint32_t EntityManager::getNetworkIdForEntity(ecs::Entity entity) const
00274 {
00275     auto it = m_entityToNetworkId.find(entity);
00276     return (it != m_entityToNetworkId.end()) ? it->second : 0;
00277 }
00278
00279 std::vector<rnp::EntityState> EntityManager::getAllEntityStates() const
00280 {
00281     std::vector<rnp::EntityState> states;
00282
00283     auto addEntityState = [&](std::uint32_t networkId, ecs::Entity entity, rnp::EntityType netType,
00284                             rnp::EntitySubtype subtype = rnp::EntitySubtype::NONE)
00285     {
00286         auto *transform = m_registry.getComponent<ecs::Transform>(entity);
00287         auto *velocity = m_registry.getComponent<ecs::Velocity>(entity);
00288
00289         if (transform && velocity)
00290         {
00291             rnp::EntityState state;
00292             state.id = networkId;
00293             state.type = static_cast<std::uint16_t>(netType);
00294             state.subtype = static_cast<std::uint8_t>(subtype);
00295             state.x = transform->x;
00296             state.y = transform->y;
00297             state.vx = velocity->x;
00298             state.vy = velocity->y;
00299
00300             // Handle health percentage
00301             state.healthPercent = 255; // Default: no health bar
00302             if (netType == rnp::EntitySubtype::ENEMY || netType == rnp::EntitySubtype::BOSS)
00303             {
00304                 auto *enemy = m_registry.getComponent<ecs::Enemy>(entity);
00305                 if (enemy && enemy->max_health > 0.0f)
00306                 {
00307                     float healthPct = (enemy->health / enemy->max_health) * 100.0f;
00308                     state.healthPercent = static_cast<std::uint8_t>(std::max(0.0f, std::min(100.0f, healthPct)));
00309                 }
00310             }
00311
00312             // Handle special state flags
00313             state.stateFlags = 0;
00314             if (netType == rnp::EntitySubtype::PLAYER)
00315             {
00316                 auto *beamCharge = m_registry.getComponent<ecs::BeamCharge>(entity);
00317                 if (beamCharge)
00318                 {
00319                     state.stateFlags = static_cast<std::uint8_t>(beamCharge->current_charge * 255.0f);
00320                 }
00321             }
00322             states.push_back(state);
00323         }
00324     };
00325
00326     for (const auto &[sessionId, entity] : m_playerEntities)
00327     {
00328         const auto *metadata = getEntityMetadata(sessionId);
00329         if (metadata && metadata->isActive)
00330         {
00331             addEntityState(sessionId, entity, rnp::EntitySubtype::PLAYER);
00332         }
00333     }
00334
00335     for (const auto &[enemyId, entity] : m_enemyEntities)
00336     {
00337         const auto *metadata = getEntityMetadata(enemyId);
00338         if (metadata && metadata->isActive)
00339         {
00340             rnp::EntityType netType;
00341             rnp::EntitySubtype subtype;
00342
00343             if (metadata->type == ServerEntityType::BOSS)

```

```

00345     {
00346         netType = rnp::EntityType::BOSS;
00347         subtype = rnp::EntitySubtype::ENEMY_BOSS;
00348     }
00349     else if (metadata->type == ServerEntityType::ENEMY_ADVANCED)
00350     {
00351         netType = rnp::EntityType::ENEMY;
00352         subtype = rnp::EntitySubtype::ENEMY_ADVANCED;
00353     }
00354     else // ENEMY_BASIC
00355     {
00356         netType = rnp::EntityType::ENEMY;
00357         subtype = rnp::EntitySubtype::ENEMY_BASIC;
00358     }
00359
00360     addEntityState(enemyId, entity, netType, subtype);
00361 }
00362 }
00363
00364 for (const auto &[projectId, entity] : m_projectileEntities)
00365 {
00366     const auto *metadata = getEntityMetadata(projectId);
00367     if (metadata && metadata->isActive)
00368     {
00369         rnp::EntitySubtype subtype = rnp::EntitySubtype::NONE;
00370
00371         if (metadata->type == ServerEntityType::PROJECTILE_PLAYER)
00372         {
00373             // Check if supercharged
00374             auto *projectile = m_registry.getComponent<ecs::Projectile>(entity);
00375             if (projectile && projectile->type == ecs::Projectile::SUPERCHARGED)
00376             {
00377                 subtype = rnp::EntitySubtype::PROJECTILE_PLAYER_SUPERCHARGED;
00378             }
00379             else
00380             {
00381                 subtype = rnp::EntitySubtype::PROJECTILE_PLAYER;
00382             }
00383         }
00384         else if (metadata->type == ServerEntityType::PROJECTILE_ENEMY)
00385         {
00386             subtype = rnp::EntitySubtype::PROJECTILE_ENEMY;
00387         }
00388
00389         addEntityState(projectId, entity, rnp::EntityType::PROJECTILE, subtype);
00390     }
00391 }
00392
00393 return states;
00394 }
00395
00396 rnp::EntityType EntityManager::getNetworkEntityType(ServerEntityType type) const
00397 {
00398     switch (type)
00399     {
00400         case ServerEntityType::PLAYER:
00401             return rnp::EntityType::PLAYER;
00402         case ServerEntityType::ENEMY_BASIC:
00403         case ServerEntityType::ENEMY_ADVANCED:
00404             return rnp::EntityType::ENEMY;
00405         case ServerEntityType::BOSS:
00406             return rnp::EntityType::BOSS;
00407         case ServerEntityType::PROJECTILE_PLAYER:
00408         case ServerEntityType::PROJECTILE_ENEMY:
00409             return rnp::EntityType::PROJECTILE;
00410         default:
00411             return rnp::EntityType::PLAYER; // Default fallback
00412     }
00413 }
00414
00415 void EntityManager::clear()
00416 {
00417     // Clear all players
00418     for (auto &[sessionId, entity] : m_playerEntities)
00419     {
00420         if (m_registry.hasComponent<ecs::Transform>(entity))
00421             m_registry.removeComponent<ecs::Transform>(entity);
00422         if (m_registry.hasComponent<ecs::Velocity>(entity))
00423             m_registry.removeComponent<ecs::Velocity>(entity);
00424         if (m_registry.hasComponent<ecs::Player>(entity))
00425             m_registry.removeComponent<ecs::Player>(entity);
00426         if (m_registry.hasComponent<ecs::BeamCharge>(entity))
00427             m_registry.removeComponent<ecs::BeamCharge>(entity);
00428         if (m_registry.hasComponent<ecs::Hitbox>(entity))
00429             m_registry.removeComponent<ecs::Hitbox>(entity);
00430     }
00431     m_playerEntities.clear();

```



```

00432
00433 // Clear all enemies
00434 for (auto &[enemyId, entity] : m_enemyEntities)
00435 {
00436     if (m_registry.hasComponent<ecs::Transform>(entity))
00437         m_registry.removeComponent<ecs::Transform>(entity);
00438     if (m_registry.hasComponent<ecs::Velocity>(entity))
00439         m_registry.removeComponent<ecs::Velocity>(entity);
00440     if (m_registry.hasComponent<ecs::Enemy>(entity))
00441         m_registry.removeComponent<ecs::Enemy>(entity);
00442     if (m_registry.hasComponent<ecs::Hitbox>(entity))
00443         m_registry.removeComponent<ecs::Hitbox>(entity);
00444 }
00445 m_enemyEntities.clear();
00446
00447 // Clear all projectiles
00448 for (auto &[projectileId, entity] : m_projectileEntities)
00449 {
00450     if (m_registry.hasComponent<ecs::Transform>(entity))
00451         m_registry.removeComponent<ecs::Transform>(entity);
00452     if (m_registry.hasComponent<ecs::Velocity>(entity))
00453         m_registry.removeComponent<ecs::Velocity>(entity);
00454     if (m_registry.hasComponent<ecs::Projectile>(entity))
00455         m_registry.removeComponent<ecs::Projectile>(entity);
00456     if (m_registry.hasComponent<ecs::Hitbox>(entity))
00457         m_registry.removeComponent<ecs::Hitbox>(entity);
00458 }
00459 m_projectileEntities.clear();
00460
00461 // Clear all powerups
00462 for (auto &[powerupId, entity] : m_powerupEntities)
00463 {
00464     if (m_registry.hasComponent<ecs::Transform>(entity))
00465         m_registry.removeComponent<ecs::Transform>(entity);
00466     if (m_registry.hasComponent<ecs::Velocity>(entity))
00467         m_registry.removeComponent<ecs::Velocity>(entity);
00468     if (m_registry.hasComponent<ecs::Hitbox>(entity))
00469         m_registry.removeComponent<ecs::Hitbox>(entity);
00470 }
00471 m_powerupEntities.clear();
00472
00473 m_entityMetadata.clear();
00474 m_entityToNetworkId.clear();
00475 m_destroyQueue.clear();
00476 }
00477
00478 size_t EntityManager::getTotalEntityCount() const
00479 {
00480     return m_playerEntities.size() + m_enemyEntities.size() + m_projectileEntities.size() +
00481         m_powerupEntities.size();
00482 }
00483
00484 void EntityManager::registerEntity(ecs::Entity entity, ServerEntityType type, std::uint32_t networkId,
00485     float lifetime, std::uint32_t ownerId)
00486 {
00487     EntityMetadata metadata;
00488     metadata.entity = entity;
00489     metadata.type = type;
00490     metadata.networkId = networkId;
00491     metadata.isActive = true;
00492     metadata.lifetime = lifetime;
00493     metadata.currentLife = 0.0f;
00494     metadata.ownerId = ownerId;
00495
00496     m_entityMetadata[networkId] = metadata;
00497     m_entityToNetworkId[entity] = networkId;
00498 }
00499
00500 void EntityManager::unregisterEntity(std::uint32_t networkId)
00501 {
00502     auto it = m_entityMetadata.find(networkId);
00503     if (it != m_entityMetadata.end())
00504     {
00505         m_entityToNetworkId.erase(it->second.entity);
00506         m_entityMetadata.erase(it);
00507     }
00508 }
00509
00510 void EntityManager::processDestroyQueue()
00511 {
00512     for (std::uint32_t networkId : m_destroyQueue)
00513     {
00514         // Check which container this entity belongs to
00515         auto enemyIt = m_enemyEntities.find(networkId);
00516         if (enemyIt != m_enemyEntities.end())
00517         {
00518             ecs::Entity entity = enemyIt->second;

```

```

00519         if (m_registry.hasComponent<ecs::Transform>(entity))
00520             m_registry.removeComponent<ecs::Transform>(entity);
00521         if (m_registry.hasComponent<ecs::Velocity>(entity))
00522             m_registry.removeComponent<ecs::Velocity>(entity);
00523         if (m_registry.hasComponent<ecs::Enemy>(entity))
00524             m_registry.removeComponent<ecs::Enemy>(entity);
00525         if (m_registry.hasComponent<ecs::Hitbox>(entity))
00526             m_registry.removeComponent<ecs::Hitbox>(entity);
00527
00528         unregisterEntity(networkId);
00529         m_enemyEntities.erase(enemyIt);
00530         continue;
00531     }
00532
00533     auto projIt = m_projectileEntities.find(networkId);
00534     if (projIt != m_projectileEntities.end())
00535     {
00536         ecs::Entity entity = projIt->second;
00537         if (m_registry.hasComponent<ecs::Transform>(entity))
00538             m_registry.removeComponent<ecs::Transform>(entity);
00539         if (m_registry.hasComponent<ecs::Velocity>(entity))
00540             m_registry.removeComponent<ecs::Velocity>(entity);
00541         if (m_registry.hasComponent<ecs::Projectile>(entity))
00542             m_registry.removeComponent<ecs::Projectile>(entity);
00543         if (m_registry.hasComponent<ecs::Hitbox>(entity))
00544             m_registry.removeComponent<ecs::Hitbox>(entity);
00545
00546         unregisterEntity(networkId);
00547         m_projectileEntities.erase(projIt);
00548         continue;
00549     }
00550
00551     auto powerupIt = m_powerupEntities.find(networkId);
00552     if (powerupIt != m_powerupEntities.end())
00553     {
00554         ecs::Entity entity = powerupIt->second;
00555         if (m_registry.hasComponent<ecs::Transform>(entity))
00556             m_registry.removeComponent<ecs::Transform>(entity);
00557         if (m_registry.hasComponent<ecs::Velocity>(entity))
00558             m_registry.removeComponent<ecs::Velocity>(entity);
00559         if (m_registry.hasComponent<ecs::Hitbox>(entity))
00560             m_registry.removeComponent<ecs::Hitbox>(entity);
00561
00562         unregisterEntity(networkId);
00563         m_powerupEntities.erase(powerupIt);
00564         continue;
00565     }
00566 }
00567
00568 m_destroyQueue.clear();
00569 }
00570
00571 } // namespace gme

```

19.259 /home/masina/Projects/Epitech/rtype/plugins/Game/RType/↵ Server/src/rtypeServer.cpp File Reference

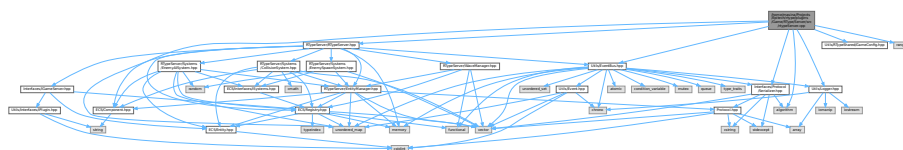
Implementation of RTypeServer with new entity management system.

```

#include "RTypeServer/RTypeServer.hpp"
#include "ECS/Component.hpp"
#include "Interfaces/Protocol/Serializer.hpp"
#include "Utils/RTypeShared/GameConfig.hpp"
#include "Utils/EventBus.hpp"
#include "Utils/Logger.hpp"
#include <algorithm>
#include <ranges>

```

Include dependency graph for rtypeServer.cpp:



Namespaces

- namespace `gme`

19.259.1 Detailed Description

Implementation of RTypeServer with new entity management system.

Definition in file [rtypeServer.cpp](#).

19.260 rtypeServer.cpp

[Go to the documentation of this file.](#)

```

00001 ///  

00002 ///  

00003 ///  

00004 ///  

00005 ///  

00006 ///  

00007 #include "RTypeServer/RTypeServer.hpp"  

00008 #include "ECS/Component.hpp"  

00009 #include "Interfaces/Protocol/Serializer.hpp"  

00010 #include "Utils/RTypeShared/GameConfig.hpp"  

00011 #include "Utils/EventBus.hpp"  

00012 #include "Utils/Logger.hpp"  

00013 #include <algorithm>  

00014 #include <ranges>  

00015 ///  

00016 namespace gme  

00017 {  

00018     RTypeServer::RTypeServer() : m_eventBus(utl::EventBus::getInstance())  

00019     {  

00020         // Register this component with the event bus  

00021         m_eventBus.registerComponent(utl::GAME_LOGIC, "RTypeServer");  

00022         // Subscribe to events  

00023         m_eventBus.subscribe(utl::GAME_LOGIC, utl::EventType::SERVER_START);  

00024         m_eventBus.subscribe(utl::GAME_LOGIC, utl::EventType::PLAYER_INPUT_RECEIVED);  

00025         m_eventBus.subscribe(utl::GAME_LOGIC, utl::EventType::GAME_START);  

00026         // Initialize entity manager  

00027         m_entityManager = std::make_unique<EntityManager>(m_registry);  

00028         // Initialize game systems  

00029         m_collisionSystem = std::make_unique<CollisionSystem>(m_registry, *m_entityManager);  

00030         m_enemyAISystem = std::make_unique<EnemyAISystem>(m_registry, *m_entityManager);  

00031         m_enemySpawnSystem = std::make_unique<EnemySpawnSystem>(*m_entityManager);  

00032         // Initialize wave manager with default waves  

00033         m_waveManager = std::make_unique<WaveManager>(*m_entityManager);  

00034         m_waveManager->setupDefaultWaves();  

00035         utl::Logger::log("RTypeServer: Initialized with entity management system", utl::LogLevel::INFO);  

00036     }  

00037     void RTypeServer::start()  

00038     {  

00039         m_gameState = State::PLAYING;  

00040         m_levelState = LevelState::WAITING_FOR_PLAYERS;  

00041         // Disable old enemy spawning system, use wave manager instead  

00042         m_enemySpawnSystem->setEnabled(false);  

00043         // DON'T start wave manager here - wait for GAME_START event from waiting room  

00044         utl::Logger::log("RTypeServer: Server started - waiting for game to begin", utl::LogLevel::INFO);  

00045     }  

00046     void RTypeServer::stop()  

00047     {  

00048         m_gameState = State::LOSE;  

00049         // Clear all entities  

00050         m_entityManager->clear();  

00051         // Reset systems  

00052         m_enemySpawnSystem->reset();  

00053         m_waveManager->reset();  

00054         // Clear player tracking  

00055         m_playerShooting.clear();  

00056         m_lastShotTime.clear();  

00057         utl::Logger::log("RTypeServer: Stopped", utl::LogLevel::INFO);  

00058     }  

00059 }

```

```

00071     }
00072
00073 void RTypeServer::update(const float deltaTime)
00074 {
00075     m_lastBroadcastTime += deltaTime;
00076
00077     auto events = m_eventBus.consumeForTarget(utl::GAME_LOGIC);
00078     for (const auto &event : events)
00079     {
00080         if (event.type == utl::EventType::SERVER_START)
00081         {
00082             processServerStartEvent(event);
00083         }
00084         else if (event.type == utl::EventType::PLAYER_INPUT_RECEIVED)
00085         {
00086             processPlayerInputEvent(event);
00087         }
00088         else if (event.type == utl::EventType::GAME_START)
00089         {
00090             processGameStartEvent(event);
00091         }
00092     }
00093
00094     // Update all game systems
00095     updateSystems(deltaTime);
00096
00097     // Update entity physics
00098     updateEntities(deltaTime);
00099
00100     // Update entity lifetimes (for projectiles with limited lifetime)
00101     m_entityManager->updateLifetimes(deltaTime);
00102
00103     // Clean up destroyed entities
00104     m_entityManager->cleanupDestroyedEntities();
00105
00106     // Broadcast world state at 60 Hz
00107     if (m_lastBroadcastTime >= BROADCAST_INTERVAL)
00108     {
00109         broadcastWorldState();
00110         m_lastBroadcastTime = 0.0f;
00111     }
00112 }
00113
00114 void RTypeServer::processServerStartEvent(const utl::Event &event)
00115 {
00116     // Create player entities from sessionIds
00117     if (event.data.size() >= sizeof(std::uint32_t))
00118     {
00119         size_t playerCount = event.data.size() / sizeof(std::uint32_t);
00120
00121         for (size_t i = 0; i < playerCount; ++i)
00122         {
00123             std::uint32_t sessionId;
00124             std::memcpy(&sessionId, event.data.data() + i * sizeof(std::uint32_t), sizeof(std::uint32_t));
00125
00126             if (!m_entityManager->hasPlayer(sessionId))
00127             {
00128                 float startX = 200.0f + (i * 200.0f);
00129                 float startY = 100.0f + (i * 100.0f);
00130
00131                 m_entityManager->createPlayer(sessionId, startX, startY);
00132                 m_playerShooting[sessionId] = false;
00133                 m_lastShotTime[sessionId] = 0.0f;
00134
00135                 utl::Logger::log("RTypeServer: Created player entity for sessionId " + std::to_string(sessionId),
00136                               utl::LogLevel::INFO);
00137             }
00138         }
00139
00140         // Start enemy spawning when game starts
00141         if (m_levelState == LevelState::WAITING_FOR_PLAYERS)
00142         {
00143             m_levelState = LevelState::IN_PROGRESS;
00144             m_enemySpawnSystem->setEnabled(true);
00145             utl::Logger::log("RTypeServer: Game started, enemy spawning enabled", utl::LogLevel::INFO);
00146         }
00147     }
00148 }
00149
00150 void RTypeServer::processPlayerInputEvent(const utl::Event &event)
00151 {
00152     try
00153     {
00154         // Input data comes as: [up, down, left, right, shoot]
00155         if (event.data.size() >= 5)
00156         {
00157             bool up = event.data[0] != 0;

```

```

00158     bool down = event.data[1] != 0;
00159     bool left = event.data[2] != 0;
00160     bool right = event.data[3] != 0;
00161     bool shoot = event.data[4] != 0;
00162
00163     std::uint32_t sessionId = event.sourceId;
00164
00165     // Create player entity if it doesn't exist
00166     if (!m_entityManager->hasPlayer(sessionId))
00167     {
00168         // Spread players vertically to avoid spawn overlap
00169         float spawnX = 200.0f;
00170         float spawnY = 200.0f + ((sessionId % 4) * 200.0f);
00171         m_entityManager->createPlayer(sessionId, spawnX, spawnY);
00172         m_playerShooting[sessionId] = false;
00173         m_lastShotTime[sessionId] = 0.0f;
00174     }
00175
00176     // Apply input to player entity
00177     if (sessionId != 0 && m_entityManager->hasPlayer(sessionId))
00178     {
00179         ecs::Entity playerEntity = m_entityManager->getPlayer(sessionId);
00180         auto *velocity = m_registry.getComponent<ecs::Velocity>(playerEntity);
00181         auto *transform = m_registry.getComponent<ecs::Transform>(playerEntity);
00182
00183         if (velocity && transform)
00184         {
00185             // Calculate velocity based on input
00186             const float SPEED = 500.0f;
00187             const float DIAGONAL_SPEED = SPEED * 0.707f;
00188
00189             velocity->x = 0.0f;
00190             velocity->y = 0.0f;
00191
00192             // Handle diagonal movement
00193             if (up && right)
00194             {
00195                 velocity->x = DIAGONAL_SPEED;
00196                 velocity->y = -DIAGONAL_SPEED;
00197             }
00198             else if (up && left)
00199             {
00200                 velocity->x = -DIAGONAL_SPEED;
00201                 velocity->y = -DIAGONAL_SPEED;
00202             }
00203             else if (down && right)
00204             {
00205                 velocity->x = DIAGONAL_SPEED;
00206                 velocity->y = DIAGONAL_SPEED;
00207             }
00208             else if (down && left)
00209             {
00210                 velocity->x = -DIAGONAL_SPEED;
00211                 velocity->y = DIAGONAL_SPEED;
00212             }
00213             else
00214             {
00215                 // Handle cardinal directions
00216                 if (up)
00217                     velocity->y = -SPEED;
00218                 if (down)
00219                     velocity->y = SPEED;
00220                 if (left)
00221                     velocity->x = -SPEED;
00222                 if (right)
00223                     velocity->x = SPEED;
00224             }
00225
00226             // Track shooting state
00227             m_playerShooting[sessionId] = shoot;
00228         }
00229     }
00230 }
00231 }
00232 catch (const std::exception &e)
00233 {
00234     utl::Logger::log("RTypeServer: Error processing input: " + std::string(e.what()), utl::LogLevel::WARNING);
00235 }
00236 }
00237
00238 void RTypeServer::processGameStartEvent(const utl::Event &event)
00239 {
00240     (void)event; // Event data contains lobby info if needed
00241
00242     utl::Logger::log("RTypeServer: GAME_START event received - starting waves!", utl::LogLevel::INFO);
00243
00244     // Start the wave manager now that the game is actually starting

```

```

00245     if (!m_waveManager->isActive())
00246     {
00247         m_waveManager->start();
00248         m_levelState = LevelState::IN_PROGRESS;
00249         utl::Logger::log("RTypeServer: Wave system started - 3 waves of 30 seconds", utl::LogLevel::INFO);
00250     }
00251     else
00252     {
00253         utl::Logger::log("RTypeServer: Wave system already active", utl::LogLevel::WARNING);
00254     }
00255 }
00256
00257 void RTypeServer::updateSystems(float deltaTime)
00258 {
00259     // Update all game systems in order
00260
00261     m_enemyAISystem->update(m_registry, deltaTime);
00262
00263     // Update wave manager instead of simple enemy spawning
00264     // Assuming 1920 as default screen width (can be made configurable later)
00265     m_waveManager->update(m_registry, deltaTime, 1920);
00266
00267     for (auto &[sessionId, _] : m_entityManager->getPlayers())
00268     {
00269         handlePlayerShooting(sessionId, deltaTime);
00270     }
00271
00272     m_collisionSystem->update(m_registry, deltaTime);
00273 }
00274
00275 void RTypeServer::handlePlayerShooting(std::uint32_t sessionId, float deltaTime)
00276 {
00277     if (m_lastShotTime.find(sessionId) == m_lastShotTime.end())
00278     {
00279         m_lastShotTime[sessionId] = 0.0f;
00280     }
00281
00282     m_lastShotTime[sessionId] += deltaTime;
00283
00284     ecs::Entity playerEntity = m_entityManager->getPlayer(sessionId);
00285     if (playerEntity == ecs::INVALID_ENTITY)
00286         return;
00287
00288     auto *beamCharge = m_registry.getComponent<ecs::BeamCharge>(playerEntity);
00289     auto *transform = m_registry.getComponent<ecs::Transform>(playerEntity);
00290
00291     if (!beamCharge || !transform)
00292         return;
00293
00294     bool isShooting = m_playerShooting.find(sessionId) != m_playerShooting.end() && m_playerShooting[sessionId];
00295
00296     if (isShooting)
00297     {
00298         const float CHARGE_RATE = 2.0f;
00299         beamCharge->current_charge += CHARGE_RATE * deltaTime;
00300         if (beamCharge->current_charge > beamCharge->max_charge)
00301             beamCharge->current_charge = beamCharge->max_charge;
00302     }
00303     else if (beamCharge->current_charge > 0.01f)
00304     {
00305         if (m_lastShotTime[sessionId] >= PROJECTILE_COOLDOWN)
00306         {
00307             const float PLAYER_SPRITE_WIDTH = 33.0f;
00308             const float PLAYER_SPRITE_HEIGHT = 17.0f;
00309             const float PLAYER_SCALE = 2.0f;
00310             const float PLAYER_WIDTH = PLAYER_SPRITE_WIDTH * PLAYER_SCALE;
00311             const float PLAYER_HEIGHT = PLAYER_SPRITE_HEIGHT * PLAYER_SCALE;
00312
00313             float projectileX = transform->x + PLAYER_WIDTH + 10.0f;
00314             float projectileY = transform->y + PLAYER_HEIGHT / 2.0f;
00315
00316             bool isSupercharged = beamCharge->current_charge >= 0.5f;
00317             float projectileSpeed = isSupercharged ? 1200.0f : 800.0f;
00318
00319             m_entityManager->createPlayerProjectile(sessionId, projectileX, projectileY, projectileSpeed, 0.0f,
00320                 isSupercharged);
00321
00322             beamCharge->current_charge = 0.0f;
00323             m_lastShotTime[sessionId] = 0.0f;
00324         }
00325     }
00326 }
00327
00328 void RTypeServer::updateEntities(float deltaTime)
00329 {
00330     const auto &players = m_entityManager->getPlayers();
00331     for (const auto &[sessionId, entity] : players)

```

```

00332     {
00333         auto *transform = m_registry.getComponent<ecs::Transform>(entity);
00334         auto *velocity = m_registry.getComponent<ecs::Velocity>(entity);
00335         auto *hitbox = m_registry.getComponent<ecs::Hitbox>(entity);
00336
00337         if (transform && velocity)
00338         {
00339             transform->x += velocity->x * deltaTime;
00340             transform->y += velocity->y * deltaTime;
00341
00342             if (hitbox)
00343             {
00344                 const float PLAYER_SPRITE_WIDTH = 33.0f;
00345                 const float PLAYER_SPRITE_HEIGHT = 17.0f;
00346                 const float PLAYER_SCALE = 2.0f;
00347                 const float PLAYER_WIDTH = PLAYER_SPRITE_WIDTH * PLAYER_SCALE;
00348                 const float PLAYER_HEIGHT = PLAYER_SPRITE_HEIGHT * PLAYER_SCALE;
00349                 const float PLAYER_HITBOX_RADIUS = hitbox->radius;
00350
00351                 float minX = PLAYER_HITBOX_RADIUS;
00352                 float maxX = utl::GameConfig::Server::SCREEN_WIDTH - (PLAYER_WIDTH -
PLAYER_HITBOX_RADIUS);
00353                 float minY = PLAYER_HITBOX_RADIUS;
00354                 float maxY = utl::GameConfig::Server::SCREEN_HEIGHT - (PLAYER_HEIGHT -
PLAYER_HITBOX_RADIUS);
00355
00356                 transform->x = std::max(minX, std::min(transform->x, maxX));
00357                 transform->y = std::max(minY, std::min(transform->y, maxY));
00358             }
00359             else
00360             {
00361                 transform->x = std::max(0.0f, std::min(transform->x, utl::GameConfig::Server::SCREEN_WIDTH));
00362                 transform->y = std::max(0.0f, std::min(transform->y, utl::GameConfig::Server::SCREEN_HEIGHT));
00363             }
00364         }
00365     }
00366
00367     // Update enemy positions
00368     const auto &enemies = m_entityManager->getEnemies();
00369     for (const auto &[enemyId, entity] : enemies)
00370     {
00371         // Skip inactive entities
00372         const auto *metadata = m_entityManager->getEntityMetadata(enemyId);
00373         if (!metadata || !metadata->isActive)
00374             continue;
00375
00376         auto *transform = m_registry.getComponent<ecs::Transform>(entity);
00377         auto *velocity = m_registry.getComponent<ecs::Velocity>(entity);
00378
00379         if (transform && velocity)
00380         {
00381             transform->x += velocity->x * deltaTime;
00382             transform->y += velocity->y * deltaTime;
00383
00384             // Destroy enemies that are too far off-screen
00385             if (transform->x < -utl::GameConfig::Server::WORLD_MARGIN ||
transform->x > utl::GameConfig::Server::SCREEN_WIDTH +
utl::GameConfig::Server::WORLD_MARGIN ||
00387                 transform->y < -utl::GameConfig::Server::WORLD_MARGIN ||
transform->y > utl::GameConfig::Server::SCREEN_HEIGHT +
utl::GameConfig::Server::WORLD_MARGIN)
00389             {
00390                 m_entityManager->destroyEnemy(enemyId);
00391             }
00392         }
00393     }
00394
00395     const auto &projectiles = m_entityManager->getProjectiles();
00396     for (const auto &[projectId, entity] : projectiles)
00397     {
00398         const auto *metadata = m_entityManager->getEntityMetadata(projectId);
00399         if (!metadata || !metadata->isActive)
00400             continue;
00401
00402         auto *transform = m_registry.getComponent<ecs::Transform>(entity);
00403         auto *velocity = m_registry.getComponent<ecs::Velocity>(entity);
00404
00405         if (transform && velocity)
00406         {
00407             transform->x += velocity->x * deltaTime;
00408             transform->y += velocity->y * deltaTime;
00409
00410             // Remove projectiles that are off-screen (with margin)
00411             if (transform->x > utl::GameConfig::Server::SCREEN_WIDTH +
utl::GameConfig::Server::WORLD_MARGIN ||
00412                 transform->x < -utl::GameConfig::Server::WORLD_MARGIN ||
00413                 transform->y > utl::GameConfig::Server::SCREEN_HEIGHT +

```



```

00414     utl::GameConfig::Server::WORLD_MARGIN ||
00415         {
00416             transform->y < -utl::GameConfig::Server::WORLD_MARGIN)
00417         {
00418             m_entityManager->destroyProjectile(projectileId);
00419         }
00420     }
00421 }
00422 void RTypeServer::broadcastWorldState()
00423 {
00424     try
00425     {
00426         rnp::PacketWorldState worldState;
00427         worldState.serverTick = static_cast<std::uint32_t>(m_lastBroadcastTime * 60.0f);
00428
00429         // Get all entity states from EntityManager
00430         worldState.entities = m_entityManager->getAllEntityStates();
00431         worldState.entityCount = static_cast<std::uint16_t>(worldState.entities.size());
00432
00433         // Create full packet with header for each client
00434         const auto &players = m_entityManager->getPlayers();
00435
00436         for (const auto &[sessionId, playerEntity] : players)
00437         {
00438             // Create packet with header for this session
00439             rnp::Serializer packetSerializer;
00440
00441             // Create header
00442             rnp::PacketHeader header;
00443             header.type = static_cast<std::uint8_t>(rnp::PacketType::WORLD_STATE);
00444
00445             // Serialize world state to calculate length
00446             rnp::Serializer tempSerializer;
00447             tempSerializer.serializeWorldState(worldState);
00448             header.length = static_cast<std::uint16_t>(tempSerializer.getData().size());
00449             header.sessionId = sessionId;
00450
00451             // Serialize header and world state
00452             packetSerializer.serializeHeader(header);
00453             packetSerializer.serializeWorldState(worldState);
00454
00455             std::vector<std::uint8_t> packet = packetSerializer.getData();
00456
00457             // Send packet via EventBus
00458             utl::Event broadcastEvent(utl::EventType::SEND_TO_CLIENT, 1, utl::NETWORK_SERVER);
00459             std::vector<std::uint8_t> eventData(sizeof(std::uint32_t) + packet.size());
00460             std::memcpy(eventData.data(), &sessionId, sizeof(std::uint32_t));
00461             std::memcpy(eventData.data() + sizeof(std::uint32_t), packet.data(), packet.size());
00462             broadcastEvent.data = eventData;
00463             m_eventBus.publish(broadcastEvent);
00464         }
00465     }
00466     catch (const std::exception &e)
00467     {
00468         utl::Logger::log("RTypeServer: Error broadcasting world state: " + std::string(e.what()),
00469             utl::LogLevel::WARNING);
00470     }
00471 }
00472
00473 } // namespace gme

```

19.261 /home/masina/Projects/Epitech/rtype/plugins/Game/RType/↵ Server/src/Systems/CollisionSystem.cpp File Reference

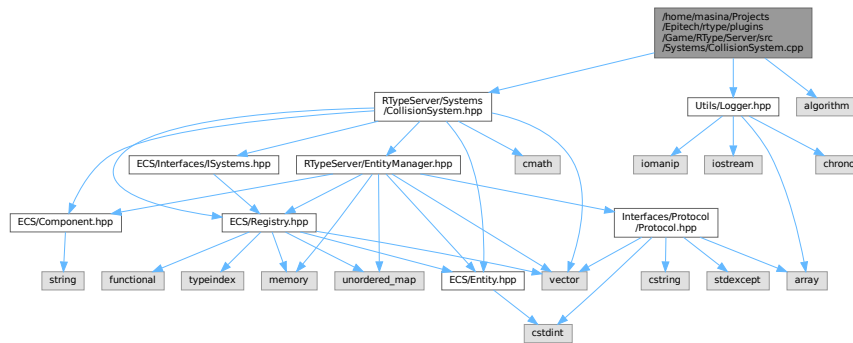
Implementation of server-side collision detection system.

#include "RTypeServer/Systems/CollisionSystem.hpp"

#include "Utils/Logger.hpp"

#include <algorithm>

Include dependency graph for CollisionSystem.cpp:



Namespaces

- namespace `gme`

19.261.1 Detailed Description

Implementation of server-side collision detection system.

Definition in file [CollisionSystem.cpp](#).

19.262 CollisionSystem.cpp

[Go to the documentation of this file.](#)

```
00001 ///
00002 /// @file CollisionSystem.cpp
00003 /// @brief Implementation of server-side collision detection system
00004 /// @namespace gme
00005 ///
00006
00007 #include "RTypeServer/Systems/CollisionSystem.hpp"
00008 #include "Utils/Logger.hpp"
00009 #include <algorithm>
00010
00011 namespace gme
00012 {
00013     void CollisionSystem::update(ecs::Registry &registry, float deltaTime)
00014     {
00015         (void)registry; // Use member registry instead
00016         (void)deltaTime; // Collision detection is instantaneous
00017
00018         m_collisionCount = 0;
00019
00020         // Check all collision types
00021         handlePlayerProjectileEnemyCollision();
00022         handleEnemyProjectilePlayerCollision();
00023         handlePlayerEnemyCollision();
00024         handlePlayerPowerupCollision();
00025     }
00026
00027 void CollisionSystem::handlePlayerProjectileEnemyCollision()
00028 {
00029     const auto &projectiles = m_entityManager.getProjectiles();
00030     const auto &enemies = m_entityManager.getEnemies();
00031
00032     for (const auto &[projId, projEntity] : projectiles)
00033     {
00034         // Check if this is a player projectile
00035         const auto *projMetadata = m_entityManager.getEntityMetadata(projId);
00036         if (!projMetadata || projMetadata->type != ServerEntityType::PROJECTILE_PLAYER)
00037             continue;
00038
00039         // Skip if projectile is already marked for destruction
00040         if (!projMetadata->isActive)
00041             continue;
00042
00043         auto *projProjectile = m_registry.getComponent<ecs::Projectile>(projEntity);
00044         if (!projProjectile || projProjectile->pierce_remaining <= 0)
```

```

00045         continue;
00046
00047     float projX, projY, projRadius;
00048     if (!getCollisionInfo(projEntity, projX, projY, projRadius))
00049         continue;
00050
00051     // Check collision with all enemies
00052     for (const auto &[enemyId, enemyEntity] : enemies)
00053     {
00054         const auto *enemyMetadata = m_entityManager.getEntityMetadata(enemyId);
00055         if (!enemyMetadata || !enemyMetadata->isActive)
00056             continue;
00057
00058         float enemyX, enemyY, enemyRadius;
00059         if (!getCollisionInfo(enemyEntity, enemyX, enemyY, enemyRadius))
00060             continue;
00061
00062         float overlapDist;
00063         if (checkCircleCollision(projX, projY, projRadius, enemyX, enemyY, enemyRadius, &overlapDist))
00064         {
00065             // Collision detected!
00066             m_collisionCount++;
00067
00068             // Apply damage to enemy
00069             applyDamageToEnemy(enemyId, projProjectile->damage);
00070
00071             // Reduce pierce count
00072             projProjectile->pierce_remaining--;
00073
00074             // Collision logged only for debugging (too verbose at INFO level)
00075             // utl::Logger::log("CollisionSystem: Projectile " + std::to_string(projId) + " hit enemy " +
00076             //                  std::to_string(enemyId) + " (pierce remaining: " +
00077             //                  std::to_string(projProjectile->pierce_remaining) + ")");
00078
00079             // If projectile has no more pierce, mark for destruction
00080             if (projProjectile->pierce_remaining <= 0)
00081             {
00082                 m_entityManager.destroyProjectile(projId);
00083                 break; // Stop checking other enemies
00084             }
00085         }
00086     }
00087 }
00088 }
00089
00090 void CollisionSystem::handleEnemyProjectilePlayerCollision()
00091 {
00092     const auto &projectiles = m_entityManager.getProjectiles();
00093     const auto &players = m_entityManager.getPlayers();
00094
00095     for (const auto &[projId, projEntity] : projectiles)
00096     {
00097         // Check if this is an enemy projectile
00098         const auto *projMetadata = m_entityManager.getEntityMetadata(projId);
00099         if (!projMetadata || projMetadata->type != ServerEntityType::PROJECTILE_ENEMY)
00100             continue;
00101
00102         if (!projMetadata->isActive)
00103             continue;
00104
00105         float projX, projY, projRadius;
00106         if (!getCollisionInfo(projEntity, projX, projY, projRadius))
00107             continue;
00108
00109         auto *projProjectile = m_registry.getComponent<ecs::Projectile>(projEntity);
00110         if (!projProjectile)
00111             continue;
00112
00113         // Check collision with all players
00114         for (const auto &[playerId, playerEntity] : players)
00115         {
00116             float playerX, playerY, playerRadius;
00117             if (!getCollisionInfo(playerEntity, playerX, playerY, playerRadius))
00118                 continue;
00119
00120             if (checkCircleCollision(projX, projY, projRadius, playerX, playerY, playerRadius))
00121             {
00122                 // Collision detected!
00123                 m_collisionCount++;
00124
00125                 // Apply damage to player
00126                 applyDamageToPlayer(playerId, projProjectile->damage);
00127
00128                 // Destroy projectile
00129                 m_entityManager.destroyProjectile(projId);
00130
00131                 // Collision logged only for debugging (too verbose at INFO level)

```

```

00132         // utl::Logger::log("CollisionSystem: Enemy projectile " + std::to_string(projId) + " hit player " +
00133         //                 std::to_string(playerId));
00134
00135         break; // Stop checking other players
00136     }
00137 }
00138 }
00139 }
00140
00141 void CollisionSystem::handlePlayerEnemyCollision()
00142 {
00143     const auto &players = m_entityManager.getPlayers();
00144     const auto &enemies = m_entityManager.getEnemies();
00145
00146     for (const auto &[playerId, playerEntity] : players)
00147     {
00148         float playerX, playerY, playerRadius;
00149         if (!getCollisionInfo(playerEntity, playerX, playerY, playerRadius))
00150             continue;
00151
00152         // Check collision with all enemies
00153         for (const auto &[enemyId, enemyEntity] : enemies)
00154         {
00155             const auto *enemyMetadata = m_entityManager.getEntityMetadata(enemyId);
00156             if (!enemyMetadata || !enemyMetadata->isActive)
00157                 continue;
00158
00159             float enemyX, enemyY, enemyRadius;
00160             if (!getCollisionInfo(enemyEntity, enemyX, enemyY, enemyRadius))
00161                 continue;
00162
00163             if (checkCircleCollision(playerX, playerY, playerRadius, enemyX, enemyY, enemyRadius))
00164             {
00165                 // Collision detected!
00166                 m_collisionCount++;
00167
00168                 // Get enemy damage
00169                 auto *enemy = m_registry.getComponent<ecs::Enemy>(enemyEntity);
00170                 float damage = enemy ? enemy->damage : 10.0f;
00171
00172                 // Apply damage to player
00173                 applyDamageToPlayer(playerId, damage);
00174
00175                 // Apply damage to enemy (ram damage)
00176                 applyDamageToEnemy(enemyId, 20.0f);
00177
00178                 // Collision logged only for debugging (too verbose at INFO level)
00179                 // utl::Logger::log("CollisionSystem: Player " + std::to_string(playerId) + " collided with enemy "
00180                 // +
00181                 //                 std::to_string(enemyId));
00182             }
00183         }
00184     }
00185 }
00186
00187 void CollisionSystem::handlePlayerPowerupCollision()
00188 {
00189     // TODO: Implement when powerup collection is needed
00190     // For now, this is a placeholder
00191 }
00192
00193 void CollisionSystem::applyDamageToEnemy(std::uint32_t enemyId, float damage)
00194 {
00195     ecs::Entity enemyEntity = m_entityManager.getEntity(enemyId);
00196     if (enemyEntity == ecs::INVALID_ENTITY)
00197         return;
00198
00199     auto *enemy = m_registry.getComponent<ecs::Enemy>(enemyEntity);
00200     if (!enemy)
00201         return;
00202
00203     enemy->health -= damage;
00204
00205     // Damage logged only for debugging (too verbose at INFO level)
00206     // utl::Logger::log("CollisionSystem: Enemy " + std::to_string(enemyId) + " took " + std::to_string(damage) +
00207     //                 " damage (health: " + std::to_string(enemy->health) + "/" +
00208     //                 std::to_string(enemy->max_health) + ")");
00209
00210     // Check if enemy is dead
00211     if (enemy->health <= 0.0f)
00212     {
00213         utl::Logger::log("CollisionSystem: Enemy " + std::to_string(enemyId) + " destroyed", utl::LogLevel::INFO);
00214         m_entityManager.destroyEntity(enemyId);
00215
00216         // TODO: Add score/rewards here
00217         // TODO: Potentially spawn powerup
00218     }

```

```

00219     }
00220
00221 void CollisionSystem::applyDamageToPlayer(std::uint32_t playerId, float damage)
00222 {
00223     // For now, just log the damage
00224     // TODO: Implement player health system
00225     // Damage logged only for debugging (too verbose at INFO level)
00226     // utl::Logger::log("CollisionSystem: Player " + std::to_string(playerId) + " took " + std::to_string(damage) +
00227     //                 " damage");
00228
00229     // TODO: When health system is implemented:
00230     // - Reduce player health
00231     // - If health <= 0, handle player death
00232     // - Broadcast player damage event to clients
00233 }
00234
00235 } // namespace gme

```

19.263 /home/masina/Projects/Epitech/rtype/plugins/Game/RType/Server/src/Systems/EnemyAISystem.cpp File Reference

Implementation of enemy AI and behavior system.

```
#include "RTypeServer/Systems/EnemyAISystem.hpp"
```

```
#include "Utils/RTypeShared/GameConfig.hpp"
```

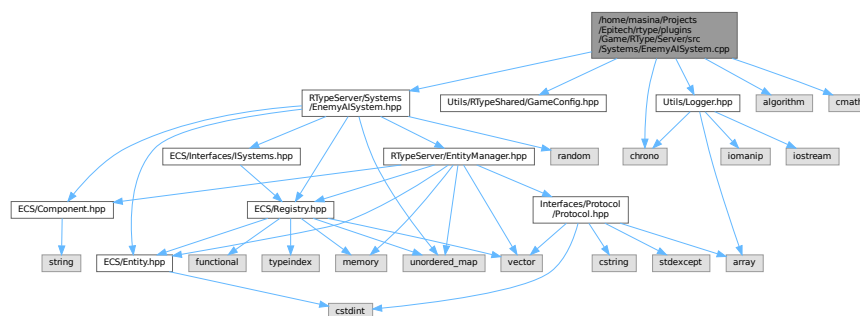
```
#include "Utils/Logger.hpp"
```

```
#include <algorithm>
```

```
#include <chrono>
```

```
#include <cmath>
```

Include dependency graph for EnemyAISystem.cpp:



Namespaces

- namespace [gme](#)

19.263.1 Detailed Description

Implementation of enemy AI and behavior system.

Definition in file [EnemyAISystem.cpp](#).

19.264 EnemyAISystem.cpp

[Go to the documentation of this file.](#)

```

00001 ///
00002 /// @file EnemyAISystem.cpp
00003 /// @brief Implementation of enemy AI and behavior system
00004 /// @namespace gme
00005 ///
00006
00007 #include "RTypeServer/Systems/EnemyAISystem.hpp"
00008 #include "Utils/RTypeShared/GameConfig.hpp"
00009 #include "Utils/Logger.hpp"
00010 #include <algorithm>

```

```

00011 #include <chrono>
00012 #include <cmath>
00013
00014 namespace gme
00015 {
00016     EnemyAISystem::EnemyAISystem(ecs::Registry &registry, EntityManager &entityManager)
00017         : m_registry(registry), m_entityManager(entityManager), m_aggressiveness(1.0f), m_shootChance(0.0f, 1.0f)
00018     {
00019         auto seed = static_cast<unsigned int>(std::chrono::system_clock::now().time_since_epoch().count());
00020         m_rng.seed(seed);
00021
00022         utl::Logger::log("EnemyAISystem: Initialized", utl::LogLevel::INFO);
00023     }
00024
00025 void EnemyAISystem::update(ecs::Registry &registry, float deltaTime)
00026 {
00027     (void)registry; // Use member registry instead
00028     const auto &enemies = m_entityManager.getEnemies();
00029
00030     for (const auto &[enemyId, enemyEntity] : enemies)
00031     {
00032         const auto *metadata = m_entityManager.getEntityMetadata(enemyId);
00033         if (!metadata || !metadata->isActive)
00034             continue;
00035
00036         // Update AI based on enemy type
00037         switch (metadata->type)
00038         {
00039             case ServerEntityType::ENEMY_BASIC:
00040                 updateBasicEnemyAI(enemyId, enemyEntity, deltaTime);
00041                 break;
00042             case ServerEntityType::ENEMY_ADVANCED:
00043                 updateAdvancedEnemyAI(enemyId, enemyEntity, deltaTime);
00044                 break;
00045             case ServerEntityType::BOSS:
00046                 updateBossAI(enemyId, enemyEntity, deltaTime);
00047                 break;
00048             default:
00049                 break;
00050         }
00051
00052         // Clamp enemy to screen bounds
00053         clampToScreen(enemyEntity);
00054     }
00055 }
00056
00057 void EnemyAISystem::updateBasicEnemyAI(std::uint32_t enemyId, ecs::Entity enemy, float deltaTime)
00058 {
00059     auto *velocity = m_registry.getComponent<ecs::Velocity>(enemy);
00060     if (!velocity)
00061         return;
00062
00063     // Basic enemies move straight left
00064     velocity->x = -200.0f;
00065     velocity->y = 0.0f;
00066
00067     // Initialize movement timer if not present
00068     if (m_enemyMovementTimers.find(enemyId) == m_enemyMovementTimers.end())
00069     {
00070         m_enemyMovementTimers[enemyId] = 0.0f;
00071     }
00072     m_enemyMovementTimers[enemyId] += deltaTime;
00073
00074     // Try to shoot occasionally
00075     tryShoot(enemyId, enemy, deltaTime);
00076 }
00077
00078 void EnemyAISystem::updateAdvancedEnemyAI(std::uint32_t enemyId, ecs::Entity enemy, float deltaTime)
00079 {
00080     auto *transform = m_registry.getComponent<ecs::Transform>(enemy);
00081     if (!transform)
00082         return;
00083
00084     // Initialize movement timer if not present
00085     if (m_enemyMovementTimers.find(enemyId) == m_enemyMovementTimers.end())
00086     {
00087         m_enemyMovementTimers[enemyId] = 0.0f;
00088     }
00089     m_enemyMovementTimers[enemyId] += deltaTime;
00090
00091     // Advanced enemies use sine wave movement
00092     applySineWaveMovement(enemyId, enemy, deltaTime, 1.5f, 100.0f);
00093
00094     // Check distance to nearest player
00095     float distanceToPlayer = getDistanceToNearestPlayer(transform->x, transform->y);
00096
00097     // If player is close, become more aggressive

```

```

00098     if (distanceToPlayer < 400.0f)
00099     {
00100         applyAggressiveMovement(enemy, deltaTime, 100.0f);
00101     }
00102
00103     // Try to shoot more frequently than basic enemies
00104     tryShoot(enemyId, enemy, deltaTime);
00105 }
00106
00107 void EnemyAISystem::updateBossAI(std::uint32_t enemyId, ecs::Entity enemy, float deltaTime)
00108 {
00109     auto *transform = m_registry.getComponent<ecs::Transform>(enemy);
00110     auto *velocity = m_registry.getComponent<ecs::Velocity>(enemy);
00111     auto *enemyComp = m_registry.getComponent<ecs::Enemy>(enemy);
00112
00113     if (!transform || !velocity || !enemyComp)
00114         return;
00115
00116     // Initialize timers if not present
00117     if (m_enemyMovementTimers.find(enemyId) == m_enemyMovementTimers.end())
00118     {
00119         m_enemyMovementTimers[enemyId] = 0.0f;
00120     }
00121     if (m_bossSpreadTimers.find(enemyId) == m_bossSpreadTimers.end())
00122     {
00123         m_bossSpreadTimers[enemyId] = 0.0f;
00124     }
00125     m_enemyMovementTimers[enemyId] += deltaTime;
00126     m_bossSpreadTimers[enemyId] += deltaTime;
00127
00128     // Boss has multiple phases based on health
00129     float healthPercent = enemyComp->health / enemyComp->max_health;
00130
00131     if (healthPercent > 0.66f)
00132     {
00133         // Phase 1: Move slowly and shoot occasionally
00134         velocity->x = -30.0f;
00135         applySineWaveMovement(enemyId, enemy, deltaTime, 0.5f, 150.0f);
00136
00137         // Reduce shoot cooldown
00138         enemyComp->shoot_cooldown = 1.5f;
00139     }
00140     else if (healthPercent > 0.33f)
00141     {
00142         // Phase 2: Move faster with zigzag pattern
00143         applyZigzagMovement(enemyId, enemy, deltaTime);
00144
00145         // Shoot more frequently
00146         enemyComp->shoot_cooldown = 1.0f;
00147     }
00148     else
00149     {
00150         // Phase 3: Aggressive movement and rapid fire
00151         velocity->x = -50.0f;
00152         applyAggressiveMovement(enemy, deltaTime, 80.0f);
00153
00154         // Rapid fire
00155         enemyComp->shoot_cooldown = 0.5f;
00156     }
00157
00158     // Boss always tries to shoot
00159     tryShoot(enemyId, enemy, deltaTime);
00160
00161     // Additional boss-specific behavior
00162     // Shoot multiple projectiles in a spread pattern when health is low
00163     if (healthPercent < 0.33f && m_bossSpreadTimers[enemyId] >= 1.0f)
00164     {
00165         // Spread pattern: shoot at angles
00166         for (int i = -1; i <= 1; i++)
00167         {
00168             if (i == 0)
00169                 continue; // Skip center (handled by normal shoot)
00170
00171             float angle = i * 15.0f * (3.14159f / 180.0f); // Convert to radians
00172             float vx = -300.0f * std::cos(angle);
00173             float vy = -300.0f * std::sin(angle);
00174
00175             m_entityManager.createEnemyProjectile(m_entityManager.getNetworkIdForEntity(enemy), transform->x,
00176                 transform->y, vx, vy);
00177         }
00178         m_bossSpreadTimers[enemyId] = 0.0f;
00179     }
00180 }
00181
00182 void EnemyAISystem::applySineWaveMovement(std::uint32_t enemyId, ecs::Entity enemy, float deltaTime,
00183     float frequency, float amplitude)
00184 {

```

```

00185     auto *transform = m_registry.getComponent<ecs::Transform>(enemy);
00186     auto *velocity = m_registry.getComponent<ecs::Velocity>(enemy);
00187
00188     if (!transform || !velocity)
00189         return;
00190
00191     // Base horizontal movement
00192     float baseVelocityX = velocity->x;
00193     if (baseVelocityX == 0.0f)
00194         baseVelocityX = -150.0f;
00195
00196     // Use per-enemy timer instead of static variable
00197     float time = m_enemyMovementTimers[enemyId];
00198     float sineValue = std::sin(time * frequency);
00199     velocity->y = sineValue * amplitude;
00200     velocity->x = baseVelocityX;
00201 }
00202
00203 void EnemyAISystem::applyAggressiveMovement(ecs::Entity enemy, float deltaTime, float speed)
00204 {
00205     auto *transform = m_registry.getComponent<ecs::Transform>(enemy);
00206     auto *velocity = m_registry.getComponent<ecs::Velocity>(enemy);
00207
00208     if (!transform || !velocity)
00209         return;
00210
00211     // Find nearest player
00212     ecs::Entity nearestPlayer = findNearestPlayer(transform->x, transform->y);
00213     if (nearestPlayer == ecs::INVALID_ENTITY)
00214     {
00215         // No player found, move left by default
00216         velocity->x = -speed;
00217         velocity->y = 0.0f;
00218         return;
00219     }
00220
00221     auto *playerTransform = m_registry.getComponent<ecs::Transform>(nearestPlayer);
00222     if (!playerTransform)
00223         return;
00224
00225     // Calculate direction to player
00226     float dx = playerTransform->x - transform->x;
00227     float dy = playerTransform->y - transform->y;
00228     float distance = std::sqrt(dx * dx + dy * dy);
00229
00230     if (distance > 0.01f)
00231     {
00232         // Normalize and apply speed
00233         velocity->x = (dx / distance) * speed * m_aggressiveness;
00234         velocity->y = (dy / distance) * speed * m_aggressiveness;
00235     }
00236 }
00237
00238 void EnemyAISystem::applyZigzagMovement(std::uint32_t enemyId, ecs::Entity enemy, float deltaTime)
00239 {
00240     auto *transform = m_registry.getComponent<ecs::Transform>(enemy);
00241     auto *velocity = m_registry.getComponent<ecs::Velocity>(enemy);
00242
00243     if (!transform || !velocity)
00244         return;
00245
00246     // Use per-enemy timer instead of static variable
00247     float zigzagTime = m_enemyMovementTimers[enemyId];
00248
00249     // Zigzag pattern: change direction every second
00250     float period = 1.5f;
00251     float phase = std::fmod(zigzagTime, period * 2.0f);
00252
00253     velocity->x = -150.0f;
00254     velocity->y = (phase < period) ? 100.0f : -100.0f;
00255 }
00256
00257 void EnemyAISystem::tryShoot(std::uint32_t enemyId, ecs::Entity enemy, float deltaTime)
00258 {
00259     auto *enemyComp = m_registry.getComponent<ecs::Enemy>(enemy);
00260     auto *transform = m_registry.getComponent<ecs::Transform>(enemy);
00261
00262     if (!enemyComp || !transform)
00263         return;
00264
00265     // Update shot timer
00266     enemyComp->last_shot_time += deltaTime;
00267
00268     // Check if can shoot
00269     if (!canShoot(enemy))
00270         return;
00271 }

```

```

00272 // Random chance to shoot (to add variety)
00273 float randomValue = m_shootChance(m_rng);
00274 if (randomValue > 0.3f) // 70% chance to NOT shoot (30% chance to shoot)
00275     return;
00276
00277 // Check if there's a player to shoot at
00278 float distanceToPlayer = getDistanceToNearestPlayer(transform->x, transform->y);
00279 if (distanceToPlayer > 1500.0f) // Don't shoot if no player is nearby
00280     return;
00281
00282 // Shoot projectile towards the left (player direction)
00283 float projectileVx = -400.0f;
00284 float projectileVy = 0.0f;
00285
00286 // For more intelligent enemies, shoot towards nearest player
00287 ecs::Entity nearestPlayer = findNearestPlayer(transform->x, transform->y);
00288 if (nearestPlayer != ecs::INVALID_ENTITY)
00289 {
00290     auto *playerTransform = m_registry.getComponent<ecs::Transform>(nearestPlayer);
00291     if (playerTransform)
00292     {
00293         float dx = playerTransform->x - transform->x;
00294         float dy = playerTransform->y - transform->y;
00295         float distance = std::sqrt(dx * dx + dy * dy);
00296
00297         // Only shoot towards the left (player direction) - don't shoot backwards
00298         if (distance > 0.01f && dx < 0.0f)
00299         {
00300             // Normalize and apply projectile speed
00301             float projectileSpeed = 500.0f;
00302             projectileVx = (dx / distance) * projectileSpeed;
00303             projectileVy = (dy / distance) * projectileSpeed;
00304         }
00305         else if (dx >= 0.0f)
00306         {
00307             // Player is behind, don't shoot
00308             return;
00309         }
00310     }
00311 }
00312
00313 m_entityManager.createEnemyProjectile(enemyId, transform->x - 10.0f, transform->y, projectileVx, projectileVy);
00314
00315 // Reset shot timer
00316 enemyComp->last_shot_time = 0.0f;
00317
00318 // Shooting logged only for debugging (too verbose at INFO level)
00319 // utl::Logger::log("EnemyAISystem: Enemy " + std::to_string(enemyId) + " fired projectile");
00320 }
00321
00322 bool EnemyAISystem::canShoot(ecs::Entity enemy) const
00323 {
00324     auto *enemyComp = m_registry.getComponent<ecs::Enemy>(enemy);
00325     if (!enemyComp)
00326         return false;
00327
00328     return enemyComp->last_shot_time >= enemyComp->shoot_cooldown;
00329 }
00330
00331 ecs::Entity EnemyAISystem::findNearestPlayer(float x, float y) const
00332 {
00333     const auto &players = m_entityManager.getPlayers();
00334
00335     ecs::Entity nearestPlayer = ecs::INVALID_ENTITY;
00336     float nearestDistance = std::numeric_limits<float>::max();
00337
00338     for (const auto &[playerId, playerEntity] : players)
00339     {
00340         auto *transform = m_registry.getComponent<ecs::Transform>(playerEntity);
00341         if (!transform)
00342             continue;
00343
00344         float dx = transform->x - x;
00345         float dy = transform->y - y;
00346         float distance = std::sqrt(dx * dx + dy * dy);
00347
00348         if (distance < nearestDistance)
00349         {
00350             nearestDistance = distance;
00351             nearestPlayer = playerEntity;
00352         }
00353     }
00354
00355     return nearestPlayer;
00356 }
00357
00358 float EnemyAISystem::getDistanceToNearestPlayer(float x, float y) const

```



```

00359 {
00360     const auto &players = m_entityManager.getPlayers();
00361
00362     float nearestDistance = std::numeric_limits<float>::max();
00363
00364     for (const auto &[playerId, playerEntity] : players)
00365     {
00366         auto *transform = m_registry.getComponent<ecs::Transform>(playerEntity);
00367         if (!transform)
00368             continue;
00369
00370         float dx = transform->x - x;
00371         float dy = transform->y - y;
00372         float distance = std::sqrt(dx * dx + dy * dy);
00373
00374         if (distance < nearestDistance)
00375         {
00376             nearestDistance = distance;
00377         }
00378     }
00379
00380     return nearestDistance;
00381 }
00382
00383 void EnemyAISystem::clampToScreen(ecs::Entity enemy)
00384 {
00385     auto *transform = m_registry.getComponent<ecs::Transform>(enemy);
00386     if (!transform)
00387         return;
00388
00389     const float MARGIN = 50.0f;
00390
00391     // Clamp to screen bounds with margin
00392     transform->x = std::max(-MARGIN, std::min(transform->x, utl::GameConfig::Server::SCREEN_WIDTH +
MARGIN));
00393     transform->y = std::max(MARGIN, std::min(transform->y, utl::GameConfig::Server::SCREEN_HEIGHT -
MARGIN));
00394 }
00395
00396 } // namespace gme

```

19.265 /home/masina/Projects/Epitech/rtype/plugins/Game/RType/Server/src/Systems/EnemySpawnSystem.cpp File Reference

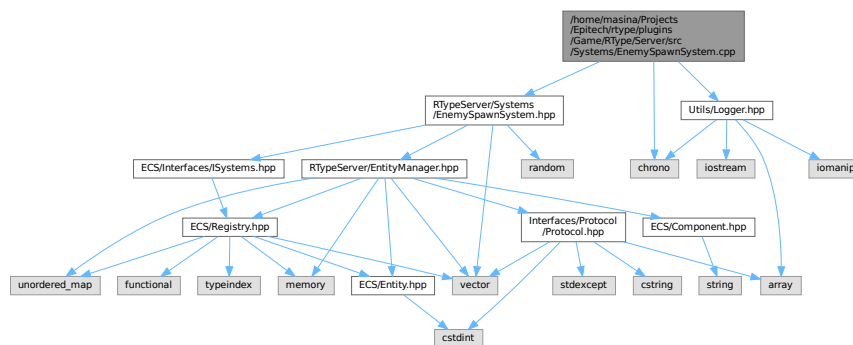
Implementation of enemy spawning system.

#include "RTypeServer/Systems/EnemySpawnSystem.hpp"

#include "Utils/Logger.hpp"

#include <chrono>

Include dependency graph for EnemySpawnSystem.cpp:



Namespaces

- namespace [gme](#)

19.265.1 Detailed Description

Implementation of enemy spawning system.

Definition in file [EnemySpawnSystem.cpp](#).

19.266 EnemySpawnSystem.cpp

[Go to the documentation of this file.](#)

```

00001 ///  

00002 ///  

00003 ///  

00004 ///  

00005 ///  

00006 ///  

00007 #include "RTypeServer/Systems/EnemySpawnSystem.hpp"  

00008 #include "Utils/Logger.hpp"  

00009 #include <chrono>  

00010  

00011 namespace gme  

00012 {  

00013     EnemySpawnSystem::EnemySpawnSystem(EntityManager &entityManager)  

00014         : m_entityManager(entityManager), m_levelTime(0.0f), m_currentWaveIndex(0), m_waveSpawnTimer(0.0f),  

00015           m_enabled(false), m_totalEnemiesSpawned(0), m_yDistribution(100.0f, 980.0f)  

00016     {  

00017         const auto seed = static_cast<unsigned int>(std::chrono::system_clock::now().time_since_epoch().count());  

00018         m_rng.seed(seed);  

00019  

00020         initializeWaves();  

00021  

00022         utl::Logger::log("EnemySpawnSystem: Initialized with " + std::to_string(m_waves.size()) + " waves",  

00023             utl::LogLevel::INFO);  

00024     }  

00025  

00026 void EnemySpawnSystem::update(ecs::Registry &registry, float deltaTime)  

00027 {  

00028     (void)registry; // Use member entityManager instead  

00029     if (!m_enabled)  

00030         return;  

00031  

00032     m_levelTime += deltaTime;  

00033     m_waveSpawnTimer += deltaTime;  

00034  

00035     // Check if we should process current wave  

00036     if (m_currentWaveIndex < m_waves.size())  

00037     {  

00038         auto &wave = m_waves[m_currentWaveIndex];  

00039  

00040         // Check if it's time to start this wave  

00041         if (m_levelTime >= wave.spawnTime)  

00042         {  

00043             processWave(wave, deltaTime);  

00044  

00045             // Check if wave is complete  

00046             int totalToSpawn = wave.basicEnemyCount + wave.advancedEnemyCount + (wave.bossSpawn ? 1 : 0);  

00047             if (wave.spawnedCount >= totalToSpawn)  

00048             {  

00049                 // Move to next wave  

00050                 m_currentWaveIndex++;  

00051                 m_waveSpawnTimer = 0.0f;  

00052  

00053                 if (m_currentWaveIndex < m_waves.size())  

00054                 {  

00055                     utl::Logger::log("EnemySpawnSystem: Wave " + std::to_string(m_currentWaveIndex) +  

00056                         " complete. Moving to wave " + std::to_string(m_currentWaveIndex + 1),  

00057                         utl::LogLevel::INFO);  

00058                 }  

00059                 else  

00060                 {  

00061                     utl::Logger::log("EnemySpawnSystem: All waves complete!", utl::LogLevel::INFO);  

00062                 }  

00063             }  

00064         }  

00065     }  

00066 }  

00067  

00068 void EnemySpawnSystem::reset()  

00069 {  

00070     m_levelTime = 0.0f;  

00071     m_currentWaveIndex = 0;  

00072     m_waveSpawnTimer = 0.0f;  

00073     m_totalEnemiesSpawned = 0;  

00074     m_enabled = false;  

00075  

00076     // Reset wave spawn counts

```

```

00077     for (auto &wave : m_waves)
00078     {
00079         wave.spawnedCount = 0;
00080     }
00081
00082     utl::Logger::log("EnemySpawnSystem: Reset", utl::LogLevel::INFO);
00083 }
00084
00085 void EnemySpawnSystem::initializeWaves()
00086 {
00087     m_waves.clear();
00088
00089     // Wave 1: 5 basic enemies (10 seconds in, 1 enemy every 1 second)
00090     SpawnWave wave1;
00091     wave1.spawnTime = 10.0f;
00092     wave1.basicEnemyCount = 5;
00093     wave1.advancedEnemyCount = 0;
00094     wave1.bossSpawn = false;
00095     wave1.spawnInterval = 1.0f;
00096     wave1.spawnedCount = 0;
00097     m_waves.push_back(wave1);
00098
00099     // Wave 2: 3 basic + 2 advanced (20 seconds in, 1 enemy every 1.5 seconds)
00100     SpawnWave wave2;
00101     wave2.spawnTime = 20.0f;
00102     wave2.basicEnemyCount = 3;
00103     wave2.advancedEnemyCount = 2;
00104     wave2.bossSpawn = false;
00105     wave2.spawnInterval = 1.5f;
00106     wave2.spawnedCount = 0;
00107     m_waves.push_back(wave2);
00108
00109     // Wave 3: 2 basic + 4 advanced (35 seconds in, 1 enemy every 1.2 seconds)
00110     SpawnWave wave3;
00111     wave3.spawnTime = 35.0f;
00112     wave3.basicEnemyCount = 2;
00113     wave3.advancedEnemyCount = 4;
00114     wave3.bossSpawn = false;
00115     wave3.spawnInterval = 1.2f;
00116     wave3.spawnedCount = 0;
00117     m_waves.push_back(wave3);
00118
00119     // Wave 4: 7 advanced enemies (55 seconds in, 1 enemy every 2 seconds)
00120     SpawnWave wave4;
00121     wave4.spawnTime = 55.0f;
00122     wave4.basicEnemyCount = 0;
00123     wave4.advancedEnemyCount = 7;
00124     wave4.bossSpawn = false;
00125     wave4.spawnInterval = 2.0f;
00126     wave4.spawnedCount = 0;
00127     m_waves.push_back(wave4);
00128
00129     // Wave 5: Boss (80 seconds in)
00130     SpawnWave wave5;
00131     wave5.spawnTime = 80.0f;
00132     wave5.basicEnemyCount = 0;
00133     wave5.advancedEnemyCount = 0;
00134     wave5.bossSpawn = true;
00135     wave5.spawnInterval = 0.0f;
00136     wave5.spawnedCount = 0;
00137     m_waves.push_back(wave5);
00138 }
00139
00140 void EnemySpawnSystem::processWave(SpawnWave &wave, float deltaTime)
00141 {
00142     (void)deltaTime; // Not used directly, but kept for consistency
00143
00144     int totalToSpawn = wave.basicEnemyCount + wave.advancedEnemyCount + (wave.bossSpawn ? 1 : 0);
00145
00146     // Check if we've spawned everything for this wave
00147     if (wave.spawnedCount >= totalToSpawn)
00148         return;
00149
00150     // Check if enough time has passed since last spawn
00151     if (m_waveSpawnTimer < wave.spawnInterval)
00152         return;
00153
00154     // Spawn next enemy
00155     float spawnX = getSpawnX();
00156     float spawnY = getRandomY();
00157
00158     // Determine which type to spawn based on what's left
00159     if (wave.bossSpawn && wave.spawnedCount == 0)
00160     {
00161         // Spawn boss first if this is a boss wave
00162         m_entityManager.createBoss(spawnX, spawnY, 1000.0f);
00163         m_totalEnemiesSpawned++;

```

```

00164         wave.spawnedCount++;
00165
00166         utl::Logger::log("EnemySpawnSystem: Spawned BOSS at (" + std::to_string(spawnX) + ", " +
00167             std::to_string(spawnY) + ")",
00168             utl::LogLevel::INFO);
00169     }
00170     else if (wave.basicEnemyCount > 0 && wave.spawnedCount < wave.basicEnemyCount)
00171     {
00172         // Spawn basic enemy
00173         m_entityManager.createBasicEnemy(spawnX, spawnY, 50.0f);
00174         m_totalEnemiesSpawned++;
00175         wave.spawnedCount++;
00176
00177         utl::Logger::log("EnemySpawnSystem: Spawned basic enemy at (" + std::to_string(spawnX) + ", " +
00178             std::to_string(spawnY) + ")",
00179             utl::LogLevel::INFO);
00180     }
00181     else if (wave.advancedEnemyCount > 0)
00182     {
00183         // Spawn advanced enemy
00184         m_entityManager.createAdvancedEnemy(spawnX, spawnY, 100.0f);
00185         m_totalEnemiesSpawned++;
00186         wave.spawnedCount++;
00187
00188         utl::Logger::log("EnemySpawnSystem: Spawned advanced enemy at (" + std::to_string(spawnX) + ", " +
00189             std::to_string(spawnY) + ")",
00190             utl::LogLevel::INFO);
00191     }
00192
00193     // Reset spawn timer
00194     m_waveSpawnTimer = 0.0f;
00195 }
00196
00197 float EnemySpawnSystem::getRandomY()
00198 {
00199     // Return a random Y position within screen bounds
00200     // Screen height is 1080, so we spawn between 100 and 980 to leave margins
00201     return m_yDistribution(m_rng);
00202 }
00203
00204 } // namespace gme

```

19.267 /home/masina/Projects/Epitech/rtype/plugins/Game/RType/Server/src/WaveManager.cpp File Reference

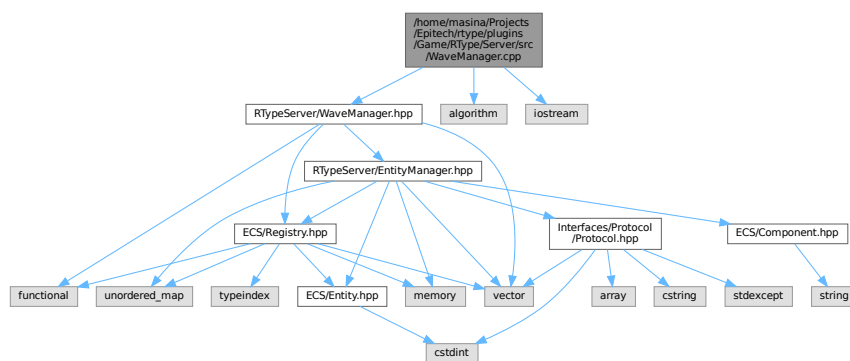
Implementation of wave-based enemy spawning system.

```
#include "RTypeServer/WaveManager.hpp"
```

```
#include <algorithm>
```

```
#include <iostream>
```

Include dependency graph for WaveManager.cpp:



Namespaces

- namespace `gme`

19.267.1 Detailed Description

Implementation of wave-based enemy spawning system.

Definition in file [WaveManager.cpp](#).

19.268 WaveManager.cpp

[Go to the documentation of this file.](#)

```
00001 /// @file WaveManager.cpp
00002 /// @brief Implementation of wave-based enemy spawning system
00003 /// @namespace gme
00004 ///
00005
00006 #include "RTypeServer/WaveManager.hpp"
00007 #include <algorithm>
00008 #include <iostream>
00009
00010 namespace gme
00011 {
00012
00013     WaveManager::WaveManager(EntityManager &entityManager)
00014         : m_entityManager(entityManager), m_currentWaveIndex(-1), m_waveTimer(0.0f), m_spawnTimer(0.0f),
00015           m_active(false), m_completed(false), m_waveInProgress(false), m_initialDelayPassed(false),
00016           m_initialDelayTimer(0.0f)
00017     {
00018     }
00019
00020     void WaveManager::update(ecs::Registry &registry, const float dt, const int screenWidth)
00021     {
00022         if (!m_active || m_completed)
00023         {
00024             return;
00025         }
00026
00027         // Handle initial delay (sync with client stage spawn)
00028         if (!m_initialDelayPassed)
00029         {
00030             m_initialDelayTimer += dt;
00031             if (m_initialDelayTimer >= INITIAL_DELAY)
00032             {
00033                 m_initialDelayPassed = true;
00034                 std::cout << "[WaveManager] Initial delay complete, waves starting!" << std::endl;
00035             }
00036             else
00037             {
00038                 return; // Wait for initial delay
00039             }
00040         }
00041
00042         // Check if we need to start the first wave
00043         if (m_currentWaveIndex < 0)
00044         {
00045             startNextWave();
00046             return;
00047         }
00048
00049         // Update wave timer
00050         m_waveTimer += dt;
00051         m_spawnTimer += dt;
00052
00053         // Process enemy spawns for current wave
00054         if (m_waveInProgress && m_currentWaveIndex < static_cast<int>(m_waves.size()))
00055         {
00056             processSpawns(dt, screenWidth);
00057
00058             const Wave &currentWave = m_waves[m_currentWaveIndex];
00059
00060             // Check if we should advance to next wave
00061             bool shouldAdvance = false;
00062
00063             if (currentWave.waitForClear)
00064             {
00065                 // Wait for all enemies to be cleared
00066                 if (isWaveCleared())
00067                 {
00068                     shouldAdvance = true;
00069                 }
00070             }
00071             else
00072             {
00073                 // Advance after duration expires
00074                 if (m_waveTimer >= currentWave.duration)
00075                 {
00076                     shouldAdvance = true;
00077                 }
00078             }
00079         }
00080     }
00081 }
```

```

00077     }
00078     }
00079
00080     if (shouldAdvance)
00081     {
00082         // Call completion callback if exists
00083         if (currentWave.onComplete)
00084         {
00085             currentWave.onComplete();
00086         }
00087
00088         std::cout << "[WaveManager] Wave " << (m_currentWaveIndex + 1) << " completed!" << std::endl;
00089
00090         // Move to next wave
00091         if (m_currentWaveIndex + 1 < static_cast<int>(m_waves.size()))
00092         {
00093             startNextWave();
00094         }
00095         else
00096         {
00097             // All waves completed
00098             m_completed = true;
00099             m_waveInProgress = false;
00100             std::cout << "[WaveManager] All waves completed!" << std::endl;
00101         }
00102     }
00103 }
00104 }
00105
00106 void WaveManager::start()
00107 {
00108     m_active = true;
00109     m_currentWaveIndex = -1;
00110     m_waveTimer = 0.0f;
00111     m_spawnTimer = 0.0f;
00112     m_completed = false;
00113     m_waveInProgress = false;
00114     m_initialDelayPassed = false;
00115     m_initialDelayTimer = 0.0f;
00116     std::cout << "[WaveManager] Wave system started with " << m_waves.size()
00117               << " waves (initial delay: " << INITIAL_DELAY << "s)" << std::endl;
00118 }
00119
00120 void WaveManager::reset()
00121 {
00122     m_active = false;
00123     m_currentWaveIndex = -1;
00124     m_waveTimer = 0.0f;
00125     m_spawnTimer = 0.0f;
00126     m_completed = false;
00127     m_waveInProgress = false;
00128     m_initialDelayPassed = false;
00129     m_initialDelayTimer = 0.0f;
00130     m_enemiesSpawned.clear();
00131     std::cout << "[WaveManager] Wave system reset" << std::endl;
00132 }
00133
00134 void WaveManager::addWave(const Wave &wave) { m_waves.push_back(wave); }
00135
00136 void WaveManager::clearWaves()
00137 {
00138     m_waves.clear();
00139     reset();
00140 }
00141
00142 void WaveManager::setupDefaultWaves()
00143 {
00144     clearWaves();
00145     createWave1();
00146     createWave2();
00147     createWave3();
00148     createWave4();
00149     createBossWave();
00150     std::cout << "[WaveManager] Default waves created (" << m_waves.size() << " waves)" << std::endl;
00151 }
00152
00153 void WaveManager::startNextWave()
00154 {
00155     m_currentWaveIndex++;
00156
00157     if (m_currentWaveIndex >= static_cast<int>(m_waves.size()))
00158     {
00159         m_completed = true;
00160         m_waveInProgress = false;
00161         return;
00162     }
00163 }

```

```

00164     const Wave &wave = m_waves[m_currentWaveIndex];
00165
00166     m_waveTimer = 0.0f;
00167     m_spawnTimer = 0.0f;
00168     m_waveInProgress = true;
00169     m_enemiesSpawned.clear();
00170     m_enemiesSpawned.resize(wave.enemies.size(), false);
00171
00172     std::cout << "[WaveManager] Starting wave " << (m_currentWaveIndex + 1) << " with " << wave.enemies.size()
00173         << " enemies" << std::endl;
00174 }
00175
00176 bool WaveManager::isWaveCleared() const
00177 {
00178     // Check if all spawned enemies are dead
00179     const auto &enemies = m_entityManager.getEnemies();
00180     return enemies.empty();
00181 }
00182
00183 void WaveManager::processSpawns(const float dt, const int screenWidth)
00184 {
00185     if (m_currentWaveIndex < 0 || m_currentWaveIndex >= static_cast<int>(m_waves.size()))
00186     {
00187         return;
00188     }
00189
00190     const Wave &currentWave = m_waves[m_currentWaveIndex];
00191
00192     // Check each enemy spawn in the current wave
00193     for (size_t i = 0; i < currentWave.enemies.size(); ++i)
00194     {
00195         if (m_enemiesSpawned[i])
00196         {
00197             continue; // Already spawned
00198         }
00199
00200         const EnemySpawn &spawn = currentWave.enemies[i];
00201
00202         // Check if it's time to spawn this enemy
00203         if (m_spawnTimer >= spawn.spawnDelay)
00204         {
00205             // Calculate spawn position (right side of screen + offset)
00206             float spawnX = static_cast<float>(screenWidth) + spawn.x;
00207             float spawnY = spawn.y;
00208
00209             // Create the enemy based on type
00210             switch (spawn.type)
00211             {
00212                 case ServerEntityType::ENEMY_BASIC:
00213                     m_entityManager.createBasicEnemy(spawnX, spawnY, spawn.health);
00214                     std::cout << "[WaveManager] Spawned Basic Enemy at (" << spawnX << ", " << spawnY << ")"
00215                         << std::endl;
00216                     break;
00217
00218                 case ServerEntityType::ENEMY_ADVANCED:
00219                     m_entityManager.createAdvancedEnemy(spawnX, spawnY, spawn.health);
00220                     std::cout << "[WaveManager] Spawned Advanced Enemy at (" << spawnX << ", " << spawnY << ")"
00221                         << std::endl;
00222                     break;
00223
00224                 case ServerEntityType::BOSS:
00225                     m_entityManager.createBoss(spawnX, spawnY, spawn.health);
00226                     std::cout << "[WaveManager] Spawned Boss at (" << spawnX << ", " << spawnY << ")" << std::endl;
00227                     break;
00228
00229                 default:
00230                     std::cerr << "[WaveManager] Unknown enemy type in spawn!" << std::endl;
00231                     break;
00232             }
00233
00234             m_enemiesSpawned[i] = true;
00235         }
00236     }
00237 }
00238
00239 // ===== Default Wave Creation =====
00240
00241 void WaveManager::createWave1()
00242 {
00243     Wave wave;
00244     wave.waveNumber = 1;
00245     wave.duration = 30.0f;
00246     wave.waitForClear = false; // Continue after 30s regardless
00247
00248     // First group - 3 Basic enemies in vertical line
00249     wave.enemies.push_back({ServerEntityType::ENEMY_BASIC, 50.0f, 150.0f, 50.0f, 2.0f});
00250     wave.enemies.push_back({ServerEntityType::ENEMY_BASIC, 50.0f, 300.0f, 50.0f, 4.0f});

```

```

00251     wave.enemies.push_back({ServerEntityType::ENEMY_BASIC, 50.0f, 450.0f, 50.0f, 6.0f});
00252
00253     // Second group - 2 Basic enemies from sides
00254     wave.enemies.push_back({ServerEntityType::ENEMY_BASIC, 80.0f, 200.0f, 50.0f, 12.0f});
00255     wave.enemies.push_back({ServerEntityType::ENEMY_BASIC, 80.0f, 400.0f, 50.0f, 12.0f});
00256
00257     // Third group - 3 Basic enemies in formation
00258     wave.enemies.push_back({ServerEntityType::ENEMY_BASIC, 100.0f, 250.0f, 50.0f, 20.0f});
00259     wave.enemies.push_back({ServerEntityType::ENEMY_BASIC, 60.0f, 300.0f, 50.0f, 21.0f});
00260     wave.enemies.push_back({ServerEntityType::ENEMY_BASIC, 100.0f, 350.0f, 50.0f, 22.0f});
00261
00262     m_waves.push_back(wave);
00263 }
00264
00265 void WaveManager::createWave2()
00266 {
00267     Wave wave;
00268     wave.waveNumber = 2;
00269     wave.duration = 30.0f;
00270     wave.waitForClear = false; // Continue after 30s regardless
00271
00272     // First wave - Advanced enemy with Basic support
00273     wave.enemies.push_back({ServerEntityType::ENEMY_ADVANCED, 80.0f, 300.0f, 100.0f, 2.0f});
00274     wave.enemies.push_back({ServerEntityType::ENEMY_BASIC, 40.0f, 200.0f, 60.0f, 3.0f});
00275     wave.enemies.push_back({ServerEntityType::ENEMY_BASIC, 40.0f, 400.0f, 60.0f, 3.0f});
00276
00277     // Second wave - V formation of Basic enemies
00278     wave.enemies.push_back({ServerEntityType::ENEMY_BASIC, 100.0f, 200.0f, 60.0f, 10.0f});
00279     wave.enemies.push_back({ServerEntityType::ENEMY_BASIC, 60.0f, 250.0f, 60.0f, 10.5f});
00280     wave.enemies.push_back({ServerEntityType::ENEMY_BASIC, 20.0f, 300.0f, 60.0f, 11.0f});
00281     wave.enemies.push_back({ServerEntityType::ENEMY_BASIC, 60.0f, 350.0f, 60.0f, 11.5f});
00282     wave.enemies.push_back({ServerEntityType::ENEMY_BASIC, 100.0f, 400.0f, 60.0f, 12.0f});
00283
00284     // Third wave - Two Advanced enemies
00285     wave.enemies.push_back({ServerEntityType::ENEMY_ADVANCED, 90.0f, 250.0f, 100.0f, 20.0f});
00286     wave.enemies.push_back({ServerEntityType::ENEMY_ADVANCED, 90.0f, 350.0f, 100.0f, 22.0f});
00287
00288     m_waves.push_back(wave);
00289 }
00290
00291 void WaveManager::createWave3()
00292 {
00293     Wave wave;
00294     wave.waveNumber = 3;
00295     wave.duration = 30.0f;
00296     wave.waitForClear = false; // Continue after 30s
00297
00298     // First assault - Multiple Basic enemies
00299     wave.enemies.push_back({ServerEntityType::ENEMY_BASIC, 50.0f, 150.0f, 70.0f, 1.0f});
00300     wave.enemies.push_back({ServerEntityType::ENEMY_BASIC, 50.0f, 300.0f, 70.0f, 1.0f});
00301     wave.enemies.push_back({ServerEntityType::ENEMY_BASIC, 50.0f, 450.0f, 70.0f, 1.0f});
00302
00303     // Second assault - Advanced enemies with escort
00304     wave.enemies.push_back({ServerEntityType::ENEMY_ADVANCED, 100.0f, 200.0f, 120.0f, 8.0f});
00305     wave.enemies.push_back({ServerEntityType::ENEMY_BASIC, 50.0f, 180.0f, 60.0f, 9.0f});
00306     wave.enemies.push_back({ServerEntityType::ENEMY_ADVANCED, 100.0f, 400.0f, 120.0f, 10.0f});
00307     wave.enemies.push_back({ServerEntityType::ENEMY_BASIC, 50.0f, 420.0f, 60.0f, 11.0f});
00308
00309     // Final assault - Mini-boss wave
00310     wave.enemies.push_back({ServerEntityType::ENEMY_ADVANCED, 120.0f, 250.0f, 150.0f, 18.0f});
00311     wave.enemies.push_back({ServerEntityType::ENEMY_ADVANCED, 120.0f, 350.0f, 150.0f, 18.0f});
00312     wave.enemies.push_back({ServerEntityType::ENEMY_BASIC, 70.0f, 200.0f, 70.0f, 20.0f});
00313     wave.enemies.push_back({ServerEntityType::ENEMY_BASIC, 70.0f, 300.0f, 70.0f, 20.0f});
00314     wave.enemies.push_back({ServerEntityType::ENEMY_BASIC, 70.0f, 400.0f, 70.0f, 20.0f});
00315
00316     m_waves.push_back(wave);
00317 }
00318
00319 void WaveManager::createWave4()
00320 {
00321     Wave wave;
00322     wave.waveNumber = 4;
00323     wave.duration = 30.0f;
00324     wave.waitForClear = false; // Continue after 30s
00325
00326     // Intense pre-boss wave with multiple formations
00327     // First formation - Top and bottom pincer
00328     wave.enemies.push_back({ServerEntityType::ENEMY_ADVANCED, 100.0f, 150.0f, 130.0f, 1.0f});
00329     wave.enemies.push_back({ServerEntityType::ENEMY_BASIC, 50.0f, 180.0f, 70.0f, 2.0f});
00330     wave.enemies.push_back({ServerEntityType::ENEMY_BASIC, 50.0f, 120.0f, 70.0f, 2.0f});
00331
00332     wave.enemies.push_back({ServerEntityType::ENEMY_ADVANCED, 100.0f, 450.0f, 130.0f, 1.0f});
00333     wave.enemies.push_back({ServerEntityType::ENEMY_BASIC, 50.0f, 420.0f, 70.0f, 2.0f});
00334     wave.enemies.push_back({ServerEntityType::ENEMY_BASIC, 50.0f, 480.0f, 70.0f, 2.0f});
00335
00336     // Second formation - Center reinforcements
00337     wave.enemies.push_back({ServerEntityType::ENEMY_BASIC, 80.0f, 250.0f, 70.0f, 10.0f});

```



```

00338     wave.enemies.push_back({ServerEntityType::ENEMY_BASIC, 80.0f, 300.0f, 70.0f, 10.5f});
00339     wave.enemies.push_back({ServerEntityType::ENEMY_BASIC, 80.0f, 350.0f, 70.0f, 11.0f});
00340     wave.enemies.push_back({ServerEntityType::ENEMY_ADVANCED, 120.0f, 300.0f, 150.0f, 12.0f});
00341
00342     // Third formation - Final assault before boss
00343     wave.enemies.push_back({ServerEntityType::ENEMY_ADVANCED, 130.0f, 200.0f, 150.0f, 20.0f});
00344     wave.enemies.push_back({ServerEntityType::ENEMY_ADVANCED, 130.0f, 400.0f, 150.0f, 20.0f});
00345     wave.enemies.push_back({ServerEntityType::ENEMY_BASIC, 70.0f, 250.0f, 70.0f, 22.0f});
00346     wave.enemies.push_back({ServerEntityType::ENEMY_BASIC, 70.0f, 300.0f, 70.0f, 22.5f});
00347     wave.enemies.push_back({ServerEntityType::ENEMY_BASIC, 70.0f, 350.0f, 70.0f, 23.0f});
00348
00349     m_waves.push_back(wave);
00350 }
00351
00352 void WaveManager::createBossWave()
00353 {
00354     Wave wave;
00355     wave.waveNumber = 5;
00356     wave.duration = 60.0f;
00357     wave.waitForClear = true; // Must kill boss to win
00358
00359     // Boss spawn with initial escort
00360     wave.enemies.push_back({ServerEntityType::BOSS, 150.0f, 300.0f, 1000.0f, 3.0f});
00361
00362     // Initial escort
00363     wave.enemies.push_back({ServerEntityType::ENEMY_BASIC, 80.0f, 200.0f, 60.0f, 4.0f});
00364     wave.enemies.push_back({ServerEntityType::ENEMY_BASIC, 80.0f, 400.0f, 60.0f, 4.0f});
00365
00366     // First wave of reinforcements (T+15s)
00367     wave.enemies.push_back({ServerEntityType::ENEMY_ADVANCED, 100.0f, 250.0f, 100.0f, 15.0f});
00368     wave.enemies.push_back({ServerEntityType::ENEMY_ADVANCED, 100.0f, 350.0f, 100.0f, 15.0f});
00369
00370     // Second wave of reinforcements (T+25s)
00371     wave.enemies.push_back({ServerEntityType::ENEMY_BASIC, 60.0f, 180.0f, 60.0f, 25.0f});
00372     wave.enemies.push_back({ServerEntityType::ENEMY_BASIC, 60.0f, 250.0f, 60.0f, 25.5f});
00373     wave.enemies.push_back({ServerEntityType::ENEMY_BASIC, 60.0f, 350.0f, 60.0f, 26.0f});
00374     wave.enemies.push_back({ServerEntityType::ENEMY_BASIC, 60.0f, 420.0f, 60.0f, 26.5f});
00375
00376     // Third wave of reinforcements (T+40s) - desperation attack
00377     wave.enemies.push_back({ServerEntityType::ENEMY_ADVANCED, 120.0f, 200.0f, 120.0f, 40.0f});
00378     wave.enemies.push_back({ServerEntityType::ENEMY_ADVANCED, 120.0f, 300.0f, 120.0f, 40.0f});
00379     wave.enemies.push_back({ServerEntityType::ENEMY_ADVANCED, 120.0f, 400.0f, 120.0f, 40.0f});
00380
00381     // Final reinforcements (T+50s) - last stand
00382     wave.enemies.push_back({ServerEntityType::ENEMY_BASIC, 50.0f, 150.0f, 70.0f, 50.0f});
00383     wave.enemies.push_back({ServerEntityType::ENEMY_BASIC, 50.0f, 225.0f, 70.0f, 50.5f});
00384     wave.enemies.push_back({ServerEntityType::ENEMY_BASIC, 50.0f, 300.0f, 70.0f, 51.0f});
00385     wave.enemies.push_back({ServerEntityType::ENEMY_BASIC, 50.0f, 375.0f, 70.0f, 51.5f});
00386     wave.enemies.push_back({ServerEntityType::ENEMY_BASIC, 50.0f, 450.0f, 70.0f, 52.0f});
00387
00388     m_waves.push_back(wave);
00389 }
00390
00391 } // namespace gme

```

19.269 /home/masina/Projects/Epitech/rtype/plugins/Network/Asio/Client/include/AsioClient/AsioClient.hpp File Reference

Asio-based implementation of INetworkClient interface.

```

#include "Interfaces/INetworkClient.hpp"
#include "Interfaces/Protocol/HandlerPacket.hpp"
#include "Interfaces/Protocol/Protocol.hpp"
#include "Utils/Event.hpp"
#include "Utils/EventBus.hpp"
#include <asio.hpp>
#include <atomic>
#include <chrono>
#include <memory>
#include <mutex>
#include <queue>
#include <string>
#include <thread>
#include <vector>

```



```

00009 #include "Interfaces/INetworkClient.hpp"
00010 #include "Interfaces/Protocol/HandlerPacket.hpp"
00011 #include "Interfaces/Protocol/Protocol.hpp"
00012 #include "Utils/Event.hpp"
00013 #include "Utils/EventBus.hpp"
00014
00015 #include <asio.hpp>
00016 #include <atomic>
00017 #include <chrono>
00018 #include <memory>
00019 #include <mutex>
00020 #include <queue>
00021 #include <string>
00022 #include <thread>
00023 #include <vector>
00024
00025 namespace eng
00026 {
00027
00028     ///
00029     /// @brief Queued packet for sending
00030     ///
00031     struct QueuedPacket
00032     {
00033         std::vector<std::uint8_t> data;
00034         bool reliable;
00035
00036         explicit QueuedPacket(const std::vector<std::uint8_t> &d, const bool rel = false) : data(d), reliable(rel)
00037         {
00038         }
00039     };
00040
00041     ///
00042     /// @brief Connection statistics
00043     ///
00044     struct ConnectionStats
00045     {
00046         std::uint32_t packetsSent = 0;
00047         std::uint32_t packetsReceived = 0;
00048         std::uint32_t bytesTransferred = 0;
00049         std::uint32_t packetsLost = 0;
00050         std::chrono::steady_clock::time_point connectionTime;
00051     };
00052
00053     ///
00054     /// @class AsioClient
00055     /// @brief Asio UDP client implementation
00056     /// @namespace eng
00057     ///
00058     class AsioClient final : public INetworkClient
00059     {
00060     public:
00061         ///
00062         /// @brief Constructor
00063         ///
00064         AsioClient();
00065
00066         ///
00067         /// @brief Destructor
00068         ///
00069         ~AsioClient() override;
00070
00071         // INetworkClient implementation
00072         void connect(const std::string &host, std::uint16_t port) override;
00073         void disconnect() override;
00074
00075         void update() override;
00076
00077         [[nodiscard]] const std::string getName() const override { return "Network_Client"; }
00078         [[nodiscard]] utl::PluginType getType() const override { return utl::PluginType::NETWORK_CLIENT; }
00079
00080         [[nodiscard]] bool isConnected() const override;
00081         [[nodiscard]] ConnectionState getConnectionState() const override;
00082         [[nodiscard]] std::uint32_t getSessionId() const override;
00083         [[nodiscard]] std::uint16_t getServerTickRate() const override;
00084         [[nodiscard]] std::uint32_t getLatency() const override;
00085
00086         void setPlayerName(const std::string &playerName) override;
00087         void setClientCapabilities(std::uint32_t caps) override;
00088
00089         void processBusEvent();
00090
00091         ///
00092         /// @brief Send custom packet to server
00093         /// @param data Packet data
00094         /// @param reliable Whether packet should be reliable
00095         ///

```

```

00096 void sendToServer(const std::vector<std::uint8_t> &data, bool reliable = false) override;
00097
00098 ///
00099 /// @brief Get connection statistics
00100 /// @return Current connection stats
00101 ///
00102 const ConnectionStats &getStats() const { return m_stats; }
00103
00104 // Lobby management methods
00105 ///
00106 /// @brief Request lobby list from server
00107 ///
00108 void requestLobbyList();
00109
00110 ///
00111 /// @brief Create a new lobby
00112 /// @param name Lobby name
00113 /// @param maxPlayers Maximum number of players
00114 /// @param gameMode Game mode identifier
00115 ///
00116 void createLobby(rnp::PacketLobbyCreate lobbyCreate);
00117
00118 ///
00119 /// @brief Join an existing lobby
00120 /// @param lobbyId ID of the lobby to join
00121 ///
00122 void joinLobby(std::uint32_t lobbyId);
00123
00124 ///
00125 /// @brief Leave current lobby
00126 ///
00127 void leaveLobby();
00128
00129 ///
00130 /// @brief Request to start the game (host only)
00131 /// @param lobbyId Lobby ID to start
00132 ///
00133 void requestStartGame(std::uint32_t lobbyId);
00134
00135 ///
00136 /// @brief Get current lobby ID
00137 /// @return Current lobby ID (0 if not in lobby)
00138 ///
00139 std::uint32_t getCurrentLobbyId() const { return m_currentLobbyId; }
00140
00141 ///
00142 /// @brief Check if player is in a lobby
00143 /// @return True if in lobby
00144 ///
00145 bool isInLobby() const { return m_currentLobbyId != 0; }
00146
00147 // Callback setters for lobby events
00148 ///
00149 /// @brief Set callback for lobby list received
00150 /// @param callback Function to call when lobby list is received
00151 ///
00152 void setOnLobbyListReceived(std::function<void(const std::vector<rnp::LobbyInfo> &)> callback);
00153
00154 ///
00155 /// @brief Set callback for lobby created
00156 /// @param callback Function to call when lobby creation response is received
00157 ///
00158 void setOnLobbyCreated(std::function<void(std::uint32_t, bool, rnp::ErrorCode)> callback);
00159
00160 ///
00161 /// @brief Set callback for lobby joined
00162 /// @param callback Function to call when lobby join response is received
00163 ///
00164 void
00165 setOnLobbyJoined(std::function<void(std::uint32_t, bool, rnp::ErrorCode, const rnp::LobbyInfo *)> callback);
00166
00167 ///
00168 /// @brief Set callback for lobby updated
00169 /// @param callback Function to call when lobby is updated
00170 ///
00171 void setOnLobbyUpdated(std::function<void(const rnp::LobbyInfo &)> callback);
00172
00173 ///
00174 /// @brief Set callback for game start
00175 /// @param callback Function to call when game starts
00176 ///
00177 void setOnGameStart(std::function<void(std::uint32_t, std::uint32_t)> callback);
00178
00179 private:
00180 // Network components
00181 std::unique_ptr<asio::io_context> m_ioContext;
00182 std::unique_ptr<asio::ip::udp::socket> m_socket;

```

```

00183     std::unique_ptr<std::thread> m_networkThread;
00184
00185     // Server connection info
00186     std::string m_serverHost;
00187     std::uint16_t m_serverPort;
00188     asio::ip::udp::endpoint m_serverEndpoint;
00189
00190     // Connection state
00191     std::atomic<ConnectionState> m_connectionState;
00192     std::uint32_t m_sessionId;
00193     std::uint16_t m_serverTickRate;
00194     std::uint32_t m_clientCaps;
00195
00196     // Client configuration
00197     std::string m_playerName;
00198
00199     // Network state
00200     std::atomic<bool> m_running;
00201
00202     // Packet handling
00203     std::unique_ptr<rnpp::HandlerPacket> m_packetHandler;
00204     std::queue<QueuedPacket> m_sendQueue;
00205     std::mutex m_sendQueueMutex;
00206
00207     // Reception buffer
00208     std::array<std::uint8_t, 1024> m_recvBuffer;
00209     asio::ip::udp::endpoint m_senderEndpoint;
00210
00211     // Ping/Latency management
00212     std::uint32_t m_lastPingNonce;
00213     std::chrono::steady_clock::time_point m_lastPingTime;
00214     std::uint32_t m_latency; // in milliseconds
00215     std::chrono::milliseconds m_pingInterval;
00216
00217     // Connection timeout
00218     std::chrono::steady_clock::time_point m_lastServerResponse;
00219     std::chrono::milliseconds m_connectionTimeout;
00220
00221     // Statistics
00222     ConnectionStats m_stats;
00223
00224     // Lobby state
00225     std::uint32_t m_currentLobbyId;
00226
00227     // Lobby callbacks
00228     std::function<void(const std::vector<rnpp::LobbyInfo> &)> m_onLobbyListReceived;
00229     std::function<void(std::uint32_t, bool, rnpp::ErrorCode)> m_onLobbyCreated;
00230     std::function<void(std::uint32_t, bool, rnpp::ErrorCode, const rnpp::LobbyInfo *)> m_onLobbyJoined;
00231     std::function<void(const rnpp::LobbyInfo &)> m_onLobbyUpdated;
00232     std::function<void(std::uint32_t, std::uint32_t)> m_onGameStart;
00233
00234     utl::EventBus &m_eventBus;
00235     std::uint32_t m_componentId = utl::NETWORK_CLIENT;
00236
00237     ///
00238     /// @brief Initialize packet handlers
00239     ///
00240     void setupPacketHandlers();
00241
00242     ///
00243     /// @brief Start receiving packets asynchronously
00244     ///
00245     void startReceive();
00246
00247     ///
00248     /// @brief Handle received packet
00249     /// @param bytesReceived Number of bytes received
00250     ///
00251     void handleReceive(std::size_t bytesReceived);
00252
00253     ///
00254     /// @brief Network thread main loop
00255     ///
00256     void networkThreadLoop() const;
00257
00258     ///
00259     /// @brief Send packet immediately
00260     /// @param data Packet data
00261     ///
00262     void sendPacketImmediate(const std::vector<std::uint8_t> &data);
00263
00264     ///
00265     /// @brief Handle CONNECT_ACCEPT packet
00266     /// @param packet Connect accept packet
00267     /// @param context Packet context
00268     /// @return Handler result
00269     ///

```

```

00270     rnp::HandlerResult handleConnectAccept(const rnp::PacketConnectAccept &packet,
00271                                           const rnp::PacketContext &context);
00272
00273     ///
00274     /// @brief Handle DISCONNECT packet
00275     /// @param packet Disconnect packet
00276     /// @param context Packet context
00277     /// @return Handler result
00278     ///
00279     rnp::HandlerResult handleDisconnect(const rnp::PacketDisconnect &packet, const rnp::PacketContext &context);
00280
00281     ///
00282     /// @brief Handle PONG packet
00283     /// @param packet Pong packet
00284     /// @param context Packet context
00285     /// @return Handler result
00286     ///
00287     rnp::HandlerResult handlePong(const rnp::PacketPingPong &packet, const rnp::PacketContext &context);
00288
00289     ///
00290     /// @brief Handle PING packet
00291     /// @param packet Ping packet
00292     /// @param context Packet context
00293     /// @return Handler result
00294     ///
00295     rnp::HandlerResult handlePing(const rnp::PacketPingPong &packet, const rnp::PacketContext &context);
00296
00297     ///
00298     /// @brief Handle ERROR packet
00299     /// @param packet Error packet
00300     /// @param context Packet context
00301     /// @return Handler result
00302     ///
00303     static rnp::HandlerResult handleError(const rnp::PacketError &packet, const rnp::PacketContext &context);
00304
00305     ///
00306     /// @brief Handle WORLD_STATE packet
00307     /// @param packet World state packet
00308     /// @param context Packet context
00309     /// @return Handler result
00310     ///
00311     rnp::HandlerResult handleWorldState(const rnp::PacketWorldState &packet,
00312                                         const rnp::PacketContext &context) const;
00313
00314     ///
00315     /// @brief Handle ENTITY_EVENT packet
00316     /// @param events Vector of event records
00317     /// @param context Packet context
00318     /// @return Handler result
00319     ///
00320     rnp::HandlerResult handleEntityEvent(const std::vector<rnp::EventRecord> &events,
00321                                         const rnp::PacketContext &context) const;
00322
00323     ///
00324     /// @brief Handle LOBBY_LIST_RESPONSE packet
00325     /// @param packet Lobby list response packet
00326     /// @param context Packet context
00327     /// @return Handler result
00328     ///
00329     rnp::HandlerResult handleLobbyListResponse(const rnp::PacketLobbyListResponse &packet,
00330                                                const rnp::PacketContext &context) const;
00331
00332     ///
00333     /// @brief Handle LOBBY_CREATE_RESPONSE packet
00334     /// @param packet Lobby create response packet
00335     /// @param context Packet context
00336     /// @return Handler result
00337     ///
00338     rnp::HandlerResult handleLobbyCreateResponse(const rnp::PacketLobbyCreateResponse &packet,
00339                                                  const rnp::PacketContext &context);
00340
00341     ///
00342     /// @brief Handle LOBBY_JOIN_RESPONSE packet
00343     /// @param packet Lobby join response packet
00344     /// @param context Packet context
00345     /// @return Handler result
00346     ///
00347     rnp::HandlerResult handleLobbyJoinResponse(const rnp::PacketLobbyJoinResponse &packet,
00348                                                const rnp::PacketContext &context);
00349
00350     ///
00351     /// @brief Handle LOBBY_UPDATE packet
00352     /// @param packet Lobby update packet
00353     /// @param context Packet context
00354     /// @return Handler result
00355     ///
00356     rnp::HandlerResult handleLobbyUpdate(const rnp::PacketLobbyUpdate &packet,

```

```
00406 } // namespace eng
```

Client/src/asioClient.cpp File Reference

Include dependency graph for asioClient.cpp:



Namespaces

- namespace [eng](#)

19.272 asioClient.cpp

[Go to the documentation of this file.](#)

```

00001 ///
00002 /// @file AsioClient.cpp
00003 /// @brief Asio-based implementation of INetworkClient interface
00004 /// @namespace eng
00005 ///
00006
00007 #include "AsioClient/AsioClient.hpp"
00008 #include "Interfaces/Protocol/HandlerPacket.hpp"
00009 #include "Interfaces/Protocol/Protocol.hpp"
00010 #include "Interfaces/Protocol/Serializer.hpp"
00011 #include "Utils/Event.hpp"
00012 #include "Utils/Logger.hpp"
00013
00014 #include <chrono>
00015 #include <cstdint>
00016 #include <random>
00017
00018 namespace eng
00019 {
00020
00021     AsioClient::AsioClient()
00022         : m_ioContext(std::make_unique<asio::io_context>()),
00023           m_socket(std::make_unique<asio::ip::udp::socket>(*m_ioContext)), m_serverPort(0),
00024           m_connectionState(ConnectionState::DISCONNECTED), m_sessionId(0), m_serverTickRate(60),
00025           m_clientCaps(0),
00026           m_running(false), m_packetHandler(std::make_unique<rng::HandlerPacket>()), m_lastPingNonce(0),
00027           m_latency(0),
00028           m_pingInterval(std::chrono::seconds(5)), m_connectionTimeout(std::chrono::seconds(30)), m_currentLobbyId(0),
00029           m_eventBus(utl::EventBus::getInstance()), m_componentId(utl::NETWORK_CLIENT)
00030     {
00031         m_stats.connectionTime = std::chrono::steady_clock::now();
00032         setupPacketHandlers();
00033         utl::Logger::log("AsioClient: Initialized", utl::LogLevel::INFO);
00034
00035         m_eventBus.registerComponent(m_componentId, "Asio Client");
00036         m_eventBus.subscribe(m_componentId, utl::EventType::SEND_PLAYER_INPUT);
00037         m_eventBus.subscribe(m_componentId, utl::EventType::REQUEST_CONNECT);
00038         m_eventBus.subscribe(m_componentId, utl::EventType::REQUEST_DISCONNECT);
00039         m_eventBus.subscribe(m_componentId, utl::EventType::SEND_ENTITY_EVENT);
00040         m_eventBus.subscribe(m_componentId, utl::EventType::LOBBY_CREATE);
00041         m_eventBus.subscribe(m_componentId, utl::EventType::LOBBY_JOIN);
00042         m_eventBus.subscribe(m_componentId, utl::EventType::LOBBY_LEAVE);
00043         m_eventBus.subscribe(m_componentId, utl::EventType::LOBBY_LIST_REQUEST);
00044         // Note: LOBBY_UPDATE and GAME_START are published by this client (not consumed)
00045         // so no subscription is needed here
00046     }
00047
00048     AsioClient::~AsioClient()
00049     {
00050         AsioClient::disconnect();
00051         utl::Logger::log("AsioClient: Destroyed", utl::LogLevel::INFO);
00052     }
00053
00054     void AsioClient::connect(const std::string &host, std::uint16_t port)
00055     {
00056         if (m_connectionState.load() != ConnectionState::DISCONNECTED)
00057         {
00058             utl::Logger::log("AsioClient: Already connected or connecting", utl::LogLevel::WARNING);
00059             return;
00060         }
00061
00062         m_serverHost = host;
00063         m_serverPort = port;
00064         m_connectionState.store(ConnectionState::CONNECTING);
00065
00066         try
00067         {
00068             // Resolve server address
00069             asio::ip::udp::resolver resolver(*m_ioContext);
00070             auto endpoints = resolver.resolve(host, std::to_string(port));
00071             m_serverEndpoint = *endpoints.begin();
00072
00073             // Open socket
00074             m_socket->open(asio::ip::udp::v4());
00075
00076             m_running.store(true);
00077
00078         }
00079     }

```



```

00076         // Start network thread
00077         m_networkThread = std::make_unique<std::thread>(&AsioClient::networkThreadLoop, this);
00078
00079         // Start receiving
00080         startReceive();
00081
00082         // Send connect packet
00083         sendConnect();
00084
00085         m_lastServerResponse = std::chrono::steady_clock::now();
00086
00087         utl::Logger::log("AsioClient: Connecting to " + host + ":" + std::to_string(port), utl::LogLevel::INFO);
00088     }
00089     catch (const std::exception &e)
00090     {
00091         m_connectionState.store(ConnectionState::DISCONNECTED);
00092         m_running.store(false);
00093         utl::Logger::log("AsioClient: Failed to connect - " + std::string(e.what()), utl::LogLevel::WARNING);
00094         throw;
00095     }
00096 }
00097
00098 void AsioClient::disconnect()
00099 {
00100     if (m_connectionState.load() == ConnectionState::DISCONNECTED)
00101     {
00102         return;
00103     }
00104
00105     m_connectionState.store(ConnectionState::DISCONNECTING);
00106
00107     // Send disconnect packet if connected
00108     if (m_sessionId != 0)
00109     {
00110         sendDisconnect();
00111         std::this_thread::sleep_for(std::chrono::milliseconds(100)); // Allow time for packet to be sent
00112     }
00113
00114     m_running.store(false);
00115
00116     // Stop io_context
00117     if (m_ioContext)
00118     {
00119         m_ioContext->stop();
00120     }
00121
00122     // Wait for network thread
00123     if (m_networkThread && m_networkThread->joinable())
00124     {
00125         m_networkThread->join();
00126     }
00127
00128     // Close socket
00129     if (m_socket && m_socket->is_open())
00130     {
00131         m_socket->close();
00132     }
00133
00134     // Reset state
00135     m_connectionState.store(ConnectionState::DISCONNECTED);
00136     m_sessionId = 0;
00137     m_serverTickRate = 60;
00138     m_latency = 0;
00139
00140     // Clear send queue
00141     {
00142         std::lock_guard<std::mutex> lock(m_sendQueueMutex);
00143         while (!m_sendQueue.empty())
00144         {
00145             m_sendQueue.pop();
00146         }
00147     }
00148
00149     utl::Logger::log("AsioClient: Disconnected", utl::LogLevel::INFO);
00150 }
00151
00152 void AsioClient::update()
00153 {
00154     // Process EventBus events
00155     processBusEvent();
00156
00157     if (m_connectionState.load() != ConnectionState::CONNECTED)
00158     {
00159         // utl::Logger::log("AsioClient: Update skipped - not connected (state: " +
00160         //                 std::to_string(static_cast<int>(m_connectionState.load())) + ")",
00161         //                 utl::LogLevel::WARNING);
00162         return;

```

```

00163     }
00164
00165     // Process send queue
00166     processSendQueue();
00167
00168     // Update connection management
00169     updateConnectionManagement();
00170 }
00171
00172 bool AsioClient::isConnected() const { return m_connectionState.load() == ConnectionState::CONNECTED; }
00173
00174 ConnectionState AsioClient::getConnectionState() const { return m_connectionState.load(); }
00175
00176 std::uint32_t AsioClient::getSessionId() const { return m_sessionId; }
00177
00178 std::uint16_t AsioClient::getServerTickRate() const { return m_serverTickRate; }
00179
00180 std::uint32_t AsioClient::getLatency() const { return m_latency; }
00181
00182 void AsioClient::setPlayerName(const std::string &playerName)
00183 {
00184     m_playerName = playerName;
00185     utl::Logger::log("AsioClient: Player name set to " + playerName, utl::LogLevel::INFO);
00186 }
00187
00188 void AsioClient::setClientCapabilities(std::uint32_t caps)
00189 {
00190     m_clientCaps = caps;
00191     utl::Logger::log("AsioClient: Client capabilities set to " + std::to_string(caps), utl::LogLevel::INFO);
00192 }
00193
00194 void AsioClient::sendToServer(const std::vector<std::uint8_t> &data, bool reliable)
00195 {
00196     if (m_connectionState.load() != ConnectionState::CONNECTED)
00197     {
00198         utl::Logger::log("AsioClient: Attempted to send while not connected", utl::LogLevel::WARNING);
00199         return;
00200     }
00201
00202     std::lock_guard<std::mutex> lock(m_sendQueueMutex);
00203     m_sendQueue.emplace(data, reliable);
00204 }
00205
00206 void AsioClient::setupPacketHandlers()
00207 {
00208     // CONNECT_ACCEPT handler
00209     m_packetHandler->onConnectAccept(
00210         [this](const rnp::PacketConnectAccept &packet, const rnp::PacketContext &context)
00211         { return handleConnectAccept(packet, context); });
00212
00213     // DISCONNECT handler
00214     m_packetHandler->onDisconnect([this](const rnp::PacketDisconnect &packet, const rnp::PacketContext &context)
00215         { return handleDisconnect(packet, context); });
00216
00217     // PONG handler
00218     m_packetHandler->onPong([this](const rnp::PacketPingPong &packet, const rnp::PacketContext &context)
00219         { return handlePong(packet, context); });
00220
00221     // PING handler (server can ping client too)
00222     m_packetHandler->onPing([this](const rnp::PacketPingPong &packet, const rnp::PacketContext &context)
00223         { return handlePing(packet, context); });
00224
00225     // ERROR handler
00226     m_packetHandler->onError([](const rnp::PacketError &packet, const rnp::PacketContext &context)
00227         { return handleError(packet, context); });
00228
00229     // WORLD_STATE handler
00230     m_packetHandler->onWorldState([this](const rnp::PacketWorldState &packet, const rnp::PacketContext &context)
00231         { return handleWorldState(packet, context); });
00232
00233     // ENTITY_EVENT handler (for server events)
00234     m_packetHandler->onEntityEvent(
00235         [this](const std::vector<rnp::EventRecord> &events, const rnp::PacketContext &context)
00236         { return handleEntityEvent(events, context); });
00237
00238     // LOBBY_LIST_RESPONSE handler
00239     m_packetHandler->onLobbyListResponse(
00240         [this](const rnp::PacketLobbyListResponse &packet, const rnp::PacketContext &context)
00241         { return handleLobbyListResponse(packet, context); });
00242
00243     // LOBBY_CREATE_RESPONSE handler
00244     m_packetHandler->onLobbyCreateResponse(
00245         [this](const rnp::PacketLobbyCreateResponse &packet, const rnp::PacketContext &context)
00246         { return handleLobbyCreateResponse(packet, context); });
00247
00248     // LOBBY_JOIN_RESPONSE handler
00249     m_packetHandler->onLobbyJoinResponse(

```

```

00250         [this](const rnp::PacketLobbyJoinResponse &packet, const rnp::PacketContext &context)
00251         { return handleLobbyJoinResponse(packet, context); });
00252
00253         // LOBBY_UPDATE handler
00254         m_packetHandler->onLobbyUpdate([this](const rnp::PacketLobbyUpdate &packet, const rnp::PacketContext
&context)
00255                                     { return handleLobbyUpdate(packet, context); });
00256
00257         // GAME_START handler
00258         m_packetHandler->onGameStart([this](const rnp::PacketGameStart &packet, const rnp::PacketContext &context)
00259                                     { return handleGameStart(packet, context); });
00260
00261         utl::Logger::log("AsioClient: Packet handlers initialized", utl::LogLevel::INFO);
00262     }
00263
00264     void AsioClient::startReceive()
00265     {
00266         m_socket->async_receive_from(asio::buffer(m_recvBuffer), m_senderEndpoint,
00267                                     [this](std::error_code ec, std::size_t bytesReceived)
00268                                     {
00269                                         if (!ec && m_running.load())
00270                                         {
00271                                             handleReceive(bytesReceived);
00272                                             startReceive(); // Continue receiving
00273                                         }
00274                                         else if (m_running.load())
00275                                         {
00276                                             utl::Logger::log("AsioClient: Receive error - " + ec.message(),
00277                                                             utl::LogLevel::WARNING);
00278                                             startReceive(); // Try to continue
00279                                         }
00280                                     });
00281     }
00282
00283     void AsioClient::handleReceive(std::size_t bytesReceived)
00284     {
00285         // PacketHeader serialized size: 1 (type) + 2 (length) + 4 (sessionId) = 7 bytes
00286         constexpr std::size_t PACKET_HEADER_SIZE = 7;
00287         if (bytesReceived < PACKET_HEADER_SIZE)
00288         {
00289             utl::Logger::log("AsioClient: Received packet too small (need " + std::to_string(PACKET_HEADER_SIZE) +
00290                             " bytes, got " + std::to_string(bytesReceived) + ")",
00291                             utl::LogLevel::WARNING);
00292             return;
00293         }
00294
00295         m_lastServerResponse = std::chrono::steady_clock::now();
00296         m_stats.packetsReceived++;
00297         m_stats.bytesTransferred += static_cast<std::uint32_t>(bytesReceived);
00298
00299         // Create packet context
00300         rnp::PacketContext context;
00301         context.receiveTime = std::chrono::steady_clock::now();
00302         context.senderAddress = m_senderEndpoint.address().to_string();
00303         context.senderPort = m_senderEndpoint.port();
00304
00305         // Extract session ID from header for context
00306         std::vector<std::uint8_t> data(m_recvBuffer.begin(), m_recvBuffer.begin() + bytesReceived);
00307         if (data.size() >= PACKET_HEADER_SIZE)
00308         {
00309             rnp::Serializer headerSerializer(data);
00310             rnp::PacketHeader header = headerSerializer.deserializeHeader();
00311             context.sessionId = header.sessionId;
00312         }
00313
00314         // Process packet
00315         rnp::HandlerResult result = m_packetHandler->processPacket(data, context);
00316         if (result != rnp::HandlerResult::SUCCESS)
00317         {
00318             utl::Logger::log("AsioClient: Packet processing failed with result " +
00319                             std::to_string(static_cast<int>(result)),
00320                             utl::LogLevel::WARNING);
00321         }
00322     }
00323
00324     void AsioClient::networkThreadLoop() const
00325     {
00326         utl::Logger::log("AsioClient: Network thread started", utl::LogLevel::INFO);
00327
00328         while (m_running.load())
00329         {
00330             try
00331             {
00332                 m_ioContext->run_for(std::chrono::milliseconds(100));
00333                 if (!m_running.load())
00334                 {
00335                     break;

```

```

00336         }
00337         m_ioContext->restart();
00338     }
00339     catch (const std::exception &e)
00340     {
00341         utl::Logger::log("AsioClient: Network thread exception - " + std::string(e.what()),
00342             utl::LogLevel::WARNING);
00343         std::this_thread::sleep_for(std::chrono::milliseconds(100));
00344     }
00345 }
00346
00347 utl::Logger::log("AsioClient: Network thread stopped", utl::LogLevel::INFO);
00348 }
00349
00350 void AsioClient::sendPacketImmediate(const std::vector<std::uint8_t> &data)
00351 {
00352     if (!m_socket || !m_socket->is_open())
00353     {
00354         utl::Logger::log("AsioClient: Cannot send packet - socket not open", utl::LogLevel::WARNING);
00355         return;
00356     }
00357
00358     utl::Logger::log("AsioClient: Sending " + std::to_string(data.size()) + " bytes to " +
00359         m_serverEndpoint.address().to_string() + ":" + std::to_string(m_serverEndpoint.port()),
00360         utl::LogLevel::INFO);
00361
00362     try
00363     {
00364         size_t bytesSent = m_socket->send_to(asio::buffer(data), m_serverEndpoint);
00365         m_stats.packetsSent++;
00366         m_stats.bytesTransferred += static_cast<std::uint32_t>(data.size());
00367         utl::Logger::log("AsioClient: Successfully sent " + std::to_string(bytesSent) + " bytes",
00368             utl::LogLevel::INFO);
00369     }
00370     catch (const std::exception &e)
00371     {
00372         utl::Logger::log("AsioClient: Failed to send packet - " + std::string(e.what()), utl::LogLevel::WARNING);
00373     }
00374 }
00375
00376 rnp::HandlerResult AsioClient::handleConnectAccept(const rnp::PacketConnectAccept &packet,
00377     const rnp::PacketContext &context)
00378 {
00379     m_sessionId = packet.sessionId;
00380     m_serverTickRate = packet.tickRateHz;
00381     m_connectionState.store(ConnectionState::CONNECTED);
00382
00383     utl::Logger::log("AsioClient: Connection accepted - Session ID: " + std::to_string(m_sessionId) +
00384         ", Tick rate: " + std::to_string(m_serverTickRate) + " Hz",
00385         utl::LogLevel::INFO);
00386
00387     rnp::Serializer serializer;
00388     m_eventBus.publish(utl::EventType::CONNECTION_ACCEPTED, serializer.getData(), m_componentId,
00389         utl::RENDERING_ENGINE); // GameClient ID
00390     return rnp::HandlerResult::SUCCESS;
00391 }
00392
00393 rnp::HandlerResult AsioClient::handleDisconnect(const rnp::PacketDisconnect &packet,
00394     const rnp::PacketContext &context)
00395 {
00396     auto reason = static_cast<rnp::DisconnectReason>(packet.reasonCode);
00397     std::string reasonStr = "Unknown";
00398
00399     switch (reason)
00400     {
00401     case rnp::DisconnectReason::CLIENT_REQUEST:
00402         reasonStr = "Client request";
00403         break;
00404     case rnp::DisconnectReason::TIMEOUT:
00405         reasonStr = "Timeout";
00406         break;
00407     case rnp::DisconnectReason::PROTOCOL_ERROR:
00408         reasonStr = "Protocol error";
00409         break;
00410     case rnp::DisconnectReason::SERVER_SHUTDOWN:
00411         reasonStr = "Server shutdown";
00412         break;
00413     case rnp::DisconnectReason::SERVER_FULL:
00414         reasonStr = "Server full";
00415         break;
00416     case rnp::DisconnectReason::BANNED:
00417         reasonStr = "Banned";
00418         break;
00419     case rnp::DisconnectReason::UNSPECIFIED:
00420         reasonStr = "Unspecified";
00421         break;
00422     }

```

```

00423
00424     utl::Logger::log("AsioClient: Disconnected by server - Reason: " + reasonStr, utl::LogLevel::INFO);
00425
00426     m_connectionState.store(ConnectionState::DISCONNECTED);
00427     m_sessionId = 0;
00428
00429     return rnp::HandlerResult::SUCCESS;
00430 }
00431
00432 rnp::HandlerResult AsioClient::handlePong(const rnp::PacketPingPong &packet, const rnp::PacketContext &context)
00433 {
00434     if (packet.nonce == m_lastPingNonce)
00435     {
00436         auto now = std::chrono::steady_clock::now();
00437         auto duration = std::chrono::duration_cast<std::chrono::milliseconds>(now - m_lastPingTime);
00438         m_latency = static_cast<std::uint32_t>(duration.count());
00439
00440         utl::Logger::log("AsioClient: Ping response received - Latency: " + std::to_string(m_latency) + "ms",
00441             utl::LogLevel::INFO);
00442     }
00443
00444     return rnp::HandlerResult::SUCCESS;
00445 }
00446
00447 rnp::HandlerResult AsioClient::handlePing(const rnp::PacketPingPong &packet, const rnp::PacketContext &context)
00448 {
00449     // Server is pinging us, send pong response
00450     sendPong(packet.nonce);
00451     return rnp::HandlerResult::SUCCESS;
00452 }
00453
00454 rnp::HandlerResult AsioClient::handleError(const rnp::PacketError &packet, const rnp::PacketContext &context)
00455 {
00456     const auto errorCode = static_cast<rnp::ErrorCode>(packet.errorCode);
00457     std::string errorStr = "Unknown error";
00458
00459     switch (errorCode)
00460     {
00461     case rnp::ErrorCode::INVALID_PAYLOAD:
00462         errorStr = "Invalid payload";
00463         break;
00464     case rnp::ErrorCode::UNAUTHORIZED_SESSION:
00465         errorStr = "Unauthorized session";
00466         break;
00467     case rnp::ErrorCode::RATE_LIMITED:
00468         errorStr = "Rate limited";
00469         break;
00470     case rnp::ErrorCode::INTERNAL_ERROR:
00471         errorStr = "Internal server error";
00472         break;
00473     default:
00474         errorStr = "Unknown error";
00475         break;
00476     }
00477
00478     utl::Logger::log("AsioClient: Server error - " + errorStr + ": " + packet.description, utl::LogLevel::WARNING);
00479
00480     return rnp::HandlerResult::SUCCESS;
00481 }
00482
00483 rnp::HandlerResult AsioClient::handleWorldState(const rnp::PacketWorldState &packet,
00484     const rnp::PacketContext &context) const
00485 {
00486     // Handle world state update
00487     utl::Logger::log("AsioClient: World state received - Tick: " + std::to_string(packet.serverTick) +
00488         ", Entities: " + std::to_string(packet.entityCount),
00489         utl::LogLevel::INFO);
00490
00491     // Forward to all interested components via EventBus (broadcast)
00492     m_eventBus.publish(utl::EventType::WORLD_STATE_RECEIVED, packet, m_componentId, 0);
00493
00494     return rnp::HandlerResult::SUCCESS;
00495 }
00496
00497 void AsioClient::sendConnect()
00498 {
00499     rnp::Serializer serializer;
00500     rnp::PacketHeader header{};
00501     header.type = static_cast<std::uint8_t>(rnp::PacketType::CONNECT);
00502     header.length = sizeof(rnp::PacketConnect);
00503     header.sessionId = 0; // No session ID yet
00504
00505     rnp::PacketConnect connect{};
00506     connect.nameLen = static_cast<std::uint8_t>(
00507         std::min(m_playerName.length(), static_cast<std::size_t>(connect.playerName.size() - 1)));
00508     std::memset(connect.playerName.data(), 0, connect.playerName.size());
00509     if (!m_playerName.empty())

```

```

00510     {
00511         std::memcpy(connect.playerName.data(), m_playerName.data(), connect.nameLen);
00512     }
00513     connect.clientCaps = m_clientCaps;
00514
00515     serializer.serializeHeader(header);
00516     serializer.serializeConnect(connect);
00517
00518     sendPacketImmediate(serializer.getData());
00519     utl::Logger::log("AsioClient: CONNECT packet sent", utl::LogLevel::INFO);
00520 }
00521
00522 void AsioClient::sendDisconnect()
00523 {
00524     if (m_sessionId == 0)
00525     {
00526         return;
00527     }
00528
00529     rnp::Serializer serializer;
00530     rnp::PacketHeader header{};
00531     header.type = static_cast<std::uint8_t>(rnp::PacketType::DISCONNECT);
00532     header.length = sizeof(rnp::PacketDisconnect);
00533     header.sessionId = m_sessionId;
00534
00535     rnp::PacketDisconnect disconnect{};
00536     disconnect.reasonCode = static_cast<std::uint16_t>(rnp::DisconnectReason::CLIENT_REQUEST);
00537
00538     serializer.serializeHeader(header);
00539     serializer.serializeDisconnect(disconnect);
00540
00541     sendPacketImmediate(serializer.getData());
00542     utl::Logger::log("AsioClient: DISCONNECT packet sent", utl::LogLevel::INFO);
00543 }
00544
00545 void AsioClient::sendPing()
00546 {
00547     if (m_sessionId == 0)
00548     {
00549         return;
00550     }
00551
00552     m_lastPingNonce = generatePingNonce();
00553     m_lastPingTime = std::chrono::steady_clock::now();
00554
00555     rnp::Serializer serializer;
00556     rnp::PacketHeader header{};
00557     header.type = static_cast<std::uint8_t>(rnp::PacketType::PING);
00558     header.length = sizeof(rnp::PacketPingPong);
00559     header.sessionId = m_sessionId;
00560
00561     rnp::PacketPingPong ping{};
00562     ping.nonce = m_lastPingNonce;
00563     ping.sendTimeMs = static_cast<std::uint32_t>(
00564         std::chrono::duration_cast<std::chrono::milliseconds>(m_lastPingTime.time_since_epoch()).count());
00565
00566     serializer.serializeHeader(header);
00567     serializer.serializePingPong(ping);
00568
00569     sendPacketImmediate(serializer.getData());
00570 }
00571
00572 void AsioClient::sendPong(std::uint32_t nonce)
00573 {
00574     if (m_sessionId == 0)
00575     {
00576         return;
00577     }
00578
00579     rnp::Serializer serializer;
00580     rnp::PacketHeader header{};
00581     header.type = static_cast<std::uint8_t>(rnp::PacketType::PONG);
00582     header.length = sizeof(rnp::PacketPingPong);
00583     header.sessionId = m_sessionId;
00584
00585     rnp::PacketPingPong pong{};
00586     pong.nonce = nonce;
00587     pong.sendTimeMs = static_cast<std::uint32_t>(
00588         std::chrono::duration_cast<std::chrono::milliseconds>(std::chrono::steady_clock::now().time_since_epoch())
00589             .count());
00590
00591     serializer.serializeHeader(header);
00592     serializer.serializePingPong(pong);
00593
00594     sendPacketImmediate(serializer.getData());
00595 }
00596

```

```

00597 void AsioClient::processSendQueue()
00598 {
00599     std::lock_guard<std::mutex> lock(m_sendQueueMutex);
00600     if (!m_sendQueue.empty())
00601     {
00602         utl::Logger::log("AsioClient: Processing send queue with " + std::to_string(m_sendQueue.size()) +
00603             " packets",
00604             utl::LogLevel::INFO);
00605     }
00606     while (!m_sendQueue.empty())
00607     {
00608         const QueuedPacket &packet = m_sendQueue.front();
00609         sendPacketImmediate(packet.data);
00610         m_sendQueue.pop();
00611     }
00612 }
00613
00614 void AsioClient::updateConnectionManagement()
00615 {
00616     auto now = std::chrono::steady_clock::now();
00617
00618     // Send periodic pings when connected
00619     if (m_connectionState.load() == ConnectionState::CONNECTED)
00620     {
00621         if (now - m_lastPingTime > m_pingInterval)
00622         {
00623             sendPing();
00624         }
00625     }
00626
00627     // Check for connection timeout
00628     if (m_connectionState.load() == ConnectionState::CONNECTING ||
00629         m_connectionState.load() == ConnectionState::CONNECTED)
00630     {
00631         if (now - m_lastServerResponse > m_connectionTimeout)
00632         {
00633             utl::Logger::log("AsioClient: Connection timed out", utl::LogLevel::WARNING);
00634             m_connectionState.store(ConnectionState::DISCONNECTED);
00635             m_sessionId = 0;
00636         }
00637     }
00638
00639     // Retry connection if in CONNECTING state and enough time has passed
00640     if (m_connectionState.load() == ConnectionState::CONNECTING)
00641     {
00642         static auto lastConnectAttempt = std::chrono::steady_clock::now();
00643         if (now - lastConnectAttempt > std::chrono::seconds(2))
00644         {
00645             sendConnect();
00646             lastConnectAttempt = now;
00647         }
00648     }
00649 }
00650
00651 std::uint32_t AsioClient::generatePingNonce()
00652 {
00653     static std::random_device rd;
00654     static std::mt19937 gen(rd());
00655     static std::uniform_int_distribution<std::uint32_t> dis(1, UINT32_MAX);
00656     return dis(gen);
00657 }
00658
00659 void AsioClient::processBusEvent()
00660 {
00661     const auto events = m_eventBus.consumeForTarget(m_componentId);
00662     for (const auto &e : events)
00663     {
00664         utl::Logger::log("AsioClient: Processing EventBus event type " +
00665             std::to_string(static_cast<std::uint32_t>(e.type)),
00666             utl::LogLevel::INFO);
00667         switch (e.type)
00668         {
00669             case utl::EventType::SEND_PLAYER_INPUT:
00670             {
00671                 sendToServer(e.data);
00672                 break;
00673             }
00674             case utl::EventType::REQUEST_CONNECT:
00675             {
00676                 rnp::Serializer serializer(e.data);
00677                 std::string playerName = serializer.readString(32);
00678                 std::string serverIP = serializer.readString(15);
00679                 unsigned short serverPortStr = std::stoi(serializer.readString(5).c_str());
00680                 utl::Logger::log("AsioClient: Received REQUEST_CONNECT event - Player: " + playerName +
00681                     ", Server: " + serverIP + ":" + std::to_string(serverPortStr),
00682                     utl::LogLevel::INFO);
00683                 if (m_connectionState == ConnectionState::DISCONNECTED)

```



```

00684         {
00685             setPlayerName(playerName);
00686             connect(serverIP, serverPortStr);
00687         }
00688         break;
00689     }
00690     case utl::EventType::REQUEST_DISCONNECT:
00691     {
00692         if (m_connectionState == ConnectionState::CONNECTED)
00693         {
00694             disconnect();
00695         }
00696         break;
00697     }
00698     case utl::EventType::SEND_ENTITY_EVENT:
00699     {
00700         // Create full packet with header and payload
00701         rnp::Serializer packetSerializer;
00702
00703         // Create header for ENTITY_EVENT packet
00704         rnp::PacketHeader header;
00705         header.type = static_cast<std::uint8_t>(rnp::PacketType::ENTITY_EVENT);
00706         header.length = static_cast<std::uint16_t>(e.data.size());
00707         header.sessionId = m_sessionId;
00708
00709         // Serialize header and payload
00710         packetSerializer.serializeHeader(header);
00711         packetSerializer.writeBytes(e.data.data(), e.data.size());
00712
00713         // Send to server
00714         sendToServer(packetSerializer.getData());
00715         break;
00716     }
00717     case utl::EventType::LOBBY_LIST_REQUEST:
00718     {
00719         requestLobbyList();
00720         break;
00721     }
00722     case utl::EventType::LOBBY_CREATE:
00723     {
00724         // Check if this is actually a START_GAME_REQUEST (just uint32_t lobbyId)
00725         if (e.data.size() == sizeof(std::uint32_t))
00726         {
00727             // This is a START_GAME_REQUEST disguised as LOBBY_CREATE
00728             std::uint32_t lobbyId;
00729             std::memcpy(&lobbyId, e.data.data(), sizeof(std::uint32_t));
00730             requestStartGame(lobbyId);
00731         }
00732         else
00733         {
00734             // Normal LOBBY_CREATE
00735             rnp::Serializer serializer(e.data);
00736             rnp::PacketLobbyCreate lobbyCreate = serializer.deserializeLobbyCreate();
00737             std::string lobbyName(lobbyCreate.lobbyName.data(), lobbyCreate.nameLen);
00738             std::uint8_t maxPlayers = lobbyCreate.maxPlayers;
00739             std::uint8_t gameMode = lobbyCreate.gameMode;
00740             utl::Logger::log("AsioClient: Received LOBBY_CREATE event - Name: " + lobbyName +
00741                             ", Max Players: " + std::to_string(maxPlayers) +
00742                             ", Game Mode: " + std::to_string(gameMode),
00743                             utl::LogLevel::INFO);
00744             createLobby(lobbyCreate);
00745         }
00746         break;
00747     }
00748     case utl::EventType::LOBBY_JOIN:
00749     {
00750         rnp::Serializer serializer(e.data);
00751         std::uint32_t lobbyId = serializer.readUInt32();
00752         joinLobby(lobbyId);
00753         break;
00754     }
00755     case utl::EventType::LOBBY_LEAVE:
00756     {
00757         utl::Logger::log("AsioClient: Received LOBBY_LEAVE event", utl::LogLevel::INFO);
00758         leaveLobby();
00759         break;
00760     }
00761     case utl::EventType::WORLD_STATE_RECEIVED:
00762     {
00763         // This event is for other components (GameMulti), not AsioClient
00764         // Just ignore it here
00765         break;
00766     }
00767     default:
00768     {
00769         utl::Logger::log("AsioClient: Unhandled event type: " + std::to_string(static_cast<int>(e.type)),
00770                         utl::LogLevel::WARNING);

```



```

00771         break;
00772     }
00773 }
00774 }
00775 }
00776
00777 rnp::HandlerResult AsioClient::handleEntityEvent(const std::vector<rnp::EventRecord> &events,
00778                                                  const rnp::PacketContext &context) const
00779 {
00780     utl::Logger::log("AsioClient: Received " + std::to_string(events.size()) + " entity events from server",
00781                     utl::LogLevel::INFO);
00782
00783     m_eventBus.publish(utl::EventType::ENTITY_EVENT_RECEIVED, events, m_componentId, 0);
00784
00785     return rnp::HandlerResult::SUCCESS;
00786 }
00787
00788 // Lobby System Implementation
00789
00790 void AsioClient::requestLobbyList()
00791 {
00792     if (m_connectionState.load() != ConnectionState::CONNECTED)
00793     {
00794         utl::Logger::log("AsioClient: Cannot request lobby list - not connected", utl::LogLevel::WARNING);
00795         return;
00796     }
00797
00798     utl::Logger::log("AsioClient: Requesting lobby list from server", utl::LogLevel::INFO);
00799
00800     rnp::PacketHeader header;
00801     header.type = static_cast<std::uint8_t>(rnp::PacketType::LOBBY_LIST_REQUEST);
00802     header.length = 0;
00803     header.sessionId = m_sessionId;
00804
00805     std::vector<std::uint8_t> packet;
00806     rnp::Serializer serializer(packet);
00807     serializer.serializeHeader(header);
00808
00809     std::cout << "Sending lobby list request packet of size " << serializer.getSize() << std::endl;
00810     sendToServer(serializer.getData(), true);
00811 }
00812
00813 void AsioClient::createLobby(rnp::PacketLobbyCreate lobbyPacket)
00814 {
00815     if (m_connectionState.load() != ConnectionState::CONNECTED)
00816     {
00817         utl::Logger::log("AsioClient: Cannot create lobby - not connected", utl::LogLevel::WARNING);
00818         return;
00819     }
00820
00821     if (m_currentLobbyId != 0)
00822     {
00823         utl::Logger::log("AsioClient: Cannot create lobby - already in lobby", utl::LogLevel::WARNING);
00824         return;
00825     }
00826
00827     std::string name(lobbyPacket.lobbyName.data(), lobbyPacket.nameLen);
00828     utl::Logger::log("AsioClient: Creating lobby '" + name + "'", utl::LogLevel::INFO);
00829
00830     utl::Logger::log("AsioClient: Lobby create packet - Name: " + name +
00831                     ", Max Players: " + std::to_string(lobbyPacket.maxPlayers) +
00832                     ", Game Mode: " + std::to_string(lobbyPacket.gameMode),
00833                     utl::LogLevel::WARNING);
00834
00835     std::vector<std::uint8_t> data;
00836     rnp::Serializer dataSerializer(data);
00837
00838     rnp::PacketHeader header;
00839     header.type = static_cast<std::uint8_t>(rnp::PacketType::LOBBY_CREATE);
00840     header.length = static_cast<std::uint16_t>(sizeof(rnp::PacketLobbyCreate));
00841     header.sessionId = m_sessionId;
00842     utl::Logger::log("AsioClient: Lobby create header - Type: " + std::to_string(header.type) + ", Length: " +
00843                     std::to_string(header.length) + ", Session ID: " + std::to_string(header.sessionId),
00844                     utl::LogLevel::WARNING);
00845
00846     dataSerializer.serializeHeader(header);
00847     dataSerializer.serializeLobbyCreate(lobbyPacket);
00848     std::cout << "Sending lobby create packet of size " << dataSerializer.getSize() << std::endl;
00849     sendToServer(dataSerializer.getData(), true);
00850 }
00851
00852 void AsioClient::joinLobby(std::uint32_t lobbyId)
00853 {
00854     if (m_connectionState.load() != ConnectionState::CONNECTED)
00855     {
00856         utl::Logger::log("AsioClient: Cannot join lobby - not connected", utl::LogLevel::WARNING);
00857         return;

```

```

00858     }
00859
00860     if (m_currentLobbyId != 0)
00861     {
00862         utl::Logger::log("AsioClient: Cannot join lobby - already in lobby", utl::LogLevel::WARNING);
00863         return;
00864     }
00865
00866     utl::Logger::log("AsioClient: Joining (lobby " + std::to_string(lobbyId), utl::LogLevel::INFO);
00867
00868     rnp::PacketLobbyJoin joinPacket;
00869     joinPacket.lobbyId = lobbyId;
00870
00871     rnp::PacketHeader header;
00872     header.type = static_cast<std::uint8_t>(rnp::PacketType::LOBBY_JOIN);
00873     header.length = static_cast<std::uint16_t>(sizeof(rnp::PacketLobbyJoin));
00874     header.sessionId = m_sessionId;
00875
00876     std::vector<std::uint8_t> packet;
00877     rnp::Serializer packetSerializer(packet);
00878     packetSerializer.serializeHeader(header);
00879     packetSerializer.serializeLobbyJoin(joinPacket);
00880
00881     sendToServer(packetSerializer.getData(), true);
00882 }
00883
00884 void AsioClient::leaveLobby()
00885 {
00886     if (m_connectionState.load() != ConnectionState::CONNECTED)
00887     {
00888         utl::Logger::log("AsioClient: Cannot leave lobby - not connected", utl::LogLevel::WARNING);
00889         return;
00890     }
00891
00892     if (m_currentLobbyId == 0)
00893     {
00894         utl::Logger::log("AsioClient: Cannot leave lobby - not in lobby", utl::LogLevel::WARNING);
00895         return;
00896     }
00897
00898     utl::Logger::log("AsioClient: Leaving lobby " + std::to_string(m_currentLobbyId), utl::LogLevel::INFO);
00899
00900     rnp::PacketHeader header;
00901     header.type = static_cast<std::uint8_t>(rnp::PacketType::LOBBY_LEAVE);
00902     header.length = 0;
00903     header.sessionId = m_sessionId;
00904
00905     std::vector<std::uint8_t> packet;
00906     rnp::Serializer serializer(packet);
00907     serializer.serializeHeader(header);
00908
00909     sendToServer(serializer.getData(), true);
00910
00911     m_currentLobbyId = 0;
00912 }
00913
00914 void AsioClient::requestStartGame(std::uint32_t lobbyId)
00915 {
00916     if (m_connectionState.load() != ConnectionState::CONNECTED)
00917     {
00918         utl::Logger::log("AsioClient: Cannot request start game - not connected", utl::LogLevel::WARNING);
00919         return;
00920     }
00921
00922     utl::Logger::log("AsioClient: Requesting to start game for lobby " + std::to_string(lobbyId),
00923                     utl::LogLevel::INFO);
00924
00925     rnp::PacketStartGameRequest request;
00926     request.lobbyId = lobbyId;
00927
00928     rnp::PacketHeader header;
00929     header.type = static_cast<std::uint8_t>(rnp::PacketType::START_GAME_REQUEST);
00930     header.length = static_cast<std::uint16_t>(sizeof(rnp::PacketStartGameRequest));
00931     header.sessionId = m_sessionId;
00932
00933     std::vector<std::uint8_t> packet;
00934     rnp::Serializer serializer(packet);
00935     serializer.serializeHeader(header);
00936     serializer.serializeStartGameRequest(request);
00937
00938     sendToServer(serializer.getData(), true);
00939     utl::Logger::log("AsioClient: START_GAME_REQUEST packet sent", utl::LogLevel::INFO);
00940 }
00941
00942 void AsioClient::setOnLobbyListReceived(std::function<void(const std::vector<rnp::LobbyInfo> &)> callback)
00943 {
00944     m_onLobbyListReceived = std::move(callback);

```

```

00945     }
00946
00947 void AsioClient::setOnLobbyCreated(std::function<void(std::uint32_t, bool, rnp::ErrorCode)> callback)
00948 {
00949     m_onLobbyCreated = std::move(callback);
00950 }
00951
00952 void AsioClient::setOnLobbyJoined(
00953     std::function<void(std::uint32_t, bool, rnp::ErrorCode, const rnp::LobbyInfo *)> callback)
00954 {
00955     m_onLobbyJoined = std::move(callback);
00956 }
00957
00958 void AsioClient::setOnLobbyUpdated(std::function<void(const rnp::LobbyInfo &)> callback)
00959 {
00960     m_onLobbyUpdated = std::move(callback);
00961 }
00962
00963 void AsioClient::setOnGameStart(std::function<void(std::uint32_t, std::uint32_t)> callback)
00964 {
00965     m_onGameStart = std::move(callback);
00966 }
00967
00968 rnp::HandlerResult AsioClient::handleLobbyListResponse(const rnp::PacketLobbyListResponse &packet,
00969     const rnp::PacketContext &context) const
00970 {
00971     utl::Logger::log("AsioClient: Received lobby list with " + std::to_string(packet.lobbyCount) + " lobbies",
00972         utl::LogLevel::INFO);
00973
00974     if (m_onLobbyListReceived)
00975     {
00976         m_onLobbyListReceived(packet.lobbies);
00977     }
00978     m_eventBus.publish(utl::EventType::LOBBY_LIST_RESPONSE, packet, m_componentId,
00979         7); // JoinRoomScene ID
00980     return rnp::HandlerResult::SUCCESS;
00981 }
00982
00983 rnp::HandlerResult AsioClient::handleLobbyCreateResponse(const rnp::PacketLobbyCreateResponse &packet,
00984     const rnp::PacketContext &context)
00985 {
00986     utl::Logger::log(
00987         "AsioClient: Received lobby create response - success: " + std::string(packet.success ? "true" : "false") +
00988         ", lobbyId: " + std::to_string(packet.lobbyId),
00989         utl::LogLevel::INFO);
00990
00991     if (packet.success)
00992     {
00993         m_currentLobbyId = packet.lobbyId;
00994     }
00995
00996     if (m_onLobbyCreated)
00997     {
00998         m_onLobbyCreated(packet.lobbyId, packet.success != 0, static_cast<rnp::ErrorCode>(packet.errorCode));
00999     }
01000     m_eventBus.publish(utl::EventType::LOBBY_CREATE_RESPONSE, packet, m_componentId,
01001         6); // CreateRoomScene ID
01002     return rnp::HandlerResult::SUCCESS;
01003 }
01004
01005 rnp::HandlerResult AsioClient::handleLobbyJoinResponse(const rnp::PacketLobbyJoinResponse &packet,
01006     const rnp::PacketContext &context)
01007 {
01008     utl::Logger::log(
01009         "AsioClient: Received lobby join response - success: " + std::string(packet.success ? "true" : "false") +
01010         ", lobbyId: " + std::to_string(packet.lobbyId),
01011         utl::LogLevel::INFO);
01012
01013     if (packet.success)
01014     {
01015         m_currentLobbyId = packet.lobbyId;
01016     }
01017
01018     if (m_onLobbyJoined)
01019     {
01020         const rnp::LobbyInfo *lobbyInfo = packet.success ? &packet.lobbyInfo : nullptr;
01021         m_onLobbyJoined(packet.lobbyId, packet.success != 0, static_cast<rnp::ErrorCode>(packet.errorCode),
01022             lobbyInfo);
01023     }
01024     m_eventBus.publish(utl::EventType::LOBBY_JOIN_RESPONSE, packet, m_componentId, 7); // JoinRoomScene
ID
01025
01026     return rnp::HandlerResult::SUCCESS;
01027 }
01028
01029 rnp::HandlerResult AsioClient::handleLobbyUpdate(const rnp::PacketLobbyUpdate &packet,
01030     const rnp::PacketContext &context) const

```

```

01031 {
01032     utl::Logger::log("AsioClient: Received lobby update for lobby " + std::to_string(packet.lobbyInfo.lobbyId),
01033                     utl::LogLevel::INFO);
01034
01035     if (m_onLobbyUpdated)
01036     {
01037         m_onLobbyUpdated(packet.lobbyInfo);
01038     }
01039
01040     // Publish to event bus so scenes can receive the update
01041     m_eventBus.publish(utl::EventType::LOBBY_UPDATE, packet, m_componentId, 8); // WaitingRoomScene ID
01042
01043     return rnp::HandlerResult::SUCCESS;
01044 }
01045
01046 rnp::HandlerResult AsioClient::handleGameStart(const rnp::PacketGameStart &packet,
01047                                                const rnp::PacketContext &context) const
01048 {
01049     utl::Logger::log("AsioClient: Game starting for lobby " + std::to_string(packet.lobbyId), utl::LogLevel::INFO);
01050
01051     if (m_onGameStart)
01052     {
01053         m_onGameStart(packet.lobbyId, context.sessionId);
01054     }
01055
01056     // Publish to event bus so WaitingRoomScene can receive the game start event
01057     utl::Event gameStartEvent(utl::EventType::GAME_START, m_componentId, 8); // target = WaitingRoomScene ID
01058     gameStartEvent.data =
01059         std::vector<std::uint8_t>(reinterpret_cast<const std::uint8_t*>(&packet),
01060                                 reinterpret_cast<const std::uint8_t*>(&packet) + sizeof(rnp::PacketGameStart));
01061     m_eventBus.publish(gameStartEvent);
01062
01063     return rnp::HandlerResult::SUCCESS;
01064 }
01065
01066 } // namespace eng

```

19.273 /home/masina/Projects/Epitech/rtype/plugins/Network/Asio/Server/include/AsioServer/AsioServer.hpp File Reference

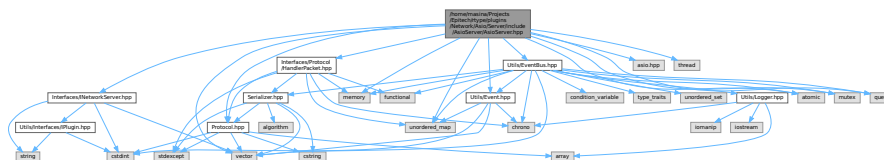
Asio-based implementation of INetworkServer interface.

```

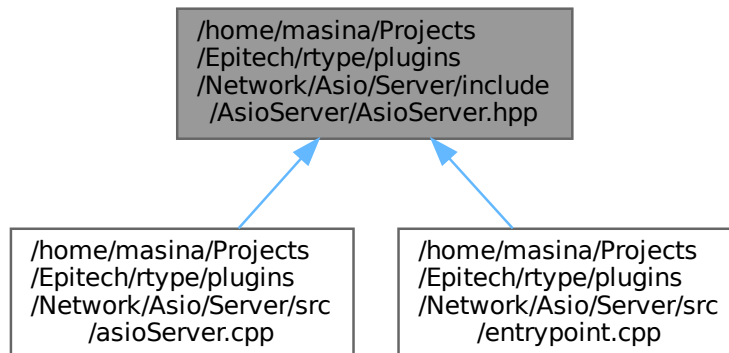
#include "Interfaces/INetworkServer.hpp"
#include "Interfaces/Protocol/HandlerPacket.hpp"
#include "Interfaces/Protocol/Protocol.hpp"
#include "Utils/Event.hpp"
#include "Utils/EventBus.hpp"
#include <asio.hpp>
#include <atomic>
#include <memory>
#include <mutex>
#include <queue>
#include <thread>
#include <unordered_map>
#include <vector>

```

Include dependency graph for AsioServer.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- struct [srv::ClientSession](#)
Client session information.
- struct [srv::Lobby](#)
Lobby structure.
- struct [srv::QueuedPacket](#)
Queued packet for sending.
- class [srv::AsioServer](#)
Asio UDP server implementation.

Namespaces

- namespace [srv](#)

Enumerations

- enum class [srv::LobbyStatus](#) : std::uint8_t {
[srv::SUCCESS](#) = 0 , [srv::WAITING](#) = 1 , [srv::IN_GAME](#) = 2 , [srv::NOT_FOUND](#) = 3 ,
[srv::FULL](#) = 4 , [srv::ALREADY_IN_LOBBY](#) = 5 }

19.273.1 Detailed Description

Asio-based implementation of INetworkServer interface.
 Definition in file [AsioServer.hpp](#).

19.274 AsioServer.hpp

[Go to the documentation of this file.](#)

```

00001 ///  

00002 ///  

00003 ///  

00004 ///  

00005 ///  

00006 ///  

00007 #pragma once  

00008  

00009 #include "Interfaces/INetworkServer.hpp"  

00010 #include "Interfaces/Protocol/HandlerPacket.hpp"  

00011 #include "Interfaces/Protocol/Protocol.hpp"
  
```

```

00012 #include "Utils/Event.hpp"
00013 #include "Utils/EventBus.hpp"
00014
00015 #include <asio.hpp>
00016 #include <atomic>
00017 #include <memory>
00018 #include <mutex>
00019 #include <queue>
00020 #include <thread>
00021 #include <unordered_map>
00022 #include <vector>
00023
00024 namespace srv
00025 {
00026
00027     ///
00028     /// @brief Client session information
00029     ///
00030     struct ClientSession
00031     {
00032         std::uint32_t sessionId;
00033         asio::ip::udp::endpoint endpoint;
00034         std::chrono::steady_clock::time_point lastSeen;
00035         std::string playerName;
00036         std::uint32_t clientCaps;
00037         bool isConnected;
00038         std::uint32_t lastPingSent;
00039         std::uint32_t latency;
00040         std::uint32_t currentLobbyId; // 0 if not in lobby
00041
00042         ClientSession()
00043             : sessionId(0), lastSeen(std::chrono::steady_clock::now()), clientCaps(0), isConnected(false),
00044               lastPingSent(0), latency(0), currentLobbyId(0)
00045         {
00046         }
00047     };
00048
00049     enum class LobbyStatus : std::uint8_t
00050     {
00051         SUCCESS = 0,
00052         WAITING = 1,
00053         IN_GAME = 2,
00054         NOT_FOUND = 3,
00055         FULL = 4,
00056         ALREADY_IN_LOBBY = 5
00057     };
00058
00059     ///
00060     /// @brief Lobby structure
00061     ///
00062     struct Lobby
00063     {
00064         std::uint32_t lobbyId;
00065         std::string lobbyName;
00066         std::uint32_t hostSessionId;
00067         std::vector<std::uint32_t> playerSessions;
00068         std::uint8_t maxPlayers;
00069         std::uint8_t gameMode;
00070         rnp::LobbyStatus status;
00071         std::chrono::steady_clock::time_point createdTime;
00072
00073         Lobby()
00074             : lobbyId(0), lobbyName(""), hostSessionId(0), maxPlayers(0), gameMode(0),
00075               status(rnp::LobbyStatus::WAITING), createdTime(std::chrono::steady_clock::now())
00076         {
00077         }
00078         Lobby(std::uint32_t id, const std::string &name, std::uint32_t host, std::uint8_t max, std::uint8_t mode)
00079             : lobbyId(id), lobbyName(name), hostSessionId(host), maxPlayers(max), gameMode(mode),
00080               status(rnp::LobbyStatus::WAITING), createdTime(std::chrono::steady_clock::now())
00081         {
00082         }
00083     };
00084
00085     ///
00086     /// @brief Queued packet for sending
00087     ///
00088     struct QueuedPacket
00089     {
00090         std::vector<std::uint8_t> data;
00091         asio::ip::udp::endpoint destination;
00092         bool reliable;
00093
00094         QueuedPacket(const std::vector<std::uint8_t> &d, const asio::ip::udp::endpoint &dest, bool rel = false)
00095             : data(d), destination(dest), reliable(rel)
00096         {
00097         }
00098     };

```

```

00099
00100 ///
00101 /// @class AsioServer
00102 /// @brief Asio UDP server implementation
00103 /// @namespace srv
00104 ///
00105 class AsioServer final : public INetworkServer
00106 {
00107
00108     public:
00109         ///
00110         /// @brief Constructor
00111         ///
00112         AsioServer();
00113
00114         ///
00115         /// @brief Destructor
00116         ///
00117         ~AsioServer() override;
00118
00119         [[nodiscard]] const std::string getName() const override { return "Network_Asio_Server"; }
00120         [[nodiscard]] utl::PluginType getType() const override { return utl::PluginType::NETWORK_CLIENT; }
00121
00122         // INetworkServer implementation
00123         void init(const std::string &host, std::uint16_t port) override;
00124         void start() override;
00125         void stop() override;
00126         void update() override;
00127         void sendToClient(std::uint32_t sessionId, const std::vector<std::uint8_t> &data,
00128             bool reliable = false) override;
00129         void sendToAllClients(const std::vector<std::uint8_t> &data, bool reliable = false) override;
00130         void disconnectClient(std::uint32_t sessionId) override;
00131
00132         [[nodiscard]] std::size_t getClientCount() const override;
00133         [[nodiscard]] std::vector<std::uint32_t> getConnectedSessions() const override;
00134         [[nodiscard]] bool isRunning() const override;
00135
00136         void setTickRate(std::uint16_t tickRate) override;
00137         void setServerCapabilities(std::uint32_t caps) override;
00138
00139     private:
00140         // Network components
00141         std::unique_ptr<asio::io_context> m_ioContext;
00142         std::unique_ptr<asio::ip::udp::socket> m_socket;
00143         std::unique_ptr<std::thread> m_networkThread;
00144
00145         // Server configuration
00146         std::string m_host;
00147         std::uint16_t m_port;
00148         std::uint16_t m_tickRate;
00149         std::uint32_t m_serverCaps;
00150
00151         // Server state
00152         std::atomic<bool> m_running;
00153         std::atomic<bool> m_started;
00154
00155         // Client management
00156         std::unordered_map<std::uint32_t, ClientSession> m_clients;
00157         std::unordered_map<std::string, std::uint32_t> m_endpointToSession;
00158         std::uint32_t m_nextSessionId;
00159         mutable std::mutex m_clientsMutex;
00160
00161         // Lobby management
00162         std::unordered_map<std::uint32_t, Lobby> m_lobbies;
00163         std::uint32_t m_nextLobbyId;
00164         mutable std::mutex m_lobbiesMutex;
00165
00166         // Packet handling
00167         std::unique_ptr<rnpp::HandlerPacket> m_packetHandler;
00168         std::queue<QueuedPacket> m_sendQueue;
00169         std::mutex m_sendQueueMutex;
00170         std::mutex m_socketMutex;
00171
00172         // Reception buffer
00173         std::array<std::uint8_t, MAX_LEN_RECV_BUFFER> m_recvBuffer;
00174         asio::ip::udp::endpoint m_senderEndpoint;
00175
00176         // Timing
00177         std::chrono::steady_clock::time_point m_lastPingTime;
00178         std::chrono::milliseconds m_pingInterval;
00179         std::chrono::milliseconds m_clientTimeout;
00180
00181         // EventBus
00182         std::uint32_t m_componentId;
00183         utl::EventBus &m_eventBus;
00184
00185         ///

```

```

00186     /// @brief Initialize packet handlers
00187     ///
00188     void setupPacketHandlers();
00189
00190     ///
00191     /// @brief Start receiving packets asynchronously
00192     ///
00193     void startReceive();
00194
00195     ///
00196     /// @brief Handle received packet
00197     /// @param bytesReceived Number of bytes received
00198     ///
00199     void handleReceive(std::size_t bytesReceived);
00200
00201     ///
00202     /// @brief Network thread main loop
00203     ///
00204     void networkThreadLoop() const;
00205
00206     ///
00207     /// @brief Generate unique session ID
00208     /// @return New session ID
00209     ///
00210     std::uint32_t generateSessionId() const;
00211
00212     ///
00213     /// @brief Get endpoint string representation
00214     /// @param endpoint UDP endpoint
00215     /// @return String representation
00216     ///
00217     static std::string endpointToString(const asio::ip::udp::endpoint &endpoint);
00218
00219     ///
00220     /// @brief Send packet immediately
00221     /// @param data Packet data
00222     /// @param destination Target endpoint
00223     ///
00224     void sendPacketImmediate(const std::vector<std::uint8_t> &data, const asio::ip::udp::endpoint &destination);
00225
00226     ///
00227     /// @brief Handle CONNECT packet
00228     /// @param packet Connect packet
00229     /// @param context Packet context
00230     /// @return Handler result
00231     ///
00232     rnp::HandlerResult handleConnect(const rnp::PacketConnect &packet, const rnp::PacketContext &context);
00233
00234     ///
00235     /// @brief Handle DISCONNECT packet
00236     /// @param packet Disconnect packet
00237     /// @param context Packet context
00238     /// @return Handler result
00239     ///
00240     rnp::HandlerResult handleDisconnect(const rnp::PacketDisconnect &packet, const rnp::PacketContext &context);
00241
00242     ///
00243     /// @brief Handle PING packet
00244     /// @param packet Ping packet
00245     /// @param context Packet context
00246     /// @return Handler result
00247     ///
00248     rnp::HandlerResult handlePing(const rnp::PacketPingPong &packet, const rnp::PacketContext &context);
00249
00250     ///
00251     /// @brief Handle PONG packet
00252     /// @param packet Pong packet
00253     /// @param context Packet context
00254     /// @return Handler result
00255     ///
00256     rnp::HandlerResult handlePong(const rnp::PacketPingPong &packet, const rnp::PacketContext &context);
00257
00258     ///
00259     /// @brief Send PONG response
00260     /// @param nonce Ping nonce
00261     /// @param destination Target endpoint
00262     /// @param sessionId Client session ID
00263     ///
00264     void sendPong(std::uint32_t nonce, const asio::ip::udp::endpoint &destination, std::uint32_t sessionId);
00265
00266     ///
00267     /// @brief Send CONNECT_ACCEPT response
00268     /// @param sessionId New session ID
00269     /// @param destination Target endpoint
00270     ///
00271     void sendConnectAccept(std::uint32_t sessionId, const asio::ip::udp::endpoint &destination);
00272

```



```

00273     ///
00274     /// @brief Send ERROR packet
00275     /// @param errorCode Error code
00276     /// @param description Error description
00277     /// @param destination Target endpoint
00278     /// @param sessionId Session ID
00279     ///
00280     void sendError(rnp::ErrorCode errorCode, const std::string &description,
00281                  const asio::ip::udp::endpoint &destination, std::uint32_t sessionId);
00282
00283     ///
00284     /// @brief Update client timeouts and send pings
00285     ///
00286     void updateClientManagement();
00287
00288     ///
00289     /// @brief Process send queue
00290     ///
00291     void processSendQueue();
00292
00293     ///
00294     /// @brief Process EventBus events for network operations
00295     ///
00296     void processEventBusEvents();
00297
00298     ///
00299     /// @brief Handle send to client event from EventBus
00300     /// @param event Send to client event
00301     ///
00302     void handleSendToClientEvent(const utl::Event &event);
00303
00304     ///
00305     /// @brief Handle broadcast event from EventBus
00306     /// @param event Broadcast event
00307     ///
00308     void handleBroadcastEvent(const utl::Event &event);
00309
00310     ///
00311     /// @brief Handle send entity event to clients from EventBus
00312     /// @param event Entity event to send to clients
00313     ///
00314     void handleSendEntityEventToClients(const utl::Event &event);
00315
00316     ///
00317     /// @brief Handle entity event packet (including player inputs)
00318     /// @param events Entity event records
00319     /// @param context Packet context
00320     /// @return Handler result
00321     ///
00322     rnp::HandlerResult handleEntityEvent(const std::vector<rnp::EventRecord> &events,
00323                                         const rnp::PacketContext &context) const;
00324
00325     ///
00326     /// @brief Handle LOBBY_LIST_REQUEST packet
00327     /// @param context Packet context
00328     /// @return Handler result
00329     ///
00330     rnp::HandlerResult handleLobbyListRequest(const rnp::PacketContext &context);
00331
00332     ///
00333     /// @brief Handle LOBBY_CREATE packet
00334     /// @param packet Lobby create packet
00335     /// @param context Packet context
00336     /// @return Handler result
00337     ///
00338     rnp::HandlerResult handleLobbyCreate(const rnp::PacketLobbyCreate &packet,
00339                                         const rnp::PacketContext &context);
00340
00341     ///
00342     /// @brief Handle LOBBY_JOIN packet
00343     /// @param packet Lobby join packet
00344     /// @param context Packet context
00345     /// @return Handler result
00346     ///
00347     rnp::HandlerResult handleLobbyJoin(const rnp::PacketLobbyJoin &packet, const rnp::PacketContext &context);
00348
00349     ///
00350     /// @brief Handle LOBBY_LEAVE packet
00351     /// @param context Packet context
00352     /// @return Handler result
00353     ///
00354     rnp::HandlerResult handleLobbyLeave(const rnp::PacketContext &context);
00355
00356     ///
00357     /// @brief Send lobby list to client
00358     /// @param sessionId Target client session ID
00359     ///

```

```

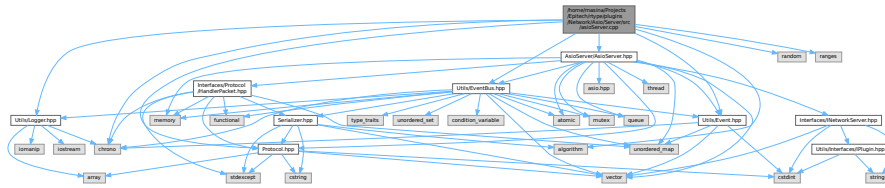
00360 void sendLobbyList(std::uint32_t sessionId);
00361
00362 ///
00363 /// @brief Create new lobby
00364 /// @param name Lobby name
00365 /// @param hostSession Host session ID
00366 /// @param maxPlayers Maximum players
00367 /// @param gameMode Game mode
00368 /// @return New lobby ID
00369 ///
00370 std::uint32_t createLobby(const std::string &name, std::uint32_t hostSession, std::uint8_t maxPlayers,
00371                          std::uint8_t gameMode);
00372
00373 ///
00374 /// @brief Join player to lobby
00375 /// @param lobbyId Target lobby ID
00376 /// @param sessionId Player session ID
00377 /// @return Success status
00378 ///
00379 LobbyStatus joinLobby(std::uint32_t lobbyId, std::uint32_t sessionId);
00380
00381 ///
00382 /// @brief Remove player from lobby
00383 /// @param sessionId Player session ID
00384 ///
00385 void leaveLobby(std::uint32_t sessionId);
00386
00387 ///
00388 /// @brief Broadcast lobby update to all lobby members
00389 /// @param lobbyId Target lobby ID
00390 ///
00391 void broadcastLobbyUpdate(std::uint32_t lobbyId);
00392
00393 ///
00394 /// @brief Handle START_GAME_REQUEST packet
00395 /// @param packet Start game request packet
00396 /// @param context Packet context
00397 /// @return Handler result
00398 ///
00399 rnp::HandlerResult handleStartGameRequest(const rnp::PacketStartGameRequest &packet,
00400                                           const rnp::PacketContext &context);
00401
00402 ///
00403 /// @brief Broadcast game start to all players in lobby
00404 /// @param lobbyId Target lobby ID
00405 ///
00406 void broadcastGameStart(std::uint32_t lobbyId);
00407
00408 ///
00409 /// @brief Convert Lobby to LobbyInfo for network transmission
00410 /// @param lobby Lobby structure
00411 /// @return LobbyInfo structure
00412 ///
00413 rnp::LobbyInfo lobbyToLobbyInfo(const Lobby &lobby);
00414
00415 ///
00416 /// @brief Send lobby create response
00417 /// @param sessionId Target client session ID
00418 /// @param lobbyId Created lobby ID (0 if failed)
00419 /// @param success Success status
00420 /// @param errorCode Error code if failed
00421 ///
00422 void sendLobbyCreateResponse(std::uint32_t sessionId, std::uint32_t lobbyId, bool success,
00423                             rnp::ErrorCode errorCode = rnp::ErrorCode::INTERNAL_ERROR);
00424
00425 ///
00426 /// @brief Send lobby join response
00427 /// @param sessionId Target client session ID
00428 /// @param lobbyId Target lobby ID
00429 /// @param success Success status
00430 /// @param errorCode Error code if failed
00431 /// @param lobbyInfo Lobby information if successful
00432 ///
00433 void sendLobbyJoinResponse(std::uint32_t sessionId, std::uint32_t lobbyId, bool success,
00434                             rnp::ErrorCode errorCode, const rnp::LobbyInfo *lobbyInfo = nullptr);
00435
00436 ///
00437 /// @brief Clean up empty lobbies
00438 ///
00439 void cleanupEmptyLobbies();
00440 };
00441
00442 } // namespace srv

```

19.275 /home/masina/Projects/Epitech/rtype/plugins/Network/Asio/Server/src/asioServer.cpp File Reference

```
#include "AsioServer/AsioServer.hpp"
#include "Interfaces/Protocol/Protocol.hpp"
#include "Utils/Event.hpp"
#include "Utils/EventBus.hpp"
#include "Utils/Logger.hpp"
#include <algorithm>
#include <chrono>
#include <random>
#include <ranges>
```

Include dependency graph for asioServer.cpp:



Namespaces

- namespace `srv`

19.276 asioServer.cpp

[Go to the documentation of this file.](#)

```
00001 ///
00002 /// @file AsioServer.cpp
00003 /// @brief Asio-based implementation of INetworkServer interface
00004 /// @namespace srv
00005 ///
00006
00007 #include "AsioServer/AsioServer.hpp"
00008 #include "Interfaces/Protocol/Protocol.hpp"
00009 #include "Utils/Event.hpp"
00010 #include "Utils/EventBus.hpp"
00011 #include "Utils/Logger.hpp"
00012
00013 #include <algorithm>
00014 #include <chrono>
00015 #include <random>
00016 #include <ranges>
00017
00018 namespace srv
00019 {
00020
00021     AsioServer::AsioServer()
00022     : m_ioContext(std::make_unique<asio::io_context>()),
00023       m_socket(std::make_unique<asio::ip::udp::socket>(*m_ioContext)), m_port(0), m_tickRate(60),
00024       m_serverCaps(0),
00025       m_running(false), m_started(false), m_nextSessionId(1), m_nextLobbyId(1),
00026       m_packetHandler(std::make_unique<HandlerPacket>()), m_pingInterval(std::chrono::seconds(5)),
00027       m_clientTimeout(std::chrono::seconds(30)), m_componentId(utl::NETWORK_SERVER),
00028       m_eventBus(utl::EventBus::getInstance())
00029     {
00030         // utl::Logger::log("AsioServer: Constructor called", utl::LogLevel::INFO);
00031         // utl::Logger::log("AsioServer: Creating I/O context and socket", utl::LogLevel::INFO);
00032         setupPacketHandlers();
00033         // utl::Logger::log("AsioServer: Packet handlers setup complete", utl::LogLevel::INFO);
00034
00035         // EventBus integration
00036         m_eventBus.registerComponent(m_componentId, "Asio Server");
00037         m_eventBus.subscribe(m_componentId, utl::EventType::SEND_TO_CLIENT);
00038         m_eventBus.subscribe(m_componentId, utl::EventType::BROADCAST_WORLD_STATE);
00039         m_eventBus.subscribe(m_componentId, utl::EventType::SEND_ENTITY_EVENTS);
00040         // utl::Logger::log("AsioServer: Initialized with EventBus integration", utl::LogLevel::INFO);
00041     }
```

```

00042
00043 AsioServer::~AsioServer()
00044 {
00045     // utl::Logger::log("AsioServer: Destructor called", utl::LogLevel::INFO);
00046     AsioServer::stop();
00047     // utl::Logger::log("AsioServer: Destroyed", utl::LogLevel::INFO);
00048 }
00049
00050 void AsioServer::init(const std::string &host, std::uint16_t port)
00051 {
00052     // utl::Logger::log("AsioServer: init() called with host=" + host + ", port=" + std::to_string(port),
00053     //                 utl::LogLevel::INFO);
00054     m_host = host;
00055     m_port = port;
00056     // utl::Logger::log("AsioServer: Configuration stored - ready to start", utl::LogLevel::INFO);
00057 }
00058
00059 void AsioServer::start()
00060 {
00061     // utl::Logger::log("AsioServer: Starting server...", utl::LogLevel::INFO);
00062
00063     if (m_started.load())
00064     {
00065         // utl::Logger::log("AsioServer: Already started", utl::LogLevel::WARNING);
00066         return;
00067     }
00068
00069     try
00070     {
00071         // utl::Logger::log("AsioServer: Creating endpoint for " + m_host + ":" + std::to_string(m_port),
00072         //                 utl::LogLevel::INFO);
00073
00074         // Create endpoint
00075         asio::ip::udp::endpoint endpoint(asio::ip::make_address(m_host), m_port);
00076         // utl::Logger::log("AsioServer: Endpoint created successfully", utl::LogLevel::INFO);
00077
00078         // Bind socket
00079         // utl::Logger::log("AsioServer: Opening socket...", utl::LogLevel::INFO);
00080         m_socket->open(endpoint.protocol());
00081         // utl::Logger::log("AsioServer: Setting socket options...", utl::LogLevel::INFO);
00082         m_socket->set_option(asio::ip::udp::socket::reuse_address(true));
00083         // utl::Logger::log("AsioServer: Binding socket to endpoint...", utl::LogLevel::INFO);
00084         m_socket->bind(endpoint);
00085         // utl::Logger::log("AsioServer: Socket bound successfully", utl::LogLevel::INFO);
00086
00087         m_running.store(true);
00088         m_started.store(true);
00089
00090         // Start network thread
00091         // utl::Logger::log("AsioServer: Starting network thread...", utl::LogLevel::INFO);
00092         m_networkThread = std::make_unique<std::thread>(&AsioServer::networkThreadLoop, this);
00093
00094         // Start receiving
00095         // utl::Logger::log("AsioServer: Starting packet reception...", utl::LogLevel::INFO);
00096         startReceive();
00097
00098         // utl::Logger::log("AsioServer: Started on " + m_host + ":" + std::to_string(m_port), utl::LogLevel::INFO);
00099     }
00100     catch (const std::exception &e)
00101     {
00102         m_running.store(false);
00103         m_started.store(false);
00104         // utl::Logger::log("AsioServer: Failed to start - " + std::string(e.what()), utl::LogLevel::WARNING);
00105         throw;
00106     }
00107 }
00108
00109 void AsioServer::stop()
00110 {
00111     if (!m_started.load())
00112     {
00113         return;
00114     }
00115
00116     m_running.store(false);
00117     m_started.store(false);
00118
00119     // Send disconnect to all clients
00120     {
00121         std::lock_guard<std::mutex> lock(m_clientsMutex);
00122         for (auto &[sessionId, client] : m_clients)
00123         {
00124             if (client.isConnected)
00125             {
00126                 rnp::Serializer serializer;
00127                 rnp::PacketHeader header{};
00128                 header.type = static_cast<std::uint8_t>(rnp::PacketType::DISCONNECT);

```

```

00129         header.length = sizeof(rnp::PacketDisconnect);
00130         header.sessionId = sessionId;
00131
00132         rnp::PacketDisconnect disconnect{};
00133         disconnect.reasonCode = static_cast<std::uint16_t>(rnp::DisconnectReason::SERVER_SHUTDOWN);
00134
00135         serializer.serializeHeader(header);
00136         serializer.serializeDisconnect(disconnect);
00137
00138         sendPacketImmediate(serializer.getData(), client.endpoint);
00139     }
00140 }
00141 }
00142
00143 // Stop io_context safely
00144 if (m_ioContext)
00145 {
00146     m_ioContext->stop();
00147 }
00148
00149 // Wait for network thread with timeout
00150 if (m_networkThread && m_networkThread->joinable())
00151 {
00152     auto joinStart = std::chrono::steady_clock::now();
00153     bool joined = false;
00154
00155     while (
00156         !joined &&
00157         std::chrono::duration_cast<std::chrono::seconds>(std::chrono::steady_clock::now() - joinStart).count() <
00158             5)
00159     {
00160         if (m_networkThread->joinable())
00161         {
00162             m_networkThread->join();
00163             joined = true;
00164         }
00165         else
00166         {
00167             std::this_thread::sleep_for(std::chrono::milliseconds(10));
00168         }
00169     }
00170
00171     if (!joined)
00172     {
00173         // utl::Logger::log("AsioServer: Network thread join timeout, detaching", utl::LogLevel::WARNING);
00174         m_networkThread->detach();
00175     }
00176 }
00177
00178 // Close socket safely
00179 {
00180     std::lock_guard<std::mutex> socketLock(m_socketMutex);
00181     if (m_socket && m_socket->is_open())
00182     {
00183         try
00184         {
00185             m_socket->close();
00186         }
00187         catch (const std::exception &e)
00188         {
00189             // utl::Logger::log("AsioServer: Exception closing socket - " + std::string(e.what()),
00190             // utl::LogLevel::WARNING);
00191         }
00192     }
00193 }
00194
00195 // Clear clients
00196 {
00197     std::lock_guard<std::mutex> lock(m_clientsMutex);
00198     m_clients.clear();
00199     m_endpointToSession.clear();
00200 }
00201
00202 // utl::Logger::log("AsioServer: Stopped", utl::LogLevel::INFO);
00203 }
00204
00205 void AsioServer::update()
00206 {
00207     processEventBusEvents();
00208
00209     if (!m_running.load())
00210     {
00211         return;
00212     }
00213
00214     // Process send queue
00215     processSendQueue();

```

```

00216
00217     // Update client management (timeouts, pings)
00218     updateClientManagement();
00219 }
00220
00221 void AsioServer::sendToClient(std::uint32_t sessionId, const std::vector<std::uint8_t> &data, bool reliable)
00222 {
00223     std::lock_guard<std::mutex> clientLock(m_clientsMutex);
00224     auto it = m_clients.find(sessionId);
00225     if (it == m_clients.end() || !it->second.isConnected)
00226     {
00227         // utl::Logger::log("AsioServer: Attempted to send to invalid session " + std::to_string(sessionId),
00228         //                 utl::LogLevel::WARNING);
00229         return;
00230     }
00231
00232     std::lock_guard<std::mutex> queueLock(m_sendQueueMutex);
00233     m_sendQueue.emplace(data, it->second.endpoint, reliable);
00234 }
00235
00236 void AsioServer::sendToAllClients(const std::vector<std::uint8_t> &data, bool reliable)
00237 {
00238     std::lock_guard<std::mutex> clientLock(m_clientsMutex);
00239     std::lock_guard<std::mutex> queueLock(m_sendQueueMutex);
00240
00241     for (const auto &client : m_clients | std::views::values)
00242     {
00243         if (client.isConnected)
00244         {
00245             m_sendQueue.emplace(data, client.endpoint, reliable);
00246         }
00247     }
00248 }
00249
00250 void AsioServer::disconnectClient(std::uint32_t sessionId)
00251 {
00252     std::uint32_t lobbyId = 0;
00253
00254     // Scope for clientsLock to release it before broadcastLobbyUpdate
00255     {
00256         std::lock_guard<std::mutex> lock(m_clientsMutex);
00257         auto it = m_clients.find(sessionId);
00258         if (it == m_clients.end())
00259         {
00260             return;
00261         }
00262
00263         utl::Logger::log("AsioServer: Disconnecting client " + std::to_string(sessionId), utl::LogLevel::INFO);
00264
00265         rnp::PacketHeader header;
00266         header.type = static_cast<std::uint8_t>(rnp::PacketType::DISCONNECT);
00267         header.length = static_cast<std::uint16_t>(sizeof(rnp::PacketDisconnect));
00268         header.sessionId = sessionId;
00269
00270         std::vector<std::uint8_t> packet;
00271         rnp::Serializer serializer(packet);
00272         rnp::PacketDisconnect disconnect{};
00273         disconnect.reasonCode = static_cast<std::uint16_t>(rnp::DisconnectReason::CLIENT_REQUEST);
00274
00275         serializer.serializeHeader(header);
00276         serializer.serializeDisconnect(disconnect);
00277
00278         sendPacketImmediate(serializer.getData(), it->second.endpoint);
00279
00280         // Remove from lobby if in one
00281         if (it->second.currentLobbyId != 0)
00282         {
00283             lobbyId = it->second.currentLobbyId;
00284         }
00285
00286         // Remove from endpoint mapping
00287         const std::string endpointStr = endpointToString(it->second.endpoint);
00288         m_endpointToSession.erase(endpointStr);
00289
00290         // Remove client
00291         m_clients.erase(it);
00292
00293         m_packetHandler->clearSession(sessionId);
00294
00295         // utl::Logger::log("AsioServer: Disconnected client " + std::to_string(sessionId), utl::LogLevel::INFO);
00296     } // clientsLock released here
00297
00298     // Leave lobby AFTER releasing clientsLock to avoid nested mutex acquisitions
00299     if (lobbyId != 0)
00300     {
00301         leaveLobby(sessionId);
00302         // Broadcast lobby update to remaining players

```

```

00303         broadcastLobbyUpdate(lobbyId);
00304     }
00305 }
00306
00307 std::size_t AsioServer::getClientCount() const
00308 {
00309     std::lock_guard clientLock(m_clientsMutex);
00310     return std::ranges::count_if(m_clients, [](const auto &pair) { return pair.second.isConnected; });
00311 }
00312
00313 std::vector<std::uint32_t> AsioServer::getConnectedSessions() const
00314 {
00315     std::lock_guard clientLock(m_clientsMutex);
00316     std::vector<std::uint32_t> sessions;
00317     for (const auto &[sessionId, client] : m_clients)
00318     {
00319         if (client.isConnected)
00320         {
00321             sessions.push_back(sessionId);
00322         }
00323     }
00324     return sessions;
00325 }
00326
00327 bool AsioServer::isRunning() const { return m_running.load(); }
00328
00329 void AsioServer::setTickRate(std::uint16_t tickRate)
00330 {
00331     m_tickRate = tickRate;
00332     // utl::Logger::log("AsioServer: Tick rate set to " + std::to_string(tickRate), utl::LogLevel::INFO);
00333 }
00334
00335 void AsioServer::setServerCapabilities(std::uint32_t caps)
00336 {
00337     m_serverCaps = caps;
00338     // utl::Logger::log("AsioServer: Server capabilities set to " + std::to_string(caps), utl::LogLevel::INFO);
00339 }
00340
00341 void AsioServer::setupPacketHandlers()
00342 {
00343     // CONNECT handler
00344     m_packetHandler->onConnect([this](const rnp::PacketConnect &packet, const rnp::PacketContext &context)
00345     { return handleConnect(packet, context); });
00346
00347     // DISCONNECT handler
00348     m_packetHandler->onDisconnect([this](const rnp::PacketDisconnect &packet, const rnp::PacketContext &context)
00349     { return handleDisconnect(packet, context); });
00350
00351     // PING handler
00352     m_packetHandler->onPing([this](const rnp::PacketPingPong &packet, const rnp::PacketContext &context)
00353     { return handlePing(packet, context); });
00354
00355     // PONG handler
00356     m_packetHandler->onPong([this](const rnp::PacketPingPong &packet, const rnp::PacketContext &context)
00357     { return handlePong(packet, context); });
00358
00359     // ENTITY_EVENT handler (for player inputs)
00360     m_packetHandler->onEntityEvent(
00361     [this](const std::vector<rnp::EventRecord> &events, const rnp::PacketContext &context)
00362     { return handleEntityEvent(events, context); });
00363
00364     // LOBBY_LIST_REQUEST handler
00365     m_packetHandler->onLobbyListRequest([this](const rnp::PacketContext &context)
00366     { return handleLobbyListRequest(context); });
00367
00368     // LOBBY_CREATE handler
00369     m_packetHandler->onLobbyCreate([this](const rnp::PacketLobbyCreate &packet, const rnp::PacketContext
00370     &context)
00371     { return handleLobbyCreate(packet, context); });
00372
00373     // LOBBY_JOIN handler
00374     m_packetHandler->onLobbyJoin([this](const rnp::PacketLobbyJoin &packet, const rnp::PacketContext &context)
00375     { return handleLobbyJoin(packet, context); });
00376
00377     // LOBBY_LEAVE handler
00378     m_packetHandler->onLobbyLeave([this](const rnp::PacketContext &context) { return handleLobbyLeave(context);
00379 });
00380
00381     // START_GAME_REQUEST handler
00382     m_packetHandler->onStartGameRequest(
00383     [this](const rnp::PacketStartGameRequest &packet, const rnp::PacketContext &context)
00384     { return handleStartGameRequest(packet, context); });
00385
00386     // utl::Logger::log("AsioServer: Packet handlers initialized", utl::LogLevel::INFO);
00387 }
00388
00389 void AsioServer::startReceive()

```

```

00388 {
00389     // Check if we should continue receiving
00390     if (!m_running.load() || !m_socket || !m_socket->is_open())
00391     {
00392         utl::Logger::log("AsioServer: Cannot start receive - server not running or socket closed",
00393             utl::LogLevel::WARNING);
00394         return;
00395     }
00396
00397     m_socket->async_receive_from(
00398         asio::buffer(m_recvBuffer), m_senderEndpoint,
00399         [this](const std::error_code ec, const std::size_t bytesReceived)
00400         {
00401             if (!ec && m_running.load())
00402             {
00403                 utl::Logger::log("AsioServer: Received " + std::to_string(bytesReceived) + " bytes from " +
00404                     m_senderEndpoint.address().to_string() + ":" +
00405                     std::to_string(m_senderEndpoint.port()),
00406                     utl::LogLevel::INFO);
00407                 handleReceive(bytesReceived);
00408                 startReceive(); // Continue receiving
00409             }
00410             else if (m_running.load() && ec != asio::error::operation_aborted)
00411             {
00412                 utl::Logger::log("AsioServer: Receive error - " + ec.message(), utl::LogLevel::WARNING);
00413                 std::this_thread::sleep_for(std::chrono::milliseconds(100));
00414                 startReceive(); // Try to continue
00415             }
00416             else
00417             {
00418                 utl::Logger::log("AsioServer: Server stopped or operation aborted, ending receive loop",
00419                     utl::LogLevel::INFO);
00420             }
00421         });
00422     }
00423
00424 void AsioServer::handleReceive(std::size_t bytesReceived)
00425 {
00426     utl::Logger::log("AsioServer: Processing received packet of " + std::to_string(bytesReceived) + " bytes",
00427         utl::LogLevel::INFO);
00428
00429     // PacketHeader serialized size: 1 (type) + 2 (length) + 4 (sessionId) = 7 bytes
00430     constexpr std::size_t PACKET_HEADER_SIZE = 7;
00431     if (bytesReceived < PACKET_HEADER_SIZE)
00432     {
00433         utl::Logger::log("AsioServer: Received packet too small (need " + std::to_string(PACKET_HEADER_SIZE) +
00434             " bytes, got " + std::to_string(bytesReceived) + ")",
00435             utl::LogLevel::WARNING);
00436         return;
00437     }
00438
00439     // Create packet context
00440     rnp::PacketContext context;
00441     context.receiveTime = std::chrono::steady_clock::now();
00442     context.senderAddress = m_senderEndpoint.address().to_string();
00443     context.senderPort = m_senderEndpoint.port();
00444
00445     utl::Logger::log("AsioServer: Packet from " + context.senderAddress + ":" + std::to_string(context.senderPort),
00446         utl::LogLevel::INFO);
00447
00448     // Extract session ID from header for context
00449     std::vector<std::uint8_t> data(m_recvBuffer.begin(), m_recvBuffer.begin() + bytesReceived);
00450     if (data.size() >= PACKET_HEADER_SIZE)
00451     {
00452         rnp::Serializer headerSerializer(data);
00453         rnp::PacketHeader header = headerSerializer.deserializeHeader();
00454         context.sessionId = header.sessionId;
00455         utl::Logger::log("AsioServer: Packet type: " + std::to_string(static_cast<int>(header.type)) +
00456             ", Session ID: " + std::to_string(header.sessionId),
00457             utl::LogLevel::INFO);
00458     }
00459
00460     // Process packet
00461     utl::Logger::log("AsioServer: Processing packet...", utl::LogLevel::INFO);
00462     rnp::HandlerResult result = m_packetHandler->processPacket(data, context);
00463     if (result != rnp::HandlerResult::SUCCESS)
00464     {
00465         utl::Logger::log("AsioServer: Packet processing failed with result " +
00466             std::to_string(static_cast<int>(result)),
00467             utl::LogLevel::WARNING);
00468     }
00469     else
00470     {
00471         utl::Logger::log("AsioServer: Packet processed successfully", utl::LogLevel::INFO);
00472     }
00473 }
00474

```



```

00475 void AsioServer::networkThreadLoop() const
00476 {
00477     // utl::Logger::log("AsioServer: Network thread started", utl::LogLevel::INFO);
00478
00479     while (m_running.load())
00480     {
00481         try
00482         {
00483             // Run I/O context and check if we should continue
00484             if (!m_running.load())
00485             {
00486                 break;
00487             }
00488
00489             size_t handlersRun = m_ioContext->run_one();
00490             if (handlersRun == 0)
00491             {
00492                 // No handlers to run, sleep briefly to avoid busy waiting
00493                 std::this_thread::sleep_for(std::chrono::milliseconds(1));
00494             }
00495         }
00496         catch (const std::exception &e)
00497         {
00498             if (m_running.load())
00499             {
00500                 // utl::Logger::log("AsioServer: Network thread exception - " + std::string(e.what()),
00501                 // utl::LogLevel::WARNING);
00502                 std::this_thread::sleep_for(std::chrono::milliseconds(100));
00503             }
00504         }
00505     }
00506
00507     // utl::Logger::log("AsioServer: Network thread stopped", utl::LogLevel::INFO);
00508 }
00509
00510 std::uint32_t AsioServer::generateSessionId() const
00511 {
00512     std::random_device rd;
00513     std::mt19937 gen(rd());
00514     std::uniform_int_distribution<std::uint32_t> dis(1, UINT32_MAX);
00515
00516     std::uint32_t sessionId = 0;
00517     while (sessionId == 0 || m_clients.contains(sessionId))
00518     {
00519         sessionId = dis(gen);
00520     }
00521
00522     return sessionId;
00523 }
00524
00525 std::string AsioServer::endpointToString(const asio::ip::udp::endpoint &endpoint)
00526 {
00527     return endpoint.address().to_string() + ":" + std::to_string(endpoint.port());
00528 }
00529
00530 void AsioServer::sendPacketImmediate(const std::vector<std::uint8_t> &data,
00531                                     const asio::ip::udp::endpoint &destination)
00532 {
00533     utl::Logger::log("AsioServer: Sending " + std::to_string(data.size()) + " bytes to " +
00534                     destination.address().to_string() + ":" + std::to_string(destination.port()),
00535                     utl::LogLevel::INFO);
00536
00537     // Protect socket access with mutex
00538     std::lock_guard<std::mutex> socketLock(m_socketMutex);
00539
00540     if (!m_socket || !m_socket->is_open() || !m_running.load())
00541     {
00542         utl::Logger::log("AsioServer: Cannot send - socket not open or server not running", utl::LogLevel::WARNING);
00543         return;
00544     }
00545
00546     try
00547     {
00548         size_t bytesSent = m_socket->send_to(asio::buffer(data), destination);
00549         utl::Logger::log("AsioServer: Successfully sent " + std::to_string(bytesSent) + " bytes",
00550                         utl::LogLevel::INFO);
00551     }
00552     catch (const std::exception &e)
00553     {
00554         if (m_running.load())
00555         {
00556             utl::Logger::log("AsioServer: Failed to send packet - " + std::string(e.what()),
00557                             utl::LogLevel::WARNING);
00558         }
00559     }
00560 }
00561

```

```

00562 rnp::HandlerResult AsioServer::handleConnect(const rnp::PacketConnect &packet, const rnp::PacketContext &context)
00563 {
00564     // utl::Logger::log("AsioServer: Handling CONNECT packet from " + context.senderAddress + ":" +
00565     //                 std::to_string(context.senderPort),
00566     //                 utl::LogLevel::INFO);
00567
00568     std::lock_guard<std::mutex> lock(m_clientsMutex);
00569     // Check if server is full (avoid getClientCount() to prevent deadlock)
00570     const size_t currentClientCount =
00571         std::ranges::count_if(m_clients, [](const auto &pair) { return pair.second.isConnected; });
00572     // utl::Logger::log("AsioServer: Current client count: " + std::to_string(currentClientCount) + "/" +
00573     //                 std::to_string(MAX_CLIENTS),
00574     //                 utl::LogLevel::INFO);
00575
00576     if (currentClientCount >= MAX_CLIENTS)
00577     {
00578         // utl::Logger::log("AsioServer: Server full, rejecting connection", utl::LogLevel::WARNING);
00579         sendError(rnp::ErrorCode::RATE_LIMITED, "Server full", m_senderEndpoint, 0);
00580         return rnp::HandlerResult::SUCCESS;
00581     }
00582
00583     // Check if client already exists
00584     const std::string endpointStr = endpointToString(m_senderEndpoint);
00585     const auto existingIt = m_endpointToSession.find(endpointStr);
00586     if (existingIt != m_endpointToSession.end())
00587     {
00588         // utl::Logger::log("AsioServer: Client already connected, resending accept (Session ID: " +
00589         //                 std::to_string(existingIt->second) + ")",
00590         //                 utl::LogLevel::INFO);
00591         // Client already connected, resend accept
00592         sendConnectAccept(existingIt->second, m_senderEndpoint);
00593         return rnp::HandlerResult::SUCCESS;
00594     }
00595
00596     // Create new session
00597     std::uint32_t sessionId = generateSessionId();
00598     // utl::Logger::log("AsioServer: Creating new session with ID: " + std::to_string(sessionId),
00599     //                 utl::LogLevel::INFO);
00600
00601     ClientSession &client = m_clients[sessionId];
00602     client.sessionId = sessionId;
00603     client.endpoint = m_senderEndpoint;
00604     client.lastSeen = context.receiveTime;
00605     client.playerName = std::string(packet.playerName.data(), packet.nameLen);
00606     client.clientCaps = packet.clientCaps;
00607     client.isConnected = true;
00608
00609     m_endpointToSession[endpointStr] = sessionId;
00610
00611     // utl::Logger::log("AsioServer: Player name: " + client.playerName +
00612     //                 ", Caps: " + std::to_string(client.clientCaps),
00613     //                 utl::LogLevel::INFO);
00614
00615     // Send accept response
00616     // utl::Logger::log("AsioServer: Sending CONNECT_ACCEPT to session " + std::to_string(sessionId),
00617     //                 utl::LogLevel::INFO);
00618     sendConnectAccept(sessionId, m_senderEndpoint);
00619
00620     // utl::Logger::log("AsioServer: Client connected - " + client.playerName + " (ID: " + std::to_string(sessionId)
00621     //                 + " +
00622     //                 ")",
00623     //                 utl::LogLevel::INFO);
00624
00625     // Publier vers GameServer
00626     m_eventBus.publish(utl::EventType::PLAYER_CONNECTED, sessionId, m_componentId,
00627                       utl::GAME_LOGIC); // GameServer ID
00628
00629     // utl::Logger::log(
00630     //     "AsioServer: Nouvelle connexion publiée vers GameServer (sessionId: " + std::to_string(sessionId) + ")",
00631     //     utl::LogLevel::INFO);
00632
00633     return rnp::HandlerResult::SUCCESS;
00634 }
00635
00636 rnp::HandlerResult AsioServer::handleDisconnect(const rnp::PacketDisconnect &packet,
00637                                                const rnp::PacketContext &context)
00638 {
00639     // utl::Logger::log("AsioServer: Handling DISCONNECT packet from session " + std::to_string(context.sessionId) +
00640     //                 " from " + context.senderAddress + ":" + std::to_string(context.senderPort),
00641     //                 utl::LogLevel::INFO);
00642
00643     std::lock_guard<std::mutex> lock(m_clientsMutex);
00644
00645     auto it = m_clients.find(context.sessionId);
00646     if (it != m_clients.end())
00647     {
00648         // utl::Logger::log("AsioServer: Client disconnected - " + it->second.playerName +

```

```

00649         //          " (ID: " + std::to_string(context.sessionId) + ") " +
00650         //          ", Reason: " + std::to_string(static_cast<int>(packet.reasonCode)),
00651         //          utl::LogLevel::INFO);
00652
00653         // Remove from endpoint mapping
00654         const std::string endpointStr = endpointToString(it->second.endpoint);
00655         // utl::Logger::log("AsioServer: Removing endpoint mapping: " + endpointStr, utl::LogLevel::INFO);
00656         m_endpointToSession.erase(endpointStr);
00657
00658         // Remove client
00659         // utl::Logger::log("AsioServer: Removing client from session list", utl::LogLevel::INFO);
00660         m_clients.erase(it);
00661     }
00662     else
00663     {
00664         // utl::Logger::log("AsioServer: DISCONNECT from unknown session " + std::to_string(context.sessionId),
00665         // utl::LogLevel::WARNING);
00666     }
00667
00668     // Publier vers GameServer
00669     m_eventBus.publish(utl::EventType::PLAYER_DISCONNECTED, context.sessionId, m_componentId,
00670                       utl::GAME_LOGIC); // GameServer ID
00671
00672     // utl::Logger::log(
00673     // "AsioServer: Déconnexion publiée vers GameServer (sessionId: " + std::to_string(context.sessionId) + ")",
00674     // utl::LogLevel::INFO);
00675
00676     return rnp::HandlerResult::SUCCESS;
00677 }
00678
00679 rnp::HandlerResult AsioServer::handlePing(const rnp::PacketPingPong &packet, const rnp::PacketContext &context)
00680 {
00681     // Update client last seen time
00682     {
00683         std::lock_guard<std::mutex> lock(m_clientsMutex);
00684         const auto it = m_clients.find(context.sessionId);
00685         if (it != m_clients.end())
00686         {
00687             it->second.lastSeen = context.receiveTime;
00688         }
00689     }
00690
00691     // Send pong response
00692     sendPong(packet.nonce, m_senderEndpoint, context.sessionId);
00693     return rnp::HandlerResult::SUCCESS;
00694 }
00695
00696 rnp::HandlerResult AsioServer::handlePong(const rnp::PacketPingPong &packet, const rnp::PacketContext &context)
00697 {
00698     // Update client latency and last seen time
00699     {
00700         std::lock_guard<std::mutex> lock(m_clientsMutex);
00701         auto it = m_clients.find(context.sessionId);
00702         if (it != m_clients.end())
00703         {
00704             it->second.lastSeen = context.receiveTime;
00705             // Calculate latency based on ping time
00706             auto now = std::chrono::steady_clock::now();
00707             auto pingTime = std::chrono::milliseconds(packet.sendTimeMs);
00708             auto currentTime = std::chrono::duration_cast<std::chrono::milliseconds>(now.time_since_epoch());
00709             it->second.latency = static_cast<std::uint32_t>((currentTime - pingTime).count());
00710         }
00711     }
00712
00713     return rnp::HandlerResult::SUCCESS;
00714 }
00715
00716 void AsioServer::sendPong(std::uint32_t nonce, const asio::ip::udp::endpoint &destination, std::uint32_t sessionId)
00717 {
00718     rnp::Serializer serializer;
00719     rnp::PacketHeader header{};
00720     header.type = static_cast<std::uint8_t>(rnp::PacketType::PONG);
00721     header.length = sizeof(rnp::PacketPingPong);
00722     header.sessionId = sessionId;
00723
00724     rnp::PacketPingPong pong{};
00725     pong.nonce = nonce;
00726     pong.sendTimeMs = static_cast<std::uint32_t>(
00727         std::chrono::duration_cast<std::chrono::milliseconds>(std::chrono::steady_clock::now().time_since_epoch())
00728         .count());
00729
00730     serializer.serializeHeader(header);
00731     serializer.serializePingPong(pong);
00732
00733     sendPacketImmediate(serializer.getData(), destination);
00734 }
00735

```

```

00736 void AsioServer::sendConnectAccept(std::uint32_t sessionId, const asio::ip::udp::endpoint &destination)
00737 {
00738     // utl::Logger::log("AsioServer: Preparing CONNECT_ACCEPT packet for session " + std::to_string(sessionId) +
00739     // " to " + destination.address().to_string() + ":" + std::to_string(destination.port()),
00740     // utl::LogLevel::INFO);
00741
00742     rnp::Serializer serializer;
00743     rnp::PacketHeader header{};
00744     header.type = static_cast<std::uint8_t>(rnp::PacketType::CONNECT_ACCEPT);
00745     header.length = sizeof(rnp::PacketConnectAccept);
00746     header.sessionId = sessionId;
00747
00748     rnp::PacketConnectAccept accept{};
00749     accept.sessionId = sessionId;
00750     accept.tickRateHz = m_tickRate;
00751     accept.mtuPayloadBytes = rnp::MAX_PAYLOAD;
00752     accept.serverCaps = m_serverCaps;
00753
00754     serializer.serializeHeader(header);
00755     serializer.serializeConnectAccept(accept);
00756
00757     // utl::Logger::log("AsioServer: Sending CONNECT_ACCEPT packet (" + std::to_string(serializer.getData().size())
00758     // +
00759     // " bytes)",
00760     // utl::LogLevel::INFO);
00761     sendPacketImmediate(serializer.getData(), destination);
00762 }
00763
00764 void AsioServer::sendError(rnp::ErrorCode errorCode, const std::string &description,
00765     const asio::ip::udp::endpoint &destination, std::uint32_t sessionId)
00766 {
00767     rnp::Serializer serializer;
00768     rnp::PacketHeader header{};
00769     header.type = static_cast<std::uint8_t>(rnp::PacketType::PACKET_ERROR);
00770     header.length = static_cast<std::uint16_t>(sizeof(rnp::PacketError) + description.length());
00771     header.sessionId = sessionId;
00772
00773     rnp::PacketError error{};
00774     error.errorCode = static_cast<std::uint16_t>(errorCode);
00775     error.msgLen = static_cast<std::uint16_t>(description.length());
00776     error.description = description;
00777
00778     serializer.serializeHeader(header);
00779     serializer.serializeError(error);
00780
00781     sendPacketImmediate(serializer.getData(), destination);
00782 }
00783
00784 void AsioServer::updateClientManagement()
00785 {
00786     const auto now = std::chrono::steady_clock::now();
00787
00788     // Send pings periodically
00789     if (now - m_lastPingTime > m_pingInterval)
00790     {
00791         std::lock_guard<std::mutex> lock(m_clientsMutex);
00792         for (auto &[sessionId, client] : m_clients)
00793         {
00794             if (client.isConnected)
00795             {
00796                 // Send ping
00797                 rnp::Serializer serializer;
00798                 rnp::PacketHeader header{};
00799                 header.type = static_cast<std::uint8_t>(rnp::PacketType::PING);
00800                 header.length = sizeof(rnp::PacketPingPong);
00801                 header.sessionId = sessionId;
00802
00803                 rnp::PacketPingPong ping{};
00804                 ping.nonce = ++client.lastPingSent;
00805                 ping.sendTimeMs = static_cast<std::uint32_t>(
00806                     std::chrono::duration_cast<std::chrono::milliseconds>(now.time_since_epoch()).count());
00807
00808                 serializer.serializeHeader(header);
00809                 serializer.serializePingPong(ping);
00810
00811                 std::lock_guard<std::mutex> queueLock(m_sendQueueMutex);
00812                 m_sendQueue.emplace(serializer.getData(), client.endpoint, false);
00813             }
00814         }
00815         m_lastPingTime = now;
00816     }
00817
00818     // Check for timeouts
00819     std::vector<std::uint32_t> timedOutClients;
00820     {
00821         std::lock_guard<std::mutex> lock(m_clientsMutex);
00822         for (const auto &[sessionId, client] : m_clients)

```

```

00823     {
00824         if (client.isConnected && (now - client.lastSeen) > m_clientTimeout)
00825         {
00826             timedOutClients.push_back(sessionId);
00827         }
00828     }
00829 }
00830
00831 // Disconnect timed out clients
00832 for (std::uint32_t sessionId : timedOutClients)
00833 {
00834     // utl::Logger::log("AsioServer: Client " + std::to_string(sessionId) + " timed out", utl::LogLevel::INFO);
00835     disconnectClient(sessionId);
00836 }
00837 }
00838
00839 void AsioServer::processSendQueue()
00840 {
00841     std::lock_guard<std::mutex> lock(m_sendQueueMutex);
00842     while (!m_sendQueue.empty())
00843     {
00844         const QueuedPacket &packet = m_sendQueue.front();
00845         sendPacketImmediate(packet.data, packet.destination);
00846         m_sendQueue.pop();
00847     }
00848 }
00849
00850 void AsioServer::processEventBusEvents()
00851 {
00852     // Consommer les événements du GameServer
00853     const auto events = m_eventBus.consumeForTarget(m_componentId);
00854
00855     for (const auto &event : events)
00856     {
00857         switch (event.type)
00858         {
00859             case utl::EventType::SEND_TO_CLIENT:
00860                 handleSendToClientEvent(event);
00861                 break;
00862
00863             case utl::EventType::BROADCAST_WORLD_STATE:
00864                 handleBroadcastEvent(event);
00865                 break;
00866
00867             case utl::EventType::SEND_ENTITY_EVENTS:
00868                 handleSendEntityEventToClients(event);
00869                 break;
00870
00871             default:
00872                 // utl::Logger::log("AsioServer: Événement non géré: " +
00873                 //                 std::to_string(static_cast<uint32_t>(event.type)),
00874                 //                 utl::LogLevel::WARNING);
00875                 break;
00876         }
00877     }
00878 }
00879
00880 void AsioServer::handleSendToClientEvent(const utl::Event &event)
00881 {
00882     // Extraire l'ID du client cible et les données
00883     if (event.data.size() >= sizeof(uint32_t))
00884     {
00885         uint32_t sessionId = 0;
00886         std::memcpy(&sessionId, event.data.data(), sizeof(uint32_t));
00887
00888         std::vector<uint8_t> messageData(event.data.begin() + sizeof(uint32_t), event.data.end());
00889
00890         // Utiliser la méthode existante
00891         sendToClient(sessionId, messageData);
00892
00893         // utl::Logger::log("AsioServer: Message envoyé au client " + std::to_string(sessionId) +
00894         //                 " (taille: " + std::to_string(messageData.size()) + " bytes)",
00895         //                 utl::LogLevel::INFO);
00896     }
00897 }
00898
00899 void AsioServer::handleBroadcastEvent(const utl::Event &event)
00900 {
00901     // Utiliser la méthode existante pour broadcaster
00902     sendToAllClients(event.data);
00903
00904     // utl::Logger::log(
00905     //     "AsioServer: Message diffuse a tous les clients (taille: " + std::to_string(event.data.size()) + "
00906     //     bytes)", utl::LogLevel::INFO);
00907 }
00908
00909 void AsioServer::handleSendEntityEventToClients(const utl::Event &event)

```

```

00910 {
00911     // Forward entity events to all clients
00912     sendToAllClients(event.data);
00913
00914     // utl::Logger::log("AsioServer: Entity events diffuses a tous les clients (taille: " +
00915     //                 std::to_string(event.data.size()) + " bytes)",
00916     //                 utl::LogLevel::INFO);
00917 }
00918
00919 rnp::HandlerResult AsioServer::handleEntityEvent(const std::vector<rnp::EventRecord> &events,
00920                                                 const rnp::PacketContext &context) const
00921 {
00922     utl::Logger::log("AsioServer: Received " + std::to_string(events.size()) + " entity events from session " +
00923                     std::to_string(context.sessionId),
00924                     utl::LogLevel::INFO);
00925
00926     // Filtrer les événements d'input et les publier vers GameServer
00927     for (const auto &eventRecord : events)
00928     {
00929         if (eventRecord.type == rnp::EventType::INPUT)
00930         {
00931             utl::Logger::log("AsioServer: Input event received, data size: " +
00932                             std::to_string(eventRecord.data.size()),
00933                             utl::LogLevel::INFO);
00934
00935             // Create event with sessionId as sourceId so RTypeServer knows which player
00936             utl::Event event(utl::EventType::PLAYER_INPUT_RECEIVED, context.sessionId, utl::GAME_LOGIC);
00937             event.data = eventRecord.data;
00938             m_eventBus.publish(event);
00939
00940             // utl::Logger::log("AsioServer: Input joueur publie vers GameServer (sessionId: " +
00941             //                 std::to_string(context.sessionId) + ")",
00942             //                 utl::LogLevel::INFO);
00943         }
00944         else
00945         {
00946             m_eventBus.publish(utl::EventType::ENTITY_EVENT_RECEIVED, eventRecord.data, m_componentId,
00947                               utl::GAME_LOGIC); // GameServer ID
00948
00949             // utl::Logger::log("AsioServer: Evenement entite publie vers GameServer (type: " +
00950             //                 std::to_string(static_cast<uint8_t>(eventRecord.type)) + ")",
00951             //                 utl::LogLevel::INFO);
00952         }
00953     }
00954
00955     return rnp::HandlerResult::SUCCESS;
00956 }
00957
00958 // Lobby System Implementation
00959
00960 rnp::HandlerResult AsioServer::handleLobbyListRequest(const rnp::PacketContext &context)
00961 {
00962     utl::Logger::log("AsioServer: Handling LOBBY_LIST_REQUEST from session " + std::to_string(context.sessionId),
00963                     utl::LogLevel::INFO);
00964
00965     // Verify session exists
00966     {
00967         std::lock_guard<std::mutex> clientsLock(m_clientsMutex);
00968         auto clientIt = m_clients.find(context.sessionId);
00969         if (clientIt == m_clients.end() || !clientIt->second.isConnected)
00970         {
00971             sendError(rnp::ErrorCode::UNAUTHORIZED_SESSION, "Invalid session", m_senderEndpoint,
00972                     context.sessionId);
00973             return rnp::HandlerResult::PROCESSING_ERROR;
00974         }
00975     } // Release the lock before calling sendLobbyList
00976
00977     sendLobbyList(context.sessionId);
00978     return rnp::HandlerResult::SUCCESS;
00979 }
00980
00981 rnp::HandlerResult AsioServer::handleLobbyCreate(const rnp::PacketLobbyCreate &packet,
00982                                                 const rnp::PacketContext &context)
00983 {
00984     utl::Logger::log("AsioServer: Handling LOBBY_CREATE from session " + std::to_string(context.sessionId),
00985                     utl::LogLevel::INFO);
00986
00987     std::uint32_t lobbyId = 0;
00988     bool success = false;
00989
00990     // Scope for clientsLock to release it before broadcastLobbyUpdate
00991     {
00992         std::lock_guard<std::mutex> clientsLock(m_clientsMutex);
00993         auto clientIt = m_clients.find(context.sessionId);
00994         if (clientIt == m_clients.end() || !clientIt->second.isConnected)
00995         {
00996             sendLobbyCreateResponse(context.sessionId, 0, false, rnp::ErrorCode::UNAUTHORIZED_SESSION);

```

```

00996         return rnp::HandlerResult::PROCESSING_ERROR;
00997     }
00998
00999     utl::Logger::log("AsioServer: LOBBY_JOIN - Client current lobby ID: " +
01000         std::to_string(clientIt->second.currentLobbyId),
01001         utl::LogLevel::INFO);
01002
01003     if (clientIt->second.currentLobbyId != 0)
01004     {
01005         sendLobbyCreateResponse(context.sessionId, 0, false, rnp::ErrorCode::ALREADY_IN_LOBBY);
01006         return rnp::HandlerResult::PROCESSING_ERROR;
01007     }
01008
01009     // Extract lobby name
01010     std::string lobbyName;
01011     if (packet.nameLen > 0 && packet.nameLen <= 31)
01012     {
01013         lobbyName = std::string(packet.lobbyName.data(), packet.nameLen);
01014     }
01015     else
01016     {
01017         lobbyName = "Lobby " + std::to_string(m_nextLobbyId);
01018     }
01019
01020     // Create lobby
01021     lobbyId = createLobby(lobbyName, context.sessionId, packet.maxPlayers, packet.gameMode);
01022     if (lobbyId > 0)
01023     {
01024         clientIt->second.currentLobbyId = lobbyId;
01025         sendLobbyCreateResponse(context.sessionId, lobbyId, true);
01026         success = true;
01027     }
01028     else
01029     {
01030         sendLobbyCreateResponse(context.sessionId, 0, false, rnp::ErrorCode::INTERNAL_ERROR);
01031     }
01032 } // clientsLock released here
01033
01034 // Broadcast lobby update AFTER releasing clientsLock to avoid deadlock
01035 if (success && lobbyId > 0)
01036 {
01037     broadcastLobbyUpdate(lobbyId);
01038 }
01039
01040 return rnp::HandlerResult::SUCCESS;
01041 }
01042
01043 rnp::HandlerResult AsioServer::handleLobbyJoin(const rnp::PacketLobbyJoin &packet,
01044     const rnp::PacketContext &context)
01045 {
01046     utl::Logger::log("AsioServer: Handling LOBBY_JOIN from session " + std::to_string(context.sessionId) +
01047         " for lobby " + std::to_string(packet.lobbyId),
01048         utl::LogLevel::INFO);
01049
01050     bool joinSuccess = false;
01051
01052     // Scope for clientsLock to release it before lobbyToLobbyInfo and broadcastLobbyUpdate
01053     {
01054         std::lock_guard<std::mutex> clientsLock(m_clientsMutex);
01055         utl::Logger::log("AsioServer: LOBBY_JOIN - Checking client session " + std::to_string(context.sessionId),
01056             utl::LogLevel::INFO);
01057         auto clientIt = m_clients.find(context.sessionId);
01058         if (clientIt == m_clients.end() || !clientIt->second.isConnected)
01059         {
01060             utl::Logger::log("AsioServer: LOBBY_JOIN failed - unauthorized session " +
01061                 std::to_string(context.sessionId),
01062                 utl::LogLevel::WARNING);
01063             sendLobbyJoinResponse(context.sessionId, packet.lobbyId, false,
01064                 rnp::ErrorCode::UNAUTHORIZED_SESSION);
01065             return rnp::HandlerResult::PROCESSING_ERROR;
01066         }
01067
01068         if (clientIt->second.currentLobbyId != 0)
01069         {
01070             utl::Logger::log("AsioServer: LOBBY_JOIN failed - session " + std::to_string(context.sessionId) +
01071                 " already in lobby " + std::to_string(clientIt->second.currentLobbyId),
01072                 utl::LogLevel::WARNING);
01073             sendLobbyJoinResponse(context.sessionId, packet.lobbyId, false, rnp::ErrorCode::ALREADY_IN_LOBBY);
01074             return rnp::HandlerResult::PROCESSING_ERROR;
01075         }
01076
01077         // Try to join lobby
01078         LobbyStatus joinStatus = joinLobby(packet.lobbyId, context.sessionId);
01079         utl::Logger::log("AsioServer: joinLobby returned status: " + std::to_string(static_cast<int>(joinStatus)),
01080             utl::LogLevel::INFO);
01081
01082         if (joinStatus == LobbyStatus::SUCCESS)

```



```

01082     {
01083         clientIt->second.currentLobbyId = packet.lobbyId;
01084         utl::Logger::log("AsioServer: Session " + std::to_string(context.sessionId) +
01085             " successfully joined lobby " + std::to_string(packet.lobbyId),
01086             utl::LogLevel::INFO);
01087         joinSuccess = true;
01088     }
01089     else
01090     {
01091         utl::Logger::log("AsioServer: Session " + std::to_string(context.sessionId) + " failed to join lobby " +
01092             std::to_string(packet.lobbyId) +
01093             " - status: " + std::to_string(static_cast<int>(joinStatus)),
01094             utl::LogLevel::WARNING);
01095         sendLobbyJoinResponse(context.sessionId, packet.lobbyId, false, rnp::ErrorCode::LOBBY_NOT_FOUND,
01096             nullptr);
01097     }
01098 } // clientsLock released here
01099
01100 // Get lobby info AFTER releasing clientsLock to avoid deadlock in lobbyToLobbyInfo
01101 if (joinSuccess)
01102 {
01103     rnp::LobbyInfo lobbyInfo;
01104     {
01105         std::lock_guard<std::mutex> lobbiesLock(m_lobbiesMutex);
01106         auto lobbyIt = m_lobbies.find(packet.lobbyId);
01107         if (lobbyIt != m_lobbies.end())
01108         {
01109             lobbyInfo = lobbyToLobbyInfo(lobbyIt->second);
01110             utl::Logger::log("AsioServer: Lobby info - currentPlayers: " +
01111                 std::to_string(static_cast<int>(lobbyInfo.currentPlayers)) + "/" +
01112                 std::to_string(static_cast<int>(lobbyInfo.maxPlayers)),
01113                 utl::LogLevel::INFO);
01114         }
01115     }
01116
01117     sendLobbyJoinResponse(context.sessionId, packet.lobbyId, true, rnp::ErrorCode::NONE, &lobbyInfo);
01118 }
01119
01120 // Broadcast lobby update AFTER releasing clientsLock to avoid deadlock
01121 if (joinSuccess)
01122 {
01123     broadcastLobbyUpdate(packet.lobbyId);
01124 }
01125
01126 return rnp::HandlerResult::SUCCESS;
01127 }
01128
01129 rnp::HandlerResult AsioServer::handleLobbyLeave(const rnp::PacketContext &context)
01130 {
01131     utl::Logger::log("AsioServer: Handling LOBBY_LEAVE from session " + std::to_string(context.sessionId),
01132         utl::LogLevel::INFO);
01133
01134     std::uint32_t lobbyId = 0;
01135
01136     // Scope for clientsLock to release it before broadcastLobbyUpdate
01137     {
01138         std::lock_guard<std::mutex> clientsLock(m_clientsMutex);
01139         auto clientIt = m_clients.find(context.sessionId);
01140         if (clientIt == m_clients.end() || !clientIt->second.isConnected)
01141         {
01142             sendError(rnp::ErrorCode::UNAUTHORIZED_SESSION, "Invalid session", m_senderEndpoint,
01143                 context.sessionId);
01144             return rnp::HandlerResult::PROCESSING_ERROR;
01145         }
01146
01147         // Check if client is in a lobby
01148         if (clientIt->second.currentLobbyId == 0)
01149         {
01150             sendError(rnp::ErrorCode::NOT_IN_LOBBY, "Not in lobby", m_senderEndpoint, context.sessionId);
01151             return rnp::HandlerResult::PROCESSING_ERROR;
01152         }
01153
01154         lobbyId = clientIt->second.currentLobbyId;
01155         clientIt->second.currentLobbyId = 0; // Clear lobby association before leaving
01156     } // clientsLock released here
01157
01158     // Leave lobby after releasing clientsLock to avoid nested mutex acquisitions
01159     leaveLobby(context.sessionId);
01160
01161     // Broadcast update to remaining players AFTER releasing clientsLock to avoid deadlock
01162     if (lobbyId != 0)
01163     {
01164         broadcastLobbyUpdate(lobbyId);
01165     }
01166
01167     return rnp::HandlerResult::SUCCESS;
01168 }

```



```

01168
01169 void AsioServer::sendLobbyList(const std::uint32_t sessionId)
01170 {
01171     utl::Logger::log("AsioServer: Sending lobby list to session " + std::to_string(sessionId), utl::LogLevel::INFO);
01172
01173     rnp::PacketLobbyListResponse response;
01174     {
01175         std::lock_guard<std::mutex> lobbiesLock(m_lobbiesMutex);
01176         response.lobbyCount = static_cast<std::uint16_t>(m_lobbies.size());
01177         response.lobbies.reserve(m_lobbies.size());
01178
01179         for (const auto &lobby : m_lobbies | std::views::values)
01180         {
01181             rnp::LobbyInfo lobbyInfo = lobbyToLobbyInfo(lobby);
01182             utl::Logger::log("AsioServer: Adding lobby " + std::to_string(lobbyInfo.lobbyId) + " with " +
01183                 std::to_string(static_cast<int>(lobbyInfo.currentPlayers)) + "/" +
01184                 std::to_string(static_cast<int>(lobbyInfo.maxPlayers)) + " players",
01185                 utl::LogLevel::INFO);
01186             response.lobbies.push_back(lobbyInfo);
01187         }
01188     }
01189
01190     // Serialize header and response data
01191     std::vector<std::uint8_t> packet;
01192     rnp::Serializer packetSerializer(packet);
01193
01194     rnp::PacketHeader header;
01195     header.type = static_cast<std::uint8_t>(rnp::PacketType::LOBBY_LIST_RESPONSE);
01196     header.length = static_cast<std::uint16_t>(sizeof(rnp::PacketLobbyListResponse));
01197     header.sessionId = sessionId;
01198
01199     packetSerializer.serializeHeader(header);
01200     packetSerializer.serializeLobbyListResponse(response);
01201
01202     utl::Logger::log("AsioServer: Sending lobby list with " + std::to_string(response.lobbyCount) +
01203         " lobbies, packet size: " + std::to_string(packetSerializer.getData().size()),
01204         utl::LogLevel::INFO);
01205
01206     sendPacketImmediate(packetSerializer.getData(), m_senderEndpoint);
01207 }
01208
01209 std::uint32_t AsioServer::createLobby(const std::string &name, std::uint32_t hostSession, std::uint8_t maxPlayers,
01210     std::uint8_t gameMode)
01211 {
01212     std::lock_guard<std::mutex> lobbiesLock(m_lobbiesMutex);
01213
01214     std::uint32_t lobbyId = m_nextLobbyId++;
01215     Lobby lobby(lobbyId, name, hostSession, maxPlayers, gameMode);
01216     lobby.playerSessions.push_back(hostSession);
01217
01218     m_lobbies[lobbyId] = std::move(lobby);
01219
01220     utl::Logger::log("AsioServer: Created lobby " + std::to_string(lobbyId) + " '" + name + "' hosted by " +
01221         std::to_string(hostSession),
01222         utl::LogLevel::INFO);
01223
01224     return lobbyId;
01225 }
01226
01227 LobbyStatus AsioServer::joinLobby(std::uint32_t lobbyId, std::uint32_t sessionId)
01228 {
01229     std::lock_guard<std::mutex> lobbiesLock(m_lobbiesMutex);
01230
01231     utl::Logger::log("AsioServer: joinLobby - Looking for lobby " + std::to_string(lobbyId), utl::LogLevel::INFO);
01232
01233     auto lobbyIt = m_lobbies.find(lobbyId);
01234     if (lobbyIt == m_lobbies.end())
01235     {
01236         utl::Logger::log("AsioServer: joinLobby - Lobby " + std::to_string(lobbyId) + " not found",
01237             utl::LogLevel::WARNING);
01238         return LobbyStatus::NOT_FOUND;
01239     }
01240
01241     Lobby &lobby = lobbyIt->second;
01242     utl::Logger::log(
01243         "AsioServer: joinLobby - Lobby found, current players: " + std::to_string(lobby.playerSessions.size()) +
01244         "/" + std::to_string(lobby.maxPlayers) + ", status: " + std::to_string(static_cast<int>(lobby.status)),
01245         utl::LogLevel::INFO);
01246
01247     if (lobby.playerSessions.size() >= lobby.maxPlayers)
01248     {
01249         utl::Logger::log("AsioServer: joinLobby - Lobby is full", utl::LogLevel::WARNING);
01250         return LobbyStatus::FULL;
01251     }
01252
01253     if (lobby.status != rnp::LobbyStatus::WAITING)
01254     {

```

```

01255         utl::Logger::log("AsioServer: joinLobby - Game already started", utl::LogLevel::WARNING);
01256         return LobbyStatus::IN_GAME;
01257     }
01258
01259     if (auto playerIt = std::ranges::find(lobby.playerSessions, sessionId); playerIt != lobby.playerSessions.end())
01260     {
01261         return LobbyStatus::ALREADY_IN_LOBBY;
01262     }
01263
01264     lobby.playerSessions.push_back(sessionId);
01265
01266     utl::Logger::log("AsioServer: Player " + std::to_string(sessionId) + " joined lobby " +
01267                     std::to_string(lobbyId) + " (now " + std::to_string(lobby.playerSessions.size()) + "/" +
01268                     std::to_string(static_cast<int>(lobby.maxPlayers)) + " players)",
01269                     utl::LogLevel::INFO);
01270     return LobbyStatus::SUCCESS;
01271 }
01272
01273 void AsioServer::leaveLobby(std::uint32_t sessionId)
01274 {
01275     std::lock_guard<std::mutex> lobbiesLock(m_lobbiesMutex);
01276
01277     for (auto &[lobbyId, lobby] : m_lobbies)
01278     {
01279         if (auto playerIt = std::ranges::find(lobby.playerSessions, sessionId);
01280             playerIt != lobby.playerSessions.end())
01281         {
01282             lobby.playerSessions.erase(playerIt);
01283
01284             utl::Logger::log("AsioServer: Player " + std::to_string(sessionId) + " left lobby " +
01285                             std::to_string(lobbyId),
01286                             utl::LogLevel::INFO);
01287
01288             // If host left, assign new host or delete lobby
01289             if (lobby.hostSessionId == sessionId)
01290             {
01291                 if (!lobby.playerSessions.empty())
01292                 {
01293                     lobby.hostSessionId = lobby.playerSessions[0];
01294                     utl::Logger::log("AsioServer: New host for lobby " + std::to_string(lobbyId) + " is " +
01295                                     std::to_string(lobby.hostSessionId),
01296                                     utl::LogLevel::INFO);
01297                 }
01298                 else
01299                 {
01300                     // No players left, lobby will be cleaned up
01301                     utl::Logger::log("AsioServer: Lobby " + std::to_string(lobbyId) + " is now empty",
01302                                     utl::LogLevel::INFO);
01303                 }
01304             }
01305             break;
01306         }
01307     }
01308
01309     cleanupEmptyLobbies();
01310 }
01311
01312 void AsioServer::broadcastLobbyUpdate(std::uint32_t lobbyId)
01313 {
01314     std::lock_guard<std::mutex> lobbiesLock(m_lobbiesMutex);
01315     auto lobbyIt = m_lobbies.find(lobbyId);
01316     if (lobbyIt == m_lobbies.end())
01317     {
01318         return;
01319     }
01320
01321     rnp::PacketLobbyUpdate update;
01322     update.lobbyInfo = lobbyToLobbyInfo(lobbyIt->second);
01323
01324     // Serialize packet data
01325     std::vector<std::uint8_t> data;
01326     rnp::Serializer serializer(data);
01327     serializer.serializeLobbyUpdate(update);
01328
01329     // Get the serialized data
01330     const std::vector<std::uint8_t> &lobbyUpdateData = serializer.getData();
01331
01332     // Send to all players in lobby
01333     for (std::uint32_t sessionId : lobbyIt->second.playerSessions)
01334     {
01335         rnp::PacketHeader header;
01336         header.type = static_cast<std::uint8_t>(rnp::PacketType::LOBBY_UPDATE);
01337         header.length = static_cast<std::uint16_t>(lobbyUpdateData.size());
01338         header.sessionId = sessionId;
01339
01340         std::vector<std::uint8_t> playerPacket;
01341         rnp::Serializer playerPacketSerializer(playerPacket);

```

```

01342     playerPacketSerializer.serializeHeader(header);
01343     playerPacketSerializer.serializeLobbyUpdate(update);
01344
01345     // Lock clients mutex to safely access client data
01346     std::lock_guard<std::mutex> clientsLock(m_clientsMutex);
01347     auto clientIt = m_clients.find(sessionId);
01348     if (clientIt != m_clients.end())
01349     {
01350         sendPacketImmediate(playerPacketSerializer.getData(), clientIt->second.endpoint);
01351     }
01352 }
01353
01354 utl::Logger::log("AsioServer: Broadcasted lobby update for lobby " + std::to_string(lobbyId),
01355                 utl::LogLevel::INFO);
01356 }
01357
01358 rnp::LobbyInfo AsioServer::lobbyToLobbyInfo(const Lobby &lobby)
01359 {
01360     rnp::LobbyInfo info;
01361     info.lobbyId = lobby.lobbyId;
01362     info.currentPlayers = static_cast<std::uint8_t>(lobby.playerSessions.size());
01363     info.maxPlayers = lobby.maxPlayers;
01364     info.gameMode = lobby.gameMode;
01365     info.status = static_cast<std::uint8_t>(lobby.status);
01366     info.hostSessionId = lobby.hostSessionId;
01367
01368     // Copy lobby name
01369     std::size_t nameLen = std::min(lobby.lobbyName.size(), static_cast<std::size_t>(31));
01370     std::memcpy(info.lobbyName.data(), lobby.lobbyName.c_str(), nameLen);
01371     info.lobbyName[nameLen] = '\0';
01372
01373     // Copy player names
01374     std::lock_guard<std::mutex> clientsLock(m_clientsMutex);
01375     for (std::size_t i = 0; i < lobby.playerSessions.size() && i < 8; ++i)
01376     {
01377         auto clientIt = m_clients.find(lobby.playerSessions[i]);
01378         if (clientIt != m_clients.end())
01379         {
01380             std::size_t playerNameLen = std::min(clientIt->second.playerName.size(), static_cast<std::size_t>(31));
01381             std::memcpy(info.playerNames[i].data(), clientIt->second.playerName.c_str(), playerNameLen);
01382             info.playerNames[i][playerNameLen] = '\0';
01383         }
01384     }
01385
01386     return info;
01387 }
01388
01389 void AsioServer::sendLobbyCreateResponse(std::uint32_t sessionId, std::uint32_t lobbyId, bool success,
01390                                         rnp::ErrorCode errorCode)
01391 {
01392     rnp::PacketLobbyCreateResponse response;
01393     response.lobbyId = lobbyId;
01394     response.success = success ? 1 : 0;
01395     response.errorCode = static_cast<std::uint16_t>(errorCode);
01396
01397     rnp::PacketHeader header;
01398     header.type = static_cast<std::uint8_t>(rnp::PacketType::LOBBY_CREATE_RESPONSE);
01399     header.length = static_cast<std::uint16_t>(sizeof(rnp::PacketLobbyCreateResponse));
01400     header.sessionId = sessionId;
01401
01402     std::vector<std::uint8_t> packet;
01403     rnp::Serializer packetSerializer(packet);
01404     packetSerializer.serializeHeader(header);
01405     packetSerializer.serializeLobbyCreateResponse(response);
01406
01407     sendPacketImmediate(packetSerializer.getData(), m_senderEndpoint);
01408
01409     utl::Logger::log("AsioServer: Sent lobby create response to session " + std::to_string(sessionId) +
01410                     " (success: " + (success ? "true" : "false") + ")",
01411                     utl::LogLevel::INFO);
01412 }
01413
01414 void AsioServer::sendLobbyJoinResponse(std::uint32_t sessionId, std::uint32_t lobbyId, bool success,
01415                                       rnp::ErrorCode errorCode, const rnp::LobbyInfo *lobbyInfo)
01416 {
01417     rnp::PacketLobbyJoinResponse response;
01418     response.lobbyId = lobbyId;
01419     response.success = success ? 1 : 0;
01420     response.errorCode = static_cast<std::uint16_t>(errorCode);
01421     if (success && lobbyInfo)
01422     {
01423         response.lobbyInfo = *lobbyInfo;
01424     }
01425
01426     rnp::PacketHeader header;
01427     header.type = static_cast<std::uint8_t>(rnp::PacketType::LOBBY_JOIN_RESPONSE);
01428     header.length = static_cast<std::uint16_t>(sizeof(rnp::PacketLobbyJoinResponse));

```

```

01429     header.sessionId = sessionId;
01430
01431     std::vector<std::uint8_t> packet;
01432     rnp::Serializer packetSerializer(packet);
01433     packetSerializer.serializeHeader(header);
01434     packetSerializer.serializeLobbyJoinResponse(response);
01435
01436     sendPacketImmediate(packetSerializer.getData(), m_senderEndpoint);
01437
01438     utl::Logger::log("AsioServer: Sent lobby join response to session " + std::to_string(sessionId) +
01439                     " (success: " + (success ? "true" : "false") + ")",
01440                     utl::LogLevel::INFO);
01441 }
01442
01443 void AsioServer::cleanupEmptyLobbies()
01444 {
01445     // NOTE: Assumes m_lobbiesMutex is already held by caller
01446     auto it = m_lobbies.begin();
01447     while (it != m_lobbies.end())
01448     {
01449         if (it->second.playerSessions.empty())
01450         {
01451             utl::Logger::log("AsioServer: Cleaning up empty lobby " + std::to_string(it->first),
01452                             utl::LogLevel::INFO);
01453             it = m_lobbies.erase(it);
01454         }
01455         else
01456         {
01457             ++it;
01458         }
01459     }
01460 }
01461
01462 rnp::HandlerResult AsioServer::handleStartGameRequest(const rnp::PacketStartGameRequest &packet,
01463                                                       const rnp::PacketContext &context)
01464 {
01465     utl::Logger::log("AsioServer: Handling START_GAME_REQUEST from session " +
01466                     std::to_string(context.sessionId) +
01467                     " for lobby " + std::to_string(packet.lobbyId),
01468                     utl::LogLevel::INFO);
01469
01470     // Verify session exists and is in the lobby
01471     {
01472         std::lock_guard<std::mutex> clientsLock(m_clientsMutex);
01473         auto clientIt = m_clients.find(context.sessionId);
01474         if (clientIt == m_clients.end() || !clientIt->second.isConnected)
01475         {
01476             sendError(rnp::ErrorCode::UNAUTHORIZED_SESSION, "Invalid session", m_senderEndpoint,
01477                     context.sessionId);
01478             return rnp::HandlerResult::PROCESSING_ERROR;
01479         }
01480
01481         if (clientIt->second.currentLobbyId != packet.lobbyId)
01482         {
01483             sendError(rnp::ErrorCode::NOT_IN_LOBBY, "Not in this lobby", m_senderEndpoint, context.sessionId);
01484             return rnp::HandlerResult::PROCESSING_ERROR;
01485         }
01486
01487         // Verify the session is the host and lobby exists
01488         {
01489             std::lock_guard<std::mutex> lobbiesLock(m_lobbiesMutex);
01490             auto lobbyIt = m_lobbies.find(packet.lobbyId);
01491             if (lobbyIt == m_lobbies.end())
01492             {
01493                 sendError(rnp::ErrorCode::LOBBY_NOT_FOUND, "Lobby not found", m_senderEndpoint, context.sessionId);
01494                 return rnp::HandlerResult::PROCESSING_ERROR;
01495             }
01496
01497             // Only host can start the game
01498             if (lobbyIt->second.hostSessionId != context.sessionId)
01499             {
01500                 sendError(rnp::ErrorCode::UNAUTHORIZED_SESSION, "Only host can start game", m_senderEndpoint,
01501                         context.sessionId);
01502                 return rnp::HandlerResult::PROCESSING_ERROR;
01503             }
01504
01505             // Update lobby status to IN_GAME
01506             lobbyIt->second.status = rnp::LobbyStatus::IN_GAME;
01507         }
01508
01509         utl::Logger::log("AsioServer: Starting game for lobby " + std::to_string(packet.lobbyId), utl::LogLevel::INFO);
01510
01511         // Get sessionIds of players in lobby
01512         std::vector<std::uint32_t> playerSessions;
01513         {
01514             std::lock_guard<std::mutex> lobbiesLock(m_lobbiesMutex);

```

```

01514         auto lobbyIt = m_lobbies.find(packet.lobbyId);
01515         if (lobbyIt != m_lobbies.end())
01516         {
01517             playerSessions = lobbyIt->second.playerSessions;
01518             utl::Logger::log("AsioServer: Player sessions in lobby: " + std::to_string(playerSessions.size()),
01519                             utl::LogLevel::INFO);
01520         }
01521     }
01522
01523     // Notify RTypeServer about game start with player sessions
01524     utl::Event startEvent(utl::EventType::SERVER_START, m_componentId, utl::GAME_LOGIC);
01525     std::vector<std::uint8_t> sessionsData;
01526     sessionsData.resize(sizeof(std::uint32_t) * playerSessions.size());
01527     for (size_t i = 0; i < playerSessions.size(); ++i)
01528     {
01529         std::memcpy(sessionsData.data() + i * sizeof(std::uint32_t), &playerSessions[i], sizeof(std::uint32_t));
01530     }
01531     startEvent.data = sessionsData;
01532     m_eventBus.publish(startEvent);
01533
01534     utl::Logger::log("AsioServer: Sent SERVER_START to RTypeServer with " + std::to_string(playerSessions.size()) +
01535                     " players",
01536                     utl::LogLevel::INFO);
01537
01538     // Broadcast game start to all players
01539     broadcastGameStart(packet.lobbyId);
01540
01541     return rnp::HandlerResult::SUCCESS;
01542 }
01543
01544 void AsioServer::broadcastGameStart(std::uint32_t lobbyId)
01545 {
01546     std::lock_guard<std::mutex> lobbiesLock(m_lobbiesMutex);
01547     auto lobbyIt = m_lobbies.find(lobbyId);
01548     if (lobbyIt == m_lobbies.end())
01549     {
01550         return;
01551     }
01552
01553     rnp::PacketGameStart gameStart;
01554     gameStart.lobbyId = lobbyId;
01555
01556     // Serialize packet
01557     std::vector<std::uint8_t> data;
01558     rnp::Serializer serializer(data);
01559     serializer.serializeGameStart(gameStart);
01560
01561     const std::vector<std::uint8_t> &gameStartData = serializer.getData();
01562
01563     // Send to all players in lobby
01564     for (std::uint32_t sessionId : lobbyIt->second.playerSessions)
01565     {
01566         rnp::PacketHeader header;
01567         header.type = static_cast<std::uint8_t>(rnp::PacketType::GAME_START);
01568         header.length = static_cast<std::uint16_t>(gameStartData.size());
01569         header.sessionId = sessionId;
01570
01571         std::vector<std::uint8_t> playerPacket;
01572         rnp::Serializer playerPacketSerializer(playerPacket);
01573         playerPacketSerializer.serializeHeader(header);
01574         playerPacketSerializer.serializeGameStart(gameStart);
01575
01576         // Lock clients mutex to safely access client data
01577         std::lock_guard<std::mutex> clientsLock(m_clientsMutex);
01578         auto clientIt = m_clients.find(sessionId);
01579         if (clientIt != m_clients.end())
01580         {
01581             sendPacketImmediate(playerPacketSerializer.getData(), clientIt->second.endpoint);
01582         }
01583     }
01584
01585     utl::Logger::log("AsioServer: Broadcasted game start for lobby " + std::to_string(lobbyId),
01586                     utl::LogLevel::INFO);
01587
01588     // Publish GAME_START event to game logic to start waves
01589     m_eventBus.publish(utl::EventType::GAME_START, gameStartData, utl::NETWORK_SERVER,
01590                       utl::GAME_LOGIC);
01591     utl::Logger::log("AsioServer: Published GAME_START event to game logic", utl::LogLevel::INFO);
01592 }
01593 } // namespace srv

```

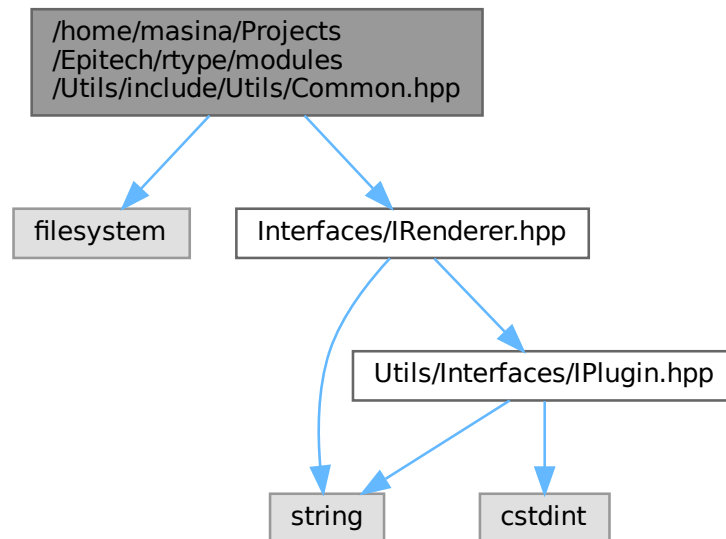
19.277 /home/masina/Projects/Epitech/rtype/modules/Utils/include/Utils/Common.hpp File Reference

This file contains common definitions and constants.

```
#include <filesystem>
```

```
#include "Interfaces/IRenderer.hpp"
```

Include dependency graph for Common.hpp:



This graph shows which files directly or indirectly include this file:



Namespaces

- namespace `utl`
- namespace `utl::Config`
- namespace `utl::Config::Player`
- namespace `utl::Config::Audio`
- namespace `utl::Config::Network`
- namespace `utl::Config::Window`
- namespace `utl::Config::Game`
- namespace `utl::Config::Color`
- namespace `utl::Path`
- namespace `utl::Path::Icons`
- namespace `utl::Path::Audio`
- namespace `utl::Path::Font`
- namespace `utl::Path::Plugin`
- namespace `utl::Path::Texture`

Variables

- constexpr auto `utl::Config::Player::DEFAULT_PLAYER_NAME` = "player"
- constexpr auto `utl::Config::Audio::DEFAULT_AUDIO_VOLUME` = 0.5F
- constexpr auto `utl::Config::Audio::DEFAULT_AUDIO_MUTED` = false
- constexpr auto `utl::Config::Network::DEFAULT_NETWORK_HOST` = "127.0.0.1"
- constexpr auto `utl::Config::Network::DEFAULT_NETWORK_PORT` = 2560
- constexpr auto `utl::Config::Window::DEFAULT_WINDOW_WIDTH` = 1920
- constexpr auto `utl::Config::Window::DEFAULT_WINDOW_HEIGHT` = 1080
- constexpr auto `utl::Config::Window::DEFAULT_WINDOW_FRAME_LIMIT` = 240
- constexpr auto `utl::Config::Window::DEFAULT_WINDOW_FULLSCREEN` = false
- constexpr auto `utl::Config::Game::DEFAULT_VIDEO_QUALITY` = 1
- constexpr auto `utl::Config::Game::DEFAULT_CONTROL_SCHEME` = 2
- constexpr auto `utl::Config::Game::DEFAULT_SKIN_INDEX` = 0
- static constexpr `eng::Color utl::Config::Color::DARK` = {.r = 0U, .g = 0U, .b = 0U, .a = 255U}
- static constexpr `eng::Color utl::Config::Color::CYAN_ELECTRIC` = {.r = 0U, .g = 191U, .b = 255U, .a = 255U}
- static constexpr `eng::Color utl::Config::Color::GRAY_BLUE_SUBTLE` = {.r = 160U, .g = 160U, .b = 160U, .a = 255U}
- static constexpr `eng::Color utl::Config::Color::STAR_BG` = {.r = 100U, .g = 100U, .b = 150U, .a = 80U}
- static constexpr `eng::Color utl::Config::Color::STAR_MID` = {.r = 150U, .g = 150U, .b = 200U, .a = 120U}
- static constexpr `eng::Color utl::Config::Color::YELLOW` = {.r = 255U, .g = 255U, .b = 200U, .a = 200U}
- static constexpr `eng::Color utl::Config::Color::CYAN_ELECTRIC_PARTICLES` = {.r = 0U, .g = 191U, .b = 255U, .a = 100U}
- static constexpr `eng::Color utl::Config::Color::CYAN_ELECTRIC_FOREGROUND` = {.r = 0U, .g = 191U, .b = 255U, .a = 180U}
- static constexpr `eng::Color utl::Config::Color::INFO_TEXT_COLOR` = {.r = 180U, .g = 180U, .b = 180U, .a = 200U}
- static constexpr `eng::Color utl::Config::Color::WHITE` = {.r = 255U, .g = 255U, .b = 255U, .a = 255U}
- static constexpr `eng::Color utl::Config::Color::WHITE_LOW` = {.r = 255U, .g = 255U, .b = 255U, .a = 0U}
- static constexpr `eng::Color utl::Config::Color::TEXT_VALUE_COLOR` = {.r = 200U, .g = 200U, .b = 255U, .a = 255U}
- static constexpr `eng::Color utl::Config::Color::WHITE_TRANS` = {.r = 255U, .g = 255U, .b = 255U, .a = 100U}
- static constexpr `eng::Color utl::Config::Color::BLUE` = {.r = 200U, .g = 200U, .b = 255U, .a = 150U}
- static constexpr `eng::Color utl::Config::Color::BLUE_SECOND` = {.r = 50U, .g = 100U, .b = 200U, .a = 60U}
- static constexpr `eng::Color utl::Config::Color::PURPLE` = {.r = 100U, .g = 50U, .b = 150U, .a = 80U}
- static constexpr `eng::Color utl::Config::Color::GREEN` = {.r = 200U, .g = 255U, .b = 200U, .a = 180U}
- static constexpr `eng::Color utl::Config::Color::GREEN_READY` = {0U, 255U, 100U, 255U}
- constexpr auto `utl::Path::Icons::ICON_APP` = "assets/icons/icon.png"
- constexpr auto `utl::Path::Audio::AUDIO_TITLE` = "assets/audio/title.mp3"
- constexpr auto `utl::Path::Audio::AUDIO_COIN` = "assets/audio/coin.mp3"
- constexpr auto `utl::Path::Audio::AUDIO_BATTLE_THEME` = "assets/audio/battle_theme.mp3"
- constexpr auto `utl::Path::Audio::AUDIO_INPUT` = "assets/audio/input.wav"
- constexpr auto `utl::Path::Audio::AUDIO_DEATH_ALLIES` = "assets/audio/deathallies.wav"

- `constexpr auto utl::Path::Audio::AUDIO_DEATH_ENEMIES = "assets/audio/deathennemies.↵wav"`
- `constexpr auto utl::Path::Audio::AUDIO_SUPERCHARGED_SHOT = "assets/audio/shoot.wav"`
- `constexpr auto utl::Path::Audio::AUDIO_BEGIN = "assets/audio/begin.wav"`
- `constexpr auto utl::Path::Audio::AUDIO_GOOFY = "assets/audio/goofy.wav"`
- `constexpr auto utl::Path::Audio::AUDIO_BOSS = "assets/audio/Bossmusic.mp3"`
- `constexpr auto utl::Path::Font::FONTS_RTYPE = "assets/fonts/r-type.otf"`
- `auto utl::Path::Plugin::PLUGIN_AUDIO_SFML`
- `auto utl::Path::Plugin::PLUGIN_NETWORK_ASIO_CLIENT`
- `auto utl::Path::Plugin::PLUGIN_RENDERER_SFML`
- `auto utl::Path::Plugin::PLUGIN_GAME_SOLO`
- `auto utl::Path::Plugin::PLUGIN_GAME_MULTI`
- `constexpr auto utl::Path::Texture::TEXTURE_PLAYER = "assets/sprites/r-typesheet42.gif"`
- `constexpr auto utl::Path::Texture::TEXTURE_SHOOT = "assets/sprites/shoot.gif"`
- `constexpr auto utl::Path::Texture::TEXTURE_SHOOT_CHARGED = "assets/sprites/shootcharged.↵gif"`
- `constexpr auto utl::Path::Texture::TEXTURE_SHOOT_LOADING = "assets/sprites/shootchargedloading.↵gif"`
- `constexpr auto utl::Path::Texture::TEXTURE_ENEMY_EASY = "assets/sprites/r-typesheet5.↵gif"`
- `constexpr auto utl::Path::Texture::TEXTURE_ENEMY_BASIC = "assets/sprites/r-typesheet5.↵gif"`
- `constexpr auto utl::Path::Texture::TEXTURE_ENEMY_ADVANCED = "assets/sprites/r-typesheet3.gif"`
- `constexpr auto utl::Path::Texture::TEXTURE_BOSS = "assets/sprites/r-typesheet14.gif"`
- `constexpr auto utl::Path::Texture::TEXTURE_ENEMY_PROJECTILE = "assets/sprites/r-typesheet20.gif"`
- `constexpr auto utl::Path::Texture::TEXTURE_EXPLOSION = "assets/sprites/r-typesheet44.gif"`
- `constexpr auto utl::Path::Texture::TEXTURE_STAGE1_FLOOR = "assets/sprites/floorstage1.↵png"`
- `constexpr auto utl::Path::Texture::TEXTURE_STAGE1_CEILING = "assets/sprites/ceillingstage1.↵png"`
- `constexpr auto utl::Path::Texture::TEXTURE_SCORE_COUNTER_BG = "assets/sprites/counter.↵png"`
- `constexpr auto utl::Path::Texture::TEXTURE_SCORE_DIGIT_0 = "assets/sprites/counter0.↵png"`
- `constexpr auto utl::Path::Texture::TEXTURE_SCORE_DIGIT_1 = "assets/sprites/counter1.↵png"`
- `constexpr auto utl::Path::Texture::TEXTURE_SCORE_DIGIT_2 = "assets/sprites/counter2.↵png"`
- `constexpr auto utl::Path::Texture::TEXTURE_SCORE_DIGIT_3 = "assets/sprites/counter3.↵png"`
- `constexpr auto utl::Path::Texture::TEXTURE_SCORE_DIGIT_4 = "assets/sprites/counter4.↵png"`
- `constexpr auto utl::Path::Texture::TEXTURE_SCORE_DIGIT_5 = "assets/sprites/counter5.↵png"`
- `constexpr auto utl::Path::Texture::TEXTURE_SCORE_DIGIT_6 = "assets/sprites/counter6.↵png"`
- `constexpr auto utl::Path::Texture::TEXTURE_SCORE_DIGIT_7 = "assets/sprites/counter7.↵png"`
- `constexpr auto utl::Path::Texture::TEXTURE_SCORE_DIGIT_8 = "assets/sprites/counter8.↵png"`
- `constexpr auto utl::Path::Texture::TEXTURE_SCORE_DIGIT_9 = "assets/sprites/counter9.↵png"`
- `constexpr auto utl::Path::Texture::ICON_PATH = "assets/icons/icon.png"`

19.277.1 Detailed Description

This file contains common definitions and constants.

Definition in file [Common.hpp](#).

19.278 Common.hpp

[Go to the documentation of this file.](#)

```

00001 ///  

00002 ///  

00003 ///  

00004 ///  

00005 ///  

00006 ///  

00007 #pragma once  

00008 ///  

00009 #include <filesystem>  

00010 ///  

00011 #include "Interfaces/IRenderer.hpp"  

00012 ///  

00013 #ifndef _WIN32  

00014 #define NOMINMAX  

00015 #define PLUGINS_EXTENSION ".dll"  

00016 #elif __linux__  

00017 #define PLUGINS_EXTENSION ".so"  

00018 #elif __APPLE__  

00019 #define PLUGINS_EXTENSION ".dylib"  

00020 #endif  

00021 ///  

00022 namespace utl  

00023 {  

00024     namespace Config  

00025     {  

00026         namespace Player  

00027         {  

00028             inline constexpr auto DEFAULT_PLAYER_NAME = "player";  

00029         } // namespace Player  

00030         namespace Audio  

00031         {  

00032             inline constexpr auto DEFAULT_AUDIO_VOLUME = 0.5F; // unused  

00033             inline constexpr auto DEFAULT_AUDIO_MUTED = false; // unused  

00034         } // namespace Audio  

00035         namespace Network  

00036         {  

00037             inline constexpr auto DEFAULT_NETWORK_HOST = "127.0.0.1";  

00038             inline constexpr auto DEFAULT_NETWORK_PORT = 2560;  

00039         } // namespace Network  

00040         namespace Window  

00041         {  

00042             inline constexpr auto DEFAULT_WINDOW_WIDTH = 1920;  

00043             inline constexpr auto DEFAULT_WINDOW_HEIGHT = 1080;  

00044             inline constexpr auto DEFAULT_WINDOW_FRAME_LIMIT = 240;  

00045             inline constexpr auto DEFAULT_WINDOW_FULLSCREEN = false;  

00046         } // namespace Window  

00047         namespace Game  

00048         {  

00049             inline constexpr auto DEFAULT_VIDEO_QUALITY = 1; // 0=Low, 1=Medium, 2=High  

00050             inline constexpr auto DEFAULT_CONTROL_SCHEME = 2; // 0=WASD, 1=ZQSD, 2=Arrows  

00051             inline constexpr auto DEFAULT_SKIN_INDEX = 0; // 0-4 different ship skins  

00052         } // namespace Game  

00053         namespace Color  

00054         {  

00055             static constexpr eng::Color DARK = {.r = 0U, .g = 0U, .b = 0U, .a = 255U};  

00056             static constexpr eng::Color CYAN_ELECTRIC = {.r = 0U, .g = 191U, .b = 255U, .a = 255U};  

00057             static constexpr eng::Color GRAY_BLUE_SUBTLE = {.r = 160U, .g = 160U, .b = 160U, .a = 255U};  

00058             static constexpr eng::Color STAR_BG = {.r = 100U, .g = 100U, .b = 150U, .a = 80U};  

00059             static constexpr eng::Color STAR_MID = {.r = 150U, .g = 150U, .b = 200U, .a = 120U};  

00060             static constexpr eng::Color YELLOW = {.r = 255U, .g = 255U, .b = 200U, .a = 200U};  

00061             static constexpr eng::Color CYAN_ELECTRIC_PARTICLES = {.r = 0U, .g = 191U, .b = 255U, .a = 100U};  

00062             static constexpr eng::Color CYAN_ELECTRIC_FOREGROUND = {.r = 0U, .g = 191U, .b = 255U, .a = 180U};  

00063             static constexpr eng::Color INFO_TEXT_COLOR = {.r = 180U, .g = 180U, .b = 180U, .a = 200U};  

00064             static constexpr eng::Color WHITE = {.r = 255U, .g = 255U, .b = 255U, .a = 255U};  

00065             static constexpr eng::Color WHITE_LOW = {.r = 255U, .g = 255U, .b = 255U, .a = 0U};  

00066             static constexpr eng::Color TEXT_VALUE_COLOR = {.r = 200U, .g = 200U, .b = 255U, .a = 255U};  

00067             static constexpr eng::Color WHITE_TRANS = {.r = 255U, .g = 255U, .b = 255U, .a = 100U};  

00068             static constexpr eng::Color BLUE = {.r = 200U, .g = 200U, .b = 255U, .a = 150U};  

00069             static constexpr eng::Color BLUE_SECOND = {.r = 50U, .g = 100U, .b = 200U, .a = 60U};  

00070             static constexpr eng::Color PURPLE = {.r = 100U, .g = 50U, .b = 150U, .a = 80U};  

00071             static constexpr eng::Color GREEN = {.r = 200U, .g = 255U, .b = 200U, .a = 180U};  

00072             static constexpr eng::Color GREEN_READY = {0U, 255U, 100U, 255U};  

00073         }  

00074     } // namespace Color  

00075 } // namespace Config  

00076 namespace Path

```

```

00077 {
00078     namespace Icons
00079     {
00080         inline constexpr auto ICON_APP = "assets/icons/icon.png";
00081     } // namespace Icons
00082     namespace Audio
00083     {
00084         inline constexpr auto AUDIO_TITLE = "assets/audio/title.mp3";
00085         inline constexpr auto AUDIO_COIN = "assets/audio/coin.mp3";
00086         inline constexpr auto AUDIO_BATTLE_THEME = "assets/audio/battle_theme.mp3";
00087         inline constexpr auto AUDIO_INPUT = "assets/audio/input.wav";
00088         inline constexpr auto AUDIO_DEATH_ALLIES = "assets/audio/deathallies.wav";
00089         inline constexpr auto AUDIO_DEATH_ENEMIES = "assets/audio/deathennemies.wav";
00090         inline constexpr auto AUDIO_SUPERCHARGED_SHOT = "assets/audio/shoot.wav";
00091         inline constexpr auto AUDIO_BEGIN = "assets/audio/begin.wav";
00092         inline constexpr auto AUDIO_GOOFY = "assets/audio/goofy.wav";
00093         inline constexpr auto AUDIO_BOSS = "assets/audio/Bossmusic.mp3";
00094     } // namespace Audio
00095     namespace Font
00096     {
00097         inline constexpr auto FONTS_RTYPE = "assets/fonts/r-type.otf";
00098     } // namespace Font
00099     namespace Plugin
00100     { // TODO(bobis33): dont use filesystem here
00101         inline auto PLUGIN_AUDIO_SFML =
00102             std::filesystem::path(PLUGINS_DIR) / ("audio_sfml" + std::string(PLUGINS_EXTENSION));
00103         inline auto PLUGIN_NETWORK_ASIO_CLIENT =
00104             std::filesystem::path(PLUGINS_DIR) / ("network_asio_client" + std::string(PLUGINS_EXTENSION));
00105         inline auto PLUGIN_RENDERER_SFML =
00106             std::filesystem::path(PLUGINS_DIR) / ("renderer_sfml" + std::string(PLUGINS_EXTENSION));
00107         inline auto PLUGIN_GAME_SOLO =
00108             std::filesystem::path(PLUGINS_DIR) / ("game_rtype_client_solo" + std::string(PLUGINS_EXTENSION));
00109         inline auto PLUGIN_GAME_MULTI =
00110             std::filesystem::path(PLUGINS_DIR) / ("game_rtype_client_multi" + std::string(PLUGINS_EXTENSION));
00111     } // namespace Plugin
00112     namespace Texture
00113     {
00114         inline constexpr auto TEXTURE_PLAYER = "assets/sprites/r-typesheet42.gif";
00115         inline constexpr auto TEXTURE_SHOOT = "assets/sprites/shoot.gif";
00116         inline constexpr auto TEXTURE_SHOOT_CHARGED = "assets/sprites/shootcharged.gif";
00117         inline constexpr auto TEXTURE_SHOOT_LOADING = "assets/sprites/shootchargedloading.gif";
00118         inline constexpr auto TEXTURE_ENEMY_EASY = "assets/sprites/r-typesheet5.gif";
00119         inline constexpr auto TEXTURE_ENEMY_BASIC = "assets/sprites/r-typesheet5.gif";
00120         inline constexpr auto TEXTURE_ENEMY_ADVANCED = "assets/sprites/r-typesheet3.gif";
00121         inline constexpr auto TEXTURE_BOSS = "assets/sprites/r-typesheet14.gif";
00122         inline constexpr auto TEXTURE_ENEMY_PROJECTILE = "assets/sprites/r-typesheet20.gif";
00123         inline constexpr auto TEXTURE_EXPLOSION = "assets/sprites/r-typesheet44.gif";
00124         inline constexpr auto TEXTURE_STAGE1_FLOOR = "assets/sprites/floorstage1.png";
00125         inline constexpr auto TEXTURE_STAGE1_CEILING = "assets/sprites/ceillingstage1.png";
00126         // HUD score
00127         inline constexpr auto TEXTURE_SCORE_COUNTER_BG = "assets/sprites/counter.png";
00128         inline constexpr auto TEXTURE_SCORE_DIGIT_0 = "assets/sprites/counter0.png";
00129         inline constexpr auto TEXTURE_SCORE_DIGIT_1 = "assets/sprites/counter1.png";
00130         inline constexpr auto TEXTURE_SCORE_DIGIT_2 = "assets/sprites/counter2.png";
00131         inline constexpr auto TEXTURE_SCORE_DIGIT_3 = "assets/sprites/counter3.png";
00132         inline constexpr auto TEXTURE_SCORE_DIGIT_4 = "assets/sprites/counter4.png";
00133         inline constexpr auto TEXTURE_SCORE_DIGIT_5 = "assets/sprites/counter5.png";
00134         inline constexpr auto TEXTURE_SCORE_DIGIT_6 = "assets/sprites/counter6.png";
00135         inline constexpr auto TEXTURE_SCORE_DIGIT_7 = "assets/sprites/counter7.png";
00136         inline constexpr auto TEXTURE_SCORE_DIGIT_8 = "assets/sprites/counter8.png";
00137         inline constexpr auto TEXTURE_SCORE_DIGIT_9 = "assets/sprites/counter9.png";
00138
00139         inline constexpr auto ICON_PATH = "assets/icons/icon.png";
00140     } // namespace Texture
00141 } // namespace Path
00142 } // namespace utl

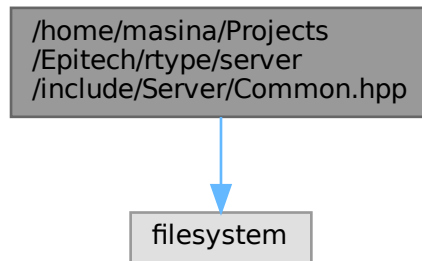
```

19.279 /home/masina/Projects/Epitech/rtype/server/include/Server/Common.hpp File Reference

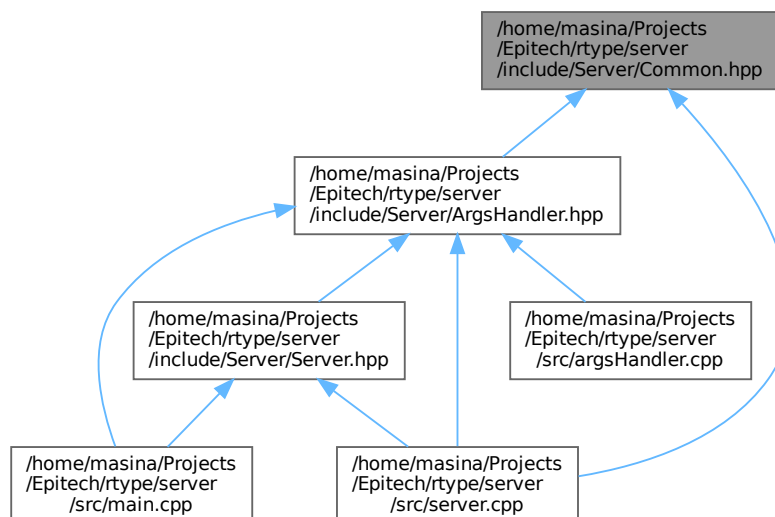
This file contains common definitions and constants.

```
#include <filesystem>
```

Include dependency graph for Common.hpp:



This graph shows which files directly or indirectly include this file:



Namespaces

- namespace `srv`
- namespace `srv::Config`
- namespace `srv::Config::Network`
- namespace `srv::Game`
- namespace `srv::Path`
- namespace `srv::Path::Plugin`

Variables

- constexpr auto `srv::Config::Network::DEFAULT_NETWORK_HOST` = "0.0.0.0"
- constexpr auto `srv::Config::Network::DEFAULT_NETWORK_PORT` = 2560
- constexpr auto `srv::Config::Network::DEFAULT_MAX_CLIENT` = 4

- constexpr auto `srv::Game::DEFAULT_TICK_RATE` = 60
- constexpr auto `srv::Game::DEFAULT_UPDATE_INTERVAL` = 1 / 20.F
- auto `srv::Path::Plugin::PLUGINS_NETWORK_ASIO_SERVER`
- auto `srv::Path::Plugin::PLUGINS_GAME_RTYPE_SERVER`

19.279.1 Detailed Description

This file contains common definitions and constants.
Definition in file [Common.hpp](#).

19.280 Common.hpp

[Go to the documentation of this file.](#)

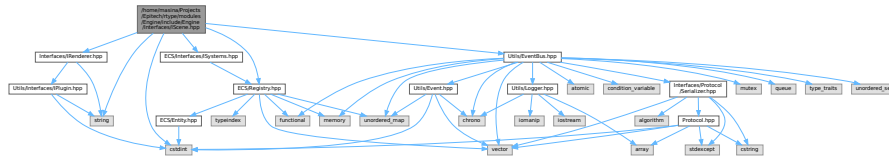
```
00001 ///  
00002 ///  
00003 ///  
00004 ///  
00005 ///  
00006 ///  
00007 #pragma once  
00008  
00009 #include <filesystem>  
00010  
00011 #ifdef _WIN32  
00012 #define PLUGINS_EXTENSION ".dll"  
00013 #elif __linux__  
00014 #define PLUGINS_EXTENSION ".so"  
00015 #elif __APPLE__  
00016 #define PLUGINS_EXTENSION ".dylib"  
00017 #endif  
00018  
00019 namespace srv  
00020 {  
00021     namespace Config::Network  
00022     {  
00023         inline constexpr auto DEFAULT_NETWORK_HOST = "0.0.0.0";  
00024         inline constexpr auto DEFAULT_NETWORK_PORT = 2560;  
00025         inline constexpr auto DEFAULT_MAX_CLIENT = 4;  
00026     } // namespace Config::Network  
00027     namespace Game  
00028     {  
00029         inline constexpr auto DEFAULT_TICK_RATE = 60;  
00030         inline constexpr auto DEFAULT_UPDATE_INTERVAL = 1 / 20.F;  
00031     } // namespace Game  
00032     namespace Path::Plugin  
00033     {  
00034         inline auto PLUGINS_NETWORK_ASIO_SERVER =  
00035             std::filesystem::path(PLUGINS_DIR) / ("network_asio_server" + std::string(PLUGINS_EXTENSION));  
00036         inline auto PLUGINS_GAME_RTYPE_SERVER =  
00037             std::filesystem::path(PLUGINS_DIR) / ("game_rtype_server" + std::string(PLUGINS_EXTENSION));  
00038     }  
00039 } // namespace Path::Plugin  
00040 } // namespace srv
```

19.281 /home/masina/Projects/Epitech/rtype/modules/↵ Engine/include/Engine/Interfaces/IScene.hpp File Reference

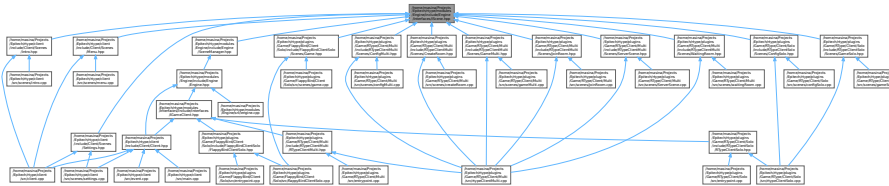
This file contains the IScene class.

```
#include <stdint>  
#include <string>  
#include "ECS/Interfaces/ISystems.hpp"  
#include "ECS/Registry.hpp"  
#include "Interfaces/IRenderer.hpp"  
#include "Utils/EventBus.hpp"
```

Include dependency graph for IScene.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- interface [eng::IScene](#)
interface class for scene
- class [eng::AScene](#)
Abstract class for scene.

Namespaces

- namespace [eng](#)

Typedefs

- using [eng::id](#) = unsigned int

19.281.1 Detailed Description

This file contains the IScene class.

Definition in file [IScene.hpp](#).

19.282 IScene.hpp

[Go to the documentation of this file.](#)

```
00001 ///
00002 /// @file IScene.hpp
00003 /// @brief This file contains the IScene class
00004 /// @namespace eng
00005 ///
00006
00007 #pragma once
00008
00009 #include <cstdint>
00010 #include <string>
00011
00012 #include "ECS/Interfaces/ISystems.hpp"
00013 #include "ECS/Registry.hpp"
00014 #include "Interfaces/IRenderer.hpp"
00015 #include "Utils/EventBus.hpp"
00016
00017 namespace eng
00018 {
00019
00020     using id = unsigned int;
00021
00022     ///
```

```

00023  /// @interface IScene
00024  /// @brief interface class for scene
00025  /// @namespace eng
00026  ///
00027  class IScene
00028  {
00029  public:
00030      virtual ~IScene() = default;
00031
00032      [[nodiscard]] virtual std::string &getName() = 0;
00033      [[nodiscard]] virtual id getId() const = 0;
00034      [[nodiscard]] virtual ecs::Registry &getRegistry() = 0;
00035
00036      virtual void setName(const std::string &newName) = 0;
00037
00038      virtual void update(float dt, const WindowSize &size) = 0;
00039      virtual void event(const Event &event) = 0;
00040
00041      virtual void addSystem(std::unique_ptr<ecs::ISystem> system) = 0;
00042      virtual void updateSystems(float dt) = 0;
00043
00044 }; // class IScene
00045
00046  ///
00047  /// @class AScene
00048  /// @brief Abstract class for scene
00049  /// @namespace eng
00050  ///
00051  class AScene : public IScene
00052  {
00053  public:
00054      explicit AScene(const id assignedId) : m_id(assignedId) {}
00055      ~AScene() override = default;
00056
00057      AScene(const AScene &other) = delete;
00058      AScene(AScene &&other) = delete;
00059      AScene &operator=(const AScene &other) = delete;
00060      AScene &operator=(AScene &&other) = delete;
00061
00062      [[nodiscard]] std::string &getName() override { return m_name; }
00063      [[nodiscard]] id getId() const override { return m_id; }
00064      [[nodiscard]] ecs::Registry &getRegistry() override { return m_registry; }
00065
00066      void setName(const std::string &newName) override { m_name = newName; }
00067
00068      void addSystem(std::unique_ptr<ecs::ISystem> system) override { m_systems.emplace_back(std::move(system)); }
00069
00070      void updateSystems(const float dt) override
00071      {
00072          for (const auto &system : m_systems)
00073          {
00074              system->update(m_registry, dt);
00075          }
00076      }
00077  private:
00078      std::string m_name = "default_name";
00079      id m_id = 1;
00080      ecs::Registry m_registry;
00081      std::vector<std::unique_ptr<ecs::ISystem>> m_systems;
00082
00083  protected:
00084      utl::EventBus &m_eventBus = utl::EventBus::getInstance();
00085      std::uint32_t m_eventComponentId = 0;
00086  }; // class AScene
00087 } // namespace eng

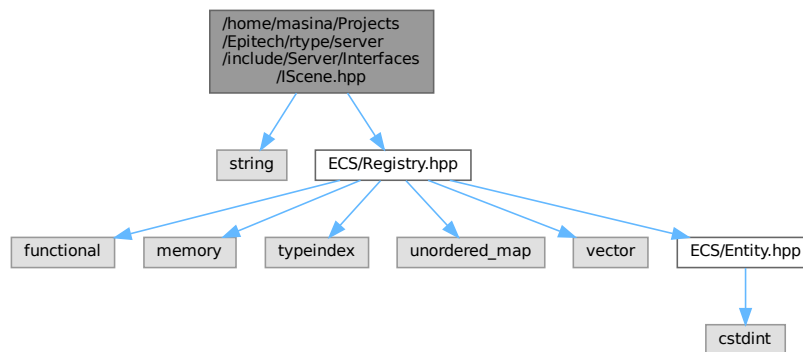
```

19.283 /home/masina/Projects/Epitech/rtype/server/include/Server/Interfaces/IScene.hpp File Reference

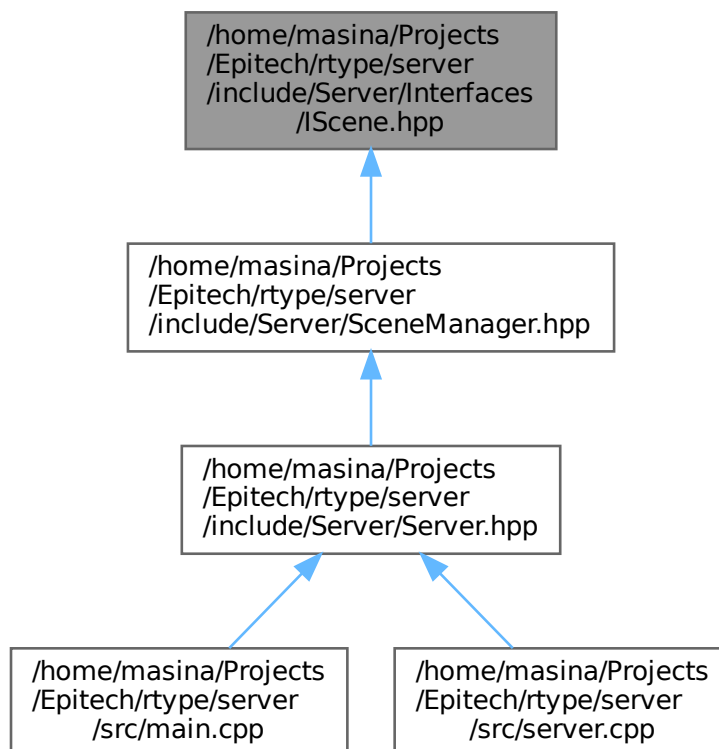
This file contains the IScene class.

```
#include <string>
#include "ECS/Registry.hpp"
```

Include dependency graph for IScene.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [srv::IScene](#)
interface class for scene
- class [srv::AScene](#)
Class for scene.

Namespaces

- namespace [srv](#)

Typedefs

- using [srv::id](#) = unsigned int

19.283.1 Detailed Description

This file contains the IScene class.

Definition in file [IScene.hpp](#).

19.284 IScene.hpp

[Go to the documentation of this file.](#)

```

00001 ///  

00002 ///  

00003 ///  

00004 ///  

00005 ///  

00006 ///  

00007 #pragma once  

00008  

00009 #include <string>  

00010  

00011 #include "ECS/Registry.hpp"  

00012  

00013 namespace srv  

00014 {  

00015  

00016     using id = unsigned int;  

00017  

00018     ///  

00019     ///  

00020     ///  

00021     ///  

00022     ///  

00023     class IScene  

00024     {  

00025     public:  

00026         virtual ~IScene() = default;  

00027  

00028         [[nodiscard]] virtual std::string &getName() = 0;  

00029         [[nodiscard]] virtual id getId() const = 0;  

00030         [[nodiscard]] virtual ecs::Registry &getRegistry() = 0;  

00031  

00032         virtual void setName(const std::string &newName) = 0;  

00033  

00034         virtual void update(float dt) = 0;  

00035     }; // class IScene  

00036  

00037     ///  

00038     ///  

00039     ///  

00040     ///  

00041     ///  

00042     class AScene : public IScene  

00043     {  

00044     public:  

00045         AScene() : m_id(s_nextId++) {}  

00046         ~AScene() override = default;  

00047  

00048         AScene(const AScene &other) = delete;  

00049         AScene(AScene &&other) = delete;  

00050         AScene &operator=(const AScene &other) = delete;  

00051         AScene &operator=(AScene &&other) = delete;  

00052  

00053         [[nodiscard]] std::string &getName() override { return m_name; }  

00054         [[nodiscard]] id getId() const override { return m_id; }  

00055         [[nodiscard]] ecs::Registry &getRegistry() override { return m_registry; }  

00056  

00057         void setName(const std::string &newName) override { m_name = newName; }  

00058  

00059     private:  

00060         std::string m_name = "default_name";  

00061         id m_id = 1;  

00062         ecs::Registry m_registry;  

00063         inline static id s_nextId = 1;  

00064     }; // class AScene

```

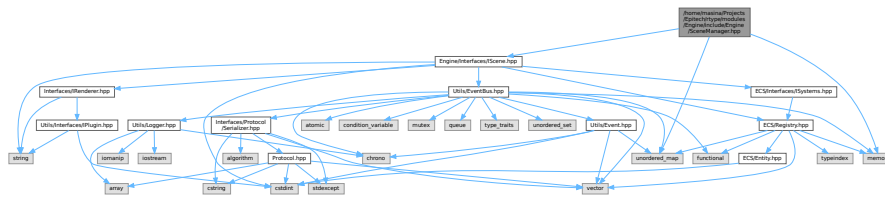


```
00065  
00066 } // namespace srv
```

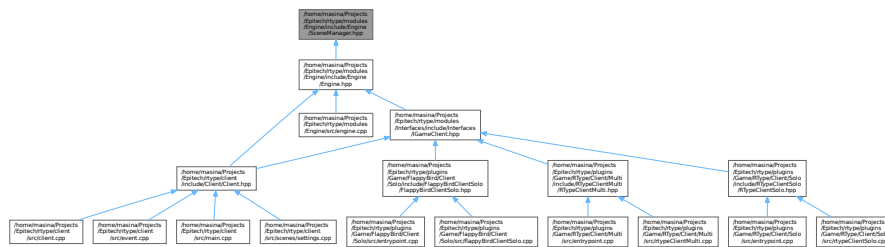
19.285 /home/masina/Projects/Epitech/rtype/modules/↔ Engine/include/Engine/SceneManager.hpp File Reference

This file contains the SceneManager class declaration.

```
#include <memory>  
#include <unordered_map>  
#include "Engine/Interfaces/IScene.hpp"  
Include dependency graph for SceneManager.hpp:
```



This graph shows which files directly or indirectly include this file:



Classes

- class [eng::SceneManager](#)
Class for managing scenes.

Namespaces

- namespace [eng](#)

19.285.1 Detailed Description

This file contains the SceneManager class declaration.

Definition in file [SceneManager.hpp](#).

19.286 SceneManager.hpp

[Go to the documentation of this file.](#)

```
00001 ///  
00002 ///  
00003 ///  
00004 ///  
00005 ///  
00006 ///  
00007 #pragma once  
00008  
00009 #include <memory>  
00010 #include <unordered_map>
```

```

00011
00012 #include "Engine/Interfaces/IScene.hpp"
00013
00014 namespace eng
00015 {
00016
00017     ///
00018     /// @class SceneManager
00019     /// @brief Class for managing scenes
00020     /// @namespace eng
00021     ///
00022     class SceneManager
00023     {
00024
00025     public:
00026         SceneManager() = default;
00027         ~SceneManager() = default;
00028
00029         SceneManager(const SceneManager &) = delete;
00030         SceneManager &operator=(const SceneManager &) = delete;
00031         SceneManager(SceneManager &&) = delete;
00032         SceneManager &operator=(SceneManager &&) = delete;
00033
00034         std::unique_ptr<IScene> &getScene(const id sceneId) { return m_scenes.at(sceneId); }
00035         std::unique_ptr<IScene> &getCurrentScene() { return m_scenes.at(m_currentSceneId); }
00036         void switchToScene(const id sceneId) { m_currentSceneId = sceneId; }
00037         void addScene(std::unique_ptr<IScene> scene) { m_scenes[scene->getId()] = std::move(scene); }
00038         id generateNextId() { return m_nextId++; }
00039
00040     private:
00041         std::unordered_map<id, std::unique_ptr<IScene>» m_scenes;
00042         id m_currentSceneId = 1;
00043         id m_nextId = 1;
00044     }; // class SceneManager
00045 } // namespace eng

```

19.287 /home/masina/Projects/Epitech/rtype/server/include/Server/SceneManager.hpp File Reference

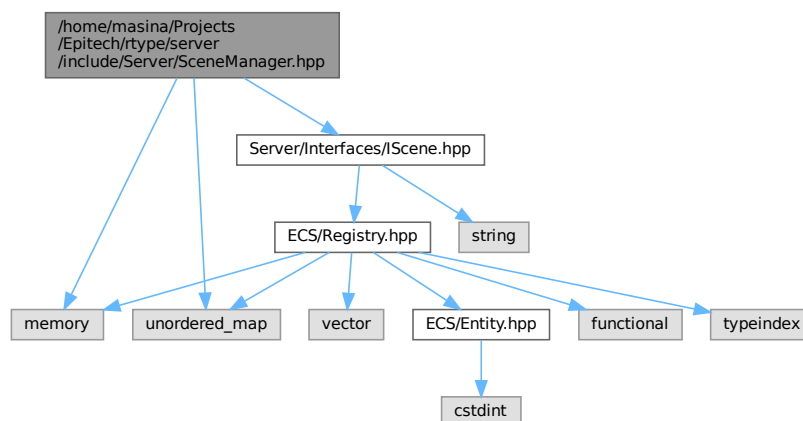
This file contains the SceneManager class declaration.

```
#include <memory>
```

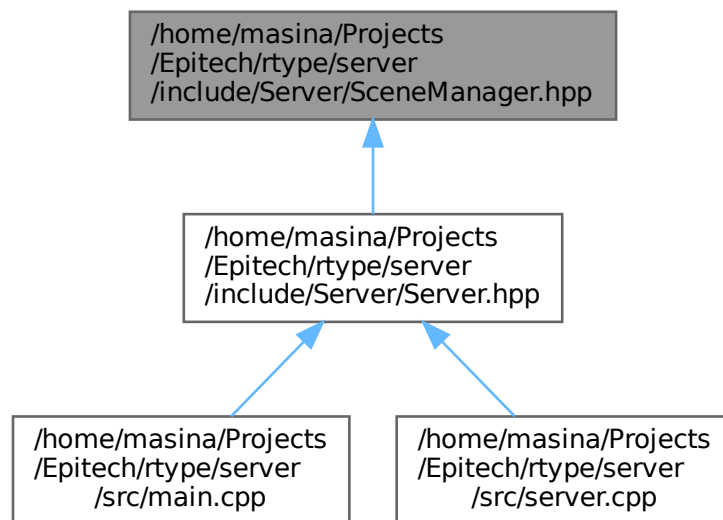
```
#include <unordered_map>
```

```
#include "Server/Interfaces/IScene.hpp"
```

Include dependency graph for SceneManager.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [srv::SceneManager](#)
Class for managing scenes.

Namespaces

- namespace [srv](#)

19.287.1 Detailed Description

This file contains the SceneManager class declaration.
Definition in file [SceneManager.hpp](#).

19.288 SceneManager.hpp

[Go to the documentation of this file.](#)

```

00001 ///  

00002 ///  

00003 ///  

00004 ///  

00005 ///  

00006 ///  

00007 #pragma once  

00008  

00009 #include <memory>  

00010 #include <unordered_map>  

00011  

00012 #include "Server/Interfaces/IScene.hpp"  

00013  

00014 namespace srv  

00015 {  

00016  

00017     ///  

00018     ///  

00019     ///  

00020     ///  

00021     ///  

00022     class SceneManager
  
```

```

00023 {
00024
00025     public:
00026         SceneManager() = default;
00027         ~SceneManager() = default;
00028
00029         SceneManager(const SceneManager &) = delete;
00030         SceneManager &operator=(const SceneManager &) = delete;
00031         SceneManager(SceneManager &&) = delete;
00032         SceneManager &operator=(SceneManager &&) = delete;
00033
00034         std::unique_ptr<IScene> &getScene(const id sceneId) { return m_scenes.at(sceneId); }
00035         std::unique_ptr<IScene> &getCurrentScene() { return m_scenes.at(m_currentSceneId); }
00036         void switchToScene(const id sceneId) { m_currentSceneId = sceneId; }
00037         void addScene(std::unique_ptr<IScene> scene) { m_scenes[scene->getId()] = std::move(scene); }
00038
00039         // template <typename... EntityDefs>
00040         // IScene &createScene(const std::string &name, const std::function<void(const Event&>> eventHandler, const
00041         // std::function<void(float)> updateHandler,
00042         // EntityDefs&&... defs) {
00043         //     auto scene = std::make_unique<IScene>();
00044         //     scene->setName(name);
00045         //     scene->setEventHandler(eventHandler);
00046         //     scene->setUpdateHandler(updateHandler);
00047         //
00048         //     Scene &ref = *scene;
00049         //     (defs(ref.getRegistry()), ...);
00050         //
00051         //     m_scenes[ref.getId()] = std::move(scene);
00052         //     return ref;
00053         // }
00054
00055     private:
00056         std::unordered_map<id, std::unique_ptr<IScene>» m_scenes;
00057         id m_currentSceneId = 1;
00058     }; // class SceneManager
00059 } // namespace srv

```

19.289 /home/masina/Projects/Epitech/rtype/server/include/Server/↵ Scenes/Level_0.hpp File Reference

This file contains the level_0 definitions.

Namespaces

- namespace `srv`

19.289.1 Detailed Description

This file contains the level_0 definitions.

Definition in file [Level_0.hpp](#).

19.290 Level_0.hpp

[Go to the documentation of this file.](#)

```

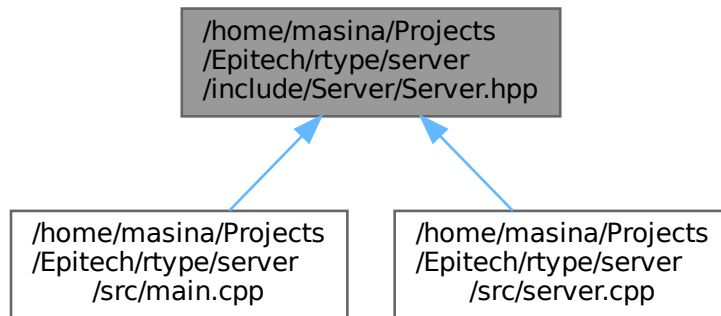
00001 ///
00002 /// @file Level_0.hpp
00003 /// @brief This file contains the level_0 definitions
00004 /// @namespace srv
00005 ///
00006
00007 #pragma once
00008
00009 namespace srv
00010 {
00011
00012 } // namespace srv

```

19.291 /home/masina/Projects/Epitech/rtype/server/include/Server/↵ Scenes/Level_1.hpp File Reference

This file contains the level_0 definitions.

This graph shows which files directly or indirectly include this file:



Classes

- class [srv::Server](#)
Class for the server.

Namespaces

- namespace [srv](#)

19.293.1 Detailed Description

This file contains the Server class declaration.

Definition in file [Server.hpp](#).

19.294 Server.hpp

[Go to the documentation of this file.](#)

```

00001 ///  

00002 ///  

00003 ///  

00004 ///  

00005 ///  

00006 ///  

00007 #pragma once  

00008  

00009 #include <memory>  

00010  

00011 #include "Interfaces/IGameServer.hpp"  

00012 #include "Interfaces/INetworkServer.hpp"  

00013 #include "Server/ArgsHandler.hpp"  

00014 #include "Server/SceneManager.hpp"  

00015 #include "Utils/AppConfigServer.hpp"  

00016 #include "Utils/Clock.hpp"  

00017 #include "Utils/PluginLoader.hpp"  

00018  

00019 namespace srv  

00020 {  

00021  

00022     ///  

00023     ///  

00024     ///  

00025     ///  

00026     ///  

00027     class Server  

00028     {  

00029  

00030     public:  

00031         explicit Server(const ArgsConfig &config);  

00032         ~Server() = default;  


```

```

00033
00034     Server(const Server &) = delete;
00035     Server &operator=(const Server &) = delete;
00036     Server(Server &&) = delete;
00037     Server &operator=(Server &&) = delete;
00038
00039     void run() const;
00040
00041 private:
00042     [[nodiscard]] static utl::srv::AppConfig setupConfig(const ArgsConfig &cfg);
00043
00044     utl::srv::AppConfig m_config;
00045
00046     std::unique_ptr<utl::PluginLoader> m_pluginLoader;
00047     std::unique_ptr<utl::Clock> m_clock;
00048     std::unique_ptr<SceneManager> m_sceneManager;
00049
00050     std::shared_ptr<INetworkServer> m_network;
00051     std::shared_ptr<gme::IGameServer> m_game;
00052 }; // class Server
00053
00054 } // namespace srv

```

19.295 /home/masina/Projects/Epitech/rtype/modules/ECS/include/ECS/Systems/Systems.hpp File Reference

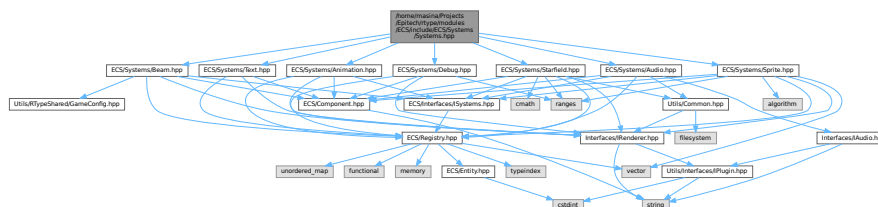
This file contains the system definitions.

```

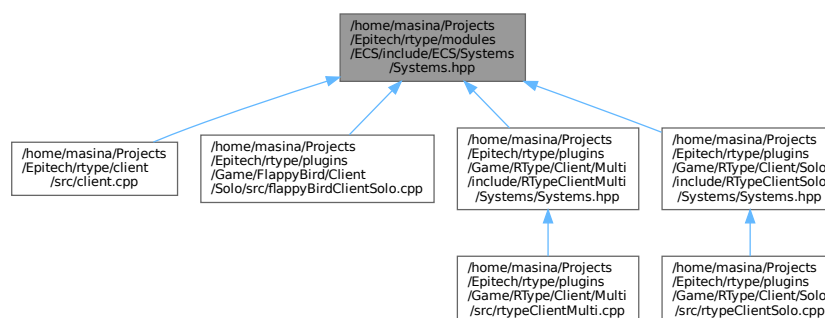
#include "ECS/Systems/Animation.hpp"
#include "ECS/Systems/Audio.hpp"
#include "ECS/Systems/Beam.hpp"
#include "ECS/Systems/Debug.hpp"
#include "ECS/Systems/Sprite.hpp"
#include "ECS/Systems/Starfield.hpp"
#include "ECS/Systems/Text.hpp"

```

Include dependency graph for Systems.hpp:



This graph shows which files directly or indirectly include this file:



Namespaces

- namespace [ecs](#)

19.295.1 Detailed Description

This file contains the system definitions.

This file contains the TextSystem definitions.

This file contains the systems definitions.

Definition in file [Systems.hpp](#).

19.296 Systems.hpp

[Go to the documentation of this file.](#)

```
00001 ///  
00002 ///  
00002 ///  
00002 @file Systems.hpp  
00003 ///  
00003 @brief This file contains the systems definitions  
00004 ///  
00004 @namespace ecs  
00005 ///  
00006 ///  
00007 #pragma once  
00008  
00009 #include "ECS/Systems/Animation.hpp"  
00010 #include "ECS/Systems/Audio.hpp"  
00011 #include "ECS/Systems/Beam.hpp"  
00012 #include "ECS/Systems/Debug.hpp"  
00013 #include "ECS/Systems/Sprite.hpp"  
00014 #include "ECS/Systems/Starfield.hpp"  
00015 #include "ECS/Systems/Text.hpp"
```

19.297 /home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Multi/include/RTypeClientMulti/Systems/Systems.hpp

File Reference

This file contains the system definitions.

```
#include "ECS/Systems/Systems.hpp"
```

```
#include "RTypeClientMulti/Systems/Collision.hpp"
```

```
#include "RTypeClientMulti/Systems>LoadingAnimation.hpp"
```

```
#include "RTypeClientMulti/Systems/PlayerControllerMulti.hpp"
```

```
#include "RTypeClientMulti/Systems/PlayerDirection.hpp"
```

```
#include "RTypeClientMulti/Systems/Projectile.hpp"
```

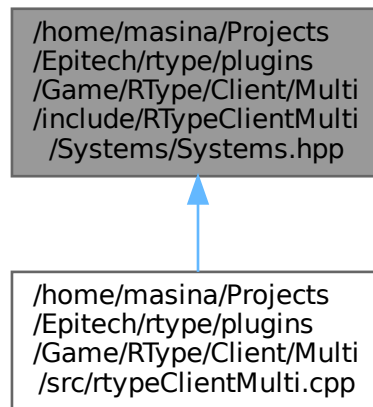
```
#include "RTypeClientMulti/Systems/Scrolling.hpp"
```

```
#include "RTypeClientMulti/Systems/Weapon.hpp"
```

Include dependency graph for Systems.hpp:



This graph shows which files directly or indirectly include this file:



Namespaces

- namespace [gme](#)

19.297.1 Detailed Description

This file contains the system definitions.

Definition in file [Systems.hpp](#).

19.298 Systems.hpp

[Go to the documentation of this file.](#)

```

00001 ///  

00002 ///  

00003 ///  

00004 ///  

00005 ///  

00006 ///  

00007 #pragma once  

00008  

00009 #include "ECS/Systems/Systems.hpp"  

00010 #include "RTypeClientMulti/Systems/Collision.hpp"  

00011 #include "RTypeClientMulti/Systems>LoadingAnimation.hpp"  

00012 #include "RTypeClientMulti/Systems/PlayerControllerMulti.hpp"  

00013 #include "RTypeClientMulti/Systems/PlayerDirection.hpp"  

00014 #include "RTypeClientMulti/Systems/Projectile.hpp"  

00015 #include "RTypeClientMulti/Systems/Scrolling.hpp"  

00016 #include "RTypeClientMulti/Systems/Weapon.hpp"
  
```

19.299 /home/masina/Projects/Epitech/rtype/plugins/Game/RType/Client/Solo/include/RTypeClientSolo/Systems/Systems.hpp File Reference

This file contains the system definitions.

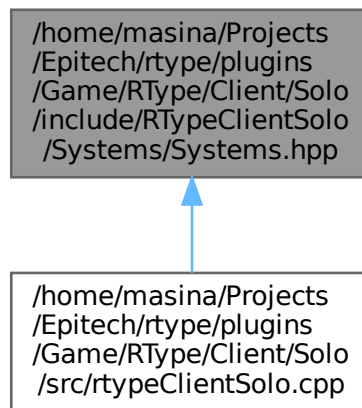
```

#include "ECS/Systems/Systems.hpp"
#include "RTypeClientSolo/Systems/Collision.hpp"
#include "RTypeClientSolo/Systems/Enemy.hpp"
#include "RTypeClientSolo/Systems/Explosion.hpp"
#include "RTypeClientSolo/Systems/HUD.hpp"
  
```

```
#include "RTypeClientSolo/Systems/LoadingAnimation.hpp"
#include "RTypeClientSolo/Systems/PlayerDirection.hpp"
#include "RTypeClientSolo/Systems/Projectile.hpp"
#include "RTypeClientSolo/Systems/Scrolling.hpp"
#include "RTypeClientSolo/Systems/Spawn.hpp"
#include "RTypeClientSolo/Systems/Weapon.hpp"
Include dependency graph for Systems.hpp:
```



This graph shows which files directly or indirectly include this file:



Namespaces

- namespace [gme](#)

19.299.1 Detailed Description

This file contains the system definitions.
Definition in file [Systems.hpp](#).

19.300 Systems.hpp

[Go to the documentation of this file.](#)

```
00001 ///  
00002 ///  
00003 ///  
00004 ///  
00005 ///  
00006 ///  
00007 #pragma once  
00008  
00009 #include "ECS/Systems/Systems.hpp"  
00010 #include "RTypeClientSolo/Systems/Collision.hpp"  
00011 #include "RTypeClientSolo/Systems/Enemy.hpp"  
00012 #include "RTypeClientSolo/Systems/Explosion.hpp"
```

```

00013 #include "RTypeClientSolo/Systems/HUD.hpp"
00014 #include "RTypeClientSolo/Systems/LoadingAnimation.hpp"
00015 #include "RTypeClientSolo/Systems/PlayerDirection.hpp"
00016 #include "RTypeClientSolo/Systems/Projectile.hpp"
00017 #include "RTypeClientSolo/Systems/Scrolling.hpp"
00018 #include "RTypeClientSolo/Systems/Spawn.hpp"
00019 #include "RTypeClientSolo/Systems/Weapon.hpp"

```

19.301 /home/masina/Projects/Epitech/rtype/server/include/Server/Systems/Systems.hpp File Reference

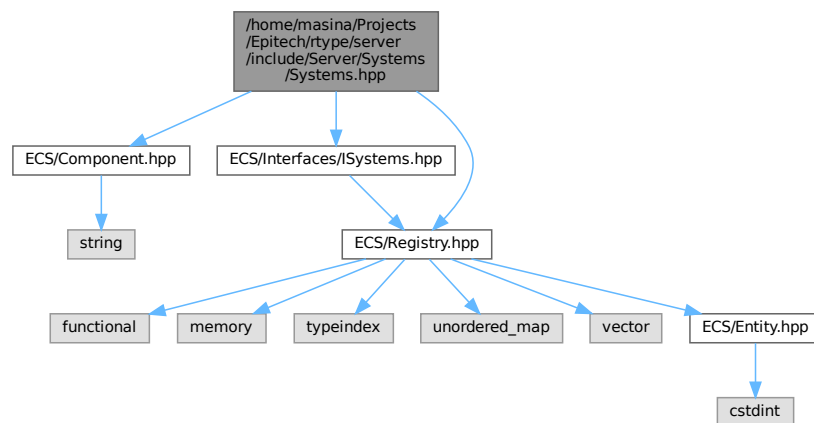
This file contains the system definitions.

```

#include "ECS/Component.hpp"
#include "ECS/Interfaces/ISystems.hpp"
#include "ECS/Registry.hpp"

```

Include dependency graph for Systems.hpp:



Namespaces

- namespace `srv`

19.301.1 Detailed Description

This file contains the system definitions.

Definition in file [Systems.hpp](#).

19.302 Systems.hpp

[Go to the documentation of this file.](#)

```

00001 ///
00002 /// @file Systems.hpp
00003 /// @brief This file contains the system definitions
00004 /// @namespace srv
00005 ///
00006
00007 #pragma once
00008
00009 #include "ECS/Component.hpp"
00010 #include "ECS/Interfaces/ISystems.hpp"
00011 #include "ECS/Registry.hpp"
00012
00013 namespace srv
00014 {
00015
00016 } // namespace srv

```



```
00051         std::cout << sessions[i];
00052         if (i < sessions.size() - 1)
00053         {
00054             std::cout << ", ";
00055         }
00056     }
00057     std::cout << '\n';
00058
00059     std::this_thread::sleep_for(std::chrono::milliseconds(16)); // ~60 FPS
00060 }
00061 }
00062
00063 utl::srv::AppConfig srv::Server::setupConfig(const ArgsConfig &cfg)
00064 {
00065     utl::srv::AppConfig config;
00066
00067     config.host = cfg.host;
00068     config.port = cfg.port;
00069
00070     return config;
00071 }
```


Index

[/home/masina/Projects/Epitech/rtype/CONTRIBUTING.md](#) 818

[/home/masina/Projects/Epitech/rtype/LICENSE.md](#) 818

[/home/masina/Projects/Epitech/rtype/README.md](#) 798

[/home/masina/Projects/Epitech/rtype/client/README.md](#) 798

[/home/masina/Projects/Epitech/rtype/client/include/Client/Assets/Project.h](#) 783, 784

[/home/masina/Projects/Epitech/rtype/client/include/Client/Client.h](#) 787, 788

[/home/masina/Projects/Epitech/rtype/client/include/Client/Client/Vec3.h](#) 789, 790

[/home/masina/Projects/Epitech/rtype/client/include/Client/Scenes/Project.h](#) 792, 793

[/home/masina/Projects/Epitech/rtype/client/include/Client/Scenes/Project/Project.h](#) 794, 795

[/home/masina/Projects/Epitech/rtype/client/include/Client/Scenes/Subjects/Epitech.h](#) 795, 796

[/home/masina/Projects/Epitech/rtype/client/src/argHandler.h](#) 798, 799

[/home/masina/Projects/Epitech/rtype/client/src/cli.h](#) 803, 804

[/home/masina/Projects/Epitech/rtype/client/src/event.h](#) 806

[/home/masina/Projects/Epitech/rtype/client/src/main.h](#) 807

[/home/masina/Projects/Epitech/rtype/client/src/scenes/line.h](#) 809, 810

[/home/masina/Projects/Epitech/rtype/client/src/scenes/line/masina.h](#) 811

[/home/masina/Projects/Epitech/rtype/client/src/scenes/line/masina/Project.h](#) 814

[/home/masina/Projects/Epitech/rtype/documentation/home/compas/masina/Projects/Epitech](#) 818

[/home/masina/Projects/Epitech/rtype/documentation/home/masina/Projects/Epitech/ontype](#) 818

[/home/masina/Projects/Epitech/rtype/modules/ECS/include/ECS/Project.h](#) 818, 820

[/home/masina/Projects/Epitech/rtype/modules/ECS/include/ECS/Project/Epitech.h](#) 822, 823

[/home/masina/Projects/Epitech/rtype/modules/ECS/include/ECS/Project/Epitech/Vec3.h](#) 823, 824

[/home/masina/Projects/Epitech/rtype/modules/ECS/include/ECS/Registry/Epitech](#) 824, 825

[/home/masina/Projects/Epitech/rtype/modules/ECS/include/ECS/Systems/Epitech/ontype](#) 827, 828

[/home/masina/Projects/Epitech/rtype/modules/ECS/include/ECS/Vec3.h](#) 830, 831

[/home/masina/Projects/Epitech/rtype/modules/ECS/include/ECS/Vec3/Vec3.h](#) 833, 834

[/home/masina/Projects/Epitech/rtype/modules/ECS/include/ECS/Vec3/Vec3/Vec3.h](#) 837, 838

[/home/masina/Projects/Epitech/rtype/modules/ECS/include/ECS/Vec3/Vec3/Vec3/Vec3.h](#) 839, 840

[/home/masina/Projects/Epitech/rtype/modules/ECS/include/ECS/Vec3/Vec3/Vec3/Vec3/Vec3.h](#) 841, 842

[/home/masina/Projects/Epitech/rtype/modules/ECS/include/ECS/Vec3/Vec3/Vec3/Vec3/Vec3/Vec3.h](#) 1175, 1176

[/home/masina/Projects/Epitech/rtype/modules/ECS/include/ECS/Vec3/Vec3/Vec3/Vec3/Vec3/Vec3/Vec3.h](#) 845, 846

[/home/masina/Projects/Epitech/rtype/modules/Engine/include/Engine.h](#) 847, 848

[/home/masina/Projects/Epitech/rtype/modules/Engine/include/Engine/Engine.h](#) 1164, 1165

[/home/masina/Projects/Epitech/rtype/modules/Engine/include/Engine/Engine/Engine.h](#) 1169

[/home/masina/Projects/Epitech/rtype/modules/Engine/src/engine.h](#) 849

[/home/masina/Projects/Epitech/rtype/modules/Interfaces/include/Interfaces.h](#) 849, 850

[/home/masina/Projects/Epitech/rtype/modules/Interfaces/include/Interfaces/Interfaces.h](#) 851, 852

[/home/masina/Projects/Epitech/rtype/modules/Interfaces/include/Interfaces/Interfaces/Interfaces.h](#) 852, 853

[/home/masina/Projects/Epitech/rtype/modules/Interfaces/include/Interfaces/Interfaces/Interfaces/Interfaces.h](#) 854, 855

[/home/masina/Projects/Epitech/rtype/modules/Interfaces/include/Interfaces/Interfaces/Interfaces/Interfaces/Interfaces.h](#) 856, 857

[/home/masina/Projects/Epitech/rtype/modules/Interfaces/include/Interfaces/Interfaces/Interfaces/Interfaces/Interfaces/Interfaces.h](#) 858, 859

[/home/masina/Projects/Epitech/rtype/modules/Interfaces/include/Interfaces/Interfaces/Interfaces/Interfaces/Interfaces/Interfaces/Interfaces.h](#) 861, 863

[/home/masina/Projects/Epitech/rtype/modules/Interfaces/include/Interfaces/Interfaces/Interfaces/Interfaces/Interfaces/Interfaces/Interfaces/Interfaces.h](#) 870, 872

[/home/masina/Projects/Epitech/rtype/modules/Interfaces/include/Interfaces/Interfaces/Interfaces/Interfaces/Interfaces/Interfaces/Interfaces/Interfaces/Interfaces.h](#) 876, 877

[/home/masina/Projects/Epitech/rtype/modules/Utils/include/Utils.h](#) 888, 889

[/home/masina/Projects/Epitech/rtype/modules/Utils/include/Utils/Utils.h](#) 889, 891

[/home/masina/Projects/Epitech/rtype/modules/Utils/include/Utils/Utils/Utils.h](#) 892

[/home/masina/Projects/Epitech/rtype/modules/Utils/include/Utils/Utils/Utils/Utils.h](#) 1158, 1161

[/home/masina/Projects/Epitech/rtype/plugins/Game/RType/ASharp/PSockets/Epitech/RType/ClientSockets/Case/RType/ServerSockets/971, 972](#)
[/home/masina/Projects/Epitech/rtype/plugins/Game/RType/ASharp/PSockets/Epitech/RType/ClientSockets/Networks/AIO/Glib/1053, 1054](#)
[/home/masina/Projects/Epitech/rtype/plugins/Game/RType/ASharp/PSockets/Epitech/RType/ClientSockets/Networks/Asio/Glib/976, 977](#)
[/home/masina/Projects/Epitech/rtype/plugins/Game/RType/ASharp/PSockets/Epitech/RType/ClientSockets/Networks/Play/Client/982, 983](#)
[/home/masina/Projects/Epitech/rtype/plugins/Game/RType/ASharp/PSockets/Epitech/RType/ClientSockets/Networks/Project/Server/987, 988](#)
[/home/masina/Projects/Epitech/rtype/plugins/Game/RType/ASharp/PSockets/Epitech/RType/ClientSockets/Networks/Script/Server/991, 993](#)
[/home/masina/Projects/Epitech/rtype/plugins/Game/RType/ASharp/PSockets/Epitech/RType/ClientSockets/Networks/Script/Server/1055, 1056](#)
[/home/masina/Projects/Epitech/rtype/plugins/Game/RType/ASharp/PSockets/Epitech/RType/ClientSockets/Networks/README/1177, 1178](#)
[/home/masina/Projects/Epitech/rtype/plugins/Game/RType/ASharp/PSockets/Epitech/RType/ClientSockets/README/Upon/1996, 997](#)
[/home/masina/Projects/Epitech/rtype/plugins/Game/RType/ASharp/PSockets/Epitech/rtype/plugins/Renderer/README/932](#)
[/home/masina/Projects/Epitech/rtype/plugins/Game/RType/ASharp/PSockets/Epitech/rtype/plugins/Renderer/README/1058](#)
[/home/masina/Projects/Epitech/rtype/plugins/Game/RType/ASharp/PSockets/Epitech/rtype/plugins/Renderer/README/1000](#)
[/home/masina/Projects/Epitech/rtype/plugins/Game/RType/ASharp/PSockets/Epitech/rtype/plugins/Renderer/README/1059](#)
[/home/masina/Projects/Epitech/rtype/plugins/Game/RType/ASharp/PSockets/Epitech/rtype/plugins/Renderer/README/1060, 1061](#)
[/home/masina/Projects/Epitech/rtype/plugins/Game/RType/ASharp/PSockets/Epitech/rtype/plugins/Renderer/README/1063, 1064](#)
[/home/masina/Projects/Epitech/rtype/plugins/Game/RType/ASharp/PSockets/Epitech/rtype/plugins/Renderer/README/1067](#)
[/home/masina/Projects/Epitech/rtype/plugins/Game/RType/ASharp/PSockets/Epitech/rtype/plugins/Renderer/README/1042](#)
[/home/masina/Projects/Epitech/rtype/plugins/Game/RType/ASharp/PSockets/Epitech/rtype/plugins/Renderer/README/1068, 1069](#)
[/home/masina/Projects/Epitech/rtype/plugins/Game/RType/ASharp/PSockets/Epitech/rtype/plugins/Renderer/README/1071, 1072](#)
[/home/masina/Projects/Epitech/rtype/plugins/Game/RType/ASharp/PSockets/Epitech/rtype/plugins/Renderer/README/1073, 1075](#)
[/home/masina/Projects/Epitech/rtype/plugins/Game/RType/ASharp/PSockets/Epitech/rtype/plugins/Renderer/README/1076, 1077](#)
[/home/masina/Projects/Epitech/rtype/plugins/Game/RType/ASharp/PSockets/Epitech/rtype/plugins/Renderer/README/1078, 1079](#)
[/home/masina/Projects/Epitech/rtype/plugins/Game/RType/ASharp/PSockets/Epitech/rtype/plugins/Renderer/README/1080, 1082](#)
[/home/masina/Projects/Epitech/rtype/plugins/Game/RType/ASharp/PSockets/Epitech/rtype/plugins/Renderer/README/1083, 1084](#)
[/home/masina/Projects/Epitech/rtype/plugins/Game/RType/ASharp/PSockets/Epitech/rtype/plugins/Renderer/README/1096, 1097](#)
[/home/masina/Projects/Epitech/rtype/plugins/Game/RType/ASharp/PSockets/Epitech/rtype/plugins/Renderer/README/1100](#)
[/home/masina/Projects/Epitech/rtype/plugins/Game/RType/ASharp/PSockets/Epitech/rtype/plugins/Renderer/README/1105, 1106](#)
[/home/masina/Projects/Epitech/rtype/plugins/Game/RType/ASharp/PSockets/Epitech/rtype/plugins/Renderer/README/1108, 1109](#)
[/home/masina/Projects/Epitech/rtype/plugins/Game/RType/ASharp/PSockets/Epitech/rtype/plugins/Renderer/README/933](#)

- srv::AsioServer, 183
- ~AudioSystem
 - ecs::AudioSystem, 223
- ~BeamSystem
 - ecs::BeamSystem, 229
 - gme::BeamSystem, 233
- ~Client
 - cli::Client, 239
- ~Clock
 - utl::Clock, 246
- ~CollisionSystem
 - gme::CollisionSystem, 254, 255
- ~ConfigMulti
 - gme::ConfigMulti, 273
- ~ConfigSolo
 - gme::ConfigSolo, 278
- ~CreateRoomScene
 - gme::CreateRoomScene, 284
- ~DebugSystem
 - ecs::DebugSystem, 290
- ~EnemyAISystem
 - gme::EnemyAISystem, 303
- ~EnemySpawnSystem
 - gme::EnemySpawnSystem, 317
- ~EnemySystem
 - gme::EnemySystem, 324
- ~Engine
 - eng::Engine, 327
- ~EntityManager
 - gme::EntityManager, 336
- ~EventBus
 - utl::EventBus, 362
- ~ExplosionSystem
 - gme::ExplosionSystem, 388
- ~FlappyBirdClientSolo
 - gme::FlappyBirdClientSolo, 394
- ~Game
 - gme::Game, 402
- ~GameMulti
 - gme::GameMulti, 410
- ~GameSolo
 - gme::GameSolo, 419
- ~HUDSystem
 - gme::HUDSystem, 448
- ~HandlerPacket
 - rnp::HandlerPacket, 426
- ~INetworkClient
 - eng::INetworkClient, 464
- ~INetworkServer
 - srv::INetworkServer, 468
- ~IPlugin
 - utl::IPlugin, 477
- ~IPool
 - ecs::Registry::IPool, 478
- ~IScene
 - eng::IScene, 490
 - srv::IScene, 493
- ~ISystem
 - ecs::ISystem, 495
- ~Intro
 - cli::Intro, 474
- ~JoinRoomScene
 - gme::JoinRoomScene, 498
- ~LoadingAnimationSystem
 - gme::LoadingAnimationSystem, 512
- ~Logger
 - utl::Logger, 520
- ~Menu
 - cli::Menu, 528
- ~PlayerControllerMulti
 - gme::PlayerControllerMulti, 558
- ~PlayerDirectionSystem
 - gme::PlayerDirectionSystem, 564
- ~PluginLoader
 - utl::PluginLoader, 568
- ~ProjectileManager
 - gme::ProjectileManager, 578
- ~ProjectileSystem
 - gme::ProjectileSystem, 584
- ~RTypeClientMulti
 - gme::RTypeClientMulti, 607
- ~RTypeClientSolo
 - gme::RTypeClientSolo, 612
- ~RTypeServer
 - gme::RTypeServer, 617
- ~Registry
 - ecs::Registry, 593
- ~SceneManager
 - eng::SceneManager, 629
 - srv::SceneManager, 632
- ~ScrollingSystem
 - gme::ScrollingSystem, 640
- ~Server
 - srv::Server, 692
- ~ServerScene
 - gme::ServerScene, 698
- ~Settings
 - cli::Settings, 704
- ~SharedLib
 - utl::SharedLib, 708
- ~SpawnSystem
 - gme::SpawnSystem, 712
- ~SpriteSystem
 - ecs::SpriteSystem, 719
- ~StageManager
 - gme::StageManager, 722, 723
- ~StarfieldSystem
 - ecs::StarfieldSystem, 730
- ~TextSystem
 - ecs::TextSystem, 742
- ~WaitingRoomScene
 - gme::WaitingRoomScene, 753
- ~WaveManager
 - gme::WaveManager, 765
- ~WeaponSystem
 - gme::WeaponSystem, 776

A

eng, 63

a

ecs::Color, 269

eng::Color, 270

AckHandler

rnp, 70

ackHandler_

rnp::HandlerPacket, 439

add

ecs::Registry::Pool< T >, 572

addComponent

ecs::Registry, 593

addScene

eng::SceneManager, 629

srv::SceneManager, 632

addSystem

eng::AScene, 135

eng::IScene, 490

addWave

gme::WaveManager, 765

advancedEnemyCount

gme::SpawnWave, 715

AGGRESSIVE

gme, 66

AGGRESSIVE_ACTIVATION_DISTANCE

utl::GameConfig::Server::Enemy::AI, 100

ALREADY_IN_LOBBY

rnp, 73

srv, 76

ANIMATION_DURATION

cli::GameConfig::Enemy::Easy, 53

cli::GameConfig::Explosion, 54

cli::GameConfig::LoadingAnimation, 55

cli::GameConfig::Projectile::Supercharged, 58

utl::GameConfig::Enemy::Easy, 90

utl::GameConfig::Explosion, 92

utl::GameConfig::LoadingAnimation, 93

utl::GameConfig::Projectile::Supercharged, 96

ANIMATION_FRAMES

cli::GameConfig::Enemy::Easy, 53

cli::GameConfig::Explosion, 54

cli::GameConfig::LoadingAnimation, 55

cli::GameConfig::Projectile::Supercharged, 58

utl::GameConfig::Enemy::Easy, 90

utl::GameConfig::Explosion, 92

utl::GameConfig::LoadingAnimation, 93

utl::GameConfig::Projectile::Supercharged, 96

AnimationSystem

ecs::AnimationSystem, 116

gme::AnimationSystem, 119

APP_EXTENSION

argsHandler.cpp, 799, 802

applyAggressiveMovement

gme::EnemyAISystem, 304

applyDamageToEnemy

gme::CollisionSystem, 255

applyDamageToPlayer

gme::CollisionSystem, 256

applySineWaveMovement

gme::EnemyAISystem, 304

applySkinChange

cli::Settings, 704

applyVideoQuality

cli::Settings, 704

applyZigzagMovement

gme::EnemyAISystem, 305

ArgsHandler

cli::ArgsHandler, 129

srv::ArgsHandler, 132

argsHandler.cpp

APP_EXTENSION, 799, 802

HELP_MESSAGE, 799, 802

VERSION_MESSAGE, 799, 802

AScene

eng::AScene, 135

srv::AScene, 140

AsioClient

eng::AsioClient, 146

AsioServer

srv::AsioServer, 182

AUDIO

utl, 81

Audio plugin, 11

AUDIO_BATTLE_THEME

utl::Path::Audio, 104

AUDIO_BEGIN

utl::Path::Audio, 104

AUDIO_BOSS

utl::Path::Audio, 105

AUDIO_COIN

utl::Path::Audio, 105

AUDIO_DEATH_ALLIES

utl::Path::Audio, 105

AUDIO_DEATH_ENEMIES

utl::Path::Audio, 105

AUDIO_GOOFY

utl::Path::Audio, 105

AUDIO_INPUT

utl::Path::Audio, 105

audio_lib_path

cli::ArgsConfig, 125

AUDIO_SUPERCHARGED_SHOT

utl::Path::Audio, 105

AUDIO_TITLE

utl::Path::Audio, 105

AudioSystem

ecs::AudioSystem, 223

audioVolume

utl::cli::AppConfig, 121

B

eng, 63

b

ecs::Color, 269

eng::Color, 270

BANNED

- rnp, 72
- BAR_HEIGHT
 - cli::GameConfig::Beam, 52
 - utl::GameConfig::Beam, 89
- BAR_WIDTH
 - cli::GameConfig::Beam, 52
 - utl::GameConfig::Beam, 89
- BAR_X
 - cli::GameConfig::Beam, 52
 - utl::GameConfig::Beam, 90
- BAR_Y
 - cli::GameConfig::Beam, 52
 - utl::GameConfig::Beam, 90
- BASIC
 - ecs::Projectile, 576
- basicEnemyCount
 - gme::SpawnWave, 715
- BeamSystem
 - ecs::BeamSystem, 229
 - gme::BeamSystem, 233
- BLUE
 - utl::Config::Color, 85
- BLUE_SECOND
 - utl::Config::Color, 85
- BOSS
 - gme, 67
 - rnp, 73
- BOSS_MUSIC_DURATION
 - gme::GameMulti, 413
- bossSpawn
 - gme::SpawnWave, 715
- BROADCAST_INTERVAL
 - gme::RTypeServer, 623
 - utl::GameConfig::Server::Network, 103
- BROADCAST_RATE
 - utl::GameConfig::Server::Network, 103
- BROADCAST_TO_CLIENTS
 - utl, 80
- BROADCAST_WORLD_STATE
 - utl, 80
- broadcastGameStart
 - srv::AsioServer, 183
- broadcastLobbyUpdate
 - srv::AsioServer, 184
- broadcastWorldState
 - gme::RTypeServer, 618
- buffer_
 - rnp::Serializer, 689
- build
 - ecs::Registry::EntityBuilder, 332
- BUILD_TYPE
 - Version.hpp, 789, 791
- BUTTON_COUNT
 - gme::WaitingRoomScene, 758
- BUTTON_LEAVE
 - gme::WaitingRoomScene, 758
- BUTTON_READY
 - gme::WaitingRoomScene, 758
- BUTTON_START
 - gme::WaitingRoomScene, 758
- bytesTransferred
 - eng::ConnectionStats, 281
- C
 - eng, 63
- calculateHitboxOffsets
 - gme::Utils, 67
 - utl, 81
- calculateHitboxOffsetsRelative
 - utl, 82
- calculateSpriteCenter
 - utl, 83
- canShoot
 - gme::EnemyAISystem, 305
- CEILING_OFFSET_Y
 - cli::GameConfig::Stage, 60
 - utl::GameConfig::Stage, 104
- CHARGE_RATE
 - cli::GameConfig::Beam, 52
 - utl::GameConfig::Beam, 90
- CHARGE_TIME
 - cli::GameConfig::Projectile::Supercharged, 58
 - utl::GameConfig::Projectile::Supercharged, 96
- checkCircleCollision
 - gme::CollisionSystem, 256
- checkCircularCollision
 - gme::CollisionSystem, 257
- clampToScreen
 - gme::EnemyAISystem, 306
- cleanupDestroyedEntities
 - gme::EntityManager, 336
- cleanupEmptyLobbies
 - srv::AsioServer, 185
- clear
 - gme::EntityManager, 336
 - utl::EventBus, 362
- clearAllSessions
 - rnp::HandlerPacket, 427
- clearPlayerEntities
 - gme::WaitingRoomScene, 753
- clearRoomEntities
 - gme::JoinRoomScene, 499
- clearSession
 - rnp::HandlerPacket, 427
- clearStats
 - utl::EventBus, 363
- clearWaves
 - gme::WaveManager, 765
- clearWindow
 - eng::IRenderer, 481
- cli, 51
 - json, 51
- cli::ArgsConfig, 123
 - audio_lib_path, 125
 - exit, 125
 - frameLimit, 125
 - fromFile, 125

- fullscreen, 125
- game_multi_lib_path, 125
- game_solo_lib_path, 125
- height, 126
- host, 126
- network_lib_path, 126
- player_name, 126
- port, 126
- renderer_lib_path, 126
- width, 126
- cli::ArgsHandler, 128
 - ~ArgsHandler, 129
 - ArgsHandler, 129
 - operator=, 130
 - ParseArgs, 130
 - ParseEnv, 130
- cli::Client, 237
 - ~Client, 239
 - Client, 239
 - handleEvents, 240
 - m_config, 242
 - m_engine, 242
 - m_gameMulti, 242
 - m_gameSolo, 242
 - m_keysPressed, 242
 - m_pluginLoader, 242
 - m_showDebug, 242
 - operator=, 240
 - run, 240
 - setupConfig, 240
 - setupScenes, 241
 - stop, 241
 - updateKeyboardInput, 241
- cli::EnvConfig, 353
- cli::GameConfig, 51
- cli::GameConfig::Animation, 52
 - FRAME_DURATION, 52
- cli::GameConfig::Beam, 52
 - BAR_HEIGHT, 52
 - BAR_WIDTH, 52
 - BAR_X, 52
 - BAR_Y, 52
 - CHARGE_RATE, 52
 - MAX_CHARGE, 52
- cli::GameConfig::Enemy, 53
- cli::GameConfig::Enemy::Easy, 53
 - ANIMATION_DURATION, 53
 - ANIMATION_FRAMES, 53
 - DAMAGE, 53
 - FRAMES_PER_ROW, 53
 - HEALTH, 53
 - SCALE, 53
 - SHOOT_COOLDOWN, 53
 - SPAWN_RATE, 53
 - SPEED, 54
 - SPRITE_HEIGHT, 54
 - SPRITE_WIDTH, 54
- cli::GameConfig::Explosion, 54
 - ANIMATION_DURATION, 54
 - ANIMATION_FRAMES, 54
 - FRAMES_PER_ROW, 54
 - LIFETIME, 54
 - SCALE, 54
 - SPRITE_HEIGHT, 54
 - SPRITE_WIDTH, 55
- cli::GameConfig::Hitbox, 55
 - ENEMY_RADIUS, 55
 - PLAYER_RADIUS, 55
 - PROJECTILE_BASIC_RADIUS, 55
 - PROJECTILE_SUPERCHARGED_RADIUS, 55
- cli::GameConfig::LoadingAnimation, 55
 - ANIMATION_DURATION, 55
 - ANIMATION_FRAMES, 55
 - OFFSET_X, 56
 - OFFSET_Y, 56
 - SPRITE_HEIGHT, 56
 - SPRITE_WIDTH, 56
- cli::GameConfig::Player, 56
 - DIAGONAL_SPEED_MULTIPLIER, 56
 - FRAMES_PER_ROW, 56
 - SCALE, 56
 - SPEED, 56
 - SPRITE_HEIGHT, 56
 - SPRITE_WIDTH, 57
 - TOTAL_FRAMES, 57
- cli::GameConfig::Projectile, 57
- cli::GameConfig::Projectile::Basic, 57
 - DAMAGE, 57
 - FIRE_COOLDOWN, 57
 - LIFETIME, 57
 - SCALE, 57
 - SPEED, 57
 - SPRITE_HEIGHT, 57
 - SPRITE_WIDTH, 58
- cli::GameConfig::Projectile::Supercharged, 58
 - ANIMATION_DURATION, 58
 - ANIMATION_FRAMES, 58
 - CHARGE_TIME, 58
 - DAMAGE, 58
 - FIRE_COOLDOWN, 58
 - LIFETIME, 58
 - SCALE, 58
 - SPEED, 58
 - SPRITE_HEIGHT, 59
 - SPRITE_WIDTH, 59
- cli::GameConfig::Screen, 59
 - MAX_Y, 59
 - MIN_Y, 59
 - REMOVE_MAX_Y, 59
 - REMOVE_MIN_Y, 59
 - REMOVE_X, 59
 - SPAWN_X, 59
- cli::GameConfig::Stage, 60
 - CEILING_OFFSET_Y, 60
 - FLOOR_OFFSET_Y, 60

- cli::Intro, 471
 - ~Intro, 474
 - event, 474
 - Intro, 474
 - m__audio, 475
 - m__clock, 475
 - m__elapsedTime, 475
 - m__logoEntity, 475
 - m__renderer, 475
 - m__soundEntity, 475
 - onLeave, 475
 - operator=, 474
 - update, 475
- cli::Menu, 525
 - ~Menu, 528
 - event, 529
 - m__animationTime, 529
 - m__contributors, 529
 - m__contributorsEntity, 529
 - m__contributorsOffset, 529
 - m__keysPressed, 529
 - m__menuOptions, 530
 - m__playMusic, 530
 - m__selectedIndex, 530
 - m__titleEntity, 530
 - m__titlePulseTime, 530
 - Menu, 528
 - onOptionSelected, 530
 - operator=, 529
 - playMusic, 529
 - update, 529
- cli::Settings, 700
 - ~Settings, 704
 - applySkinChange, 704
 - applyVideoQuality, 704
 - event, 704
 - loadFromConfig, 704
 - m__animationTime, 705
 - m__appConfig, 705
 - m__audioVolume, 705
 - m__controlScheme, 705
 - m__controlValueEntity, 706
 - m__keysPressed, 706
 - m__playMusic, 706
 - m__qualityValueEntity, 706
 - m__renderer, 706
 - m__selectedIndex, 706
 - m__settingsOptions, 706
 - m__skinIndex, 706
 - m__skinSpriteEntity, 706
 - m__titleEntity, 706
 - m__titlePulseTime, 707
 - m__videoQuality, 707
 - m__volumeValueEntity, 707
 - onLeave, 707
 - operator=, 705
 - playMusic, 705
 - Settings, 703, 704
 - update, 705
 - updateSettingsDisplay, 705
- Client
 - cli::Client, 239
- CLIENT_REQUEST
 - rnp, 72
- clientCaps
 - rnp::PacketConnect, 531
 - srv::ClientSession, 244
- ClientSession
 - srv::ClientSession, 244
- Clock
 - utl::Clock, 246
- close
 - utl::SharedLib, 709
- Closed
 - eng, 63
- closeWindow
 - eng::IRenderer, 481
- CollisionSystem
 - gme::CollisionSystem, 254, 255
- color
 - eng::CircleShape, 236
 - eng::Text, 739
- COLOR_ERROR
 - utl::Logger, 520
- COLOR_INFO
 - utl::Logger, 520
- COLOR_RESET
 - utl::Logger, 520
- COLOR_WARNING
 - utl::Logger, 520
- ColorIndex
 - utl::Logger, 520
- COMPLETED
 - gme, 66
- COMPRESSED
 - rnp, 74
- ConfigMulti
 - gme::ConfigMulti, 273
- ConfigSolo
 - gme::ConfigSolo, 278
- CONNECT
 - rnp, 74
- connect
 - eng::AsioClient, 147
 - eng::INetworkClient, 464
- CONNECT_ACCEPT
 - rnp, 74
- ConnectAcceptHandler
 - rnp, 70
- connectAcceptHandler_
 - rnp::HandlerPacket, 439
- CONNECTED
 - eng, 62
- ConnectHandler
 - rnp, 70
- connectHandler_

- rnp::HandlerPacket, [439](#)
- CONNECTING
 - eng, [62](#)
- CONNECTION_ACCEPTED
 - utl, [80](#)
- CONNECTION_REJECTED
 - utl, [80](#)
- ConnectionState
 - eng, [62](#)
- connectionTime
 - eng::ConnectionStats, [281](#)
- connectServer
 - gme::ServerScene, [698](#)
- consume
 - utl::EventBus, [363](#)
- consumeForTarget
 - utl::EventBus, [364](#)
- consumeType
 - utl::EventBus, [364](#)
- content
 - ecs::Text, [737](#)
 - eng::Text, [739](#)
- Contributing to R-Type, [29](#)
- controlScheme
 - utl::cli::AppConfig, [121](#)
- createAdvancedEnemy
 - gme::EntityManager, [336](#)
- createAudio
 - eng::IAudio, [452](#)
- createBasicEnemy
 - gme::EntityManager, [337](#)
- createBasicProjectile
 - gme::ProjectileManager, [578](#)
- createBoss
 - gme::EntityManager, [338](#)
- createBossWave
 - gme::WaveManager, [766](#)
- createCircleShape
 - eng::IRenderer, [481](#)
- createComets
 - ecs::StarfieldSystem, [731](#)
- createDamageEvent
 - rnp::Serializer, [647](#)
- createDespawnEvent
 - rnp::Serializer, [647](#)
- createdTime
 - srv::Lobby, [516](#)
- createEnemyProjectile
 - gme::EntityManager, [338](#)
- createEntity
 - ecs::Registry, [594](#)
- createExplosion
 - gme::CollisionSystem, [257](#)
- createFont
 - eng::IRenderer, [481](#)
- createInputEvent
 - rnp::Serializer, [647](#)
- createLobby
 - eng::AsioClient, [148](#)
 - srv::AsioServer, [185](#)
- createNebulae
 - ecs::StarfieldSystem, [731](#)
- createPipePair
 - gme::Game, [403](#)
- createPlanets
 - ecs::StarfieldSystem, [732](#)
- createPlayer
 - gme::EntityManager, [339](#)
 - gme::Game, [403](#)
 - gme::GameSolo, [420](#)
 - gme::PlayerControllerMulti, [558](#)
- createPlayerProjectile
 - gme::EntityManager, [339](#)
- createPowerup
 - gme::EntityManager, [340](#)
- createProjectile
 - gme::ProjectileManager, [579](#)
- createRoom
 - gme::CreateRoomScene, [285](#)
- createRoom.cpp
 - keyToChar, [1006](#)
- CreateRoomScene
 - gme::CreateRoomScene, [284](#), [285](#)
- createScoreEvent
 - rnp::Serializer, [648](#)
- createScoreHUD
 - gme::HUDSystem, [449](#)
- createShootingStars
 - ecs::StarfieldSystem, [732](#)
- createSpawnEvent
 - rnp::Serializer, [649](#)
- createSprite
 - eng::IRenderer, [481](#)
- createStarfield
 - ecs::StarfieldSystem, [733](#)
- createStars
 - ecs::StarfieldSystem, [734](#)
- createStarsVaried
 - ecs::StarfieldSystem, [734](#)
- createSuperchargedProjectile
 - gme::ProjectileManager, [580](#)
- createText
 - eng::IRenderer, [482](#)
- createTexture
 - eng::IRenderer, [482](#)
- createWave1
 - gme::WaveManager, [766](#)
- createWave2
 - gme::WaveManager, [766](#)
- createWave3
 - gme::WaveManager, [767](#)
- createWave4
 - gme::WaveManager, [767](#)
- createWindow
 - eng::IRenderer, [482](#)
- CRITICAL

- utl, 80
- current_charge
 - ecs::BeamCharge, 226
- current_frame
 - ecs::Animation, 113
 - ecs::Explosion, 385
 - ecs::LoadingAnimation, 509
- current_lifetime
 - ecs::Explosion, 385
 - ecs::Projectile, 576
- current_time
 - ecs::Animation, 113
 - ecs::Explosion, 385
 - ecs::LoadingAnimation, 509
- currentLife
 - gme::EntityMetadata, 350
- currentLobbyId
 - srv::ClientSession, 244
- currentPlayers
 - rnp::LobbyInfo, 518
- currentQueueSize
 - utl::EventStats, 381
- currentRotation
 - gme::InterpolationData, 470
- currentX
 - gme::InterpolationData, 470
- currentY
 - gme::InterpolationData, 470
- CUSTOM
 - rnp, 73
- cv
 - utl::EventBus::EventQueue, 378
- CYAN_ELECTRIC
 - utl::Config::Color, 85
- CYAN_ELECTRIC_FOREGROUND
 - utl::Config::Color, 85
- CYAN_ELECTRIC_PARTICLES
 - utl::Config::Color, 86
- D
 - eng, 63
- DAMAGE
 - cli::GameConfig::Enemy::Easy, 53
 - cli::GameConfig::Projectile::Basic, 57
 - cli::GameConfig::Projectile::Supercharged, 58
 - rnp, 73
 - utl::GameConfig::Enemy::Easy, 91
 - utl::GameConfig::Projectile::Basic, 95
 - utl::GameConfig::Projectile::Supercharged, 97
 - utl::GameConfig::Server::Enemy::Advanced, 99
 - utl::GameConfig::Server::Enemy::Basic, 101
 - utl::GameConfig::Server::Enemy::Boss, 101
- damage
 - ecs::Enemy, 300
 - ecs::Projectile, 576
- DARK
 - utl::Config::Color, 86
- data
 - ecs::Registry::Pool< T >, 572
 - eng::QueuedPacket, 587
 - rnp::EventRecord, 379
 - srv::QueuedPacket, 588
 - utl::Event, 358
- DebugSystem
 - ecs::DebugSystem, 290
- DEFAULT
 - eng, 64
- DEFAULT_AUDIO_MUTED
 - utl::Config::Audio, 84
- DEFAULT_AUDIO_VOLUME
 - utl::Config::Audio, 84
- DEFAULT_CONTROL_SCHEME
 - utl::Config::Game, 87
- DEFAULT_MAX_CLIENT
 - srv::Config::Network, 77
- DEFAULT_NETWORK_HOST
 - srv::Config::Network, 77
 - utl::Config::Network, 88
- DEFAULT_NETWORK_PORT
 - srv::Config::Network, 77
 - utl::Config::Network, 88
- DEFAULT_PLAYER_NAME
 - utl::Config::Player, 88
- DEFAULT_SKIN_INDEX
 - utl::Config::Game, 87
- DEFAULT_TICK_RATE
 - srv::Game, 77
- DEFAULT_UPDATE_INTERVAL
 - srv::Game, 77
- DEFAULT_VIDEO_QUALITY
 - utl::Config::Game, 87
- DEFAULT_WINDOW_FRAME_LIMIT
 - utl::Config::Window, 88
- DEFAULT_WINDOW_FULLSCREEN
 - utl::Config::Window, 88
- DEFAULT_WINDOW_HEIGHT
 - utl::Config::Window, 88
- DEFAULT_WINDOW_WIDTH
 - utl::Config::Window, 89
- Delete
 - eng, 63
- description
 - rnp::PacketError, 536
- deserializeConnect
 - rnp::Serializer, 649
- deserializeConnectAccept
 - rnp::Serializer, 650
- deserializeDisconnect
 - rnp::Serializer, 651
- deserializeEntityEvents
 - rnp::Serializer, 651
- deserializeEntityState
 - rnp::Serializer, 652
- deserializeError
 - rnp::Serializer, 652
- deserializeEventRecord

- rnp::Serializer, [653](#)
- deserializeGameStart
 - rnp::Serializer, [654](#)
- deserializeHeader
 - rnp::Serializer, [654](#)
- deserializeInputData
 - rnp::Serializer, [655](#)
- deserializeLobbyCreate
 - rnp::Serializer, [655](#)
- deserializeLobbyCreateResponse
 - rnp::Serializer, [656](#)
- deserializeLobbyInfo
 - rnp::Serializer, [657](#)
- deserializeLobbyJoin
 - rnp::Serializer, [657](#)
- deserializeLobbyJoinResponse
 - rnp::Serializer, [658](#)
- deserializeLobbyListResponse
 - rnp::Serializer, [659](#)
- deserializeLobbyUpdate
 - rnp::Serializer, [659](#)
- deserializePingPong
 - rnp::Serializer, [660](#)
- deserializeStartGameRequest
 - rnp::Serializer, [660](#)
- deserializeWorldState
 - rnp::Serializer, [661](#)
- DESPAWN
 - rnp, [73](#)
- destination
 - srv::QueuedPacket, [588](#)
- destroyEnemy
 - gme::EntityManager, [340](#)
- destroyPlayer
 - gme::EntityManager, [341](#)
- destroyPowerup
 - gme::EntityManager, [341](#)
- destroyProjectile
 - gme::EntityManager, [341](#)
- DIAGONAL_SPEED_MULTIPLIER
 - cli::GameConfig::Player, [56](#)
 - utl::GameConfig::Player, [94](#)
- DISCONNECT
 - rnp, [74](#)
- disconnect
 - eng::AsioClient, [149](#)
 - eng::INetworkClient, [464](#)
- DISCONNECT_CLIENT
 - utl, [80](#)
- disconnectClient
 - srv::AsioServer, [186](#)
 - srv::INetworkServer, [468](#)
- DISCONNECTED
 - eng, [62](#)
- DisconnectHandler
 - rnp, [70](#)
- disconnectHandler_
 - rnp::HandlerPacket, [439](#)
- DISCONNECTING
 - eng, [62](#)
- DisconnectReason
 - rnp, [72](#)
- displayWindow
 - eng::IRenderer, [482](#)
- Dot
 - eng, [63](#)
- Down
 - eng, [63](#)
- down_pressed
 - ecs::KeyboardInput, [505](#)
- drawCircleShape
 - eng::IRenderer, [482](#)
- drawPoint
 - eng::IRenderer, [483](#)
- drawSprite
 - eng::IRenderer, [483](#)
- drawText
 - eng::IRenderer, [484](#)
- dt
 - gme::GameMulti::PendingInput, [551](#)
 - gme::PlayerControllerMulti::PendingInput, [552](#)
- Duration
 - utl::Clock, [246](#)
- duration
 - gme::Wave, [761](#)
- E
 - eng, [63](#)
- ecs, [60](#)
 - Entity, [61](#)
 - INVALID_ENTITY, [61](#)
- ecs::Animation, [111](#)
 - current_frame, [113](#)
 - current_time, [113](#)
 - frame_duration, [113](#)
 - frame_height, [113](#)
 - frame_width, [113](#)
 - frames_per_row, [113](#)
 - total_frames, [113](#)
- ecs::AnimationSystem, [113](#)
 - ~AnimationSystem, [116](#)
 - AnimationSystem, [116](#)
 - operator=, [116](#)
 - update, [116](#)
- ecs::ASystem, [216](#)
 - isEnabled, [217](#)
 - m_isEnable, [218](#)
 - setEnabled, [217](#)
- ecs::Audio, [218](#)
 - loop, [220](#)
 - path, [220](#)
 - play, [220](#)
 - volume, [220](#)
- ecs::AudioSystem, [220](#)
 - ~AudioSystem, [223](#)
 - AudioSystem, [223](#)

- [m_audio](#), [224](#)
 - [m_audioVolume](#), [224](#)
 - [m_playSelectionMusic](#), [224](#)
 - [m_registry](#), [224](#)
 - [m_selectionSoundEntity](#), [224](#)
 - [m_selectionSoundName](#), [225](#)
 - [operator=](#), [223](#)
 - [update](#), [224](#)
- [ecs::BeamCharge](#), [225](#)
 - [current_charge](#), [226](#)
 - [max_charge](#), [226](#)
- [ecs::BeamSystem](#), [227](#)
 - [~BeamSystem](#), [229](#)
 - [BeamSystem](#), [229](#)
 - [m_renderer](#), [230](#)
 - [operator=](#), [229](#)
 - [update](#), [229](#)
- [ecs::Ceiling](#), [234](#)
- [ecs::Color](#), [267](#)
 - [a](#), [269](#)
 - [b](#), [269](#)
 - [g](#), [269](#)
 - [r](#), [269](#)
- [ecs::DebugSystem](#), [287](#)
 - [~DebugSystem](#), [290](#)
 - [DebugSystem](#), [290](#)
 - [m_renderer](#), [291](#)
 - [m_showDebug](#), [291](#)
 - [operator=](#), [290](#)
 - [update](#), [290](#)
- [ecs::Enemy](#), [297](#)
 - [damage](#), [300](#)
 - [health](#), [300](#)
 - [last_shot_time](#), [300](#)
 - [max_health](#), [300](#)
 - [shoot_cooldown](#), [300](#)
 - [speed](#), [300](#)
- [ecs::Explosion](#), [382](#)
 - [current_frame](#), [385](#)
 - [current_lifetime](#), [385](#)
 - [current_time](#), [385](#)
 - [frame_duration](#), [385](#)
 - [frame_height](#), [385](#)
 - [frame_width](#), [385](#)
 - [frames_per_row](#), [385](#)
 - [lifetime](#), [385](#)
 - [total_frames](#), [385](#)
- [ecs::Floor](#), [396](#)
- [ecs::Font](#), [397](#)
 - [path](#), [399](#)
- [ecs::Hitbox](#), [442](#)
 - [offsetX](#), [444](#)
 - [offsetY](#), [445](#)
 - [radius](#), [445](#)
- [ecs::IComponent](#), [454](#)
 - [id](#), [455](#)
- [ecs::ISystem](#), [494](#)
 - [~ISystem](#), [495](#)
 - [isEnabled](#), [495](#)
 - [setEnabled](#), [495](#)
 - [update](#), [495](#)
- [ecs::KeyboardInput](#), [502](#)
 - [down_pressed](#), [505](#)
 - [left_pressed](#), [505](#)
 - [right_pressed](#), [505](#)
 - [space_pressed](#), [505](#)
 - [up_pressed](#), [505](#)
- [ecs::Layer](#), [505](#)
 - [layer](#), [506](#)
- [ecs::LoadingAnimation](#), [507](#)
 - [current_frame](#), [509](#)
 - [current_time](#), [509](#)
 - [frame_duration](#), [509](#)
 - [frame_height](#), [509](#)
 - [frame_width](#), [509](#)
 - [frames_per_row](#), [509](#)
 - [total_frames](#), [509](#)
- [ecs::Pixel](#), [552](#)
- [ecs::Player](#), [553](#)
 - [is_cli](#), [555](#)
- [ecs::Projectile](#), [574](#)
 - [BASIC](#), [576](#)
 - [current_lifetime](#), [576](#)
 - [damage](#), [576](#)
 - [lifetime](#), [576](#)
 - [pierce_remaining](#), [576](#)
 - [SUPERCHARGED](#), [576](#)
 - [Type](#), [576](#)
 - [type](#), [576](#)
- [ecs::Rect](#), [588](#)
 - [pos_x](#), [591](#)
 - [pos_y](#), [591](#)
 - [size_x](#), [591](#)
 - [size_y](#), [591](#)
- [ecs::Registry](#), [591](#)
 - [~Registry](#), [593](#)
 - [addComponent](#), [593](#)
 - [createEntity](#), [594](#)
 - [getAll](#), [596](#)
 - [getComponent](#), [598](#)
 - [getPool](#), [600](#)
 - [hasComponent](#), [601](#)
 - [m_components](#), [604](#)
 - [m_entities](#), [604](#)
 - [m_lastEntity](#), [604](#)
 - [m_onComponentAddedCallbacks](#), [604](#)
 - [onComponentAdded](#), [602](#)
 - [operator=](#), [603](#)
 - [Registry](#), [593](#)
 - [removeComponent](#), [603](#)
- [ecs::Registry::EntityBuilder](#), [330](#)
 - [build](#), [332](#)
 - [EntityBuilder](#), [332](#)
 - [m_entity](#), [333](#)
 - [m_registry](#), [333](#)
 - [with](#), [332](#)

- ecs::Registry::IPool, 478
 - ~IPool, 478
 - remove, 479
- ecs::Registry::Pool< T >, 570
 - add, 572
 - data, 572
 - get, 572
 - has, 572
 - remove, 572
- ecs::Scale, 625
 - x, 626
 - y, 627
- ecs::Score, 633
 - value, 634
- ecs::Scrolling, 635
 - fit_width, 637
 - original_height, 637
 - original_width, 637
 - speed_x, 637
- ecs::SpriteSystem, 716
 - ~SpriteSystem, 719
 - m_renderer, 720
 - operator=, 719
 - SpriteSystem, 719
 - update, 719
- ecs::StarfieldSystem, 727
 - ~StarfieldSystem, 730
 - createComets, 731
 - createNebulae, 731
 - createPlanets, 732
 - createShootingStars, 732
 - createStarfield, 733
 - createStars, 734
 - createStarsVaried, 734
 - m_renderer, 736
 - m_timeAccumulator, 736
 - operator=, 735
 - StarfieldSystem, 730, 731
 - update, 735
- ecs::Text, 736
 - content, 737
 - font_size, 737
- ecs::TextSystem, 739
 - ~TextSystem, 742
 - m_renderer, 743
 - operator=, 742
 - TextSystem, 742
 - update, 742
- ecs::Texture, 743
 - path, 744
- ecs::Transform, 745
 - rotation, 746
 - x, 747
 - y, 747
- ecs::Velocity, 747
 - x, 748
 - y, 749
- endpoint
 - srv::ClientSession, 244
- endpointToString
 - srv::AsioServer, 187
- enemies
 - gme::Wave, 762
- ENEMY
 - rnp, 72
- ENEMY_ADVANCED
 - gme, 67
 - rnp, 72
- ENEMY_BASIC
 - gme, 67
 - rnp, 72
- ENEMY_BOSS
 - rnp, 72
- ENEMY_DAMAGE
 - utl::GameConfig::Server::Projectile, 103
- ENEMY_LIFETIME
 - utl::GameConfig::Server::Projectile, 103
- ENEMY_RADIUS
 - cli::GameConfig::Hitbox, 55
 - utl::GameConfig::Hitbox, 93
- ENEMY_SMOOTH_FACTOR
 - gme::GameMulti, 413
- ENEMY_SPEED
 - utl::GameConfig::Server::Projectile, 103
- EnemyAISystem
 - gme::EnemyAISystem, 303, 304
- EnemyBehavior
 - gme, 66
- EnemySpawnSystem
 - gme::EnemySpawnSystem, 317, 318
- EnemySystem
 - gme::EnemySystem, 324
- eng, 61
 - A, 63
 - B, 63
 - C, 63
 - Closed, 63
 - CONNECTED, 62
 - CONNECTING, 62
 - ConnectionState, 62
 - D, 63
 - DEFAULT, 64
 - Delete, 63
 - DISCONNECTED, 62
 - DISCONNECTING, 62
 - Dot, 63
 - Down, 63
 - E, 63
 - Enter, 63
 - Escape, 63
 - EventType, 62
 - F, 63
 - G, 63
 - H, 63
 - I, 63
 - id, 62

J, [63](#)
 K, [63](#)
 Key, [63](#)
 KeyPressed, [63](#)
 KeyReleased, [63](#)
 L, [63](#)
 LControl, [64](#)
 Left, [63](#)
 M, [63](#)
 N, [63](#)
 None, [63](#)
 Num0, [64](#)
 Num1, [64](#)
 Num2, [64](#)
 Num3, [64](#)
 Num4, [64](#)
 Num5, [64](#)
 Num6, [64](#)
 Num7, [64](#)
 Num8, [64](#)
 Num9, [64](#)
 O, [63](#)
 P, [63](#)
 Paused, [64](#)
 Playing, [64](#)
 Q, [63](#)
 R, [63](#)
 Right, [63](#)
 RUN, [64](#)
 S, [63](#)
 Space, [63](#)
 State, [64](#)
 Status, [64](#)
 STOP, [64](#)
 Stopped, [64](#)
 T, [63](#)
 U, [63](#)
 Unknown, [63](#)
 Up, [63](#)
 V, [63](#)
 W, [63](#)
 X, [63](#)
 Y, [64](#)
 Z, [64](#)
 eng::AScene, [133](#)
 ~AScene, [135](#)
 addSystem, [135](#)
 AScene, [135](#)
 getId, [135](#)
 getName, [136](#)
 getRegistry, [136](#)
 m_eventBus, [136](#)
 m_eventComponentId, [136](#)
 m_id, [136](#)
 m_name, [137](#)
 m_registry, [137](#)
 m_systems, [137](#)
 operator=, [136](#)
 setName, [136](#)
 updateSystems, [136](#)
 eng::AsioClient, [141](#)
 ~AsioClient, [147](#)
 AsioClient, [146](#)
 connect, [147](#)
 createLobby, [148](#)
 disconnect, [149](#)
 generatePingNonce, [150](#)
 getConnectionState, [150](#)
 getCurrentLobbyId, [150](#)
 getLatency, [150](#)
 getName, [150](#)
 getServerTickRate, [150](#)
 getSessionId, [150](#)
 getStats, [150](#)
 getType, [151](#)
 handleConnectAccept, [151](#)
 handleDisconnect, [151](#)
 handleEntityEvent, [152](#)
 handleError, [153](#)
 handleGameStart, [154](#)
 handleLobbyCreateResponse, [154](#)
 handleLobbyJoinResponse, [155](#)
 handleLobbyListResponse, [156](#)
 handleLobbyUpdate, [156](#)
 handlePing, [157](#)
 handlePong, [158](#)
 handleReceive, [159](#)
 handleWorldState, [159](#)
 isConnected, [160](#)
 isInLobby, [160](#)
 joinLobby, [160](#)
 leaveLobby, [161](#)
 m_clientCaps, [174](#)
 m_componentId, [174](#)
 m_connectionState, [174](#)
 m_connectionTimeout, [174](#)
 m_currentLobbyId, [174](#)
 m_eventBus, [175](#)
 m_ioContext, [175](#)
 m_lastPingNonce, [175](#)
 m_lastPingTime, [175](#)
 m_lastServerResponse, [175](#)
 m_latency, [175](#)
 m_networkThread, [175](#)
 m_onGameStart, [175](#)
 m_onLobbyCreated, [175](#)
 m_onLobbyJoined, [175](#)
 m_onLobbyListReceived, [176](#)
 m_onLobbyUpdated, [176](#)
 m_packetHandler, [176](#)
 m_pingInterval, [176](#)
 m_playerName, [176](#)
 m_recvBuffer, [176](#)
 m_running, [176](#)
 m_senderEndpoint, [176](#)
 m_sendQueue, [176](#)

- m_sendQueueMutex, 176
- m_serverEndpoint, 177
- m_serverHost, 177
- m_serverPort, 177
- m_serverTickRate, 177
- m_sessionId, 177
- m_socket, 177
- m_stats, 177
- networkThreadLoop, 162
- processBusEvent, 162
- processSendQueue, 163
- requestLobbyList, 164
- requestStartGame, 164
- sendConnect, 165
- sendDisconnect, 166
- sendPacketImmediate, 167
- sendPing, 167
- sendPong, 168
- sendToServer, 169
- setClientCapabilities, 169
- setOnGameStart, 170
- setOnLobbyCreated, 170
- setOnLobbyJoined, 170
- setOnLobbyListReceived, 170
- setOnLobbyUpdated, 170
- setPlayerName, 171
- setupPacketHandlers, 171
- startReceive, 172
- update, 173
- updateConnectionManagement, 173
- eng::CircleShape, 235
 - color, 236
 - name, 236
 - outline_color, 237
 - outline_thickness, 237
 - radius, 237
 - x, 237
 - y, 237
- eng::Color, 269
 - a, 270
 - b, 270
 - g, 270
 - r, 270
- eng::ConnectionStats, 280
 - bytesTransferred, 281
 - connectionTime, 281
 - packetsLost, 281
 - packetsReceived, 281
 - packetsSent, 281
- eng::Engine, 325
 - ~Engine, 327
 - Engine, 327
 - getAudio, 327
 - getClock, 328
 - getNetwork, 328
 - getRenderer, 328
 - getSceneManager, 328
 - getState, 328
 - m_audio, 329
 - m_clock, 329
 - m_network, 329
 - m_renderer, 329
 - m_sceneManager, 330
 - m_state, 330
 - operator=, 328, 329
 - render, 329
 - setState, 329
 - stop, 329
- eng::Event, 354
 - key, 354
 - type, 354
- eng::IAudio, 450
 - createAudio, 452
 - isPlaying, 453
 - playAudio, 453
 - setLoop, 453
 - setVolume, 453
 - stopAudio, 454
- eng::INetworkClient, 461
 - ~INetworkClient, 464
 - connect, 464
 - disconnect, 464
 - getConnectionState, 464
 - getLatency, 464
 - getServerTickRate, 464
 - getSessionId, 464
 - isConnected, 464
 - sendToServer, 464
 - setClientCapabilities, 464
 - setPlayerName, 464
 - update, 465
- eng::IRenderer, 479
 - clearWindow, 481
 - closeWindow, 481
 - createCircleShape, 481
 - createFont, 481
 - createSprite, 481
 - createText, 482
 - createTexture, 482
 - createWindow, 482
 - displayWindow, 482
 - drawCircleShape, 482
 - drawPoint, 483
 - drawSprite, 483
 - drawText, 484
 - getWindowSize, 484
 - pollEvent, 485
 - renderGui, 485
 - rotateSprite, 485
 - setCircleShapePosition, 485
 - setFrameLimit, 485
 - setSpriteColor, 486
 - setSpriteFrame, 486
 - setSpriteOrigin, 486
 - setSpritePosition, 486
 - setSpriteRotation, 487

- setSpriteScale, [487](#)
- setSpriteTexture, [487](#)
- setTextColor, [488](#)
- setTextContent, [488](#)
- setTextPosition, [488](#)
- setWindowIcon, [489](#)
- windowIsOpen, [489](#)
- eng::IScene, [489](#)
 - ~IScene, [490](#)
 - addSystem, [490](#)
 - event, [490](#)
 - getId, [491](#)
 - getName, [491](#)
 - getRegistry, [491](#)
 - setName, [491](#)
 - update, [491](#)
 - updateSystems, [491](#)
- eng::QueuedPacket, [586](#)
 - data, [587](#)
 - QueuedPacket, [587](#)
 - reliable, [587](#)
- eng::SceneManager, [627](#)
 - ~SceneManager, [629](#)
 - addScene, [629](#)
 - generateNextId, [629](#)
 - getCurrentScene, [629](#)
 - getScene, [629](#)
 - m_currentSceneId, [630](#)
 - m_nextId, [630](#)
 - m_scenes, [630](#)
 - operator=, [630](#)
 - SceneManager, [629](#)
 - switchToScene, [630](#)
- eng::Text, [738](#)
 - color, [739](#)
 - content, [739](#)
 - font_name, [739](#)
 - name, [739](#)
 - size, [739](#)
 - x, [739](#)
 - y, [739](#)
- eng::WindowSize, [782](#)
 - height, [782](#)
 - width, [782](#)
- Engine
 - eng::Engine, [327](#)
- ensureEnemyDeathChannel
 - gme::CollisionSystem, [258](#)
- ensurePlayerDeathAudio
 - gme::CollisionSystem, [258](#)
- ensureSuperShotAudio
 - gme::WeaponSystem, [776](#)
- Enter
 - eng, [63](#)
- entities
 - rnp::PacketWorldState, [549](#)
- Entity
 - ecs, [61](#)
- entity
 - gme::EntityMetadata, [350](#)
- ENTITY_EVENT
 - rnp, [74](#)
- ENTITY_EVENT_RECEIVED
 - utl, [80](#)
- ENTITY_EVENTS_RECEIVED
 - utl, [80](#)
- EntityBuilder
 - ecs::Registry::EntityBuilder, [332](#)
- entityCount
 - rnp::PacketWorldState, [550](#)
- EntityEventHandler
 - rnp, [70](#)
- entityEventHandler_
 - rnp::HandlerPacket, [439](#)
- entityId
 - rnp::EventRecord, [379](#)
- entityId1
 - gme::CollisionPair, [250](#)
- entityId2
 - gme::CollisionPair, [250](#)
- EntityManager
 - gme::EntityManager, [336](#)
- EntitySubtype
 - rnp, [72](#)
- EntityType
 - rnp, [72](#)
- entryPoint
 - entrypoint.cpp, [931–934](#)
- entrypoint.cpp
 - entryPoint, [931–934](#)
- EntryPointFn
 - utl, [79](#)
- ErrorCode
 - rnp, [73](#)
- errorCode
 - rnp::PacketError, [536](#)
 - rnp::PacketLobbyCreateResponse, [541](#)
 - rnp::PacketLobbyJoinResponse, [543](#)
- ErrorHandler
 - rnp, [70](#)
- errorHandler_
 - rnp::HandlerPacket, [439](#)
- Escape
 - eng, [63](#)
- Event
 - utl::Event, [356, 357](#)
- event
 - cli::Intro, [474](#)
 - cli::Menu, [529](#)
 - cli::Settings, [704](#)
 - eng::IScene, [490](#)
 - gme::ConfigMulti, [273](#)
 - gme::ConfigSolo, [278](#)
 - gme::CreateRoomScene, [285](#)
 - gme::Game, [404](#)
 - gme::GameMulti, [411](#)

- gme::GameSolo, 420
- gme::JoinRoomScene, 499
- gme::ServerScene, 698
- gme::WaitingRoomScene, 753
- EventBus
 - utl::EventBus, 362
- EventPriority
 - utl, 80
- events
 - utl::EventBus::EventQueue, 378
 - utl::EventBus::PriorityEventQueue, 574
- EventType
 - eng, 62
 - rnp, 73
 - utl, 80
- eventTypeCount
 - utl::EventStats, 381
- exit
 - cli::ArgsConfig, 125
 - srv::ArgsConfig, 128
- ExplosionSystem
 - gme::ExplosionSystem, 388, 389
- F
 - eng, 63
- filterEventType
 - utl::EventBus, 365
- findNearestPlayer
 - gme::EnemyAISystem, 306
- FINISHED
 - rnp, 74
- FIRE_COOLDOWN
 - cli::GameConfig::Projectile::Basic, 57
 - cli::GameConfig::Projectile::Supercharged, 58
 - utl::GameConfig::Projectile::Basic, 95
 - utl::GameConfig::Projectile::Supercharged, 97
- fit_width
 - ecs::Scrolling, 637
- FlappyBirdClientSolo
 - gme::FlappyBirdClientSolo, 394
- FLOOR_OFFSET_Y
 - cli::GameConfig::Stage, 60
 - utl::GameConfig::Stage, 104
- font_name
 - eng::Text, 739
- font_size
 - ecs::Text, 737
- FONTS_RTYPE
 - utl::Path::Font, 106
- formatLogMessage
 - utl::Logger, 521
- FRAME_DURATION
 - cli::GameConfig::Animation, 52
 - utl::GameConfig::Animation, 89
- frame_duration
 - ecs::Animation, 113
 - ecs::Explosion, 385
 - ecs::LoadingAnimation, 509
- frame_height
 - ecs::Animation, 113
 - ecs::Explosion, 385
 - ecs::LoadingAnimation, 509
- frame_limit
 - ecs::Animation, 113
 - ecs::Explosion, 385
 - ecs::LoadingAnimation, 509
- frameLimit
 - cli::ArgsConfig, 125
 - utl::cli::AppConfig, 121
- FRAMES_PER_ROW
 - cli::GameConfig::Enemy::Easy, 53
 - cli::GameConfig::Explosion, 54
 - cli::GameConfig::Player, 56
 - utl::GameConfig::Enemy::Easy, 91
 - utl::GameConfig::Explosion, 92
 - utl::GameConfig::Player, 94
- frames_per_row
 - ecs::Animation, 113
 - ecs::Explosion, 385
 - ecs::LoadingAnimation, 509
- fromFile
 - cli::ArgsConfig, 125
 - srv::ArgsConfig, 127
- FULL
 - srv, 76
- fullscreen
 - cli::ArgsConfig, 125
 - utl::cli::AppConfig, 121
- G
 - eng, 63
- g
 - ecs::Color, 269
 - eng::Color, 270
- Game
 - gme::Game, 402, 403
- Game plugin, 13
- game.cpp
 - getGapSize, 936
 - Large, 936
 - Medium, 936
 - PipeType, 936
 - randomPipeType, 936
 - Small, 936
- GAME_CLIENT
 - utl, 81
- game_lib_path
 - srv::ArgsConfig, 128
- GAME_LOGIC
 - utl, 84
- game_multi_lib_path
 - cli::ArgsConfig, 125
- GAME_SERVER
 - utl, 81
- game_solo_lib_path
 - cli::ArgsConfig, 125
- GAME_START
 - rnp, 74

- utl, 81
- gameMode
 - rnp::LobbyInfo, 518
 - rnp::PacketLobbyCreate, 539
 - srv::Lobby, 516
- GameMulti
 - gme::GameMulti, 410
- GameSolo
 - gme::GameSolo, 419, 420
- GameStartHandler
 - rnp, 70
- gameStartHandler_
 - rnp::HandlerPacket, 439
- generateNextId
 - eng::SceneManager, 629
- generatePingNonce
 - eng::AsioClient, 150
- generateSessionId
 - srv::AsioServer, 187
- get
 - ecs::Registry::Pool< T >, 572
- getAge
 - utl::Event, 357
- getAggressiveness
 - gme::EnemyAISystem, 307
- getAll
 - ecs::Registry, 596
- getAllEntityStates
 - gme::EntityManager, 342
- getAudio
 - eng::Engine, 327
- getClientCount
 - srv::AsioServer, 188
 - srv::INetworkServer, 468
- getClock
 - eng::Engine, 328
- getCollisionCount
 - gme::CollisionSystem, 259
- getCollisionInfo
 - gme::CollisionSystem, 259
- getColorForDuration
 - utl::Logger, 522
- GetComponent
 - ecs::Registry, 598
- getConnectedSessions
 - srv::AsioServer, 188
 - srv::INetworkServer, 468
- getConnectionState
 - eng::AsioClient, 150
 - eng::INetworkClient, 464
- getCurrentEditField
 - gme::ServerScene, 698
- getCurrentLobbyId
 - eng::AsioClient, 150
- getCurrentScene
 - eng::SceneManager, 629
 - srv::SceneManager, 632
- getCurrentWave
 - gme::StageManager, 723
 - gme::WaveManager, 767
- getCurrentWaveIndex
 - gme::EnemySpawnSystem, 318
- getData
 - rnp::Serializer, 662
- getDeltaSeconds
 - utl::Clock, 247
- getDistanceToNearestPlayer
 - gme::EnemyAISystem, 307
- getElapsed
 - utl::Clock, 247
- getEnemies
 - gme::EntityManager, 342
- getEnemy
 - gme::EntityManager, 343
- getEntityMetadata
 - gme::EntityManager, 343, 344
- getEntryPoint
 - utl::PluginLoader, 568
- getEnvMap
 - utl, 83
- getEventsPerSecond
 - utl::EventStats, 381
- getGapSize
 - game.cpp, 936
- getId
 - eng::AScene, 135
 - eng::IScene, 491
 - srv::AScene, 140
 - srv::IScene, 493
- getInstance
 - utl::EventBus, 365
- getLatency
 - eng::AsioClient, 150
 - eng::INetworkClient, 464
- getLevelTime
 - gme::EnemySpawnSystem, 318
- getLobbyId
 - gme::WaitingRoomScene, 754
- getMainSceneId
 - gme::FlappyBirdClientSolo, 394
 - gme::IGameClient, 457
 - gme::RTypeClientMulti, 607
 - gme::RTypeClientSolo, 612
- getName
 - eng::AScene, 136
 - eng::AsioClient, 150
 - eng::IScene, 491
 - gme::FlappyBirdClientSolo, 394
 - gme::RTypeClientMulti, 607
 - gme::RTypeClientSolo, 612
 - gme::RTypeServer, 618
 - srv::AScene, 140
 - srv::AsioServer, 188
 - srv::IScene, 493
 - utl::IPlugin, 477
- getNetwork

- eng::Engine, 328
- getNetworkEntityType
 - gme::EntityManager, 344
- getNetworkIdForEntity
 - gme::EntityManager, 344
- getPlayer
 - gme::EntityManager, 344
- getPlayers
 - gme::EntityManager, 344
- getPool
 - ecs::Registry, 600
- getProjectile
 - gme::EntityManager, 345
- getProjectiles
 - gme::EntityManager, 345
- getQueueSize
 - utl::EventBus, 366
- getRandomY
 - gme::EnemySpawnSystem, 318
- getRegistry
 - eng::AScene, 136
 - eng::IScene, 491
 - srv::AScene, 140
 - srv::IScene, 493
- getRenderer
 - eng::Engine, 328
- getScene
 - eng::SceneManager, 629
 - srv::SceneManager, 632
- getSceneManager
 - eng::Engine, 328
- getServerTickRate
 - eng::AsioClient, 150
 - eng::INetworkClient, 464
- getSessionId
 - eng::AsioClient, 150
 - eng::INetworkClient, 464
- getSize
 - rnp::Serializer, 663
- getSpawnX
 - gme::EnemySpawnSystem, 318
- getState
 - eng::Engine, 328
 - gme::IGameServer, 460
 - gme::RTypeServer, 618
- getStats
 - eng::AsioClient, 150
 - rnp::HandlerPacket, 427
 - utl::EventBus, 366
- getTotalEnemiesSpawned
 - gme::EnemySpawnSystem, 318
- getTotalEntityCount
 - gme::EntityManager, 345
- getTotalGameTime
 - gme::StageManager, 723
- getTotalWaves
 - gme::WaveManager, 767
- getType
 - eng::AsioClient, 151
 - gme::FlappyBirdClientSolo, 394
 - gme::RTypeClientMulti, 607
 - gme::RTypeClientSolo, 612
 - gme::RTypeServer, 618
 - srv::AsioServer, 188
 - utl::IPlugin, 477
- getWaveTimer
 - gme::StageManager, 723
- getWindowSize
 - eng::IRenderer, 484
- GIT_COMMIT_HASH
 - Version.hpp, 789, 791
- GIT_TAG
 - Version.hpp, 789, 791
- gme, 64
 - AGGRESSIVE, 66
 - BOSS, 67
 - COMPLETED, 66
 - ENEMY_ADVANCED, 67
 - ENEMY_BASIC, 67
 - EnemyBehavior, 66
 - IN_PROGRESS, 66
 - LevelState, 66
 - LOOSE, 66
 - LOSE, 67
 - PLAYER, 67
 - PLAYING, 67
 - POWERUP, 67
 - PROJECTILE_ENEMY, 67
 - PROJECTILE_PLAYER, 67
 - ServerEntityType, 66
 - SINE_WAVE, 66
 - State, 67
 - STATIONARY, 66
 - STRAIGHT, 66
 - WAITING_FOR_PLAYERS, 66
 - WIN, 67
 - ZIGZAG, 66
- gme::AnimationSystem, 117
 - ~AnimationSystem, 119
 - AnimationSystem, 119
 - operator=, 119
 - update, 119
- gme::BeamSystem, 230
 - ~BeamSystem, 233
 - BeamSystem, 233
 - m_renderer, 234
 - operator=, 233
 - update, 233
- gme::CollisionPair, 250
 - entityId1, 250
 - entityId2, 250
 - overlapDistance, 250
- gme::CollisionSystem, 251
 - ~CollisionSystem, 254, 255
 - applyDamageToEnemy, 255
 - applyDamageToPlayer, 256

- checkCircleCollision, 256
- checkCircularCollision, 257
- CollisionSystem, 254, 255
- createExplosion, 257
- ensureEnemyDeathChannel, 258
- ensurePlayerDeathAudio, 258
- getCollisionCount, 259
- getCollisionInfo, 259
- handleEnemyProjectilePlayerCollision, 260
- handlePlayerEnemyCollision, 260
- handlePlayerPowerupCollision, 261
- handlePlayerProjectileEnemyCollision, 261
- isEnabled, 261
- m_collisionCount, 266
- m_enemyDeathAudioEntities, 266
- m_entityManager, 266
- m_nextEnemyDeathChannel, 266
- m_playerDeathAudioEntity, 266
- m_registry, 266
- m_renderer, 266
- m_showDebug, 266
- m_wasPlayerPresent, 267
- operator=, 262
- playEnemyDeathSound, 262
- playPlayerDeathSound, 263
- removeEnemy, 263
- removeProjectile, 264
- resetCollisionCount, 264
- setEnabled, 265
- update, 265
- gme::ConfigMulti, 270
 - ~ConfigMulti, 273
 - ConfigMulti, 273
 - event, 273
 - m_animationTime, 274
 - m_keysPressed, 274
 - m_menuOptions, 274
 - m_playMusic, 274
 - m_selectedIndex, 274
 - m_titleEntity, 274
 - m_titlePulseTime, 275
 - onOptionSelected, 275
 - operator=, 274
 - playMusic, 274
 - update, 274
- gme::ConfigSolo, 275
 - ~ConfigSolo, 278
 - ConfigSolo, 278
 - event, 278
 - m_animationTime, 279
 - m_fpsEntity, 279
 - m_keysPressed, 279
 - m_menuOptions, 279
 - m_playMusic, 279
 - m_selectedIndex, 279
 - m_titleEntity, 280
 - m_titlePulseTime, 280
 - onOptionSelected, 280
- operator=, 279
- playMusic, 279
- update, 279
- gme::CreateRoomScene, 281
 - ~CreateRoomScene, 284
 - createRoom, 285
 - CreateRoomScene, 284, 285
 - event, 285
 - m_animationTime, 286
 - m_maxPlayers, 286
 - m_maxPlayersValueEntity, 286
 - m_options, 286
 - m_playMusic, 287
 - m_roomName, 287
 - m_roomNameValueEntity, 287
 - m_selectedIndex, 287
 - m_titleEntity, 287
 - onBackToMulti, 287
 - onCreate, 287
 - onRoomCreated, 287
 - operator=, 285
 - playMusic, 285
 - processEventBus, 286
 - update, 286
 - updateValueDisplay, 286
- gme::EnemyAISystem, 300
 - ~EnemyAISystem, 303
 - applyAggressiveMovement, 304
 - applySineWaveMovement, 304
 - applyZigzagMovement, 305
 - canShoot, 305
 - clampToScreen, 306
 - EnemyAISystem, 303, 304
 - findNearestPlayer, 306
 - getAggressiveness, 307
 - getDistanceToNearestPlayer, 307
 - m_aggressiveness, 311
 - m_bossSpreadTimers, 312
 - m_enemyMovementTimers, 312
 - m_entityManager, 312
 - m_registry, 312
 - m_rng, 312
 - m_shootChance, 312
 - operator=, 308
 - setAggressiveness, 308
 - tryShoot, 308
 - update, 309
 - updateAdvancedEnemyAI, 309
 - updateBasicEnemyAI, 310
 - updateBossAI, 311
- gme::EnemySpawn, 312
 - health, 313
 - spawnDelay, 313
 - type, 313
 - x, 313
 - y, 313
- gme::EnemySpawnSystem, 314
 - ~EnemySpawnSystem, 317

- EnemySpawnSystem, 317, 318
- getCurrentWaveIndex, 318
- getLevelTime, 318
- getRandomY, 318
- getSpawnX, 318
- getTotalEnemiesSpawned, 318
- initializeWaves, 319
- isEnabled, 319
- m_currentWaveIndex, 321
- m_enabled, 321
- m_entityManager, 321
- m_levelTime, 321
- m_rng, 321
- m_totalEnemiesSpawned, 321
- m_waves, 321
- m_waveSpawnTimer, 321
- m_yDistribution, 321
- operator=, 319
- processWave, 319
- reset, 320
- setEnabled, 320
- update, 320
- gme::EnemySystem, 322
 - ~EnemySystem, 324
 - EnemySystem, 324
 - isEnabled, 324
 - m_renderer, 325
 - operator=, 324
 - setEnabled, 324
 - update, 324
- gme::EntityManager, 334
 - ~EntityManager, 336
 - cleanupDestroyedEntities, 336
 - clear, 336
 - createAdvancedEnemy, 336
 - createBasicEnemy, 337
 - createBoss, 338
 - createEnemyProjectile, 338
 - createPlayer, 339
 - createPlayerProjectile, 339
 - createPowerup, 340
 - destroyEnemy, 340
 - destroyPlayer, 341
 - destroyPowerup, 341
 - destroyProjectile, 341
 - EntityManager, 336
 - getAllEntityStates, 342
 - getEnemies, 342
 - getEnemy, 343
 - getEntityMetadata, 343, 344
 - getNetworkEntityType, 344
 - getNetworkIdForEntity, 344
 - getPlayer, 344
 - getPlayers, 344
 - getProjectile, 345
 - getProjectiles, 345
 - getTotalEntityCount, 345
 - hasPlayer, 345
 - m_destroyQueue, 348
 - m_enemyEntities, 348
 - m_entityMetadata, 348
 - m_entityToNetworkId, 348
 - m_nextEnemyId, 348
 - m_nextPowerupId, 348
 - m_nextProjectileId, 348
 - m_playerEntities, 349
 - m_powerupEntities, 349
 - m_projectileEntities, 349
 - m_registry, 349
 - operator=, 346
 - processDestroyQueue, 346
 - registerEntity, 347
 - unregisterEntity, 347
 - updateLifetimes, 348
- gme::EntityMetadata, 349
 - currentLife, 350
 - entity, 350
 - isActive, 350
 - lifetime, 350
 - networkId, 351
 - ownerId, 351
 - type, 351
- gme::ExplosionSystem, 386
 - ~ExplosionSystem, 388
 - ExplosionSystem, 388, 389
 - isEnabled, 389
 - m_renderer, 391
 - operator=, 389
 - removeExplosion, 389, 390
 - setEnabled, 390
 - update, 390, 391
- gme::FlappyBirdClientSolo, 391
 - ~FlappyBirdClientSolo, 394
 - FlappyBirdClientSolo, 394
 - getMainSceneId, 394
 - getName, 394
 - getType, 394
 - init, 395
 - m_appConfig, 396
 - m_engine, 396
 - m_mainSceneId, 396
 - operator=, 395
 - setupScenes, 395
 - update, 395
- gme::Game, 399
 - ~Game, 402
 - createPipePair, 403
 - createPlayer, 403
 - event, 404
 - Game, 402, 403
 - m_audio, 405
 - m_flapSound, 405
 - m_gameOver, 405
 - m_gameOverShown, 405
 - m_keysPressed, 405
 - m_looseSound, 405

- m_looseText, [405](#)
- m_pipes, [406](#)
- m_playerEntity, [406](#)
- m_playMusic, [406](#)
- m_renderer, [406](#)
- m_showDebug, [406](#)
- operator=, [404](#)
- playMusic, [404](#)
- resetGame, [404](#)
- update, [405](#)
- gme::GameMulti, [406](#)
 - ~GameMulti, [410](#)
 - BOSS_MUSIC_DURATION, [413](#)
 - ENEMY_SMOOTH_FACTOR, [413](#)
 - event, [411](#)
 - GameMulti, [410](#)
 - handleWorldStateUpdate, [411](#)
 - m_audio, [413](#)
 - m_beginSoundEntity, [413](#)
 - m_bossMusicEntity, [414](#)
 - m_bossMusicStarted, [414](#)
 - m_bossMusicTimer, [414](#)
 - m_enemyData, [414](#)
 - m_enemyEntities, [414](#)
 - m_eventComponentId, [414](#)
 - m_firstWorldState, [414](#)
 - m_inputHistory, [414](#)
 - m_keysPressed, [414](#)
 - m_lastAckSeqId, [414](#)
 - m_loadedFonts, [414](#)
 - m_loadedTextures, [414](#)
 - m_localPlayerEntity, [415](#)
 - m_playerController, [415](#)
 - m_playerSkinMap, [415](#)
 - m_playMusic, [415](#)
 - m_projectileData, [415](#)
 - m_projectileEntities, [415](#)
 - m_remotePlayerData, [415](#)
 - m_remotePlayers, [415](#)
 - m_renderer, [415](#)
 - m_sessionId, [415](#)
 - m_showDebug, [415](#)
 - m_skinIndex, [416](#)
 - m_stageManager, [416](#)
 - m_starfieldCreated, [416](#)
 - operator=, [411](#)
 - playMusic, [411](#)
 - preloadCommonTextures, [411](#)
 - processEventBus, [412](#)
 - PROJECTILE_SMOOTH_FACTOR, [416](#)
 - REMOTE_PLAYER_SMOOTH_FACTOR, [416](#)
 - setupEventSubscriptions, [412](#)
 - update, [412](#)
 - updateInterpolation, [413](#)
 - updatePlayerSkin, [413](#)
- gme::GameMulti::PendingInput, [550](#)
 - dt, [551](#)
 - inputData, [551](#)
 - seqId, [551](#)
- gme::GameSolo, [416](#)
 - ~GameSolo, [419](#)
 - createPlayer, [420](#)
 - event, [420](#)
 - GameSolo, [419](#), [420](#)
 - handlePlayerInputs, [420](#)
 - m_audio, [422](#)
 - m_keysPressed, [422](#)
 - m_lastAppliedSkinIndex, [422](#)
 - m_loadedFonts, [422](#)
 - m_loadedTextures, [422](#)
 - m_playerEntity, [422](#)
 - m_playmusic, [422](#)
 - m_renderer, [422](#)
 - m_showDebug, [422](#)
 - m_skinIndex, [422](#)
 - m_stageManager, [422](#)
 - operator=, [421](#)
 - playMusic, [421](#)
 - preloadCommonTextures, [421](#)
 - update, [421](#)
 - updatePlayerSkin, [421](#)
- gme::HUDSystem, [445](#)
 - ~HUDSystem, [448](#)
 - createScoreHUD, [449](#)
 - HUDSystem, [448](#)
 - m_registry, [450](#)
 - m_renderer, [450](#)
 - m_scoreBgEntity, [450](#)
 - m_scoreDigitEntities, [450](#)
 - operator=, [449](#)
 - update, [449](#)
 - updateScore, [449](#)
- gme::IGameClient, [455](#)
 - getMainSceneId, [457](#)
 - init, [457](#)
 - update, [457](#)
- gme::IGameServer, [458](#)
 - getState, [460](#)
 - start, [460](#)
 - stop, [460](#)
 - update, [461](#)
- gme::InterpolationData, [469](#)
 - currentRotation, [470](#)
 - currentX, [470](#)
 - currentY, [470](#)
 - smoothFactor, [470](#)
 - targetRotation, [470](#)
 - targetVx, [470](#)
 - targetVy, [470](#)
 - targetX, [470](#)
 - targetY, [470](#)
- gme::JoinRoomScene, [495](#)
 - ~JoinRoomScene, [498](#)
 - clearRoomEntities, [499](#)
 - event, [499](#)

- handleLobbyJoinResponse, 499
- handleLobbyListResponse, 499
- JoinRoomScene, 498, 499
- m_animationTime, 501
- m_noRoomsEntity, 501
- m_playMusic, 501
- m_roomEntities, 501
- m_rooms, 502
- m_selectedIndex, 502
- onBackToMulti, 502
- onJoin, 502
- onRefreshRequest, 502
- operator=, 500
- playMusic, 500
- processEventBus, 500
- refreshRoomList, 500
- setRooms, 500
- setupEventSubscriptions, 500
- update, 501
- updateRoomDisplay, 501
- gme::LoadingAnimationSystem, 509
 - ~LoadingAnimationSystem, 512
 - LoadingAnimationSystem, 512
 - m_renderer, 514
 - operator=, 513
 - update, 513
- gme::PlayerControllerMulti, 555
 - ~PlayerControllerMulti, 558
 - createPlayer, 558
 - handleInput, 559
 - INPUT_THROTTLE_INTERVAL, 560
 - isSpacePressed, 559
 - m_componentId, 560
 - m_eventBus, 560
 - m_keysPressed, 560
 - m_lastAckSeqId, 560
 - m_nextSeqId, 560
 - m_playerEntity, 561
 - m_renderer, 561
 - operator=, 559
 - PlayerControllerMulti, 558
 - sendInputsIfChanged, 559
 - sendInputToServer, 559
 - update, 560
- gme::PlayerControllerMulti::PendingInput, 551
 - dt, 552
 - inputData, 552
 - seqId, 552
- gme::PlayerDirectionSystem, 561
 - ~PlayerDirectionSystem, 564
 - m_skinIndex, 566
 - operator=, 565
 - PlayerDirectionSystem, 564
 - update, 565
- gme::ProjectileManager, 576
 - ~ProjectileManager, 578
 - createBasicProjectile, 578
 - createProjectile, 579
 - createSuperchargedProjectile, 580
 - operator=, 581
 - ProjectileManager, 578
- gme::ProjectileSystem, 582
 - ~ProjectileSystem, 584
 - operator=, 585
 - ProjectileSystem, 584
 - update, 585
- gme::RTypeClientMulti, 604
 - ~RTypeClientMulti, 607
 - getMainSceneId, 607
 - getName, 607
 - getType, 607
 - init, 608
 - m_appConfig, 609
 - m_engine, 609
 - m_mainSceneId, 609
 - m_showDebug, 609
 - operator=, 608
 - RTypeClientMulti, 607
 - setupScenes, 608
 - update, 609
- gme::RTypeClientSolo, 609
 - ~RTypeClientSolo, 612
 - getMainSceneId, 612
 - getName, 612
 - getType, 612
 - init, 613
 - m_appConfig, 614
 - m_engine, 614
 - m_mainSceneId, 614
 - operator=, 613
 - RTypeClientSolo, 612
 - setupScenes, 613
 - update, 614
- gme::RTypeServer, 615
 - ~RTypeServer, 617
 - BROADCAST_INTERVAL, 623
 - broadcastWorldState, 618
 - getName, 618
 - getState, 618
 - getType, 618
 - handlePlayerShooting, 618
 - m_collisionSystem, 623
 - m_enemyAISystem, 623
 - m_enemySpawnSystem, 623
 - m_entityManager, 623
 - m_eventBus, 623
 - m_gameState, 624
 - m_lastBroadcastTime, 624
 - m_lastShotTime, 624
 - m_levelState, 624
 - m_playerShooting, 624
 - m_registry, 624
 - m_waveManager, 624
 - operator=, 619
 - processGameStartEvent, 619
 - processPlayerInputEvent, 620

- processServerStartEvent, 620
- PROJECTILE_COOLDOWN, 624
- RTypeServer, 617
- start, 621
- stop, 621
- update, 621
- updateEntities, 622
- updateSystems, 622
- gme::ScrollingSystem, 637
 - ~ScrollingSystem, 640
 - m_renderer, 642
 - operator=, 641
 - ScrollingSystem, 640
 - update, 641
- gme::ServerScene, 694
 - ~ServerScene, 698
 - connectServer, 698
 - event, 698
 - getCurrentEditField, 698
 - m_animationTime, 699
 - m_host, 699
 - m_playerName, 699
 - m_playerNameValueEntity, 699
 - m_playMusic, 699
 - m_port, 699
 - m_selectedIndex, 700
 - m_serverIPValueEntity, 700
 - m_serverOptions, 700
 - m_serverPortValueEntity, 700
 - onBackToMenu, 700
 - onConnect, 700
 - operator=, 698, 699
 - playMusic, 699
 - ServerScene, 697, 698
 - update, 699
 - updateValueDisplay, 699
- gme::SpawnSystem, 710
 - ~SpawnSystem, 712
 - isEnabled, 712
 - m_enemySpawnTimer, 714
 - m_renderer, 714
 - m_waveTimer, 714
 - operator=, 712
 - setEnabled, 712
 - spawnEnemy, 713
 - SpawnSystem, 712
 - spawnWave, 713
 - update, 714
- gme::SpawnWave, 715
 - advancedEnemyCount, 715
 - basicEnemyCount, 715
 - bossSpawn, 715
 - spawnedCount, 716
 - spawnInterval, 716
 - spawnTime, 716
- gme::StageManager, 720
 - ~StageManager, 722, 723
 - getCurrentWave, 723
 - getTotalGameTime, 723
 - getWaveTimer, 723
 - isStageSpawned, 723, 724
 - m_currentWave, 726
 - m_stageSpawned, 726
 - m_stageTimer, 726
 - m_totalGameTimer, 726
 - m_waveTimer, 726
 - operator=, 724
 - ORIGINAL_WIDTH, 726
 - SCROLL_SPEED, 726
 - spawnStage, 724, 725
 - SPRITE_HEIGHT, 726
 - STAGE_SPAWN_DELAY, 726
 - StageManager, 722, 723
 - stopScrolling, 725
 - TOTAL_WAVES, 726
 - update, 725
 - WAVE_DURATION, 726
- gme::Utils, 67
 - calculateHitboxOffsets, 67
- gme::WaitingRoomScene, 749
 - ~WaitingRoomScene, 753
 - BUTTON_COUNT, 758
 - BUTTON_LEAVE, 758
 - BUTTON_READY, 758
 - BUTTON_START, 758
 - clearPlayerEntities, 753
 - event, 753
 - getLobbyId, 754
 - handleGameStart, 754
 - handleLobbyUpdate, 754
 - isHost, 754
 - leaveLobby, 755
 - m_animationTime, 758
 - m_currentLobbyInfo, 759
 - m_hasLobbyInfo, 759
 - m_isHost, 759
 - m_leaveButtonEntity, 759
 - m_lobbyId, 759
 - m_lobbyIdEntity, 759
 - m_playerCountEntity, 759
 - m_playerEntities, 759
 - m_playMusic, 759
 - m_readyButtonEntity, 759
 - m_renderer, 759
 - m_selectedButton, 760
 - m_startButtonEntity, 760
 - m_statusEntity, 760
 - onGameStart, 760
 - onLeaveLobby, 760
 - operator=, 755
 - playMusic, 755
 - processEventBus, 755
 - setIsHost, 755
 - setLobbyId, 756
 - setLobbyInfo, 756
 - setupEventSubscriptions, 757

- startGame, 757
- update, 758
- updatePlayerDisplay, 758
- WaitingRoomScene, 753
- gme::Wave, 760
 - duration, 761
 - enemies, 762
 - onComplete, 762
 - waitForClear, 762
 - waveNumber, 762
- gme::WaveManager, 762
 - ~WaveManager, 765
 - addWave, 765
 - clearWaves, 765
 - createBossWave, 766
 - createWave1, 766
 - createWave2, 766
 - createWave3, 767
 - createWave4, 767
 - getCurrentWave, 767
 - getTotalWaves, 767
 - INITIAL_DELAY, 771
 - isActive, 768
 - isCompleted, 768
 - isReady, 768
 - isWaveCleared, 768
 - m_active, 771
 - m_completed, 771
 - m_currentWaveIndex, 771
 - m_enemiesSpawned, 772
 - m_entityManager, 772
 - m_initialDelayPassed, 772
 - m_initialDelayTimer, 772
 - m_spawnTimer, 772
 - m_waveInProgress, 772
 - m_waves, 772
 - m_waveTimer, 772
 - operator=, 768, 769
 - processSpawns, 769
 - reset, 769
 - setupDefaultWaves, 770
 - start, 770
 - startNextWave, 770
 - update, 771
 - WaveManager, 765
- gme::WeaponSystem, 773
 - ~WeaponSystem, 776
 - ensureSuperShotAudio, 776
 - hideLoadingAnimation, 777
 - m_fireCooldown, 781
 - m_isCharging, 781
 - m_superShotAudioEntity, 781
 - operator=, 777, 778
 - reset, 778
 - showLoadingAnimation, 778, 779
 - tryFireBasic, 779
 - tryFireSupercharged, 780
 - update, 780, 781
 - WeaponSystem, 776
- GRAY_BLUE_SUBTLE
 - utl::Config::Color, 86
- GREEN
 - utl::Config::Color, 86
- GREEN_READY
 - utl::Config::Color, 86
- H
 - eng, 63
- handle
 - utl::SharedLib, 709
- handleBroadcastEvent
 - srv::AsioServer, 188
- handleConnect
 - srv::AsioServer, 189
- handleConnectAccept
 - eng::AsioClient, 151
- handleDisconnect
 - eng::AsioClient, 151
 - srv::AsioServer, 190
- handleEnemyProjectilePlayerCollision
 - gme::CollisionSystem, 260
- handleEntityEvent
 - eng::AsioClient, 152
 - srv::AsioServer, 191
- handleError
 - eng::AsioClient, 153
- handleEvents
 - cli::Client, 240
- handleGameStart
 - eng::AsioClient, 154
 - gme::WaitingRoomScene, 754
- handleInput
 - gme::PlayerControllerMulti, 559
- handleLobbyCreate
 - srv::AsioServer, 191
- handleLobbyCreateResponse
 - eng::AsioClient, 154
- handleLobbyJoin
 - srv::AsioServer, 192
- handleLobbyJoinResponse
 - eng::AsioClient, 155
 - gme::JoinRoomScene, 499
- handleLobbyLeave
 - srv::AsioServer, 193
- handleLobbyListRequest
 - srv::AsioServer, 194
- handleLobbyListResponse
 - eng::AsioClient, 156
 - gme::JoinRoomScene, 499
- handleLobbyUpdate
 - eng::AsioClient, 156
 - gme::WaitingRoomScene, 754
- handlePing
 - eng::AsioClient, 157
 - srv::AsioServer, 195
- handlePlayerEnemyCollision
 - gme::CollisionSystem, 260

- handlePlayerInputs
 - gme::GameSolo, [420](#)
- handlePlayerPowerupCollision
 - gme::CollisionSystem, [261](#)
- handlePlayerProjectileEnemyCollision
 - gme::CollisionSystem, [261](#)
- handlePlayerShooting
 - gme::RTypeServer, [618](#)
- handlePong
 - eng::AsioClient, [158](#)
 - srv::AsioServer, [196](#)
- handleReceive
 - eng::AsioClient, [159](#)
 - srv::AsioServer, [196](#)
- HandlerPacket
 - rnp::HandlerPacket, [426](#)
- HandlerResult
 - rnp, [73](#)
- handleSendEntityEventToClients
 - srv::AsioServer, [197](#)
- handleSendToClientEvent
 - srv::AsioServer, [197](#)
- handleStartGameRequest
 - srv::AsioServer, [198](#)
- handleWorldState
 - eng::AsioClient, [159](#)
- handleWorldStateUpdate
 - gme::GameMulti, [411](#)
- has
 - ecs::Registry::Pool< T >, [572](#)
- hasComponent
 - ecs::Registry, [601](#)
- hasExpired
 - utl::Event, [357](#)
- hasPlayer
 - gme::EntityManager, [345](#)
- HEALTH
 - cli::GameConfig::Enemy::Easy, [53](#)
 - utl::GameConfig::Enemy::Easy, [91](#)
 - utl::GameConfig::Server::Enemy::Advanced, [99](#)
 - utl::GameConfig::Server::Enemy::Basic, [101](#)
 - utl::GameConfig::Server::Enemy::Boss, [101](#)
- health
 - ecs::Enemy, [300](#)
 - gme::EnemySpawn, [313](#)
- healthPercent
 - rnp::EntityState, [352](#)
- HEIGHT
 - utl::GameConfig::Player, [94](#)
- height
 - cli::ArgsConfig, [126](#)
 - eng::WindowSize, [782](#)
 - utl::cli::AppConfig, [121](#)
- HELP_MESSAGE
 - argsHandler.cpp, [799](#), [802](#)
- hideLoadingAnimation
 - gme::WeaponSystem, [777](#)
- HIGH
 - utl, [80](#)
- HITBOX_RADIUS
 - utl::GameConfig::Server::Enemy::Advanced, [99](#)
 - utl::GameConfig::Server::Enemy::Basic, [101](#)
 - utl::GameConfig::Server::Enemy::Boss, [102](#)
- host
 - cli::ArgsConfig, [126](#)
 - srv::ArgsConfig, [128](#)
 - utl::cli::AppConfig, [121](#)
 - utl::srv::AppConfig, [123](#)
- hostSessionId
 - rnp::LobbyInfo, [518](#)
 - srv::Lobby, [516](#)
- hostToNetwork16
 - rnp::EndianUtils, [292](#)
- hostToNetwork32
 - rnp::EndianUtils, [293](#)
- hostToNetworkFloat
 - rnp::EndianUtils, [294](#)
- How to create your own plugin, [9](#)
- HUDSystem
 - gme::HUDSystem, [448](#)
- I
 - eng, [63](#)
- ICON_APP
 - utl::Path::Icons, [106](#)
- ICON_PATH
 - utl::Path::Texture, [107](#)
- id
 - ecs::IComponent, [455](#)
 - eng, [62](#)
 - rnp::EntityState, [352](#)
 - srv, [76](#)
- IN_GAME
 - rnp, [74](#)
 - srv, [76](#)
- IN_PROGRESS
 - gme, [66](#)
- INFO
 - utl, [81](#)
- INFO_TEXT_COLOR
 - utl::Config::Color, [86](#)
- init
 - gme::FlappyBirdClientSolo, [395](#)
 - gme::IGameClient, [457](#)
 - gme::RTypeClientMulti, [608](#)
 - gme::RTypeClientSolo, [613](#)
 - srv::AsioServer, [199](#)
 - srv::INetworkServer, [468](#)
 - utl::Logger, [523](#)
- INITIAL_DELAY
 - gme::WaveManager, [771](#)
- initializeWaves
 - gme::EnemySpawnSystem, [319](#)
- INPUT
 - rnp, [73](#)

- INPUT_THROTTLE_INTERVAL
 - gme::PlayerControllerMulti, 560
- inputData
 - gme::GameMulti::PendingInput, 551
 - gme::PlayerControllerMulti::PendingInput, 552
- INTERNAL_ERROR
 - rnp, 73
- Intro
 - cli::Intro, 474
- INVALID_ENTITY
 - ecs, 61
- INVALID_PACKET
 - rnp, 73
- INVALID_PAYLOAD
 - rnp, 73
- IPlugin.hpp
 - PLUGIN_EXPORT, 909
- is_cli
 - ecs::Player, 555
- isActive
 - gme::EntityMetadata, 350
 - gme::WaveManager, 768
- isBroadcast
 - utl::Event, 358
- isCompleted
 - gme::WaveManager, 768
- isCompressed
 - rnp::PacketContext, 534
- isConnected
 - eng::AsioClient, 160
 - eng::INetworkClient, 464
 - srv::ClientSession, 244
- isEnabled
 - ecs::ASystem, 217
 - ecs::ISystem, 495
 - gme::CollisionSystem, 261
 - gme::EnemySystem, 324
 - gme::ExplosionSystem, 389
 - gme::SpawnSystem, 712
- isEnabled
 - gme::EnemySpawnSystem, 319
- isHost
 - gme::WaitingRoomScene, 754
- isInLobby
 - eng::AsioClient, 160
- isPlaying
 - eng::IAudio, 453
- isRateLimited
 - rnp::HandlerPacket, 427
- isReady
 - gme::WaveManager, 768
- isReliable
 - rnp::PacketContext, 534
- isRunning
 - srv::AsioServer, 199
 - srv::INetworkServer, 468
 - utl::EventBus, 366
- isSpacePressed
 - gme::PlayerControllerMulti, 559
- isStageSpawned
 - gme::StageManager, 723, 724
- isTargeted
 - utl::Event, 358
- isWaveCleared
 - gme::WaveManager, 768
- J
 - eng, 63
- joinLobby
 - eng::AsioClient, 160
 - srv::AsioServer, 199
- JoinRoomScene
 - gme::JoinRoomScene, 498, 499
- json
 - cli, 51
 - srv, 76
- K
 - eng, 63
- Key
 - eng, 63
- key
 - eng::Event, 354
- KeyPressed
 - eng, 63
- KeyReleased
 - eng, 63
- keyToChar
 - createRoom.cpp, 1006
 - ServerScene.cpp, 1025
- L
 - eng, 63
- Large
 - game.cpp, 936
- last_shot_time
 - ecs::Enemy, 300
- lastPacketTime_
 - rnp::HandlerPacket, 439
- lastPingSent
 - srv::ClientSession, 244
- lastSeen
 - srv::ClientSession, 244
- latency
 - srv::ClientSession, 244
- layer
 - ecs::Layer, 506
- LControl
 - eng, 64
- leaveLobby
 - eng::AsioClient, 161
 - gme::WaitingRoomScene, 755
 - srv::AsioServer, 200
- Left
 - eng, 63
- left_pressed

- ecs::KeyboardInput, 505
- length
 - rnp::PacketHeader, 538
- LevelState
 - gme, 66
- LibHandle
 - utl, 79
- LICENSE, 31
- LIFETIME
 - cli::GameConfig::Explosion, 54
 - cli::GameConfig::Projectile::Basic, 57
 - cli::GameConfig::Projectile::Supercharged, 58
 - utl::GameConfig::Explosion, 92
 - utl::GameConfig::Projectile::Basic, 95
 - utl::GameConfig::Projectile::Supercharged, 97
- lifetime
 - ecs::Explosion, 385
 - ecs::Projectile, 576
 - gme::EntityMetadata, 350
- loadFromConfig
 - cli::Settings, 704
- LoadingAnimationSystem
 - gme::LoadingAnimationSystem, 512
- loadLibrary
 - utl::PluginLoader, 568
- loadPlugin
 - utl::PluginLoader, 569
- lobbies
 - rnp::PacketLobbyListResponse, 545
- Lobby
 - srv::Lobby, 516
- LOBBY_CREATE
 - rnp, 74
 - utl, 81
- LOBBY_CREATE_RESPONSE
 - rnp, 74
 - utl, 81
- LOBBY_FULL
 - rnp, 73
- LOBBY_JOIN
 - rnp, 74
 - utl, 81
- LOBBY_JOIN_RESPONSE
 - rnp, 74
 - utl, 81
- LOBBY_LEAVE
 - rnp, 74
 - utl, 81
- LOBBY_LIST_REQUEST
 - rnp, 74
 - utl, 80
- LOBBY_LIST_RESPONSE
 - rnp, 74
 - utl, 81
- LOBBY_NOT_FOUND
 - rnp, 73
- LOBBY_UPDATE
 - rnp, 74
- utl, 81
- lobbyCount
 - rnp::PacketLobbyListResponse, 545
- LobbyCreateHandler
 - rnp, 71
- lobbyCreateHandler__
 - rnp::HandlerPacket, 439
- LobbyCreateResponseHandler
 - rnp, 71
- lobbyCreateResponseHandler__
 - rnp::HandlerPacket, 440
- lobbyId
 - rnp::LobbyInfo, 518
 - rnp::PacketGameStart, 537
 - rnp::PacketLobbyCreateResponse, 541
 - rnp::PacketLobbyJoin, 541
 - rnp::PacketLobbyJoinResponse, 543
 - rnp::PacketStartGameRequest, 548
 - srv::Lobby, 516
- lobbyInfo
 - rnp::PacketLobbyJoinResponse, 543
 - rnp::PacketLobbyUpdate, 546
- LobbyJoinHandler
 - rnp, 71
- lobbyJoinHandler__
 - rnp::HandlerPacket, 440
- LobbyJoinResponseHandler
 - rnp, 71
- lobbyJoinResponseHandler__
 - rnp::HandlerPacket, 440
- LobbyLeaveHandler
 - rnp, 71
- lobbyLeaveHandler__
 - rnp::HandlerPacket, 440
- LobbyListRequestHandler
 - rnp, 71
- lobbyListRequestHandler__
 - rnp::HandlerPacket, 440
- LobbyListResponseHandler
 - rnp, 71
- lobbyListResponseHandler__
 - rnp::HandlerPacket, 440
- lobbyName
 - rnp::LobbyInfo, 518
 - rnp::PacketLobbyCreate, 539
 - srv::Lobby, 516
- LobbyStatus
 - rnp, 74
 - srv, 76
- lobbyToLobbyInfo
 - srv::asioServer, 200
- LobbyUpdateHandler
 - rnp, 71
- lobbyUpdateHandler__
 - rnp::HandlerPacket, 440
- log
 - utl::Logger, 523
- LOG_LEVEL_COLOR

- utl::Logger, [525](#)
- LOG_LEVEL_STRING
 - utl::Logger, [525](#)
- logExecutionTime
 - utl::Logger, [524](#)
- Logger
 - utl::Logger, [520](#)
- LogLevel
 - utl, [81](#)
- loop
 - ecs::Audio, [220](#)
- LOOSE
 - gme, [66](#)
- LOSE
 - gme, [67](#)
- LOW
 - utl, [80](#)
- M
 - eng, [63](#)
- m_active
 - gme::WaveManager, [771](#)
- m_aggressiveness
 - gme::EnemyAISystem, [311](#)
- m_animationTime
 - cli::Menu, [529](#)
 - cli::Settings, [705](#)
 - gme::ConfigMulti, [274](#)
 - gme::ConfigSolo, [279](#)
 - gme::CreateRoomScene, [286](#)
 - gme::JoinRoomScene, [501](#)
 - gme::ServerScene, [699](#)
 - gme::WaitingRoomScene, [758](#)
- m_appConfig
 - cli::Settings, [705](#)
 - gme::FlappyBirdClientSolo, [396](#)
 - gme::RTypeClientMulti, [609](#)
 - gme::RTypeClientSolo, [614](#)
- m_audio
 - cli::Intro, [475](#)
 - ecs::AudioSystem, [224](#)
 - eng::Engine, [329](#)
 - gme::Game, [405](#)
 - gme::GameMulti, [413](#)
 - gme::GameSolo, [422](#)
- m_audioVolume
 - cli::Settings, [705](#)
 - ecs::AudioSystem, [224](#)
- m_beginSoundEntity
 - gme::GameMulti, [413](#)
- m_bossMusicEntity
 - gme::GameMulti, [414](#)
- m_bossMusicStarted
 - gme::GameMulti, [414](#)
- m_bossMusicTimer
 - gme::GameMulti, [414](#)
- m_bossSpreadTimers
 - gme::EnemyAISystem, [312](#)
- m_clientCaps
 - eng::AsioClient, [174](#)
- m_clients
 - srv::AsioServer, [213](#)
- m_clientsMutex
 - srv::AsioServer, [213](#)
- m_clientTimeout
 - srv::AsioServer, [214](#)
- m_clock
 - cli::Intro, [475](#)
 - eng::Engine, [329](#)
 - srv::Server, [693](#)
- m_collisionCount
 - gme::CollisionSystem, [266](#)
- m_collisionSystem
 - gme::RTypeServer, [623](#)
- m_completed
 - gme::WaveManager, [771](#)
- m_componentId
 - eng::AsioClient, [174](#)
 - gme::PlayerControllerMulti, [560](#)
 - srv::AsioServer, [214](#)
- m_componentMutex
 - utl::EventBus, [374](#)
- m_componentNames
 - utl::EventBus, [374](#)
- m_components
 - ecs::Registry, [604](#)
- m_config
 - cli::Client, [242](#)
 - srv::Server, [693](#)
- m_connectionState
 - eng::AsioClient, [174](#)
- m_connectionTimeout
 - eng::AsioClient, [174](#)
- m_contributors
 - cli::Menu, [529](#)
- m_contributorsEntity
 - cli::Menu, [529](#)
- m_contributorsOffset
 - cli::Menu, [529](#)
- m_controlScheme
 - cli::Settings, [705](#)
- m_controlValueEntity
 - cli::Settings, [706](#)
- m_currentLobbyId
 - eng::AsioClient, [174](#)
- m_currentLobbyInfo
 - gme::WaitingRoomScene, [759](#)
- m_currentSceneId
 - eng::SceneManager, [630](#)
 - srv::SceneManager, [633](#)
- m_currentWave
 - gme::StageManager, [726](#)
- m_currentWaveIndex
 - gme::EnemySpawnSystem, [321](#)
 - gme::WaveManager, [771](#)
- m_destroyQueue
 - gme::EntityManager, [348](#)

- m_elapsedTime
 - cli::Intro, [475](#)
- m_enabled
 - gme::EnemySpawnSystem, [321](#)
- m_endpointToSession
 - srv::AsioServer, [214](#)
- m_enemiesSpawned
 - gme::WaveManager, [772](#)
- m_enemyAISystem
 - gme::RTypeServer, [623](#)
- m_enemyData
 - gme::GameMulti, [414](#)
- m_enemyDeathAudioEntities
 - gme::CollisionSystem, [266](#)
- m_enemyEntities
 - gme::EntityManager, [348](#)
 - gme::GameMulti, [414](#)
- m_enemyMovementTimers
 - gme::EnemyAISystem, [312](#)
- m_enemySpawnSystem
 - gme::RTypeServer, [623](#)
- m_enemySpawnTimer
 - gme::SpawnSystem, [714](#)
- m_engine
 - cli::Client, [242](#)
 - gme::FlappyBirdClientSolo, [396](#)
 - gme::RTypeClientMulti, [609](#)
 - gme::RTypeClientSolo, [614](#)
- m_entities
 - ecs::Registry, [604](#)
- m_entity
 - ecs::Registry::EntityBuilder, [333](#)
- m_entityManager
 - gme::CollisionSystem, [266](#)
 - gme::EnemyAISystem, [312](#)
 - gme::EnemySpawnSystem, [321](#)
 - gme::RTypeServer, [623](#)
 - gme::WaveManager, [772](#)
- m_entityMetadata
 - gme::EntityManager, [348](#)
- m_entityToNetworkId
 - gme::EntityManager, [348](#)
- m_eventBus
 - eng::AScene, [136](#)
 - eng::AsioClient, [175](#)
 - gme::PlayerControllerMulti, [560](#)
 - gme::RTypeServer, [623](#)
 - srv::AsioServer, [214](#)
- m_eventComponentId
 - eng::AScene, [136](#)
 - gme::GameMulti, [414](#)
- m_eventQueue
 - utl::EventBus, [374](#)
- m_filteredTypes
 - utl::EventBus, [374](#)
- m_filterMutex
 - utl::EventBus, [374](#)
- m_fireCooldown
 - gme::WeaponSystem, [781](#)
- m_firstWorldState
 - gme::GameMulti, [414](#)
- m_flapSound
 - gme::Game, [405](#)
- m_fpsEntity
 - gme::ConfigSolo, [279](#)
- m_game
 - srv::Server, [693](#)
- m_gameMulti
 - cli::Client, [242](#)
- m_gameOver
 - gme::Game, [405](#)
- m_gameOverShown
 - gme::Game, [405](#)
- m_gameSolo
 - cli::Client, [242](#)
- m_gameState
 - gme::RTypeServer, [624](#)
- m_handles
 - utl::PluginLoader, [570](#)
- m_hasLobbyInfo
 - gme::WaitingRoomScene, [759](#)
- m_host
 - gme::ServerScene, [699](#)
 - srv::AsioServer, [214](#)
- m_id
 - eng::AScene, [136](#)
 - srv::AScene, [141](#)
- m_initialDelayPassed
 - gme::WaveManager, [772](#)
- m_initialDelayTimer
 - gme::WaveManager, [772](#)
- m_inputHistory
 - gme::GameMulti, [414](#)
- m_ioContext
 - eng::AsioClient, [175](#)
 - srv::AsioServer, [214](#)
- m_isCharging
 - gme::WeaponSystem, [781](#)
- m_isEnable
 - ecs::ASystem, [218](#)
- m_isHost
 - gme::WaitingRoomScene, [759](#)
- m_isPaused
 - utl::Clock, [249](#)
- m_keysPressed
 - cli::Client, [242](#)
 - cli::Menu, [529](#)
 - cli::Settings, [706](#)
 - gme::ConfigMulti, [274](#)
 - gme::ConfigSolo, [279](#)
 - gme::Game, [405](#)
 - gme::GameMulti, [414](#)
 - gme::GameSolo, [422](#)
 - gme::PlayerControllerMulti, [560](#)
- m_lastAckSeqId
 - gme::GameMulti, [414](#)

- gme::PlayerControllerMulti, 560
- m_lastAppliedSkinIndex
 - gme::GameSolo, 422
- m_lastBroadcastTime
 - gme::RTypeServer, 624
- m_lastEntity
 - ecs::Registry, 604
- m_lastPingNonce
 - eng::AsioClient, 175
- m_lastPingTime
 - eng::AsioClient, 175
 - srv::AsioServer, 214
- m_lastServerResponse
 - eng::AsioClient, 175
- m_lastShotTime
 - gme::RTypeServer, 624
- m_latency
 - eng::AsioClient, 175
- m_leaveButtonEntity
 - gme::WaitingRoomScene, 759
- m_levelState
 - gme::RTypeServer, 624
- m_levelTime
 - gme::EnemySpawnSystem, 321
- m_loadedFonts
 - gme::GameMulti, 414
 - gme::GameSolo, 422
- m_loadedTextures
 - gme::GameMulti, 414
 - gme::GameSolo, 422
- m_lobbies
 - srv::AsioServer, 214
- m_lobbiesMutex
 - srv::AsioServer, 214
- m_lobbyId
 - gme::WaitingRoomScene, 759
- m_lobbyIdEntity
 - gme::WaitingRoomScene, 759
- m_localPlayerEntity
 - gme::GameMulti, 415
- m_logoEntity
 - cli::Intro, 475
- m_looseSound
 - gme::Game, 405
- m_looseText
 - gme::Game, 405
- m_mainSceneId
 - gme::FlappyBirdClientSolo, 396
 - gme::RTypeClientMulti, 609
 - gme::RTypeClientSolo, 614
- m_maxPlayers
 - gme::CreateRoomScene, 286
- m_maxPlayersValueEntity
 - gme::CreateRoomScene, 286
- m_maxQueueSize
 - utl::EventBus, 374
- m_menuOptions
 - cli::Menu, 530
- gme::ConfigMulti, 274
- gme::ConfigSolo, 279
- m_mutex
 - utl::PluginLoader, 570
- m_name
 - eng::AScene, 137
 - srv::AScene, 141
- m_network
 - eng::Engine, 329
 - srv::Server, 693
- m_networkThread
 - eng::AsioClient, 175
 - srv::AsioServer, 215
- m_nextEnemyDeathChannel
 - gme::CollisionSystem, 266
- m_nextEnemyId
 - gme::EntityManager, 348
- m_nextId
 - eng::SceneManager, 630
- m_nextLobbyId
 - srv::AsioServer, 215
- m_nextPowerupId
 - gme::EntityManager, 348
- m_nextProjectileId
 - gme::EntityManager, 348
- m_nextSeqId
 - gme::PlayerControllerMulti, 560
- m_nextSessionId
 - srv::AsioServer, 215
- m_noRoomsEntity
 - gme::JoinRoomScene, 501
- m_onComponentAddedCallbacks
 - ecs::Registry, 604
- m_onGameStart
 - eng::AsioClient, 175
- m_onLobbyCreated
 - eng::AsioClient, 175
- m_onLobbyJoined
 - eng::AsioClient, 175
- m_onLobbyListReceived
 - eng::AsioClient, 176
- m_onLobbyUpdated
 - eng::AsioClient, 176
- m_options
 - gme::CreateRoomScene, 286
- m_packetHandler
 - eng::AsioClient, 176
 - srv::AsioServer, 215
- m_pausedDuration
 - utl::Clock, 249
- m_pausedTime
 - utl::Clock, 249
- M_PI
 - PlayerDirection.hpp, 981, 983
- m_pingInterval
 - eng::AsioClient, 176
 - srv::AsioServer, 215
- m_pipes

- gme::Game, 406
- m_playerController
 - gme::GameMulti, 415
- m_playerCountEntity
 - gme::WaitingRoomScene, 759
- m_playerDeathAudioEntity
 - gme::CollisionSystem, 266
- m_playerEntities
 - gme::EntityManager, 349
 - gme::WaitingRoomScene, 759
- m_playerEntity
 - gme::Game, 406
 - gme::GameSolo, 422
 - gme::PlayerControllerMulti, 561
- m_playerName
 - eng::AsioClient, 176
 - gme::ServerScene, 699
- m_playerNameValueEntity
 - gme::ServerScene, 699
- m_playerShooting
 - gme::RTypeServer, 624
- m_playerSkinMap
 - gme::GameMulti, 415
- m_playMusic
 - cli::Menu, 530
 - cli::Settings, 706
 - gme::ConfigMulti, 274
 - gme::ConfigSolo, 279
 - gme::CreateRoomScene, 287
 - gme::Game, 406
 - gme::GameMulti, 415
 - gme::JoinRoomScene, 501
 - gme::ServerScene, 699
 - gme::WaitingRoomScene, 759
- m_playmusic
 - gme::GameSolo, 422
- m_playSelectionMusic
 - ecs::AudioSystem, 224
- m_pluginLoader
 - cli::Client, 242
 - srv::Server, 693
- m_plugins
 - utl::PluginLoader, 570
- m_port
 - gme::ServerScene, 699
 - srv::AsioServer, 215
- m_powerupEntities
 - gme::EntityManager, 349
- m_priorityQueue
 - utl::EventBus, 374
- m_projectileData
 - gme::GameMulti, 415
- m_projectileEntities
 - gme::EntityManager, 349
 - gme::GameMulti, 415
- m_qualityValueEntity
 - cli::Settings, 706
- m_readyButtonEntity
 - gme::WaitingRoomScene, 759
- m_recvBuffer
 - eng::AsioClient, 176
 - srv::AsioServer, 215
- m_registry
 - ecs::AudioSystem, 224
 - ecs::Registry::EntityBuilder, 333
 - eng::AScene, 137
 - gme::CollisionSystem, 266
 - gme::EnemyAISystem, 312
 - gme::EntityManager, 349
 - gme::HUDSystem, 450
 - gme::RTypeServer, 624
 - srv::AScene, 141
- m_remotePlayerData
 - gme::GameMulti, 415
- m_remotePlayers
 - gme::GameMulti, 415
- m_renderer
 - cli::Intro, 475
 - cli::Settings, 706
 - ecs::BeamSystem, 230
 - ecs::DebugSystem, 291
 - ecs::SpriteSystem, 720
 - ecs::StarfieldSystem, 736
 - ecs::TextSystem, 743
 - eng::Engine, 329
 - gme::BeamSystem, 234
 - gme::CollisionSystem, 266
 - gme::EnemySystem, 325
 - gme::ExplosionSystem, 391
 - gme::Game, 406
 - gme::GameMulti, 415
 - gme::GameSolo, 422
 - gme::HUDSystem, 450
 - gme::LoadingAnimationSystem, 514
 - gme::PlayerControllerMulti, 561
 - gme::ScrollingSystem, 642
 - gme::SpawnSystem, 714
 - gme::WaitingRoomScene, 759
- m_rng
 - gme::EnemyAISystem, 312
 - gme::EnemySpawnSystem, 321
- m_roomEntities
 - gme::JoinRoomScene, 501
- m_roomName
 - gme::CreateRoomScene, 287
- m_roomNameValueEntity
 - gme::CreateRoomScene, 287
- m_rooms
 - gme::JoinRoomScene, 502
- m_running
 - eng::AsioClient, 176
 - srv::AsioServer, 215
 - utl::EventBus, 374
- m_sceneManager
 - eng::Engine, 330
 - srv::Server, 694

- m_scenes
 - eng::SceneManager, 630
 - srv::SceneManager, 633
- m_scoreBgEntity
 - gme::HUDSystem, 450
- m_scoreDigitEntities
 - gme::HUDSystem, 450
- m_selectedButton
 - gme::WaitingRoomScene, 760
- m_selectedIndex
 - cli::Menu, 530
 - cli::Settings, 706
 - gme::ConfigMulti, 274
 - gme::ConfigSolo, 279
 - gme::CreateRoomScene, 287
 - gme::JoinRoomScene, 502
 - gme::ServerScene, 700
- m_selectionSoundEntity
 - ecs::AudioSystem, 224
- m_selectionSoundName
 - ecs::AudioSystem, 225
- m_senderEndpoint
 - eng::AsioClient, 176
 - srv::AsioServer, 215
- m_sendQueue
 - eng::AsioClient, 176
 - srv::AsioServer, 215
- m_sendQueueMutex
 - eng::AsioClient, 176
 - srv::AsioServer, 216
- m_serverCaps
 - srv::AsioServer, 216
- m_serverEndpoint
 - eng::AsioClient, 177
- m_serverHost
 - eng::AsioClient, 177
- m_serverIPValueEntity
 - gme::ServerScene, 700
- m_serverOptions
 - gme::ServerScene, 700
- m_serverPort
 - eng::AsioClient, 177
- m_serverPortValueEntity
 - gme::ServerScene, 700
- m_serverTickRate
 - eng::AsioClient, 177
- m_sessionId
 - eng::AsioClient, 177
 - gme::GameMulti, 415
- m_settingsOptions
 - cli::Settings, 706
- m_shootChance
 - gme::EnemyAISystem, 312
- m_showDebug
 - cli::Client, 242
 - ecs::DebugSystem, 291
 - gme::CollisionSystem, 266
 - gme::Game, 406
 - gme::GameMulti, 415
 - gme::GameSolo, 422
 - gme::RTypeClientMulti, 609
- m_skinIndex
 - cli::Settings, 706
 - gme::GameMulti, 416
 - gme::GameSolo, 422
 - gme::PlayerDirectionSystem, 566
- m_skinSpriteEntity
 - cli::Settings, 706
- m_socket
 - eng::AsioClient, 177
 - srv::AsioServer, 216
- m_socketMutex
 - srv::AsioServer, 216
- m_soundEntity
 - cli::Intro, 475
- m_spawnTimer
 - gme::WaveManager, 772
- m_stageManager
 - gme::GameMulti, 416
 - gme::GameSolo, 422
- m_stageSpawned
 - gme::StageManager, 726
- m_stageTimer
 - gme::StageManager, 726
- m_starfieldCreated
 - gme::GameMulti, 416
- m_start
 - utl::Clock, 249
- m_startButtonEntity
 - gme::WaitingRoomScene, 760
- m_started
 - srv::AsioServer, 216
- m_state
 - eng::Engine, 330
- m_stats
 - eng::AsioClient, 177
 - utl::EventBus, 374
- m_statsMutex
 - utl::EventBus, 375
- m_statusEntity
 - gme::WaitingRoomScene, 760
- m_subscribers
 - utl::EventBus, 375
- m_subscribersMutex
 - utl::EventBus, 375
- m_superShotAudioEntity
 - gme::WeaponSystem, 781
- m_systems
 - eng::AScene, 137
- m_tickRate
 - srv::AsioServer, 216
- m_timeAccumulator
 - ecs::StarfieldSystem, 736
- m_titleEntity
 - cli::Menu, 530
 - cli::Settings, 706

- gme::ConfigMulti, 274
 - gme::ConfigSolo, 280
 - gme::CreateRoomScene, 287
- m_titlePulseTime
 - cli::Menu, 530
 - cli::Settings, 707
 - gme::ConfigMulti, 275
 - gme::ConfigSolo, 280
- m_totalEnemiesSpawned
 - gme::EnemySpawnSystem, 321
- m_totalGameTimer
 - gme::StageManager, 726
- m_videoQuality
 - cli::Settings, 707
- m_volumeValueEntity
 - cli::Settings, 707
- m_wasPlayerPresent
 - gme::CollisionSystem, 267
- m_waveInProgress
 - gme::WaveManager, 772
- m_waveManager
 - gme::RTypeServer, 624
- m_waves
 - gme::EnemySpawnSystem, 321
 - gme::WaveManager, 772
- m_waveSpawnTimer
 - gme::EnemySpawnSystem, 321
- m_waveTimer
 - gme::SpawnSystem, 714
 - gme::StageManager, 726
 - gme::WaveManager, 772
- m_yDistribution
 - gme::EnemySpawnSystem, 321
- main
 - main.cpp, 807, 808
- main.cpp
 - main, 807, 808
- MAX_CHARGE
 - cli::GameConfig::Beam, 52
 - utl::GameConfig::Beam, 90
- max_charge
 - ecs::BeamCharge, 226
- MAX_CLIENTS
 - srv, 76
- max_health
 - ecs::Enemy, 300
- MAX_IP_LENGTH
 - srv, 76
- MAX_LEN_RECV_BUFFER
 - srv, 76
- MAX_PAYLOAD
 - rnp, 75
- MAX_SHOOT_DISTANCE
 - utl::GameConfig::Server::Enemy::AI, 100
- MAX_Y
 - cli::GameConfig::Screen, 59
 - utl::GameConfig::Screen, 98
- maxPlayers
 - rnp::LobbyInfo, 518
 - rnp::PacketLobbyCreate, 540
 - srv::Lobby, 516
- maxQueueSize
 - utl::EventStats, 381
- Medium
 - game.cpp, 936
- Menu
 - cli::Menu, 528
- MIN_Y
 - cli::GameConfig::Screen, 59
 - utl::GameConfig::Screen, 98
- msgLen
 - rnp::PacketError, 536
- mtuPayloadBytes
 - rnp::PacketConnectAccept, 532
- mutex
 - utl::EventBus::EventQueue, 378
 - utl::EventBus::PriorityEventQueue, 574
- N
 - eng, 63
- name
 - eng::CircleShape, 236
 - eng::Text, 739
- nameLen
 - rnp::PacketConnect, 531
 - rnp::PacketLobbyCreate, 540
- Network plugin, 17
- Network Protocol Specification, 25
- NETWORK_CLIENT
 - utl, 81, 84
- network_lib_path
 - cli::ArgsConfig, 126
 - srv::ArgsConfig, 128
- NETWORK_SERVER
 - utl, 81, 84
- networkId
 - gme::EntityMetadata, 351
- networkThreadLoop
 - eng::AsioClient, 162
 - srv::AsioServer, 201
- networkToHost16
 - rnp::EndianUtils, 295
- networkToHost32
 - rnp::EndianUtils, 296
- networkToHostFloat
 - rnp::EndianUtils, 297
- nonce
 - rnp::PacketPingPong, 547
- NONE
 - rnp, 72–74
- None
 - eng, 63
- NORMAL
 - utl, 80
- NOT_FOUND
 - srv, 76
- NOT_IN_LOBBY

- rnp, [73](#)
- NOT_LOBBY_HOST
 - rnp, [73](#)
- now
 - utl::Clock, [247](#)
- Num0
 - eng, [64](#)
- Num1
 - eng, [64](#)
- Num2
 - eng, [64](#)
- Num3
 - eng, [64](#)
- Num4
 - eng, [64](#)
- Num5
 - eng, [64](#)
- Num6
 - eng, [64](#)
- Num7
 - eng, [64](#)
- Num8
 - eng, [64](#)
- Num9
 - eng, [64](#)
- O
 - eng, [63](#)
- OFFSET_X
 - cli::GameConfig::LoadingAnimation, [56](#)
 - utl::GameConfig::LoadingAnimation, [93](#)
- OFFSET_Y
 - cli::GameConfig::LoadingAnimation, [56](#)
 - utl::GameConfig::LoadingAnimation, [93](#)
- offsetX
 - ecs::Hitbox, [444](#)
- offsetY
 - ecs::Hitbox, [445](#)
- onBackToMenu
 - gme::ServerScene, [700](#)
- onBackToMulti
 - gme::CreateRoomScene, [287](#)
 - gme::JoinRoomScene, [502](#)
- onComplete
 - gme::Wave, [762](#)
- onComponentAdded
 - ecs::Registry, [602](#)
- onConnect
 - gme::ServerScene, [700](#)
 - rnp::HandlerPacket, [428](#)
- onConnectAccept
 - rnp::HandlerPacket, [428](#)
- onCreate
 - gme::CreateRoomScene, [287](#)
- onDisconnect
 - rnp::HandlerPacket, [429](#)
- onEntityEvent
 - rnp::HandlerPacket, [429](#)
- onError
 - rnp::HandlerPacket, [430](#)
- onGameStart
 - gme::WaitingRoomScene, [760](#)
 - rnp::HandlerPacket, [430](#)
- onJoin
 - gme::JoinRoomScene, [502](#)
- onLeave
 - cli::Intro, [475](#)
 - cli::Settings, [707](#)
- onLeaveLobby
 - gme::WaitingRoomScene, [760](#)
- onLobbyCreate
 - rnp::HandlerPacket, [430](#)
- onLobbyCreateResponse
 - rnp::HandlerPacket, [431](#)
- onLobbyJoin
 - rnp::HandlerPacket, [431](#)
- onLobbyJoinResponse
 - rnp::HandlerPacket, [432](#)
- onLobbyLeave
 - rnp::HandlerPacket, [432](#)
- onLobbyListRequest
 - rnp::HandlerPacket, [432](#)
- onLobbyListResponse
 - rnp::HandlerPacket, [433](#)
- onLobbyUpdate
 - rnp::HandlerPacket, [433](#)
- onOptionSelected
 - cli::Menu, [530](#)
 - gme::ConfigMulti, [275](#)
 - gme::ConfigSolo, [280](#)
- onPing
 - rnp::HandlerPacket, [434](#)
- onPong
 - rnp::HandlerPacket, [434](#)
- onRefreshRequest
 - gme::JoinRoomScene, [502](#)
- onRoomCreated
 - gme::CreateRoomScene, [287](#)
- onStartGameRequest
 - rnp::HandlerPacket, [435](#)
- onWorldState
 - rnp::HandlerPacket, [435](#)
- operator bool
 - utl::SharedLib, [709](#)
- operator<<
 - utl::Clock, [249](#)
- operator=
 - cli::ArgsHandler, [130](#)
 - cli::Client, [240](#)
 - cli::Intro, [474](#)
 - cli::Menu, [529](#)
 - cli::Settings, [705](#)
 - ecs::AnimationSystem, [116](#)
 - ecs::AudioSystem, [223](#)
 - ecs::BeamSystem, [229](#)
 - ecs::DebugSystem, [290](#)
 - ecs::Registry, [603](#)

- ecs::SpriteSystem, 719
- ecs::StarfieldSystem, 735
- ecs::TextSystem, 742
- eng::AScene, 136
- eng::Engine, 328, 329
- eng::SceneManager, 630
- gme::AnimationSystem, 119
- gme::BeamSystem, 233
- gme::CollisionSystem, 262
- gme::ConfigMulti, 274
- gme::ConfigSolo, 279
- gme::CreateRoomScene, 285
- gme::EnemyAISystem, 308
- gme::EnemySpawnSystem, 319
- gme::EnemySystem, 324
- gme::EntityManager, 346
- gme::ExplosionSystem, 389
- gme::FlappyBirdClientSolo, 395
- gme::Game, 404
- gme::GameMulti, 411
- gme::GameSolo, 421
- gme::HUDSystem, 449
- gme::JoinRoomScene, 500
- gme::LoadingAnimationSystem, 513
- gme::PlayerControllerMulti, 559
- gme::PlayerDirectionSystem, 565
- gme::ProjectileManager, 581
- gme::ProjectileSystem, 585
- gme::RTypeClientMulti, 608
- gme::RTypeClientSolo, 613
- gme::RTypeServer, 619
- gme::ScrollingSystem, 641
- gme::ServerScene, 698, 699
- gme::SpawnSystem, 712
- gme::StageManager, 724
- gme::WaitingRoomScene, 755
- gme::WaveManager, 768, 769
- gme::WeaponSystem, 777, 778
- rnp::HandlerPacket, 435
- srv::ArgsHandler, 132
- srv::AScene, 140
- srv::SceneManager, 632
- srv::Server, 692
- utl::Clock, 248
- utl::EventBus, 367
- utl::Logger, 524
- utl::PluginLoader, 569, 570
- utl::SharedLib, 709
- original_height
 - ecs::Scrolling, 637
- ORIGINAL_WIDTH
 - gme::StageManager, 726
- original_width
 - ecs::Scrolling, 637
- outline_color
 - eng::CircleShape, 237
- outline_thickness
 - eng::CircleShape, 237
- overlapDistance
 - gme::CollisionPair, 250
- ownerId
 - gme::EntityMetadata, 351
- P
 - eng, 63
- PACKET_ERROR
 - rnp, 74
- PacketFlags
 - rnp, 74
- packetsLost
 - eng::ConnectionStats, 281
- packetsReceived
 - eng::ConnectionStats, 281
- packetsSent
 - eng::ConnectionStats, 281
- PacketType
 - rnp, 74
- packetTypeCount
 - rnp::HandlerStats, 442
- ParseArgs
 - cli::ArgsHandler, 130
 - srv::ArgsHandler, 132
- parseEntityEvents
 - rnp::HandlerPacket, 436
- ParseEnv
 - cli::ArgsHandler, 130
 - srv::ArgsHandler, 133
- path
 - ecs::Audio, 220
 - ecs::Font, 399
 - ecs::Texture, 744
- pause
 - utl::Clock, 248
- Paused
 - eng, 64
- PHASE1_COOLDOWN
 - utl::GameConfig::Server::Enemy::Boss, 102
- PHASE1_SINE_AMPLITUDE
 - utl::GameConfig::Server::Enemy::Boss, 102
- PHASE1_SINE_FREQUENCY
 - utl::GameConfig::Server::Enemy::Boss, 102
- PHASE1_SPEED
 - utl::GameConfig::Server::Enemy::Boss, 102
- PHASE2_COOLDOWN
 - utl::GameConfig::Server::Enemy::Boss, 102
- PHASE3_AGGRESSION
 - utl::GameConfig::Server::Enemy::Boss, 102
- PHASE3_COOLDOWN
 - utl::GameConfig::Server::Enemy::Boss, 102
- PHASE3_SPEED
 - utl::GameConfig::Server::Enemy::Boss, 102
- PHASE_2_THRESHOLD
 - utl::GameConfig::Server::Enemy::Boss, 102
- PHASE_3_THRESHOLD
 - utl::GameConfig::Server::Enemy::Boss, 102
- pierce_remaining
 - ecs::Projectile, 576

- PING
 - rnp, [74](#)
- PingHandler
 - rnp, [71](#)
- pingHandler_
 - rnp::HandlerPacket, [440](#)
- PipeType
 - game.cpp, [936](#)
- play
 - ecs::Audio, [220](#)
- playAudio
 - eng::IAudio, [453](#)
- playEnemyDeathSound
 - gme::CollisionSystem, [262](#)
- PLAYER
 - gme, [67](#)
 - rnp, [72](#)
- PLAYER_CONNECTED
 - utl, [80](#)
- PLAYER_DISCONNECTED
 - utl, [80](#)
- PLAYER_INPUT_RECEIVED
 - utl, [80](#)
- player_name
 - cli::ArgsConfig, [126](#)
 - utl::cli::AppConfig, [121](#)
- PLAYER_RADIUS
 - cli::GameConfig::Hitbox, [55](#)
 - utl::GameConfig::Hitbox, [93](#)
- PlayerControllerMulti
 - gme::PlayerControllerMulti, [558](#)
- PlayerDirection.hpp
 - M_PI, [981](#), [983](#)
- PlayerDirectionSystem
 - gme::PlayerDirectionSystem, [564](#)
- playerName
 - rnp::PacketConnect, [531](#)
 - srv::ClientSession, [244](#)
 - utl::EventConnect, [376](#)
- playerNames
 - rnp::LobbyInfo, [518](#)
- playerSessions
 - srv::Lobby, [516](#)
- PLAYING
 - gme, [67](#)
- Playing
 - eng, [64](#)
- playMusic
 - cli::Menu, [529](#)
 - cli::Settings, [705](#)
 - gme::ConfigMulti, [274](#)
 - gme::ConfigSolo, [279](#)
 - gme::CreateRoomScene, [285](#)
 - gme::Game, [404](#)
 - gme::GameMulti, [411](#)
 - gme::GameSolo, [421](#)
 - gme::JoinRoomScene, [500](#)
 - gme::ServerScene, [699](#)
 - gme::WaitingRoomScene, [755](#)
- playPlayerDeathSound
 - gme::CollisionSystem, [263](#)
- PLUGIN_AUDIO_SFML
 - utl::Path::Plugin, [106](#)
- PLUGIN_EXPORT
 - IPlugin.hpp, [909](#)
- PLUGIN_GAME_MULTI
 - utl::Path::Plugin, [106](#)
- PLUGIN_GAME_SOLO
 - utl::Path::Plugin, [106](#)
- PLUGIN_NETWORK_ASIO_CLIENT
 - utl::Path::Plugin, [106](#)
- PLUGIN_RENDERER_SFML
 - utl::Path::Plugin, [107](#)
- PluginLoader
 - utl::PluginLoader, [568](#)
- PluginLoader.hpp
 - PLUGINS_EXTENSION, [913](#)
- PLUGINS_EXTENSION
 - PluginLoader.hpp, [913](#)
- PLUGINS_GAME_RTYPE_SERVER
 - srv::Path::Plugin, [78](#)
- PLUGINS_NETWORK_ASIO_SERVER
 - srv::Path::Plugin, [78](#)
- PluginType
 - utl, [81](#)
- pollEvent
 - eng::IRenderer, [485](#)
- PONG
 - rnp, [74](#)
- PongHandler
 - rnp, [71](#)
- pongHandler_
 - rnp::HandlerPacket, [440](#)
- port
 - cli::ArgsConfig, [126](#)
 - srv::ArgsConfig, [128](#)
 - utl::cli::AppConfig, [121](#)
 - utl::srv::AppConfig, [123](#)
- pos_x
 - ecs::Rect, [591](#)
- pos_y
 - ecs::Rect, [591](#)
- POWERUP
 - gme, [67](#)
 - rnp, [73](#)
- preloadCommonTextures
 - gme::GameMulti, [411](#)
 - gme::GameSolo, [421](#)
- priority
 - utl::Event, [358](#)
- priorityCount
 - utl::EventStats, [382](#)
- PriorityEventQueue
 - utl::EventBus::PriorityEventQueue, [574](#)
- processBusEvent
 - eng::AsioClient, [162](#)

- processDestroyQueue
 - gme::EntityManager, [346](#)
- processEventBus
 - gme::CreateRoomScene, [286](#)
 - gme::GameMulti, [412](#)
 - gme::JoinRoomScene, [500](#)
 - gme::WaitingRoomScene, [755](#)
- processEventBusEvents
 - srv::AsioServer, [201](#)
- processGameStartEvent
 - gme::RTypeServer, [619](#)
- PROCESSING_ERROR
 - rnp, [73](#)
- processPacket
 - rnp::HandlerPacket, [436](#)
- processPlayerInputEvent
 - gme::RTypeServer, [620](#)
- processSendQueue
 - eng::AsioClient, [163](#)
 - srv::AsioServer, [202](#)
- processServerStartEvent
 - gme::RTypeServer, [620](#)
- processSpawns
 - gme::WaveManager, [769](#)
- processWave
 - gme::EnemySpawnSystem, [319](#)
- PROJECT_NAME
 - Version.hpp, [789](#), [791](#)
- PROJECT_VERSION
 - Version.hpp, [789](#), [791](#)
- PROJECT_VERSION_MAJOR
 - Version.hpp, [790](#), [792](#)
- PROJECT_VERSION_MINOR
 - Version.hpp, [790](#), [792](#)
- PROJECT_VERSION_PATCH
 - Version.hpp, [790](#), [792](#)
- PROJECTILE
 - rnp, [72](#)
- PROJECTILE_BASIC_RADIUS
 - cli::GameConfig::Hitbox, [55](#)
 - utl::GameConfig::Hitbox, [93](#)
- PROJECTILE_COOLDOWN
 - gme::RTypeServer, [624](#)
- PROJECTILE_ENEMY
 - gme, [67](#)
 - rnp, [72](#)
- PROJECTILE_PLAYER
 - gme, [67](#)
 - rnp, [72](#)
- PROJECTILE_PLAYER_SUPERCHARGED
 - rnp, [72](#)
- PROJECTILE_SMOOTH_FACTOR
 - gme::GameMulti, [416](#)
- PROJECTILE_SUPERCHARGED_RADIUS
 - cli::GameConfig::Hitbox, [55](#)
 - utl::GameConfig::Hitbox, [93](#)
- ProjectileManager
 - gme::ProjectileManager, [578](#)
- ProjectileSystem
 - gme::ProjectileSystem, [584](#)
- PROTOCOL_ERROR
 - rnp, [72](#)
- publish
 - utl::EventBus, [367](#), [368](#)
- PURPLE
 - utl::Config::Color, [86](#)
- Q
 - eng, [63](#)
- QueuedPacket
 - eng::QueuedPacket, [587](#)
 - srv::QueuedPacket, [588](#)
- R
 - eng, [63](#)
- r
 - ecs::Color, [269](#)
 - eng::Color, [270](#)
- R-Type, [1](#)
- R-Type | Client, [5](#)
- R-Type | Server, [7](#)
- radius
 - ecs::Hitbox, [445](#)
 - eng::CircleShape, [237](#)
- RAM_DAMAGE
 - utl::GameConfig::Server::Enemy::Boss, [102](#)
- randomPipeType
 - game.cpp, [936](#)
- RATE_LIMITED
 - rnp, [73](#)
- rateLimitInterval_
 - rnp::HandlerPacket, [440](#)
- readByte
 - rnp::Serializer, [664](#)
- readBytes
 - rnp::Serializer, [665](#)
- readFile
 - utl, [83](#)
- readFloat
 - rnp::Serializer, [666](#)
- readPos_
 - rnp::Serializer, [689](#)
- readString
 - rnp::Serializer, [667](#)
- readUInt16
 - rnp::Serializer, [667](#)
- readUInt32
 - rnp::Serializer, [668](#)
- reasonCode
 - rnp::PacketDisconnect, [535](#)
- receiveTime
 - rnp::PacketContext, [534](#)
- refreshRoomList
 - gme::JoinRoomScene, [500](#)
- registerComponent
 - utl::EventBus, [369](#)
- registerEntity

- gme::EntityManager, 347
- Registry
 - ecs::Registry, 593
- RELIABLE
 - rnp, 74
- reliable
 - eng::QueuedPacket, 587
 - srv::QueuedPacket, 588
- REMOTE_PLAYER_SMOOTH_FACTOR
 - gme::GameMulti, 416
- remove
 - ecs::Registry::IPool, 479
 - ecs::Registry::Pool< T >, 572
- REMOVE_MAX_Y
 - cli::GameConfig::Screen, 59
 - utl::GameConfig::Screen, 98
- REMOVE_MIN_Y
 - cli::GameConfig::Screen, 59
 - utl::GameConfig::Screen, 98
- REMOVE_X
 - cli::GameConfig::Screen, 59
 - utl::GameConfig::Screen, 98
- removeComponent
 - ecs::Registry, 603
- removeEnemy
 - gme::CollisionSystem, 263
- removeExplosion
 - gme::ExplosionSystem, 389, 390
- removeProjectile
 - gme::CollisionSystem, 264
- render
 - eng::Engine, 329
- RENDERER
 - utl, 81
- Renderer plugin, 21
- renderer_lib_path
 - cli::ArgsConfig, 126
- renderGui
 - eng::IRenderer, 485
- RENDERING_ENGINE
 - utl, 84
- REQUEST_CONNECT
 - utl, 80
- REQUEST_DISCONNECT
 - utl, 80
- requestLobbyList
 - eng::AsioClient, 164
- requestStartGame
 - eng::AsioClient, 164
- reset
 - gme::EnemySpawnSystem, 320
 - gme::WaveManager, 769
 - gme::WeaponSystem, 778
 - rnp::Serializer, 669
 - utl::EventStats, 381
- resetCollisionCount
 - gme::CollisionSystem, 264
- resetGame
 - gme::Game, 404
- resetStats
 - rnp::HandlerPacket, 438
- restart
 - utl::Clock, 248
- resultCount
 - rnp::HandlerStats, 442
- resume
 - utl::Clock, 249
- Right
 - eng, 63
- right_pressed
 - ecs::KeyboardInput, 505
- rnp, 68
 - AckHandler, 70
 - ALREADY_IN_LOBBY, 73
 - BANNED, 72
 - BOSS, 73
 - CLIENT_REQUEST, 72
 - COMPRESSED, 74
 - CONNECT, 74
 - CONNECT_ACCEPT, 74
 - ConnectAcceptHandler, 70
 - ConnectHandler, 70
 - CUSTOM, 73
 - DAMAGE, 73
 - DESPAWN, 73
 - DISCONNECT, 74
 - DisconnectHandler, 70
 - DisconnectReason, 72
 - ENEMY, 72
 - ENEMY_ADVANCED, 72
 - ENEMY_BASIC, 72
 - ENEMY_BOSS, 72
 - ENTITY_EVENT, 74
 - EntityEventHandler, 70
 - EntitySubtype, 72
 - EntityType, 72
 - ErrorCode, 73
 - ErrorHandler, 70
 - EventType, 73
 - FINISHED, 74
 - GAME_START, 74
 - GameStartHandler, 70
 - HandlerResult, 73
 - IN_GAME, 74
 - INPUT, 73
 - INTERNAL_ERROR, 73
 - INVALID_PACKET, 73
 - INVALID_PAYLOAD, 73
 - LOBBY_CREATE, 74
 - LOBBY_CREATE_RESPONSE, 74
 - LOBBY_FULL, 73
 - LOBBY_JOIN, 74
 - LOBBY_JOIN_RESPONSE, 74
 - LOBBY_LEAVE, 74
 - LOBBY_LIST_REQUEST, 74
 - LOBBY_LIST_RESPONSE, 74

- LOBBY_NOT_FOUND, 73
- LOBBY_UPDATE, 74
- LobbyCreateHandler, 71
- LobbyCreateResponseHandler, 71
- LobbyJoinHandler, 71
- LobbyJoinResponseHandler, 71
- LobbyLeaveHandler, 71
- LobbyListRequestHandler, 71
- LobbyListResponseHandler, 71
- LobbyStatus, 74
- LobbyUpdateHandler, 71
- MAX_PAYLOAD, 75
- NONE, 72–74
- NOT_IN_LOBBY, 73
- NOT_LOBBY_HOST, 73
- PACKET_ERROR, 74
- PacketFlags, 74
- PacketType, 74
- PING, 74
- PingHandler, 71
- PLAYER, 72
- PONG, 74
- PongHandler, 71
- POWERUP, 73
- PROCESSING_ERROR, 73
- PROJECTILE, 72
- PROJECTILE_ENEMY, 72
- PROJECTILE_PLAYER, 72
- PROJECTILE_PLAYER_SUPERCHARGED, 72
- PROTOCOL_ERROR, 72
- RATE_LIMITED, 73
- RELIABLE, 74
- SCORE, 73
- SERVER_FULL, 72
- SERVER_SHUTDOWN, 72
- SESSION_INVALID, 73
- SPAWN, 73
- START_GAME_REQUEST, 75
- StartGameRequestHandler, 71
- SUCCESS, 73
- TIMEOUT, 72
- UNAUTHORIZED_SESSION, 73
- UNSPECIFIED, 72
- UNSUPPORTED_TYPE, 73
- WAITING, 74
- WORLD_STATE, 74
- WorldStateHandler, 72
- rnp::EndianUtils, 291
 - hostToNetwork16, 292
 - hostToNetwork32, 293
 - hostToNetworkFloat, 294
 - networkToHost16, 295
 - networkToHost32, 296
 - networkToHostFloat, 297
- rnp::EntityState, 351
 - healthPercent, 352
 - id, 352
 - stateFlags, 352
 - subtype, 352
 - type, 352
 - vx, 352
 - vy, 352
 - x, 353
 - y, 353
- rnp::EventRecord, 378
 - data, 379
 - entityId, 379
 - type, 379
- rnp::HandlerPacket, 423
 - ~HandlerPacket, 426
 - ackHandler_, 439
 - clearAllSessions, 427
 - clearSession, 427
 - connectAcceptHandler_, 439
 - connectHandler_, 439
 - disconnectHandler_, 439
 - entityEventHandler_, 439
 - errorHandler_, 439
 - gameStartHandler_, 439
 - getStats, 427
 - HandlerPacket, 426
 - isRateLimited, 427
 - lastPacketTime_, 439
 - lobbyCreateHandler_, 439
 - lobbyCreateResponseHandler_, 440
 - lobbyJoinHandler_, 440
 - lobbyJoinResponseHandler_, 440
 - lobbyLeaveHandler_, 440
 - lobbyListRequestHandler_, 440
 - lobbyListResponseHandler_, 440
 - lobbyUpdateHandler_, 440
 - onConnect, 428
 - onConnectAccept, 428
 - onDisconnect, 429
 - onEntityEvent, 429
 - onError, 430
 - onGameStart, 430
 - onLobbyCreate, 430
 - onLobbyCreateResponse, 431
 - onLobbyJoin, 431
 - onLobbyJoinResponse, 432
 - onLobbyLeave, 432
 - onLobbyListRequest, 432
 - onLobbyListResponse, 433
 - onLobbyUpdate, 433
 - onPing, 434
 - onPong, 434
 - onStartGameRequest, 435
 - onWorldState, 435
 - operator=, 435
 - parseEntityEvents, 436
 - pingHandler_, 440
 - pongHandler_, 440
 - processPacket, 436
 - rateLimitInterval_, 440

- resetStats, 438
- setRateLimitInterval, 438
- startGameRequestHandler_, 441
- stats_, 441
- updateStats, 438
- worldStateHandler_, 441
- rnp::HandlerStats, 441
 - packetTypeCount, 442
 - resultCount, 442
 - totalBytesReceived, 442
 - totalPacketsDropped, 442
 - totalPacketsProcessed, 442
 - totalPacketsReceived, 442
- rnp::LobbyInfo, 517
 - currentPlayers, 518
 - gameMode, 518
 - hostSessionId, 518
 - lobbyId, 518
 - lobbyName, 518
 - maxPlayers, 518
 - playerNames, 518
 - status, 518
- rnp::PacketConnect, 530
 - clientCaps, 531
 - nameLen, 531
 - playerName, 531
- rnp::PacketConnectAccept, 532
 - mtuPayloadBytes, 532
 - serverCaps, 532
 - sessionId, 532
 - tickRateHz, 532
- rnp::PacketContext, 533
 - isCompressed, 534
 - isReliable, 534
 - receiveTime, 534
 - senderAddress, 534
 - senderPort, 534
 - sessionId, 534
- rnp::PacketDisconnect, 534
 - reasonCode, 535
- rnp::PacketError, 535
 - description, 536
 - errorCode, 536
 - msgLen, 536
- rnp::PacketGameStart, 537
 - lobbyId, 537
- rnp::PacketHeader, 537
 - length, 538
 - sessionId, 538
 - type, 538
- rnp::PacketLobbyCreate, 539
 - gameMode, 539
 - lobbyName, 539
 - maxPlayers, 540
 - nameLen, 540
- rnp::PacketLobbyCreateResponse, 540
 - errorCode, 541
 - lobbyId, 541
 - success, 541
- rnp::PacketLobbyJoin, 541
 - lobbyId, 541
- rnp::PacketLobbyJoinResponse, 542
 - errorCode, 543
 - lobbyId, 543
 - lobbyInfo, 543
 - success, 543
- rnp::PacketLobbyListResponse, 543
 - lobbies, 545
 - lobbyCount, 545
- rnp::PacketLobbyUpdate, 545
 - lobbyInfo, 546
- rnp::PacketPingPong, 547
 - nonce, 547
 - sendTimeMs, 547
- rnp::PacketStartGameRequest, 547
 - lobbyId, 548
- rnp::PacketWorldState, 548
 - entities, 549
 - entityCount, 550
 - serverTick, 550
- rnp::Serializer, 642
 - buffer_, 689
 - createDamageEvent, 647
 - createDespawnEvent, 647
 - createInputEvent, 647
 - createScoreEvent, 648
 - createSpawnEvent, 649
 - deserializeConnect, 649
 - deserializeConnectAccept, 650
 - deserializeDisconnect, 651
 - deserializeEntityEvents, 651
 - deserializeEntityState, 652
 - deserializeError, 652
 - deserializeEventRecord, 653
 - deserializeGameStart, 654
 - deserializeHeader, 654
 - deserializeInputData, 655
 - deserializeLobbyCreate, 655
 - deserializeLobbyCreateResponse, 656
 - deserializeLobbyInfo, 657
 - deserializeLobbyJoin, 657
 - deserializeLobbyJoinResponse, 658
 - deserializeLobbyListResponse, 659
 - deserializeLobbyUpdate, 659
 - deserializePingPong, 660
 - deserializeStartGameRequest, 660
 - deserializeWorldState, 661
 - getData, 662
 - getSize, 663
 - readByte, 664
 - readBytes, 665
 - readFloat, 666
 - readPos_, 689
 - readString, 667
 - readUInt16, 667
 - readUInt32, 668

- reset, [669](#)
- serializeConnect, [669](#)
- serializeConnectAccept, [670](#)
- serializeDisconnect, [671](#)
- serializeEntityEvents, [671](#)
- serializeEntityState, [672](#)
- serializeError, [673](#)
- serializeEventRecord, [673](#)
- serializeGameStart, [674](#)
- serializeHeader, [675](#)
- serializeInputData, [676](#)
- serializeLobbyCreate, [677](#)
- serializeLobbyCreateResponse, [677](#)
- serializeLobbyInfo, [678](#)
- serializeLobbyJoin, [679](#)
- serializeLobbyJoinResponse, [680](#)
- serializeLobbyListResponse, [680](#)
- serializeLobbyUpdate, [681](#)
- serializePingPong, [681](#)
- Serializer, [646](#)
- serializeStartGameRequest, [682](#)
- serializeWorldState, [683](#)
- writeByte, [683](#)
- writeBytes, [684](#)
- writeFloat, [685](#)
- writePos_, [690](#)
- writeString, [686](#)
- writeUInt16, [687](#)
- writeUInt32, [688](#)
- rotateSprite
 - eng::IRenderer, [485](#)
- rotation
 - ecs::Transform, [746](#)
- RTypeClientMulti
 - gme::RTypeClientMulti, [607](#)
- RTypeClientSolo
 - gme::RTypeClientSolo, [612](#)
- RTypeServer
 - gme::RTypeServer, [617](#)
- RUN
 - eng, [64](#)
- run
 - cli::Client, [240](#)
 - srv::Server, [692](#)
- S
 - eng, [63](#)
- s_nextId
 - srv::AScene, [141](#)
- SCALE
 - cli::GameConfig::Enemy::Easy, [53](#)
 - cli::GameConfig::Explosion, [54](#)
 - cli::GameConfig::Player, [56](#)
 - cli::GameConfig::Projectile::Basic, [57](#)
 - cli::GameConfig::Projectile::Supercharged, [58](#)
 - utl::GameConfig::Enemy::Easy, [91](#)
 - utl::GameConfig::Explosion, [92](#)
 - utl::GameConfig::Player, [94](#)
 - utl::GameConfig::Projectile::Basic, [96](#)
 - utl::GameConfig::Projectile::Supercharged, [97](#)
- SceneManager
 - eng::SceneManager, [629](#)
 - srv::SceneManager, [632](#)
- SCORE
 - rnp, [73](#)
- SCREEN_HEIGHT
 - utl::GameConfig::Server, [98](#)
- SCREEN_WIDTH
 - utl::GameConfig::Server, [98](#)
- SCROLL_SPEED
 - gme::StageManager, [726](#)
- ScrollingSystem
 - gme::ScrollingSystem, [640](#)
- SEND_ENTITY_EVENT
 - utl, [80](#)
- SEND_ENTITY_EVENTS
 - utl, [80](#)
- SEND_PLAYER_INPUT
 - utl, [80](#)
- SEND_TO_CLIENT
 - utl, [80](#)
- sendConnect
 - eng::AsioClient, [165](#)
- sendConnectAccept
 - srv::AsioServer, [202](#)
- sendDisconnect
 - eng::AsioClient, [166](#)
- senderAddress
 - rnp::PacketContext, [534](#)
- senderPort
 - rnp::PacketContext, [534](#)
- sendError
 - srv::AsioServer, [203](#)
- sendInputsIfChanged
 - gme::PlayerControllerMulti, [559](#)
- sendInputToServer
 - gme::PlayerControllerMulti, [559](#)
- sendLobbyCreateResponse
 - srv::AsioServer, [204](#)
- sendLobbyJoinResponse
 - srv::AsioServer, [205](#)
- sendLobbyList
 - srv::AsioServer, [206](#)
- sendPacketImmediate
 - eng::AsioClient, [167](#)
 - srv::AsioServer, [207](#)
- sendPing
 - eng::AsioClient, [167](#)
- sendPong
 - eng::AsioClient, [168](#)
 - srv::AsioServer, [207](#)
- sendTimeMs
 - rnp::PacketPingPong, [547](#)
- sendToAllClients
 - srv::AsioServer, [208](#)
 - srv::INetworkServer, [468](#)
- sendToClient

- srv::AsioServer, 208
 - srv::INetworkServer, 468
- sendToServer
 - eng::AsioClient, 169
 - eng::INetworkClient, 464
- seqId
 - gme::GameMulti::PendingInput, 551
 - gme::PlayerControllerMulti::PendingInput, 552
- serializeConnect
 - rnp::Serializer, 669
- serializeConnectAccept
 - rnp::Serializer, 670
- serializeDisconnect
 - rnp::Serializer, 671
- serializeEntityEvents
 - rnp::Serializer, 671
- serializeEntityState
 - rnp::Serializer, 672
- serializeError
 - rnp::Serializer, 673
- serializeEventRecord
 - rnp::Serializer, 673
- serializeGameStart
 - rnp::Serializer, 674
- serializeHeader
 - rnp::Serializer, 675
- serializeInputData
 - rnp::Serializer, 676
- serializeLobbyCreate
 - rnp::Serializer, 677
- serializeLobbyCreateResponse
 - rnp::Serializer, 677
- serializeLobbyInfo
 - rnp::Serializer, 678
- serializeLobbyJoin
 - rnp::Serializer, 679
- serializeLobbyJoinResponse
 - rnp::Serializer, 680
- serializeLobbyListResponse
 - rnp::Serializer, 680
- serializeLobbyUpdate
 - rnp::Serializer, 681
- serializePingPong
 - rnp::Serializer, 681
- Serializer
 - rnp::Serializer, 646
- serializeStartGameRequest
 - rnp::Serializer, 682
- serializeWorldState
 - rnp::Serializer, 683
- Server
 - srv::Server, 692
- SERVER_DISCONNECTED
 - utl, 80
- SERVER_FULL
 - rnp, 72
- SERVER_SHUTDOWN
 - rnp, 72
- SERVER_START
 - utl, 80
- SERVER_STATUS_UPDATE
 - utl, 80
- SERVER_STOP
 - utl, 80
- serverCaps
 - rnp::PacketConnectAccept, 532
- ServerEntityType
 - gme, 66
- serverIP
 - utl::EventConnect, 376
- serverPort
 - utl::EventConnect, 376
- ServerScene
 - gme::ServerScene, 697, 698
- ServerScene.cpp
 - keyToChar, 1025
- serverTick
 - rnp::PacketWorldState, 550
- SESSION_INVALID
 - rnp, 73
- sessionId
 - rnp::PacketConnectAccept, 532
 - rnp::PacketContext, 534
 - rnp::PacketHeader, 538
 - srv::ClientSession, 244
- setAggressiveness
 - gme::EnemyAISystem, 308
- setCircleShapePosition
 - eng::IRenderer, 485
- setClientCapabilities
 - eng::AsioClient, 169
 - eng::INetworkClient, 464
- setEnabled
 - ecs::ASystem, 217
 - ecs::ISystem, 495
 - gme::CollisionSystem, 265
 - gme::EnemySystem, 324
 - gme::ExplosionSystem, 390
 - gme::SpawnSystem, 712
- setEnabled
 - gme::EnemySpawnSystem, 320
- setFrameLimit
 - eng::IRenderer, 485
- setIsHost
 - gme::WaitingRoomScene, 755
- setLobbyId
 - gme::WaitingRoomScene, 756
- setLobbyInfo
 - gme::WaitingRoomScene, 756
- setLoop
 - eng::IAudio, 453
- setMaxQueueSize
 - utl::EventBus, 370
- setName
 - eng::AScene, 136

- eng::IScene, [491](#)
- srv::AScene, [141](#)
- srv::IScene, [493](#)
- setOnGameStart
 - eng::AsioClient, [170](#)
- setOnLobbyCreated
 - eng::AsioClient, [170](#)
- setOnLobbyJoined
 - eng::AsioClient, [170](#)
- setOnLobbyListReceived
 - eng::AsioClient, [170](#)
- setOnLobbyUpdated
 - eng::AsioClient, [170](#)
- setPlayerName
 - eng::AsioClient, [171](#)
 - eng::INetworkClient, [464](#)
- setRateLimitInterval
 - rnp::HandlerPacket, [438](#)
- setRooms
 - gme::JoinRoomScene, [500](#)
- setServerCapabilities
 - srv::AsioServer, [209](#)
 - srv::INetworkServer, [468](#)
- setSpriteColor
 - eng::IRenderer, [486](#)
- setSpriteFrame
 - eng::IRenderer, [486](#)
- setSpriteOrigin
 - eng::IRenderer, [486](#)
- setSpritePosition
 - eng::IRenderer, [486](#)
- setSpriteRotation
 - eng::IRenderer, [487](#)
- setSpriteScale
 - eng::IRenderer, [487](#)
- setSpriteTexture
 - eng::IRenderer, [487](#)
- setState
 - eng::Engine, [329](#)
- setTextColor
 - eng::IRenderer, [488](#)
- setTextContent
 - eng::IRenderer, [488](#)
- setTextPosition
 - eng::IRenderer, [488](#)
- setTickRate
 - srv::AsioServer, [209](#)
 - srv::INetworkServer, [469](#)
- Settings
 - cli::Settings, [703](#), [704](#)
- setupConfig
 - cli::Client, [240](#)
 - srv::Server, [693](#)
- setupDefaultWaves
 - gme::WaveManager, [770](#)
- setupEventSubscriptions
 - gme::GameMulti, [412](#)
 - gme::JoinRoomScene, [500](#)
 - gme::WaitingRoomScene, [757](#)
- setupPacketHandlers
 - eng::AsioClient, [171](#)
 - srv::AsioServer, [209](#)
- setupScenes
 - cli::Client, [241](#)
 - gme::FlappyBirdClientSolo, [395](#)
 - gme::RTypeClientMulti, [608](#)
 - gme::RTypeClientSolo, [613](#)
- setVolume
 - eng::IAudio, [453](#)
- setWindowIcon
 - eng::IRenderer, [489](#)
- SharedLib
 - utl::SharedLib, [708](#)
- SHOOT_CHANCE
 - utl::GameConfig::Server::Enemy::AI, [100](#)
- SHOOT_COOLDOWN
 - cli::GameConfig::Enemy::Easy, [53](#)
 - utl::GameConfig::Enemy::Easy, [91](#)
 - utl::GameConfig::Server::Enemy::Advanced, [99](#)
 - utl::GameConfig::Server::Enemy::Basic, [101](#)
- shoot_cooldown
 - ecs::Enemy, [300](#)
- showLoadingAnimation
 - gme::WeaponSystem, [778](#), [779](#)
- SINE_AMPLITUDE
 - utl::GameConfig::Server::Enemy::Advanced, [99](#)
- SINE_FREQUENCY
 - utl::GameConfig::Server::Enemy::Advanced, [99](#)
- SINE_WAVE
 - gme, [66](#)
- size
 - eng::Text, [739](#)
- size_x
 - ecs::Rect, [591](#)
- size_y
 - ecs::Rect, [591](#)
- skinIndex
 - utl::cli::AppConfig, [121](#)
- Small
 - game.cpp, [936](#)
- smoothFactor
 - gme::InterpolationData, [470](#)
- sourceCount
 - utl::EventStats, [382](#)
- sourceId
 - utl::Event, [359](#)
- Space
 - eng, [63](#)
- space_pressed
 - ecs::KeyboardInput, [505](#)
- SPAWN
 - rnp, [73](#)
- SPAWN_RATE

- cli::GameConfig::Enemy::Easy, 53
- utl::GameConfig::Enemy::Easy, 91
- SPAWN_X
 - cli::GameConfig::Screen, 59
 - utl::GameConfig::Screen, 98
- spawnDelay
 - gme::EnemySpawn, 313
- spawnedCount
 - gme::SpawnWave, 716
- spawnEnemy
 - gme::SpawnSystem, 713
- spawnInterval
 - gme::SpawnWave, 716
- spawnStage
 - gme::StageManager, 724, 725
- SpawnSystem
 - gme::SpawnSystem, 712
- spawnTime
 - gme::SpawnWave, 716
- spawnWave
 - gme::SpawnSystem, 713
- SPEED
 - cli::GameConfig::Enemy::Easy, 54
 - cli::GameConfig::Player, 56
 - cli::GameConfig::Projectile::Basic, 57
 - cli::GameConfig::Projectile::Supercharged, 58
 - utl::GameConfig::Enemy::Easy, 91
 - utl::GameConfig::Player, 94
 - utl::GameConfig::Projectile::Basic, 96
 - utl::GameConfig::Projectile::Supercharged, 97
 - utl::GameConfig::Server::Enemy::Advanced, 100
 - utl::GameConfig::Server::Enemy::Basic, 101
- speed
 - ecs::Enemy, 300
- speed_x
 - ecs::Scrolling, 637
- SPREAD_ANGLE_DEGREES
 - utl::GameConfig::Server::Enemy::Boss, 103
- SPREAD_INTERVAL
 - utl::GameConfig::Server::Enemy::Boss, 103
- SPREAD_SPEED
 - utl::GameConfig::Server::Enemy::Boss, 103
- SPRITE_HEIGHT
 - cli::GameConfig::Enemy::Easy, 54
 - cli::GameConfig::Explosion, 54
 - cli::GameConfig::LoadingAnimation, 56
 - cli::GameConfig::Player, 56
 - cli::GameConfig::Projectile::Basic, 57
 - cli::GameConfig::Projectile::Supercharged, 59
 - gme::StageManager, 726
 - utl::GameConfig::Enemy::Easy, 91
 - utl::GameConfig::Explosion, 92
 - utl::GameConfig::LoadingAnimation, 94
 - utl::GameConfig::Player, 95
 - utl::GameConfig::Projectile::Basic, 96
 - utl::GameConfig::Projectile::Supercharged, 97
- SPRITE_WIDTH
 - cli::GameConfig::Enemy::Easy, 54
 - cli::GameConfig::Explosion, 55
 - cli::GameConfig::LoadingAnimation, 56
 - cli::GameConfig::Player, 57
 - cli::GameConfig::Projectile::Basic, 58
 - cli::GameConfig::Projectile::Supercharged, 59
 - utl::GameConfig::Enemy::Easy, 91
 - utl::GameConfig::Explosion, 92
 - utl::GameConfig::LoadingAnimation, 94
 - utl::GameConfig::Player, 95
 - utl::GameConfig::Projectile::Basic, 96
 - utl::GameConfig::Projectile::Supercharged, 97
- SpriteSystem
 - ecs::SpriteSystem, 719
- srv, 75
 - ALREADY_IN_LOBBY, 76
 - FULL, 76
 - id, 76
 - IN_GAME, 76
 - json, 76
 - LobbyStatus, 76
 - MAX_CLIENTS, 76
 - MAX_IP_LENGTH, 76
 - MAX_LEN_RECV_BUFFER, 76
 - NOT_FOUND, 76
 - SUCCESS, 76
 - WAITING, 76
- srv::ArgsConfig, 126
 - exit, 128
 - fromFile, 127
 - game_lib_path, 128
 - host, 128
 - network_lib_path, 128
 - port, 128
- srv::ArgsHandler, 131
 - ~ArgsHandler, 132
 - ArgsHandler, 132
 - operator=, 132
 - ParseArgs, 132
 - ParseEnv, 133
- srv::AScene, 137
 - ~AScene, 140
 - AScene, 140
 - getId, 140
 - getName, 140
 - getRegistry, 140
 - m_id, 141
 - m_name, 141
 - m_registry, 141
 - operator=, 140
 - s_nextId, 141
 - setName, 141
- srv::AsioServer, 177
 - ~AsioServer, 183
 - AsioServer, 182
 - broadcastGameStart, 183
 - broadcastLobbyUpdate, 184
 - cleanupEmptyLobbies, 185

- createLobby, 185
- disconnectClient, 186
- endpointToString, 187
- generateSessionId, 187
- getClientCount, 188
- getConnectedSessions, 188
- getName, 188
- getType, 188
- handleBroadcastEvent, 188
- handleConnect, 189
- handleDisconnect, 190
- handleEntityEvent, 191
- handleLobbyCreate, 191
- handleLobbyJoin, 192
- handleLobbyLeave, 193
- handleLobbyListRequest, 194
- handlePing, 195
- handlePong, 196
- handleReceive, 196
- handleSendEntityEventToClients, 197
- handleSendToClientEvent, 197
- handleStartGameRequest, 198
- init, 199
- isRunning, 199
- joinLobby, 199
- leaveLobby, 200
- lobbyToLobbyInfo, 200
- m_clients, 213
- m_clientsMutex, 213
- m_clientTimeout, 214
- m_componentId, 214
- m_endpointToSession, 214
- m_eventBus, 214
- m_host, 214
- m_ioContext, 214
- m_lastPingTime, 214
- m_lobbies, 214
- m_lobbiesMutex, 214
- m_networkThread, 215
- m_nextLobbyId, 215
- m_nextSessionId, 215
- m_packetHandler, 215
- m_pingInterval, 215
- m_port, 215
- m_recvBuffer, 215
- m_running, 215
- m_senderEndpoint, 215
- m_sendQueue, 215
- m_sendQueueMutex, 216
- m_serverCaps, 216
- m_socket, 216
- m_socketMutex, 216
- m_started, 216
- m_tickRate, 216
- networkThreadLoop, 201
- processEventBusEvents, 201
- processSendQueue, 202
- sendConnectAccept, 202
- sendError, 203
- sendLobbyCreateResponse, 204
- sendLobbyJoinResponse, 205
- sendLobbyList, 206
- sendPacketImmediate, 207
- sendPong, 207
- sendToAllClients, 208
- sendToClient, 208
- setServerCapabilities, 209
- setTickRate, 209
- setupPacketHandlers, 209
- start, 210
- startReceive, 211
- stop, 211
- update, 212
- updateClientManagement, 213
- srv::ClientSession, 243
 - clientCaps, 244
 - ClientSession, 244
 - currentLobbyId, 244
 - endpoint, 244
 - isConnected, 244
 - lastPingSent, 244
 - lastSeen, 244
 - latency, 244
 - playerName, 244
 - sessionId, 244
- srv::Config, 76
- srv::Config::Network, 77
 - DEFAULT_MAX_CLIENT, 77
 - DEFAULT_NETWORK_HOST, 77
 - DEFAULT_NETWORK_PORT, 77
- srv::EnvConfig, 353
- srv::Game, 77
 - DEFAULT_TICK_RATE, 77
 - DEFAULT_UPDATE_INTERVAL, 77
- srv::INetworkServer, 465
 - ~INetworkServer, 468
 - disconnectClient, 468
 - getClientCount, 468
 - getConnectedSessions, 468
 - init, 468
 - isRunning, 468
 - sendToAllClients, 468
 - sendToClient, 468
 - setServerCapabilities, 468
 - setTickRate, 469
 - start, 469
 - stop, 469
 - update, 469
- srv::IScene, 491
 - ~IScene, 493
 - getId, 493
 - getName, 493
 - getRegistry, 493
 - setName, 493
 - update, 494
- srv::Lobby, 514

- createdTime, 516
- gameMode, 516
- hostSessionId, 516
- Lobby, 516
- lobbyId, 516
- lobbyName, 516
- maxPlayers, 516
- playerSessions, 516
- status, 516
- srv::Path, 77
- srv::Path::Plugin, 77
 - PLUGINS_GAME_RTYPE_SERVER, 78
 - PLUGINS_NETWORK_ASIO_SERVER, 78
- srv::QueuedPacket, 587
 - data, 588
 - destination, 588
 - QueuedPacket, 588
 - reliable, 588
- srv::SceneManager, 630
 - ~SceneManager, 632
 - addScene, 632
 - getCurrentScene, 632
 - getScene, 632
 - m_currentSceneId, 633
 - m_scenes, 633
 - operator=, 632
 - SceneManager, 632
 - switchToScene, 633
- srv::Server, 690
 - ~Server, 692
 - m_clock, 693
 - m_config, 693
 - m_game, 693
 - m_network, 693
 - m_pluginLoader, 693
 - m_sceneManager, 694
 - operator=, 692
 - run, 692
 - Server, 692
 - setupConfig, 693
- STAGE_SPAWN_DELAY
 - gme::StageManager, 726
- StageManager
 - gme::StageManager, 722, 723
- STAR_BG
 - utl::Config::Color, 86
- STAR_MID
 - utl::Config::Color, 86
- StarfieldSystem
 - ecs::StarfieldSystem, 730, 731
- start
 - gme::IGameServer, 460
 - gme::RTypeServer, 621
 - gme::WaveManager, 770
 - srv::AsioServer, 210
 - srv::INetworkServer, 469
- START_GAME_REQUEST
 - rnp, 75
- startGame
 - gme::WaitingRoomScene, 757
- StartGameRequestHandler
 - rnp, 71
- startGameRequestHandler_
 - rnp::HandlerPacket, 441
- startNextWave
 - gme::WaveManager, 770
- startReceive
 - eng::AsioClient, 172
 - srv::AsioServer, 211
- startTime
 - utl::EventStats, 382
- State
 - eng, 64
 - gme, 67
- stateFlags
 - rnp::EntityState, 352
- STATIONARY
 - gme, 66
- stats_
 - rnp::HandlerPacket, 441
- Status
 - eng, 64
- status
 - rnp::LobbyInfo, 518
 - srv::Lobby, 516
- STOP
 - eng, 64
- stop
 - cli::Client, 241
 - eng::Engine, 329
 - gme::IGameServer, 460
 - gme::RTypeServer, 621
 - srv::AsioServer, 211
 - srv::INetworkServer, 469
 - utl::EventBus, 370
- stopAudio
 - eng::IAudio, 454
- Stopped
 - eng, 64
- stopScrolling
 - gme::StageManager, 725
- STRAIGHT
 - gme, 66
- subscribe
 - utl::EventBus, 370
- subtype
 - rnp::EntityState, 352
- SUCCESS
 - rnp, 73
 - srv, 76
- success
 - rnp::PacketLobbyCreateResponse, 541
 - rnp::PacketLobbyJoinResponse, 543
- SUPERCHARGED
 - ecs::Projectile, 576
- switchToScene

- eng::SceneManager, [630](#)
- srv::SceneManager, [633](#)
- T
 - eng, [63](#)
- targetCount
 - utl::EventStats, [382](#)
- targetId
 - utl::Event, [359](#)
- targetRotation
 - gme::InterpolationData, [470](#)
- targetVx
 - gme::InterpolationData, [470](#)
- targetVy
 - gme::InterpolationData, [470](#)
- targetX
 - gme::InterpolationData, [470](#)
- targetY
 - gme::InterpolationData, [470](#)
- Technical Analysis & Technology Comparison, [23](#)
- TEXT_VALUE_COLOR
 - utl::Config::Color, [87](#)
- TextSystem
 - ecs::TextSystem, [742](#)
- TEXTURE_BOSS
 - utl::Path::Texture, [107](#)
- TEXTURE_ENEMY_ADVANCED
 - utl::Path::Texture, [107](#)
- TEXTURE_ENEMY_BASIC
 - utl::Path::Texture, [108](#)
- TEXTURE_ENEMY_EASY
 - utl::Path::Texture, [108](#)
- TEXTURE_ENEMY_PROJECTILE
 - utl::Path::Texture, [108](#)
- TEXTURE_EXPLOSION
 - utl::Path::Texture, [108](#)
- TEXTURE_PLAYER
 - utl::Path::Texture, [108](#)
- TEXTURE_SCORE_COUNTER_BG
 - utl::Path::Texture, [108](#)
- TEXTURE_SCORE_DIGIT_0
 - utl::Path::Texture, [108](#)
- TEXTURE_SCORE_DIGIT_1
 - utl::Path::Texture, [108](#)
- TEXTURE_SCORE_DIGIT_2
 - utl::Path::Texture, [108](#)
- TEXTURE_SCORE_DIGIT_3
 - utl::Path::Texture, [108](#)
- TEXTURE_SCORE_DIGIT_4
 - utl::Path::Texture, [109](#)
- TEXTURE_SCORE_DIGIT_5
 - utl::Path::Texture, [109](#)
- TEXTURE_SCORE_DIGIT_6
 - utl::Path::Texture, [109](#)
- TEXTURE_SCORE_DIGIT_7
 - utl::Path::Texture, [109](#)
- TEXTURE_SCORE_DIGIT_8
 - utl::Path::Texture, [109](#)
- TEXTURE_SCORE_DIGIT_9
 - utl::Path::Texture, [109](#)
- TEXTURE_SHOOT
 - utl::Path::Texture, [109](#)
- TEXTURE_SHOOT_CHARGED
 - utl::Path::Texture, [109](#)
- TEXTURE_SHOOT_LOADING
 - utl::Path::Texture, [109](#)
- TEXTURE_STAGE1_CEILING
 - utl::Path::Texture, [109](#)
- TEXTURE_STAGE1_FLOOR
 - utl::Path::Texture, [110](#)
- tickRateHz
 - rnp::PacketConnectAccept, [532](#)
- TIMEOUT
 - rnp, [72](#)
- TimePoint
 - utl::Clock, [246](#)
- timestamp
 - utl::Event, [359](#)
- TOTAL_FRAMES
 - cli::GameConfig::Player, [57](#)
- total_frames
 - ecs::Animation, [113](#)
 - ecs::Explosion, [385](#)
 - ecs::LoadingAnimation, [509](#)
- TOTAL_WAVES
 - gme::StageManager, [726](#)
- totalBytesReceived
 - rnp::HandlerStats, [442](#)
- totalEvents
 - utl::EventBus::EventQueue, [378](#)
- totalEventsConsumed
 - utl::EventStats, [382](#)
- totalEventsExpired
 - utl::EventStats, [382](#)
- totalEventsPublished
 - utl::EventStats, [382](#)
- totalPacketsDropped
 - rnp::HandlerStats, [442](#)
- totalPacketsProcessed
 - rnp::HandlerStats, [442](#)
- totalPacketsReceived
 - rnp::HandlerStats, [442](#)
- tryFireBasic
 - gme::WeaponSystem, [779](#)
- tryFireSupercharged
 - gme::WeaponSystem, [780](#)
- tryShoot
 - gme::EnemyAISystem, [308](#)
- Type
 - ecs::Projectile, [576](#)
- type
 - ecs::Projectile, [576](#)
 - eng::Event, [354](#)
 - gme::EnemySpawn, [313](#)
 - gme::EntityMetadata, [351](#)
 - rnp::EntityState, [352](#)
 - rnp::EventRecord, [379](#)

- rnp::PacketHeader, 538
 - utl::Event, 359
- U
 - eng, 63
- UNAUTHORIZED_SESSION
 - rnp, 73
- unfilterEventType
 - utl::EventBus, 371
- UNKNOWN
 - utl, 81
- Unknown
 - eng, 63
- unregisterComponent
 - utl::EventBus, 372
- unregisterEntity
 - gme::EntityManager, 347
- UNSPECIFIED
 - rnp, 72
- unsubscribe
 - utl::EventBus, 372
- UNSUPPORTED_TYPE
 - rnp, 73
- Up
 - eng, 63
- up_pressed
 - ecs::KeyboardInput, 505
- update
 - cli::Intro, 475
 - cli::Menu, 529
 - cli::Settings, 705
 - ecs::AnimationSystem, 116
 - ecs::AudioSystem, 224
 - ecs::BeamSystem, 229
 - ecs::DebugSystem, 290
 - ecs::ISystem, 495
 - ecs::SpriteSystem, 719
 - ecs::StarfieldSystem, 735
 - ecs::TextSystem, 742
 - eng::AsioClient, 173
 - eng::INetworkClient, 465
 - eng::IScene, 491
 - gme::AnimationSystem, 119
 - gme::BeamSystem, 233
 - gme::CollisionSystem, 265
 - gme::ConfigMulti, 274
 - gme::ConfigSolo, 279
 - gme::CreateRoomScene, 286
 - gme::EnemyAISystem, 309
 - gme::EnemySpawnSystem, 320
 - gme::EnemySystem, 324
 - gme::ExplosionSystem, 390, 391
 - gme::FlappyBirdClientSolo, 395
 - gme::Game, 405
 - gme::GameMulti, 412
 - gme::GameSolo, 421
 - gme::HUDSystem, 449
 - gme::IGameClient, 457
 - gme::IGameServer, 461
 - gme::JoinRoomScene, 501
 - gme::LoadingAnimationSystem, 513
 - gme::PlayerControllerMulti, 560
 - gme::PlayerDirectionSystem, 565
 - gme::ProjectileSystem, 585
 - gme::RTypeClientMulti, 609
 - gme::RTypeClientSolo, 614
 - gme::RTypeServer, 621
 - gme::ScrollingSystem, 641
 - gme::ServerScene, 699
 - gme::SpawnSystem, 714
 - gme::StageManager, 725
 - gme::WaitingRoomScene, 758
 - gme::WaveManager, 771
 - gme::WeaponSystem, 780, 781
 - srv::AsioServer, 212
 - srv::INetworkServer, 469
 - srv::IScene, 494
- updateAdvancedEnemyAI
 - gme::EnemyAISystem, 309
- updateBasicEnemyAI
 - gme::EnemyAISystem, 310
- updateBossAI
 - gme::EnemyAISystem, 311
- updateClientManagement
 - srv::AsioServer, 213
- updateConnectionManagement
 - eng::AsioClient, 173
- updateEntities
 - gme::RTypeServer, 622
- updateInterpolation
 - gme::GameMulti, 413
- updateKeyboardInput
 - cli::Client, 241
- updateLifetimes
 - gme::EntityManager, 348
- updatePlayerDisplay
 - gme::WaitingRoomScene, 758
- updatePlayerSkin
 - gme::GameMulti, 413
 - gme::GameSolo, 421
- updateRoomDisplay
 - gme::JoinRoomScene, 501
- updateScore
 - gme::HUDSystem, 449
- updateSettingsDisplay
 - cli::Settings, 705
- updateStats
 - rnp::HandlerPacket, 438
 - utl::EventBus, 372
- updateSystems
 - eng::AScene, 136
 - eng::IScene, 491
 - gme::RTypeServer, 622
- updateValueDisplay
 - gme::CreateRoomScene, 286
 - gme::ServerScene, 699
- utl, 78

- AUDIO, 81
- BROADCAST_TO_CLIENTS, 80
- BROADCAST_WORLD_STATE, 80
- calculateHitboxOffsets, 81
- calculateHitboxOffsetsRelative, 82
- calculateSpriteCenter, 83
- CONNECTION_ACCEPTED, 80
- CONNECTION_REJECTED, 80
- CRITICAL, 80
- DISCONNECT_CLIENT, 80
- ENTITY_EVENT_RECEIVED, 80
- ENTITY_EVENTS_RECEIVED, 80
- EntryPointFn, 79
- EventPriority, 80
- EventType, 80
- GAME_CLIENT, 81
- GAME_LOGIC, 84
- GAME_SERVER, 81
- GAME_START, 81
- getEnvMap, 83
- HIGH, 80
- INFO, 81
- LibHandle, 79
- LOBBY_CREATE, 81
- LOBBY_CREATE_RESPONSE, 81
- LOBBY_JOIN, 81
- LOBBY_JOIN_RESPONSE, 81
- LOBBY_LEAVE, 81
- LOBBY_LIST_REQUEST, 80
- LOBBY_LIST_RESPONSE, 81
- LOBBY_UPDATE, 81
- LogLevel, 81
- LOW, 80
- NETWORK_CLIENT, 81, 84
- NETWORK_SERVER, 81, 84
- NORMAL, 80
- PLAYER_CONNECTED, 80
- PLAYER_DISCONNECTED, 80
- PLAYER_INPUT_RECEIVED, 80
- PluginType, 81
- readFile, 83
- RENDERER, 81
- RENDERING_ENGINE, 84
- REQUEST_CONNECT, 80
- REQUEST_DISCONNECT, 80
- SEND_ENTITY_EVENT, 80
- SEND_ENTITY_EVENTS, 80
- SEND_PLAYER_INPUT, 80
- SEND_TO_CLIENT, 80
- SERVER_DISCONNECTED, 80
- SERVER_START, 80
- SERVER_STATUS_UPDATE, 80
- SERVER_STOP, 80
- UNKNOWN, 81
- WARNING, 81
- WORLD_STATE_RECEIVED, 80
- utl::cli, 84
- utl::cli::AppConfig, 120
- audioVolume, 121
- controlScheme, 121
- frameLimit, 121
- fullscreen, 121
- height, 121
- host, 121
- player_name, 121
- port, 121
- skinIndex, 121
- videoQuality, 121
- width, 122
- utl::Clock, 245
 - ~Clock, 246
 - Clock, 246
 - Duration, 246
 - getDeltaSeconds, 247
 - getElapsed, 247
 - m_isPaused, 249
 - m_pausedDuration, 249
 - m_pausedTime, 249
 - m_start, 249
 - now, 247
 - operator<=, 249
 - operator=, 248
 - pause, 248
 - restart, 248
 - resume, 249
 - TimePoint, 246
- utl::Config, 84
- utl::Config::Audio, 84
 - DEFAULT_AUDIO_MUTED, 84
 - DEFAULT_AUDIO_VOLUME, 84
- utl::Config::Color, 85
 - BLUE, 85
 - BLUE_SECOND, 85
 - CYAN_ELECTRIC, 85
 - CYAN_ELECTRIC_FOREGROUND, 85
 - CYAN_ELECTRIC_PARTICLES, 86
 - DARK, 86
 - GRAY_BLUE_SUBTLE, 86
 - GREEN, 86
 - GREEN_READY, 86
 - INFO_TEXT_COLOR, 86
 - PURPLE, 86
 - STAR_BG, 86
 - STAR_MID, 86
 - TEXT_VALUE_COLOR, 87
 - WHITE, 87
 - WHITE_LOW, 87
 - WHITE_TRANS, 87
 - YELLOW, 87
- utl::Config::Game, 87
 - DEFAULT_CONTROL_SCHEME, 87
 - DEFAULT_SKIN_INDEX, 87
 - DEFAULT_VIDEO_QUALITY, 87
- utl::Config::Network, 88
 - DEFAULT_NETWORK_HOST, 88
 - DEFAULT_NETWORK_PORT, 88

- utl::Config::Player, 88
 - DEFAULT_PLAYER_NAME, 88
- utl::Config::Window, 88
 - DEFAULT_WINDOW_FRAME_LIMIT, 88
 - DEFAULT_WINDOW_FULLSCREEN, 88
 - DEFAULT_WINDOW_HEIGHT, 88
 - DEFAULT_WINDOW_WIDTH, 89
- utl::Event, 355
 - data, 358
 - Event, 356, 357
 - getAge, 357
 - hasExpired, 357
 - isBroadcast, 358
 - isTargeted, 358
 - priority, 358
 - sourceId, 359
 - targetId, 359
 - timestamp, 359
 - type, 359
- utl::EventBus, 359
 - ~EventBus, 362
 - clear, 362
 - clearStats, 363
 - consume, 363
 - consumeForTarget, 364
 - consumeType, 364
 - EventBus, 362
 - filterEventType, 365
 - getInstance, 365
 - getQueueSize, 366
 - getStats, 366
 - isRunning, 366
 - m_componentMutex, 374
 - m_componentNames, 374
 - m_eventQueue, 374
 - m_filteredTypes, 374
 - m_filterMutex, 374
 - m_maxQueueSize, 374
 - m_priorityQueue, 374
 - m_running, 374
 - m_stats, 374
 - m_statsMutex, 375
 - m_subscribers, 375
 - m_subscribersMutex, 375
 - operator=, 367
 - publish, 367, 368
 - registerComponent, 369
 - setMaxQueueSize, 370
 - stop, 370
 - subscribe, 370
 - unfilterEventType, 371
 - unregisterComponent, 372
 - unsubscribe, 372
 - updateStats, 372
 - waitForEvents, 373
- utl::EventBus::EventQueue, 377
 - cv, 378
 - events, 378
 - mutex, 378
 - totalEvents, 378
- utl::EventBus::PriorityEventQueue, 572
 - events, 574
 - mutex, 574
 - PriorityEventQueue, 574
- utl::EventConnect, 375
 - playerName, 376
 - serverIP, 376
 - serverPort, 376
- utl::EventStats, 379
 - currentQueueSize, 381
 - eventTypeCount, 381
 - getEventsPerSecond, 381
 - maxQueueSize, 381
 - priorityCount, 382
 - reset, 381
 - sourceCount, 382
 - startTime, 382
 - targetCount, 382
 - totalEventsConsumed, 382
 - totalEventsExpired, 382
 - totalEventsPublished, 382
- utl::GameConfig, 89
- utl::GameConfig::Animation, 89
 - FRAME_DURATION, 89
- utl::GameConfig::Beam, 89
 - BAR_HEIGHT, 89
 - BAR_WIDTH, 89
 - BAR_X, 90
 - BAR_Y, 90
 - CHARGE_RATE, 90
 - MAX_CHARGE, 90
- utl::GameConfig::Enemy, 90
- utl::GameConfig::Enemy::Easy, 90
 - ANIMATION_DURATION, 90
 - ANIMATION_FRAMES, 90
 - DAMAGE, 91
 - FRAMES_PER_ROW, 91
 - HEALTH, 91
 - SCALE, 91
 - SHOOT_COOLDOWN, 91
 - SPAWN_RATE, 91
 - SPEED, 91
 - SPRITE_HEIGHT, 91
 - SPRITE_WIDTH, 91
- utl::GameConfig::Explosion, 92
 - ANIMATION_DURATION, 92
 - ANIMATION_FRAMES, 92
 - FRAMES_PER_ROW, 92
 - LIFETIME, 92
 - SCALE, 92
 - SPRITE_HEIGHT, 92
 - SPRITE_WIDTH, 92
- utl::GameConfig::Hitbox, 92
 - ENEMY_RADIUS, 93
 - PLAYER_RADIUS, 93
 - PROJECTILE_BASIC_RADIUS, 93

PROJECTILE_SUPERCHARGED_RADIUS, [utl::GameConfig::Server::Enemy::AI, 100](#)
[93](#)
[utl::GameConfig::LoadingAnimation, 93](#)
 ANIMATION_DURATION, [93](#)
 ANIMATION_FRAMES, [93](#)
 OFFSET_X, [93](#)
 OFFSET_Y, [93](#)
 SPRITE_HEIGHT, [94](#)
 SPRITE_WIDTH, [94](#)
[utl::GameConfig::Player, 94](#)
 DIAGONAL_SPEED_MULTIPLIER, [94](#)
 FRAMES_PER_ROW, [94](#)
 HEIGHT, [94](#)
 SCALE, [94](#)
 SPEED, [94](#)
 SPRITE_HEIGHT, [95](#)
 SPRITE_WIDTH, [95](#)
 WIDTH, [95](#)
[utl::GameConfig::Projectile, 95](#)
[utl::GameConfig::Projectile::Basic, 95](#)
 DAMAGE, [95](#)
 FIRE_COOLDOWN, [95](#)
 LIFETIME, [95](#)
 SCALE, [96](#)
 SPEED, [96](#)
 SPRITE_HEIGHT, [96](#)
 SPRITE_WIDTH, [96](#)
[utl::GameConfig::Projectile::Supercharged, 96](#)
 ANIMATION_DURATION, [96](#)
 ANIMATION_FRAMES, [96](#)
 CHARGE_TIME, [96](#)
 DAMAGE, [97](#)
 FIRE_COOLDOWN, [97](#)
 LIFETIME, [97](#)
 SCALE, [97](#)
 SPEED, [97](#)
 SPRITE_HEIGHT, [97](#)
 SPRITE_WIDTH, [97](#)
[utl::GameConfig::Screen, 97](#)
 MAX_Y, [98](#)
 MIN_Y, [98](#)
 REMOVE_MAX_Y, [98](#)
 REMOVE_MIN_Y, [98](#)
 REMOVE_X, [98](#)
 SPAWN_X, [98](#)
[utl::GameConfig::Server, 98](#)
 SCREEN_HEIGHT, [98](#)
 SCREEN_WIDTH, [98](#)
 WORLD_MARGIN, [99](#)
[utl::GameConfig::Server::Enemy, 99](#)
[utl::GameConfig::Server::Enemy::Advanced, 99](#)
 DAMAGE, [99](#)
 HEALTH, [99](#)
 HITBOX_RADIUS, [99](#)
 SHOOT_COOLDOWN, [99](#)
 SINE_AMPLITUDE, [99](#)
 SINE_FREQUENCY, [99](#)
 SPEED, [100](#)
 AGGRESSIVE_ACTIVATION_DISTANCE, [100](#)
 MAX_SHOOT_DISTANCE, [100](#)
 SHOOT_CHANCE, [100](#)
 ZIGZAG_PERIOD, [100](#)
 ZIGZAG_SPEED_X, [100](#)
 ZIGZAG_SPEED_Y, [100](#)
[utl::GameConfig::Server::Enemy::Basic, 100](#)
 DAMAGE, [101](#)
 HEALTH, [101](#)
 HITBOX_RADIUS, [101](#)
 SHOOT_COOLDOWN, [101](#)
 SPEED, [101](#)
[utl::GameConfig::Server::Enemy::Boss, 101](#)
 DAMAGE, [101](#)
 HEALTH, [101](#)
 HITBOX_RADIUS, [102](#)
 PHASE1_COOLDOWN, [102](#)
 PHASE1_SINE_AMPLITUDE, [102](#)
 PHASE1_SINE_FREQUENCY, [102](#)
 PHASE1_SPEED, [102](#)
 PHASE2_COOLDOWN, [102](#)
 PHASE3_AGGRESSION, [102](#)
 PHASE3_COOLDOWN, [102](#)
 PHASE3_SPEED, [102](#)
 PHASE_2_THRESHOLD, [102](#)
 PHASE_3_THRESHOLD, [102](#)
 RAM_DAMAGE, [102](#)
 SPREAD_ANGLE_DEGREES, [103](#)
 SPREAD_INTERVAL, [103](#)
 SPREAD_SPEED, [103](#)
[utl::GameConfig::Server::Network, 103](#)
 BROADCAST_INTERVAL, [103](#)
 BROADCAST_RATE, [103](#)
[utl::GameConfig::Server::Projectile, 103](#)
 ENEMY_DAMAGE, [103](#)
 ENEMY_LIFETIME, [103](#)
 ENEMY_SPEED, [103](#)
[utl::GameConfig::Stage, 104](#)
 CEILING_OFFSET_Y, [104](#)
 FLOOR_OFFSET_Y, [104](#)
[utl::IPlugin, 475](#)
 ~IPlugin, [477](#)
 getName, [477](#)
 getType, [477](#)
[utl::Logger, 519](#)
 ~Logger, [520](#)
 COLOR_ERROR, [520](#)
 COLOR_INFO, [520](#)
 COLOR_RESET, [520](#)
 COLOR_WARNING, [520](#)
 ColorIndex, [520](#)
 formatLogMessage, [521](#)
 getColorForDuration, [522](#)
 init, [523](#)
 log, [523](#)
 LOG_LEVEL_COLOR, [525](#)

- LOG_LEVEL_STRING, 525
- logExecutionTime, 524
- Logger, 520
- operator=, 524
- utl::Path, 104
- utl::Path::Audio, 104
 - AUDIO_BATTLE_THEME, 104
 - AUDIO_BEGIN, 104
 - AUDIO_BOSS, 105
 - AUDIO_COIN, 105
 - AUDIO_DEATH_ALLIES, 105
 - AUDIO_DEATH_ENEMIES, 105
 - AUDIO_GOOFY, 105
 - AUDIO_INPUT, 105
 - AUDIO_SUPERCHARGED_SHOT, 105
 - AUDIO_TITLE, 105
- utl::Path::Font, 105
 - FONTS_RTYPE, 106
- utl::Path::Icons, 106
 - ICON_APP, 106
- utl::Path::Plugin, 106
 - PLUGIN_AUDIO_SFML, 106
 - PLUGIN_GAME_MULTI, 106
 - PLUGIN_GAME_SOLO, 106
 - PLUGIN_NETWORK_ASIO_CLIENT, 106
 - PLUGIN_RENDERER_SFML, 107
- utl::Path::Texture, 107
 - ICON_PATH, 107
 - TEXTURE_BOSS, 107
 - TEXTURE_ENEMY_ADVANCED, 107
 - TEXTURE_ENEMY_BASIC, 108
 - TEXTURE_ENEMY_EASY, 108
 - TEXTURE_ENEMY_PROJECTILE, 108
 - TEXTURE_EXPLOSION, 108
 - TEXTURE_PLAYER, 108
 - TEXTURE_SCORE_COUNTER_BG, 108
 - TEXTURE_SCORE_DIGIT_0, 108
 - TEXTURE_SCORE_DIGIT_1, 108
 - TEXTURE_SCORE_DIGIT_2, 108
 - TEXTURE_SCORE_DIGIT_3, 108
 - TEXTURE_SCORE_DIGIT_4, 109
 - TEXTURE_SCORE_DIGIT_5, 109
 - TEXTURE_SCORE_DIGIT_6, 109
 - TEXTURE_SCORE_DIGIT_7, 109
 - TEXTURE_SCORE_DIGIT_8, 109
 - TEXTURE_SCORE_DIGIT_9, 109
 - TEXTURE_SHOOT, 109
 - TEXTURE_SHOOT_CHARGED, 109
 - TEXTURE_SHOOT_LOADING, 109
 - TEXTURE_STAGE1_CEILING, 109
 - TEXTURE_STAGE1_FLOOR, 110
- utl::PluginLoader, 566
 - ~PluginLoader, 568
 - getEntryPoint, 568
 - loadLibrary, 568
 - loadPlugin, 569
 - m_handles, 570
 - m_mutex, 570
 - m_plugins, 570
 - operator=, 569, 570
 - PluginLoader, 568
- utl::SharedLib, 707
 - ~SharedLib, 708
 - close, 709
 - handle, 709
 - operator bool, 709
 - operator=, 709
 - SharedLib, 708
- utl::srv, 110
- utl::srv::AppConfig, 122
 - host, 123
 - port, 123
- V
 - eng, 63
- value
 - ecs::Score, 634
- Version.hpp
 - BUILD_TYPE, 789, 791
 - GIT_COMMIT_HASH, 789, 791
 - GIT_TAG, 789, 791
 - PROJECT_NAME, 789, 791
 - PROJECT_VERSION, 789, 791
 - PROJECT_VERSION_MAJOR, 790, 792
 - PROJECT_VERSION_MINOR, 790, 792
 - PROJECT_VERSION_PATCH, 790, 792
- VERSION_MESSAGE
 - argsHandler.cpp, 799, 802
- videoQuality
 - utl::cli::AppConfig, 121
- volume
 - ecs::Audio, 220
- vx
 - rnp::EntityState, 352
- vy
 - rnp::EntityState, 352
- W
 - eng, 63
- waitForClear
 - gme::Wave, 762
- waitForEvents
 - utl::EventBus, 373
- WAITING
 - rnp, 74
 - srv, 76
- WAITING_FOR_PLAYERS
 - gme, 66
- WaitingRoomScene
 - gme::WaitingRoomScene, 753
- WARNING
 - utl, 81
- WAVE_DURATION
 - gme::StageManager, 726
- WaveManager
 - gme::WaveManager, 765
- waveNumber

- gme::Wave, [762](#)
- WeaponSystem
 - gme::WeaponSystem, [776](#)
- WHITE
 - utl::Config::Color, [87](#)
- WHITE_LOW
 - utl::Config::Color, [87](#)
- WHITE_TRANS
 - utl::Config::Color, [87](#)
- WIDTH
 - utl::GameConfig::Player, [95](#)
- width
 - cli::ArgsConfig, [126](#)
 - eng::WindowSize, [782](#)
 - utl::cli::AppConfig, [122](#)
- WIN
 - gme, [67](#)
- windowIsOpen
 - eng::IRenderer, [489](#)
- with
 - ecs::Registry::EntityBuilder, [332](#)
- WORLD_MARGIN
 - utl::GameConfig::Server, [99](#)
- WORLD_STATE
 - rnp, [74](#)
- WORLD_STATE_RECEIVED
 - utl, [80](#)
- WorldStateHandler
 - rnp, [72](#)
- worldStateHandler_
 - rnp::HandlerPacket, [441](#)
- writeByte
 - rnp::Serializer, [683](#)
- writeBytes
 - rnp::Serializer, [684](#)
- writeFloat
 - rnp::Serializer, [685](#)
- writePos_
 - rnp::Serializer, [690](#)
- writeString
 - rnp::Serializer, [686](#)
- writeUInt16
 - rnp::Serializer, [687](#)
- writeUInt32
 - rnp::Serializer, [688](#)

X

- eng, [63](#)

x

- ecs::Scale, [626](#)
- ecs::Transform, [747](#)
- ecs::Velocity, [748](#)
- eng::CircleShape, [237](#)
- eng::Text, [739](#)
- gme::EnemySpawn, [313](#)
- rnp::EntityState, [353](#)

Y

- eng, [64](#)

y

- ecs::Scale, [627](#)
- ecs::Transform, [747](#)
- ecs::Velocity, [749](#)
- eng::CircleShape, [237](#)
- eng::Text, [739](#)
- gme::EnemySpawn, [313](#)
- rnp::EntityState, [353](#)

YELLOW

- utl::Config::Color, [87](#)

Z

- eng, [64](#)

ZIGZAG

- gme, [66](#)

ZIGZAG_PERIOD

- utl::GameConfig::Server::Enemy::AI, [100](#)

ZIGZAG_SPEED_X

- utl::GameConfig::Server::Enemy::AI, [100](#)

ZIGZAG_SPEED_Y

- utl::GameConfig::Server::Enemy::AI, [100](#)